

이 문서에 잘못된 정보가 있을 수 있습니다. 투비소프트는 이 문서가 제공하는 정보의 정확성을 유지하기 위해 노력하고 특별한 언급 없이 이 문서를 지속적으로 변경하고 보완할 것입니다. 그러나 이 문서에 잘못된 정보가 포함되어 있지 않다는 것을 보증하지 않습니다. 이 문서에 기술된 정보로 인해 발생할 수 있는 직접적인 또는 간접적인 손해, 데이터, 프로그램, 기타 무형의 재산에 관한 손실, 사용 이익의 손실 등에 대해 비록 이와 같은 손해 가능성에 대해 사전에 알고 있었다고 해도 손해 배상 등 기타 책임을 지지 않습니다.

사용자는 본 문서를 구입하거나, 전자 문서로 내려 받거나, 사용을 시작함으로써, 여기에 명시된 내용을 이해하며, 이에 동의하는 것으로 간주합니다.

각 회사의 제품명을 포함한 각 상표는 각 개발사의 등록 상표이며 특허법과 저작권법 등에 의해 보호를 받고 있습니다. 따라서 본 문서에 포함된 기타 모든 제품들과 회사 이름은 각각 해당 소유주의 상표로서 참조용으로만 사용됩니다.

발행처 | (주)투비소프트

발행일 | 2025/03/07

주소 | (06083) 서울시 강남구 봉은사로 617 인탑스빌딩 2-5층

전화 | 02-2140-7700

홈페이지 | www.tobesoft.com

고객지원센터 | support.tobesoft.co.kr

제품기술문의 | 1588-7895 (오전 10시부터 오후 5시까지)

유지보수정책 | 유지보수기간과 범위는 제품 라이선스 계약에 따라 다릅니다.

컴포넌트 활용 워크북

17.1.2.200

변경 이력

버전	변경일	내용
17.1.2.200.12	2024-02-21	VideoPlayer 동영상 링크 오류를 수정했습니다.
17.1.2.200.1	2020-09-08	Row 위치 변경하기 트리 형식에 마스크 문자 적용하기 병합된 셀의 체크박스 체크하기 Grid 컴포넌트로 달력 형태의 일정표 만들기
17.1.2.200	2020-09-02	한 행씩 스크롤하기 셀에 입력한 문자열 길이 표시하기 트리를 펼치거나 접을 때 하위 레벨 데이터 보이게 하기
17.1.2.100	2020-07-14	DataObject 오브젝트 신규 간단한 데이터 표시하기 다국어로 Button 라벨 표시하기 메소드 파라미터 지정해서 request 메소드 실행하기
17.1.0.200	2020-02-05	팝업창 크기 제한하기 이미지가 ImageViewer보다 클 경우 스크롤 처리하기 지도에 표시되는 언어 및 지역 특성 설정하기

버전	변경일	내용
17.1.0.100	2019-12-23	chart는 오픈소스로 전환됨에 따라 관련 콘텐츠 삭제 chart 정보는 https://www.playnexacro.com 를 참고하세요. 선 그리기 선이나 도형을 그룹으로 묶어서 처리하기 간단한 그림판 만들기 Grid 컴포넌트에서 BIGDECIMAL 타입 사용하기 1경(京)보다 큰 값 연산하기 XML 형식의 Open API 연결해서 Grid 컴포넌트에 표시하기
17.0.0.2600	2019-11-27	특정 키 입력 시 onclick 이벤트 처리하기 간단한 토글 버튼 만들기 체크 여부 확인하기 dropbutton 크기 변경하기
17.0.0.2300.1	2019-09-23	구글 폰트 사용하기 구분선 추가하기

버전	변경일	내용
17.0.0.2300	2019-08-16	맨 위로 가기 버튼 만들기 포틀릿 화면 구현하기 병합된 Head 영역의 컬럼 타이틀 텍스트 구하기 병합된 셀의 자식셀 속성 지정하기 키보드 입력 시 발생하는 onclick 이벤트 구분하기 마우스 오버 시 항목 펼치기 스크립트에서 속성값 변경 시 이벤트 처리하기 선택한 영역의 합계 보여주기 클릭한 셀 표시하기 컴포넌트의 속성을 모두 수정하기 이미지를 찾지 못할 때 대체 이미지 보여주기
17.0.0.2200	2019-07-23	Grid 컴포넌트의 Row 높이를 직접 지정하기 컬럼 너비보다 큰 텍스트만 툴팁으로 보여주기 트리 형식에서 padding 속성 적용하기 체크박스 병합해서 표시하기 데이터 너비에 맞추어 컬럼 너비 조절하기 특정 Grid 컴포넌트만 합계 Row 추가하기 마지막 좌표값 처리하기
17.0.0.2102	2019-07-08	17.0.0.2102 AS 버전 배포에 따라 관련 콘텐츠 삭제 fittocontents 속성 조합 시 표시 방식
17.0.0.2100.1	2019-06-27	문서명 변경 개발 가이드 -> 컴포넌트 활용 워크북 타이틀바 영역 변경하기 평일로 다음날 설정하기 데이터 정렬 시 null 값을 가장 위에 표시하기 자식 노드 한 번에 펼치기 아코디언 UI 만들기 특정 구분자를 줄바꿈 문자로 바꾸기
17.0.0.2100	2019-06-18	fittocontents 속성 조합 시 표시 방식
17.0.0.1900	2019-04-18	FileDialog 실행하고 선택한 파일 표시하기 선택한 파일 업로드하기 GoogleMap 컴포넌트 사용 시 apikey 관련 항목 추가
17.0.0.1800	2019-04-03	DatePickerControl 스타일 지정하기 body 밴드 영역 클릭 시 상세 데이터 표현하기 그룹화된 데이터와 관련된 계산값 구하기
17.0.0.1700	2019-02-20	트리 구조에서 자식 정보 확인하기 펼쳐지는 메뉴 효과 만들기 cssclass 속성값을 여러 개 지정하기 스크롤바가 생기지 않게 Grid 컴포넌트의 높이 조정하기 MS 오피스 뷰어 사용하기 편집중인 값 가져오기 대소문자 구분 없이 검색하거나 필터링하기
17.0.0.1600.1	2019-01-30	전체 예제가 포함된 프로젝트 코드(압축파일) 내려받기 내려받기 링크 변경 (github.

버전	변경일	내용
		com 으로 소스관리 이전)
17.0.0.1600	2019-01-22	GaugeChart 항목 추가 입력한 날짜 보정하기 속성 지정해서 선택한 셀 테두리선 표현하기 Bind 아이템 목록 확인하기 셀 실제 속성값 가져오기
17.0.0.1400.2	2019-01-09	세계 시간 표시하기 PoupDiv 컴포넌트를 사용해 선택한 셀 테두리선 표현하기 Base64로 인코딩된 이미지 보여주기

버전	변경일	내용
17.0.0.900.1	2018-07-17	여러 항목 선택하는 콤보 구현하기 트리 레벨에 따라 글자색 바꾸어 표시하기 아이콘 이미지 바꾸어 표시하기 선택한 Cell 속성 확인하기 ComboControl 내에서 선택한 항목의 스타일 바꾸어 표시하기 두 개의 Grid 컴포넌트의 스크롤을 동시에 움직이기 체크된 Row 숫자 반환하기 Like 조건으로 검색하기 중복된 항목 제거하기 숫자 형식 여부 확인하기 영문이나 숫자 여부 확인하기 중복된 요소 제거하기 숫자 배열 정렬하기 컬럼 정보 확인하고 type 속성값 변경하기
17.0.0.1000	2018-07-23	Grid에서 특정 텍스트만 입력하기 가로값이 동일한 셀을 하나의 셀로 표시하기 GroupBox 스타일 적용하기
17.0.0.1000.1	2018-08-03	삭제된 Row 정보 확인하기 스크립트로 Dataset 생성하기 변경된 컬럼정보 원복하기 RowType 변경하기 대소문자 구분 없이 필터링하기 날짜, 시간 유효성 확인하기 function 찾아서 실행하기 2차원 배열 만들기 외부 웹에디터 사용하기 Radio 버튼 표현하기
17.0.0.1100	2018-08-21	ListView 항목 추가
17.0.0.1400	2018-11-20	RadarChart 항목 추가 애니메이션 효과 적용하기 원하는 영역의 셀을 하나의 셀로 표시하기 선택한 영역의 셀을 하나의 셀로 표시하기

버전	변경일	내용
17.0.0.300	2018-01-05	Smart Scroll 기능 사용하기 항목 추가
17.0.0.300.1	2018-01-26	Grid에서 간단한 설문조사 구현하기 간단한 splitter 구현하기 항목 추가
17.0.0.400.1	2018-02-09	화면을 스텝 단위로 나누기 항목에 stepselector 관련 설명을 추가했습니다.
17.0.0.500	2018-02-22	테마, XCSS 스펙 변경에 따라 일부 설명을 수정했습니다. 테마 활용 스타일 활용 (XCSS)
17.0.0.500.1	2018-03-07	이미지 리소스 위치 변경에 따라 일부 설명 및 샘플을 수정했습니다.
17.0.0.500.2	2018-03-09	쿠키 정보 확인하기 일부 데이터를 감추어 표시하기 영문 자판으로 한글 입력 시 자동 전환하기 이메일 주소값 검증하기 항목 추가
17.0.0.500.3	2018-03-15	RGB 색상 코드 변경하기 백분율 구하기 숫자 범위 체크하기 변경된 Column 상태의 포맷 복사하기 선택한 Row 전체 컬럼을 입력 상태로 변환하기 PopupDiv에서 선택한 값을 데이터로 입력하기 항목 추가
17.0.0.500.5	2018-03-16	활성화된 화면(Form) 이름(ID) 속성 정보 얻기 화면 크기에 따라 Grid 포맷 바꾸기 스텝 기능을 활용해 슬라이드쇼 만들기 여러줄로 문자열 표시하기 항목 추가
17.0.0.500.6	2018-03-23	전각문자와 반각문자 변환하기 문자열이 지정된 길이가 되도록 다른 문자로 채우기 URI 변환하기 유니코드 변환하기 데이터가 없는 경우 Calendar 대신 대체 텍스트 표시하기 조건에 따라 displaytype 속성값 변경하기 마우스 휠 동작을 이용한 페이지징 처리 1건만 있는 소계 삭제하기 포맷 정보 확인하기 음력 날짜로 변환하기 항목 추가
17.0.0.600	2018-04-02	컬럼값이 "Y", "N"인 경우 checkboxcontrol 사용하기 특정 위치의 부모 트리만 펼치기 타이머 구현하기 항목 추가
17.0.0.600.2	2018-04-09	틀 고정하기 - 선택한 Column 영수증 계산하기 그룹에 레벨 지정해서 subtotal 계산하기 URL 링크 클릭 시 해당 사이트로 이동하기 컬럼 크기 조정 기능 제한하기 항목 추가
17.0.0.600.3	2018-04-16	문자열에 있는 하이픈(-) 제거하기 주민등록번호 유효성 검사하기 법인등록번호 유효성 검사하기

버전	변경일	내용
		문자열을 분리해 배열로 만들기 입력된 날짜에서 정보 구하기 조건에 따라 데이터 평균, 합계 계산하기 expr 내에서 사용할 수 있는 속성 스크립트로 suppress 속성 적용하기 데이터가 없는 영역을 채우기 바인딩된 값을 키 입력 시 바로 업데이트하기 innerdataset 항목 추가하기 innerHTML을 사용해 데이터 표현하기 이미지 크기 조정하기 항목 추가
17.0.0.700	2018-04-17	Animation Objects 항목 추가
17.0.0.700.1	2018-04-23	NULL 값 체크하기 컴포넌트 목록 표시하기 Bar 차트 만들기 항목 추가
17.0.0.700.2	2018-04-30	앞뒤 공백문자 제거하기 DECODE, IIF 기능 구현하기 컴포넌트의 속성, 메소드, 이벤트 확인하기 특정 Row를 깜빡이게 표시하기 데이터 변경 시 Cell 글자색을 바꾸어 표시하기 행번호 표시하기 동적으로 Grid 컴포넌트 생성하고 데이터 바인딩하기 드래그앤드롭으로 Grid 컴포넌트의 문자열 옮기기 한글만 입력할 수 있도록 하기
17.0.0.800	2018-05-24	Chart 항목 추가
17.0.0.900	2018-06-21	분산형 차트 표현하기 예제 일부를 수정했습니다.

버전	변경일	내용
17.0.0.100	2017-10-13	17.0.0.100 공개로 전환
17.0.0.100.9	2017-10-24	Grid간의 복사/붙여넣기 샘플 및 설명 수정
17.0.0.100.10	2017-11-02	글자 길이에 맞추어 배치하기 항목 추가
17.0.0.100.12	2017-11-03	- 기존 스타일 활용(XCSS)를 테마 활용으로 장 제목 변경 - 스타일 활용 (XCSS) 장 추가 - 소계 이름과 배경색 설정하기 항목 추가

차례

저작권 및 면책조항	i
변경 이력	iii
차례	viii
1. 컴포넌트 활용 워크북 소개	1
1.1 전체 예제가 포함된 프로젝트 코드(압축파일) 내려받기	1
1.2 컴포넌트 활용 워크북 활용 안내	1
파트 I. 기본 활용	3
2. 컴포넌트 배치	4
2.1 툴바에서 컴포넌트 아이콘 선택하고 배치하기	4
2.2 컴포넌트 선택하고 위치 변경하기	6
2.3 컴포넌트 선택하고 크기 변경하기	7
2.4 2개 이상 컴포넌트 선택하고 위치나 크기 변경하기	8
2.5 컴포넌트 복사하고 붙여넣기	10
2.6 보이지 않는 컴포넌트 (오브젝트)	11
2.7 비활성화되어 있는 컴포넌트 사용하기	12
3. 속성값 설정	14
3.1 선택한 컴포넌트를 속성창에 표기하기	14
3.2 선택한 컴포넌트의 기본값 확인하기	15
3.3 속성창에서 Button 컴포넌트의 text 속성값 변경하기	16
3.4 스크립트에서 Button 컴포넌트의 text 속성값 변경하기	18
4. 이벤트 함수 생성	20
4.1 속성창에서 Button 컴포넌트의 onclick 이벤트 함수 생성하기	20
4.2 Edit 컴포넌트에 입력된 텍스트 확인하기	22
4.3 하나의 이벤트 함수를 여러 컴포넌트에서 사용하기	23
5. 데이터 바인딩	26
5.1 툴바에서 Dataset 오브젝트 아이콘 선택하고 배치하기	26

5.2	Dataset Editor 실행하기	27
5.3	Dataset 오브젝트에 컬럼 추가하기	28
5.4	Dataset 오브젝트에 데이터 추가하기	30
5.5	Grid 컴포넌트에 Dataset 오브젝트 바인딩하기	31
5.6	Edit 컴포넌트에 Dataset 오브젝트 바인딩하기	32
5.7	Grid 컴포넌트와 Edit 컴포넌트에 Dataset 오브젝트 바인딩하기	34
6.	테마 활용	36
6.1	기본 테마 수정하기	36
6.2	기본 테마 복사해서 내 테마로 만들기	36
6.3	내 테마에서 Button 컴포넌트 배경색 변경하기	39
7.	스타일 활용 (XCSS)	42
7.1	넥사코로 스타일 시트(XCSS) 만들기	42
7.2	XCSS 파일 재사용하기	46
7.3	cssclass 속성으로 Button 컴포넌트 배경색 변경하기	47
8.	공통 스크립트	50
8.1	공통 스크립트 소개	50
8.2	XJS 작성하기	50
8.3	XJS 사용하기	52
8.4	XJS 가져오기	53
8.5	서비스 등록하기	53
8.6	App에서 스크립트 사용하기	54
9.	디버그 기능	56
9.1	전통적 디버깅	56
9.2	디버그 기능을 활용한 디버깅	57
9.2.1	디버그 메뉴와 툴바	57
9.2.2	디버그 정보창	59
9.3	디버그 기능 사용하기	59
9.3.1	디버깅 모드로 애플리케이션 실행/종료하기	59
9.3.2	중단점 추가/삭제하기	60
9.3.3	조건부 중단점 추가/삭제하기	61
9.3.4	중단점 활성화/비활성화하기	63
9.3.5	디버깅 모드에서 코드 실행하기(Continue)	63
9.3.6	Step 명령으로 코드 실행하기(Step Into, Step Over, Step Out)	64
	Step Into 사용법	65
	Step Over 사용법	66
	Step Out 사용법	66
9.3.7	커서 위치까지 한번에 코드 실행하기(Run to Cursor)	67
9.3.8	변수 값 확인하기(Variables)	68

9.3.9 변수 값 확인하기(Script Editor)	69
9.3.10 변수 값의 변화 관찰하기(Watch)	70
9.3.11 함수 호출 정보 확인하기(Call Stack)	72

파트 II. Application, Form, Frames 74

10. Application	75
10.1 Application 소개	75
10.1.1 사용 예	75
10.1.2 Application 접근하기	76
10.2 경고, 확인 대화상자 띄우기	76
10.2.1 예제	76
10.2.2 예제에서 사용한 핵심 기능	77
10.2.3 예제 구현 방법	77
10.3 사용자 정보 저장하기	78
10.3.1 예제	78
10.3.2 예제에서 사용한 핵심 기능	78
10.3.3 예제 구현 방법	79
10.4 쿠키 정보 확인하기	80
10.4.1 예제	80
10.4.2 예제에서 사용한 핵심 기능	80
10.4.3 예제 구현 방법	81
10.5 활성화된 화면(Form) 이름(ID) 속성 정보 얻기	82
10.5.1 예제	82
10.5.2 예제에서 사용한 핵심 기능	83
10.5.3 예제 구현 방법	83
10.6 구글 폰트 사용하기	85
10.6.1 예제	85
10.6.2 예제에서 사용한 핵심 기능	86
10.6.3 예제 구현 방법	86
11. Form	92
11.1 Form 소개	92
11.1.1 사용 예	92
11.1.2 Form 만들기	92
11.2 화면 크기에 따라 레이아웃 바꾸기	93
11.2.1 예제	93
11.2.2 예제에서 사용한 핵심 기능	95
11.2.3 예제 구현 방법	95
11.3 화면을 스텝 단위로 나누기	98

11.3.1	예제	99
11.3.2	예제에서 사용한 핵심 기능	100
11.3.3	예제 구현 방법	100
11.4	화면 이동하기	102
11.4.1	예제	102
11.4.2	예제에서 사용한 핵심 기능	103
11.4.3	예제 구현 방법	103
11.5	오브젝트 접근하기	104
11.5.1	예제	104
11.5.2	예제에서 사용한 핵심 기능	105
11.5.3	예제 구현 방법	106
11.6	엔터키로 컴포넌트 이동하기	108
11.6.1	예제	108
11.6.2	예제에서 사용한 핵심 기능	108
11.6.3	예제 구현 방법	109
11.7	타이머로 시간 표시하기	110
11.7.1	예제	110
11.7.2	예제에서 사용한 핵심 기능	111
11.7.3	예제 구현 방법	111
11.8	화면 크기에 따라 Grid 포맷 바꾸기	114
11.8.1	예제	114
11.8.2	예제에서 사용한 핵심 기능	115
11.8.3	예제 구현 방법	115
11.9	스텝 기능을 활용해 슬라이드쇼 만들기	118
11.9.1	예제	118
11.9.2	예제에서 사용한 핵심 기능	119
11.9.3	예제 구현 방법	119
11.10	컴포넌트 목록 표시하기	121
11.10.1	예제	121
11.10.2	예제에서 사용한 핵심 기능	122
11.10.3	예제 구현 방법	123
11.11	서버와 transaction 처리하기	126
11.11.1	예제	126
11.11.2	예제에서 사용한 핵심 기능	127
11.11.3	예제 구현 방법	127
11.12	Bind 아이템 목록 확인하기	129
11.12.1	예제	129
11.12.2	예제에서 사용한 핵심 기능	130
11.12.3	예제 구현 방법	130

11.13	컴포넌트의 속성을 모두 수정하기	132
11.13.1	예제	132
11.13.2	예제에서 사용한 핵심 기능	133
11.13.3	예제 구현 방법	133
12.	Frames	135
12.1	Frame 소개	135
12.1.1	사용 예	135
12.1.2	Frame 구성하기	136
12.1.3	formurl	138
12.2	ChildFrame 생성하기	139
12.2.1	예제	139
12.2.2	예제에서 사용한 핵심 기능	140
12.2.3	예제 구현 방법	140
12.3	FrameSet 생성하기	142
12.3.1	예제	143
12.3.2	예제에서 사용한 핵심 기능	143
12.3.3	예제 구현 방법	144
12.4	ChildFrame 정렬하기	146
12.4.1	예제	146
12.4.2	예제에서 사용한 핵심 기능	147
12.4.3	예제 구현 방법	147
12.5	모달창으로 ChildFrame 생성하기	149
12.5.1	예제	149
12.5.2	예제에서 사용한 핵심 기능	150
12.5.3	예제 구현 방법	151
12.6	open 메소드로 속성값 전달하기	152
12.6.1	예제	153
12.6.2	예제에서 사용한 핵심 기능	153
12.6.3	예제 구현 방법	153
12.7	타이틀바 영역 변경하기	154
12.7.1	예제	155
12.7.2	예제에서 사용한 핵심 기능	155
12.7.3	예제 구현 방법	156
12.8	팝업창 크기 제한하기	157
12.8.1	예제	157
12.8.2	예제에서 사용한 핵심 기능	158
12.8.3	예제 구현 방법	158

13. Button	161
13.1 Button 소개	161
13.1.1 사용 예	161
13.1.2 Button 만들기	162
13.2 버튼에 아이콘 넣기	162
13.2.1 예제	162
13.2.2 예제에서 사용한 핵심 기능	163
13.2.3 예제 구현 방법	163
13.3 이미지 버튼 만들기	164
13.3.1 예제	165
13.3.2 예제에서 사용한 핵심 기능	165
13.3.3 예제 구현 방법	165
13.4 버튼에 들어간 아이콘 위치 변경하기	167
13.4.1 예제	167
13.4.2 예제에서 사용한 핵심 기능	168
13.4.3 예제 구현 방법	168
13.5 컴포넌트의 속성, 메소드, 이벤트 확인하기	170
13.5.1 예제	170
13.5.2 예제에서 사용한 핵심 기능	171
13.5.3 예제 구현 방법	171
13.6 키보드 입력 시 발생하는 onclick 이벤트 구분하기	172
13.6.1 예제	172
13.6.2 예제에서 사용한 핵심 기능	173
13.6.3 예제 구현 방법	173
13.7 특정 키 입력 시 onclick 이벤트 처리하기	173
13.7.1 예제	174
13.7.2 예제에서 사용한 핵심 기능	174
13.7.3 예제 구현 방법	174
13.8 간단한 토글 버튼 만들기	175
13.8.1 예제	175
13.8.2 예제에서 사용한 핵심 기능	176
13.8.3 예제 구현 방법	176
14. Calendar	178
14.1 Calendar 소개	178
14.1.1 사용 예	178
14.1.2 Calendar 만들기	180
14.2 날짜 형식 바꾸기	181
14.2.1 예제	181
14.2.2 예제에서 사용한 핵심 기능	182

14.2.3 예제 구현 방법	183
14.3 오늘 날짜 구하기	183
14.3.1 예제	184
14.3.2 예제에서 사용한 핵심 기능	184
14.3.3 예제 구현 방법	185
14.4 Trailing day 보여주기	185
14.4.1 예제	186
14.4.2 예제에서 사용한 핵심 기능	186
14.4.3 예제 구현 방법	187
14.5 날짜 간 차이 계산하기	188
14.5.1 예제	188
14.5.2 예제에서 사용한 핵심 기능	188
14.5.3 예제 구현 방법	189
14.6 연도와 주로 날짜 구간 계산하기	190
14.6.1 예제	190
14.6.2 예제에서 사용한 핵심 기능	190
14.6.3 예제 구현 방법	191
14.7 공휴일 지정하기	192
14.7.1 예제	192
14.7.2 예제에서 사용한 핵심 기능	193
14.7.3 예제 구현 방법	193
14.8 음력 날짜로 변환하기	195
14.8.1 예제	196
14.8.2 예제에서 사용한 핵심 기능	196
14.8.3 예제 구현 방법	196
14.9 입력한 날짜 보정하기	198
14.9.1 예제	198
14.9.2 예제에서 사용한 핵심 기능	198
14.9.3 예제 구현 방법	199
14.10 DatePickerControl 스타일 지정하기	200
14.10.1 예제	201
14.10.2 예제에서 사용한 핵심 기능	201
14.10.3 예제 구현 방법	201
14.11 평일로 다음날 설정하기	203
14.11.1 예제	203
14.11.2 예제에서 사용한 핵심 기능	204
14.11.3 예제 구현 방법	204
15. CheckBox	206
15.1 CheckBox 소개	206

15.1.1	사용 예	206
15.1.2	CheckBox 만들기	207
15.2	TRUE, FALSE 값 변경하기	207
15.2.1	예제	207
15.2.2	예제에서 사용한 핵심 기능	208
15.2.3	예제 구현 방법	208
15.3	체크 여부 확인하기	209
15.3.1	예제	210
15.3.2	예제에서 사용한 핵심 기능	210
15.3.3	예제 구현 방법	210
16.	Combo	213
16.1	Combo 소개	213
16.1.1	사용 예	213
16.1.2	Combo 만들기	214
16.2	Combo 구성하기 - 고정 아이템 목록	215
16.2.1	예제	216
16.2.2	예제에서 사용한 핵심 기능	216
16.2.3	예제 구현 방법	216
16.3	Combo 구성하기 - 동적 아이템 목록	218
16.3.1	예제	218
16.3.2	예제에서 사용한 핵심 기능	219
16.3.3	예제 구현 방법	219
16.4	키워드 검색을 통하여 관련 아이템 추출하기	223
16.4.1	예제	223
16.4.2	예제에서 사용한 핵심 기능	224
16.4.3	예제 구현 방법	224
16.5	멀티 콤보 구현하기	228
16.5.1	예제	228
16.5.2	예제에서 사용한 핵심 기능	228
16.5.3	예제 구현 방법	229
16.6	여러 항목 선택하는 콤보 구현하기	231
16.6.1	예제	231
16.6.2	예제에서 사용한 핵심 기능	231
16.6.3	예제 구현 방법	231
16.7	대소문자 구분 없이 검색하거나 필터링하기	234
16.7.1	예제	234
16.7.2	예제에서 사용한 핵심 기능	234
16.7.3	예제 구현 방법	235
16.8	마우스 오버 시 항목 펼치기	236

16.8.1	예제	236
16.8.2	예제에서 사용한 핵심 기능	236
16.8.3	예제 구현 방법	237
16.9	스크립트에서 속성값 변경 시 이벤트 처리하기	238
16.9.1	예제	238
16.9.2	예제에서 사용한 핵심 기능	238
16.9.3	예제 구현 방법	239
16.10	dropdown 크기 변경하기	240
16.10.1	예제	240
16.10.2	예제에서 사용한 핵심 기능	241
16.10.3	예제 구현 방법	241
17.	Edit	243
17.1	Edit 소개	243
17.1.1	사용 예	243
17.1.2	Edit 만들기	244
17.2	문자열 길이 구하기	245
17.2.1	예제	245
17.2.2	예제에서 사용한 핵심 기능	245
17.2.3	예제 구현 방법	246
17.3	자동 포커스 이동	246
17.3.1	예제	247
17.3.2	예제에서 사용한 핵심 기능	247
17.3.3	예제 구현 방법	248
17.4	입력 언어 변경하기	248
17.4.1	예제	248
17.4.2	예제에서 사용한 핵심 기능	249
17.4.3	예제 구현 방법	250
17.5	정규표현식(Regular Expression)을 사용한 비밀번호 형식 체크	250
17.5.1	예제	251
17.5.2	예제에서 사용한 핵심 기능	251
17.5.3	예제 구현 방법	252
17.6	Edit에 Dataset 바인딩하기	254
17.6.1	예제	255
17.6.2	예제에서 사용한 핵심 기능	256
17.6.3	예제 구현 방법	256
17.7	자동완성 기능으로 키워드 제시하기	258
17.7.1	예제	259
17.7.2	예제에서 사용한 핵심 기능	260
17.7.3	예제 구현 방법	260

17.8	목록에서 선택한 값을 여러 개 Edit 컴포넌트에 입력하기	264
17.8.1	예제	265
17.8.2	예제에서 사용한 핵심 기능	265
17.8.3	예제 구현 방법	266
17.9	바인딩된 값을 키 입력 시 바로 업데이트하기	268
17.9.1	예제	268
17.9.2	예제에서 사용한 핵심 기능	269
17.9.3	예제 구현 방법	269
17.10	한글만 입력할 수 있도록 하기	270
17.10.1	예제	270
17.10.2	예제에서 사용한 핵심 기능	271
17.10.3	예제 구현 방법	271
18.	FileDialog	272
18.1	FileDialog 소개	272
18.1.1	지원 범위	272
18.2	FileDialog 실행하고 선택한 파일 표시하기	273
18.2.1	예제	273
18.2.2	예제에서 사용한 핵심 기능	274
18.2.3	예제 구현 방법	274
19.	FileUpTransfer	278
19.1	FileUpTransfer 소개	278
19.1.1	사용 예	278
19.1.2	FileUpload 컴포넌트와 차이	278
19.2	선택한 파일 업로드하기	279
19.2.1	예제	279
19.2.2	예제에서 사용한 핵심 기능	280
19.2.3	예제 구현 방법	281
20.	GoogleMap	287
20.1	GoogleMap 소개	287
20.1.1	사용 예	287
20.1.2	지도 보여주기	288
20.1.3	apikey	289
20.2	지도에 마커 추가/삭제하기	290
20.2.1	예제	290
20.2.2	예제에서 사용한 핵심 기능	291
20.2.3	예제 구현 방법	291
20.3	지도에 원 추가/삭제하기	294
20.3.1	예제	294

20.3.2	예제에서 사용한 핵심 기능	295
20.3.3	예제 구현 방법	295
20.4	지도에 표시되는 언어 및 지역 특성 설정하기	298
20.4.1	예제	298
20.4.2	예제에서 사용한 핵심 기능	299
20.4.3	예제 구현 방법	299
21.	Graphics	300
21.1	Graphics 컴포넌트 소개	300
21.1.1	Graphics.json 등록하기	300
21.2	선 그리기	301
21.2.1	예제	301
21.2.2	예제에서 사용한 핵심 기능	302
21.2.3	예제 구현 방법	302
21.3	선이나 도형을 그룹으로 묶어서 처리하기	303
21.3.1	예제	303
21.3.2	예제에서 사용한 핵심 기능	304
21.3.3	예제 구현 방법	304
21.4	간단한 그림판 만들기	308
21.4.1	예제	308
21.4.2	예제에서 사용한 핵심 기능	309
21.4.3	예제 구현 방법	309
22.	Grid 기본	315
22.1	Grid 소개	315
22.1.1	사용 예	316
22.1.2	Grid 만들기	317
22.2	Grid에 Dataset 바인딩하기	317
22.2.1	예제	317
22.2.2	예제에서 사용한 핵심 기능	318
22.2.3	예제 구현 방법	318
22.3	조건에 맞는 레코드 필터링하기	319
22.3.1	예제	319
22.3.2	예제에서 사용한 핵심 기능	320
22.3.3	예제 구현 방법	321
22.4	조건에 맞는 특정 레코드 검색하기	323
22.4.1	예제	323
22.4.2	예제에서 사용한 핵심 기능	324
22.4.3	예제 구현 방법	324
22.5	레코드 추가/삭제하기	327
22.5.1	예제	327

22.5.2	예제에서 사용한 핵심 기능	328
22.5.3	예제 구현 방법	328
22.6	여러 레코드 선택하기	330
22.6.1	예제	330
22.6.2	예제에서 사용한 핵심 기능	331
22.6.3	예제 구현 방법	331
22.7	여러 레코드 삭제하기	333
22.7.1	예제	334
22.7.2	예제에서 사용한 핵심 기능	334
22.7.3	예제 구현 방법	334
22.8	다른 데이터셋의 레코드 추가하기	336
22.8.1	예제	336
22.8.2	예제에서 사용한 핵심 기능	338
22.8.3	예제 구현 방법	338
22.9	레코드 개수 구하기	340
22.9.1	예제	341
22.9.2	예제에서 사용한 핵심 기능	341
22.9.3	예제 구현 방법	341
22.10	변경된 데이터가 있는지 확인하기	343
22.10.1	예제	343
22.10.2	예제에서 사용한 핵심 기능	344
22.10.3	예제 구현 방법	344
22.11	변경 내용 데이터셋에 반영하기 / 되돌리기	347
22.11.1	예제	348
22.11.2	예제에서 사용한 핵심 기능	348
22.11.3	예제 구현 방법	349
22.12	텍스트의 길이가 긴 경우 툴팁(tooltip)으로 보여주기	351
22.12.1	예제	351
22.12.2	예제에서 사용한 핵심 기능	352
22.12.3	예제 구현 방법	352
22.13	Smart Scroll 기능 사용하기	353
22.13.1	예제	354
22.13.2	예제에서 사용한 핵심 기능	354
22.13.3	예제 구현 방법	355
22.14	속성 지정해서 선택한 셀 테두리선 표현하기	357
22.14.1	예제	357
22.14.2	예제에서 사용한 핵심 기능	358
22.14.3	예제 구현 방법	358
22.15	스크롤바가 생기지 않게 Grid 컴포넌트의 높이 조정하기	360

22.15.1	예제	360
22.15.2	예제에서 사용한 핵심 기능	360
22.15.3	예제 구현 방법	361
22.16	Grid 컴포넌트의 Row 높이를 직접 지정하기	363
22.16.1	예제	363
22.16.2	예제에서 사용한 핵심 기능	364
22.16.3	예제 구현 방법	364
22.17	컬럼 너비보다 큰 텍스트만 툴팁으로 보여주기	366
22.17.1	예제	367
22.17.2	예제에서 사용한 핵심 기능	367
22.17.3	예제 구현 방법	367
22.18	한 행씩 스크롤하기	369
22.18.1	예제	369
22.18.2	예제에서 사용한 핵심 기능	369
22.18.3	예제 구현 방법	370
22.19	Row 위치 변경하기	371
22.19.1	예제	371
22.19.2	예제에서 사용한 핵심 기능	372
22.19.3	예제 구현 방법	372
23.	Grid 레이아웃	375
23.1	Cell 크기 조절하기 (Row 크기 일괄 조절)	375
23.1.1	예제	375
23.1.2	예제에서 사용한 핵심 기능	376
23.1.3	예제 구현 방법	376
23.2	Cell 크기 조절하기 (특정 Row 크기 조절)	378
23.2.1	예제	378
23.2.2	예제에서 사용한 핵심 기능	378
23.2.3	예제 구현 방법	379
23.3	다중 헤드와 다중 레코드 표현하기	381
23.3.1	예제	381
23.3.2	예제에서 사용한 핵심 기능	382
23.3.3	예제 구현 방법	382
23.4	자동 줄 바꾸기와 자동 Cell 크기 조절하기	386
23.4.1	예제	386
23.4.2	예제에서 사용한 핵심 기능	387
23.4.3	예제 구현 방법	387
23.5	Head Column에 여러 줄 입력하기	390
23.5.1	예제	390
23.5.2	예제에서 사용한 핵심 기능	391

23.5.3 예제 구현 방법	391
23.6 틀 고정하기 - Column	393
23.6.1 예제	394
23.6.2 예제에서 사용한 핵심 기능	395
23.6.3 예제 구현 방법	395
23.7 틀 고정하기 - Row	399
23.7.1 예제	400
23.7.2 예제에서 사용한 핵심 기능	400
23.7.3 예제 구현 방법	401
23.8 Column 위치 이동하기	403
23.8.1 예제	403
23.8.2 예제에서 사용한 핵심 기능	404
23.8.3 예제 구현 방법	404
23.9 변경된 Column 상태의 포맷 복사하기	406
23.9.1 예제	406
23.9.2 예제에서 사용한 핵심 기능	406
23.9.3 예제 구현 방법	407
23.10 포맷 정보 확인하기	408
23.10.1 예제	408
23.10.2 예제에서 사용한 핵심 기능	408
23.10.3 예제 구현 방법	409
23.11 틀 고정하기 - 선택한 Column	410
23.11.1 예제	411
23.11.2 예제에서 사용한 핵심 기능	411
23.11.3 예제 구현 방법	411
23.12 컬럼 크기 조정 기능 제한하기	414
23.12.1 예제	414
23.12.2 예제에서 사용한 핵심 기능	415
23.12.3 예제 구현 방법	415
23.13 병합된 Head 영역의 컬럼 타이틀 텍스트 구하기	416
23.13.1 예제	416
23.13.2 예제에서 사용한 핵심 기능	417
23.13.3 예제 구현 방법	417
23.14 병합된 셀의 자식셀 속성 지정하기	419
23.14.1 예제	419
23.14.2 예제에서 사용한 핵심 기능	420
23.14.3 예제 구현 방법	420
24. Grid 셀	423
24.1 포맷 변경해서 적용하기	423

24.1.1	예제	423
24.1.2	예제에서 사용한 핵심 기능	424
24.1.3	예제 구현 방법	424
24.2	바인딩된 Dataset 오브젝트의 포맷을 생성하여 보여주기	429
24.2.1	예제	429
24.2.2	예제에서 사용한 핵심 기능	430
24.2.3	예제 구현 방법	430
24.3	다양한 형식으로 데이터 표현하기	434
24.3.1	예제	434
24.3.2	예제에서 사용한 핵심 기능	435
24.3.3	예제 구현 방법	435
24.4	다양한 형식으로 데이터 편집하기	439
24.4.1	예제	440
24.4.2	예제에서 사용한 핵심 기능	441
24.4.3	예제 구현 방법	441
24.5	레코드 그룹핑하기	446
24.5.1	예제	447
24.5.2	예제에서 사용한 핵심 기능	447
24.5.3	예제 구현 방법	448
24.6	레코드 정렬하기	449
24.6.1	예제	450
24.6.2	예제에서 사용한 핵심 기능	450
24.6.3	예제 구현 방법	450
24.7	값이 동일한 셀을 하나의 셀로 표시하기 - Suppress	452
24.7.1	예제	453
24.7.2	예제에서 사용한 핵심 기능	454
24.7.3	예제 구현 방법	454
24.8	셀 속성 설정하기	456
24.8.1	예제	457
24.8.2	예제에서 사용한 핵심 기능	458
24.8.3	예제 구현 방법	459
24.9	셀 병합하기	462
24.9.1	예제	462
24.9.2	예제에서 사용한 핵심 기능	464
24.9.3	예제 구현 방법	464
24.10	서브 셀 속성 설정하기	468
24.10.1	예제	469
24.10.2	예제에서 사용한 핵심 기능	470
24.10.3	예제 구현 방법	470

24.11	일부 데이터를 감추어 표시하기	475
24.11.1	예제	475
24.11.2	예제에서 사용한 핵심 기능	475
24.11.3	예제 구현 방법	476
24.12	선택한 Row 전체 컬럼을 입력 상태로 변환하기	477
24.12.1	예제	477
24.12.2	예제에서 사용한 핵심 기능	477
24.12.3	예제 구현 방법	477
24.13	PopupDiv에서 선택한 값을 데이터로 입력하기	479
24.13.1	예제	479
24.13.2	예제에서 사용한 핵심 기능	480
24.13.3	예제 구현 방법	480
24.14	데이터가 없는 경우 Calendar 대신 대체 텍스트 표시하기	483
24.14.1	예제	483
24.14.2	예제에서 사용한 핵심 기능	483
24.14.3	예제 구현 방법	484
24.15	조건에 따라 displaytype 속성값 변경하기	485
24.15.1	예제	486
24.15.2	예제에서 사용한 핵심 기능	486
24.15.3	예제 구현 방법	486
24.16	컬럼값이 "Y", "N"인 경우 checkboxcontrol 사용하기	488
24.16.1	예제	488
24.16.2	예제에서 사용한 핵심 기능	488
24.16.3	예제 구현 방법	488
24.17	URL 링크 클릭 시 해당 사이트로 이동하기	491
24.17.1	예제	491
24.17.2	예제에서 사용한 핵심 기능	491
24.17.3	예제 구현 방법	491
24.18	expr 내에서 사용할 수 있는 속성	493
24.18.1	예제	493
24.18.2	예제에서 사용한 핵심 기능	494
24.18.3	예제 구현 방법	494
24.19	스크립트로 suppress 속성 적용하기	495
24.19.1	예제	496
24.19.2	예제에서 사용한 핵심 기능	496
24.19.3	예제 구현 방법	496
24.20	데이터가 없는 영역을 채우기	498
24.20.1	예제	498
24.20.2	예제에서 사용한 핵심 기능	499

24.20.3 예제 구현 방법	499
24.21 특정 Row를 깜빡이게 표시하기	500
24.21.1 예제	501
24.21.2 예제에서 사용한 핵심 기능	501
24.21.3 예제 구현 방법	501
24.22 데이터 변경 시 Cell 글자색을 바꾸어 표시하기	505
24.22.1 예제	506
24.22.2 예제에서 사용한 핵심 기능	506
24.22.3 예제 구현 방법	506
24.23 행번호 표시하기	508
24.23.1 예제	508
24.23.2 예제에서 사용한 핵심 기능	508
24.23.3 예제 구현 방법	508
24.24 트리 레벨에 따라 글자색 바꾸어 표시하기	510
24.24.1 예제	510
24.24.2 예제에서 사용한 핵심 기능	510
24.24.3 예제 구현 방법	511
24.25 아이콘 이미지 바꾸어 표시하기	514
24.25.1 예제	514
24.25.2 예제에서 사용한 핵심 기능	514
24.25.3 예제 구현 방법	515
24.26 선택한 Cell 속성 확인하기	517
24.26.1 예제	517
24.26.2 예제에서 사용한 핵심 기능	518
24.26.3 예제 구현 방법	518
24.27 ComboControl 내에서 선택한 항목의 스타일 바꾸어 표시하기	520
24.27.1 예제	520
24.27.2 예제에서 사용한 핵심 기능	520
24.27.3 예제 구현 방법	521
24.28 Grid에서 특정 텍스트만 입력하기	523
24.28.1 예제	523
24.28.2 예제에서 사용한 핵심 기능	523
24.28.3 예제 구현 방법	524
24.29 가로값이 동일한 셀을 하나의 셀로 표시하기	525
24.29.1 예제	525
24.29.2 예제에서 사용한 핵심 기능	526
24.29.3 예제 구현 방법	526
24.30 원하는 영역의 셀을 하나의 셀로 표시하기	528
24.30.1 예제	528

24.30.2	예제에서 사용한 핵심 기능	528
24.30.3	예제 구현 방법	529
24.31	선택한 영역의 셀을 하나의 셀로 표시하기	531
24.31.1	예제	532
24.31.2	예제에서 사용한 핵심 기능	532
24.31.3	예제 구현 방법	532
24.32	셀 실제 속성값 가져오기	535
24.32.1	예제	535
24.32.2	예제에서 사용한 핵심 기능	535
24.32.3	예제 구현 방법	536
24.33	cssclass 속성값을 여러 개 지정하기	537
24.33.1	예제	537
24.33.2	예제에서 사용한 핵심 기능	537
24.33.3	예제 구현 방법	538
24.34	편집중인 값 가져오기	540
24.34.1	예제	540
24.34.2	예제에서 사용한 핵심 기능	540
24.34.3	예제 구현 방법	541
24.35	자식 노드 한 번에 펼치기	542
24.35.1	예제	542
24.35.2	예제에서 사용한 핵심 기능	543
24.35.3	예제 구현 방법	543
24.36	트리 형식에서 padding 속성 적용하기	545
24.36.1	예제	545
24.36.2	예제에서 사용한 핵심 기능	546
24.36.3	예제 구현 방법	546
24.37	체크박스 병합해서 표시하기	548
24.37.1	예제	548
24.37.2	예제에서 사용한 핵심 기능	548
24.37.3	예제 구현 방법	549
24.38	셀에 입력한 문자열 길이 표시하기	551
24.38.1	예제	551
24.38.2	예제에서 사용한 핵심 기능	551
24.38.3	예제 구현 방법	551
24.39	트리 형식에 마스크 문자 적용하기	554
24.39.1	예제	554
24.39.2	예제에서 사용한 핵심 기능	554
24.39.3	예제 구현 방법	555
24.40	병합된 셀의 체크박스 체크하기	556

24.40.1	예제	556
24.40.2	예제에서 사용한 핵심 기능	557
24.40.3	예제 구현 방법	557
25.	Grid 응용	559
25.1	Grid에서 엑셀로 복사/붙여넣기	559
25.1.1	예제	559
25.1.2	예제에서 사용한 핵심 기능	560
25.1.3	예제 구현 방법	562
25.2	합계/소계 구하기	566
25.2.1	예제	567
25.2.2	예제에서 사용한 핵심 기능	567
25.2.3	예제 구현 방법	568
25.3	소계 이름과 배경색 설정하기	572
25.3.1	예제	572
25.3.2	예제에서 사용한 핵심 기능	573
25.3.3	예제 구현 방법	573
25.4	트리 형태로 표현하기	577
25.4.1	예제	578
25.4.2	예제에서 사용한 핵심 기능	579
25.4.3	예제 구현 방법	580
25.5	트리에서 편집하기	581
25.5.1	예제	582
25.5.2	예제에서 사용한 핵심 기능	582
25.5.3	예제 구현 방법	583
25.6	헤더 이벤트를 이용한 정렬하기	585
25.6.1	예제	585
25.6.2	예제에서 사용한 핵심 기능	586
25.6.3	예제 구현 방법	587
25.7	전체 선택/해제 하기	590
25.7.1	예제	590
25.7.2	예제에서 사용한 핵심 기능	591
25.7.3	예제 구현 방법	591
25.8	Grid간의 복사/붙여넣기	595
25.8.1	예제	595
25.8.2	예제에서 사용한 핵심 기능	596
25.8.3	예제 구현 방법	598
25.9	페이지 기능 구현하기	603
25.9.1	예제	603
25.9.2	예제에서 사용한 핵심 기능	604

25.9.3 예제 구현 방법	605
25.10 Grid에서 간단한 설문조사 구현하기	612
25.10.1 예제	613
25.10.2 예제에서 사용한 핵심 기능	613
25.10.3 예제 구현 방법	614
25.11 마우스 휠 동작을 이용한 페이지징 처리	618
25.11.1 예제	618
25.11.2 예제에서 사용한 핵심 기능	618
25.11.3 예제 구현 방법	619
25.12 1건만 있는 소계 삭제하기	621
25.12.1 예제	621
25.12.2 예제에서 사용한 핵심 기능	622
25.12.3 예제 구현 방법	622
25.13 특정 위치의 부모 트리만 펼치기	623
25.13.1 예제	624
25.13.2 예제에서 사용한 핵심 기능	624
25.13.3 예제 구현 방법	625
25.14 영수증 계산하기	627
25.14.1 예제	628
25.14.2 예제에서 사용한 핵심 기능	628
25.14.3 예제 구현 방법	628
25.15 그룹에 레벨 지정해서 subtotal 계산하기	631
25.15.1 예제	631
25.15.2 예제에서 사용한 핵심 기능	631
25.15.3 예제 구현 방법	632
25.16 동적으로 Grid 컴포넌트 생성하고 데이터 바인딩하기	633
25.16.1 예제	633
25.16.2 예제에서 사용한 핵심 기능	634
25.16.3 예제 구현 방법	634
25.17 드래그앤드롭으로 Grid 컴포넌트의 문자열 옮기기	636
25.17.1 예제	636
25.17.2 예제에서 사용한 핵심 기능	637
25.17.3 예제 구현 방법	637
25.18 두 개의 Grid 컴포넌트의 스크롤을 동시에 움직이기	640
25.18.1 예제	640
25.18.2 예제에서 사용한 핵심 기능	641
25.18.3 예제 구현 방법	641
25.19 체크된 Row 숫자 반환하기	642
25.19.1 예제	642

25.19.2	예제에서 사용한 핵심 기능	643
25.19.3	예제 구현 방법	643
25.20	Like 조건으로 검색하기	645
25.20.1	예제	645
25.20.2	예제에서 사용한 핵심 기능	646
25.20.3	예제 구현 방법	646
25.21	중복된 항목 제거하기	647
25.21.1	예제	647
25.21.2	예제에서 사용한 핵심 기능	648
25.21.3	예제 구현 방법	648
25.22	Radio 버튼 표현하기	650
25.22.1	예제	651
25.22.2	예제에서 사용한 핵심 기능	651
25.22.3	예제 구현 방법	651
25.23	세계 시간 표시하기	653
25.23.1	예제	653
25.23.2	예제에서 사용한 핵심 기능	654
25.23.3	예제 구현 방법	654
25.24	PoupDiv 컴포넌트를 사용해 선택한 셀 테두리선 표현하기	657
25.24.1	예제	657
25.24.2	예제에서 사용한 핵심 기능	657
25.24.3	예제 구현 방법	657
25.25	트리 구조에서 자식 정보 확인하기	658
25.25.1	예제	658
25.25.2	예제에서 사용한 핵심 기능	659
25.25.3	예제 구현 방법	659
25.26	데이터 정렬 시 null 값을 가장 위에 표시하기	661
25.26.1	예제	662
25.26.2	예제에서 사용한 핵심 기능	662
25.26.3	예제 구현 방법	663
25.27	데이터 너비에 맞추어 컬럼 너비 조절하기	665
25.27.1	예제	665
25.27.2	예제에서 사용한 핵심 기능	665
25.27.3	예제 구현 방법	665
25.28	특정 Grid 컴포넌트만 합계 Row 추가하기	667
25.28.1	예제	667
25.28.2	예제에서 사용한 핵심 기능	667
25.28.3	예제 구현 방법	668
25.29	선택한 영역의 합계 보여주기	670

25.29.1	예제	670
25.29.2	예제에서 사용한 핵심 기능	670
25.29.3	예제 구현 방법	670
25.30	클릭한 셀 표시하기	673
25.30.1	예제	673
25.30.2	예제에서 사용한 핵심 기능	673
25.30.3	예제 구현 방법	673
25.31	트리를 펼치거나 접을 때 하위 레벨 데이터 보이게 하기	675
25.31.1	예제	675
25.31.2	예제에서 사용한 핵심 기능	676
25.31.3	예제 구현 방법	676
25.32	Grid 컴포넌트로 달력 형태의 일정표 만들기	680
25.32.1	예제	680
25.32.2	예제에서 사용한 핵심 기능	681
25.32.3	예제 구현 방법	681
26.	GroupBox	689
26.1	GroupBox 소개	689
26.1.1	사용 예	689
26.1.2	GroupBox 만들기	690
26.2	UI 요소 그룹짓기	690
26.2.1	예제	691
26.2.2	예제에서 사용한 핵심 기능	691
26.2.3	예제 구현 방법	691
26.3	GroupBox 스타일 적용하기	692
26.3.1	예제	692
26.3.2	예제에서 사용한 핵심 기능	693
26.3.3	예제 구현 방법	693
27.	ImageViewer	695
27.1	ImageViewer 소개	695
27.1.1	사용 예	695
27.1.2	ImageViewer 만들기	696
27.2	이미지 폴더에 있는 이미지 보여주기	697
27.2.1	예제	697
27.2.2	예제에서 사용한 핵심 기능	698
27.2.3	예제 구현 방법	698
27.3	이미지 크기 조정하기	700
27.3.1	예제	700
27.3.2	예제에서 사용한 핵심 기능	701
27.3.3	예제 구현 방법	701

27.4	Base64로 인코딩된 이미지 보여주기	703
27.4.1	예제	703
27.4.2	예제에서 사용한 핵심 기능	704
27.4.3	예제 구현 방법	704
27.5	이미지를 찾지 못할 때 대체 이미지 보여주기	705
27.5.1	예제	705
27.5.2	예제에서 사용한 핵심 기능	706
27.5.3	예제 구현 방법	706
27.6	이미지가 ImageViewer보다 클 경우 스크롤 처리하기	707
27.6.1	예제	707
27.6.2	예제에서 사용한 핵심 기능	707
27.6.3	예제 구현 방법	708
28.	ListBox	710
28.1	ListBox 소개	710
28.1.1	사용 예	710
28.1.2	ListBox 만들기	711
28.2	다중선택	713
28.2.1	예제	713
28.2.2	예제에서 사용한 핵심 기능	714
28.2.3	예제 구현 방법	714
29.	ListView	717
29.1	ListView 소개	717
29.1.1	사용 예	717
29.1.2	ListView 만들기	718
29.2	기본, 상세 데이터 표현하기	722
29.2.1	예제	723
29.2.2	예제에서 사용한 핵심 기능	723
29.2.3	예제 구현 방법	724
29.3	너비에 따라 다른 형식으로 데이터 표현하기	727
29.3.1	예제	727
29.3.2	예제에서 사용한 핵심 기능	728
29.3.3	예제 구현 방법	728
29.4	버튼 클릭 이벤트 처리하기	734
29.4.1	예제	734
29.4.2	예제에서 사용한 핵심 기능	734
29.4.3	예제 구현 방법	735
29.5	body 밴드 영역 클릭 시 상세 데이터 표현하기	737
29.5.1	예제	737
29.5.2	예제에서 사용한 핵심 기능	738

29.5.3 예제 구현 방법	738
30. MaskEdit	742
30.1 MaskEdit 소개	742
30.1.1 사용 예	742
30.1.2 MaskEdit 만들기	743
30.2 정해진 형식으로 입력받기	743
30.2.1 예제	743
30.2.2 예제에서 사용한 핵심 기능	744
30.2.3 예제 구현 방법	744
30.3 주민등록번호 안 보이게 처리하기	745
30.3.1 예제	745
30.3.2 예제에서 사용한 핵심 기능	745
30.3.3 예제 구현 방법	746
31. Menu	747
31.1 Menu 소개	747
31.1.1 사용 예	747
31.1.2 Menu 만들기	748
31.2 메뉴 구성하기	749
31.2.1 예제	750
31.2.2 예제에서 사용한 핵심 기능	750
31.2.3 예제 구현 방법	751
31.3 단축키로 메뉴 실행하기	758
31.3.1 예제	758
31.3.2 예제에서 사용한 핵심 기능	759
31.3.3 예제 구현 방법	759
32. PopupMenu	761
32.1 PopupMenu 소개	761
32.1.1 사용 예	761
32.1.2 PopupMenu 만들기	762
32.2 팝업 위치 변경하기	763
32.2.1 예제	763
32.2.2 예제에서 사용한 핵심 기능	764
32.2.3 예제 구현 방법	765
32.3 구분선 추가하기	767
32.3.1 예제	768
32.3.2 예제에서 사용한 핵심 기능	768
32.3.3 예제 구현 방법	768
33. ProgressBar	770

33.1	ProgressBar 소개	770
33.1.1	사용 예	770
33.1.2	ProgressBar 만들기	771
33.2	진행 상태 표시하기	771
33.2.1	예제	771
33.2.2	예제에서 사용한 핵심 기능	772
33.2.3	예제 구현 방법	772
34.	Radio	776
34.1	Radio 소개	776
34.1.1	사용 예	776
34.1.2	Radio 만들기	777
34.2	기본값 설정하기	777
34.2.1	예제	778
34.2.2	예제에서 사용한 핵심 기능	778
34.2.3	예제 구현 방법	778
34.3	가로 방향으로 나열하기	779
34.3.1	예제	779
34.3.2	예제에서 사용한 핵심 기능	779
34.3.3	예제 구현 방법	780
34.4	innerdataset 항목 추가하기	782
34.4.1	예제	782
34.4.2	예제에서 사용한 핵심 기능	782
34.4.3	예제 구현 방법	782
34.5	fittedcontents 속성 조합 시 표시 방식	783
35.	Sketch	784
35.1	Sketch 소개	784
35.1.1	사용 예	784
35.1.2	Sketch 만들기	785
35.2	Sketch 옵션 추가하기	786
35.2.1	예제	786
35.2.2	예제에서 사용한 핵심 기능	787
35.2.3	예제 구현 방법	787
36.	Spin	789
36.1	Spin 소개	789
36.1.1	사용 예	789
36.1.2	Spin 만들기	790
36.2	Spin 사용하기	790
36.2.1	예제	791

36.2.2	예제에서 사용한 핵심 기능	791
36.2.3	예제 구현 방법	792
37.	Static	793
37.1	Static 소개	793
37.1.1	사용 예	793
37.1.2	Static 만들기	794
37.2	텍스트를 동적으로 표현하기	794
37.2.1	예제	794
37.2.2	예제에서 사용한 핵심 기능	794
37.2.3	예제 구현 방법	795
37.3	자동줄넘김(wordwrap) 기능	797
37.3.1	예제	798
37.3.2	예제에서 사용한 핵심 기능	798
37.3.3	예제 구현 방법	799
37.4	텍스트 꾸미기	800
37.4.1	예제	800
37.4.2	예제에서 사용한 핵심 기능	801
37.4.3	예제 구현 방법	801
37.5	가로바 만들기	803
37.5.1	예제	804
37.5.2	예제에서 사용한 핵심 기능	804
37.5.3	예제 구현 방법	805
37.6	글자 길이에 맞추어 배치하기	806
37.6.1	예제	807
37.6.2	예제에서 사용한 핵심 기능	807
37.6.3	예제 구현 방법	808
37.7	여러줄로 문자열 표시하기	810
37.7.1	예제	811
37.7.2	예제에서 사용한 핵심 기능	811
37.7.3	예제 구현 방법	811
37.8	Bar 차트 만들기	812
37.8.1	예제	812
37.8.2	예제에서 사용한 핵심 기능	812
37.8.3	예제 구현 방법	813
37.9	아코디언 UI 만들기	816
37.9.1	예제	816
37.9.2	예제에서 사용한 핵심 기능	816
37.9.3	예제 구현 방법	817
37.10	특정 구분자를 줄바꿈 문자로 바꾸기	819

37.10.1	예제	819
37.10.2	예제에서 사용한 핵심 기능	820
37.10.3	예제 구현 방법	820
38.	TextArea	822
38.1	TextArea 소개	822
38.1.1	사용 예	822
38.1.2	TextArea 만들기	823
38.2	자동줄넘김(wordwrap) 기능	824
38.2.1	예제	824
38.2.2	예제에서 사용한 핵심 기능	824
38.2.3	예제 구현 방법	825
39.	VideoPlayer	827
39.1	VideoPlayer 소개	827
39.1.1	사용 예	827
39.1.2	VideoPlayer 만들기	828
39.2	컨트롤 기능 만들기	829
39.2.1	예제	829
39.2.2	예제에서 사용한 핵심 기능	830
39.2.3	예제 구현 방법	830
40.	WebBrowser	835
40.1	WebBrowser 소개	835
40.1.1	사용 예	836
40.1.2	WebBrowser 만들기	836
40.2	엘리먼트 속성값 가져오기	837
40.2.1	예제	837
40.2.2	예제에서 사용한 핵심 기능	838
40.2.3	예제 구현 방법	838
40.3	엘리먼트 속성값 설정하기	840
40.3.1	예제	840
40.3.2	예제에서 사용한 핵심 기능	841
40.3.3	예제 구현 방법	842
40.4	스크립트의 변수값 가져오기	843
40.4.1	예제	843
40.4.2	예제에서 사용한 핵심 기능	844
40.4.3	예제 구현 방법	845
40.5	스크립트의 함수 호출하기	846
40.5.1	예제	846
40.5.2	예제에서 사용한 핵심 기능	847

40.5.3 예제 구현 방법	847
40.6 usernotify 이벤트를 이용한 사용자 데이터 전달하기	848
40.6.1 예제	849
40.6.2 예제에서 사용한 핵심 기능	849
40.6.3 예제 구현 방법	850
40.7 innerHTML을 사용해 데이터 표현하기	851
40.7.1 예제	852
40.7.2 예제에서 사용한 핵심 기능	852
40.7.3 예제 구현 방법	852
40.8 외부 웹에디터 사용하기	853
40.8.1 예제	853
40.8.2 예제에서 사용한 핵심 기능	854
40.8.3 예제 구현 방법	854
40.9 MS 오피스 뷰어 사용하기	856
40.9.1 예제	857
40.9.2 예제에서 사용한 핵심 기능	858
40.9.3 예제 구현 방법	858

파트 IV. Container Component 859

41. Div	860
41.1 Div 소개	860
41.1.1 사용 예	860
41.1.2 Div 만들기	861
41.2 다른 화면 보여주기	862
41.2.1 예제	863
41.2.2 예제에서 사용한 핵심 기능	863
41.2.3 예제 구현 방법	864
41.3 Div Contents 접근하기	866
41.3.1 예제	866
41.3.2 예제에서 사용한 핵심 기능	866
41.3.3 예제 구현 방법	866
41.4 간단한 splitter 구현하기	868
41.4.1 예제	869
41.4.2 예제에서 사용한 핵심 기능	870
41.4.3 예제 구현 방법	870
41.5 마지막 좌표값 처리하기	871
41.5.1 예제	871
41.5.2 예제에서 사용한 핵심 기능	872

41.5.3 예제 구현 방법	872
41.6 맨 위로 가기 버튼 만들기	875
41.6.1 예제	875
41.6.2 예제에서 사용한 핵심 기능	875
41.6.3 예제 구현 방법	876
41.7 포틀릿 화면 구현하기	877
41.7.1 예제	877
41.7.2 예제에서 사용한 핵심 기능	878
41.7.3 예제 구현 방법	878
42. PopupDiv	882
42.1 PopupDiv 소개	882
42.1.1 사용 예	882
42.1.2 PopupDiv 만들기	883
42.2 PopupDiv 배치하기	884
42.2.1 예제	884
42.2.2 예제에서 사용한 핵심 기능	885
42.2.3 예제 구현 방법	885
43. Tab	887
43.1 Tab 소개	887
43.1.1 사용 예	888
43.1.2 Tab 만들기	889
43.2 멀티 라인 Tab 구성하기	890
43.2.1 예제	890
43.2.2 예제에서 사용한 핵심 기능	891
43.2.3 예제 구현 방법	891
43.3 탭 버튼 위치 조정하기	892
43.3.1 예제	892
43.3.2 예제에서 사용한 핵심 기능	893
43.3.3 예제 구현 방법	893
43.4 탭 버튼 비활성화	894
43.4.1 예제	894
43.4.2 예제에서 사용한 핵심 기능	895
43.4.3 예제 구현 방법	895
파트 V. Animation Objects	897
44. Animation	898
44.1 Animation 소개	898
44.1.1 사용 예	898

44.1.2 Animation 만들기	898
44.2 크기와 위치 변경하기	900
44.2.1 예제	900
44.2.2 예제에서 사용한 핵심 기능	900
44.2.3 예제 구현 방법	901
44.3 숫자 카운터 효과 만들기	902
44.3.1 예제	902
44.3.2 예제에서 사용한 핵심 기능	902
44.3.3 예제 구현 방법	903
44.4 easing 효과 만들기	904
44.4.1 예제	905
44.4.2 예제에서 사용한 핵심 기능	905
44.4.3 예제 구현 방법	906
44.5 순서대로 애니메이션 처리하기	909
44.5.1 예제	909
44.5.2 예제에서 사용한 핵심 기능	910
44.5.3 예제 구현 방법	910
44.6 펼쳐지는 메뉴 효과 만들기	911
44.6.1 예제	911
44.6.2 예제에서 사용한 핵심 기능	912
44.6.3 예제 구현 방법	912
45. AnimationTimeline	915
45.1 AnimationTimeline	915
45.1.1 사용 예	915
45.1.2 AnimationTimeline 만들기	916
45.2 순서대로 애니메이션 처리하기	916
45.2.1 예제	917
45.2.2 예제에서 사용한 핵심 기능	917
45.2.3 예제 구현 방법	917
파트 VI. Data Objects	919
46. Dataset	920
46.1 Dataset 소개	920
46.2 데이터 바인딩(Data Binding)	920
46.3 조건에 따라 데이터 평균, 합계 계산하기	922
46.3.1 예제	923
46.3.2 예제에서 사용한 핵심 기능	923
46.3.3 예제 구현 방법	923

46.4	컬럼 정보 확인하고 type 속성값 변경하기	925
46.4.1	예제	925
46.4.2	예제에서 사용한 핵심 기능	926
46.4.3	예제 구현 방법	926
46.5	삭제된 Row 정보 확인하기	928
46.5.1	예제	928
46.5.2	예제에서 사용한 핵심 기능	929
46.5.3	예제 구현 방법	929
46.6	스크립트로 Dataset 생성하기	932
46.6.1	예제	932
46.6.2	예제에서 사용한 핵심 기능	932
46.6.3	예제 구현 방법	932
46.7	변경된 컬럼정보 원복하기	934
46.7.1	예제	934
46.7.2	예제에서 사용한 핵심 기능	934
46.7.3	예제 구현 방법	934
46.8	RowType 변경하기	936
46.8.1	예제	936
46.8.2	예제에서 사용한 핵심 기능	937
46.8.3	예제 구현 방법	937
46.9	대소문자 구분 없이 필터링하기	939
46.9.1	예제	939
46.9.2	예제에서 사용한 핵심 기능	939
46.9.3	예제 구현 방법	939
46.10	그룹화된 데이터와 관련된 계산값 구하기	941
46.10.1	예제	941
46.10.2	예제에서 사용한 핵심 기능	941
46.10.3	예제 구현 방법	942
47.	ExcelExportObject / ExcellImportObject	944
47.1	nexacro-xeni 소개	944
47.2	ExcelExportObject 소개	945
47.3	ExcellImportObject 소개	946
47.4	Grid를 엑셀 파일로 내보내기(익스포트)	947
47.4.1	예제	948
47.4.2	예제에서 사용한 핵심 기능	949
47.4.3	예제 구현 방법	949
47.5	엑셀 파일 읽어서 Grid로 보여주기(임포트)	952
47.5.1	예제	952
47.5.2	예제에서 사용한 핵심 기능	953

47.5.3	예제 구현 방법	954
47.6	Grid의 이미지를 엑셀 파일로 내보내기	955
47.6.1	예제	955
47.6.2	예제에서 사용한 핵심 기능	956
47.6.3	예제 구현 방법	956
47.7	여러 Grid 데이터를 엑셀 파일로 내보내기	959
47.7.1	예제	959
47.7.2	예제에서 사용한 핵심 기능	961
47.7.3	예제 구현 방법	961
48.	DataObject	965
48.1	DataObject 오브젝트 소개	965
48.2	간단한 데이터 표시하기	965
48.2.1	예제	966
48.2.2	예제에서 사용한 핵심 기능	966
48.2.3	예제 구현 방법	966
	데이터를 가져올 url 설정하기	966
	Dataset 오브젝트와 DataObject 오브젝트 연결하기	967
	Grid 컴포넌트에 데이터 표시하기	969
48.3	다국어로 Button 라벨 표시하기	970
48.3.1	예제	970
48.3.2	예제에서 사용한 핵심 기능	970
48.3.3	예제 구현 방법	970
	JSON 데이터 작성하기	970
	Combo 컴포넌트에 표시할 기준 데이터 가져오기	973
	Button 컴포넌트에 연결되는 라벨 데이터 가져오기	974
48.4	메소드 파라미터 지정해서 request 메소드 실행하기	976
48.4.1	예제	976
48.4.2	예제에서 사용한 핵심 기능	977
48.4.3	예제 구현 방법	977
	화면 구성하기	977
	request 메소드 준비하기	977
	이벤트 처리하기	978
	실행 결과 확인하기	980
파트 VII.	넥사크로 오브젝트	982
49.	nexacro.Decimal	983
49.1	nexacro.Decimal 오브젝트 소개	983
49.2	Grid 컴포넌트에서 BIGDECIMAL 타입 사용하기	983

49.2.1	예제	984
49.2.2	예제에서 사용한 핵심 기능	984
49.2.3	예제 구현 방법	984
49.3	1경(京)보다 큰 값 연산하기	986
49.3.1	예제	986
49.3.2	예제에서 사용한 핵심 기능	987
49.3.3	예제 구현 방법	987
50.	nexacro.DomParser	989
50.1	nexacro.DomParser 오브젝트 소개	989
50.2	XML 형식의 Open API 연결해서 Grid 컴포넌트에 표시하기	989
50.2.1	예제	989
50.2.2	예제에서 사용한 핵심 기능	990
50.2.3	예제 구현 방법	990

파트 VIII. 자바스크립트 표준 내장 객체 995

51.	Array	996
51.1	문자열을 분리해 배열로 만들기	996
51.1.1	예제	996
51.1.2	예제에서 사용한 핵심 기능	996
51.1.3	예제 구현 방법	997
51.2	중복된 요소 제거하기	998
51.2.1	예제	998
51.2.2	예제에서 사용한 핵심 기능	998
51.2.3	예제 구현 방법	998
51.3	숫자 배열 정렬하기	1,000
51.3.1	예제	1,000
51.3.2	예제에서 사용한 핵심 기능	1,000
51.3.3	예제 구현 방법	1,001
51.4	2차원 배열 만들기	1,001
51.4.1	예제	1,002
51.4.2	예제에서 사용한 핵심 기능	1,002
51.4.3	예제 구현 방법	1,002
52.	Date	1,004
52.1	타이머 구현하기	1,004
52.1.1	예제	1,004
52.1.2	예제에서 사용한 핵심 기능	1,004
52.1.3	예제 구현 방법	1,005
52.2	입력된 날짜에서 정보 구하기	1,007

52.2.1	예제	1,007
52.2.2	예제에서 사용한 핵심 기능	1,007
52.2.3	예제 구현 방법	1,007
53.	Function	1,010
53.1	DECODE, IIF 기능 구현하기	1,010
53.1.1	예제	1,010
53.1.2	예제에서 사용한 핵심 기능	1,011
53.1.3	예제 구현 방법	1,011
54.	Global	1,014
54.1	RGB 색상 코드 변경하기	1,014
54.1.1	예제	1,014
54.1.2	예제에서 사용한 핵심 기능	1,015
54.1.3	예제 구현 방법	1,015
54.2	URI 변환하기	1,017
54.2.1	예제	1,017
54.2.2	예제에서 사용한 핵심 기능	1,017
54.2.3	예제 구현 방법	1,018
54.3	유니코드 변환하기	1,018
54.3.1	예제	1,019
54.3.2	예제에서 사용한 핵심 기능	1,019
54.3.3	예제 구현 방법	1,019
54.4	function 찾아서 실행하기	1,020
54.4.1	예제	1,020
54.4.2	예제에서 사용한 핵심 기능	1,020
54.4.3	예제 구현 방법	1,020
55.	Math	1,022
55.1	백분율 구하기	1,022
55.1.1	예제	1,022
55.1.2	예제에서 사용한 핵심 기능	1,022
55.1.3	예제 구현 방법	1,023
56.	Number	1,025
56.1	숫자 범위 체크하기	1,025
56.1.1	예제	1,025
56.1.2	예제에서 사용한 핵심 기능	1,025
56.1.3	예제 구현 방법	1,026
57.	String	1,028
57.1	영문 자판으로 한글 입력 시 자동 전환하기	1,028
57.1.1	예제	1,028

57.1.2	예제에서 사용한 핵심 기능	1,029
57.1.3	예제 구현 방법	1,029
57.2	이메일 주소값 검증하기	1,032
57.2.1	예제	1,032
57.2.2	예제에서 사용한 핵심 기능	1,032
57.2.3	예제 구현 방법	1,033
57.3	전각문자와 반각문자 변환하기	1,034
57.3.1	예제	1,034
57.3.2	예제에서 사용한 핵심 기능	1,034
57.3.3	예제 구현 방법	1,035
57.4	문자열이 지정된 길이가 되도록 다른 문자로 채우기	1,037
57.4.1	예제	1,037
57.4.2	예제에서 사용한 핵심 기능	1,037
57.4.3	예제 구현 방법	1,038
57.5	문자열에 있는 하이픈(-) 제거하기	1,038
57.5.1	예제	1,038
57.5.2	예제에서 사용한 핵심 기능	1,039
57.5.3	예제 구현 방법	1,039
57.6	주민등록번호 유효성 검사하기	1,040
57.6.1	예제	1,040
57.6.2	예제에서 사용한 핵심 기능	1,040
57.6.3	예제 구현 방법	1,040
57.7	법인등록번호 유효성 검사하기	1,043
57.7.1	예제	1,044
57.7.2	예제에서 사용한 핵심 기능	1,044
57.7.3	예제 구현 방법	1,044
57.8	NULL 값 체크하기	1,045
57.8.1	예제	1,045
57.8.2	예제에서 사용한 핵심 기능	1,046
57.8.3	예제 구현 방법	1,046
57.9	앞뒤 공백문자 제거하기	1,048
57.9.1	예제	1,048
57.9.2	예제에서 사용한 핵심 기능	1,048
57.9.3	예제 구현 방법	1,049
57.10	숫자 형식 여부 확인하기	1,050
57.10.1	예제	1,050
57.10.2	예제에서 사용한 핵심 기능	1,050
57.10.3	예제 구현 방법	1,050
57.11	영문이나 숫자 여부 확인하기	1,052

57.11.1 예제	1,052
57.11.2 예제에서 사용한 핵심 기능	1,052
57.11.3 예제 구현 방법	1,053
57.12 날짜, 시간 유효성 확인하기	1,054
57.12.1 예제	1,054
57.12.2 예제에서 사용한 핵심 기능	1,054
57.12.3 예제 구현 방법	1,055

1.

컴포넌트 활용 워크북 소개

컴포넌트 활용 워크북은 초급부터 중급개발자까지 넥사크로플랫폼의 기본 활용법과 각 오브젝트 또는 컴포넌트의 사용법을 안내하고 있습니다.



개발도구(넥사크로 스튜디오) 사용법은 [\[개발도구 가이드\]](#)를 참고해주세요.

1.1 전체 예제가 포함된 프로젝트 코드(압축파일) 내려받기

전체 예제가 포함된 프로젝트 코드(ZIP)는 아래 링크에서 내려받을 수 있습니다.

https://github.com/TOBESOFT-DOCS/sample_nexacroplatform_17

https://github.com/TOBESOFT-DOCS/sample_nexacroplatform_17/archive/master.zip



전체 예제가 포함된 프로젝트 코드(ZIP)를 내려받아 실행하는 경우 네트워크가 연결되어있지 않다면 매뉴얼 표시 영역이 빈 공간으로 보일 수 있습니다. 예제에 대한 가이드는 온라인 상태에서 확인할 수 있습니다.

1.2 컴포넌트 활용 워크북 활용 안내

컴포넌트 활용 워크북에서 제공하는 콘텐츠는 실제 실행할 수 있는 예제와 샘플 코드를 제공합니다.

- 웹브라우저에서 동작하는 예제만 제공하고 있습니다.
- 데스크탑에서만 동작하는 기능에 대해서는 컴포넌트 활용 워크북에서 다루고 있지 않습니다.
- 모바일 위주 예제는 [Device API 활용 워크북]를 참고해주세요.

① [예제] 항목 아래에서 링크를 확인합니다. 링크를 클릭하면 브라우저의 새 탭이 열립니다.

20.2.1 예제

다음 Grid는 Salary 컬럼의 합계를 구하고 Company 컬럼으로 그룹을 지어 그룹별 소계를 계산한 예제입니다.

합계는 Grid의 가장 하단에 표시되고 소계는 각 그룹의 아래에 새로운 Row가 생성되어 표시됩니다.

Name	Address	Company	Department	Salary	
John	Seoul, KOREA	1dollar	Sales	₩15,000	1dollar group
Lisa	London, United Kingdom		Quality Assurance	₩3,000	
Michael	Mumbai, India		Personnel	₩1,300	
				₩19,300	
James	Barcelona, Spain	EBOT	Consulting	₩25,000	EBOT group
Donald	Califonia, USA		Research Institute	₩1,000,000	
				₩1,025,000	
Joe	São Paulo, Brazil	hangul	R&D	₩8,600	hangul group
Lilli	Ankara, Turkey		Quality Assurance	₩4,400	
Colt	Sydney, NSW, Australia		Sales	₩62,000	
				₩75,000	
TOTAL				₩1,119,300	



② 새 탭에서는 예제 설명(화면 왼쪽)과 실제 동작하는 예제(화면 오른쪽)를 보여줍니다.

실제 동작하는 예제는 디자인 테마 설정에 따라 예제 설명과 다르게 보일 수 있습니다. 디자인 테마는 제품에서 기본 제공되는 테마를 사용합니다.

본 예제는 17.0.0.50 버전에서 테스트되었습니다.

예제 설명

다음 Grid는 Salary 컬럼의 합계를 구하고 Company 컬럼으로 그룹을 지어 그룹별 소계를 계산한 예제입니다.

합계는 Grid의 가장 하단에 표시되고 소계는 각 그룹의 아래에 새로운 Row가 생성되어 표시됩니다.

Name	Address	Company	Department	Salary
John	Seoul, KOREA	1dollar	Sales	₩15,000
Lisa	London, United Kingdom		Quality Assurance	₩3,000
Michael	Mumbai, India		Personnel	₩1,300
				₩19,300
James	Barcelona, Spain	EBOT	Consulting	₩25,000
Donald	Califonia, USA		Research Institute	₩1,000,000
				₩1,025,000
Joe	São Paulo, Brazil	hangul	R&D	₩8,600
Lilli	Ankara, Turkey		Quality Assurance	₩4,400
Colt	Sydney, NSW, Australia		Sales	₩62,000
				₩75,000
TOTAL				₩1,119,300

예제 소스 소스

Name	Address	Company	Department	Salary
John	Seoul, KOREA	1dollar	Sales	₩15,000
Lisa	London, United Kingdom		Quality Assurance	₩3,000
Michael	Mumbai, India		Personnel	₩1,300
				₩19,300
James	Barcelona, Spain	EBOT	Consulting	₩25,000
Donald	Califonia, USA		Research Institute	₩1,000,000
				₩1,025,000
Joe	São Paulo, Brazil	hangul	R&D	₩8,600
Lilli	Ankara, Turkey		Quality Assurance	₩4,400
Colt	Sydney, NSW, Australia		Sales	₩62,000
				₩75,000
TOTAL				₩1,119,300

파트 I.

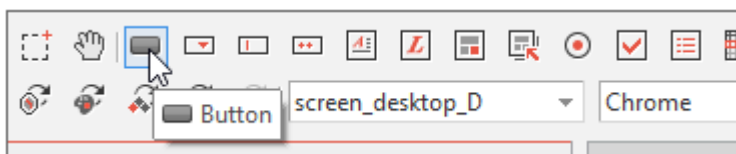
기본 활용

2.

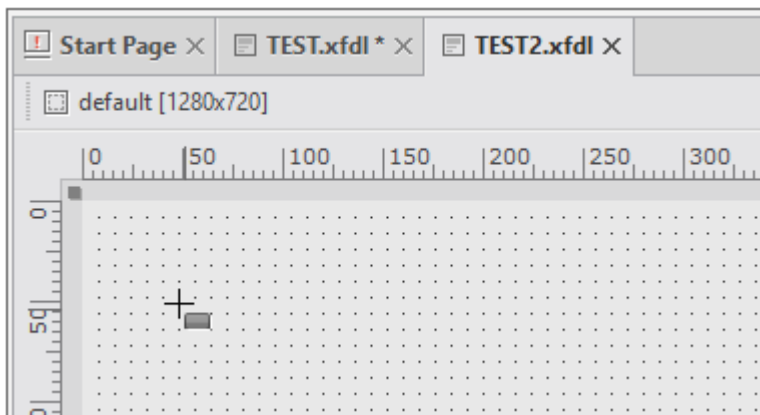
컴포넌트 배치

2.1 툴바에서 컴포넌트 아이콘 선택하고 배치하기

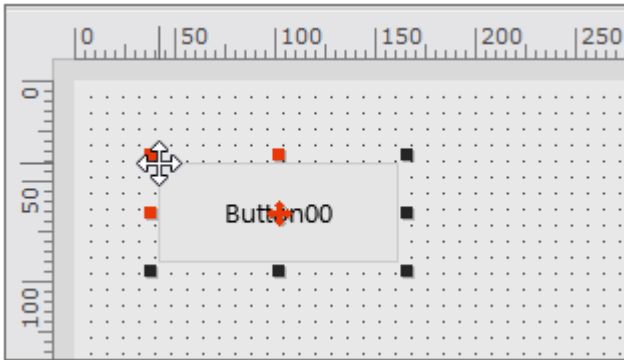
- 1 컴포넌트 툴바에서 원하는 컴포넌트를 선택합니다. 컴포넌트 아이콘 위에 마우스를 가져가면 컴포넌트 이름이 툴팁으로 표시됩니다. 원하는 컴포넌트를 찾아서 마우스로 클릭하면 선택된 상태가 됩니다.



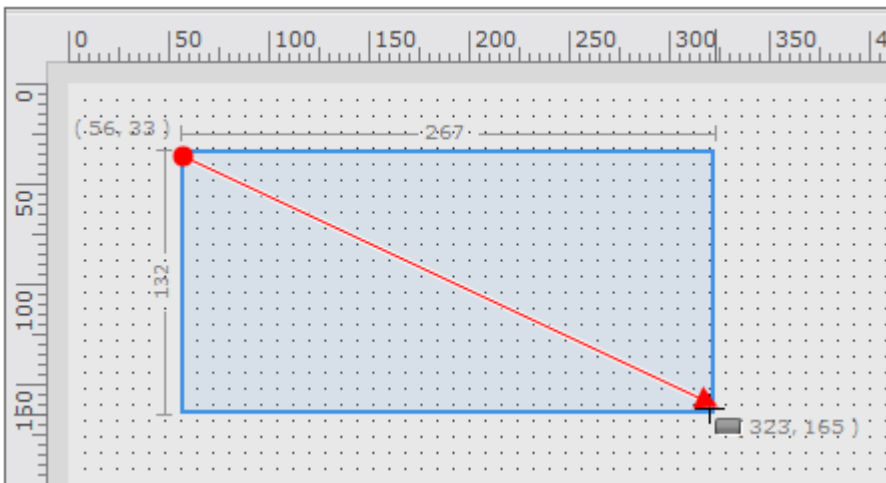
컴포넌트 툴바에서 컴포넌트를 선택한 상태에서는 디자인 화면 내에서 마우스 포인터가 해당 컴포넌트 아이콘으로 변경됩니다.



- 2 디자인 화면 내 원하는 위치 위에서 마우스를 클릭합니다. 마우스를 클릭한 지점이 컴포넌트의 좌측 상단 모서리 시작점이 됩니다. 마우스를 클릭하면 해당 컴포넌트의 기본 크기로 화면에 배치됩니다.



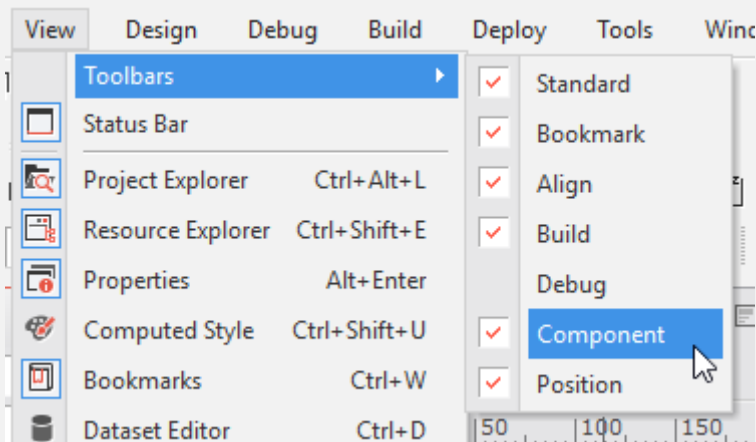
마우스를 클릭하지 않고 왼쪽 버튼을 누른 상태에서 원하는 방향으로 마우스를 드래그하면 해당 방향으로 컴포넌트의 크기를 조절할 수 있습니다. 컴포넌트를 배치하는 동시에 컴포넌트의 크기를 조절할 수 있습니다.



- ③ 컴포넌트를 화면에 배치하면 선택된 상태가 됩니다. 컴포넌트를 선택한 상태에서 컴포넌트의 크기를 바꾸거나 위치를 변경할 수 있고 속성창에서 컴포넌트의 속성값을 수정할 수 있습니다.

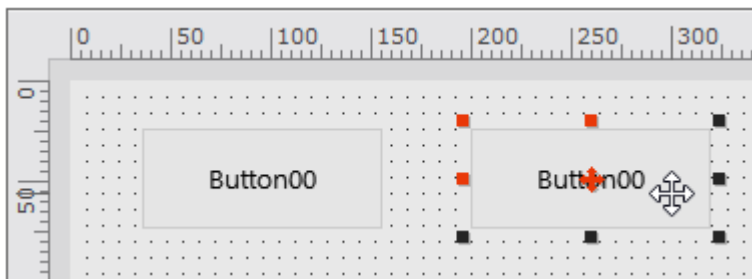


컴포넌트 툴바가 보이지 않는다면 메뉴[View > Toolbars > Component] 항목이 체크되어 있는지 확인하세요. 최초 설치 시에는 체크되어 있지만 작업 중 해당 항목 체크를 취소한 경우에는 컴포넌트 툴바가 보이지 않습니다.

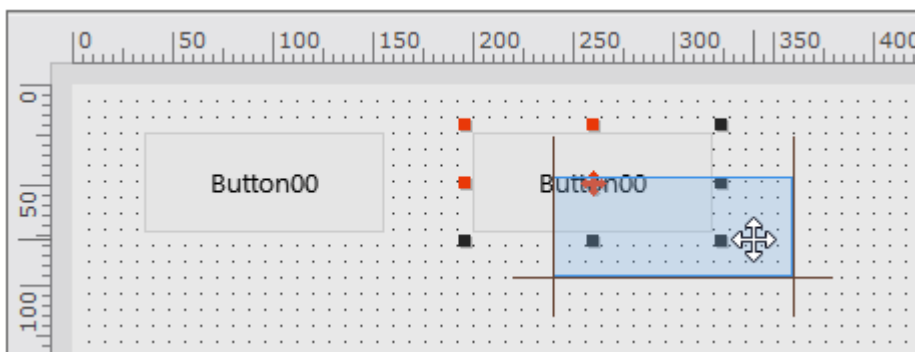


2.2 컴포넌트 선택하고 위치 변경하기

- ① 디자인 화면에 배치된 컴포넌트를 클릭합니다. 컴포넌트는 선택된 상태로 변경됩니다. 컴포넌트가 선택된 상태에서는 크기를 변경할때 사용하는 트래커(검은색 또는 빨간색 점)이 표시되며 컴포넌트 위에 마우스포인터를 가져가면 (+) 모양으로 마우스 포인터가 변경됩니다.



- ② 선택된 컴포넌트 위에 마우스 포인터를 위치시키고 마우스 왼쪽 버튼을 누른 상태에서 원하는 방향으로 이동합니다. 마우스 포인터를 이동하는 중에는 원래 컴포넌트는 그대로 보여지고 같은 크기의 이미지 그림자가 마우스 포인터를 따라 이동합니다.



- ③ 마우스 왼쪽 버튼에서 손을 떼면 컴포넌트가 원하는 위치로 옮겨집니다.



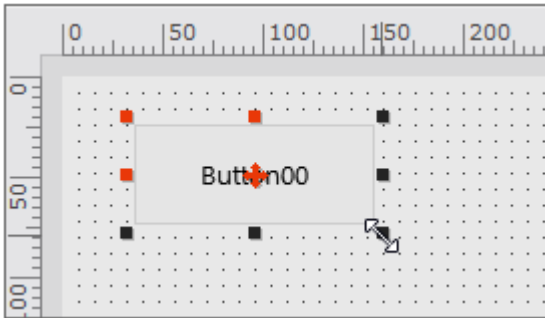
마우스 왼쪽 버튼을 누른 상태에서 **Esc** 키를 입력하면 컴포넌트 이동 작업이 취소됩니다.



컴포넌트를 선택한 후 속성창에서 Position 관련 속성값을 수정해서 컴포넌트의 위치를 변경할 수 있습니다.

2.3 컴포넌트 선택하고 크기 변경하기

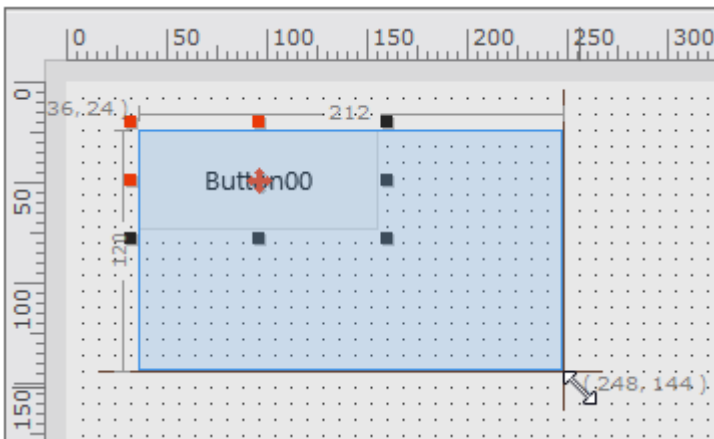
- ① 디자인 화면에 배치된 컴포넌트를 클릭합니다. 컴포넌트는 선택된 상태로 변경됩니다. 컴포넌트가 선택된 상태에서는 크기를 변경할때 사용하는 트래커(검은색 또는 빨간색 점)이 표시됩니다. 트래커 위에 마우스포인터를 가져가면 각 트래커에 해당하는 방향 모양으로 마우스 포인터가 변경됩니다.



각 모서리에 표시된 트래커를 선택한 경우에는 수직, 수평, 대각선 방향으로 크기를 변경할 수 있습니다.

오른쪽이나 왼쪽면에 표시된 트래커를 선택한 경우에는 수평 방향으로만 크기를 변경할 수 있고 위, 아래쪽에 표시된 트래커를 선택한 경우에는 수직 방향으로만 크기를 변경할 수 있습니다.

- ② 트래커 위에 마우스 포인터를 위치시키고 마우스 왼쪽 버튼을 누른 상태에서 원하는 방향으로 이동합니다. 마우스 포인터를 이동하는 중에는 원래 컴포넌트는 그대로 보여지고 마우스 포인터의 이동 거리만큼 변경된 크기의 이미지 그림자가 마우스 포인터를 따라 보여집니다.



- ③ 마우스 왼쪽 버튼에서 손을 떼면 컴포넌트가 원하는 크기로 변경됩니다.



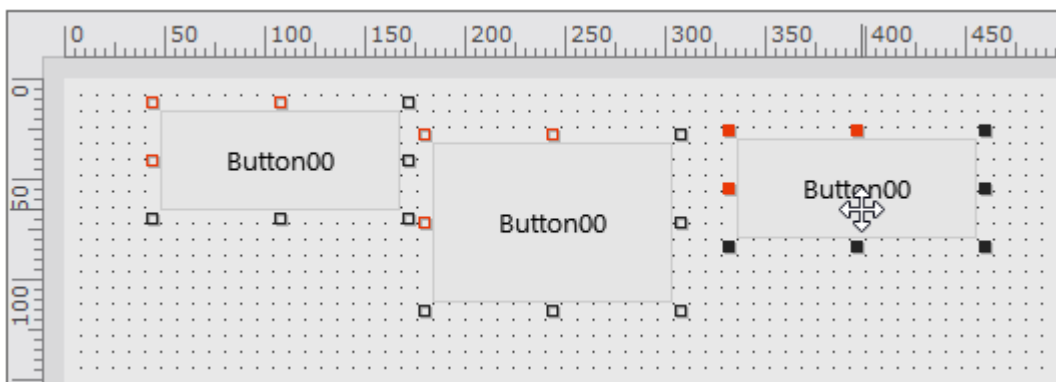
마우스 왼쪽 버튼을 누른 상태에서 **Esc** 키를 입력하면 컴포넌트 크기 변경 작업이 취소됩니다.



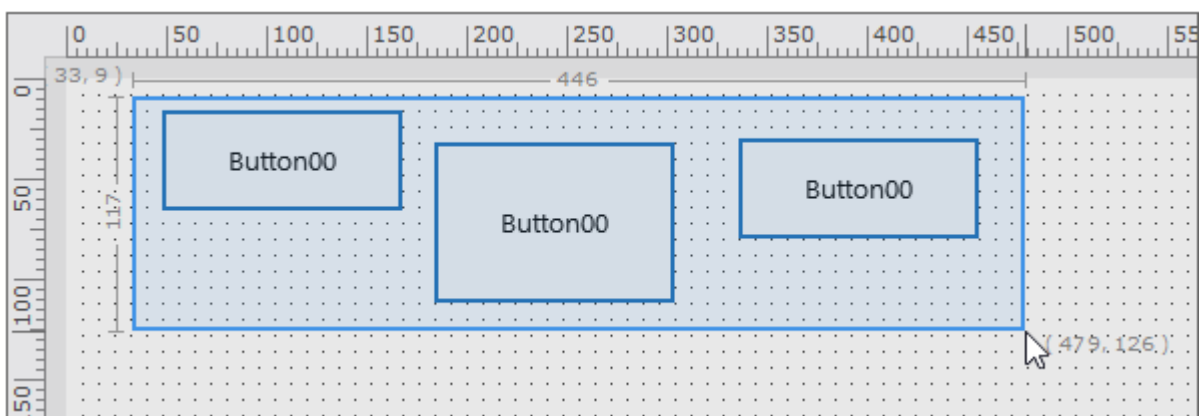
컴포넌트를 선택한 후 속성창에서 Position 관련 속성값을 수정해서 컴포넌트의 크기를 변경할 수 있습니다.

2.4 2개 이상 컴포넌트 선택하고 위치나 크기 변경하기

- 1 2개 이상 컴포넌트를 선택합니다. **Ctrl** 또는 **Shift** 키를 누른 상태에서 컴포넌트를 클릭하면 2개 이상의 컴포넌트를 선택할 수 있습니다.

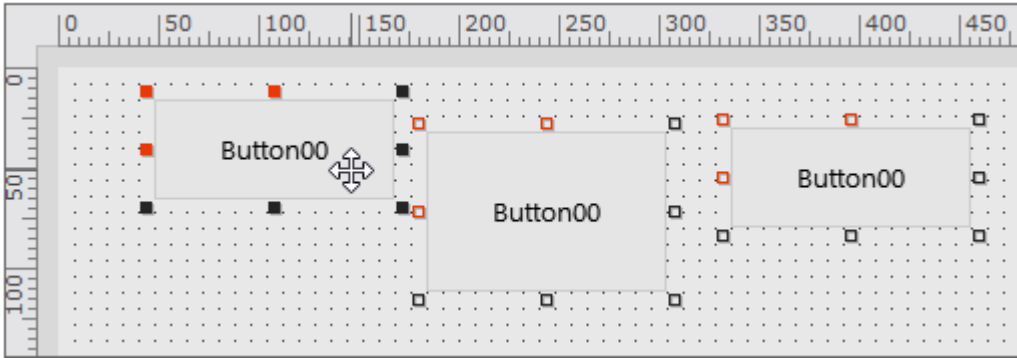


컴포넌트 영역 밖에서 마우스 왼쪽 버튼을 누른 상태에서 컴포넌트를 감싸는 형태로 마우스를 드래그해서 컴포넌트를 선택할 수도 있습니다.



- 2 기준이 되는 컴포넌트를 선택합니다. 2개 이상의 컴포넌트를 선택하면 트래커가 채워져 있는 컴포넌트도 있고 트래커가 투명한 사각형으로만 표시되는 컴포넌트도 있습니다. 트래커가 채워져 있는 컴포넌트가 기준이 되는 컴포넌트입니다. 컴포넌트가 선택된 상태에서 원하는 컴포넌트를 클릭하면 기준이 되는 컴포넌트가 변경됩니

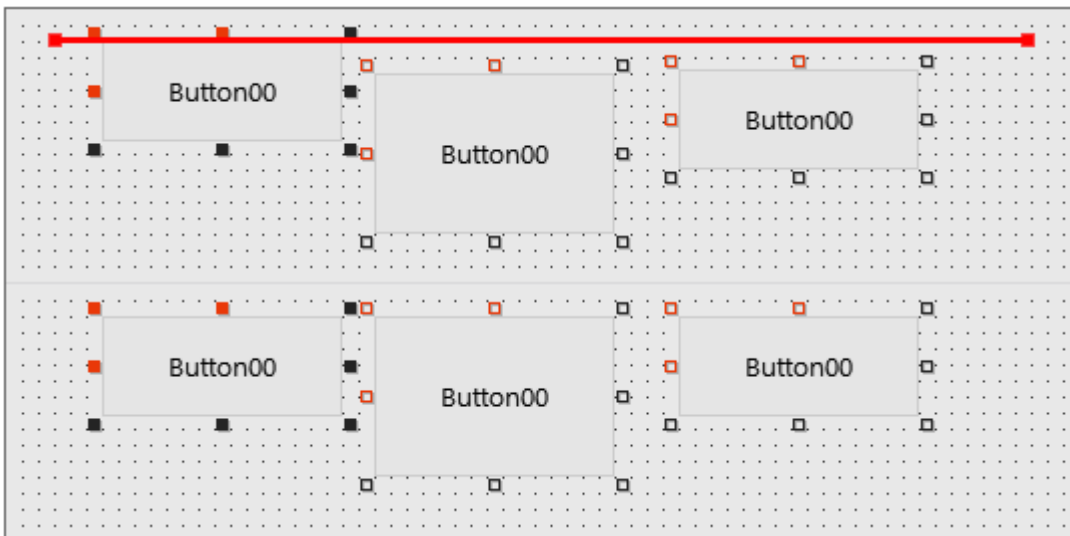
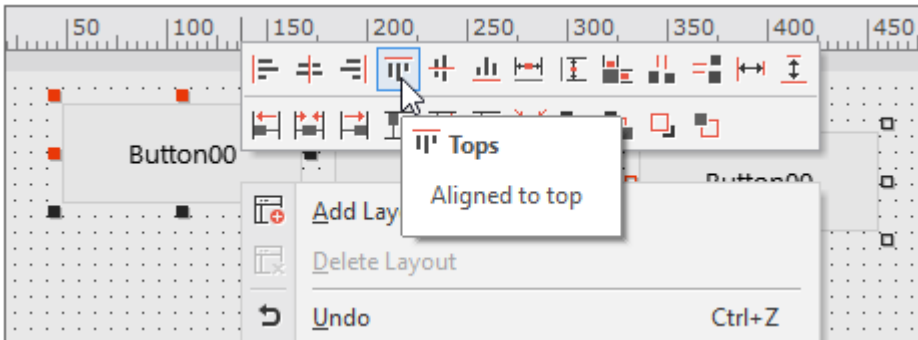
다.



- ③ 정렬 툴바에서 원하는 기능을 선택하거나 컴포넌트 위에 마우스 포인터를 올린 상태에서 왼쪽 마우스 버튼을 클릭하면 표시되는 미니 정렬 툴바에서 원하는 기능을 선택합니다.

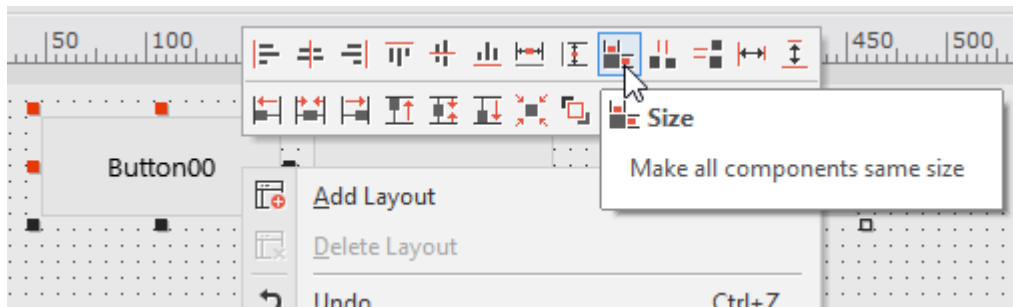
이번 예제에서는 첫 번째 버튼을 기준으로 컴포넌트의 Y 좌표값을 정렬하고 컴포넌트의 크기를 변경합니다.

- ④ Y 좌표값을 먼저 정렬합니다. 미니 정렬 툴바에서 [Tops] 아이콘을 선택합니다. 첫 번째 컴포넌트의 Y 좌표값을 기준으로 나머지 컴포넌트의 위치를 조절합니다.

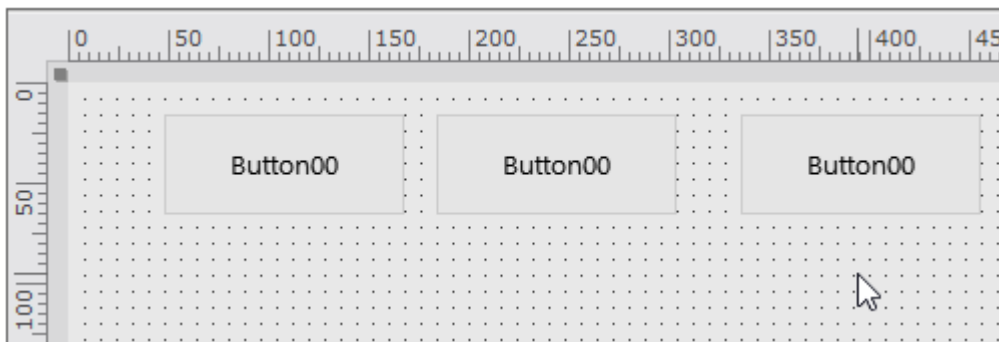


- ⑤ 컴포넌트의 크기를 조절합니다. 미니 정렬 툴바에서 [Size] 아이콘을 선택합니다. 첫 번째 컴포넌트의 크기를

기준으로 나머지 컴포넌트의 크기를 조절합니다.

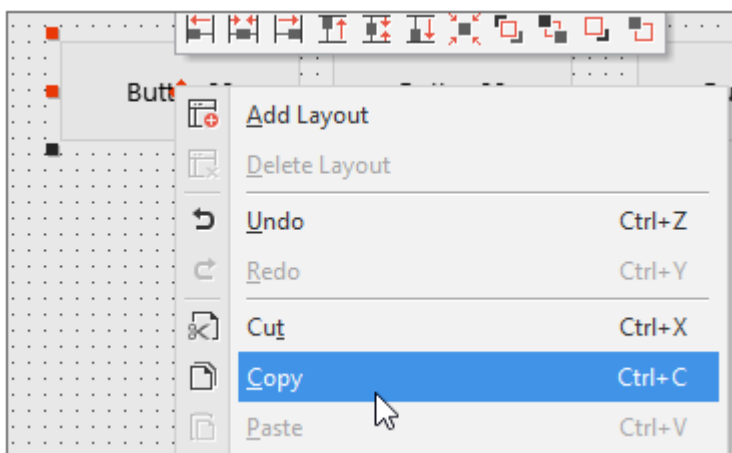


- ⑥ Esc 키를 입력하거나 디자인 화면에서 다른 영역을 클릭하면 선택 상태는 해제됩니다.



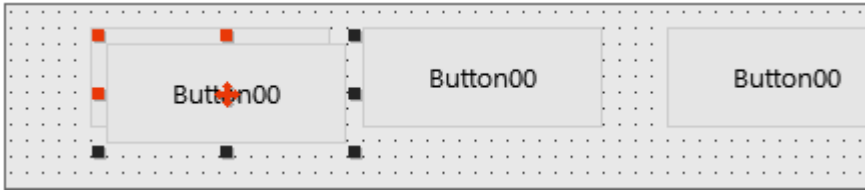
2.5 컴포넌트 복사하고 붙여넣기

- ① 컴포넌트를 선택하고 **Ctrl+C** 키를 입력하거나 왼쪽 마우스를 클릭해 표시되는 메뉴에서 [Copy] 항목을 선택합니다. 선택한 컴포넌트가 클립보드에 복사됩니다.



- ② **Ctrl+V** 키를 입력하거나 왼쪽 마우스를 클릭해 표시되는 메뉴에서 [Paste] 항목을 선택합니다. 클립보드에 복

사했던 컴포넌트를 화면에 추가합니다.



- 3 붙여넣기해서 화면에 표시된 컴포넌트는 복사한 컴포넌트 위치의 오른쪽 아래쪽에 추가됩니다. 추가된 컴포넌트를 선택하고 원하는 위치로 이동하거나 크기를 변경합니다.

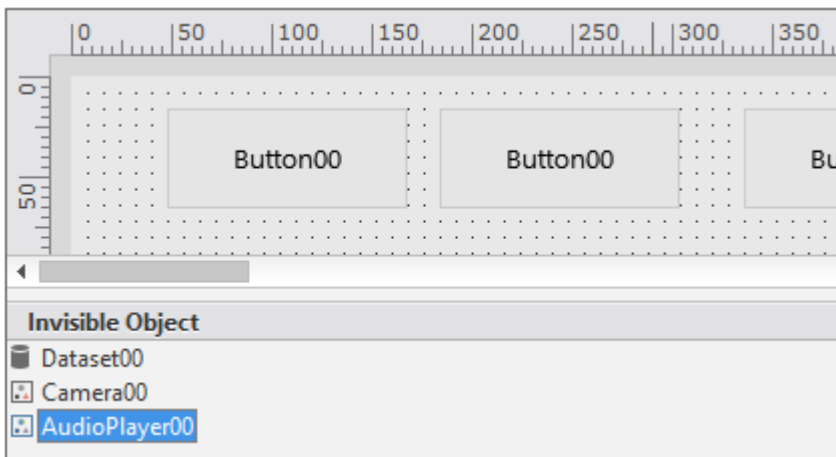


2개 이상 컴포넌트를 선택한 후 복사해서 붙여넣기할 수 있습니다.

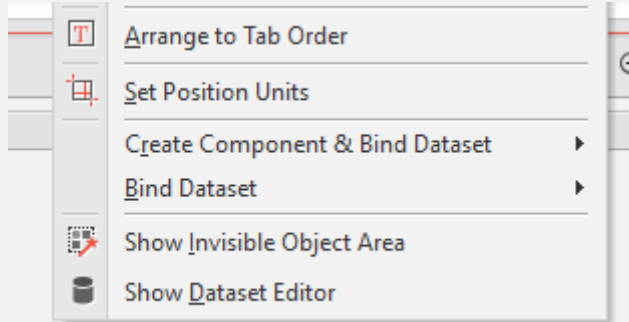
2.6 보이지 않는 컴포넌트 (오브젝트)

컴포넌트 툴바에서 아이콘을 선택해 디자인 화면에 배치해도 보이지 않는 컴포넌트가 있습니다. 애플리케이션에서 특정 기능을 제공하지만 디자인 화면에 표시되지 않고 애플리케이션 실행 시에도 실행 화면에 나타나지 않습니다.

데이터를 테이블 형태로 저장하는 Dataset 오브젝트나 모바일 기기에서 사용하는 DeviceAPI 오브젝트가 보이지 않는 컴포넌트입니다. 보이지 않는 컴포넌트를 선택하고 디자인 화면에 추가하면 별도의 [Invisible Object] 영역에 표시됩니다. 보이지 않는 컴포넌트는 Position 속성을 가지지 않습니다.



[Invisible Object] 영역이 보이지 않는다면 디자인 화면에서 오른쪽 마우스 버튼을 클릭해서 표시되는 메뉴 항목 중 [Show Invisible Object Area] 항목을 선택하거나 메뉴[Design > Show Invisible Object Area] 항목을 선택합니다.



2.7 비활성화되어 있는 컴포넌트 사용하기

넥사크로 스튜디오를 설치하면 기본적으로 사용할 수 있는 UI 관련 컴포넌트 이외에 비활성화되어 감춰져 있는 컴포넌트가 있습니다. 이는 대부분 특정 기능을 수행하는 용도의 컴포넌트로 WebBrowser, VideoPlayer, Sketch, GoogleMap, ExcelExport/Import 등을 비롯하여 약 서른 개 정도가 제공되며 사용자의 필요에 따라 간단한 등록 절차만으로 사용할 수 있습니다.

이 컴포넌트가 처음부터 사용할 수 있도록 활성화되어 있지 않은 이유는 불필요한 컴포넌트의 추가로 프로젝트 코드가 커지는 것을 방지하기 위함입니다. 따라서 사용자는 꼭 필요한 컴포넌트만 추가하여 사용하는 것이 좋습니다.

비활성화되어 있는 컴포넌트를 사용 등록하여 활성화 상태로 만들려면 다음과 같은 절차를 따릅니다. 이 설명에서는 GoogleMap 컴포넌트를 예로 설명합니다.

① Project Explorer에서 [TypeDefinition > Objects]를 선택하고 마우스 오른쪽 버튼을 클릭한 후 컨텍스트 메뉴에서 [Edit] 항목을 선택해 오브젝트 편집 창을 엽니다.

② Objects 테이블에서 하단의 + 버튼을 클릭하여 목록에서 nexacro.GoogleMap을 선택합니다.

아이콘은 컴포넌트에 알맞은 모양으로 선택하고 나머지는 기본 값을 사용합니다.

Regist	Component Name	Icon	Width	Height	Regist
-	Dataset	nexacro.NormalDataset	21	0	Dataset
-	WebBrowser	nexacro.WebBrowser	59	0	WebBrowser
-	Sketch	nexacro.Sketch	50	0	Sketch
-	GoogleMap	nexacro.GoogleMap	66	300	GoogleMap
+					



Objects 테이블의 각 컴포넌트명 옆에 있는 - 버튼을 클릭하면 해당 컴포넌트가 프로젝트에서 제거되어 사용할 수 없게됩니다.

- 3 등록이 완료되면 OK 버튼을 클릭합니다. 그러면 프로젝트가 자동으로 제너레이트 되고 툴바에 새로 추가한 GoogleMap 컴포넌트 아이콘이 나타납니다.

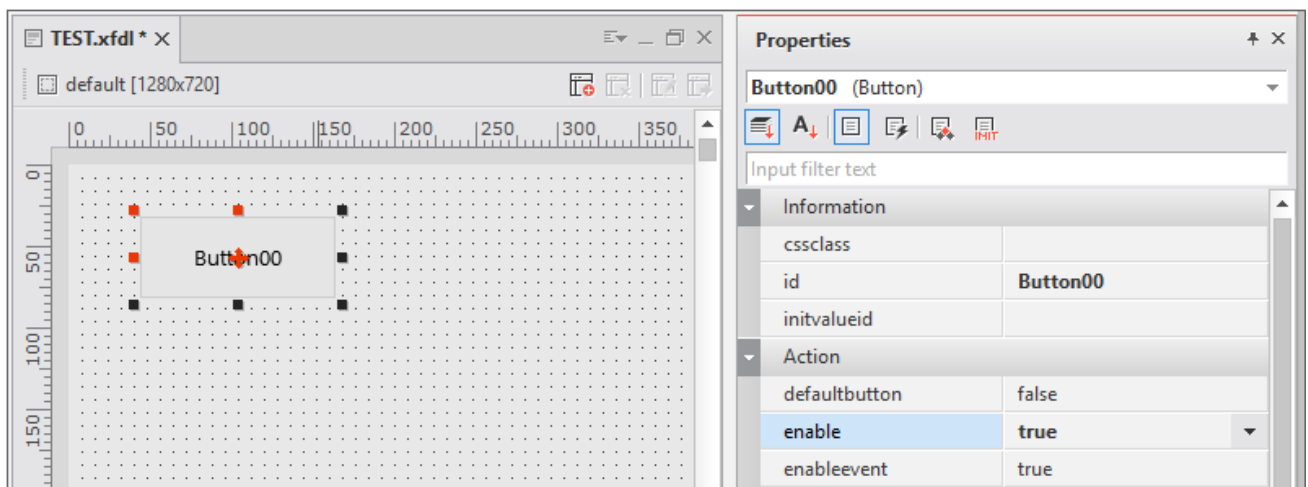


3.

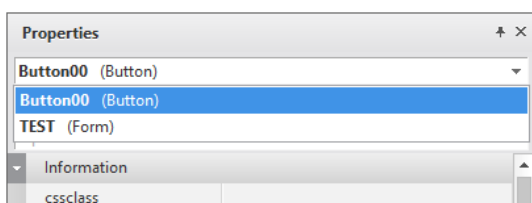
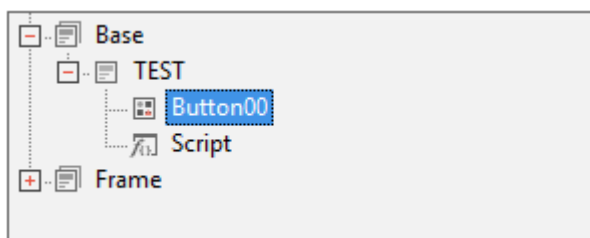
속성값 설정

3.1 선택한 컴포넌트를 속성창에 표기하기

디자인 화면에 배치된 컴포넌트를 선택하면 속성창에 선택한 컴포넌트의 속성값이 표시됩니다. 속성창에서는 속성값을 확인하거나 수정할 수 있습니다.



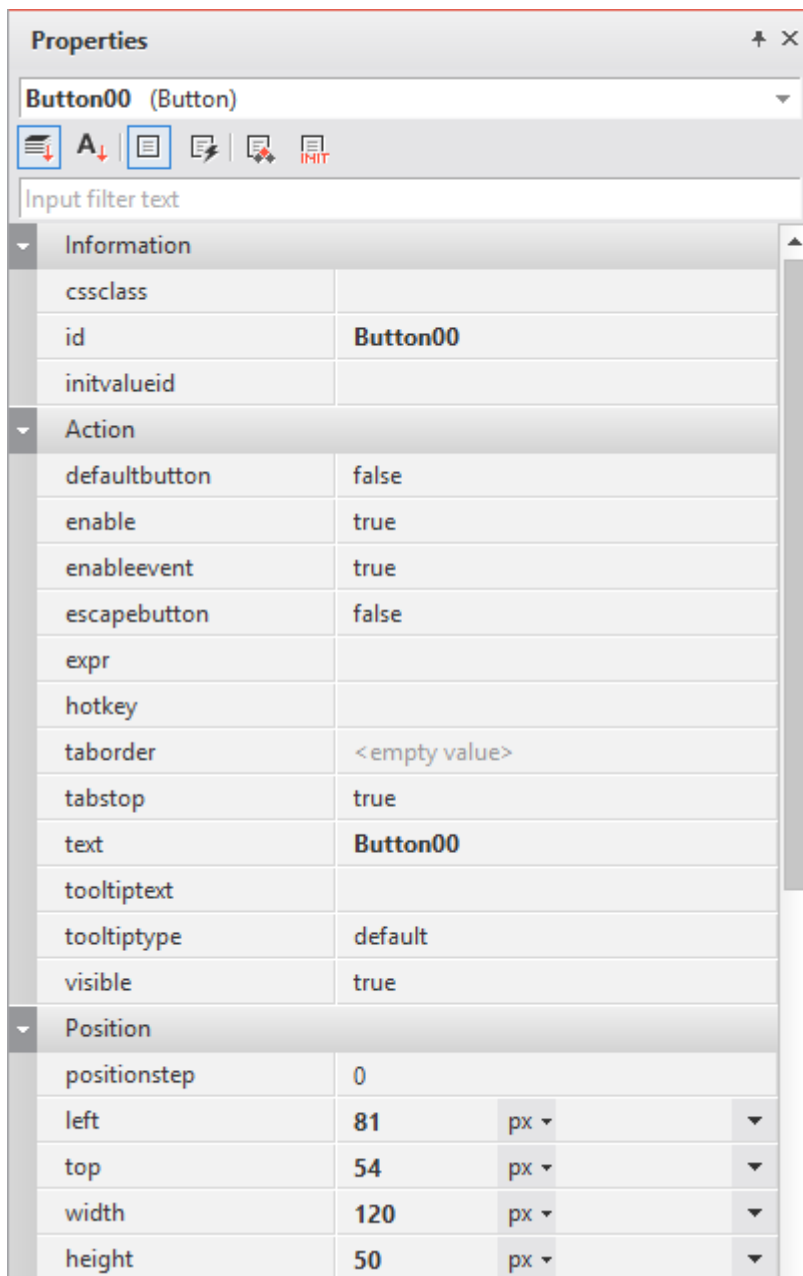
디자인 화면에서 컴포넌트를 선택하지 않더라도 [Project Explorer]에서 해당 컴포넌트를 선택하거나 속성창 목록에서 원하는 컴포넌트를 선택할 수 있습니다.



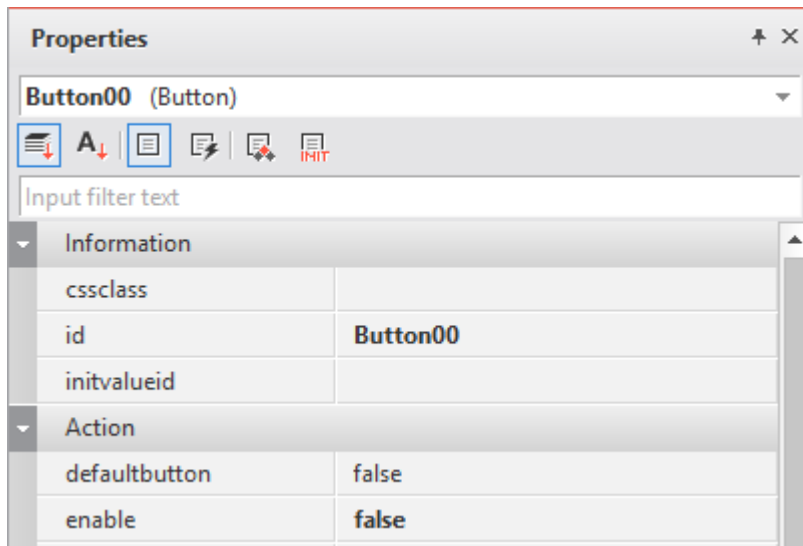
3.2 선택한 컴포넌트의 기본값 확인하기

컴포넌트를 디자인 화면에 처음 배치하면 기본값으로 속성값이 설정됩니다. 컴포넌트를 선택한 상태에서 넥사크로 스튜디오 오른쪽에 표시되는 속성창에서 컴포넌트의 속성값을 확인할 수 있습니다.

컴포넌트의 기본값을 수정한 경우에는 굵은 글꼴로 속성값이 표시됩니다. 디자인 화면에 처음 배치된 상태에서 id, text, left, top, width, height 속성값이 굵은 글꼴로 표시되는 것은 컴포넌트를 배치하는 과정에서 id, text 속성값이 지정되고 Position 관련 속성값이 배치된 위치에 따라 지정되기 때문에 굵은 글꼴로 속성값이 표시됩니다.

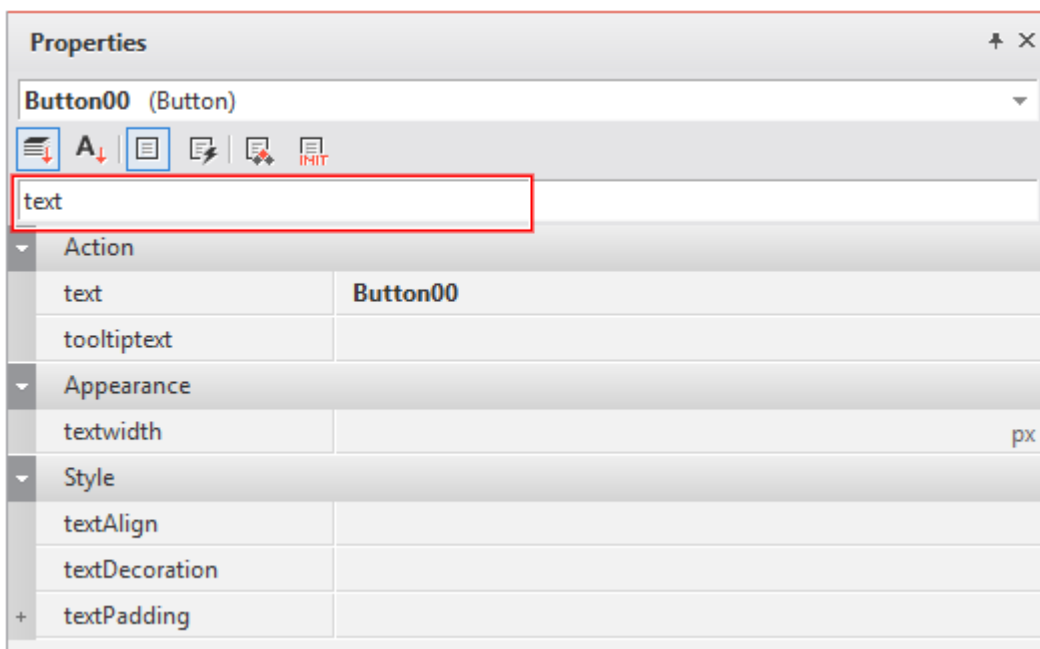


초기 상태에서 enable 속성값을 false로 수정하면 아래와 같이 굵은 글꼴로 표시됩니다.



3.3 속성창에서 Button 컴포넌트의 text 속성값 변경하기

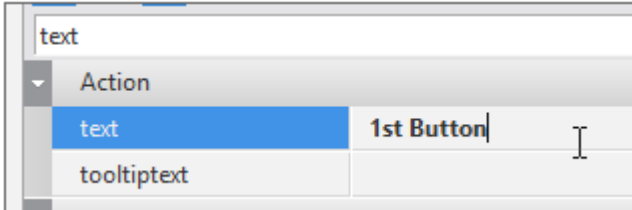
- ① 디자인 화면에 Button 컴포넌트를 배치하고 배치한 Button 컴포넌트를 선택합니다.
- ② 속성창에서 text 속성 항목을 찾습니다. 속성창 상단 검색창에 원하는 항목을 입력하면 바로 찾을 수 있습니다.



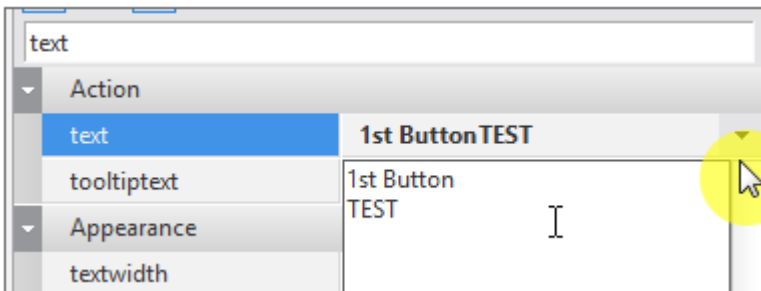


속성창 상단 검색창은 검색어로 입력된 항목을 포함하는 속성 항목을 찾습니다. 검색어로 "text"를 입력하면 속성명에 "text"가 포함된 모든 속성 항목을 보여줍니다.

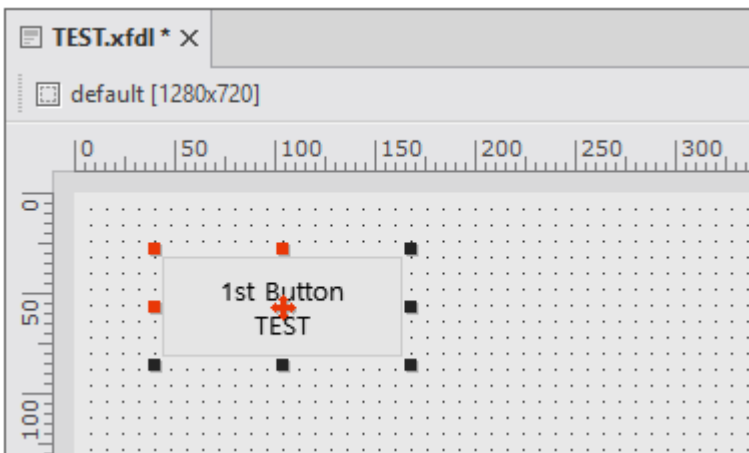
- 3 text 속성 항목에서 속성값이 있는 부분을 마우스로 클릭하고 원하는 텍스트를 입력합니다.



텍스트 입력 항목에 2줄 이상 텍스트를 입력하고자 할 때는 오른쪽에 있는 버튼을 클릭하면 2줄 이상의 텍스트를 입력할 수 있는 에디터가 표시됩니다. 줄바꿈은 **Ctrl+Enter** 키를 입력하면 바뀝니다.

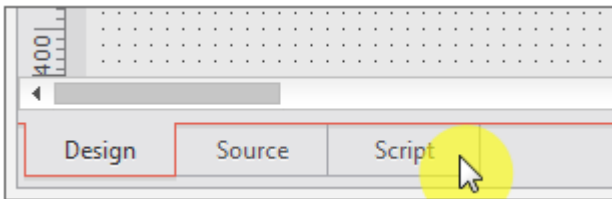


- 4 텍스트 입력 후 다른 속성 항목으로 이동하거나 디자인 화면으로 포커스를 옮기면 Button 컴포넌트의 텍스트가 변경된 것을 확인할 수 있습니다.



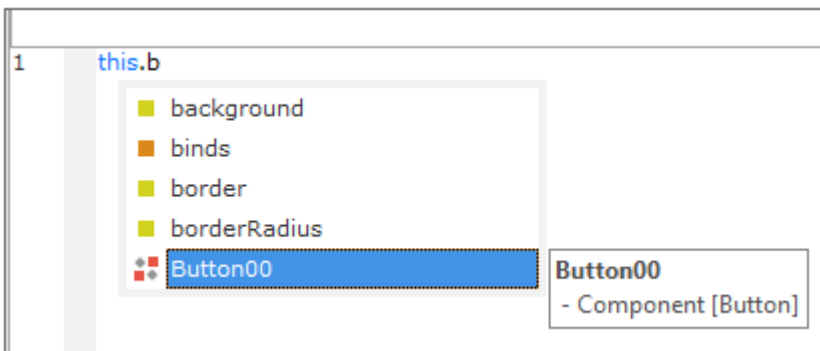
3.4 스크립트에서 Button 컴포넌트의 text 속성값 변경하기

- 1 스크립트 화면으로 변경합니다. 디자인 화면 하단에 보이는 3개의 탭 버튼 중에서 "Script" 탭을 선택합니다.



- 2 스크립트 화면에서 코드를 입력합니다. 넥사크로 스튜디오 스크립트 화면에서는 코드 작성 시 도움을 주는 인텔리센스 기능을 지원합니다.

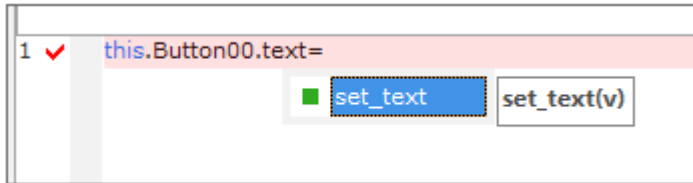
"this"라는 텍스트 입력 후 "."을 입력하면 Form 오브젝트의 속성, 메소드, 이벤트와 함께 배치된 컴포넌트의 목록을 보여줍니다. "." 입력 후 입력되는 글자에 따라 표시되는 목록을 필터링해서 보여줍니다. 목록에서 원하는 컴포넌트를 선택합니다.



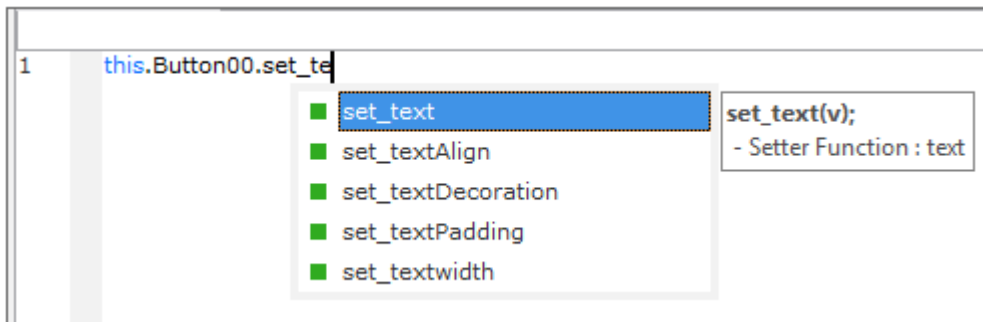
- 3 "this."을 입력한 것과 마찬가지로 "this.Button00."을 입력하면 Button 컴포넌트의 속성, 메소드, 이벤트 목록을 보여줍니다. 해당 목록 중에서 text 속성 항목을 선택합니다.



- 4 "this.Button00.text"까지 입력한 상태에서 등호("=")를 입력하면 속성값을 설정할 수 있는 코드가 자동으로 작성됩니다.



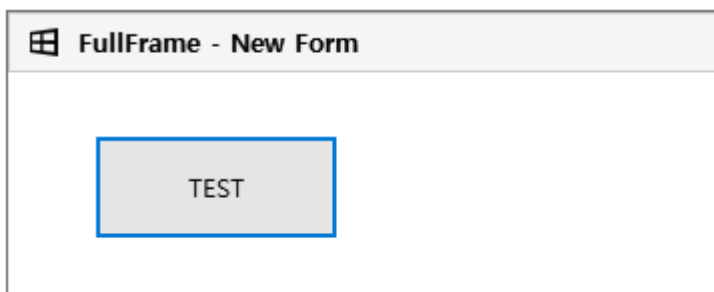
이전 단계에서 "this.Button00." 다음에 "set_"으로 시작하는 텍스트를 입력해도 같은 코드를 작성할 수 있습니다.



- ⑤ text 속성값으로 지정할 값을 입력합니다. 완성된 코드는 아래와 같습니다.

```
this.Button00.set_text("TEST");
```

- ⑥ 변경된 내용을 저장하고 툴바에서 [QuickView] 항목을 선택합니다. 넥사크로 브라우저가 실행되면서 화면에 Button 컴포넌트가 표시됩니다. 화면을 로딩하면서 앞에서 작성한 코드를 실행하고 Button 컴포넌트의 text 속성이 바뀐 상태에서 표시되는 것을 확인할 수 있습니다.



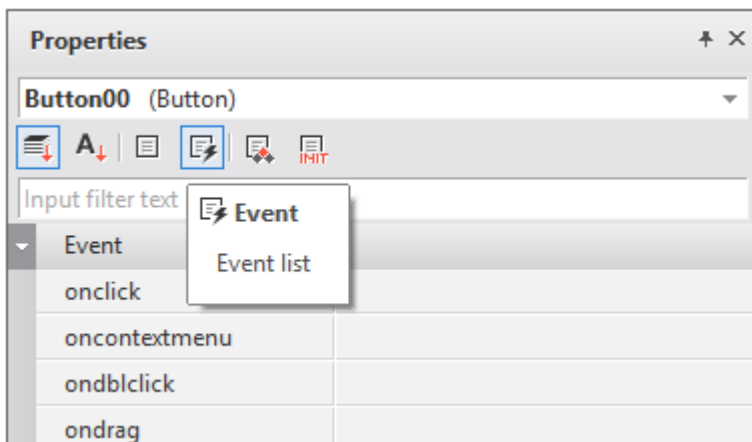
4.

이벤트 함수 생성

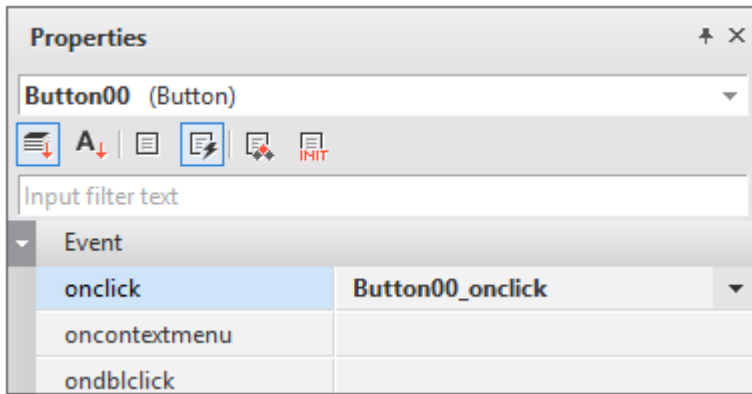
사용자가 컴포넌트를 마우스로 클릭하거나 포커스를 이동하는 등 어떤 동작을 하거나 컴포넌트의 상태가 변경될때 이벤트가 발생합니다. 이벤트 함수를 지정해놓으면 해당하는 이벤트 발생 시 원하는 동작을 처리할 수 있습니다.

4.1 속성창에서 Button 컴포넌트의 onclick 이벤트 함수 생성하기

- 1 디자인 화면에 Button 컴포넌트를 배치합니다. 배치된 Button 컴포넌트를 선택하고 속성창에서 이벤트 아이콘을 선택합니다. 속성창에 컴포넌트의 이벤트 목록이 표시됩니다.



- 2 onclick 이벤트 항목 오른쪽에 함수명을 입력합니다. 함수명을 입력하지 않고 빈칸을 더블클릭하면 ["컴포넌트 id"+"_"+"이벤트명"]으로 함수명을 자동 생성합니다.



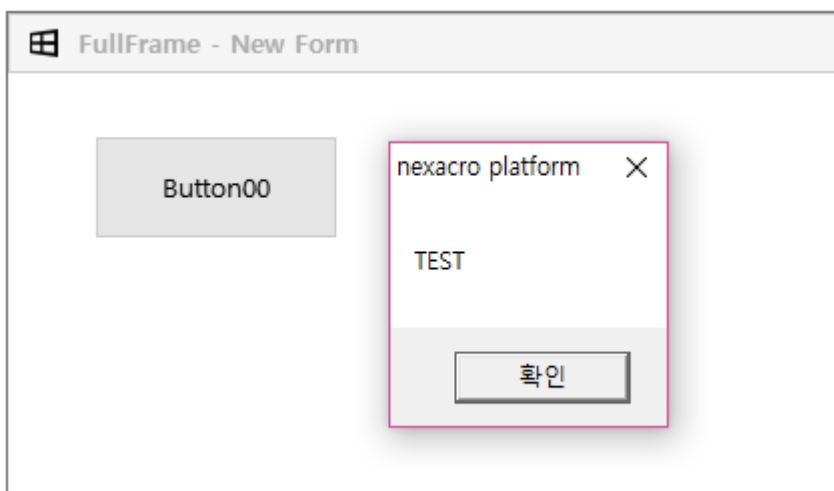
- 3 디자인 화면에서 스크립트 화면으로 이동하면 생성된 이벤트 함수를 확인할 수 있습니다. 아래와 같은 코드가 생성됩니다.

```
this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
};
```

- 4 이벤트 함수 내에 경고 대화상자를 표시하는 alert 메소드를 추가합니다. Button 컴포넌트를 클릭하는 이벤트가 발생 시 Button00_onclick 이벤트 함수가 호출되며 이벤트 함수 내에 alert 메소드가 실행됩니다.

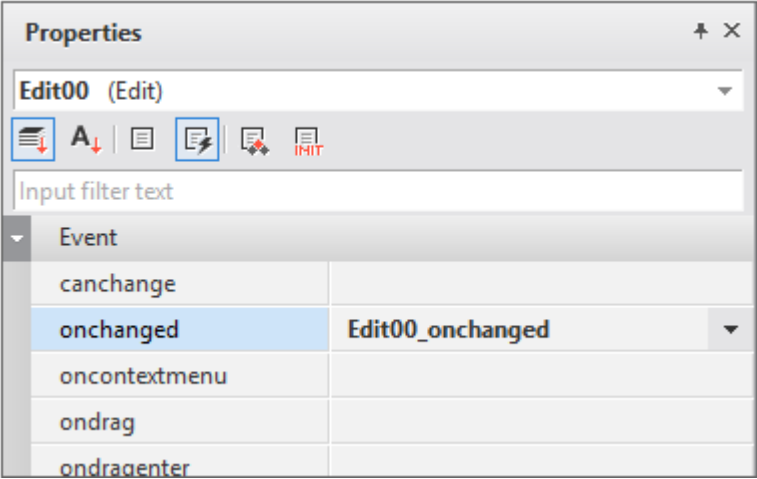
```
this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.alert("TEST");
};
```

- 5 변경된 내용을 저장하고 툴바에서 [QuickView] 항목을 선택합니다. 넥사크로 브라우저가 실행되면서 화면에 Button 컴포넌트가 표시됩니다. Button 컴포넌트를 클릭하면 경고 대화상자가 표시됩니다.



4.2 Edit 컴포넌트에 입력된 텍스트 확인하기

- ① 디자인 화면에 Edit 컴포넌트를 배치합니다. 배치된 Edit 컴포넌트를 선택하고 속성창에서 이벤트 아이콘을 선택합니다. 이벤트 항목 중 `onchanged` 이벤트를 선택하고 이벤트 함수를 생성합니다.



- ② 디자인 화면에서 스크립트 화면으로 이동하면 생성된 이벤트 함수를 확인할 수 있습니다. 이벤트 함수 코드 내에 아래와 같은 코드를 추가합니다. Edit 컴포넌트에 값을 입력하고 `Enter` 키를 입력하거나 다른 컴포넌트로 포커스를 이동하면 입력된 텍스트를 Output 창에 표시합니다.

```
this.Edit00_onchanged = function(obj:nexacro.Edit,e:nexacro.ChangeEventInfo)
{
    trace(e.posttext);
};
```

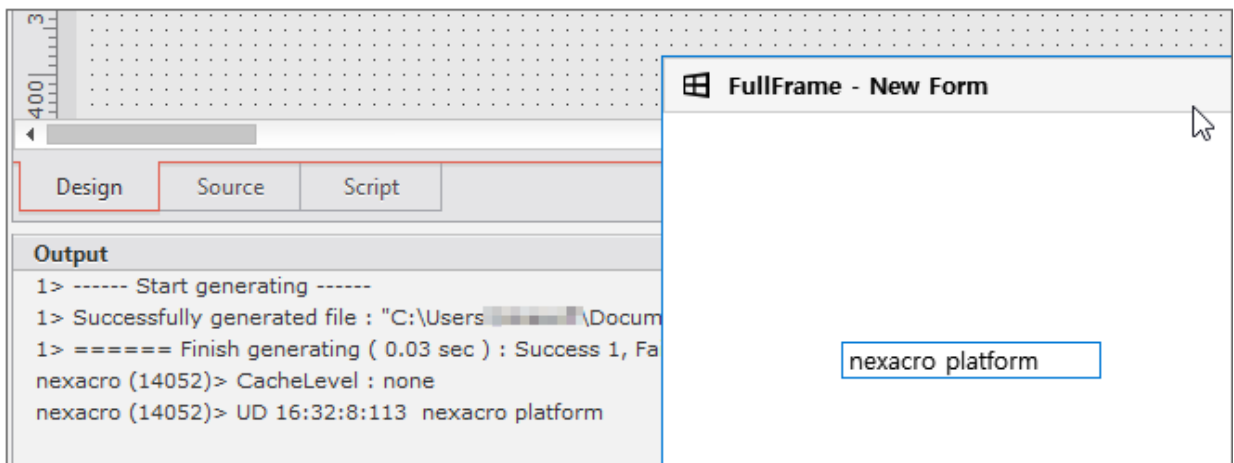
이벤트 함수 실행 시에 2개의 파라미터를 전달합니다. 첫 번째 파라미터는 이벤트가 발생한 컴포넌트이며 두 번째 파라미터는 이벤트와 관련된 정보를 담고 있는 `EventInfo` 오브젝트입니다. `EventInfo` 오브젝트에 어떤 정보가 담겨있는지는 도움말에서 확인할 수 있습니다. 도움말에서 `onchanged` 이벤트 항목을 찾아가면 `ChangeEventInfo` 오브젝트로 연결되는 링크를 확인할 수 있습니다.

Parameters		
Parameters	Type	
obj	Edit	Event generated Object.
e	ChangeEventInfo	Event Object.

ChangeEventInfo 오브젝트에서 사용할 수 있는 속성 정보를 확인할 수 있습니다.

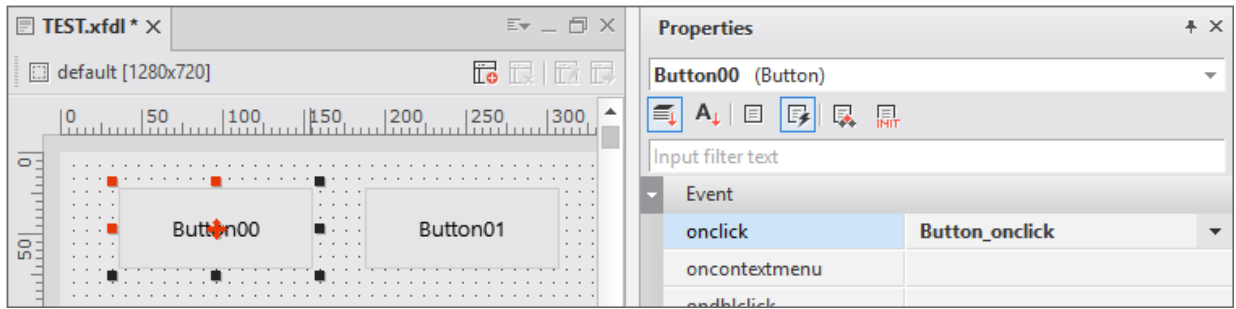
Property	
Name	
eventid	This property holds the ID of an eve
fromobject	This property holds an event-trigger
fromreferenceobject	It is Property to indicate an Object in
posttext	It is property to have a text string wh
postvalue	It is Property of a value which Comp
pretext	It is Property of a text string which C
prevalue	It is Property of a value which Comp

- ③ 변경된 내용을 저장하고 툴바에서 [QuickView] 항목을 선택합니다. 넥사크로 브라우저가 실행되면서 화면에 Edit 컴포넌트가 표시됩니다. Edit 컴포넌트에 텍스트를 입력하고 Enter 키를 입력합니다. 넥사크로 스튜디오의 Output 창에서 입력된 텍스트를 확인할 수 있습니다.



4.3 하나의 이벤트 함수를 여러 컴포넌트에서 사용하기

- ① 디자인 화면에 Button 컴포넌트를 2개 배치합니다. 배치된 Button 컴포넌트 중 하나를 선택하고 속성창에서 이벤트 아이콘을 선택합니다. 이벤트 항목 중 onclick 이벤트를 선택하고 이벤트 함수를 생성합니다. 이벤트 함수명은 자동으로 생성하지 않고 "Button_onclick"으로 지정합니다.

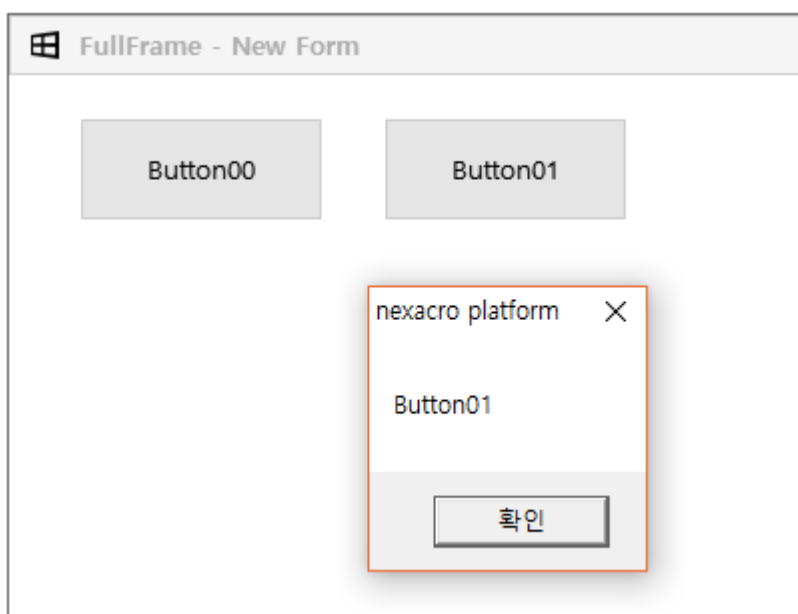


- ② 디자인 화면에서 스크립트 화면으로 이동하면 생성된 이벤트 함수를 확인할 수 있습니다. 이벤트 함수를 다음과 같이 작성합니다.

```
this.Button_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.alert(e.fromobject.id);
};
```

Button 컴포넌트 클릭 시 해당 컴포넌트의 id 값을 경고 대화상자에 표시합니다.

- ③ 이전에 선택하지 않았던 Button 컴포넌트를 선택하고 속성창에서 이벤트 항목 중 onclick 이벤트를 선택하고 이벤트 함수를 생성합니다. 이벤트 함수명은 자동으로 생성하지 않고 이전 단계에서 생성한 "Button_onclick"으로 지정합니다.
- ④ 변경된 내용을 저장하고 툴바에서 [QuickView] 항목을 선택합니다. 넥사크로 브라우저가 실행되면서 화면에 2개의 Button 컴포넌트가 표시됩니다. Button 컴포넌트를 클릭하면 각 Button 컴포넌트의 id값을 경고대화상자에 표시합니다.



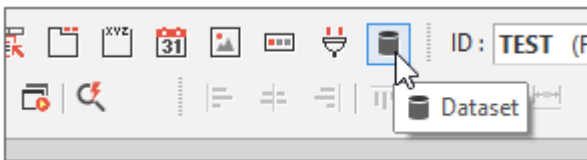
예제에서는 Button 컴포넌트 클릭 시 id값을 표시하는 간단한 예제를 보여주었지만 실제 코드에서는 사용자가 클릭한 Button 컴포넌트의 id값을 확인해 다른 작업을 수행하도록 할 수 있습니다.

5.

데이터 바인딩

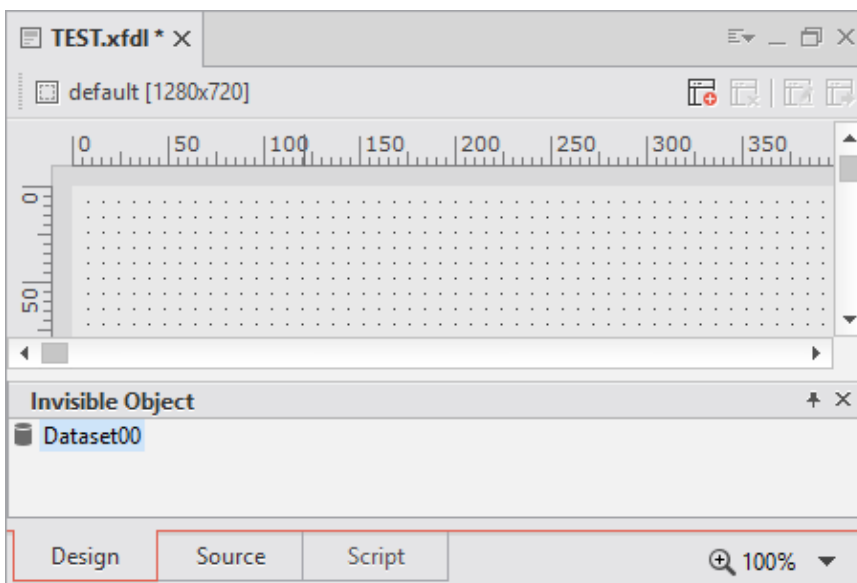
5.1 툴바에서 Dataset 오브젝트 아이콘 선택하고 배치하기

- 1 컴포넌트 툴바에서 Dataset 오브젝트를 선택합니다. 컴포넌트 아이콘 위에 마우스를 가져가면 컴포넌트 이름이 툴팁으로 표시됩니다.



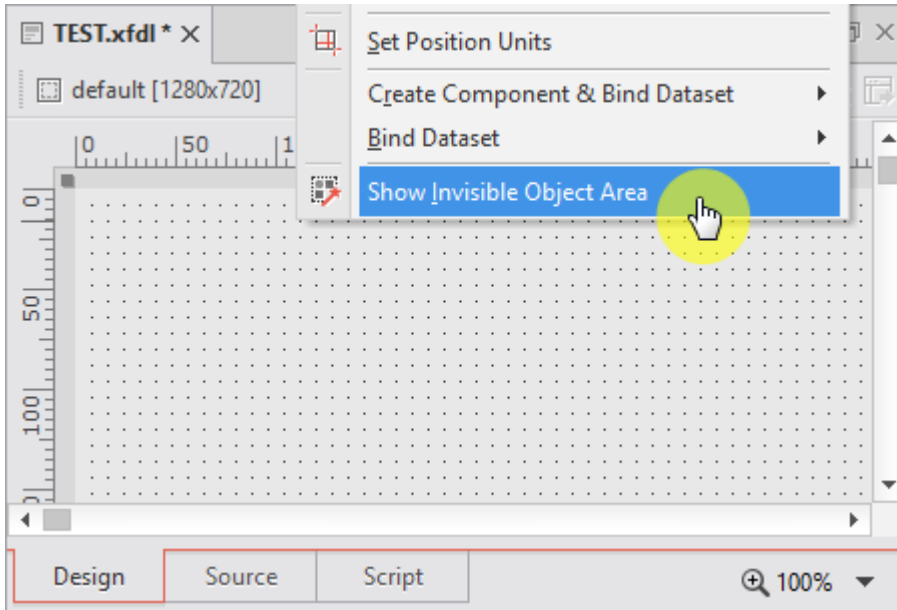
컴포넌트 툴바에서 Dataset 오브젝트를 선택한 상태에서는 디자인 화면 내에서 마우스 포인터가 Dataset 오브젝트 아이콘으로 변경됩니다.

- 2 디자인 화면 내 아무 곳이나 마우스를 클릭합니다. Dataset 오브젝트는 화면에 표시되지 않기 때문에 배치된 위치는 아무 의미가 없습니다. 마우스를 클릭하면 [Invisible Object] 영역에 Dataset 오브젝트가 추가됩니다.





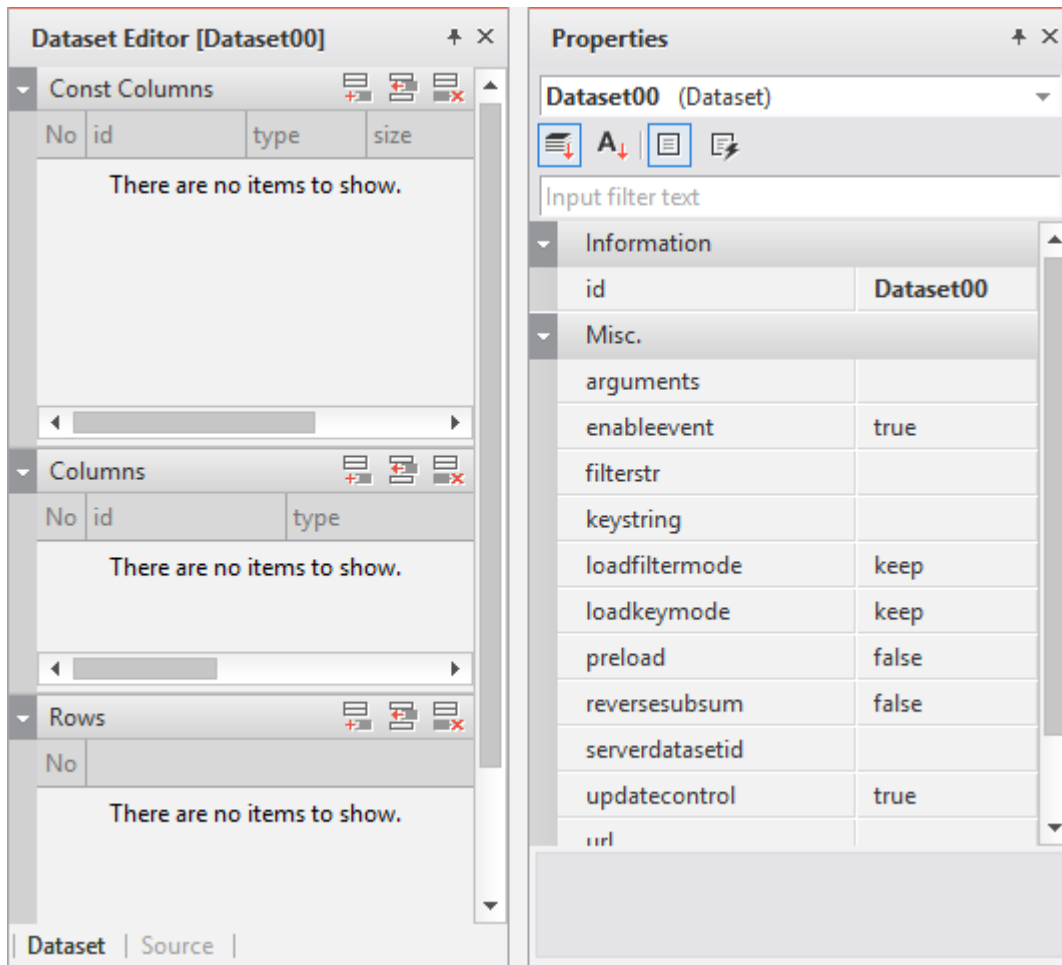
[Invisible Object] 영역이 보이지 않으면 디자인 화면 위에서 마우스 오른쪽 버튼을 클릭한 후 표시되는 컨텍스트 메뉴에서 [Show Invisible Object Area] 항목을 선택합니다.



5.2 Dataset Editor 실행하기

Dataset 오브젝트에서 처리할 데이터 컬럼 형식과 데이터를 지정할 수 있는 도구를 제공합니다. Dataset Editor를 실행해서 적절한 컬럼을 생성하고 데이터를 추가할 수 있습니다.

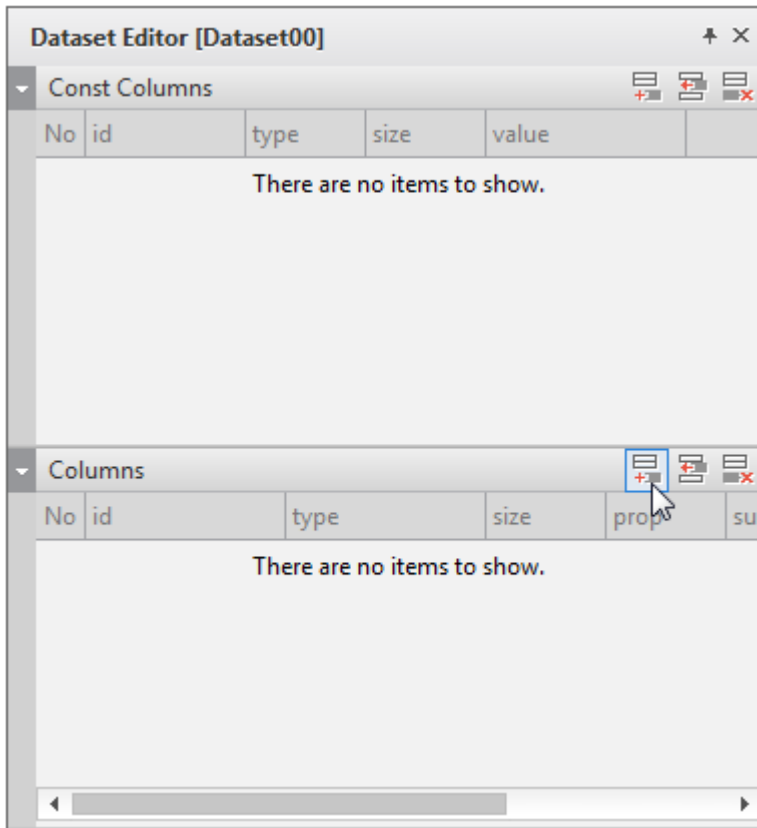
- ① [Invisible Object] 영역에서 Dataset 오브젝트를 더블클릭합니다.
- ② 속성창 왼쪽에 [Dataset Editor]가 실행됩니다. 사용하지 않을 때는 오른쪽 [x] 버튼을 클릭해 창을 닫습니다. [Dataset Editor]는 Dataset 오브젝트를 더블클릭하거나 메뉴[View > Dataset Editor]를 선택하면 다시 창을 표시합니다.



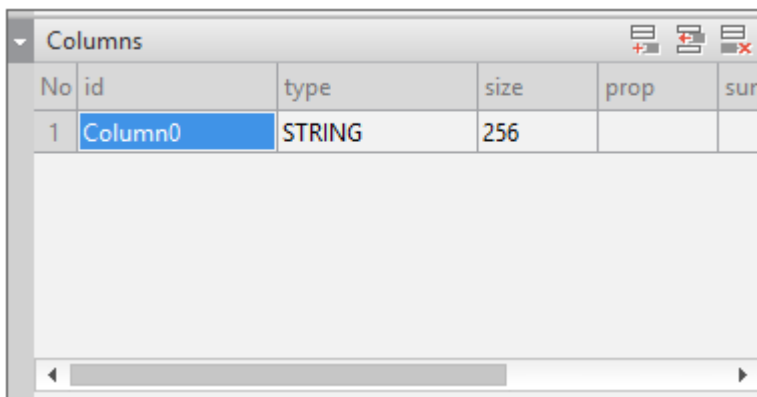
5.3 Dataset 오브젝트에 컬럼 추가하기

Dataset 오브젝트에 데이터를 담기 위해서는 어떤 데이터를 담을지에 대한 정보를 설정해주어야 합니다. 이를 컬럼이라 하고 Dataset Editor에서 컬럼을 추가할 수 있습니다.

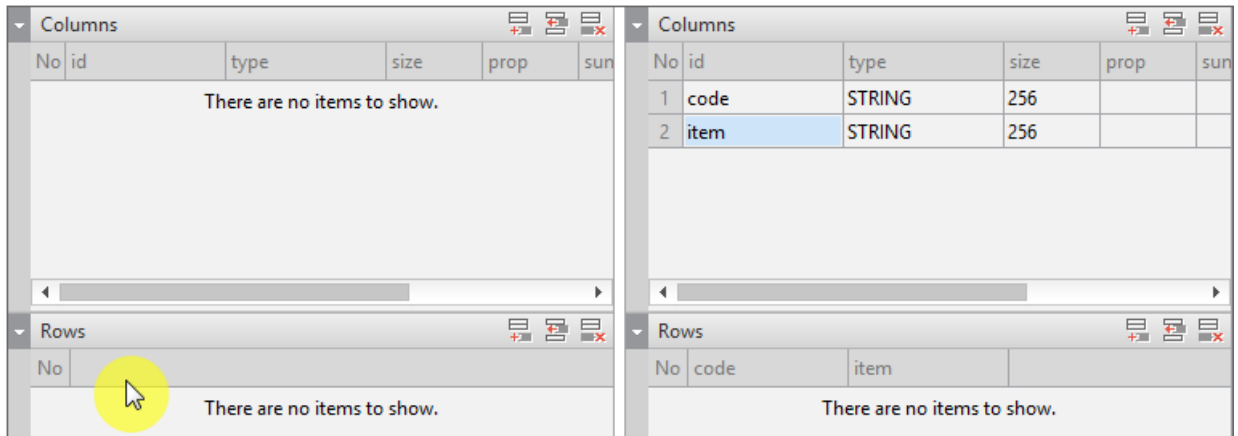
- ① [Invisible Object] 영역에서 Dataset 오브젝트를 더블클릭합니다.
- ② 속성창 왼쪽에 [Dataset Editor]가 실행됩니다. [Dataset Editor] 중간에 있는 [Columns] 항목에서 컬럼 추가 아이콘을 클릭합니다.



- 3 [Columns] 항목에 새로운 컬럼이 추가됩니다. 관련된 속성값(id, type, size 등)은 해당하는 셀을 마우스로 클릭한 후 수정할 수 있습니다.



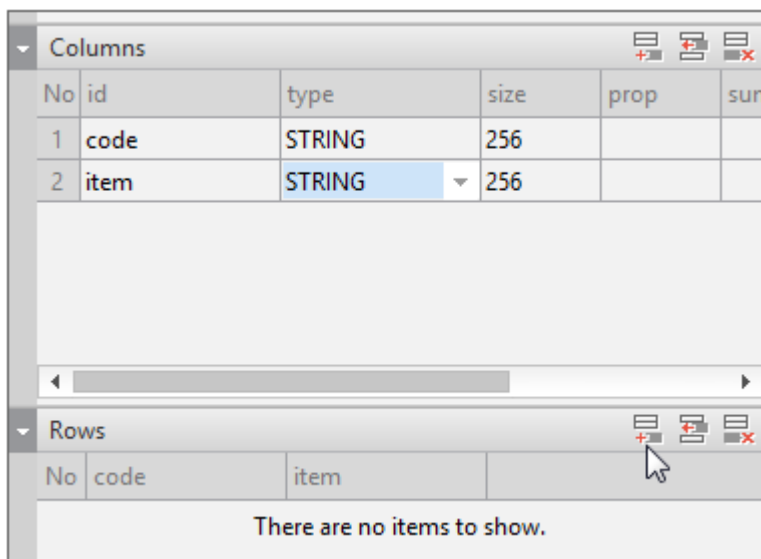
- 4 2개 컬럼을 추가하고 id값을 "code", "item"으로 수정합니다. 컬럼을 추가하면 아래에 있는 [Rows] 항목에 추가한 컬럼 정보가 표기되는 것을 확인할 수 있습니다.



5.4 Dataset 오브젝트에 데이터 추가하기

컬럼 정보가 만들어지면 데이터를 추가할 수 있습니다.

- ① [Invisible Object] 영역에서 Dataset 오브젝트를 더블클릭합니다.
- ② 속성창 왼쪽에 [Dataset Editor]가 실행됩니다. [Dataset Editor] 아래에 있는 [Rows] 항목에서 데이터 추가 아이콘을 클릭합니다.



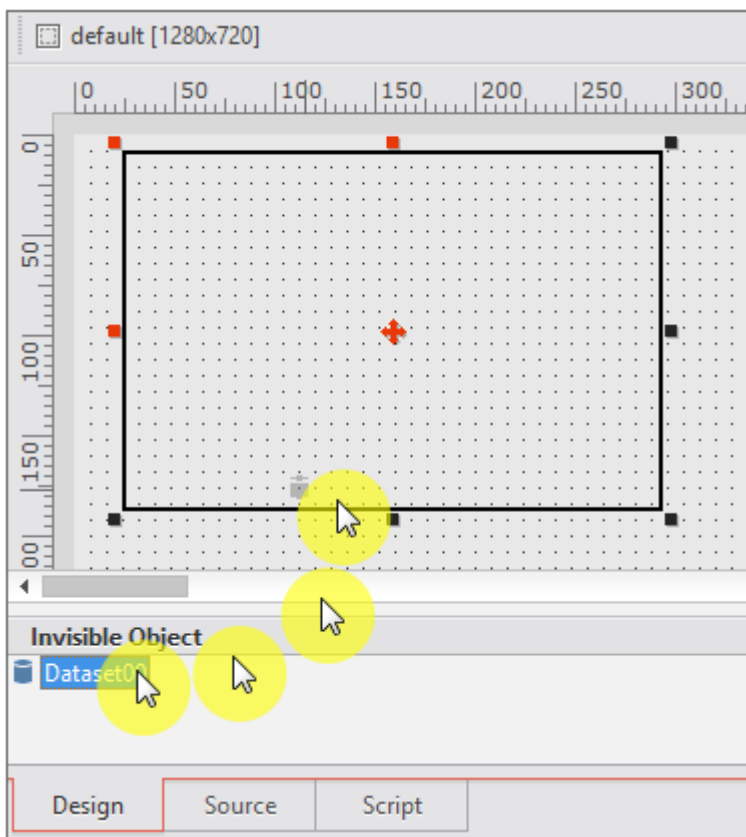
- ③ 데이터가 추가되면 해당하는 셀을 마우스로 클릭해 원하는 값을 입력합니다.

Rows		
No	code	item
1	0001	Toy
2	0002	Food
3	[Undefined]	[Undefined]

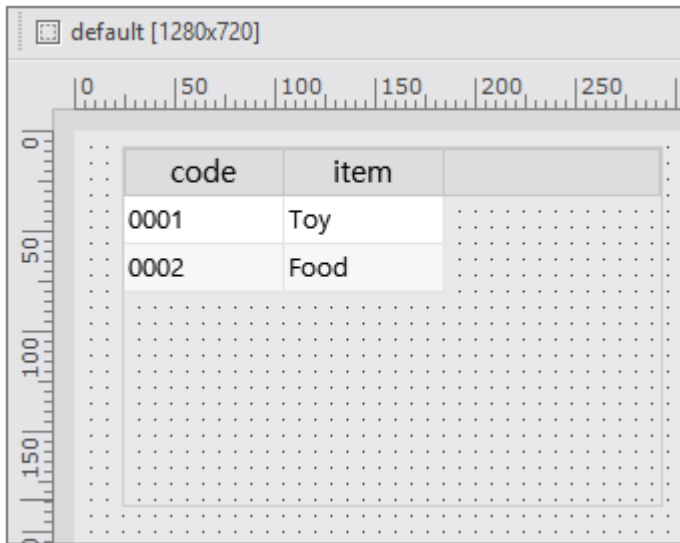
5.5 Grid 컴포넌트에 Dataset 오브젝트 바인딩하기

컴포넌트와 Dataset 오브젝트를 바인딩한다는 것은 데이터를 공유하도록 연결하는 것입니다. 바인딩된 상태에서 Dataset 오브젝트의 데이터가 추가되거나 변경되면 컴포넌트에 표시되는 데이터도 같이 변경됩니다. 반대로 컴포넌트에서 데이터를 추가, 삭제하거나 변경하면 Dataset 오브젝트의 데이터도 같이 변경됩니다.

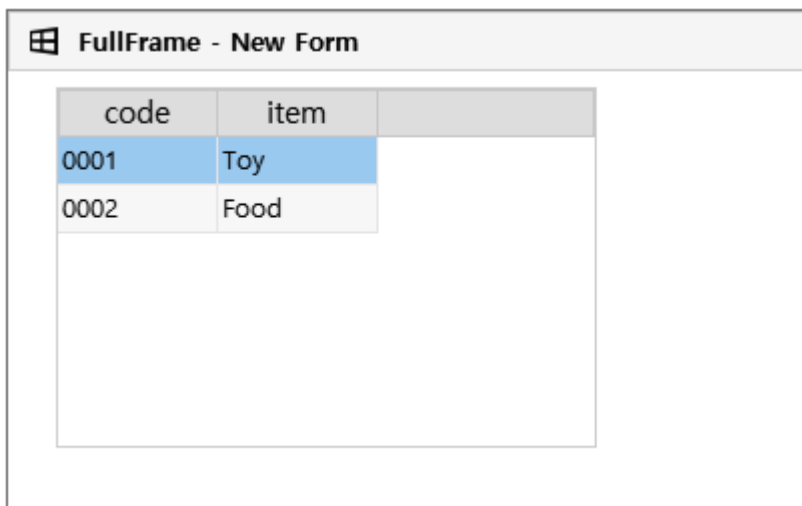
- 1 디자인 화면에 Grid 컴포넌트를 배치합니다.
- 2 [Invisible Object] 영역에서 Dataset 오브젝트를 마우스 왼쪽 버튼을 눌러 선택하고 드래그해서 디자인 화면에 있는 Grid 컴포넌트 위로 가져옵니다.



- ③ Grid 컴포넌트 위에서 누르고 있던 마우스 왼쪽 버튼을 놓으면 자동으로 데이터 바인딩이 처리됩니다. 아무것도 표시되지 않던 Grid 컴포넌트에는 Dataset 오브젝트에 설정한 컬럼 정보와 데이터가 표시됩니다.



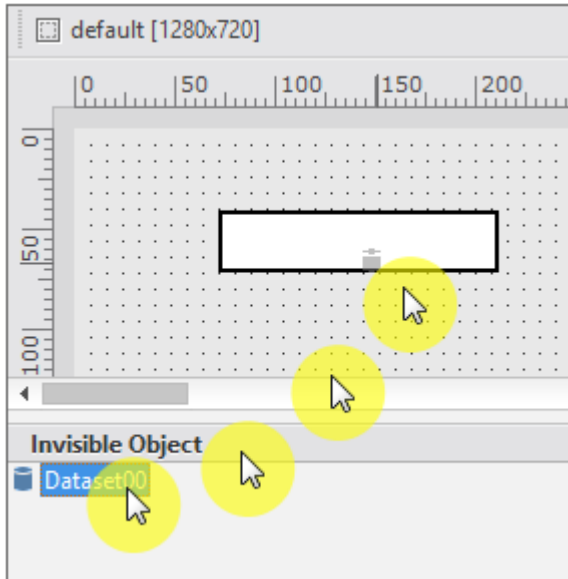
- ④ 변경된 내용을 저장하고 툴바에서 [QuickView] 항목을 선택합니다. 넥사크로 브라우저가 실행되면서 화면에 Grid 컴포넌트가 표시되고 바인딩된 데이터도 확인할 수 있습니다.



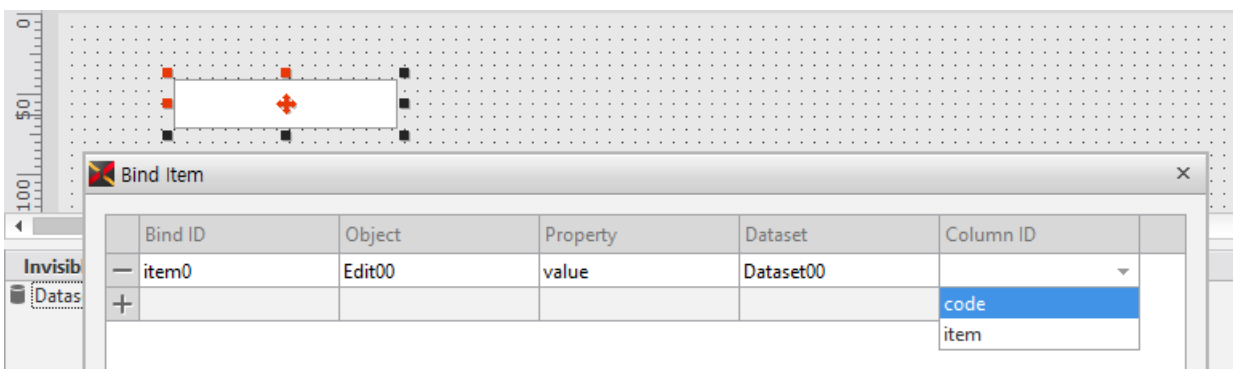
5.6 Edit 컴포넌트에 Dataset 오브젝트 바인딩하기

Grid 컴포넌트의 경우에는 Dataset 오브젝트의 컬럼 정보와 데이터를 표 형식으로 모두 표시합니다. 하지만 Edit 컴포넌트처럼 하나의 데이터만 표시할 수 있는 경우에는 바인딩한 Dataset 오브젝트의 어떤 정보를 표시할 수 있을까요? 화면에는 보이지 않지만 Dataset 오브젝트 내부에는 선택된 행(Row) 정보를 가지고 있고 Edit 컴포넌트같은 경우에는 선택된 행에서 원하는 컬럼 정보를 선택해서 표시합니다.

- ① 디자인 화면에 Edit 컴포넌트를 배치합니다.
- ② [Invisible Object] 영역에서 Dataset 오브젝트를 마우스 왼쪽 버튼을 눌러 선택하고 드래그해서 디자인 화면에 있는 Edit 컴포넌트 위로 가져옵니다.

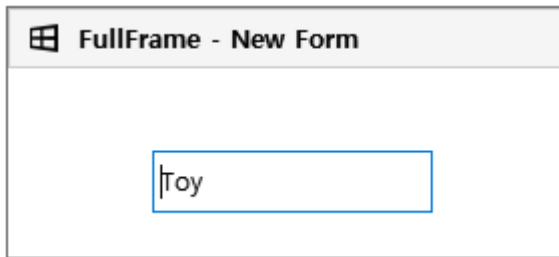


- ③ Edit 컴포넌트 위에서 누르고 있던 마우스 왼쪽 버튼을 놓으면 [Bind Item] 설정창이 표시됩니다. [Bind Item] 설정창에서는 Edit 컴포넌트에서 바인딩할 속성과 Dataset 오브젝트에서 바인딩할 컬럼을 선택합니다.



Edit 컴포넌트는 넥사크로 스튜디오 디자인 화면에서는 바인딩된 값이 표시되지 않습니다. 애플리케이션 실행 시 선택된 행(Row) 정보에 따라 값을 표시합니다.

- ④ 변경된 내용을 저장하고 툴바에서 [QuickView] 항목을 선택합니다. 넥사크로 브라우저가 실행되면서 화면에 Edit 컴포넌트가 표시되고 바인딩된 값을 확인할 수 있습니다. 행(Row) 정보를 건드리지 않았기 때문에 첫 번째 행(Row) 데이터인 "Toy"가 표시됩니다.



5.7 Grid 컴포넌트와 Edit 컴포넌트에 Dataset 오브젝트 바인딩하기

Edit 컴포넌트에 Dataset 컴포넌트를 바인딩하는 경우에는 행(Row) 정보에 따라 표시되는 데이터가 달라진다고 했습니다. 이번에는 Grid 컴포넌트와 Edit 컴포넌트에 같은 Dataset 오브젝트를 바인딩해서 Grid 컴포넌트의 행(Row)을 선택할 때마다 Edit 컴포넌트에 표시되는 값이 달라지는 예제를 다룹니다.

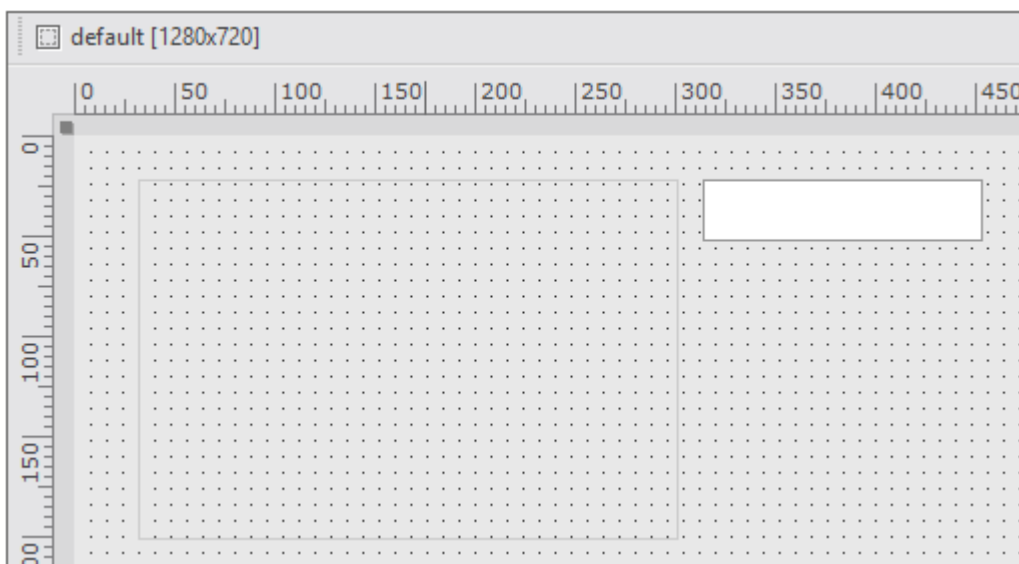


Grid 컴포넌트와 Edit 컴포넌트에 Dataset 오브젝트를 바인딩하는 방법은 이전 예제를 참고해주세요.

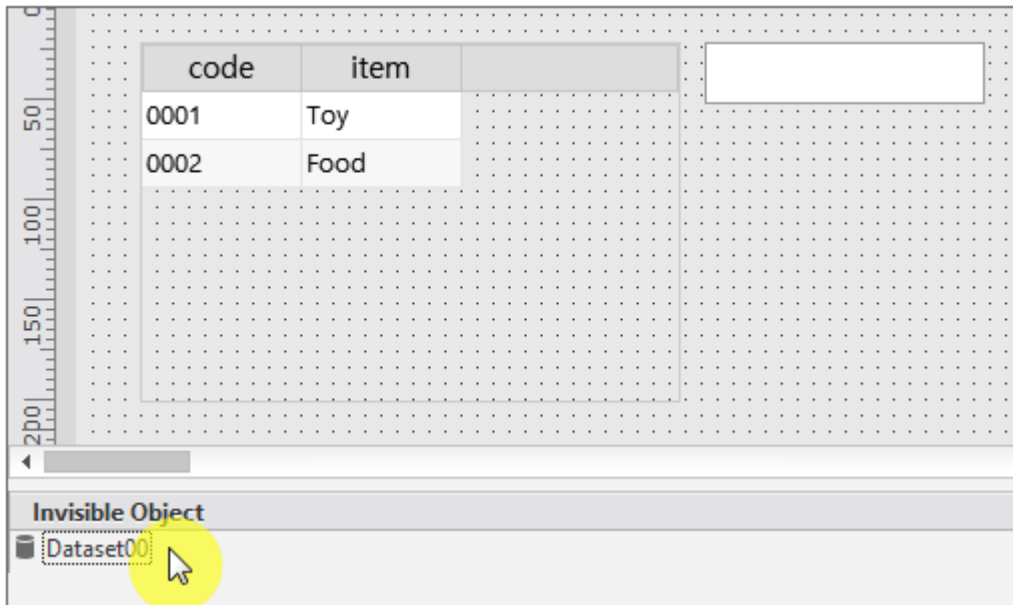
[Grid 컴포넌트에 Dataset 오브젝트 바인딩하기](#)

[Edit 컴포넌트에 Dataset 오브젝트 바인딩하기](#)

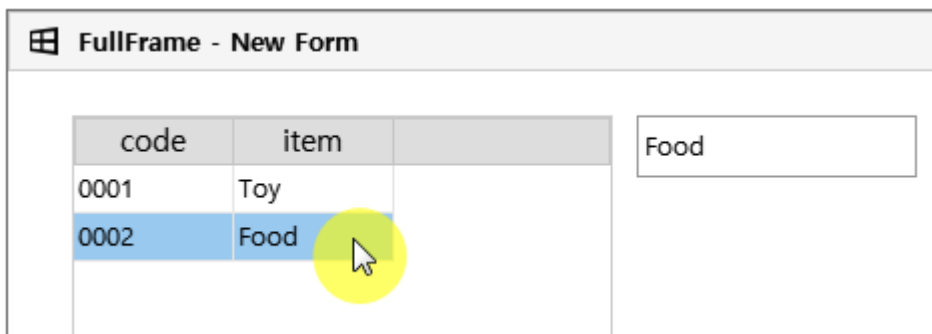
- 1 디자인 화면에 Grid 컴포넌트와 Edit 컴포넌트를 배치합니다.



- 2 Dataset 오브젝트를 선택하고 Grid 컴포넌트와 Edit 컴포넌트에 바인딩합니다.



- ③ 변경된 내용을 저장하고 툴바에서 [QuickView] 항목을 선택합니다. 넥사크로 브라우저가 실행되면서 화면에 Grid 컴포넌트와 Edit 컴포넌트가 표시되고 바인딩된 값을 확인할 수 있습니다. 행(Row) 정보를 건드리지 않았기 때문에 첫 번째 행(Row)이 선택되어 있고 Edit 컴포넌트에는 "Toy"가 표시됩니다.
- ④ 마우스로 Grid 컴포넌트의 두 번째 행(Row)을 클릭합니다. Edit 컴포넌트의 값이 달라지는 것을 확인할 수 있습니다. 다시 첫 번째 행(Row)을 선택하면 Edit 컴포넌트의 값이 달라집니다.



6.

테마 활용

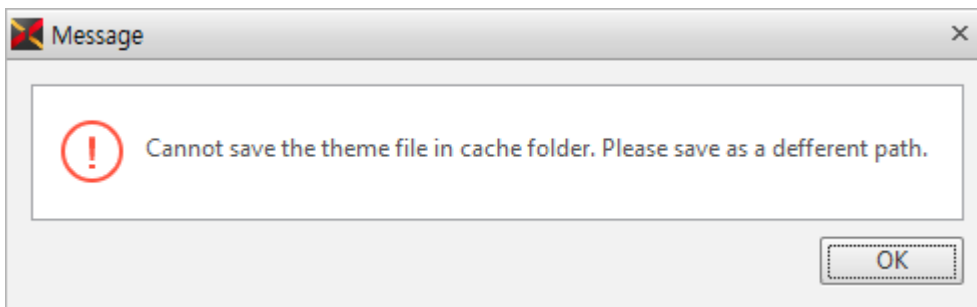
넥사크로플랫폼은 각 운영체제에 적절한 기본 테마를 제공하고 있습니다. 하지만 업무 환경에 따라 적절한 디자인 작업은 애플리케이션 개발 시 필요한 작업입니다. 각 컴포넌트에 적절한 스타일 속성값을 지정하고 이를 반영할 수 있습니다.

6.1 기본 테마 수정하기



앗! 기본 테마는 수정할 수 없습니다.

넥사크로 스튜디오에서 기본 테마를 열어 수정할 수 있습니다. 하지만 수정한 내용을 기본 테마로 저장할 수는 없습니다. 기본 테마를 수정한 후 저장하려 하면 다른 이름으로 저장하기 대화상자가 표시됩니다.

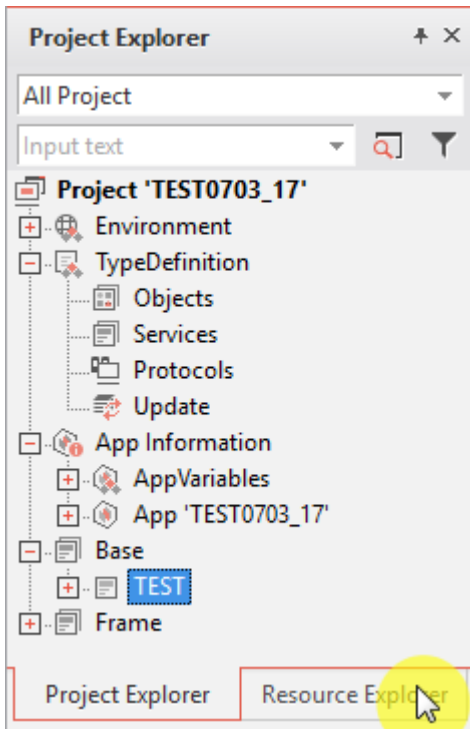


6.2 기본 테마 복사해서 내 테마로 만들기

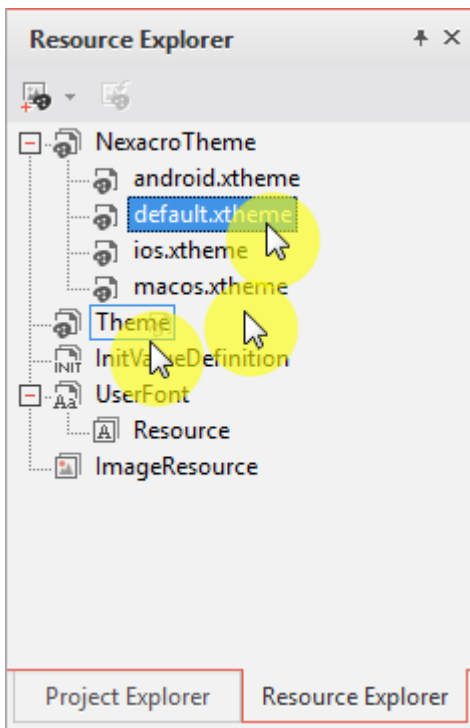
테마를 새로 만들수도 있습니다. 하지만 테마를 생성하기 위해 지정해야 하는 속성이 너무 많기 때문에 처음부터 테마를 만들기는 쉽지 않은 작업입니다. 이런 경우 기본 테마를 복사해놓고 이를 기반으로 필요한 항목을 수정해 사용합니

다.

- 1 넥사크로 스튜디오의 Project Explorer 창에서 하단의 Resource Explorer 탭을 클릭합니다.

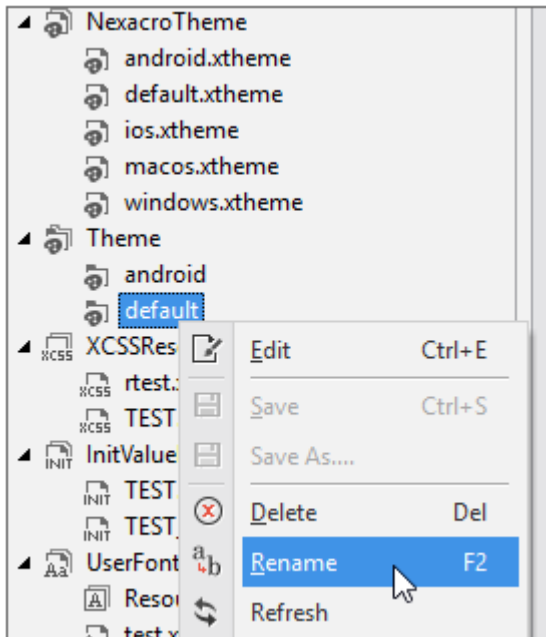


- 2 복사할 기본 테마를 마우스 왼쪽 버튼으로 누른 상태에서 드래그해서 Theme 폴더로 이동합니다.



- 3 Theme 폴더 아래에 default.xtheme 파일이 복사됐습니다. 그대로 사용해도 되지만 기본 테마와 파일명이 같

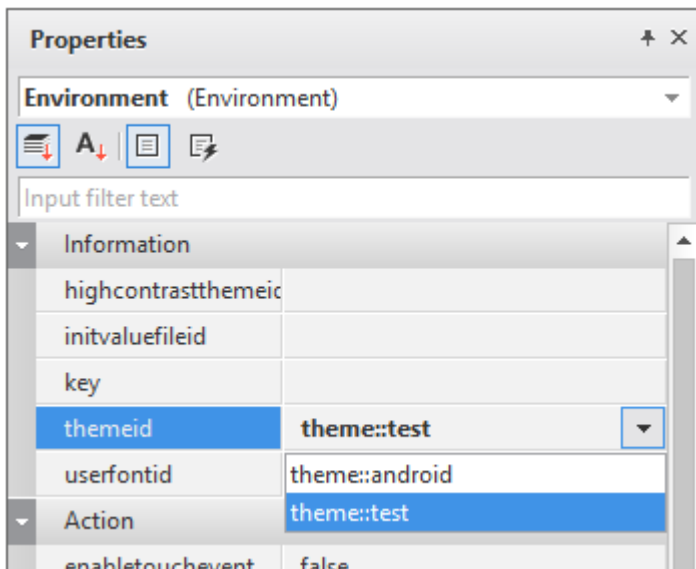
으면 혼란스러울 수 있으니 테마명을 변경합니다. 마우스 오른쪽 버튼을 클릭하고 컨텍스트 메뉴에서 [Rename] 항목을 선택하고 "test"로 수정합니다.



default.xtheme 파일이 그대로 복사되지 않고 이미지 파일과 XCSS 파일이 압축이 풀린 상태로 복사됩니다.

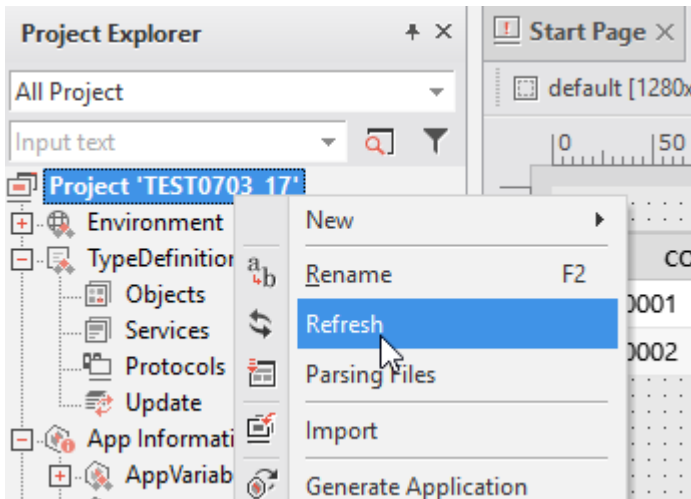
4

[Project Explorer]로 다시 돌아가서 "Environment" 항목을 선택하고 속성창에서 themeid 속성값을 Theme 폴더 아래에 복사한 테마로 변경합니다.



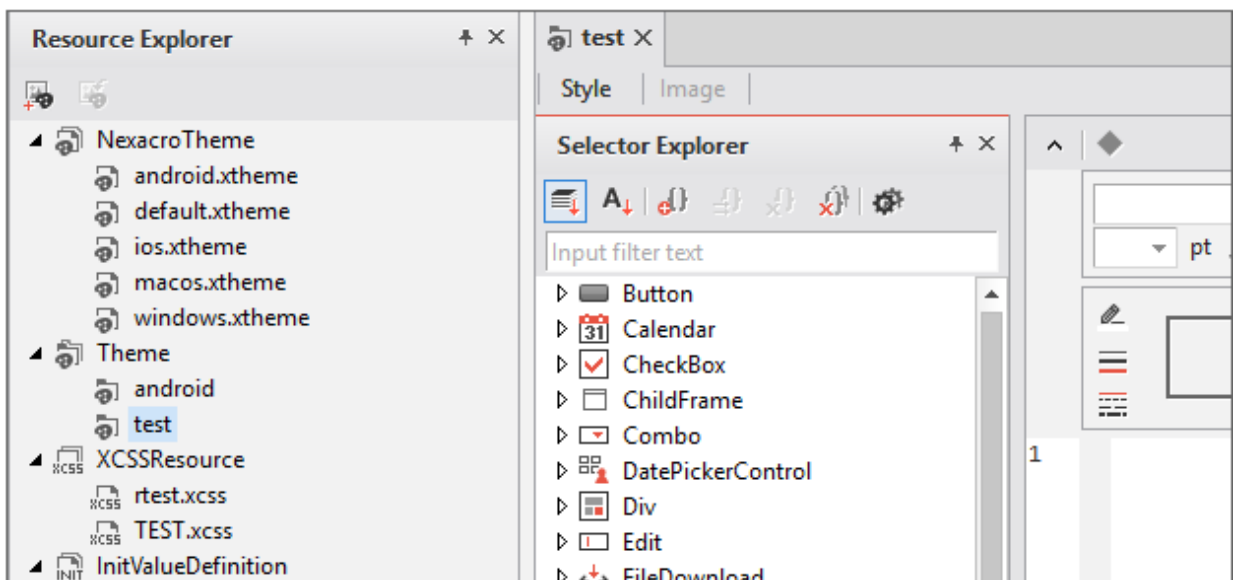


themeid를 변경한 이후 디자인 화면이 변경되지 않는 경우가 있습니다. 이런 경우에는 [Project Explorer]에서 프로젝트를 마우스 오른쪽 버튼으로 클릭하고 컨텍스트 메뉴에서 "Refresh" 항목을 선택합니다.

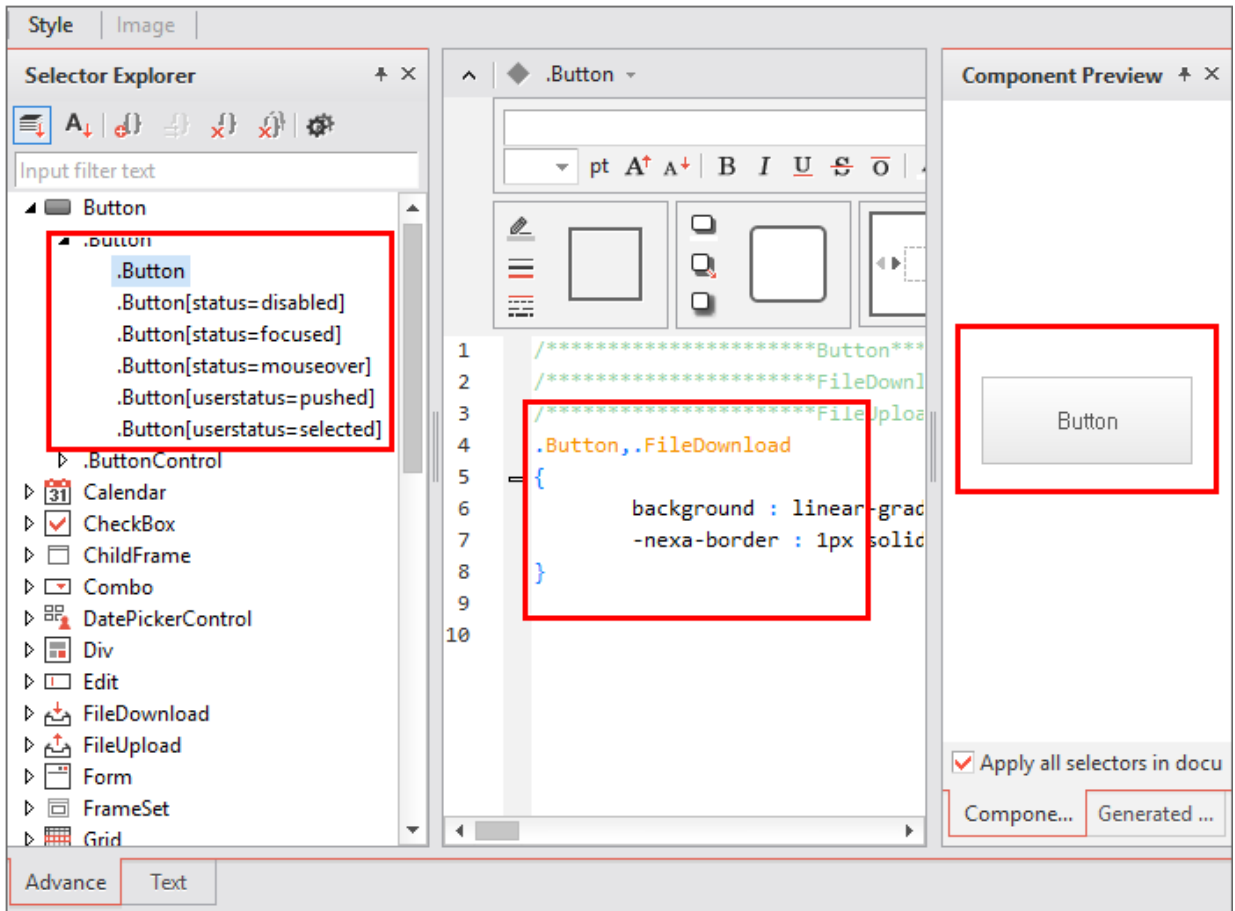


6.3 내 테마에서 Button 컴포넌트 배경색 변경하기

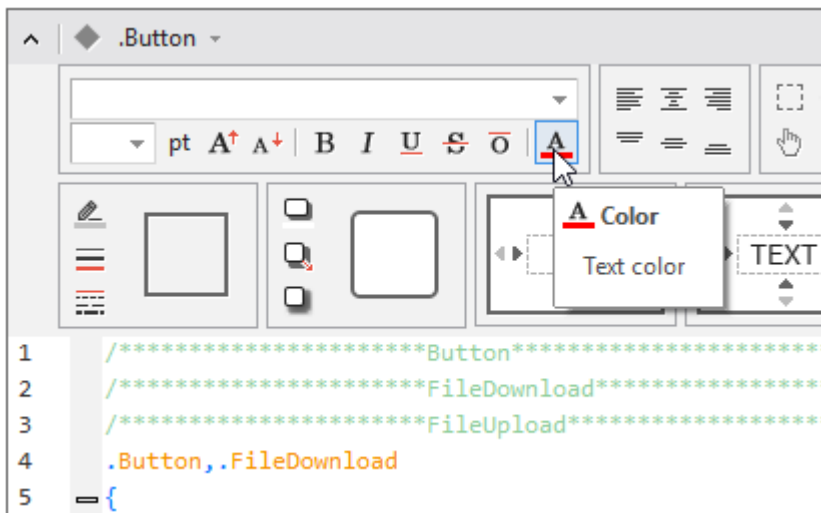
- 1 Theme 폴더에 복사한 테마 파일을 마우스로 더블클릭합니다. 파일이 열리면서 theme.xcss 파일을 편집할 수 있는 에디터가 열립니다.



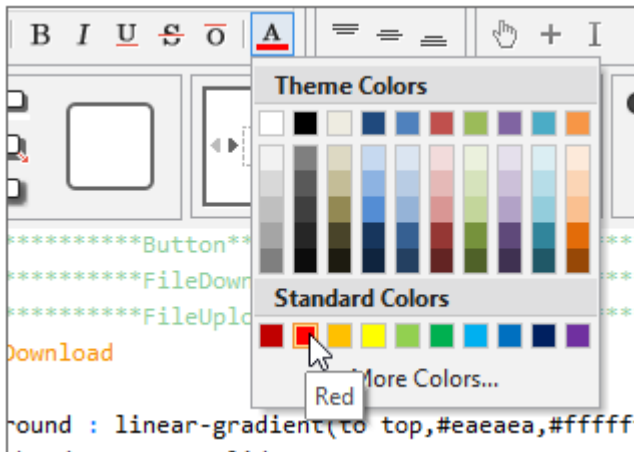
- 2 [Nexacro Style Sheet] 목록에서 Button을 찾아 마우스로 더블클릭합니다. [Xcss Editor]에는 .Button 선택자의 코드가 보여지고 [Preview]에는 Button 컴포넌트의 미리보기 화면이 표시됩니다.



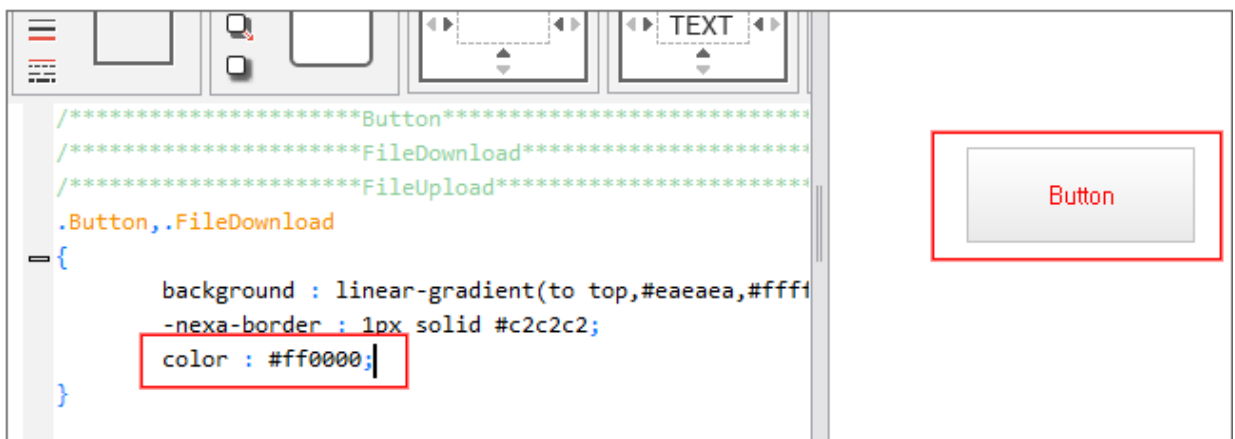
- ③ [Xcss Editor]에 표시된 코드를 직접 수정하거나 상단 툴바에서 원하는 항목을 선택해 수정할 수 있습니다. 여기에서는 상단 툴바에서 "Color" 항목을 선택합니다.



- ④ 원하는 배경색을 선택합니다.



코드에서 color 속성값과 [Preview]에 표시되는 Button 컴포넌트의 글자색이 변경된 것을 확인할 수 있습니다.



- 5 디자인 화면에서 Button 컴포넌트를 배치해봅니다. 테마에서 수정한 배경색이 반영되었는지 확인합니다. 기존에 배치한 Button 컴포넌트의 색도 모두 변경됩니다.

7.

스타일 활용 (XCSS)

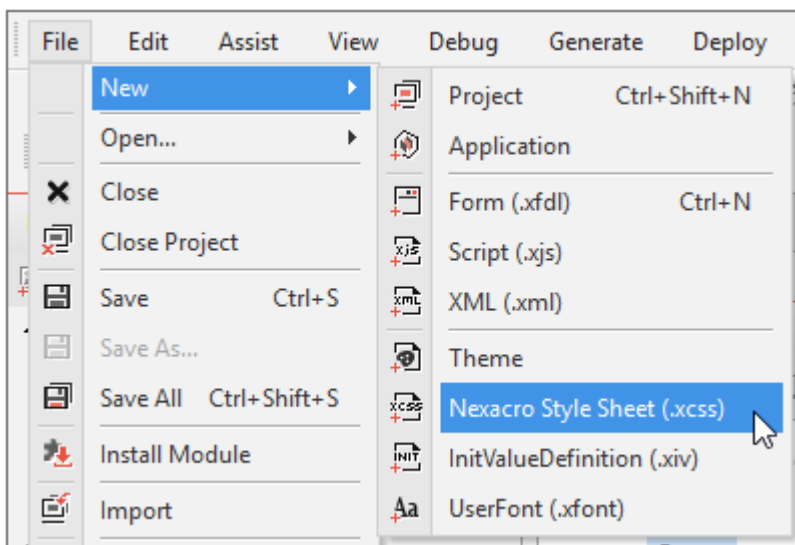
넥사크로 스타일 시트(XCSS)는 넥사크로플랫폼 애플리케이션의 화면 요소를 디자인할 때 사용하는 넥사크로플랫폼용 CSS입니다. 웹 문서의 스타일을 정의하는 CSS(Cascading Style Sheets)와 기본적인 사용법 및 스타일 속성이 유사하여 CSS를 다뤄본 개발자나 디자이너가 큰 어려움 없이 사용할 수 있습니다. XCSS는 테마와 별개로 스타일을 적용할 때 유용하며 클래스 선택자와 컴포넌트의 cssclass 속성을 사용하면 특정 컴포넌트에만 스타일 효과를 줄 수도 있습니다. 테마도 내부적으로는 XCSS와 이미지로 구성되어 있습니다.

이번 장에서는 넥사크로 스튜디오를 사용하여 XCSS를 만들고 사용하는 방법에 대해 설명합니다.

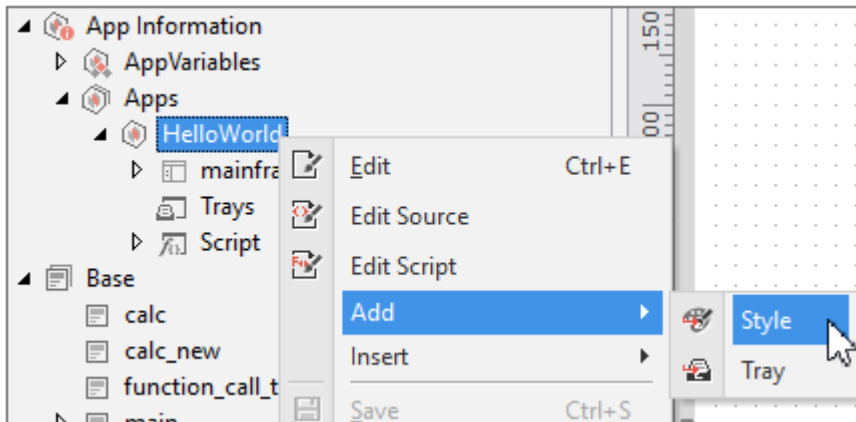
7.1 넥사크로 스타일 시트(XCSS) 만들기

넥사크로플랫폼 애플리케이션의 화면 요소를 디자인할 때 XCSS를 사용할 수 있습니다. XCSS는 다음과 같은 절차로 만듭니다.

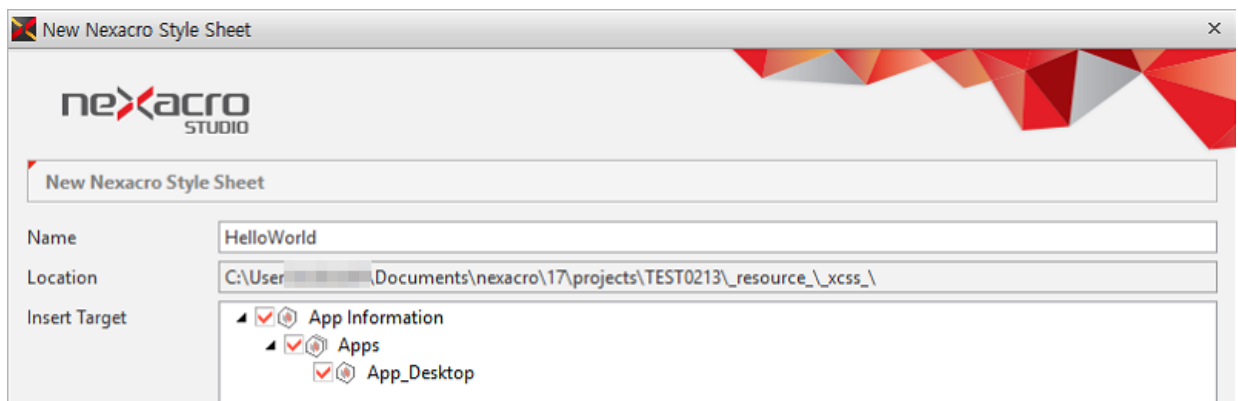
- 1 넥사크로 스튜디오의 Menu에서 [File > New > Nexacro Style Sheet(.xcss)]를 선택합니다.



참고로 Project Explorer에서도 다음과 같이 Application 이름을 선택한 후 마우스 오른쪽 버튼을 클릭하여 컨텍스트 메뉴에서 [Add > Style]을 선택하면 같은 결과를 얻을 수 있습니다.

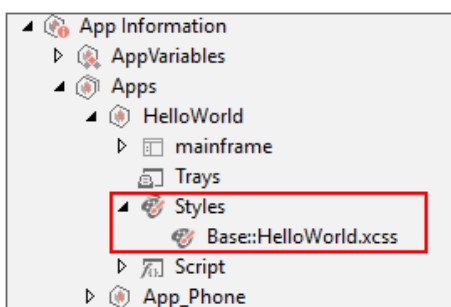


- ② New Style Sheet 화면에서 XCSS 이름과 경로, 적용 대상을 설정합니다.



항목	설명
Name	새로운 스타일 시트 파일의 이름을 입력합니다.
Location	스타일 시트 파일이 위치할 경로를 선택합니다.
Insert Target	스타일 시트를 적용할 대상을 선택합니다. 애플리케이션 단위로 적용할 수 있습니다.

Finish 버튼을 클릭하면 다음과 같이 Project Explorer에서 새로 만들어진 XCSS 파일을 확인할 수 있습니다. 만들어진 파일은 Xcss Editor에서 자동으로 오픈됩니다.



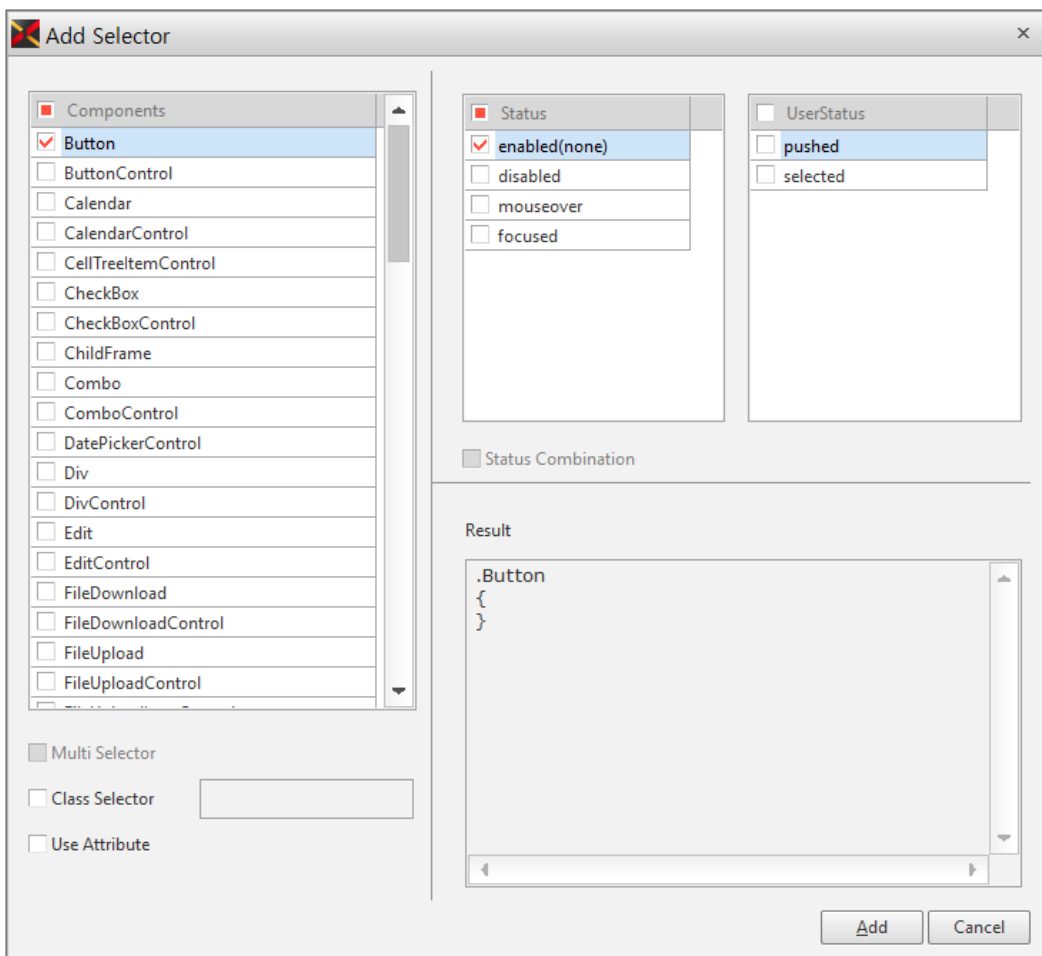
3 Xcss Editor에서 새로운 선택자를 추가합니다.

생성된 XCSS는 아무 내용이 없는 빈 파일입니다. 스타일 효과를 설정하려면 컴포넌트에 해당하는 선택자를 추가해야 합니다. Xcss Editor의 왼쪽 위에서 Add Selector (Insert) 버튼을 클릭하여 새로운 선택자를 추가합니다.



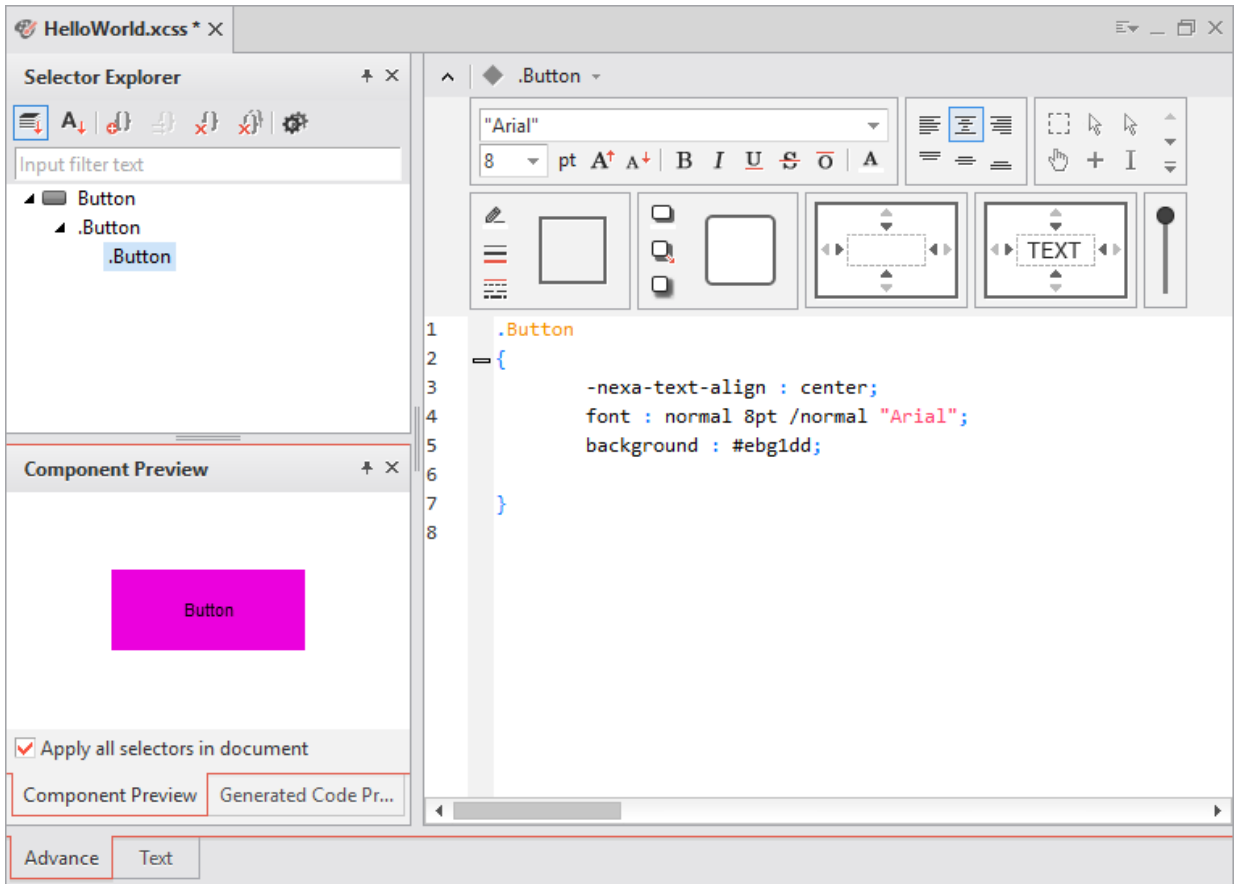
4 Add Selector 창에서 스타일 효과를 적용할 컴포넌트와 상태 등을 선택합니다.

선택한 컴포넌트에 어떤 스타일 속성이 있는지 확인하려면 Use Attribute 체크박스를 클릭합니다. Use Attribute 옵션을 체크하면 선택한 컴포넌트에서 사용할 수 있는 스타일 속성이 모두 생성됩니다. 단, 아무 값도 설정되지 않은 빈 속성이므로 속성값은 다음 단계인 편집 모드에서 사용자가 직접 입력해줘야 합니다.



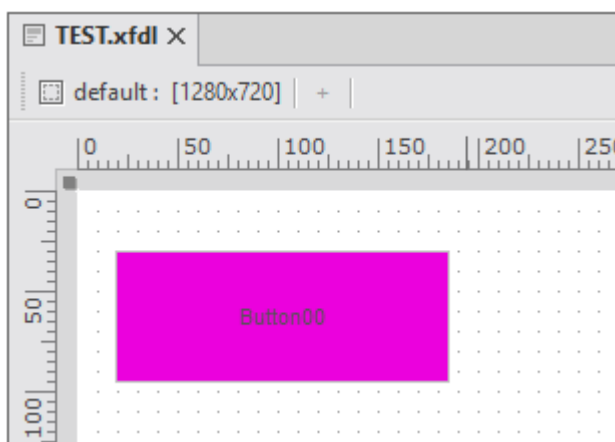
- 5 Nexacro Style Sheet 목록에서 스타일을 디자인할 선택자를 선택한 후 Xcss Editor에서 적용할 스타일을 만듭니다.

Xcss Editor의 상단의 메뉴를 이용하면 자동으로 스타일 코드가 생성되며 Preview 창에서 즉시 효과를 확인할 수 있습니다. Xcss Editor에서 사용자가 직접 코드를 입력하는 방법도 가능합니다.



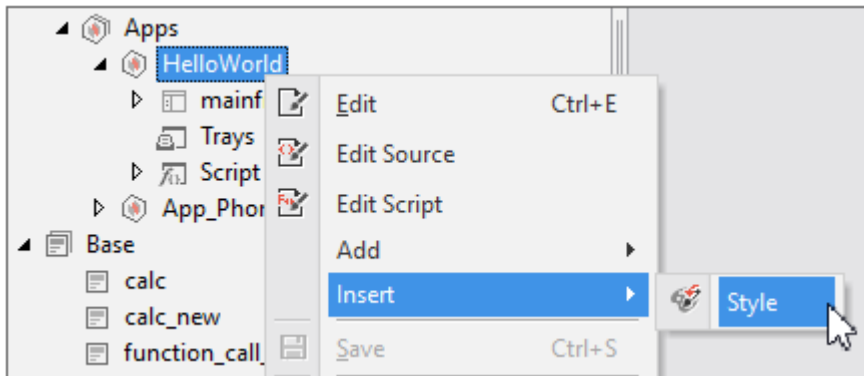
위와 같이 .Button 선택자의 스타일 코드를 작성한 후 HelloWorld.xcss 파일을 저장합니다.

- 6 폼 디자인 화면에서 Button 컴포넌트를 배치하고 XCSS의 스타일 효과가 반영되는지 확인합니다. XCSS 스타일 속성은 작성하고 저장되는 즉시 효과가 나타납니다.



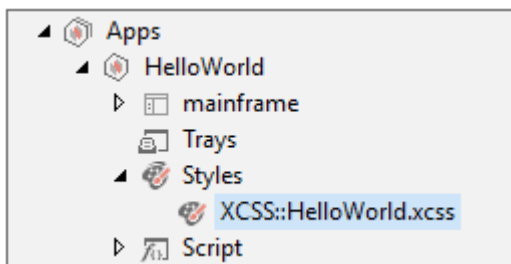
7.2 XCSS 파일 재사용하기

- 1 Project Explorer에서 다음과 같이 Applcaton 이름을 선택한 후 마우스 오른쪽 버튼을 클릭하여 컨텍스트 메뉴에서 [Insert > Style]을 선택합니다.

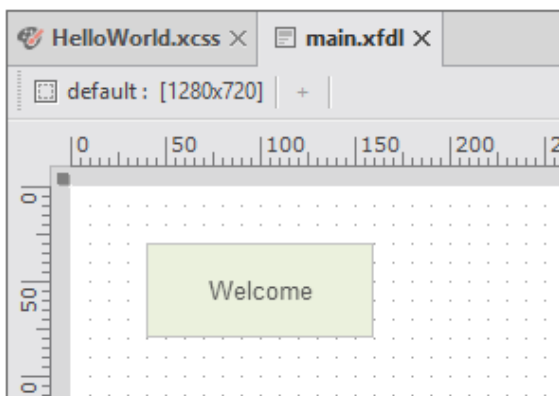


- 2 Choose a Nexacro Style Sheet Files 창이 오픈되면 기존에 작성한 XCSS 파일을 선택 후 열기 버튼을 클릭합니다.
- 3 XCSS 파일의 임포트가 성공했는지 확인합니다.

임포트를 성공적으로 수행하면 Project Explorer에서 임포트한 XCSS 파일을 확인할 수 있습니다. XCSS는 프로젝트 애플리케이션의 Styles 밑에 다음과 같이 추가됩니다.



- 4 폼 디자인 화면에서 컴포넌트를 배치하고 XCSS의 스타일 효과가 반영되는지 확인합니다.





임포트한 XCSS 스타일이 폼 디자인 화면에 적용되려면 컴포넌트에 해당하는 선택자가 XCSS에 정의되어 있어야 합니다.

7.3 cssclass 속성으로 Button 컴포넌트 배경색 변경하기

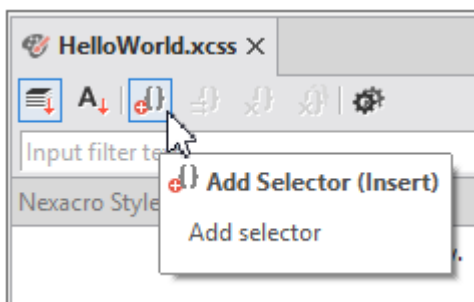
XCSS와 컴포넌트의 cssclass 속성을 이용하여 원하는 컴포넌트에만 스타일 효과를 줄 수 있습니다. cssclass 속성에 XCSS에서 정의한 클래스 선택자(Class Selector)를 설정하면 XCSS에서 해당 클래스 선택자 명으로 정의한 스타일이 즉시 적용됩니다.

cssclass 속성을 사용하려면 우선 Button 컴포넌트에 적용할 XCSS 파일을 만들고 클래스 선택자를 정의해야 합니다. XCSS를 만드는 과정은 생략하고 여기서는 클래스 선택자를 선언하는 부분부터 설명합니다.

① XCSS 파일을 생성합니다. ([넥사크로 스타일 시트\(XCSS\) 만들기](#)를 참조하십시오.)

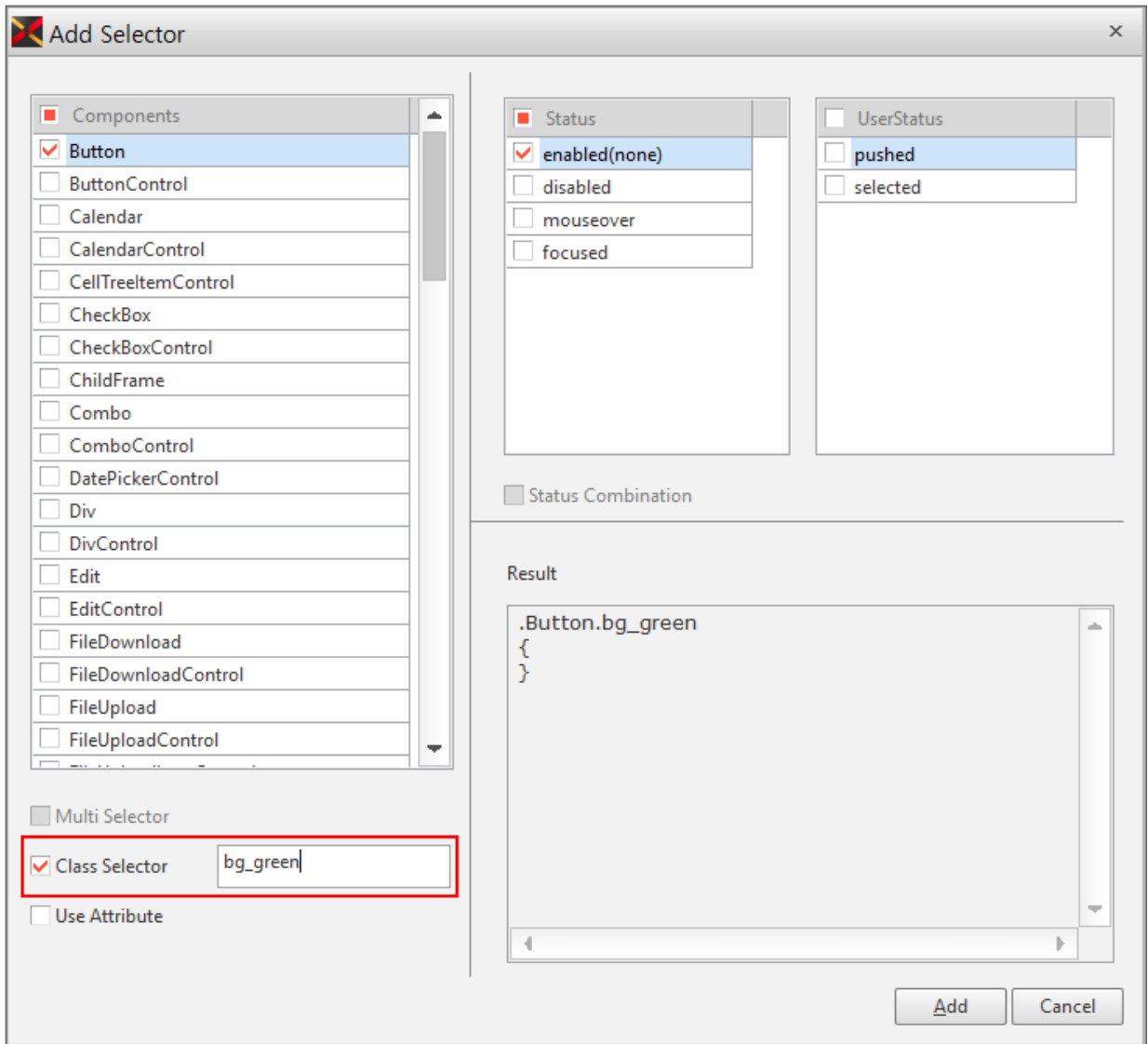
② Xcss Editor에서 Add Selector 버튼을 클릭하여 새로운 선택자를 추가합니다.

생성된 XCSS는 아무 내용이 없는 빈 파일입니다. 스타일 효과를 적용하려면 컴포넌트에 해당하는 선택자를 추가해야 합니다. Xcss Editor의 왼쪽 위에서 Add Selector (Insert) 버튼을 클릭하여 새로운 선택자를 추가합니다.



③ Add Selector 창에서 스타일을 정의할 컴포넌트를 선택하고 Class Selector 명을 설정합니다.

해당 컴포넌트에 어떤 스타일 속성이 있는지 모를 경우에는 Use Attribute 체크박스를 선택하여 선택한 컴포넌트에서 사용할 수 있는 스타일 속성을 모두 생성합니다. 단, 아무 값도 설정되지 않은 빈 속성이 생성됩니다.



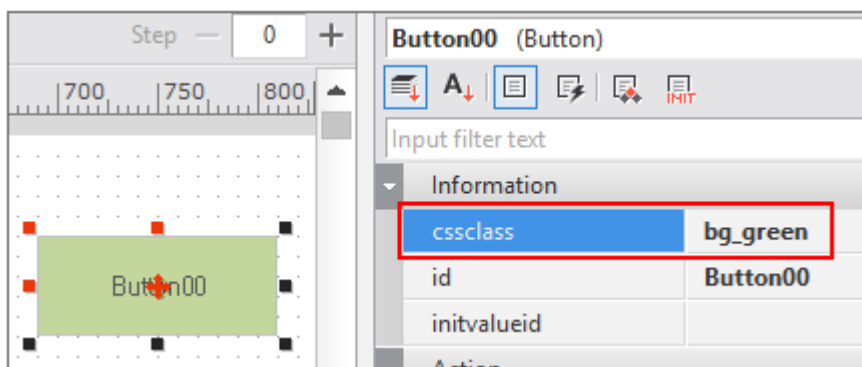
- ④ Xcss Editor에서 컴포넌트의 스타일 속성을 설정합니다.

Xcss Editor를 사용해 다음과 같이 background 속성을 #c3d69b로 -nexa-text-align 속성을 center로 설정한 후 XCSS 파일을 저장합니다.

```
.Button.bg_green
{
    background : #c3d69b;
    -nexa-text-align : center;
}
```

- ⑤ 폼 디자인 화면에서 Button 컴포넌트를 생성한 후 cssclass 속성을 bg_green으로 설정합니다.

Button 컴포넌트에 bg_green 선택자로 정의한 스타일이 적용되는지 확인합니다.



8.

공통 스크립트

8.1 공통 스크립트 소개

공통 스크립트는 프로젝트에서 자주 사용하는 기능을 모아 놓은 일종의 스크립트 라이브러리입니다. 특정 폼에만 필요한 스크립트는 폼 파일에 작성하지만 애플리케이션 전체에서 사용하는 스크립트는 별도의 파일(XJS)에 작성하여 각 폼에서 이를 참조해 사용하는 것이 좋습니다.



XJS는 넥사크로플랫폼에서 사용하는 스크립트 파일의 확장자입니다.

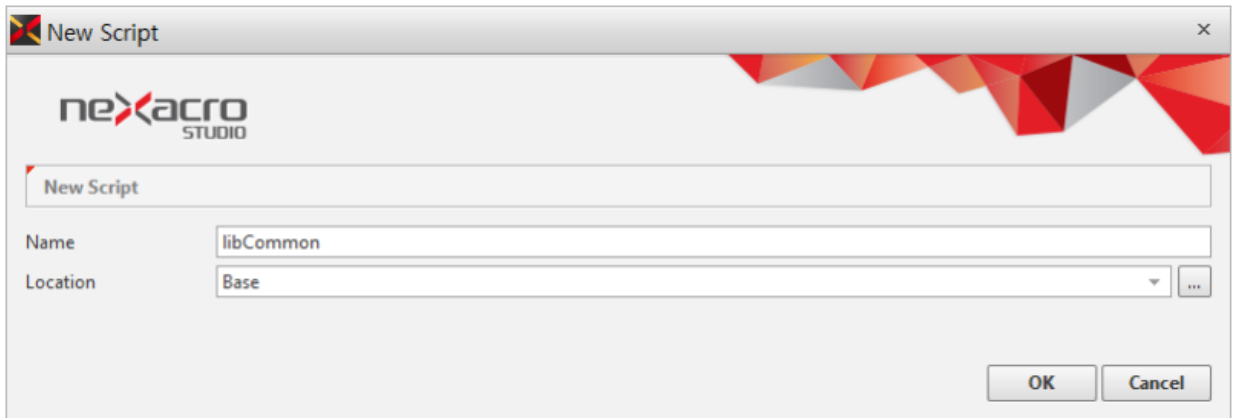
공통 스크립트를 한번 만들어 놓으면 같은 기능을 여러 번 개발하는 중복을 피할 수 있고, 여러 명이 작업할 때 통일된 기능을 사용할 수 있어 효율적입니다. 또한, 공통 스크립트 파일을 그대로 다른 프로젝트에서도 사용할 수 있으므로 코드의 재사용, 개발 시간 단축의 효과를 얻을 수 있습니다. 그러나 불필요하게 남용하면 애플리케이션의 크기가 커져 리소스 낭비와 속도 지연 등의 원인이 될 수 있으므로 필요한 곳에만 사용하는 것이 좋습니다.

일반적으로 공통 스크립트로 구현하는 기능은 문자 및 날짜 체크, 문자열 처리, 형식 검증, 각종 연산, 인/디코딩, 통신 연결 등의 공통으로 사용하는 기능은 무엇이든 될 수 있으며 프로젝트에 따라 내용은 달라질 수 있습니다.

8.2 XJS 작성하기

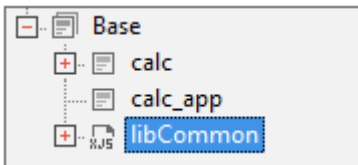
공통 스크립트는 XJS 파일에 작성합니다. 프로젝트에 XJS 파일을 생성한 후 필요한 기능을 작성하면 각 폼에서 이를 참조하여 사용할 수 있습니다. XJS를 작성하는 절차는 다음과 같습니다.

- 1 메뉴에서 [File > New > Script (.xjs)]를 선택하여 New Script 팝업창을 엽니다.



- ② Name에는 libCommon, Location은 기본 값인 Base를 설정한 후 OK 버튼을 클릭합니다. 그러면 XJS 파일이 생성되고 스크립트 편집 창에서 자동으로 오픈됩니다.

생성된 XJS 파일은 다음과 같이 Project Explorer에서도 확인할 수 있습니다.



- ③ 스크립트 편집 창에 오픈된 XJS 파일에 필요한 기능을 스크립트로 작성합니다.

함수를 선언할 때에는 this 키워드를 붙여줍니다. this를 붙여주지 않으면 전역으로 처리됩니다.

```
/* libCommon.xjs */
this.isNumber = function(str)
{
    var retVal = nexacro.isNumeric(str);
    return retVal;
}

this.fn_add = function(x, y)
{
    var retAdd = nexacro.toNumber(x) + nexacro.toNumber(y);
    return retAdd;
}
```

8.3 XJS 사용하기

폼에서 공통 스크립트를 사용하려면 해당 페이지에서 include 문을 선언해 주는 것으로 간편히 사용할 수 있습니다.

- 1 Project Explorer에서 폼을 더블 클릭하여 오픈한 후 스크립트 편집 창의 상단에 include 문으로 XJS 사용을 선언합니다.

```
include "Base::libCommon.xjs";
```

include 문을 쓰고 "" 안에 XJS 파일의 서비스 ID와 파일명을 써줍니다. 그리고 코드의 끝에는 반드시 세미콜론 (;)을 붙여 해당 줄이 끝났음을 명시해 줍니다.



공통으로 참조하는 스크립트가 많은 경우 여러 파일을 include로 참조할 수 있습니다. 또한, 참조하는 파일이 많은 경우에는 스크립트 코드 안에서 다시 다른 코드를 참조할 수도 있습니다. 예를 들어, libCommon.xjs 스크립트 파일을 다음과 같이 구성할 수 있습니다. 이렇게 구성하면 참조할 파일이 늘어나더라도 업무에서 사용하는 폼 파일은 건드리지 않고 libCommon.xjs 파일만 수정해 기능을 추가할 수도 있습니다.

```
/* libCommon.xjs */
include "Lib::libAuth.xjs";
include "Lib::libGrid.xjs";
include "SERVICE::libService.xjs";
include "Util::libUtil.xjs";
```

- 2 폼 스크립트에서 함수를 호출합니다.

폼 스크립트에서 다음과 같이 libCommon.xjs에 정의된 fn_add 함수를 사용할 수 있습니다.

```
/* calc.xfdl */

include "Base::libCommon.xjs";

this.btn_calc_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var result = this.fn_add(5, 2);    //result = 7
};
```

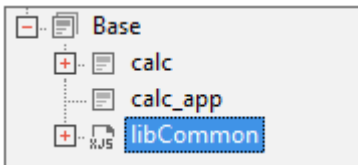
8.4 XJS 가져오기

이전에 작성했던 XJS 파일을 프로젝트로 가져와 사용할 수 있습니다.

- 1 XJS 파일을 프로젝트 경로 내의 Base 디렉터리로 복사합니다.

XJS 파일을 Base 디렉터리 이외의 경로에 복사하면 프로젝트에서 사용할 수 없습니다. XJS 파일을 Base 디렉터리 이외의 경로에서 사용하려면 User Service에 해당 경로를 등록해야 합니다. 자세한 내용은 다음 장을 참조하십시오.

- 2 Project Explorer를 refresh 하여 Base 하위에 복사한 XJS 파일이 표시되는지 확인합니다.

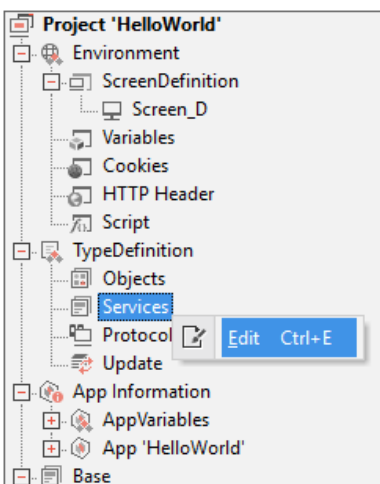


- 3 폼 스크립트에서 XJS를 include하여 사용합니다.

8.5 서비스 등록하기

XJS 파일을 별도의 경로에 두고 사용하려면 해당 경로를 서비스로 등록합니다.

- 1 Project Explorer의 [TypeDefinition > Services]를 선택 후 마우스 오른쪽 버튼을 클릭하여 컨텍스트 메뉴에서 [Edit] 항목을 선택하고 서비스 편집 창을 엽니다.

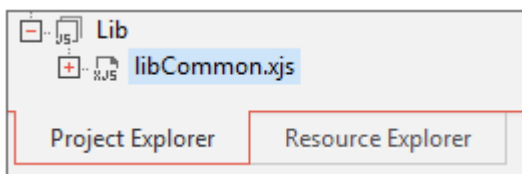


- ② User Service에서 + 버튼을 클릭하여 서비스 ID(PrefixID), 파일 형식(Type), 경로(URL) 정보를 입력합니다. 그 이외의 항목은 디폴트 값으로 둡니다.

	PrefixID	Type	URL	CacheLevel
—	Base	form	./Base/	session
—	Lib	js	./Lib/	session

- ③ Project Explorer에서 서비스 경로를 확인합니다.

새로운 서비스가 등록되면 서비스와 해당 경로에 위치한 XJS 파일이 Project Explorer에 표시되고 관리가 가능해집니다.



8.6 App에서 스크립트 사용하기

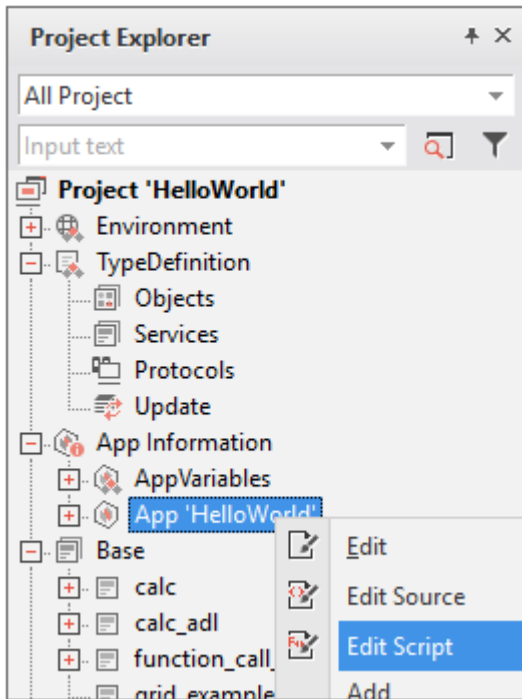
App(XADL)은 프로젝트의 메인에 해당되는 부분으로 애플리케이션을 로딩할 때 호출됩니다. XADL 파일의 스크립트 코드에는 주로 애플리케이션의 시작이나 종료 시에 필요한 기능을 작성하는데 필요에 따라 공통 스크립트를 이곳에 정의해 사용하기도 합니다.

XADL에 공통 스크립트를 작성하면 초기에 application 객체와 함께 로딩됩니다. 따라서 별도의 include 절차가 필요 없으며 객체를 this 객체로 정의하면 application 객체의 자식으로 동작합니다.



XADL 파일을 변경할 때마다 index.html도 자동으로 업데이트되므로 index.html에 추가 코딩을 한 경우에는 별도로 관리를 해줘야 합니다.

- ① Project Explorer에서 프로젝트의 **App '[프로젝트명]'**을 선택한 후 마우스 오른쪽 버튼을 클릭하여 컨텍스트 메뉴에서 [Edit Script]를 선택합니다.



- ② XADL 파일이 스크립트 편집 창으로 오픈되면 다음과 같이 기능을 구현합니다.

```
/* calc.xadl */

this.adl_add = function(x, y)
{
    var retAdd = nexacro.toNumber(x) + nexacro.toNumber(y);
    return retAdd;
}
```

- ③ 폼에서 XADL에 정의된 함수를 사용합니다.

nexacro.getApplication 메소드를 사용하여 application 객체를 얻은 후 calc.xadl에 정의된 adl_add 함수를 다음과 같이 사용할 수 있습니다.

```
/* calc_adl.xfdl */

var objApp = nexacro.getApplication();
result = objApp.adl_add(5, 2);    //result = 7
```

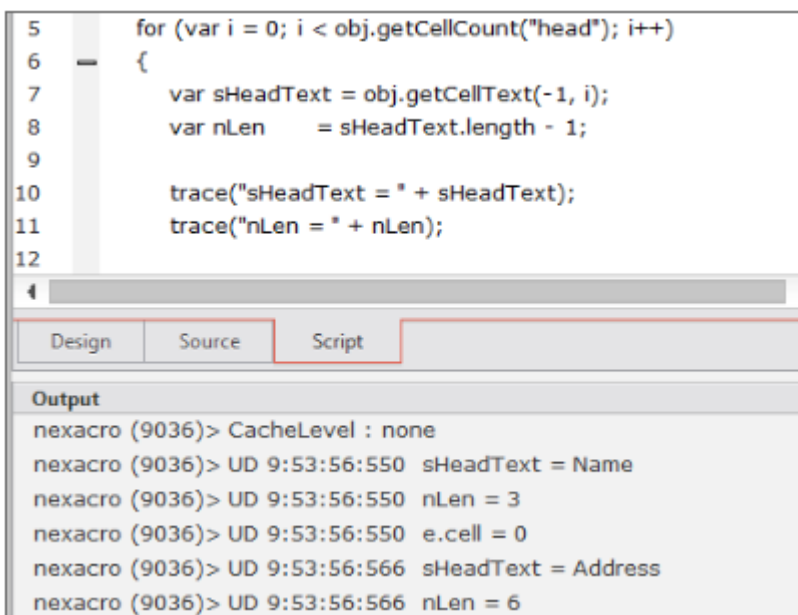
9.

디버그 기능

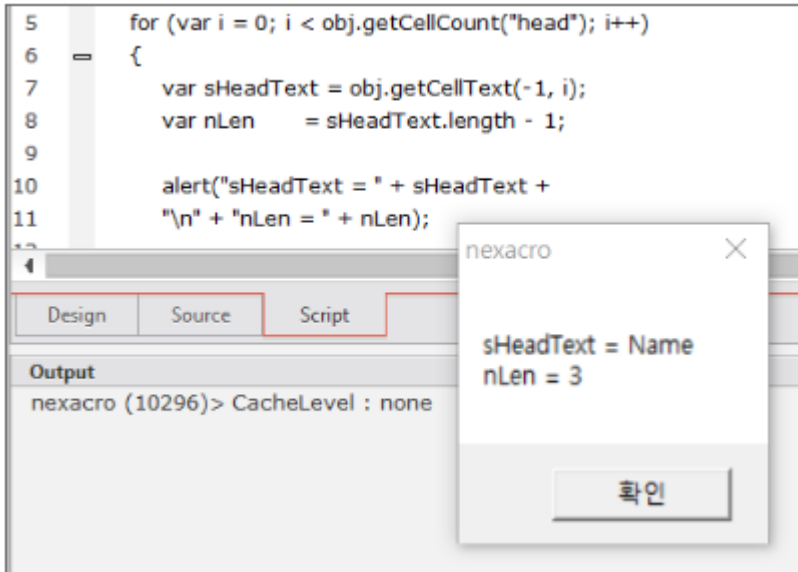
9.1 전통적 디버깅

디버그란 애플리케이션 프로그램 내에 존재하는 비정상적인 동작이나 논리상의 오류와 같은 버그를 찾아내어 수정하는 작업을 말합니다. 처음부터 완벽한 결과물을 만들어 낼 수 없는 개발의 특성상 시작 단계에서부터 완료 시까지 계속해서 디버깅 과정이 이뤄집니다.

개발자가 가장 손쉽게 사용할 수 있는 디버깅 방법은 확인이 필요한 객체나 변수 등을 화면에 출력하는 것입니다. 이렇게 출력창에 남겨진 기록을 로그라 하며 넥사크로플랫폼에서는 trace 메소드를 사용할 수 있습니다. trace는 추적한다는 의미 그대로 개발자가 로직의 흐름이나 변수의 값을 추적할 수 있도록 로그를 남기는 기능을 합니다. 다음과 같이 텍스트 형태로 기술하면 Output 창과 로그 파일에 정보를 출력합니다.



개발 과정에서 trace만큼 많이 사용하는 것이 alert 메소드입니다. trace는 남겨진 로그에서 자기가 필요로 하는 부분을 찾아 확인해야 하는 번거로움이 있지만 alert은 화면에서 바로 팝업으로 오픈하여 필요한 결과만을 확인할 수 있으므로 편리하다는 장점이 있습니다.



trace나 alert은 사용하기 쉽다는 장점이 있으나 프로젝트가 많은 파일로 구성되거나 코드가 길고 복잡한 경우에는 오히려 디버깅을 복잡하게 만듭니다. 또한, 여러 변수나 객체 정보를 출력할 때 공간상의 제약이 많으며 정보를 확인하기 불편합니다. 더구나 스크립트로 일일이 코딩을 해야 하며 디버깅이 끝난 다음에는 개발자가 직접 삭제해줘야 하므로 그 과정에서 새로운 문제를 일으킬 수도 있습니다.

넥사크로 스튜디오에서는 개발자가 좀 더 효율적으로 디버깅할 수 있도록 자체 디버그 기능을 포함하고 있습니다. 이번 장에서는 넥사크로 스튜디오에서 제공하는 디버그 기능과 그 사용법에 대해 설명합니다.

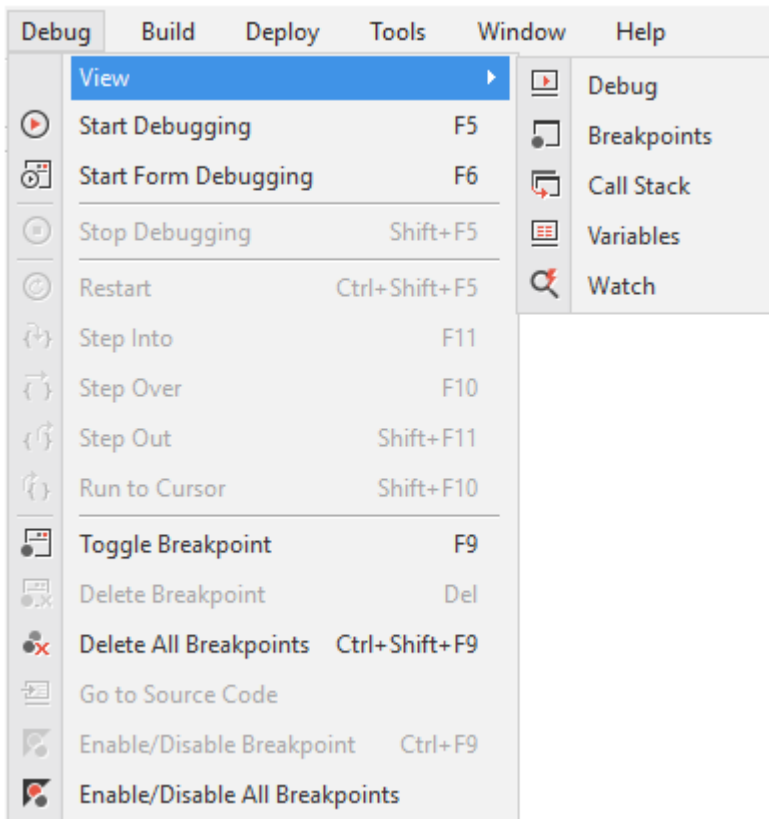
9.2 디버그 기능을 활용한 디버깅

앞서 설명한 trace나 alert 메소드는 사용이 간편하지만, 프로세스의 흐름이나 많은 정보를 담고 있는 객체를 확인하기에 한계가 있습니다. 또한, 현재 코드상에 어떤 문제가 있는지 명확히 파악하기 힘듭니다. 하지만 디버그 기능을 사용하면 문제가 어디서부터 어떤 경로를 통해 해당 지점까지 오게 되었는지 등의 구체적인 정보를 추적하기에 좀 더 수월합니다.

넥사크로 스튜디오의 디버그 기능은 명령어를 실행할 수 있는 메뉴 및 툴바와 객체, 함수, 로그 등의 정보를 볼 수 있는 디버그 정보창으로 구성됩니다.

9.2.1 디버그 메뉴와 툴바

넥사크로 스튜디오의 디버그 메뉴와 툴바에서는 디버깅에 필요한 명령어를 사용할 수 있습니다. 디버그 메뉴와 툴바의 구성은 조건부 중단점 관련 명령을 제외하고는 같습니다.



	항목	설명
1	Start Debugging (F5) Continue (F5)	애플리케이션 디버깅을 시작합니다. 디버깅을 계속해서 진행합니다. (디버깅이 시작되면 Start Debugging 메뉴가 Continue 메뉴로 전환됩니다)
2	Start Form Debugging (F6)	현재 폼에 대한 디버깅을 시작합니다.
3	Stop Debugging (Shift + F5)	디버깅을 종료합니다.
4	Restart (Ctrl + Shift + F5)	디버깅을 종료한 후 다시 시작합니다.
5	Step Into (F11)	한 줄씩 코드를 실행합니다. 중간에 함수가 호출되면 해당 함수로 넘어가 코드를 실행합니다.
6	Step Over (F10)	함수 단위로 코드를 실행합니다.
7	Step Out (Shift + F11)	함수에서 나갑니다.
8	Run to Cursor (Shift + F10)	현재 위치에서 커서의 위치까지 실행합니다.
9	Toggle Breakpoint (F9)	중단점을 지정하거나 해제합니다.
10	Set Conditional Breakpoint	조건부 중단점을 지정합니다.
11	Edit Breakpoint	지정된 중단점에 조건을 설정합니다.
12	Delete All Breakpoints (Ctrl + Shift + F9)	지정된 모든 중단점을 해제합니다.
13	Go to Source Code	Breakpoints 창의 중단점 목록에서 해당 중단점의 소스 위치로 이동합니다.

	항목	설명
14	Enable/Disable Breakpoint (Ctrl + F9)	지정된 중단점을 활성화 혹은 비활성화합니다.
15	Enable/Disable All Breakpoints	지정된 모든 중단점을 활성화 혹은 비활성화합니다.

주요 디버그 명령은 단축키를 지원합니다. 실제 디버깅할 때에는 디버그 명령을 메뉴나 툴바에서 선택해서 사용하기 보다는 단축키를 사용하는 것이 속도 면에서 훨씬 효과적입니다.

9.2.2 디버그 정보창

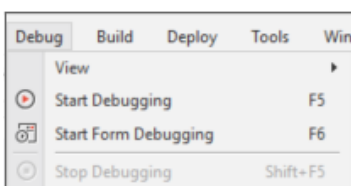
기능	설명
Debug	디버깅 모드에서 발생하는 메시지를 표시합니다.
Breakpoints	디버깅 모드에서 일시 정지할 위치를 스크립트 코드상에 지정할 수 있습니다. 중단점으로 지정된 위치는 Breakpoints 창에서 목록 형태로 확인할 수 있습니다.
Call Stack	현재 상태까지 거처온 함수를 역순으로 표시해 줍니다. 함수를 더블 클릭하면 해당 소스의 위치로 이동합니다.
Variables	현재 상태에서 확인이 가능한 변수, 객체의 상태 값을 조회합니다. 변수인 경우에는 해당 값을 보여주고, 객체인 경우에는 해당 객체의 속성을 트리 형태로 조회할 수 있습니다.
Watch	특정 변수나 객체의 값을 등록해 놓고 변화되는 값을 관찰할 수 있습니다. 특히 스텝 기능을 사용할 때 유용합니다.

9.3 디버그 기능 사용하기

9.3.1 디버깅 모드로 애플리케이션 실행/종료하기

디버깅 모드는 디버깅을 위한 애플리케이션의 실행 형태로 디버그 기능을 사용해 애플리케이션의 오류를 추적하고 해결할 수 있는 환경을 제공합니다.

디버깅 모드는 Debug 메뉴에서 [Start Debugging] 혹은 [Start Form Debugging]을 선택하여 실행할 수 있습니다. [Start Debugging]은 프로젝트 전체를, [Start Form Debugging] 현재 오픈된 폼만을 실행할 때 사용합니다.



디버깅 모드를 종료하려면 Debug 메뉴에서 [Stop Debugging]을 선택합니다.



중단점을 설정하지 않은 상태에서도 디버깅 모드로 애플리케이션을 실행할 수 있습니다. 그러나 디버깅 기능들을 사용할 수 없으므로 실행 자체가 무의미합니다. 그러므로 중단점을 먼저 지정한 후 디버깅 모드로 실행하시기 바랍니다.

9.3.2 중단점 추가/삭제하기

중단점은 디버깅 모드에서 일시 정지할 위치를 지정할 때 사용합니다. 이렇게 지정된 위치는 스크립트 편집 창과 Breakpoints 창에서 확인할 수 있습니다. 스크립트 편집 창에서는 코드의 왼쪽에 위치한 줄 번호 옆에 마커가 찍혀 있어 중단점의 설정 상태를 확인할 수 있습니다. Breakpoints 창에서는 목록 형태로 중단점을 표시하는데 이를 더블 클릭하면 중단점이 찍혀 있는 소스 위치로 이동할 수 있습니다.

중단점을 한번 지정하면 넥사크로 스튜디오를 종료해도 사용자가 삭제하기 전까진 상태가 계속 유지됩니다.



마커란?

중단점이 지정되었음을 알려주는 일종의 표식입니다. 중단점의 설정 상태에 따라 다음의 네 가지 모양을 갖습니다.

- (Enable): 중단점을 지정한 줄에서 실행을 멈춥니다.
- (Disable): 중단점을 비활성화합니다. 애플리케이션 실행 시 무시됩니다.
- (Conditional Enable): 중단점에 설정된 조건이 일치할 경우에만 실행을 멈춥니다.
- (Conditional Disable): 조건부 중단점을 비활성화합니다. 애플리케이션 실행 시 무시됩니다.

중단점을 지정하면 해당 줄의 앞에 마커가 추가되고 Breakpoints 창에 중단점 정보가 등록됩니다. 또한, 한번 지정된 중단점은 사용자가 삭제하기 전까진 넥사크로 스튜디오를 종료해도 상태가 계속 유지됩니다.

중단점을 추가/삭제하는 방법을 예를 들어 설명합니다.

```

15  this.btn_calc onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
16  -{
17      var result = 0;
18      var x = this.edt_operand_x.value;
19      var y = this.edt_operand_y.value;
20      ●
21      if(!fn_validate(x) || !fn_validate(y))
22      - {
23          this.stt_msg.set_text("Operand is not number.");
24      }
25      else
26      - {
27          result = fn_add(x, y);
28          this.stt_result.set_text(result);
29      }
30  };
  
```

- ① 스크립트 편집 창에서 20번 줄을 선택합니다.
- ② 20번 줄의 줄 번호를 클릭하거나 메뉴에서 [Toggle Breakpoint]를 선택하면 중단점이 지정됩니다. 중단점으로 지정되면 줄 번호 옆에 마커가 생깁니다.



중단점을 삭제하려면 삭제하려는 위치의 줄 번호를 클릭하거나 툴바의 [Toggle Breakpoint]를 선택합니다. [Delete Breakpoint]를 선택해도 결과는 같습니다. 중단점을 삭제하면 줄번호 옆의 마커가 없어지고 Breakpoints 창의 중단점 정보도 없어집니다.

지정한 모든 중단점을 한 번에 삭제하려면 메뉴나 툴바에서 [Delete All Breakpoints]를 선택합니다.

9.3.3 조건부 중단점 추가/삭제하기

사용자는 필요에 따라 중단점을 특정 조건에만 적중하도록 설정할 수 있습니다. 이렇게 조건을 설정한 중단점을 조건부 중단점이라 합니다.

조건 표현식을 설정하면 애플리케이션 수행이 해당 라인에 도달 시 설정된 조건에 맞는 경우에만 중단점이 적중합니다. 무시 횟수를 설정하면 코드 실행이 해당 라인에 도달해도 중단점은 무시되며 설정한 횟수를 넘어간 이후부터 적중합니다.

조건부 중단점의 기본적인 사용법은 중단점과 같으나 조건 표현식(Conditional Expression)과 무시 횟수(Number of Ignore)를 설정할 수 있다는 차이가 있습니다.

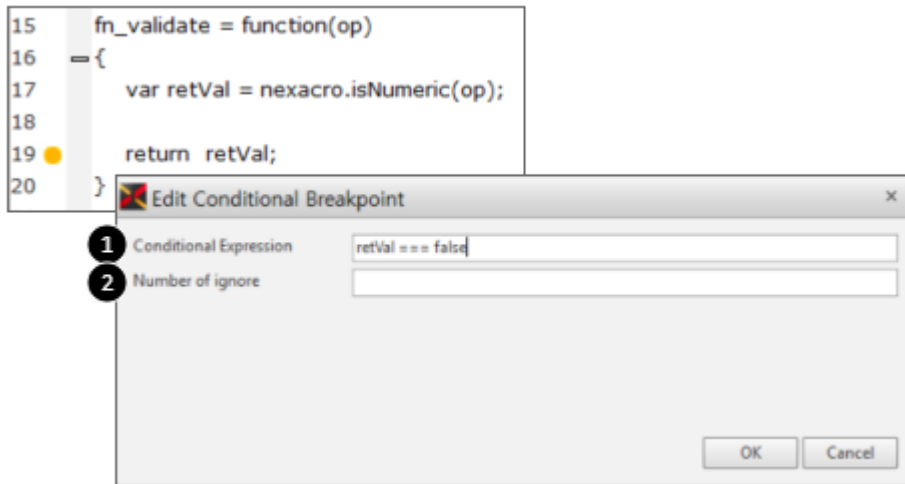
조건부 중단점을 추가/삭제하는 방법을 예를 들어 설명합니다.

- ① 스크립트 편집 창을 클릭하여 활성화 시킵니다.
- ② 스크립트 편집 창에서 19번 줄을 선택하고 툴바 혹은 Breakpoints 창에서 [Set Conditional Breakpoint]를 선택합니다.

```

15  fn_validate = function(op)
16  = {
17      var retVal = nexacro.isNumeric(op);
18
19      return retVal;
20  }
```

- ③ Add Conditional Breakpoint 팝업이 오픈되면 Conditional Expression에 조건 표현식을 입력합니다.



	이름	설명
1	Conditional Expression	중단점을 적중시킬 조건 표현식을 설정합니다.
2	Number of ignore	설정된 횟수만큼 중단점을 무시합니다.

입력을 완료하면 줄 번호 옆에 주황색 마커가 생기고 Breakpoints 창에서 다음과 같이 설정 정보를 확인할 수 있습니다.

File	Line	Condition	Hit Count
D:\Project\NP17\HelloWorld\Base\calc.xfdl	26		0
D:\Project\NP17\HelloWorld\Base\calc.xfdl	19	retVal === true	0
D:\Project\NP17\HelloWorld\Base\calc.xfdl	43		0
D:\Project\NP17\HelloWorld\Base\calc.xfdl	38		0
D:\Project\NP17\HelloWorld\Base\calc.xfdl	34		0



조건부 중단점을 삭제하려면 삭제하려는 위치의 줄 번호를 클릭하거나 툴바의 [Toggle Breakpoint]를 선택합니다. [Delete Breakpoint]를 선택해도 결과는 같습니다. 중단점을 삭제하면 줄번호 옆의 마커가 없어지고 Breakpoints 창의 중단점 정보도 없어집니다. 지정한 모든 중단점을 한 번에 삭제하려면 메뉴나 툴바에서 [Delete All Breakpoints]를 선택합니다.



이미 중단점으로 지정된 줄이라도 툴바의 [Edit Breakpoint]를 사용하여 조건식이나 무시 횟수를 설정하면 조건부 중단점으로 변경됩니다.

9.3.4 중단점 활성화/비활성화하기

지정한 중단점은 필요에 따라 활성화 혹은 비활성화 상태로 설정할 수 있습니다. 이 기능은 지정한 중단점을 다음에 다시 사용해야 할 필요가 있을 때 일시적으로 중단점이 동작하지 못하도록 할 때 사용합니다.

중단점을 활성화 혹은 비활성화하면 해당 줄의 마커 표시와 Breakpoints 창의 중단점 정보가 상태에 맞게 변경됩니다.

```

35     if(!fn_validate(x) || !fn_validate(y))
36     =   {
37         this.stt_msg.set_text("Operand is not number.");
38 ○      trace("Operand is not number.");
39     }

```

중단점을 활성/비활성화하는 방법을 예를 들어 설명합니다.

- ① 스크립트 편집창에서 38번 줄에 중단점을 지정합니다.
- ② 중단점이 지정된 38번 줄을 선택한 후 툴바에서 [Enable/Disable Breakpoint]를 선택합니다.



비활성화된 중단점을 다시 활성화하려면 툴바의 [Enable/Disable Breakpoint]를 다시 한번 선택합니다. 그리고 지정된 모든 중단점을 한 번에 활성화 혹은 비활성화하려면 툴바에서 [Enable/Disable All Breakpoints]를 선택합니다.

9.3.5 디버깅 모드에서 코드 실행하기(Continue)

디버깅 모드에서 계속해서 코드를 실행하려면 Continue 명령을 사용합니다. Continue를 실행하면 현재 중단점부터 다음 중단점 사이의 코드를 실행하고 다음 중단점 위치에서 멈춥니다.

Continue 명령으로 코드를 실행하는 방법을 예를 들어 설명합니다.

- ① 스크립트 편집창에서 34번 줄과 44번 줄에 중단점을 지정합니다.

```

29  this.btn_calc_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
30  -{
31      var result = 0;
32      var x = this.edt_operand_x.value;
33      var y = this.edt_operand_y.value;
34  ●
35      if(!fn_validate(x) || !fn_validate(y))
36      - {
37          this.stt_msg.set_text("Operand is not number.");
38      }
39      else
40      - {
41          result = fn_add(x, y);
42          this.stt_result.set_text(result);
43      }
44  ● };

```

- ② 디버깅 모드로 애플리케이션을 실행하면 34번 줄까지 코드가 실행된 후 멈춥니다.
- ③ 이때 메뉴나 툴바에서 [Continue]를 선택하면 35번 줄부터 43번 줄까지의 코드를 모두 실행하고 그다음 중단점인 44번 줄에서 멈춥니다.

9.3.6 Step 명령으로 코드 실행하기(Step Into, Step Over, Step Out)

중단점을 사용할 때에는 필요한 곳에만 사용하는 것이 좋습니다. 코드가 길고 복잡한 애플리케이션을 디버깅할 때 중단점을 남발하면 중단점을 지정한 횟수만큼 사용자가 직접 Continue 명령을 입력해야 하므로 실행 효율성이 떨어집니다. 따라서 특정 부분에서 줄 단위로 코드를 실행하며 디버깅이 필요한 경우에는 Step 명령을 사용하는 것이 편리합니다.

Step 명령은 일종의 임시 중단점 기능을 합니다. 쉽게 말해 중단점을 지정하지 않는 위치에서 애플리케이션 실행을 멈춰 디버깅할 때 사용할 수 있습니다. Step 명령은 동작 방식에 따라 Step Into, Step Over, Step Out으로 나뉩니다.



Step 명령을 사용하여 디버깅 시에도 진행 중간에 중단점이 존재하면 Step 명령과 상관없이 중단점 위치에서 실행이 멈춥니다.

다음은 사용자로부터 숫자 두 개를 입력받아 더한 결과를 출력하는 예제 샘플입니다. btn_calc_onclick 함수는 버튼 클릭을 통해 발생하는 이벤트 함수로 사용자가 숫자 17과 2를 입력하고 연산 버튼을 클릭해서 함수 호출이 일어났다고 가정합니다.

다음은 사용자로부터 숫자 두 개를 입력받아 더한 결과를 출력하는 예제 샘플입니다. 이 예제를 사용하여 Step 명령으로 코드를 실행하는 방법에 대해 설명합니다.

```

1  fn_validate = function(op)
2  = {
3      var retVal = nexacro.isNumeric(op);
4
5      return retVal;
6  }
7
8  fn_add = function(x, y)
9  = {
10     var retAdd = nexacro.toNumber(x) + nexacro.toNumber(y);
11
12     return retAdd;
13 }
14
15 this.btn_calc_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
16 = {
17     var result = 0;
18     var x = this.edt_operand_x.value;
19     var y = this.edt_operand_y.value;
20
21     if(!fn_validate(x) || !fn_validate(y))
22     = {
23         this.stt_msg.set_text("Operand is not number.");
24     }
25     else
26     = {
27         result = fn_add(x, y);
28         this.stt_result.set_text(result);
29     }
30 };

```

Step Into 사용법

코드를 한 줄씩 실행합니다. 함수 호출을 만나면 해당 함수 내부로 이동하여 코드를 한 줄씩 실행합니다.

- ① 스크립트 편집 창에서 20번 줄에 중단점을 지정합니다.
- ② 디버깅 모드로 애플리케이션을 실행하면 20번 줄까지 코드가 실행된 후 멈춥니다.
- ③ 메뉴나 툴바에서 [Step Into]를 선택하면 아래와 같이 fn_validate 함수의 첫 번째 줄로 이동합니다. 만일 한 번 더 [Step Into]를 선택하면 3번 줄을 실행하는데 여기서 다시 isNumeric 함수의 호출이 발생하므로 isNumeric 함수의 첫 번째 줄로 이동하게 됩니다.

```

1  fn_validate = function(op)  op = "17"
2  = {
3  ▶    var retVal = nexacro.isNumeric(op);
4
5      return retVal;
6  }

```


Step Over 사용법

코드를 한 줄씩 실행합니다. 실행 중 함수 호출을 만나면 함수 내부로 들어가지 않고 함수를 실행해 버린 후 다음 라인으로 이동합니다.

- 1 스크립트 편집 창에서 20번 줄에 중단점을 지정합니다.
- 2 디버깅 모드로 애플리케이션을 실행하면 20번 줄까지 코드가 실행된 후 멈춥니다.
- 3 메뉴나 툴바에서 [Step Over]를 선택하면 21번 줄의 fn_validate 함수를 실행하고 27번 줄로 이동합니다. 한 번 더 [Step Over]를 선택하면 27번 줄의 코드를 실행하고 그다음 줄인 28번 줄로 이동하여 멈춥니다.

```

15  this.btn_calc_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
16  = {
17      var result = 0;    result = 0
18      var x = this.edt_operand_x.value;    x = "17"
19      var y = this.edt_operand_y.value;    y = "2"
20  ●
21      if(!fn_validate(x) || !fn_validate(y))    x = "17", y = "2"
22      = {
23          this.stt_msg.set_text("Operand is not number.");
24      }
25      else
26      = {
27  ▶ |      result = fn_add(x, y);
28          this.stt_result.set_text(result);
29      }
30  };
  
```

Step Out 사용법

현재 실행 중인 함수의 나머지 코드 부분을 실행하고 함수를 호출했던 곳으로 돌아가 그다음 줄로 이동한 후 멈춥니다.

Step Out은 독자적으로 사용하기보다는 Step Into와 같이 사용합니다. Step Into를 수행하며 함수 내부로 이동하여 코드를 수행하다 더는 살펴볼 필요가 없을 때 해당 함수에서의 디버깅을 건너뛰는 용도로 사용합니다.

- 1 스크립트 편집창에서 20번 줄에 중단점을 지정합니다.
- 2 디버깅 모드로 애플리케이션을 실행하면 20번 줄까지 코드가 실행된 후 멈춥니다.
- 3 메뉴나 툴바에서 [Step Into]를 선택하면 21번 줄을 실행하는데 여기서 fn_validate 함수를 호출하므로 fn_validate 함수의 첫 번째 줄로 이동합니다.

```

1  fn_validate = function(op)    op = "17"
2  = {
3  ▶ |      var retVal = nexacro.isNumeric(op);
4
5      return retVal;
6  }
  
```

- ④ 메뉴나 툴바에서 [Step Out]을 선택하면 3번 줄부터 fn_validate 함수의 끝인 5번 줄까지 실행한 후 fn_validate 함수를 호출했던 btn_calc_onclick 함수의 21번 줄로 이동합니다.

```

15  this.btn_calc_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
16  = {
17      var result = 0; result = 0
18      var x = this.edt_operand_x.value; x = "17"
19      var y = this.edt_operand_y.value; y = "2"
20
21  | if(!fn_validate(x) || !fn_validate(y))
22  = {
23      this.stt_msg.set_text("Operand is not number.");
24  }
25  else
26  = {
27      result = fn_add(x, y);
28      this.stt_result.set_text(result);
29  }
30  };

```

9.3.7 커서 위치까지 한번에 코드 실행하기(Run to Cursor)

현재 적중된 중단점에서 커서 위치까지 코드를 실행한 후 멈춥니다. 만일 현재 중단점과 커서 사이에 다른 중단점이 존재하면 해당 중단점에서 실행을 멈춥니다. 또한, 커서의 위치가 분기 상 도달할 수 없는 위치일 경우에는 다음 중단점까지 실행한 후 멈춥니다.

Run to Cursor 명령으로 코드를 실행하는 방법을 예를 들어 설명합니다.

```

29  this.btn_calc_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
30  = {
31      var result = 0; result = 19
32      var x = this.edt_operand_x.value; x = "17"
33      var y = this.edt_operand_y.value; y = "2"
34  |
35  | if(!fn_validate(x) || !fn_validate(y)) x = "17", y = "2"
36  = {
37      this.stt_msg.set_text("Operand is not number.");
38      trace("Operand is not number.");
39  }
40  else
41  = {
42  ↓ Cursor result = fn_add(x, y); result = 19, x = "17", y = "2"
43  | this.stt_result.set_text(result);
44  }

```

- ① 스크립트 편집 창에서 34번 줄에 중단점을 지정합니다.
- ② 디버깅 모드로 애플리케이션을 실행합니다. 입력과 버튼 클릭이 이뤄지면 34번 줄에서 실행이 멈춥니다.

- ③ 43번 줄에 커서를 놓고 메뉴나 툴바에서 [Run to Cursor]을 선택합니다. 그러면 35번 줄부터 42번 줄까지 코드를 실행하고 43번 줄에서 실행이 멈춥니다.

9.3.8 변수 값 확인하기(Variables)

디버깅의 가장 기본은 객체나 변수의 값을 확인하는 것입니다. 이 정보는 Variables 창에서 확인할 수 있습니다.

Variables는 중단점이 적중한 위치에서 접근 가능한 객체나 변수의 값을 표시합니다. 멤버를 포함하는 객체의 경우 목록에서 +/- 버튼으로 상세 내용을 펼치거나 접어서 확인할 수 있습니다. 값이 변경된 항목은 적색 볼드체로 표시됩니다. Variables은 디버깅 모드를 수행하면 자동으로 동작합니다.

Variables	
Name	Value
this	[Form]
obj	[Button]
e	[ClickEventInfo]
result	19
x	"17"
y	"2"

Debug
Watch
Variables

Variables로 변수값을 확인하는 방법을 예를 들어 설명합니다.

- ① 스크립트 편집 창에서 세 변수 x, y, result의 값을 확인하기 위해 34번 줄에 중단점을 지정합니다.

```

29  this.btn_calc_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
30  -{
31      var result = 0;
32      var x = this.edt_operand_x.value;
33      var y = this.edt_operand_y.value;
34  ●
35      if(!fn_validate(x) || !fn_validate(y))
36      - {
37          this.stt_msg.set_text("Operand is not number.");
38      }
39      else
40      - {
41          result = fn_add(x, y);
42          this.stt_result.set_text(result);
43      }
44  };
```

- ② 디버깅 모드로 애플리케이션을 실행합니다. 입력과 버튼 클릭이 이뤄지면 34번 줄에서 실행이 멈춥니다. 이때, Variables 창을 보면 다음과 같이 x, y, result 값을 확인할 수 있습니다.

Variables	
Name	Value
this	[Form]
obj	[Button]
e	[ClickEventInfo]
result	0
x	"17"
y	"2"

9.3.9 변수 값 확인하기(Script Editor)

스크립트 편집기에서는 디버깅 중에 객체/변수값을 직관적으로 확인할 수 있도록 Assist Tip과 Inline Assist Variable 기능을 제공합니다.

Assist Tip은 스크립트 편집기에서 값을 확인하고 싶은 객체나 변수에 마우스 커서를 오버시키면 팝업 형태로 해당 정보를 표시해주는 기능입니다. 해당 항목이 변수면 단순히 값만 표시해 주고 객체면 구성되는 멤버와 그 값까지 트리 형태로 표시해 줍니다.

Inline Assist Variable은 스크립트 편집기에서 함수의 선언부부터 현재 적중된 중단점 사이에 존재하는 모든 객체와 변수의 값을 각 라인의 옆에 표시해 주는 기능입니다. 또한, 적중된 중단점이 속한 함수뿐만 아니라 그와 연관된 Call Stack 상의 함수에서도 해당 함수가 호출되기 이전까지의 객체와 변수값을 표시해 줍니다.

스크립트 편집기에서 변수값을 확인하는 방법을 예를 들어 설명합니다.

- 1 스크립트 편집 창에서 세 변수 x, y, result의 값을 확인하기 위해 34번 줄에 중단점을 지정합니다.

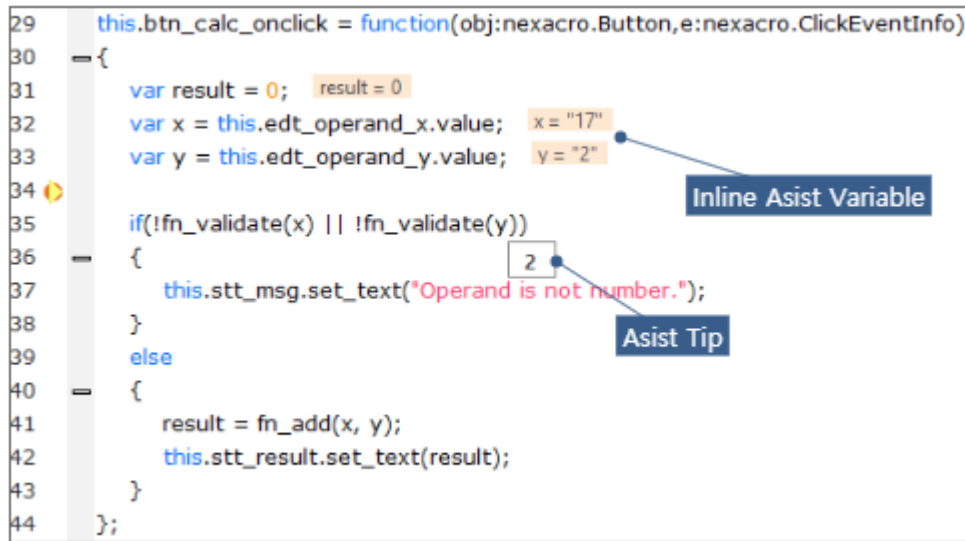
```

29  this.btn_calc_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
30  -{
31      var result = 0;
32      var x = this.edt_operand_x.value;
33      var y = this.edt_operand_y.value;
34  ●
35      if(!fn_validate(x) || !fn_validate(y))
36      - {
37          this.stt_msg.set_text("Operand is not number.");
38      }
39      else
40      - {
41          result = fn_add(x, y);
42          this.stt_result.set_text(result);
43      }
44  };

```

- 2 디버깅 모드로 애플리케이션을 실행합니다. 입력과 버튼 클릭이 이뤄지면 34번 줄에서 실행이 멈춥니다. 이때, 스크립트 편집기를 보면 적중된 중단점 위쪽 코드 줄 옆에 '변수 = 값' 형태로 Inline Assist Variable이 표시됩

니다. Assist Tip으로 특정 변수값만을 확인하고 싶으면 마우스 커서를 객체 혹은 변수명 위로 오버시키면 사각형 상자안에 값이 표시됩니다.



Assist Tip과 Inline Assist Variable 기능의 사용 여부는 [Options > Debug > Assistance]에서 설정할 수 있습니다.

9.3.10 변수 값의 변화 관찰하기(Watch)

특정 객체 또는 변수를 Watch에 등록해놓고 디버깅이 진행되는 동안 해당 값의 변화를 살펴볼 수 있습니다. 값이 변경된 항목은 적색 볼드체로 표시됩니다.

일단 객체/변수를 Watch에 등록하면 넥사크로 스튜디오를 종료해도 사용자가 삭제하기 전까진 목록이 계속 유지됩니다.

Watch			
Expression	Value	+	-
x	"17"		
y	"2"		
result	19		
☒ this.stt_result	[Static]		
name	stt_result		
id	stt_result		
☒ parent	[Form]		
left	236		
Debug		Watch	
		Variables	

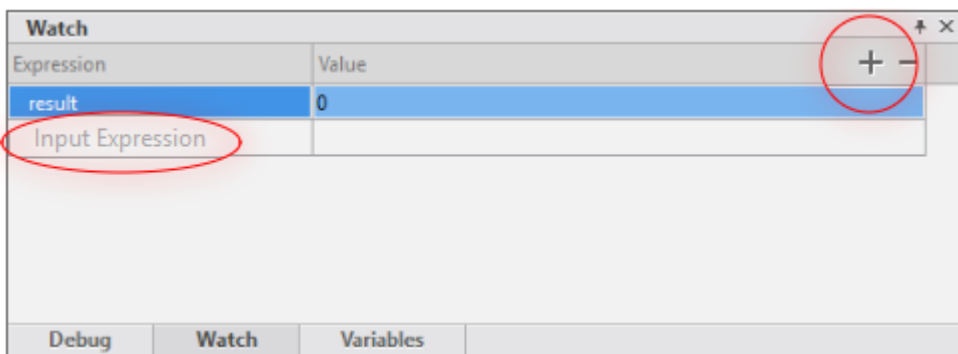
Watch로 변수값을 관찰하는 방법을 예를 들어 설명합니다.

- 1 스크립트 편집 창에서 result 변수값의 변화를 관찰하기 위해 34번 줄과 44번 줄에 중단점을 지정합니다.

```

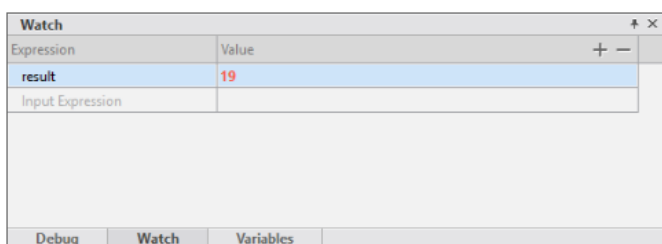
29  this.btn_calc_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
30  -{
31      var result = 0;
32      var x = this.edt_operand_x.value;
33      var y = this.edt_operand_y.value;
34  ●
35      if(!fn_validate(x) || !fn_validate(y))
36      - {
37          this.stt_msg.set_text("Operand is not number.");
38      }
39      else
40      - {
41          result = fn_add(x, y);
42          this.stt_result.set_text(result);
43      }
44  ● };
  
```

- 2 디버깅 모드로 애플리케이션을 실행합니다. 입력과 버튼 클릭이 이뤄지면 34번 줄에서 실행이 멈춥니다. 그럼 Watch 창을 선택한 후 + 버튼을 클릭하거나 마지막 행(Input Expression)을 더블 클릭하여 다음과 같이 result 변수를 등록합니다.



스크립트 편집기에서도 추가할 수 있는데 우선 객체 명이나 변수 명을 드래그하여 선택한 후 Watch 창에 드래그 앤 드롭하거나 마우스 오른쪽 버튼을 클릭하여 컨텍스트 메뉴에서 [Add Text to Watch]를 선택합니다.

- 3 Continue 명령을 사용하여 코드를 실행합니다. 두 번째 중단점인 44번 줄에서 실행이 멈추면 다시 Watch 창을 확인하여 result 값이 어떻게 변화됐는지 확인합니다.





객체나 변수를 Watch에 추가하면 사용자가 삭제하기 전까지 계속 유지됩니다. Watch에 등록된 변수를 삭제하려면 Watch 창에서 삭제하려는 항목을 선택한 후 - 버튼을 클릭하거나 키보드의 Delete 키를 입력합니다.

9.3.11 함수 호출 정보 확인하기(Call Stack)

각종 객체와 함수가 섞여 있는 복잡한 코드를 디버깅하려면 함수 간의 관계를 파악하는 것이 중요합니다. 현재 보고 있는 코드가 어떤 함수에 존재하는 것인지, 어떤 함수의 호출로부터 왔는지 등을 한눈에 파악할 수 있다면 흐름을 파악하는 데 큰 도움이 됩니다.

Call Stack은 함수 호출 히스토리를 제공하여 함수 간의 호출 관계를 확인할 수 있게 해줍니다. 현재 위치까지 호출된 함수를 역순으로 표시하는데 항목의 함수명을 더블 클릭하면 함수를 호출한 위치의 코드로 이동할 수 있습니다.

Call Stack은 디버깅 모드를 실행하면 자동으로 동작하며 코드 실행 흐름에 따라 함수 호출 정보가 갱신됩니다.

함수 호출 정보를 확인하는 방법을 예를 들어 설명합니다.

- 1 스크립트 편집 창에서 함수 호출 정보를 확인하기 위해 fn_add 함수 내에 있는 26번 줄에 중단점을 지정합니다.

```

22  fn_add = function(x, y)
23  -{
24      var retAdd = nexacro.toNumber(x) + nexacro.toNumber(y);
25
26  ●  return retAdd;
27  }
28
29  this.btn_calc_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
30  -{
31      var result = 0;
32      var x = this.edt_operand_x.value;
33      var y = this.edt_operand_y.value;
34
35      if(!fn_validate(x) || !fn_validate(y))
36  -  {
37          this.stt_msg.set_text("Operand is not number.");
38      }
39      else
40  -  {
41          result = fn_add(x, y);
42          this.stt_result.set_text(result);
43      }
44  };
```

- 2 디버깅 모드로 애플리케이션을 실행합니다. 입력과 버튼 클릭이 이뤄지면 26번 줄에서 실행이 멈춥니다. 이때, Call Stack 창에서 함수 호출 정보를 확인하면 다음과 같습니다.

Call Stack			
Function	File	Line	
fn_add(x, y)	calc.xfdl	26	
btn_calc_onclick(obj, e)	calc.xfdl	41	
_pEventListener_fireEvent(obj, evt)	SystemBase.js	2507	
_pComponent.on_fire_onclick(button, alt_key, ctrl_key, shift_key, ...)	CompEventBase.js	3786	
_pComponent_on_click(elem, button, alt_key, ctrl_key, shift_key, ...)	CompEventBase.js	441	
__pWindow_on_default_sys_lbuttonup(elem, strButton, altKey, ct...	Platform.js	3217	
nexacro_syshandler_onlbuttonup_forward(_window, elem, strBut...	Platform_Runtime.js	82	
Error List Call Stack Breakpoints			

현재 위치인 fn_add 함수가 최상단에 위치하고 그다음으로 fn_add 함수를 호출한 btn_calc_onclick 함수가 위치합니다. 그리고 그 아래에도 여러 함수가 보이는데 이는 넥사크로플랫폼 내부에서 사용되는 함수들이므로 특별한 경우가 아니면 신경을 쓰지 않아도 됩니다.



스크립트 함수 정의 형태에 따라 Call Stack 창에 함수명이 표시되지 않을 수도 있습니다. 예를 들면, 다음과 같이 선언된 함수는 함수명이 표시되지 않고 인수만 (a, b) 형태로 표시됩니다.

```
function myFunc(a, b) {}
```

따라서 Call Stack에 함수명이 정상적으로 표시되지 않을 경우에는 다음과 같은 형태를 사용하기 바랍니다.

```
var myFunction = function(a, b) {}
```


파트 II.

Application, Form, Frames

10.

Application

10.1 Application 소개

Application은 실행되는 애플리케이션과 관련된 기본 환경 설정 정보를 담고 있습니다.

10.1.1 사용 예

프로젝트 생성 시 "Application" 프로젝트를 선택했다면 프로젝트를 생성하면서 기본 Application을 생성합니다. 생성된 Application은 [Project Explorer > App Information] 항목에서 확인할 수 있습니다. 해당 항목을 선택하고 관련 속성을 수정할 수 있습니다.



현재 버전은 "Application" 프로젝트만 있습니다.

The screenshot displays the IDE's Project Explorer and Properties panels. In the Project Explorer, the project 'sample_nexacroplatform_16' is expanded, showing its internal structure including Environment, TypeDefinition, App Information, AppVariables, App 'sample_nexacroplatform_16' (selected), mainframe, Trays, Styles, Script, Base, and Sample. The Properties panel on the right shows the configuration for the selected application. It includes sections for Information, Misc., and File Information.

App 'sample_nexacroplatform_16' (Application)	
Input filter text	
Information	
screenid	screen_desktop_D
Misc.	
enginesetupkey	
engineurl	
engineversion	2.0
id	sample_nexacroplatform_16
licenseurl	
version	
File Information	
filename	sample_nexacroplatform_16

10.1.2 Application 접근하기

앞에서 언급한 것처럼 [Project Explorer]에서 접근하는 방식은 속성을 수정하거나 이벤트 함수를 지정하는 경우에 사용합니다. 하지만 Form에서 스크립트로 접근해야 할 때에는 아래와 같은 코드를 사용합니다.

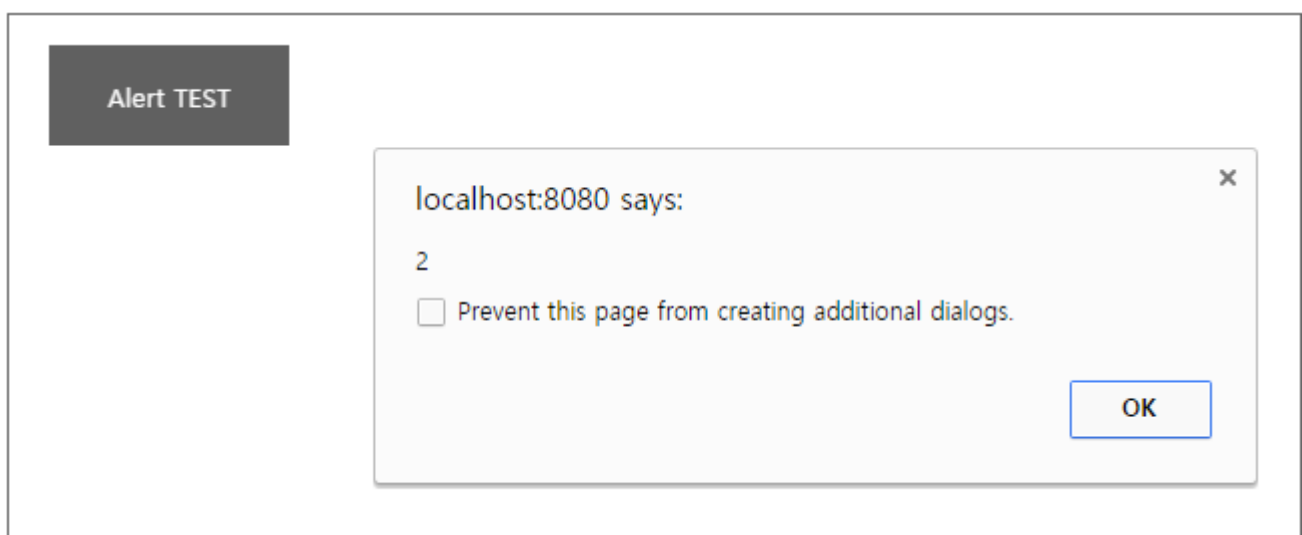
```
nexacro.getApplication();
nexacro.getApplication().alert("warning");
```

10.2 경고, 확인 대화상자 띄우기

일반적으로 애플리케이션은 사용자가 정보를 입력하고 이를 처리한 후 결과를 보여주는 것이 주된 역할입니다. 하지만 사용자가 입력된 값이 잘못되었거나 뭔가 메시지를 직관적으로 주기 위해서 경고 또는 확인 대화상자를 사용합니다.

10.2.1 예제

버튼을 클릭하면 경고 대화상자가 나타납니다. 반복구문을 사용해 10번의 대화상자가 표시됩니다. 10개의 대화상자가 동시에 나타나지 않고 사용자가 화면에 나타난 대화상자를 닫거나 [확인] 버튼을 클릭했을 때만 다음 대화상자가 나타납니다.



10.2.2 예제에서 사용한 핵심 기능

alert

alert 메소드는 경고 메시지를 보여주기 위한 용도로 사용됩니다. 예제에서 반복구문을 사용했는데 10개의 대화상자가 동시에 뜨지 않는 이유는 alert 메소드는 사용자가 해당 메시지를 확인하고 버튼을 누르기 전에는 다른 프로세스를 진행하지 않고 멈춘 상태로 대기합니다.



alert 메소드 실행 시 표시되는 대화상자는 브라우저에 따라 다르게 보일 수 있습니다. 자신이 원하는 형태로 경고 메시지를 보여주기 원한다면 alert 메소드 대신 popupDiv 같은 컴포넌트를 사용해야 합니다.



구글 크롬 브라우저를 사용하는 경우에는 예제 실행 시 경고 대화상자에서 "Prevent this page from creating additional dialogs"라는 메시지를 볼 수 있습니다. 해당 항목을 체크한 경우에는 더는 경고 대화상자가 나타나지 않습니다. 다시 대화상자가 나오게 하려면 브라우저를 재시작해야 합니다.

10.2.3 예제 구현 방법

① 메인 화면 구성하기

Button 컴포넌트를 예제 화면과 같이 배치합니다.

② Button 컴포넌트 이벤트 함수 작성하기

버튼 클릭 시 처리할 이벤트 함수를 작성합니다. 반복 구문을 사용해 경고 대화상자가 10번 나타날 수 있도록 합니다.

```
this.btnAlert_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    for(var i=0 ; i<10 ; i++)
    {
        nexacro.getApplication().alert(i);
    }
};
```

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭하고 경고 대화상자가 나타나는 것을 확인합니다.

10.3 사용자 정보 저장하기

각 화면 간 데이터를 공유할 필요가 있으면 `addVariable`에 변수를 저장할 수도 있지만 `addVariable`은 애플리케이션이 종료된 경우에는 다시 해당 값을 사용할 수 없습니다. 하지만 로컬스토리지에 사용자 정보를 저장하면 애플리케이션이 종료되더라도 해당 PC에 저장된 정보를 다시 사용할 수 있습니다.



로컬스토리지 정보는 런타임과 웹브라우저가 자신만의 저장소에서 정보를 관리합니다. 때문에 런타임에서 저장된 데이터를 크롬 브라우저에서 가져올 수는 없습니다.



로컬스토리지 정보는 웹브라우저에서는 도메인 단위로 관리하고 런타임에서는 프로젝트 단위로 관리합니다. 같은 키값을 사용하더라도 도메인이나 프로젝트가 다르다면 처리되지 않습니다.

10.3.1 예제

[SET DATA] 버튼을 클릭하면 Edit 컴포넌트에 입력된 값을 로컬스토리지에 저장합니다. 애플리케이션을 닫고 다시 실행해서 [GET DATA] 버튼을 클릭하면 이전에 입력했던 값이 Edit 컴포넌트에 출력됩니다.

 [sample_application_04.xfdl](#)

10.3.2 예제에서 사용한 핵심 기능

`setPrivateProfile`

로컬스토리지에 데이터를 저장합니다. 저장 공간은 웹브라우저에 따라 다릅니다.

getPrivateProfile

로컬스토리지에 저장된 값을 가져옵니다. 지정된 키 값에 해당하는 값이 없더라도 에러로 처리하지는 않습니다.

10.3.3 예제 구현 방법

① 메인 화면 구성하기

Button 컴포넌트와 Edit 컴포넌트를 예제 화면과 같이 배치합니다.

② Button 컴포넌트 이벤트 함수 작성하기

버튼 클릭 시 처리할 이벤트 함수를 작성합니다. 키 값은 미리 정해진 값으로 설정합니다. 키 값만 다르다면 여러 개의 값을 관리할 수 있습니다.

```
this.btnSet_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Edit00.set_value(nexacro.setPrivateProfile('TEST', this.editSet.value));
};

this.btnGet_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Edit00.set_value(nexacro.getPrivateProfile('TEST'));
};
```

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 Edit 컴포넌트에 임의의 값을 입력하고 [SET DATA] 버튼을 클릭하면 값이 저장됩니다. 정상적으로 값이 저장되면 true 값을 Edit 컴포넌트에 표시합니다. 그리고 [GET DATA] 버튼을 클릭하면 저장된 값을 Edit 컴포넌트에 표시합니다.



같은 키 값을 사용해 setPrivateProfile 메소드를 호출하면 이전에 저장된 값을 덮어씁니다.

10.4 쿠키 정보 확인하기

웹브라우저가 가지고 있는 쿠키 정보를 확인할 수 있습니다.



window.document 오브젝트는 넥사크로 브라우저에서는 접근할 수 없습니다.
넥사크로 브라우저로 실행 시에는 해당 예제가 동작하지 않습니다.

10.4.1 예제

[Cookie check] 버튼 클릭 시 TextArea 영역에 쿠키 정보를 표시합니다.

Cookie check

Cookie0 : TEST
 Cookie1 : nexacro platform 17



sample_application_05.xfdl

10.4.2 예제에서 사용한 핵심 기능

`window.document.cookie`

현재 Document안의 cookies 정보를 확인합니다. name/value 값을 구할 수 있습니다.

10.4.3 예제 구현 방법

① 메인 화면 구성하기

Button 컴포넌트와 TextArea 컴포넌트를 예제 화면과 같이 배치합니다.

② Button 컴포넌트 이벤트 함수 작성하기

버튼 클릭 시 실행할 이벤트 함수를 작성합니다. window.document 오브젝트에서 cookie라는 값을 받아옵니다. 각 쿠키값은 ";" 문자로 구분되어 있기 때문에 split 메소드를 사용해 배열로 변환한 뒤 TextArea 컴포넌트에 표시해 줍니다.

```
this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var arrCookie = window.document.cookie;
    arrCookie = arrCookie.split(";");
    var arrTempCookie;
    this.TextArea00.set_value("");
    for(var i=0; i<arrCookie.length; i++)
    {
        arrTempCookie = arrCookie[i].split("=");
        this.TextArea00.set_value(this.TextArea00.value
            + ( arrTempCookie[0].trim() + " : " + arrTempCookie[1] ) + "\n");
    }
};
```

③ QuickView로 확인하기

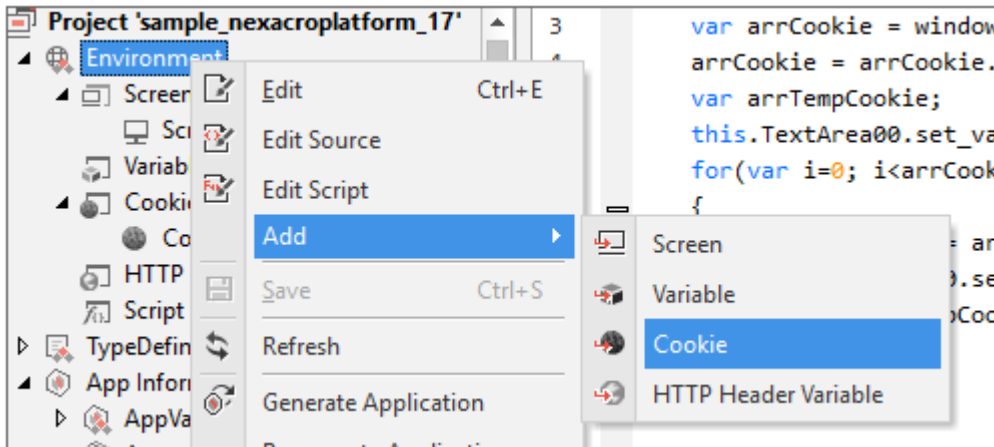
QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭하면 TextArea에 쿠키값을 표시합니다. 하지만 상황에 따라 아무값도 표시되지 않는 경우도 있습니다.



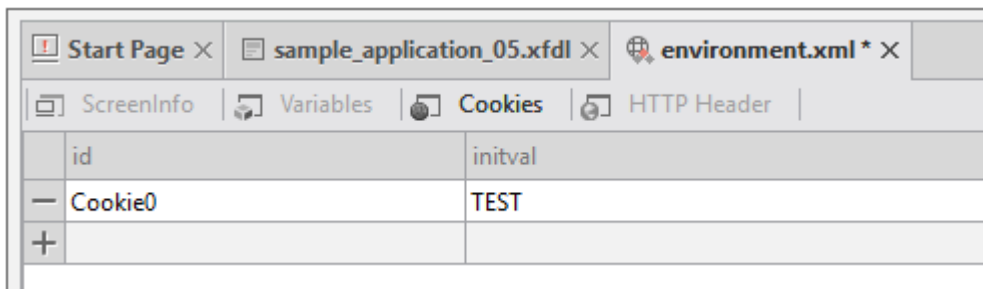
넥사크로 스튜디오에서 로컬 웹 서버를 사용해 실행하는 경우에는 별도로 쿠키값을 설정하지 않은 경우에 쿠키값이 표시되지 않습니다.

④ 쿠키값 설정하기

쿠키값이 표시되지 않는 경우에는 임시로 쿠키값을 설정해봅니다. Project Explorer에서 Environment 항목을 선택하고 컨텍스트 메뉴에서 [Add > Cookie] 항목을 선택합니다.



쿠키값으로 추가할 값을 입력합니다. 아래 입력 양식에서 id와 initval 항목을 입력합니다.



5 다시 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭하면 TextArea에 앞 단계에서 입력한 Cookie 정보가 표시됩니다.

10.5 활성화된 화면(Form) 이름(ID) 속성 정보 얻기

여러 화면이 열려 있는 경우에는 현재 활성화된 화면이 어떤 것인지 쉽게 알 수 없습니다. 이런 경우 활성화된 Form을 확인하는 기능 예제입니다.

10.5.1 예제

[Check] 버튼 클릭 시 Form의 id, titletext 속성값, Button 컴포넌트의 부모 오브젝트의 id 속성값을 Edit 컴포넌트에 표시합니다. [Div Check] 버튼 클릭 시에는 활성화된 화면(Form)은 Div 내 innerForm 이지만 getActiveForm 메소드에서는 Div 컴포넌트를 포함하고 있는 Form 오브젝트를 반환합니다.

sample_nexacroplatform_17

nexacroplatform developer guide

sample_application_06

Check

Div Check



sample_application_06.xfdl

10.5.2 예제에서 사용한 핵심 기능

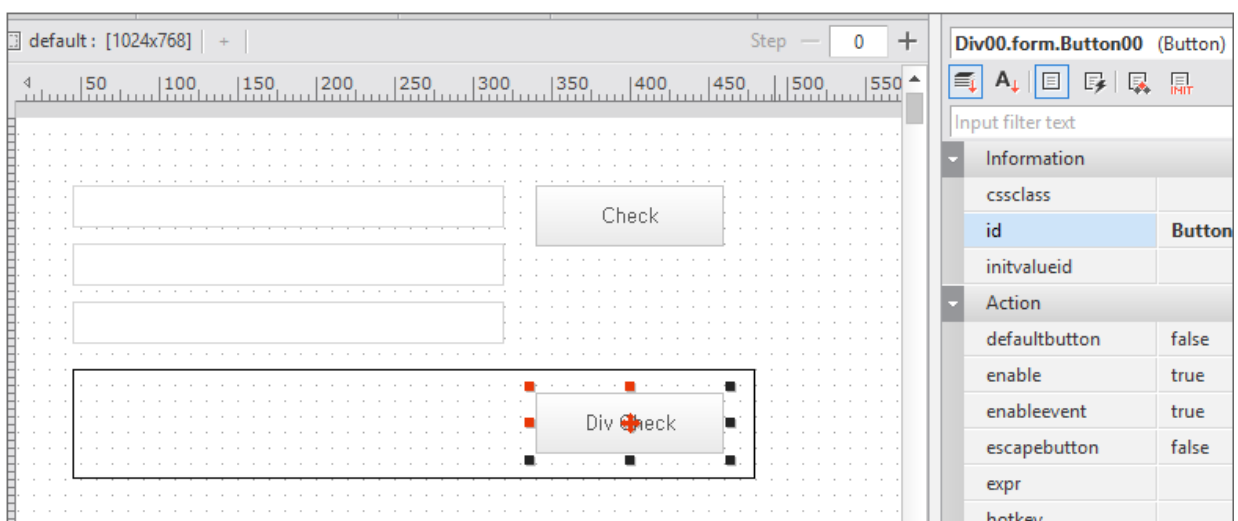
getActiveForm

활성화(현재 포커스가 있는)된 Form 오브젝트를 반환합니다.

10.5.3 예제 구현 방법

① 메인 화면 구성하기

Edit 컴포넌트와 Button 컴포넌트를 배치합니다. Div 컴포넌트를 배치하고 그 안에 Button 컴포넌트를 배치합니다.



2 Button 컴포넌트의 onclick 이벤트 함수 생성하기

2개의 Button 컴포넌트 클릭 시 호출되는 이벤트 함수 내에서 check_active_form 라는 함수를 다시 호출합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.check_active_form();
};

this.Div00_Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.check_active_form();
};
```

check_active_form 라는 함수 내에서는 getActiveForm 메소드를 실행해 활성화된 Form 정보를 Edit 컴포넌트에 표시합니다. 마지막 Edit02 컴포넌트에는 현재 포커스를 가지고 있는 컴포넌트(Button)의 부모 오브젝트의 id 값을 표시하도록 했습니다.

```
this.check_active_form = function()
{
    var objApp = nexacro.getApplication();
    var objForm = objApp.getActiveForm();

    this.Edit00.set_value(objForm.name);
    this.Edit01.set_value(objForm.titletext);
    this.Edit02.set_value(this.getFocus().parent.name);
}
```



예제의 특성 상 하나의 Form 내에서 Div 컴포넌트를 배치했기 때문에 실제 활성화된 Form 자체가 바뀌지는 않았습니다. MDI 형식으로 여러 개의 Form을 배치하는 경우에 유용한 예제입니다.

3 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 [Check] 버튼과 [Div Check] 버튼을 클릭하고 표시되는 정보를 확인합니다.

10.6 구글 폰트 사용하기

사용자 PC에 해당 폰트가 설치되어 있지 않더라도 공개된 폰트 리소스를 웹에서 내려받아 바로 표시할 수 있습니다. 구글에서는 구글 폰트 사이트를 통해 무료로 사용할 수 있는 웹 폰트 정보를 제공하고 있습니다.

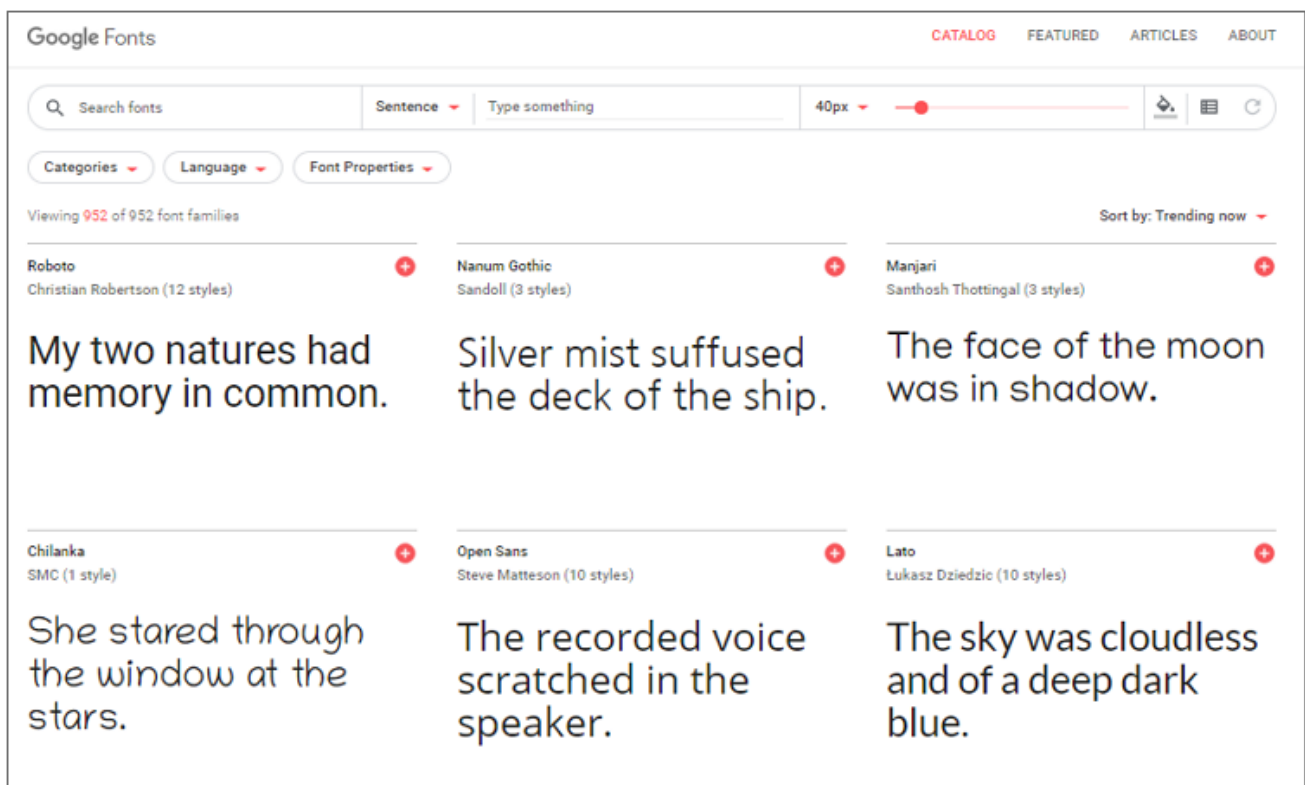


웹에서 사용할 수 있도록 폰트 리소스 용량이 크지 않게 만들어졌습니다. 하지만, 네트워크 환경이 좋지 않은 경우에는 애플리케이션 처리 속도에 영향을 미칠 수도 있습니다.



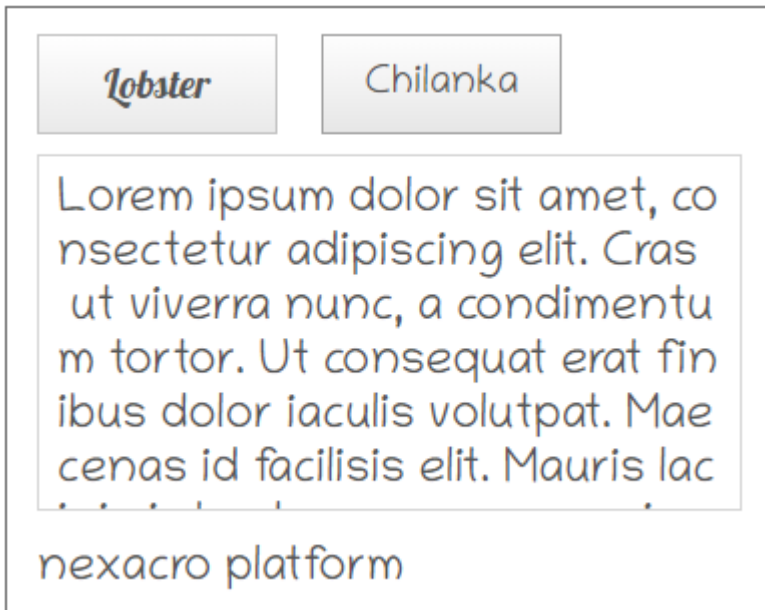
구글 폰트는 WOFF2 파일 형식을 기본값으로 사용하지만, IE 브라우저와 일부 웹브라우저에서는 WOFF2 파일 형식을 지원하지 않아 WOFF 파일 형식으로 내려받습니다. 파일 압축률의 차이가 있어, 파일 크기가 웹브라우저에 따라 다를 수 있습니다.

https://www.w3schools.com/Css/css3_fonts.asp



10.6.1 예제

버튼을 클릭하면 선택한 폰트가 Form에 적용되고 각 컴포넌트에 반영됩니다. 단, Button 컴포넌트는 자체적으로 font 속성을 지정해서 해당 속성값이 유지됩니다.



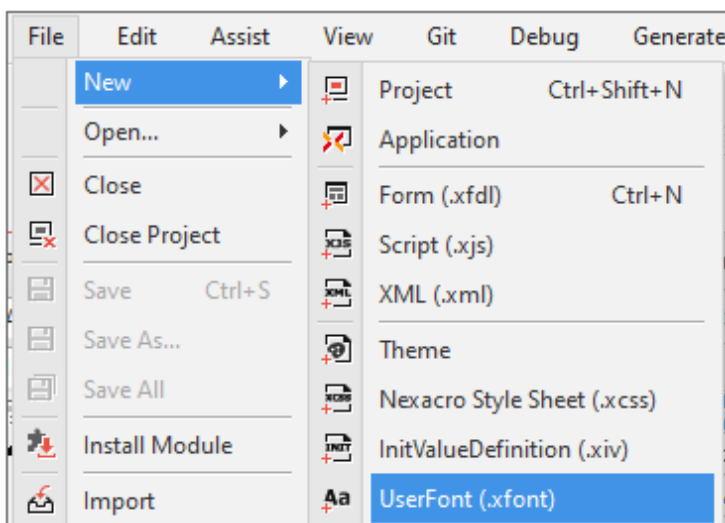
10.6.2 예제에서 사용한 핵심 기능

userfontid

애플리케이션에 적용할 UserFont ID를 설정합니다.

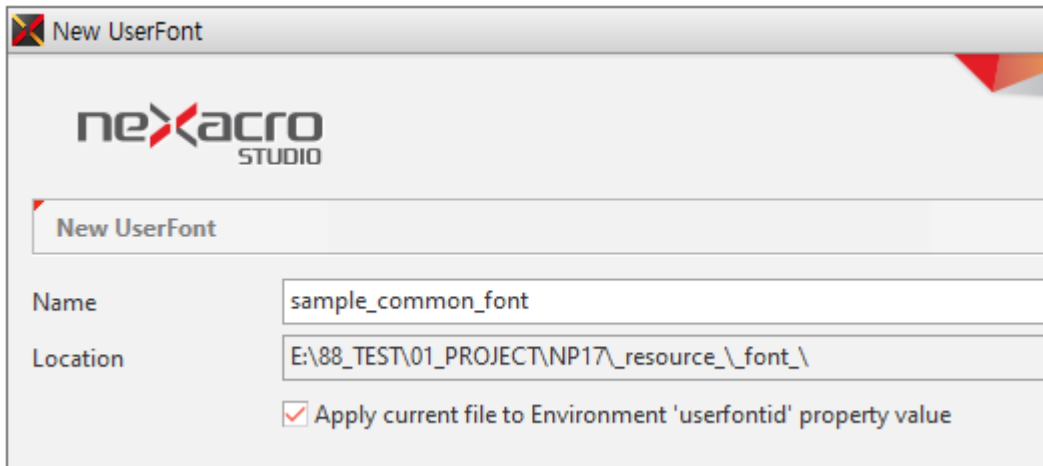
10.6.3 예제 구현 방법

- 1 UserFont 파일을 새로 생성합니다. 메뉴[File > New > UserFont] 항목을 선택합니다.



② 생성할 파일명을 지정합니다.

userfontid 속성에 생성한 UserFont 파일을 바로 적용하려면 아래 체크박스를 체크 상태로 변경합니다.



③ 생성된 파일을 확인합니다. 예제 스크립트가 주석 내에 생성되어 있습니다.

이번 예제에서는 구글 폰트를 사용하기 때문에 주석 내 코드는 건드리지 않습니다.



④ 구글 폰트 웹사이트에 접속합니다.

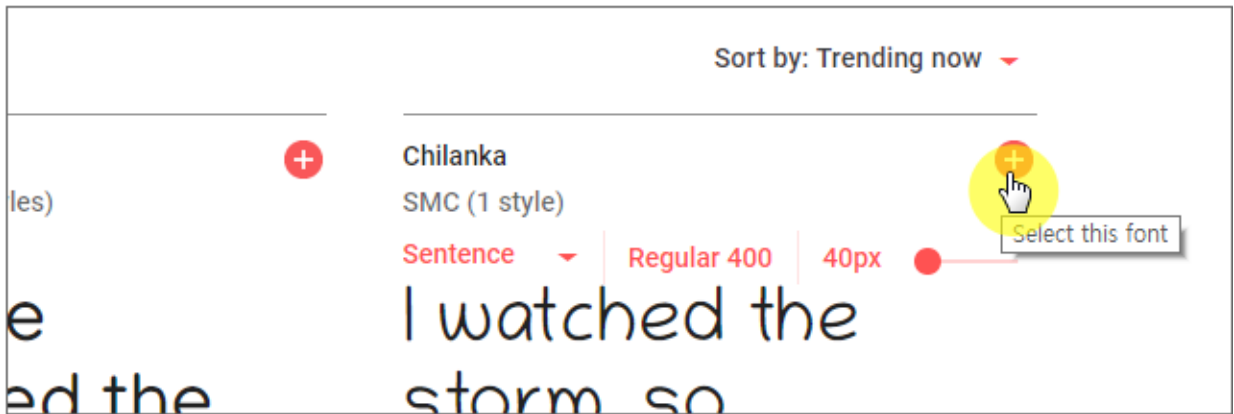
<https://fonts.google.com>



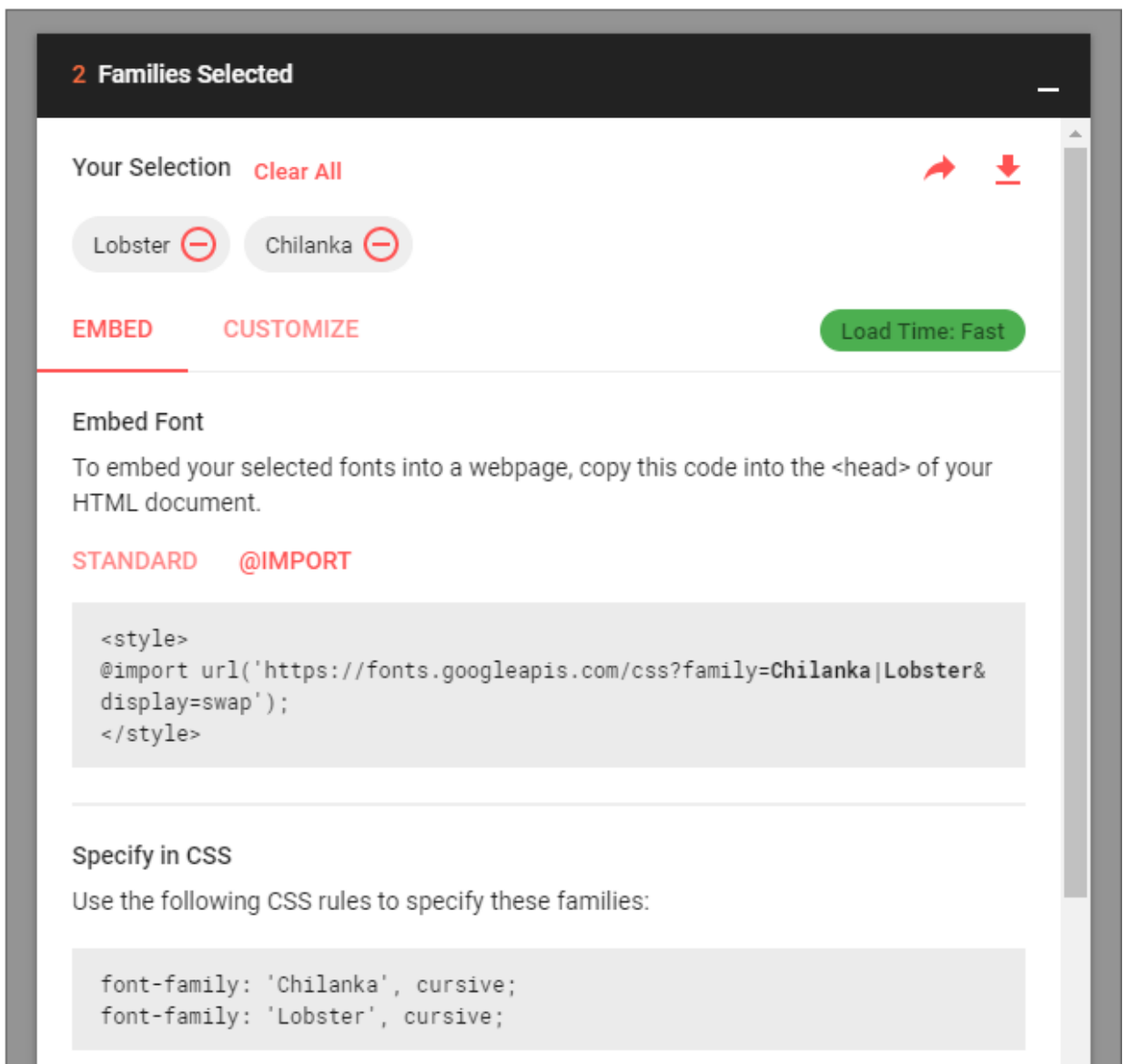
구글 폰트 웹사이트 관련 내용은 2019년 9월 기준입니다. 이후 서비스 정책에 따라 사용법이 변경될 수 있습니다.

<https://fonts.google.com>

⑤ 폰트 목록에서 원하는 항목을 선택하고 [+] 아이콘을 클릭합니다.



- ⑥ 선택한 폰트는 오른쪽 하단에 박스에 담겨져 있습니다. 박스를 클릭하면 아래와 같은 내용을 볼 수 있습니다.



- ⑦ @IMPORT 항목을 복사해서 앞에서 만든 xfont 파일에 붙여넣습니다.

```

/*****
@font-face {
  font-family : 'NanumGothic';
  font-style : normal;
  ...
  url(../_resource/_font_/NanumGothicBold.woff) format('woff'),
  url(../_resource/_font_/NanumGothicBold.ttf) format('truetype');
}
*****/
@import url('https://fonts.googleapis.com/css?family=Chilanka|Lobster&display=swap');

```

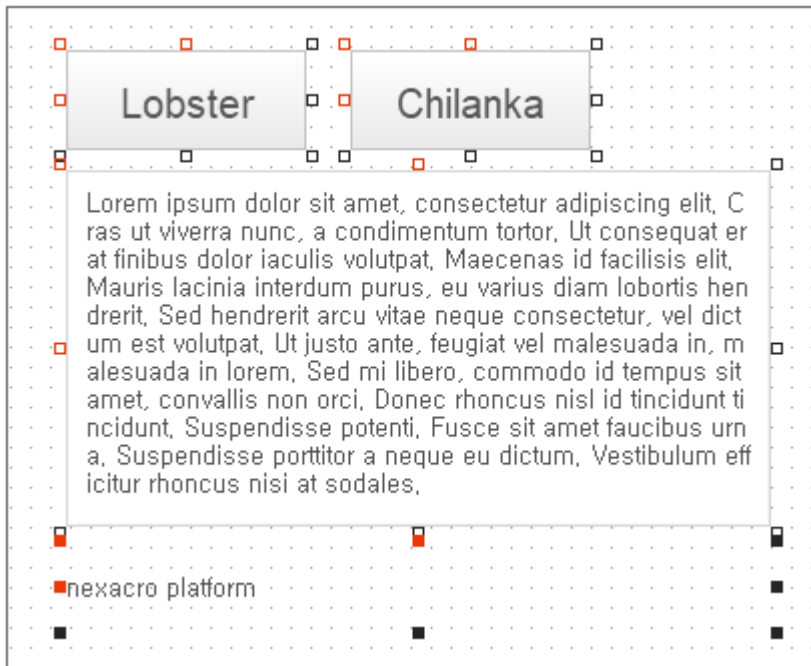


xfont 파일에 작성한 내용은 부트스트랩 파일에 반영됩니다. 서버에 배포 시에는 xfont 파일을 배포하는 것이 아니라 index.html 파일을 포함한 부트스트랩 파일을 업데이트해주어야 합니다.

- ⑧ userfontid 속성값이 지정되었는지 확인합니다. 이전 단계에서 xfont 파일 생성하면서 지정하도록 체크했기 때문에 아래와 같이 속성값이 설정되어 있어야 합니다.

Properties	
Environment (Environment)	
Input filter text	
▼ Information	
highcontrastthemeid	
initvaluefileid	
key	
themeid	theme::default
userfontid	font::sample_common_font.xfont

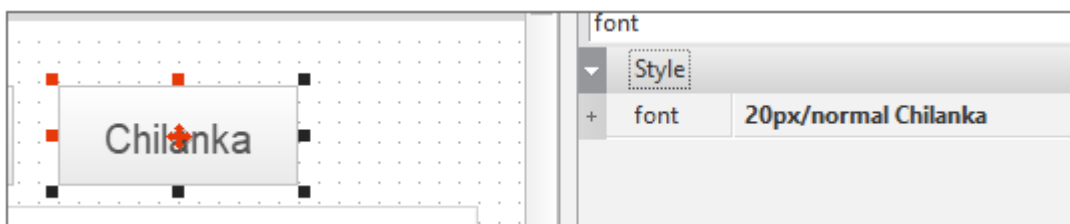
- ⑨ 이제 폰트를 적용할 화면을 만듭니다. Button 컴포넌트 2개를 배치하고 TextArea, Static 컴포넌트를 배치합니다.



- ⑩ Button 컴포넌트의 font 속성을 아래와 같이 설정합니다.

Button00: 20px/normal Lobster

Button01: 20px/normal Chilanka



font 속성을 설정해도 넥사크로 스튜디오에서는 폰트가 적용되지 않습니다. 넥사크로 스튜디오에서는 사용자 PC에 설치된 폰트만 적용할 수 있습니다.

- ⑪ Button 컴포넌트의 onclick 이벤트 함수를 아래와 같이 작성합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.set_font("24px/normal 'Lobster'");
};

this.Button01 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.set_font("24px/normal 'Chilanka'");
};
```

```
};
```

- 12 QuickView(Ctrl + F6)로 실행한 후 버튼을 눌렀을 때 컴포넌트에 폰트 설정이 적용되는지 확인합니다.

11.

Form

11.1 Form 소개

넥사크로플랫폼에서 Form은 화면을 구성하는 가장 기본적인 단위입니다. 독립적으로 하나의 화면을 구성하기도 하고 Div, PopupDiv, Tab 컴포넌트처럼 다른 컴포넌트의 콘텐츠로 사용되기도 합니다. 넥사크로스튜디오에서 화면을 배치하려면 먼저 Form을 생성하고 그 위에 컴포넌트를 배치해야 합니다.

11.1.1 사용 예

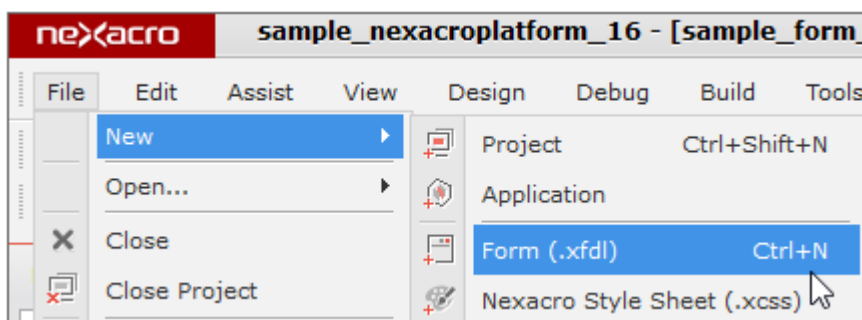
Form은 직접 화면에 보이는 부분이 아니라 컴포넌트를 배치하고 관련된 이벤트 등을 처리하는 용도로 사용하기 때문에 예를 들어 설명하기 어렵습니다. 마치 공기가 눈에 보이지 않는 것처럼 말이죠.

11.1.2 Form 만들기

스크립트에서 동적으로 생성할 수도 있지만 일반적인 경우는 넥사크로 스튜디오 메뉴에서 Form을 생성합니다.

① 메뉴 [File > New > Form]

Form 이름과 서비스를 지정합니다. 프로젝트 규모가 큰 경우에는 업무에 따라 서비스를 구분하기 때문에 적절한 서비스를 선택합니다. 서비스는 Form 생성 후 변경할 수 있습니다.



② Design | Source | Script

Form을 생성하면 넥사크로 스튜디오에서 생성된 Form을 열어줍니다. 열린 Form은 3가지 편집 모드를 제공합니다.

[Design] 모드에서는 컴포넌트를 배치하거나 컴포넌트를 선택하고 속성값을 지정할 수 있습니다. 수정된 속성값은 바로 반영되어 [Design] 모드 화면에서 확인할 수 있습니다.

[Source] 모드에서는 XML 파일 형식으로 생성된 코드값을 보여줍니다. 화면의 배치 순서나 컴포넌트의 속성, 등록된 이벤트 등을 확인할 수 있습니다. 코드를 직접 수정할 수 있지만 [Design] 모드나 속성창에서 수정한 값이 자동으로 반영되기 때문에 따로 수정할 필요는 없습니다.

[Script] 모드에서는 사용자가 함수를 생성하거나 이벤트 함수를 편집합니다.

11.2 화면 크기에 따라 레이아웃 바꾸기

모바일 디바이스가 처음 등장했을 때는 데스크탑에서 보던 화면을 그대로 보여주었습니다. 사용자가 많지 않았고 작은 화면에서도 원하는 부분을 확대해서 보면 사용하는 데 문제가 없다고 생각했습니다. 하지만 모바일 디바이스 사용자가 늘어나면서 사용자는 작은 화면에 최적화된 화면을 원하기 시작했습니다. 그런 요구에 따라 "모바일 퍼스트"라는 말도 등장했습니다.

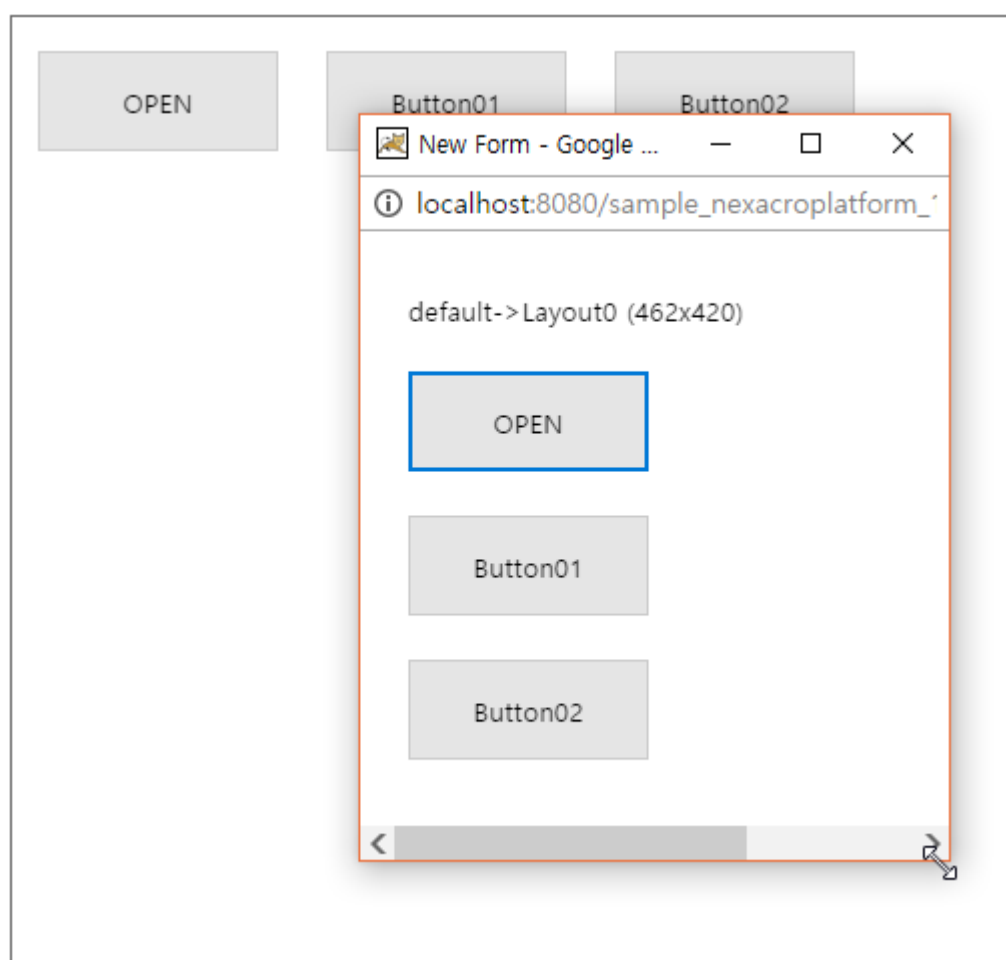
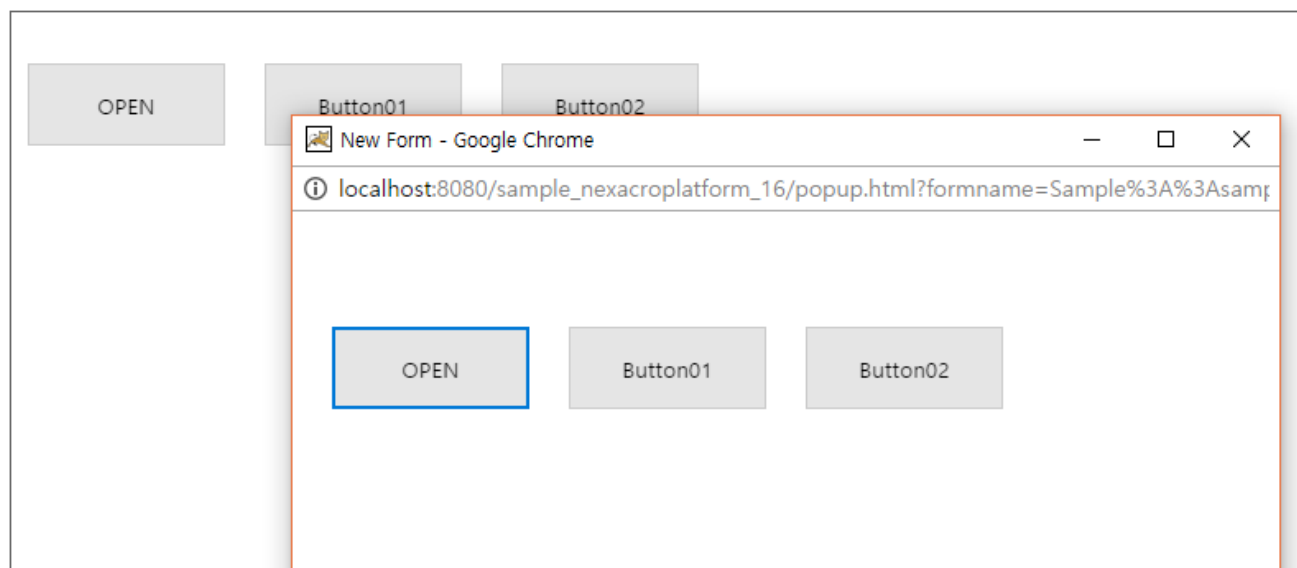
넥사크로플랫폼에서는 디바이스에 따라 스크린 정보를 처리할 수 있고 화면 크기에 따라 레이아웃 정보를 사용할 수 있습니다. 이번 예제에서는 화면 크기에 따라 레이아웃을 어떻게 바꿀 수 있는지 알아보겠습니다.



예제에서는 기능 테스트를 위해 크기에 사용자가 강제로 창 크기를 조절하는 상황을 설정했습니다. 실제로는 모니터 크기나 디바이스 화면 종류, 크기에 따라 적절한 레이아웃을 사용할 수 있도록 설계합니다.

11.2.1 예제

버튼 클릭 시 open 메소드를 사용해 같은 Form을 새 창으로 띄웁니다. 띄운 창의 모서리 부분을 마우스로 잡고 움직여 화면 크기를 줄이면 버튼의 배치가 바뀌는 것을 확인할 수 있습니다. 다시 화면 크기를 크게 조절하면 버튼의 배치가 바뀝니다.



11.2.2 예제에서 사용한 핵심 기능

Layout

Form 오브젝트의 하위에 저장되는 오브젝트입니다. 넥사크로 스튜디오에서 지정할 수 있으며 스크립트로는 추가 생성하거나 변경할 수 없습니다. Form을 생성하면 default 레이아웃이 자동으로 생성됩니다. default 레이아웃의 name 속성은 변경할 수 없습니다.

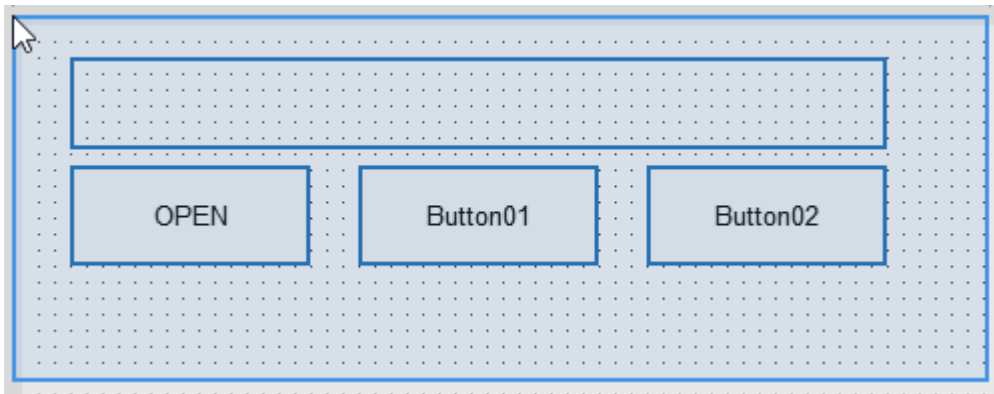
onlayoutchanged

화면 크기 등이 변경되면서 레이아웃이 변경됐을 때 발생하는 이벤트입니다. 이전 레이아웃 정보와 바뀐 레이아웃 정보를 확인할 수 있습니다.

11.2.3 예제 구현 방법

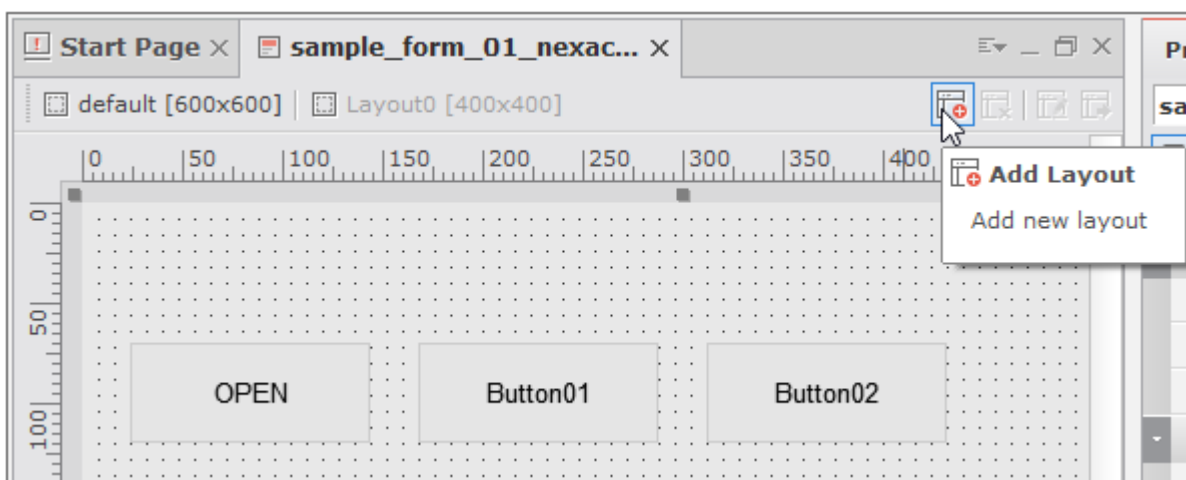
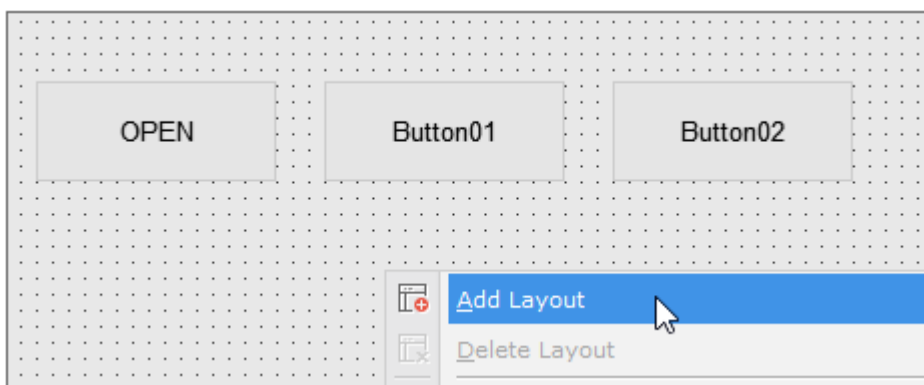
① 화면 구성하기

Button 컴포넌트를 예제 화면과 같이 배치합니다. 화면 크기 조정 시 편의를 위해 Form 생성 시 width, height 속성값은 600, 600으로 지정합니다. Button 컴포넌트 위에 레이아웃 변경 시 관련 정보를 표시하기 위해 Static 컴포넌트를 배치합니다.



② 레이아웃 추가하기

레이아웃은 Form 생성 시 지정할 수 있고 Form 생성 이후에 추가할 수 있습니다. 이번 예제에서는 Form을 생성한 후 추가하는 방식을 사용합니다. [Design] 모드 화면에서 오른쪽 마우스 클릭 후 컨텍스트 메뉴에서 [Add Layout] 항목을 선택하거나 화면 오른쪽 상단에 있는 미니 툴바에서 [Add Layout] 항목을 선택합니다.



3 레이아웃 정보 입력하기

[Add Layout] 항목을 클릭하면 레이아웃 정보를 입력할 수 있는 [New Layout] 창이 나타납니다. 예제에서는 width, height 값만 입력합니다.

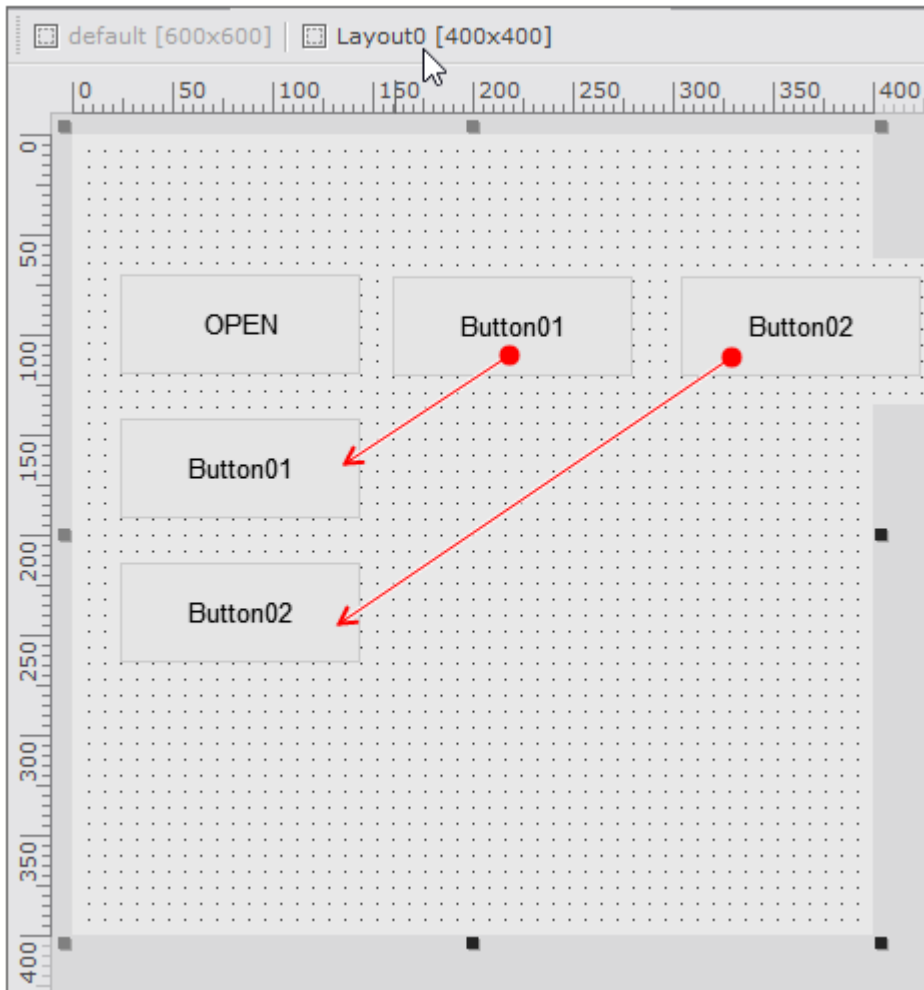
Layout Information	
name	Layout1
screenid	
width	400 px
height	400 px
stepcount	
stepindex	
description	

항목을 입력하면 레이아웃이 추가되고 [Design] 모드 화면 상단에 추가된 레이아웃 탭이 생성됩니다.

4 새로운 레이아웃의 콘텐츠 배치 설정하기

추가된 레이아웃을 선택하고 버튼 컴포넌트의 배치를 아래와 같이 수정합니다. 레이아웃을 선택하면 해당 레이

아웃의 width, height 설정값에 따라 [Design] 모드에 보이는 Form 크기가 변경됩니다.



5 Button 컴포넌트 이벤트 함수 작성하기

실행된 웹브라우저의 크기를 조절해도 되지만 편의를 위해 같은 Form을 가지는 창을 open 메소드를 사용해 띄웁니다. 띄운 창의 크기를 조절해 레이아웃 변경 기능을 확인합니다.

```
this.btnOpen_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    nexacro.open("TEST", "Sample::sample_form_01.xfd1", this.getOwnerFrame());
};
```



예제에서는 Form 이름을 직접 지정해주었지만 ChildFrame에 바로 연결된 Form이라면 this.parent.formurl 속성값으로 지정할 수 있습니다.

6 onlayoutchanged 이벤트 함수 작성하기

레이아웃이 변경됐을 때 Static 컴포넌트에 관련 정보를 표기합니다.

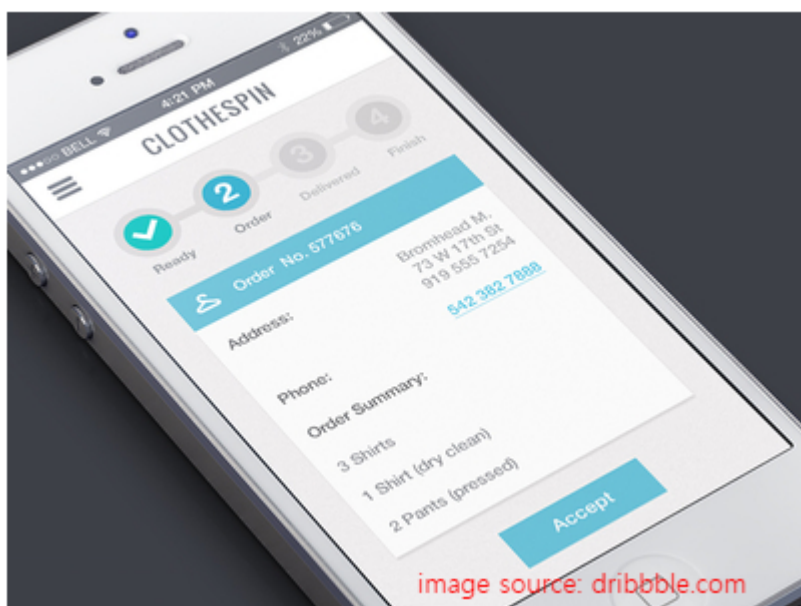
```
this.Form_onlayoutchanged = function(obj:nexacro.Form,e:nexacro.LayoutChangeEventInfo)
{
    this.staticLayoutInfo.set_text(e.oldlayout + ">" + e.newlayout + " (" + this.width +
    "x" + this.height + ")");
};
```

7 QuickView로 확인하기

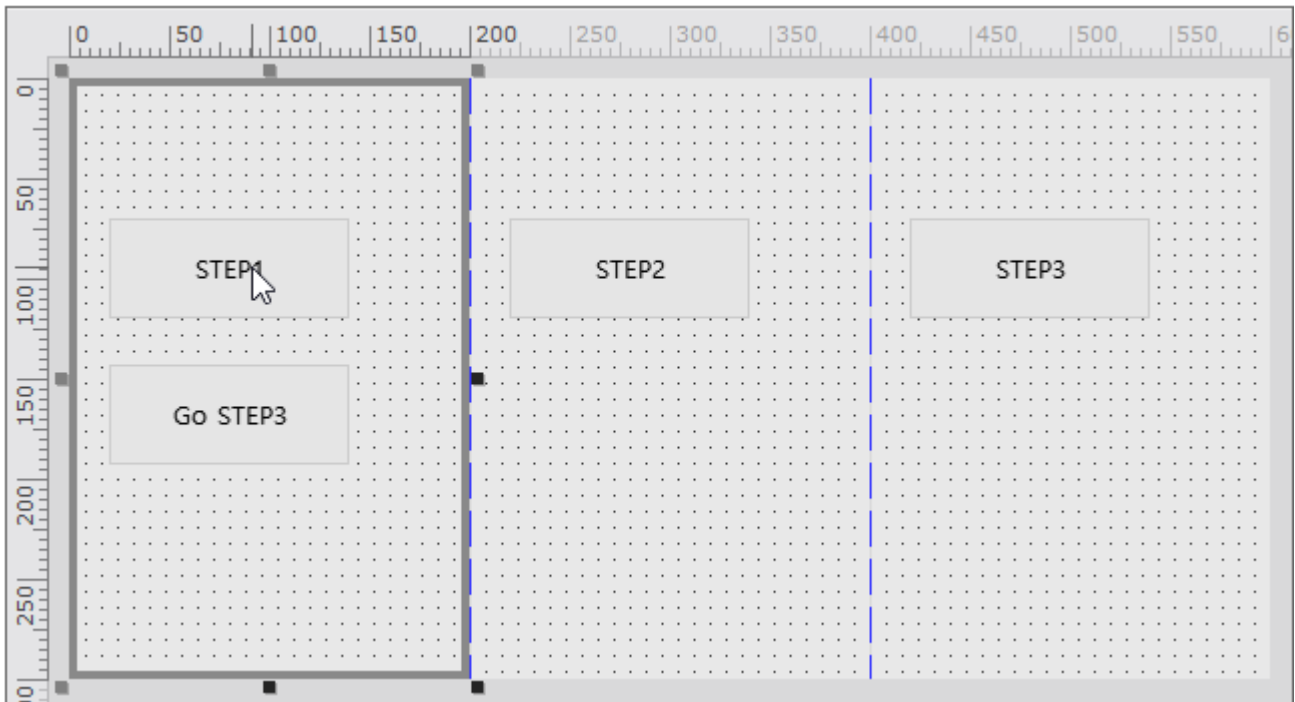
QuickView(Ctrl + F6)로 실행한 후 [OPEN] 버튼을 클릭해 새 창을 띄웁니다. 새 창의 크기를 변경하면서 버튼의 배치가 변경되는지 확인합니다.

11.3 화면을 스텝 단위로 나누기

간단한 레이아웃을 가지는 화면이라면 가로로 배치된 컴포넌트를 세로로 배치하는 것으로 화면 크기에 따른 문제를 해결할 수 있지만 복잡한 입력화면이라면 그것만으로 부족할 수 있습니다. 이런 경우에는 스텝 단위로 화면을 나눌 수 있습니다. 사용자가 입력하는 항목은 똑같지만, 화면을 넘기면서 단계적으로 입력할 수 있게 구성하는 것입니다.



넥사크로 스튜디오에서 Form 오브젝트의 stepcount 속성값을 2 이상으로 지정하면 Form의 너비가 지정된 값만큼 늘어납니다. 예를 들어 Form 오브젝트의 width 속성값을 200으로 지정하고 stepcount 속성값을 3으로 지정하면 600만큼 너비를 표시합니다. 그리고 200픽셀마다 파란 점선으로 구분해 표시합니다. 실제 화면에서 보이는 너비는 200이지만 스텝 단위로 화면을 넘기기 때문에 600만큼의 내용을 보여줄 수 있습니다.

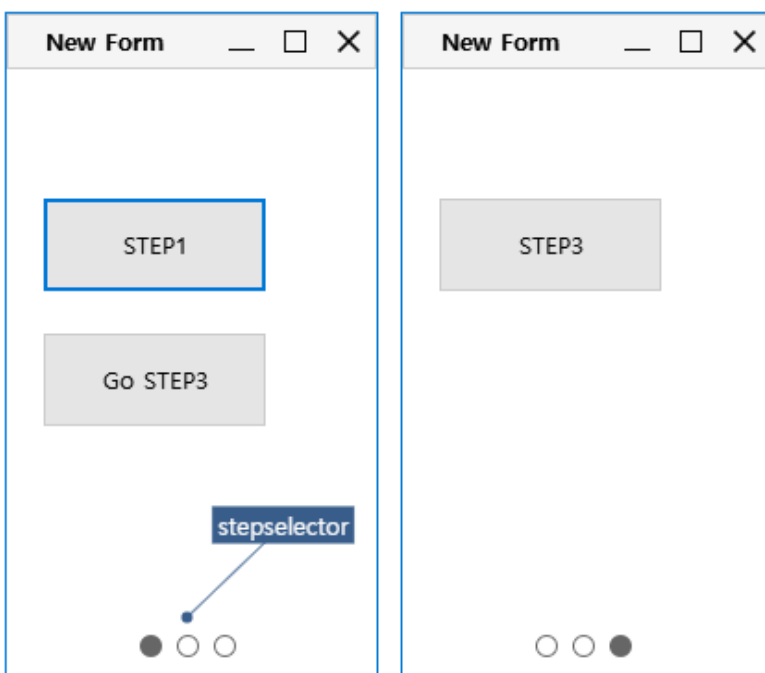


11.3.1 예제

화면 하단에서 stepselector를 선택하면 선택한 화면으로 이동합니다. 첫 번째 화면에서 [Go STEP3] 버튼을 클릭하면 세 번째 화면으로 이동합니다.



모바일 디바이스에서 실행한 경우에는 제스처 동작으로 스텝을 이동할 수 있습니다.





StepControl(stepselector)은 독립적으로 사용할 수 없습니다. Form 오브젝트에서 stepcount 속성값을 지정해 화면을 분할하는 경우에만 적용됩니다.

StepControl의 속성 중 일부는 Form 오브젝트의 속성으로 제공되고 있습니다. stepalign, stepitemgap, stepitemsize 등의 속성값을 지정해 스타일을 조정할 수 있으며 stepshowtype 속성값을 지정해 화면에 보여지는 시점을 결정할 수 있습니다.

11.3.2 예제에서 사용한 핵심 기능

stepcount

속성값에 지정한 값만큼 화면을 분할해 보여줍니다. 넥사크로 스튜디오 속성창에서는 Form 오브젝트를 선택하고 수정하지만 스크립트로 접근하는 경우에는 stepselector 오브젝트를 사용합니다.

positionstep

Form 오브젝트에서 stepcount 속성값을 지정해 화면을 분할한 경우 해당 컴포넌트가 어느 화면에 표시될지를 설정하는 속성입니다. 넥사크로 스튜디오 [Design] 모드에서는 컴포넌트를 배치하는 영역의 인덱스값으로 속성값을 수정합니다. 인덱스값은 0부터 시작합니다.

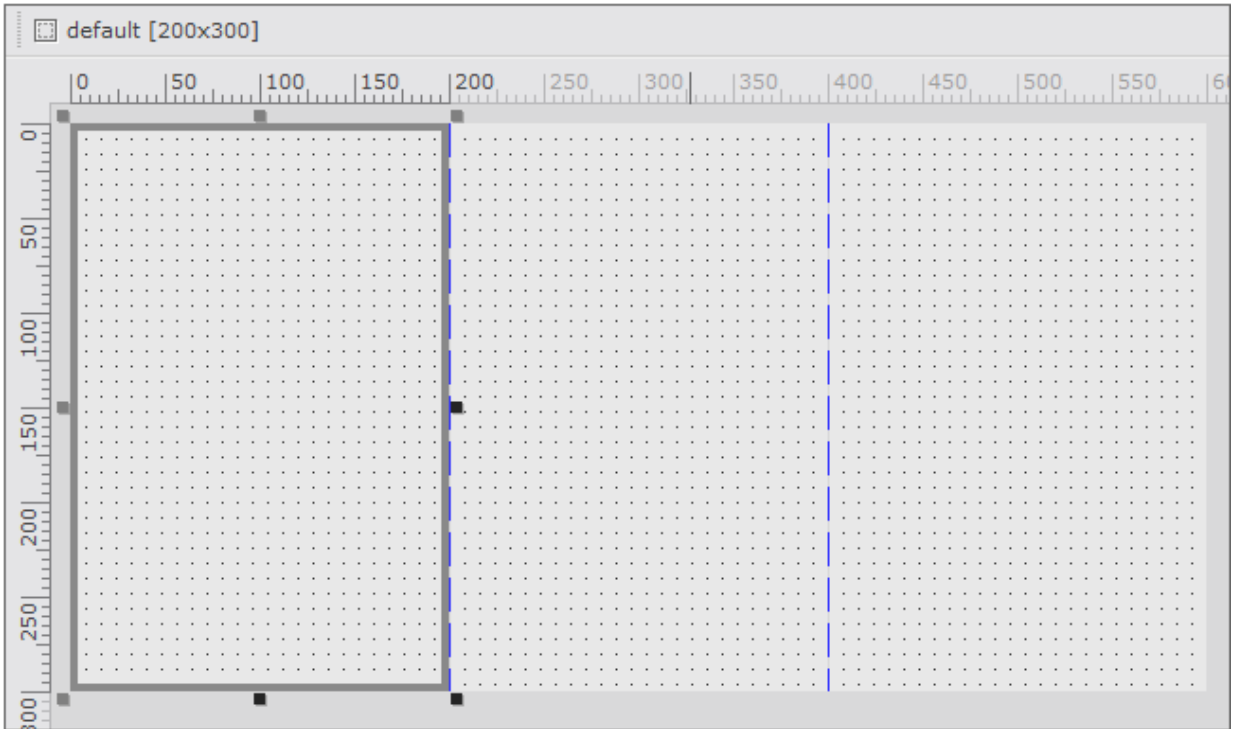
setStepIndex

특정 인덱스의 화면으로 이동하고자 할 때 사용하는 메소드입니다. 인덱스값이 0부터 시작하니 세 번째 화면으로 가고자 한다면 2로 값을 설정합니다.

11.3.3 예제 구현 방법

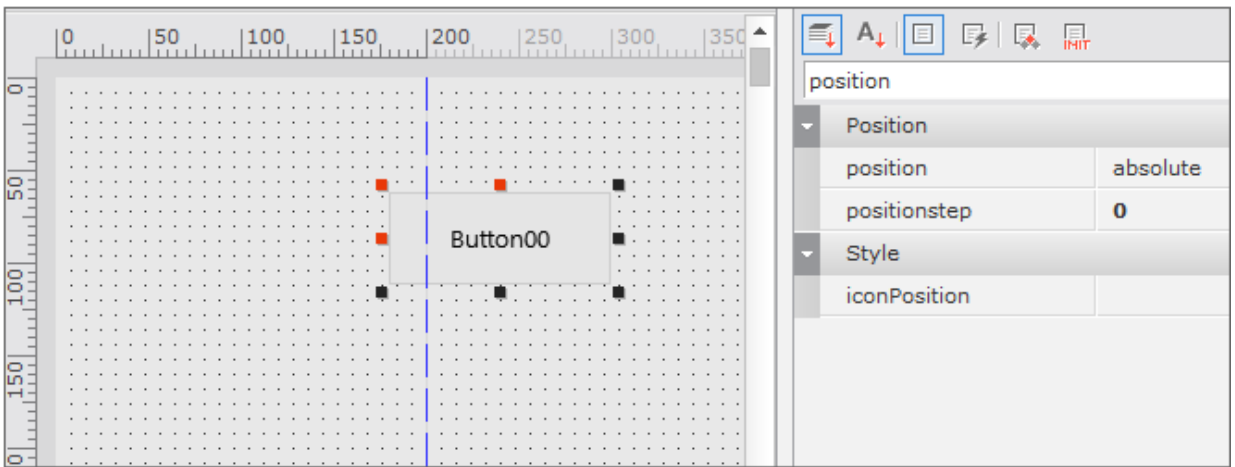
① Form 화면 구성하기

Form을 생성하고 width, height 속성값을 200, 300으로 지정합니다. 그리고 stepcount 속성값을 3으로 지정합니다. 화면이 3개로 분할되면서 처음 생성한 Form의 크기만한 영역이 stepcount 속성값에 지정한 숫자만큼 생성됩니다.



② 컴포넌트 배치하기

Button 컴포넌트를 배치합니다. 컴포넌트가 배치된 영역에 따라 positionstep 속성값이 달라집니다. 컴포넌트가 명확하게 특정 영역에 위치하지 못하고 걸쳐 있는 경우에는 앞부분이 걸친 영역이 positionstep 속성값으로 지정됩니다.



애플리케이션 실행 시 컴포넌트가 걸쳐있는 경우에는 positionstep 속성값으로 지정된 영역에만 표시되고 나머지 영역에서는 보이지 않습니다. 넥사크로스튜디오에서 보이는 것은 디자인 작업 편의를 위해 화면을 펼쳐 놓은 것이고 각 영역은 별개의 화면입니다.

③ Button 컴포넌트 이벤트 함수 작성하기

실행 후 stepselector를 사용해 화면을 이동할 수 있지만 메소드를 [Go STEP3] 버튼 클릭 시 지정된 인덱스 화면으로 이동하는 이벤트 함수를 작성합니다. 예를 들어 회원 가입 양식을 작성하는 경우 추가 입력 항목을 건너 뛰는 경우에는 바로 전송 화면으로 이동할 수 있습니다.

```
this.btnGoStep3_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.setStepIndex(2);
};
```

④ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 stepselector를 사용하거나 [Go STEP3] 버튼을 클릭해봅니다.

11.4 화면 이동하기

화면을 이동(Form 이동)하는 방법은 ChildFrame 오브젝트의 formurl 속성을 변경하거나 컨테이너 컴포넌트의 url 속성값을 변경할 수 있습니다. 하지만 Form의 parent 오브젝트가 어떤 것이 될지 알 수 없다면 Form 오브젝트의 go 메소드를 사용할 수 있습니다. 이번 예제에서는 go 메소드를 사용해 화면을 이동해봅니다.

11.4.1 예제

화면에서 버튼을 클릭하면 다른 Form 화면으로 이동합니다. go 메소드를 사용한 경우에는 보이는 화면을 이동하지만 ChildFrame 오브젝트의 formurl 속성을 변경하지는 않습니다. 이동한 화면에서 [Check formurl] 버튼을 클릭해 ChildFrame 오브젝트의 formurl 속성값(또는 컨테이너 컴포넌트의 url 속성값)을 확인합니다.

Go TEST (SUB)

Check formurl

Sample::sample_form_03.xfdl



11.4.2 예제에서 사용한 핵심 기능

go

지정된 Form을 화면에 표시하는 메소드입니다.



go 메소드는 값을 전달하는 기능을 제공하지 않습니다. 화면 전환 시 값을 같이 넘겨주어야 하는 경우에는 변수나 Dataset을 전달할 수 있는 방법이 없습니다. 전환된 화면에서 같은 값을 공유해야 한다면 AppVariables 또는 Environment의 Variables로 저장한 후 전환된 화면에서 접근해야 합니다.

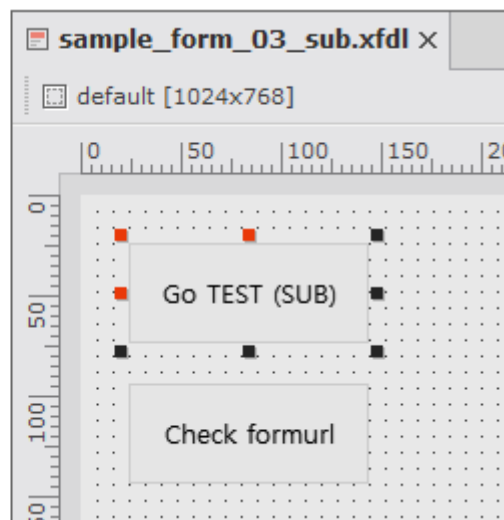
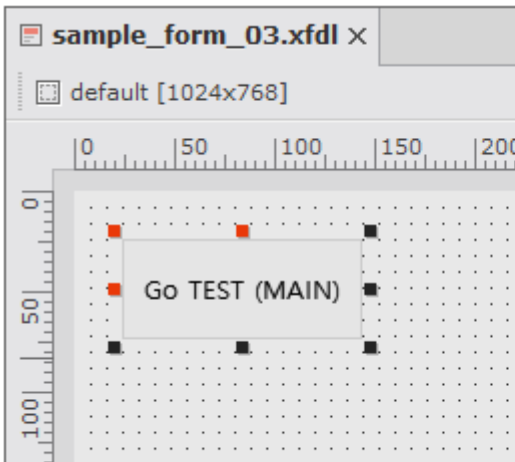
hasOwnProperty

오브젝트가 특정 속성을 가졌는지 확인하는 메소드입니다. 예제에서는 parent 속성값에 해당하는 오브젝트가 ChildFrame인지 Div와 같은 컨테이너 컴포넌트인지 확인하기 위해 formurl 속성값을 가졌는지를 확인합니다. formurl 속성값은 ChildFrame 오브젝트만 가지는 속성입니다.

11.4.3 예제 구현 방법

① Form 화면 구성하기

첫 번째 Form에는 버튼 하나를 배치하고 두 번째 Form에는 버튼 2개와 Edit 컴포넌트를 배치합니다.



② Button 컴포넌트 이벤트 함수 작성하기 (첫 번째 Form)

Button 컴포넌트 클릭 시 지정된 화면으로 이동합니다.

```

this.btnGo_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.go("Sample::sample_form_03_sub.xfdl");
};

```

③ Button 컴포넌트 이벤트 함수 작성하기 (두 번째 Form)

두 번째 Form에는 2개의 Button 컴포넌트가 배치되어 있습니다. [Go TEST (SUB)] Button 컴포넌트 클릭 시에는 지정된 화면으로 이동하며 [Check formurl] Button 컴포넌트 클릭 시에는 ChildFrame 오브젝트의 formurl 속성값(또는 컨테이너 컴포넌트의 url 속성값)을 보여줍니다.

```

this.btnCheck_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    if(this.parent.hasOwnProperty("formurl"))
    {
        this.Edit00.set_value(this.parent.formurl);
    } else {
        this.Edit00.set_value(this.parent.url);
    }
};

```

④ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 화면이 이동하는지 확인합니다.

11.5 오브젝트 접근하기

Form 오브젝트 내 생성된 컴포넌트나 오브젝트는 id를 지정해 접근할 수 있습니다. 하지만 특정 컴포넌트가 아니라 전체 요소에 대해서 뭔가 속성을 지정해야 할 수도 있습니다.

11.5.1 예제

[Count] 버튼은 두 개의 Button 컴포넌트와 두 개의 Dataset 컴포넌트가 있는 화면의 components, objects, all 속성의 length 값을 확인합니다. [Add Button] 버튼 클릭 시에는 Button 오브젝트를 추가로 생성합니다. 화면에는 보이지 않지만 [Count] 버튼을 클릭하면 Button 오브젝트가 추가된 것을 확인할 수 있습니다. 마지막으로 [Hide Butt

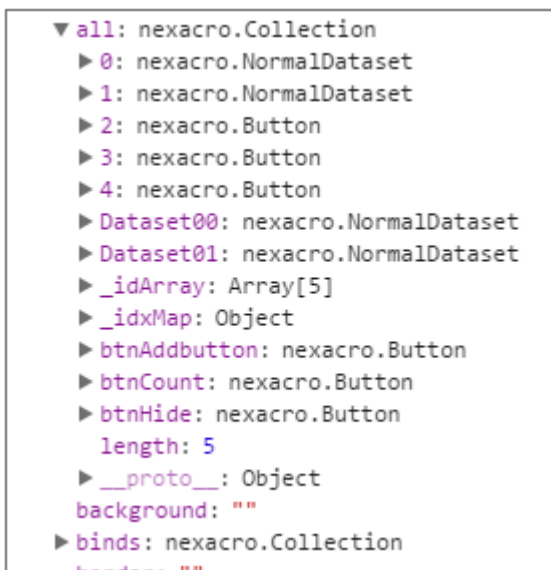
on] 버튼 클릭 시 해당 버튼을 제외한 나머지 버튼이 화면에서 사라집니다.



11.5.2 예제에서 사용한 핵심 기능

components, objects, all

컴포넌트, 오브젝트 목록을 Collection 형식으로 제공합니다. 아래 그림처럼 디버깅 상태에서 실제 제공되는 형식을 확인할 수 있습니다. all 속성의 경우 objects, components 순서로 표시됩니다.



예를 들어 btnAddbutton 컴포넌트가 Button 컴포넌트 중에서 우선순위가 가장 높은 경우 해당 컴포넌트는 아래와 같이 사용할 수 있습니다.

```

this.btnAddbutton;
this.all[2];

```

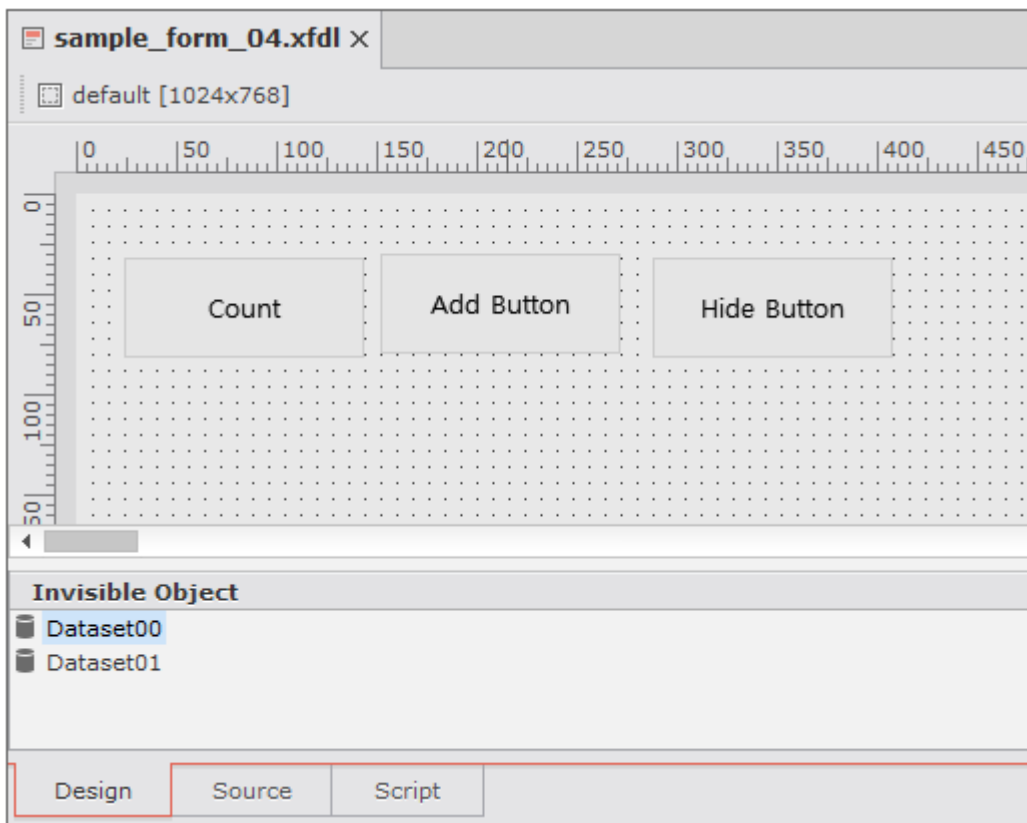



this.all[0], this.all[1]은 Dataset 오브젝트를 가리키고 있으며 그 다음 우선순위로 this.all[2]가 btnAddbutton 컴포넌트를 가리킵니다.

11.5.3 예제 구현 방법

1 Form 화면 구성하기

Button 컴포넌트를 예제 화면과 같이 배치합니다. Dataset 오브젝트도 두 개 추가해줍니다. Dataset 오브젝트를 어떻게 처리하는지 확인하기 위한 용도라 따로 컬럼값을 지정하지는 않습니다. Button 컴포넌트 하단에 정보를 출력하기 위해 TextArea 컴포넌트를 배치합니다.



2 Button 컴포넌트 이벤트 함수 작성하기

[Count] 버튼 클릭 시 components, objects, all 속성 정보를 보여줍니다.

```
this.btnCount_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.TextArea00.set_value("components.length: "+this.components.length+"\n"
        +"objects.length: "+this.objects.length+"\n"
        +"all.length: "+this.all.length);
}
```

```
};
```

[Add Button] 버튼 클릭 시 스크립트에서 Button을 생성하고 Form 오브젝트에 추가합니다. 여러 번 추가할 수 있도록 임의의 변수를 만들어 addChild 메소드에 지정하는 id값을 계속 변경해줍니다.

```
var i=0;
this.btnAddbutton_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var btnobj = new Button();
    this.addChild("btn"+i, btnobj);
    i++;
};
```

[Hide Button] 버튼 클릭 시 해당 버튼을 제외한 나머지 버튼이 화면에서 보이지 않도록 합니다. 다시 버튼을 클릭하면 화면에 나타납니다.

```
this.btnHide_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var tempobj;
    for(var j=0;j<this.components.length;j++)
    {
        tempobj = this.components[j];
        if(tempobj.visible == true && tempobj != obj)    {
            tempobj.set_visible(false);
        } else {
            tempobj.set_visible(true);
        }
    }

    if(obj.text == "Hide Button")
    {
        obj.set_text("Show Button");
    } else {
        obj.set_text("Hide Button");
    }
};
```

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 오브젝트의 속성을 확인합니다.

11.6 엔터키로 컴포넌트 이동하기

웹브라우저에서 컴포넌트를 이동할 때는 탭(Tab)키를 사용합니다. 하지만 엔터키를 사용하는 경우도 있습니다. 각 컴포넌트의 이벤트를 확인해 엔터키 입력 시 동작을 지정할 수도 있지만 Form 오브젝트에서 이벤트를 처리해줄 수도 있습니다. 각 컴포넌트에서 발생하는 이벤트는 상위 컴포넌트로 전달이 됩니다. 이를 활용한 예제입니다.

11.6.1 예제

두 개의 Edit 컴포넌트는 문자열 입력 후 탭키 또는 엔터키를 입력하면 다음 컴포넌트로 포커스를 이동합니다. [SUBMIT] 버튼은 클릭 시 Edit 컴포넌트의 텍스트를 모두 삭제합니다.

첫 번째 Edit 컴포넌트는 autoskip, maxlength, inputtype 속성값을 지정해 숫자만 입력할 수 있고 5자리를 입력하면 다음 컴포넌트로 포커스를 이동합니다.




11.6.2 예제에서 사용한 핵심 기능

onkeydown

해당 컴포넌트나 오브젝트에 포커스가 있는 상태에서 키 입력 시 발생하는 이벤트입니다. 해당 이벤트 함수에서 true 값을 반환하면 상위 컴포넌트로 이벤트가 전파되지 않습니다. 이번 예제에서는 Button 컴포넌트는 엔터키를 입력했을 때 Form 오브젝트에 지정된 이벤트를 실행하지 않도록 하기 위해 Button 컴포넌트의 onkeydown 이벤트 함수 내에서 true 값을 반환합니다.

getFocus

Form 오브젝트 내에서 현재 포커스를 가지고 있는 컴포넌트를 반환합니다. 예제에서는 KeyEventInfo 오브젝트의 fromreferenceobject 속성값을 참조해도 같은 컴포넌트를 반환합니다.

getNextComponent

탭키 입력 시 포커스가 이동할 컴포넌트를 반환합니다. taborder 속성값에 따라 반환하는 컴포넌트가 달라집니다. 단지 대상 컴포넌트를 반환하는 용도이며 실제 포커스가 변경되지는 않습니다.

setFocus

해당 컴포넌트로 포커스를 이동합니다.



탭키를 사용하는 경우에는 `getFocus`, `getNextComponent`, `setFocus` 메소드에서 처리하는 일련의 과정을 자동으로 처리하기 때문에 별도로 이벤트 함수를 작성할 필요가 없습니다.

maxlength, autoskip

입력하는 문자열의 최대값을 지정합니다. 지정된 최대값만큼 문자열이 입력되면 더는 입력되지 않습니다. `autoskip` 속성값을 `true`로 설정한 경우에는 입력한 문자열이 최대값이 도달하면 자동으로 다음 포커스를 가지는 컴포넌트로 이동합니다.

inputtype

첫 번째 Edit 컴포넌트는 `inputtype` 속성값을 'number'로 지정했습니다. 숫자만 입력할 수 있습니다.

11.6.3 예제 구현 방법

① Form 화면 구성하기

Edit, Button 컴포넌트를 예제 화면과 같이 배치합니다. 첫 번째 Edit 컴포넌트의 `autoskip`, `inputtype`, `maxlength` 속성값을 `true`, `number`, 5로 지정합니다.

② Form 이벤트 함수 작성하기

넥사크로 스튜디오에서 Form을 선택하고 속성창에서 `onkeydown` 속성 항목을 더블클릭합니다. 엔터키 입력 여부는 `KeyEventInfo` 오브젝트의 `keycode` 속성값으로 비교합니다. 엔터키를 입력하는 경우에는 13이라는 값으로 비교합니다.

```

this.Form_onkeydown = function(obj:nexacro.Form,e:nexacro.KeyEventInfo)
{
    if(e.keycode == 13)
    {
        var objFocus = this.getFocus();
        var objComp = obj.getNextComponent(objFocus, true);
        objComp.setFocus();
    }
};

```

③ Button 컴포넌트 이벤트 함수 작성하기

onkeydown 함수는 컴포넌트의 이벤트가 먼저 발생하고 상위 Form 오브젝트의 이벤트가 발생합니다. 이때 true를 반환하면 이벤트가 상위 Form 오브젝트로 전파되지 않습니다. Button 컴포넌트에 포커스가 있을때 엔터를 입력하면 클릭 이벤트가 발생하는데 이를 처리하기 위해 상위 Form 오브젝트로 이벤트가 전파되지 않도록 합니다.

```
this.btnSubmit_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.edit01.set_value('');
    this.edit02.set_value('');
};

this.btnSubmit_onkeydown = function(obj:nexacro.Button,e:nexacro.KeyEventInfo)
{
    return true;
};
```

④ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 오브젝트의 속성을 확인합니다.

11.7 타이머로 시간 표시하기

타이머 기능은 뭔가 변경되는 모습을 사용자에게 보여줄 때 유용하게 사용합니다. 이번 예제에서 보여줄 시계뿐 아니라 그래프를 애니메이션처럼 보여준다는지 데이터가 변경되는 모습을 시각적으로 잘 드러내기 위해 사용하기도 합니다.

자바스크립트에서도 setInterval 메소드를 사용하면 간단한 타이머를 구현할 수 있지만 한 번에 하나만 사용할 수 있습니다. 넥사크로플랫폼에서는 별도 id를 지정해 여러 개의 타이머를 구현할 수 있다는 것이 큰 차이입니다.

11.7.1 예제

검은색 Static 컴포넌트와 두 개의 Button 컴포넌트가 한 쌍입니다. [Time Start] 버튼 클릭 시 시간이 표시되며 [Time Stop] 버튼 클릭 시 표시된 시간의 업데이트가 중단됩니다. 빨간색 Static 컴포넌트의 쌍도 같은 형식인데 타이머 이벤트가 별도로 동작하는 것이 차이입니다.



11.7.2 예제에서 사용한 핵심 기능

setTimer

키값과 타이머 이벤트를 처리할 시간 간격을 지정해 타이머를 실행합니다. 시간 간격은 밀리세컨드 단위로 지정할 수 있습니다. 예를 들어 1000을 지정하면 1초 간격으로 타이머가 실행됩니다.

ontimer

setTimer에서 지정한 시간 간격에 따라 이벤트 함수가 실행됩니다. 이벤트 함수 내에서 TimerEventInfo 오브젝트의 timerid 속성값으로 지정된 키값도 확인할 수 있습니다.

killTimer

실행된 타이머를 중지합니다. setTimer 메소드 실행 시 설정한 키값을 지정해주어야 합니다.

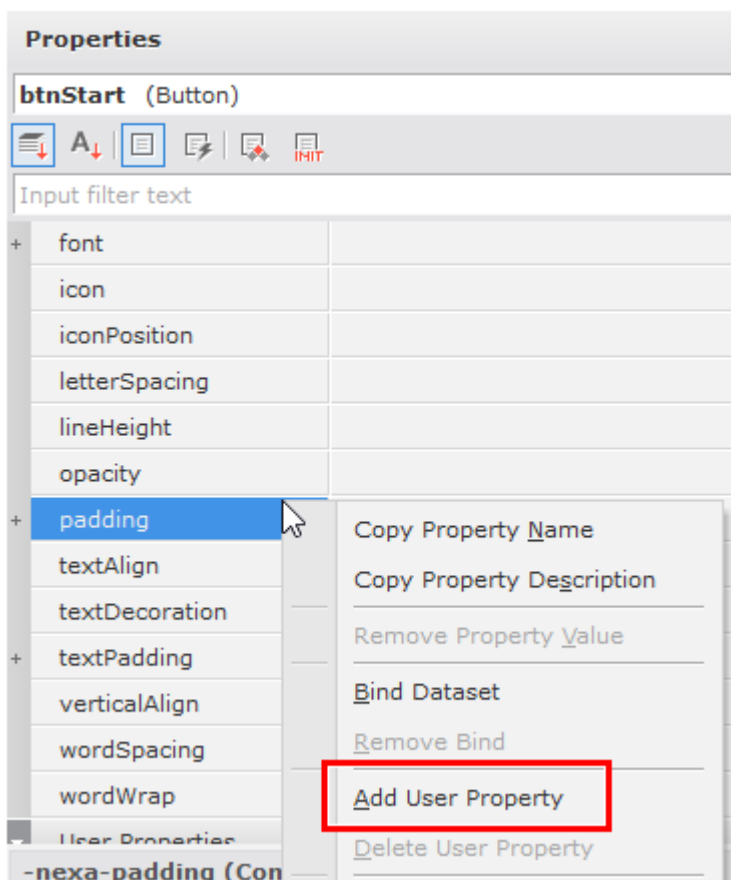
11.7.3 예제 구현 방법

① Form 화면 구성하기

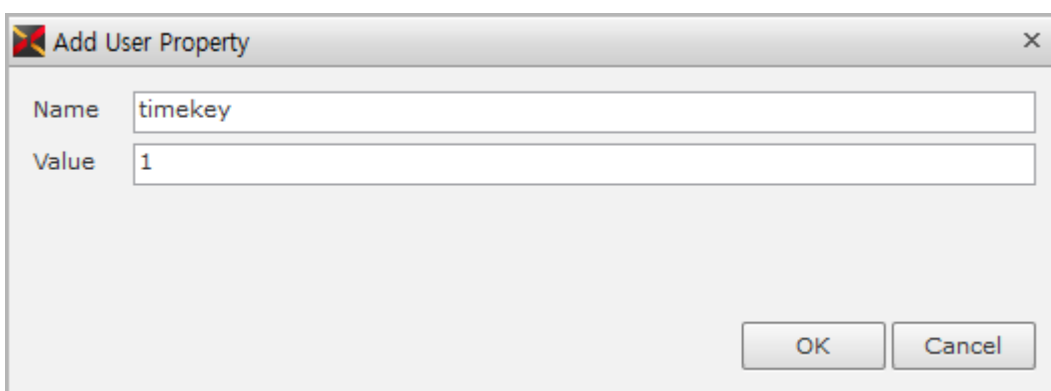
Static, Button 컴포넌트를 예제 화면과 같이 배치합니다. Static 컴포넌트의 text 속성값은 "READY"로 지정하고 background 속성값과 color 속성값을 구분해 지정합니다.

② 사용자 속성 추가하기

[Time Start] 버튼 클릭 시 전달할 키값을 Button 컴포넌트의 User Properties로 추가합니다. Button 컴포넌트를 선택하고 속성창에서 오른쪽 마우스 버튼을 클릭하면 컨텍스트 메뉴가 나타납니다. 여기에서 [Add User Property] 항목을 선택합니다.



그리고 입력창에 Name, Value 항목을 입력합니다. 예제에서는 Name 항목은 "timekey" 그리고 Value 항목은 첫 번째 버튼은 1 두 번째 버튼은 2를 입력합니다. 입력된 값은 속성창에서 [User Properties] 항목으로 보이며 다른 속성값처럼 사용할 수 있습니다.



3 타이머 생성 함수 작성하기

[Time Start] 버튼 클릭 시 타이머를 생성하는 함수를 작성합니다. 키값은 클릭한 Button 컴포넌트의 사용자 속성으로 추가한 timekey 속성을 지정합니다. 이렇게 하면 버튼마다 사용자 속성에 지정된 값으로 다른 키값을 지정해줍니다.

```

this.btnStart_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.setTimer(obj.timekey, 1000);
};

```



두 개의 Button 컴포넌트 모두 btnStart_onclick를 onclick 이벤트로 지정합니다. 따로 이벤트 함수를 지정하고 키값을 지정할 수도 있지만 사용자 속성으로 키값을 처리하기 때문에 하나의 이벤트 함수를 만들고 두 개의 Button 컴포넌트에서 사용합니다.

4 타이머 이벤트 함수 작성하기

타이머에 지정된 시간 간격마다 이벤트가 발생합니다. 이때 처리할 함수를 작성합니다. Form 오브젝트의 ontimer 이벤트를 선택하고 이벤트 함수를 생성합니다.

```

this.Form_ontimer = function(obj:nexacro.Form,e:nexacro.TimerEventInfo)
{
    if(e.timerid == 1)
    {
        this.staticTime.set_text(this.Clock());
    } else if(e.timerid == 2) {
        this.staticTime01.set_text(this.Clock());
    }
};

```

timerid 속성값에 따라 Static 컴포넌트의 text 속성값을 지정하는데 이때 Clock이라는 함수를 사용합니다. Clock 함수는 locale 속성값에 따라 현재 시간을 반환하는 함수입니다. 화면에 나타나는 시간 형식은 locale 속성값에 따라 달라질 수 있습니다.

```

this.Clock = function()
{
    var objDate = new Date();
    return objDate.toLocaleTimeString();
}

```

5 타이머 중지 함수 작성하기

[Time Stop] 버튼 클릭 시 타이머를 중지하는 함수를 작성합니다. 타이머를 중지했다가 다시 실행하는 것은 아니므로 타이머 자체를 삭제한다는 표현이 더 정확합니다.


```

this.btnEnd_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.killTimer(obj.timekey);
};

```

⑥ QuickView로 확인하기

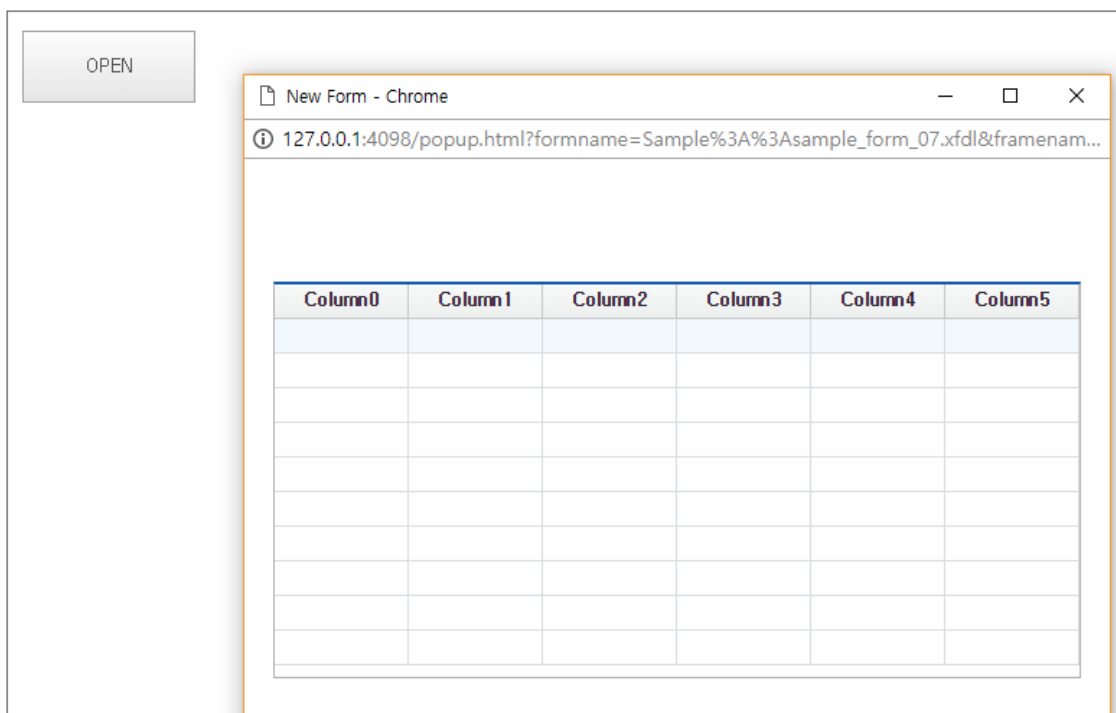
QuickView(Ctrl + F6)로 실행한 후 각 버튼을 클릭해 타이머가 정상적으로 실행되고 Static 컴포넌트에 시간이 표시되는지 확인해봅니다.

11.8 화면 크기에 따라 Grid 포맷 바꾸기

[화면 크기에 따라 레이아웃 바꾸기](#)와 비슷한 예제입니다. 하지만 이번에는 컴포넌트의 배치를 바꾸는 것이 아니라 Grid 포맷 속성을 바꾸어서 필요한 컬럼만 보이도록 합니다.

11.8.1 예제

버튼 클릭 시 open 메소드를 사용해 같은 Form을 새 창으로 띄웁니다. 띄운 창의 모서리 부분을 마우스로 잡고 움직여 화면 크기를 줄이면 Grid 컴포넌트에 보여지는 컬럼이 바뀌는 것을 확인할 수 있습니다.





11.8.2 예제에서 사용한 핵심 기능

onlayoutchanged

화면 크기 등이 변경되면서 레이아웃이 변경됐을 때 발생하는 이벤트입니다. 레이아웃이 바뀌는 시점에 Grid 컴포넌트의 속성을 바꾸어줍니다. 특정 컴포넌트의 visible 속성값을 수정해주면 화면에 보여지는 시점도 조절할 수 있습니다.

11.8.3 예제 구현 방법

① 화면 구성하기

Button 컴포넌트와 Grid 컴포넌트를 예제 화면과 같이 배치합니다.

② 레이아웃 추가하기

레이아웃은 Form 생성 시 지정할 수 있고 Form 생성 이후에 추가할 수 있습니다. 이번 예제에서는 Form을 생성한 후 추가하는 방식을 사용합니다. [Design] 모드 화면에서 오른쪽 마우스 클릭 후 컨텍스트 메뉴에서 [Add Layout] 항목을 선택하거나 화면 오른쪽 상단에 있는 미니 툴바에서 [Add Layout] 항목을 선택합니다.

③ 레이아웃 정보 입력하기

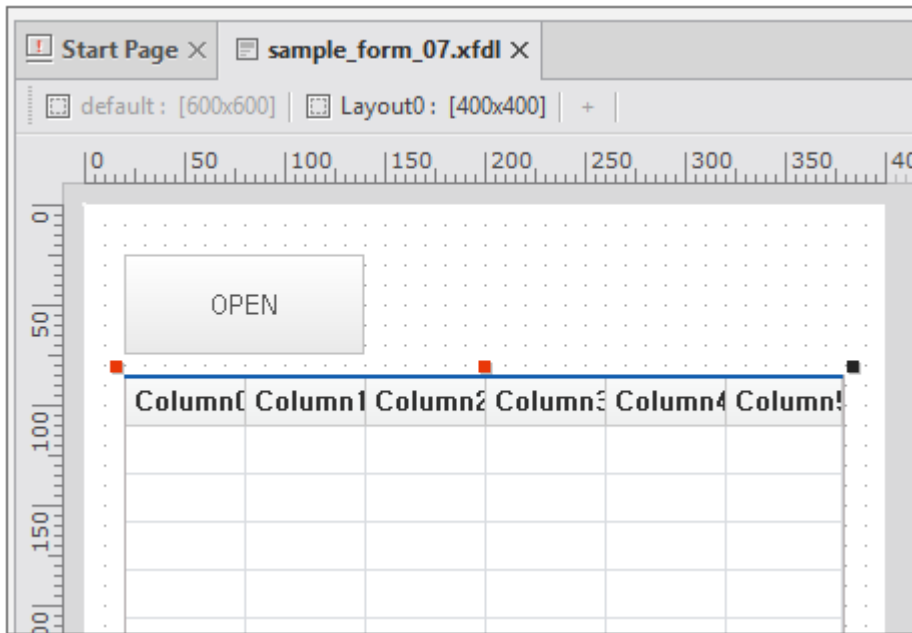
[Add Layout] 항목을 클릭하면 레이아웃 정보를 입력할 수 있는 [New Layout] 창이 나타납니다. 예제에서는 width, height 값만 입력합니다.

Layout Information	
name	Layout1
screenid	
width	400 px
height	400 px
stepcount	
stepindex	
description	

항목을 입력하면 레이아웃이 추가되고 [Design] 모드 화면 상단에 추가된 레이아웃 탭이 생성됩니다.

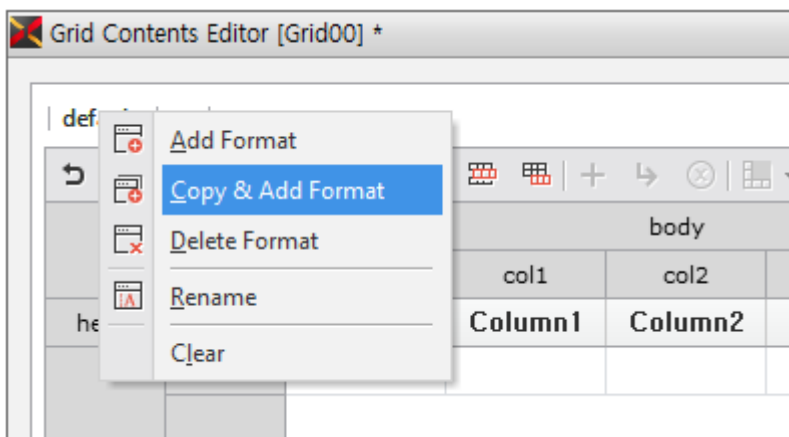
4 새로운 레이아웃의 콘텐츠 배치 설정하기

추가된 레이아웃을 선택하고 Grid 컴포넌트의 크기만 적절하게 조정합니다.



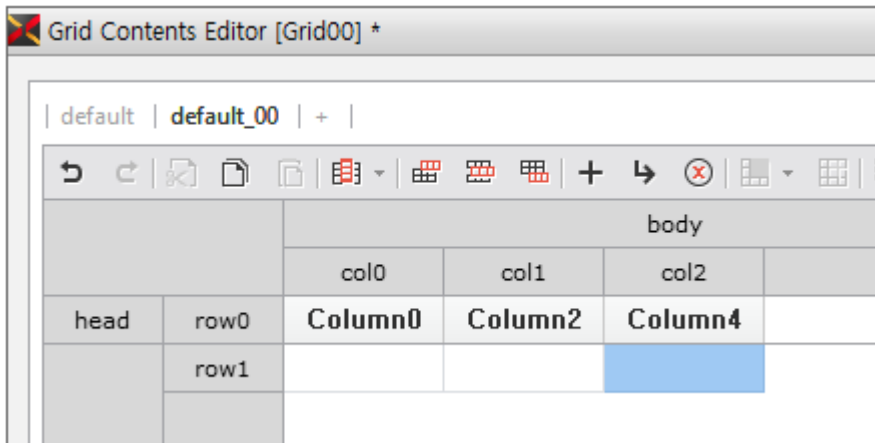
5 Grid 포맷 복사해서 추가하기

Grid 컴포넌트를 더블클릭해 Grid Content Editor를 실행합니다. 아예 새로운 포맷을 만드는 것이 아니라 기존 포맷을 살짝 변경할 것이어서 에디터 상단에서 [+] 버튼을 클릭하지 않고 default 포맷 항목 위에서 오른쪽 마우스 버튼을 클릭하면 표시되는 컨텍스트 메뉴에서 [Copy & Add Format] 항목을 선택합니다.



6 Grid 포맷 변경하기

추가된 포맷에서 홀수 컬럼만 삭제합니다. Column1, Column3, Column5를 선택하고 삭제합니다. 그러면 3개의 컬럼만 남습니다.



7 Button 컴포넌트 이벤트 함수 작성하기

실행된 웹브라우저의 크기를 조절해도 되지만 편의를 위해 같은 Form을 가지는 창을 open 메소드를 사용해 띄워줍니다. 띄운 창의 크기를 조절해 레이아웃 변경 기능을 확인합니다.

```
this.btnOpen_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    nexacro.open("TEST", "Sample::sample_form_07.xfd1", this.getOwnerFrame());
};
```



예제에서는 Form 이름을 직접 지정해주었지만 ChildFrame에 바로 연결된 Form이라면 this.parent.formurl 속성값으로 지정할 수 있습니다.

8 onlayoutchanged 이벤트 함수 작성하기

레이아웃이 변경됐을 때 Grid 컴포넌트의 formatid 속성값을 변경합니다.

```
this.Form_onlayoutchanged = function(obj:nexacro.Form,e:nexacro.LayoutChangeEventInfo)
{
    if(e.newlayout == 'default') {
        this.Grid00.set_formatid("default");
    } else if(e.newlayout == 'Layout0') {
        this.Grid00.set_formatid("default_00");
    }
};
```

9 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 [OPEN] 버튼을 클릭해 새 창을 띄웁니다. 새 창의 크기를 변경하면서 Gri

d 컴포넌트에 표시되는 컬럼이 변경되는지 확인합니다.

10 visible 속성 조정하기

처음 화면을 실행할 때 사용하지 않는 Grid 컴포넌트가 보여지는 것이 조금 불편합니다. 새 창에서도 [OPEN] 버튼이 없었으면 좋겠습니다. Form 오브젝트의 onload 이벤트 함수 내에서 필요한 컴포넌트의 visible 속성을 조정합니다.

```
this.sample_form_07_onload = function(obj:nexacro.Form,e:nexacro.LoadEventInfo)
{
    if(this.parent.name == "TEST") {
        this.Grid00.set_visible(true);
        this.btnOpen.set_visible(false);
    } else {
        this.Grid00.set_visible(false);
        this.btnOpen.set_visible(true);
    }
};
```

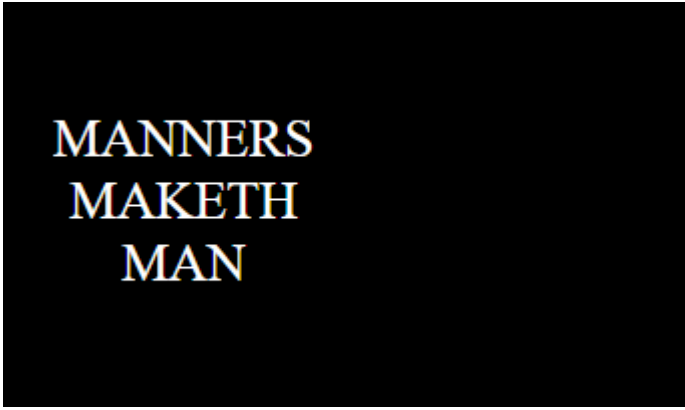
앞 단계에서 open 메소드 실행 시 첫번째 값으로 "TEST"라는 이름을 주었습니다. 새창을 띄울때 그 부모가 되는 ChildFrame 오브젝트의 이름을 "TEST"로 한다는 의미입니다. onload 이벤트 함수 내에서는 그 이름을 가지고 현재 Form 오브젝트가 어떤 상황인지 판단합니다.

11.9 스텝 기능을 활용해 슬라이드쇼 만들기

stepcount 속성을 이용해 스텝을 나누는 것은 사용자가 직접 화면을 전환하기 위한 용도로 주로 사용하지만 타이머를 설정하면 자동으로 화면이 바뀌면서 슬라이드쇼 효과를 만들 수 있습니다.

11.9.1 예제

첫 화면이 로딩되고 2초마다 화면이 전환됩니다.



11.9.2 예제에서 사용한 핵심 기능

stepselector

StepControl 오브젝트에 접근하기 위해 사용하는 속성입니다. 해당 속성을 사용하면 StepControl 오브젝트 하위의 속성이나 메소드를 사용할 수 있습니다.

stept

stepindex 속성값을 1씩 증가하거나 감소하면서 다음 화면으로 이동시키는 메소드입니다. bRotate 파라미터는 기본값이 false이고 반복해서 슬라이드를 표시하려면 true로 설정합니다.

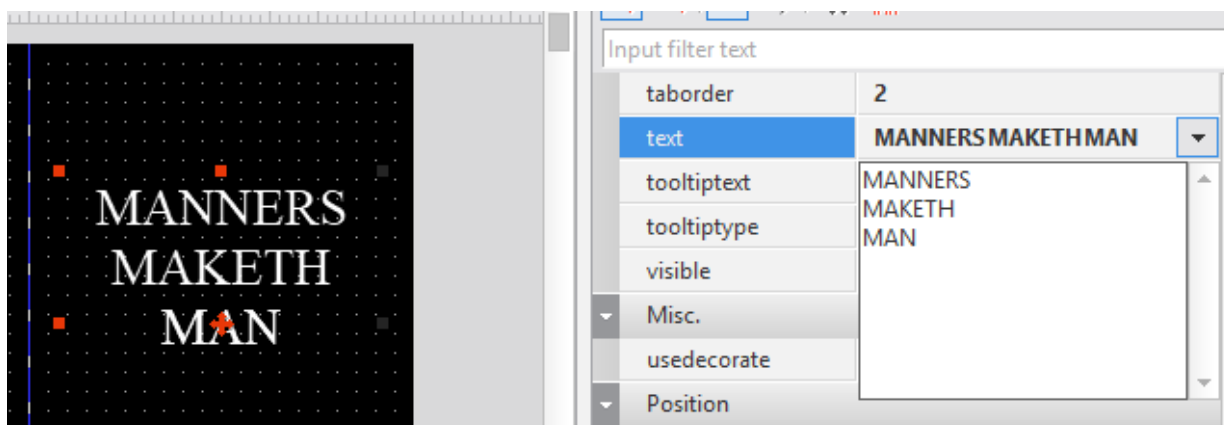
11.9.3 예제 구현 방법

① Form 화면 구성하기

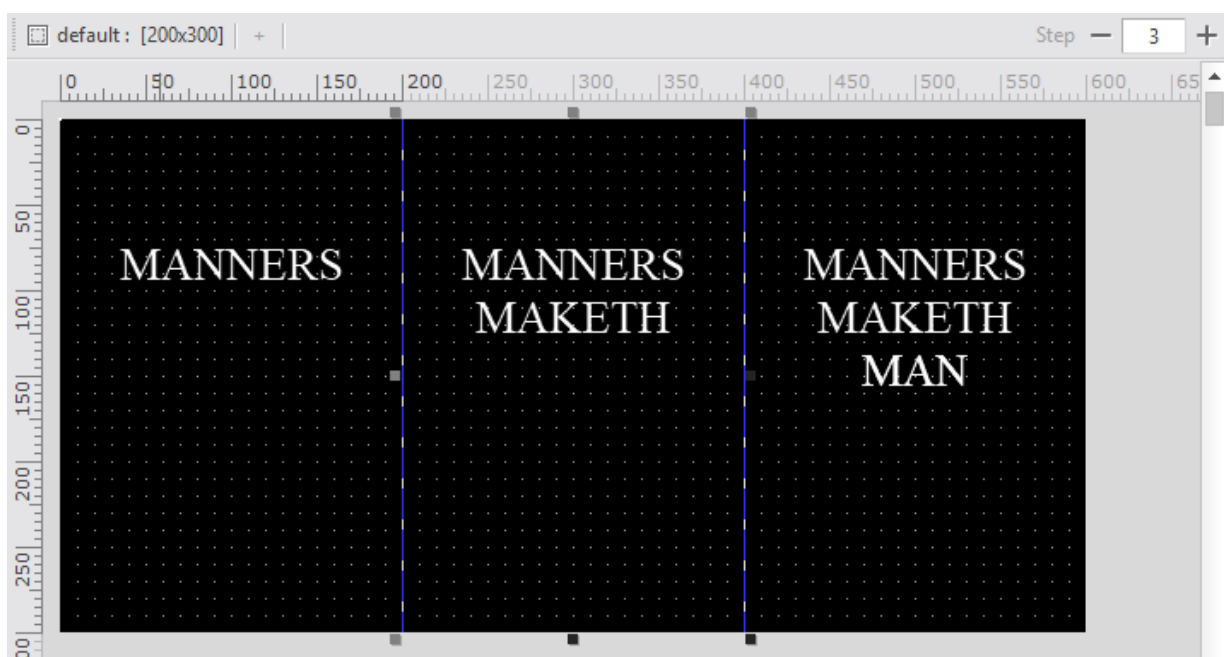
Form을 생성하고 width, height 속성값을 200, 300으로 지정합니다. 그리고 stepcount 속성값을 3으로 지정합니다. 화면이 3개로 분할되면서 처음 생성한 Form의 크기만한 영역이 stepcount 속성값에 지정한 숫자만큼 생성됩니다.

② 컴포넌트 배치하기

Static 컴포넌트를 배치합니다. 같은 크기의 컴포넌트를 배치하고 text 속성값만 다르게 지정합니다. 속성창에서 text 속성값을 선택하고 오른쪽에 있는 버튼을 클릭하면 넓게 입력할 수 있는 창이 펼쳐집니다. Ctrl+ENTER 키를 입력하면 줄을 바꾸어 입력할 수 있습니다.



입력한 컴포넌트의 모습은 아래 그림과 같습니다. Form 오브젝트의 배경색을 검정색으로 Static 컴포넌트의 color 속성값은 흰색으로 지정했습니다.



③ 타이머 이벤트 함수 작성하기

Form 오브젝트의 onload 함수에서 setTimer 메소드를 실행합니다.

```
this.sample_form_08 onload = function(obj:nexacro.Form,e:nexacro.LoadEventInfo)
{
    this.setTimer(1, 2000);
};
```

2초마다 ontimer 이벤트가 발생하며 stepindex 속성값을 변경합니다. stepindex 속성값이 변경되면 다음 스텝 화면으로 이동하게 됩니다.

```

this.sample_form_08_ontimer = function(obj:nexacro.Form,e:nexacro.TimerEventInfo)
{
    if(e.timerid == 1){
        this.stepsselector.stepIt(true, true);
    }
};

```

4 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 슬라이드가 2초마다 바뀌는지 확인합니다.

11.10 컴포넌트 목록 표시하기

컴포넌트의 목록을 표시하는 기능입니다. 해당 기능 자체로는 사용할 일이 많지 않지만, Form 내의 모든 컴포넌트의 특정 속성값을 변경해야 하는 경우에는 전체 목록을 가져와서 반복문을 사용해 처리하게 됩니다. 예제의 코드를 활용하면 원하는 기능을 구현할 수 있습니다.

11.10.1 예제

[check list] 버튼을 클릭하면 화면에 표시되는 컴포넌트 목록을 Grid에 표시합니다. [check all list]는 화면에 보이지 않는 컴포넌트(Dataset 오브젝트 같은)나 바인딩 아이템을 포함한 목록을 표시합니다.

check list

check all list

Static00

☐ CheckBox00

tabpage1

tabpage2

Button00

No	Parent	ID	path
1	[object Form]	Button00	Button00
2	[object Form]	Button01	Button01
3	[object Form]	Grid00	Grid00
4	[object Form]	Tab00	Tab00
5	[object Tab]	tabpage1	Tab00,tabpage1
6	[object TabpageControl]	Button03	Tab00,tabpage1,form,Button03
7	[object Tab]	tabpage2	Tab00,tabpage2
8	[object TabpageControl]	MaskEdit00	Tab00,tabpage2,form,MaskEdit00
9	[object Form]	Div00	Div00
10	[object Div]	TextArea00	Div00,form,TextArea00
11	[object Div]	Static00	Div00,form,Static00
12	[object Div]	Div00	Div00,form,Div00
13	[object Div]	CheckBox00	Div00,form,Div00,form,CheckBox00


[sample_form_09.xfdl](#)

11.10.2 예제에서 사용한 핵심 기능

components

Form에 포함된 하위 컴포넌트 목록을 가지고 있는 속성입니다. 인덱스 또는 id로 원하는 컴포넌트에 접근할 수 있습니다.

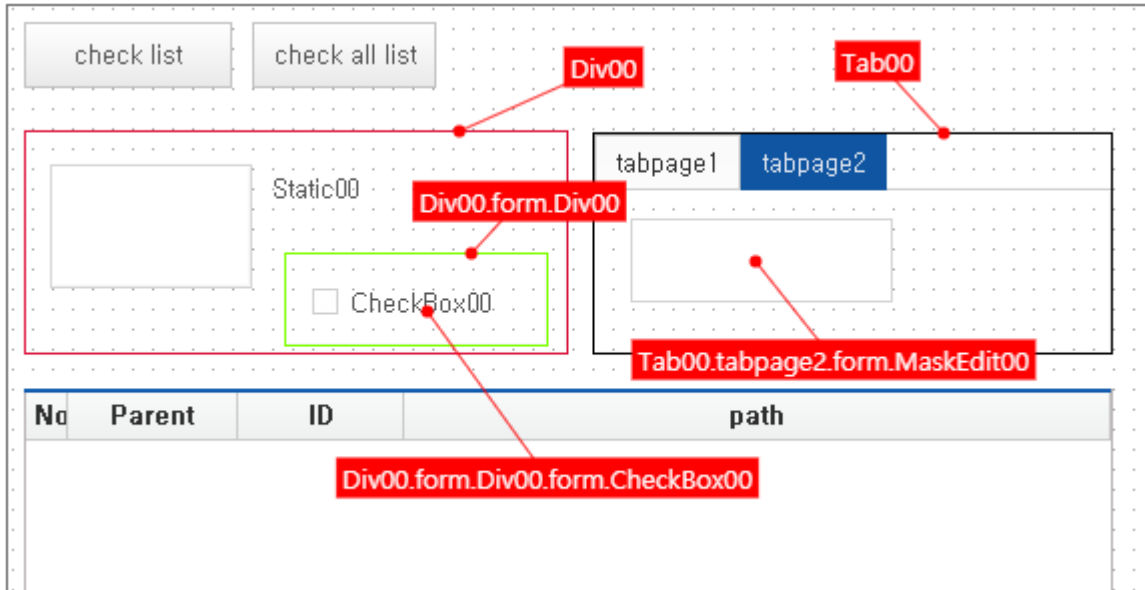
all

components 속성으로 접근하지 못하는 Invisible Object, Bind 정보를 포함한 목록을 가지고 있는 속성입니다.

11.10.3 예제 구현 방법

1 Form 화면 구성하기

Form을 생성하고 각 컴포넌트를 배치합니다. 예제에서는 Div 컴포넌트와 Tab 컴포넌트 안에 컴포넌트를 배치하는 예제를 사용하고 있습니다. 상단 기능 버튼과 하단 목록을 표시하는 Grid 컴포넌트 외에는 자유롭게 배치하면서 테스트할 수 있습니다.



2 Dataset 바인딩하기

Dataset 오브젝트의 담겨지는 데이터는 컴포넌트 목록 정보를 직접 대입합니다. Dataset 오브젝트는 Column 0, Column1, Column2 3개의 컬럼을 가지고 Row 정보는 입력하지 않습니다. Grid 컴포넌트와 Div00 안에 있는 TextArea 컴포넌트에 바인딩합니다. Dataset 오브젝트 아이콘을 드래그해서 TextArea 컴포넌트 위에 가져다놓으면 바인딩할 속성과 컬럼을 지정할 수 있습니다.

Bind Item					
	Bind ID	Object	Property	Dataset	Column ID
-	item0	Div00.form.TextArea00	value	Dataset00	Column2
+					

3 No 컬럼 추가하기

순번을 표시하는 No 컬럼을 Grid 컴포넌트에 추가합니다. Grid 컴포넌트를 더블클릭해 Grid Contents Editor를 실행하고 가장 앞에 컬럼을 추가한 후 head text는 "No", 셀 expr 속성값은 "expr:rowidx+1"로 지정합니다. rowidx는 0부터 시작하는 인덱스값을 반환하는데 1을 더해주면 1부터 순번을 표시할 수 있습니다.

col0	col1	col2			id	body0
No	Parent	ID			Action	
					autosizecol	default
					autosizerow	default
					controlautosizingtype	both
					displaytype	normal
					edittype	none
					expr	expr:rowidx+1
					suppress	0

4 onclick 이벤트 함수 작성하기

Button 컴포넌트 클릭 시 컴포넌트 또는 오브젝트 목록을 화면에 표시하는 기능을 처리합니다. 실제 데이터 처리는 get_com_info 함수에서 처리합니다.

```

this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Dataset00.clearData();
    this.get_com_info(this);
};

this.Button01 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Dataset00.clearData();
    this.get_com_info(this, true);
};

```

get_com_info 함수는 3개의 값을 받아서 처리합니다. 첫번째 값은 오브젝트입니다. Div 컴포넌트에 안에 있는 컴포넌트가 있는 경우 Form에서는 Div 컴포넌트가 있는 것만 확인하고 Div 컴포넌트 안에 있는 컴포넌트의 목록은 확인할 수 없습니다. 때문에 Div, Tab 컴포넌트 같은 경우에는 안쪽에 컴포넌트를 가지고 있는지 확인하는 과정을 거칩니다. 조건에 따라 다시 get_com_info 함수를 호출하는 코드입니다. 이런 방식을 재귀함수(Recursive)라고 합니다.

```

this.get_com_info = function(obj, bAll, bRecursive)
{
    var i;
    var components = bAll?obj.all:obj.components;
    this.Dataset00.set_enableevent(false);

    for( i = 0 ; i < components.length ; i++ )
    {

```

```

var nRow = this.Dataset00.addRow();
this.Dataset00.setColumn(nRow,0,components[i].id);
this.Dataset00.setColumn(nRow,1,bRecursive?obj.parent.valueOf():this.valueOf());
this.Dataset00.setColumn(nRow,2,this.get_url(components[i],""));
var strType = components[i].valueOf();

if(strType == "[object Div]" || strType == "[object Tab]){
    if (strType == "[object Tab]") {
        for(var j = 0 ; j < components[i].tabpages.length ; j++ )
        {
            var nRow = this.Dataset00.addRow();
            this.Dataset00.setColumn(nRow,0,components[i].tabpages[j].id);
            this.Dataset00.setColumn(nRow,1,components[i].valueOf());
            this.Dataset00.setColumn(nRow,2,this.get_url(components[i].tabpages[j
],""));

            this.get_com_info(components[i].tabpages[j].form, bAll, true);
        }
    } else {
        this.get_com_info(components[i].form, bAll, true);
    }
}
}
this.Dataset00.set_enableevent(true);
};

```

get_url 함수는 목록에 표시되는 아이템의 경로를 표시합니다.

```

this.get_url = function(obj, str)
{
    if(obj == "[object Form]")
    {
        var s = "form";
        if(obj.name == "sample_form_09")
        {
            return str;
        }
    }
    else {
        var s = obj.name;
    }
    if(str != "")

```

```

{
    s += "."+str;
}
return this.get_url(obj.parent, s);
};

```

5 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 Button 컴포넌트를 클릭해 실제 목록과 맞게 출력되는지 확인해봅니다.

11.11 서버와 transaction 처리하기

transaction 메소드는 주로 Form 오브젝트의 메소드를 사용합니다. 하지만 AppVariables와 연계해 사용해야 할 때에는 application 오브젝트의 transaction 메소드를 사용합니다.

11.11.1 예제

버튼을 클릭하면 transaction 메소드를 실행해 서버에 있는 데이터와 통신을 시작합니다. 가져온 데이터는 지정된 Dataset에 입력합니다. 선택한 동기화 여부 조건에 따라 통신이 처리되는 과정이 달라집니다.

선택한 동기화 여부 조건에 따라 "Before Transaction", "Transaction rowcount:", "After Transaction" 3개의 경고 대화상자가 보이는 순서가 달라집니다.

Transaction TEST	☒ ASync ☐ Sync	Transaction TEST	☐ ASync ☒ Sync
Before Transaction After Transaction Transaction rowcount: 23208		Before Transaction Transaction rowcount: 23208 After Transaction	



sample_form_10.xfdl

11.11.2 예제에서 사용한 핵심 기능

transaction

transaction 메소드는 데이터를 주고받는 용도로 사용하지만, 이번 예제에서는 데이터를 받는 용도로만 사용했습니다. XML 파일을 읽어서 지정된 Dataset에서 해당 값을 받아 처리합니다. bAsync 옵션 설정에 따라 transaction 처리를 동기식(transaction 처리를 끝내고 다음 처리를 할지) 또는 비동기식(transaction 처리 여부와 상관없이 다음 처리 진행)으로 진행할지 결정합니다. default 값은 비동기식(true)입니다.

11.11.3 예제 구현 방법

① 메인 화면 구성하기

Button 컴포넌트와 Radio, TextArea 컴포넌트를 예제 화면과 같이 배치합니다. Radio 컴포넌트의 innerdataset 속성값은 아래와 같습니다.

InnerDataset			
No	codecolumn	datacolumn	
1	1	ASync	
2		Sync	



Boolean 값 처리 시 false 값을 가지는 조건은 0, -0, null, false, NaN, undefined, 빈 문자열입니다. innerdataset은 각 컬럼값을 String 형식으로 처리하기 때문에 false 값을 가지려면 빈 문자열로 설정하는 방법을 취합니다. Dataset을 따로 만들면 컬럼 형식을 지정할 수 있어서 원하는 형식으로 지정하고 적절한 값을 대입할 수 있습니다.

Dataset 오브젝트를 추가하고 컬럼 id가 id, name인 2개 컬럼을 추가합니다.

② Button 컴포넌트 이벤트 함수 작성하기

버튼 클릭 시 처리할 이벤트 함수를 작성합니다. transaction 메소드 실행 전과 후 그리고 콜백함수 처리 시에 TextArea 컴포넌트에 메시지를 추가하는 함수를 실행하는데 bAsync 옵션 설정에 따라 메시지를 처리하는 순서가 달라집니다.

```
var bAsync = true;
this.btnTransaction_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.TextArea00.set_value("");
}
```

```

        this.fn_addlog("Before Transaction");
        this.transaction("TEST","FileSample::sample_form_10_file.xml", null, "Dataset00=TEST",
        null, "fn_test", bAsync);
        this.fn_addlog("After Transaction");
    };

    this.fn_addlog = function(strMessage)
    {
        this.TextArea00.set_value(this.TextArea00.text + strMessage + '\n');
    }

```

3 콜백함수 작성하기

transaction 메소드를 실행하면 지정된 URL에 접근해 데이터를 수신합니다. 데이터 수신이 정상적으로 처리 되면 콜백함수로 지정된 함수를 실행합니다. 예제에서는 "fn_test"라는 콜백함수명을 지정했습니다.

넥사크로플랫폼에서 사용하는 콜백함수는 이름은 달라질 수 있지만 형식은 아래와 같습니다.

[function name] = function(strSvcID, nErrorCode, strErrorMsg) {}

```

this.fn_test = function(strSvcID, nErrorCode, strErrorMsg)
{
    this.fn_addlog("Transaction rowcount: "+this.Dataset00.rowcount);
}

```

4 Radio 컴포넌트 이벤트 작성하기

Radio 컴포넌트의 아이템 선택 시 bAsync 변수에 값을 대입합니다. 해당 변수는 transaction 메소드 호출 시 옵션값으로 지정됩니다.

```

this.radioTransaction_onitemchanged = function(obj:nexacro.Radio,e:nexacro.
ItemChangeEventInfo)
{
    bAsync = obj.value;
};

```

5 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 bAsync 옵션 선택 후 버튼을 클릭해 경고 대화상자가 나타나는 순서를 확인합니다.

ASync를 선택한 경우에는 transaction 메소드를 실행하는 동안 다른 프로세스를 멈추지 않습니다. 그래서경

고 대화상자가 보이는 순서는 다음과 같습니다.

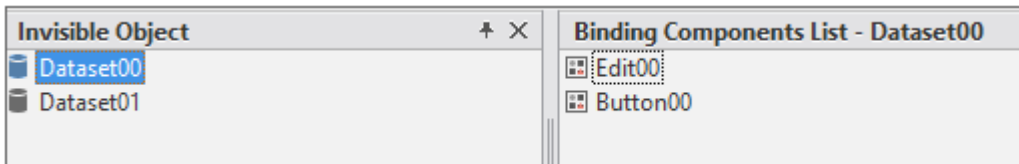
[Before Transaction] -> [After Transaction] -> [Transaction rowcount: xxx]

Sync로 선택한 경우에는 transaction 메소드를 처리한 다음 다른 프로세스를 진행합니다. 그래서 경고 대화상자가 보이는 순서는 다음과 같습니다.

[Before Transaction] -> [Transaction rowcount: xxx] -> [After Transaction]

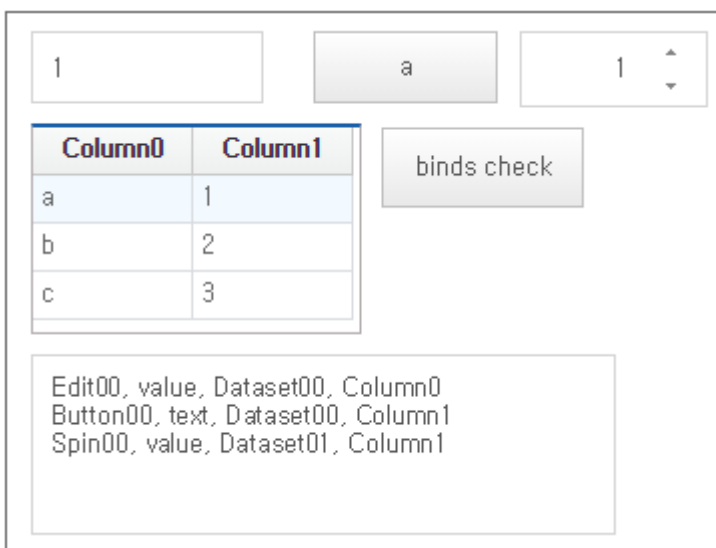
11.12 Bind 아이템 목록 확인하기

넥사크로 스튜디오 Invisible Object 영역에서 Dataset 오브젝트를 선택하면 Binding Components List 영역에 아이템을 표시합니다. 같은 내용을 스크립트에서 확인하는 방법을 살펴봅니다.



11.12.1 예제

[binds check] 버튼을 클릭하면 Form 오브젝트 내에서 Bind 아이템 목록을 TextArea 컴포넌트에 표시합니다. Grid 컴포넌트처럼 bindddataset 속성에 직접 바인딩 설정을 하는 컴포넌트는 표시하지 않습니다.





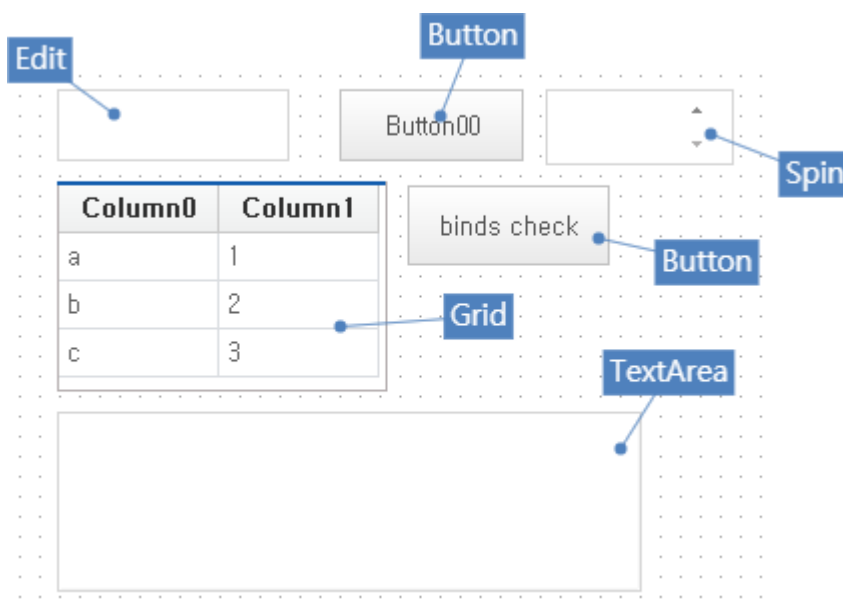
11.12.2 예제에서 사용한 핵심 기능

binds

Form 오브젝트 내 바인딩 정보를 가지고 있는 속성입니다. 각 바인딩 정보는 BindItem 오브젝트에 담겨 있습니다.

11.12.3 예제 구현 방법

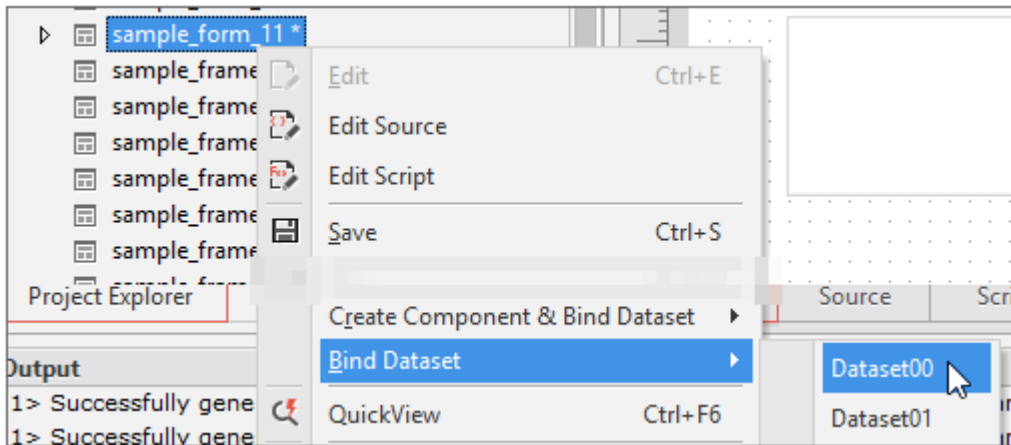
- 1 화면에 컴포넌트를 아래와 같이 배치합니다.



- 2 2개의 Dataset 오브젝트를 추가합니다.

Dataset00 오브젝트에는 두 개의 컬럼을 추가하고 첫 번째 컬럼 데이터는 a, b, c 두 번째 컬럼 데이터는 1, 2, 3 으로 설정합니다. Dataset01 오브젝트도 두 개의 컬럼을 추가하고 첫 번째 컬럼 데이터는 1, 2, 3 두 번째 컬럼 데이터는 a, b, c 로 설정합니다.

- 3 Project Explorer에서 Form 오브젝트를 선택하고 컨텍스트 메뉴에서 [Bind Dataset > Dataset00]을 선택합니다.



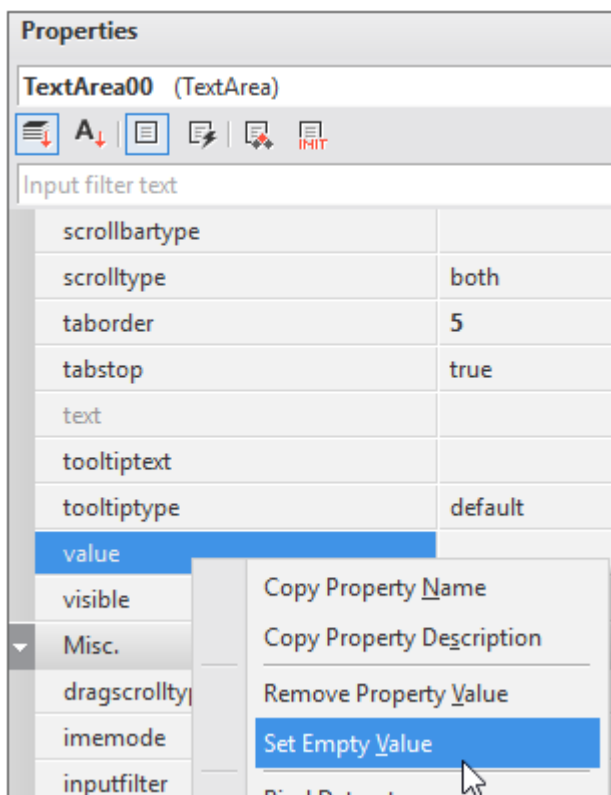
- ④ Bind Item 창에서 아래와 같이 값을 입력합니다. Bind Item 창에서는 여러 개의 Bind 아이템 목록을 관리하는데 편리하게 사용할 수 있습니다.

Bind Item					
	Bind ID	Object	Property	Dataset	Column ID
—	item0	Edit00	value	Dataset00	Column0
—	item1	Button00	text	Dataset00	Column1
—	item3	Spin00	value	Dataset01	Column1
+					

- ⑤ Dataset01 오브젝트를 선택하고 Grid 컴포넌트에 바인딩합니다.
- ⑥ [binds check] 버튼 컴포넌트를 선택하고 onclick 이벤트 함수를 아래와 같이 추가합니다.

```
this.Button01 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    for(var i = 0; i < this.binds.length; i++)
    {
        this.TextArea00.set_value(this.TextArea00.value+this.binds[i].compid+", "+ this.
        binds[i].propid+", "+ this.binds[i].datasetid + ", " + this.binds[i].columnid+"\n");
    }
};
```

- ⑦ TextArea 컴포넌트를 선택하고 속성창에서 해당 값을 빈값으로 설정합니다. value 속성항목을 선택하고 컨텍스트 메뉴에서 [Set Empty Value]를 선택합니다.



Empty Value 설정을 하지 않으면 undefined로 값을 처리합니다.

- 8 QuickView(Ctrl + F6)로 실행한 후 [binds check] 버튼 컴포넌트를 클릭해 TextArea 컴포넌트에 Bind 아이템 목록이 표시되는지 확인합니다.

11.13 컴포넌트의 속성을 모두 수정하기

Form 내에 있는 컴포넌트의 속성을 같은 값으로 수정하는 예제입니다.

11.13.1 예제

[Check] 버튼을 클릭하면 Edit, Combo, Button 컴포넌트의 배경색이 수정됩니다.

Edit	Combo ▼	Button
Edit	Combo ▼	Button
Edit	Combo ▼	Button
Edit	Combo ▼	Button
Edit	Combo ▼	Button
Check		

 sample_form_12.xfdl

11.13.2 예제에서 사용한 핵심 기능

instanceof

해당하는 컴포넌트가 어떤 오브젝트인지 확인할 수 있습니다. Edit 컴포넌트의 경우 Edit 또는 nexacro.Edit 모두 true 값을 반환합니다.



<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/instanceof>

11.13.3 예제 구현 방법

- 1 화면에 컴포넌트를 아래와 같이 배치합니다.

Edit	Combo ▼	Button
Edit	Combo ▼	Button
Edit	Combo ▼	Button
Edit	Combo ▼	Button
Edit	Combo ▼	Button
Check		

- ② [Check] 버튼의 onclick 이벤트 함수를 아래와 같이 작성합니다.

[Check] 버튼은 속성 변경 시 제외합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    for(var i = 0; i < this.components.length; i++)
    {
        if(this.components[i] instanceof nexacro.Edit)
        {
            this.components[i].set_background("lavenderblush");
        }
        else if(this.components[i] instanceof nexacro.Combo)
        {
            this.components[i].set_background("beige");
        }
        else if(this.components[i] instanceof nexacro.Button)
        {
            if(this.components[i].id != obj.id)
            {
                this.components[i].set_background("lightgreen");
            }
        }
    }
};
```

- ③ QuickView(Ctrl + F6)로 실행한 후 [Check] 버튼 컴포넌트를 클릭해 나머지 컴포넌트의 배경색이 바뀌는지 확인합니다.

12.

Frames

12.1 Frame 소개

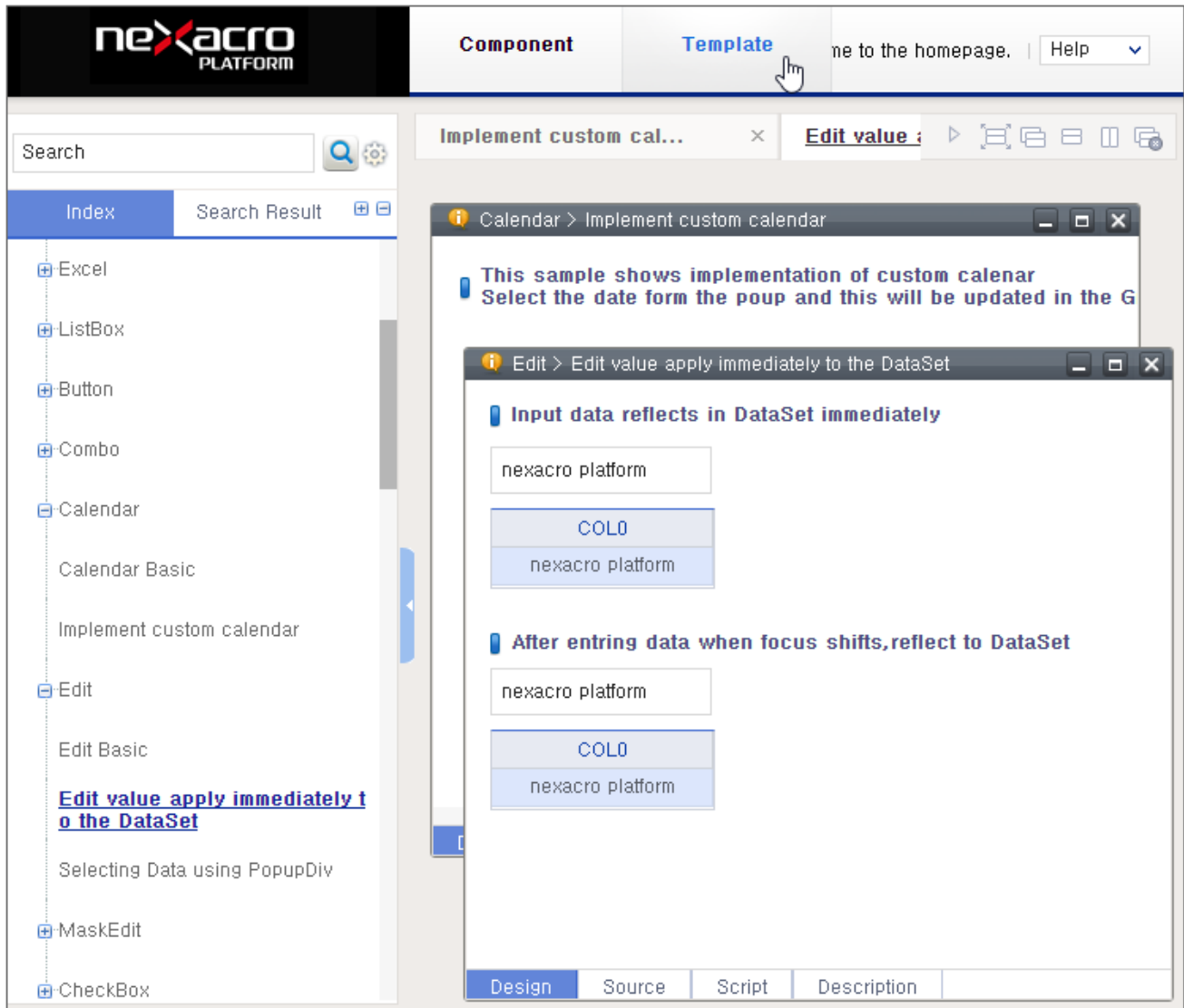
독립적인 창 구조를 가지고 동작하는 애플리케이션을 구성할 때 Frame 구조를 가집니다. MainFrame은 그 이름처럼 Frame 구조를 이루는 기본 오브젝트입니다. MainFrame은 하나만 존재할 수 있습니다.

웹브라우저에서 실행하는 경우에는 웹브라우저가 Frame의 역할을 담당합니다. 콘텐츠의 로딩 상태나 화면의 크기, 타이틀바 설정을 웹브라우저에서 처리합니다. 웹브라우저의 특성에 따라 화면에 보이는 모습은 조금씩 다를 수 있습니다.

Frame 형태에 따라 Div 컴포넌트처럼 Container 컴포넌트를 사용하는 것인지 구분이 되지 않을 수 있습니다. Frame은 전체 애플리케이션의 구조를 지정하는 것이라면 Container 컴포넌트는 각 콘텐츠의 구조를 설정합니다. Container 컴포넌트는 스크립트로 그 구조를 변경할 수 있지만, Frame 구조는 쉽게 변경하기 어렵습니다. Frame 구조는 제공하는 콘텐츠의 일관성을 유지하는 역할을 하고 있어 구조를 변경하지 않은 것이 좋습니다.

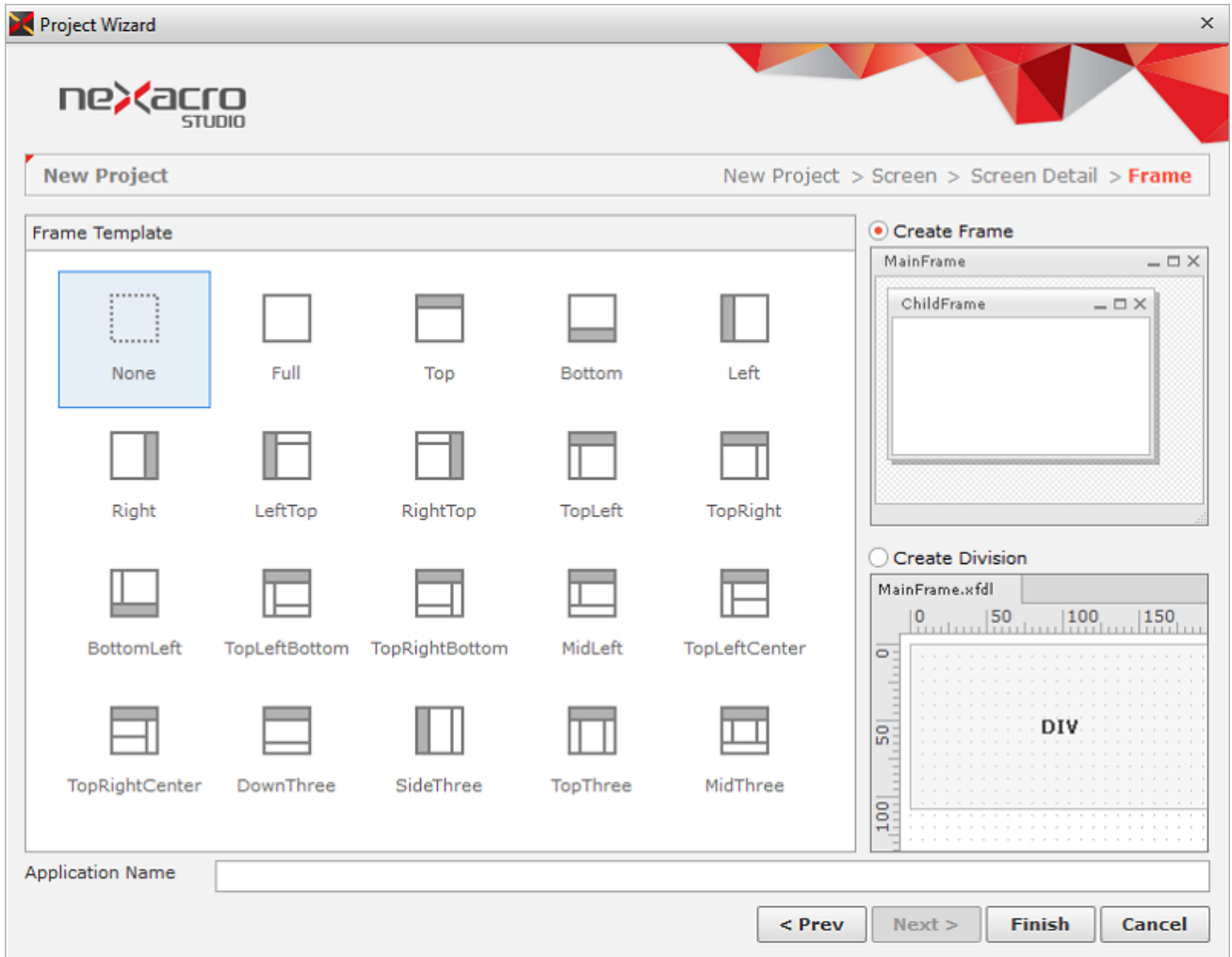
12.1.1 사용 예

최근에는 메뉴를 다양한 형식으로 표현하고 있지만, 아직도 여전히 상단 메인 메뉴, 왼쪽 서브 메뉴 구조를 유지하는 곳이 많습니다. 특히 콘텐츠의 양이 많은 경우에는 책을 읽는 것처럼 자연스럽게 접근할 수 있는 방식이기도 합니다.

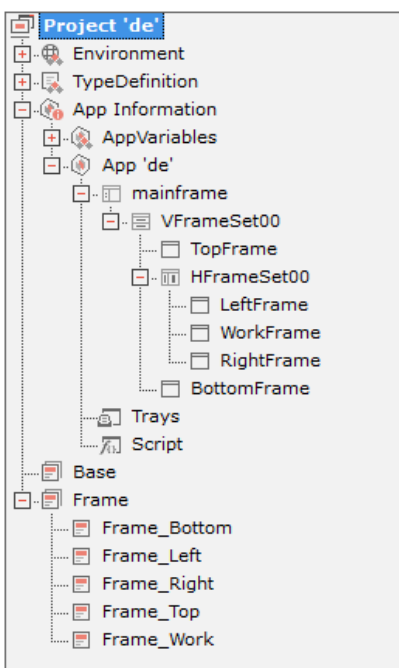


12.1.2 Frame 구성하기

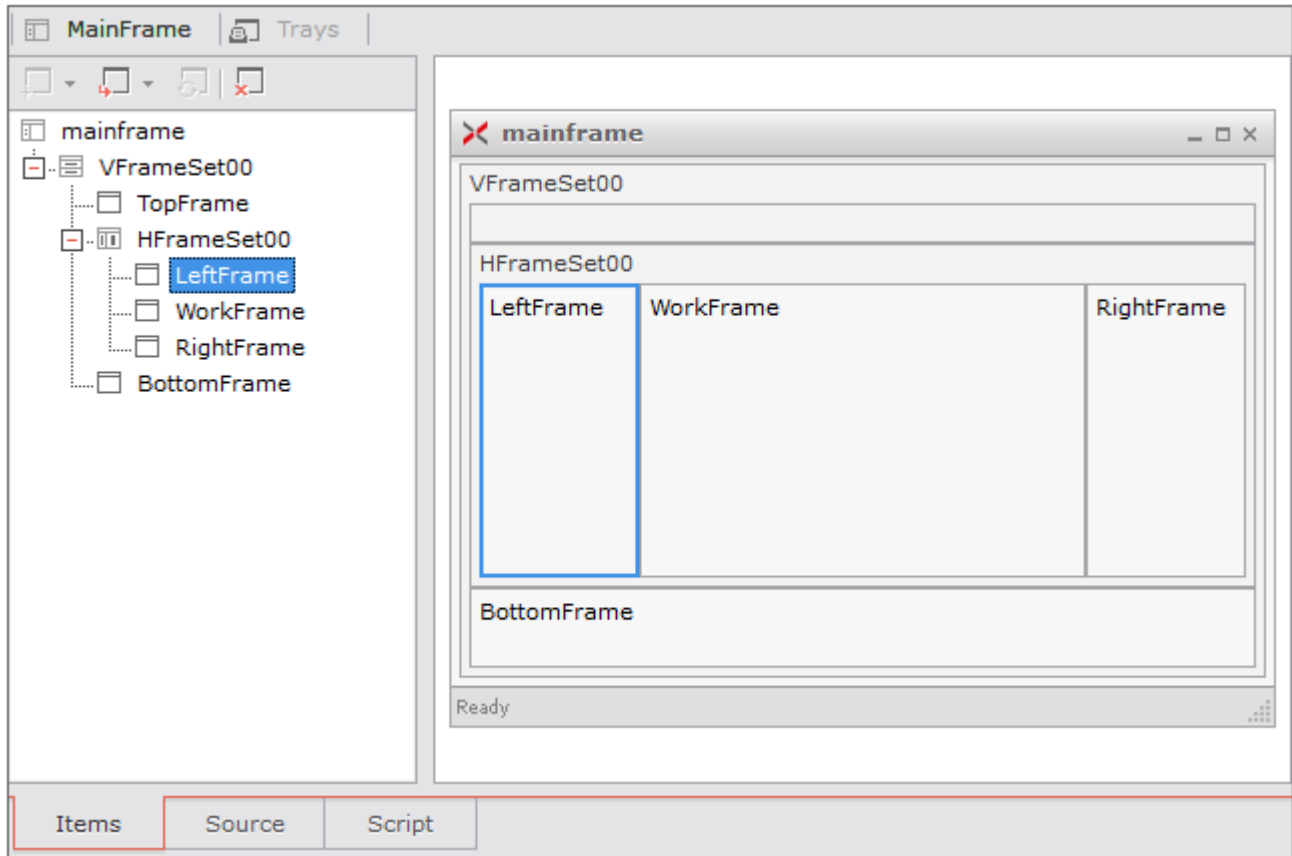
프로젝트 생성 시 Frame 템플릿을 지정합니다. 다른 항목을 선택하지 않고 넘어가는 경우(Frame Template은 "None", "Create Frame" 선택)에는 [MainFrame > ChildFrame] 형식으로 App을 구성합니다. 이때는 Form은 생성되지 않고 App만 생성합니다. Form은 직접 생성해 ChildFrame에 연결해주어야 합니다.



"None" 이외의 다른 Frame 템플릿을 선택하면 해당 Frame 구조와 함께 Form도 생성합니다. 아래 예제는 "MidThree" 템플릿을 선택했을 때 생성된 프로젝트 구조입니다.



프로젝트 생성 시 Frame 구조를 선택하지 않았더라도 프로젝트 생성 후 Frame 구조를 변경할 수 있습니다. 다양한 템플릿을 제공하지만 원하는 형태가 아닌 경우라면 "None"을 선택 후 Frame 편집창에서 원하는 형태로 수정할 수 있습니다.



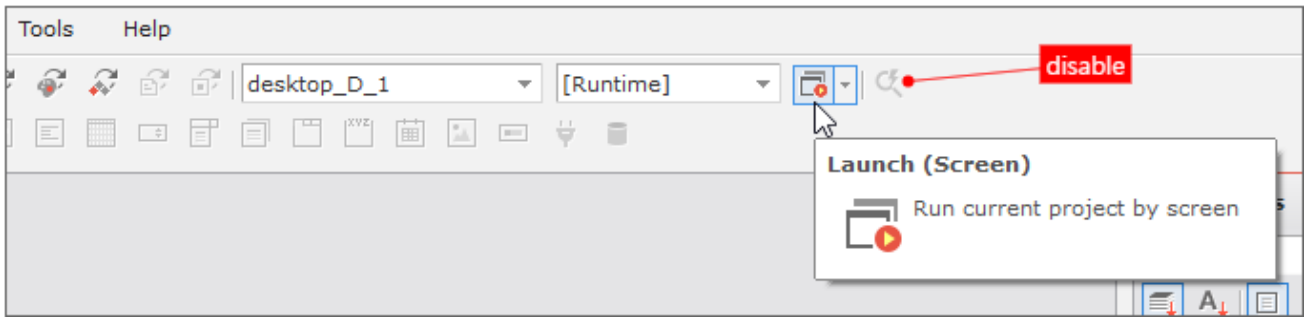
12.1.3 formurl

Frame 중에서 formurl 속성은 ChildFrame만 가지고 있습니다. 나머지 MainFrame, FrameSet은 ChildFrame과 연결된 Form을 어떻게 보여주고 ChildFrame을 어떻게 배치할 것인지에 대한 기능을 제공하는 것일 뿐 콘텐츠를 연결해 제공하지는 않습니다.

넥사크로 스튜디오에서 메뉴에서 [QuickView]로 Form을 실행한 경우에는 임의의 Frame 설정으로 화면을 보여줍니다. MainFrame 아래 ChildFrame이 있고 formurl로 Form을 연결한 상태로 나타납니다. 하지만 메뉴에서 [Launch]로 실행한 경우에는 설정된 Frame 구조에 따라 실행되고 ChildFrame의 formurl 속성값에 지정된 Form이 화면에 보입니다.



넥사크로 스튜디오의 편집창에 열린 Form이 없는 경우에는 [QuickView] 메뉴나 툴바가 활성화되지 않습니다.

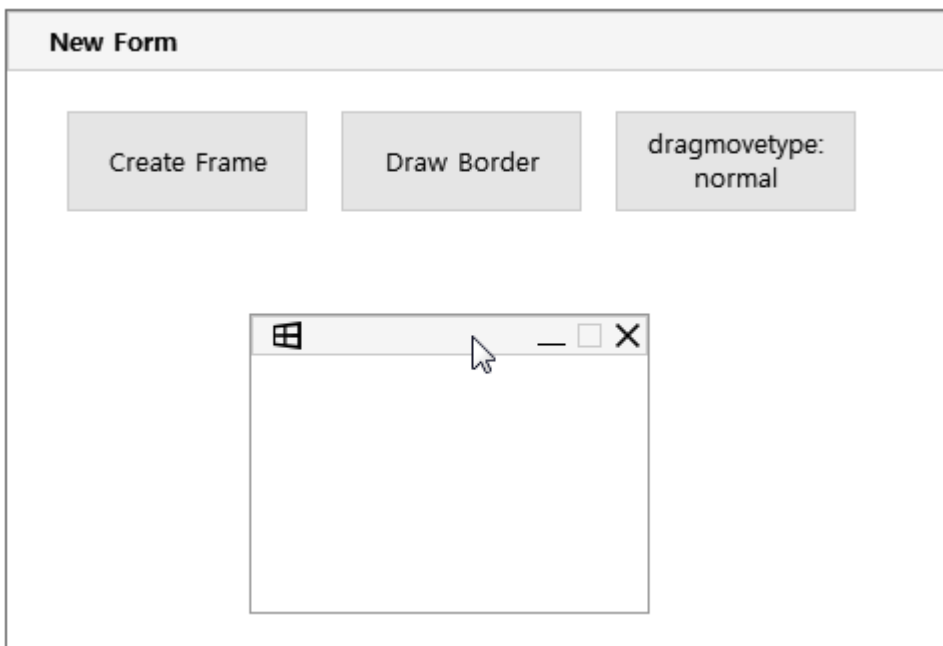


12.2 ChildFrame 생성하기

Form 안에 Frame을 생성하는 것은 원칙적으로 추천하지 않는 방법입니다. 다만 팝업창처럼 독립적으로 동작하는 기능을 처리하고자 하는 경우에 사용할 수 있습니다.

12.2.1 예제

ChildFrame을 생성하고 스타일 속성을 변경하거나 드래그 지원 여부를 변경합니다. [Create Frame] 버튼을 클릭하면 팝업창이 표시되고 [Draw Border] 버튼을 클릭하면 팝업창의 경계선이 보여집니다. 이 상태에서는 윈도우 타이틀 영역을 잡고 팝업창을 자유롭게 움직일 수 있는데 [dragmovetype] 버튼을 클릭하면 창이 dragmovetype 속성값이 변경되어 움직이지 않습니다. [dragmovetype] 버튼을 다시 클릭하면 팝업창을 다시 움직일 수 있습니다.





Default 테마에서 ChildFrame은 border 속성이 0px로 지정되어 있어서 경계선이 보이지 않습니다. 실제 운영 단계에서 빈 ChildFrame만 생성하고 사용하는 경우는 없으므로 border 속성값을 지정할 필요는 없습니다. 하지만 이번 예제에서는 ChildFrame 자체를 생성하고 관련 속성을 살펴보는 것이므로 border 속성값을 스크립트로 지정해봅니다.



ChildFrame을 생성하지 않고 관련 속성에 접근해 border, dragmovetype 속성값을 변경하려 하면 스크립트 에러가 발생합니다. 예제에서는 border 속성을 처리할 때는 ChildFrame 생성 여부를 확인해 스크립트 에러가 발생하지 않지만 dragmovetype 속성을 처리할 때는 생성 여부를 확인하지 않기 때문에 스크립트 에러가 발생합니다. 두 가지 차이를 비교해보세요.



넥사크로플랫폼에서는 같은 id 속성값을 가지는 자식을 여러 개 만들 수 없습니다. 그래서 이번 예제에서 ChildFrame을 두 개 이상 생성하려 하면 스크립트 에러가 발생합니다.



sample_frame_01.xfdl

12.2.2 예제에서 사용한 핵심 기능

dragmovetype

ChildFrame을 마우스 드래그로 이동시키는 기능 활성화 여부를 설정하는 속성입니다. default 값은 "normal"입니다. 타이틀바 영역을 선택해 마우스 드래그로 이동시킬 수 있습니다. 타이틀바 영역을 보이지 않도록 한 경우에는 Form 영역을 드래그할 수 있습니다. Form 영역은 텍스트 선택이나 항목 선택을 마우스 드래그로 처리할 수 있으므로 이벤트가 겹치지 않도록 default 속성에서는 타이틀바 여부에 따라 마우스 드래그 방식이 결정되도록 처리합니다.

replace

String 오브젝트의 메소드입니다. Button 컴포넌트의 text 속성처럼 String 형식을 가지는 속성이라면 해당 메소드를 사용할 수 있습니다. replace는 특정 문자열을 다른 문자열로 바꾸고 바뀐 값을 반환합니다. 주의할 점은 원래의 값 자체를 바꾸는 것이 아니라 바뀐 값을 반환한다는 겁니다. 해당 값을 바꾸고 싶다면 replace 메소드를 사용해 얻은 값으로 다시 지정해주어야 합니다.

12.2.3 예제 구현 방법

① 메인 화면 구성하기

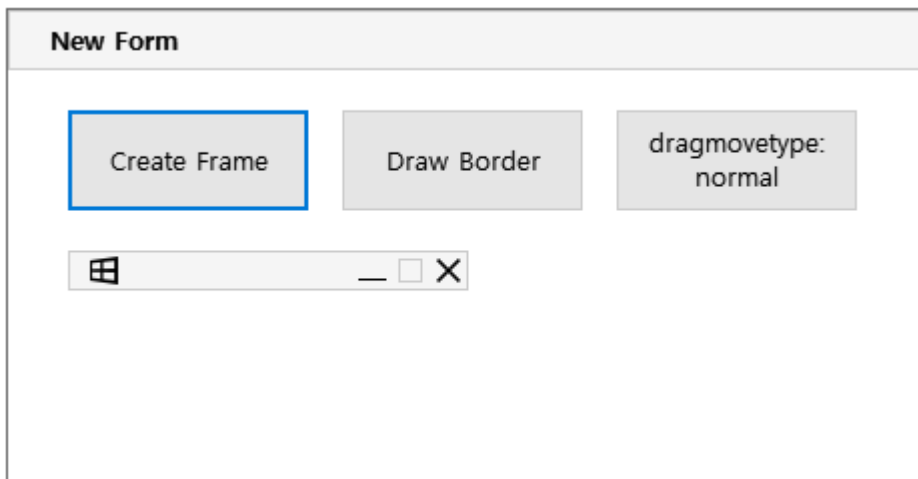
Button 컴포넌트를 예제 화면과 같이 배치합니다. ChildFrame은 넥사크로 스튜디오에서 생성하지 않고 버튼 클릭 시 스크립트에서 생성합니다.

2 ChildFrame 생성하기

첫 번째 버튼 클릭 시 ChildFrame을 생성합니다. ChildFrame을 생성할 위치와 크기만 지정합니다.

```
this.btnCreate_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var objChildFrame = new ChildFrame();
    objChildFrame.init("childframeNew", 30, 90, 200, 150, null, null);
    this.addChild("childframeNew", objChildFrame);
    objChildFrame.show();
};
```

ChildFrame의 border 속성값이 0px이기 때문에 타이틀바 영역만 보이고 Form 영역은 보이지 않는 것처럼 보입니다. 실제 Form 영역은 있지만, 배경색이 같으므로 없는 것처럼 보이는 것입니다.



3 ChildFrame border 속성 지정하기

생성된 ChildFrame의 border 속성을 지정합니다. ChildFrame 생성 시 지정한 id 속성값으로 접근합니다.

```
this.btnBorder_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    if(this.childframeNew) {
        this.childframeNew.set_border("1px solid #999999");
    } else {
        trace("childframeNew undefined");
    }
};
```

조건문을 사용해 ChildFrame이 생성됐는지 확인하고 그렇지 않다면 메시지를 표기합니다.

4 ChildFrame dragmovetype 속성 지정하기

생성된 ChildFrame의 dragmovetype 속성을 지정합니다. 속성값을 "none"으로 변경하면 마우스 드래그로 ChildFrame을 움직일 수 없습니다. 해당 속성값의 현재값을 비교해 "normal"이면 "none"으로 변경하고 "none"이면 "normal"로 변경해 토글 기능처럼 동작합니다.

```
this.btnDrag_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    if(this.childframeNew.dragmovetype == "normal") {
        this.childframeNew.set_dragmovetype("none");
        obj.set_text(obj.text.replace("normal","none"));
    } else {
        this.childframeNew.set_dragmovetype("normal");
        obj.set_text(obj.text.replace("none","normal"));
    }
};
```

현재 속성값을 Button 컴포넌트의 text 속성으로 표기하기 위해 replace 메소드를 사용합니다. 표기되는 값에서 dragmovetype 속성값에 해당하는 문자열만 조건에 따라 변경합니다.

5 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭하거나 마우스 드래그로 동작을 확인합니다. 상황에 따라 스크립트 에러가 발생할 수 있습니다. 스크립트 에러는 런타임으로 실행 시에는 넥사크로 스튜디오 [Output] 창에서 확인할 수 있고 웹브라우저에서는 콘솔창에서 확인할 수 있습니다.

12.3 FrameSet 생성하기

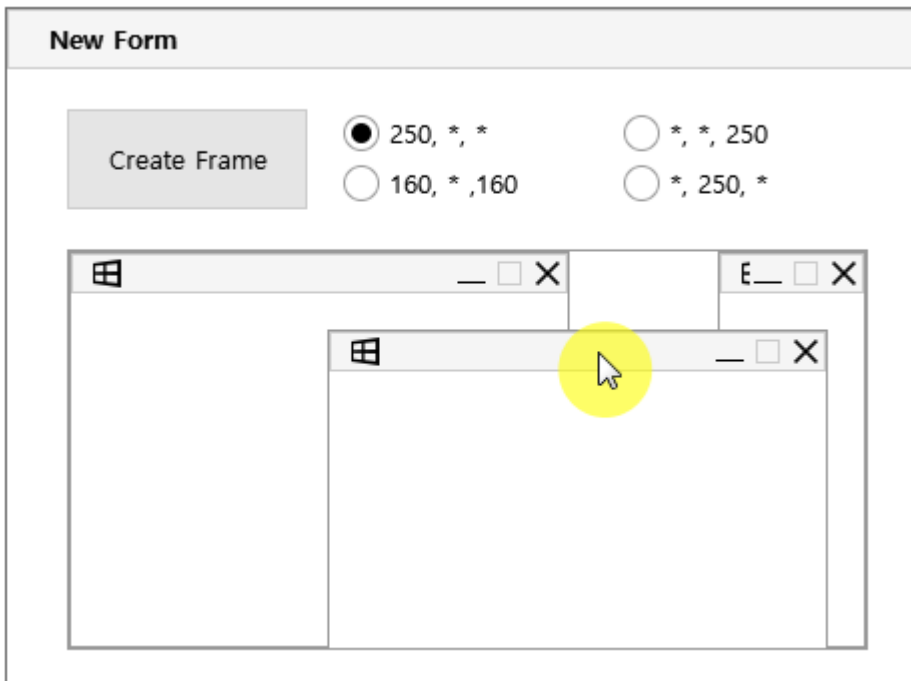
FrameSet은 화면 배열을 처리하기 위해 사용하는 오브젝트입니다. 화면을 배열하는 방식에 따라 HFrameSet(가로 방향 배열), VFrameSet(세로 방향 배열), TileFrameSet(타일 형태 배열)으로 세분화됩니다. HFrameSet을 선택한 경우에는 추가된 ChildFrame이 가로 방향으로 배치되고 HFrameSet에 맞게 ChildFrame의 크기가 자동으로 조정됩니다. 또한, 마우스 드래그로 배치 순서를 바꾸더라도 자동으로 맞는 위치에 정렬됩니다.

12.3.1 예제

버튼을 클릭하면 HFrameSet을 생성하고 3개의 ChildFrame을 생성해 HFrameSet에 추가합니다. ChildFrame의 크기는 자동으로 설정되는데 Radio 컴포넌트 항목을 선택하면 크기 비율이 변경됩니다.



애플리케이션 개발 시 동적으로 FrameSet을 생성하는 경우는 거의 없습니다. 이번 예제는 FraemSet의 속성을 보여주기 위한 예제일 뿐 권장하는 사양은 아닙니다.



sample_frame_02.xfdl

12.3.2 예제에서 사용한 핵심 기능

addChild

동적으로 자식 컴포넌트나 오브젝트를 추가하는 메소드입니다. Div 컴포넌트와 같은 컨테이너 컴포넌트나 Form은 컴포넌트나 오브젝트를 추가할 수 있고 FrameSet은 FrameSet 또는 ChildFrame을 추가할 수 있습니다.

separatesize

coma(,)로 구분해 FrameSet 내 포함된 ChildFrame의 너비를 지정합니다. 속성값을 지정하지 않았다면 같은 너비로 처리합니다.

12.3.3 예제 구현 방법

① 메인 화면 구성하기

Button 컴포넌트와 Radio 컴포넌트를 예제 화면과 같이 배치합니다. Radio 컴포넌트의 항목값은 `separatesize` 속성값으로 지정할 때 사용합니다. `innerdataset` 값은 아래와 같습니다.

InnerDataset		
No	codecolumn	datacolumn
1	250,*,*	250,*,*
2	160,*,160	160,*,160
3	*,*,250	*,*,250
4	*,250,*	*,250,*

② Frame 생성하기

Form 오브젝트의 `onload` 이벤트 함수 내에서 `HFrameSet`을 생성합니다.

```
this.sample_frame_02_onload = function(obj:nexacro.Form,e:nexacro.LoadEventInfo)
{
    var objHFrameset = new HFrameSet();
    objHFrameset.init("hframesetNew", 30, 90, 400, 200);
    this.addChild("hframesetNew", objHFrameset);
    objHFrameset.set_border("1px solid #999999");
    objHFrameset.show();
};
```

그리고 버튼 클릭 시 `ChildFrame` 생성, 추가합니다. 화면에 보일 수 있도록 `border` 속성값을 추가로 지정했습니다.

```
this.btnCreate_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.addChildFrame(this.hframesetNew, "childFrame00");
    this.addChildFrame(this.hframesetNew, "childFrame01");
    this.addChildFrame(this.hframesetNew, "childFrame02");
};
```

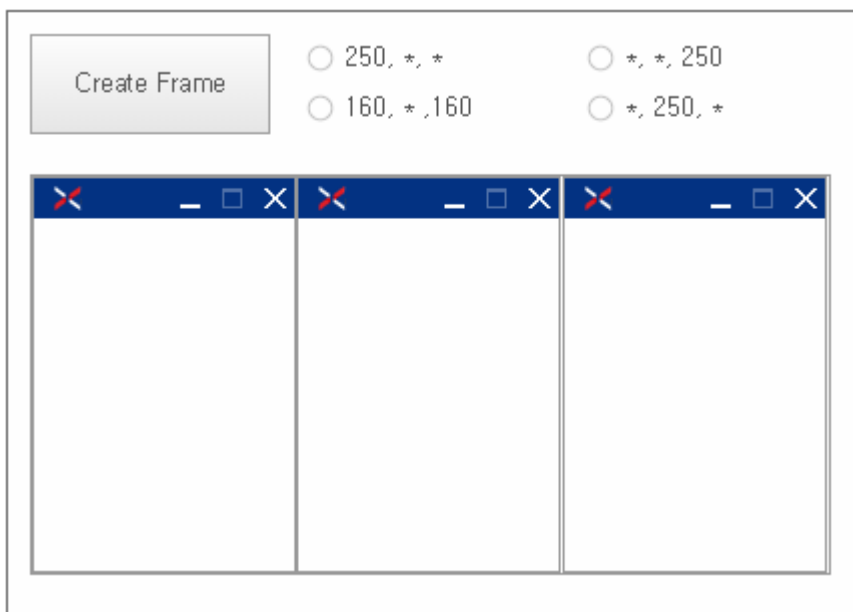
`ChildFrame`은 같은 형식을 3번 반복하기 때문에 `addChildFrame`이라는 함수를 만들어 사용했습니다. `ChildFrame`은 `Form`이 아닌 `HFrameSet`의 자식으로 추가합니다.

```

this.addChildFrame = function(obj, strId:String)
{
    var objChildFrame = new ChildFrame();
    objChildFrame.init(strId, 0, 0, 0, 0);
    obj.addChild(strId, objChildFrame);
    objChildFrame.set_border("1px solid #999999");
    objChildFrame.show();
}

```

버튼을 클릭해보면 아래와 같이 배치된 3개의 ChildFrame을 볼 수 있습니다.



3 separatesize 속성 지정하기

Radio 컴포넌트의 항목을 선택하면 배치된 ChildFrame의 너비가 변경됩니다. 숫자로 값을 지정한 경우에는 p x 단위로 너비가 수정되고 * 문자로 지정된 경우에는 숫자로 지정된 너비를 제외한 나머지를 계산해 처리합니다. 예를 들어 HFrameSet 너비가 400이고 separatesize 속성값을 "250, *, *"로 지정했다면 첫 번째 ChildFrame의 너비는 250이 되고 나머지 ChildFrame의 너비는 (400-250)/2가 됩니다. 실제 값을 계산할 때는 border 속성이나 다른 속성도 고려되어야 하므로 수치는 달라질 수 있습니다.

```

this.radioSeparate_onitemchanged = function(obj:nexacro.Radio,e:nexacro.
ItemChangeEventInfo)
{
    this.hframesetNew.set_separatesize(obj.value);
};

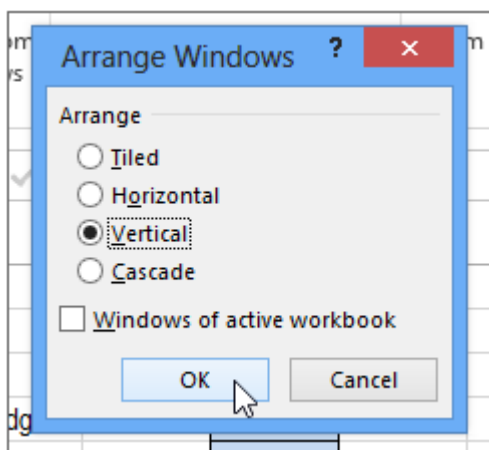
```

4 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭하거나 마우스 드래그로 동작을 확인합니다.

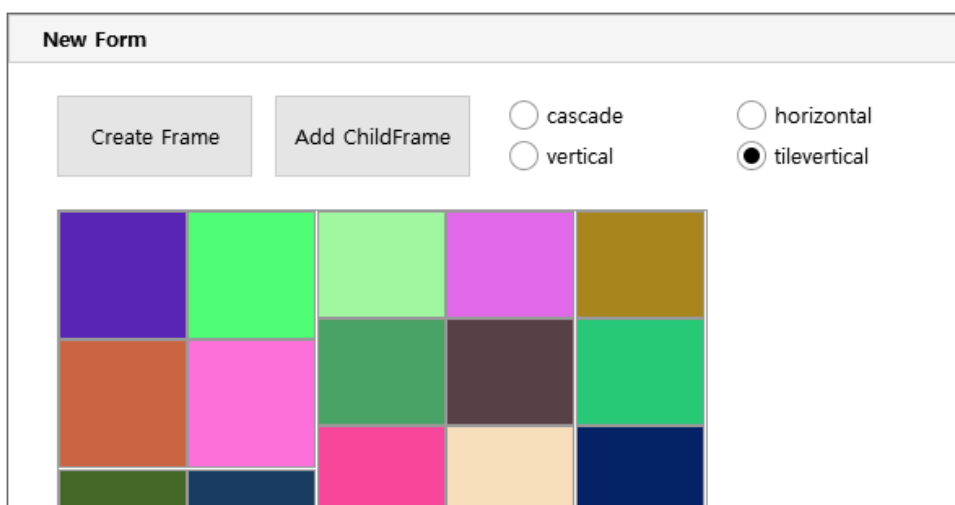
12.4 ChildFrame 정렬하기

문서 편집 소프트웨어는 아래 그림처럼 창을 여러 개 띄어놓은 상태에서 정렬하는 기능을 제공합니다. 가로나 세로, 계단식, 바둑판식으로 창을 정렬하는 기능을 제공합니다. 넥사크로플랫폼도 FrameSet에 ChildFrame을 추가한 경우 창 정렬방식을 지정할 수 있습니다.



12.4.1 예제

첫 번째 버튼을 클릭하면 FrameSet을 생성하고 두 번째 버튼을 클릭하면 ChildFrame을 생성하고 FrameSet에 추가합니다. FrameSet은 강제로 ChildFrame의 위치나 크기를 조정하지 않기 때문에 지정한 위치와 크기에 ChildFrame이 생성됩니다. Radio 컴포넌트의 항목을 설정하면 지정한 정렬 방식이 적용됩니다.





12.4.2 예제에서 사용한 핵심 기능

arrange

FrameSet을 사용하는 경우 정렬 방식을 지정합니다. 해당 메소드를 실행하는 순간 적용되며 다른 ChildFrame이 추가되는 경우에는 영향을 받지 않습니다. 필요하다면 한 번 더 메소드를 실행해야 합니다. HFrameSet처럼 정렬 방향이 지정된 경우에는 arrange 메소드를 사용할 수 없으며 FrameSet만 가지고 있는 메소드입니다.

showtitlebar

예제에서는 창 정렬 방식을 잘 보여주기 위해 타이틀바 영역을 보이지 않도록 처리했습니다.

12.4.3 예제 구현 방법

① 메인 화면 구성하기

Button 컴포넌트와 Radio 컴포넌트를 예제 화면과 같이 배치합니다. Radio 컴포넌트의 항목값은 separatesize 속성값으로 지정할 때 사용합니다. innerdataset 값은 아래와 같습니다.

InnerDataset			
No	codecolumn	datacolumn	
1	cascade	cascade	
2	vertical	vertical	
3	horizontal	horizontal	
4	tilevertical	tilevertical	

② Frame 생성하기

버튼 클릭 시 FrameSet을 생성합니다. 화면에 보일 수 있도록 border 속성값을 추가로 지정했습니다.

```
this.btnCreate_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var objFrameset = new FrameSet();
    objFrameset.init("framesetNew", 30, 90, 400, 400);
    this.addChild("framesetNew", objFrameset);
    objFrameset.set_border("1px solid #999999");
}
```

```
objFrameset.show();
};
```

3 ChildFrame 생성하기

버튼 클릭 시 ChildFrame을 생성합니다. 여러 개 ChildFrame을 생성할 수 있도록 클릭할 때마다 늘어나는 인덱스 변수를 정의하고 id 값과 위치값을 설정합니다. 타이틀바가 보이지 않도록 showtitlebar 속성값을 지정합니다. 또한 배경색도 매번 다른 색으로 지정합니다.

```
var frameIdx = 0;

this.btnAddChild_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var objChildFrame = new ChildFrame();
    objChildFrame.init("childframe"+frameIdx, frameIdx, frameIdx, 40, 40);
    this.framesetNew.addChild("childframe"+frameIdx, objChildFrame);
    objChildFrame.set_border("1px solid #999999");
    objChildFrame.set_showtitlebar(false);
    objChildFrame.set_background('#'+Math.random().toString(16).slice(-6));
    objChildFrame.show();
    frameIdx = frameIdx + 10;
};
```



배경색을 매번 다르게 지정하기 위해 Math.random 함수를 사용했습니다. 좀 더 정확하게 하려면 0에서 255 사이의 정수를 랜덤하게 생성한 후 16진수로 변환한 3개의 값을 합쳐 6자리 코드를 만들어야 하지만 예제에서는 편의상 간단하게 다른 색만 표현하는 코드를 사용했습니다.

4 arrange 처리하기

Radio 컴포넌트의 항목을 선택하면 배치된 ChildFrame의 정렬 방식이 변경됩니다.

```
this.radioArrange_onitemchanged = function(obj:nexacro.Radio,e:nexacro.ItemChangeEventInfo)
{
    this.framesetNew.arrange(obj.value);
};
```

5 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 FrameSet과 ChildFrame을 생성한 후 정렬 방식을 변경해봅니다.

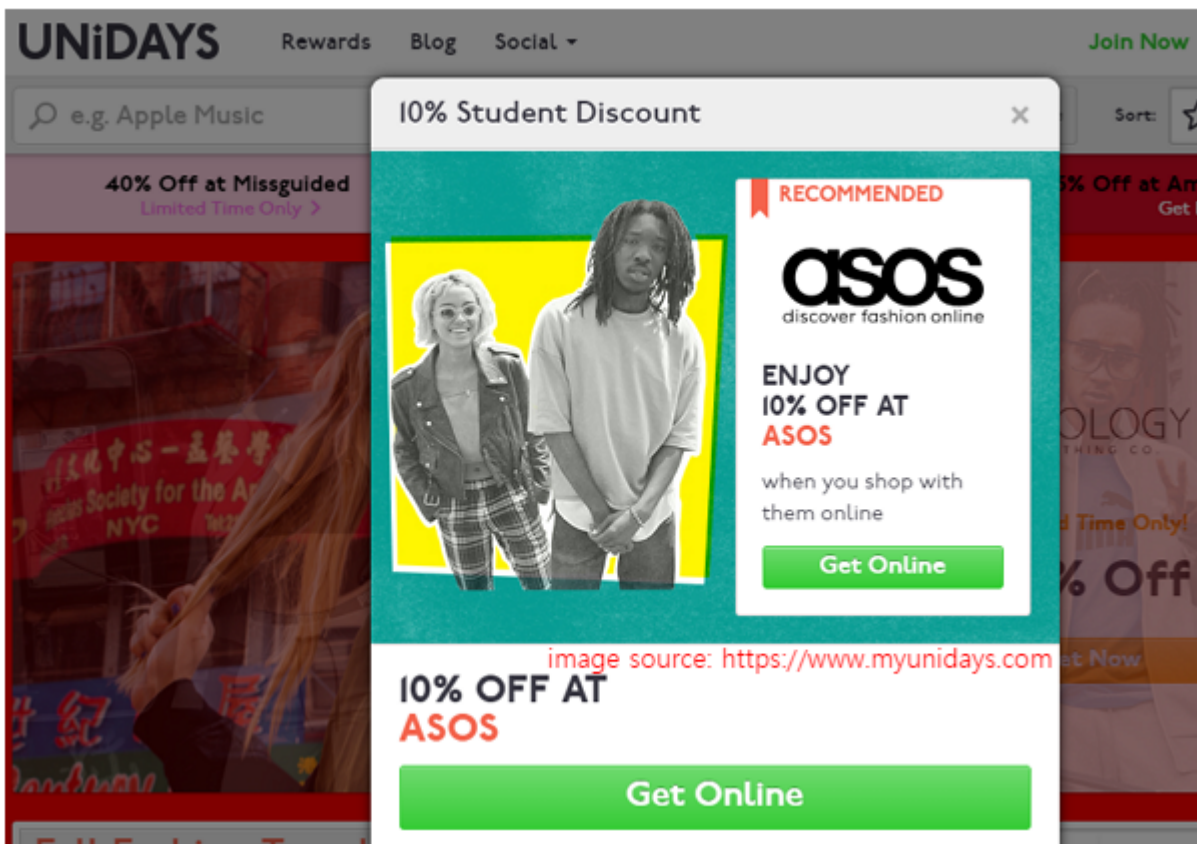


정렬 방식 중 "cascade" 정렬 방식은 프레임 크기에 따라 다르게 보일 수 있습니다.

12.5 모달창으로 ChildFrame 생성하기

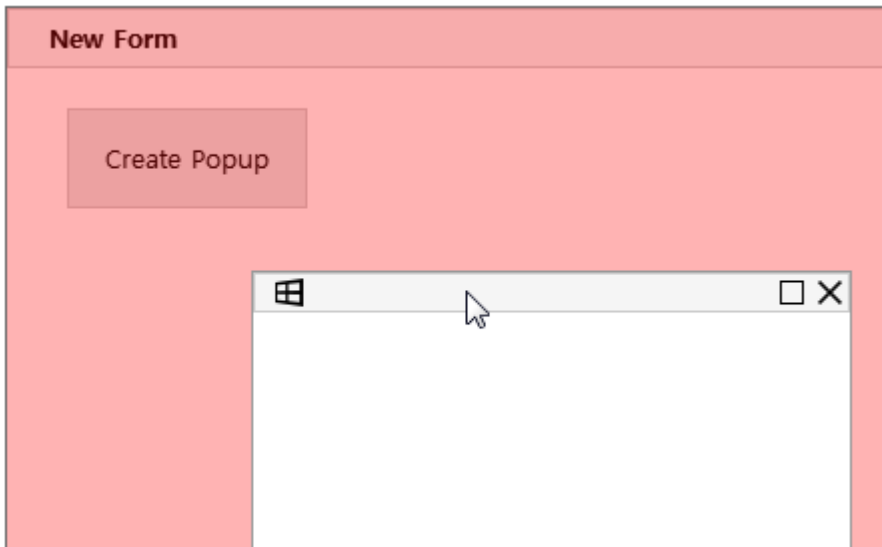
웹사이트에서 로그인 버튼을 클릭하면 로그인창이 팝업 형태로 보이는 경우가 있습니다. 이런 경우 열린 로그인창 외에 다른 영역은 뿌옇게 처리되는 경우를 본적이 있을 겁니다. 아니면 이메일을 등록하면 10% 할인 쿠폰을 준다는 광고창도 마찬가지입니다.

모달창은 팝업창과 비슷하지만 간단한 정보를 보여주거나 입력받은 정보를 처리하는 용도로 사용합니다. 해당 작업에 집중하기 위해 다른 영역은 뿌옇게 비활성화 상태처럼 보여줍니다.



12.5.1 예제

버튼을 클릭하면 ChildFrame을 생성하고 모달창으로 화면 가운데 띄웁니다.



12.5.2 예제에서 사용한 핵심 기능

openalign

ChildFrame 초기화 단계에서 위치를 지정하지만 `openalign` 속성을 지정하면 원하는 위치에 정렬해 표시할 수 있습니다. `showModal` 메소드와 같이 사용할 때만 적용되는 속성입니다.

resizable

ChildFrame의 창 크기를 조절할 수 있도록 허용할 것인지 여부를 설정합니다. default 값은 `false`로 크기를 조절할 수 없습니다. 해당 속성값을 `true`로 설정하면 창 경계선 영역에 마우스를 가져가면 크기를 조절할 수 있는 상태로 마우스 커서가 변경됩니다. 창에 상태바를 보이도록 허용한 경우에는 창 왼쪽 하단에 크기를 조정할 수 있는 컨트롤 영역이 추가됩니다.



overlaycolor

팝업창으로 ChildFrame을 띄울 때 나머지 영역 배경색을 지정하는 속성입니다. 색상값만 지정할 수도 있고 투명도를 같이 지정할 수 있습니다. 투명도를 지정하려면 `RGBA()` 형식으로 값을 지정합니다.



RGBA는 CSS3부터 지원되는 기능입니다. IE 브라우저를 사용한다면 9 이상 버전을 사용해야 합니다.

<https://www.w3.org/wiki/CSS/Properties/color/RGBA>

<https://www.w3.org/TR/css3-color/#rgba-color>

12.5.3 예제 구현 방법

① 메인 화면 구성하기

Button 컴포넌트를 예제 화면과 같이 배치합니다.

② 버튼 클릭 이벤트 처리하기

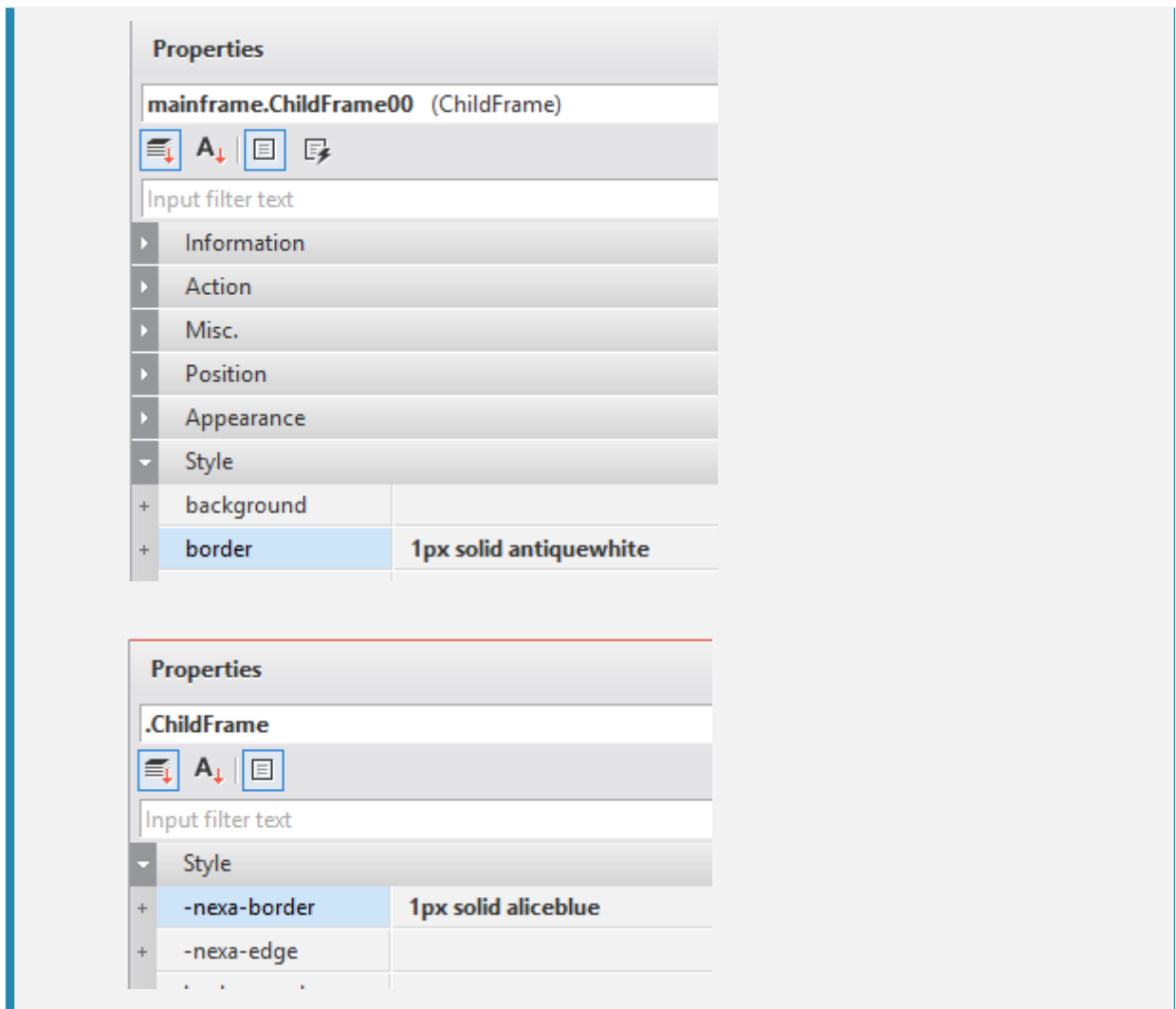
버튼 클릭 시 ChildFrame을 생성합니다. 화면에 보일 수 있도록 border 속성값을 추가로 지정하고 상태바가 보이도록 showstatusbar 속성값을 지정했습니다. 모달창으로 화면에 보일 때 가운데 위치할 수 있도록 open align 속성값을 지정하고 모달창 주변의 배경색을 수정했습니다.

```
this.btnPopup_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var objChildFrame = new ChildFrame();
    objChildFrame.init("childframe00", 0, 0, 300, 300, null, null);
    objChildFrame.set_border("1px solid #999999");
    objChildFrame.set_showstatusbar(true);
    objChildFrame.set_openalign('center middle');
    objChildFrame.set_resizable(true);
    objChildFrame.set_overlaycolor("rgba(255,0,0,0.3)");
    objChildFrame.set_background("#ffffff");
    objChildFrame.showModal(this.getOwnerFrame());
};
```



ChildFrame의 setter 관련 정보는 도움말에서 ChildFrame 속성 항목에서 찾아볼 수 있습니다. 하지만 border 속성은 보이지 않는데요. 도움말에서 속성 이름이 border가 아니라 -nexa-border로 되어 있습니다. border 같은 경우에는 스크립트에서 사용하는 이름과 XCSS에서 지정하는 이름이 다르기 때문에 속성명 표시는 XCSS를 기준으로 표시하고 있습니다.

속성창에서도 디자인 모드에서는 border로 속성이 표시되고 XCSS 편집시에는 -nexa-border로 속성이 표시됩니다.



3 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 모달창으로 ChildFrame을 생성해봅니다.

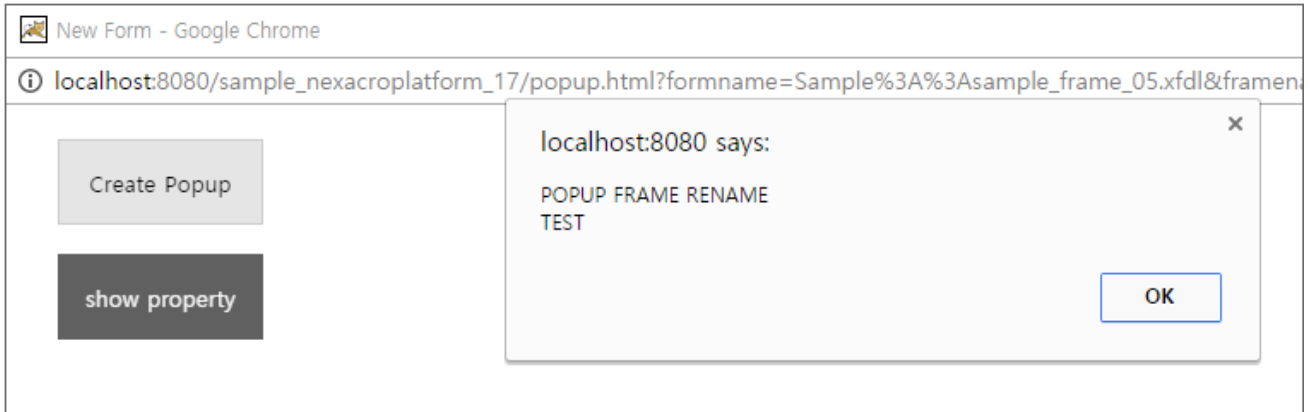
12.6 open 메소드로 속성값 전달하기

Div 컴포넌트 같은 컨테이너 컴포넌트는 팝업창으로 나타나지만 같은 브라우저 내에서 동작하기 때문에 서로 접근할 수 있습니다. 하지만 open 메소드를 사용하는 경우에는 브라우저 자체를 새 창으로 처리합니다. 이런 경우에는 서로 접근할 수 있는 방법이 없습니다. 그래서 open 메소드 실행 시 필요한 속성값을 전달할 수 있도록 준비되어 있습니다.

open 메소드로 전달된 속성값은 실행되는 Form의 상위 프레임인 ChildFrame의 속성으로 전달됩니다. 새로운 속성값을 사용자 속성처럼 전달할 수 있으며 기존 속성값을 덮어 쓸 수 있습니다.

12.6.1 예제

"Create Popup" 버튼을 클릭하면 새 창이 열립니다. 새 창에서 "Show property" 버튼을 클릭하면 전달된 속성값을 확인할 수 있습니다.



12.6.2 예제에서 사용한 핵심 기능

open

open 메소드는 동적으로 ChildFrame을 생성하고 modeless 창으로 표시합니다. 파라미터 중 objArgument List 항목을 지정해 원하는 속성값을 전달할 수 있습니다. 속성값은 "{속성명:속성값, 속성명:속성값}" 형식으로 전달합니다.



속성값으로 전달할 수 있는 것은 문자열 뿐 아니라 Dataset이나 오브젝트 형태도 전달할 수 있습니다.

12.6.3 예제 구현 방법

① 메인 화면 구성하기

Button 컴포넌트를 예제 화면과 같이 배치합니다.

② 버튼 클릭 이벤트 처리하기

"Create Popup" 버튼 클릭 시 open 메소드를 실행합니다. 따로 Form을 생성하지 않고 같은 Form을 재활용합니다. 4번째 파라미터로 속성값 목록을 전달합니다. 전달되는 속성값 중 name 속성은 ChildFrame 오브젝

트가 원래 가지고 있는 속성값을 덮어쓰게 되고 uname 속성은 새로운 속성을 만들고 속성값을 설정합니다.

```
this.btnPopup_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    nexacro.open("POPUP FRAME", "Sample::sample_frame_05.xfd1",
        nexacro.getApplication().getActiveFrame(),
        {name:"POPUP FRAME RENAME", uname:"TEST"});
};
```

"Show property" 버튼 클릭 시 전달된 속성값을 제대로 처리하는지 확인합니다. 속성값은 Form의 parent인 ChildFrame의 속성값이기 때문에 "this.parent.속성값" 형식으로 접근합니다.

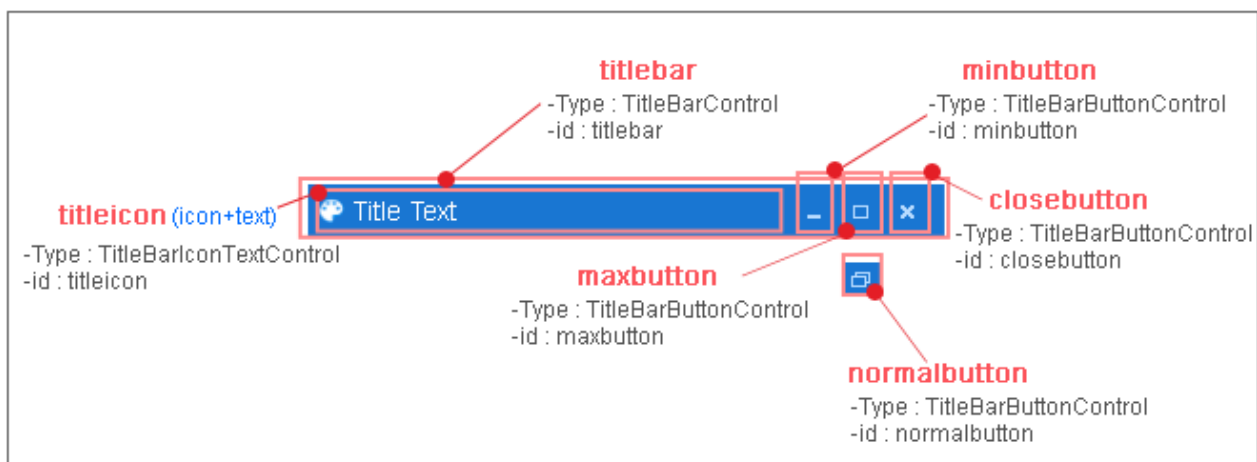
```
this.btnShow_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.alert(this.parent.name + "\n" + this.parent.uname);
};
```

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 모달창으로 ChildFrame을 생성하고 전달된 속성값을 확인합니다.

12.7 타이틀바 영역 변경하기

타이틀바 영역은 별도 컨트롤로 구성되어 있으며 각 컨트롤 별로 접근할 수 있습니다. 도움말에서는 TitleBarControl 항목을 찾으면 해당 구조를 확인할 수 있습니다.

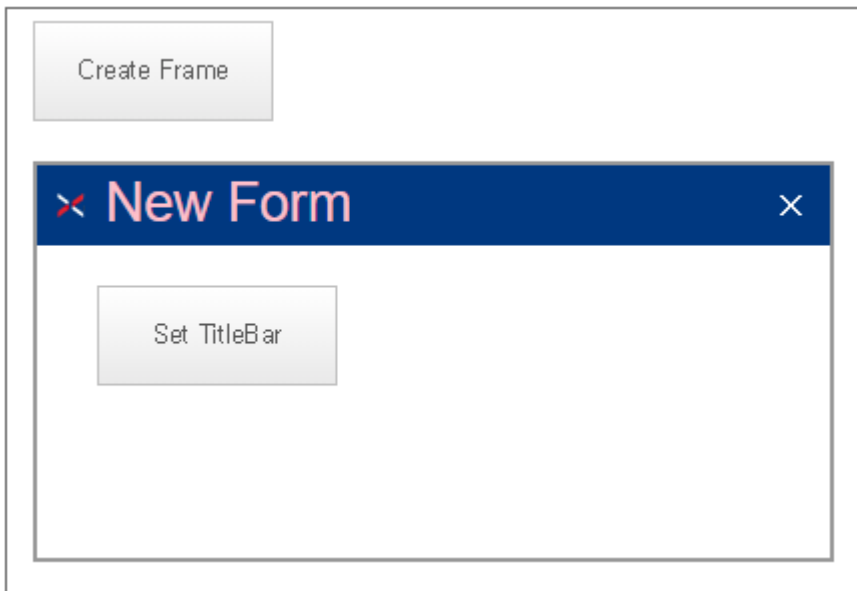




타이틀바 영역은 애플리케이션 내에서 보여지는 영역을 의미합니다. 웹브라우저의 타이틀바 영역과는 다릅니다.

12.7.1 예제

"Create Frame" 버튼을 클릭하면 Button 컴포넌트 아래에 프레임이 생성되고 생성된 창에서 "Set TitleBar" 버튼을 클릭하면 타이틀바 영역의 일부 속성을 변경합니다.



sample_frame_08.xfdl

12.7.2 예제에서 사용한 핵심 기능

titlebar

프레임의 타이틀바 오브젝트 속성입니다. TitleBarControl 오브젝트에서 하위 속성을 확인할 수 있습니다.

titleicon

타이틀바 영역에서 텍스트와 아이콘이 표시되는 영역입니다. 아이콘 이미지와 텍스트 속성을 변경할 수 있습니다. TitleBarIconTextControl 오브젝트에서 하위 속성을 확인할 수 있습니다.

maxbutton, minbutton

최소화, 최대화 버튼입니다. 예제에서는 해당 버튼을 보이지 않도록 설정했습니다. TitleBarButtonControl 오브젝트에서 하위 속성을 확인할 수 있습니다.

12.7.3 예제 구현 방법

- ① Button 컴포넌트를 예제 화면과 같이 배치합니다.
- ② Button 컴포넌트의 onclick 이벤트 함수를 아래와 같이 작성합니다.

HFrameSet을 배치하고 그 안에 ChildFrame을 배치합니다. ChildFrame의 formurl 속성은 같은 Form으로 지정합니다.

```
this.btnCreate_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var objHFrameSet = new HFrameSet();
    objHFrameSet.init("hframesetNew", 30, 90, 400, 200);
    this.addChild("hframesetNew", objHFrameSet);
    objHFrameSet.set_border("1px solid #999999");
    objHFrameSet.show();

    var objChildFrame = new ChildFrame();
    objChildFrame.init("TEST", 0, 0, 0, 0);
    objHFrameSet.addChild("TEST", objChildFrame);
    objChildFrame.set_border("1px solid #999999");
    objChildFrame.set_formurl("Sample::sample_frame_08.xfdl");
    objChildFrame.show();
};
```

버튼을 클릭하면 같은 Form이 화면에 표시됩니다. 하지만, 버튼을 클릭했을 때 다른 동작을 처리하도록 코드를 수정합니다. parent.id 속성값이 ChildFrame의 id 속성값인 "TEST"인 경우에는 타이틀바 영역을 변경하는 동작을 처리하도록 합니다.

```
this.btnCreate_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    if(this.parent.id=="TEST")
    {
        this.parent.titlebar.titleicon.set_font("20pt 'Arial'");
        this.parent.titlebar.titleicon.set_color("pink");
        this.parent.titlebar.minbutton.set_visible(false);
        this.parent.titlebar.maxbutton.set_visible(false);
        this.parent.set_titlebarheight(40);
    }
    else
```

```

    {
        ...
    }
};

```

- ③ 버튼에 표시되는 텍스트도 변경합니다. onload 이벤트 함수를 아래와 같이 작성합니다.

```

this.sample_frame_08 onload = function(obj:nexacro.Form,e:nexacro.LoadEventInfo)
{
    if(this.parent.id=="TEST")
    {
        this.btnCreate.set_text('Set TitleBar');
    }
};

```

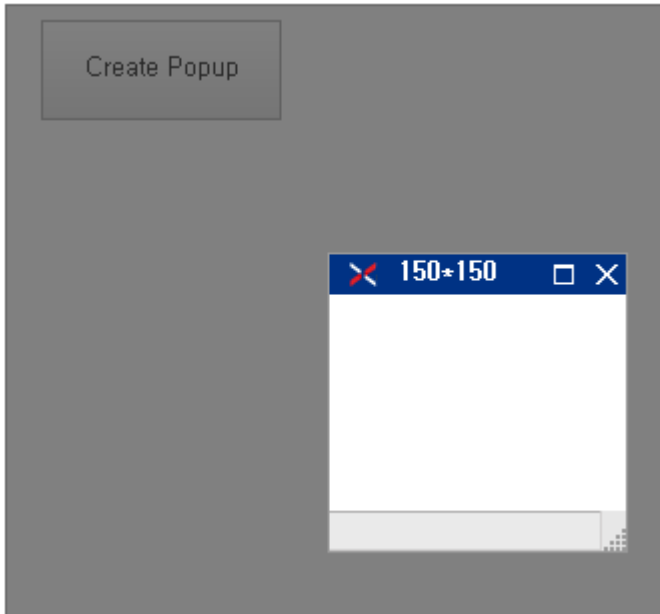
- ④ QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 ChildFrame을 배치하고 ChildFrame 내에 표시된 Form에서 버튼 클릭 시 타이틀바 영역이 변경되는지 확인합니다.

12.8 팝업창 크기 제한하기

팝업창으로 띄운 창 크기를 사용자가 변경할 수 있지만, 지정된 범위는 벗어나지 않게 합니다.

12.8.1 예제

"Create Popup" 버튼을 클릭하면 Button 컴포넌트 아래에 프레임이 생성되고 생성된 팝업창 크기 변경 시 특정 크기보다 작거나 크게 만들 수 없습니다.



12.8.2 예제에서 사용한 핵심 기능

cx, cy

SizeEventInfo 오브젝트에서 반환하는 값으로 컴포넌트의 크기가 변경될 때 변경된 너비, 높이 값을 반환합니다.

12.8.3 예제 구현 방법

- ① Button 컴포넌트를 예제 화면과 같이 배치합니다.
- ② Button 컴포넌트의 onclick 이벤트 함수를 아래와 같이 작성합니다.

ChildFrame 오브젝트를 배치하고 addEventHandler 메소드를 사용해 onsize 이벤트 발생 시 크기를 제한하는 스크립트가 동작하도록 합니다. showModal 메소드 실행 시 최소, 최대 크기값을 같이 전달합니다.

```
this.btnPopup onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var objChildFrame = new ChildFrame();
    objChildFrame.init("childframe00", 0, 0, 300, 300, null, null);
    objChildFrame.set_border("1px solid #999999");
    objChildFrame.set_showstatusbar(true);
```

```

objChildFrame.set_openalign('center middle');
objChildFrame.set_resizable(true);
objChildFrame.set_background("#ffffff");
objChildFrame.set_titletext(objChildFrame.width+"*"+objChildFrame.height);
objChildFrame.addEventHandler( "onsize", this.ChildFrame00_onsize, this);
objChildFrame.showModal(this.getOwnerFrame(), {minsize:'150', maxsize:'400'});
};

```

사용자가 팝업창 크기를 변경할 때마다 onsize 이벤트가 발생하며 현재 팝업창의 크기와 변경하려는 크기를 비교해서 범위를 벗어나는 경우 최소 또는 최대 크기값으로 변경합니다.

```

this.ChildFrame00_onsize = function(obj:nexacro.ChildFrame, e:nexacro.SizeEventInfo)
{
    obj.set_titletext(obj.width+"*"+obj.height);
    if(e.cx < obj.minsize) {
        obj.set_width(obj.minsize);
    }
    if(e.cx > obj.maxsize) {
        obj.set_width(obj.maxsize);
    }

    if(e.cy < obj.minsize){
        obj.set_height(obj.minsize);
    }
    if(e.cy > obj.maxsize){
        obj.set_height(obj.maxsize);
    }
}
}

```

- ③ QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 ChildFrame을 배치하고 팝업창의 크기를 변경해봅니다.

파트 III.

Component

13.

Button

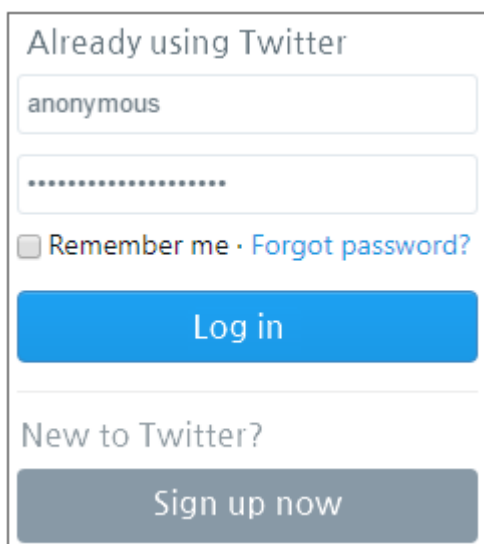
13.1 Button 소개

버튼은 사용자가 마우스로 버튼을 클릭하여 사전에 설정해 놓은 동작을 수행하기 위한 컴포넌트입니다. 사용자는 버튼을 클릭했을 때 어떤 동작이 발생할 것을 직관적으로 알 수 있습니다. 동작은 스크립트로 작성하며 버튼의 이벤트 함수를 통해 수행됩니다.

13.1.1 사용 예

다음은 대표적인 SNS(Social Network Service)중 하나인 트위터의 로그인 화면입니다. 사용자 계정 정보를 입력하는 편집창 밑으로 로그인 버튼과 가입하기 버튼이 있습니다.

로그인 버튼을 누르면 편집창에 입력한 계정으로 사용자 인증을 받게 되고, 가입하기 버튼을 클릭하면 사용자 정보를 입력할 수 있는 메뉴가 나타납니다.

A screenshot of the Twitter login interface. At the top, it says "Already using Twitter". Below this is a text input field containing the word "anonymous". Underneath the input field is a password field represented by a series of dots. Below the password field is a checkbox labeled "Remember me" followed by a link "Forgot password?". A large blue button labeled "Log in" is positioned below the checkbox. At the bottom of the form, it says "New to Twitter?" followed by a grey button labeled "Sign up now".

Already using Twitter

☐ Remember me · [Forgot password?](#)

Log in

New to Twitter?

Sign up now

13.1.2 Button 만들기

Button00

① Button 컴포넌트 생성하기

툴바에서 Button  을 선택한 후, Form 위에 적당한 크기로 드래그하여 생성합니다. 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

② QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

버튼은 사용자의 마우스 움직임에 따라 상태가 변경됩니다. 그러나 이벤트 핸들러를 작성하지 않았기 때문에 어떤 동작도 수행하지 않습니다.

13.2 버튼에 아이콘 넣기

버튼을 꾸미기 위해 이미지를 아이콘 형태로 버튼에 삽입할 수 있습니다.

13.2.1 예제

다음은 이미지를 삽입한 버튼 예제입니다. 아이콘 이미지가 위치하고 그 옆에 버튼 text 영역이 존재합니다. 버튼을 누르면 버튼과 이벤트 정보를 팝업으로 출력합니다.

 OK

 [sample_button_02.xfdl](#)

13.2.2 예제에서 사용한 핵심 기능

icon

버튼에서 텍스트와 함께 표시할 아이콘 이미지의 경로를 설정하는 속성입니다.

13.2.3 예제 구현 방법

① Button 컴포넌트 생성하기

툴바에서 Button  컴포넌트를 선택한 후 예제 화면과 같이 Form에 배치합니다.

② 아이콘 설정하기

버튼을 선택한 후 Properties 창에서 icon 속성을 설정합니다.

icon	imagerc::img_sta_des.png
iconPosition	imagerc::icon_reload_7a7a7a_32.png
letterSpacing	imagerc::icon_reload_black_32.png
opacity	imagerc::icon_reload_red_32.png
padding	imagerc::img_imgviewer_D.png
padding	imagerc::img_imgviewer_N.png
textAlign	imagerc::img_sta_des.png
textDecoration	imagerc::launch_1.png



ImageResource를 추가하는 방법은 아래 링크를 참고하세요.

[이미지 폴더에 있는 이미지 보여주기](#)

③ OK 버튼 기능 구현하기

OK 버튼의 id를 btn_ok로 설정하고 onclick 이벤트 핸들러를 추가합니다.

- OK 버튼의 onclick 이벤트 핸들러

이벤트 종류와 버튼 이름을 팝업으로 출력합니다.

```
this.btn_ok onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var strMsg = "[" + e.eventid + " event] " + this.btn_ok.text + " button clicked.";
```

```
alert(strMsg);
};
```

④ QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. OK 버튼에 아이콘 이미지가 출력되고 버튼을 누르면 이벤트와 버튼의 정보를 팝업으로 출력합니다.

13.3 이미지 버튼 만들기

Button 컴포넌트를 꾸미기 위해 배경색이나 경계선 등의 속성을 사용합니다. 이런 스타일 속성만으로 부족한 부분이 있는 경우에는 이미지 자체를 버튼처럼 사용합니다. 넥사크로플랫폼에서는 ImageView에 원하는 이미지를 지정하고 클릭 이벤트와 필요한 속성을 지정해 이미지 버튼을 구현할 수도 있고 Button 컴포넌트의 배경이미지를 지정해 이미지 버튼을 구현할 수 있습니다. 이번 예제에서는 배경이미지를 지정해 이미지 버튼을 구현하는 방법을 살펴봅니다.



ImageView 컴포넌트를 사용해 이미지 버튼을 만들 수도 있지만 Button 컴포넌트에 공통으로 적용되는 테마나 스타일이 적용되지 못하는 단점이 있습니다. 다른 버튼과 같은 속성을 적용하려면 Button 컴포넌트를 사용하거나 cssclass 속성값을 적용할 수 있습니다.

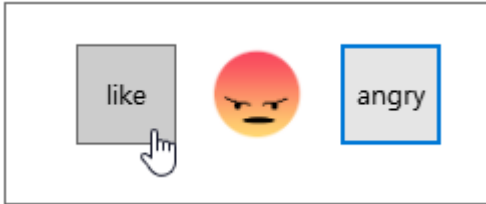


샘플 이미지에 사용한 아이콘은 아래 주소에서 참조했습니다.

<https://webdesignceylon.blogspot.kr/2016/03/facebook-new-like-icon-set-2016-png-psd.html>

13.3.1 예제

3개의 버튼을 배치합니다. 첫 번째, 세 번째 버튼은 일반 Button 컴포넌트이고 가운데 버튼은 background 속성을 적용했습니다. 첫 번째, 세 번째 버튼 클릭 시 가운데 이미지 버튼에 설정된 이미지가 변경됩니다.



sample_button_03.xfdl

13.3.2 예제에서 사용한 핵심 기능

background

배경색 또는 배경 이미지를 지정합니다. 배경색을 지정할 때도 그라데이션을 적용할지 단일 색으로 지정할지에 따라 하위 속성이 달라집니다. 예제에서는 배경 이미지 파일을 지정하고 다른 속성은 기본값을 적용했습니다. 넥사크로 스튜디오에서는 원하는 파일을 탐색기에서 선택할 수 있는데 스크립트에서는 아래와 같이 속성값을 지정합니다.

```
this.btnImage.set_background('url("imagerc::like50.png")');
```

13.3.3 예제 구현 방법

① 화면 구성하기

Button 컴포넌트를 예제 화면과 같이 배치합니다. 아직은 background 속성값을 지정하기 전이라 모든 버튼이 같은 모습으로 보입니다. 버튼의 크기는 width, height 값을 50으로 지정합니다.

② background 속성 지정하기

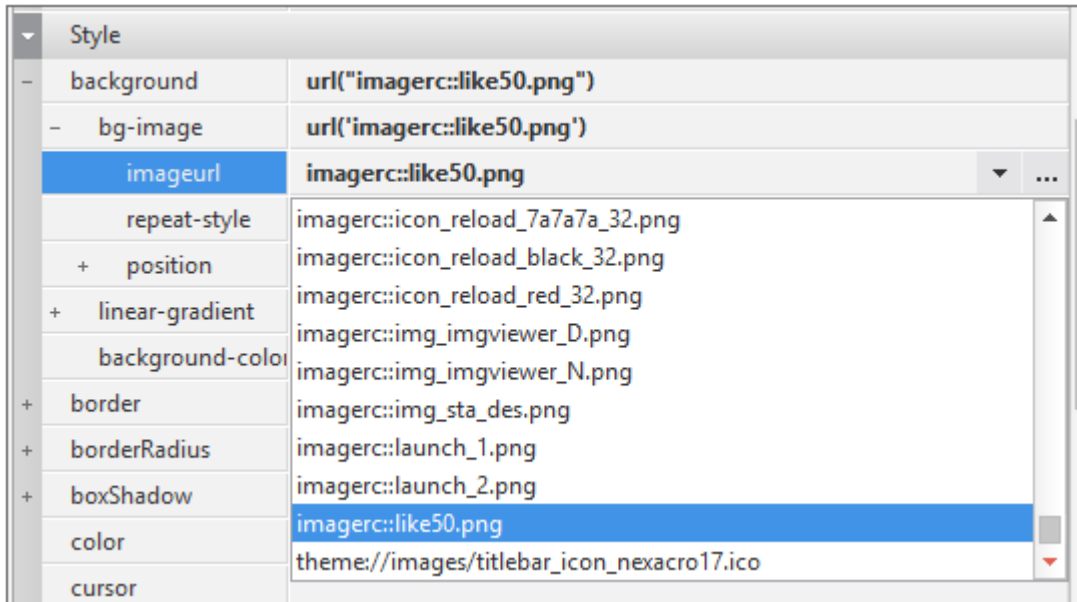
가운데 버튼을 선택하고 background 속성값을 지정합니다. 필요한 이미지 파일은 ImageResource로 추가합니다.



ImageResource를 추가하는 방법은 아래 링크를 참고하세요.

[이미지 폴더에 있는 이미지 보여주기](#)

background 속성은 하위 속성을 지정할 수 있습니다. background 속성명 앞에 있는 "+" 버튼을 클릭하면 하위 속성이 펼쳐집니다. 그중에서 imageUrl 속성값을 지정합니다. 파일 탐색기를 열어 원하는 이미지 파일을 선택합니다.



3 border, text 속성 수정하기

default 테마를 사용한다면 Button 컴포넌트는 border 속성값이 지정되어 있습니다. 이미지 버튼으로 지정하는 경우에는 경계선이 보이지 않도록 해야 하므로 해당 속성값을 "0px none"으로 지정합니다. 그리고 text 속성값 역시 빈 값으로 지정합니다.

4 Button 컴포넌트 이벤트 함수 작성하기

Button 컴포넌트를 선택한 후 onclick 이벤트 함수를 작성합니다. 각 버튼에 지정된 이미지 파일을 background 속성값으로 지정합니다.

```
this.btnLike_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.btnImage.set_background('url("imagerc::like50.png")');
};

this.btnAngry_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.btnImage.set_background('url("imagerc::angry50.png")');
};
```

5 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 이미지를 바꾸어가면서 확인해봅니다.

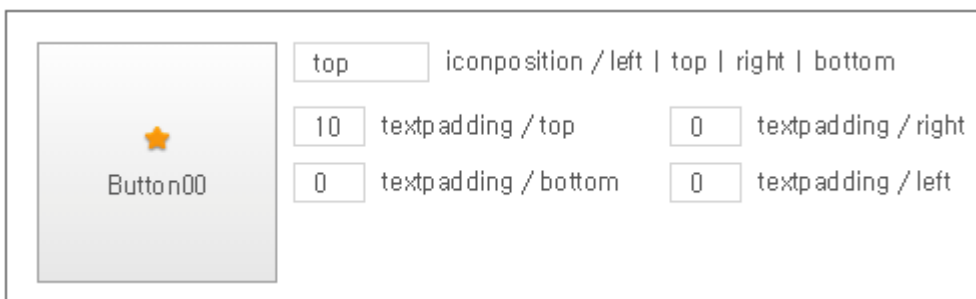
13.4 버튼에 들어간 아이콘 위치 변경하기

Button 컴포넌트에 아이콘을 추가하는 것은 버튼의 용도를 좀 더 명확하게 해줍니다. 일반적으로 아이콘이 먼저 위치하고 그 다음에 텍스트가 보이는데 필요에 따라 아이콘의 위치를 조정할 수 있습니다. 이런 경우 어떻게 아이콘 위치를 변경하는지 살펴봅니다.



13.4.1 예제

정사각형 형태의 버튼을 배치하고 iconposition, textpadding 속성값을 수정하면서 어떻게 적용되는지 살펴봅니다. 값을 수정한 후에 버튼을 클릭하면 아이콘 위치가 변경됩니다.



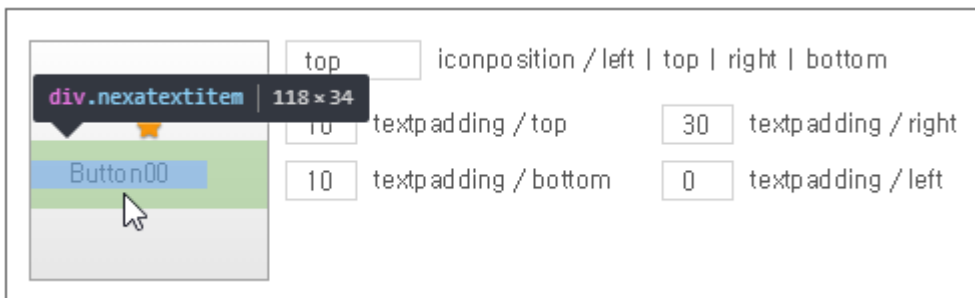
13.4.2 예제에서 사용한 핵심 기능

iconPosition (-nexa-icon-position)

설정된 아이콘 이미지를 어느 방향에 배치할지를 지정합니다. 지정된 값이 없는 경우에는 default 속성값이 적용됩니다. 아이콘은 하나만 지정할 수 있기 때문에 양쪽에 아이콘을 배치하고자 하는 경우에는 background 속성을 사용해 이미지로 처리할 수 있습니다.

textPadding (-nexa-text-padding)

Button 컴포넌트 내 text 요소를 둘러싼 여백을 지정합니다. 아래 그림에서 top, right, bottom 방향의 textpadding 속성값을 지정했을때 초록색으로 표시된 부분이 지정된 여백 영역입니다. 해당 영역이 변경되면서 text 요소의 위치가 변경됩니다.



13.4.3 예제 구현 방법

① 화면 구성하기

Button 컴포넌트와 Edit, Static 컴포넌트를 예제 화면과 같이 배치합니다. Button 컴포넌트의 아이콘의 위치를 조정하고 이를 확인하기 위해 Button 컴포넌트는 정사각형 형태로 width, height 값을 수정합니다.

② icon 속성 지정하기

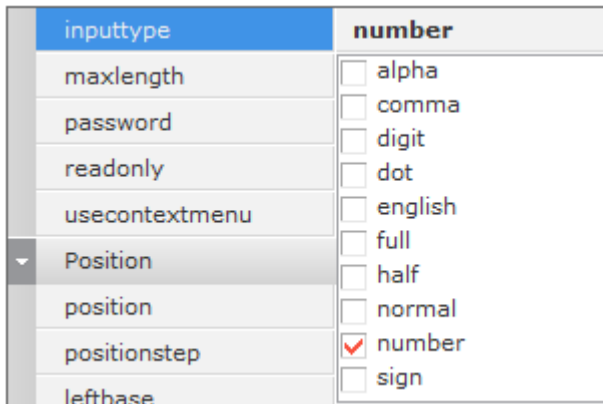
icon 속성값은 적절한 이미지 파일을 지정해줍니다. 이번 예제에서는 [버튼에 아이콘 넣기](#) 예제에서 사용한 이미지를 재사용합니다.

+	font	
	icon	imagerc::img_sta_des.png
	iconPosition	

③ textpadding 입력창 속성 지정하기

textpadding 속성값을 입력하는 Edit 컴포넌트는 숫자만 입력할 수 있어야 합니다. 이런 경우에는 Spin 컴포넌트를 사용해도 되지만 이번 예제에서는 Edit 컴포넌트를 사용합니다. 먼저 Edit 컴포넌트의 value 속성값을

0으로 지정하고 inputtype 속성값을 "number"로 지정합니다. 넥사크로 스튜디오에서 inputtype 속성값은 목록에서 원하는 항목을 1개 이상 선택할 수 있습니다.



inputtype 속성값을 지정하면 숫자 외 다른 값을 입력할 수 없도록 제한할 수 있습니다. 하지만 값을 지우는 것은 막을 수 없습니다. 사용자가 값을 지우는 경우에는 다른 값을 대입하도록 onchanged 이벤트를 추가합니다. 이벤트 함수 내에서 이벤트를 처리하는 컴포넌트를 직접 지정하지 않고 이벤트 함수 파라미터로 전달되는 obj라는 변수를 받아서 처리합니다. 예를 들어 textpadding top 속성값을 지정하는 Edit 컴포넌트에서 onchanged 이벤트가 발생했다면 obj 변수는 해당 Edit 컴포넌트를 가리킵니다.

이벤트 함수 내에서는 Edit 컴포넌트의 value 속성값이 숫자인지 확인하고 숫자가 아니라면 0이라는 값을 지정합니다.

```
this.editTextPadding_onchanged = function(obj:nexacro.Edit,e:nexacro.ChangeEventInfo)
{
    if(!nexacro.isNumeric(obj.value))
    {
        obj.set_value(0);
    }
};
```

4 Button 컴포넌트 이벤트 함수 작성하기

이번 예제에서는 따로 명령을 실행하는 버튼을 만들지 않고 아이콘 위치를 변경할 버튼을 실행 버튼으로 사용합니다. 버튼 클릭 시 아래 함수가 실행되며 입력된 값을 적용합니다.

```
this.btnIcon_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    obj.set_iconPosition(this.editIconPosition.value);
    obj.set_textPadding(this.editTextPaddingTop.value+"px "
        +this.editTextPaddingRight.value+"px "
        +this.editTextPaddingBottom.value+"px ");
}
```



```
+this.editTextPaddingLeft.value+"px");
};
```



textpadding 속성값을 지정할때 "px"라는 단위를 같이 지정하지 않으면 속성값이 적용되지 않습니다. 예제에서는 사용자가 입력한 값에 "px"라는 문자열을 추가해 속성값으로 지정합니다.

5 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 값을 입력하고 버튼을 클릭해 아이콘과 텍스트의 위치를 확인합니다.

13.5 컴포넌트의 속성, 메소드, 이벤트 확인하기

Button 컴포넌트 뿐 아니라 다른 컴포넌트나 오브젝트에 속한 속성, 메소드, 이벤트 정보를 확인하고자 한다면 for in 반복문을 활용할 수 있습니다.

13.5.1 예제

버튼을 클릭하면 TextArea 컴포넌트에 Button 컴포넌트와 ClickEventInfo 오브젝트와 관련된 정보를 표시합니다.

Button00

```
name: Button00
id: Button00
parent: [object Form]
left: 30
top: 20
width: 100
height: 40
right: null
bottom: null
text: Button00
onclick: [object EventListener]
constructor: function()
tabstop: true
```

```
eventid: onclick
id: onclick
fromobject: [object Button]
fromreferenceobject: [object Button]
altkey: false
ctrlkey: false
button: lbutton
shiftkey: false
```



13.5.2 예제에서 사용한 핵심 기능

for in

특정 오브젝트가 가지고 있는 모든 속성을 반복문으로 확인할 수 있습니다. for 문을 그대로 사용하는 것보다 속도는 느릴 수 있기 때문에 특정한 용도에만 사용하는 것을 권장합니다.

13.5.3 예제 구현 방법

① 화면 구성하기

Button 컴포넌트와 TextArea 컴포넌트를 예제 화면과 같이 배치합니다.

② onclick 이벤트 함수 작성하기

Button 클릭 시 전달되는 Button 컴포넌트(obj)와 ClickEventInfo 오브젝트(e)에 속한 모든 속성을 확인하고 TextArea 컴포넌트의 value 속성값으로 표시합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.TextArea00.set_value(this.checkList(obj));
    this.TextArea01.set_value(this.checkList(e));
};
```

checkList 함수는 오브젝트를 받아서 for in 반복문을 처리합니다. for in 반복문에서는 보이지 않는 모든 속성을 다 표시하기 때문에 "_"으로 시작하는 속성은 제외합니다. 그리고 "function"이 포함된 속성값은 해당 function의 코드 모두를 노출하기 때문에 속성값 대신 "function()"이라는 문자열을 대신 표기합니다.

```
this.checkList = function(obj)
{
    var strReturn = "";
    for(var x in obj)
    {
        if(x.substring(0,1)!="_")
        {
            if(obj[x]!=null && obj[x].toString().indexOf("function")>-1) {
```

```

        strReturn += x+": "+function()+"\n";
    }    else    {
        strReturn += x+": "+obj[x)+"\n";
    }
}
}
return strReturn;
}

```

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 값을 입력하고 버튼을 클릭해 아이콘과 텍스트의 위치를 확인합니다.

13.6 키보드 입력 시 발생하는 onclick 이벤트 구분하기

Button 컴포넌트에 포커스가 있는 경우 ENTER 키 또는 SPACE 키 입력 시 onclick 이벤트가 발생합니다. 마우스 클릭 시에만 이벤트를 처리하고 싶다면 ClickEventInfo 오브젝트의 button 속성값으로 확인할 수 있습니다.

13.6.1 예제

ENTER, SPACE 키 입력 시에는 "SPACE OR ENTER KEY"라고 표시하고 마우스 버튼 클릭 시에는 "CLICK"라고 표시합니다.



 sample_button_06.xfdl

13.6.2 예제에서 사용한 핵심 기능

button

마우스 버튼 또는 터치 동작이 아닌 경우에는 "none" 값을 반환합니다.

13.6.3 예제 구현 방법

① Button 컴포넌트와 Static 컴포넌트를 예제 화면과 같이 배치합니다.

② 아래와 같이 Button 컴포넌트의 onclick 이벤트를 함수를 작성합니다.

마우스 버튼 클릭이 아닌 경우에는 ClickEventInfo 오브젝트의 button 속성값이 "none" 값을 반환합니다.

```
this.Button00.onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Static00.set_text('SPACE OR ENTER KEY');
    if(e.button!='none')
    {
        this.Static00.set_text('CLICK');
    }
};
```

③ QuickView(Ctrl + F6)로 실행한 후 SPACE 키나 ENTER 키 입력 시 onclick 이벤트가 어떻게 처리되는지 확인합니다.

13.7 특정 키 입력 시 onclick 이벤트 처리하기

구글 웹사이트에서는 검색창에 키워드를 입력 후 Button으로 포커스를 옮기지 않고 바로 ENTER 키를 입력하면 조회가 처리됩니다. 이와 같이 특정 키 입력 시 버튼을 클릭한 것처럼 처리하는 방법을 살펴봅니다.

13.7.1 예제

Edit 컴포넌트에 포커스가 있는 상태에서 **ENTER**, **ESC**, **CTRL+H** 키 입력 시 onclick 이벤트를 처리합니다.



13.7.2 예제에서 사용한 핵심 기능

defaultbutton

true로 설정하면 **ENTER** 키 입력 시 onclick 이벤트를 발생합니다. TextArea 컴포넌트처럼 줄바꿈을 사용하는 컴포넌트를 사용하는 경우에는 ENTER 키 입력 시 줄바꿈 동작보다 Button 컴포넌트의 onclick 이벤트가 먼저 발생합니다.

escapebutton

true로 설정하면 **ESC** 키 입력 시 onclick 이벤트를 발생합니다.

hotkey

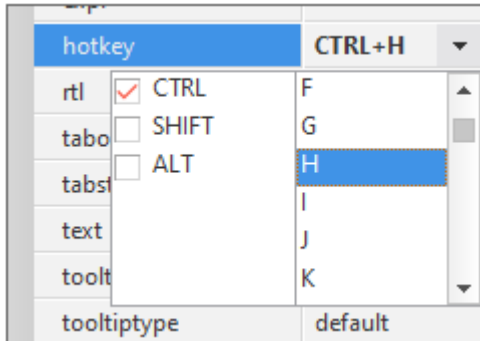
원하는 키 조합을 선택해 해당 키 입력 시 onclick 이벤트를 발생합니다. **CTRL**, **ALT**, **SHIFT** 와 다른 키를 조합해서 사용할 수 있습니다.

13.7.3 예제 구현 방법

- ① Edit 컴포넌트와 Button 컴포넌트를 예제 화면과 같이 배치합니다.
- ② 아래와 같이 Button 컴포넌트의 onclick 이벤트 함수를 작성합니다.
onclick 이벤트가 발생하면 Edit 컴포넌트에 문자열을 채워줍니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Edit00.set_value('KEY PRESS CHECK');
};
```

- ③ Button 컴포넌트의 defaultbutton, escapebutton 속성값을 true로 설정합니다.
- ④ 속성창에서 Button 컴포넌트의 hotkey 속성값을 아래와같이 설정합니다.



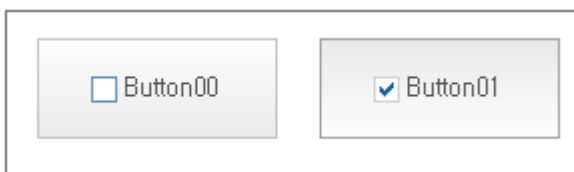
- ⑤ QuickView(Ctrl + F6)로 실행한 후 Button 컴포넌트에 포커스가 가지 않은 상태에서 **ENTER** 키나 **ESC** 키, **CTR** + **H** 입력 시 onclick 이벤트가 어떻게 처리되는지 확인합니다.

13.8 간단한 토글 버튼 만들기

물리적인 스위치처럼 버튼 클릭 시 눌러진 상태를 표시할 수 있습니다.

13.8.1 예제

Button 컴포넌트 클릭 시 상태값을 변경해서 다른 스타일이 적용됩니다.



 [sample_button_08.xfdl](#)

13.8.2 예제에서 사용한 핵심 기능

getSelectStatus, setSelectStatus

getSelectStatus 메소드는 Button 컴포넌트의 Status가 Seleted인지 확인할 수 있고, setSelectStatus 메소드를 사용하면 Button 컴포넌트의 Seleted Status를 설정할 수 있습니다.

toggleSelectStatus

Button 컴포넌트의 Status가 Seleted인 경우에는 상태를 해제하고 Selected가 아닌 경우에는 Selected Status로 설정합니다.

13.8.3 예제 구현 방법

- 1 Button 컴포넌트를 예제 화면과 같이 배치합니다.

- 2 아래와 같이 첫 번째 Button 컴포넌트의 onclick 이벤트 함수를 작성합니다.

getSelectStatus 메소드로 상태를 확인하고 반환받은 값에 NOT 연산 처리한 값을 setSelectStatus 메소드에 전달합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    obj.setSelectStatus(!obj.getSelectStatus());
};
```

- 3 아래와 같이 두 번째 Button 컴포넌트의 onclick 이벤트 함수를 작성합니다.

toggleSelectStatus 메소드는 첫 번째 Button 컴포넌트의 onclick 이벤트 함수에 작성한 내용을 알아서 처리하는 메소드입니다. 비교를 위해 두 가지 스크립트를 작성했습니다.

```
this.Button01 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    obj.toggleSelectStatus();
};
```

- 4 xcss 파일을 열어서 아래와 같이 코드를 추가합니다.

기본 테마에서도 버튼 배경색을 살짝 다르게 표현하고 있는데, 거기에 추가해서 Seleted Status와 아닌 경우 아이콘 이미지를 다르게 표시합니다.

```
.Button.sample_button_08
{
    -nexa-icon : url('theme://images/chk_WF_Box_0.png');
    -nexa-text-padding : 3px;
}

.Button.sample_button_08[userstatus=selected]
{
    -nexa-icon : url('theme://images/chk_WF_Box_F.png');
    -nexa-text-padding : 3px;
}
```

- ⑤ QuickView(Ctrl + F6)로 실행한 후 Button 컴포넌트 클릭 시 버튼의 상태가 어떻게 바뀌는지 확인합니다.

14.

Calendar

14.1 Calendar 소개

일상생활 속에서도 날짜를 입력해야 할 일이 많습니다. 예를 들어 회원 가입 시에 생년월일을 입력하기도 하고 극장에서 영화를 예매할 때 관람 일자를 선택하기도 합니다. 다른 항목과 다르게 날짜는 입력하는 형식이 정해져 있습니다. Calendar 컴포넌트는 사용자가 좀 더 편하고 빠르게 원하는 항목을 입력하고 확인할 수 있는 기능을 제공합니다.



HTML5 Input 요소에 추가된 type 속성값 중에서 'date'를 사용하면 달력과 같은 기능을 사용할 수 있습니다. 날짜를 선택하는 기능은 동작하지만 웹브라우저에 따라 사용자에게 보이는 방식이 다릅니다. 때문에 넥사크로플랫폼에서 사용하는 Calendar 컴포넌트는 HTML5 Input 요소와 별개로 독자적으로 구현된 컴포넌트를 사용합니다.

14.1.1 사용 예

Calendar 컴포넌트는 상황에 따라 입력하는 방식이 달라집니다.

- 날짜를 바로 확인하고 선택하는 경우
비행기 왕복 티켓을 예매하는 경우에는 현재일자를 기준으로 출발일자와 도착일자를 한눈에 확인하고 선택할 수 있어야 합니다. 달력이 펼쳐진 상태에서 사용자가 바로 날짜를 선택할 수 있습니다.

Choose Your Dates

Depart Date

< August 2016 >

S	M	T	W	T	F	S
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31			

Return Date

< September 2016 >

S	M	T	W	T	F	S
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	

Select

image source: koreanair.com

• Navigate the dates using the arrow keys and use the Enter key to select the date.

- 조회 조건에서 날짜를 선택하는 경우

조회 조건은 사용자가 원하는 데이터를 필터링하기 위한 도구입니다. 조회 조건을 입력할 때만 팝업창으로 나타났다가 선택하면 사라지는 형식을 사용합니다.

Visit Statistics

2016-08-20 ~ 2016-08-26

2016-08-20 ~ 2016-08-26

2016/08

SUN	MON	TUE	WED	THU	FRI	SAT
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31			

image source: 3rabbitz.com

- 특정 날짜를 빠르게 선택해야 하는 경우

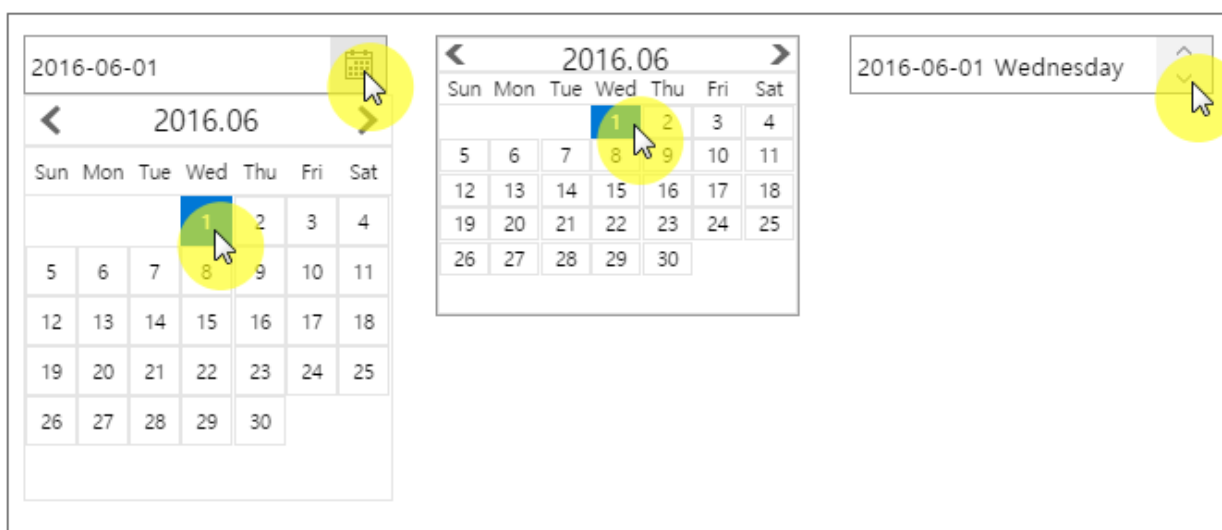
달력 형식에서 날짜를 선택하는 것은 원하는 날짜를 직관적으로 찾을 수 있는 장점은 있지만, 과거의 특정 날짜를 선택하기에는 불편함이 있습니다. 이런 경우에는 직접 날짜값을 입력하거나 콤보 형식으로 제공되는 목록에서 선택하는 것이 더 빠를 수 있습니다.

14.1.2 Calendar 만들기

1 Calendar 컴포넌트 생성하기

툴바에서 Calendar를 선택한 후, Form 위에 드래그 앤 드롭해 생성합니다. type 속성값에 따라 화면에 보이는 형식이 달라지기 때문에 Calendar 컴포넌트를 3개 생성하고 각각 type 속성값을 다르게 지정합니다. Calendar 컴포넌트에서 지원하는 type 속성값은 아래와 같습니다.

- normal (default)
- monthonly
- spin



type 속성값이 "normal"인 경우에는 달력 모양 아이콘을 클릭하면 달력이 펼쳐집니다. 원하는 날짜를 클릭해서 선택하면 달력은 사라집니다. type 속성값이 "monthonly"인 경우에는 달력이 펼쳐진 상태입니다. 원하는 날짜를 선택하면 선택된 값을 처리하는 스크립트를 별도 작성해야 합니다. type 속성값이 "spin"인 경우에는 달력 모양 아이콘 대신 날짜값을 이동할 수 있는 스피ن 버튼이 나타납니다.

type 속성값을 'monthonly'로 지정한 경우에는 달력이 펼쳐진 형태로 표시됩니다. 컴포넌트의 크기가 자동으로

로 변경되지는 않기 때문에 속성값 수정 후 컴포넌트의 크기를 조정해주어야 합니다.

2 QuickView로 확인하기

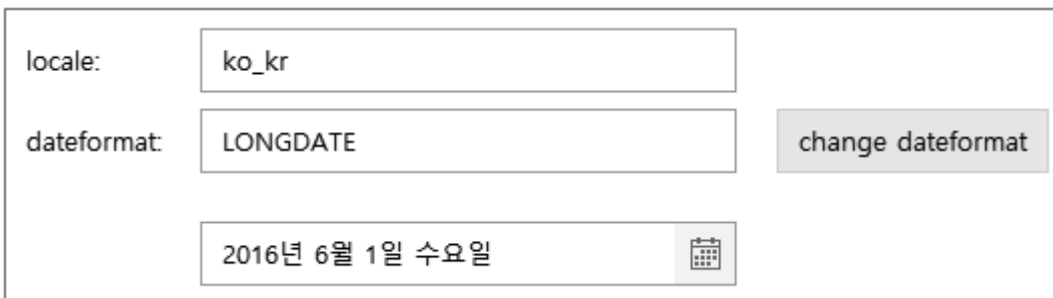
QuickView(Ctrl + F6)로 실행합니다. Calendar 생성 후 type 속성을 별도로 설정하지 않으면 'normal'로 설정되어 동작합니다.

14.2 날짜 형식 바꾸기

나라마다 날짜를 표기하는 형식이 다릅니다. 나라별 형식은 보통 locale 속성값을 수정하면 자동으로 처리됩니다. 보통 가장 많이 사용하는 형식을 기본값으로 설정하는데 해당 형식 외에 다른 형식을 원하면 날짜 형식을 바꿀 수 있습니다.

14.2.1 예제

사용자가 dateFormat 속성값을 변경했을 때 화면에 표시되는 날짜 형식이 바뀌는 것을 확인할 수 있습니다. 입력창에 입력할 수 있는 날짜 형식은 정해져 있습니다. 형식에 어긋나는 값을 입력하면 입력된 값이 dateFormat으로 적용되지 않고 그대로 입력한 값을 보여줍니다.




sample_calendar_01.xfdl

dateFormat 항목에 입력할 수 있는 형식은 아래와 같습니다.

```
strFormat ::= <maskchar> | 'LONGDATE' | 'SHORTDATE'
```

```
<maskchar> ::= [<Year>] [<Month>] [<Day>] [<Week>] [<Hour>] [<Minute>] [<Second>] [<MiliSec>]
```

'LONGDATE', 'SHORTDATE'로 속성값을 지정하는 경우에는 locale 속성값에 따라 사전에 지정된 형식이 적용됩니다. 년도, 월, 일 항목을 배치하는 순서가 지역에 따라 달라집니다. 예제에서 locale 항목을 바꾸면서 어떻게 적용되는지 확인할 수 있습니다.

ru-RU	ja-JP	en-US
LONGDATE	LONGDATE	LONGDATE
1 Июнь 2016 Среда г. 	2016年6月1日 	Wednesday, June 01, 2016

<maskchar> 속성값을 지정하는 경우에는 locale 속성과 상관없이 지정된 날짜 형식으로 변경됩니다. 강제적으로 보이는 날짜 형식을 고정하기 위해 사용합니다. 단, <week> 항목은 설정된 locale에 따라 표시됩니다.

ru-RU	ja-JP	en-US
yyyy/MM/dd dddd	yyyy/MM/dd dddd	yyyy/MM/dd dddd
2016/06/01 Среда 	2016/06/01 水曜日 	2016/06/01 Wednesday

14.2.2 예제에서 사용한 핵심 기능

locale

국가 및 언어를 설정합니다. 언어[국가] 코드의 조합으로 사용합니다. 예를 들어 미국에서 사용하는 영어를 지정하고 싶다면 'en_US'를 속성값으로 지정합니다. 대소문자를 가리지 않고 언더바(_) 대신 하이픈(-)을 사용할 수 있습니다. 국가코드를 지정하지 않고 언어코드만 지정해도 동작합니다. 'EN_US', 'en-US', 'EN' 모두 같은 형식으로 처리합니다.



locale 속성을 언어코드로만 지정하는 경우에는 해당 언어의 대표적인 국가로 자동 설정됩니다. 예를 들어 '영어'를 설정하면 대표적인 국가인 '미국'을 지정합니다. 언어가 같더라도 국가에 따라 표기가 다를 수 있습니다. 뉴질랜드에서 사용하는 locale 코드는 'en_NZ'인데 날짜를 표기하는 형식이 'en_US'와 다릅니다.

dateformat

컴포넌트에 포커스가 없을 때 화면에 표시되는 날짜 형식을 설정합니다. 화면에 보여주는 형식을 지정하는 것이기 때문에 실제 컴포넌트가 가지고 있는 value값에는 영향을 미치지 않습니다.



dateformat 속성은 대소문자를 구분합니다. 입력할 수 있는 항목은 아래와 같습니다.

'LONGDATE', 'SHORTDATE'
 [<Year>] 'yy', 'yyyy'
 [<Month>] 'M', 'MM'

```
[<Day>] 'd', 'dd'
[<Week>] 'ddd'
[<Hour>] 'h', 'hh', 'H', 'HH'
[<Minute>] 'm', 'mm'
[<Second>] 's', 'ss'
[<MiliSec>] 'sss'
```

14.2.3 예제 구현 방법

① 화면 구성하기

Static, Edit, Calendar, Button 컴포넌트를 예제 화면과 같이 배치합니다. Static 컴포넌트는 입력 항목의 타이틀을 표기하고 Edit 컴포넌트는 locale 속성값과 dateformat 속성값을 입력받습니다.

② Button 컴포넌트 이벤트 함수 작성하기

Button 컴포넌트를 선택한 후 onclick 이벤트 함수를 작성합니다. Button 컴포넌트 클릭 시 Edit 컴포넌트에 입력한 속성값을 Calendar 컴포넌트에 적용합니다.

```
this.btnChange_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.calDisplay.set_locale(this.editLocale.value);
    this.calDisplay.set_dateformat(this.editDateformat.value);
};
```

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 locale, dateformat을 입력하면서 Calendar 컴포넌트에 보이는 값을 확인합니다.

14.3 오늘 날짜 구하기

오늘 날짜를 표시하는 기능은 어느 시스템에서든 기본적인 기능입니다. 윈도우 운영체제를 사용하고 있다면 화면 오른쪽 하단에 시계와 오늘 날짜가 표시됩니다. 달력을 실행하면 오늘 날짜가 선택된 상태로 표시됩니다. 날짜를 기준으로 검색조건을 작성할 때도 오늘 날짜 기준으로 이전 1주일, 이전 1개월 이런 식으로 지정하기 때문에 오늘 날짜를 처리하는 기능은 가장 기본적인 기능입니다.

14.3.1 예제

버튼 클릭 시 오늘 날짜를 화면에 출력합니다. 화면이 출력될 때 바로 오늘 날짜가 보이길 원한다면 Form 오브젝트의 onload 이벤트 함수에서 스크립트를 처리할 수 있습니다. 이번 예제는 오늘 날짜를 구하는 방법을 설명하는 것이기 때문에 버튼 클릭 시 동작하는 방식으로 처리합니다.



14.3.2 예제에서 사용한 핵심 기능

Date

Date 오브젝트로 날짜와 시간을 처리합니다. 생성된 Date 오브젝트는 현재 시각에 대한 정보를 담고 있습니다. getFullYear와 같은 메소드를 사용해 원하는 항목값만 추출할 수 있습니다.

getFullYear, getMonth, getDate

Date 오브젝트에서 연도, 월, 일 값을 Integer 형태로 추출하는 메소드입니다. Integer 형태로 값을 추출하기 때문에 YYYYMMDD 형태로 값을 표현하기 위해서는 toString 메소드를 사용해 문자열로 변환 후 값을 합쳐 주어야 합니다. getMonth 메소드는 1월은 0, 12월은 11로 반환하기 때문에 실제 사용하는 월을 표기하기 위해서는 1을 더해주어야 합니다.

padLeft

String 오브젝트에서 지정한 길이로 문자열을 만들기 위해 사용하는 메소드입니다. 예를 들어 "1"이라는 문자를 "01"로 표현하려고 할 때 아래와 같이 사용합니다. 첫 번째 파라미터(2)는 만들 문자열 길이입니다. 두 번째 파라미터("0")는 왼쪽에 채울 문자열입니다. 원래 문자열의 길이를 뺀 나머지 길이만큼 해당 문자열이 채워집니다.

```
var dummy = "1".padLeft(2,"0"); // "01"
var dummy = "1".padLeft(5,"*"); // "*****1"
```

value

Calendar 컴포넌트에 표시되는 날짜값을 갖는 속성입니다. value 속성값은 editformat 속성값에 따라 입력할 수 있는 값이 달라집니다. editformat 속성의 default 값이 'yyyyMMdd' 8자리값이기 때문에 아래 예제에서는 해당 형태로 문자열을 만들어 value 값으로 설정합니다.

14.3.3 예제 구현 방법

① 화면 구성하기

Calendar, Button 컴포넌트를 예제 화면과 같이 배치합니다.

② Button 컴포넌트 이벤트 함수 작성하기

Button 컴포넌트를 선택한 후 onclick 이벤트 함수를 작성합니다. Button 컴포넌트 클릭 시 오늘 날짜를 계산해 Calendar 컴포넌트에 적용합니다.

```
this.btnToday onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var currDate = new Date();
    var year = currDate.getFullYear().toString().padLeft(4, "0");
    var month = (currDate.getMonth()+1).toString().padLeft(2, "0");
    var day = currDate.getDate().toString().padLeft(2, "0");

    this.calToday.set_value(year+month+day);
};
```

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 오늘 날짜가 정상 출력되는지 확인합니다.

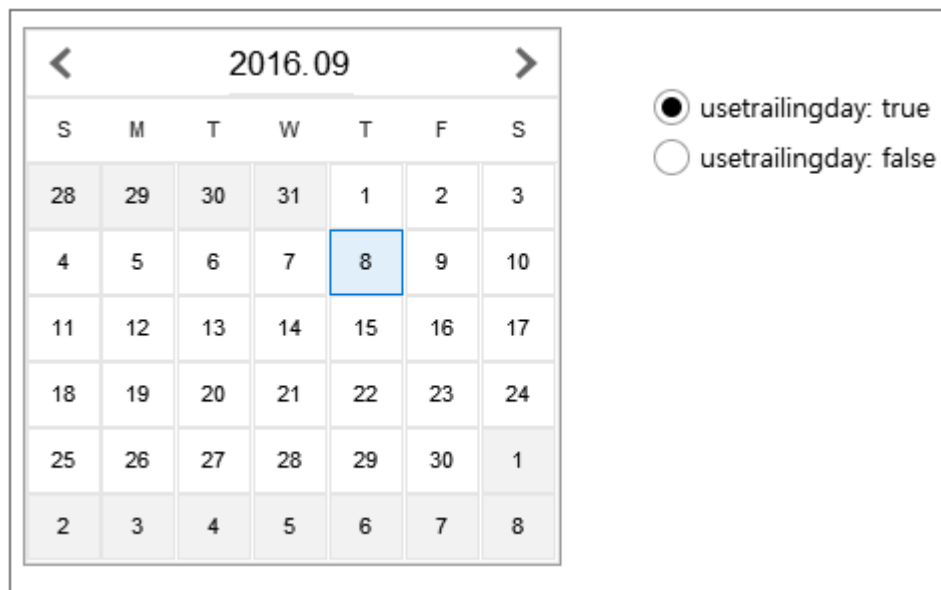
14.4 Trailing day 보여주기

Calendar 컴포넌트는 월 단위로 날짜를 표시합니다. 이때 이달에 해당하는 날짜만 표시할지 아니면 이전, 이후 월에 해당하는 날짜를 같이 보여줄지 옵션으로 설정할 수 있습니다. default 값은 false이며 이달에 해당하는 날짜만 표시합니다.

현재월에 해당하는 날짜만 선택해야 하는 경우 사용자가 선택할 수 있는 날짜만 보여주고 선택하게 할 수 있다는 장점이 있습니다.

14.4.1 예제

Radio 컴포넌트를 사용해 해당 속성값을 선택한 후 Calendar 컴포넌트가 어떻게 보이는지 확인할 수 있습니다.



 [sample_calendar_03.xfdl](#)

14.4.2 예제에서 사용한 핵심 기능

usetrailingday

Calendar 컴포넌트에 표시되는 날짜 표기를 해당 월에 해당하는 날짜만 표시할지 이전달, 다음달에 해당하는 날짜도 표시할지 설정합니다.

type

Calendar 컴포넌트는 type 속성값에 따라 화면에 표시되는 모습이 달라집니다. 이번 예제에서는 usetrailing day 속성값이 어떻게 적용되는지 바로 확인하기 위해 type 속성값을 'monthonly'로 설정했습니다.

Boolean

Radio 컴포넌트에서 선택한 값은 문자열 형태로 전달됩니다. usetrailingday 속성은 true 또는 false처럼 Boolean 속성값을 지정해주어야 하는데 이때 Boolean 메소드를 사용하면 값을 변환할 수 있습니다.



Boolean 메소드를 사용할 때 false 값으로 처리되는 경우는 아래와 같습니다. 그 외의 값은 모두 true 값으로 처리합니다. 예제에서도 false 값으로 처리하기 위해 Radio 컴포넌트의 codecolumn 데이터를 공백문자로 설정했습니다.

```
Boolean()
Boolean(0)
Boolean(null)
Boolean('')
Boolean(false)
```

14.4.3 예제 구현 방법

① 화면 구성하기

Calendar, Radio 컴포넌트를 예제 화면과 같이 배치합니다.

② 속성값 지정하기

Calendar 컴포넌트의 type 속성값을 'monthonly'로 수정합니다. 컴포넌트의 height 값도 적절히 수정해주어야 합니다. width값과 같은 값으로 설정하면 보기 좋게 출력됩니다. Radio 컴포넌트는 innerdataset 값을 아래와 같이 설정합니다. 이때 false에 해당하는 codecolumn 값은 공백문자로 설정해야 합니다.

InnerDataset		
No	codecolumn	datacolumn
1	true	usetrailingday: true
2		usetrailingday: false



코드상에서는 ''도 공백문자로 처리되지만 넥사크로 스튜디오에서 codecolumn 항목에 ''을 입력하면 인용부호 자체를 문자로 인식합니다. 때문에 ''대신 공백문자를 입력해야 합니다.

③ Radio 컴포넌트 이벤트 함수 작성하기

Radio 컴포넌트를 선택한 후 onitemchanged 이벤트 함수를 작성합니다. Radio 컴포넌트의 항목 선택 시 해당하는 값을 Calendar 컴포넌트의 usetrailingday 속성값으로 설정합니다.

```
this.radioTrailingday_onitemchanged = function(obj:nexacro.Radio,e:nexacro.
ItemChangeEventInfo)
{
    this.calToday.set_usetrailingday(Boolean(this.radioTrailingday.value));
};
```

4 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 선택에 따라 Calendar 컴포넌트의 모습이 바뀌는지 확인합니다.

14.5 날짜 간 차이 계산하기

날짜 간 차이를 계산하는 것은 생활 속에서도 자주 사용하는 기능입니다. 예를 들어 시험을 앞두고 있다면 오늘 날짜를 기준으로 얼마나 공부할 수 있는 날이 남아있는지 궁금할 수 있습니다. 그리고 여자친구를 만나고 있다면 처음 만난 지 며칠이 되었는지 확인해야 할 경우도 있습니다. 이런 경우뿐 아니라 회사에서 급여를 계산할 때도 날짜를 기준으로 처리하는 경우가 자주 발생합니다.

14.5.1 예제

두 개의 Calendar 컴포넌트에서 날짜를 입력받아 차이를 계산해 출력합니다.

From	2016-09-01 Thursday	Calculate	day	115
To	2016-12-25 Sunday			

 sample_calendar_04.xfdl

14.5.2 예제에서 사용한 핵심 기능

getYear, getMonth, getDay

화면에 표시되는 날짜값은 locale 속성값 설정에 따라 달라질 수 있어서 년, 월, 일을 가져오기 위해 별도 메소드를 사용했습니다. getMonth 메소드는 1월을 0으로 처리하고 있어서 별도 연산을 추가했습니다.

getTime

Date 오브젝트의 메소드로 시간값을 밀리세컨드 단위로 반환합니다. 두 개의 날짜를 연산으로 비교하기 위해 밀리초 단위로 변환하기 위해 사용합니다.

abs

날짜 계산 시 From 항목에 들어가는 날짜가 To 항목에 들어가는 날짜보다 나중이면 결과값이 음수로 표기되는데 이를 바로잡기 위해 Math 오브젝트의 abs 메소드를 사용합니다. abs 메소드는 입력된 값의 절댓값을 반환합니다.

14.5.3 예제 구현 방법

① 화면 구성하기

Calendar, Edit, Button, Static 컴포넌트를 예제 화면과 같이 배치합니다.

② Button 컴포넌트 이벤트 함수 작성하기

Button 컴포넌트를 선택한 후 onclick 이벤트 함수를 작성합니다. Button 컴포넌트 클릭 시 두 개의 Calendar 컴포넌트에서 선택된 값을 비교해 Edit 컴포넌트에 계산된 값을 출력합니다.

```
this.btnCal_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var fromDate = new Date();
    var toDate = new Date();
    var calDate;
    var day = 1000*60*60*24;

    fromDate.setFullYear(this.calFrom.getYear());
    fromDate.setMonth(this.calFrom.getMonth()-1);
    fromDate.setDate(this.calFrom.getDay());

    toDate.setFullYear(this.calTo.getYear());
    toDate.setMonth(this.calTo.getMonth()-1);
    toDate.setDate(this.calTo.getDay());

    calDate = fromDate.getTime() - toDate.getTime();

    this.editDay.set_value(Math.abs(calDate/day));
}
```



getTime 메소드가 밀리세컨드값을 반환하는데 이를 날짜값으로 변환하기 위해 day 변수를 사용합니다. day 변수에 담긴 값은 1일을 밀리세컨드로 환산한 값입니다. (24시간 * 60분 * 60초 * 1000)

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 날짜를 선택하고 버튼을 클릭해 날짜 간 차이를 계산합니다.

14.6 연도와 주로 날짜 구간 계산하기

업무에 따라 특정 날짜 구간을 주 단위로 처리하는 경우가 있습니다. 예를 들어 정기적으로 회의를 진행한다면 '2016년 32주차 정기 회의'라고 표시하는 경우가 있습니다. 이런 경우 2016년의 32주차라는 것이 어떤 날짜 구간을 의미하는 것인지 계산해야 할 필요가 있는 것이지요.

14.6.1 예제

연도와 주차를 숫자로 입력받아 해당하는 날짜 구간 시작일을 Calendar 컴포넌트에 표시합니다.

Year

Week

<
2016. 10
>

S	M	T	W	T	F	S
25	26	27	28	29	30	1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30	31	1	2	3	4	5

 sample_calendar_05.xfdl

14.6.2 예제에서 사용한 핵심 기능

getDay

Date 오브젝트의 getDay 메소드는 0부터 6까지 요일값을 반환합니다. 0은 일요일, 6은 토요일입니다. 예제에서는 주차의 시작을 일요일로 처리하기 때문에 1주차를 계산하기 위해 해당 연도의 1월 1일을 설정한 다음 해당 일이 일요일인지 확인합니다.



Calendar 컴포넌트에도 getDay 메소드가 있습니다. 이 메소드는 요일이 아니라 날짜값을 반환합니다. Calendar 컴포넌트에서 요일값을 구하려면 getDayOfWeek 메소드를 사용해야 합니다.

addDate

Date 오브젝트의 날짜 연산을 처리하는 메소드입니다. 파라미터로 지정한 숫자만큼 더한 날짜값을 반환합니다. 해당 Date 오브젝트의 날짜값이 변경되며 성공적으로 처리한 경우에는 밀리세컨드 단위로 계산된 날짜값을 반환합니다. 파라미터로 지정한 숫자가 음수면 지정한 숫자만큼 뺀 날짜값을 반환합니다.

parseInt

파라미터로 지정한 문자열을 정수 형태의 숫자로 변환하는 메소드입니다. 예를 들어 Edit 컴포넌트에 입력된 값은 문자열로 처리되는데 입력된 값을 숫자로 연산해야 하는 경우 변환을 위한 메소드가 필요합니다. 문자열을 변환하지 않고 연산하는 경우 원하지 않는 결과가 나올 수 있습니다.

```
"3"+1; //"31"
parseInt("3")+1; // 4
```

14.6.3 예제 구현 방법

① 화면 구성하기

Calendar, Edit, Button, Static 컴포넌트를 예제 화면과 같이 배치합니다. Calendar 컴포넌트는 계산된 값을 바로 확인할 수 있도록 type 속성값은 monthonly로 설정하고 usetrilingday 속성값은 true로 설정합니다.

② Button 컴포넌트 이벤트 함수 작성하기

Button 컴포넌트를 선택한 후 onclick 이벤트 함수를 작성합니다. Button 컴포넌트 클릭 시 입력된 연도와 주를 계산해 결과값을 Calendar 컴포넌트에 표시합니다.

```
this.btnCal_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var dDate = new Date(parseInt(this.editYear.value),0,1);
    var nDay = dDate.getDay();

    if(nDay != 0) {
```

```

        dDate.addDate(-nDay);
    }

    dDate.addDate(7 * (parseInt(this.editWeek.value)-1));

    this.calWeek.set_value(dDate.getFullYear()
        +(dDate.getMonth()+1).toString().padLeft(2, "0")
        +dDate.getDate().toString().padLeft(2, "0")
    );
}

```

3 QuickView로 확인하기

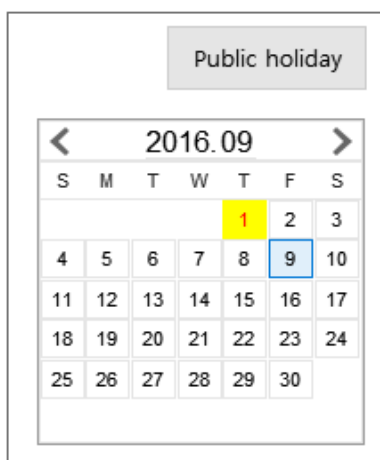
QuickView(Ctrl + F6)로 실행한 후 연도와 주차를 입력하고 해당 날짜 구간을 확인합니다.

14.7 공휴일 지정하기

달력에서 공휴일을 표시하는 기능은 자주 사용하면서도 논리적으로 구현하기는 어려운 부분입니다. 외부에서 제공하는 이벤트 일정 API를 사용하거나 개발자가 자체적으로 데이터를 만들어 제공하기도 합니다. Calendar 컴포넌트에서는 innerdataset 속성값을 지정해 공휴일 정보를 표시하는 방법을 사용합니다.

14.7.1 예제

버튼 클릭 시 공휴일 정보를 적용해 Calendar 컴포넌트에 표시합니다. 현재 연도의 매달 1일을 지정했습니다. 해당 날짜 정보는 필요에 따라 변경할 수 있습니다.

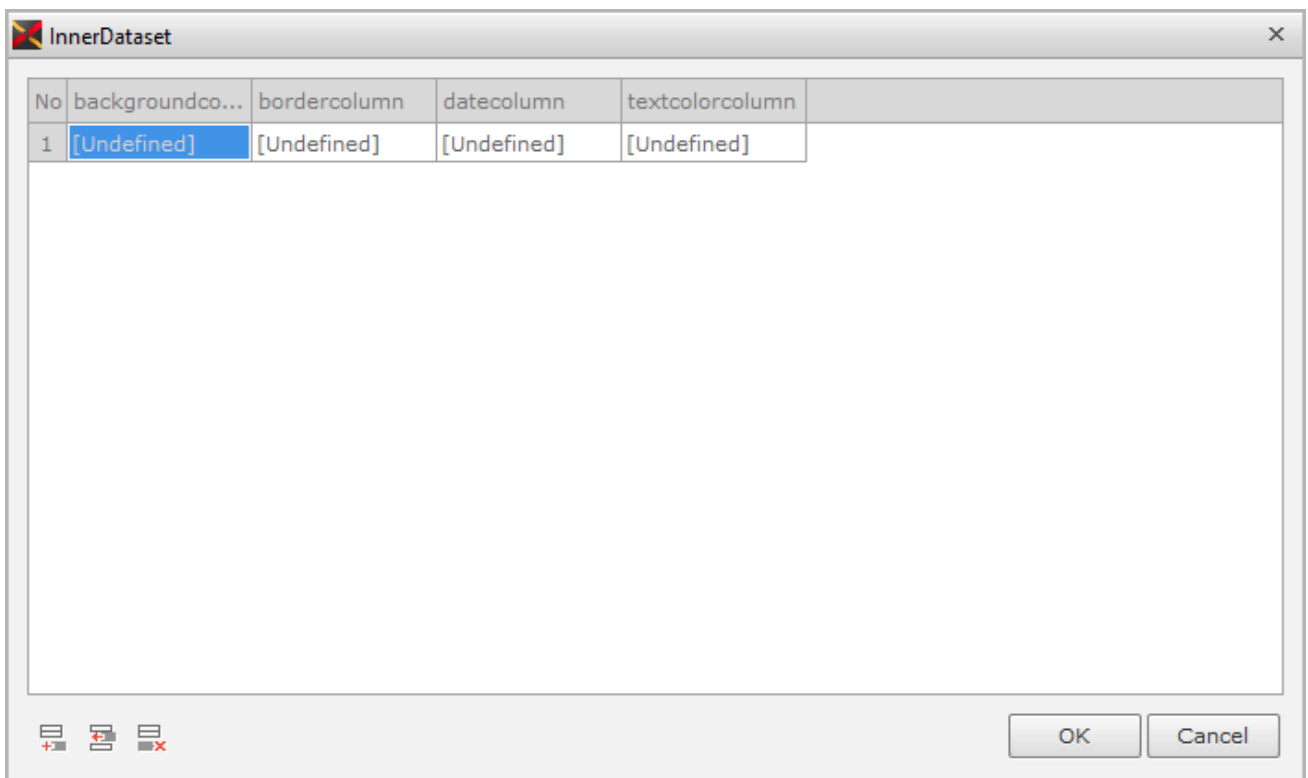




14.7.2 예제에서 사용한 핵심 기능

innerdataset

Calendar 컴포넌트에서 특정 날짜 표기를 처리하기 위해 해당 데이터를 담고 있는 Dataset을 지정할 수 있습니다. Dataset 컴포넌트를 추가해 지정할 수도 있으며 넥사크로 스튜디오에서 innerdataset 편집기를 사용해 데이터를 추가할 수 있습니다.



backgroundcolumn, textcolorcolumn, bordercolumn

Calendar 컴포넌트에서 해당 날짜에 해당하는 영역의 배경색, 텍스트색, 테두리 정보를 지정하는 속성입니다. bordercolumn 데이터는 DatePickerControl 컨트롤의 border 속성을 기준으로 작성되어야 합니다.

14.7.3 예제 구현 방법

① 화면 구성하기

Calendar, Button 컴포넌트를 예제 화면과 같이 배치합니다. Calendar 컴포넌트는 계산된 값을 바로 확인할 수 있도록 type 속성값은 monthonly로 설정합니다.

2 Dataset 컴포넌트 설정하기

Dataset 컴포넌트를 추가하고 backgroundcolumn, bordercolumn, datecolumn, textcolorcolumn 4개 컬럼을 설정합니다. 그리고 속성창에서 Calendar 컴포넌트의 innerdataset 항목을 설정합니다. innerdataset 속성값을 설정하고 나면 나머지 항목도 목록에서 설정할 수 있습니다.

Binding	
backgroundcolumn	backgroundcolumn
bordercolumn	bordercolumn
datecolumn	datecolumn
innerdataset	dsHoliday
textcolorcolumn	dsHoliday

3 Button 컴포넌트 이벤트 함수 작성하기

Button 컴포넌트를 선택한 후 onclick 이벤트 함수를 작성합니다. Button 컴포넌트 클릭 시 Dataset의 데이터를 초기화하고 현재 년도의 매월 1일 날짜값을 Dataset에 추가합니다. 이때 backgroundcolumn, bordercolumn, textcolorcolumn 항목은 설정하지 않으면 테마에 지정된 값이 적용됩니다.

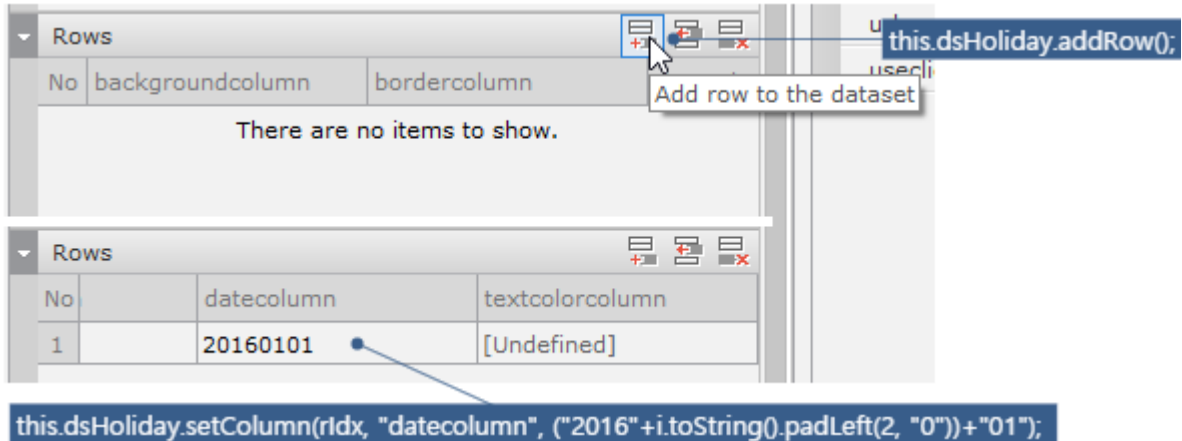
```
this.btnCal_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.dsHoliday.clearData();

    var colorBordercolumn = "1px solid green";
    var colorBackgroundcolumn = "yellow";
    var colorTextcolorcolumn = "red";

    var currDate = new Date();
    var year = currDate.getFullYear().toString().padLeft(4, "0");

    for(var i=1;i<13;i++) {
        var rIdx = this.dsHoliday.addRow();
        this.dsHoliday.setColumn(rIdx, "backgroundcolumn", colorBackgroundcolumn);
        this.dsHoliday.setColumn(rIdx, "textcolorcolumn", colorTextcolorcolumn);
        this.dsHoliday.setColumn(rIdx, "bordercolumn", colorBordercolumn);
        this.dsHoliday.setColumn(rIdx, "datecolumn", (year+i.toString().padLeft(2, "0"))+
"01");
    }
}
```

1월부터 12월까지 매월 1일을 공휴일로 지정하기 위해 for 문을 사용했습니다. i라는 변수의 초기값을 1로 지정하고 13보다 작을 때까지 1씩 늘어나며 구문 내의 코드를 반복해서 실행합니다. Dataset에 새로운 값을 추가하기 때문에 addRow 메소드로 Row를 추가하고 setColumn 메소드를 사용해 지정된 컬럼에 데이터를 지정합니다. 넥사크로 스튜디오에서 아래 그림과 같이 Row를 추가하고 값을 설정하는 것과 같습니다.



사용자가 버튼을 클릭하거나 항목을 입력했을 때 이전에 추가된 Dataset의 값을 모두 삭제하고자 할 때는 clearData 메소드를 사용합니다. 이번 예제에서는 같은 데이터가 추가되기 때문에 초기화가 없다면 같은 데이터가 계속 쌓이게 됩니다.



backgroundcolumn, bordercolumn, textcolorcolumn 항목은 설정하지 않으면 테마에 지정된 값이 적용됩니다.

4 QuickView로 확인하기

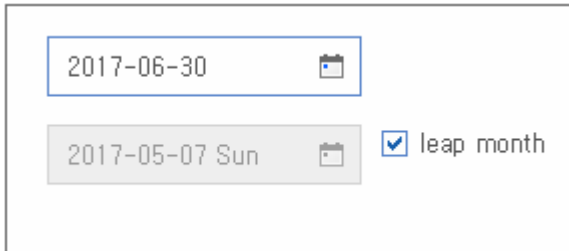
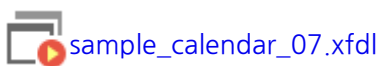
QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 지정된 공휴일을 지정합니다.

14.8 음력 날짜로 변환하기

양력 날짜를 음력으로 변환하는 것은 명확한 규칙을 가지고 있지는 않습니다. 그래서 대부분의 음력 날짜 변환하는 스크립트를 보면 어느 시점부터 어느 시점까지만 변환할 수 있는 제한을 둡니다. 그 기간동안의 기본 데이터를 스크립트 안에 지정해놓고 계산합니다. 이번 예제에서는 1900년부터 2040년까지 음력 날짜로 변환하는 기능을 살펴봅니다.

14.8.1 예제

양력 날짜를 입력하면 음력 날짜를 표시합니다.

14.8.2 예제에서 사용한 핵심 기능

concat

Calendar 컴포넌트에 value 값을 설정할때 "20181212"와 같은 식으로 값을 설정하는데, 년도, 월, 일로 나뉘어진 문자열을 하나로 합치기 위해 concat 메소드를 사용합니다.

14.8.3 예제 구현 방법

① Form 화면 구성하기

Calendar 컴포넌트를 2개 배치합니다. 첫번째 Calendar 컴포넌트는 양력 날짜를 입력받기 위한 용도입니다. 두번째 Calendar 컴포넌트는 음력 날짜를 출력하기 위한 용도입니다. enable 속성값을 false로 지정합니다. 그리고 CheckBox 컴포넌트를 하나 더 배치하는데 음력 날짜에는 윤달이라는 개념이 있어서, 윤달일 경우에 체크박스에 표시해줍니다.

② Calendar 컴포넌트의 onchanged 이벤트 함수 작성하기

양력 날짜를 입력하면 바로 음력 날짜가 출력되도록 onchanged 이벤트 함수를 작성합니다. 입력받은 값으로 음력 날짜를 계산해서 Calendar 컴포넌트와 CheckBox 컴포넌트에 표시합니다.

```
this.Calendar00_onchanged = function(obj:nexacro.Calendar,e:nexacro.ChangeEventInfo)
{
    var c = new String(obj.value);
    var y = c.substr(0,4);
```

```

var m = c.substr(4,2);
var d = c.substr(6,2);
var lunar = this.lunarCalc(y, m, d, 1);
lunar.month = lunar.month < 10 ? "0"+lunar.month : lunar.month;
lunar.day = lunar.day < 10 ? "0"+lunar.day : lunar.day;

this.Calendar01.set_value(String().concat(lunar.year, lunar.month, lunar.day));
this.CheckBox00.set_value(lunar.leapMonth);
};

```

3 음력 날짜 계산하기

onchanged 이벤트에서 호출되는 함수는 lunarCalc 입니다. 여기에서는 입력된 양력 날짜를 음력 날짜로 계산합니다. 정확하게는 계산이라기보다는 lunarMonthTable 라는 배열에서 해당하는 날짜 정보를 찾아서 반환하는 것입니다.

```

this.lunarCalc = function (year, month, day, type, leapmonth)
{
    var solYear, solMonth, solDay;
    var lunYear, lunMonth, lunDay;
    var lunLeapMonth, lunMonthDay;
    var i, lunIndex;

    var solMonthDay = [31, 0, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31];

    /* range check */
    if (year < 1900 || year > 2040)
    {
        alert('This example supports solar to lunar from 1900 to 2040 only');
        return;
    }
}

```



전체 스크립트 코드는 소스를 참조해주세요.

4 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 첫번째 Calendar에서 날짜를 선택하면 음력 날짜를 확인할 수 있습니다.

14.9 입력한 날짜 보정하기

달력에서 직접 날짜를 선택하는 경우는 선택한 날짜값을 그대로 처리하지만, 직접 텍스트 편집 영역에 값을 입력하거나 스크립트에서 값을 설정하는 경우에는 날짜 범위를 벗어나거나 누락된 값이 들어올 수 있습니다. 이런 경우에는 내부적으로 날짜값을 규칙에 따라 보정하고 보정된 값으로 처리합니다.

14.9.1 예제

Radio 컴포넌트에서 입력할 값을 선택하면 Calendar 컴포넌트에 보정된 값이 표시됩니다. 날짜 보정 규칙은 아래와 같습니다.

- 년, 월, 일 항목이 빈 값인 경우에는 "01"로 바꾸어줍니다.
"2018"만 입력한 경우에는 "20180101"로 처리합니다.
- "00" 또는 "0"을 입력한 경우에는 -1을 뺀 값으로 처리합니다.
"201800"을 입력한 경우에는 "20180001"로 처리한 후 "20171201"로 처리합니다.
- 월, 일 범위를 초과해 입력한 경우에는 초과한 만큼 더한 값으로 처리합니다.
"20181238"을 입력한 경우에는 "20181231"에서 7을 더해서 "20190107"로 처리합니다.

empty >> "01"	00 >> -1	over >> +x
☒ 2018 ☐ 201802 ☐ 0000	☒ 201800 ☐ 20180000 ☐ 00000000	☒ 20181238 ☐ 00001345 ☐ 99999999
2018-01-01	2017-12-01	2019-01-07

sample_calendar_08.xfdl

14.9.2 예제에서 사용한 핵심 기능

value

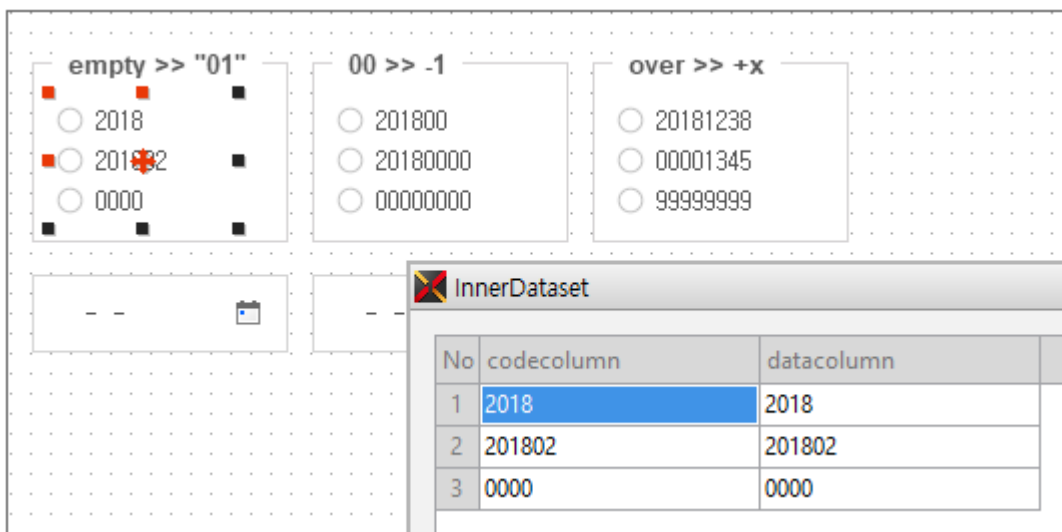
Calendar 컴포넌트의 경우 사용자가 입력한 값과 실제 처리되는 value 속성값이 다를 수 있습니다. 예제에서 설명한 규칙 외에도 editformat 속성에 따라 입력한 값과 달라질 수 있습니다. 일반적인 경우에는 사용자가 입력할 수 있는 경로를 제한하기 때문에 예외적인 상황이 발생하지는 않지만, value 속성의 특징을 미리 알지 못하면 원하는 데이터가 들어오지 않았을 때 원인을 확인할 수 없습니다.

14.9.3 예제 구현 방법

- 1 화면에 GroupBox 컴포넌트를 배치하고, Radio 컴포넌트를 그 안에 배치합니다.

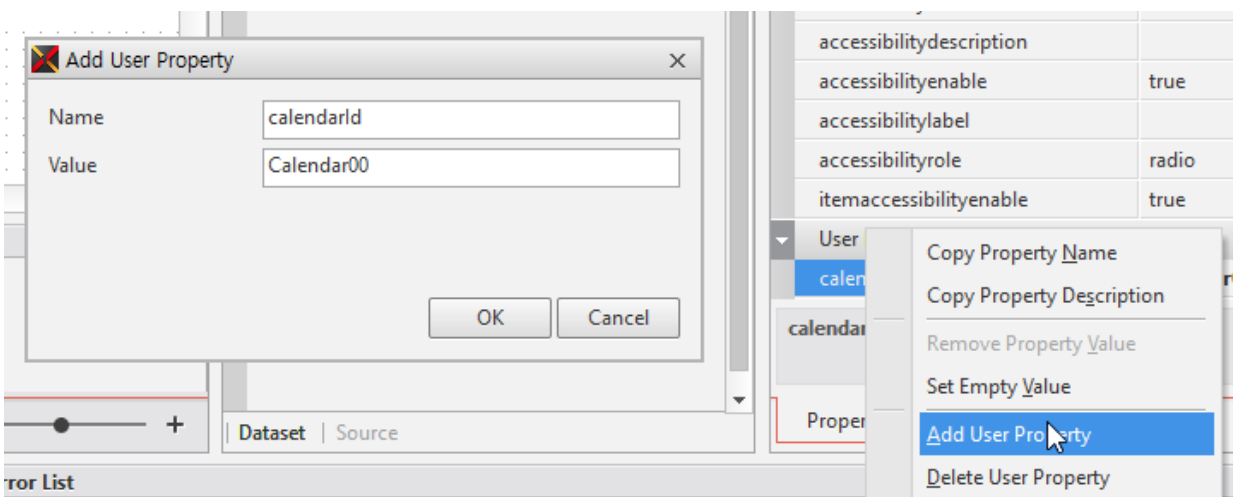
예제에서는 규칙에 따라 Radio 컴포넌트를 구분해 표시하기 위해 GroupBox 컴포넌트를 사용했습니다. 동작에는 영향을 미치지 않습니다.

- 2 Radio 컴포넌트의 innerdataset 데이터를 예제와 같이 지정합니다.



- 3 Radio 컴포넌트 아래에 Calendar 컴포넌트를 배치합니다.

- 4 Radio 컴포넌트를 선택하고 속성창 컨텍스트 메뉴를 [Add User Property] 항목을 선택합니다.



- 5 [Add User Property] 입력창이 뜨면 Name 항목은 "calendarId", Value 항목은 "Calendar00"으로 입력합니다.

사용자 속성으로 추가한 calendarId 속성은 Radio 컴포넌트의 이벤트 처리 시 사용할 겁니다.

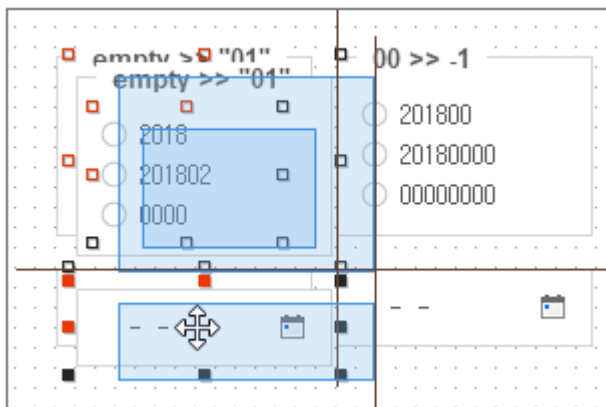
- ⑥ Radio 컴포넌트의 onitemchanged 이벤트 함수를 아래와 같이 추가합니다.

이벤트가 발생한 Radio 컴포넌트의 사용자 속성인 calendarId 속성값에 해당하는 Calendar 컴포넌트의 value 속성값을 선택한 아이템 값으로 설정합니다.

```
this.Radio_onitemchanged = function(obj:nexacro.Radio,e:nexacro.ItemChangeEventInfo)
{
    this[obj.calendarId].set_value(e.postvalue);
};
```

- ⑦ GroupBox, Radio, Calendar 컴포넌트를 모두 선택하고 복사한 후 붙여넣기를 하고 적절한 위치에 배치합니다.

예제에서는 3개의 컴포넌트의 배치가 3번 반복되는 구성이라 복사해 붙여넣기를 사용했습니다. 간단한 예제이니 1번부터 단계를 반복할 수도 있습니다.



- ⑧ GroupBox 컴포넌트의 text 속성값을 수정하고 Radio 컴포넌트의 innerdataset 속성 데이터를 설정합니다. 앞에서 만든 Radio 컴포넌트를 복사했다면 사용자 속성인 calendarId 속성값도 수정합니다.

- ⑨ QuickView(Ctrl + F6)로 실행한 후 선택한 값이 어떻게 보여지는지 확인합니다.

14.10 DatePickerControl 스타일 지정하기

Calendar 컴포넌트에서 달력 형태로 날짜를 선택할 수 있는 영역은 DatePickerControl 입니다. Calendar 컴포넌트 뿐 아니라 Grid 컴포넌트 등에서도 사용할 수 있기 때문에 별도의 컨트롤로 분리되어 있습니다. Calendar 컴포넌트에서 DatePickerControl 영역의 스타일을 지정하는 방법을 살펴봅니다.

14.10.1 예제

왼쪽은 기본 테마를 적용한 상태이며, 오른쪽은 일부 스타일 속성을 변경한 상태입니다.

< 2019. 04 >						
Sun	Mon	Tue	Wed	Thu	Fri	Sat
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30				

< 2019. 04 >						
Sun	Mon	Tue	Wed	Thu	Fri	Sat
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30				



sample_calendar_09.xfdl

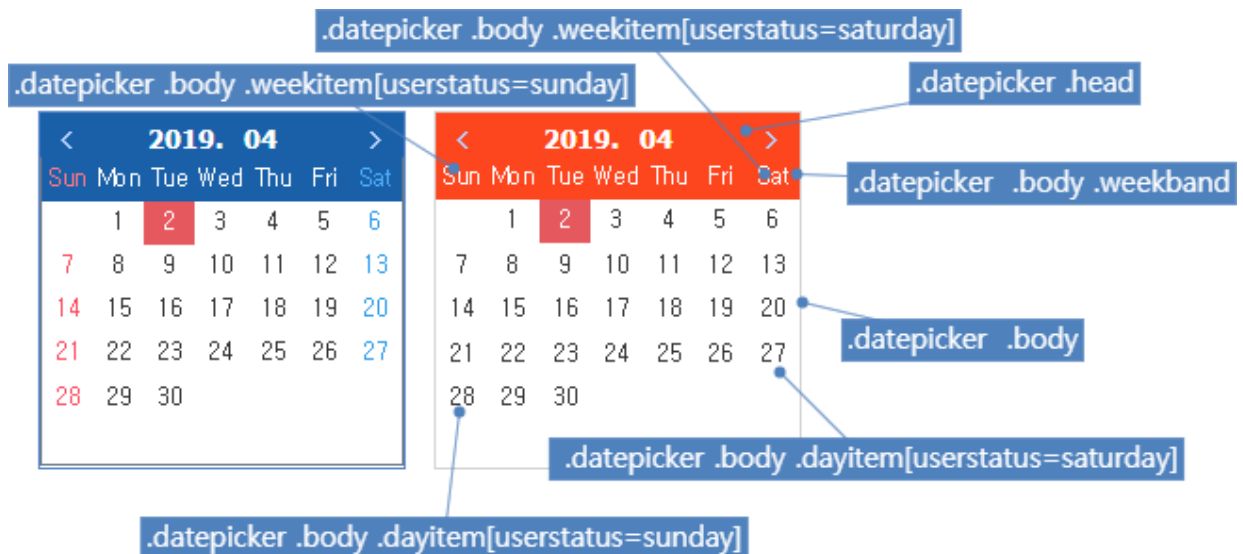
14.10.2 예제에서 사용한 핵심 기능

datepicker

Calendar 컴포넌트의 하위 컨트롤인 DatePickerControl에 접근하기 위한 속성입니다. 예제에서는 스타일 속성을 설정하기 위한 자식 선택자로 사용합니다.

14.10.3 예제 구현 방법

- 1 화면에 Calendar 컴포넌트를 배치하고 locale 속성값은 "en_US", type 속성값은 "monthonly"로 지정합니다. width, height 속성값은 200으로 지정합니다.
- 2 xcss 파일을 열어서 아래와 같이 코드를 추가합니다.



DatePickerControl은 두 부분으로 나눌 수 있습니다. 년도와 월을 표시하며 버튼을 가지고 있는 head, 요일과 날짜를 표시하는 body. 그리고 body는 다시 요일을 표시하는 weekband와 각각의 dayitem으로 구성됩니다. 아래 코드에서는 head, weekband 영역의 배경색을 바꾸었고 body 영역의 테두리선을 보이지 않게 했으며, 토요일과 일요일을 다른 날과 같은 색으로 표시하도록 했습니다.

```
.Calendar.sample_calendar_09 .datepicker .head
{
    background : orangered;
}

.Calendar.sample_calendar_09 .datepicker .body .weekband
{
    background : orangered;
}
```

```
.Calendar.sample_calendar_09 .datepicker .body
{
    -nexa-border : 1px none transparent;
}
```

```
.Calendar.sample_calendar_09 .datepicker .body .dayitem[userstatus=saturday]
{
    color : #333333;
}

.Calendar.sample_calendar_09 .datepicker .body .dayitem[userstatus=sunday]
{
}
```

```

    color : #333333;
}
.Calendar.sample_calendar_09 .datepicker .body .weekitem[userstatus=saturday]
{
    color : #ffffff;
}
.Calendar.sample_calendar_09 .datepicker .body .weekitem[userstatus=sunday]
{
    color : #ffffff;
}

```

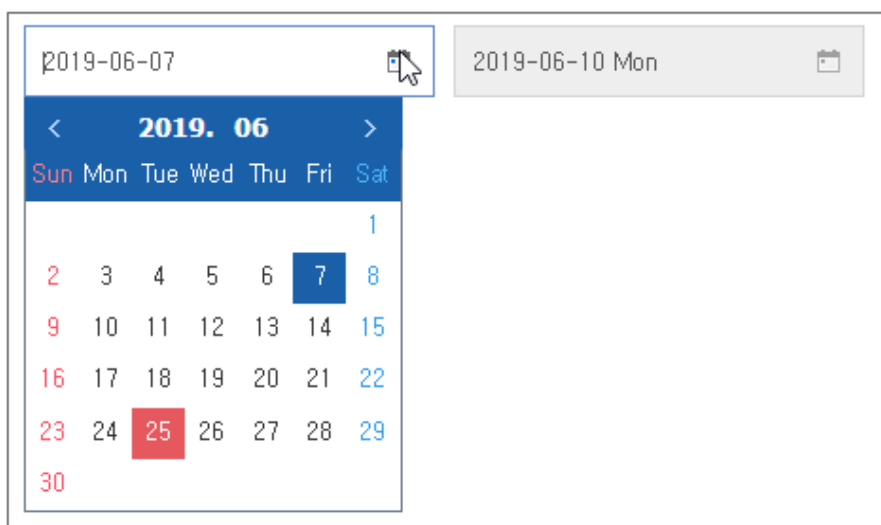
- ③ 오른쪽 Calendar 컴포넌트의 cssclass 속성값을 "sample_calendar_09"로 설정합니다.
- ④ QuickView(Ctrl + F6)로 실행한 후 두 개의 Calendar 컴포넌트에 적용된 스타일을 확인합니다.

14.11 평일로 다음날 설정하기

Calendar 컴포넌트에서 선택한 날이 금요일인 경우 토요일이 아닌 다음 주 월요일을 설정합니다.

14.11.1 예제

일요일에서 목요일 사이의 날짜를 선택하면 다음날을 두 번째 Calendar 컴포넌트에 표시하고 금요일, 토요일을 선택한 경우에는 월요일을 표시합니다.





14.11.2 예제에서 사용한 핵심 기능

getDayOfWeek

선택한 날짜의 요일값을 반환합니다. 일요일은 0을 반환하고 토요일은 6을 반환합니다.

14.11.3 예제 구현 방법

- 1 화면에 Calendar 컴포넌트를 배치하고 locale 속성값은 "en_US"로 설정합니다. 두 번째 Calendar 컴포넌트의 readonly 속성값은 false로 지정합니다.

- 2 첫 번째 Calendar 컴포넌트의 onchanged 이벤트 함수를 아래와 같이 작성합니다.

getDayOfWeek 메소드를 사용해 선택한 날짜의 요일값을 확인하고 금요일, 토요일의 경우 따로 처리합니다.

```
this.Calendar00_onchanged = function(obj:nexacro.Calendar,e:nexacro.ChangeEventInfo)
{
    var nWeek = obj.getDayOfWeek();
    if(nWeek == 5)
    {
        this.Calendar01.set_value (this.AddDateOffset(e.postvalue, 3));
    }
    else if(nWeek == 6)
    {
        this.Calendar01.set_value (this.AddDateOffset(e.postvalue, 2));
    }
    else
    {
        this.Calendar01.set_value (this.AddDateOffset(e.postvalue, 1));
    }
    this.setFocus(true, false);
};
```

다음날 처리를 위해 지정한 날만큼 더한 값을 반환합니다.

```
this.AddDateOffset = function(sDate, nOffset)
{
    var nYear = parseInt(sDate.substr(0, 4));
    var nMonth = parseInt(sDate.substr(4, 2));
    var nDate = parseInt(sDate.substr(6, 2)) + nOffset;

    return this.MakeDate(nYear, nMonth, nDate);
}

this.MakeDate = function(nYear, nMonth, nDate)
{
    var objDate = new Date(nYear, nMonth-1, nDate);

    var sYear = objDate.getFullYear().toString();
    var sMonth = (objDate.getMonth() + 1).toString().padLeft(2, "0");
    var sDate = (objDate.getDate()).toString().padLeft(2, "0");

    return sYear + sMonth + sDate;
}
```

- ③ QuickView(Ctrl + F6)로 실행한 후 첫 번째 Calendar 컴포넌트에서 원하는 날짜를 선택해봅니다.

15.

CheckBox

15.1 CheckBox 소개

CheckBox는 선택과 해제라는 두 가지 상태값을 보여주고 설정하기 위한 컴포넌트입니다. CheckBox는 선택 상태를 표시하는 작은 사각형 아이콘과 텍스트로 구성되는데 사용자가 체크박스를 선택하면 작은 사각형 아이콘에 v로 체크 표시가 되고 다시 한 번 더 선택하면 체크 표시가 없어집니다.

CheckBox는 그리드에서 사용이 가능합니다. 그리드 셀의 displaytype, edittype 속성을 체크박스로 설정하면 설정한 컬럼은 체크박스로 동작합니다. 그리드에서는 체크박스를 어떤 작업을 하기 위해 로우를 선택하는 용도로 많이 사용합니다.

CheckBox는 Dataset을 바인딩해 사용할 수 있습니다. 바인딩된 컬럼의 값이 true 혹은 1이면 v로 체크 표시되고, 값이 존재하지 않거나 false 혹은 0이면 체크가 해제됩니다.

CheckBox의 상태 값으로 사용하는 true(혹은 1) 값과 false(혹은 0) 값은 다른 값으로 변경할 수 있습니다. 상태 값을 변경하려면 truevalue, falsevalue 속성을 사용하는데 예를 들어, truevalue 속성에 male을, falsevalue 속성에 female을 설정했을 때 바인딩된 컬럼의 값이 male이면 체크 상태로, female이면 체크 해제 상태로 표시됩니다.

15.1.1 사용 예

아래는 구글의 광고 설정 관리 화면입니다. 사용자는 주제별로 분류된 광고 중에 받아 보고 싶은 광고만을 선택하여 받을 수 있습니다. 여기서 사용자 선택에 대한 표시나 입력을 받을 때 체크박스를 사용하고 있음을 알 수 있습니다.

Your topics

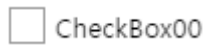
- | | | |
|---|---|--|
| ☐ Basketball | ☒ Blues | ☒ Books & Literature |
| ☒ Cats | ☐ Classical Music | ☒ Combat Sports |
| ☒ Dogs | ☒ Fashion & Style | ☐ Fitness |
| ☒ Mobile & Wireless | ☐ Movies | ☒ Outdoors |
| ☒ Performing Arts | ☒ Politics | ☒ Pop Music |
| ☐ Baseball | ☐ Celebrities & Entertainment News | ☐ Cooking & Recipes |
| ☐ Fast Food | ☐ Loans | ☐ Restaurants |
| ☒ Shopping | ☐ Soccer | ☐ TV Comedies |

+ NEW TOPIC

VIEW 1 MORE INTEREST

WHERE DID THESE COME FROM?

15.1.2 CheckBox 만들기



1 CheckBox 컴포넌트 생성하기

툴바에서 CheckBox 를 선택한 후, Form 위에 적당한 크기로 드래그하여 생성합니다. 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

2 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행합니다. CheckBox 컴포넌트가 표현할 어떤 값도 설정되어 있지 않으므로 사각형 아이콘은 빈 박스로 표시됩니다.

15.2 TRUE, FALSE 값 변경하기

CheckBox의 상태를 결정하는 값은 true(혹은 1)와 false(혹은 0)입니다. 그러나 설정에 따라 다른 값을 사용할 수도 있습니다. 예를 들면, yes, no 혹은 male, female과 같은 문자열이나 Y, N 같은 문자를 체크값으로 사용할 수 있습니다.

CheckBox에서 체크 설정값을 변경할 수 있도록 truevalue와 falsevalue 속성을 제공합니다. truevalue 속성에는 체크 설정값을, falsevalue 속성에는 체크 해제 값을 설정합니다. 설정값은 문자열 형식이며 대, 소문자를 구별합니다.

15.2.1 예제

다음은 기본적으로 사용하는 true, false 값이 아닌 Yes, No를 적용한 CheckBox 예제입니다. 동작은 동일하나 체크 값으로 Yes, No를 사용하고 있습니다. 체크 값이 변경될 때 체크박스 아래 Edit 컴포넌트에 변경된 값을 보여줍니다.

CheckBox에서 사용하는 체크 값을 변경하려면 truevalue, falsevalue 속성을 설정합니다.



15.2.2 예제에서 사용한 핵심 기능

truevalue

체크박스에서 체크를 설정하는 데 사용할 값을 문자열로 설정하는 속성입니다. 속성에 아무 값도 설정하지 않으면 "true" 또는 "1"값이 사용됩니다.

falsevalue

체크박스에서 체크 설정을 해제하는데 사용할 값을 문자열로 설정하는 속성입니다. 속성에 아무 값도 설정하지 않으면 "false" 또는 "0"값이 사용됩니다.

15.2.3 예제 구현 방법

① CheckBox 컴포넌트 생성하기

툴바에서 CheckBox 컴포넌트와 Edit 컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택하고 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

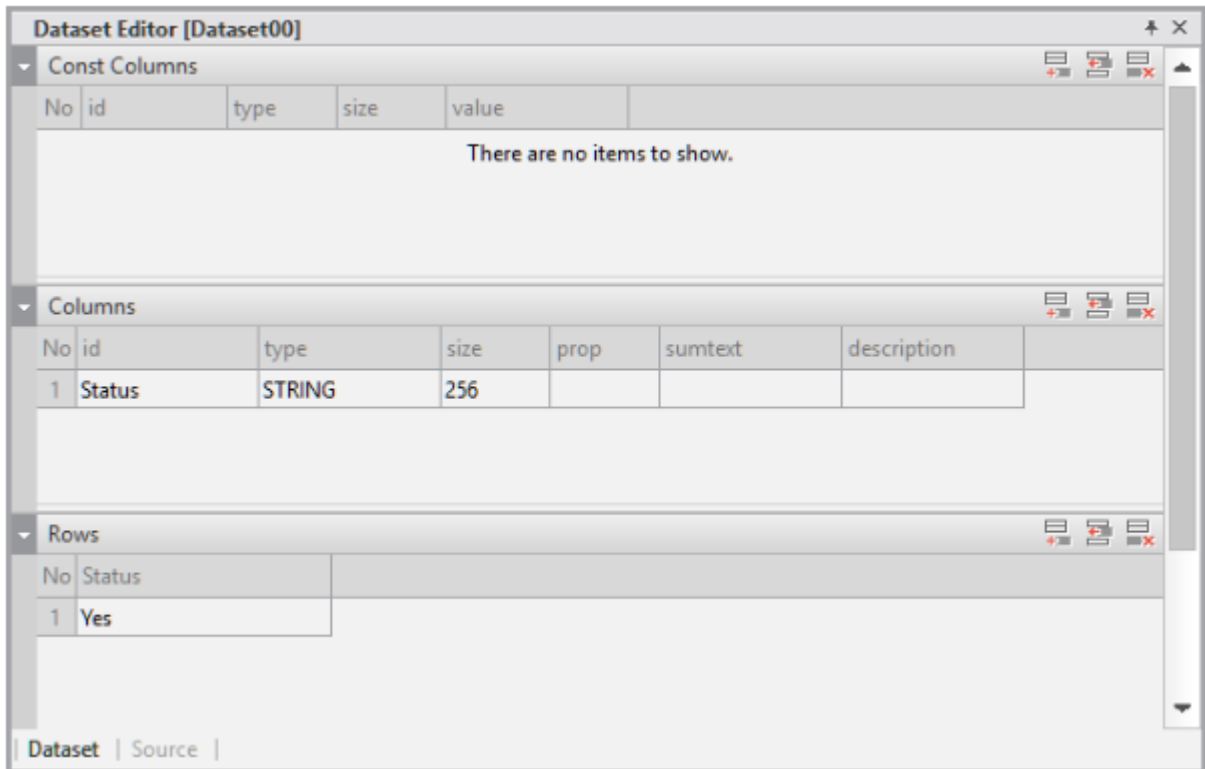
② CheckBox 속성 설정하기

속성	값
truevalue	Yes
falsevalue	No

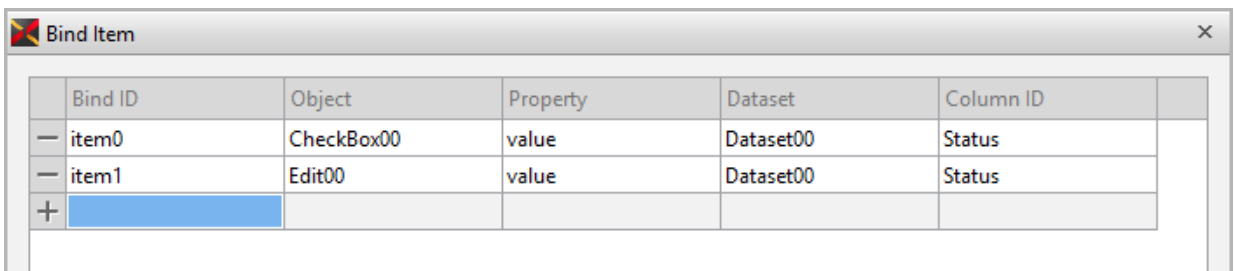
③ Dataset 생성과 바인딩하기

CheckBox에 바인딩할 값을 갖는 Dataset을 생성하고 바인딩합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. Dataset Editor를 열어 다음과 같이 입력합니다.



3. Invisible Object 영역에 있는 Dataset을 Form에 있는 CheckBox 컴포넌트와 Edit 컴포넌트로 드래그 앤 드롭시키면 Bind Item 창이 뜨는데 다음과 같이 바인딩합니다.



4 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행합니다. 바인딩된 컬럼의 값이 "Yes"로 설정되어 있으므로 사각형 아이콘은 체크 상태로 표시됨을 확인합니다.

15.3 체크 여부 확인하기

truevalue와 falsevalue 속성을 지정한 경우에는 value 속성값만 봐서는 체크된 상태인지 여부를 확인하기 어려울 수 있습니다. 이런 경우에는 isChecked 메소드를 사용해 체크 여부를 확인할 수 있습니다.

15.3.1 예제

Grid 컴포넌트에서 값을 선택하면 해당하는 값이 CheckBox 컴포넌트에 반영됩니다. CheckBox02 에는 truevalue 속성값을 "A", falsevalue 속성값을 "B"로 설정했습니다.

	CheckBox00	CheckBox01	CheckBox02
☒ CheckBox00	0	true	A
☐ CheckBox01	1	false	B
☒ CheckBox02	TEST	TRUE	C
	TRUE	FALSE	D

CheckBox00: true / 1
 CheckBox01: false / false
 CheckBox02: true / B



sample_checkbox_03.xfdl

15.3.2 예제에서 사용한 핵심 기능

isChecked

체크 상태를 반환합니다. truevalue와 falsevalue 속성이 적용된 경우에는 해당 속성값을 반영해서 상태를 반환합니다.

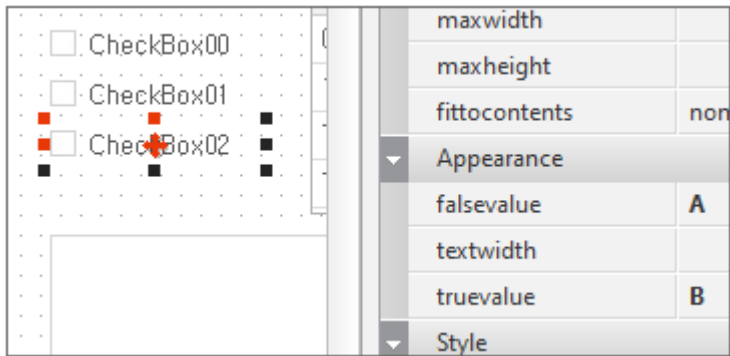
onbindingvaluechanged

Dataset 오브젝트의 상태가 변경되면서 바인딩된 컴포넌트의 속성값이 변경되는 경우 발생합니다. 바인딩된 컴포넌트의 속성값이 모두 변경된 후에 처리되기 때문에 마지막 값을 확인하거나 변경된 속성으로 인해 레이아웃이 변경된 경우 등에 사용합니다.

15.3.3 예제 구현 방법

- 1 CheckBox, Grid, TextArea 컴포넌트를 배치합니다.

마지막 컴포넌트(CheckBox02)의 truevalue, falsevalue 속성값을 "B", "A"로 설정합니다.

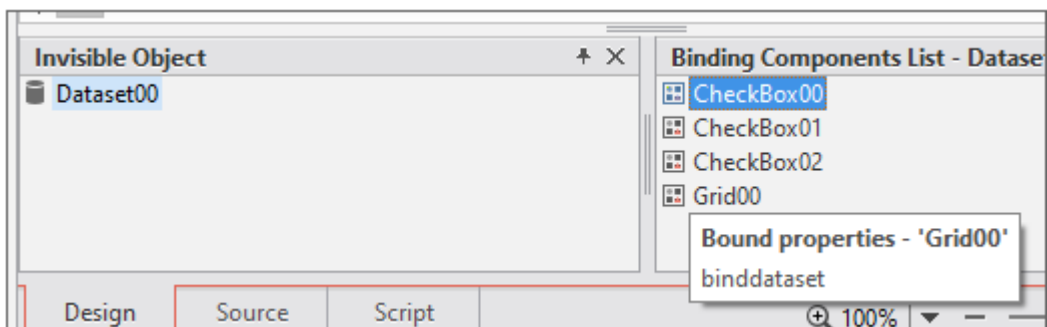


- 2 Dataset 오브젝트를 배치하고 아래와 같이 데이터를 설정합니다.

Columns					
No	id	type	size	prop	sumtext
1	CheckBox00	STRING	256		
2	CheckBox01	STRING	256		
3	CheckBox02	STRING	256		

Rows			
No	CheckBox00	CheckBox01	CheckBox02
1	0	true	A
2	1	false	B
3	TEST	TRUE	C
4	TRUE	FALSE	D

- 3 Grid 컴포넌트와 CheckBox 컴포넌트에 Dataset 오브젝트를 바인딩합니다.



- 4 Form 오브젝트에 onbindingvaluechanged 이벤트 함수를 아래와 같이 추가합니다.

Grid 컴포넌트에서 특정 Row를 선택하면 Dataset 오브젝트의 상태가 변경되고 바인딩된 컴포넌트에 전파됩니다. 바인딩된 컴포넌트의 속성값이 모두 반영되면 onbindingvaluechanged 이벤트가 발생합니다.

```
this.sample_checkbox_03_onbindingvaluechanged = function(obj:nexacro.Form,e:nexacro.
BindingValueChangedEventInfo)
{
    this.TextArea00.set_value("CheckBox00: "+this.CheckBox00.isChecked()+" / "+this.
CheckBox00.value);
    this.TextArea00.insertText("\nCheckBox01: "+this.CheckBox01.isChecked()+" / "+this.
CheckBox01.value);
    this.TextArea00.insertText("\nCheckBox02: "+this.CheckBox02.isChecked()+" / "+this.
CheckBox02.value);
};
```

- 5 QuickView(Ctrl + F6)로 실행합니다. 값이 변경될 때 CheckBox 컴포넌트가 어떤 방식으로 체크 상태로 변경되는지 확인합니다.

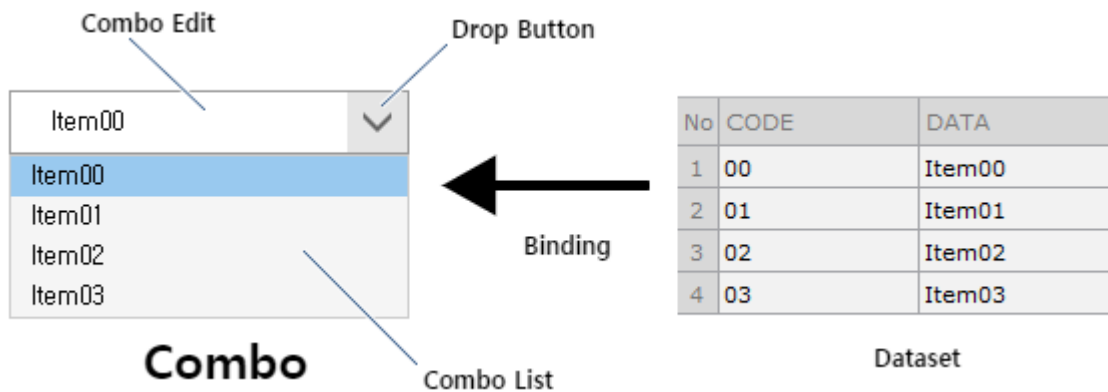
16.

Combo

16.1 Combo 소개

Combo는 여러 아이템 목록 중에서 한 개 값을 선택하기 위한 용도로 사용하는 컴포넌트입니다.

텍스트를 입/출력할 수 있는 콤보 에디트가 있고 그 옆에 드롭 버튼이 배치되어 있어 버튼을 클릭하면 Combo에 바인딩되어 있는 Dataset의 아이템 목록이 리스트박스 형태로 출력되는 구조를 갖습니다.

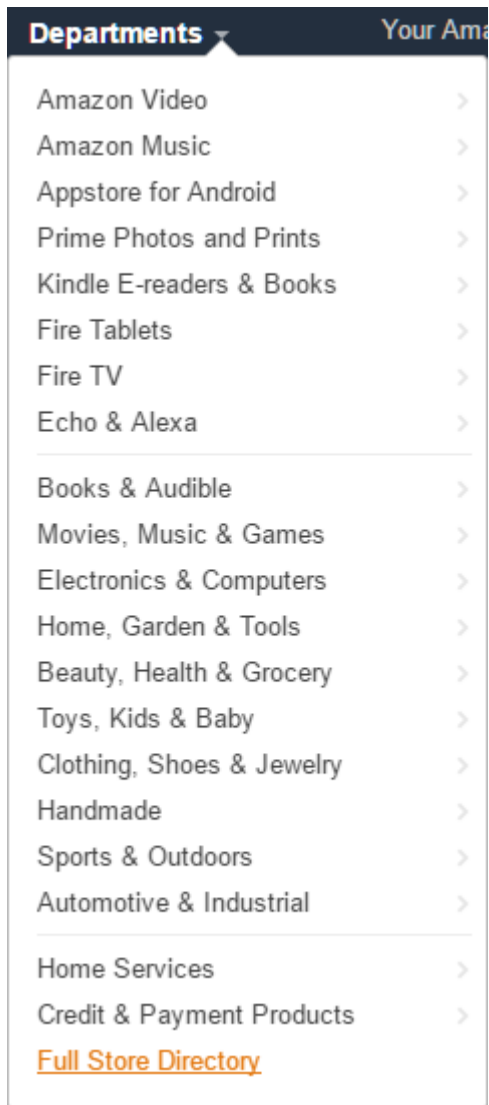


Combo는 아이템의 개수가 많아 원하는 아이템을 찾기 힘든 경우를 위해 네 가지 검색 타입을 제공하는데 type 속성을 설정하는 것만으로 손쉽게 검색 기능을 사용할 수 있습니다. dropdown은 검색어 입력을 받지 않고 오직 아이템 목록을 선택만 할 수 있고, search와 filter는 콤보 에디트에 입력된 검색어로 시작하는 아이템만을 목록으로 보여줍니다. 그리고 filterlike는 콤보 에디트에 입력된 검색어가 들어간 모든 아이템을 목록으로 보여줍니다. 개발자는 상황에 맞는 적절한 Combo 타입을 사용하여 사용자의 편의성을 높일 수 있습니다.

Combo는 Dataset을 바인딩해 동적으로 아이템 목록을 구성할 수 있습니다. 또한, 아이템이 항상 고정적인 경우에는 InnerDataset 편집기를 사용해 개발자가 직접 아이템 목록을 입력하는 것도 가능합니다.

16.1.1 사용 예

다음은 국가별 언어 설정 기능을 Combo를 사용해 구현한 예를 보여줍니다. 사용자는 선택할 수 있는 아이템 목록을 살펴보고 그중에서 하나의 아이템만을 선택할 수 있습니다.



16.1.2 Combo 만들기



① Combo 컴포넌트 생성하기

툴바에서 Combo  를 선택한 후, 폼 위에 적당한 크기로 드래그하여 생성합니다. 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

② 쿼뷰로 확인하기

쿼뷰(Ctrl + F6)를 실행하여 결과를 확인합니다.

아무 속성도 설정하지 않은 Combo 컴포넌트는 아무 값도 출력되지 않고 검색어도 입력할 수 없습니다.

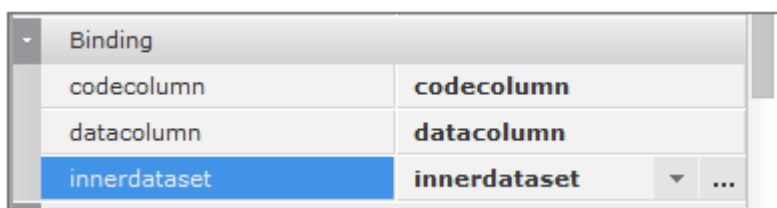
16.2 Combo 구성하기 - 고정 아이템 목록

일반적으로 Combo를 구성할 때 아이템 목록을 데이터베이스로부터 로컬의 Dataset으로 받아와 사용합니다. 그러나 아이템이 몇 개 안되고 값이 항상 같다면 InnerDataset 편집기를 사용하여 간단히 아이템 목록을 구성할 수도 있습니다. 예를 들어, 성별, 날짜, 주소의 지역 등과 같은 데이터는 변환 일이 거의 없으므로 InnerDataset 편집기로 간단히 아이템을 목록을 구성하는 게 편합니다.

InnerDataset 편집기는 사용자가 별도로 데이터셋을 생성하는 번거로움 없이 내부적으로 innerdataset이라는 Dataset을 자동 생성해 줍니다. 아래는 InnerDataset 편집기로 아이템 목록을 구성한 후의 XFDL 코드로 자동으로 생성된 innerdataset과 codecolumn, datacolumn을 확인할 수 있습니다. 이 속성에 대한 설명은 [예제에서 사용한 핵심 기능](#)을 참조하시기 바랍니다.

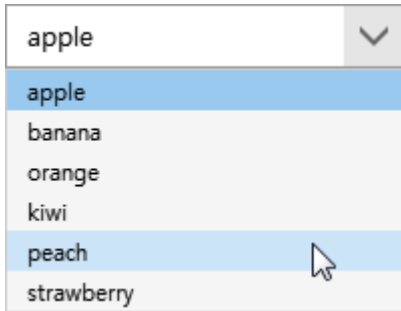
```
<Combo id="Combo00" taborder="0" text="male" left="32" top="40" width="200" height="32" right="
"" bottom="" leftbase="" topbase="" bottombase="" rightbase="" widthbase="" heightbase=""
codecolumn="codecolumn" datacolumn="datacolumn" value="m" index="0">
  <Dataset id="innerdataset">
    <ColumnInfo>
      <Column id="codecolumn" size="256"/>
      <Column id="datacolumn" size="256"/>
    </ColumnInfo>
    <Rows>
      <Row>
        <Col id="codecolumn">00</Col>
        .
        .
      </Row>
    </Rows>
  </Dataset>
</Combo>
```

아래 그림은 InnerDataset 편집기로 Combo를 구성한 후 자동으로 설정된 innerdataset과 codecolumn, datacolumn 속성의 설정 상태를 보여줍니다. 툴에서 자동으로 처리되므로 개발자가 별도로 설정할 필요는 없습니다.



16.2.1 예제

다음은 6개 아이템 목록을 갖는 Combo 예제입니다. Combo의 드롭 버튼을 클릭하면 아이템 목록이 나오며 그중에서 하나의 아이템을 선택할 수 있습니다. 아이템을 선택하면 콤보 에디트에 선택한 아이템이 반영됩니다.



16.2.2 예제에서 사용한 핵심 기능

codecolumn

Combo의 아이템 항목에서 코드값으로 사용할 컬럼을 설정하는 속성입니다. 코드는 Combo 내부적으로 아이템을 처리할 때 사용하는 일종의 ID 값입니다.

datacolumn

Combo의 아이템 항목에서 데이터값으로 사용할 컬럼을 설정하는 속성입니다. 데이터는 Combo에 출력되어 사용자에게 보일 아이템 정보입니다.

innerdataset

Combo에 바인딩(아이템 항목을 구성)할 Dataset의 ID를 설정하는 속성입니다.

16.2.3 예제 구현 방법

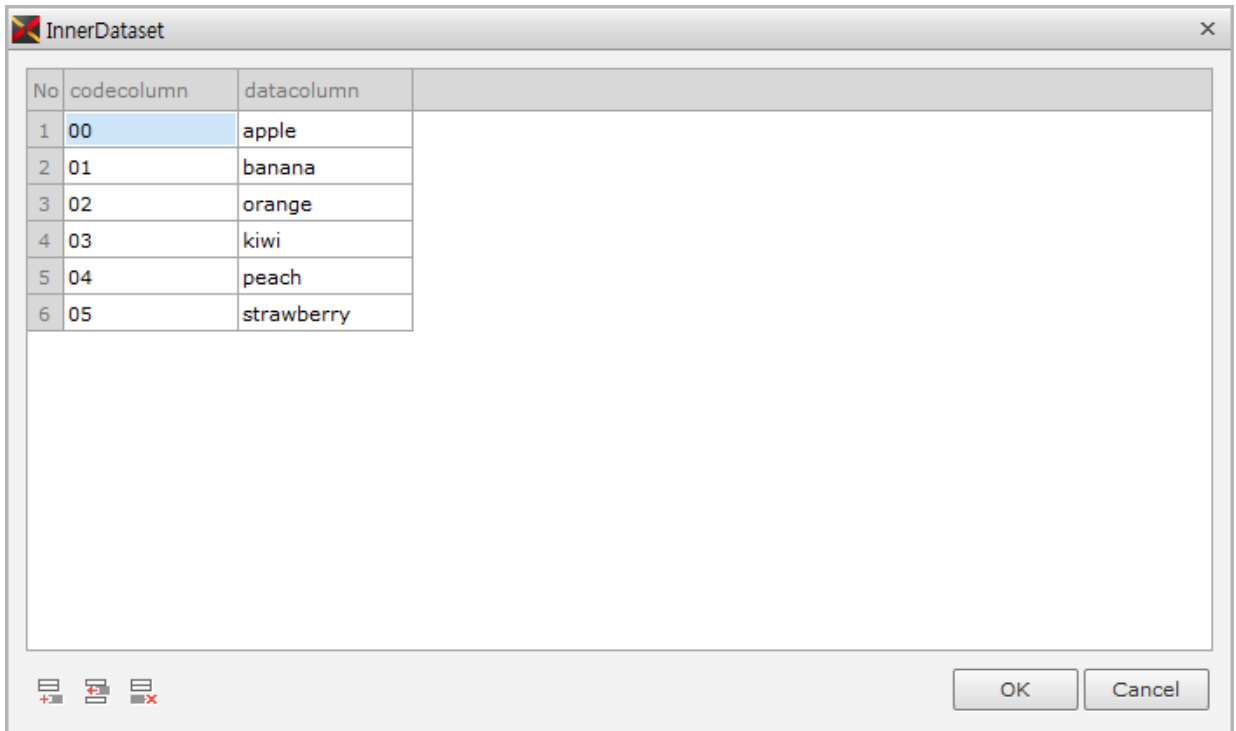
Dataset 바인딩없이 InnerDataset 편집기를 사용하여 간단히 아이템 목록을 구성하는 방법에 대해 설명합니다.

① Combo 생성하기

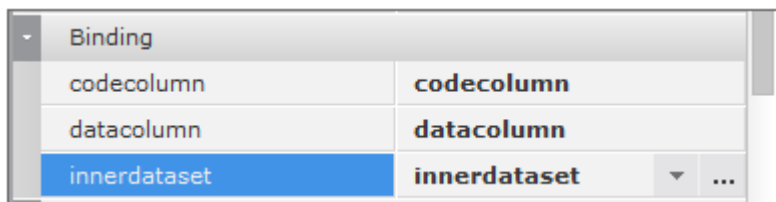
툴바에서 Combo 를 선택한 후, 폼 위에서 적당한 크기로 드래그하여 생성합니다. 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

② InnerDataset 편집기로 아이템 목록 생성하기

단계 1에서 생성한 Combo를 선택한 후 Properties 창에서 innerdataset 속성을 찾습니다. innerdataset 속성을 선택하면 아래와 같이 속성 옆에 [...] 버튼이 나타나고 그 버튼을 클릭하면 다음과 같이 InnerDataset 편집기가 나타납니다. 편집기에는 아래와 같이 입력하고 [OK] 버튼을 클릭하면 바인딩이 완료됩니다.



Properties 창에서 아래와 같이 codecolumn, datacolumn, innerdataset 속성에 자동으로 값이 설정되었는지 확인합니다.



InnerDataset 편집기를 사용하면 내부적으로 데이터셋을 생성하고 Combo에 바인딩시켜주는 작업이 자동으로 수행됩니다.

③ Combo 초기값 설정하기

Combo 동작시 최초로 선택되어 보이는 초기값은 index 속성으로 설정합니다. index에 설정할 값은 현재 바인딩되어 있는 Dataset의 아이템 순서대로 앞에서부터 0, 1, 2...의 순서로 정의됩니다.

예제에서는 apple을 초기값으로 사용하기 위해 Properties 창에서 index를 0으로 설정합니다. text와 value 속성은 index 값을 설정하면 자동으로 설정됩니다.

4 쿼뷰로 확인하기

성공적으로 Combo가 만들어졌는지 QuickView(Ctrl + F6)로 확인합니다.

콤보 에디트에 apple이 초기값으로 선택되어 있는지 확인하고 드롭 버튼을 눌러 목록에서 다른 아이템을 선택하여 해당 값으로 값이 변경되는지 확인합니다.

16.3 Combo 구성하기 - 동적 아이템 목록

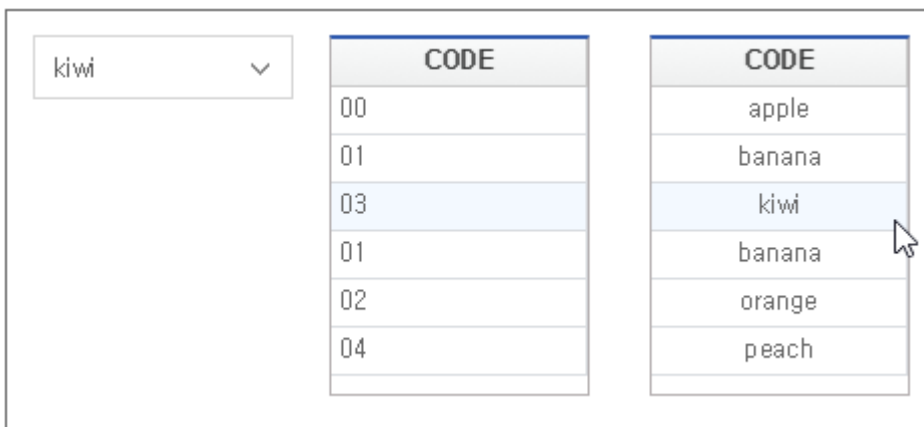
일반적으로 Combo를 구성할 때 아이템 목록을 데이터베이스로부터 가져와 사용합니다. 이는 Combo의 아이템을 데이터베이스에서 관리하여 아이템 정보가 변경되어도 화면을 수정할 필요 없이 아이템 목록을 동적으로 구성할 수 있기 때문입니다.

데이터베이스로부터 데이터를 받아 Combo의 아이템 목록을 구성하기 위해서는 Dataset이 필요합니다. 이번 장에서는 Combo에 Dataset을 바인딩시켜 동적으로 아이템을 구성하는 방법에 대해 설명합니다.

16.3.1 예제

다음은 6개 아이템 목록을 갖는 Combo 예제입니다. Combo의 드롭 버튼을 클릭하면 아이템 목록이 나오며 그중에서 하나의 아이템을 선택할 수 있습니다. 아이템을 선택하면 콤보 에디트에 선택한 아이템이 반영됩니다.

Dataset 컴포넌트를 바인딩해서 목록을 보여주고 있기 때문에 Grid 컴포넌트를 표시할 때 같은 데이터를 공유할 수 있습니다. 오른쪽에 배치한 Grid 컴포넌트는 00부터 05까지 코드만 가지고 있는데 Combo 컴포넌트에 바인딩된 Dataset의 정보를 가지고 텍스트 형태로 데이터를 보여줍니다. 또한 Combo 컴포넌트의 value 속성값을 Grid 컴포넌트에 바인딩된 Dataset 오브젝트와 바인딩해서 Grid 컴포넌트에서 선택한 항목을 Combo 컴포넌트에서 보여주고 Combo 컴포넌트에서 선택한 값을 Grid 컴포넌트에 업데이트합니다.





16.3.2 예제에서 사용한 핵심 기능

codecolumn

Combo의 아이템 항목에서 코드값으로 사용할 컬럼을 설정하는 속성입니다. 코드는 Combo 내부적으로 아이템을 처리할 때 사용하는 일종의 ID 값입니다.

datacolumn

Combo의 아이템 항목에서 데이터값으로 사용할 컬럼을 설정하는 속성입니다. 데이터는 Combo에 출력되어 사용자에게 보일 아이템 정보입니다.

innerdataset

Combo에 바인딩(아이템 항목을 구성)할 Dataset의 ID를 설정하는 속성입니다.

16.3.3 예제 구현 방법

① Combo 생성하기

툴바에서 Combo 를 선택한 후, 폼 위에서 적당한 크기로 드래그하여 생성합니다. 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

② Dataset 생성하기

Combo에 출력할 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset 을 선택한 후, 폼 위 적당한 공간을 클릭하여 Dataset을 생성합니다. Dataset은 형태가 없는 논리적인 오브젝트입니다. 따라서 다른 컴포넌트와 달리 폼에 표시되지 않고 Invisible Object 영역에 표시됩니다.
2. 생성한 Dataset에 아이템 정보를 입력합니다. Invisible Objects 영역에 있는 Dataset00을 더블 클릭하면 Dataset 편집기가 뜨는데 아래와 같이 입력합니다. 입력이 완료되면 Dataset 편집기 창을 닫습니다. 창을 닫으면 작업 내용은 자동으로 저장됩니다.

Dataset Editor

↑

×

There are no items to show.

Columns

+

+

+

No	id	type	size	prop	sumtext	description	
1	CODE	STRING	256				
2	DATA	STRING	256				

Rows

+

+

+

No	CODE	DATA	
1	00	apple	
2	01	banana	
3	02	orange	
4	03	kiwi	
5	04	peach	
6	05	strawberry	

Dataset

|

Source

|

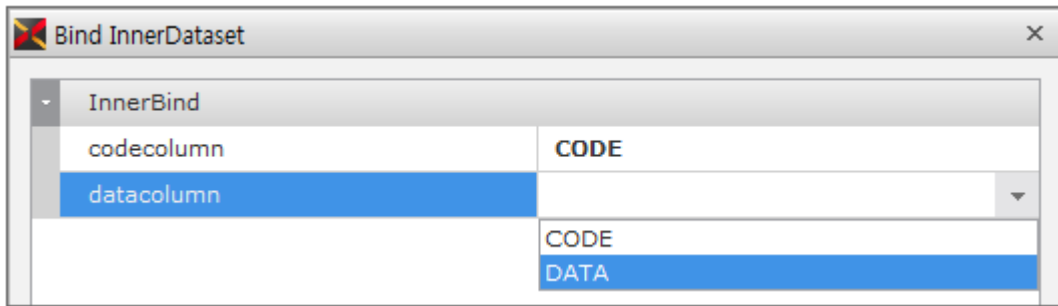


일반적으로 데이터셋에 데이터를 직접 입력하는 경우는 많지 않습니다. 트랜잭션을 통해 데이터베이스로부터 데이터를 받아오는 작업이 필요하지만 본 예제에서는 Combo를 구성하는 방법에 대해서만 다루기 때문에 그 부분은 처리된 걸로 간주하고 생략합니다.

3 Combo에 데이터셋 바인딩하기

데이터셋을 Combo에 바인딩합니다. 바인딩을 위해서는 Combo의 `codecolumn`, `datacolumn` 속성에 알맞은 데이터셋의 컬럼을 설정해야 합니다.

Invisible Objects 영역에 있는 Dataset 오브젝트(ds_combo)을 폼에 배치한 Combo 컴포넌트로 드래그 앤드롭하고 [Bind InnerDataset "ds_combo"]을 선택하면 아래와 같이 Bind InnerDataset 창이 뜹니다.



codecolumn에 'CODE'를 datacolumn에 'DATA'를 입력 혹은 선택하고 [OK] 버튼을 클릭하면 바인딩이 완료됩니다. 이 예제는 설정에 대한 이해가 쉽도록 데이터셋의 컬럼 이름을 CODE와 DATA로 하였습니다.

4 Combo 초기값 설정하기

Combo 동작 시 최초로 선택되어 보이는 초기값은 index 속성으로 설정합니다. index에 설정할 값은 현재 바인딩되어 있는 데이터셋의 아이템 순서대로 앞에서부터 0, 1, 2...의 순서로 정의됩니다.

예제에서는 apple을 초기값으로 사용하기 위해 index를 '0'으로 설정합니다. text와 value 속성은 index 값을 설정하면 자동으로 설정됩니다.

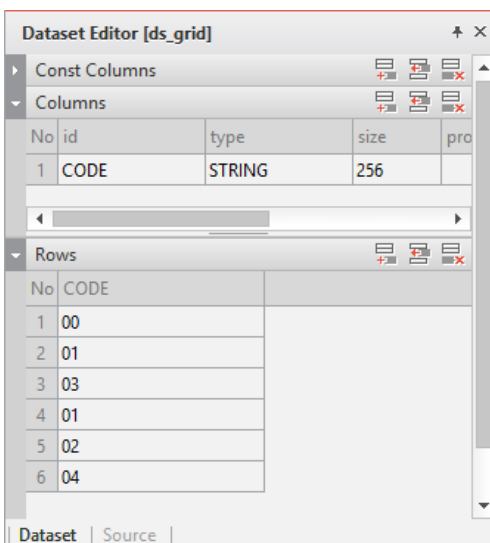
5 쿼뷰로 확인하기

성공적으로 Combo가 만들어졌는지 쿼뷰(Ctrl + F6)로 확인합니다.

콤보 에디트에 apple이 초기값으로 선택되어 있는지 확인하고 드롭 버튼을 눌러 목록에서 다른 아이템을 선택하여 해당 값으로 값이 변경되는지 확인합니다.

6 Grid 컴포넌트 배치하고 Dataset 오브젝트 추가하기

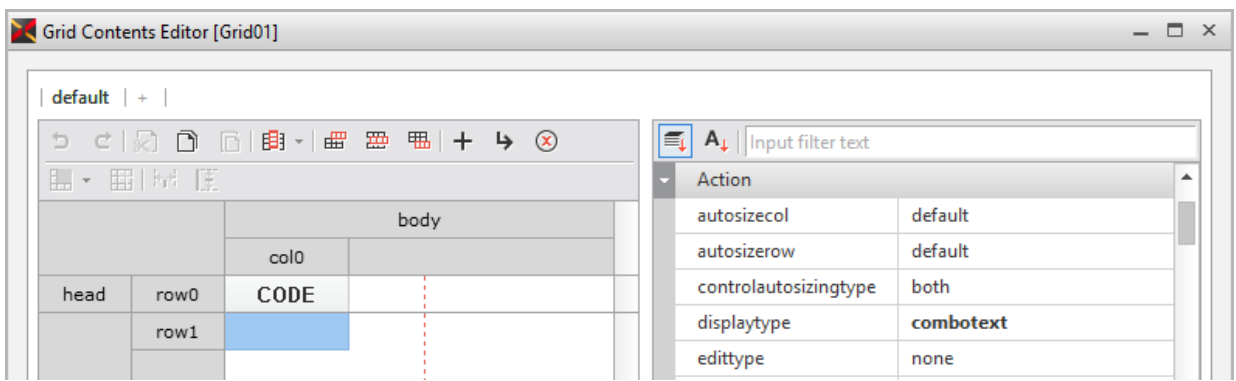
Combo 컴포넌트 옆에 예제와 같이 2개의 Grid 컴포넌트를 배치합니다. 그리고 Dataset 오브젝트를 하나 더 생성합니다. Dataset 오브젝트 생성 정보는 아래와 같습니다. 데이터는 00~05 사이의 값을 임의로 입력합니다.



7 Grid 컴포넌트에 생성된 Dataset 오브젝트(ds_grid)를 바인딩하기

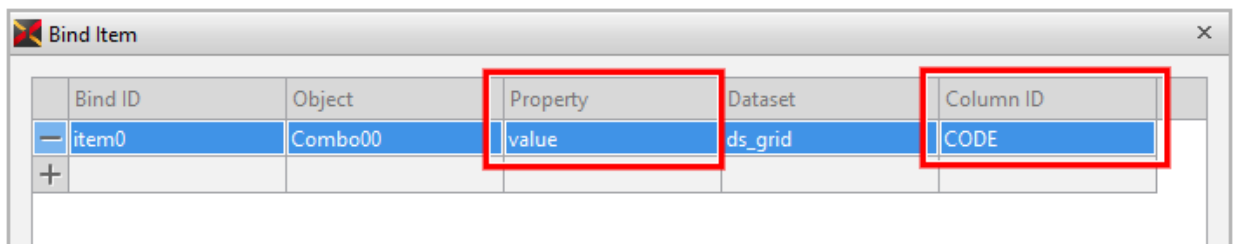
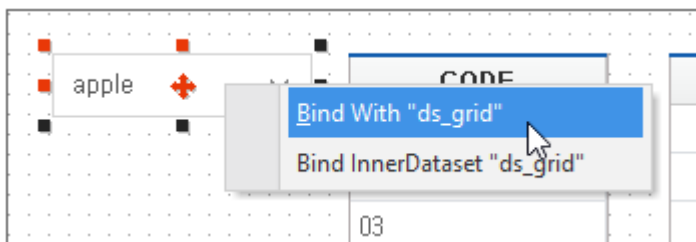
Dataset 오브젝트를 마우스 왼쪽 버튼으로 누른 상태로 드래그해서 Grid 컴포넌트 위로 가져가면 바인딩 작업이 처리됩니다. 첫번째 Grid 컴포넌트는 바인딩까지만 처리하고 두번째 Grid 컴포넌트는 해당 CODE 값을 텍스트로 보여주도록 수정합니다. Grid 컴포넌트를 더블클릭해서 Grid Contents Editor를 실행하고 아래 속성값을 수정합니다.

속성	속성값
displaytype	combotext
combodataset	ds_combo
combocodecol	CODE
combodatacol	DATA



8 Combo 컴포넌트에 Dataset 오브젝트(ds_grid)를 바인딩하기

Invisible Objects 영역에 있는 Dataset 오브젝트(ds_grid)을 폼에 배치한 Combo 컴포넌트로 드래그 앤 드롭하고 [Bind With "ds_grid"]을 선택하면 아래와 같이 Bind Item 창이 뜹니다. Bind Item 창에서 속성 항목과 컬럼 ID 항목을 수정합니다.



9 쿼뷰로 확인하기

Grid 컴포넌트가 예제 이미지처럼 코드와 텍스트 모두 표현되는지 확인하고 Grid 컴포넌트의 항목이나 Combo 컴포넌트의 항목을 변경했을 때 어떻게 표현되는지 확인합니다.

16.4 키워드 검색을 통하여 관련 아이템 추출하기

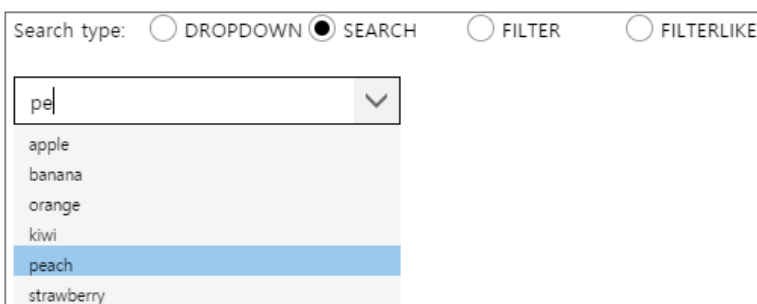
Combo의 type 속성을 설정하면 아이템을 직접 선택하는 방법 외에 검색어를 입력하여 목록에서 아이템을 검색하고, 선택하는 기능을 구현할 수 있습니다. 예를 들어, 속성값을 'filter'나 'filterlike'로 설정하면 포털 사이트에서 제공하는 자동완성 기능과 유사한 효과를 보여줄 수 있습니다.



16.4.1 예제

다음은 라디오 버튼으로 Combo의 type 속성 중 하나를 선택하면 해당 타입으로 Combo가 동작하는 예제입니다. 설정할 수 있는 타입은 dropdown, search, filter, filterlike 입니다.

- DROPDOWN은 Combo가 포커스를 받으면 모든 아이템을 콤보 리스트에 표시합니다. 이때는 콤보 에디트에 문자열을 입력할 수 없으며 오직 마우스를 이용한 선택만 가능합니다.
- SEARCH는 콤보 에디트가 활성화되고 문자열을 입력할 수 있습니다. 문자열을 입력하면 콤보 리스트에 모든 아이템을 표시하고 입력한 문자열로 시작하는 첫 번째 아이템으로 포커스가 이동합니다.
- FILTER는 콤보 에디트가 활성화되어 문자열을 입력할 수 있습니다. 문자열을 입력하면 콤보 리스트에 문자열로 시작하는 아이템만 표시하고 그 중 첫 번째 아이템으로 포커스가 이동합니다.
- FILTERLIKE는 SQL의 like 문과 같은 기능으로 입력된 문자열이 들어가는 모든 아이템을 콤보 리스트에 표시합니다. 예를 들어, 'p'를 입력하면 'p'가 들어가 있는 apple과 peach가 목록에 표시되고 첫 번째 아이템인 apple로 포커스가 이동합니다.





16.4.2 예제에서 사용한 핵심 기능

type

Combo의 검색 기능을 설정하는 속성입니다. 설정값에 따라 아이템 목록을 보여주는 방법을 설정할 수 있습니다. 검색 기능은 search, filter, filterlike로 설정했을 경우에 사용이 가능합니다.

dropdown	(default) Combo가 포커스를 받으면 전체 아이템을 콤보 리스트에 표시합니다. 콤보 에디트에 문자열을 입력할 수 없습니다.
search	전체 아이템을 콤보 리스트에 표시하고 입력한 문자열로 시작하는 첫 번째 아이템으로 포커스가 이동합니다.
filter	입력한 문자열로 시작하는 아이템을 필터링하여 해당하는 아이템만 콤보 리스트에 표시하고 그 중 첫 번째 아이템으로 포커스가 이동합니다. 검색 조건을 만족하는 아이템이 없으면 콤보 리스트를 표시하지 않습니다.
filterlike	입력한 문자열을 포함하는 아이템을 필터링하여 해당하는 아이템만 콤보 리스트에 표시합니다. 검색 조건을 만족하는 아이템이 없으면 콤보 리스트를 표시하지 않습니다.

onitemchanged

Combo에서 값이 변경된 후 발생하는 이벤트입니다. 여기서 값이란 Combo의 index, text, value 속성의 값을 말하며 이 속성이 사용자의 마우스, 키보드 조작으로 변경되는 경우에 이벤트가 발생합니다. 스크립트에 의해 값이 변경되는 경우에는 이벤트가 발생하지 않습니다.

16.4.3 예제 구현 방법

① Combo 생성하기

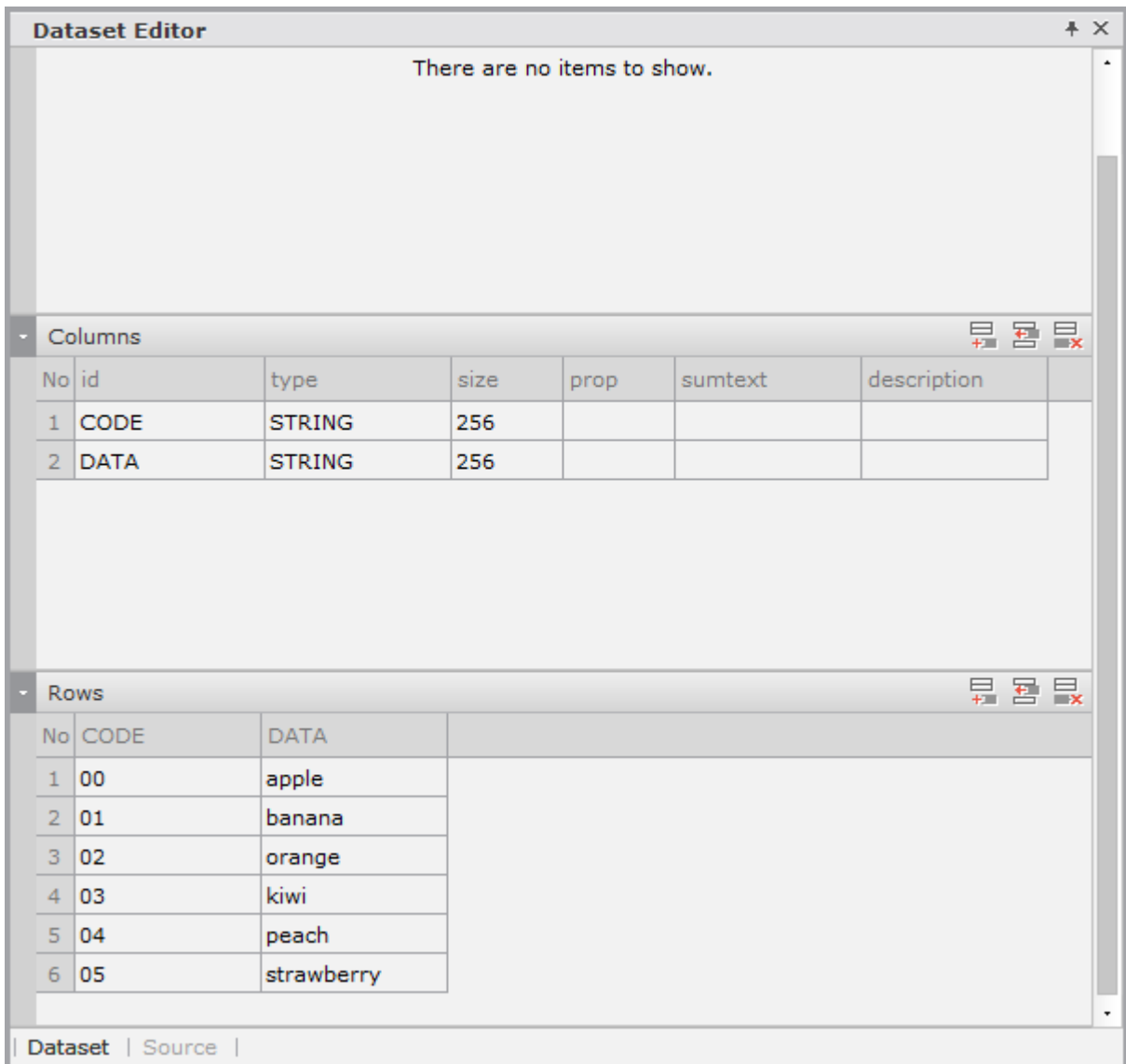
툴바에서 Combo  를 선택한 후, 폼 위에서 적당한 크기로 드래그하여 생성합니다. 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

② Dataset 생성하기

Combo에 출력할 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간을 클릭하여 Dataset을 생성합니다. Dataset은 형태가 없는 논리적인 오브젝트입니다. 따라서 다른 컴포넌트와 달리 폼에 표시되지 않고 Invisible Object 영역에 표시됩니다.
2. Invisible Object 창에서 데이터셋을 선택한 후 Properties 창에서 id를 'ds_combo'로 변경합니다.

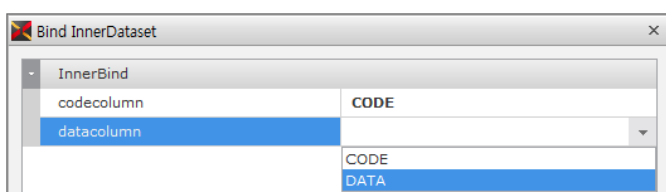
3. 그다음 생성한 데이터셋에 아이템 정보를 입력합니다. Invisible Objects 영역에 있는 ds_combo를 더블 클릭하면 Dataset 편집기가 뜨는데 아래와 같이 입력합니다. 입력이 완료되면 Dataset 편집기 창을 닫습니다. 창을 닫으면 작업 내용은 자동으로 저장됩니다.



3 Combo에 Dataset 바인딩하기

Dataset을 Combo에 바인딩합니다. 바인딩을 위해서는 Combo의 codecolumn, datacolumn 속성에 알맞은 Dataset의 컬럼을 설정해야 합니다.

Invisible Objects 영역에 있는 ds_combo를 폼에 배치한 Combo 컴포넌트로 드래그 앤 드롭하고 [Bind InnerDataset "ds_combo"]를 선택하면 아래와 같이 Bind InnerDataset 창이 뜹니다.

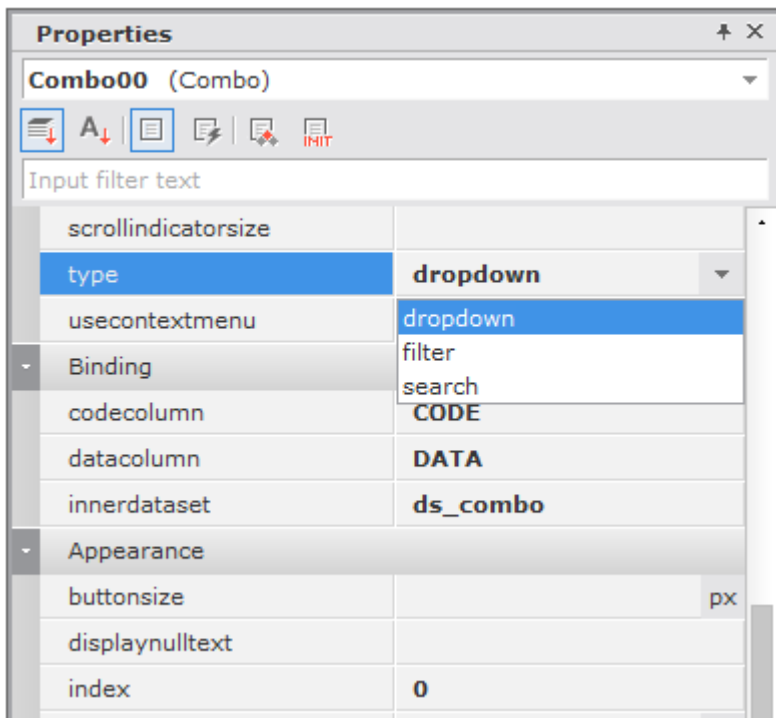


codecolumn에 'CODE'를 datacolumn에 'DATA'를 입력 혹은 선택하고 [OK]를 클릭하면 바인딩이 완료됩니다.

4 Combo 초기값 설정하기

Combo의 Properties 영역에서 type 속성을 'dropdown'으로 설정합니다. 아무 값도 설정하지 않으면 dropdown으로 동작합니다.

초기값을 설정하기 위해 index 속성은 '0'으로 설정합니다. '0'으로 설정하면 Dataset의 첫번째 값인 apple이 초기값으로 설정됩니다.



5 라디오 메뉴 만들기

Combo의 4가지 타입인 dropdown, search, filter, filterlike를 선택하여 설정할 수 있도록 라디오 메뉴를 만듭니다.

1. 툴바에서 Radio  를 선택한 후, 폼 위에서 적당한 크기로 드래그하여 생성합니다. 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.
2. 생성한 Radio 컴포넌트를 선택하고 마우스 오른쪽 버튼을 눌러 컨텍스트 메뉴에서 [Bind Innerdataset > Innerdataset Editor] 메뉴를 선택합니다. 그러면 InnerDataset 편집기가 나오는데 아래와 같이 입력한 후 [OK] 버튼을 클릭합니다.

No	codecolumn	datacolumn
1	00	DROPDOWN
2	01	SEARCH

No	codecolumn	datacolumn
3	02	FILTER
4	03	FILTERLIKE

6 라디오 속성 설정하기

예제의 라디오 메뉴와 같이 가로 방향으로 아이템을 나열하기 위해 `direction` 속성을 'vertical'로 설정합니다. 그리고 초기값을 DROPDOWN으로 하기 위해 `index` 속성은 '0'으로 설정합니다.

7 라디오 메뉴 선택 기능 구현하기

라디오 버튼을 사용해 Combo의 타입을 설정합니다. 사용자가 라디오 버튼으로 아이템을 선택할 때 Combo 타입의 설정이 변경되도록 `onitemchanged` 이벤트 함수를 등록하고 스크립트를 작성합니다.

```
/* 라디오 버튼의 onitemchanged 이벤트 함수 */

this.Radio00_onitemchanged = function(obj:Radio,e:ItemChangeEventInfo)
{
    switch(e.postindex)
    {
        case 0:    /* DROPDOWN */
            this.Combo00.set_type("dropdown");
            break;
        case 1:    /* SEARCH */
            this.Combo00.set_type("search");
            break;
        case 2:    /* FILTER */
            this.Combo00.set_type("filter");
            break;
        case 3: /* FILTERLIKE */
            this.Combo00.set_type("filterlike");
            break;
        default:
            trace("UNKNOWN TYPE");
    }
};
```

8 퀵뷰로 확인하기

성공적으로 Combo가 만들어졌는지 퀵뷰(Ctrl + F6)로 확인합니다.

라디오 메뉴에서 DROPDOWN, SEARCH, FILTER, FILTERLIKE를 하나씩 선택한 후 콤보 에디트에 검색어를

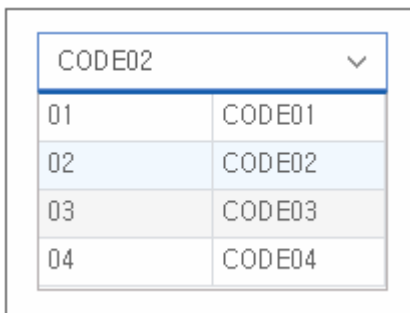
입력하며 타입에 따라 검색 결과가 목록에 표시되는지 확인합니다.

16.5 멀티 콤보 구현하기

Combo 컴포넌트에 표시되는 목록은 하나의 데이터만 표시할 수 있기 때문에 자세한 내용을 볼 수 없는 단점이 있습니다. 버튼 클릭 시 보여지는 기본 아이템 목록 대신에 Grid 컴포넌트를 활용해 좀 더 자세한 정보를 볼 수 있도록 기능을 구현할 수 있습니다.

16.5.1 예제

Combo 컴포넌트의 버튼 클릭 후 보여지는 Grid 컴포넌트 항목을 클릭하면 Combo 데이터로 입력됩니다.



16.5.2 예제에서 사용한 핵심 기능

ondropdown

Combo 컴포넌트의 dropdown을 클릭했을 때 아이템 리스트가 표시되는 시점에 발생하는 이벤트입니다. false 값을 반환하면 아이템 리스트를 표시하지 않습니다. 예제에서는 아이템 리스트를 표시하지 않고 Grid 컴포넌트의 visible 속성값을 true로 변경합니다.

16.5.3 예제 구현 방법

1 화면 배치하기

Combo 컴포넌트와 Grid 컴포넌트를 배치합니다. 예제에서는 데이터 갯수가 4개로 한정되어 있다는 전제아래 Grid 컴포넌트의 크기를 조정했습니다. Grid 컴포넌트가 처음에는 보이지 않아야 하기 때문에 visible 속성 값을 false로 변경합니다.

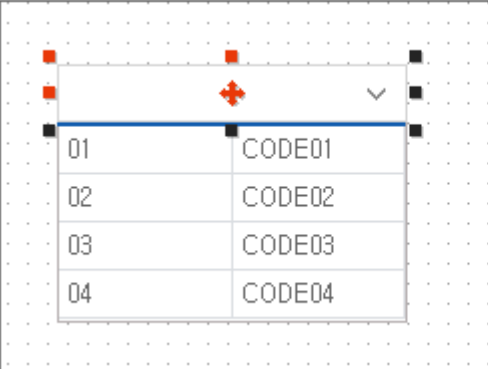
2 Dataset 생성하기

Dataset 오브젝트를 생성하고 데이터를 아래와 같이 추가합니다.

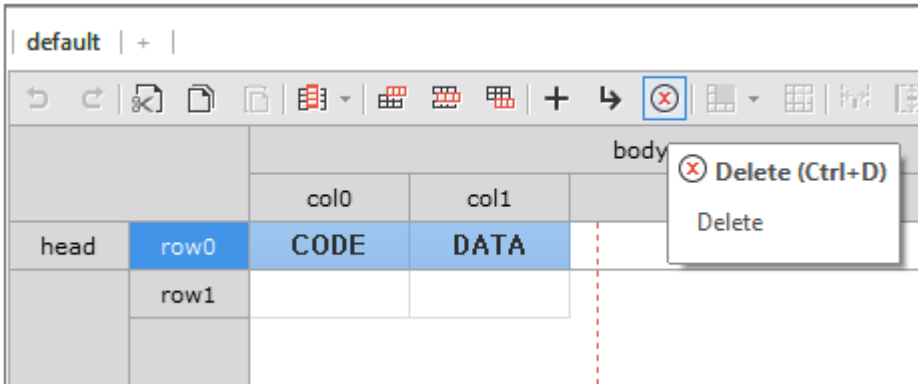
Dataset Editor [Dataset00]				
▶ Const Columns				
▼ Columns				
No	id	type	size	prop
1	CODE	STRING	256	
2	DATA	STRING	256	
▼ Rows				
No	CODE	DATA		
1	01	CODE01		
2	02	CODE02		
3	03	CODE03		
4	04	CODE04		

3 데이터 바인딩하기

Combo 컴포넌트와 Grid 컴포넌트에 생성된 Dataset 오브젝트를 바인딩합니다.

	Binding	
	codecolumn	CODE
	datacolumn	DATA
	innerdataset	Dataset00
	Appearance	
	buttonsize	
	itemheight	
	popupsize	

Grid 컴포넌트에서는 head 영역이 보이지 않도록 합니다. Grid Contents Editor를 실행한 후 head row를 선택하고 Delete 버튼을 클릭합니다.



④ ondropdown 이벤트 함수 생성하기

ondropdown 이벤트 함수에서 Grid 컴포넌트의 visible 속성값을 true로 변경해주어야 하고 아이템 리스트가 뜨지 않도록 false 값을 반환합니다.

```
this.Combo00_ondropdown = function(obj:nexacro.Combo,e:nexacro.EventInfo)
{
    this.Grid00.set_visible(true);
    return false;
};
```

⑤ oncellclick 이벤트 함수 생성하기

Grid 컴포넌트가 아이템 리스트 대신 보여지고 항목 선택 시 해당 값을 Combo 컴포넌트로 전달하고 다시 visible 속성값을 false로 변경합니다.

```
this.Grid00_oncellclick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    this.Grid00.set_visible(false);
    this.Combo00.set_value(this.Dataset00.getColumn(e.row, "CODE"));
};
```

⑥ 쿼뷰로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. dropdown을 클릭해서 아이템을 선택하고 정상적으로 Combo 컴포넌트에 반영되는지 확인합니다.

16.6 여러 항목 선택하는 콤보 구현하기

콤보 목록에서 여러 항목을 선택해야 하는 경우 사용할 수 있는 팁입니다.

16.6.1 예제

Combo 컴포넌트의 버튼 클릭 후 보여지는 Grid 컴포넌트 항목에서 원하는 항목을 체크하고 다른 컬럼을 클릭하면 Combo 데이터로 입력됩니다.

CODE02, CODE03		
☐	01	CODE01
☒	02	CODE02
☒	03	CODE03
☐	04	CODE04



16.6.2 예제에서 사용한 핵심 기능

getColumn

지정된 ROW 인덱스의 지정된 컬럼값을 반환합니다. 예제에서는 체크된 항목값을 확인하기 위한 용도로 사용하고 있습니다.

16.6.3 예제 구현 방법

① 화면 배치하기

Combo 컴포넌트와 Grid 컴포넌트를 배치합니다. 예제에서는 데이터 갯수가 4개로 한정되어 있다는 전제 아래 Grid 컴포넌트의 크기를 조정했습니다. Grid 컴포넌트가 처음에는 보이지 않아야 하기 때문에 visible 속성 값을 false로 변경합니다.

② Dataset 생성하기

Dataset 오브젝트를 생성하고 데이터를 아래와 같이 추가합니다. check 컬럼은 선택 여부를 확인하기 위한 용도로 데이터를 입력하지 않아도 괜찮습니다.

Columns			
No	id	type	size
1	check	STRING	256
2	CODE	STRING	256
3	DATA	STRING	256

Rows			
No	check	CODE	DATA
1	[Undefined]	01	CODE01
2	[Undefined]	02	CODE02
3	[Undefined]	03	CODE03
4	[Undefined]	04	CODE04

③ 데이터 바인딩하기

Grid 컴포넌트에 생성된 Dataset 오브젝트를 바인딩합니다. Grid 컴포넌트에서는 head 영역이 보이지 않도록 합니다. Grid Contents Editor를 실행한 후 head row를 선택하고 Delete 버튼을 클릭합니다.

첫번째 컬럼 Cell 오브젝트의 displaytype, edittype 속성값을 아래와 같이 수정합니다.

	body			
	col0	col1	col2	
row0	☐			

cssclass	
id	body0
Action	
autosizecol	default
autosizerow	default
controlautosizingtype	both
displaytype	checkboxcontrol
edittype	checkbox

④ ondropdown 이벤트 함수 생성하기

ondropdown 이벤트 함수에서 Grid 컴포넌트의 visible 속성값을 true로 변경해주어야 하고 아이템 리스트가 뜨지 않도록 false 값을 반환합니다.

```
this.Combo00_ondropdown = function(obj:nexacro.Combo,e:nexacro.EventInfo)
{
    this.Grid00.set_visible(true);
    return false;
}
```

};

5 oncellick 이벤트 함수 생성하기

Grid 컴포넌트가 아이템 리스트 대신 보여지고 항목 선택 시 해당 값을 Combo 컴포넌트로 전달하고 다시 visible 속성값을 false로 변경합니다. 첫번째 컬럼은 체크박스 영역이기 때문에 이벤트가 적용되지 않도록 합니다.

```
this.Grid00_oncellick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    if(e.col!=0)
    {
        this.Grid00.set_visible(false);
        this.fn_comboEdit_set();
    }
};
```

연결된 fn_comboEdit_set 함수를 생성합니다. 첫번째 컬럼이 체크되어 있는 DATA 컬럼의 텍스트를 조합해서 반환합니다.

```
this.fn_comboEdit_set = function()
{
    var str_txt = "";

    for (i=0;i<this.Dataset00.getRowCount();i++)
    {
        if (this.Dataset00.getColumn(i, "check")==1)
        {
            str_txt += this.Dataset00.getColumn(i,"DATA") + ",";
        }
    }
    this.Combo00.set_text(str_txt.substr(0, str_txt.length-1));
};
```



생성된 텍스트를 Combo 컴포넌트의 value가 아닌 text 속성값으로 설정합니다. Combo 컴포넌트의 innerdataset 속성값을 설정하지 않았기 때문에 단지 텍스트 편집 영역에 데이터가 보여질 뿐입니다. 선택된 텍스트를 활용하고자 할때는 text 속성값을 가져와서 사용합니다.

6 쿼뷰로 확인하기

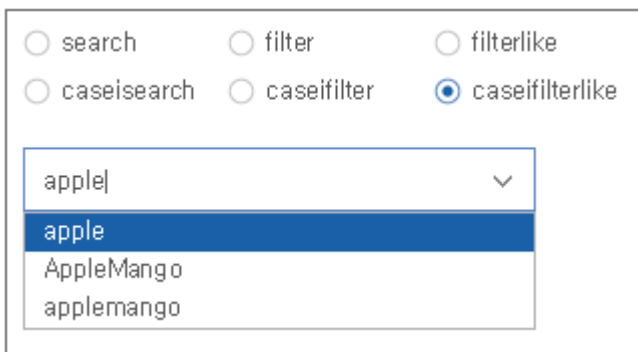
QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. dropdown을 클릭해서 아이템을 선택하고 정상적으로 Combo 컴포넌트에 반영되는지 확인합니다.

16.7 대소문자 구분 없이 검색하거나 필터링하기

Combo의 type 속성값을 설정하면 원하는 값을 찾을 수 있으나 대소문자를 구분하고 있습니다. 영어 대소문자 구분 없이 검색하고자 할 때는 type 속성값을 다르게 지정해줄 수 있습니다.

16.7.1 예제

Radio 컴포넌트에서 선택하는 항목에 따라 대소문자 구분 여부를 선택할 수 있습니다. "search", "filter", "filterlike"는 대소문자를 구분해서 입력한 항목 그대로 처리하고 "caseisearch", "caseifilter", "caseifilterlike"는 대소문자 구분없이 처리합니다.



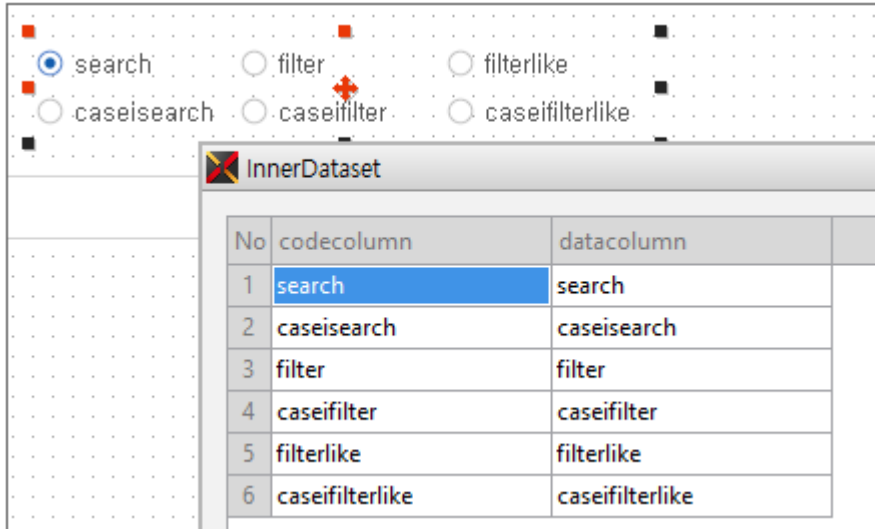
16.7.2 예제에서 사용한 핵심 기능

type

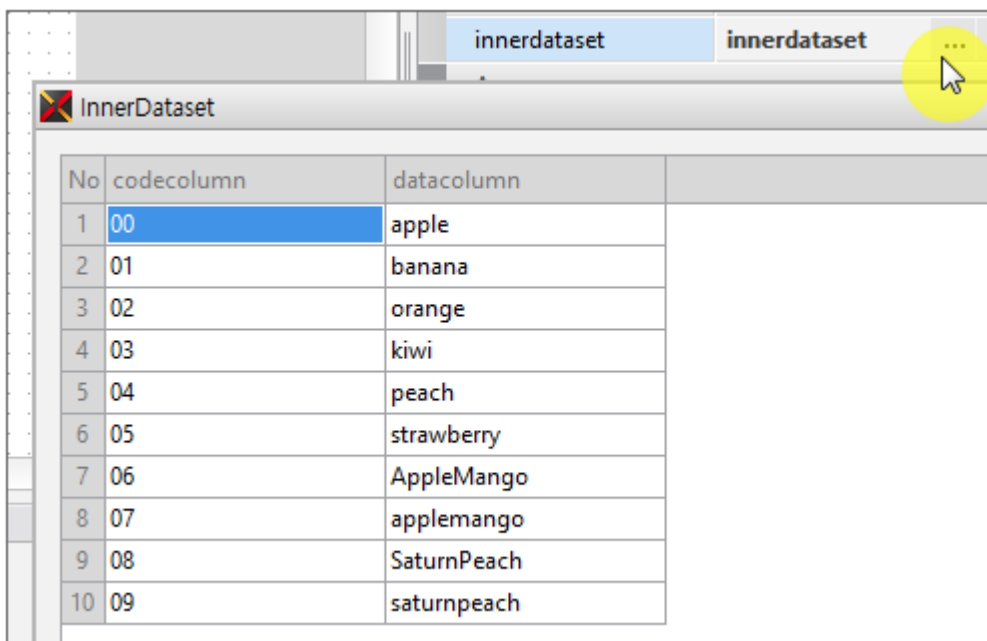
기존 속성값에서 대소문자 구분 없이 처리할 수 있는 "caseisearch", "caseifilter", "caseifilterlike" 값을 지정할 수 있습니다 ("caseisearch", "caseifilter", "caseifilterlike" 속성값은 17.0.0.1700 버전부터 지원)

16.7.3 예제 구현 방법

- ① 화면에 Radio 컴포넌트, Combo 컴포넌트를 배치합니다.
- ② Radio 컴포넌트의 innerdataset 속성값을 아래와 같이 지정합니다.



- ③ Combo 컴포넌트의 innerdataset 속성값을 아래와 같이 지정합니다.



- ④ Radio 컴포넌트의 onitemchanged 이벤트 함수를 아래와 같이 추가합니다.

```
this.Radio00_onitemchanged = function(obj:nexacro.Radio,e:nexacro.ItemChangeEventInfo)
{
    this.Combo00.set_type(e.postvalue);
};
```

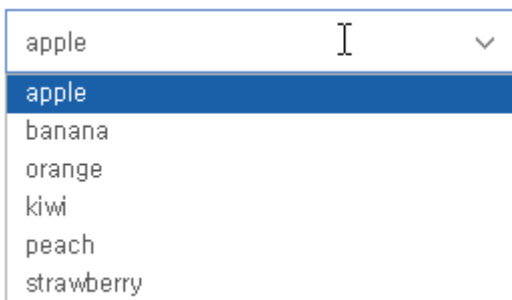
- 5 QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. 각 type 속성값에 따라 검색 결과를 확인합니다. 데이터 중에서 대소문자로 구분된 항목(AppleMango, SaturnPeach)를 검색하고 결과를 확인합니다.

16.8 마우스 오버 시 항목 펼치기

dropdown 메소드를 사용해 특정 상황에서 combolist 항목을 펼칠 수 있습니다.

16.8.1 예제

Combo 컴포넌트에 마우스를 가져가면 항목이 펼쳐집니다.



16.8.2 예제에서 사용한 핵심 기능

isDropdown

combolist가 화면에 표시되어 있는지 여부를 확인합니다.

dropdown

combolist를 화면에 표시합니다.

16.8.3 예제 구현 방법

- ① 화면에 Combo 컴포넌트를 배치합니다.
- ② Combo 컴포넌트의 innerdataset 속성값을 아래와 같이 지정합니다.

InnerDataset		
No	codecolumn	datacolumn
1	00	apple
2	01	banana
3	02	orange
4	03	kiwi
5	04	peach
6	05	strawberry

- ③ Combo 컴포넌트의 onmouseenter 이벤트 함수를 아래와 같이 추가합니다.

```
this.Combo00_onmouseenter = function(obj:nexacro.Combo,e:nexacro.MouseEventInfo)
{
    if(!obj.isDropdown()) {
        obj.dropdown();
    }
};
```



onmouseleave 이벤트를 함수를 아래와 같이 추가하면 마우스 커서가 Combo 컴포넌트를 벗어날 때 combolist가 닫힙니다.
해당 동작은 버전에 따라 변경될 수 있으며 예제 코드에서도 주석으로 막아놓았습니다.

```
/*
this.Combo00_onmouseleave = function(obj:nexacro.Combo,e:nexacro.MouseEventInfo)
{
    if(obj.isDropdown()) {
        obj.dropdown();
    }
};
```

```

    }
};
*/

```

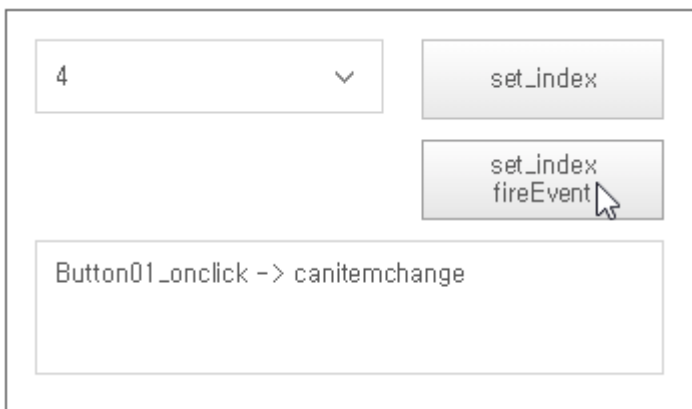
- ④ QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. 마우스 커서를 Combo 컴포넌트로 가져가면 목록이 펼쳐집니다.

16.9 스크립트에서 속성값 변경 시 이벤트 처리하기

Combo 컴포넌트의 index 속성값을 스크립트에서 수정한 경우에는 이벤트가 발생하지 않습니다. 하지만, 필요에 의해 이벤트가 발생해야 한다면 강제로 이벤트를 만들 수 있습니다.

16.9.1 예제

첫 번째 버튼 클릭 시에는 index는 변경되지만, 이벤트는 발생하지 않습니다. 두 번째 버튼 클릭 시에는 index가 변경되며 이벤트가 발생합니다.



 [sample_combo_09.xfdl](#)

16.9.2 예제에서 사용한 핵심 기능

fireEvent

설정된 EventInfo 오브젝트로 지정된 오브젝트에 대한 이벤트를 발생합니다.



도움말에서 fireEvent는 "Composite Component 와 같은 사용자 컴포넌트에서 이벤트를 발생시킬 때 사용하는 메소드입니다"라고 설명이 되어 있습니다. 때문에 일반 컴포넌트에서의 동작은 향후 기능이 변경되면서 보장하지 않을 수 있습니다.

16.9.3 예제 구현 방법

- ① 화면에 Combo 컴포넌트와 Button 컴포넌트, TextArea 컴포넌트를 배치합니다.
- ② Combo 컴포넌트의 innerdataset 속성값을 아래와 같이 지정합니다.

InnerDataset		
No	codecolumn	datacolumn
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5

- ③ 첫 번째 Button 컴포넌트의 onclick 이벤트 함수를 아래와 같이 추가합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Combo00.set_index(3);
    this.TextArea00.set_value("Button00 onclick");
};
```

- ④ 두 번째 Button 컴포넌트의 onclick 이벤트 함수를 아래와 같이 추가합니다.

```
this.Button01 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var preindex = this.Combo00.index;
    var pretext = this.Combo00.text;
    var prevalue = "";
```

```

this.Combo00.set_index(3);
var postindex = this.Combo00.index;
var posttext = this.Combo00.text;
var postvalue = "";
this.TextArea00.set_value("Button01_onclick");

var evt = new nexacro.ItemChangeEventInfo(this, "canitemchange", preindex, pretext,
prevalue, postindex, posttext, postvalue);
this.Combo00.canitemchange.fireEvent(this.Combo00, evt, true);

};

```

- ⑤ Combo 컴포넌트의 canitemchange 이벤트 함수를 아래와 같이 추가합니다.

```

this.Combo00.canitemchange = function(obj:nexacro.Combo,e:nexacro.ItemChangeEventInfo)
{
    this.TextArea00.insertText(" -> canitemchange");
};

```

- ⑥ QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. 첫 번째 버튼과 두 번째 버튼 클릭 시 이벤트 발생 여부를 확인합니다.

16.10 dropbutton 크기 변경하기

Combo 컴포넌트 오른쪽에 표시되는 작은 아이콘이 dropbutton 입니다. type 속성값에 따라 필터링이 되는 경우에도 dropbutton을 클릭하면 모든 항목을 표시합니다. 예제에서는 dropbutton 의 크기를 변경하는 방법을 살펴봅니다.

16.10.1 예제

첫 번째 버튼 클릭 시에는 크기를 20으로 지정하고 두 번째 버튼 클릭 시에는 0으로 지정해서 보이지 않도록 합니다.





16.10.2 예제에서 사용한 핵심 기능

buttonsize

dropbutton의 크기를 지정합니다. width, height 속성값을 지정해주는 것과 같은 동작인데, Combo 컴포넌트의 크기를 벗어나지 않도록 하는 등의 내부적인 처리가 포함되어 있습니다. Combo 컴포넌트의 크기보다 큰 값을 지정하면 알아서 값을 보정합니다.

16.10.3 예제 구현 방법

- ① 화면에 Combo 컴포넌트와 Button 컴포넌트를 배치합니다.
- ② Combo 컴포넌트의 innerdataset 속성값을 아래와 같이 지정합니다.

InnerDataset		
No	codecolumn	datacolumn
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5

- ③ 첫 번째 Button 컴포넌트의 onclick 이벤트 함수를 아래와 같이 추가합니다.

width와 height 속성값을 지정하고 싶다면 문자열 내에 공백문자로 구분해서 2개의 값을 지정합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Combo00.set_buttonsize("40 20");
};
```

- ④ 두 번째 Button 컴포넌트의 onclick 이벤트 함수를 아래와 같이 추가합니다.

속성값을 0으로 지정하면 화면에 표시되지 않습니다. dropbutton 컨트롤 자체를 삭제한 것은 아니기 때문에 다시 buttonsize를 수정하면 다른 크기로 표시됩니다.


```

this.Button01_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Combo00.set_buttonsize(0);
};

```

- ⑤ dropdown의 크기가 잘 보일 수 있도록 Form 오브젝트의 onload 이벤트 함수에서 배경색을 지정합니다.

```

this.sample_combo_10_onload = function(obj:nexacro.Form,e:nexacro.LoadEventInfo)
{
    this.Combo00.dropdown.set_background("cornflowerblue");
};

```

- ⑥ QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. 첫 번째 버튼과 두 번째 버튼 클릭 시 이벤트 발생 여부를 확인합니다.

17.

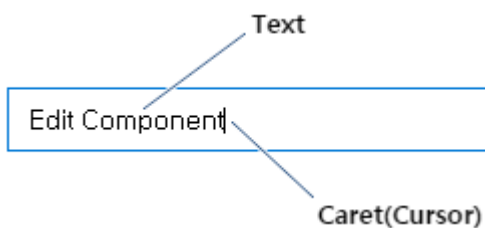
Edit

17.1 Edit 소개

Edit는 한 줄로 된 문자열을 입력받거나 출력할 때 사용하는 컴포넌트입니다. 문자열의 길이를 제한하거나 숫자나 특정 문자만 입력하게 할 수 있으며 입력 언어를 설정하는 등의 다양한 부가 기능을 속성의 설정을 통해 사용할 수 있습니다.

Edit는 한 줄의 문자열만 표현할 수 있도록 디자인되었기 때문에 줄 넘김이 안됩니다. 여러 줄 혹은 단락을 입력해야 하는 경우에는 TextArea 컴포넌트를 사용하십시오.

아래 그림은 기본 형태의 Edit 컴포넌트를 보여줍니다. Edit에 포커스가 생기면 텍스트(Text)를 캐럿(Caret) 위치부터 입력할 수 있는 구조입니다.



17.1.1 사용 예

다음은 회원가입 메뉴의 예를 보여줍니다. Edit가 사용되는 전형적인 사례로 가입 정보를 입력받을 때 Edit 컴포넌트를 사용하고 있음을 알 수 있습니다.

Sign Up
It's free and always will be.

john doe

johndoe@gmail.com
1 email or phone number

johndoe@gmail.com

●●●●●●●●●●●●●●●● 2 password

Birthday

Mar ▼ 1 ▼ 1919 ▼ Why do I need to provide my birthday?

☒ Female ☐ Male

By clicking Sign Up, you agree to our [Terms](#) and that you have read our [Data Policy](#), including our [Cookie Use](#).

Sign Up

상황에 따라 특정한 형식으로 입/출력을 하는 경우에는 Edit의 속성을 사용하여 처리할 수 있습니다. 예를 들어 비밀번호 같은 경우는 Edit의 password 속성을 설정하면 입력한 문자가 별표(*)로 표시되어 타인이 알아볼 수 없게 합니다. 또한, maxlength 속성을 설정하면 입력할 수 있는 문자의 길이를 제한할 수 있습니다.

메일주소나 전화번호와 같이 정해진 형식으로 입력해야 하는 경우에는 MaskEdit을 사용하는 것이 사용자의 입력 실수를 줄일 수 있어 좋습니다. MaskEdit 컴포넌트는 마스킹 기능을 사용할 수 있는 Edit 컴포넌트입니다.

17.1.2 Edit 만들기

Edit Component

1 Edit 컴포넌트 생성하기

툴바에서 Edit  를 선택한 후, Form 위에 적당한 크기로 드래그하여 생성합니다. 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

② 퀵뷰로 확인하기

퀵뷰(Ctrl + F6)를 실행하여 결과를 확인합니다.

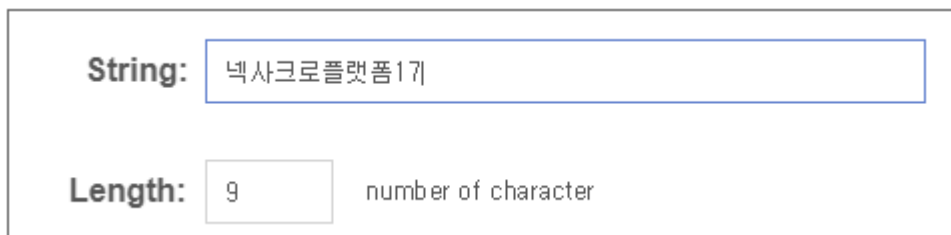
17.2 문자열 길이 구하기

문자열에 대한 가공 처리를 위해 문자열의 길이를 알아야 하거나 문자열 길이에 맞게 Edit 컴포넌트의 크기를 조절해서 보여줘야 하는 경우 Edit의 length 속성을 사용하면 손쉽게 문자열 길이를 구할 수 있습니다.

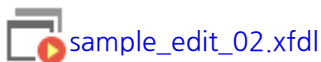
length 속성은 숫자, 공백, 영어, 한국어 등의 모든 문자를 한 문자당 1로 계산합니다.

17.2.1 예제

사용자로부터 입력받은 문자열의 길이를 실시간으로 보여주는 예제입니다. 사용자가 문자열을 입력하면 키 입력 이벤트가 발생할 때마다 길이를 계산하여 출력합니다.



length는 무조건 문자당 1씩으로 계산하므로 입력된 예제의 문자열 "넥사크로플랫폼17"의 길이는 9가 됩니다.



17.2.2 예제에서 사용한 핵심 기능

length

문자열의 길이값을 갖는 속성입니다. 문자에 상관없이 문자 하나당 무조건 1씩 계산합니다.

onkeyup

Edit에 포커스가 있는 상태에서 눌렀던 키보드가 떼어질 때 발생하는 이벤트입니다.

17.2.3 예제 구현 방법

① 화면 구성하기

Edit , Static  컴포넌트를 생성하여 예제 화면과 같이 폼에 배치합니다. 생성한 Edit 컴포넌트의 ID를 다음과 같이 설정합니다.

컴포넌트	ID
Edit00	edit_input
Edit01	edit_length

② Edit에 onkeyup 이벤트 함수 등록하기

폼에 있는 edit_input을 선택한 후 onkeyup 이벤트 함수를 추가합니다.

키보드가 눌렀다 떴어질 때마다 사용자가 입력한 문자의 수는 length 속성을 사용해 edit_length에 출력합니다.

```
this.edit_input_onkeyup = function(obj:nexacro.Edit,e:nexacro.KeyEventInfo)
{
    this.edit_length.set_value(this.edit_input.value.length);
};
```

③ 퀵뷰로 확인하기

퀵뷰(Ctrl + F6)로 실행한 후 문자열을 입력하여 실제 길이와 일치하는지 확인합니다.

17.3 자동 포커스 이동

복잡한 화면에서 사용자가 여러 내용을 입력하기 위해 컴포넌트 사이를 마우스나 탭키로 이동하며 입력하는 것은 약간 불편한 일이 아닙니다. 이럴 때 Edit 컴포넌트의 autoskip 속성을 사용하여 설정한 길이 이상 문자를 입력받으면 자동으로 다음 컴포넌트로 포커스가 이동하는 기능을 구현할 수 있습니다.

17.3.1 예제

다음은 사용자로부터 전화번호를 입력받는 예제입니다.

❶ edit_first는 3자리, ❷ edit_second와 ❸ edit_third는 4자리를 입력하면 자동으로 다음 컴포넌트로 포커스가 이동합니다.

The image shows a form with the title "Enter your phone number." It contains three input fields and an "OK" button. The first field contains the text "123", the second field contains "4567", and the third field is empty with a vertical cursor. Below each field is a label: "❶ edit_first" under the first, "❷ edit_second" under the second, and "❸ edit_third" under the third. Blue lines connect these labels to their corresponding input fields.

포커스의 이동 순서는 각 컴포넌트의 `taborder` 속성에 설정된 값에 따라 달라집니다. `taborder` 값은 '0'이상의 정수로 설정할 수 있는데 설정된 값이 작은 것이 우선순위가 높으며 값이 커질수록 우선순위가 낮아집니다. 아무 값도 설정하지 않으면 컴포넌트가 생성된 순서로 `taborder`가 자동 설정됩니다.



sample_edit_03.xfdl

17.3.2 예제에서 사용한 핵심 기능

autoskip

Edit에 입력 중인 문자열의 길이가 `maxlength` 값에 도달하면 자동으로 포커스를 이동시킬지 설정하는 속성입니다.

maxlength

Edit에 입력할 수 있는 문자열의 최대 길이를 설정하는 속성입니다. 0으로 설정 시 최대 길이를 제한하지 않습니다.

taborder

Tab 키 입력 혹은 `autoskip`으로 포커스가 이동할 때 포커스의 이동 순서를 설정하는 속성입니다. 0 이상의 정수로 설정하는데 값이 작을수록 우선순위가 높습니다.

17.3.3 예제 구현 방법

① 화면 구성하기

Edit , Button  컴포넌트를 예제의 그림과 같이 적절히 배치합니다.

② Edit 컴포넌트 속성 설정하기

각 Edit 컴포넌트별로 autoskip, maxlength, taborder 속성을 아래 표와 같이 설정합니다.

ID	autoskip 속성	maxlength 속성	taborder 속성
① edit_first	true	3	0
② edit_second	true	4	1
③ edit_third	true	4	2

③ 퀵뷰로 확인하기

퀵뷰(Ctrl + F6)로 실행한 후 앞에서부터 전화번호를 입력합니다. 자동으로 포커스가 다음 컴포넌트로 이동하는지 확인합니다.

17.4 입력 언어 변경하기

사용자는 필요에 따라 IME(Input Method Editor, 입력기) 설정을 변경하여 Edit 컴포넌트에 입력할 언어를 설정할 수 있습니다.

17.4.1 예제

다음은 Edit 컴포넌트에 언어별로 문자를 입력하는 예제입니다.

런타임이나 Internet Explorer에서 hangul과 english 항목에 입력을 할 때 한/영 변환을 하지 않아도 자동으로 IME가 전환되어 각각 한글과 영어로 입력됩니다.

japanese에 일본어를 입력하려면 사용자가 윈도우의 IME를 일본어로 변경해야 합니다. 윈도우에 일본어 IME가 설치되어 있더라도 넥사크로플랫폼 환경에서 한글 IME와 일본어 IME 간의 자동 전환은 불가능합니다.

hangul:	한글
english:	english
japanese:	ひらがな



IME 기능은 넥사크로 브라우저와 Internet Explorer 브라우저에서만 동작합니다. 그 외의 브라우저에서는 버전에 따라 동작이 다를 수 있습니다.

한글 IME와 일본어 IME간의 자동 전환은 불가능합니다. 사용자가 수동으로 윈도우의 IME를 설정해줘야 합니다.



17.4.2 예제에서 사용한 핵심 기능

imemode

Edit의 기본 입력 언어를 설정하는 속성입니다.

none	(default) 기본 입력 언어를 변경하지 않고 현재 시스템에 설정된 언어를 유지합니다.
alpha	입력언어를 영문으로 설정합니다. IME가 동아시아 입력기(한글/일본어/한자) 일 경우만 적용됩니다.
alpha,full	입력언어를 영문 전각으로 설정합니다. IME가 동아시아 입력기(한글/일본어/한자) 일 경우만 적용됩니다.
hangul	입력언어를 한글로 설정합니다. IME가 한글 입력기일 경우만 적용됩니다.
hangul,full	입력언어를 한글 전각으로 설정합니다. IME가 한글 입력기일 경우만 적용됩니다.
katakana	입력언어를 일본어 가나로 설정합니다. IME가 일본어 입력기일 경우만 적용됩니다.
katakana,full	입력언어를 일본어 가나 전각으로 설정합니다. IME가 일본어 입력기일 경우만 적용됩니다.
hiragana	입력언어를 일본어 히라가나 전각으로 설정합니다. IME가 일본어 입력기일 경우만 적용됩니다.
direct	입력언어를 일본어 직접입력으로 설정합니다. IME2002의 일본어 입력기일 경우에만 적용됩니다. IME2007과 2010일 경우는 "alpha"로 적용됩니다.

17.4.3 예제 구현 방법

① 화면 구성하기

Edit , Static  컴포넌트를 예제의 그림과 같이 적절히 배치합니다.

② Edit 속성 설정하기

각 Edit 컴포넌트의 imemode 속성을 차례로 한글은 'hangul'로, 영어는 'alpha'로, 일어는 'hiragana'로 설정합니다.

③ 퀵뷰로 확인하기

퀵뷰(Ctrl + F6)로 실행한 후 각 Edit에 언어에 맞게 입력 언어가 변경되는지 확인합니다. 일본어는 사용자가 윈도우 IME를 수동으로 변경해줘야 합니다.

17.5 정규표현식(Regular Expression)을 사용한 비밀번호 형식 체크

보안의 중요성이 커지면서 크래킹 공격에도 쉽게 뚫리지 않는 복잡한 비밀번호에 대한 요구가 많아지고 있습니다. 그에 따라 비밀번호의 복잡도가 증가하고 있고 그걸 체크하는 코드의 복잡도 역시 증가하고 있습니다. 이럴 때 정규표현식을 사용하면 단 몇 줄의 코드만으로 간단히 비밀번호의 형식을 체크할 수 있습니다.

이번 장에서는 입력받은 비밀번호가 형식에 맞는지 정규표현식을 사용해 체크하는 방법을 설명합니다. 정규표현식에 대한 자세한 설명이나 사용법은 매뉴얼의 범주를 넘어가므로 여기서는 다루지 않습니다.



정규표현식이란?

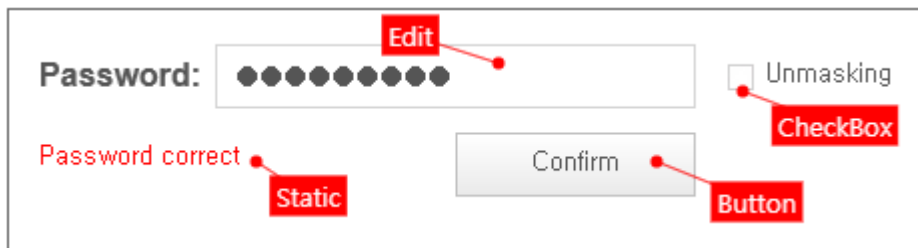
특정한 규칙을 가진 문자열의 집합을 표현할 때 사용하는 형식언어입니다. 프로그래밍 언어와 텍스트 편집기에서 문자열의 검색과 치환을 위한 용도로 사용됩니다. 문법이 일반적인 프로그래밍 언어의 형식과는 다른 데다 복잡한 내용을 간단하게 줄여서 작성하는 만큼 가독성이 좋지 못하다는 단점이 있습니다. 그러나 문자열을 다루는 데 있어 편리하고 코드량을 줄여주기 때문에 자바스크립트를 비롯한 대부분의 프로그래밍 언어에서 지원하고 있습니다.

17.5.1 예제

다음은 사용자로부터 비밀번호를 입력받아 정규표현식으로 체크하고 그 결과를 빨간색 텍스트로 알려주는 예제입니다.

비밀번호를 입력한 후 Confirm 버튼을 클릭하면 형식에 맞는지 결과를 알려줍니다. 비밀번호 형식은 알파벳 소문자 a부터 z 사이, 숫자 0부터 9 사이의 문자를 허용하고 문자열의 길이는 8부터 10까지만 허용합니다.

Unmasking 체크박스는 사용자 스스로 비밀번호를 형식에 맞게 입력했는지 확인할 수 있도록 별표로 표시되는 비밀번호를 일반 텍스트로 표시해주는 기능입니다.



 sample_edit_05.xfdl

17.5.2 예제에서 사용한 핵심 기능

password

Edit에 표시되는 문자를 알아볼 수 없게 별표(*)로 표시할지 설정하는 속성입니다.

RegExp

정규표현식 패턴 검색을 위한 정규표현식 객체를 생성하는 메소드입니다. 정규표현식을 인수로 입력하여 정규표현식 객체를 생성한 후 exec 메소드를 사용하여 문자열에 대한 패턴 검색을 수행할 수 있습니다.

```
var objRegExp = new RegExp("[a-z0-9]{8,10}", "i");    //regular expression pattern and
searching flag
var objArr = objRegExp.exec("#strString*01234");    //string for pattern searching
```

exec

정규표현식 패턴을 이용해 문자열을 검색하고 검색 결과를 배열로 반환하는 메소드입니다.

```
var objRegExp = new RegExp("a", "g");
var objArr = objRegExp.exec("abcd"); // objArr = a
```

17.5.3 예제 구현 방법

① 화면 구성하기

Static , Edit , Button , CheckBox  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. Edit의 password 속성을 'true'로 설정합니다.

② Confirm 버튼의 onclick 이벤트 함수 등록하기

사용자로부터 입력받은 비밀번호의 형식이 올바른지 체크하여 결과를 반환해주는 fn_checkPassword 함수를 호출하고 그 결과를 Static 컴포넌트의 text 속성값으로 지정해 사용자에게 알려줍니다.

fn_checkPassword 함수는 비밀번호가 올바른 형식일 경우 'Y'를 반환하고 아닐 경우에는 'N'을 반환합니다.

```
this.Button00 onclick = function(obj:Button,e:ClickEventInfo)
{
    var sRtn = fn_checkPassword(this.edit_password.value);

    if(sRtn === 'Y')
        this.Static01.set_text("Password correct");
    else
        this.Static01.set_text("Password incorrect");

};
```

③ Unmasking 체크박스의 onchanged 이벤트 함수 등록하기

사용자가 입력한 비밀번호가 올바른지 스스로 확인할 수 있게 Edit의 password 속성을 설정/해제할 수 있게 합니다.

```
this.CheckBox00 onchanged = function(obj:nexacro.CheckBox,e:nexacro.
CheckBoxChangeEventInfo)
{
    if(e.postvalue == 1)
    {
        this.edit_password.set_password("false");
    }
    else
    {
        this.edit_password.set_password("true");
    }
}
```

};

onchanged 이벤트는 체크박스의 체크 상태가 변경되면 발생하는 이벤트로 postvalue 속성에는 체크박스가 체크되면 1이 체크 해제되면 0이 저장됩니다. 따라서 e.postvalue로 체크박스의 값이 변경되었는지 확인 후 그것에 맞게 Edit의 password 속성을 설정합니다.

4 비밀번호 형식 체크 함수 작성하기

비밀번호 형식을 체크해서 그 결과를 반환하는 fn_checkPassword 함수를 작성합니다.

```

1  function fn_checkPassword(sValue)
2  {
3      var sRtn = "N";
4
5      //Regular expression pattern
6      var sRegExp = "[a-z0-9]{8,10}";
7
8      //Create regular expression object
9      var objRegExp = new RegExp(sRegExp, "i");
10
11     //Validate password
12     var objResult = objRegExp.exec(sValue);
13
14     if (objResult == null)
15     {
16         sRtn = "N";
17     }
18     else
19     {
20         if ((objResult.index == 0) && (objResult == sValue))
21             sRtn = "Y";
22         else
23             sRtn = "N";
24     }
25
26     return sRtn;
27 }
```

이 함수에서 눈여겨 봐야 할 것은 5~9줄에 있는 정규표현식 부분입니다.

1. 우선 정규표현식 문자열을 아래와 같이 작성합니다. 알파벳 소문자 a부터 z 사이, 숫자 0부터 9 사이의 문자만을 허용하고, 문자열의 길이는 8부터 10까지만 허용한다는 의미입니다.

```
var sRegExp = "[a-z0-9]{8,10}";
```

2. 정규표현식 패턴 검색을 위해 RegExp 객체를 생성합니다. flag "i"는 대, 소문자를 구별하지 않고 처리하는 옵션입니다. 따라서 "i" flag를 설정하면 알파벳 대, 소문자 입력을 모두 허용한다는 의미가 됩니다.

```
var objRegExp = new RegExp(sRegExp, "i");
```

3. exec 메소드를 호출하여 사용자로부터 입력받은 문자열이 정규표현식에 맞는지 검사합니다. exec 메소드는 정규표현식에 맞지 않으면 null을 반환합니다.

```
var objResult = objRegExp.exec(sValue);
```

5 퀵뷰로 확인하기

퀵뷰(Ctrl + F6)로 확인합니다.

비밀번호를 입력하고 Confirm 버튼을 클릭합니다. 형식에 맞으면 "password correct", 맞지 않으면 "password incorrect" 메시지를 표시합니다. 비밀번호의 형식을 바꿔가며 입력하여 정상적으로 체크되고 있는지 확인합니다.

17.6 Edit에 Dataset 바인딩하기

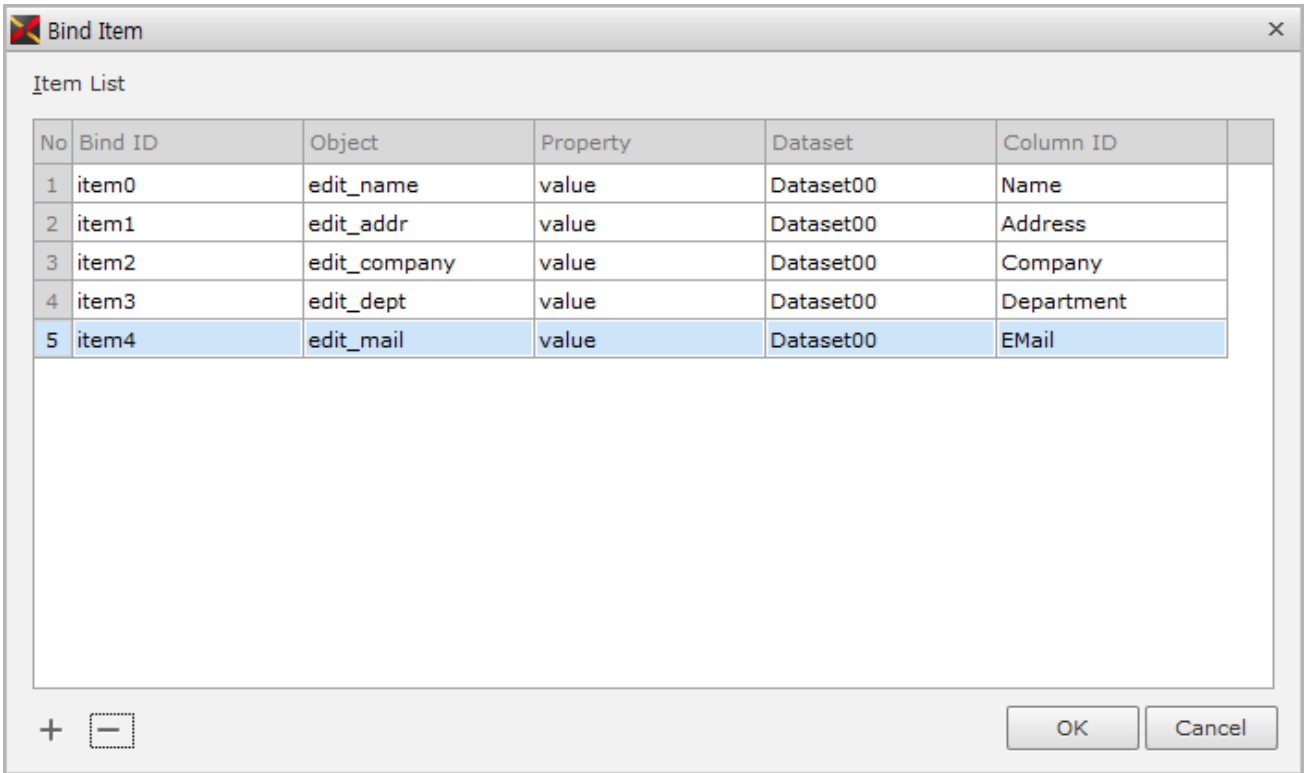
Edit 컴포넌트에 Dataset을 바인딩할 때는 Edit의 속성마다 하나의 Dataset 컬럼만을 바인딩시킬 수 있습니다. Edit 단독으로 Dataset을 바인딩해서 사용하는 경우는 많지 않고 본 장의 예제와 같이 다른 컴포넌트와 같이 복합적으로 사용하는 경우가 많습니다.

Edit에 Dataset을 바인딩하는 방법은 다음과 같습니다.

- Dataset 아이콘을 컴포넌트에 드래그 앤 드롭: 바인딩 작업을 직관적으로 할 수 있습니다. 생성된 컴포넌트에 Dataset을 바인딩 할 때 사용합니다.
- [Create Component & Bind Dataset] 메뉴: 폼 디자인 화면에서 마우스 오른쪽 버튼을 클릭하여 컨텍스트 메뉴에서 선택할 수 있습니다. 컴포넌트 생성과 바인딩을 한 번에 할 때 사용합니다.
- [Bind Dataset] 메뉴: 폼 디자인 화면에서 마우스 오른쪽 버튼을 클릭하여 컨텍스트 메뉴에서 선택할 수 있습니다. 생성된 컴포넌트에 Dataset을 바인딩 할 때 사용합니다.

어떤 방법을 사용하던 다음 그림과 같이 Bind Item 창이 뜨며 각 컴포넌트의 속성에 Dataset의 컬럼을 바인딩시키는

작업을 할 수 있습니다. 다음 그림은 각 Edit 컴포넌트에 Dataset의 컬럼을 바인딩한 예를 보여줍니다.



17.6.1 예제

다음은 Edit와 Grid 컴포넌트에 Dataset을 바인딩하여 데이터를 화면에 보여주는 예제입니다. Grid의 레코드를 선택하면 Edit에도 같은 정보가 표시됩니다.

Name	Address	Company	Department	E-Mail
john	Califonia, USA	1dollar	Pre-sales	pre-sales@1dollar.com
james	Seoul, KOREA	EBOT	Consulting	consulting@ebot.com
donald	Barcelona, Spain	EBOT	Research Institute	research@ebot.co.kr
lisa	London, UK	1dollar	Quality Assurance	quality@1dollar.co.jp

Name:

1 edit_name

Address:

2 edit_addr

Company:

3 edit_company

Department:

4 edit_dept

E-Mail:

5 edit_mail



Grid와 Edit는 같은 Dataset을 바인딩하고 있으므로 데이터를 조작 혹은 수정하면 그 결과가 Grid와 Edit에 모두 반영되어 보입니다. 예를 들어, Grid에서 레코드 선택을 변경하면 Edit에서는 자동으로 변경된 레코드의 데이터를 표시하게 됩니다. 그리고 Edit에서 값을 변경하면 Grid에도 반영됩니다.

17.6.2 예제에서 사용한 핵심 기능

없음.

17.6.3 예제 구현 방법

① 화면 구성하기

툴바에서 Grid , Static , Edit  를 각각 폼에 생성하고 예제와 같이 적당한 위치에 배치합니다.

생성한 Edit 컴포넌트의 id를 각각 다음과 같이 설정합니다.

출력 정보	ID
Name	edit_name
Address	edit_addr
Company	edit_company
Department	edit_dept
E-Mail	edit_mail

② Dataset 생성하기

Edit와 Grid에 바인딩할 Dataset을 생성합니다.

툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.

그다음 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset00을 더블 클릭하면 Dataset 편집기가 뜨는데 컬럼과 로우를 아래 그림과 같이 입력합니다.

Dataset Editor [Dataset00]							
Const Columns							
Columns							
No	id	type	size	prop	sumtext	description	
1	Name	STRING	256				
2	Address	STRING	256				
3	Company	STRING	256				
4	Department	STRING	256				
5	EMail	STRING	256				
Rows							
No	Name	Address	Company	Department	EMail		
1	john	Califonia, USA	1dollar	Pre-sales	pre-sales@1dollar.com		
2	james	Seoul, KOREA	EBOT	Consulting	consulting@ebot.com		
3	donald	Barcelona, Spain	EBOT	Research Institute	research@ebot.co.kr		
4	lisa	London, UK	1dollar	Quality Assurance	quality@1dollar.co.jp		

3 Edit에 Dataset 바인딩하기

Edit 컴포넌트에 Dataset을 바인딩합니다. Invisible Objects 영역에 있는 Dataset00을 폼에 생성한 각각의 Edit 컴포넌트로 드래그 앤 드롭하면 아래와 같이 Bind Item 창이 뜹니다. Edit는 하나의 속성에 하나의 컬럼만 바인딩할 수 있습니다. Bind Item 창에서 각 Edit 컴포넌트의 value 속성에 대응하는 Dataset00의 컬럼 ID를 선택한 후 [OK] 버튼을 클릭하면 바인딩이 완료됩니다.

Bind Item					
Item List					
No	Bind ID	Object	Property	Dataset	Column ID
1	item0	edit_name	value	Dataset00	Name
2	item1	edit_addr	value	Dataset00	Address
3	item2	edit_company	value	Dataset00	Company
4	item3	edit_dept	value	Dataset00	Department
5	item4	edit_mail	value	Dataset00	EMail

아래는 바인딩이 완료된 후의 XFDL 코드입니다. <BindItem>을 보면 각 Edit의 value 속성에 Dataset00의 각 컬럼이 바인딩된 것을 확인할 수 있습니다.

```
<?xml version="1.0" encoding="utf-8"?>
<FDL version="2.0">
  <Form id="sample_edit_00_nexacro16" left="0" top="0" width="1024" height="768" titletext="New Form">
    .
    .
    <Bind>
      <BindItem id="item0" compid="edit_name" propid="value" datasetid="Dataset00"
columnid="Name"/>
      <BindItem id="item1" compid="edit_addr" propid="value" datasetid="Dataset00"
columnid="Address"/>
      <BindItem id="item2" compid="edit_company" propid="value" datasetid="Dataset00"
columnid="Company"/>
      <BindItem id="item3" compid="edit_dept" propid="value" datasetid="Dataset00"
columnid="Department"/>
      <BindItem id="item4" compid="edit_mail" propid="value" datasetid="Dataset00"
columnid="EMail"/>
    </Bind>
  </Form>
</FDL>
```

④ 쿼뷰로 확인하기

성공적으로 각 Edit 컴포넌트에 Dataset의 컬럼이 바인딩됐는지 쿼뷰(Ctrl + F6)로 확인합니다. Grid에서 특정 레코드를 선택해 Edit에 해당 레코드 정보가 올바르게 표시되는지도 확인합니다.

17.7 자동완성 기능으로 키워드 제시하기

자동완성 기능은 사용자의 입력과 동시에 선택할 수 있는 아이템 목록을 제시하는 기능입니다.

사용자가 입력하는 내용을 기반으로 데이터베이스에 저장된 데이터로부터 후보값을 검색하여 제시합니다. 그리고 사용자가 모든 내용을 입력하지 않고도 검색어를 후보 목록에서 손쉽게 선택할 수 있게 하여 자동으로 입력 내용을 완성하여 줍니다.



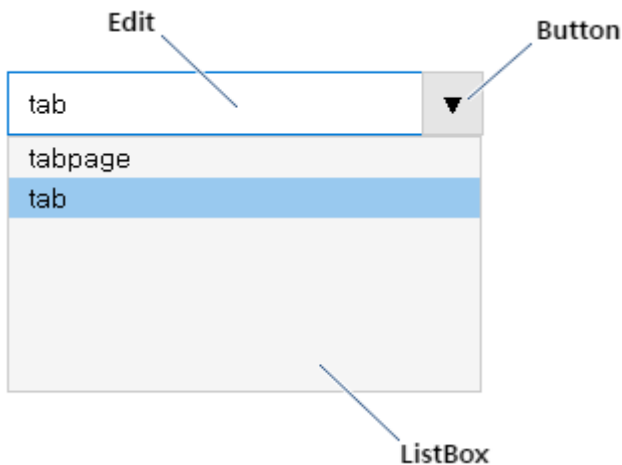
Dataset의 filter 메소드를 사용하여 검색어와 같은 글자가 들어가는 아이템 목록을 보여주는 자동완성 기능을 구현하는 방법을 설명합니다.

17.7.1 예제

예제의 자동완성 입력 폼은 Edit, Button, ListBox 컴포넌트로 구성하여 Combo 컴포넌트와 유사한 모습을 갖습니다.

사용자가 검색어를 입력하면 ListBox에 관련 아이템 목록이 보입니다. 목록에서 아이템을 선택하면 Edit에 선택된 아이템이 표시됩니다.

검색어를 입력하지 않은 상태에서 버튼을 클릭하면 모든 아이템 목록을 확인할 수 있습니다.



17.7.2 예제에서 사용한 핵심 기능

filter

Dataset에 로딩된 데이터 중에 조건에 만족하는 데이터만 보이도록 필터링하는 메소드입니다. 문자열 형태로 조건식을 입력하면 조건에 맞게 필터링합니다. 조건식을 빈 문자열("")로 설정 후 호출하면 현재의 필터링이 해제되며 필터링 적용 전의 원본 데이터를 출력합니다. 아무 값도 설정하지 않고 호출하면 이전에 필터링했던 조건식으로 필터링합니다.

oninput

키보드로 값을 입력할 때 발생하는 이벤트입니다.

17.7.3 예제 구현 방법

① 화면 구성하기

Edit , Button , ListBox  컴포넌트를 예제의 그림과 같이 적절히 배치합니다.

생성한 컴포넌트의 id를 각각 다음과 같이 설정합니다.

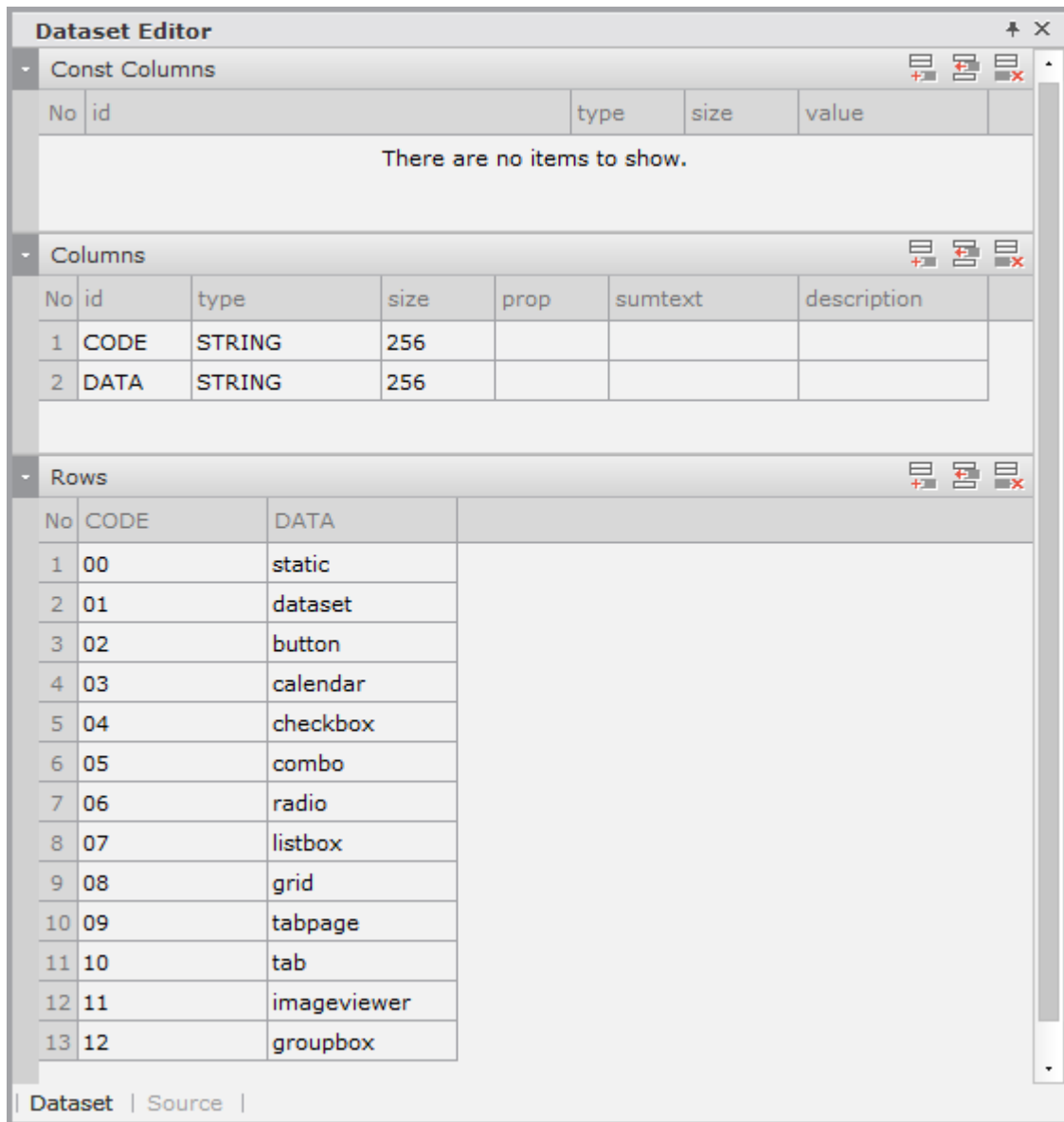
컴포넌트	ID
Edit00	edit_input
Button00	btn_list
ListBox00	list_output
Dataset00	ds_data

② 아이템 목록 생성하기

ListBox에 출력할 아이템 목록을 가진 Dataset을 생성합니다.

툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭합니다. Invisible Object 영역에 있는 Dataset00을 선택한 후 Properties 창에서 id를 'ds_data'로 설정합니다.

그다음 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 ds_data를 더블 클릭하여 Dataset 편집기에 다음과 같이 입력합니다.



3 ListBox에 Dataset 바인딩하기

Dataset을 ListBox에 바인딩합니다. 하단의 Invisible Objects 영역에 있는 ds_data를 폼에 위치한 ListBox 컴포넌트로 드래그 앤 드롭하고 [bind innerDataset 'ds_data']를 선택하면 Bind InnerDataset 창이 뜹니다.

codecolumn에 'CODE'를 datacolumn에 'DATA'를 입력하고 [OK]를 클릭하면 바인딩이 완료됩니다.

4 컴포넌트별 이벤트 함수 작성하기

사용자 입력 및 각 컴포넌트를 동작시키기 위해 컴포넌트별 이벤트 함수를 작성합니다.

- Edit 컴포넌트의 oninput 이벤트 함수

사용자가 키보드로 입력할 때마다 문자 혹은 문자열을 포함한 아이템을 필터링하여 ListBox에 보여줍니다.

```

this.edit_input_oninput = function(obj:nexacro.Edit,e:nexacro.InputEventInfo)
{
    var strLength = this.edit_input.getLength();

    this.list_output.set_visible(true);

    if(strLength > 0)
    {
        this.ds_data.filter("DATA.toString().search('"+this.edit_input.value+"' ) >= 0");

        if(this.ds_data.rowcount == 0)
        {
            this.list_output.set_visible(false);
        }

    }
    else
    {
        this.list_output.set_visible(false);
    }
};

```

- Edit 컴포넌트의 onkeydown 이벤트 함수

ListBox에 표시된 아이템을 선택하기 위해 키보드 방향키와 **ENTER**키를 감지하여 선택 동작을 수행합니다.

```

this.edit_input_onkeydown = function(obj:nexacro.Edit,e:nexacro.KeyEventInfo)
{
    this.list_output.set_visible(true);

    switch(e.keycode)
    {
        case 13:    //ENTER
            if (this.list_output.index != -1)
            {
                this.edit_input.set_value(this.list_output.text)
            }

            this.list_output.set_visible(false);
    }
}

```

```

        break;

    case 27:    //ESC
        this.list_output.set_visible(false);

        break;

    case 38:    //UP
        if (this.list_output.index > 0)
        {
            this.list_output.set_index(this.list_output.index - 1);
        }

        break;

    case 40:    //DOWN
        this.list_output.set_index(this.list_output.index + 1);

        break;

    default:
        trace("keycode = " + e.keycode);
    }
};

```

- Button 컴포넌트의 onclick 이벤트 함수

현재 상태의 후보값을 보여주기 위해 ListBox를 화면에 표시합니다. ListBox가 비활성화되어 있고 사용자가 입력한 내용이 없으면 ds_data의 filter를 초기화하고 ListBox를 활성화합니다. 필터를 초기화하면 ds_data의 모든 아이템을 목록에서 확인할 수 있습니다.

```

this.btn_list onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    if(this.list_output.visible == false)
    {
        if(!this.edit_input.getLength())
        {
            this.ds_data.filter("");
        }
    }
}

```

```

        this.list_output.set_visible(true);
    }
    else
    {
        this.list_output.set_visible(false);
    }
};

```

- ListBox 컴포넌트의 onitemclick 이벤트 함수

사용자가 ListBox의 아이템을 선택했을 때 선택한 값을 Edit에 표시하는 기능을 수행합니다.

```

this.list_output_onitemclick = function(obj:nexacro.ListBox,e:nexacro.ClickEventInfo)
{
    this.edit_input.set_value(obj.text);
    this.list_output.set_visible(false);
};

```

5 퀵뷰로 확인하기

퀵뷰(Ctrl + F6)로 실행한 후 Edit에 검색어를 입력합니다. ListBox가 뜨며 검색어를 포함한 항목만 필터링되어 나오는지 확인합니다.

17.8 목록에서 선택한 값을 여러 개 Edit 컴포넌트에 입력하기

Edit 컴포넌트는 항목을 직접 입력하는 용도로 사용됩니다. 하지만 경우에 따라서 사전에 만들어놓은 목록에서 항목을 선택하고 여러 개 Edit 컴포넌트에 입력하는 경우도 있습니다. 우편번호를 조회하는 경우 해당하는 주소를 선택하면 우편번호항목과 주소항목이 같이 보여지고 선택 시 각각의 Edit 컴포넌트에 입력됩니다. 우편번호를 알고 있는 경우에는 각 항목을 직접 입력할 수 있지만 모르는 경우 목록에서 선택해서 입력하는 방식입니다.

17.8.1 예제

CODE, DATA 2개의 Edit 컴포넌트가 있고 CODE 항목을 입력하려 할때 팝업 형태로 보기 목록을 보여주고 항목을 선택하면 2개의 Edit 컴포넌트에 값을 입력합니다.

CODE	DATA
03	walnut
CODE	DATA
01	apple
02	pear
03	walnut
04	acorn
05	strawberry



sample_edit_08.xfdl

17.8.2 예제에서 사용한 핵심 기능

trackPopupByComponent

컴포넌트를 기준으로 PopupDiv 컴포넌트의 위치를 지정합니다. 예제에서는 첫 번째 Edit 컴포넌트를 기준으로 위치를 지정합니다. X좌표값은 같은 위치로 지정하기 위해 0으로 지정하고 Y좌표값은 Edit 컴포넌트 바로 아래 위치로 지정하기 위해 Edit 컴포넌트의 높이값으로 지정합니다.

oncellick

Grid 컴포넌트의 셀을 클릭했을때 클릭한 위치의 ROW 정보를 비롯한 관련 정보를 전달합니다. 해당 ROW 정보를 받아서 getColumn 메소드에서 선택한 값을 찾을 수 있습니다.

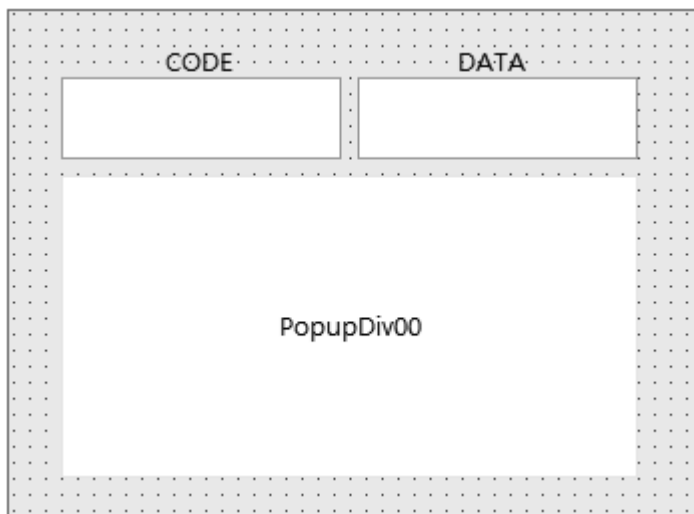
getColumn

ROW 정보를 받아서 원하는 컬럼에 해당하는 값을 찾을 수 있습니다.

17.8.3 예제 구현 방법

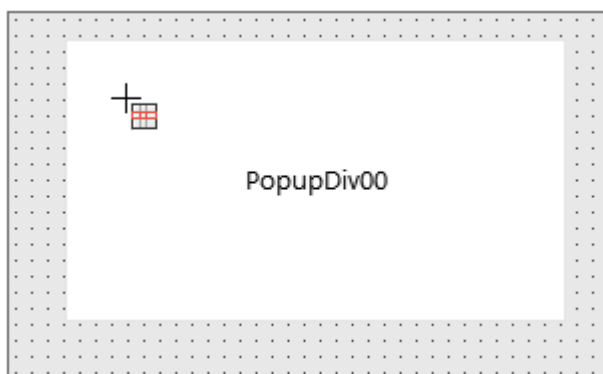
1 화면 구성하기

Static 컴포넌트, Edit 컴포넌트, PopupDiv 컴포넌트를 예제와 같이 배치합니다. PopupDiv 컴포넌트의 위치는 호출할때 지정하기 때문에 어디든 상관없지만 Edit 컴포넌트의 크기가 맞추기 편하게 Edit 컴포넌트 하단에 배치합니다.

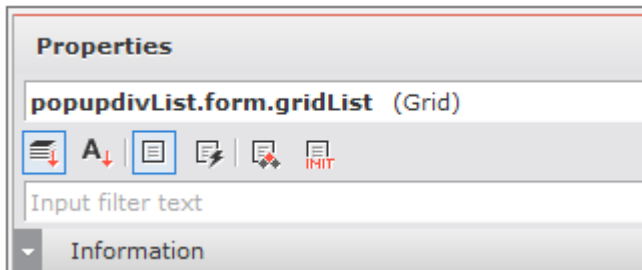


2 PopupDiv 내 Grid 컴포넌트 배치하기

PopupDiv 컴포넌트 내 Grid 컴포넌트를 배치합니다. PopupDiv 컴포넌트 같은 컨테이너 컴포넌트 내에 다른 컴포넌트를 배치할 때는 넥사크로 스튜디오에서 배치할 컴포넌트를 선택한 후에 PopupDiv 컴포넌트 위에서 클릭해주면 됩니다. PopupDiv 컴포넌트 영역 밖에서 클릭하면 Form 오브젝트 아래 배치됩니다.



정상적으로 배치된 경우에는 속성창 상단 오브젝트 목록에서 Grid 컴포넌트가 PopupDiv 컴포넌트에 포함된 것을 확인할 수 있습니다.



3 Grid 컴포넌트 데이터 생성하기

Dataset 오브젝트를 추가하고 Column, Row 정보는 아래와 같이 입력합니다. 그리고 Dataset 오브젝트를 선택한 후 드래그앤드롭으로 Grid 컴포넌트와 바인딩해줍니다.

Columns				
No	id	type	size	prop
1	CODE	STRING	256	
2	DATA	STRING	256	
Rows				
No	CODE	DATA		
1	01	apple		
2	02	pear		
3	03	walnut		
4	04	acorn		
5	05	stawberry		

4 Grid 컴포넌트 속성 설정하기

Dataset 바인딩 후 화면에 깔끔하게 표시되도록 autofitttype 속성값을 "col"로 수정합니다. 그리고 width, height 속성값을 "100%"로 지정하고 left, top 속성값을 0으로 지정하면 PopupDiv 컴포넌트 크기에 딱 맞게 표시됩니다.

5 Edit 컴포넌트 이벤트 함수 작성하기

Edit 컴포넌트를 선택한 후 oneditclick 이벤트 함수를 작성합니다. Edit 컴포넌트를 클릭했을때 이벤트가 처리되면서 PopupDiv 컴포넌트에 배치한 Grid 컴포넌트가 나타납니다.

```
this.editCode_oneditclick = function(obj:nexacro.Edit,e:nexacro.EditClickEventInfo)
{
    this.popupdivList.trackPopupByComponent(this.editCode, 0, parseInt(this.editCode.
height));
};
```

6 Grid 컴포넌트 이벤트 함수 작성하기

Grid 컴포넌트를 선택한 후에 oncellick 이벤트 함수를 작성합니다. Grid 컴포넌트에 표시된 목록에서 항목 선택 시(Cell 클릭 시) Edit 컴포넌트에 선택된 문자열을 표시해줍니다. 항목을 선택하면 더 이상 PopupDiv 컴포넌트를 사용하지 않습니다. closePopup 메소드로 팝업으로 열린 창을 닫습니다.

```
this.popupdivList_gridList_oncellick = function(obj:nexacro.Grid,e:nexacro.
GridClickEventInfo)
{
    this.popupdivList.closePopup();
    this.editCode.set_value(this.dsList.getColumn(e.row, "CODE"));
    this.editData.set_value(this.dsList.getColumn(e.row, "DATA"));
};
```

7 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 Edit 컴포넌트를 클릭해 팝업창에서 원하는 항목을 선택해봅니다.

17.9 바인딩된 값을 키 입력 시 바로 업데이트하기

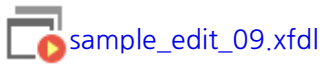
Edit 컴포넌트에 바인딩된 값은 값을 입력하고 Enter 키를 입력하거나 포커스를 이동했을때 Dataset 오브젝트에 반영됩니다. 하지만 updateToDataset 메소드를 사용하면 원하는 시점에 데이터를 변경할 수 있습니다.

17.9.1 예제

Grid 컴포넌트의 데이터를 선택하면 바인딩된 값이 Edit 컴포넌트에 표시됩니다. Edit 컴포넌트의 문자열을 수정하면 키 입력 시 바로 Grid 컴포넌트에 반영됩니다.

Column0
Mango
app
Coconuts
Orange

app|



17.9.2 예제에서 사용한 핵심 기능

updateToDataset

컴포넌트의 value 속성값에 바인딩된 Dataset 오브젝트의 데이터를 업데이트합니다. 예제에서는 onkeyup 이벤트 함수 내에서 메소드를 실행해 키 입력 시마다 데이터가 반영되도록 했습니다.

17.9.3 예제 구현 방법

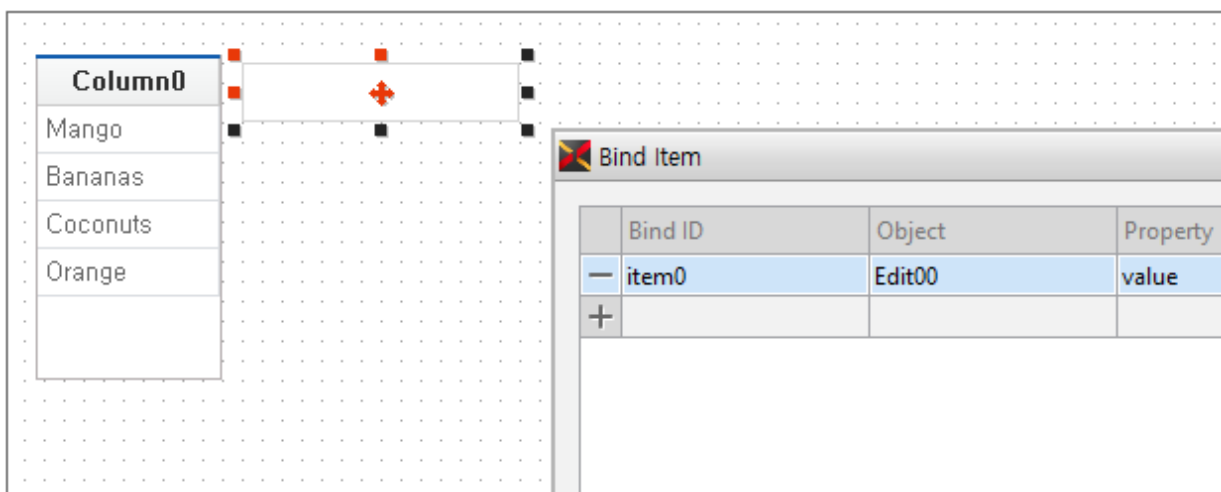
① 화면 구성하기

Grid 컴포넌트를 배치하고 Dataset 오브젝트를 생성합니다. Dataset 오브젝트는 하나의 컬럼만 생성하고 데이터를 입력합니다.

Columns			
No	id	type	size
1	Column0	STRING	256
Rows			
No	Column0		
1	Mango		
2	Bananas		
3	Coconuts		
4	Orange		

② Edit 컴포넌트 설정하기

Edit 컴포넌트를 배치하고 Dataset 오브젝트를 끌어다가 바인딩합니다.



onkeyup 이벤트를 함수를 아래와 같이 작성합니다. 키 입력 시 바인딩된 Dataset 오브젝트를 업데이트합니다.

```
this.Edit00_onkeyup = function(obj:nexacro.Edit,e:nexacro.KeyEventInfo)
{
    this.Edit00.updateToDataset();
};
```

3 QuickView로 확인하기

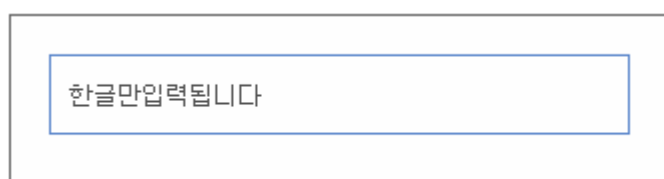
QuickView(Ctrl + F6)로 실행한 후 Edit 컴포넌트에 입력된 항목이 키 입력 시 Grid 컴포넌트에 반영되는지 확인합니다.

17.10 한글만 입력할 수 있도록 하기

입력할 수 있는 문자열을 스크립트에서 제한하는 방식입니다.

17.10.1 예제

Edit 컴포넌트에 한글이 아닌 다른 문자를 입력하면 입력된 문자를 삭제하고 한글만 남깁니다.





sample_edit_10.xfdl

17.10.2 예제에서 사용한 핵심 기능

oninput

Edit 컴포넌트의 편집상태(포커스가 있는)에서 입력 시도 시 발생하는 이벤트입니다.

17.10.3 예제 구현 방법

① Edit 컴포넌트 배치하고 oninput 이벤트 함수 작성하기

Edit 컴포넌트를 배치하고 oninput 이벤트 함수를 추가합니다. 입력된 문자열이 한글이 아닌 경우에는 ""로 치환합니다.

```
this.Edit00_oninput = function(obj:nexacro.Edit,e:nexacro.InputEventInfo)
{
    obj.set_value(obj.value.replace(/[\^ㄱ-ㅎㅌ-ㅣ가-힣]/g, ""));
};
```

② QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 Edit 컴포넌트에 한글 아닌 문자열이 입력되는지 확인합니다.

18.

FileDialog

18.1 FileDialog 소개

FileDialog 컴포넌트는 사용자가 가지고 있는 파일을 선택할 때 파일 선택 대화상자를 표시하는 기능을 제공합니다. 실행되는 파일 대화상자는 시스템에서 제공하는 대화상자를 표시합니다. FileDialog 컴포넌트에서는 대화상자를 표시하고 파일 선택 범위를 줄일 수 있는 필터링 기능 등을 제공합니다.

18.1.1 지원 범위



웹브라우저 지원은 17.0.0.1900 버전부터 지원합니다.

FileDialog 컴포넌트는 운영체제와 실행환경의 영향을 받기 때문에 각 속성, 메소드, 이벤트의 지원 범위가 다를 수 있으며, 파라미터별로도 차이가 있을 수 있습니다.

아래 표는 17.0.0.1900 버전 기준의 지원 범위입니다. 이후 버전에서는 지원 범위가 변경될 수 있습니다.

	nexacro Browser (desktop)	nexacro Browser (mobile)	IE10, 11, Chrome, Firefox, Opera	Android, iOS default Web Browser	Edge, Safari
속성					
accept	X	X	O	O	X
defaulttextextension	O	O	X	X	X
dialogtype	X	O	X	X	X
filter	O	O	X	X	X
filterindex	O	O	X	X	X
메소드					
open	O	O	O	O	O
이벤트					
onclose	O	O	O	O	O

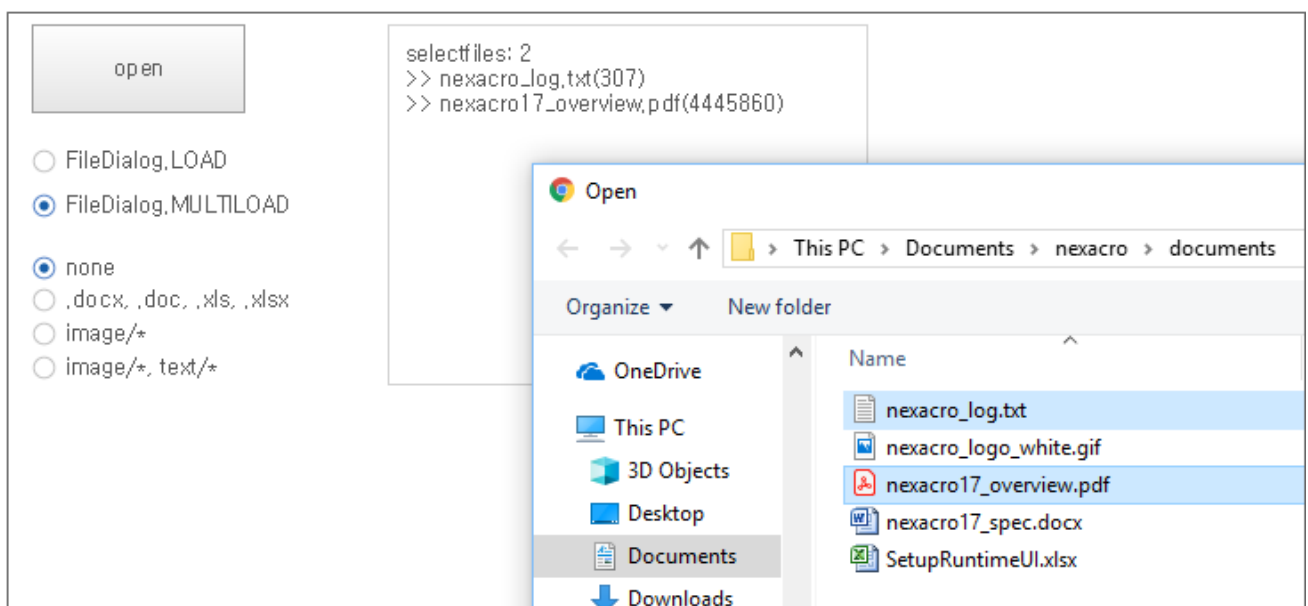
	nexacro Browser (desktop)	nexacro Browser (mobile)	IE10, 11, Chrome, Firefox, Opera	Android, iOS default Web Browser	Edge, Safari
FileDialogEventInfo					
path	O	O	X	X	X
reason	O	O	O	O	O
virtualfiles	O	O	O	O	O

18.2 FileDialog 실행하고 선택한 파일 표시하기

파일 선택 대화 상자를 선택한 옵션에 따라 실행하고 선택한 파일을 표시합니다.

18.2.1 예제

여러 개 파일을 선택할지, 파일 확장자로 필터링을 할지 선택한 후에 [open] 버튼을 클릭하면 파일 선택 대화상자를 표시합니다. 파일을 선택하고 파일 선택 대화상자를 닫으면 TextArea 영역에 선택한 파일명과 크기를 표시합니다.



sample_filedialog_01.xfdl

18.2.2 예제에서 사용한 핵심 기능

open

파일 선택 대화상자를 표시합니다. 파라미터로 strTitle(파일 선택 대화상자 타이틀), constOpenMode(파일 선택 옵션)을 지정할 수 있는데, strTitle 파라미터는 웹브라우저 실행 시 적용되지 않으며 constOpenMode도 웹브라우저 실행 시에는 2가지만 사용할 수 있습니다.

virtualfiles

파일 선택 대화상자를 닫으면 onclose 이벤트가 발생하고 FileDialogEventInfo 오브젝트의 virtualfiles 속성에서 선택한 파일 정보를 확인할 수 있습니다. virtualfiles 속성에는 VirtualFile 오브젝트가 배열 형태로 담겨져 있습니다. 웹브라우저 환경에서는 VirtualFile 오브젝트를 직접 생성해 사용할 수는 없습니다.

accept

파일 확장자 또는 MIME 타입 형태로 파일 형식을 지정할 수 있습니다.



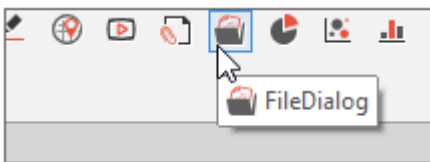
MIME 타입 목록은 아래 링크를 참고하세요.

https://developer.mozilla.org/en-US/docs/Web/HTTP/Basics_of_HTTP/MIME_types/Complete_list_of_MIME_types

18.2.3 예제 구현 방법

- 1 화면에 Button 컴포넌트를 배치하고 Button 컴포넌트 아래에 Radio 컴포넌트를 2개 배치합니다. 그리고 오른쪽에 선택한 파일 목록을 표시할 TextArea 컴포넌트를 배치합니다.

- 2 툴바에서 FileDialog를 선택하고 화면에 추가합니다. FileDialog는 Invisible Object 영역에 표시됩니다.



- 3 Radio 컴포넌트의 innerdataset을 아래와 같이 편집합니다. 하나는 open 메소드의 constOpenMode 파라미터를 지정하고 다른 하나는 accept 속성값을 지정합니다.

No	codecolumn	datacolumn
1	0	FileDialog.LOAD
2	1	FileDialog.MULTILOAD

No	codecolumn	datacolumn
1		none
2	.docx, .doc, .xls, .xlsx	.docx, .doc, .xls, .xlsx
3	image/*	image/*
4	image/*, text/*	image/*, text/*

- ④ Radio 컴포넌트의 onitemchanged 이벤트 함수를 아래와 같이 작성합니다. 첫 번째 Radio 컴포넌트에서 선택한 항목을 open 메소드의 파라미터로 지정하고 두 번째 Radio 컴포넌트에서 선택한 항목은 accept 속성값으로 지정합니다.

```
var constOpenMode = FileDialog.LOAD;

this.Radio00_onitemchanged = function(obj:nexacro.Radio,e:nexacro.ItemChangeEventInfo)
{
    if(e.postvalue==0)
    {
        constOpenMode = FileDialog.LOAD;
    }
    else
    {
        constOpenMode = FileDialog.MULTILOAD;
    }
};

this.Radio01_onitemchanged = function(obj:nexacro.Radio,e:nexacro.ItemChangeEventInfo)
{
    this.FileDialog00.set_accept(e.postvalue);
};
```

- ⑤ Button 컴포넌트의 onclick 이벤트 함수를 아래와 같이 작성합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.FileDialog00.open('nexacro17', constOpenMode);
};
```

6 FileDialog 오브젝트의 onclose 이벤트 함수를 아래와 같이 작성합니다.

웹브라우저에서는 파일을 선택하지 않고 파일 선택 대화상자를 닫았을때 이벤트가 발생하지 않지만 넥사크로 브라우저에서는 이벤트가 발생하기 때문에 예외 처리를 해주어야 합니다. 아래 예제에서는 예외 처리를 해주지 않았습니다.

```
this.FileDialog00_onclose = function(obj:nexacro.FileDialog,e:nexacro.FileDialogEventInfo)
{
    this.TextArea00.set_value("selectfiles: "+e.virtualfiles.length+"\n");
    for (var i = 0, len = e.virtualfiles.length, vFile; i < len; i++)
    {
        vFile = e.virtualfiles[i];
        vFile.addEventHandler("onsuccess", this.FileDialog_VirtualFile_onsuccess, this);

        vFile.open(null, VirtualFile.openRead);
    }
};

this.FileDialog_VirtualFile_onsuccess = function(obj:nexacro.VirtualFile, e:nexacro.
VirtualFileEventInfo)
{
    switch (e.reason)
    {
        case 1:
            obj.getFileSize();
            break;
        case 9:
            this.TextArea00.set_value(this.TextArea00.value+obj.filename+"("+e.filesize+
            ")\\n");
            obj.close();
            break;
    }
}
```

onclose 이벤트가 발생하면 FileDialogEventInfo 오브젝트의 virtualfiles 속성에서 선택한 파일 정보를 확인할 수 있습니다. 파일명만 필요한 경우에는 filename 속성값을 사용하며, 파일 크기를 확인해야 하는 경우에는 getFileSize 메소드를 실행하고 VirtualFile 오브젝트의 onsuccess 이벤트 함수 내에서 파일 크기를 확인합니다.

VirtualFile 오브젝트의 getFileSize 메소드를 사용하기 위해서는 먼저 open 메소드를 실행해주어야 하고 onsuccess 이벤트를 등록해주어야 합니다. open 메소드가 성공하면 그 다음 getFileSize 메소드를 실행합니다.

FileDialogEventInfo 오브젝트의 virtualfiles 속성에서 가져온 VirtualFile 오브젝트는 사용할 수 있는 기능 제한이 있습니다. open 메소드를 사용할 때 strFileName 파라미터를 지정할 수 없으며, constOptions 파라미터는 VirtualFile.openRead 만 사용할 수 있습니다.

- 7 QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. FileDialog.LOAD 옵션을 선택하면 파일을 하나만 선택할 수 있고, FileDialog.MULTILOAD 옵션을 선택하면 여러 파일을 선택할 수 있습니다.

19.

FileUpTransfer

19.1 FileUpTransfer 소개

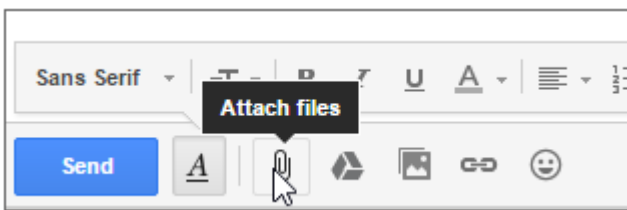
FileUpTransfer 오브젝트는 사용자가 작성하거나 가지고 있는 파일을 서버 저장소에 업로드하는 기능을 제공합니다. 웹이 등장하면서 파일을 전송하는 것은 중요한 이슈였으나 HTML 기본 기능으로는 원하는 작업을 수행하기 힘들었기 때문에 다양한 서드파티 서비스가 등장했습니다. 대용량 파일을 다루거나 여러 개의 파일을 한 번에 업로드 하는 작업을 위한 서비스가 제공되기도 합니다.



FileUpTransfer 오브젝트는 17.0.0.1900 버전부터 지원합니다.

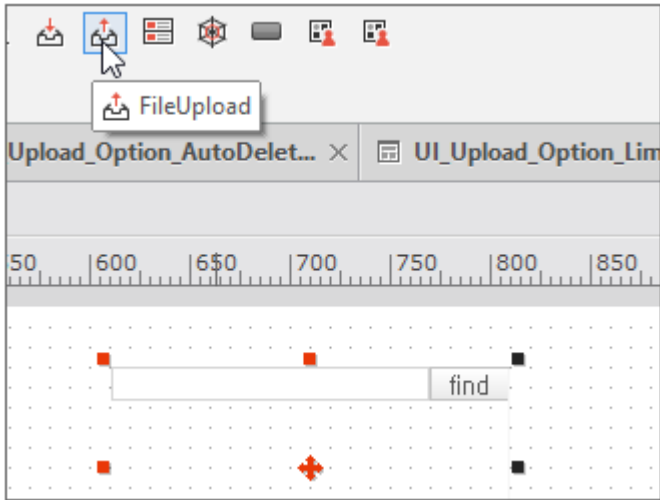
19.1.1 사용 예

게시판에 글을 작성하면서 첨부자료를 업로드하거나 이메일 작성 시 첨부파일을 추가하는 작업 시 사용합니다.



19.1.2 FileUpload 컴포넌트와 차이

FileUpload 컴포넌트는 기본 UI를 제공하는 컴포넌트로 드래그앤드롭이나 다른 형식의 인터페이스를 구현하기에 제약이 있습니다. 이런 제약을 해결하고자 프로젝트에서 필요한 UI를 만들고 파일을 선택하고 전송하는 기능을 구현할 수 있도록 FileUpTransfer 오브젝트를 Invisible Object로 제공합니다.



이미 FileUpload 컴포넌트를 적용해 사용하는 프로젝트가 있기 때문에 FileUpload 컴포넌트는 현재 상태를 유지하지만 FileUpTransfer 오브젝트를 사용하는 것을 권장합니다.

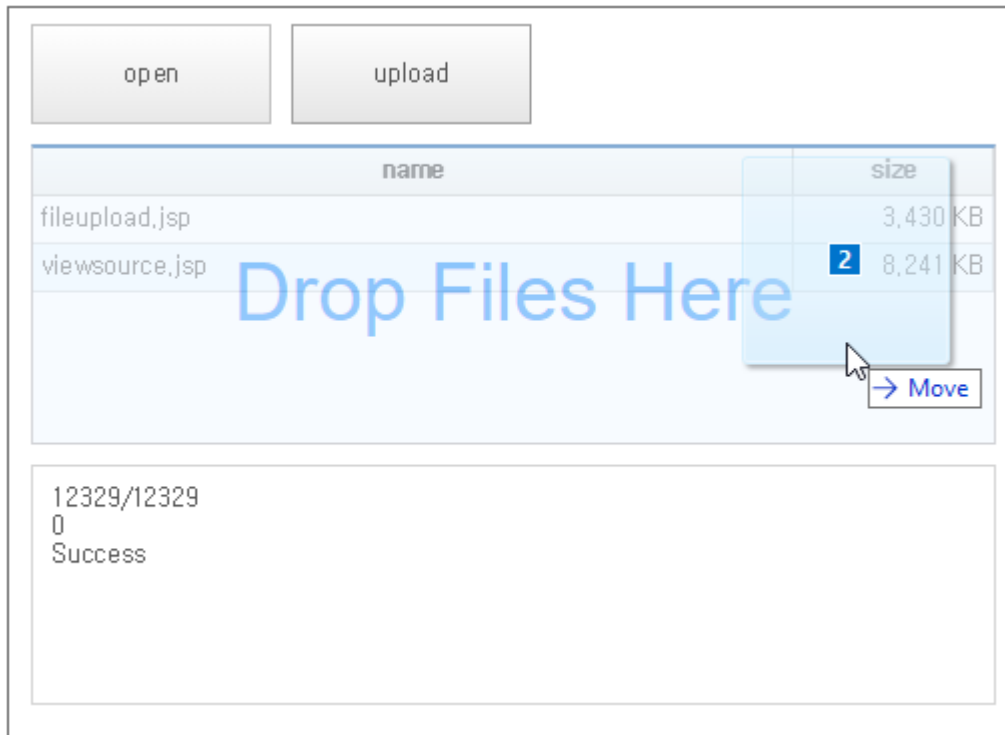
19.2 선택한 파일 업로드하기

파일 선택 대화 상자에서 파일을 선택하거나 드래그앤드롭으로 추가한 파일을 지정된 서버에 업로드합니다.

19.2.1 예제

[open] 버튼을 클릭하면 파일 선택 대화상자를 표시합니다. 파일을 선택하고 파일 선택 대화상자를 닫으면 Grid 컴포넌트에 파일 목록을 추가합니다. 윈도우 탐색기에서 파일을 선택하고 Grid 컴포넌트 영역 위로 드래그앤드롭해도 파일을 추가할 수 있습니다.

파일 목록을 추가한 상태에서 [upload] 버튼을 클릭하면 해당 파일을 서버로 전송하고 전송 결과를 TextArea 컴포넌트에 표시합니다.



sample_fileuptransfer_01.xfdl

19.2.2 예제에서 사용한 핵심 기능

datatype, fileList (17.0.0.1900 버전부터 지원)

DragEventInfo, GridDragEventInfo, ListViewDragEventInfo, MenuDragEventInfo 오브젝트에 추가된 속성입니다. 윈도우 탐색기에서 파일을 선택하고 드래그앤드롭하는 경우에는 datatype 속성값이 "file"로 처리되고 fileList 속성값에서 선택한 파일 목록을 확인할 수 있습니다.

addFile (17.0.0.2100 버전부터 파라미터 변경)

VirtualFile 오브젝트 형태로 파일 정보를 추가합니다. 추가된 파일 정보는 FileUpTransfer 오브젝트의 fileList 속성에서 확인할 수 있습니다.

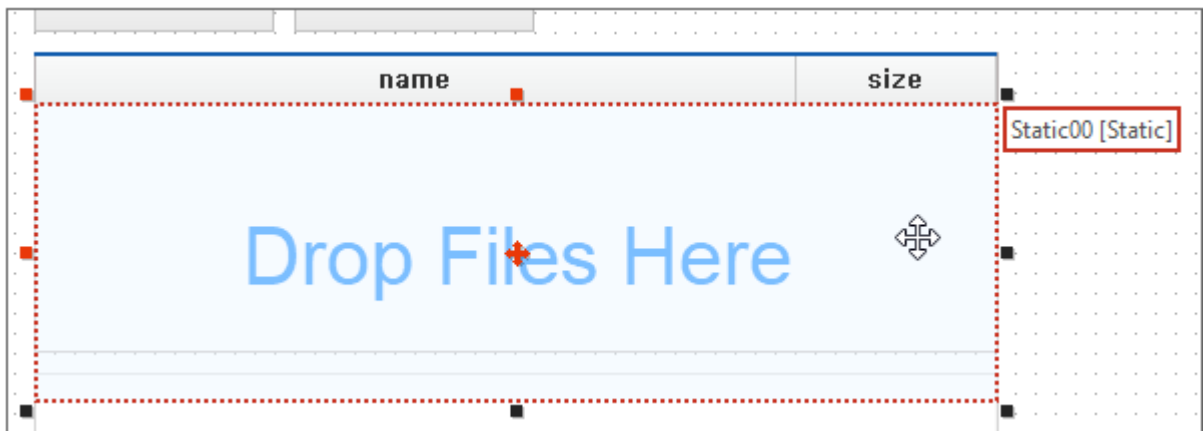
upload

POST 방식으로 파일을 전송합니다.

19.2.3 예제 구현 방법

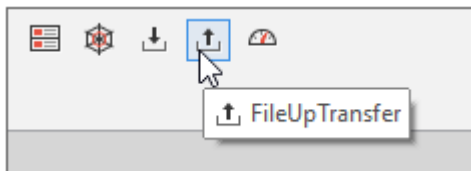
- ① 화면에 2개의 Button 컴포넌트를 배치하고 Button 컴포넌트 아래에 Grid 컴포넌트와 TextArea 컴포넌트를 배치합니다.
- ② Dataset 오브젝트를 추가하고 'name', 'size' 두 개의 컬럼을 생성합니다.
- ③ Grid 컴포넌트에 Dataset 오브젝트를 바인딩합니다.
- ④ Grid 컴포넌트와 같은 크기, 같은 위치로 Static 컴포넌트를 배치합니다.

Grid 컴포넌트 아래에 Static 컴포넌트가 배치되어야 합니다. Static 컴포넌트를 먼저 배치하고 그 위에 Grid 컴포넌트를 배치하든지 아니면 Static 컴포넌트를 나중에 배치하고 메뉴[Design > Arrange > Send Backward]를 사용해 순서를 변경할 수 있습니다.



Grid 컴포넌트 뒤에 배치하는 Static 컴포넌트는 사용자에게 파일을 드래그앤드롭할 수 있는 영역이라는 것을 표시하는 역할을 합니다.

- ⑤ 툴바에서 FileUpTransfer를 선택하고 화면에 추가합니다. FileUpTransfer는 Invisible Object 영역에 표시됩니다.



- ⑥ [open] Button 컴포넌트의 onclick 이벤트를 함수를 아래와 같이 작성합니다.


```

this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.FileDialog00.open('nexacro17', FileDialog.MULTILOAD);
};

```

- 7 FileDialog 오브젝트의 onclose 이벤트를 함수를 아래와 같이 작성합니다.

파일을 추가하는 방법이 파일 선택 대화상자와 드래그앤드롭 두 가지로 나누어져서 추가한 파일을 처리하는 부분을 addFileList 라는 function으로 분리했습니다.

```

this.FileDialog00_onclose = function(obj:nexacro.FileDialog,e:nexacro.FileDialogEventInfo)
{
    this.addFileList(e.virtualfiles);
};

this.addFileList = function(filelist)
{
    for (var i = 0, len = filelist.length, vFile; i < len; i++)
    {
        vFile = filelist[i];
        vFile.addEventHandler("onsuccess", this.FileList_onsuccess, this);
        vFile.addEventHandler("onerror", this.FileList_onerror , this);

        vFile.open(null, 1);
    }
}

```

- 8 Grid 컴포넌트의 drag, drop 관련 이벤트를 함수를 아래와 같이 작성합니다.

사용자가 윈도우 탐색기에서 파일을 선택하고 드래그해서 Grid 컴포넌트 영역으로 들어왔을때 투명도 속성을 조정해서 뒤에 있는 Static 컴포넌트가 보이도록 합니다. Grid 컴포넌트 영역 밖으로 나가거나 드롭 동작이 처리되면 투명도를 다시 원래 상태로 돌려놓습니다.

```

this.Grid00_ondragenter = function(obj:nexacro.Grid,e:nexacro.DragEventInfo)
{
    if(e.datatype == "file")
    {
        this.Grid00.set_opacity(0.5);
    }
};

```

```

this.Grid00_ondragleave = function(obj:nexacro.Grid,e:nexacro.DragEventInfo)
{
    this.Grid00.set_opacity(1);
};

this.Grid00_ondrop = function(obj:nexacro.Grid,e:nexacro.GridDragEventInfo)
{
    this.Grid00.set_opacity(1);
    if(e.datatype == "file")
    {
        this.addFileList(e.filelist);
    }
};

```

ondrop 이벤트 함수에서는 addFileList function에 filelist 속성을 전달합니다.

- 9 VirtualFile 오브젝트의 onsuccess, onerror 이벤트를 함수를 아래와 같이 작성합니다.

onsuccess 이벤트 함수에서는 Dataset 오브젝트에 파일명과 파일 크기를 표시하고 FileUpTransfer 오브젝트의 addFile 메소드를 실행해 파일 정보를 추가합니다.



17.0.0.2100 버전부터 addFile 메소드 사용시 ID 파라미터와 VirtualFile 오브젝트를 같이 지정해주어야 합니다. ID 파라미터가 지정되지 않는 경우 정상 동작하지 않습니다.

```

this.FileList_onsuccess = function(obj:nexacro.VirtualFile, e:nexacro.VirtualFileEventInfo)
{
    switch (e.reason)
    {
        case 1:
            obj.getFileSize();
            break;
        case 9:
            var nRowIdx = this.Dataset00.addRow();
            this.Dataset00.setColumn(nRowIdx, 0, obj.filename);
            this.Dataset00.setColumn(nRowIdx, 1, this.cutFileSize(e.filesize));
            this.FileUpTransfer00.addFile(obj.filename, obj);
            break;
    }
}

```

```

}

this.FileList_onerror = function(obj:nexacro.VirtualFile, e:nexacro.
VirtualFileErrorEventInfo)
{
    trace("errortype: "+e.errortype);
    trace("errmsg: "+e.errormsg);
    trace("statusCode: "+e.statuscode);
}

```

- ⑩ 파일 크기를 적절한 단위로 표시하는 function을 아래와 같이 추가합니다.



코드는 아래 링크에서 참조했습니다.

[https://developer.mozilla.org/en-US/docs/Web/API/File/Using_files_from_web_applications#Example_Showing_file\(s\)_size](https://developer.mozilla.org/en-US/docs/Web/API/File/Using_files_from_web_applications#Example_Showing_file(s)_size)

```

this.cutFileSize = function(filesize)
{
    var sOutput = filesize + " bytes";
    for (var aMultiples = ["KB", "MB", "GB", "TB", "PB", "EB", "ZB", "YB"], nMultiple = 0,
nApprox = filesize / 1024; nApprox > 1; nApprox /= 1024, nMultiple++)
    {
        sOutput = nApprox.toFixed(3) + " " + aMultiples[nMultiple];
    }
    return sOutput;
};

```

- ⑪ [upload] Button 컴포넌트의 onclick 이벤트 함수를 아래와 같이 작성합니다.

```

this.Button01_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.TextArea00.set_value("");
    this.FileUpTransfer00.upload('http://demo.nexacro.com/developer_guide/17/Service/
fileupload.jsp');
};

```



서버에서 사용하는 코드는 아래 링크에서 참조했습니다.

http://www.tutorialspoint.com/jsp/jsp_file_uploading.htm

적용한 코드는 아래 링크에서 확인할 수 있습니다.

https://github.com/TOBESOFT-DOCS/sample_nexacroplatform_17/blob/master/Service/fileupload.jsp

- ⑫ FileUpTransfer 오브젝트의 onprogress, onsuccess, onerror 이벤트를 함수를 아래와 같이 작성합니다.

```
this.FileUpTransfer00_onprogress = function(obj:nexacro.FileUpTransfer,e:nexacro.
FileUpTransferProgressEventInfo)
{
    this.fn_addlog(e.loaded+"/"+e.total);
};

this.FileUpTransfer00_onsuccess = function(obj:nexacro.FileUpTransfer,e:nexacro.
FileUpTransferEventInfo)
{
    this.fn_addlog(e.code);
    this.fn_addlog(e.message);
};

this.FileUpTransfer00_onerror = function(obj:nexacro.FileUpTransfer,e:nexacro.
FileUpTransferErrorEventInfo)
{
    this.fn_addlog(e.errormsg);
    this.fn_addlog(e.statuscode);
};

this.fn_addlog = function(strMessage)
{
    this.TextArea00.insertText(strMessage + '\n');
}
```



onprogress 이벤트는 IE8, IE9에서는 지원하지 않으며 실행 환경에 따라 이벤트 발생 시점이 달라질 수 있습니다.



서버 내 코드에서 5MB 이상의 파일은 에러 처리하고 있습니다. 하지만, 파일 크기와 상관없이 `onprogress` 이벤트가 처리되는 것은 확인할 수 있습니다.



로컬 또는 다른 도메인 환경에서 테스트하는 경우에는 권한 확인을 위해 사전 호출(preflight request)을 하면서 upload 메소드 파라미터로 지정된 도메인에 2번 접근하게 됩니다. 자세한 내용은 아래 링크를 참고하세요.

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/OPTIONS>

13

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. 파일 선택 대화상자 또는 드래그앤드롭으로 파일을 추가하고 업로드를 시도해봅니다.

20.

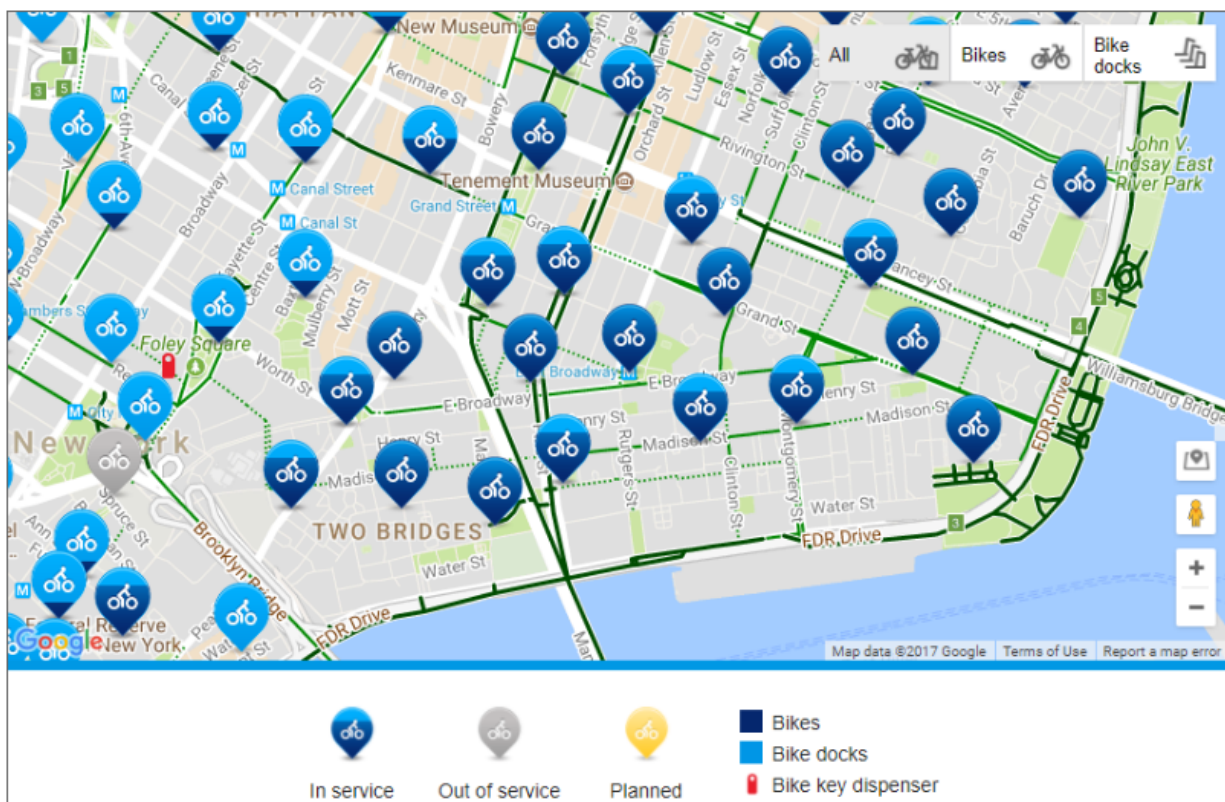
GoogleMap

20.1 GoogleMap 소개

GoogleMap은 구글에서 제공하는 지도를 화면에 출력할 때 사용하는 컴포넌트입니다. 구글 지도를 사용하려면 HT ML 페이지를 만들고 구글에서 제공하는 API를 사용해서 로딩하는 과정이 필요하지만 GoogleMap 컴포넌트를 사용하면 좀 더 손쉽게 구글 지도를 제어할 수 있습니다.

20.1.1 사용 예

다음은 Citi Bike 라는 구글 지도를 이용한 자전거 대여 서비스의 화면입니다. 지도상에 자전거를 대여할 수 있는 위치와 대여 가능 상태를 마커로 표시해 주어 사용자가 지도만 보고도 정보를 파악할 수 있습니다.



20.1.2 지도 보여주기

GoogleMap 컴포넌트를 사용해 화면에 지도를 출력하는 예제를 만들어 봅니다.



① GoogleMap 컴포넌트 생성하기

폼에 GoogleMap과 Button 컴포넌트를 생성합니다.

② Show Map 버튼 이벤트 함수 작성하기

Show Map 버튼의 onclick 이벤트 함수를 다음과 같이 작성합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.GoogleMap00.load(false, 11.96832946, 121.922996308, 0, 12);
    this.GoogleMap00.set_apikey(nexacro.getApplication().googleMapAPIKey);
    this.GoogleMap00.set_showzoom(true);
};
```

load 메소드의 인수는 앞에서부터 사용자 gps 사용 여부, 북위 좌표, 동경 좌표, 지도 모드, 줌 레벨입니다. 인수에 대한 더 자세한 설명은 레퍼런스가이드를 참조하시기 바랍니다.

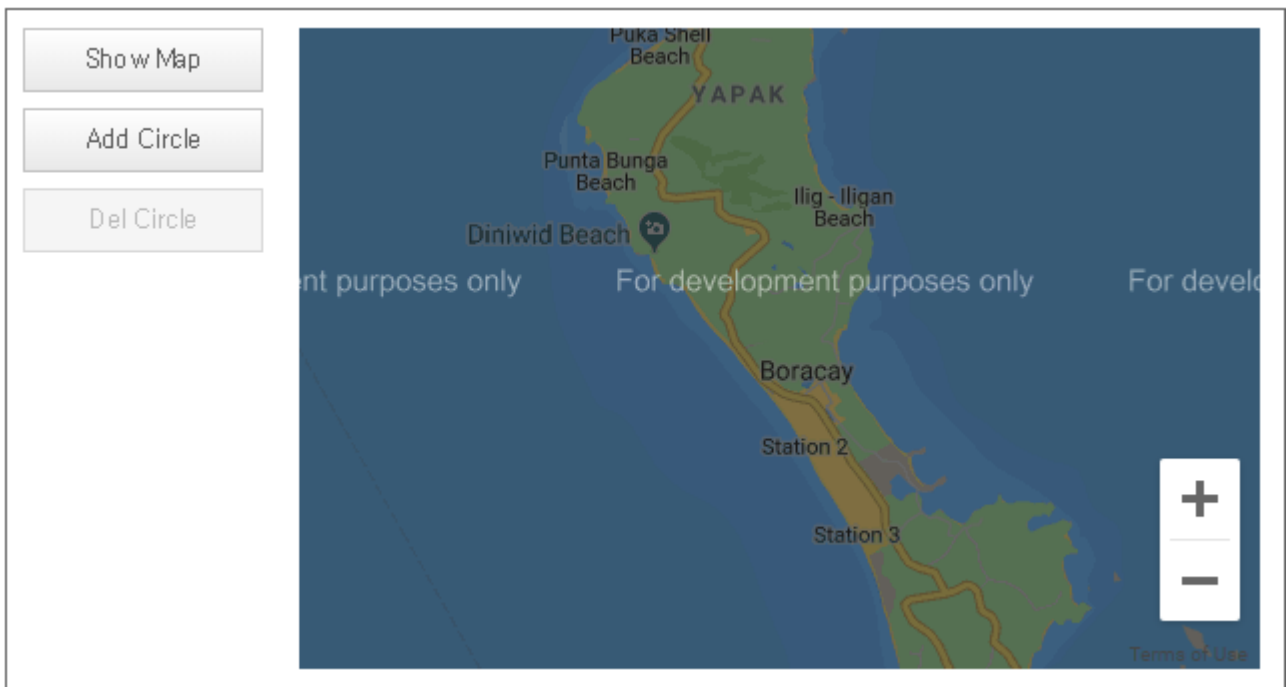
showzoom 속성을 true로 설정하면 지도상에 줌 버튼을 표시합니다. + 버튼을 누르면 확대 - 버튼을 누르면 축소됩니다.

③ 퀵뷰로 확인하기

쿼뷰(Ctrl + F6)를 실행하여 결과를 확인합니다. Show Map 버튼을 클릭하면 화면에 보라카이 지도가 출력되며 마우스와 키보드를 이용해 지도를 이동 및 확대/축소할 수 있습니다.

20.1.3 apikey

GooglaMap 컴포넌트를 사용하기 위해서는 API KEY값을 load 메소드 호출 전에 입력해주어야 합니다. API KEY 값을 입력하지 않은 경우에는 아래와 같이 워터마크가 표시된 상태로 지도가 노출됩니다.



API KEY 발급과 가격 정책에 대해서는 아래 링크를 참고하세요.
<https://cloud.google.com/maps-platform/pricing/?hl=ko>



안드로이드 운영체제는 AndroidManifest.xml 파일에서 API KEY 값을 설정합니다. 아래 링크를 참고하세요.
http://docs.tobesoft.com/deployment_guide_nexacro_17_ko#3f1a23f2e31fc50d

20.2 지도에 마커 추가/삭제하기

GoogleMap 컴포넌트에 로딩한 지도의 특정 좌표에 마커를 추가하거나 삭제할 수 있습니다. 마커는 지도상에 어떤 표시를 할 때 사용합니다.

20.2.1 예제

다음은 구글 지도에서 보라카이 섬(위도 11.96832946, 경도 121.922996308)을 출력하고 같은 좌표에 마커를 표시하는 예입니다. 예제의 버튼은 각각 다음과 같은 기능을 합니다.

버튼	설명
Show Map	보라카이 섬 지도를 화면에 출력합니다.
Add Marker	지도에서 위도 11.96832946, 경도 121.922996308 좌표에 마커를 추가합니다.
Del Marker	지도에 표시된 마커를 삭제합니다.



20.2.2 예제에서 사용한 핵심 기능

load

구글 지도를 화면에 로딩하는 메소드입니다. 사용자 GPS 정보 사용 여부, 위도, 경도, 지도 모드, 줌 등의 설정이 가능합니다.

```
this.GoogleMap00.load(false, 11.96832946, 121.922996308, 0, 13);
```

showzoom

구글 지도에 줌(확대/축소) 컨트롤의 표시 여부를 설정하는 속성입니다.

GoogleMapMarker

지도의 특정 좌표에 마커를 표시할 때 사용하는 오브젝트입니다. 마커의 속성(draggable, imageUrl, latitude, longitude, text, visible)을 설정할 수 있습니다.

addItem

구글 지도에 표시할 아이템(마커, 원, 선, 다각형)을 추가하는 메소드입니다. 여러 개의 아이템을 추가할 때에는 아이템 ID를 다르게 설정해야 합니다.

removeItem

구글 지도에 표시한 아이템(마커, 원, 선, 다각형)을 제거하는 메소드입니다. 추가한 아이템이 없는 상태에서 사용하면 false를 반환하고 onerror 이벤트가 발생합니다.

20.2.3 예제 구현 방법

① 컴포넌트 생성하기

예제와 같이 폼에 GoogleMap과 Button 컴포넌트를 생성합니다.

Add Marker, Del Marker 버튼의 enable 속성을 false로 설정합니다.

② 버튼 이벤트 함수 작성하기

Show Map, Add Marker, Del Marker 버튼의 onclick 이벤트 함수를 각각 다음과 같이 작성합니다.

- Show Map 버튼 이벤트 함수

특정 좌표의 지도를 화면에 출력합니다.

```

this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.GoogleMap00.set_showzoom(true);
    this.GoogleMap00.set_apikey(nexacro.getApplication().googleMapAPIKey);
    this.GoogleMap00.load(false, 11.96832946, 121.922996308, 0, 13);
};

```

- Add Marker 버튼 이벤트 함수

GoogleMapMarker 객체를 생성하고 속성을 설정한 후 addItem 메소드로 마커를 지도에 추가합니다.

```

this.Button01_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var objGoogleMapMaker = new nexacro.GoogleMapMarker();

    objGoogleMapMaker.set_latitude(11.96832946);
    objGoogleMapMaker.set_longitude(121.922996308);
    objGoogleMapMaker.set_text("Default Marker");
    objGoogleMapMaker.set_draggable(true);
    objGoogleMapMaker.set_visible(true);

    this.GoogleMap00.addItem("MapMarker", objGoogleMapMaker);

    this.Button01.set_enable(false);
    this.Button02.set_enable(true);
};

```

- Del Marker 버튼 이벤트 함수

지도에 추가한 마커를 제거합니다.

```

this.Button02_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.GoogleMap00.removeItem("MapMarker");

    this.Button01.set_enable(true);
    this.Button02.set_enable(false);
};

```

- GoogleMap onload 이벤트 함수

GoogleMap의 지도를 정상적으로 로딩하면 발생하는 이벤트입니다.

```
this.GoogleMap00_onload = function(obj:nexacro.GoogleMap,e:nexacro.GoogleMapEventInfo)
{
    trace("loading succeed");

    if(this.GoogleMap00.items[0])
    {
        this.GoogleMap00.removeItem(this.GoogleMap00.items[0]);
    }

    this.Button01.set_enable(true);
    this.Button02.set_enable(false);
};
```

- GoogleMap onerror 이벤트 함수

GoogleMap의 동작에 실패하면 발생하는 이벤트입니다.

```
this.GoogleMap00_onerror = function(obj:nexacro.GoogleMap,e:nexacro.
GoogleMapErrorEventInfo)
{
    trace("error: " + e.errormsg);
};
```

④ 퀵뷰로 확인하기

퀵뷰(Ctrl + F6)를 실행하여 결과를 확인합니다.

Show Map 버튼을 클릭하면 화면에 보라카이 지도가 출력되며 마우스와 키보드를 이용해 지도를 이동 및 확대/축소할 수 있습니다. Add Marker 버튼을 클릭하면 화면에 마커를 추가하고, Del Marker 버튼을 클릭하면 마커를 삭제합니다.

20.3 지도에 원 추가/삭제하기

GoogleMap 컴포넌트에 로딩한 지도의 특정 좌표에 원을 추가하거나 삭제할 수 있습니다.

20.3.1 예제

다음은 구글 지도에서 필리핀의 보라카이 섬(위도 11.96832946, 경도 121.922996308)을 출력하고 같은 좌표에 원을 표시하는 예입니다. 예제의 버튼은 각각 다음과 같은 기능을 합니다.

버튼	설명
Show Map	보라카이 섬 지도를 화면에 출력합니다.
Add Circle	지도에서 위도 11.96832946, 경도 121.922996308 좌표에 원을 추가합니다.
Del Circle	지도에 표시된 원을 삭제합니다.



 sample_googlemap_03.xfdl

20.3.2 예제에서 사용한 핵심 기능

load

구글 지도를 화면에 로딩하는 메소드입니다. 사용자 GPS 정보 사용 여부, 위도, 경도, 지도 모드, 줌 등의 설정이 가능합니다.

```
this.GoogleMap00.load(false, 11.96832946, 121.922996308, 0, 13);
```

showzoom

구글 지도에 줌(확대/축소) 컨트롤의 표시 여부를 설정하는 속성입니다.

GoogleMapCircle

지도의 특정 좌표에 원을 표시할 때 사용하는 오브젝트입니다. 원의 속성을 설정할 수 있습니다.

addItem

구글 지도에 표시할 아이템(마커, 원, 선, 다각형)을 추가하는 메소드입니다. 여러 개의 아이템을 추가할 때에는 아이템 ID를 다르게 설정해야 합니다.

removeItem

구글 지도에 표시한 아이템(마커, 원, 선, 다각형)을 제거하는 메소드입니다. 추가한 아이템이 없는 상태에서 사용하면 false를 반환하고 onerror 이벤트가 발생합니다.

20.3.3 예제 구현 방법

① 컴포넌트 생성하기

예제와 같이 폼에 GoogleMap과 Button 컴포넌트를 생성합니다.

Add Circle, Del Circle 버튼의 enable 속성을 false로 설정합니다.

② 버튼 이벤트 함수 작성하기

Show Map, Add Circle, Del Circle 버튼의 onclick 이벤트 함수를 각각 다음과 같이 작성합니다.

- Show Map 버튼 이벤트 함수

특정 좌표의 지도를 화면에 출력합니다.

```

this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.GoogleMap00.set_showzoom(true);
    this.GoogleMap00.set_apikey(nexacro.getApplication().googleMapAPIKey);
    this.GoogleMap00.load(false, 11.96832946, 121.922996308, 0, 13);
};

```

- Add Circle 버튼 이벤트 함수

GoogleMapCircle 객체를 생성하고 속성을 설정한 후 addItem 메소드로 원을 지도에 추가합니다.

```

this.Button01_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var objGoogleMapCircle = new nexacro.GoogleMapCircle();

    objGoogleMapCircle.set_latitude(11.96832946);
    objGoogleMapCircle.set_longitude(121.922996308);
    objGoogleMapCircle.set_radius(1000);
    objGoogleMapCircle.set_visible(true);

    this.GoogleMap00.addItem("MapCircle", objGoogleMapCircle);

    this.Button01.set_enable(false);
    this.Button02.set_enable(true);
};

```

- Del Circle 버튼 이벤트 함수

지도에 추가한 원을 제거합니다.

```

this.Button02_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.GoogleMap00.removeItem("MapCircle");

    this.Button01.set_enable(true);
    this.Button02.set_enable(false);
};

```

- GoogleMap onload 이벤트 함수

GoogleMap의 지도를 정상적으로 로딩하면 발생하는 이벤트입니다.

```
this.GoogleMap00_onload = function(obj:nexacro.GoogleMap,e:nexacro.GoogleMapEventInfo)
{
    trace("loading succeed");

    if(this.GoogleMap00.items[0])
    {
        this.GoogleMap00.removeItem(this.GoogleMap00.items[0]);
    }

    this.Button01.set_enable(true);
    this.Button02.set_enable(false);
};
```

- GoogleMap onerror 이벤트 함수

GoogleMap의 동작에 실패하면 발생하는 이벤트입니다.

```
this.GoogleMap00_onerror = function(obj:nexacro.GoogleMap,e:nexacro.
GoogleMapErrorEventInfo)
{
    trace("error: " + e.errormsg);
};
```

④ 퀵뷰로 확인하기

퀵뷰(Ctrl + F6)를 실행하여 결과를 확인합니다.

Show Map 버튼을 클릭하면 화면에 보라카이 지도가 출력되며 마우스와 키보드를 이용해 지도를 이동 및 확대/축소할 수 있습니다. Add Circle 버튼을 클릭하면 화면에 원을 추가하고, Del Circle 버튼을 클릭하면 원을 삭제합니다.

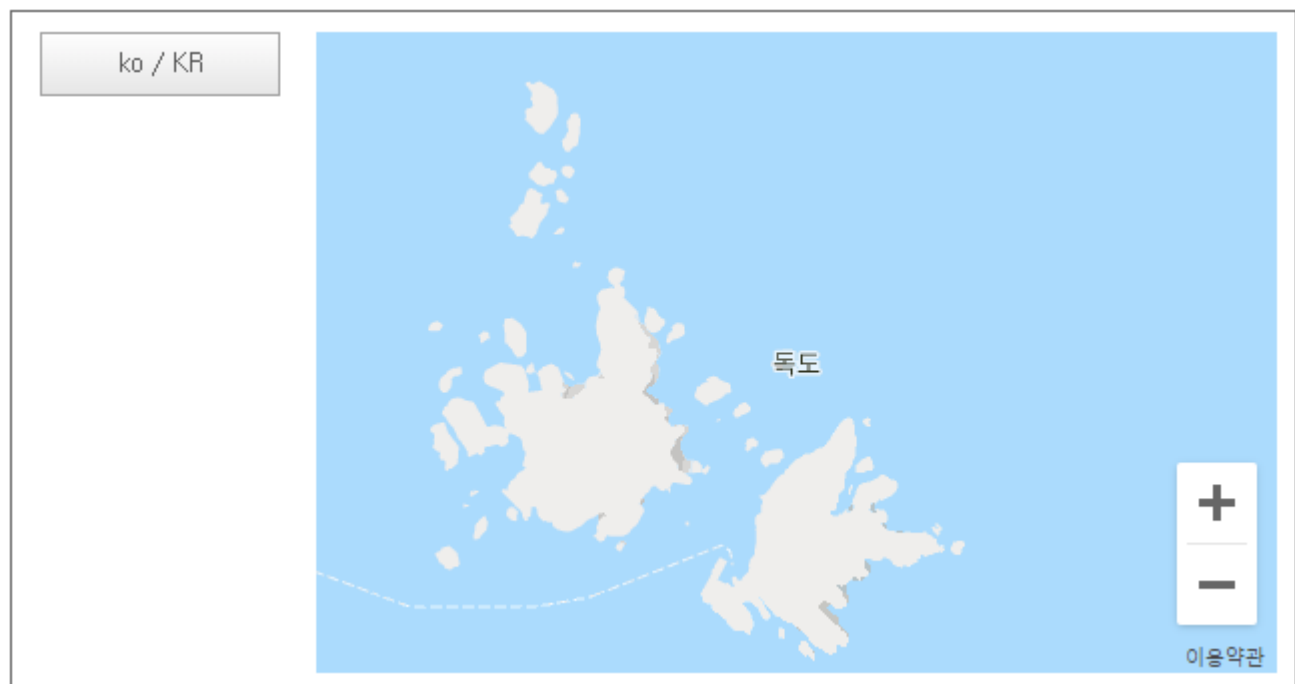
20.4 지도에 표시되는 언어 및 지역 특성 설정하기

GoogleMap 컴포넌트의 표시 언어와 지역적인 특성을 설정할 수 있습니다. 한국 영토인 독도 같은 경우 지역 특성 설정에 따라 다르게 표시됩니다.

language	region	표시되는 값
ko	KR	독도
en	KR	Dokdo
ja	KR	独島
en	US	Liancourt Rocks
ja	JP	竹島
ko	JP	다케시마

20.4.1 예제

버튼 클릭 시 언어 및 지역 설정된 지도 화면을 표시합니다. 언어 및 지역은 화면 로딩 시 설정된 후 변경할 수 없습니다. 다른 언어 및 지역을 설정하려면 화면을 새로 고침 후 사용해야 합니다.



20.4.2 예제에서 사용한 핵심 기능

language (17.1.0.200 버전부터 지원)

구글 지도에 표시할 언어를 설정합니다.

region (17.1.0.200 버전부터 지원)

구글 지도에 표시할 지역을 설정합니다. 해당 설정에 따라 같은 지명도 다르게 표시될 수 있습니다.



지원 언어와 지역에 대한 정보는 아래 링크를 참고하세요.

<https://developers.google.com/maps/documentation/javascript/localization>

20.4.3 예제 구현 방법

① Button 컴포넌트와 GoogleMap 컴포넌트를 예제 화면과 같이 배치합니다.

② Button 컴포넌트의 onclick 이벤트 함수를 아래와 같이 추가합니다.

지도 위치는 독도가 표시되는 좌표로 설정했습니다. 로딩 전에 language, region 속성값을 지정합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.GoogleMap00.set_showzoom(true);
    this.GoogleMap00.set_apikey(nexacro.getApplication().googleMapAPIKey);
    this.GoogleMap00.set_language('ko');
    this.GoogleMap00.set_region('KR');
    this.GoogleMap00.load(false, 37.243158, 131.869202, 3, 15);
};
```

③ QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 지도에 표시되는 내용을 확인합니다.

21.

Graphics

21.1 Graphics 컴포넌트 소개

Graphics 컴포넌트는 선이나 도형을 그릴 수 있는 기능을 제공합니다. Graphics 컴포넌트는 HTML5 Canvas 요소를 마치 다른 컴포넌트처럼 개발자가 익숙하게 사용할 수 있습니다. 컴포넌트의 속성, 메소드, 이벤트를 사용해 스크립트 상에서 쉽게 원하는 이미지를 만들 수 있습니다.



Graphics 컴포넌트는 17.1.0.100 버전부터 지원합니다.

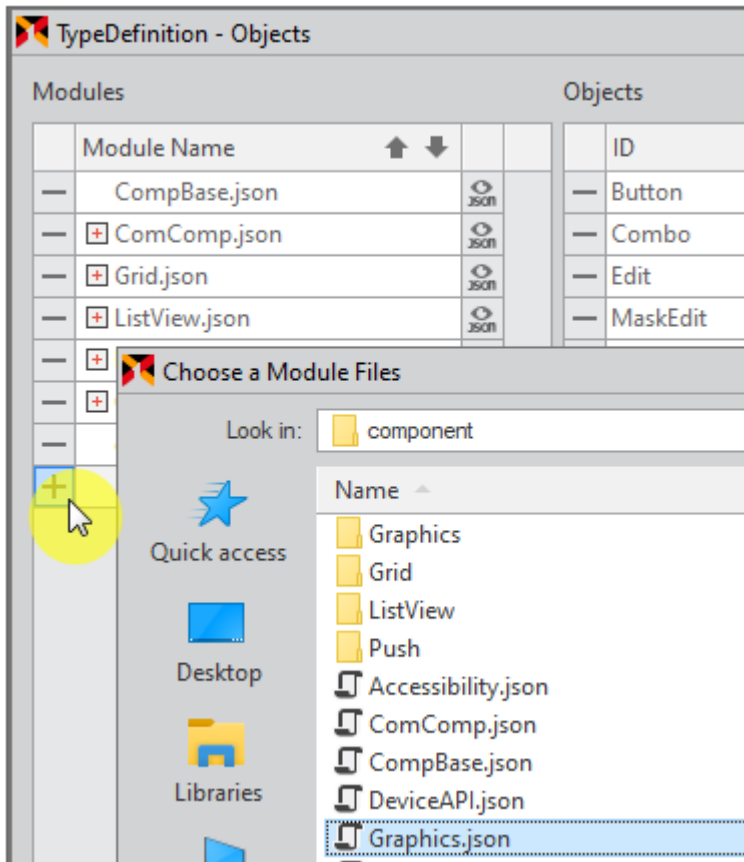
21.1.1 Graphics.json 등록하기

넥사크로 스튜디오 설치 시 Graphics 컴포넌트는 기본 설정되지 않은 상태입니다. 프로젝트 TypeDefinition 에서 Graphics.json를 등록하고 Graphics 컴포넌트를 오브젝트로 설정해주어야 합니다.

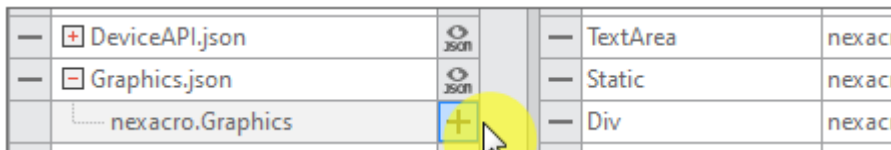
- ① 새로운 프로젝트를 만들거나 기존 프로젝트를 실행합니다.
- ② Project Explorer에서 TypeDefinition 항목 아래에 Objects 항목을 선택하고 컨텍스트 메뉴에서 [Edit]를 선택합니다.
- ③ Modules 항목에서 [+] 버튼을 클릭하고 탐색기에서 Graphics.json 파일을 선택합니다.

Graphics.json 파일은 아래 경로에서 확인할 수 있습니다.

[넥사크로 스튜디오 설치 폴더]\nexacro17lib\component\Graphics.json



- 4 Modules 항목에 Graphics.json이 등록된 것을 확인하고 nexacro.Graphics 컴포넌트를 Objects 목록에 추가합니다.



21.2 선 그리기

Graphics 컴포넌트 위에 선을 그리는 가장 기본적인 동작을 살펴봅니다.

21.2.1 예제

빈 화면에 아래와 같이 빨간색 선을 그립니다.



21.2.2 예제에서 사용한 핵심 기능

strokepen

선의 색이나 형식을 지정합니다. default 값이 없으므로 해당 속성값을 지정하지 않으면 화면에 선이 표시되지 않습니다.

21.2.3 예제 구현 방법

- ① Graphics 컴포넌트를 화면에 배치합니다.
- ② Form의 onload 이벤트 함수를 추가합니다.
Graphics 컴포넌트가 로딩되고 그 위에 선이 그려질 수 있도록 스크립트를 추가합니다.
- ③ onload 이벤트 함수의 스크립트를 아래와 같이 작성합니다.

```
var objGLine = new nexacro.GraphicsLine();
this.Graphics00.addChild( "GraphicsLine00", objGLine );
objGLine.set_x(10);
objGLine.set_y(10);
objGLine.set_x2(50);
objGLine.set_y2(50);
objGLine.set_strokepen('1px solid red');
this.Graphics00.redraw();
```

Graphics 컴포넌트 위에 필요한 컨트롤(선이나 도형)을 자식으로 등록하는 방식입니다. 예제에서는 선을 추가할 것이기 때문에 GraphicsLine 오브젝트를 생성하고 Graphics 컴포넌트의 addChild 메소드를 사용해 추가합니다.

```
var objGLine = new nexacro.GraphicsLine();
this.Graphics00.addChild( "GraphicsLine00", objGLine );
```

GraphicsLine 오브젝트의 속성을 지정해서 시작점(x, y)와 끝나는점(x2, y2)를 지정하고 선의 굵기와 색상(stroke)을 설정합니다. 지정된 속성으로 그려진 선이나 도형을 표시하려면 Graphics 컴포넌트의 redraw 메소드를 호출해주어야 합니다.

```
this.Graphics00.redraw();
```

- ④ QuickView(Ctrl + F6)로 실행한 후 선이 표시되는지 확인합니다.

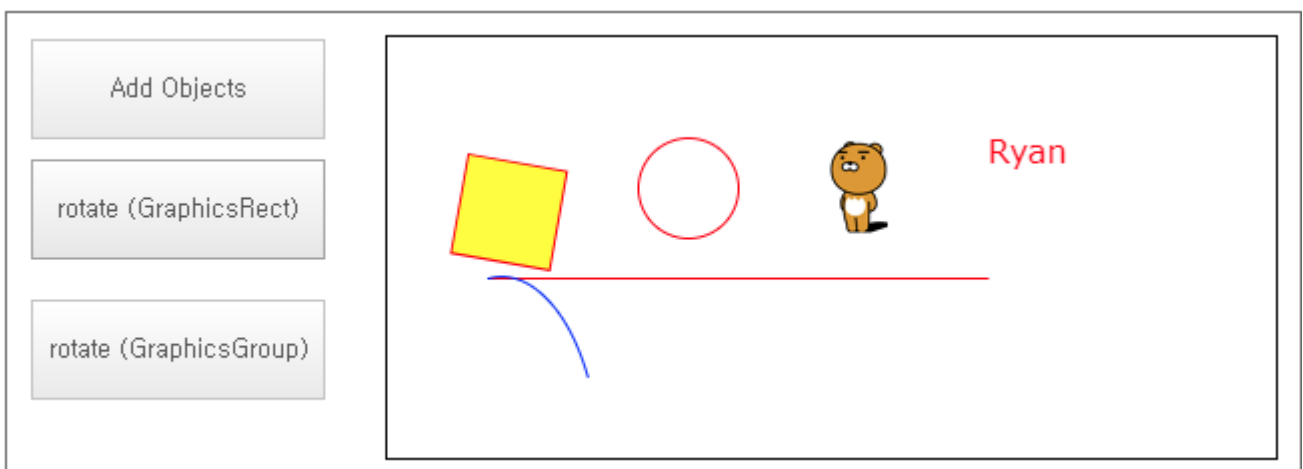
21.3 선이나 도형을 그룹으로 묶어서 처리하기

개별적인 선이나 도형을 그리는 오브젝트를 Graphics 컴포넌트에 직접 추가할 수도 있습니다. 하지만, 이렇게 구성을 한 경우에는 전체 도형의 위치를 변경해야 할 경우 개별 오브젝트의 메소드를 매번 호출해야 하는 불편함이 있습니다.

GraphicsGroup 오브젝트를 사용하면 Graphics 컴포넌트에는 GraphicsGroup 오브젝트만 추가하고 나머지 오브젝트는 GraphicsGroup 오브젝트의 자식으로 추가합니다. 이렇게 하면 GraphicsGroup 오브젝트만 이동하게 되면 나머지 도형이 같이 따라 움직이게 됩니다.

21.3.1 예제

[Add Objects] 버튼을 클릭하면 여러 형태의 오브젝트를 표시합니다. 사각형은 Graphics 컴포넌트의 자식으로 추가했고, 나머지는 GraphicsGroup 오브젝트의 자식으로 추가했습니다. [rotate] 버튼 클릭 시 사각형과 나머지 도형이 어떻게 움직이는지 확인할 수 있습니다.





21.3.2 예제에서 사용한 핵심 기능

setImageLoadEventHandler

GraphicsImage 오브젝트 사용 시 이미지 로딩 후 처리가 필요한 부분이 있을 때 사용합니다. 예제에서는 이미지의 크기(width, height)를 이미지 로딩 후 실제 이미지 값에서 가져오도록 처리했습니다.

getObjectByID

Graphics 컴포넌트, GraphicsGroup 오브젝트, GraphicsPaths 오브젝트의 자식을 ID 값으로 찾습니다. 예제에서는 Graphics 컴포넌트를 기준으로 찾았습니다. 예제처럼 Graphics 컴포넌트를 기준으로 찾을 수도 있고 GraphicsGroup 오브젝트를 기준으로 찾을 수도 있습니다.

21.3.3 예제 구현 방법

- ① Graphics 컴포넌트를 화면에 배치합니다.
- ② Button 컴포넌트 3개를 Graphics 컴포넌트 옆에 배치합니다.
- ③ 첫 번째 Button 컴포넌트의 onclick 이벤트 함수를 작성합니다.

```
this.Button01_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.addGraphicsControl();
    this.Graphics00.redraw();
};
```

- ④ addGraphicsControl 함수의 스크립트를 아래와 같이 작성합니다.

```
this.addGraphicsControl = function()
{
    //GraphicsRect
    var objGRect = new nexacro.GraphicsRect();
    this.Graphics00.addChild( "GraphicsRect00", objGRect );
    objGRect.set_x(50);
    objGRect.set_y(50);
```

```
objGRect.set_width(50);
objGRect.set_height(50);
objGRect.set_strokepen("1px solid red");

// GraphicsEllipse
var objGEllipse = new nexacro.GraphicsEllipse();
objGEllipse.set_x(150);
objGEllipse.set_y(75);
objGEllipse.set_width(50);
objGEllipse.set_height(50);
objGEllipse.set_strokepen("1px solid red");

// GraphicsText
var objGText = new nexacro.GraphicsText();
objGText.set_x(300);
objGText.set_y(50);
objGText.set_color('red');
objGText.set_font('12pt/normal Verdana');
objGText.set_text('Ryan');

// GraphicsImage
var objGImage = new nexacro.GraphicsImage();
objGImage.set_x(200);
objGImage.set_y(50);
objGImage.setImageLoadEventHandler(this.fn_checkGImage, this);
objGImage.set_image("url('imagerc:img_50.png')");

// GraphicsLine
var objGLine = new nexacro.GraphicsLine();
objGLine.set_x(50);
objGLine.set_y(120);
objGLine.set_x2(250);
objGLine.set_y2(0);
objGLine.set_strokepen('1px solid red');

// GraphicsPaths
var objGPaths = new nexacro.GraphicsPaths();
objGPaths.set_x(50);
objGPaths.set_y(120);
objGPaths.set_strokepen('1px solid blue');
```



```

var objGPath = new nexacro.GraphicsPath();
objGPaths.addChild( "GraphicsPath00", objGPath );
objGPath.moveTo(0,0);
objGPath.arcTo( 50, 50, 50, 100, 0, true, false );

// GraphicsGroup
var objGGroup = new nexacro.GraphicsGroup();
this.Graphics00.addChild( "GraphicsGroup00", objGGroup );
objGGroup.addChild("GraphicsEllipse00", objGEllipse);
objGGroup.addChild("GraphicsText00", objGText);
objGGroup.addChild("GraphicsImage00", objGImage);
objGGroup.addChild("GraphicsLine00", objGLine);
objGGroup.addChild("GraphicsPaths00", objGPaths);
}

```

GraphicsRect 오브젝트는 Graphics 컴포넌트의 자식으로 직접 추가합니다. 하지만, 나머지 오브젝트는 Graphics 컴포넌트의 자식으로 추가하지 않습니다.

```
this.Graphics00.addChild( "GraphicsRect00", objGRect );
```

GraphicsImage 오브젝트의 width, height 속성값은 직접 지정할 수도 있지만, 실제 이미지의 값으로 설정하고자 한다면 아래와 같이 이미지가 로딩되고 난 이후에 별도 함수에서 속성값을 처리하도록 합니다.

```

var objGImage = new nexacro.GraphicsImage();
objGImage.setImageLoadEventHandler(this.fn_checkGImage, this);
objGImage.set_image("url('imagerc::img_50.png')");

```

GraphicsGroup 오브젝트를 추가하고 Graphics 컴포넌트의 자식으로 추가합니다. 나머지 도형을 처리하는 오브젝트는 GraphicsGroup 오브젝트의 자식으로 추가합니다.

```

var objGGroup = new nexacro.GraphicsGroup();
this.Graphics00.addChild( "GraphicsGroup00", objGGroup );
objGGroup.addChild("GraphicsEllipse00", objGEllipse);
objGGroup.addChild("GraphicsText00", objGText);
...

```

- ⑤ GraphicsImage 오브젝트에서 이미지 로딩 후 호출하는 함수의 스크립트를 작성합니다.

이미지 로딩이 완료되는 시점에 처리되기 때문에 redraw 메소드를 실행합니다.

```

this.fn_checkGImage = function()
{
    var objGImage = this.Graphics00.getObjectByID("GraphicsImage00");
    objGImage.set_width(objGImage.imagewidth);
    objGImage.set_height(objGImage.imageheight);
    this.Graphics00.redraw();
}

```

- ⑥ 두 번째 Button 컴포넌트의 onclick 이벤트 함수를 작성합니다.

GraphicsRect 오브젝트만 찾아서 rotate 메소드를 실행합니다.

```

this.Button02 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var objGRect = this.Graphics00.getObjectByID("GraphicsRect00");
    objGRect.set_fillstyle("yellow");
    objGRect.rotate(10);
    this.Graphics00.redraw();
};

```

- ⑦ 세 번째 Button 컴포넌트의 onclick 이벤트 함수를 작성합니다.

GraphicsGroup 오브젝트만 찾아서 rotate 메소드를 실행합니다.

```

this.Button03 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var objGGroup = this.Graphics00.getObjectByID("GraphicsGroup00");
    objGGroup.rotate(10);
    this.Graphics00.redraw();
};

```

- ⑧ QuickView(Ctrl + F6)로 실행한 후 각 버튼을 클릭해서 도형이 어떻게 반응하는지 확인합니다.

21.4 간단한 그림판 만들기

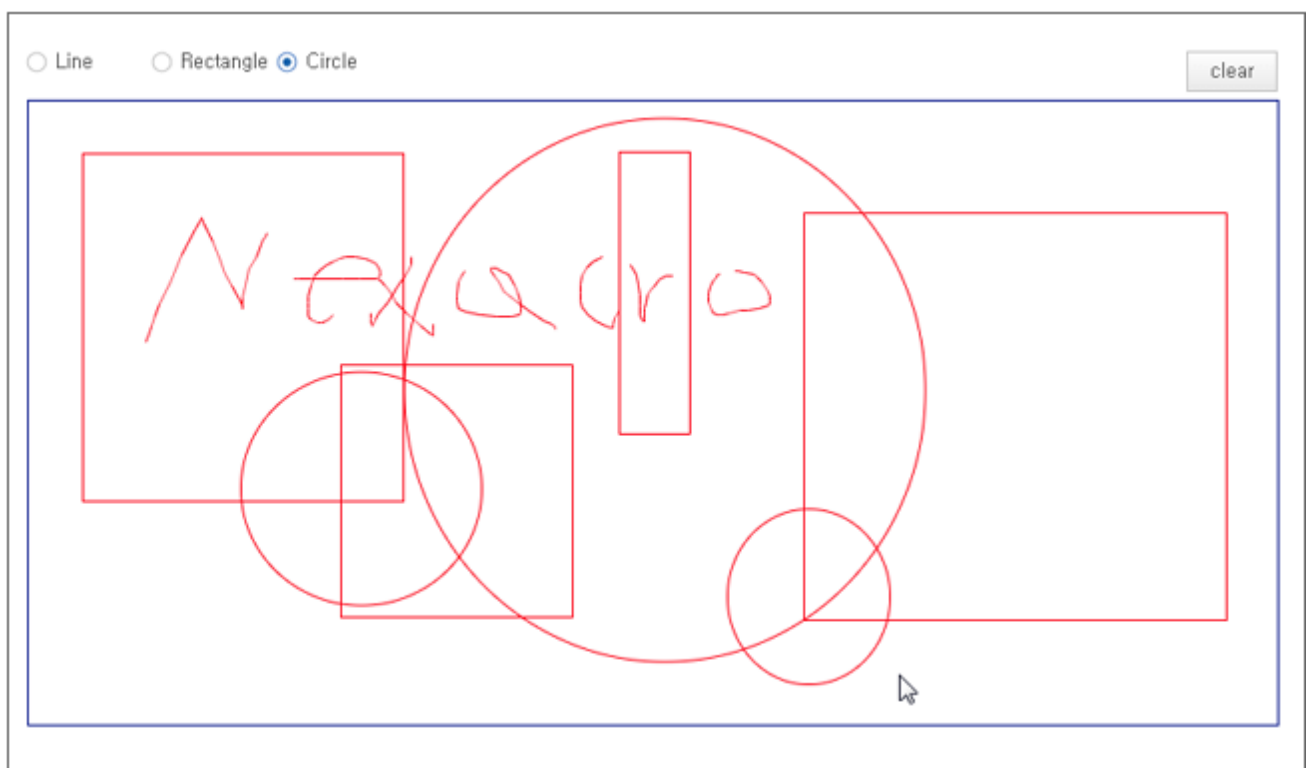
그림판처럼 선을 그리거나 도형을 추가할 수 있는 기능을 구현해봅니다.



스크립트 코드는 공개된 자료를 참고했습니다.
<https://icechou.tistory.com/158>

21.4.1 예제

빈 화면에 Line, Rectangle, Circle 중 하나의 항목을 선택하고 마우스로 드래그하면 도형이 표시됩니다. [clear] 버튼을 클릭하면 화면에 그려진 도형이 모두 사라집니다.



sample_graphics_03.xfdl

21.4.2 예제에서 사용한 핵심 기능

clear

모든 자식 오브젝트를 초기화합니다. clear 메소드 실행 후 redraw 메소드를 호출해주어야 실제 화면에서 초기화가 처리됩니다.

21.4.3 예제 구현 방법

- ① Graphics 컴포넌트를 화면에 배치합니다.
- ② Radio 컴포넌트와 Button 컴포넌트를 Graphics 컴포넌트 위에 배치합니다.
Radio 컴포넌트의 innerdataset 항목으로 Line, Rectangle, Circle 항목을 설정합니다.
- ③ 상태값을 처리할 변수를 선언하고 Graphics 컴포넌트에서 마우스 상태에 따른 이벤트 함수를 추가합니다.

Graphics 컴포넌트 위에서 마우스 버튼이 눌러진 시점(onmousedown)에 Radio 컴포넌트의 값을 체크해서 어떤 동작을 할지 결정하고 버튼이 눌러진 상태에서 마우스가 움직이면(onmousemove) 도형에 따라 그리기 동작을 처리합니다. 마우스 버튼을 떼면(onmouseup) 상태값을 초기화합니다.

```
var bMouseButtonPressed = false;
var preClientX;
var preClientY;
var nNUm = 0;
var temprightangle;
var tempellipse;
var drawType;
this.Graphics00_onmousedown = function(obj:nexacro.Graphics,e:nexacro.MouseEventInfo)
{
    bMouseButtonPressed = true;
    preClientX = e.clientX;
    preClientY = e.clientY;
    drawType = this.Radio00.value;
};

this.Graphics00_onmouseup = function(obj:nexacro.Graphics,e:nexacro.MouseEventInfo)
{
    bMouseButtonPressed = false;
    temprightangle = null;
```

```

    tempellipse = null;
};

this.Graphics00_onmousemove = function(obj:nexacro.Graphics,e:nexacro.MouseEventInfo)
{
    switch(drawType) {
    case 'Line':
        this.fn_drawLine(e);
        break;
    case 'Rectangle':
        this.fn_drawRect(e);
        break;
    case 'Circle':
        this.fn_drawEllipse(e);
        break;
    default:
    }
};

```

- ④ 선을 그리기 위한 스크립트를 아래와 같이 작성합니다.

```

this.fn_drawLine = function(e:nexacro.MouseEventInfo)
{
    var nowClientX = e.clientX;
    var nowClientY = e.clientY;
    if(bMouseButtonPressed)
    {
        var objGLine = new nexacro.GraphicsLine();
        objGLine.set_x1(preClientX);
        objGLine.set_x2(nowClientX);
        objGLine.set_y1(preClientY);
        objGLine.set_y2(nowClientY);
        objGLine.set_strokepen('1px solid red');
        this.Graphics00.addChild( "GraphicsControl_"+nNum, objGLine );
        this.Graphics00.redraw();

        preClientX = nowClientX;
        preClientY = nowClientY;
        nNum++;
    }
}

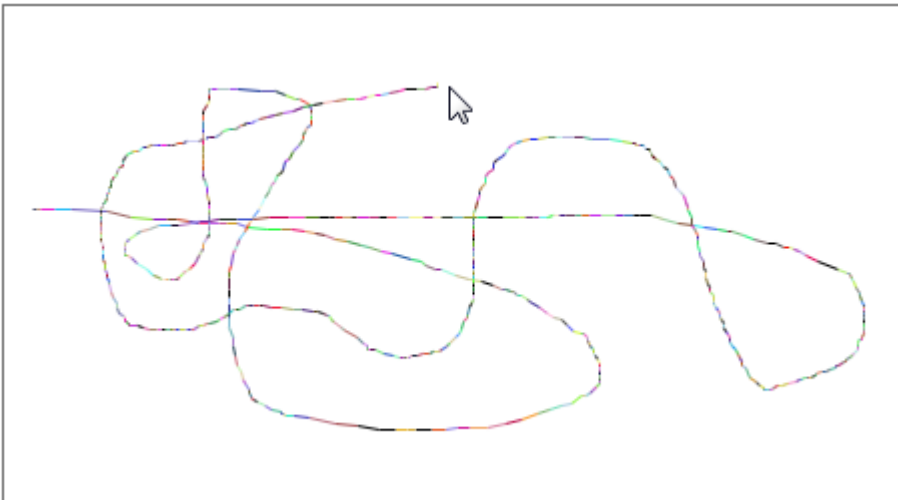
```

```
};
```

Graphics 컴포넌트의 onmousemove 이벤트가 발생할 때마다 새로운 GraphicsLine 오브젝트를 생성하고 선을 이어 그려줍니다.

예를 들어 색상 코드값을 랜덤으로 처리해서 strokepen 속성값을 지정했다면 아래와 같은 색으로 선이 그려집니다. 하나의 선이 아니라 여러 개의 선이 이어져서 하나의 선처럼 보이는 상태입니다.

```
var colorCode = "#" + Math.round(Math.random()*0xffffffff).toString(16);
objGLine.set_strokepen('1px solid '+colorCode);
```



- 5 사각형을 그리기 위한 스크립트를 아래와 같이 작성합니다.

```
this.fn_drawRect = function(e:nexacro.MouseEventInfo)
{
    var nowClientX = e.clientX;
    var nowClientY = e.clientY;
    if(bMouseButtonPressed)
    {
        var objGRect = new nexacro.GraphicsRect();
        var left = preClientX;
        var top = preClientY;
        var width = nowClientX - preClientX;
        var height = nowClientY - preClientY;

        if(nowClientX < preClientX)
        {
            left = nowClientX;
            width *= -1;
        }
    }
}
```

```

    }
    if(nowClientY < preClientY)
    {
        top = nowClientY;
        height *= -1;
    }

    objGRect.set_x(left);
    objGRect.set_y(top);
    objGRect.set_width(width);
    objGRect.set_height(height);
    objGRect.set_strokepen('1px solid red');
    if(temprectangle)
    {
        this.Graphics00.removeChild(temprectangle);
    }
    temprectangle = "GraphicsControl_"+nNum;
    this.Graphics00.addChild("GraphicsControl_"+nNum, objGRect );
    this.Graphics00.redraw();

    nNum++;
}
};

```

마우스 포인터가 시작점에서 오른쪽 아래 방향이 아니라 왼쪽이나 위로 움직이는 경우 시작점을 보정해주는 코드를 추가했습니다.

```

if(nowClientX < preClientX)
{
    left = nowClientX;
    width *= -1;
}

```

그리고 마우스 포인터가 움직일 때 임시로 사각형을 표시해주고 삭제합니다. 이렇게 하면 최종적으로 사각형이 그려질 위치를 사용자가 알 수 있습니다.

```

if(temprectangle)
{
    this.Graphics00.removeChild(temprectangle);
}

```

- 6 원을 그리기 위한 스크립트를 아래와 같이 작성합니다.

```

this.fn_drawEllipse = function(e:nexacro.MouseEventInfo)
{
    var nowClientX = e.clientX;
    var nowClientY = e.clientY;
    if(bMouseButtonPressed)
    {
        var objGEllipse = new nexacro.GraphicsEllipse();
        var left = preClientX;
        var top = preClientY;
        var width = nowClientX - preClientX;
        var height = nowClientY - preClientY;

        if(nowClientX < preClientX)
        {
            left = nowClientX;
            width *= -1;
        }
        if(nowClientY < preClientY)
        {
            top = nowClientY;
            height *= -1;
        }

        objGEllipse.set_x(left+(width/2));
        objGEllipse.set_y(top+(height/2));
        objGEllipse.set_width(width);
        objGEllipse.set_height(height);
        objGEllipse.set_strokepen('1px solid red');
        if(tempellipse)
        {
            this.Graphics00.removeChild(tempellipse);
        }
        tempellipse = "GraphicsControl_"+nNum;
        this.Graphics00.addChild("GraphicsControl_"+nNum, objGEllipse );
        this.Graphics00.redraw();

        nNum++;
    }
};

```


- ⑦ [clear] 버튼 클릭 시 onclick 이벤트 함수를 작성합니다.

```
this.Button00.onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Graphics00.clear();
    this.Graphics00.redraw();
};
```

- ⑧ QuickView(Ctrl + F6)로 실행한 후 그림판 위에 자유롭게 그림이나 도형을 그려봅니다.

22.

Grid 기본

22.1 Grid 소개

Grid는 데이터를 격자 모양의 테이블 형태로 표현하는 컴포넌트로 데이터를 질서 있게 나열하고자 할 때 사용합니다. 단순히 데이터를 보여주거나 내부적으로 연산을 처리할 수도 있으며 사용자가 원하는 형식으로 모양을 변경하여 보여줄 수 있습니다.

Grid는 head, body, summary라는 3개의 밴드(band) 영역으로 구성되며, 각 밴드는 셀(cell)이라는 기본 단위로 구성됩니다.

- head 밴드: Grid의 상단에 위치하며 주로 컬럼 명을 표시하는 용도로 사용합니다.
- body 밴드: 바인딩된 데이터가 표시되는 영역입니다.
- summary 밴드: Grid의 하단에 위치하며 합계, 평균, 최고값, 최소값 등 연산한 결과를 표시하는 용도로 사용됩니다.

Column						
Name	Address	Company	Department	EMail	Salary	Head
John	Seoul, KOREA	nexaweb	Sales	sales@tobesoft.co.kr	₩15,000	Body
James	Barcelona, Spain	tobesoft	Consulting	consulting@tobesoft.com	₩25,000	
Donald	Califonia, USA	tobesoft	Research Institute	research@nexaweb.com	₩1,000,000	
Lisa	London, United Kingdom	nexaweb	Quality Assurance	quality@nexaweb.co.jp	₩3,000	
Joe	São Paulo, Brazil	hangul	R&D	rnd@hangul.com	₩8,600	
Michael	Mumbai, India	nexaweb	Personnel	personnel@nexaweb.com	₩1,300	
Lilli	Ankara, Turkey	nexaweb	Sales	sales@nexaweb.com	₩4,400	
Colt	Sydney, NSW, Australia	miplatform	Sales	sales2@nexaweb.com	₩62,000	Summary
TOTAL					₩1,119,300	

셀은 Grid를 구성하는 오브젝트중 하나로 Grid와 별도의 속성을 갖습니다. Dataset의 컬럼을 바인딩해 사용할 수도 있고, 셀의 displaytype, edittype 속성을 설정하면 Button, Combo, CheckBox, Image, ProgressBar, Edit 등의 컴포넌트 형태로 표현하거나 트리(tree) 형태로도 표현할 수 있습니다.

tree	bar	button	checkbox	combo	currency	date	image	number	text
John	20	Profile	☒	Male	₩15,000	2016-01-05		15000 nexaweb	
James	40	Profile	☐	Male	₩25,000	2010-08-26		25000 tobesoft	
Donald	60	Profile	☐	Male	₩1,000,000	2012-03-21		1000000 tobesoft	
Lisa	100	Profile	☒	Female	₩3,000	< 2012.03 >			3000 nexaweb
Joe	80	Profile	☒	Female	₩8,600	S M T W T F S			8600 hangul
Michael	0	Profile	☐	Male	₩1,300		1	2	3
Lilli	50	Profile	☐	Female	₩4,400	4 5 6 7 8 9 10			4400 nexaweb
Colt	90	Profile	☒	Male	₩62,000	11 12 13 14 15 16 17			62000 hangul
						18 19 20 21 22 23 24			
						25 26 27 28 29 30 31			

Grid는 데이터를 시각적으로 표현하기 위한 형태를 제공할 뿐이며 Grid 자체는 데이터를 갖지 않습니다. 따라서 데이터를 Grid로 표현하려면 형태를 제공하는 Grid와 데이터 정보를 갖는 Dataset을 연결해주는 바인딩 작업이 반드시 필요합니다.

Dataset은 눈에 보이지 않는 오브젝트로 데이터로 구성된 집합입니다. Dataset은 데이터만을 가지고 있으므로 그 자체만으로는 의미가 없고 Grid와 같은 UI 컴포넌트와 함께 사용해야 합니다.

Dataset과 바인딩에 관한 자세한 설명은 [Dataset](#)을 참조하시기 바랍니다.

22.1.1 사용 예

다음은 Grid를 사용한 예입니다. Grid를 사용하면 복잡하고 많은 데이터 정보를 사용자에게 일목요연하게 전달할 수 있습니다.

Desktop Chipsets	Category	Max TDP	Halogen Free	USB Ports	PCIe* 2.0 Ports	SATA Ports
Intel® Z170 Chipset	Performance		Yes	14 (up to six USB 3.0)	Up to eight	Six SATA 3 Gbps (up to six SATA 6 Gbps)
Intel® X99 Chipset	Performance	7 W	Yes	14 (up to six USB 3.0)	Up to eight	Ten SATA 3 Gbps (up to ten SATA 6 Gbps)
Intel® Z97 Chipset	Performance	4.1 W	Yes	14 (up to six USB 3.0)	Up to eight	Six SATA 3 Gbps (up to six SATA 6 Gbps)
Intel® Z87 Chipset	Performance	4.1 W	Yes	14 (up to six USB 3.0)	Up to eight	Six SATA 3 Gbps (up to six SATA 6 Gbps)
Intel® Q87 Chipset	Performance and Business	4.1 W	Yes	14 (up to six USB 3.0)	Up to eight	Six SATA 3 Gbps (up to six SATA 6 Gbps)

22.1.2 Grid 만들기

1 Grid 컴포넌트 생성하기

툴바에서 Grid  를 선택한 후, Form 위에 적당한 크기로 드래그하여 생성합니다.

2 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Grid에 표시할 데이터가 없으므로 아무것도 표시되지 않습니다. Grid의 모양을 확인하려면 border 속성을 설정하여 테두리를 만들어줍니다.

22.2 Grid에 Dataset 바인딩하기

Grid는 형태가 Dataset과 논리적으로 같은 컴포넌트입니다. 따라서 Grid는 Dataset과 완전히 매핑되는데 이런 식으로 Dataset과 컴포넌트가 1대1로 매핑되는 바인딩을 풀 바인딩(full binding)이라고 합니다. 여러 가지 컴포넌트 중 오직 Grid만이 Dataset과 풀 바인딩됩니다.

Grid에 Dataset이 바인딩된 경우에는 Grid의 동작이 Dataset에 영향을 미치게 됩니다. 예를 들어 바인딩된 Grid에서로우나 셀의 포커스를 이동하는 경우 Dataset의 rowposition도 같이 변경됩니다.

22.2.1 예제

다음은 Grid 컴포넌트에 Name, Address, Company, Department, Salary 정보를 갖는 Dataset을 바인딩한 예입니다.

Name	Address	Company	Department	Salary
John	Seoul, KOREA	1dollar	Sales	₩15,000
James	Barcelona, Spain	EBOT	Consulting	₩25,000
Donald	Califonia, USA	EBOT	Research Institute	₩1,000,000
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
Joe	São Paulo, Brazil	hangul	Quality Assurance	₩8,600
Michael	Mumbai, India	1dollar	Quality Assurance	₩1,300
Lilli	Ankara, Turkey	hangul	Quality Assurance	₩4,400
Colt	Sydney, NSW, Australia	hangul	R&D	₩62,000



22.2.2 예제에서 사용한 핵심 기능

binddataset

Grid에 바인딩할 DataSet의 ID를 설정하는 속성입니다.

22.2.3 예제 구현 방법

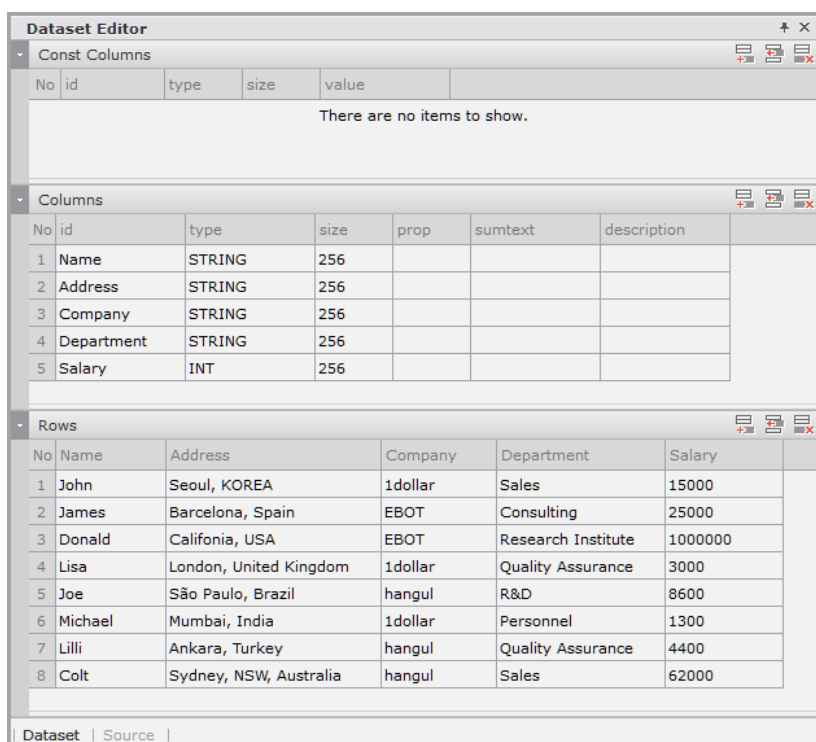
① Grid 컴포넌트 생성하기

툴바에서 Grid 를 선택한 후, Form위에 적당한 크기로 드래그하여 배치합니다.

② Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 정보를 갖는 Dataset을 생성합니다.

1. 툴바에서 Dataset 을 선택한 후, Form 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터를 입력합니다. Invisible Objects 영역에 있는 Dataset00을 더블 클릭하면 Dataset 편집기가 뜨는데 Columns과 Rows에 다음과 같이 입력합니다.



Dataset Editor

Const Columns

No	id	type	size	value
There are no items to show.				

Columns

No	id	type	size	prop	sumtext	description
1	Name	STRING	256			
2	Address	STRING	256			
3	Company	STRING	256			
4	Department	STRING	256			
5	Salary	INT	256			

Rows

No	Name	Address	Company	Department	Salary
1	John	Seoul, KOREA	1dollar	Sales	15000
2	James	Barcelona, Spain	EBOT	Consulting	25000
3	Donald	Califonia, USA	EBOT	Research Institute	1000000
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000
5	Joe	São Paulo, Brazil	hangul	R&D	8600
6	Michael	Mumbai, India	1dollar	Personnel	1300
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000

Dataset | Source |

③ Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset00을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

④ QuickView로 확인하기

Grid에 Dataset이 성공적으로 바인딩됐는지 QuickView(Ctrl + F6)로 확인합니다.

예제는 Grid에 표현되는 데이터 목록이 잘려 보이지 않도록 Grid의 속성을 설정하였기 때문에 사용자의 결과와 달라 보일 수 있습니다.

22.3 조건에 맞는 레코드 필터링하기

Grid에서 출력되는 여러 레코드 중 원하는 레코드만 보고 싶은 경우에는 filter 메소드를 사용합니다. 원하는 조건 표현식(Conditional Expression)을 작성하여 filter 메소드로 호출하면 Grid에 조건에 맞는 레코드만 표시해 볼 수 있습니다.

filter 메소드를 수행한 후 필터링 조건을 변경하여 다시 호출하면 이전에 적용되어 있던 필터링 조건은 무시되고 새로운 필터링 조건이 적용됩니다.

필터링 기능은 filterstr 속성을 사용해서도 구현할 수 있습니다. filterstr 속성은 filter 메소드와 같은 기능을 합니다.

22.3.1 예제

다음은 Grid에서 출력하길 원하는 데이터를 조건 표현식으로 필터링하는 예제입니다.

사용자는 조건 표현식을 입력하고 Filter 버튼을 눌러 Grid에서 필터링된 결과를 확인합니다. Cancel 버튼을 누르면 필터링이 제거되고 원본 데이터로 돌아갑니다.

Name	Address	Company	Department	Salary
John	Seoul, KOREA	1dollar	Sales	₩15,000
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000

Conditional Expression:

Filter

Cancel



조건 표현식은 [Column 명][기호][값]의 형태로 작성합니다. 예를 들어,

Name == "James"는 이름이 James인 레코드를 필터링합니다.

Salary < 20000는 급여가 20,000보다 작은 레코드를 필터링합니다.

또한, 필요에 따라 여러 가지 조건을 복합적으로 작성할 수도 있습니다.

Department == "Sales" || Department == "Consulting"은 부서가 Sales 혹은 Consulting인 레코드를 필터링합니다.

Company=="hangul" && Salary < 10000는 회사가 hangul이면서 급여가 10000 미만인 레코드를 필터링합니다.

22.3.2 예제에서 사용한 핵심 기능

filter

Dataset에서 조건에 만족하는 레코드만 보이게 필터링하는 메소드입니다.

```
this.Dataset00.filter();
this.Dataset00.filter("");
this.Dataset00.filter("dept_cd == 'A1'");
this.Dataset00.filter("amount > 1000 && model_cd = 'A-1'");
this.Dataset00.filter("amount > 1000 || amount < 50 ");
```

22.3.3 예제 구현 방법

1 화면 구성하기

툴바에서 Grid , Static , Edit , Button  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택한 후 Form 위에 놓고 그대로 클릭하면 기본 크기로 생성됩니다.

2 Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, Form 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset00을 더블 클릭하면 Dataset 편집기가 뜨는데 컬럼과 로우를 아래 그림과 같이 입력합니다.

Dataset Editor

Const Columns

No	id	type	size	value
There are no items to show.				

Columns

No	id	type	size	prop	sumtext	description
1	Name	STRING	256			
2	Address	STRING	256			
3	Company	STRING	256			
4	Department	STRING	256			
5	Salary	INT	256			

Rows

No	Name	Address	Company	Department	Salary
1	John	Seoul, KOREA	1dollar	Sales	15000
2	James	Barcelona, Spain	EBOT	Consulting	25000
3	Donald	Califonia, USA	EBOT	Research Institute	1000000
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000
5	Joe	São Paulo, Brazil	hangul	R&D	8600
6	Michael	Mumbai, India	1dollar	Personnel	1300
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000

Dataset | Source |

③ Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset00을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

① Edit 컴포넌트에 기본 조건식 설정하기

Edit 컴포넌트의 value 속성에 아래와 같이 기본 조건식을 설정합니다. 조건식은 Company 컬럼이 "1dollar" 이고 Salary 컬럼이 "2000" 이상인 레코드를 의미합니다. 샘플 동작을 위한 조건식으로 임의로 변경하여도 상관없습니다.

```
Company == "1dollar" && Salary > 2000
```

② Button 이벤트 함수 작성하기

- filter 버튼의 onclick 이벤트 함수

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var strCondExpr = this.Edit00.value;

    this.Dataset00.filter(strCondExpr);
};
```

- cancel 버튼의 onclick 이벤트 함수

```
this.Button01 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Dataset00.filter("");
};
```

③ QuickView로 확인하기

Grid에 Dataset이 성공적으로 바인딩됐는지 QuickView(Ctrl + F6)로 확인합니다.

Conditional Expression에 조건식을 입력하고 filter 버튼을 눌러 조건에 맞는 레코드만 필터링되는지 확인합니다. cancel 버튼을 누르면 조건식 필터링이 해제되어 원래 상태로 돌아옵니다.

본 예제는 Grid에 표현되는 데이터 목록이 잘려 보이지 않도록 Grid의 속성을 설정하였기 때문에 사용자의 결과와 달라 보일 수 있습니다.

22.4 조건에 맞는 특정 레코드 검색하기

Grid에서는 findRow, findRowExpr 메소드를 사용하여 검색할 범위를 지정하고 그 안에서 조건에 맞는 레코드를 검색할 수 있습니다.

22.4.1 예제

아래는 Grid에서 findRow, findRowExpr 메소드를 사용하여 원하는 데이터를 검색하는 예제입니다. findRow와 findRowExpr 메소드는 검색한 레코드의 인덱스를 반환하는데 예제에서는 사용자가 바로 알아볼 수 있도록 해당 인덱스로 포커스를 이동시킵니다.

- findRow 기능

사용자가 ❶에서 컬럼을 선택하고, ❷에 조건 값을 기술한 후 ❸findRow 버튼을 누릅니다. 조건에 맞는 로우로 포커스가 이동합니다.

- findRowExpr 기능

사용자가 ❹에 조건 표현식을 기술하고 ❺findRowExpr 버튼을 누릅니다. 조건 표현식에 맞는 레코드로 포커스가 이동합니다.

Name	Address	Company	Department	Salary
John	Seoul, KOREA	1dollar	Sales	₩15,000
James	Barcelona, Spain	EBOT	Consulting	₩25,000
Donald	Califonia, USA	EBOT	Research Institute	₩1,000,000
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
Joe	São Paulo, Brazil	hangul	Quality Assurance	₩8,600
Michael	Mumbai, India	1dollar	Quality Assurance	₩1,300
Lilli	Ankara, Turkey	hangul	Quality Assurance	₩4,400
Colt	Sydney, NSW, Australia	hangul	R&D	₩62,000

❶
Name

❷
Lisa

❸
findRow

❹
Company == "1dollar" && Salary < 3000

❺
findRowExpr

22.4.2 예제에서 사용한 핵심 기능

findRow

DataSet에서 지정한 컬럼에서 입력된 값과 일치하는 데이터를 갖는 첫 번째 로우의 인덱스를 반환하는 메소드입니다.

```
var nRow;
nRow = this.Dataset00.findRow( "column00", "100");
nRow = this.Dataset00.findRow( "column00", "100", 10);
nRow = this.Dataset00.findRow( "column00", "100", 10, 10000);
```

findRowExpr

특정 컬럼의 값 중에서 주어진 조건표현식을 만족하는 첫 번째 로우의 인덱스를 반환하는 메소드입니다.

```
var nRow;
nRow = this.Dataset00.findRowExpr("dept_cd == 'A2'");
nRow = this.Dataset00.findRowExpr("dept_cd == 'A2'", 2);
nRow = this.Dataset00.findRowExpr("dept_cd == 'A2'", 2, 10);
nRow = this.Dataset00.findRowExpr("dept_cd == 'A2' && pos_cd > '03'", 2, 10);
```

22.4.3 예제 구현 방법

① 화면 구성하기

툴바에서 Grid , Combo , Edit , Button  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택한 후 Form 위에 놓고 그대로 클릭하면 기본 크기로 생성됩니다.

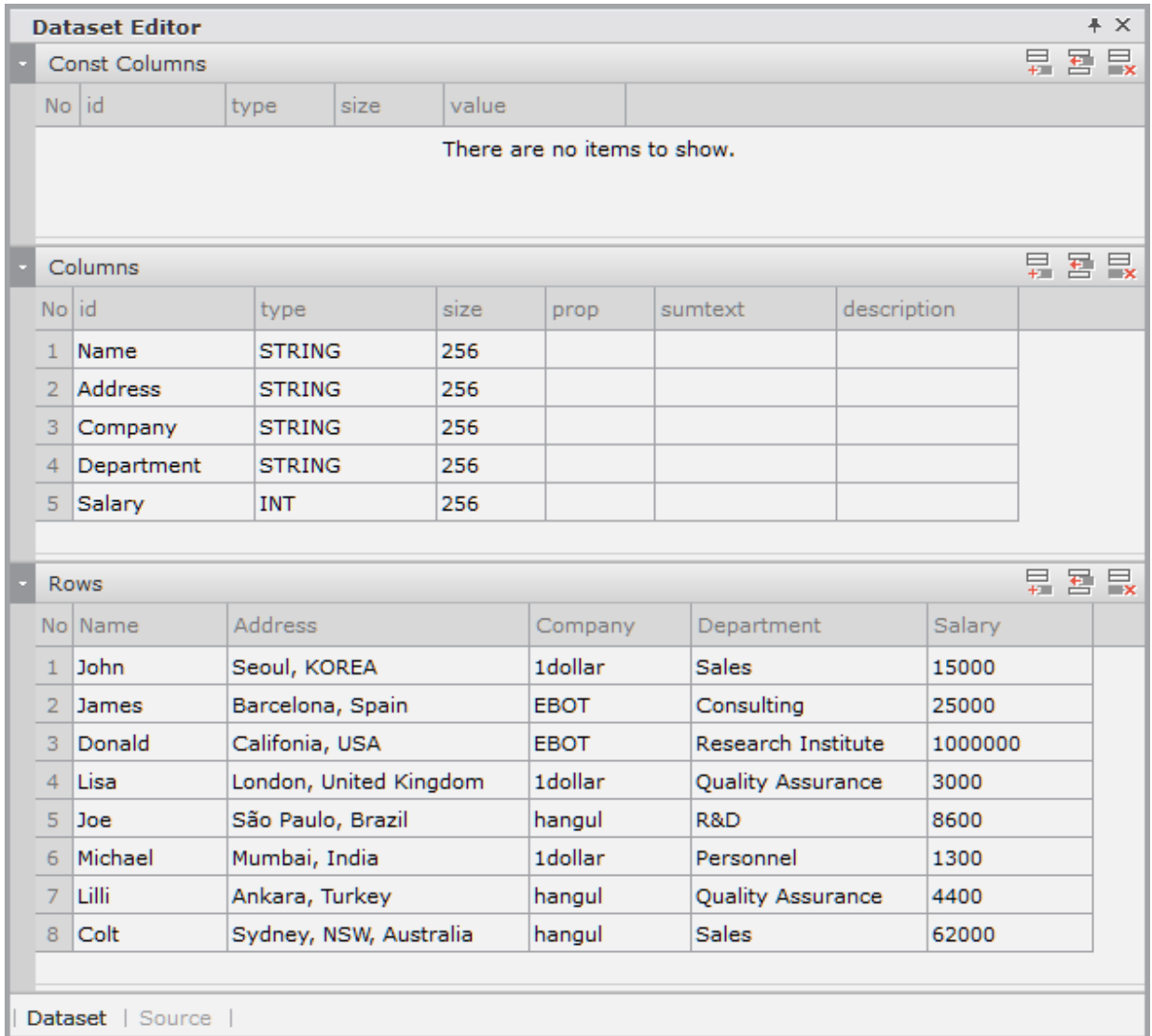
생성한 컴포넌트의 ID를 다음과 같이 변경합니다.

컴포넌트	ID
Combo00	combo_category
Edit00	edit_keyword
Edit01	edit_condexpr
Button00	btn_findRow
Button01	btn_findRowExpr

② Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, Form 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset00을 더블 클릭하면 Dataset Editor가 뜨는데 컬럼과 로우를 다음 그림과 같이 입력합니다.



Dataset Editor

Const Columns

No	id	type	size	value
There are no items to show.				

Columns

No	id	type	size	prop	sumtext	description
1	Name	STRING	256			
2	Address	STRING	256			
3	Company	STRING	256			
4	Department	STRING	256			
5	Salary	INT	256			

Rows

No	Name	Address	Company	Department	Salary
1	John	Seoul, KOREA	1dollar	Sales	15000
2	James	Barcelona, Spain	EBOT	Consulting	25000
3	Donald	Califonia, USA	EBOT	Research Institute	1000000
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000
5	Joe	São Paulo, Brazil	hangul	R&D	8600
6	Michael	Mumbai, India	1dollar	Personnel	1300
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000

Dataset | Source |

③ Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset00을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

④ findRow 기능 구현

- Combo 메뉴 만들기

Combo를 선택한 후 Properties 창에서 innerdataset 속성 옆의 [...] 버튼을 클릭합니다. InnerDataset 편집기에 다음과 같이 입력한 후 [OK] 버튼을 누르면 바인딩이 완료됩니다.

No	codecolumn	datacolumn
1	00	Name
2	01	Address
3	02	Company
4	03	Department
5	04	Salary

- findRow 버튼의 onclick 이벤트 함수

```
this.btn_findRow_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var strCategory = this.combo_category.text;
    var strKeyword = this.edit_keyword.value;

    var nRow = this.Dataset00.findRow(strCategory, strKeyword);

    this.Grid00.selectRow(nRow, true);

    if(nRow == -1)
        alert("\"" + strKeyword + "\" not found.");
};
```

5 findRowExpr 기능 구현

Edit 컴포넌트의 value 속성에 다음과 같이 조건식을 설정합니다. 조건식은 Company 컬럼이 '1dollar'이고 Salary 컬럼이 '3000' 이하인 레코드를 의미하며 사용자가 임의로 변경하여도 상관없습니다.

```
Company == '1dollar' && Salary < 3000
```

- findRowExpr 버튼의 onclick 이벤트 함수

```
this.btn_findRowExpr_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var strKeyword = this.edit_condexpr.value;

    var nRow = this.Dataset00.findRowExpr(strKeyword);

    this.Grid00.selectRow(nRow, true);
```

```
if(nRow == -1)
    alert("\"" + strKeyword + "\" not found.");
};
```

6 QuickView로 확인하기

Grid에 Dataset이 성공적으로 바인딩됐는지 QuickView(Ctrl + F6)로 확인합니다.

findRow 버튼을 눌러 조건에 맞는 레코드를 찾아 포커스가 이동하는지 확인합니다. Combo에서 컬럼을 선택하고 검색 조건을 입력하여 findRow 기능을 확인합니다.

findRowExpr 버튼을 눌러 조건에 맞는 레코드를 찾아 포커스가 이동하는지 확인합니다. 조건식을 변경하여 findRowExpr 기능을 확인합니다.

본 예제는 Grid에 표현되는 데이터 목록이 잘려 보이지 않도록 Grid의 속성을 설정하였기 때문에 사용자의 결과와 달라 보일 수 있습니다.

22.5 레코드 추가/삭제하기

Grid에서는 addRow, deleteRow 메소드를 사용하여 레코드를 추가하거나 삭제할 수 있습니다.

22.5.1 예제

새로운 로우를 추가하려면 addRow 버튼을 누르고, 로우를 삭제하려면 deleteRow 버튼을 누릅니다.

addRow는 마지막 로우의 뒤에 새로운 로우를 추가합니다. deleteRow는 현재 선택된 로우를 삭제합니다.

Name	Address	Company	Department	Salary	^
Donald	Califonia, USA	EBOT	Research Institute	₩1,000,000	
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000	
Joe	São Paulo, Brazil	hangul	Quality Assurance	₩8,600	
Michael	Mumbai, India	1dollar	Quality Assurance	₩1,300	
Lilli	Ankara, Turkey	hangul	Quality Assurance	₩4,400	
Colt	Sydney, NSW, Australia	hangul	R&D	₩62,000	
					v

addRow

deleteRow



22.5.2 예제에서 사용한 핵심 기능

addRow

Dataset의 마지막 로우 뒤에 새로운 로우를 추가하는 메소드입니다.

deleteRow

Dataset에서 지정한 로우를 삭제하는 메소드입니다.

22.5.3 예제 구현 방법

① 화면 구성하기

툴바에서 Grid , Button  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택한 후 Form 위에 놓고 그대로 클릭하면 기본 크기로 생성됩니다.

② Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, Form 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset00을 더블 클릭하면 Dataset 편집기가 뜨는데 컬럼과 로우를 다음 그림과 같이 입력합니다.

Dataset Editor

Const Columns

No	id	type	size	value	
There are no items to show.					

Columns

No	id	type	size	prop	sumtext	description	
1	Name	STRING	256				
2	Address	STRING	256				
3	Company	STRING	256				
4	Department	STRING	256				
5	Salary	INT	256				

Rows

No	Name	Address	Company	Department	Salary	
1	John	Seoul, KOREA	1dollar	Sales	15000	
2	James	Barcelona, Spain	EBOT	Consulting	25000	
3	Donald	Califonia, USA	EBOT	Research Institute	1000000	
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000	
5	Joe	São Paulo, Brazil	hangul	R&D	8600	
6	Michael	Mumbai, India	1dollar	Personnel	1300	
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400	
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000	

Dataset

Source

3 Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset00을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

4 addRow 기능 구현

- addRow 버튼의 onclick 이벤트 함수

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Dataset00.addRow();
};
```


5 deleteRow 기능 구현

- deleteRow 버튼의 onclick 이벤트 함수

```
this.Button01_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Dataset00.deleteRow(this.Dataset00.rowposition);
};
```

6 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

addRow 버튼을 눌러 Grid의 가장 밑에 새로운 로우가 생성되는지 확인합니다. 버튼을 누른 만큼 로우가 생성됩니다.

deleteRow 버튼을 눌러 현재 선택된 로우가 삭제되는지 확인합니다.

22.6 여러 레코드 선택하기

Grid의 selecttype 속성을 사용하여 Grid에서 데이터를 선택하는 방법을 설정할 수 있습니다.

데이터를 선택하는 방법으로는 row, multirow, cell, area, multiarea가 있습니다. row는 한 줄을 선택할 때, multirow는 여러 줄을 선택할 때, 그리고 cell은 하나의 Cell을 선택할 때 사용합니다. 그리고 area는 영역으로 선택할 때, multiarea는 영역을 여러 선택할 때 사용합니다.

22.6.1 예제

아래는 Select type의 설정에 따라 Grid에서 데이터를 선택하는 동작을 보여주는 예제입니다.

Name	Address	Company	Department	Salary
John	Seoul, KOREA	1dollar	Sales	₩15,000
James	Barcelona, Spain	EBOT	Consulting	₩25,000
Donald	Califonia, USA	EBOT	Research Institute	₩1,000,000
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
Joe	São Paulo, Brazil	hangul	Quality Assurance	₩8,600
Michael	Mumbai, India	1dollar	Quality Assurance	₩1,300
Lilli	Ankara, Turkey	hangul	Quality Assurance	₩4,400
Colt	Sydney, NSW, Australia	hangul	R&D	₩62,000

Select type: ▼



sample_grid_06.xfdl

Select type 콤보에서 원하는 선택 타입을 설정한 후 Grid에서 로우 혹은 셀을 선택합니다. 다중 선택을 할 경우에는 마우스를 클릭 후 드래그하거나 키보드의 **Ctrl** 키와 **Shift** 키를 누른 상태에서 마우스를 클릭합니다.

Select type	설명
row	Grid에서 하나의 로우를 선택할 수 있습니다.
multirow	Grid에서 마우스 드래그 혹은 키보드를 사용하여 여러 개의 로우를 선택할 수 있습니다.
cell	Grid에서 하나의 셀을 선택할 수 있습니다.
area	Grid에서 마우스 드래그를 통한 영역 선택을 할 수 있습니다.
multiarea	Grid에서 마우스 드래그 혹은 키보드를 사용하여 영역을 여러 선택을 할 수 있습니다.

22.6.2 예제에서 사용한 핵심 기능

selecttype

Grid에서 사용자가 Row 혹은 Cell을 선택하는 방법을 설정하는 속성입니다.

posttext

새로 변경된 아이템의 text 값을 갖는 속성입니다.

22.6.3 예제 구현 방법

① 화면 구성하기

툴바에서 Grid , Static , Combo  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택한 후 Form 위에 놓고 그대로 클릭하면 기본 크기로 생성됩니다.

② Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, Form 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset00을 더블 클릭하면 Dataset 편집기가 뜨는데 컬럼과 로우를 다음 그림과 같이 입력합니다.

Dataset Editor

Const Columns

No	id	type	size	value
There are no items to show.				

Columns

No	id	type	size	prop	sumtext	description
1	Name	STRING	256			
2	Address	STRING	256			
3	Company	STRING	256			
4	Department	STRING	256			
5	Salary	INT	256			

Rows

No	Name	Address	Company	Department	Salary
1	John	Seoul, KOREA	1dollar	Sales	15000
2	James	Barcelona, Spain	EBOT	Consulting	25000
3	Donald	Califonia, USA	EBOT	Research Institute	1000000
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000
5	Joe	São Paulo, Brazil	hangul	R&D	8600
6	Michael	Mumbai, India	1dollar	Personnel	1300
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000

Dataset | Source |

3 Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset00을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

4 Select type 설정을 위한 Combo 메뉴 만들기

1. Combo 메뉴 만들기

Combo를 선택한 후 Properties 창에서 innerdataset 속성 옆의 [...] 버튼을 클릭합니다. InnerDataset 편집기에 다음과 같이 입력한 후 [OK] 버튼을 누르면 바인딩이 완료됩니다.

No	codecolumn	datacolumn
1	00	row
2	01	multirow

No	codecolumn	datacolumn
3	02	cell
4	03	area
5	04	multiarea

2. Combo에 이벤트 추가하기

Combo를 선택한 후 onitemchanged 이벤트를 함수를 아래와 같이 추가합니다.

```
this.combo_selecttype_onitemchanged = function(obj:nexacro.Combo,e:nexacro.ItemChangeEventInfo)
{
    var strSelectType = e.posttext;

    this.Grid00.set_selecttype(strSelectType);

};
```

5 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

- row를 선택한 후 Grid에서 하나의 로우가 선택되는지 확인합니다.
- multirow를 선택한 후 Grid에서 여러 개의 로우가 선택되는지 확인합니다. 마우스 드래그나 **Ctrl** 키, **Shift** 키를 사용하여 여러 로우가 선택되는지 확인합니다.
- cell을 선택한 후 Grid에서 하나의 셀이 선택되는지 확인합니다.
- area를 선택한 후 Grid에서 마우스 드래그를 통한 영역 선택이 되는지 확인합니다.
- multiarea를 선택한 후 Grid에서 마우스 드래그를 통한 영역 선택이 되는지 확인합니다. **Ctrl** 키, **Shift** 키를 사용하여 여러 영역이 추가로 선택되는지 확인합니다.

22.7 여러 레코드 삭제하기

Grid에서 getSelectedDatasetRows 메소드를 사용하면 현재 선택된 레코드의 정보를 얻을 수 있고 deleteMultiRows 메소드를 사용하면 여러 개의 레코드를 한 번에 삭제할 수 있습니다.

22.7.1 예제

아래는 사용자가 Grid의 레코드를 선택하고 Delete 버튼을 누르면 선택된 레코드가 모두 삭제되는 예제입니다.

Name	Address	Company	Department	Salary
John	Seoul, KOREA	1dollar	Sales	₩15,000
James	Barcelona, Spain	EBOT	Consulting	₩25,000
Donald	Califonia, USA	EBOT	Research Institute	₩1,000,000
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
Joe	São Paulo, Brazil	hangul	Quality Assurance	₩8,600
Michael	Mumbai, India	1dollar	Quality Assurance	₩1,300
Lilli	Ankara, Turkey	hangul	Quality Assurance	₩4,400
Colt	Sydney, NSW, Australia	hangul	R&D	₩62,000

Delete



22.7.2 예제에서 사용한 핵심 기능

selecttype

Grid에서 사용자가 Row 혹은 Cell을 선택하는 방법을 설정하는 속성입니다.

getSelectedDatasetRows

Grid에서 선택된 Row에 해당하는 Dataset의 Row 인덱스를 배열로 반환하는 메소드입니다.

deleteMultiRows

배열로 된 Row 목록을 입력받아 해당 Row를 삭제하는 메소드입니다.

22.7.3 예제 구현 방법

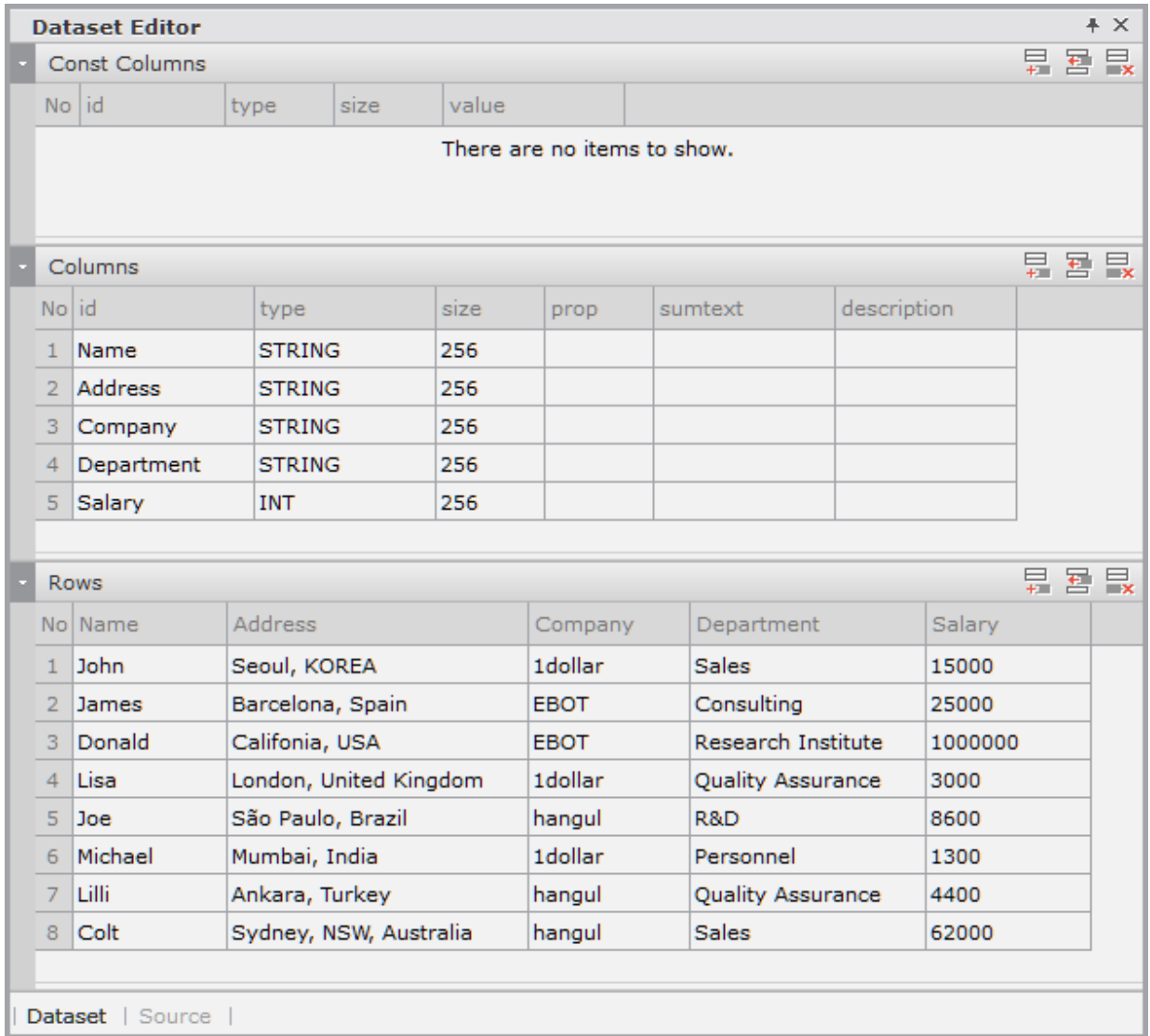
① 화면 구성하기

툴바에서 Grid , Button  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택한 후 Form 위에 놓고 그대로 클릭하면 기본 크기로 생성됩니다.

2 Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, Form 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset00을 더블 클릭하면 Dataset 편집기가 뜨는데 컬럼과 로우를 다음 그림과 같이 입력합니다.



Dataset Editor

Const Columns

No	id	type	size	value
There are no items to show.				

Columns

No	id	type	size	prop	sumtext	description
1	Name	STRING	256			
2	Address	STRING	256			
3	Company	STRING	256			
4	Department	STRING	256			
5	Salary	INT	256			

Rows

No	Name	Address	Company	Department	Salary
1	John	Seoul, KOREA	1dollar	Sales	15000
2	James	Barcelona, Spain	EBOT	Consulting	25000
3	Donald	Califonia, USA	EBOT	Research Institute	1000000
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000
5	Joe	São Paulo, Brazil	hangul	R&D	8600
6	Michael	Mumbai, India	1dollar	Personnel	1300
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000

Dataset | Source

3 Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset00을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

4 Grid 속성 설정하기

Grid를 다중 선택이 가능하도록 selecttype 속성을 'multirow'로 설정합니다.

5 Delete 기능 구현

Button을 선택한 후 onclick 이벤트 함수를 다음과 같이 추가합니다.

- Delete 버튼의 onclick 이벤트 함수

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var arrSelectedRow = this.Grid00.getSelectedDatasetRows();

    this.Dataset00.deleteMultiRows(arrSelectedRow);
};
```

6 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Grid에서 하나 혹은 여러 개의 로우를 선택한 후 Delete 버튼을 눌러 선택된 로우가 삭제되는지 확인합니다.

22.8 다른 데이터셋의 레코드 추가하기

Dataset 객체의 appendData 메소드를 사용하면 다른 Dataset의 데이터를 현재 사용 중인 Dataset의 끝에 추가할 수 있습니다. 인수의 설정에 따라 데이터가 추가되는 방식도 설정할 수 있습니다.

22.8.1 예제

아래는 ❶Dataset에 다른 ❷Dataset을 추가하는 예제입니다.

예제의 기능은 레코드가 추가되는 방식에 따라 Append dataset과 Append dataset with column name 두 가지로 나뉩니다. ❸Append dataset은 컬럼 이름에 상관없이 무조건 앞의 컬럼부터 순서대로 데이터를 추가합니다.

❹Append dataset with column name은 컬럼 이름이 같은 컬럼에만 데이터를 추가합니다.

- Append dataset 버튼을 클릭하면 ❶Dataset에 ❷Dataset의 모든 데이터가 추가됩니다. 데이터는 빨간 박스 ❺와 같이 가장 앞의 Name 컬럼부터 순서대로 추가됩니다. Salary 컬럼의 데이터가 ₩0으로 표시되는 이유는 추가된 데이터(Mail 컬럼)가 숫자가 아닌 문자열이기 때문입니다.

Name	Address	Company	Department	Salary
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
Joe	São Paulo, Brazil	hangul	Quality Assurance	₩8,600
Michael	Mumbai, India	1dollar	Quality Assurance	₩1,300
Lilli	Ankara, Turkey	hangul	Quality Assurance	₩4,400
Colt	Sydney, NSW, Australia	hangul	R&D	₩62,000
Muhammad	Syria	alzazeera		₩0
Gosling	Busan, KOREA	java		₩0
Jisung	Seoul, KOREA	apple		₩0

Append dataset Append dataset with column name

Name	Addr	Comp	Dept	Mail	Salary
Muhammad	Syria	alzazeera		muhammad@alzazeera.com	₩37,000
Gosling	Busan, KOREA	java		gosling@java.com	₩20,000
Jisung	Seoul, KOREA	apple		jisung@apple.com	₩840,000

- Append dataset with column name 버튼을 클릭하면 ② Dataset의 데이터 중에서 ① Dataset의 컬럼 이름과 일치되는 Name과 Salary 컬럼의 데이터만을 ⑤와 같이 추가합니다.

Name	Address	Company	Department	Salary
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
Joe	São Paulo, Brazil	hangul	Quality Assurance	₩8,600
Michael	Mumbai, India	1dollar	Quality Assurance	₩1,300
Lilli	Ankara, Turkey	hangul	Quality Assurance	₩4,400
Colt	Sydney, NSW, Australia	hangul	R&D	₩62,000
Muhammad				₩37,000
Gosling				₩20,000
Jisung				₩840,000

Append dataset Append dataset with column name

Name	Addr	Comp	Dept	Mail	Salary
Muhammad	Syria	alzazeera		muhammad@alzazeera.com	₩37,000
Gosling	Busan, KOREA	java		gosling@java.com	₩20,000
Jisung	Seoul, KOREA	apple		jisung@apple.com	₩840,000



22.8.2 예제에서 사용한 핵심 기능

appendData

현재 Dataset의 마지막 Row 다음에 지정한 Dataset의 데이터를 추가하는 메소드입니다. 새로 추가된 데이터의 Row 개수를 반환합니다.

22.8.3 예제 구현 방법

① 화면 구성하기

툴바에서 Grid , Button  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택한 후 Form 위에 놓고 그대로 클릭하면 기본 크기로 생성됩니다.

② Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, Form 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset00을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.

예제의 ① Grid에 바인딩할 Dataset

Dataset Editor

Const Columns

No

id

type

size

value

There are no items to show.

Columns

No

id

type

size

prop

sumtext

description

1

Name

STRING

256

2

Address

STRING

256

3

Company

STRING

256

4

Department

STRING

256

5

Salary

INT

256

Rows

No

Name

Address

Company

Department

Salary

1

John

Seoul, KOREA

1dollar

Sales

15000

2

James

Barcelona, Spain

EBOT

Consulting

25000

3

Donald

Califonia, USA

EBOT

Research Institute

1000000

4

Lisa

London, United Kingdom

1dollar

Quality Assurance

3000

5

Joe

São Paulo, Brazil

hangul

R&D

8600

6

Michael

Mumbai, India

1dollar

Personnel

1300

7

Lilli

Ankara, Turkey

hangul

Quality Assurance

4400

8

Colt

Sydney, NSW, Australia

hangul

Sales

62000

Dataset

Source

예제의 ②Grid에 바인딩할 Dataset

Dataset Editor

Const Columns

No	id	type	size	value	
There are no items to show.					

Columns

No	id	type	size	prop	sumtext	description	
1	Name	STRING	256				
2	Addr	STRING	256				
3	Comp	STRING	256				
4	Dept	STRING	256				
5	Mail	STRING	256				
6	Salary	INT	256				

Rows

No	Name	Addr	Comp	Dept	Mail	Salary	
1	Muhammad	Syria	alzazeera	[Undefined]	muhammad@alzazeera.com	37000	
2	Gosling	Busan, KOREA	java	[Undefined]	gosling@java.com	20000	
3	Jisung	Seoul, KOREA	apple	[Undefined]	jisung@apple.com	840000	

Dataset | Source |

③ Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 2개의 Dataset을 각각 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

④ Append dataset 기능 구현

- Append dataset 버튼의 onclick 이벤트 함수

```
this.btn_appendData_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var nRowCnt = this.Dataset00.appendData(this.Dataset01);
};
```

⑤ Append dataset with column name 기능 구현

- Append dataset with column name 버튼의 onclick 이벤트 함수

```
this.btn_appendDataCol_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var nRowCnt = this.Dataset00.appendData(this.Dataset01, true);
};
```

⑥ QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Append dataset 버튼을 눌러 ②Grid의 데이터가 ①Grid에 모두 추가되는지 확인합니다.

Append dataset with column name 버튼을 눌러 ②Grid의 데이터 중에 ①Grid의 컬럼 이름과 같은 데이터만 추가되는지 확인합니다. 예제에서 이름이 같은 컬럼은 Name과 Salary입니다. 그 외의 데이터는 추가되지 않습니다.

22.9 레코드 개수 구하기

Grid에 출력된 로우의 개수를 구할 때는 Dataset의 getRowCount 메소드를 사용합니다.

22.9.1 예제

[Get row count] 버튼을 누르면 현재 Grid의 총 로우 개수를 구해 Edit 컴포넌트에 표시합니다.

Name	Address	Company	Department	Salary
John	Seoul, KOREA	1dollar	Sales	₩15,000
James	Barcelona, Spain	EBOT	Consulting	₩25,000
Donald	Califonia, USA	EBOT	Research Institute	₩1,000,000
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
Joe	São Paulo, Brazil	hangul	Quality Assurance	₩8,600
Michael	Mumbai, India	1dollar	Quality Assurance	₩1,300
Lilli	Ankara, Turkey	hangul	Quality Assurance	₩4,400
Colt	Sydney, NSW, Australia	hangul	R&D	₩62,000

Get row count
8

 sample_grid_09.xfdl

22.9.2 예제에서 사용한 핵심 기능

getRowCount

Dataset에서 전체 Row의 개수를 구하는 메소드입니다.

22.9.3 예제 구현 방법

① 화면 구성하기

툴바에서 Grid , Button  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택한 후 Form 위에 놓고 그대로 클릭하면 기본 크기로 생성됩니다.

② Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, Form 위 적당한 공간에 클릭하여 Dataset을 생성합니다.

2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset00을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.

Dataset Editor

Const Columns

No	id	type	size	value
There are no items to show.				

Columns

No	id	type	size	prop	sumtext	description
1	Name	STRING	256			
2	Address	STRING	256			
3	Company	STRING	256			
4	Department	STRING	256			
5	Salary	INT	256			

Rows

No	Name	Address	Company	Department	Salary
1	John	Seoul, KOREA	1dollar	Sales	15000
2	James	Barcelona, Spain	EBOT	Consulting	25000
3	Donald	Califonia, USA	EBOT	Research Institute	1000000
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000
5	Joe	São Paulo, Brazil	hangul	R&D	8600
6	Michael	Mumbai, India	1dollar	Personnel	1300
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000

Dataset | Source |

3 Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset00을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

4 Get row count 기능 구현

- Get row count 버튼의 onclick 이벤트 함수

```
this.btn_getRowCount_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var intRowCnt = this.Dataset00.getRowCount();
```

```
this.Edit00.set_value(intRowCnt);
};
```

5 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Get row count 버튼을 눌러 구해진 Row 개수와 Grid에 보이는 전체 Row 개수를 비교하여 일치하는지 확인합니다.

22.10 변경된 데이터가 있는지 확인하기

트랜잭션은 반드시 필요하지만 시스템의 성능을 떨어뜨리는 요소이기도 합니다. 따라서 트랜잭션 횟수를 최소한으로 줄이는 것이 시스템 개발 시 중요합니다.

getRowType 메소드를 사용하면 데이터가 변경되었는지 확인할 수 있어 불필요한 트랜잭션을 수행하지 않도록 할 수 있습니다. 또한 현재 데이터가 추가, 삭제되었는지 등의 상태도 체크할 수 있어 개발시 유용하게 사용할 수 있습니다.

22.10.1 예제

아래는 Grid의 데이터가 변경됐는지 getRowType 메소드로 확인하는 예제입니다.

Grid의 Cell을 수정한 후 Get row type 버튼을 누르면 어떤 Row가 수정됐는지 확인할 수 있습니다. 단, 데이터를 수정했다가 다시 원래 값으로 되돌리면 수정되지 않은 것으로 간주합니다. Cell을 수정하려면 Cell을 먼저 선택한 후 다시 클릭합니다.

Name	Address	Company	Department	Salary
John	Seoul, KOREA	1dollar	Sales	₩1,115,000
James	Barcelona, Spain	EBOT	Consulting	₩2,233,300
Donald	Califonia, USA	EBOT	Research Institute	₩1,000,000
Lisa	London, United Kingdom	1dollar	Qualitv Assurance	₩33,000
Joe	São Paulo, Brazil			
Michael	Mumbai, India			
Lilli	Ankara, Turkey			
Colt	Sydney, NSW, Austr			

Get row type

localhost:8080 내용:

Row [0] is changed.
 Row [1] is changed.
 Row [2] is not changed.
 Row [3] is changed.
 Row [4] is not changed.
 Row [5] is not changed.
 Row [6] is changed.
 Row [7] is changed.

Grid is changed.

확인



22.10.2 예제에서 사용한 핵심 기능

getRowType

Dataset에서 지정된 Row의 상태를 반환하는 메소드입니다.

getRowCount

Dataset에서 전체 Row의 개수를 구하는 메소드입니다.

edittype

Grid의 Cell에서 편집창의 형식을 설정하는 속성입니다.

22.10.3 예제 구현 방법

① 화면 구성하기

툴바에서 Grid , Button  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택한 후 Form 위에 놓고 그대로 클릭하면 기본 크기로 생성됩니다.

② Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, Form 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset00을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.

Dataset Editor

Const Columns

No	id	type	size	value	
There are no items to show.					

Columns

No	id	type	size	prop	sumtext	description	
1	Name	STRING	256				
2	Address	STRING	256				
3	Company	STRING	256				
4	Department	STRING	256				
5	Salary	INT	256				

Rows

No	Name	Address	Company	Department	Salary	
1	John	Seoul, KOREA	1dollar	Sales	15000	
2	James	Barcelona, Spain	EBOT	Consulting	25000	
3	Donald	Califonia, USA	EBOT	Research Institute	1000000	
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000	
5	Joe	São Paulo, Brazil	hangul	R&D	8600	
6	Michael	Mumbai, India	1dollar	Personnel	1300	
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400	
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000	

Dataset

Source

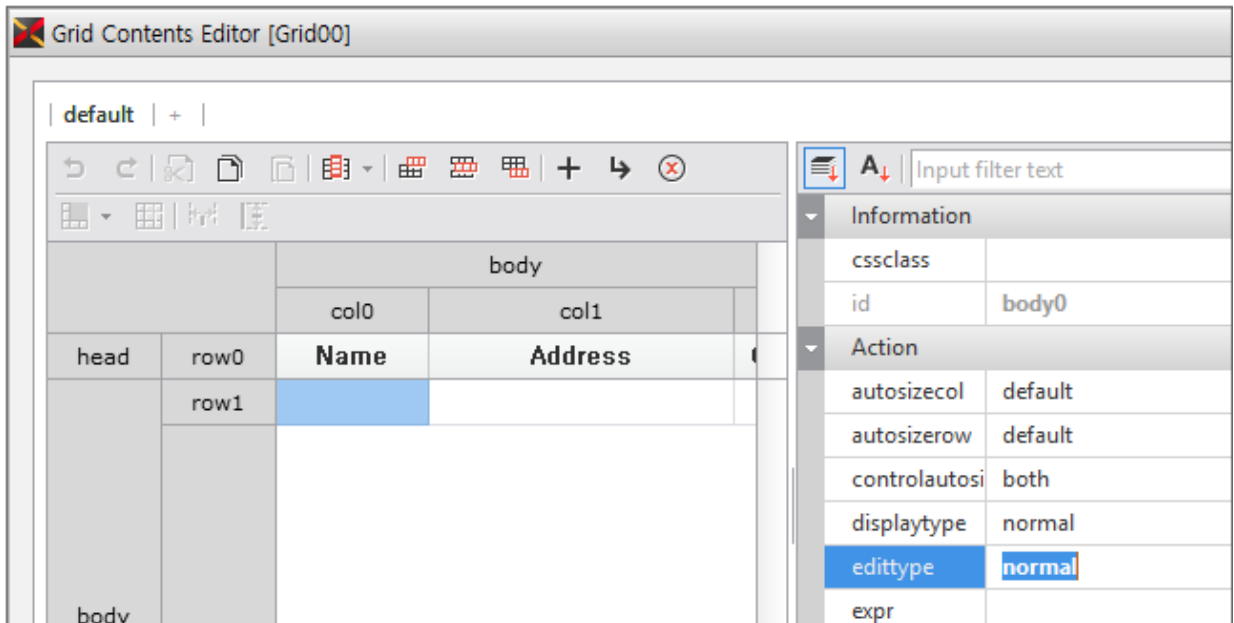
3 Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset00을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

4 Grid를 편집할 수 있도록 설정하기

Grid의 selecttype 속성은 "cell"로 설정하고, Grid 컴포넌트를 더블클릭해 Grid Contents Editor를 실행하고 각 Cell의 edittype 속성을 "normal"로 설정합니다.



5 Get row type 기능 구현

Button을 선택한 후 onclick 이벤트 함수를 아래와 같이 추가합니다.

- Get row type 버튼의 onclick 이벤트 함수

```
this.btn_getRowType_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var bChangedData = false;
    var nRowCnt = this.Dataset00.getRowCount();

    var strResult = "";

    for(var i=0; i<nRowCnt; i++)
    {

        switch( this.Dataset00.getRowType(i) )
        {
            case 0:
                strResult += "Row [" + i + "] is not exist.\n";
                break;
            case 1:
            case 16:
                strResult += "Row [" + i + "] is not changed.\n";
                break;
            case 2:
```

```

        case 4:
        case 8:
            strResult += "Row [" + i + "] is changed.\n";
            bChangedData = true;
            break;
        default:
            strResult += "Row [" + i + "] Unknown status\n";
    }

}

if(bChangedData == true)
    strResult += "\nGrid is changed.";
else
    strResult += "\nGrid is not changed.";

alert(strResult);

};

```

⑥ QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Get row type 버튼을 눌러 Grid에서 변경된 데이터가 있는지 팝업창에서 확인합니다.

22.11 변경 내용 데이터셋에 반영하기 / 되돌리기

Grid에서 데이터를 수정할 때에는 수정 내용을 실제로 Dataset에 반영하기 위한 과정이 필요합니다. 이는 불필요한 트랜잭션의 발생을 줄여주며 데이터 수정이 잘못되었을 경우에 복구할 수 있는 안전장치 역할을 합니다.

Dataset의 applyChange 메소드는 그리드에서 변경된 내용을 데이터셋에 반영합니다. applyChange 메소드는 데이터 수정으로 인해 변경된 Grid의 상태를 ROWTYPE_NORMAL 상태로 되돌려 수정된 내용을 확정합니다.

Dataset의 reset 메소드는 Grid의 데이터를 마지막 저장 상태로 되돌립니다. 다시 말해 데이터를 수정하기 전 상태나 applyChange를 마지막으로 수행했던 상태로 되돌립니다.

22.11.1 예제

아래는 Grid의 데이터를 수정한 후 Dataset에 반영하는 applyChange 기능과 Dataset에 반영하기 전의 상태로 데이터를 복구하는 Reset 기능을 보여주는 예제입니다.

Name	Address	Company	Department	Salary
John	Seoul, KOREA	1dollar	Sales	₩1,115,000
James	Barcelona, Spain	EBOT	Consulting	₩25,000
Donald	Califonia, USA	EBOT	Research Institute	₩1,000,000
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩23,000
Joe	São Paulo, Brazil	hangul	Quality Assurance	₩8,600
Michael	Mumbai, India	2dollar	Quality Assurance	₩1,300
Lilli	Ankara, Turkey	hangul	Quality Assurance	₩54,400
Colt	Sydney, NSW, Australia	hangul	R&D	₩62,000

applyChange

Reset



Grid의 데이터를 수정한 후 Reset 버튼을 클릭하면 수정하기 전 상태로 복구됩니다.

Grid의 데이터를 수정한 후 applyChange 버튼을 클릭하면 수정 내용이 Dataset에 반영됩니다. 한번 Dataset에 반영되면 다시 Reset 버튼을 눌러도 이전 상태로 복구할 수 없습니다.

22.11.2 예제에서 사용한 핵심 기능

applyChange

Dataset의 모든 Row 타입을 ROWTYPE_NORMAL로 변경하여 수정된 데이터를 Dataset에 최종 반영합니다.

Reset

Dataset의 데이터를 마지막 저장 상태로 복구하는 메소드입니다.

22.11.3 예제 구현 방법

1 화면 구성하기

툴바에서 Grid , Button  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택한 후 Form 위에 놓고 그대로 클릭하면 기본 크기로 생성됩니다.

2 Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, Form 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset00을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.

Dataset Editor

Const Columns

No	id	type	size	value
There are no items to show.				

Columns

No	id	type	size	prop	sumtext	description
1	Name	STRING	256			
2	Address	STRING	256			
3	Company	STRING	256			
4	Department	STRING	256			
5	Salary	INT	256			

Rows

No	Name	Address	Company	Department	Salary
1	John	Seoul, KOREA	1dollar	Sales	15000
2	James	Barcelona, Spain	EBOT	Consulting	25000
3	Donald	Califonia, USA	EBOT	Research Institute	1000000
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000
5	Joe	São Paulo, Brazil	hangul	R&D	8600
6	Michael	Mumbai, India	1dollar	Personnel	1300
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000

Dataset | Source |

③ Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset00을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

④ applyChange 기능 구현

- applyChange 버튼의 onclick 이벤트 함수

```
this.btn_apply_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Dataset00.applyChange();
};
```

⑤ Reset 기능 구현

- Reset 버튼의 onclick 이벤트 함수

```
this.btn_reset_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Dataset00.reset();
};
```

⑥ QuickView로 확인하기

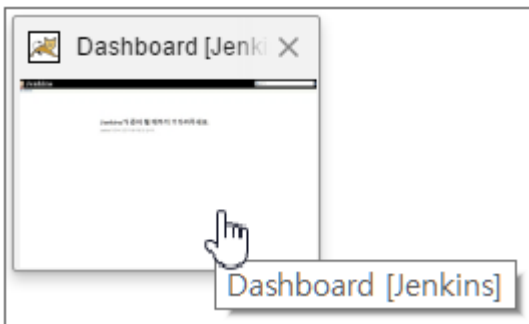
QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Grid의 데이터를 수정한 후 Reset 버튼을 눌러 데이터가 초기 상태로 복구되는지 확인합니다.

Grid의 데이터를 수정한 후 applyChange 버튼을 눌러 Dataset에 반영하고, 다시 데이터를 수정한 후 Reset 버튼을 눌러 applyChange 버튼을 눌렀던 상태로 복구되는지 확인합니다.

22.12 텍스트의 길이가 긴 경우 툴팁(tooltip)으로 보여주기

텍스트 데이터의 크기가 Grid의 각 Cell 넓이보다 클 때 모든 내용을 표시하지 못하는 문제가 발생합니다. 이는 화면상의 제약 때문에 Grid와 Cell의 크기를 충분히 크게 확보하지 못했거나 혹은 예상치 못한 텍스트 데이터의 크기 때문일 수 있습니다. 이런 경우에 잘려 보이는 텍스트 내용을 툴팁을 사용해 모두 표시해 줄 수 있습니다.



툴팁은 말 풍선 혹은 풍선 도움말이라고도 하며 마우스 커서를 UI 요소 위에 올리면 해당 요소에 대한 도움말을 띄워주는 기능을 합니다. Grid를 비롯한 거의 모든 컴포넌트들은 툴팁 기능을 지원합니다.

22.12.1 예제

다음은 텍스트 데이터의 크기가 커서 Cell에서 잘려 보이는 부분을 툴팁을 이용해 표시해주는 Grid 예제입니다.

Name	Address	Compar	Department	Salary
John	Seoul, KOREA	1dollar	Sales	₩15,000
James	Barcelona, Spain	EBOT	Consulting	₩25,000
Donald	Califonia, USA	EBOT	Research	₩1,000,0
Lisa	London, United	1dollar	Quality	₩3,000
Joe	São Paulo, Brazil	hangul	R&D	₩8,600
Michael	Mumbai, India	1dollar	Personnel	₩1,300
Lilli	Ankara, Turkey	hangul	Quality	₩4,400
Colt	Sydney, NSW, A	hangul	Sales	₩62,000

Sydney, NSW, Australia



sample_grid_36.xfdl

텍스트 크기가 셀 영역을 넘어가는 Address, Department, Salary 컬럼에만 툴팁 기능을 적용하였습니다. 마우스 커서를 컬럼 위에 올리면 Cell에 표시된 텍스트 내용이 툴팁에도 표시되는 것을 확인할 수 있습니다.

22.12.2 예제에서 사용한 핵심 기능

tooltiptext

Grid의 Cell 영역에 마우스 커서를 올려 놓았을 때 표시되는 풍선 도움말을 설정하는 속성입니다.

22.12.3 예제 구현 방법

1 화면 구성하기

툴바에서 Grid  컴포넌트를 예제의 그림과 같이 적절히 배치합니다.

2 Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, Form 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.

Dataset Editor

Const Columns

No

id

type

size

value

There are no items to show.

Columns

No

id

type

size

prop

sumtext

description

1

Name

STRING

256

2

Address

STRING

256

3

Company

STRING

256

4

Department

STRING

256

5

Salary

STRING

256

Rows

No

Name

Address

Company

Department

Salary

1

John

Seoul, KOREA

1dollar

Sales

15000

2

James

Barcelona, Spain

EBOT

Consulting

25000

3

Donald

Califonia, USA

EBOT

Research Institute

1000000

4

Lisa

London, United Kingdom

1dollar

Quality Assurance

3000

5

Joe

São Paulo, Brazil

hangul

R&D

8600

6

Michael

Mumbai, India

1dollar

Personnel

1300

7

Lilli

Ankara, Turkey

hangul

Quality Assurance

4400

8

Colt

Sydney, NSW, Australia

hangul

Sales

62000

Dataset

|

Source

|

Dataset | Source |

3 Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

4 툴팁 설정하기

Grid Contents Editor를 열어 Address, Salary 컬럼의 Cell을 선택한 후 tooltiptext 속성을 설정합니다. 각 컬럼에 맞게 Dataset을 바인딩합니다. text 속성에 설정된 값과 동일하게 설정하여 툴팁에서도 Cell의 내용이 표시될 수 있도록 합니다.

다음은 Address 컬럼의 Cell에서 tooltiptext 속성을 설정하는 예를 보여줍니다. 마찬가지로 Salary 컬럼도 동일하게 설정합니다.

text	bind:Address
tooltiptext	bind:Address

5 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Address, Salary 컬럼에 마우스 커서를 위치시킨 후 해당 Cell의 텍스트 데이터가 툴팁에 표시되는지 확인합니다.

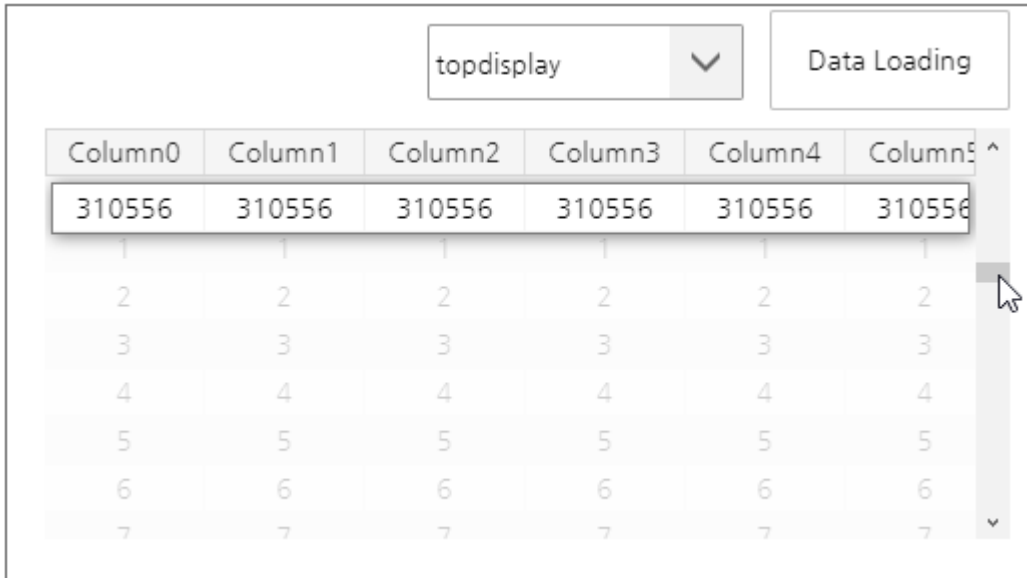
22.13 Smart Scroll 기능 사용하기

Grid 컴포넌트는 화면에 표시할 데이터 Row가 많아지면 스크롤바가 표시됩니다. 이런 경우 스크롤바를 움직여서 데이터를 탐색하게 되는데 스크롤바를 움직이면서 화면에 표시되는 데이터를 계속 표시해 주어야 합니다. 브라우저 입장에서 스크롤바를 움직이는 동안 데이터를 표시해 주는 작업이 부담이 될 수 있고 사용자 입장에서는 스크롤바를 움직이면서 원하는 데이터를 찾기가 쉽지 않습니다.

Smart Scroll 기능은 이런 불편을 어느 정도 해소해 주는 팁 같은 기능입니다. Grid 컴포넌트에 fastscrolltype 속성 값을 설정하면 Smart Scroll 기능을 적용할 수 있습니다.

22.13.1 예제

[Data Loading] 버튼 클릭 시 For 반복문으로 생성한 1,000,000개 데이터를 Dataset 오브젝트에 추가합니다. Combo 컴포넌트에 표시되는 fastscrolltype 속성값을 변경하면 스크롤바 트랙바를 이동했을때 Smart Scroll 기능이 적용된 것을 확인할 수 있습니다.

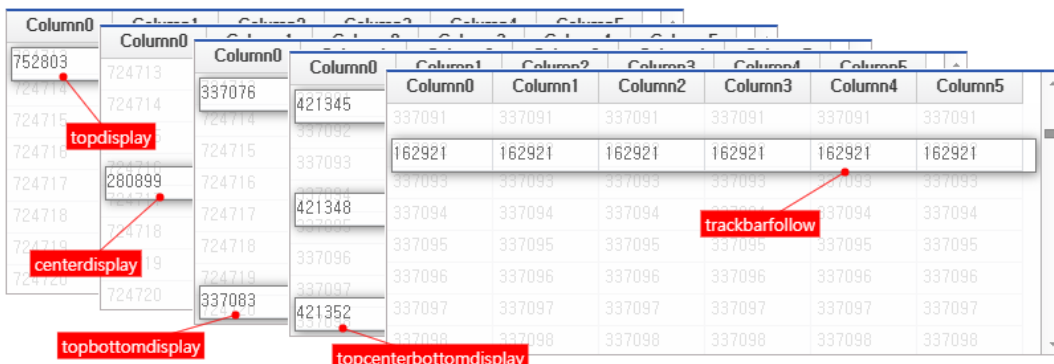


 sample_grid_39.xfdl

22.13.2 예제에서 사용한 핵심 기능

fastscrolltype (17.0.0.200 버전부터 지원)

Smart Scroll 기능 구현 시 어느 위치의 데이터를 보여줄 시 지정합니다. Grid 컴포넌트를 기준으로 첫 번째 Row, 가운데 Row, 표시되는 화면의 마지막 Row 중에서 선택할 수 있으며 두 개 이상의 데이터를 보여주는 "topbottomdisplay", "topcenterbottomdisplay" 속성값을 지원합니다. 스크롤바 트랙바 위치를 따라 데이터를 보여주는 "trackbarfollow" 속성도 사용할 수 있습니다.





fastscrolltype 속성은 트랙바를 누른 상태에서 스크롤바를 움직이는 경우에만 적용됩니다. 키보드 방향키 또는 페이지 업/다운 키를 사용하거나 스크롤바 decbutton, incbutton을 눌러 스크롤바를 움직이는 경우에는 적용되지 않습니다.

22.13.3 예제 구현 방법

① 화면 구성하기

Grid, Combo, Button 컴포넌트를 예제 화면과 같이 배치합니다. Combo 컴포넌트의 innerdataset 속성값은 아래와 같이 지정합니다. fastscrolltype 속성에서 지원하는 속성값입니다.

InnerDataset		
No	codecolumn	datacolumn
1	default	default
2	topdisplay	topdisplay
3	centerdisplay	centerdisplay
4	topbottomdisplay	topbottomdisplay
5	topcenterbottomdisplay	topcenterbottomdisplay
6	trackbarfollow	trackbarfollow

② Dataset 오브젝트 추가하기

Dataset 오브젝트를 추가합니다. Columns 속성값만 6개의 컬럼을 추가하고 Rows 속성값은 추가하지 않습니다. 다음 단계에서 For 반복문으로 채워줄 겁니다. Dataset 오브젝트를 선택하고 마우스 드래그앤드롭으로 Grid 컴포넌트에 바인딩해줍니다.

Dataset Editor [dsTest]					
Const Columns					
Columns					
No	id	type	size	prop	sumtext
1	Column0	STRING	256		
2	Column1	STRING	256		
3	Column2	STRING	256		
4	Column3	STRING	256		
5	Column4	STRING	256		
6	Column5	STRING	256		

3 Button 컴포넌트 이벤트 함수 작성하기

Button 컴포넌트 클릭 시 Dataset에 데이터를 로딩해주는 이벤트를 작성합니다. For 반복문에서 Dataset 오브젝트의 setColumn 메소드를 호출하는 방식입니다. For 반복문을 시작하기 전에 enableevent 속성값을 false로 지정하는데 그렇게 하지 않으면 setColumn 메소드를 호출할 때마다 Grid 컴포넌트에 변경된 데이터에 대한 이벤트가 전달되어 For 반복문 처리속도가 늦어집니다.

For 반복문이 완료되면 enableevent 속성값을 다시 true로 수정합니다. 속성값 변경 시 Dataset 오브젝트의 onrowsetchanged 이벤트가 발생하는데 이때 바인딩된 Grid 컴포넌트에 데이터를 반영하게 됩니다.

```
this.btnData_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var rIdx;
    this.dsTest.set_enableevent(false);
    for(var i=0;i<1000000;i++) {
        rIdx = this.dsTest.addRow();
        this.dsTest.setColumn(rIdx, "Column0", i);
        this.dsTest.setColumn(rIdx, "Column1", i);
        this.dsTest.setColumn(rIdx, "Column2", i);
        this.dsTest.setColumn(rIdx, "Column3", i);
        this.dsTest.setColumn(rIdx, "Column4", i);
        this.dsTest.setColumn(rIdx, "Column5", i);
    }
    this.dsTest.set_enableevent(true);
};
```



For 반복문 내에서 처리하는 데이터 수가 적은 경우에는 enableevent 속성값을 변경하는 것이 큰 영향을 미치지 않습니다. 하지만 데이터 수가 커지는 경우에는 문제가 될 수 있습니다. 시스템에 따라 다르겠지만 1,000건 이상의 데이터를 처리하는 경우에는 애플리케이션 동작이 멈출 수도 있습니다.

4 Combo 컴포넌트 이벤트 함수 작성하기

Combo 컴포넌트의 아이템 선택 시 Grid 컴포넌트의 fastvscrolltype 속성값을 변경합니다. 속성값 변경 시 바로 Grid 컴포넌트의 Smart Scroll 기능이 동작합니다.

```
this.cbType_onitemchanged = function(obj:nexacro.Combo,e:nexacro.ItemChangeEventInfo)
{
    this.gridData.set_fastvscrolltype(e.postvalue);
};
```

5 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 데이터를 로딩하고 Combo 컴포넌트에서 fastscrolltype 속성값을 바꾸어가며 확인합니다.

22.14 속성 지정해서 선택한 셀 테두리선 표현하기

showselection 속성값을 true로 지정하면 선택한 셀 테두리선을 표현할 수 있습니다.

22.14.1 예제

선택한 셀 영역의 테두리선을 표시합니다. 셀 선택이 해제되면 테두리선도 사라집니다. [selectArea] 버튼을 클릭하면 0번째 셀 선택을 추가합니다. [clearSelect] 버튼을 클릭하면 셀 선택 상태를 해제합니다.

아래 예제 이미지는 다음과 같은 방식으로 선택했습니다.

1. Salary 컬럼 첫 번째 항목(15,000) 클릭
2. Ctrl 키를 누른 상태에서 Name 컬럼의 세 번째 항목(Donald) 클릭
3. Shift 키를 누른 상태에서 Company 컬럼의 네 번째 항목(1dollar) 클릭
4. Ctrl 키를 누른 상태에서 Name 컬럼의 여섯 번째 항목(Michael) 클릭
5. Shift 키를 누른 상태에서 Department 컬럼의 일곱 번째 항목(Quality Assurance) 클릭
6. [selectArea] 버튼 클릭

Name	Address	Company	Department	Salary
John	Seoul, KOREA	1 dollar	Sales	15,000
James	Barcelona, Spain	EBOT	Consulting	25,000
Donald	Califonia, USA	EBOT	Research Institute	1,000,000
Lisa	London, United Kin	1 dollar	Quality Assurance	3,000
Joe	São Paulo, Brazil	hangul	Quality Assurance	8,600
Michael	Mumbai, India	1 dollar	Quality Assurance	1,300
Lilli	Ankara, Turkey	hangul	Quality Assurance	4,400
Colt	Sydney, NSW, Aust	hangul	R&D	62,000

selectArea

clearSelect



22.14.2 예제에서 사용한 핵심 기능

showselection (17.0.0.1600 버전부터 지원)

속성값을 true로 지정하면 선택한 셀 영역의 테두리선을 보여줍니다. 테두리선을 표현하기 위한 스타일 속성 값이 같이 지정되어 있어야 합니다.

clearSelect

셀 선택 상태를 초기화합니다. 선택한 셀 테두리선도 같이 사라집니다.

22.14.3 예제 구현 방법

- 1 Grid 컴포넌트를 화면에 배치합니다.
- 2 Dataset 오브젝트를 추가하고 아래와 같이 데이터를 설정합니다. 선택된 영역의 테두리선을 표현하는 예제이기 때문에 데이터 자체는 큰 의미가 없습니다.

Dataset Editor

Const Columns

No	id	type	size	value	
There are no items to show.					

Columns

No	id	type	size	prop	sumtext	description	
1	Name	STRING	256				
2	Address	STRING	256				
3	Company	STRING	256				
4	Department	STRING	256				
5	Salary	STRING	256				

Rows

No	Name	Address	Company	Department	Salary	
1	John	Seoul, KOREA	1dollar	Sales	15000	
2	James	Barcelona, Spain	EBOT	Consulting	25000	
3	Donald	Califonia, USA	EBOT	Research Institute	1000000	
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000	
5	Joe	São Paulo, Brazil	hangul	R&D	8600	
6	Michael	Mumbai, India	1dollar	Personnel	1300	
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400	
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000	

Dataset | Source |

- ③ Dataset을 Grid 컴포넌트에 바인딩합니다. Invisible Objects 영역에 있는 Dataset을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.
- ④ Grid 컴포넌트의 showselection 속성값을 true로 변경하고 selecttype 속성값을 "multiarea"로 설정합니다.
- ⑤ xcss 파일을 열어서 아래 코드를 추가합니다. Advanced 탭에서 선택자를 추가하거나 Text 탭에서 직접 코드 값을 입력할 수 있습니다.

```
.Grid.sample_grid_83 .body .selection
{
    -nexa-border : 2px dotted hotpink;
}
```

기본 테마를 수정해서 사용하는 경우에는 아래와 같은 형식으로 코드를 추가합니다.

```
.Grid .body .selection
{
    -nexa-border : 2px solid #125dae;
}
```

- ⑥ Grid 컴포넌트의 cssclass 속성값을 "sample_grid_83"으로 설정합니다.
- ⑦ Grid 컴포넌트 아래에 Button 컴포넌트 2개를 배치합니다.
- ⑧ Button 컴포넌트의 onclick 이벤트 함수를 아래와 같이 추가합니다. selectArea 메소드는 지정한 인덱스의 셀을 선택합니다. 예제에서는 첫 번째 셀을 선택합니다. clearSelect 메소드는 선택한 상태를 해제합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Grid00.selectArea(0,0,0,0);
};

this.Button01 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Grid00.clearSelect();
};
```

- ⑨ QuickView(Ctrl + F6)로 실행한 후 셀 영역을 선택하고 경계선이 표시되는지 확인합니다.

22.15 스크롤바가 생기지 않게 Grid 컴포넌트의 높이 조정하기

미리 정해진 데이터라면 스크롤바가 생기지 않게 Grid 컴포넌트의 높이를 설정해서 배포하면 되지만, 데이터값이 실시간으로 들어와서 변경되는 경우에는 Row 건수가 늘어나거나 하나의 셀 안에 많은 데이터가 들어가면서 지정된 Grid 컴포넌트의 높이보다 데이터가 많아지면서 스크롤바가 생길 수 있습니다.

`getRealRowFullSize` 메소드는 Grid 컴포넌트에 표시되는 데이터 영역의 실제 높이값을 구해서 Grid 컴포넌트의 높이값을 적절하게 조정할 수 있습니다.

22.15.1 예제

[`getRealRowFullSize`] 버튼을 클릭하면 5개의 Row에 맞게 Grid 컴포넌트의 높이가 변경됩니다. TextArea 컴포넌트에 2줄 이상의 텍스트를 입력하고 특정 셀을 선택한 후 [Set Text] 버튼을 클릭하면 해당 데이터가 선택한 셀에 반영되면서 Grid 컴포넌트에 스크롤바가 생길 수 있습니다. 이때 다시 [`getRealRowFullSize`] 버튼을 클릭하면 Grid 컴포넌트의 높이가 변경됩니다.

Column0	Column1	Column2
1	one	I
2	two	II
3	TEST TEST TEST	III
4	four	IV

getRealRowFullSize

Set Text

TEST
TEST
TEST



sample_grid_85.xfdl

22.15.2 예제에서 사용한 핵심 기능

getRealRowFullSize

head, body, summary 영역의 Row 높이의 합을 반환합니다. 파라미터를 지정하면 각 영역의 높이를 반환하며 지정하지 않으면 전체 영역의 높이를 반환합니다.

autosizingtype, extendsizetype

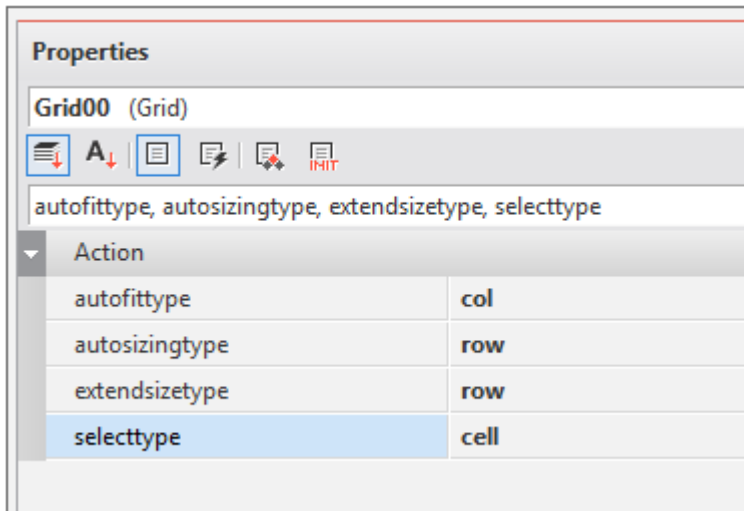
속성값으로 "col", "row", "both" 값을 설정하면 데이터에 맞게 크기를 자동 조절합니다. 이때 전체 Row의 높이가 같이 조절되기 때문에 변경된 Row 높이만 조절하고 싶다면 extendsizetype 속성값을 지정해주어야 합니다.

22.15.3 예제 구현 방법

- ① Grid 컴포넌트와 Button 컴포넌트, TextArea 컴포넌트를 화면에 배치합니다.
- ② Dataset 오브젝트를 추가하고 3개의 컬럼을 생성한 후 데이터를 추가합니다.

Columns					
No	id	type	size	prop	sum
1	Column0	STRING	256		
2	Column1	STRING	256		
3	Column2	STRING	256		
Rows					
No	Column0	Column1	Column2		
1	1	one	I		
2	2	two	II		
3	3	three	III		
4	4	four	IV		
5	5	five	V		

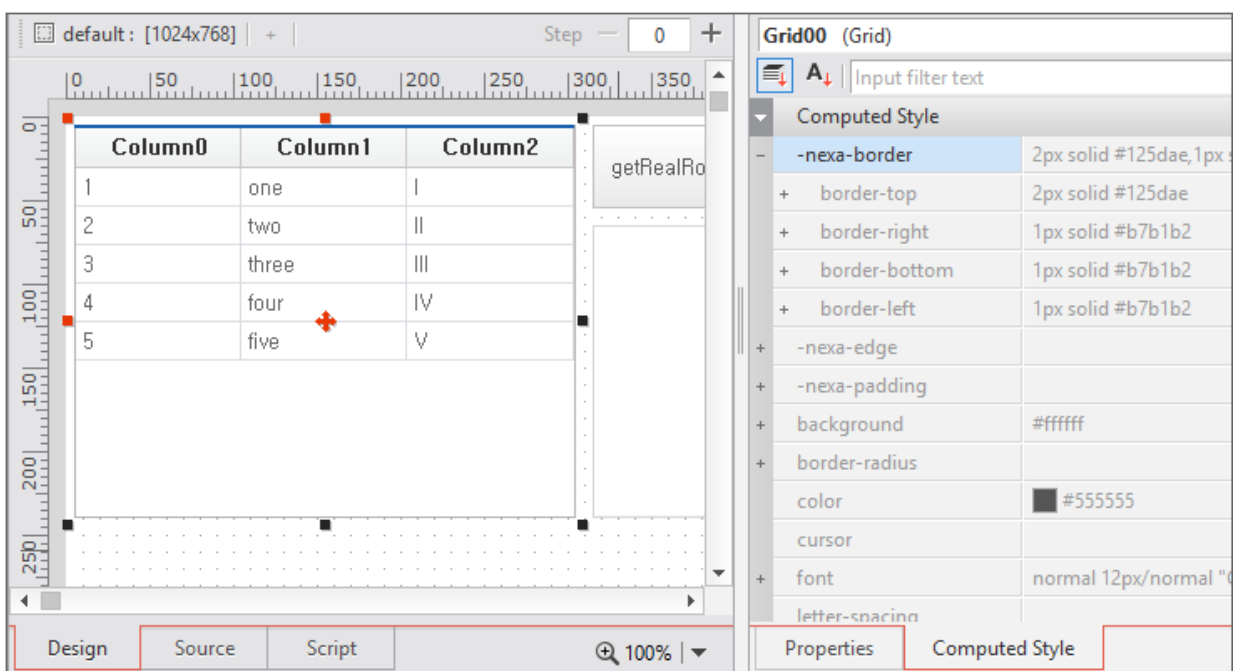
- ③ Dataset 오브젝트를 선택하고 Grid 컴포넌트로 드래그앤드롭해서 binddataset 속성으로 등록합니다.
- ④ Grid 컴포넌트의 autofitttype, autosizingtype, extendsizetype, selecttype 속성값을 아래와 같이 설정합니다. selecttype 속성은 [Set Text] 버튼 클릭 시 선택한 셀 하나에만 데이터를 업데이트하기 위해서 속성값을 "cell"로 지정합니다.



- 5 [getRealRowFullSize] 버튼의 onclick 이벤트 함수를 아래와 같이 작성합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var nH = this.Grid00.getRealRowFullSize();
    //computed style (border-top + border-bottom = 3)
    this.Grid00.set_height(nH + 3);
};
```

Grid 컴포넌트의 height 속성값 지정 시 더해주는 3은 border 속성값입니다. border 속성값은 애플리케이션 실행 중에는 확인할 수 없으며 Grid 컴포넌트를 선택하고 속성창에서 [Computed Style] 탭을 선택하면 실제 적용될 border 속성값을 알 수 있습니다. border 속성값은 적용되는 테마에 따라 달라집니다.



- ⑥ [Set Text] 버튼의 onclick 이벤트 함수를 아래와 같이 작성합니다.

selecttype 속성값을 "cell"로 지정했기 때문에 하나의 셀만 선택할 수 있습니다. selectstartrow 속성값은 하나의 셀만 선택하더라도 배열로 값을 반환합니다.

```
this.Button01_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Dataset00.setColumn(this.Grid00.selectstartrow[0], this.Grid00.selectstartcol[0],
    this.TextArea00.value);
};
```

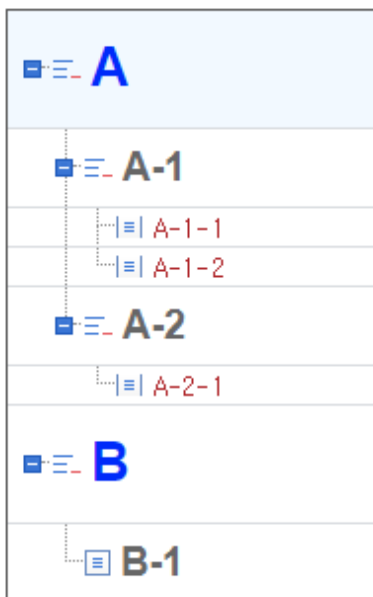
- ⑦ QuickView(Ctrl + F6)로 실행한 후 데이터를 변경하고 [getRealRowFullSize] 버튼을 클릭해 Grid 컴포넌트의 높이가 적절하게 조절되는지 확인합니다.

22.16 Grid 컴포넌트의 Row 높이를 직접 지정하기

특정 Row 또는 조건에 맞는 Row의 높이를 직접 지정하는 방법을 살펴봅니다.

22.16.1 예제

트리 형태로 표현되는 Grid 컴포넌트의 레벨에 따라 Row 높이와 글꼴 스타일을 다르게 적용합니다.





22.16.2 예제에서 사용한 핵심 기능

setRealRowSize

특정 Row의 높이값을 지정할 수 있습니다. 예제에서는 트리 레벨에 따라 다른 값으로 Row를 지정합니다.

extendsizetype

Row 크기를 각각 조정하기 때문에 해당 속성값을 "row"로 지정해주어야 합니다. 그렇지 않으면 원하는 모양과 다른 형태로 표시될 수 있습니다.

22.16.3 예제 구현 방법

- ① Grid 컴포넌트를 화면에 배치합니다.
- ② Dataset 오브젝트를 추가하고 2개의 컬럼을 생성한 후 데이터를 추가합니다.

Columns				
No	id	type	size	prop
1	Column0	STRING	256	
2	Column1	STRING	256	
Rows				
No	Column0	Column1		
1	0	A		
2	1	A-1		
3	2	A-1-1		
4	2	A-1-2		
5	1	A-2		
6	2	A-2-1		
7	0	B		
8	1	B-1		

- ③ Dataset 오브젝트를 선택하고 Grid 컴포넌트로 드래그앤드롭해서 binddataset 속성으로 등록합니다.
- ④ Grid 컴포넌트의 더블클릭해서 Grid Contents Editor를 실행하고 아래 속성값을 지정합니다.

```
displaytype: treeitemcontrol
editttype: tree
treelevel: bind:Column0
```

- ⑤ Grid 컴포넌트에서 아래 속성값을 수정합니다.

```
extendsizetype: row
treeinitstatus: expand,all
treeusecheckbox: false
treeuseline: true
```

- ⑥ Dataset 오브젝트의 onload 이벤트 함수를 아래와 같이 작성합니다. 어느 시점에 Grid 컴포넌트의 Row 높이를 조정할 것인지에 따라 다른 이벤트에서 처리할 수도 있습니다.

```
this.Dataset00_onload = function(obj:nexacro.NormalDataset,e:nexacro.DSLoadEventInfo)
{
    var tempColValue;

    for(var i=0; i < this.Grid00.rowcount; i++)
    {
        tempColValue = this.Dataset00.getColumn(this.Grid00.getDatasetRow(i), "Column0");

        if(tempColValue==0) {
            this.Grid00.setRealRowSize(i,"60");
        }
        else if(tempColValue==1) {
            this.Grid00.setRealRowSize(i,"40");
        }
        else if(tempColValue==2) {
            this.Grid00.setRealRowSize(i,"20");
        }
    }
};
```

- ⑦ xcss 파일을 열어서 아래 코드를 추가합니다. Advanced 탭에서 선택자를 추가하거나 Text 탭에서 직접 코드 값을 입력할 수 있습니다.

```
.Grid .body .row .cell.sample_grid_94_level_0 .celltreeitem .treeitemtext
{
    font : normal bold 20pt/normal "Arial";
```

```

    color : blue;
}

.Grid .body .row .cell.sample_grid_94_level_1 .celltreeitem .treeitemtext
{
    font : normal bold 15pt/normal "Arial";
}

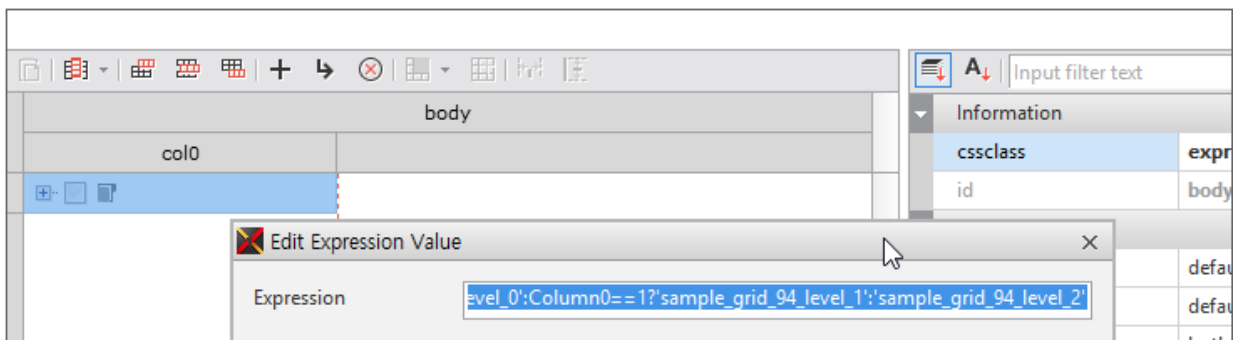
.Grid .body .row .cell.sample_grid_94_level_2 .celltreeitem .treeitemtext
{
    color : firebrick;
}

```

- ⑧ Grid Contents Editor에서 cssclass 속성값을 아래와 같이 지정합니다.

Column0 컬럼의 값에 따라 다른 스타일 속성을 적용합니다.

```
Column0==0?'sample_grid_94_level_0':Column0==1?'sample_grid_94_level_1':'sample_grid_94_level_2'
```



- ⑨ QuickView(Ctrl + F6)로 실행한 후 트리 형태의 Grid 컴포넌트에서 트리 레벨에 따라 Row 높이와 스타일이 다르게 적용되는지 확인합니다.

22.17 컬럼 너비보다 큰 텍스트만 툴팁으로 보여주기

텍스트의 길이가 긴 경우 툴팁(tooltip)으로 보여주기에서는 특정 컬럼은 모두 툴팁으로 보여주었는데, 실제 텍스트 길이에 따라 필요한 경우만 툴팁으로 보여주는 방법을 살펴봅니다.

22.17.1 예제

컬럼의 실제 너비보다 텍스트 길이가 큰 경우에만 툴팁을 표시합니다. 컬럼 너비와 텍스트 너비 정보를 TextArea 컴포넌트에 표시합니다.

Column0	Column1	Column2
1	test test test test	abc
2	test1	nexacro
3	test234567890123	abc

nexacro font size test
 strText: nexacro font size test
 nCellSize: 60
 nTextLen: 143,1513671875

 sample_grid_95.xfdl

22.17.2 예제에서 사용한 핵심 기능

getTextSize

지정된 폰트 설정에 따라 텍스트가 어느 정도의 영역을 차지하는지 계산합니다. 가로, 세로 크기 정보를 오브젝트로 반환하며 nx, ny 속성으로 값을 확인할 수 있습니다. font 정보가 있어야 하기 때문에 테마 또는 XCSS 파일에서 지정한 스타일을 참조하는 경우에는 사용할 수 없습니다.

getFormatColSize

컬럼 크기를 계산합니다. Grid Contents Editor에서 설정한 값을 반환합니다. 동적으로 컬럼 크기를 조정하도록 설정했다면 getRealColSize 메소드를 사용합니다.

22.17.3 예제 구현 방법

- 1 Grid 컴포넌트와 TextArea 컴포넌트를 화면에 배치합니다.
- 2 Dataset 오브젝트를 추가하고 3개의 컬럼을 생성한 후 데이터를 추가합니다.

Columns					
No	id	type	size	prop	sumtext
1	Column0	STRING	256		
2	Column1	STRING	256		
3	Column2	STRING	256		

Rows			
No	Column0	Column1	Column2
1	1	test test test test 123	abc
2	2	test1	nexacro font size test
3	3	test2345678901234567890	abcdefghijklmnopqrstuvwxyz

③ Dataset 오브젝트를 선택하고 Grid 컴포넌트로 드래그앤드롭해서 binddataset 속성으로 등록합니다.

④ Grid 컴포넌트에서 font 속성값을 지정합니다.

font: bold 14px/normal "Arial"

⑤ Grid 컴포넌트의 onmousemove 이벤트 함수를 아래와 같이 작성합니다.

```
this.Grid00_onmousemove = function(obj:nexacro.Grid,e:nexacro.GridMouseEventInfo)
{
    var nCellSize = obj.getFormatColSize(e.cell);
    var strText    = obj.getCellText(e.row, e.cell);
    var nTextLen   = nexacro.getTextSize( strText, obj.font );

    this.TextArea00.set_value("strText: "+strText+"\n");
    this.TextArea00.insertText("nCellSize: "+nCellSize+"\n");
    this.TextArea00.insertText("nTextLen: "+nTextLen.nx+"\n");

    if( nCellSize < nTextLen.nx){
        obj.setCellProperty( "Body", e.cell, "tooltiptext", strText);
    } else {
        obj.setCellProperty( "Body", e.cell, "tooltiptext", "");
    }
};
```

⑥ QuickView(Ctrl + F6)로 실행한 후 셀 영역에 마우스 커서를 가져가면 텍스트 길이에 따라 툴팁이 표시되고, TextArea 컴포넌트에 관련 정보를 출력합니다.

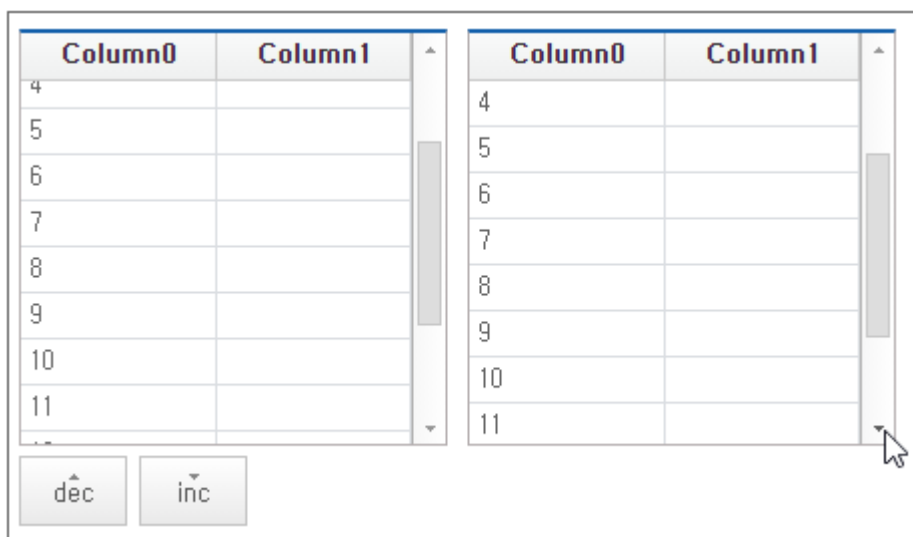
22.18 한 행씩 스크롤하기

데이터가 많아서 스크롤바가 생겼을때 스크롤을 이동하는 방법은 여러가지가 있습니다. 마우스 휠 동작으로 이동할 수 있고, 스크롤바 양쪽에 버튼을 클릭해서 이동할 수도 있습니다. 아니면 스크롤바에 있는 트랙바를 잡은 상태에서 원하는 방향으로 끌어서 이동할 수도 있습니다.

Grid 컴포넌트는 원하는 목적에 따라 스크롤 형식을 설정할 수 있습니다. 예제에서는 한 행씩 스크롤하는 방법을 살펴봅니다.

22.18.1 예제

Grid 컴포넌트 아래에 있는 버튼을 클릭하거나 마우스 휠 동작으로 움직이거나 트랙바를 잡아서 끌어 이동할때 오른 쪽 Grid 컴포넌트는 한 행씩 움직입니다.



 [sample_grid_102.xfdl](#)

22.18.2 예제에서 사용한 핵심 기능

wheelscrollrow

마우스 휠 동작 시 한번 스크롤 시 이동하는 Row를 설정합니다.

scrollpixel

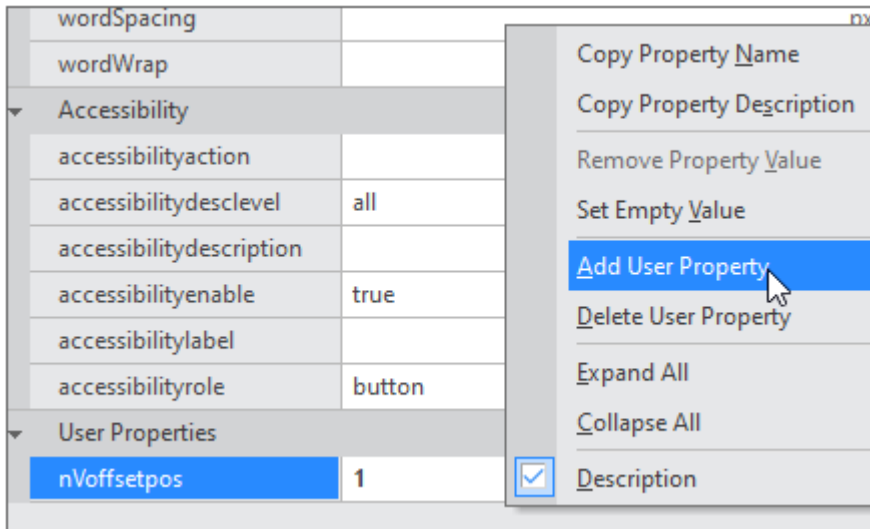
스크롤 동작 단위를 설정합니다. 기본값은 "default"이며 데스크탑에서 "none"으로 동작합니다. Row 단위로 이동하도록 설정하면 "all"로 속성값을 설정합니다.

22.18.3 예제 구현 방법

- ① Grid 컴포넌트를 화면에 배치합니다.
- ② Dataset 오브젝트를 추가하고 2개의 컬럼을 생성한 후 데이터를 추가합니다.
15개 정도 Row를 추가하고 첫 번째 컬럼값을 1부터 15까지 설정합니다.

Columns		
No	id	type
1	Column0	STRING
2	Column1	STRING
Rows		
No	Column0	Column1
1	1	[Undefin
2	2	[Undefin
3	3	[Undefin
4	4	[Undefin

- ③ Dataset 오브젝트를 선택하고 Grid 컴포넌트로 드래그앤드롭해서 binddataset 속성으로 등록합니다.
- ④ Grid 컴포넌트를 복사해서 새로운 Grid 컴포넌트를 추가합니다.
- ⑤ 첫 번째 Grid 컴포넌트의 scrollpixel 속성값을 "all"로 수정합니다.
- ⑥ 두 번째 Grid 컴포넌트의 wheelscrollrow 속성값을 "1"로 수정합니다.
- ⑦ Grid 컴포넌트 아래 2개의 Button 컴포넌트를 추가합니다.
- ⑧ Button 컴포넌트에 User Property를 "nVoffsetpos"라는 이름으로 추가하고 각각 -1, 1 값을 설정합니다.
Button 클릭 시 User Property 값과 연산을 통해 이동 방향을 조정하도록 합니다.



- 9 Button 컴포넌트의 onclick 이벤트 함수를 아래와 같이 작성합니다.

```
this.Btn_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Grid00.scrollBy(0,this.Grid00.getRealRowSize(0)*obj.nVoffsetpos);
    this.Grid01.scrollBy(0,this.Grid01.getRealRowSize(0)*obj.nVoffsetpos);
};
```

- 10 QuickView(Ctrl + F6)로 실행한 후 Grid 아래 버튼을 클릭하거나 마우스 휠 또는 스크롤 버튼을 클릭해서 스크롤 위치를 이동해봅니다.

22.19 Row 위치 변경하기

바인딩된 Dataset 오브젝트의 Row 인덱스를 변경합니다. 예제에서는 Grid 컴포넌트의 ondrag 이벤트 발생 시 Row 인덱스를 확인하고 ondrop 이벤트에서 Row 위치를 변경합니다.

22.19.1 예제

Grid 컴포넌트 아래에 있는 버튼을 클릭하거나 마우스 휠 동작으로 움직이거나 트랙바를 잡아서 끌어 이동할때 오른 쪽 Grid 컴포넌트는 한 행씩 움직입니다.

Column0	Column1
1	test1
2	test2
3	test3
4	test4



sample_grid_106.xfdl

22.19.2 예제에서 사용한 핵심 기능

moveRow

지정한 Row를 특정 위치로 이동합니다. 이동하는 위치에 따라 나머지 Row의 위치도 같이 변경됩니다.

22.19.3 예제 구현 방법

- 1 Grid 컴포넌트를 화면에 배치합니다.
- 2 Dataset 오브젝트를 추가하고 아래와 같이 데이터를 설정합니다.

Columns				
No	id	type	size	prop
1	Column0	STRING	256	
2	Column1	STRING	256	

Rows		
No	Column0	Column1
1	1	test1
2	2	test2
3	3	test3
4	4	test4

- 3 Dataset 오브젝트를 선택하고 Grid 컴포넌트로 드래그앤드롭해서 binddataset 속성으로 등록합니다.

- ④ Static 컴포넌트를 추가하고 아래와 같이 속성값을 설정합니다.

Static 컴포넌트는 Row 이동 시 이동하는 위치를 표시하기 위해 사용합니다.

```
visible: false
border: 2px dotted black
```

- ⑤ Grid 컴포넌트의 ondrag 이벤트 함수를 아래와 같이 작성합니다.

이벤트가 시작하는 지점에 Row 정보를 확인하고 Row 인덱스값을 GridDragEventInfo 오브젝트의 userdata 로 설정합니다. 그리고 선택한 Row 정보를 Static 컴포넌트에 표시합니다.

```
this.Grid00_ondrag = function(obj:nexacro.Grid,e:nexacro.GridDragEventInfo)
{
    e.set_userdata(e.row);
    var strVal = this.Dataset00.getColumn(e.row, "Column0");
    strVal += " : " + this.Dataset00.getColumn(e.row, "Column1");
    this.Static00.set_text(strVal);
    this.Static00.set_visible(true);
    var nL = obj.getOffsetLeft() + e.clientX;
    var nT = obj.getOffsetTop() + e.clientY;
    this.Static00.move(nL, nT);
    return true;
};
```

- ⑥ Form 오브젝트의 ondragmove 이벤트 함수를 아래와 같이 작성합니다.

Static 컴포넌트가 마우스 포인터 위치를 따라가도록 합니다. Grid 컴포넌트가 아닌 Form 오브젝트의 ondrag move 이벤트 함수에서 처리해주기 때문에 Grid 컴포넌트 영역을 벗어나도 계속 드래그 동작을 유지합니다.

```
this.sample_grid_106_ondragmove = function(obj:nexacro.Form,e:nexacro.DragEventInfo)
{
    this.Static00.move(e.clientX, e.clientY + 10);
};
```

- ⑦ Grid 컴포넌트의 ondrop 이벤트 함수를 아래와 같이 작성합니다.

userdata로 받아온 값을 이전 Row 인덱스로 설정하고 현재 위치를 새로운 인덱스로 설정해 Row의 위치를 변경합니다.

```
this.Grid00_ondrop = function(obj:nexacro.Grid,e:nexacro.GridDragEventInfo)
{
    this.Static00.set_visible(false);
    var nRow = e.userdata;
    this.Dataset00.moveRow(nRow, e.row);
};
```

- ⑧ QuickView(Ctrl + F6)로 실행한 후 Grid 컴포넌트의 Row를 선택하고 끌어다가 새로운 Row 위치에 놓습니다.

23.

Grid 레이아웃

23.1 Cell 크기 조절하기 (Row 크기 일괄 조절)

Grid에서 한 Cell에 많은 양의 데이터가 들어 있으면 전체 내용을 다 볼 수 없는 경우가 있습니다. 이럴 때 엑셀에서처럼 자유롭게 컬럼과 로우 크기를 조절하는 기능이 있으면 사용자가 좀 더 편하게 Grid를 이용할 수 있습니다.

cellsizingtype 속성은 사용자가 마우스를 Cell이나 Head의 경계 부분에 가져가면 포인터가 바뀌고 그 상태에서 마우스 버튼을 누르고 움직이면 셀의 크기를 변경할 수 있습니다. cellsizingtype 속성은 none, col, row, both 중 하나로 설정할 수 있습니다. col로 설정하면 너비만 변경할 수 있고 row로 설정하면 높이만 변경할 수 있습니다. both로 설정하면 너비와 높이 모두 변경이 가능하며 none으로 설정하면 셀 크기 변경을 할 수 없습니다.

23.1.1 예제

다음은 컬럼과 로우의 크기를 변경할 수 있는 Grid 예제입니다.

Name	Address	Company	Department	Salary	
John	Seoul, KOREA	1dollar	Sales	₩15,000	
James	Barcelona, Spain	EBOT	Consulting	₩25,000	
Donald	Califonia, USA	EBOT	Research Institute	₩1,000,000	
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000	
Joe	São Paulo, Brazil	hangul	Quality Assurance	₩8,600	



sample_grid_12.xfdl

cellsizingtype 속성이 both로 설정되어 있으므로 컬럼이나 로우의 크기를 모두 변경할 수 있습니다. 여기서 한가지 유의할 점은 한 로우의 높이를 변경하면 모든 로우 높이가 동일하게 적용된다는 것입니다. 로우 별로 높이를 따로 조절하려면 extendsizetype 속성을 따로 설정해줘야 합니다. 이에 대한 설명은 [Cell 크기 조절하기 \(특정 Row 크기 조](#)

절)를 참조하시기 바랍니다.

너비를 변경하려면 Head Column의 경계 부분에서 포인터가 변경되었을때 마우스 버튼을 누른 상태로 이동하고, 높이를 변경하려면 각Row의 경계 부분에서 포인터가 변경되었을때 마우스 버튼을 누른 상태로 이동합니다.

23.1.2 예제에서 사용한 핵심 기능

cellsizingtype

사용자가 마우스로 Row 또는 Column의 크기를 변경할 수 있게 설정하는 속성입니다.

none	(default) Column 및 Row의 크기를 변경할 수 없습니다.
col	마우스로 Head Column의 경계를 드래그해 가로 크기만 변경할 수 있습니다.
row	마우스로 Row의 경계를 드래그해 세로 크기만 변경할 수 있습니다.
both	마우스로 Head Column의 경계나 Row의 경계를 드래그해 가로 혹은 세로 크기를 모두 변경할 수 있습니다.

23.1.3 예제 구현 방법

① 화면 구성하기

툴바에서 Grid  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택한 후 Form 위에 놓고 그대로 클릭하면 기본 크기로 생성됩니다.

② Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset00을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.

Dataset Editor

Const Columns

No

id

type

size

value

There are no items to show.

Columns

No

id

type

size

prop

sumtext

description

1

Name

STRING

256

2

Address

STRING

256

3

Company

STRING

256

4

Department

STRING

256

5

Salary

INT

256

Rows

No

Name

Address

Company

Department

Salary

1

John

Seoul, KOREA

1dollar

Sales

15000

2

James

Barcelona, Spain

EBOT

Consulting

25000

3

Donald

Califonia, USA

EBOT

Research Institute

1000000

4

Lisa

London, United Kingdom

1dollar

Quality Assurance

3000

5

Joe

São Paulo, Brazil

hangul

R&D

8600

6

Michael

Mumbai, India

1dollar

Personnel

1300

7

Lilli

Ankara, Turkey

hangul

Quality Assurance

4400

8

Colt

Sydney, NSW, Australia

hangul

Sales

62000

Dataset

Source

3 Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset00을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

4 Grid 설정하기

Grid를 선택한 후 cellsizingtype 속성을 both로 설정합니다.

5 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Grid의 Head Column의 경계나 Grid의 Cell의 경계를 마우스로 드래그해 너비가 변경되는지 확인합니다.

23.2 Cell 크기 조절하기 (특정 Row 크기 조절)

Grid에서 한 Cell에 많은 양의 데이터가 들어 있으면 전체 내용을 다 볼 수 없는 경우가 있습니다. 이럴 때 엑셀에서처럼 자유롭게 컬럼과 로우 크기를 조절하는 기능이 있으면 사용자가 좀 더 편하게 Grid를 이용할 수 있습니다.

Grid의 `cellsizingtype` 속성은 사용자가 마우스로 Cell과 Head의 경계 부분을 드래그해 레코드의 크기를 변경할 수 있게 합니다. 설정에 따라 너비 혹은 높이만 조절하거나 둘 다 조절하게 할 수도 있습니다.

기본적으로 Cell의 높이는 하나를 변경하면 모두 같이 적용됩니다. 높이를 Cell 별로 조절하려면 `extendsizetype` 속성을 설정해야 합니다.

23.2.1 예제

다음은 컬럼과 로우의 크기를 변경할 수 있는 Grid 예제입니다.

Name	Address	Company	Department	Salary	
John	Seoul, KOREA	1dollar	Sales	₩15,000	
James	Barcelona, Spain	EBOT	Consulting	₩25,000	
Donald	California, USA	EBOT	Research Institute	₩1,000,000	
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000	
Joe	São Paulo, Brazil	hangul	Quality Assurance	₩8,600	
Michael	Mumbai, India	1dollar	Quality Assurance	₩1,300	



`extendsizetype` 속성을 row로 설정했기 때문에 각 Row 별로 높이를 자유롭게 변경할 수 있습니다.

너비를 변경하려면 Head Column의 경계 부분을 마우스로 드래그하고, 높이를 변경하려면 각Row의 경계 부분을 마우스로 드래그합니다.

23.2.2 예제에서 사용한 핵심 기능

cellsizingtype

사용자가 마우스로 Row 또는 Column의 크기를 변경할 수 있게 설정하는 속성입니다.

none	(default) Column 및 Row의 크기를 변경할 수 없습니다.
col	마우스로 Head Column의 경계를 드래그해 가로 크기만 변경할 수 있습니다.
row	마우스로 Row의 경계를 드래그해 세로 크기만 변경할 수 있습니다.
both	마우스로 Head Column의 경계나 Row의 경계를 드래그해 가로 혹은 세로 크기를 모두 변경할 수 있습니다.

extendsizetype

Row의 높이가 변경될 때 Row 높이를 어떻게 조절할지 설정하는 속성입니다.

none	(default) Row 높이가 변경될 때 모든 Row의 높이를 같이 설정합니다. Row 별로 높이를 따로 설정할 수 없습니다.
row	Row 높이가 변경될 때 각 Row 별로 높이를 설정합니다.

23.2.3 예제 구현 방법

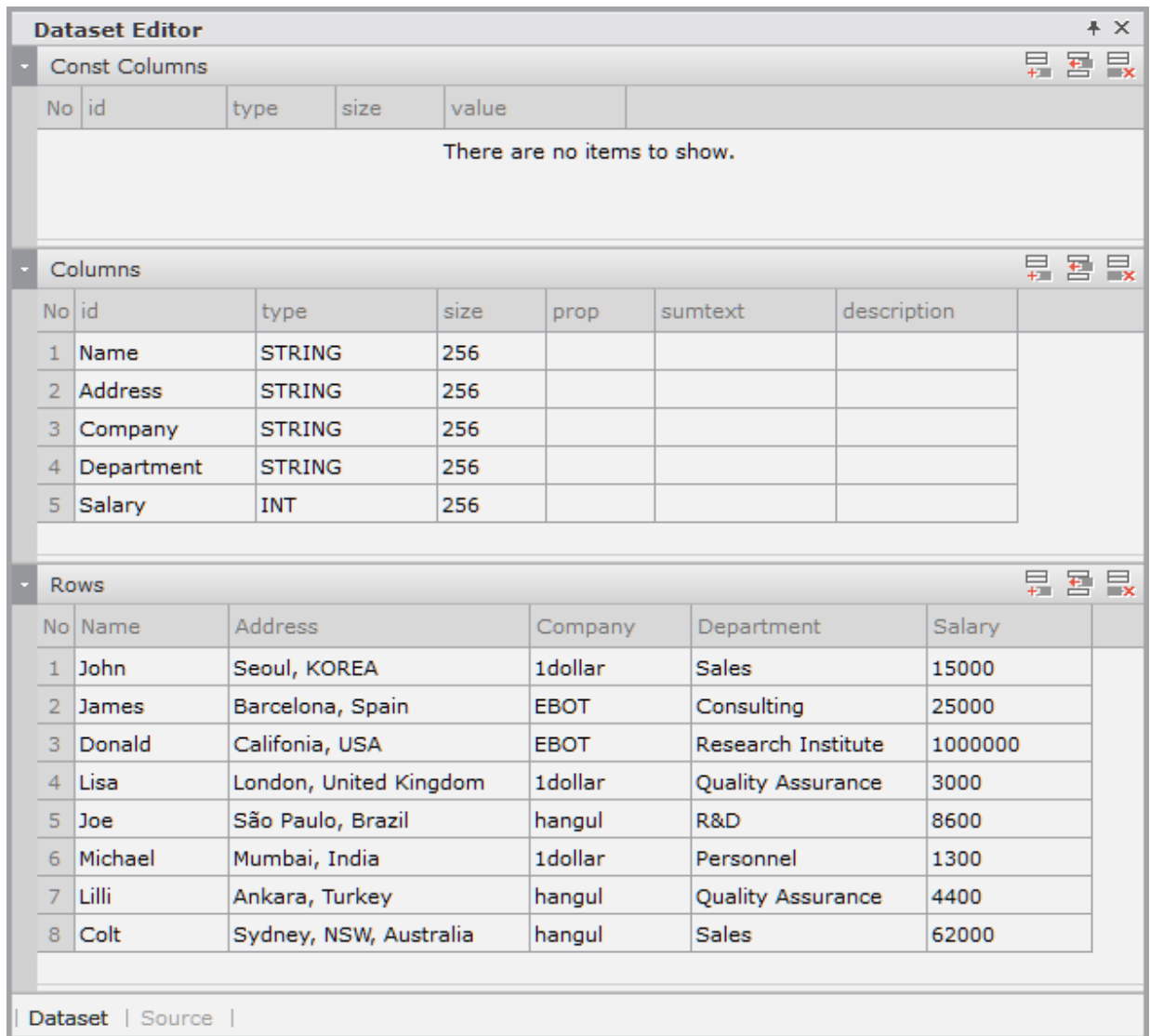
① 화면 구성하기

툴바에서 Grid  컴포넌트를 예제의 그림과 같이 적절히 배치합니다.

② Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset00을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.



3 Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset00을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

4 Grid 설정하기

Properties 창에서 cellsizingtype 속성을 both로, extendsizetype 속성을 row로 설정합니다.

5 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Grid의 Head Column의 경계를 마우스로 드래그해 너비가 변경되는지 확인합니다.

Grid의 Cell 경계를 마우스로 드래그해 높이가 변경되는지 확인합니다.

23.3 다중 헤드와 다중 레코드 표현하기

Grid에서 하나의 레코드에 출력할 정보가 많거나 가로의 너비가 좁아 한 화면에 다 출력하기 힘들 때도 있습니다. 이 때, Grid의 다중 헤드와 레코드를 사용하면 좀 더 효율적으로 화면을 구성할 수 있습니다.

23.3.1 예제

아래는 Head 밴드와 Body 밴드에서 다중 헤드와 다중 레코드를 사용한 예제입니다.

EEmail와 Department Column을 각각 Address와 Company Column의 밑에 배치하여 좁은 가로 너비를 효율적으로 활용하였습니다.

Name	Address	Company	Salary
	EEmail	Department	
John	Seoul, KOREA	1dollar	₩15,000
	sales@1dollar.com	Sales	
James	Barcelona, Spain	EBOT	₩25,000
	consulting@ebot.com	Consulting	
Donald	Califonia, USA	EBOT	₩1,000,000
	research@ebot.com	Research Institute	
Lisa	London, United Kingdom	1dollar	₩3,000
	quality@1dollar.com	Quality Assurance	
Joe	São Paulo, Brazil	hangul	₩8,600
	rnd@hangul.com	R&D	
Michael	Mumbai, India	1dollar	₩1,300
	personnel@1dollar.com	Personnel	
Lilli	Ankara, Turkey	1dollar	₩4,400
	sales@1dollar.com	Sales	
Colt	Sydney, NSW, Australia	hangul	₩62,000
	sales2@hangul.com	Sales	



sample_grid_14.xfdl

23.3.2 예제에서 사용한 핵심 기능

없음.

23.3.3 예제 구현 방법

1 화면 구성하기

툴바에서 Grid  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택한 후 Form 위에 놓고 그대로 클릭하면 기본 크기로 생성됩니다.

2 Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset00을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.

Dataset Editor

Const Columns

No

id

type

size

value

There are no items to show.

Columns

No

id

type

size

prop

sumtext

description

1

Name

STRING

256

2

Address

STRING

256

3

Company

STRING

256

4

Department

STRING

256

5

EMail

STRING

256

6

Salary

INT

256

Rows

No

Name

Address

Company

Department

EMail

Salary

1

John

Seoul, KOREA

1dollar

Sales

sales@1dollar.com

15000

2

James

Barcelona, Spain

EBOT

Consulting

consulting@ebot.com

25000

3

Donald

Califonia, USA

EBOT

Research Institute

research@ebot.com

1000000

4

Lisa

London, United Kingdom

1dollar

Quality Assurance

quality@1dollar.com

3000

5

Joe

São Paulo, Brazil

hangul

R&D

rnd@hangul.com

8600

6

Michael

Mumbai, India

1dollar

Personnel

personnel@1dollar.com

1300

7

Lilli

Ankara, Turkey

1dollar

Sales

sales@1dollar.com

4400

8

Colt

Sydney, NSW, Australia

hangul

Sales

sales2@hangul.com

62000

Dataset

Source

Dataset | Source |

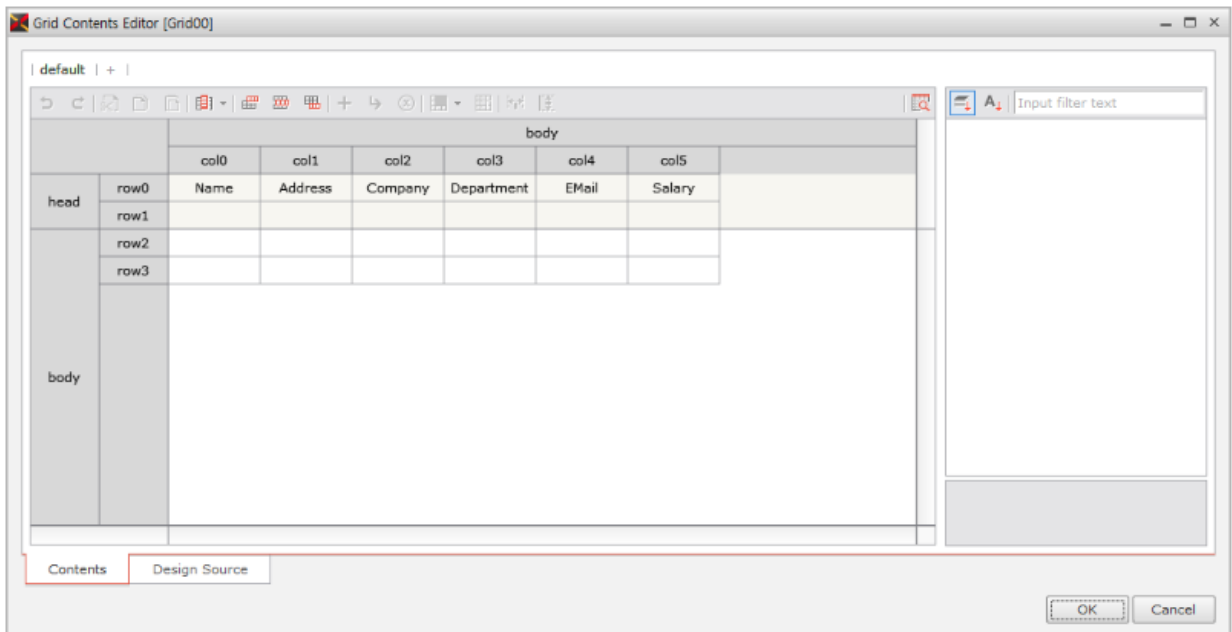
3 Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

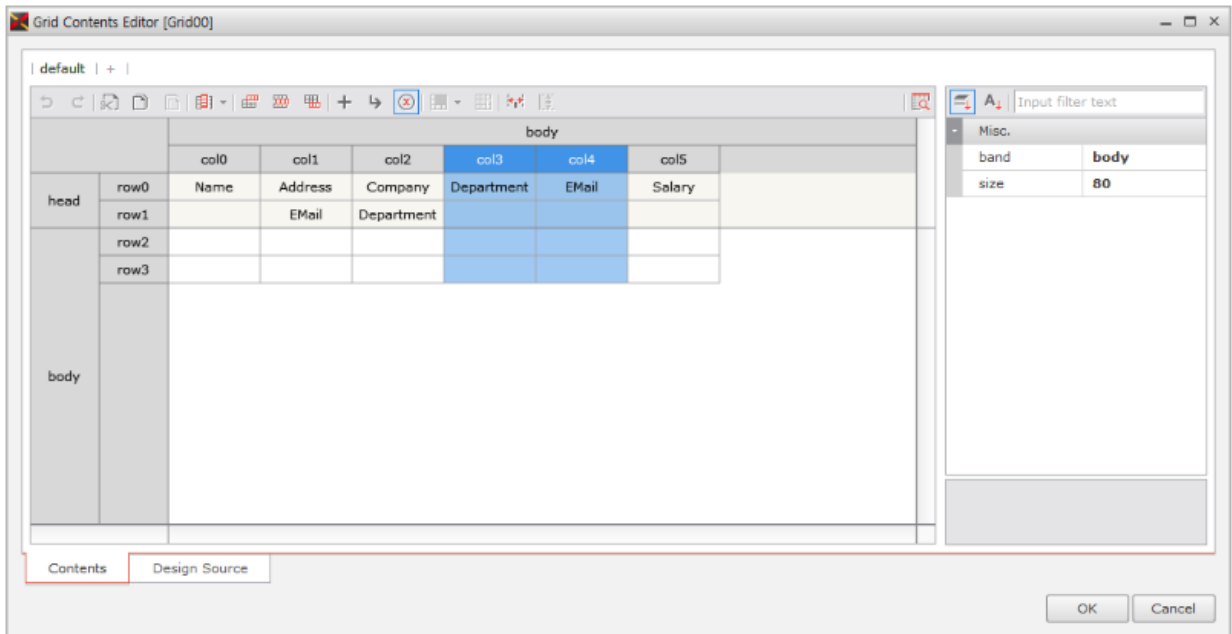
Invisible Objects 영역에 있는 Dataset00을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

4 다중 헤드, 다중 레코드 생성

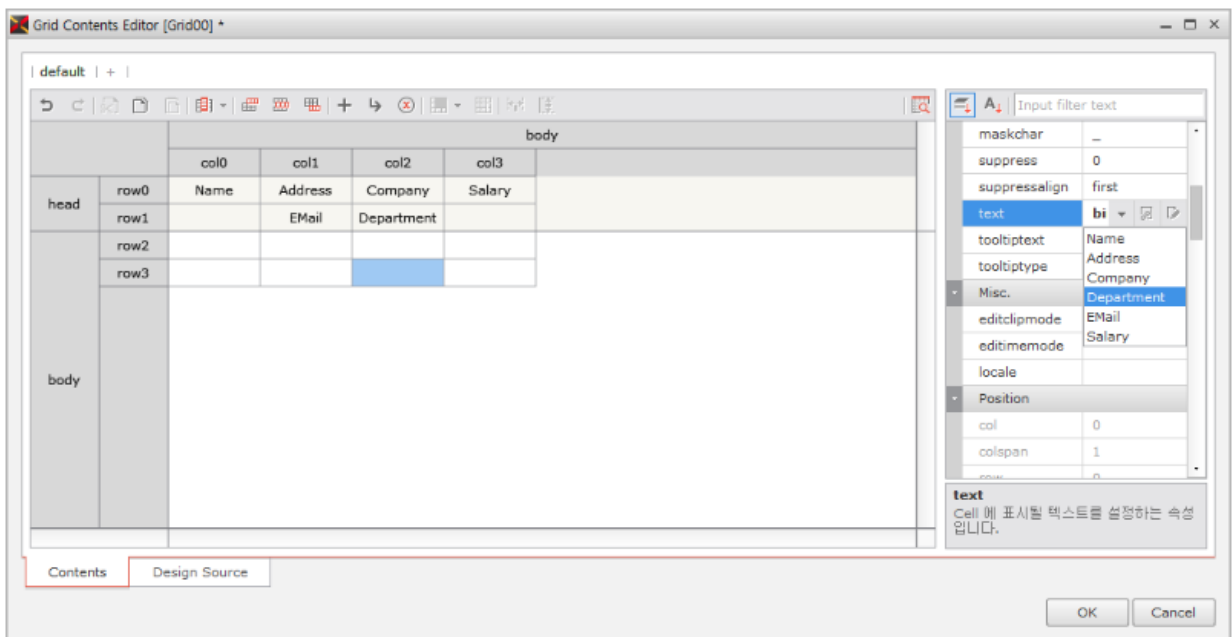
1. Grid 컴포넌트를 더블 클릭하여 Grid Contents Editor를 엽니다.
2. Head row, Body row를 하나씩 추가합니다. row를 추가하기 위해서는 Grid Contents Editor에서 Add Head Row  버튼과 Add Body Row  버튼을 한 번씩 눌러줍니다. Grid Contents Editor에서 마우스 오른쪽 버튼을 눌러도 row를 추가할 수 있는 같은 컨텍스트 메뉴가 나옵니다.



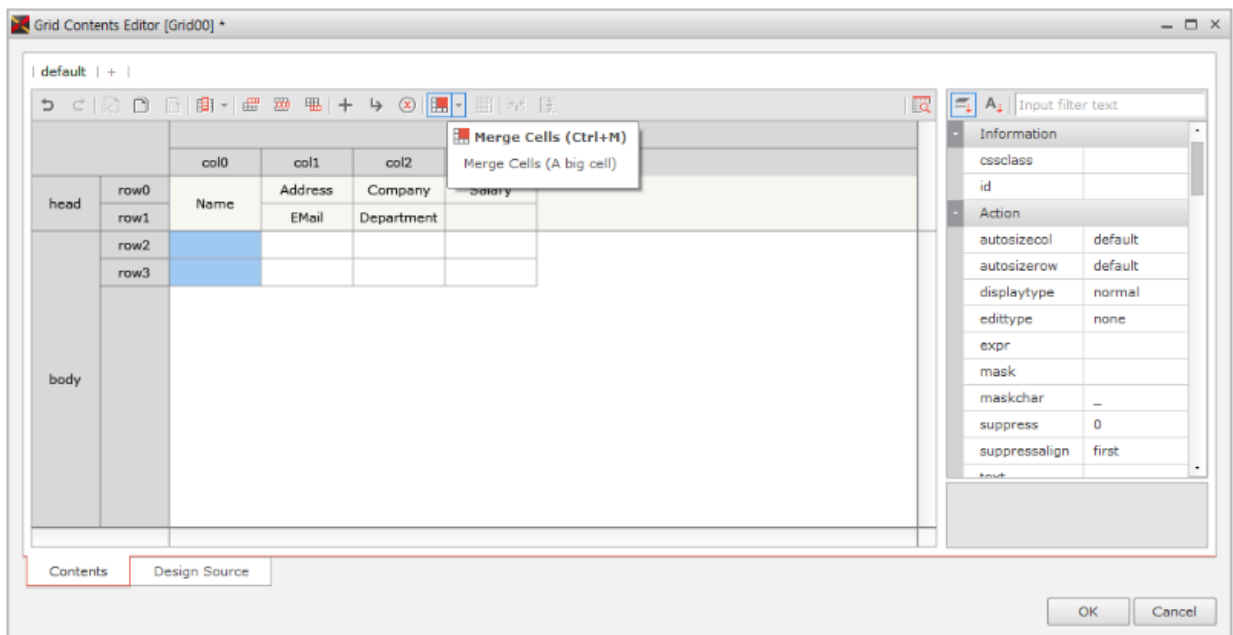
3. 새로 추가된 head row에서 Address, Company Column 밑의 cell을 각각 선택하여 text 속성에 EEmail, Department라고 입력합니다.
4. 필요 없어진 Department, EEmail Column을 선택한 후 Delete  버튼을 눌러 삭제합니다.



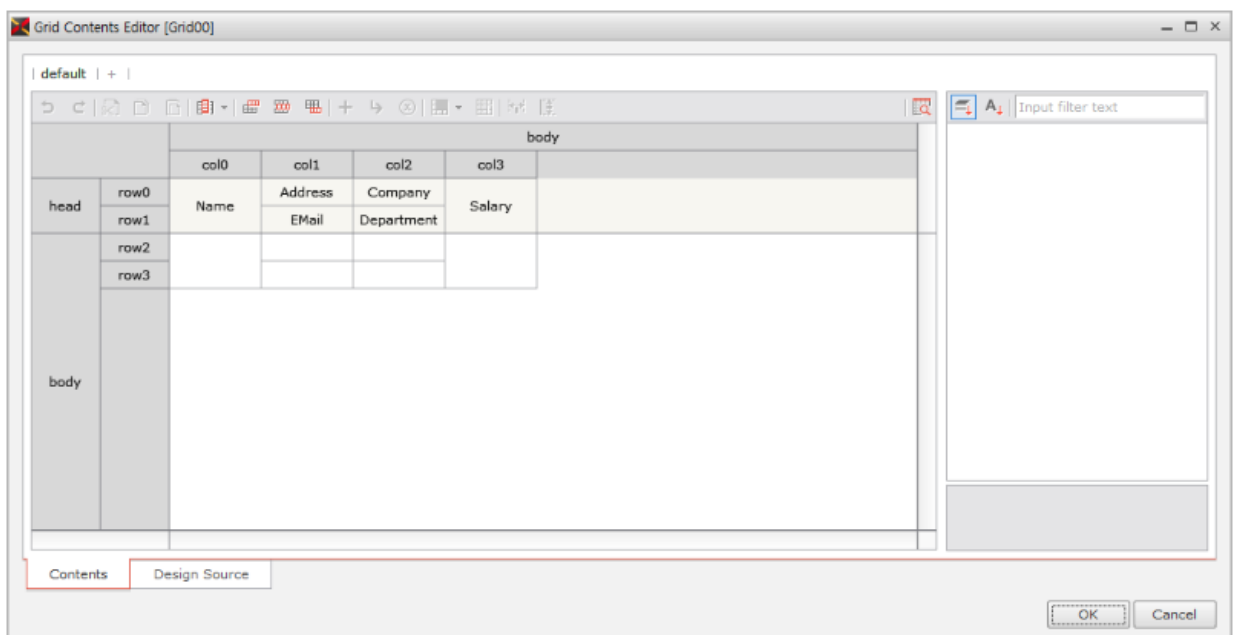
5. EMail, Department Column에 해당하는 Body row3의 Cell을 선택한 후 각 Column에 맞게 Dataset의 EMail, Department로 바인딩합니다.



6. head row와 body row의 추가로 인해 Name, Salary Column이 두 줄씩 생겼으므로 이를 하나로 합쳐줍니다. 병합할 Cell을 선택 후에 Merge Cells  버튼을 누르면 하나의 Cell로 합쳐집니다.



Cell 병합까지 완료되면 최종적으로 아래와 같은 모양이 됩니다. OK 버튼을 누릅니다.



5 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

다중 헤드와 레코드로 출력되는지 확인합니다.

23.4 자동 줄 바꾸기와 자동 Cell 크기 조절하기

데이터의 길이가 Cell의 크기보다 너무 길거나 여러 줄로 되어 있으면 데이터가 잘려 보일 수 있습니다. 이럴 때 자동 줄 바꿈, Cell 높이 조절 기능을 사용하면 데이터 길이에 상관없이 Grid에 표현할 수 있습니다.

자동 줄 바꿈은 데이터 길이가 Cell의 너비보다 큰 경우 데이터가 잘려 보이지 않게 자동으로 Cell의 너비에 맞게 자동으로 줄 바꿈이 되는 기능입니다. Cell의 `wordwrap` 속성을 사용하여 글자, 단어 단위로 줄 바꿈을 할 수 있습니다.

자동 Cell 높이 조절은 데이터 길이가 길거나 여러 줄로 된 경우 데이터가 잘려보이지 않게 자동으로 Cell의 높이를 설정해주는 기능입니다. Grid의 `autosizingtype` 속성을 설정하여 Column, Row를 설정할 수 있습니다.

23.4.1 예제

아래는 자동 줄 바꿈 기능과 자동 Cell 높이 조절 기능을 적용한 Grid 예제입니다.

자동 줄 바꿈 기능은 `wordwrap`의 설정에 따라 다음과 같이 달리 보입니다. ❶은 `wordwrap` 속성을 `char`로 설정한 경우로 Cell의 너비를 넘어서는 글자는 글자 단위로 줄 바꿈이 됩니다. ❷은 `wordwrap` 속성을 `english`로 설정한 경우로 Cell의 너비를 넘어서는 글자는 단어 단위로 줄 바꿈이 됩니다.



`wordwrap` 속성은 오직 영어만 단어 단위 줄 바꿈을 지원합니다. 다른 언어일 경우에는 자동으로 글자 단위 줄 바꿈이 수행됩니다.

Grid의 `autosizingtype` 속성이 `row`로 설정되어 있어 자동 줄 바꿈이 발생해도 Cell의 높이가 자동 조절되어 데이터가 잘리지 않습니다.

Name	Address	Company	Department	Salary
John	Seoul, KOREA	1dollar	Sales	₩15,000
James	Barcelona, Spain	EBOT	Consulting	₩25,000
Donald	Califonia, USA	EDOT	Research Institute	₩1,000,000
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
Joe	São Paulo, Brazil	hangul	R&D	₩8,600
Michael	Mumbai, India	1dollar	Personnel	₩1,300
Lilli	Ankara, Turkey	1dollar	Quality Assurance	₩4,400
Colt	Sydney, NSW, Australia	hangul	Sales	₩62,000



23.4.2 예제에서 사용한 핵심 기능

autosizingtype

Grid에 데이터가 모두 표시되도록 Row, Column의 크기를 자동 조절할지 설정하는 속성입니다.

enumType ::= 'none' 'col' 'row' 'both'	
none	(default) 크기를 자동 조절하지 않고 디자인 시 설정한 크기를 유지합니다.
col	autosizebandtype 속성에 설정한 영역의 각 Column 너비만 자동 조절합니다.
row	autosizebandtype 속성에 설정한 영역의 각 Row 높이만 자동 조절합니다.
both	autosizebandtype 속성에 설정한 영역의 각 Column 너비와 각 Row 높이를 자동 조절합니다.

wordwrap

Grid의 Cell에 표시되는 데이터가 Cell의 너비보다 길 경우 자동으로 줄 바꿈이 되도록 설정하는 속성입니다.

strwordwrap ::= 'none' 'char' 'english'	
none	(default) 자동으로 줄 바꿈을 하지 않습니다.
char	글자 단위로 줄 바꿈을 자동 수행합니다.
english	단어 단위로 줄 바꿈을 자동 수행합니다. 단, 영어만 해당하며 다른 언어의 경우에는 글자 단위로 줄 바꿈을 수행합니다.

23.4.3 예제 구현 방법

① 화면 구성하기

툴바에서 Grid  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택한 후 Form 위에 놓고 그대로 클릭하면 기본 크기로 생성됩니다.

② Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset을 더블 클

릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.

The screenshot shows the 'Dataset Editor' window with three main sections: 'Const Columns', 'Columns', and 'Rows'.

Const Columns: A table with columns 'No', 'id', 'type', 'size', and 'value'. It contains the message 'There are no items to show.'

Columns: A table with columns 'No', 'id', 'type', 'size', 'prop', 'sumtext', and 'description'. It contains 5 rows of data:

No	id	type	size	prop	sumtext	description
1	Name	STRING	256			
2	Address	STRING	256			
3	Company	STRING	256			
4	Department	STRING	256			
5	Salary	INT	256			

Rows: A table with columns 'No', 'Name', 'Address', 'Company', 'Department', and 'Salary'. It contains 8 rows of data:

No	Name	Address	Company	Department	Salary
1	John	Seoul, KOREA	1dollar	Sales	15000
2	James	Barcelona, Spain	EBOT	Consulting	25000
3	Donald	Califonia, USA	EBOT	Research Institute	1000000
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000
5	Joe	São Paulo, Brazil	hangul	R&D	8600
6	Michael	Mumbai, India	1dollar	Personnel	1300
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000

At the bottom, there are tabs for 'Dataset' and 'Source'.

3 Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

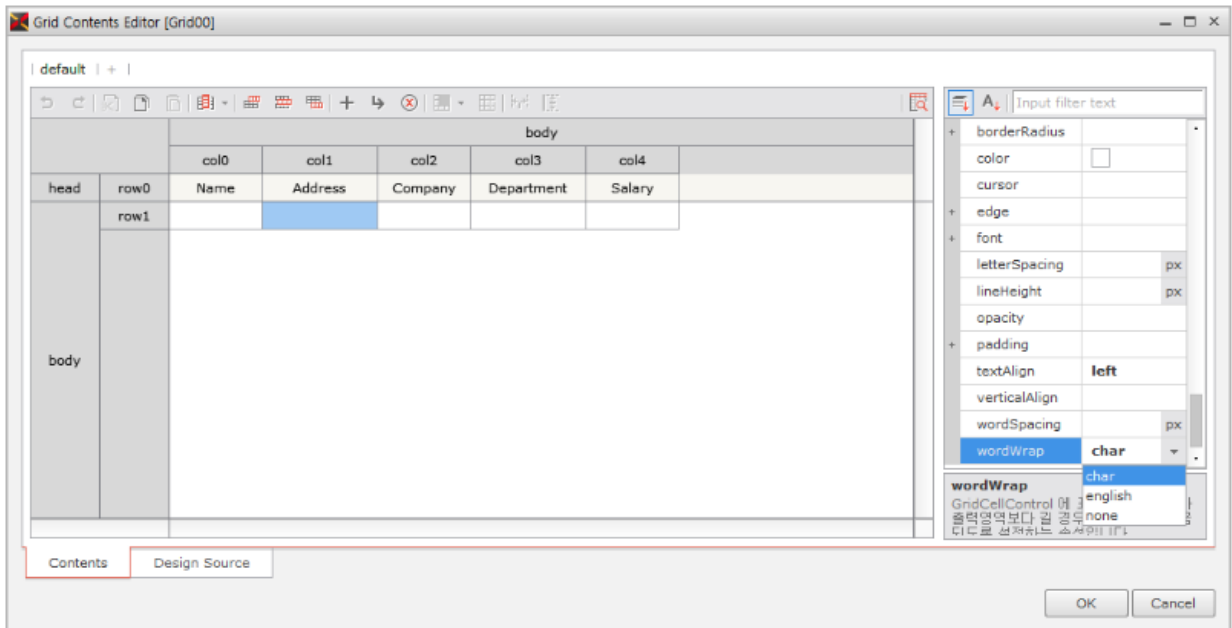
4 Grid의 autosizingtype 속성 설정하기

Properties 창에서 autosizingtype 속성을 row로 설정합니다.

5 Cell의 wordwrap 속성 설정하기

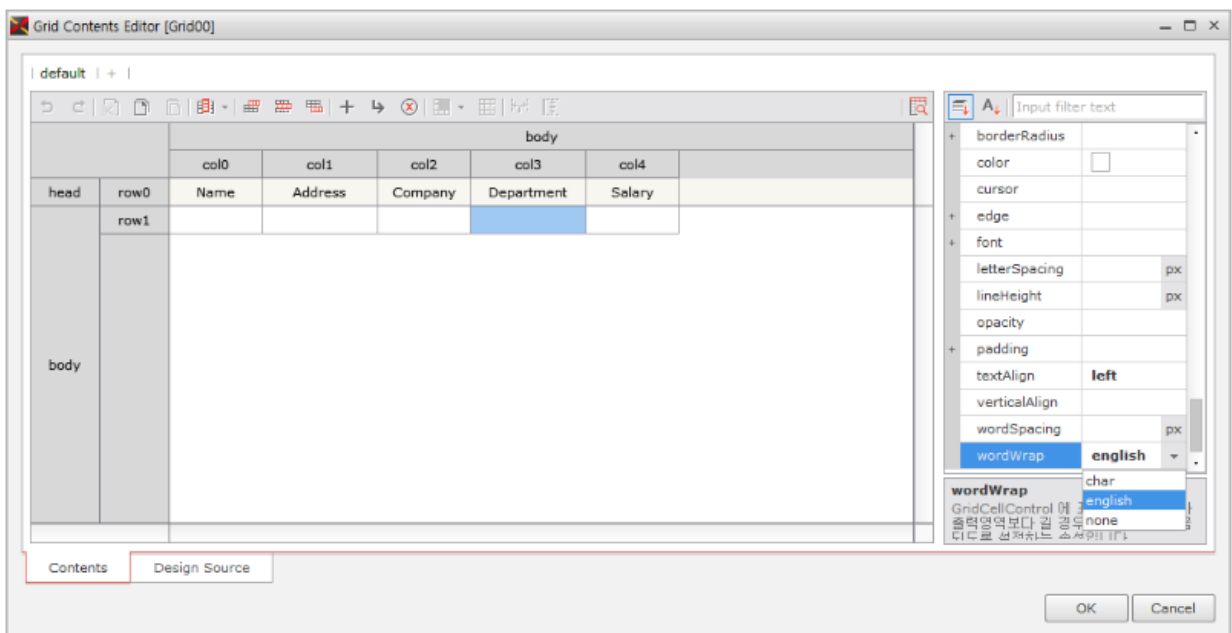
1. Grid를 더블 클릭하여 Grid Contents Editor를 띄웁니다.
2. Address Column에 해당하는 Body 밴드의 row를 선택합니다.

3. wordWrap 속성을 아래 그림과 같이 char로 설정합니다.



4. Department Column에 해당하는 Body 밴드의 row를 선택합니다.

5. wordWrap 속성을 아래 그림과 같이 english로 설정합니다.



6 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Grid의 Address Column이 글자 단위, Department Column이 단어 단위로 각각 줄바꿈이 되었는지 확인합니다.

줄 바꿈 된 데이터가 Grid에서 잘려 보이지 않는지 확인합니다.

23.5 Head Column에 여러 줄 입력하기

Grid의 Head Column은 기본적으로 한 줄의 텍스트로 처리됩니다. 그러나 텍스트가 너무 길거나 텍스트를 의미있게 표현하기 위해 여러 줄로 입력해야 할 경우가 있습니다. Grid Contents Editor는 간단한 키보드 입력 처리만으로 여러 줄을 입력할 방법을 제공합니다.

23.5.1 예제

아래는 Grid Contents Editor를 사용해 Head Column의 텍스트를 여러 줄로 입력한 Grid 예제입니다.

Grid에서 ❶은 두 줄, ❷은 세 줄로 처리하여 한 줄로 되어 있는 Grid의 Head Column에 비해 가로 사이즈를 줄이는 효과를 얻을 수 있습니다.

Name	Address, Country ❶	Company and Department ❷	Salary
John	Seoul, KOREA	1dollar / Sales	₩15,000
James	Barcelona, Spain	EBOT / Consulting	₩25,000
Donald	Califonia, USA	EBOT / Research Institute	₩1,000,000
Lisa	London, United Kingdom	1dollar / Quality Assurance	₩3,000
Joe	São Paulo, Brazil	hangul / R&D	₩8,600
Michael	Mumbai, India	1dollar / Personnel	₩1,300
Lilli	Ankara, Turkey	hangul / Quality Assurance	₩4,400
Colt	Sydney, NSW, Australia	hangul / Sales	₩62,000

23.5.2 예제에서 사용한 핵심 기능

없음.

23.5.3 예제 구현 방법

① 화면 구성하기

툴바에서 Grid  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택한 후 Form 위에 놓고 그대로 클릭하면 기본 크기로 생성됩니다.

② Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.

Dataset Editor

Const Columns

No	id	type	size	value	
There are no items to show.					

Columns

No	id	type	size	prop	sumtext	description	
1	Name	STRING	256				
2	Address	STRING	256				
3	Company	STRING	256				
4	Salary	STRING	256				

Rows

No	Name	Address	Company	Salary	
1	John	Seoul, KOREA	1dollar / Sales	15000	
2	James	Barcelona, Spain	EBOT / Consulting	25000	
3	Donald	Califonia, USA	EBOT / Research Institute	1000000	
4	Lisa	London, United Kingdom	1dollar / Quality Assurance	3000	
5	Joe	São Paulo, Brazil	hangul / R&D	8600	
6	Michael	Mumbai, India	1dollar / Personnel	1300	
7	Lilli	Ankara, Turkey	hangul / Quality Assurance	4400	
8	Colt	Sydney, NSW, Australia	hangul / Sales	62000	

Dataset

Source

Dataset | Source |

③ Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

④ Head Column 설정하기

Head 밴드의 Address Column과 Company Column을 아래와 같이 수정합니다.

Address	Address, Country
Company	Company and Department

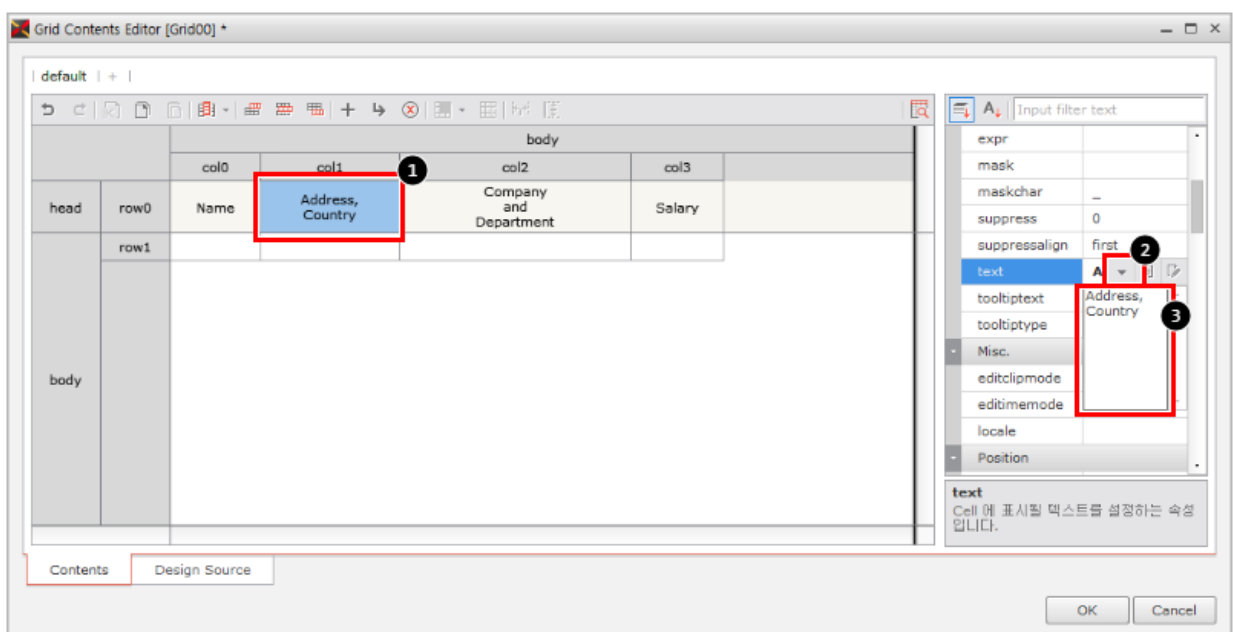
Grid를 선택한 후 더블클릭하여 Grid Contents Editor를 띄웁니다.

- Address Column 설정

Head 밴드의 ① Address Column을 선택합니다. Properties 창의 text 속성에서 ② 버튼을 누르면 버튼 밑에 ③ 편집할 수 있는 작은 창이 뜹니다. ③에 다음을 차례로 입력합니다.

Address, Ctrl+Enter
Country Enter

입력을 다 끝내면 아래 그림과 같이 좌측의 Head Column에 수정 내용이 반영됐는지 확인합니다.

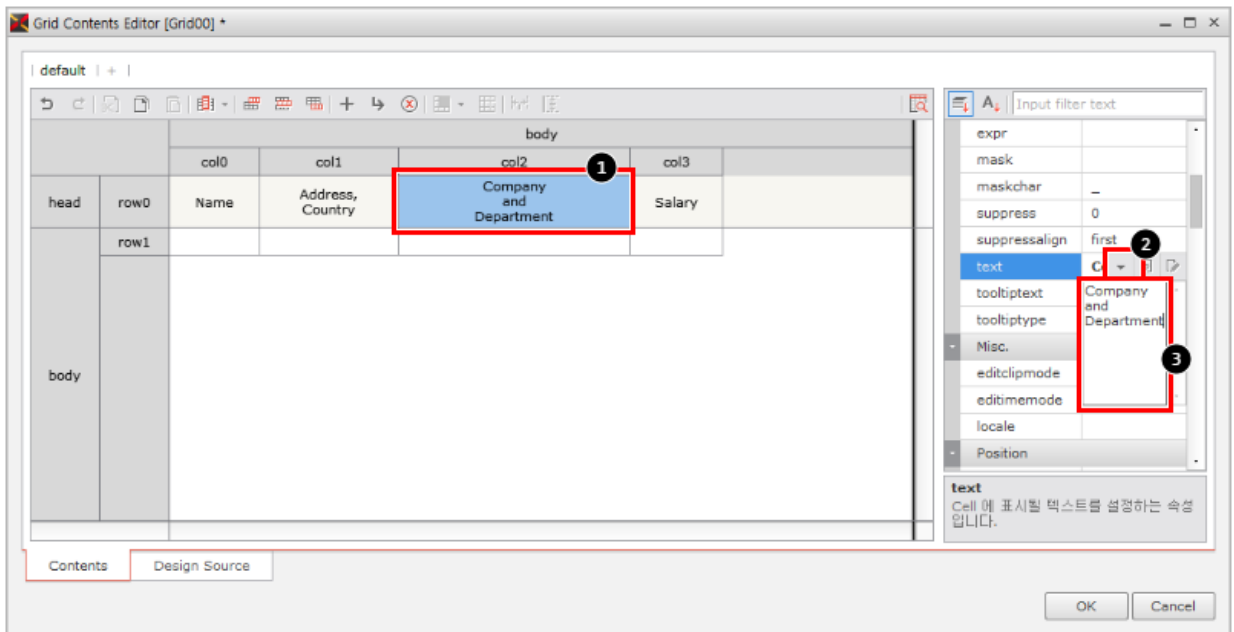


- Company Column의 설정

Head 밴드의 ① Company Column을 선택합니다. Properties 창의 text 속성에서 ② 버튼을 누르면 버튼 밑에 ③ 편집할 수 있는 작은 창이 뜹니다. ③에 다음을 차례로 입력합니다.

Company **Ctrl+Enter**
and **Ctrl+Enter**
Department **Enter**

입력을 다 끝내면 아래 그림과 같이 좌측의 Head Column에 수정 내용이 반영됐는지 확인합니다.



모든 수정이 완료되면 OK 버튼을 눌러 Grid Contents Editor를 종료합니다.

⑤ QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Address와 Company의 Head Column이 각각 두 줄, 세 줄로 표시되는지 확인합니다.

23.6 틀 고정하기 - Column

Grid의 크기는 데이터양에 비례하여 늘어나기 때문에 제한된 크기의 화면에서는 모든 데이터를 한 번에 보기 힘듭니다. 특히 데이터를 서로 비교하며 봐야 할 경우에 스크롤바를 이동하며 보는 것은 여간 불편한 일이 아닙니다. 이런 경우 계속 화면상에 놓고 봐야 할 데이터를 Grid의 한쪽에 고정해놓고 볼 수 있다면 Grid의 활용성을 더욱 높일 수 있습

니다.

이런 기능을 틀 고정이라 합니다. 틀 고정은 Grid의 특정 Column이나 Row를 스크롤에 상관없이 한 자리에 고정해 볼 수 있는 기능입니다. 엑셀에서 자주 사용하는 틀 고정 기능과 유사합니다.

Grid는 기본적으로 틀 고정을 지원하지 않지만, Grid를 구역에 따라 나누는 Band를 활용하여 틀 고정 기능을 구현할 수 있습니다. 아래는 Grid의 밴드 구간을 표시한 그림입니다.

		left		body			right	
		col0		col1	col2	col3	col4	
head	row0	Name		Address	Company	Department	Salary	
body	row1							
summary								

빨간색으로 표시된 부분이 left, right 밴드로 이 밴드에 속한 Column은 고정되어 움직이지 않습니다.

여기서 한가지 유의할 점은 body 밴드에 속한 Column이나 Row는 틀 고정이 안 된다는 것입니다. 다시 말해 Grid 중간에 있는 Column, Row는 틀 고정이 안 되고 Grid의 왼쪽(left 밴드), 오른쪽(right 밴드) 그리고 위쪽(head 밴드), 아래쪽(summary 밴드)에 있는 Column, Row만이 소속 밴드를 변경하여 틀 고정이 가능합니다.

이번 장에서는 Column을 틀 고정하는 방법에 대해 설명합니다.

23.6.1 예제

아래는 Name과 Salary Column을 틀 고정한 Grid 예제입니다.

빨간색 박스로 표시된 Name, Salary Column을 각각 left, right 밴드로 설정하여 틀 고정하였기 때문에 스크롤바를 움직이면 가운데 위치한 Address, Company, Department Column만 이동됩니다.

Name	Address	Company	Department	Salary
John	DREA	1dollar	Sales	₩15,000
James	a, Spain	EBOT	Consulting	₩25,000
Donald	USA	EBOT	Research Instit	₩1,000,000
Lisa	United Kingdom	1dollar	Quality Assura	₩3,000
Joe	o, Brazil	hangul	R&D	₩8,600
Michael	India	1dollar	Personnel	₩1,300
Lilli	Turkey	hangul	Quality Assura	₩4,400
Colt	NSW, Australia	hangul	Sales	₩62,000



23.6.2 예제에서 사용한 핵심 기능

band

Grid에서 Column이 소속될 밴드 명을 설정하는 속성입니다. left, right 밴드는 각각 Grid의 왼쪽, 오른쪽 고정 영역에 표시되며 스크롤시 고정되어 움직이지 않습니다.

enumBand ::= 'left' 'body' 'right'	
left	해당 Column을 left 밴드에 속하게 설정합니다.
body	(default) 해당 Column을 body 밴드에 속하게 설정합니다.
right	해당 Column을 right 밴드에 속하게 설정합니다.

band 속성의 값을 얻거나 설정할 경우에는 각각 getFormatColProperty, setFormatColProperty 메소드를 사용합니다.

```
this.Grid00.setFormatColProperty( 0, "band", "left" );
```

23.6.3 예제 구현 방법

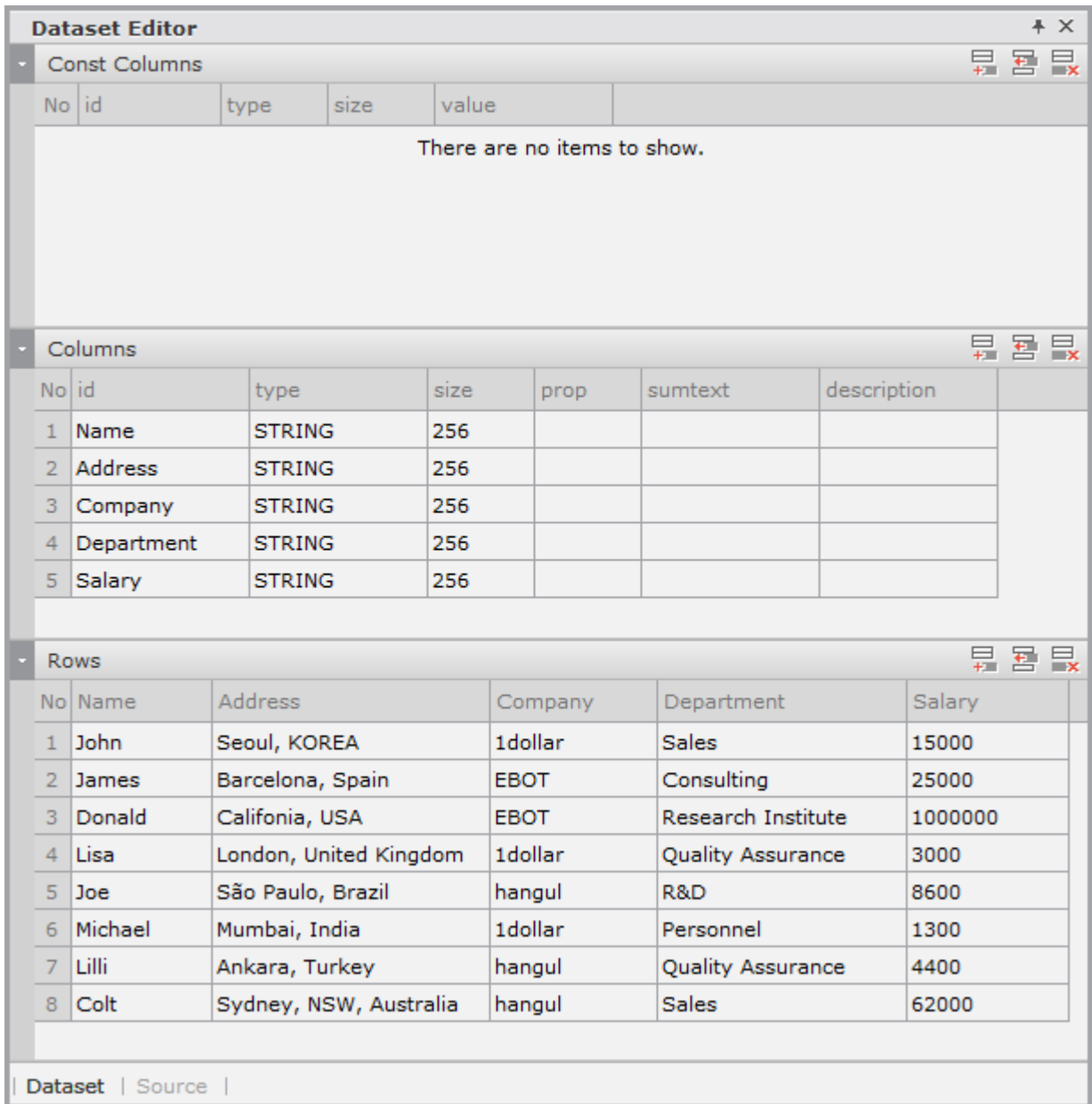
① 화면 구성하기

툴바에서 Grid  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택한 후 Form 위에 놓고 그대로 클릭하면 기본 크기로 생성됩니다.

2 Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.



The Dataset Editor window is titled "Dataset Editor". It contains three main sections: "Const Columns", "Columns", and "Rows".

Const Columns: This section is currently empty, displaying the message "There are no items to show."

Columns: This section contains a table with the following data:

No	id	type	size	prop	sumtext	description
1	Name	STRING	256			
2	Address	STRING	256			
3	Company	STRING	256			
4	Department	STRING	256			
5	Salary	STRING	256			

Rows: This section contains a table with the following data:

No	Name	Address	Company	Department	Salary
1	John	Seoul, KOREA	1dollar	Sales	15000
2	James	Barcelona, Spain	EBOT	Consulting	25000
3	Donald	Califonia, USA	EBOT	Research Institute	1000000
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000
5	Joe	São Paulo, Brazil	hangul	R&D	8600
6	Michael	Mumbai, India	1dollar	Personnel	1300
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000

At the bottom of the window, there is a tab bar with "Dataset" and "Source" tabs. The "Dataset" tab is currently selected.

3 Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

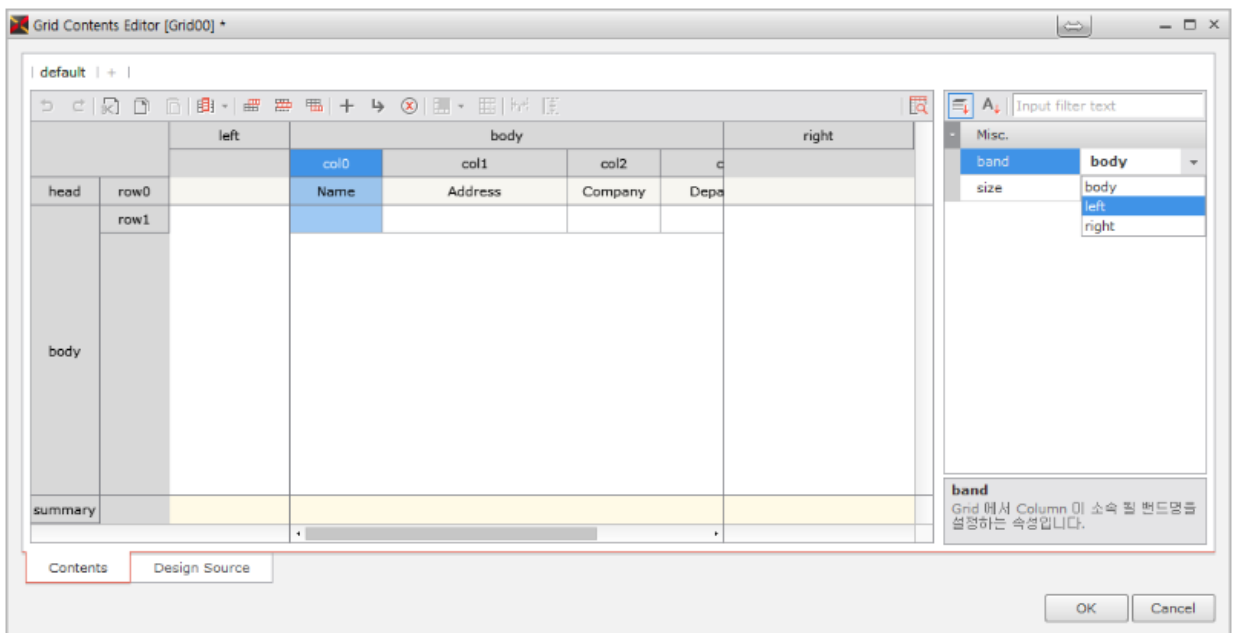
Invisible Objects 영역에 있는 Dataset을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료

됩니다.

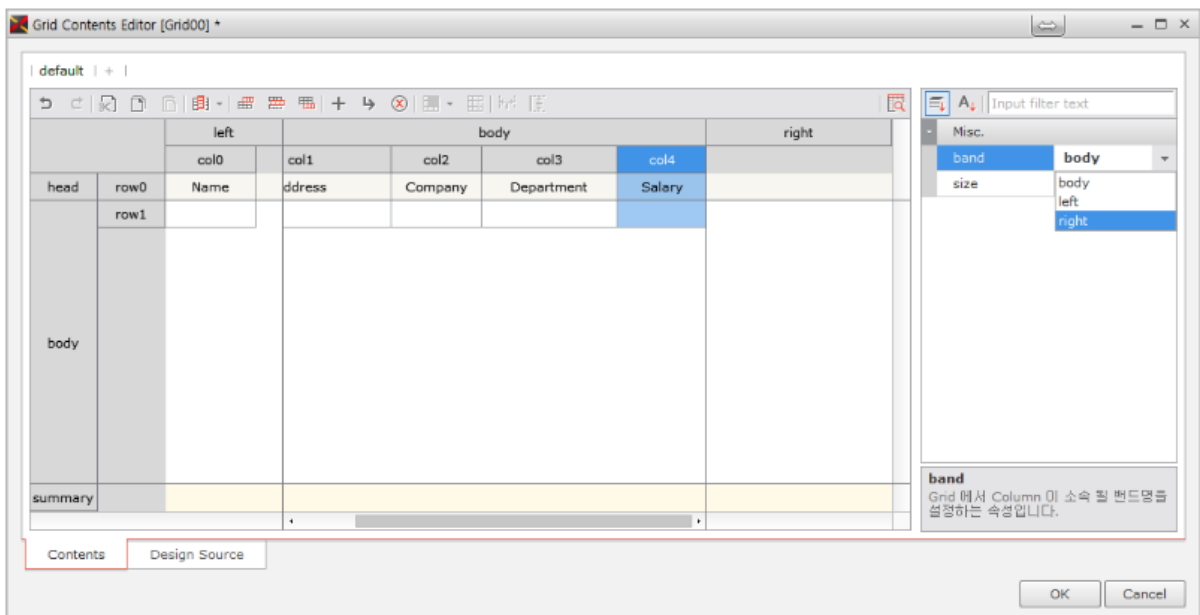
4. 틀 고정할 Column 설정하기

Body 밴드에 속해 있는 Name Column을 left 밴드로, Salary Column을 right 밴드로 설정합니다.

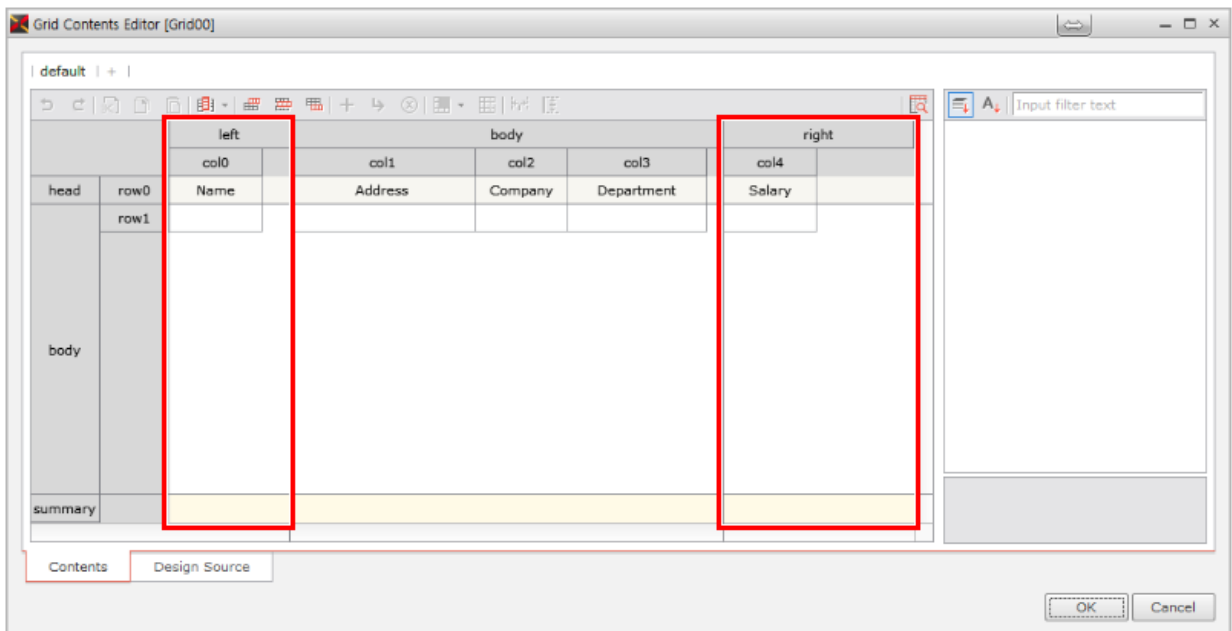
1. Grid를 더블클릭하여 Grid Contents Editor를 엽니다.
2. body 밴드에서 Name Column이 속해 있는 col0을 선택한 후 오른쪽 Properties 창에서 band 속성을 left로 설정합니다.



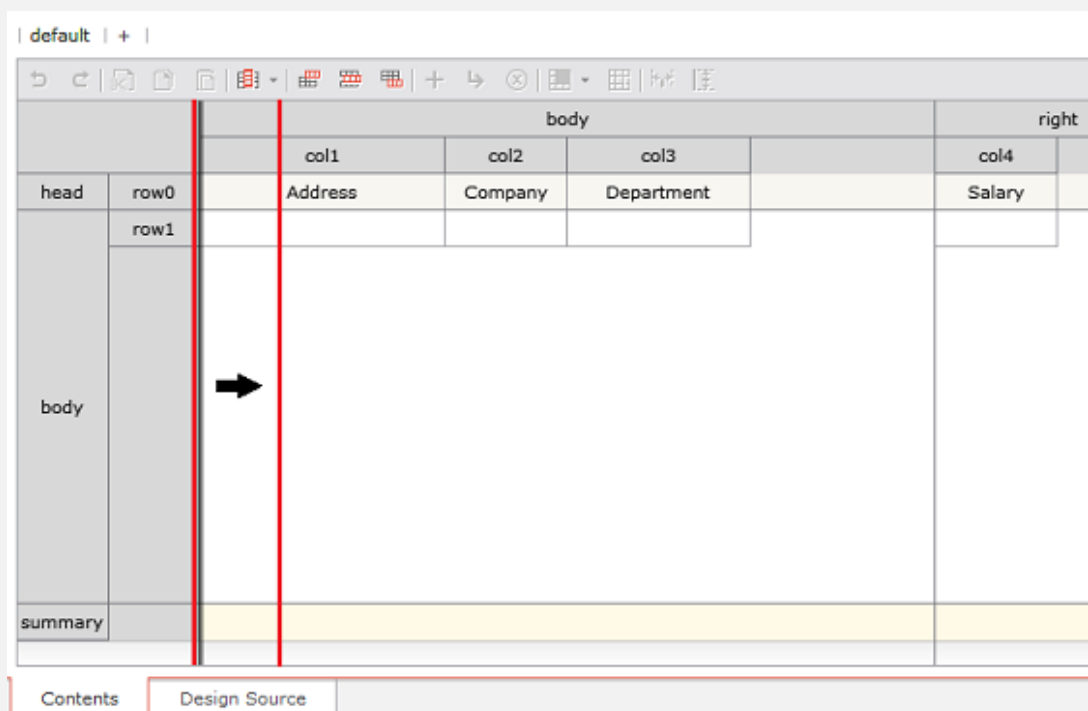
3. body 밴드에서 Salary Column이 속한 col4를 선택한 후 오른쪽 Properties 창에서 band 속성을 right로 설정합니다.



4. 아래 그림의 빨간색 박스와 같이 Name, Salary Column이 각각 left, right 밴드로 변경되었는지 확인한 후 OK 버튼을 눌러 설정을 완료합니다.



Grid Contents Editor에서 body 밴드의 Column을 left 혹은 right 밴드로 설정했을 때 화면상에서 옮겨진 결과가 보이지 않을 수 있습니다. 이런 경우에는 left, body, right 밴드의 인접 라인을 아래와 같이 드래그하면 left, right 밴드를 확인할 수 있습니다.



5

QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Grid의 스크롤바를 움직여 왼쪽의 Name Column과 오른쪽의 Salary Column이 고정되어 움직이지 않는지 확인합니다.

23.7 틀 고정하기 - Row

Grid의 크기는 데이터양에 비례하여 늘어나기 때문에 제한된 크기의 화면에서는 모든 데이터를 한 번에 보기 힘듭니다. 특히 데이터를 서로 비교하며 봐야 할 경우에 스크롤바를 이동하며 보는 것은 여간 불편한 일이 아닙니다. 이런 경우 계속 화면상에 놓고 봐야 할 데이터를 Grid의 한쪽에 고정해놓고 볼 수 있다면 Grid의 활용성을 더욱 높일 수 있습니다.

이런 기능을 틀 고정이라 합니다. 틀 고정은 Grid의 특정 Column이나 Row를 스크롤에 상관없이 한 자리에 고정해 볼 수 있는 기능입니다. 엑셀에서 자주 사용하는 틀 고정 기능과 유사합니다.

Grid는 별도로 틀 고정 기능을 지원하지 않지만, Grid를 구역에 따라 나누는 Band를 활용하여 틀 고정 기능을 구현할 수 있습니다. Row 틀 고정은 head나 summary 밴드에 row를 추가하는 것만으로 별도의 설정없이 자동으로 구현됩니다.

아래는 Grid의 밴드 구간을 표시한 그림입니다.

		left	body					right
			col0	col1	col2	col3	col4	
head	row0		Name	Address	Company	Department	Salary	
body	row1							
summary	row2					Average		
	row3					Max		
	row4					Min		

빨간색으로 표시된 부분이 head, summary 밴드로 이 밴드에 속한 Row는 스크롤에 상관없이 고정되어 움직이지 않습니다. head와 summary 밴드 모두 틀 고정은 가능하지만, left, right 밴드와는 틀 고정의 용도가 다릅니다.

head의 경우는 Column 이름을 표시하는 용도로만 사용하는 것이 좋습니다. head에 row를 추가하여 데이터를 연산한 결과나 문자열 등을 표시할 수도 있지만, head 영역에 보이기 때문에 보는 입장에서 혼란스러울 수 있습니다.

따라서 이 영역에서 Row 를 고정은 하지 않습니다.

summary의 경우는 일반적으로 합계, 평균 등과 같이 데이터를 연산한 값이나 고정 문자열 등을 보여주기 위한 용도로 사용됩니다.

결론적으로 하단 Row 를 고정은 Grid에 바인딩된 Row 데이터를 고정하는 용도가 아니라 Grid에 항상 보여줘야 하는 내용을 고정하는 용도로 사용됩니다. 참고로 head, summary의 Row에 Dataset을 바인딩시켜도 아무것도 표시하지 않습니다.

이번 장에서는 하단 Row를 틀 고정하는 방법에 대해 설명합니다.

23.7.1 예제

아래는 Summary 밴드에 Salary Column의 Average(평균값), Max(최고값), Min(최소값) Row를 추가하여 틀 고정한 Grid 예제입니다.

빨간색 박스로 표시된 Average, Max, Min Row는 summary 밴드로 추가하였기 때문에 스크롤바를 움직이면 가운데 위치한 body 밴드에 속한 Row만 이동됩니다.

Name	Address	Company	Department	Salary	
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000	
Joe	São Paulo, Brazil	hangul	R&D	₩8,600	
Michael	Mumbai, India	1dollar	Personnel	₩1,300	
Lilli	Ankara, Turkey	hangul	Quality Assurance	₩4,400	
Colt	Sydney, NSW, Australia	hangul	Sales	₩62,000	
			Average	₩139,912	
			Max	₩1,000,000	
			Min	₩1,300	



sample_grid_18.xfdl

23.7.2 예제에서 사용한 핵심 기능

band

Grid에서 Row가 소속된 밴드 명을 갖는 읽기전용 속성입니다. head, summary 밴드는 각각 Grid의 위쪽, 아래쪽 고정 영역에 표시되며 스크롤 시 고정되어 움직이지 않습니다.

Row의 band 속성 값을 얻으려면 getFormatRowProperty 메소드를 사용합니다.

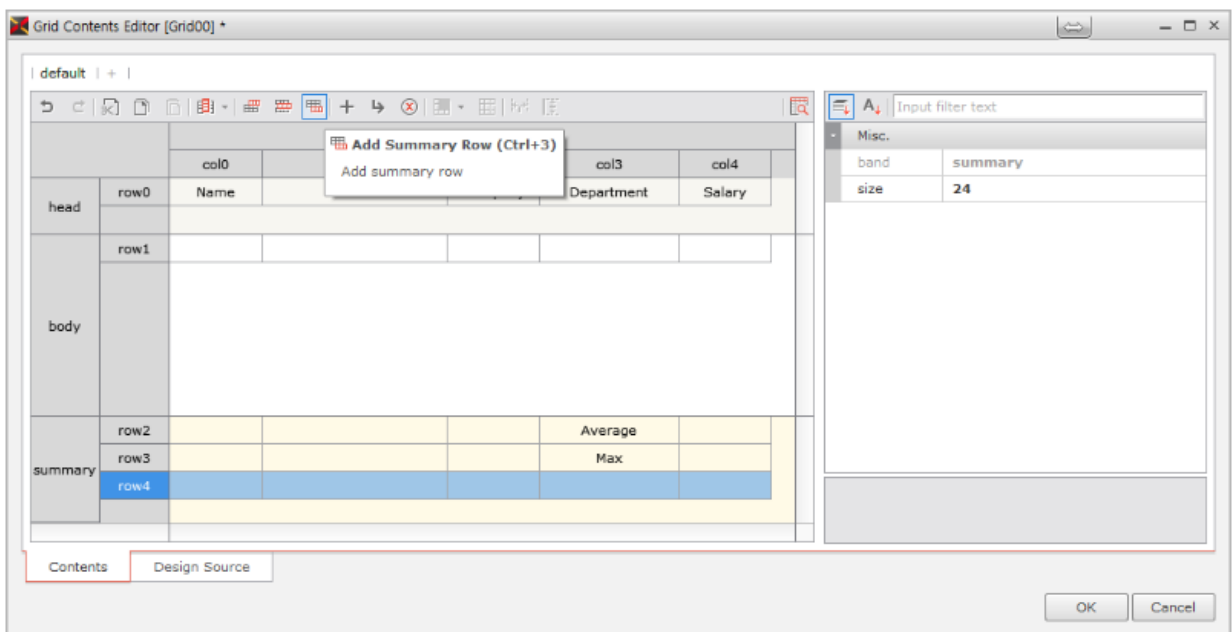
3 Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

4 summary 밴드에 Row 추가하기

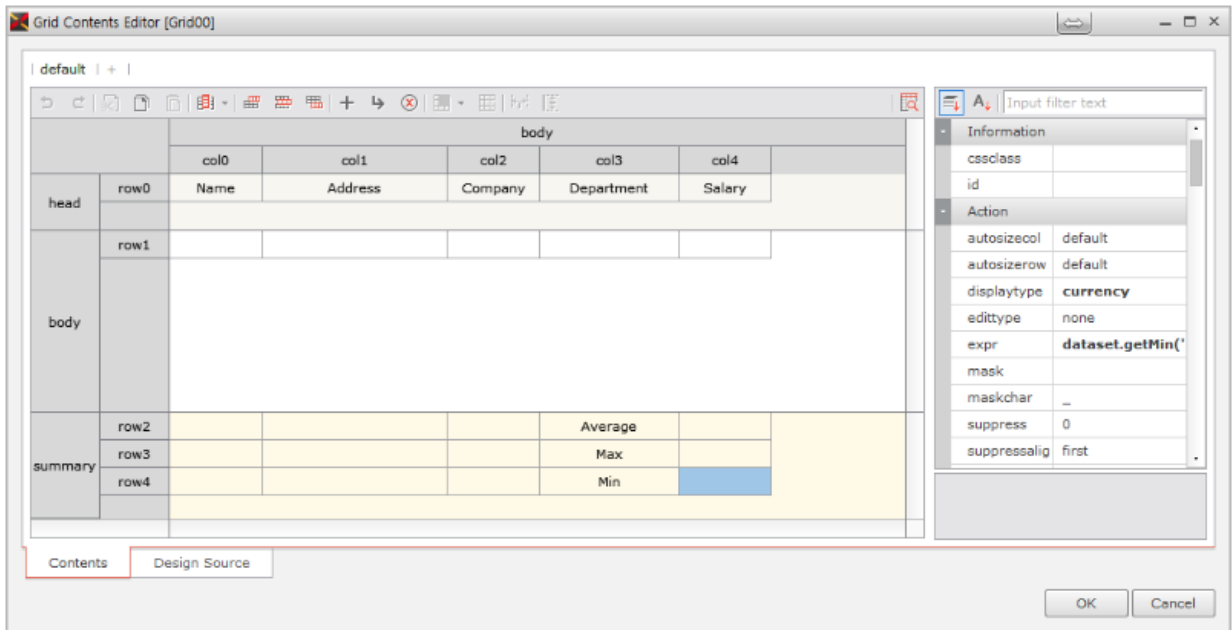
1. Grid를 더블클릭하여 Grid Contents를 연 후 아래 그림과 같이 Summary 밴드에 Average, Max, Min Row를 추가합니다. Grid Contents Editor의 메뉴중  버튼(Add Summary Row)을 누르면 Summary 밴드에 Row가 추가됩니다.



2. Average, Max, Min Row의 설정

각 Cell의 속성을 아래 표와 같이 설정합니다.

Column	Row	속성	설정값
col3	row2	text	Average
col3	row3	text	Max
col3	row4	text	Min
col4	row2	expr	dataset.getAvg('parseInt(Salary)')
		displaytype	currency
col4	row3	expr	dataset.getMax('parseInt(Salary)')
		displaytype	currency
col4	row4	expr	dataset.getMin('parseInt(Salary)')
		displaytype	currency



설정이 모두 완료되면 Grid Contents Editor의 OK 버튼을 누릅니다.

5 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Grid의 스크롤바를 위, 아래로 움직여 Average, Max, Min Row가 고정되어 움직이지 않는지 확인합니다.

23.8 Column 위치 이동하기

Grid의 cellmovingtype 속성을 설정하면 사용자가 마우스 드래그 앤 드롭을 이용하여 Column의 위치를 변경할 수 있습니다.

23.8.1 예제

아래는 cellmovingtype 속성을 col1로 설정한 Grid 예제입니다. 사용자는 Column의 위치를 원하는 곳으로 이동시킬 수 있습니다.

Column의 위치 이동은 Column 헤더를 마우스로 드래그하여 원하는 위치에 있는 Column 헤더에 드롭합니다. 예를 들어 Department 헤더를 드래그하여 Address 헤더 위에서 드롭하면 그 위치에 Department Column이 위치하게 되고 Address Column은 한 칸 오른쪽으로 밀려납니다. Column이 새로운 위치로 이동하게 되면 그 위치와 그 오른쪽에 있던 Column들은 전부 오른쪽으로 한 칸씩 밀려납니다.



윈도우에서의 드래그 앤 드롭처럼 마우스로 드래그하는 동안에 Column이 이동하는 모습은 보이지 않습니다. 또한, 마우스 드래그 도중 마우스 커서가 Column 헤더 영역을 벗어나면 드래그 동작이 취소됩니다.

Salary	Company	Name	Address	Department
₩15,000	1dollar	John	Seoul, KOREA	Sales
₩25,000	EBOT	James	Barcelona, Spain	Consulting
₩1,000,000	EBOT	Donald	Califonia, USA	Research Institute
₩3,000	1dollar	Lisa	London, United Kingdom	Quality Assurance
₩8,600	hangul	Joe	São Paulo, Brazil	R&D
₩1,300	1dollar	Michael	Mumbai, India	Personnel
₩4,400	hangul	Lilli	Ankara, Turkey	Quality Assurance
₩62,000	hangul	Colt	Sydney, NSW, Australia	Sales



sample_grid_19.xfdl

23.8.2 예제에서 사용한 핵심 기능

cellmovingtype

Grid에서 Column의 Head 영역을 드래그하여 Column 순서를 변경할 수 있게 설정하는 속성입니다.

23.8.3 예제 구현 방법

① 화면 구성하기

툴바에서 Grid  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택한 후 Form 위에 놓고 그대로 클릭하면 기본 크기로 생성됩니다.

② Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.

Dataset Editor

Const Columns

No	id	type	size	value	
There are no items to show.					

Columns

No	id	type	size	prop	sumtext	description	
1	Name	STRING	256				
2	Address	STRING	256				
3	Company	STRING	256				
4	Department	STRING	256				
5	Salary	STRING	256				

Rows

No	Name	Address	Company	Department	Salary	
1	John	Seoul, KOREA	1dollar	Sales	15000	
2	James	Barcelona, Spain	EBOT	Consulting	25000	
3	Donald	Califonia, USA	EBOT	Research Institute	1000000	
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000	
5	Joe	São Paulo, Brazil	hangul	R&D	8600	
6	Michael	Mumbai, India	1dollar	Personnel	1300	
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400	
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000	

Dataset

|

Source

|

3 Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

4 cellmovingtype 속성 설정하기

Grid를 선택한 후 오른쪽 Properties 창에서 cellmovingtype 속성을 col로 설정합니다.

5 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. 위치를 변경하려는 Column의 헤더를 마우스로 드래그하여 원하는 위치에 드롭합니다.

23.9 변경된 Column 상태의 포맷 복사하기

Column 위치 이동하기 예제와 비슷한 내용이지만, 사용자가 변경한 상태 정보를 활용하는 예제입니다.

23.9.1 예제

마우스로 드래그해서 첫번째 Grid 컴포넌트의 컬럼 위치를 변경하고 [Change] 버튼 클릭 시 변경된 컬럼 위치가 두 번째 Grid 컴포넌트에 적용됩니다.

Department	Vacation	Name
Sales	1	John
IT	0	James
Education	0	Donald
Marketing	1	Lisa

Change

Name	Department	Vacation
John	Sales	1
James	IT	0
Donald	Education	0
Lisa	Marketing	1



sample_grid_43.xfdl

23.9.2 예제에서 사용한 핵심 기능

formats

넥사크로 스튜디오에서 Dataset 오브젝트를 Grid 컴포넌트에 바인딩했을 때 formats 속성이 자동으로 생성됩니다. Dataset 오브젝트의 정보를 가지고 자동으로 생성해주는 정보입니다. 직접 생성된 formats 정보를 지정할 수도 있습니다.

getCurFormatString

Grid 컴포넌트의 컬럼 위치를 변경한 경우 formats 속성값을 확인해보면 변경된 내용이 반영되어 있지 않습니다. 변경된 내용의 format 정보를 확인하려면 getCurFormatString 메소드를 사용해야 합니다.

23.9.3 예제 구현 방법

① 화면 구성하기

Grid 컴포넌트를 두 개 배치하고 Dataset 오브젝트를 추가합니다. Dataset 오브젝트의 데이터는 아래와 같이 지정합니다. 그리고 두 개의 Grid 컴포넌트에 같은 Dataset 오브젝트를 바인딩합니다.

Dataset Editor [Dataset00]					
▶ Const Columns					
▼ Columns					
No	id	type	size	prop	su
1	Name	STRING	256		
2	Department	STRING	256		
3	Vacation	STRING	256		
▼ Rows					
No	Name	Department	Vacation		
1	John	Sales	1		
2	James	IT	0		
3	Donald	Education	0		
4	Lisa	Marketing	1		

② Button 컴포넌트 추가하고 이벤트 함수 작성하기

Button 컴포넌트를 배치하고 onclick 이벤트 함수를 작성합니다. 첫번째 Grid 컴포넌트의 format 정보를 가져와서 두번째 Grid 컴포넌트의 formats 속성값으로 지정합니다. 이때 getCurFormatString 메소드에서 가져온 값은 <Formats>라는 태그를 가지고 있기 때문에 해당 태그를 추가해줍니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var sFormatContents = this.Grid00.getCurFormatString();
    this.Grid01.set_formats("<Formats> "+sFormatContents+" </Formats>");
};
```

③ QuickView로 확인하기

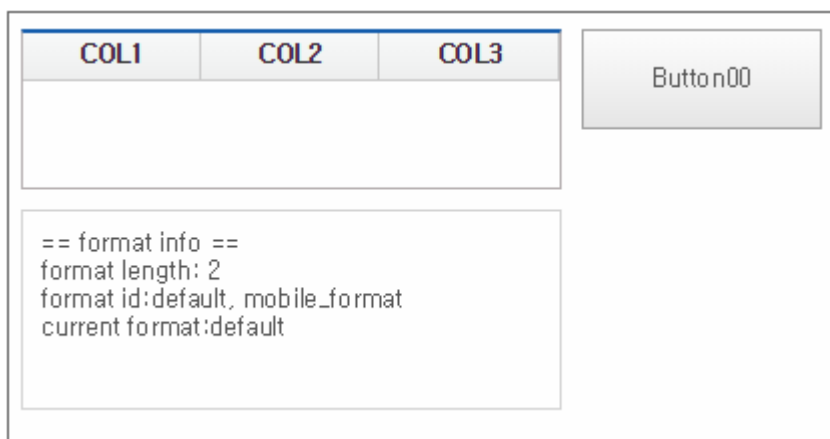
QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. 첫번째 Grid 컴포넌트의 컬럼 헤더를 마우스로 드래그해서 위치를 변경합니다. 그리고 버튼을 클릭하면 두번째 Grid 컴포넌트의 컬럼 위치가 변경된 것을 확인할 수 있습니다.

23.10 포맷 정보 확인하기

Grid 컴포넌트의 formats 속성값은 XML 형식으로 지정하도록 되어 있습니다. 넥사크로 스튜디오에서는 Grid Contents Editor를 실행해 어떤 formatid를 사용할 수 있는지 확인할 수 있지만 애플리케이션 실행 중에는 이를 확인할 수 없습니다. 예제에서는 간단하게 사용할 수 있는 포맷 정보를 확인하는 기능을 살펴봅니다.

23.10.1 예제

버튼을 클릭하면 Grid 컴포넌트가 가지고 있는 포맷 정보를 TextArea 컴포넌트에 표시합니다.



23.10.2 예제에서 사용한 핵심 기능

getFormatString

포맷 정보를 XML 형식의 문자열로 반환합니다. 반환된 문자열 자체가 XML이 아니기 때문에 DOMParser를 사용해 XML 문서로 변환한 후 관련 정보를 확인합니다.

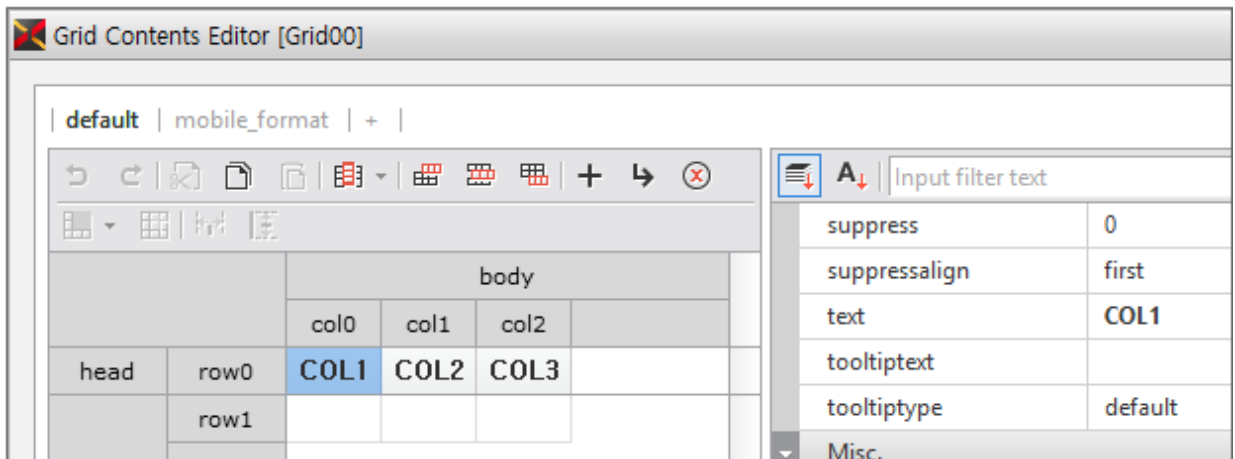
23.10.3 예제 구현 방법

1 화면 구성하기

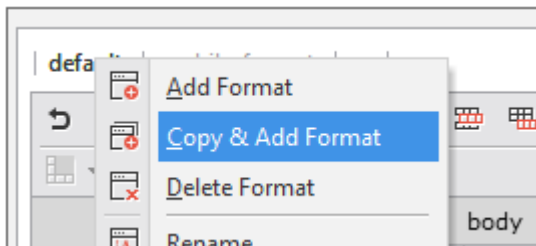
Grid 컴포넌트를 배치하고 Button 컴포넌트와 TextArea 컴포넌트를 배치합니다. 이번 예제에서는 바인딩 없이 포맷 정보만 사용하기 때문에 Dataset 오브젝트는 생성하지 않습니다.

2 Grid 컴포넌트의 format 설정하기

Grid 컴포넌트를 더블클릭해서 Grid Contents Editor를 실행합니다. [Add Head Row] 항목을 선택해 Head Row를 추가하고 [Add Body Column] 항목을 선택해 컬럼을 3개 추가합니다. 그리고 Head Row의 셀 영역을 선택하고 text 속성값을 지정해줍니다.



default 포맷 위에서 마우스 오른쪽 버튼을 클릭하면 표시되는 컨텍스트 메뉴에서 [Copy & Add Format] 항목을 선택합니다. 복사된 포맷 이름을 변경하고 세번째 컬럼을 삭제합니다.



3 Button 컴포넌트의 onclick 이벤트 함수 작성하기

Button 컴포넌트를 클릭했을때 Grid 컴포넌트의 formats 속성값으로 지정된 정보를 가져와서 XML 형태로 바꾸고(parseFromString) "Format" 태그에 해당하는 정보를 컬렉션 형태로 생성합니다. 해당 정보에서 포맷의 갯수나 id 정보를 확인합니다.


```

this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var parser, xmlDoc;
    parser = new DOMParser();
    xmlDoc = parser.parseFromString(this.Grid00.getFormatString(), "text/xml");
    var formatList = xmlDoc.getElementsByTagName('Format');
    var formatStr = "== format info =="\n"
        +"format length: "+formatList.length+"\n"
        +"format id:"+this.getFormatid(formatList, formatList.length)+"\n"
        +"current format:"+this.Grid00.formatid;
    this.TextArea00.set_value(formatStr);
};

```

포맷이 여러개인 경우 id 정보값을 표시하기 위해 함수를 하나 더 추가합니다.

```

this.getFormatid = function(arr, length)
{
    var str = "";
    for(var i=0;i<length;i++)
    {
        str += arr[i].getAttribute("id");
        if(i!=length-1) str += ", ";
    }
    return str;
}

```

④ QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 버튼을 클릭하면 포맷 정보가 TextArea 컴포넌트에 표시됩니다.

23.11 틀 고정하기 - 선택한 Column

화면 설계 시 틀 고정하기 기능을 사용할 수 있습니다. 하지만 사용자에게 틀 고정 위치를 지정하도록 허용해야 하는 경우도 있습니다. 이번 예제에서는 선택한 Column 기준으로 틀 고정하는 기능을 설명합니다.

23.11.1 예제

고정하기 원하는 컬럼 head 영역을 클릭하면 해당 컬럼이 고정된 상태로 변경됩니다.

Column0	Column1	Column2		Column4	
		Column2	Column3		

 [sample_grid_52.xfdl](#)

23.11.2 예제에서 사용한 핵심 기능

setFormatColProperty

컬럼 속성을 설정하는 메소드입니다. band, size 2가지 파라미터를 지정할 수 있는데, 예제에서는 band 속성을 변경합니다.

23.11.3 예제 구현 방법

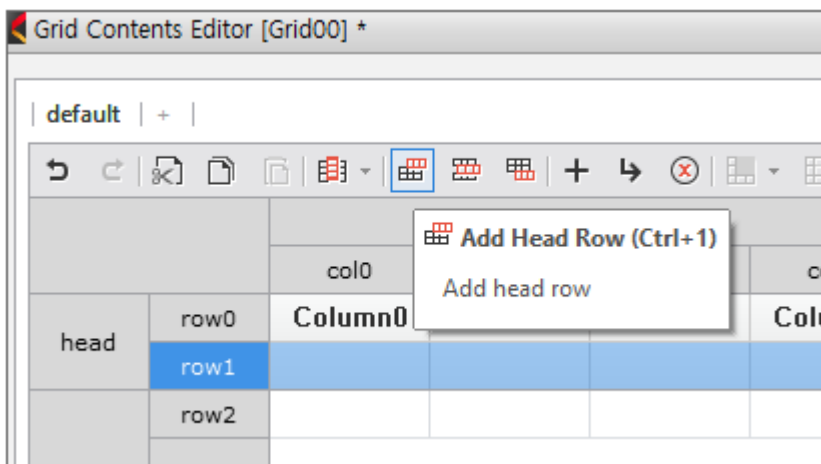
① 화면 구성하기

Grid 컴포넌트를 배치하고 Dataset 오브젝트를 생성합니다. 예제에서는 컬럼 고정 기능만 살펴볼 것이기 때문에 Dataset 오브젝트의 데이터는 지정하지 않습니다.

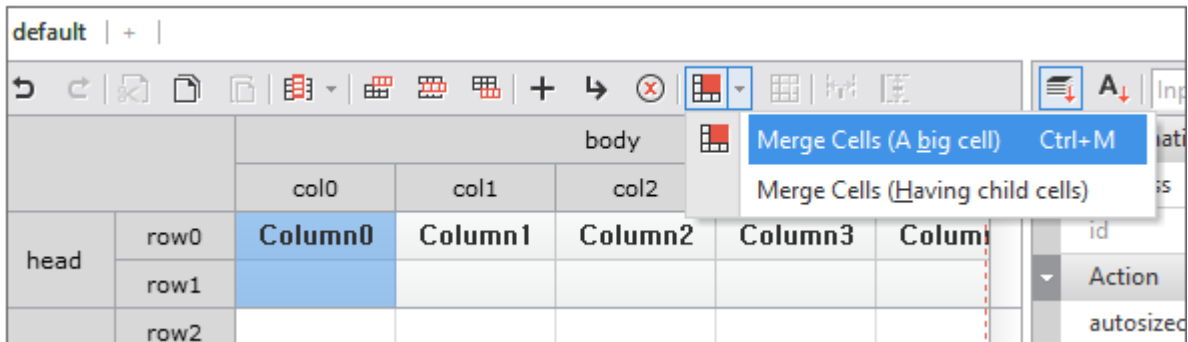
Dataset Editor [Dataset00]					
▶ Const Columns					
▼ Columns					
No	id	type	size	prop	sum
1	Column0	STRING	256		
2	Column1	STRING	256		
3	Column2	STRING	256		
4	Column3	STRING	256		
5	Column4	STRING	256		
6	Column5	STRING	256		
7	Column6	STRING	256		
8	Column7	STRING	256		
▼ Rows					
No	Column0	Column1	Column2		
1	[Undefined]	[Undefined]	[Undefined]		
2	[Undefined]	[Undefined]	[Undefined]		
3	[Undefined]	[Undefined]	[Undefined]		
4	[Undefined]	[Undefined]	[Undefined]		

2 Grid Content Editor 실행하기

Head Row를 하나 추가합니다.



그리고 아래와 같이 Head row를 합쳐줍니다.



아래와 같은 형식이 되도록 구성합니다. 순서는 상관없습니다. 2개 이상의 Head 영역을 합친 상태에서 동작하는 기능을 확인하기 위함입니다.

body							
col0	col1	col2	col3	col4	col5	col6	col7
Column0	Column1	Column2		Column4	Column5		
		Column2	Column3		Column5	Column6	Column7

3 onheadclick 이벤트 함수 작성

Grid 컴포넌트의 head 영역을 클릭했을때 틀 고정 기능이 동작하도록 합니다.

```

this.Grid00_onheadclick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    var iCnt=0;

    for (var j = e.cell; j >= 0; j--)
    {
        var iColCur = this.Grid00.getCellProperty("Head", j, "col");
        var iCell = e.cell;

        for (var i = iCell; i >= 0; i--)
        {
            if (this.Grid00.getCellProperty("Head", i, "row") == 0
                && iColCur == this.Grid00.getCellProperty("Head", i, "col"))
            {
                if (iCnt==0)
                {
                    var iCol = this.Grid00.getCellProperty("Head", i, "col");
                    var iSpan = this.Grid00.getCellProperty("Head", i, "colspan")-1;
                    var iTotCol = iCol + iSpan;
                }
            }
        }
    }
}

```

```

        for (var k = 0; k <= iTotCol; k++)
        {
            this.Grid00.setFormatColProperty(k, "band", "left");
        }
        for(var l = iTotCol;l<this.Grid00.getCellCount("head"); l++)
        {
            this.Grid00.setFormatColProperty(k, "band", "body");
        }
    }
    iCnt++;
}
}
};

```

④ QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 head 영역을 클릭하면서 틀 고정여부 확인합니다.

23.12 컬럼 크기 조정 기능 제한하기

컬럼 크기를 사용자가 직접 조정할 수 있게 허용할 수 있습니다. 하지만 너무 작게 조정하면 데이터를 보기 힘들 수 있다면 기능 일부를 제한할 수 있습니다.

23.12.1 예제

사용자가 컬럼 크기를 조정할 수 있지만, 너무 작게 조정하면 지정된 크기로 재조정됩니다.

Column0	Column1	Column2
←→		



23.12.2 예제에서 사용한 핵심 기능

oncolresized

컬럼 너비가 변경된 경우 발생하는 이벤트입니다. GridSizeChangedEventInfo 오브젝트를 통해 변경되기 전 크기값(oldvalue)을 확인하거나 변경된 크기값(newvalue)을 확인할 수 있습니다.

23.12.3 예제 구현 방법

① 화면 구성하기

Grid 컴포넌트를 배치하고 Dataset 오브젝트를 생성합니다. 예제에서는 컬럼 크기 변경 기능만 살펴볼 것이기 때문에 Dataset 오브젝트의 데이터는 지정하지 않습니다.

Columns				
No	id	type	size	prop
1	Column0	STRING	256	
2	Column1	STRING	256	
3	Column2	STRING	256	
Rows				
No	Column0	Column1	Column2	
1	[Undefined]	[Undefined]	[Undefined]	
2	[Undefined]	[Undefined]	[Undefined]	

② cellsizingtype 속성값 지정하기

컬럼 크기를 조정할 수 있도록 cellsizingtype 속성값을 "col"로 지정합니다.

cellsizingtype	col
enable	both
enableevent	col
extendsizetype	none
fastvscrolltype	row

③ oncolresized 이벤트 함수 작성하기

컬럼 크기 조정 시 지정된 크기보다 작은 경우 지정된 크기로 고정하는 기능을 추가합니다. 예제에서는 컬럼의 크기가 80으로 지정되어 있습니다.

```
this.Grid00_ocolresized = function(obj:nexacro.Grid,e:nexacro.GridSizeChangedEventInfo)
{
    this.Grid00.setFormatColProperty(e.formatindex,"size",e.newvalue);
    if( e.newvalue < 70 ){
        this.Grid00.setFormatColProperty(e.formatindex,"size",70);
        this.Grid00.formats = "<Formats>" + this.Grid00.getCurFormatString() + "</Formats>";
        obj.setFocus();
    }
};
```

④ QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 컬럼 크기를 작게 줄였을때 어떻게 변하는지 확인합니다.

23.13 병합된 Head 영역의 컬럼 타이틀 텍스트 구하기

Grid 컴포넌트에서 컬럼 타이틀(Head) 영역과 Body 영역이 같은 경우에는 컬럼 인덱스 정보로 해당 컬럼의 Head 텍스트를 확인할 수 있습니다. 하지만, Head 컬럼이 병합된 경우에는 인덱스가 달라지면서 원하는 정보를 바로 가져올 수 없습니다. 이런 경우 어떤 식으로 컬럼 타이틀을 확인하는지 알아보니다.

23.13.1 예제

Body 영역의 셀을 클릭하면 해당하는 컬럼 타이틀을 셀에 표시합니다.

A	B		C	
			D	E
A	B	B	D	



23.13.2 예제에서 사용한 핵심 기능

getCellCount

셀의 갯수를 반환합니다. 병합된 셀은 하나로 계산합니다.

getHeadValue

Head 밴드 영역에서 특정 셀의 text 속성값을 반환합니다. `getCellProperty("Head", i, "text")`와 같은 값을 반환합니다.

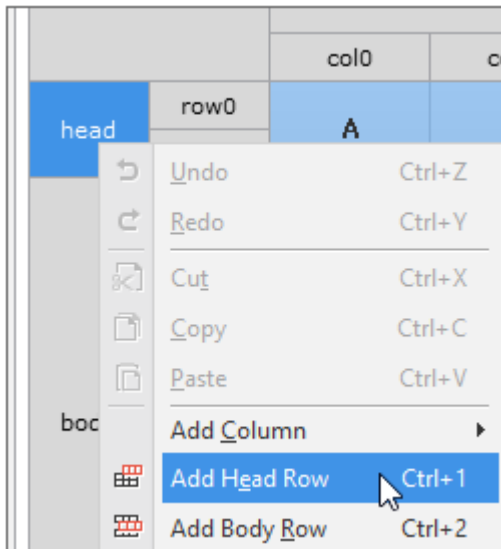
23.13.3 예제 구현 방법

- 1 Grid 컴포넌트를 배치하고 Dataset 오브젝트를 생성합니다. 예제에서는 컬럼 타이틀 정보를 어떻게 가져오는지만 살펴볼 것이기 때문에 Dataset 오브젝트의 데이터는 지정하지 않습니다.

Columns				
No	id	type	size	prop
1	Column0	STRING	256	
2	Column1	STRING	256	
3	Column2	STRING	256	
4	Column3	STRING	256	
5	Column4	STRING	256	

Rows			
No	Column0	Column1	Column2
1	[Undefined]	[Undefined]	[Undefined]

- 2 생성한 Dataset 오브젝트를 Grid 컴포넌트로 드래그앤드롭해서 바인딩 처리합니다.
- 3 Grid 컴포넌트를 더블클릭해서 Grid Contents Editor를 실행합니다.
- 4 head 영역을 선택하고 컨텍스트 메뉴에서 [Add Head Row]를 선택해 Row를 추가합니다.



- 5 아래 그림과 같이 Head 영역의 셀을 병합하고 Head 텍스트를 A부터 E까지 text 속성값을 수정합니다.

		body				
		col0	col1	col2	col3	col4
head	row0	A	B		C	
	row1				D	E
	row2					

- 6 Grid 컴포넌트의 onclick 이벤트 함수를 아래와 같이 작성합니다.

셀 클릭 시 셀의 인덱스 정보를 가지고 Head 영역의 컬럼 타이틀 텍스트를 확인하고 해당 텍스트를 셀의 text 속성값으로 설정합니다.

```
this.Grid00_onclick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    var str = this.fn_getHeadText(obj, e.cell);
    obj.setCellProperty("body", e.cell, "text", str);
};

this.fn_getHeadText = function(obj, nCell)
{
    for(j=nCell; j>=0; j--) {
        var nCellColIndex = obj.getCellProperty("Body", j, "col");
        var icell = obj.getCellCount("Head");

        for(i=icell; i>=0; i--) {
```

```

        if(obj.getCellProperty("Head", i, "row")==0 && nCellColIndex == obj.
getCellProperty("Head", i, "col")) {
            return obj.getHeadValue(i);
        } else if(obj.getCellProperty("Head", i, "row")==1 && nCellColIndex == obj.
getCellProperty("Head", i, "col")) {
            return obj.getHeadValue(i);
        }
    }
}
}

```

- ⑦ QuickView(Ctrl + F6)를 실행하여 Body 영역의 셀 클릭 시 컬럼 타이틀 텍스트가 표시되는지 확인합니다.

23.14 병합된 셀의 자식셀 속성 지정하기

넥사크로 스튜디오에서 병합된 셀의 자식셀을 선택하고 속성을 지정하는 방법을 살펴봅니다.

23.14.1 예제

병합된 셀에서 아래에 있는 자식셀의 color 속성값을 다르게 표시합니다. 해당 셀을 클릭하면 Edit 컴포넌트에 자식셀의 color 속성값을 표시합니다.

Column0	Column1
1 ONE	A a
2 TWO	B b
3 THREE	C c

lightcoral

23.14.2 예제에서 사용한 핵심 기능

getSubCellProperty

서브셀의 속성값을 반환합니다. 넥사크로 스튜디오에서 설정한 값을 확인할 수 있습니다.

23.14.3 예제 구현 방법

- 1 Grid 컴포넌트와 Edit 컴포넌트를 배치하고 Dataset 오브젝트를 생성합니다.

Columns					
No	id	type	size	prop	sumtext
1	Column0	STRING	256		
2	Column1	STRING	256		
3	Column2	STRING	256		
4	Column3	STRING	256		

Rows			
No	Column0	Column1	Column2
1	1	A	ONE
2	2	B	TWO
3	3	C	THREE

- 2 생성한 Dataset 오브젝트를 Grid 컴포넌트로 드래그앤드롭해서 바인딩 처리합니다.
- 3 Grid 컴포넌트를 더블클릭해서 Grid Contents Editor를 실행합니다.
- 4 3번째와 4번째 컬럼을 삭제합니다.

Body 영역을 선택하고 컨텍스트 메뉴에서 [Add Body Row]를 선택해 Row를 추가합니다.

		col0	col1	
head	row0	Column0	Column1	
body	row1			

Undo	Ctrl+Z
Redo	Ctrl+Y
Cut	Ctrl+X
Copy	Ctrl+C
Paste	Ctrl+V
Add Column	
Add Head Row	Ctrl+1
Add Body Row	Ctrl+2
Add Summary Row	Ctrl+3

- 5 두 번째 Row에 Column2, Column3 값을 text 속성값으로 설정합니다.

	body			
	col0	col1		
row0	Column0	Column1		
row1				
row2				

suppress	0
suppressalign	first
text	bind:Column2
tooltiptext	Column0
tooltiptype	Column1
Misc.	Column2
locale	Column3

- 6 자식셀을 유지한 상태로 첫 번째 Row와 두 번째 Row를 컬럼별로 병합합니다.

	body			
	col0	col1		
row0	Column0	Column1		
row1				
row2				

Merge Cells (A big cell) Ctrl+M
Merge Cells (Having child cells)
suppressalign
text
tooltiptext
tooltiptype
Misc.
locale

- 7 color 속성을 변경할 자식셀을 선택합니다. ALT키를 누른 상태에서 마우스로 해당 셀을 클릭하면 특정 자식셀만 선택할 수 있습니다.

		body		
		col0	col1	
head	row0	Column0	Column1	
	row1			
	row2			

- 8 속성창에서 원하는 값으로 color 속성값을 설정합니다.

- 9 Grid 컴포넌트의 onclickclick 이벤트 함수를 아래와 같이 작성합니다.

셀 클릭 시 셀의 인덱스 정보를 가지고 두 번째 자식셀의 color 속성값을 Edit 컴포넌트에 표시합니다.

```
this.Grid00_onclick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    this.Edit00.set_value(obj.getSubCellProperty("body", e.cell, 1, "color"))
};
```

- 10 QuickView(Ctrl + F6)를 실행하여 자식셀의 color 속성이 표현되는지 확인합니다.

24.

Grid 셀

24.1 포맷 변경해서 적용하기

Grid에서 데이터를 어떤 형태로 표현할지 정의한 형식을 포맷이라고 합니다. Grid를 사용하기 위해 꼭 필요한 것이 데이터와 포맷이며 두 가지 요소가 모두 있어야 데이터를 Grid에 표현할 수 있습니다.

Grid는 다중 포맷을 지원합니다. Grid에 여러 포맷을 정의해 놓고 필요에 따라 변경해가며 사용할 수 있습니다. 포맷은 개발 시 미리 정의해 놓고 사용할 수도 있고 애플리케이션 수행 시 생성하거나 변경할 수도 있습니다. 다중 포맷을 사용하면 사용자나 상황에 맞게 Grid의 모양을 변경할 수 있다는 장점이 있습니다.

Grid를 처음 생성하면 어떤 데이터가 어떤 형태로 표현될지 알 수 없기 때문에 포맷이 정의되어 있지 않습니다. 포맷이 만들어지려면 사용자가 직접 Grid Contents Editor를 사용해 만들어 주거나 Dataset을 바인딩 해주면 됩니다.

Grid에 Dataset을 바인딩하면 Dataset의 Column 정보를 기준으로 default라는 포맷이 자동 생성되기 때문에 다중 포맷을 사용하지 않는 경우에는 개발자가 별도로 포맷을 만들어 줄 필요가 없습니다.

24.1.1 예제

아래는 Grid에 default, format00이라는 두 개의 포맷을 만들어 놓고 Select format 콤보박스를 사용해 포맷을 전환하는 예제입니다.

사용자는 아래의 Select format 콤보박스를 사용하여 포맷을 전환하며 같은 데이터가 포맷에 따라 어떻게 보이는지 확인할 수 있습니다. 선택할 수 있는 포맷은 Dataset을 바인딩해 자동으로 생성된 default 포맷과 Grid Contents Editor를 사용해 별도로 생성한 format00 포맷이 있습니다.

Name	Address	Company	Department	Salary
John	Seoul, KOREA	1dollar	Sales	₩15,000
James	Barcelona, Spain	EBOT	Consulting	₩25,000
Donald	Califonia, USA	EBOT	Research Institute	₩1,000,000
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
Joe	São Paulo, Brazil	hangul	R&D	₩8,600
Michael	Mumbai, India	1dollar	Personnel	₩1,300
Lilli	Ankara, Turkey	hangul	Quality Assurance	₩4,400
Colt	Sydney, NSW, Australia	hangul	Sales	₩62,000

Select format:

default



default

format00



24.1.2 예제에서 사용한 핵심 기능

formatid

Grid에 정의된 포맷 중에 Grid에 적용할 포맷의 ID를 설정하는 속성입니다. Grid의 포맷 정보는 모두 format 속성에 문자열 형태로 정의되어 있습니다.

24.1.3 예제 구현 방법

① 화면 구성하기

툴바에서 Grid , Static , Combo  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택하고 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

② Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset을 더블 클

릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.

The screenshot shows the 'Dataset Editor' window with two main sections: 'Const Columns' and 'Columns'.

Const Columns:

No	id	type	size	value
There are no items to show.				

Columns:

No	id	type	size	prop	sumtext	description
1	Name	STRING	256			
2	Address	STRING	256			
3	Company	STRING	256			
4	Department	STRING	256			
5	Salary	STRING	256			

Rows:

No	Name	Address	Company	Department	Salary
1	John	Seoul, KOREA	1dollar	Sales	15000
2	James	Barcelona, Spain	EBOT	Consulting	25000
3	Donald	Califonia, USA	EBOT	Research Institute	1000000
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000
5	Joe	São Paulo, Brazil	hangul	R&D	8600
6	Michael	Mumbai, India	1dollar	Personnel	1300
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000

At the bottom, there are tabs for 'Dataset' and 'Source'.

3 Dataset 바인딩

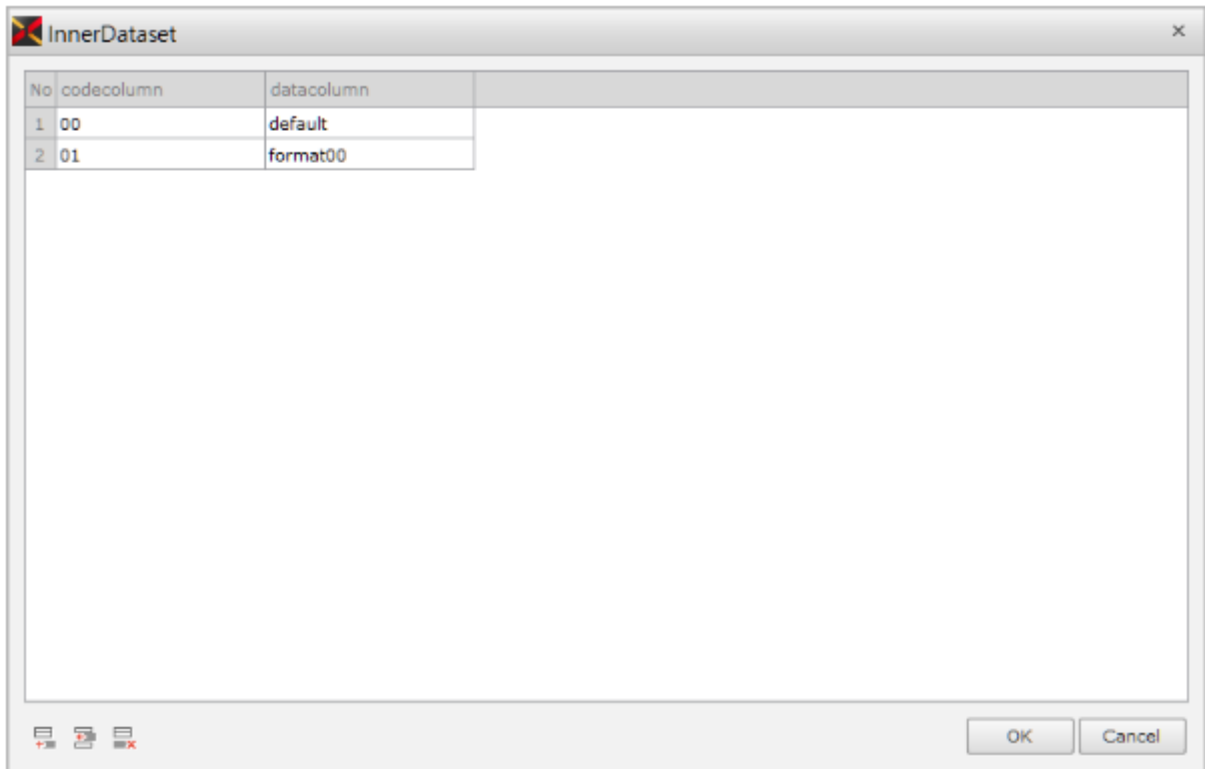
Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

4 Select format 설정을 위한 Combo 메뉴 만들기

1. Combo 메뉴 만들기

단계 1에서 생성한 Combo를 선택한 후 Properties 창에서 innerdataset 속성을 찾습니다. innerdataset 속성 옆에 ... 버튼을 클릭하여 InnerDataset Editor 창이 뜨면 아래와 같이 입력합니다. 입력이 완료되면 OK 버튼을 눌러 바인딩을 완료합니다.



2. Combo에 이벤트 추가하기

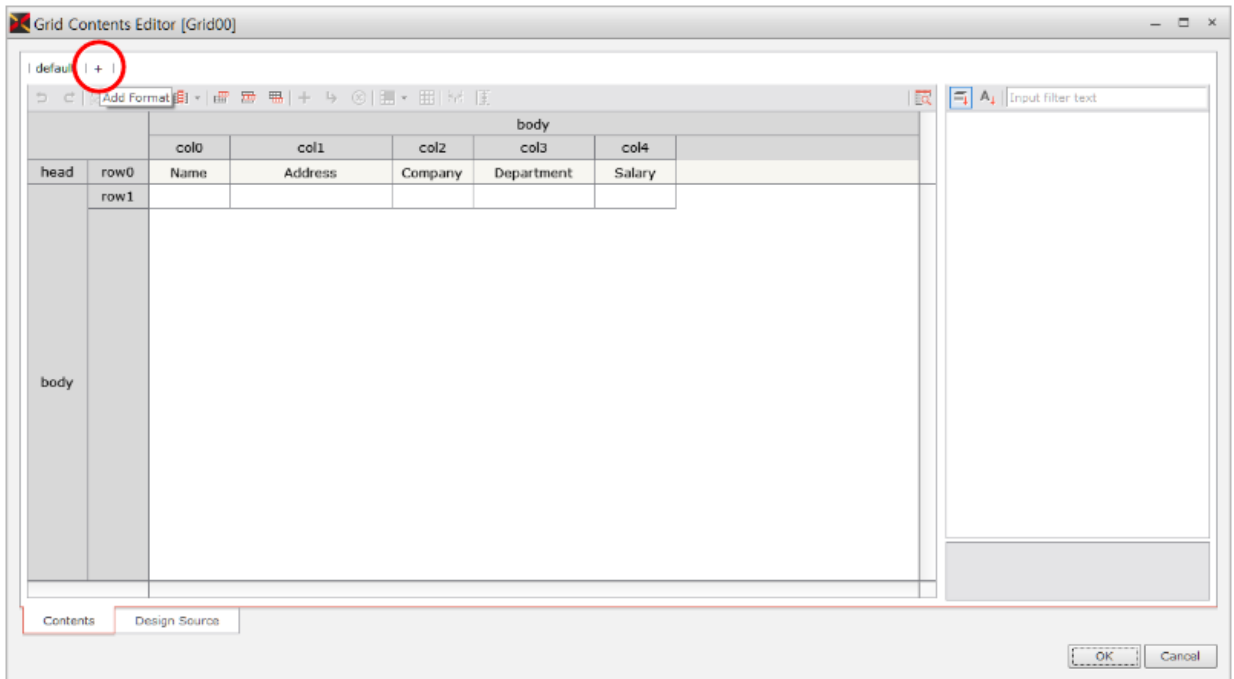
Form에 있는 Combo를 선택한 후 onitemchanged 이벤트 핸들러를 아래와 같이 추가합니다.

```
this.Combo00_onitemchanged = function(obj:nexacro.Combo,e:nexacro.ItemChangeEventInfo)
{
    var strFormat = e.posttext;

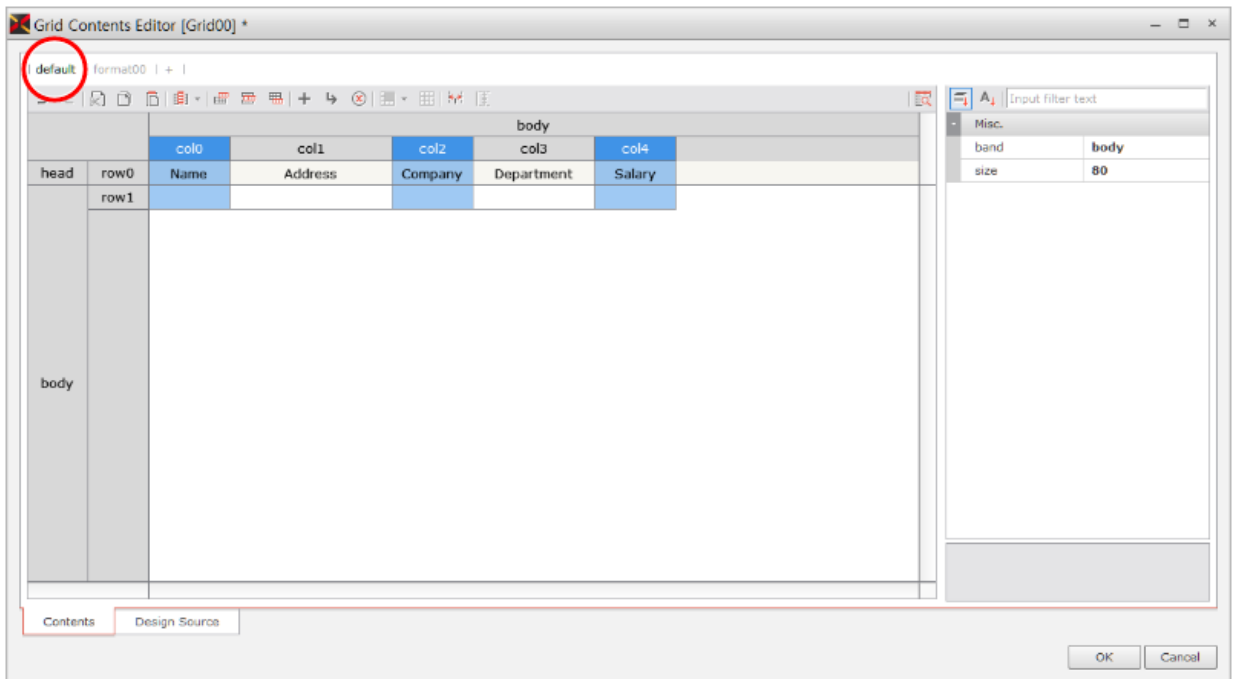
    this.Grid00.set_formatid(strFormat);
};
```

5 Grid에 포맷 생성하기

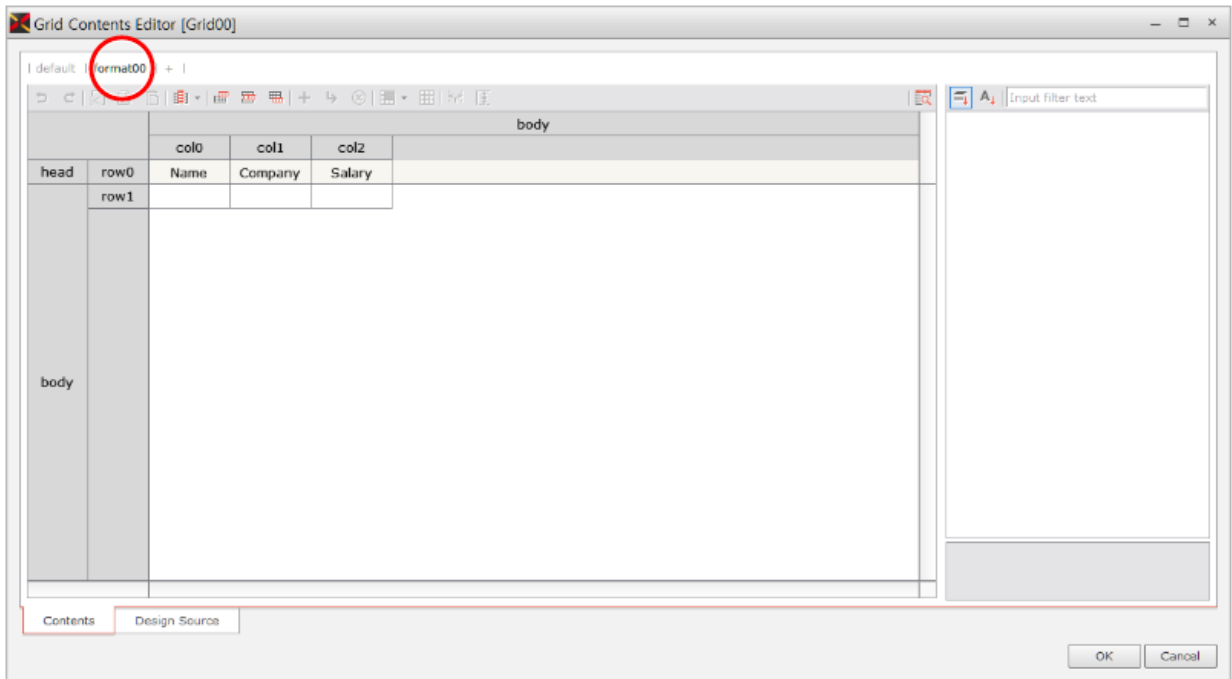
1. Grid를 선택한 후 더블클릭하여 Grid Contents Editor를 띄웁니다.
2. Grid Contents Editor 왼쪽 상단에서 + 버튼을 누르면 format00이라는 빈 포맷이 생성됩니다.



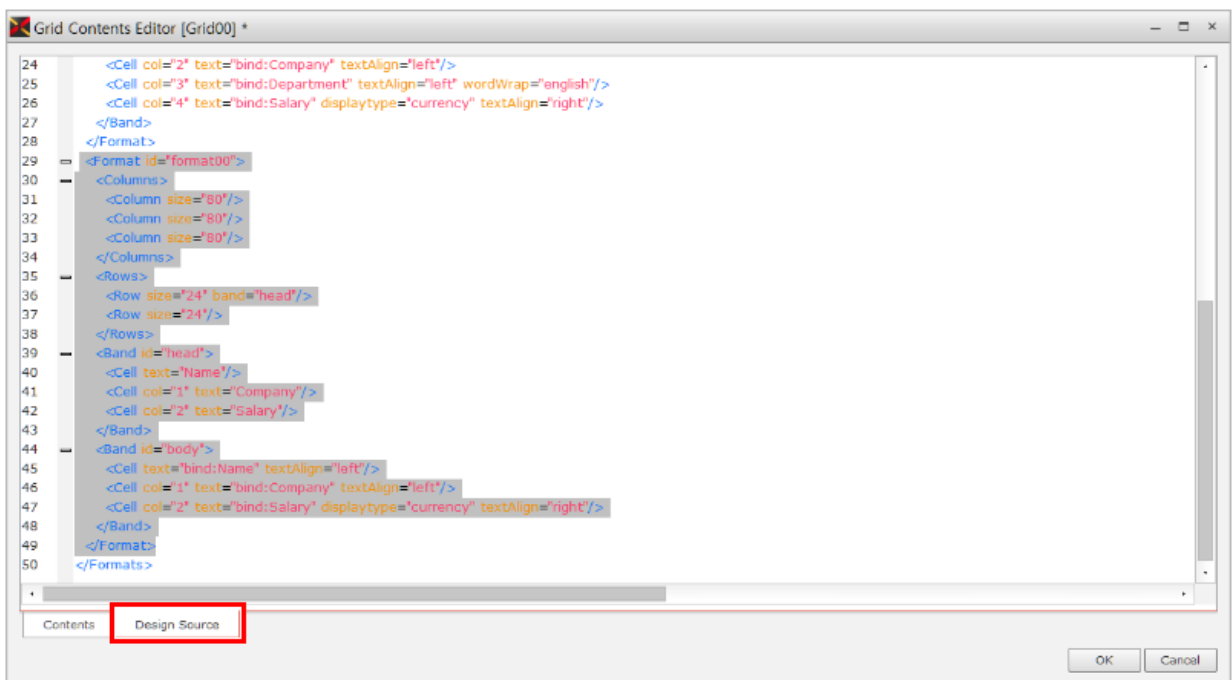
3. format00 포맷을 설정합니다. 예제에서는 편의를 위해 default 포맷에 있는 Column을 아래와 같이 선택한 후 **Ctrl** + **C**로 복사합니다.



4. default 포맷에서 복사한 Column을 아래 그림과 같이 format00 포맷에 **Ctrl** + **V**로 붙여넣습니다.



5. format00 포맷이 정상적으로 생성되었는지 확인하기 위해 Grid Contents Editor에서 아래와 같이 Design Source 탭을 눌러 소스에 아래와 같이 format00 정보가 추가되었는지 확인합니다.



6. format00 포맷이 정상적으로 생성된 것을 확인한 후 Grid Contents Editor의 OK 버튼을 누릅니다.

6 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Select format 콤보박스에서 default, format00을 번갈아 선택해 보며 Grid가 어떻게 변하는지 확인합니다.

24.2 바인딩된 Dataset 오브젝트의 포맷을 생성하여 보여주기

Grid에서 데이터를 어떤 형태로 표현할지 정의한 형식을 포맷이라고 합니다. Grid를 사용하기 위해 꼭 필요한 것이 데이터와 포맷이며 두 가지 요소가 모두 있어야 데이터를 Grid에 표현할 수 있습니다.

Grid의 포맷은 바인딩할 Dataset이 하나뿐이고 추후 다른 Dataset을 바인딩하지 않는다면 개발 시에 생성합니다. 그러나 애플리케이션 수행 중에 다른 Dataset을 바인딩해야 한다면 Grid의 포맷을 생성하는 별도의 과정이 필요합니다. 그 과정에 사용되는 메소드가 createFormat입니다.

createFormat은 Grid에 바인딩한 Dataset의 Column 정보를 기준으로 새로운 포맷을 생성하는 메소드입니다. 애플리케이션 수행 중에 Dataset을 바인딩한 후 createFormat 메소드를 수행하지 않는다면 Grid는 데이터를 어떤 형식으로 출력할지 알 수 없으므로 정상적으로 동작하지 않습니다. 따라서 Dataset의 바인딩 후에는 반드시 createFormat을 수행해줘야 합니다.

동적으로 Grid의 포맷을 생성하는 기능은 하나의 Grid에 여러 Dataset을 상황에 따라 바인딩하는 경우에 매우 유용합니다.

24.2.1 예제

다음은 Select dataset 콤보박스를 사용해 Grid에 Dataset00, Dataset01이라는 두 개의 Dataset을 바인딩하는 예제입니다.

Select dataset 콤보박스에서 데이터셋을 선택하면 Grid에 해당 Dataset이 바인딩됩니다. 그리고 createFormat 메소드가 수행되어 각 Dataset의 Column 구조에 맞게 Grid의 포맷이 생성됩니다. createFormat 메소드 수행 후 생성된 포맷은 Grid에 즉시 반영됩니다.

Name	Address	Company	Department	Salary
John	Seoul, KOREA	1dollar	Sales	15000
James	Barcelona, Spain	EBOT	Consulting	25000
Donald	Califonia, USA	EBOT	Research Institute	1000000
Lisa	London, United K	1dollar	Quality Assurance	3000
Joe	São Paulo, Brazil	hangul	R&D	8600
Michael	Mumbai, India	1dollar	Personnel	1300
Lilli	Ankara, Turkey	hangul	Quality Assurance	4400
Colt	Sydney, NSW, Au	hangul	Sales	62000

Select Dataset: Dataset00 

Dataset00
Dataset01



다음은 Dataset00과 Dataset01의 Column 구조를 보여줍니다. 각기 다른 데이터와 Column 구조로 되어 있어 같은 포맷을 공유해 사용할 수 없습니다.

No	Name	Address	Company	Department	Salary
1	John	Seoul, KOREA	1dollar	Sales	15000
2	James	Barcelona, Spain	EBOT	Consulting	25000
3	Donald	Califonia, USA	EBOT	Research Institute	1000000
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000
5	Joe	São Paulo, Brazil	hangul	R&D	8600
6	Michael	Mumbai, India	1dollar	Personnel	1300
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000

No	CODE	DATA
1	00	apple
2	01	banana
3	02	orange
4	03	kiwi
5	04	peach
6	05	strawberry

24.2.2 예제에서 사용한 핵심 기능

createFormat

Grid에 바인딩한 Dataset의 Column 정보를 기준으로 새로운 포맷을 생성하는 메소드입니다.

binddataset

Grid에 바인딩할 Dataset의 ID를 설정하는 속성입니다.

24.2.3 예제 구현 방법

① 화면 구성하기

툴바에서 Grid , Static , Combo  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택하고 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

2 Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다. 두 개의 Dataset을 생성해야 하므로 같은 작업을 한 번 더 반복합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.

- Dataset00의 아이템 목록 작성

Dataset Editor

Const Columns

No	id	type	size	value
There are no items to show.				

Columns

No	id	type	size	prop	sumtext	description
1	Name	STRING	256			
2	Address	STRING	256			
3	Company	STRING	256			
4	Department	STRING	256			
5	Salary	STRING	256			

Rows

No	Name	Address	Company	Department	Salary
1	John	Seoul, KOREA	1dollar	Sales	15000
2	James	Barcelona, Spain	EBOT	Consulting	25000
3	Donald	Califonia, USA	EBOT	Research Institute	1000000
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000
5	Joe	São Paulo, Brazil	hangul	R&D	8600
6	Michael	Mumbai, India	1dollar	Personnel	1300
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000

Dataset | Source |

- Dataset01의 아이템 목록 작성

Dataset Editor

Const Columns

No	id	type	size	value
There are no items to show.				

Columns

No	id	type	size	prop	sumtext	description
1	CODE	STRING	256			
2	DATA	STRING	256			

Rows

No	CODE	DATA
1	00	apple
2	01	banana
3	02	orange
4	03	kiwi
5	04	peach
6	05	strawberry

Dataset | Source |

3 Dataset00 바인딩

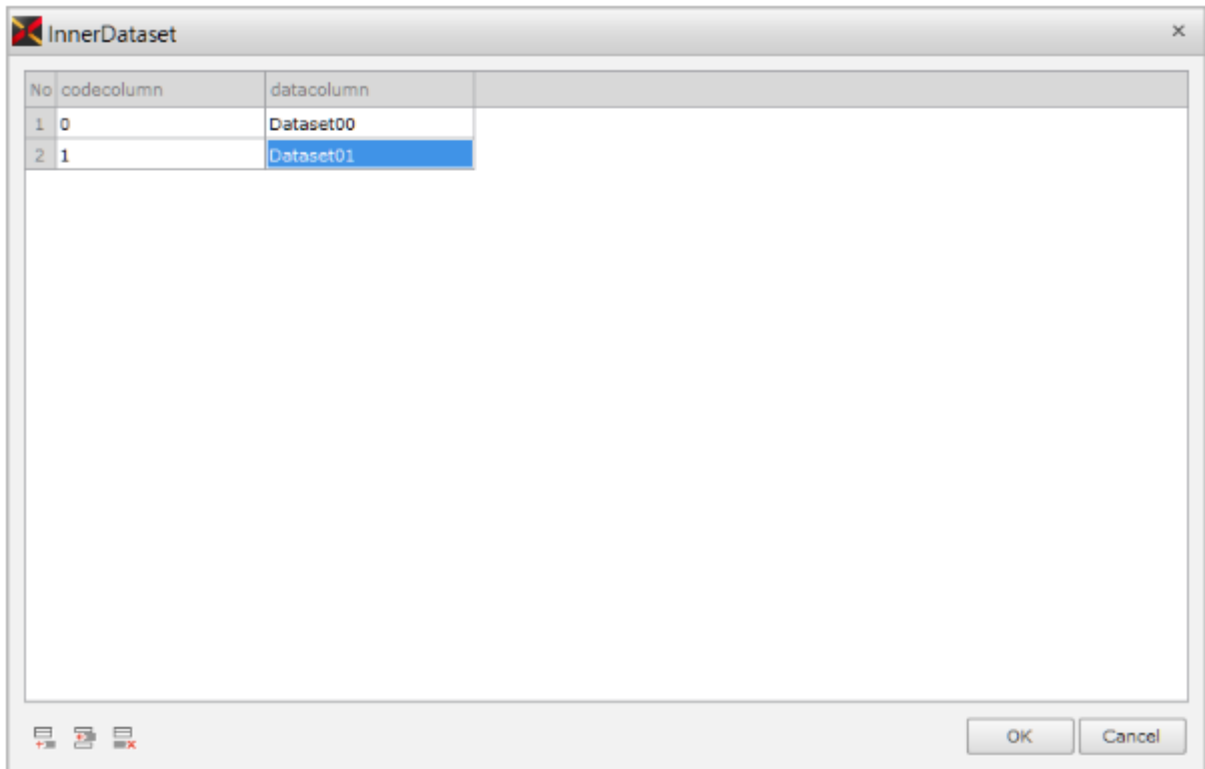
Dataset00을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

4 Select dataset 설정을 위한 Combo 메뉴 만들기

1. Combo 메뉴 만들기

단계 1에서 생성한 Combo를 선택한 후 Properties 창에서 innerdataset 속성을 찾습니다. innerdataset 속성 옆에 ... 버튼을 클릭하여 InnerDataset Editor 창이 뜨면 아래와 같이 입력합니다. 입력이 완료되면 OK 버튼을 눌러 바인딩을 완료합니다.



2. Combo에 이벤트 추가하기

Form에 있는 Combo를 선택한 후 onitemchanged 이벤트 핸들러를 아래와 같이 추가합니다.

```
this.Combo00_onitemchanged = function(obj:nexacro.Combo,e:nexacro.ItemChangeEventInfo)
{
    //Binding selected dataset
    this.Grid00.set_binddataset(e.posttext);

    //Creating new format for bound dataset
    this.Grid00.createFormat();
};
```

5 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Select dataset 콤보박스에서 dataset00, dataset01을 번갈아 선택하며 Grid가 정상적으로 출력되는지 확인합니다.

24.3 다양한 형식으로 데이터 표현하기

Grid에서 Cell의 displaytype 속성을 설정하면 데이터를 다양한 형식으로 표현할 수 있습니다.

displaytype은 Cell의 내용을 화면에 표시하는 형식을 설정하는 속성으로 normal, currency, date, number, text 등과 같은 텍스트 형태를 비롯하여 combo, checkbox, button, tree 등과 같은 컴포넌트 형태로도 설정할 수 있습니다.

여러 타입 중 combo, date, tree 등의 컴포넌트 형태로 설정할 경우에는 별도로 Cell의 속성을 설정해야 합니다. Cell의 속성에 대한 설정은 Grid Contents Editor에서 가능하며 해당 Cell을 선택한 후 Properties 창의 GridCombo, GridDate, GridTree 카테고리에서 세부 설정이 가능합니다.

예를 들어, Combo 타입으로 설정하는 경우에는 Cell의 설정에서 콤보에 출력할 데이터셋을 바인딩하고, code와 data를 설정해줘야 합니다.

CellCombo	
comboautoselect	false
combobuttonsize	px px  
combocodecol	CODE
combodatacol	VALUE
combodataset	ds_combo
combodisplaynulltext	
combodisplaynulltype	none

Tree 타입으로 설정하는 경우에는 Cell의 설정에서 트리의 깊이를 나타내는 treelevel, 트리의 상태를 나타내는 treestate, 트리의 체크박스를 설정하는 treecheck 속성을 설정해야 합니다.

CellTreeItem	
treecheck	bind:checkbox
treelevel	bind:tree
treestartlevel	0
treestate	1

24.3.1 예제

다음은 Cell의 displaytype 속성을 설정한 Grid 예제입니다. 두 개의 Grid 중 위는 displaytype 속성을 설정한 Grid이고 아래는 아무 설정도 하지 않은 Grid입니다.

두 Grid를 비교해 보며 데이터가 Cell의 displaytype 속성 설정에 따라 어떻게 표현되는지 확인할 수 있습니다.

normal	none	bar	button	checkbox	combo	currency	date	image	number	text	tree
John		20	Profile	✓	Male	₩15,000	2016-01-05		15000	1dollar	0
James		40	Profile		Male	₩25,000	2010-08-26		25000	EBOT	1
Donald		80	Profile		Male	₩1,000,000	2012-03-21		1000000	EBOT	1
Lisa		100	Profile	✓	Female	₩3,000	2010-12-25		3000	1dollar	2
Joe		30	Profile	✓	Female	₩8,600	1919-03-01		8600	hangul	2
Michael		0	Profile		Male	₩1,300	2014-04-16		1300	1dollar	0
Lilli		50	Profile		Female	₩4,400	2020-01-01		4400	hangul	1
Colt		90	Profile	✓	Male	₩62,000	1991-12-21		62000	hangul	2

normal	none	bar	button	checkbox	combo	currency	date	image	number	text	tree
John	Korea	20	Profile	1	M	15000	20160105	Image::c	15000	1dollar	0
James	Spain	40	Profile	0	M	25000	20100826	Image::c	25000	EBOT	1
Donald	USA	80	Profile	0	M	1000000	20120321	Image::c	1000000	EBOT	1
Lisa	United	100	Profile	1	W	3000	20101225	Image::c	3000	1dollar	2
Joe	Brazil	30	Profile	1	W	8600	19190301	Image::c	8600	hangul	2
Michael	India	0	Profile	0	M	1300	20140416	Image::c	1300	1dollar	0
Lilli	Turkey	50	Profile	0	W	4400	20200101	Image::l	4400	hangul	1
Colt	Austra	90	Profile	1	M	62000	19911221	Image::l	62000	hangul	2

sample_grid_22.xfdl

24.3.2 예제에서 사용한 핵심 기능

displaytype

Grid의 Cell에서 데이터가 표시되는 형식을 설정하는 속성입니다. 아무 값도 설정하지 않으면 normal이 기본 값으로 설정됩니다.

24.3.3 예제 구현 방법

① 화면 구성하기

툴바에서 Grid 컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택하고 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

Grid 생성 작업을 두 번 반복하면 Grid00, Grid01이 생성됩니다. Grid00은 셀의 displaytype을 설정할 Grid이고 Grid01은 Grid00과 비교를 위한 아무 설정도 하지 않는 Grid입니다.

2 Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다. Grid와 Combo 용으로 두 개의 Dataset을 생성해야 하므로 같은 작업을 한 번 더 반복합니다. 그리고 Grid를 위한 Dataset의 id는 ds_data로 하고, Combo를 위한 Dataset의 id는 ds_combo로 설정합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.

- ds_data의 아이템 목록 작성

Dataset Editor

Const Columns

No	id	type	size	value								
There are no items to show.												

Columns

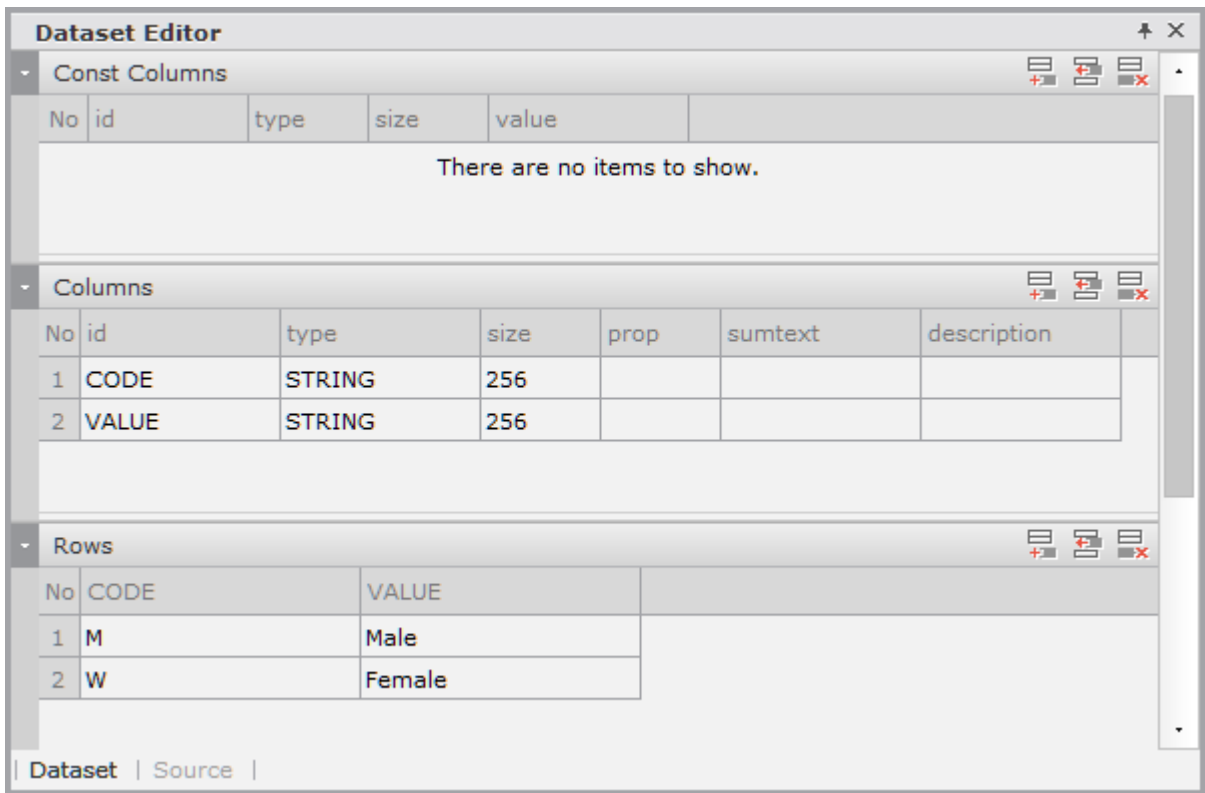
No	id	type	size	prop	sumtext	description						
1	normal	STRING	256									
2	none	STRING	256									
3	bar	STRING	256									
4	button	STRING	256									
5	checkbox	STRING	256									
6	combo	STRING	256									
7	currency	STRING	256									
8	date	STRING	256									
9	image	STRING	256									
10	number	STRING	256									
11	text	STRING	256									
12	tree	STRING	256									

Rows

No	normal	none	bar	button	checkbox	combo	currency	date	image	number	text	tree
1	John	Korea	20	Profile	1	M	15000	20160105	Image::component_Button.png	15000	1dollar	0
2	James	Spain	40	Profile	0	M	25000	20100826	Image::component_Grid.png	25000	EBOT	1
3	Donald	USA	80	Profile	0	M	1000000	20120321	Image::component_MaskEdit.png	1000000	EBOT	1
4	Lisa	United Kingdom	100	Profile	1	W	3000	20101225	Image::component_Static.png	3000	1dollar	2
5	Joe	Brazil	30	Profile	1	W	8600	19190301	Image::component_Tab.png	8600	hangul	2
6	Michael	India	0	Profile	0	M	1300	20140416	Image::icon_Play.png	1300	1dollar	0
7	Lilli	Turkey	50	Profile	0	W	4400	20200101	Image::launch_1.png	4400	hangul	1
8	Colt	Australia	90	Profile	1	M	62000	19911221	Image::launch_2.png	62000	hangul	2

Dataset | Source |

- ds_combo의 아이템 목록 작성

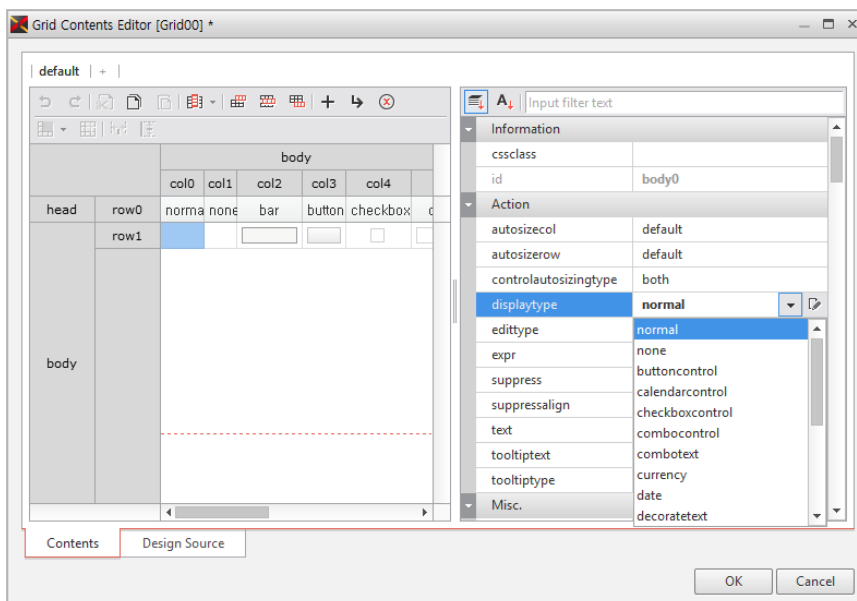


3 Grid에 Dataset 바인딩하기

ds_data를 Grid 컴포넌트에 바인딩합니다. Invisible Objects 영역에 있는 ds_data를 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다. Grid00, Grid01에 똑같이 ds_data를 바인딩시킵니다.

4 Cell 설정하기

1. Grid00을 더블클릭하여 Grid Contents Editor를 엽니다. 아래 그림과 같이 좌측의 col0부터 col11까지 Cell을 선택한 후 우측 Properties 창에서 displaytype을 설정합니다.

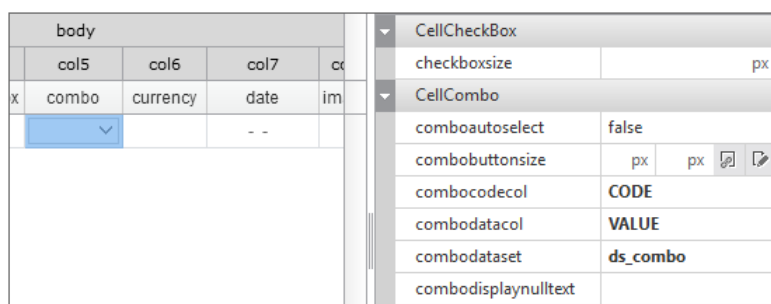


아래 표와 같이 각 Cell의 속성을 설정합니다. head 밴드의 Cell 설정은 헤드 컬럼에 표시될 타이틀을 설정하는 부분이고 body 밴드의 Cell 설정은 displaytype 속성을 설정합니다.

Band	Row	Column	Property	Value
head	row0	col0	text	normal
		col1		none
		col2		bar
		col3		button
		col4		checkbox
		col5		combo
		col6		currency
		col7		date
		col8		image
		col9		number
		col10		text
		col11		tree
body	row1	col0	displaytype	normal
		col1		none
		col2		progressbarcontrol
		col3		buttoncontrol
		col4		checkboxcontrol
		col5		combocontrol
		col6		currency
		col7		date
		col8		imagecontrol
		col9		number
		col10		text
		col11		treeitemcontrol

2. Grid Combo 설정하기

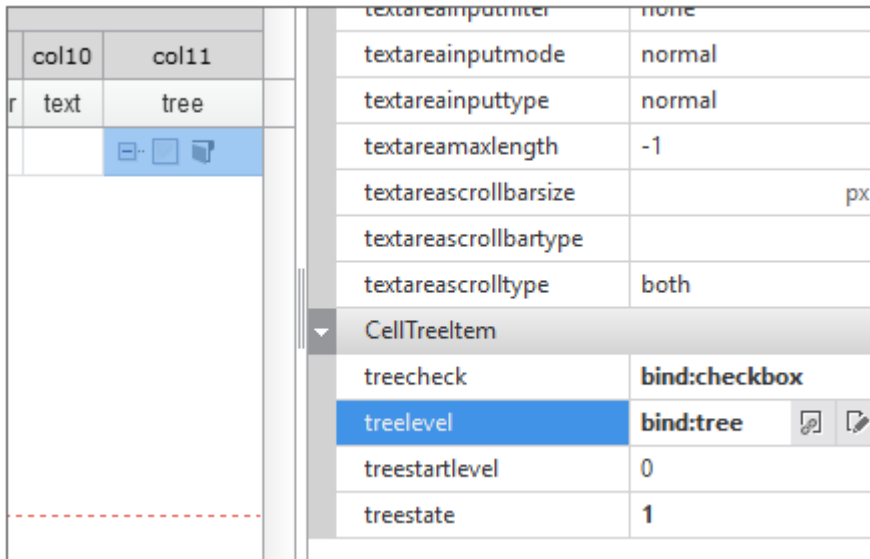
Combo로 설정할 Cell을 선택한 후 Properties 창에서 combodataset 속성을 찾습니다. combodataset 속성 옆에 ▼ 버튼을 클릭하여 ds_combo를 선택하여 바인딩합니다. 그리고 combocodecol, combodatacol 속성도 아래와 같이 설정합니다.



이 예제에서는 Cell의 속성에 대한 설정만을 다루기 때문에 Combo의 동작에 대한 구현은 다루지 않습니다.

3. Grid Tree 설정하기

Tree로 설정할 Cell을 선택한 후 Properties 창에서 GridTree 카테고리를 찾습니다. treecheck 속성 옆의  버튼을 클릭하여 checkbox를 선택하고, 마찬가지로 treelevel 속성 옆의  버튼을 클릭하여 tree를 선택합니다. treecheck는 트리의 체크박스 값으로 사용할 Column을 바인딩하는 속성이고, treelevel은 트리의 레벨 값으로 사용할 Column을 바인딩하는 속성입니다.



5 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

위의 Grid와 아래 Grid의 수행 결과를 비교해 보며 데이터가 Grid의 Cell 타입에 따라 어떻게 표현되는지 확인합니다.

24.4 다양한 형식으로 데이터 편집하기

Grid에서 Cell의 edittype 속성을 설정하면 데이터를 다양한 형식으로 편집할 수 있습니다.

edittype은 Cell에 어떤 형식으로 입력할지 설정하는 속성으로 normal, date, mask, text, textarea 등과 같은 텍스트 형태를 비롯하여 combo, checkbox, tree 등과 같은 컴포넌트 형태로도 설정할 수 있습니다.

여러 타입 중 combo, date, tree 등의 컴포넌트 형태로 설정할 경우에는 별도로 Cell의 속성을 설정해야 합니다. Cell의 속성에 대한 설정은 Grid Contents Editor에서 가능하며 해당 Cell을 선택한 후 Properties 창의 GridCombo, GridDate, GridTree 카테고리에서 세부 설정이 가능합니다.

예를 들어, Combo 타입으로 설정하는 경우에는 Cell의 설정에서 콤보에 출력할 데이터셋을 바인딩하고, code와 data를 설정해줘야 합니다.

CellCombo	
comboautoselect	false
combobuttonsize	px px  
combocodecol	CODE
combodatacol	VALUE
combodataset	ds_combo
combodisplaynulltext	
combodisplaynulltype	none

Tree 타입으로 설정하는 경우에는 Cell의 설정에서 트리의 깊이를 나타내는 treelevel, 트리의 상태를 나타내는 treestate, 트리의 체크박스를 설정하는 treecheck 속성을 설정해야 합니다.

CellTreeltem	
treecheck	bind:checkbox
treelevel	bind:tree
treestartlevel	0
treestate	1

24.4.1 예제

다음은 Cell의 edittype 속성을 설정한 Grid 예제입니다. 각 데이터가 Cell의 edittype 속성 설정에 따라 어떻게 표시되는지 확인할 수 있습니다.

normal	button	check	combo	date	mask	readonly	text	textarea	tree
Johr	Profile	☒	Male ▾	2016-01-05 화	AA105	secret	1dollar	Seoul, KOREA	 ☒ 0
James	Profile	☐	Male ▾	2010-08-26 목	CA826	secret	EBOT	Barcelona,	 1
Donald	Profile	☐	Male ▾	2012-03-21 수	CA321	secret	EBOT	California, USA	 1



sample_grid_23.xfdl

24.4.2 예제에서 사용한 핵심 기능

edittype

Grid의 Cell에서 편집 창의 형식을 설정하는 속성입니다. 아무 값도 설정하지 않으면 none이 기본값으로 설정됩니다.

autoenter

Grid에서 입력이 가능한 Cell을 선택하면 자동으로 입력 모드로 변경되는 방법을 설정하는 속성입니다.



key 값 설정은 런타임 버전에만 동작하며 웹 브라우저에서는 지원하지 않습니다.

wordwrap

Grid의 Cell에 표시되는 데이터가 Cell의 넓이보다 길 경우 자동으로 줄 넘김이 되도록 설정하는 속성입니다.

24.4.3 예제 구현 방법

① 화면 구성하기

툴바에서 Grid  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택하고 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

② Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다. Grid와 Combo 용으로 두 개의 Dataset을 생성해야 하므로 같은 작업을 한 번 더 반복합니다. 그리고 Grid를 위한 Dataset의 id는 ds_data로 하고, Combo를 위한 Dataset의 id는 ds_combo로 설정합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.
 - ds_data의 아이템 목록 작성

Dataset Editor [ds_data]

Const Columns

No id type size value

There are no items to show.

Columns

No id type size prop sumtext description

1	normal	STRING	256			
2	none	STRING	256			
3	button	STRING	256			
4	checkbox	STRING	256			
5	combo	STRING	256			
6	date	STRING	256			
7	mask	STRING	256			
8	readonly	STRING	256			
9	text	STRING	256			
10	textarea	STRING	256			
11	tree	STRING	256			

Rows

No	normal	none	button	checkbox	combo	date	mask	readonly	text	textarea	tree
1	John	Korea	Profile	1	M	20160105	AA105	secret	1dollar	Seoul, KOREA	0
2	James	Spain	Profile	0	M	20100826	CA826	secret	EBOT	Barcelona, Spain	1
3	Donald	USA	Profile	0	M	20120321	CA321	secret	EBOT	Califonia, USA	1
4	Lisa	United Kingdom	Profile	1	W	20101225	AA225	secret	1dollar	London, United Kingdom	2
5	Joe	Brazil	Profile	1	W	19190301	AA301	secret	hangul	São Paulo, Brazil	2
6	Michael	India	Profile	0	M	20140416	CA416	secret	1dollar	Mumbai, India	0
7	Lilli	Turkey	Profile	0	W	20200101	AA101	secret	hangul	Ankara, Turkey	1
8	Colt	Australia	Profile	1	M	19911221	CA221	secret	hangul	Sydney, NSW, Austailia	2

Dataset | Source |

- ds_combo의 아이템 목록 작성

Dataset Editor

Const Columns

No id type size value

There are no items to show.

Columns

No id type size prop sumtext description

1	CODE	STRING	256			
2	VALUE	STRING	256			

Rows

No	CODE	VALUE
1	M	Male
2	W	Female

Dataset | Source |

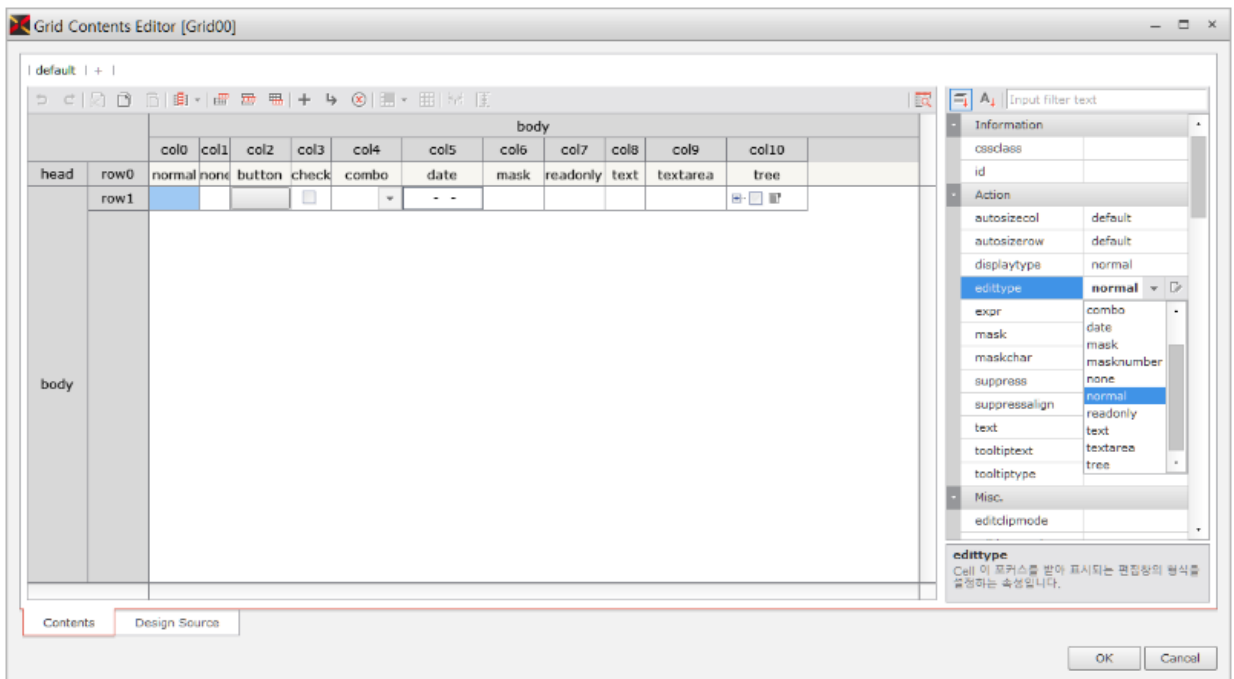
3 Grid에 Dataset 바인딩하기

ds_data를 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 ds_data를 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

4 Cell 설정하기

1. Grid00을 더블클릭하여 Grid Contents Editor를 엽니다. 아래 그림과 같이 좌측의 col0부터 col10까지 Cell을 선택한 후 우측 Properties 창에서 edittype 속성을 설정합니다.



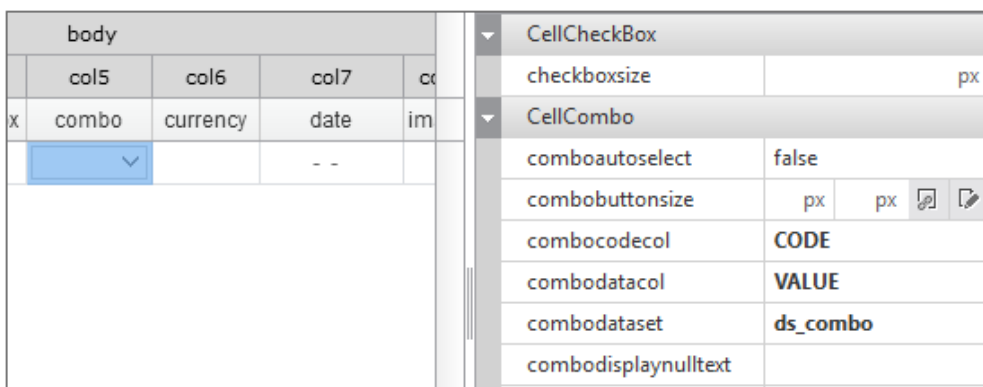
아래 표와 같이 각 Cell의 속성을 설정합니다. head 밴드의 Cell 설정은 헤드 컬럼에 표시될 타이틀을 설정하는 부분이고 body 밴드의 Cell 설정은 edittype과 displaytype 속성을 설정합니다.

Band	Row	Column	Property	Value
head	row0	col0	text	normal
		col1		none
		col2		button
		col3		checkbox
		col4		combo
		col5		date
		col6		mask
		col7		readonly
		col8		text
		col9		textarea
		col10		tree

Band	Row	Column	Property	Value
body	row1	col0	edittype (displaytype)	normal (normal)
		col1		none (none)
		col2		buttoncontrol (button)
		col3		checkboxcontrol (checkbox)
		col4		combocontrol (combo)
		col5		date (date)
		col6		mask (mask)
		col7		readonly (normal)
		col8		text (text)
		col9		textareacontrol (textarea)
		col10		treeitemcontrol (tree)
body	row1	col5	calendarpopupsize	250

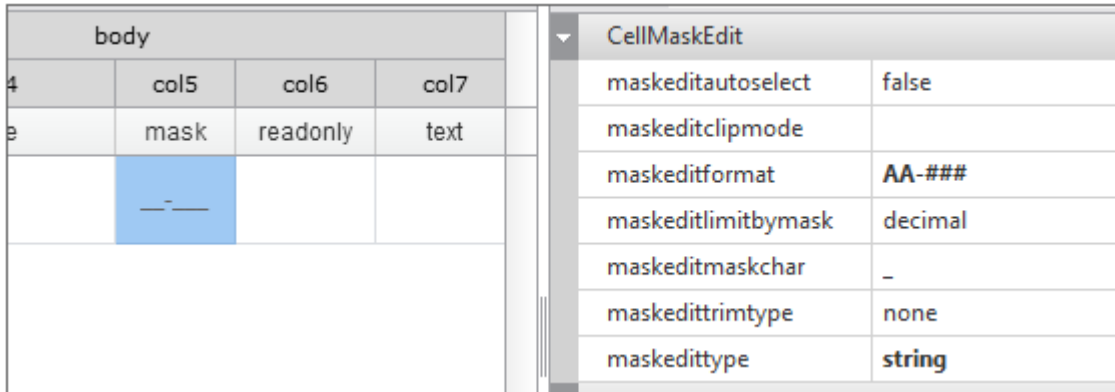
2. Grid Combo 설정하기

Combo로 설정할 Cell을 선택한 후 Properties 창에서 combodataset 속성을 찾습니다. combodataset 속성 옆에 ▼ 버튼을 클릭하여 ds_combo를 선택하여 바인딩합니다. 그리고 combocodecol, combodatacol 속성도 아래와 같이 설정합니다.



3. Grid Mask 설정하기

mask로 설정할 Cell을 선택한 후 Properties 창에서 mask 관련 속성을 다음과 같이 설정합니다.



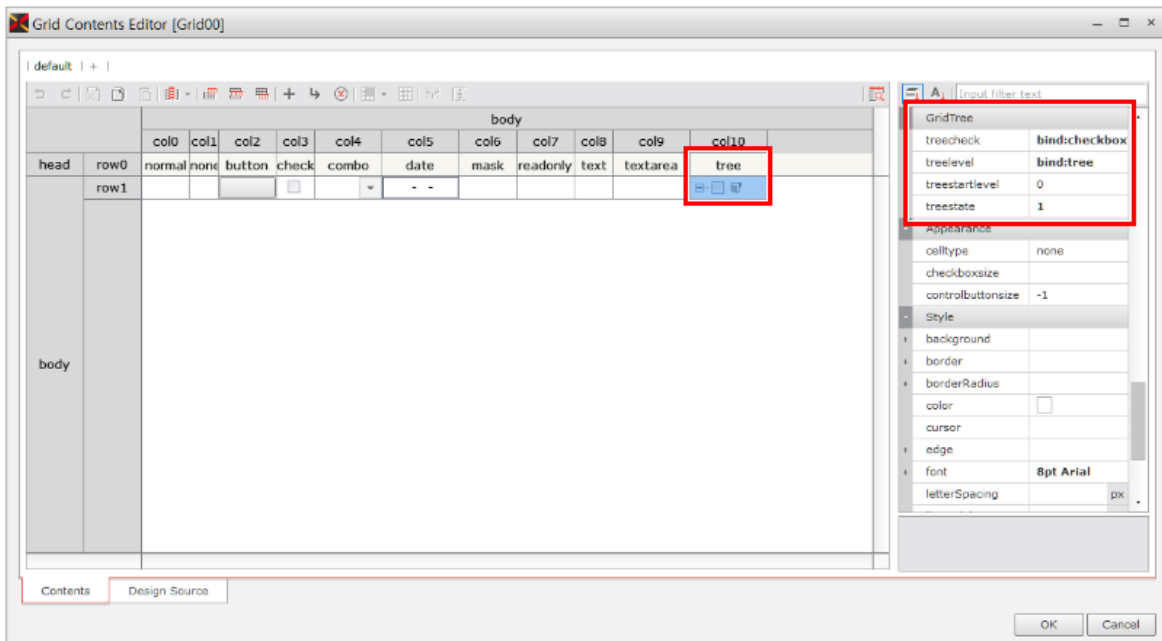
AA-###은 (영어 알파벳 대문자 2자)-(0~9까지의 10진수 숫자 3자리)로만 입력받는다는 의미입니다.

4. Grid Textarea 설정하기

textarea로 설정할 Cell을 선택한 후 Properties 창에서 wordWrap 속성을 english로 설정합니다. english로 설정하면 영어 단어 단위로 줄 바꿈이 이뤄집니다.

5. Grid Tree 설정하기

Tree로 설정할 Cell을 선택한 후 Properties 창에서 GridTree 카테고리를 찾습니다. treecheck 속성 옆의  버튼을 클릭하여 checkbox를 선택하고, 마찬가지로 treelevel 속성 옆의  버튼을 클릭하여 tree를 선택합니다. treecheck는 트리의 체크박스 값으로 사용할 Column을 바인딩하는 속성이고, treelevel은 트리의 레벨 값으로 사용할 Column을 바인딩하는 속성입니다.



5 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

각 타입별로 Column을 편집해 보며 edittype 속성의 설정에 따라 어떻게 동작하는지 확인합니다.

24.5 레코드 그룹핑하기

그룹핑은 SQL의 GROUP BY 문과 유사하게 Grid의 데이터를 작은 그룹으로 분류하는 기능으로 주로 그룹별로 통계 정보를 계산하기 위해 사용합니다. 그룹으로의 분류는 사용자가 지정한 공통된 값을 갖는 Column을 기준으로 나누고 공통되는 값이 없는 레코드 역시 각각 그룹이 만들어집니다.

그룹핑시 여러 Column을 지정하면 다중 그룹핑이 가능합니다. 예를 들어, 대그룹, 중그룹, 소그룹과 같이 나누어 그룹별 연산을 수행할 수 있습니다. 또한, 여러 Column을 지정하여 하나의 그룹으로 묶는 것도 가능한데 두 개 이상의 Column을 그룹핑하여 연산을 수행할 수 있습니다. 다중 그룹핑은 논리적으로 생성되는 레코드가 많아져 Grid가 지저분해질 수 있으므로 레코드 수 혹은 공통되는 Column을 가진 레코드에 따라 판단하는 것이 좋습니다. 다중 그룹핑에 대한 예제는 별도의 예제에서 다룹니다.

그룹핑을 하면 부가적으로 그룹별 연산 기능을 이용할 수 있습니다. 기본적으로는 그룹별 연산이 이뤄지지 않으나 INT 타입인 Column이 존재하면 해당 타입의 Column에 대해 자동으로 그룹별 합계 연산이 이뤄집니다. 어떤 연산을 수행할지는 prop 속성을 통하여 결정되며 사용자가 설정할 수 있습니다. 그룹별 연산의 결과는 각 그룹의 바로 밑에 논리적 레코드가 만들어져 출력됩니다. 논리적 레코드는 Dataset에 실재하지 않는 논리적으로만 생성된 레코드로 Dataset의 데이터 처리 작업에는 영향을 미치지 않습니다. prop 속성에 대한 더 자세한 내용은 별도의 예제에서 다룹니다.

연산의 종류는 다음과 같습니다.

연산	설명
NONE	(default) 아무 값도 표시하지 않습니다.
AVG	그룹의 평균값을 계산합니다.
COUNT	그룹의 개수를 계산합니다.
KEY	그룹의 키 값을 표시합니다.
MAX	그룹의 최대 값을 표시합니다.
MIN	그룹의 최소 값을 표시합니다.
SUM	그룹의 합계를 계산합니다.
TEXT	sumtext 속성값을 표시합니다. sumtext 속성은 Grid의 Summary 영역에 표시할 문자를 설정하는 속성입니다.

그룹핑은 Dataset의 keystack 메소드를 사용해 수행할 수 있습니다. 자세한 사용법은 관련 요소를 참조하시기 바랍니다.

24.5.1 예제

다음은 keystring 메소드를 사용해 그룹핑을 수행하는 Grid 예제입니다.

그룹핑은 Company Column을 기준으로 해당 Column 값이 같은 레코드들이 재정렬되어 모입니다. 그리고 각 그룹 별로 합계가 계산되어 그룹 바로 밑에 새로운 레코드로 추가되었음을 확인할 수 있습니다.

Name	Address	Company	Department	Salary	
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000	1dollar Group
Michael	Mumbai, India	1dollar	Personnel	₩1,300	
		1dollar		₩19,300	
James	Barcelona, Spain	EBOT	Consulting	₩25,000	
Donald	Califonia, USA	EBOT	Research Institute	₩1,000,000	
		EBOT		₩1,025,000	
Joe	São Paulo, Brazil	hangul	R&D	₩8,600	
Lilli	Ankara, Turkey	hangul	Quality Assurance	₩4,400	
GROUP by Company					



24.5.2 예제에서 사용한 핵심 기능

keystring

Dataset에 로드된 데이터를 그룹/정렬할 기준이 되는 조건식을 설정하는 속성입니다. 그룹 타입을 생략하면 G 옵션이 적용됩니다.

```

this.Dataset00.set_keystring( "column0" );
this.Dataset00.set_keystring( "G:column0" );
this.Dataset00.set_keystring( "G:+column0-column1" );
this.Dataset00.set_keystring( "G:+column0,S:-column1" );

```

24.5.3 예제 구현 방법

① 화면 구성하기

툴바에서 Grid , Button  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택하고 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

② Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.



Salary Column의 type을 INT로 설정해야 그룹별 합계가 계산됩니다.

Dataset Editor

Const Columns

No	id	type	size	value
There are no items to show.				

Columns

No	id	type	size	prop	sumtext	description
1	Name	STRING	256			
2	Address	STRING	256			
3	Company	STRING	256			
4	Department	STRING	256			
5	Salary	INT	256			

Rows

No	Name	Address	Company	Department	Salary
1	John	Seoul, KOREA	1dollar	Sales	15000
2	James	Barcelona, Spain	EBOT	Consulting	25000
3	Donald	Califonia, USA	EBOT	Research Institute	1000000
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000
5	Joe	São Paulo, Brazil	hangul	R&D	8600
6	Michael	Mumbai, India	1dollar	Personnel	1300
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000

Dataset | Source |

3 Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

4 그룹핑 기능 구현

Form에 있는 Button을 선택한 후 onclick 이벤트 핸들러를 아래와 같이 추가합니다.

- GROUP by Company 버튼의 onclick 이벤트 핸들러

```
this.btn_group_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Dataset00.set_keystring("G:Company");
};
```

5 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

위치를 변경하려는 Column의 헤더를 마우스로 드래그하여 원하는 위치에 드롭합니다.

24.6 레코드 정렬하기

정렬은 데이터를 특정한 조건에 따라 일정한 순서가 되도록 다시 배열하는 작업을 의미합니다. 그리드에서는 레코드를 특정 컬럼을 기준으로 정렬해서 제공해야 사용자 입장에서 데이터를 살펴보기에 좋으므로 정렬은 필수 기능입니다.

정렬의 종류에는 오름차순 정렬과 내림차순 정렬이 있습니다. 오름차순은 숫자나 문자가 낮은 것부터 차례로 배열되는 정렬이고 내림차순은 높은 것부터 차례로 배열되는 정렬입니다. 정렬 수행 시 컬럼을 지정하면 오름차순 혹은 내림차순으로 자유롭게 정렬할 수 있습니다.

정렬 시 여러 컬럼을 지정하면 다중 정렬이 가능합니다. 예를 들어, Column1과 Column2를 같이 지정하면 우선 Column1을 정렬하고, Column1이 같은 레코드들은 다시 Column2로 정렬합니다.

그리드에서 정렬 기능은 데이터셋의 keystring 메소드를 사용해 구현합니다. 자세한 사용법은 관련 요소를 참조하시기 바랍니다.

24.6.1 예제

다음은 keystring 메소드를 사용해 정렬을 수행하는 Grid 예제입니다.

SORT by Name 버튼을 누르면 Name Column을 기준으로 오름차순으로 정렬됩니다.

Name	Address	Company	Department	Salary
Colt	Sydney, NSW, Australia	hangul	Sales	₩62,000
Donald	California, USA	EBOT	Research Institute	₩1,000,000
James	Barcelona, Spain	EBOT	Consulting	₩25,000
Joe	São Paulo, Brazil	hangul	R&D	₩8,600
John	Seoul, KOREA	1dollar	Sales	₩15,000
Lilli	Ankara, Turkey	hangul	Quality Assurance	₩4,400
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
Michael	Mumbai, India	1dollar	Personnel	₩1,300

SORT
by Name



24.6.2 예제에서 사용한 핵심 기능

keystring

Dataset에 로드된 데이터를 그룹/정렬할 기준이 되는 조건식을 설정하는 속성입니다. 그룹 타입을 생략하면 G 옵션이 적용됩니다.

```
this.Dataset00.set_keystring( "S:column0" );
this.Dataset00.set_keystring( "S:column0+column0" );
this.Dataset00.set_keystring( "G:+column0,S:-column1" );
```

24.6.3 예제 구현 방법

① 화면 구성하기

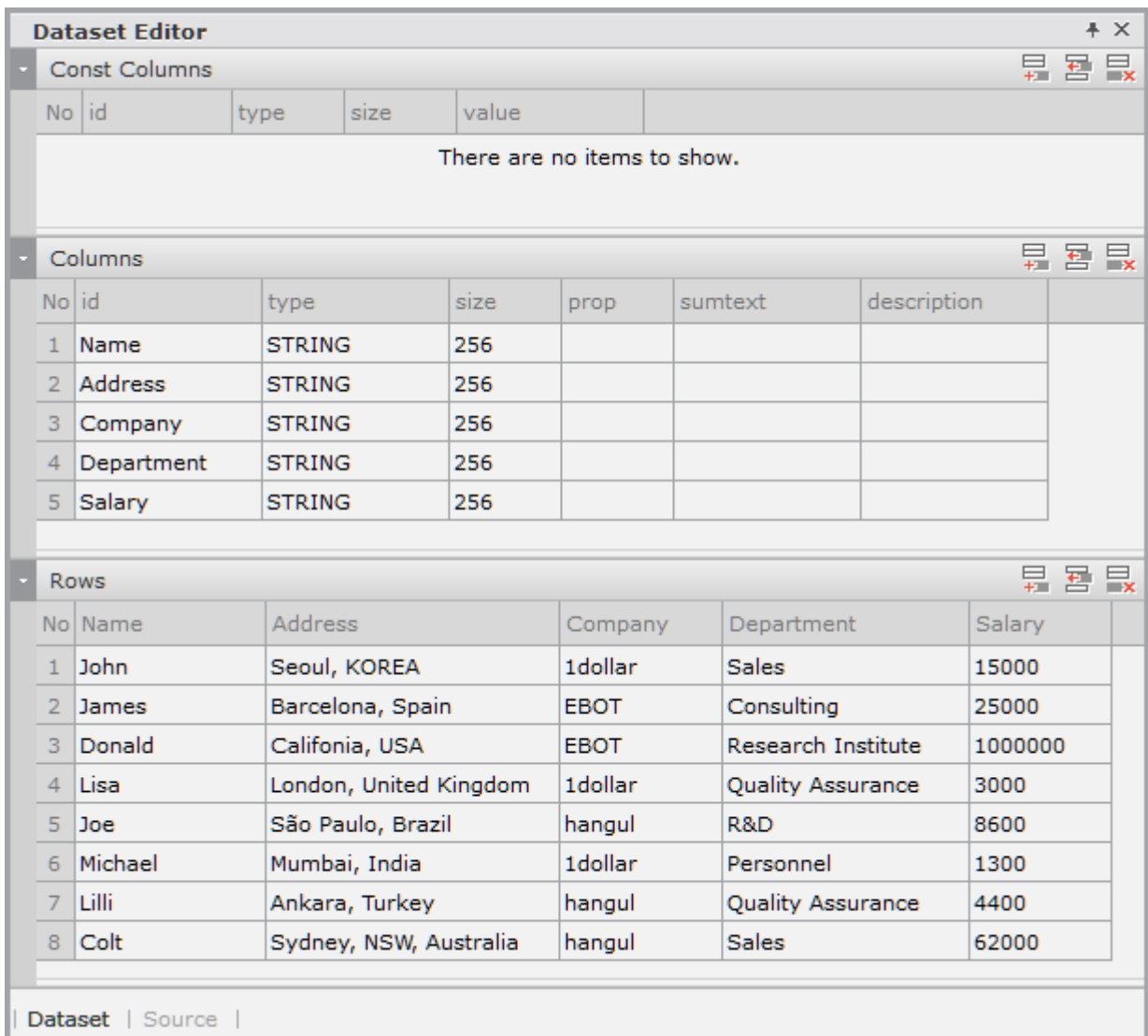
툴바에서 Grid , Button  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택하고 그냥

폼에 대고 클릭하면 기본 크기로 생성됩니다.

2 Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 항목 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 항목 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.



Dataset Editor

Const Columns

No	id	type	size	value
There are no items to show.				

Columns

No	id	type	size	prop	sumtext	description
1	Name	STRING	256			
2	Address	STRING	256			
3	Company	STRING	256			
4	Department	STRING	256			
5	Salary	STRING	256			

Rows

No	Name	Address	Company	Department	Salary
1	John	Seoul, KOREA	1dollar	Sales	15000
2	James	Barcelona, Spain	EBOT	Consulting	25000
3	Donald	Califonia, USA	EBOT	Research Institute	1000000
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000
5	Joe	São Paulo, Brazil	hangul	R&D	8600
6	Michael	Mumbai, India	1dollar	Personnel	1300
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000

Dataset | Source |

3 Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

4 정렬 기능 구현

Form에 있는 Button을 선택한 후 onclick 이벤트 함수를 아래와 같이 추가합니다.

- SORT by Name 버튼의 onclick 이벤트 함수

```
this.btn_sort onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Dataset00.set_keystring("S:Name");
};
```

5 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Sort by Name 버튼을 눌러 Name 컬럼이 오름차순으로 정렬되는지 확인합니다.

24.7 값이 동일한 셀을 하나의 셀로 표시하기 - Suppress

Grid는 공통되는 특징을 가진 다량의 데이터를 사용자에게 전달하기 위한 컴포넌트입니다. 다량의 데이터를 Grid에 출력하기 때문에 복잡하며 사용자가 레코드를 비교해 보기가 쉽지 않습니다.

Grid에서는 데이터의 복잡함을 줄이고 효율적으로 사용자에게 전달하기 위해 suppress 기능을 제공합니다. suppress 기능은 한 Column에 같은 값을 가진 Cell이 여러 개 반복적으로 나열되어 있을 경우 하나의 Cell로 합쳐 보여줍니다. 예를 들어, 아래 그림과 같이 1dollar나 hangul과 같이 반복되는 Cell들을 하나의 Cell로 합쳐 보여준다면 좀 더 Grid의 가독성을 높일 수 있습니다.

Name	Address	Company	Department	Salary
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
Michael	Mumbai, India			₩1,300
John	Seoul, KOREA		Sales	₩15,000
James	Barcelona, Spain	EBOT	Consulting	₩25,000
Donald	Califonia, USA	hangul	Research Institute	₩1,000,000
Joe	São Paulo, Brazil		Quality Assurance	₩8,600
Lilli	Ankara, Turkey			₩4,400
Colt	Sydney, NSW, Australia		R&D	₩62,000

suppress 기능은 같은 값을 갖는 Cell이 연속해서 나열되어 있을 경우에 효율적으로 동작합니다. 예를 들어, 아래 Grid의 Company Column과 같이 값이 연속해서 출력되어 있지 않은 경우에는 suppress 기능의 효과를 볼 수 없습니다. 따라서 suppress 기능을 사용하기에 앞서 suppress를 수행하려는 Column을 먼저 정렬해줘야 합니다.

Name	Address	Company	Department	Salary
John	Seoul, KOREA	1dollar	Sales	₩15,000
James	Barcelona, Spain	EBOT	Consulting	₩25,000
Donald	Califonia, USA	EBOT	Research Institute	₩1,000,000
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
Joe	São Paulo, Brazil	hangul	Quality Assurance	₩8,600
Michael	Mumbai, India	1dollar	Quality Assurance	₩1,300
Lilli	Ankara, Turkey	hangul	Quality Assurance	₩4,400
Colt	Sydney, NSW, Australia	hangul	R&D	₩62,000



suppress 기능은 Cell의 displaytype이 progressbarcontrol, buttoncontrol, checkboxcontrol, treeitemcontrol인 경우는 동작하지 않습니다.

suppress는 레벨 값으로 설정하는데 레벨 값에 따라 어떤 Column의 Cell에서 먼저 suppress가 수행될지 결정합니다. 예를 들어, Company Cell의 suppress 값을 1로 설정하고 Department Cell의 suppress 값을 2로 설정했을 경우 Company Column의 Cell에서 먼저 suppress를 수행하고 그다음 레벨인 Department Column의 Cell에서 suppress가 수행됩니다. 그리고 suppress 레벨을 설정하지 않은 다른 Column의 Cell에서는 suppress를 수행하지 않습니다. 이런 식으로 suppress 값으로 레벨을 설정함으로써 각 Column의 Cell에 우선순위를 줄 수 있습니다.

24.7.1 예제

다음은 suppress 기능을 수행한 Grid 예제입니다.

Suppress 버튼을 누르면 먼저 공통되는 값을 갖는 Company와 Department를 기준으로 정렬한 후 suppress 기능을 수행하여 같은 값을 갖는 Cell을 합쳐줍니다.

Company Column의 Cell에 suppress 값을 1로 설정하고 Department Column의 Cell에 suppress 값을 2로 설정했기 때문에 Company Column에서 먼저 suppress가 수행되고 난 후 그 결과를 바탕으로 다시 Department Column에서 suppress가 수행됩니다. 그 이외의 Column에서는 suppress를 설정하지 않았기 때문에 아무런 변화가 없습니다.

Name	Address	Company	Department	Salary
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
Michael	Mumbai, India			₩1,300
John	Seoul, KOREA		Sales	₩15,000
James	Barcelona, Spain	EBOT	Consulting	₩25,000
Donald	Califonia, USA		Research Institute	₩1,000,000
Joe	São Paulo, Brazil	hangul	Quality Assurance	₩8,600
Lilli	Ankara, Turkey			₩4,400
Colt	Sydney, NSW, Australia		R&D	₩62,000
Suppress				



24.7.2 예제에서 사용한 핵심 기능

suppress

Grid Cell에 suppress 기능을 설정하는 속성입니다.

```
var retVal;

retVal = this.Grid00.setCellProperty("body", 2, "suppress", "1");
retVal = this.Grid00.setCellProperty("body", 3, "suppress", "2");
```

24.7.3 예제 구현 방법

① 화면 구성하기

툴바에서 Grid , Button  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택하고 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

② Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.

2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.

Dataset Editor [Dataset00]

Const Columns

No	id	type	size	value
There are no items to show.				

Columns

No	id	type	size	prop	sumtext	description
1	Name	STRING	256			
2	Address	STRING	256			
3	Company	STRING	256			
4	Department	STRING	256			
5	Salary	INT	256			

Rows

No	Name	Address	Company	Department	Salary
1	John	Seoul, KOREA	1dollar	Sales	15000
2	James	Barcelona, Spain	EBOT	Consulting	25000
3	Donald	Califonia, USA	EBOT	Research Institute	1000000
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000
5	Joe	São Paulo, Brazil	hangul	Quality Assurance	8600
6	Michael	Mumbai, India	1dollar	Quality Assurance	1300
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400
8	Colt	Sydney, NSW, Australia	hangul	R&D	62000

Dataset | Source

3 Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

4 suppress 기능 구현

suppress 기능을 구현하기 위해 suppress를 수행하려는 Column을 먼저 정렬하고 Cell의 suppress 속성을 설정합니다.

1. Company Column을 먼저 정렬하고 그 결과를 바탕으로 다시 Department Column을 정렬합니다.
2. Company Cell의 suppress 속성을 1로, Department Cell의 suppress 속성을 2로 설정합니다. Compa

ny Column의 Cell이 먼저 suppress 되고 그 결과를 바탕으로 다시 Department Column의 Cell이 suppress 됩니다.

- suppress 버튼의 onclick 이벤트 핸들러

```
this.btn_suppress onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    //Sorting records
    this.Dataset00.set_keystring("S:Company+Department");

    //Suppressing cells
    var retVal = this.Grid00.setCellProperty("body", 2, "suppress", "1");
    var retVal = this.Grid00.setCellProperty("body", 3, "suppress", "2");

};
```

⑤ QuickView로 확인하기

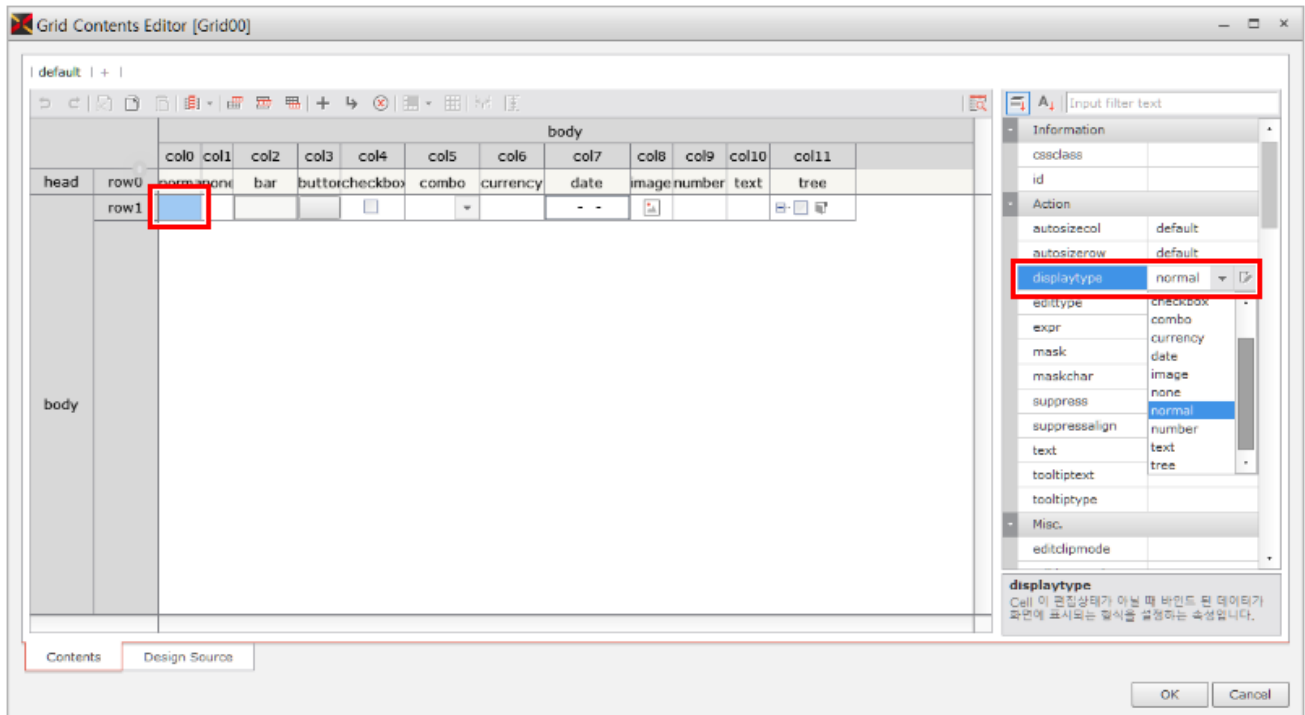
QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

24.8 셀 속성 설정하기

Grid 컴포넌트는 GridBandControl, GridCellControl, GridColumnControl, GridRowControl와 같은 여러 오브젝트로 구성됩니다. 그 중 GridCellControl 오브젝트는 Grid에 바인딩된 데이터를 실제 Grid에 출력하는 중요한 역할을 합니다. 따라서 Grid에 원하는 형태로 데이터를 출력하거나 연산처리를 하기 위해서는 GridCellControl 오브젝트를 설정해야 합니다.

GridCellControl 오브젝트를 설정하려면 Grid Contents Editor를 이용하는 방법과 스크립트에서 메소드를 호출하는 두 가지 방법이 있습니다.

- Grid Contents Editor를 이용하는 방법은 설정하기 원하는 Cell을 선택한 후 Properties 창에서 원하는 속성을 설정합니다. 이 방법은 애플리케이션 개발 시에 Cell 속성을 설정하는 경우에 사용합니다.



- 스크립트에서 메소드를 호출하는 방법은 `getCellProperty`, `setCellProperty` 메소드를 사용하여 속성을 설정합니다. `getCellProperty` 메소드는 지정한 Cell의 속성값을 반환하는 메소드이고, `setCellProperty` 메소드는 지정한 Cell의 속성값을 설정하는 메소드입니다. 이 방법은 애플리케이션 수행 중에 Cell 속성을 설정하는 경우에 사용됩니다.

```
this.btn_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var nCellIdx = this.Grid00.currentcell;

    var varCellProp = this.Grid00.getCellProperty("body",nCellIdx,"displaytype");

    var bRet = this.Grid00.setCellProperty("body",nCellIdx,"displaytype","button");
};
```

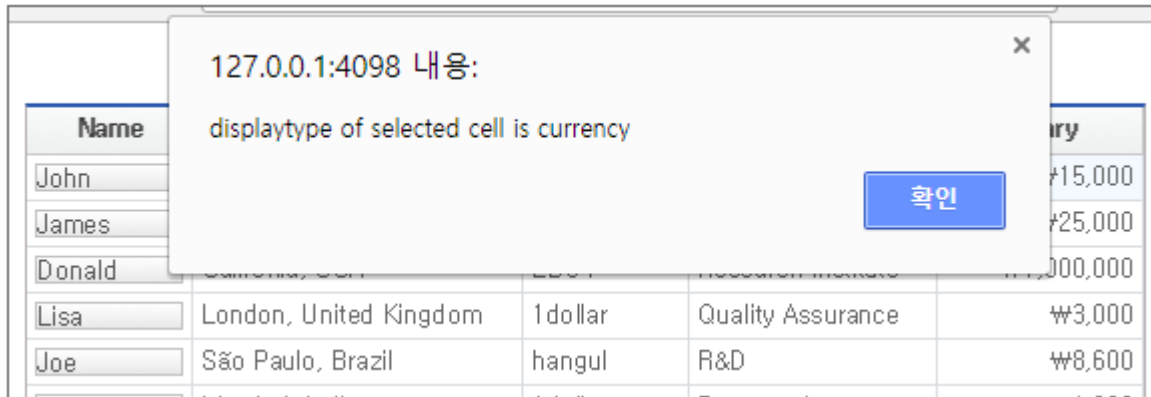
Grid Contents Editor를 이용하는 방법은 다른 Grid 예제에서 계속해서 다뤄왔기 때문에 이번 예제에서는 스크립트에서 메소드를 사용하여 Cell 속성을 설정하는 방법을 중심으로 설명합니다.

24.8.1 예제

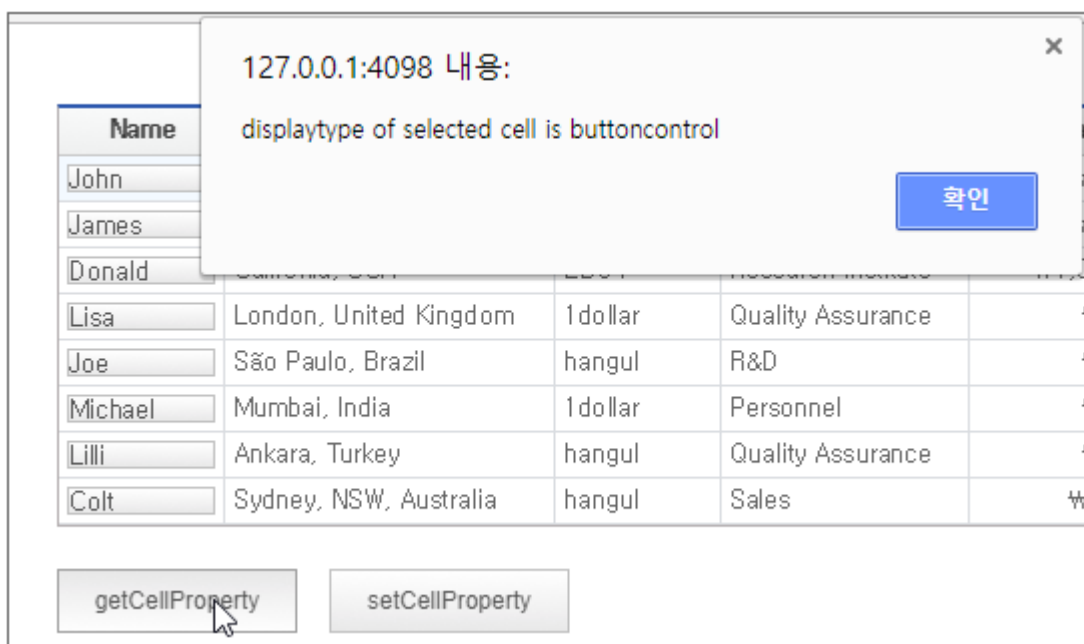
다음은 `getCellProperty`, `setCellProperty` 메소드를 사용하여 지정한 Cell의 속성값을 가져오거나 설정하는 Grid 예제입니다. 예제에는 `getCellProperty`와 `setCellProperty` 메소드를 호출하는 두 개의 버튼이 존재합니다.

- `getCellProperty` 버튼은 현재 선택한 Cell의 `displaytype` 속성값을 화면에 팝업으로 출력합니다. 아래 그림은 S

Salary Column의 Cell을 선택한 후 getCellProperty를 수행한 결과입니다. Cell의 displaytype 속성이 currency로 설정되어 있음을 확인할 수 있습니다.



- setCellProperty 버튼은 현재 선택한 Cell의 displaytype 속성값을 buttoncontrol로 설정합니다. 아래 그림은 Salary Column의 Cell을 선택한 후 setCellProperty를 수행한 결과입니다. Cell의 모양이 버튼으로 변경되었고 getCellProperty 버튼을 눌러 확인해보면 buttoncontrol로 나옴을 알 수 있습니다.



 sample_grid_27.xfdl

24.8.2 예제에서 사용한 핵심 기능

getCellProperty

Grid Cell의 특정 속성 값을 반환하는 메소드입니다.

```
var strHeadCellColor = this.Grid00.getCellProperty( "head", 0, "color" );
```

setCellProperty

Grid Cell의 특정 속성 값을 설정하는 메소드입니다.

```
var bSucc = this.Grid00.setCellProperty( "head", 0, "color", "black");
```

currentcell

Grid에서 현재 선택된 Cell의 인덱스를 갖는 속성입니다. 인덱스는 0부터 시작하며 선택된 Cell이 없는 경우 -1 값을 갖습니다.

24.8.3 예제 구현 방법

① 화면 구성하기

툴바에서 Grid , Button  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택하고 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

② Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.

Dataset Editor

Const Columns

No	id	type	size	value	
There are no items to show.					

Columns

No	id	type	size	prop	sumtext	description	
1	Name	STRING	256				
2	Address	STRING	256				
3	Company	STRING	256				
4	Department	STRING	256				
5	Salary	STRING	256				

Rows

No	Name	Address	Company	Department	Salary	
1	John	Seoul, KOREA	1dollar	Sales	15000	
2	James	Barcelona, Spain	EBOT	Consulting	25000	
3	Donald	Califonia, USA	EBOT	Research Institute	1000000	
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000	
5	Joe	São Paulo, Brazil	hangul	R&D	8600	
6	Michael	Mumbai, India	1dollar	Personnel	1300	
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400	
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000	

Dataset

|

Source

|

3 Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

4 getCellProperty 버튼 기능 구현

getCellProperty 버튼의 id를 btn_getCellProperty로 설정하고 onclick 이벤트 핸들러를 추가합니다.

- getCellProperty 버튼의 onclick 이벤트 핸들러

현재 선택된 Cell의 displaytype 속성값을 가져와 팝업으로 출력합니다.

```

this.btn_getCellProperty_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    //Getting index of selected cell
    var nCellIdx = this.Grid00.currentcell;

    //Getting displaytype property of selected cell
    var varCellProp = this.Grid00.getCellProperty("body", nCellIdx, "displaytype");

    alert("displaytype of selected cell is " + varCellProp);

};

```

5 setCellProperty 버튼 기능 구현

setCellProperty 버튼의 id를 btn_setCellProperty로 설정하고 onclick 이벤트 핸들러를 추가합니다.

- setCellProperty 버튼의 onclick 이벤트 핸들러

현재 선택된 Cell의 displaytype 속성을 buttoncontrol1으로 설정합니다.

```

this.btn_setCellProperty_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    //Getting index of selected cell
    var nCellIdx = this.Grid00.currentcell;

    //Setting displaytype property of selected cell
    var bRet = this.Grid00.setCellProperty("body", nCellIdx, "displaytype", "buttoncontrol1");

};

```

6 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Grid의 Cell을 선택한 후 getCellProperty 버튼을 눌러 displaytype 속성의 값을 확인합니다. 마찬가지로 Cell을 선택한 후 setCellProperty 버튼을 눌러 displaytype 속성이 buttoncontrol1으로 설정되는지 결과를 확인합니다.

24.9 셀 병합하기

그리드에서는 같은 열이나 행에 있는 두 개 이상의 셀을 하나의 셀로 병합하는 기능을 제공합니다. 이는 엑셀의 셀 병합 기능과 같은 기능으로 그리드에서 데이터를 표현하는데 매우 유용합니다.

셀 병합은 ❶ A big cell(하나의 큰 셀로 실제 병합하는 방식)과 ❷ Having child cells(하나의 셀로 병합되지만, 내부적으로는 여러 개의 ❸ 서브 셀로 구성되는 방식)의 두 가지로 나뉩니다.

- A big cell은 여러 셀을 하나로 합칠 때 가장 앞의 (셀 인덱스가 앞서는) 셀을 기준으로 다른 셀들과 합쳐집니다. 다시 말해 기준이 되는 가장 앞의 셀을 제외한 다른 셀의 속성은 사라지고 기준이 되는 셀의 속성을 갖는 큰 셀로 병합됩니다. 이 방법은 셀을 실제로 하나로 병합할 필요가 있을 때 사용합니다.
- Having child cells는 외부적으로는 셀이 하나로 합쳐지지만, 내부적으로는 각 셀의 속성을 그대로 유지합니다. 따라서 아래 ❷와 같이 서브 셀들이 각각의 text를 그대로 화면에 출력하는 형태가 됩니다. 이 방법은 외부적으로 하나의 셀처럼 보여야 하고 내부적으로 별개의 셀로 동작할 필요가 있을 때 사용합니다.

	col0	col1	col2	col3	col4
row0		ID		Mail	Primary
row1		First			Secondary
row2	Name	Middle	Contact		Mobile
row3		Last		Tel.	Office



서브 셀은 셀 병합 시 병합된 셀 내부에 존재하는 셀을 말합니다. 서브 셀이 모여 하나의 셀을 구성하며 병합된 후에도 속성을 그대로 유지하기 때문에 각각의 셀을 컨트롤 할 수 있습니다.



셀 인덱스는 0부터 시작합니다. 각 밴드별로 좌상단부터 우하단까지 순차적으로 부여됩니다.

24.9.1 예제

다음은 셀 병합을 수행한 Grid 예제입니다.

ID		Contact	Mail	Primary
Name	First Middle Last		Tel.	Secondary
JOHNSON		Contact	Mail	john@email.com
Name	John Doe		Tel.	john@email.org 1012345678
JJ		Contact	Mail	jane@email.com
Name	Jane Roe		Tel.	1056781234 256781234
jaspers		Contact	Mail	dick@email.com dick@email.org
Name	Dick Fitzgerald Public		Tel.	1012345678 212345678

A big cell

Having child cells



sample_grid_28.xfdl

그림에서 빨간색으로 표시된 부분이 A big cell 방식으로 병합한 셀이고, 파란색으로 표시된 부분이 Having child cells 방식으로 병합한 셀입니다.

A big cell 방식으로 병합한 셀은 실제 하나의 셀로 동작합니다. Having child cells 방식으로 병합한 셀은 하나의 셀처럼 보이나 내부적으로 서브 셀이 각각 데이터를 출력합니다.

그리드 예제의 실제 포맷은 아래와 같습니다. 위 그림의 병합된 셀 부분과 아래의 포맷을 비교해 보면 두 방식의 차이점을 확인할 수 있습니다.

		body			
		col0	col1	col2	col3
head	row0	ID		Contact	Primary
	row1	Name	First		Secondary
	row2		Middle		Mobile
	row3		Last		Office
body	row4			Contact	Mail
	row5	Name			
	row6				
	row7				Tel.

24.9.2 예제에서 사용한 핵심 기능

없음.

24.9.3 예제 구현 방법

1 화면 구성하기

툴바에서 Grid  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택하고 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

2 Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.

Dataset Editor

Const Columns

No

id

type

size

value

There are no items to show.

Columns

No

id

type

size

prop

sumtext

description

1

ID

STRING

256

2

First

STRING

256

3

Middle

STRING

256

4

Last

STRING

256

5

Mail_P

STRING

256

6

Mail_S

STRING

256

7

Tel_M

STRING

256

8

Tel_O

STRING

256

Rows

No

ID

First

Middle

Last

Mail_P

Mail_S

Tel_M

Tel_O

1

JOHNSON

John

Doe

john@email.com

john@email.org

1012345678

2

JJ

Jane

Roe

jane@email.com

1056781234

256781234

3

jaspers

Dick

Fitzgerald

Public

dick@email.com

dick@email.org

1012345678

212345678

4

son

Robert

Walker

Smith

robert@email.com

robert@email.org

256781234

5

Daniel

Jack

Robertson

jack@email.com

1012345678

212345678

6

BILLBILL

Billy

Cook

bill@email.com

bill@email.org

1056781234

7

mixnut

Elizabeth

Soap

elizabeth@email.com

elizabeth@email.org

1012345678

8

sync

Harry

Hussein

Blow

harry@email.com

1056781234

256781234

Dataset | Source

Dataset | Source |

3 Grid 포맷 만들기

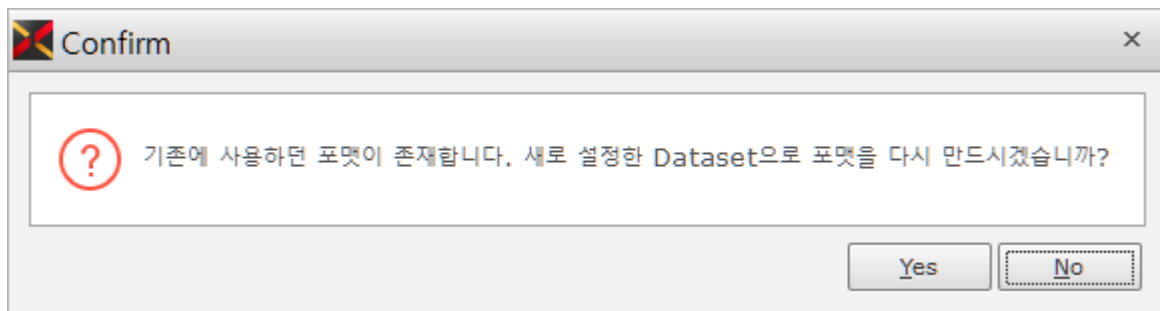
그리드를 더블 클릭하여 Grid Contents Editor를 열어 아래와 같이 Column과 Row를 구성합니다.

		body				
		col0	col1	col2	col3	col4
head	row0	ID		Contact	Mail	Primary
	row1	Name	First			Secondary
	row2		Middle		Tel.	Mobile
	row3		Last			Office
body	row4			Contact	Mail	
	row5	Name				
	row6				Tel.	
	row7					

4 Dataset 바인딩

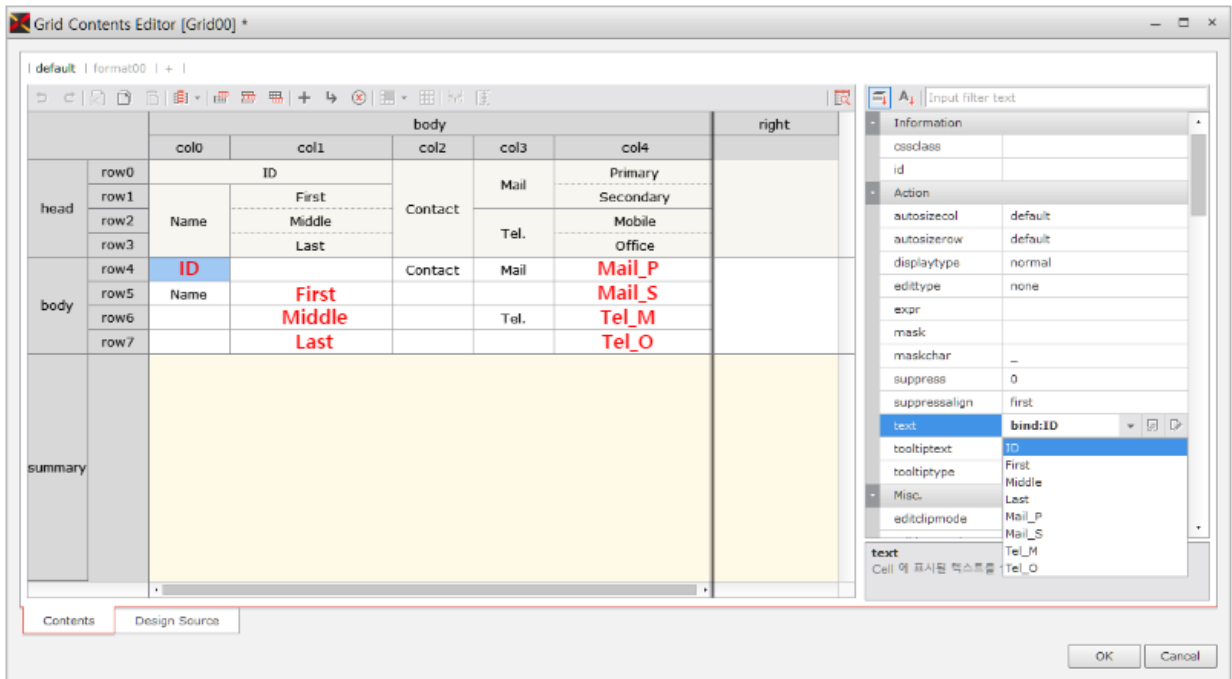
1. 그리드에 데이터셋을 바인딩합니다.

그리드를 선택한 후 Properties 창에서 binddataset 속성에 Dataset00을 바인딩합니다. 바인딩을 시도하면 데이터셋에 맞게 포맷을 재구성하려 시도하기 때문에 아래와 같은 Confirm 창이 뜹니다. 그러나 우리는 앞서 별도의 포맷을 사용하기 위해 만들어 놔기 때문에 No를 선택합니다.



2. 셀에 데이터를 바인딩합니다.

그리드에 데이터셋을 바인딩한 상태에서 Grid Contents Editor를 열어 body 밴드에 속한 각 셀의 text 속성에 데이터를 바인딩합니다. 아래 빨간 글씨에 있는 위치의 셀을 선택하여 글씨에 해당하는 값을 text 속성에서 선택합니다.



5 셀 병합하기

아래와 같이 셀을 병합합니다. 빨간색으로 표시된 부분은 A big cell 방식으로 병합하고 파란색으로 표시된 부분은 Having child cells 방식으로 병합합니다.

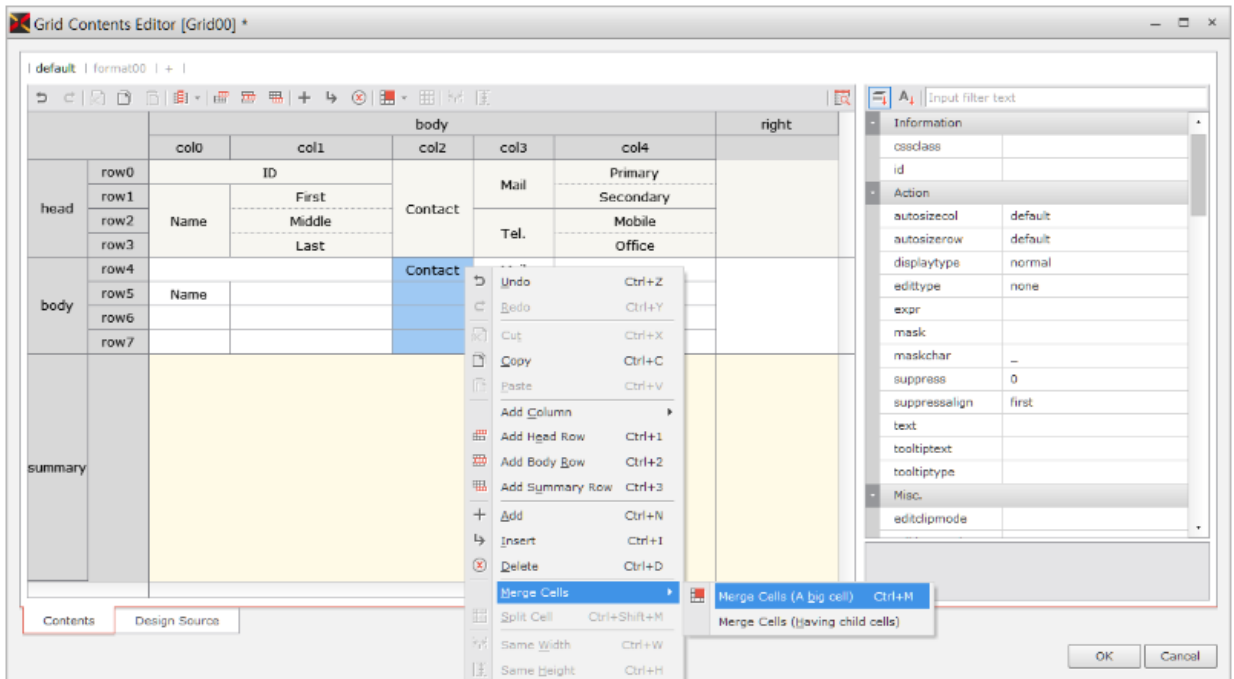
		body				
		col0	col1	col2	col3	col4
head	row0	ID		Contact	Mail	Primary
	row1	Name	First			Secondary
	row2		Middle		Tel.	Mobile
	row3		Last			Office
body	row4			Contact	Mail	
	row5	Name				
	row6				Tel.	
	row7					

셀 병합은 Grid Contents Editor에서 병합하고자 하는 셀을 마우스 드래그로 선택한 후 마우스 오른쪽 버튼을 눌러 컨텍스트 메뉴에서 [Merge Cells]을 선택합니다. Merge Cells를 선택하면 세부 메뉴를 선택할 수 있는데 Merge Cells(A big cell)과 Merge Cells(Having child cells) 중에 원하는 병합 방식을 선택합니다.

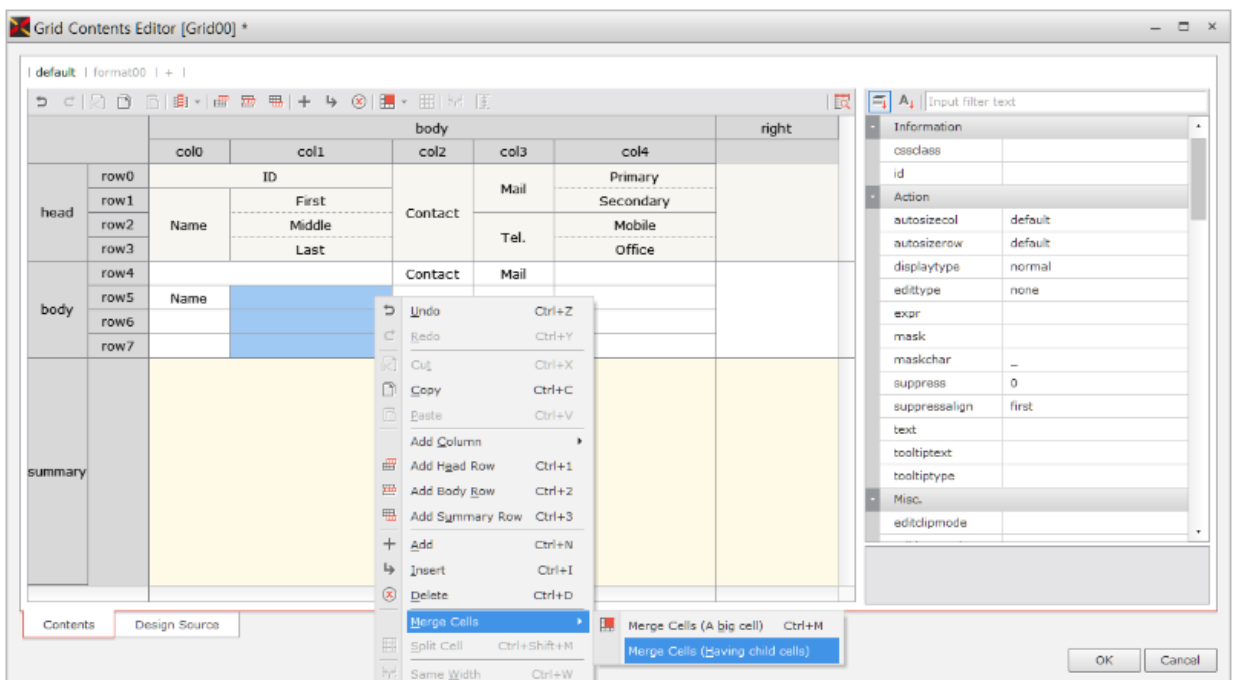
병합이 잘못되었을 경우 Split Cell 메뉴로 병합을 풀 수 있습니다. Split Cell은 병합된 셀을 선택한 후 마우스 오른쪽 버튼을 눌러 컨텍스트 메뉴에서 [Split Cell]을 선택합니다.

병합이 모두 완료되면 OK 버튼을 눌러 Grid Contents Editor를 닫습니다.

- A big cell 병합 방법



- Having child cells 병합 방법



6 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Grid의 각 셀이 병합되었는지 병합된 셀에 데이터가 정상적으로 출력되는지 확인합니다.

24.10 서브 셀 속성 설정하기

서브 셀은 병합된 셀 내부에 있는 셀입니다. 서브 셀이 모여 하나의 셀을 구성하며 병합된 후에도 속성을 그대로 유지하기 때문에 각각의 서브 셀을 컨트롤 할 수 있습니다.

그리드에서 서브 셀을 컨트롤하기 위해서 `getSubCellProperty`, `setSubCellProperty` 메소드를 사용합니다. 두 메소드를 사용하려면 반드시 셀 인덱스와 서브 셀 인덱스를 알아야 합니다. 그래야 서브 셀의 속성값을 읽어오거나 설정할 수 있습니다.

		body						
		col0	col1		col2	col3	col4	
head	row0	ID			Contact	Mail	Primary	
	row1	Name	First				Secondary	
	row2		Middle			Tel.	Mobile	
	row3		Last				Office	
body	row4	Sub Cell 0 Sub Cell 1 Sub Cell 2			Contact	Mail		
	row5							
	row6					Name	Tel.	
	row7							

Merged Cell

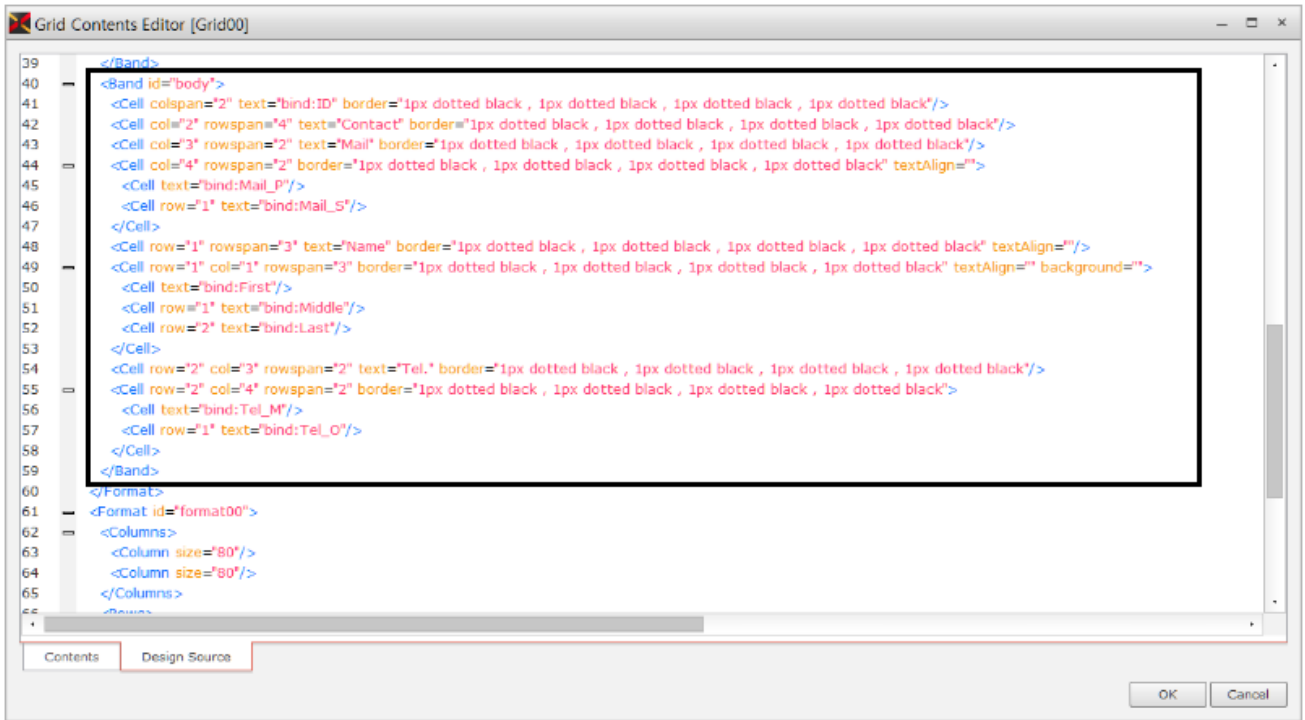
셀 인덱스는 셀 병합이 발생하면 달라집니다. 셀 병합으로 인해 줄어드는 셀 수만큼 감소하여 조정되기 때문에 주의해야 합니다. 서브 셀은 병합된 셀의 셀 인덱스를 공유하면서 별도의 서브 셀 인덱스를 갖습니다. 서브 셀 인덱스는 병합된 셀 내에 정의된 순서대로 0부터 시작합니다.

아래는 병합한 셀을 포함하고 있는 그리드의 구조를 보여줍니다. Body 밴드의 각 셀에는 숫자가 마킹되어 있는데 빨간색이 셀 인덱스를 파란색이 서브 셀 인덱스를 나타냅니다. 그림에는 표시되어 있지 않지만 헤드 밴드도 Body 밴드와 같은 구조이므로 인덱스 구조 또한 같습니다.

		body				
		col0	col1	col2	col3	col4
head	row0	ID		Contact	Mail	Primary
	row1	Name	First			Secondary
	row2		Middle		Tel.	Mobile
	row3		Last			Office
body	row4			Contact	Mail	
	row5	Name			Tel.	
	row6					
	row7					

Cell Index Sub Cell Index

셀과 서브 셀 인덱스는 좌상단에서 우하단쪽으로 순서대로 매겨집니다. 인덱스는 Grid Contents Editor에서 그리드 포맷을 보고 세면 되지만 구조가 복잡한 경우에는 Design Source 탭에서 직접 그리드 코드를 보는 방법도 있습니다. 그리드 코드에서 셀의 인덱스는 나열된 순서대로 매겨지기 때문에 소스에서 확인하는 것이 가장 확실합니다.



24.10.1 예제

다음은 병합된 셀을 갖는 그리드에서 서브 셀의 속성을 설정하고 값을 읽어오는 예제입니다.

ID		Contact	Mail	Primary Secondary
Name	First Middle Last		Tel.	Mobile Office
jaspers		Contact	Mail	dick@email.com
Name	Dick Fitzgerald Public		Tel.	dick@email.org
son		Contact	Mail	1012345678
Name	Robert Walker Smith		Tel.	212345678
Daniel		Contact	Mail	robert@email.com
Name	Jack Robertson		Tel.	robert@email.org
		Contact	Mail	256781234
Name			Tel.	jack@email.com
		Contact	Mail	1012345678
Name			Tel.	212345678

getSubCellProperty

setSubCellProperty

1. 먼저 `getSubCellProperty` 버튼을 눌러 First Name에 해당하는 서브 셀의 background 속성값을 확인합니다. 아무 색깔도 설정되지 않았기에 undefined로 나옵니다.
2. `setSubCellProperty` 버튼을 눌러 해당 서브 셀의 background 속성값을 khaki로 변경합니다.
3. 다시 `getSubCellProperty` 버튼을 눌러 해당 서브 셀의 background 속성값을 확인하여 khaki로 변경되었는지 확인합니다.

두 개의 버튼은 구체적으로 다음과 같은 기능을 합니다.

- `getSubCellProperty` 버튼은 Body 밴드에서 First Name에 해당하는 셀의 background 속성값을 읽어와 팝업으로 출력합니다. First Name은 5번째 셀에서 0번째 서브 셀입니다. `setSubCellProperty` 버튼을 누르지 않은 상태에서 `getSubCellProperty` 버튼을 누르면 background 속성에 아무 값도 설정되어 있지 않기 때문에 'undefined'로 나옵니다.
- `setSubCellProperty` 버튼은 Body 밴드에서 First Name에 해당하는 셀의 background 속성값을 'khaki'로 설정하여 배경색을 변경합니다. 버튼을 누르면 First Name 셀의 배경색이 Khaki로 변경됩니다.



24.10.2 예제에서 사용한 핵심 기능

`getSubCellProperty`

Grid의 Cell이 서브 셀 구조일 경우 지정한 서브 셀의 속성값을 반환하는 메소드입니다.

`setSubCellProperty`

Grid의 Cell이 서브 셀 구조일 경우 지정한 서브 셀의 속성값을 설정하는 메소드입니다.

24.10.3 예제 구현 방법

① 화면 구성하기

툴바에서 Grid , Button  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택하고 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

② Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset을 더블 클

릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.

The screenshot shows the 'Dataset Editor' window. It has three main sections: 'Const Columns', 'Columns', and 'Rows'.

Const Columns: A table with columns: No, id, type, size, value. It contains the message: "There are no items to show."

Columns: A table with columns: No, id, type, size, prop, sumtext, description. It contains 8 rows of column definitions:

No	id	type	size	prop	sumtext	description
1	ID	STRING	256			
2	First	STRING	256			
3	Middle	STRING	256			
4	Last	STRING	256			
5	Mail_P	STRING	256			
6	Mail_S	STRING	256			
7	Tel_M	STRING	256			
8	Tel_O	STRING	256			

Rows: A table with columns: No, ID, First, Middle, Last, Mail_P, Mail_S, Tel_M, Tel_O. It contains 8 rows of data:

No	ID	First	Middle	Last	Mail_P	Mail_S	Tel_M	Tel_O
1	JOHNSON	John		Doe	john@email.com	john@email.org	1012345678	
2	JJ	Jane		Roe	jane@email.com		1056781234	256781234
3	jaspers	Dick	Fitzgerald	Public	dick@email.com	dick@email.org	1012345678	212345678
4	son	Robert	Walker	Smith	robert@email.com	robert@email.org		256781234
5	Daniel	Jack		Robertson	jack@email.com		1012345678	212345678
6	BILLBILL	Billy		Cook	bill@email.com	bill@email.org	1056781234	
7	mixnut	Elizabeth		Soap	elizabeth@email.com	elizabeth@email.org	1012345678	
8	sync	Harry	Hussein	Blow	harry@email.com		1056781234	256781234

At the bottom, there are tabs for 'Dataset' and 'Source'.

3 Grid 포맷 만들기

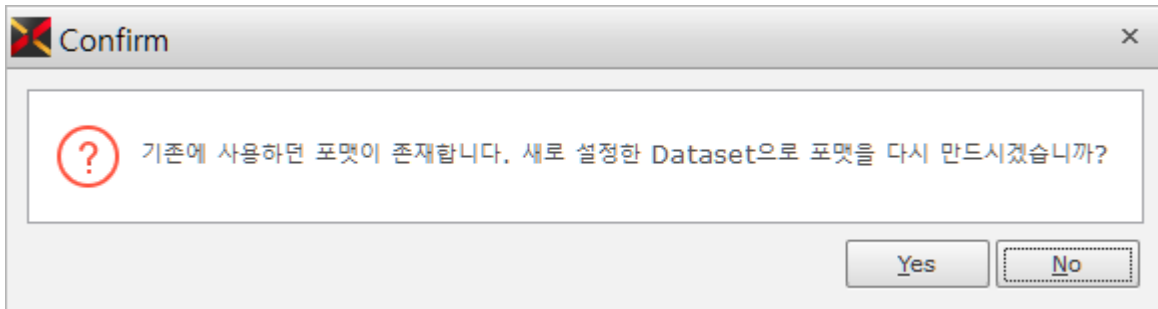
그리드를 더블 클릭하여 Grid Contents Editor를 열어 아래와 같이 Column과 Row를 구성합니다.

		body				
		col0	col1	col2	col3	col4
head	row0	ID		Contact	Mail	Primary
	row1	Name	First			Secondary
	row2		Middle		Tel.	Mobile
	row3		Last			Office
body	row4			Contact	Mail	
	row5	Name				
	row6				Tel.	
	row7					

4 Dataset 바인딩

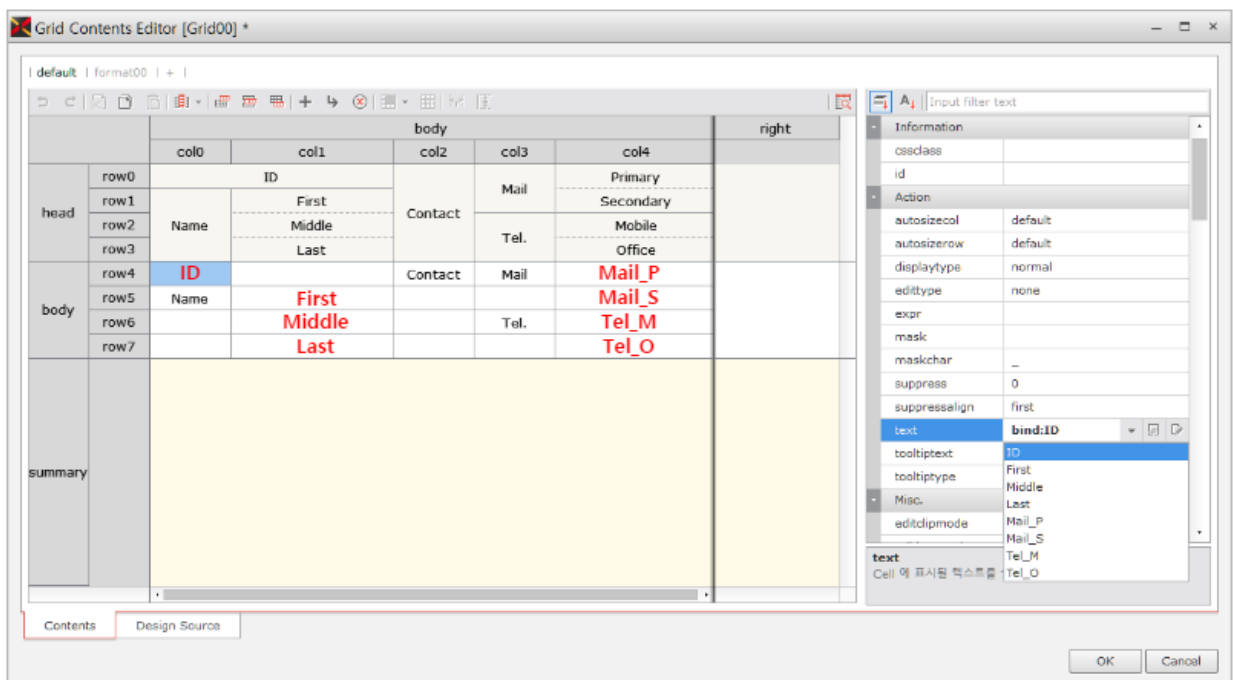
1. 그리드에 데이터셋을 바인딩합니다.

그리드를 선택한 후 Properties 창에서 binddataset 속성에 Dataset00을 바인딩합니다. 바인딩을 시도하면 데이터셋에 맞게 포맷을 재구성하려 시도하기 때문에 아래와 같은 Confirm 창이 뜹니다. 그러나 우리는 앞서 별도의 포맷을 사용하기 위해 만들어 냈기 때문에 No를 선택합니다.



2. 셀에 데이터를 바인딩합니다.

그리드에 데이터셋을 바인딩한 상태에서 Grid Contents Editor를 열어 body 밴드에 속한 각 셀의 text 속성에 데이터를 바인딩합니다. 아래 빨간 글씨에 있는 위치의 셀을 선택하여 글씨에 해당하는 값을 text 속성에서 선택합니다.



5 셀 병합하기

아래와 같이 셀을 병합합니다. 빨간색으로 표시된 부분은 A big cell 방식으로 병합하고 파란색으로 표시된 부분은 Having child cells 방식으로 병합합니다.

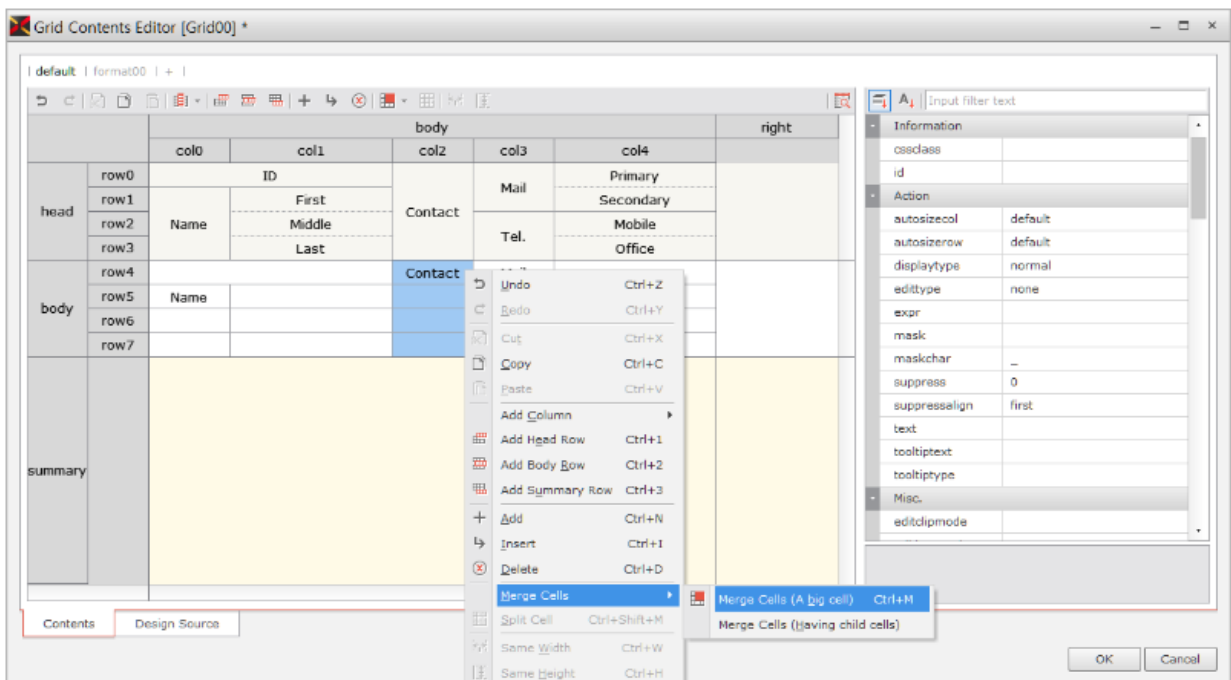
		body				
		col0	col1	col2	col3	col4
head	row0	ID		Contact	Mail	Primary
	row1	Name	First			Secondary
	row2		Middle		Tel.	Mobile
	row3		Last			Office
body	row4			Contact	Mail	
	row5	Name				
	row6				Tel.	
	row7					

셀 병합은 Grid Contents Editor에서 병합하고자 하는 셀을 마우스 드래그로 선택한 후 마우스 오른쪽 버튼을 눌러 컨텍스트 메뉴에서 [Merge Cells]을 선택합니다. Merge Cells를 선택하면 세부 메뉴를 선택할 수 있는데 Merge Cells(A big cell)과 Merge Cells(Having child cells) 중에 원하는 병합 방식을 선택합니다.

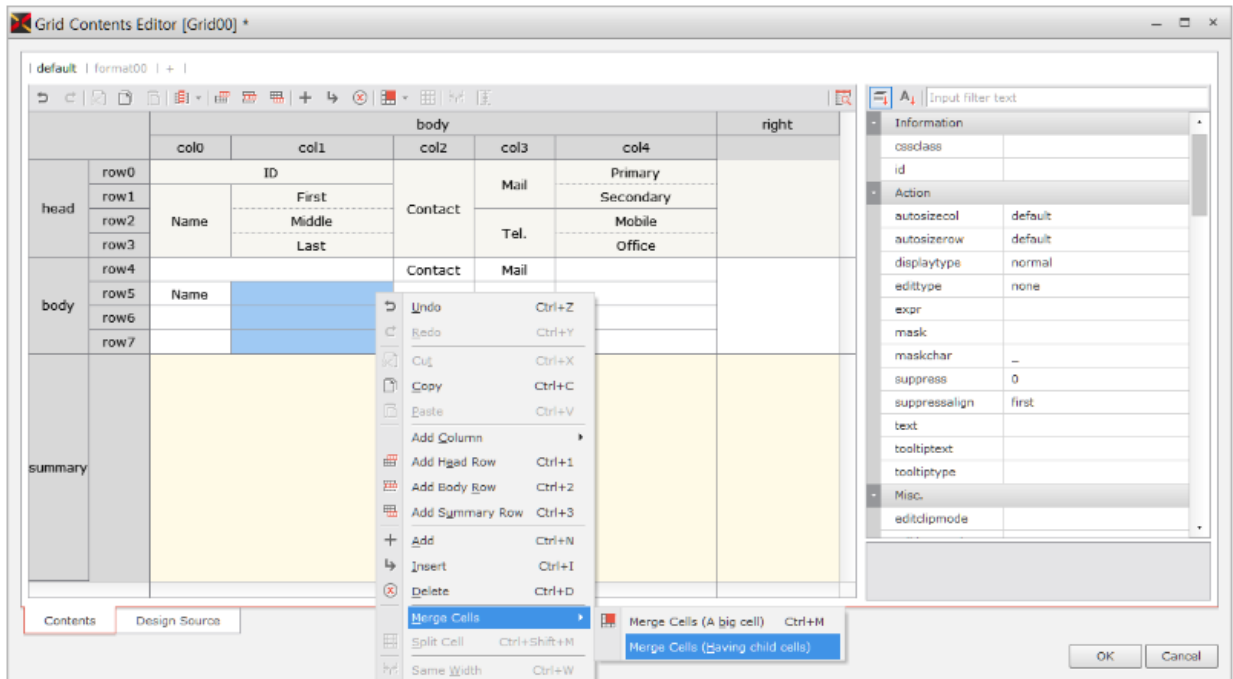
병합이 잘못되었을 경우 Split Cell 메뉴로 병합을 풀 수 있습니다. Split Cell은 병합된 셀을 선택한 후 마우스 오른쪽 버튼을 눌러 컨텍스트 메뉴에서 [Split Cell]을 선택합니다.

병합이 모두 완료되면 OK 버튼을 눌러 Grid Contents Editor를 닫습니다.

- A big cell 병합 방법



- Having child cells 병합 방법



6 getSubCellProperty 기능 구현

- Body 밴드에서 First 컬럼이 바인딩된 셀의 background 속성값을 읽어와 팝업으로 출력합니다. First 컬럼이 바인딩된 셀은 5번 셀의 0번 서브 셀입니다.
- getCellProperty 버튼의 onclick 이벤트 함수

```
this.btn_getCellProperty_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var varProperty = this.Grid00.getSubCellProperty("body", 5, 0, "background");

    alert("Background value of first name sub cell: " + varProperty);
};
```

7 setSubCellProperty 기능 구현

- Body 밴드에서 First 컬럼이 바인딩된 셀의 background 속성값을 khaki로 설정하여 배경색을 변경합니다. First 컬럼이 바인딩된 셀은 5번 셀의 0번 서브 셀입니다.
- setSubCellProperty 버튼의 onclick 이벤트 함수

```
this.btn_setCellProperty_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var bRet = this.Grid00.setSubCellProperty("body", 5, 0, "background", "khaki");
};
```

8 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

우선 setSubCellProperty 버튼을 눌러 First 컬럼의 배경색이 변경되는지 확인합니다. 그런 후 getSubCellProperty 버튼을 눌러 First 컬럼의 배경색 설정값을 확인합니다.

24.11 일부 데이터를 감추어 표시하기

Grid에 표시하는 데이터 중에서 전화번호나 주소 정보와 같이 모든 데이터를 표시하기 곤란할 경우, 일부를 감추어야 할 필요가 있습니다. 표시되는 문자열 일부를 지정해 감추는 방법을 살펴봅니다.

24.11.1 예제

Grid에 표시되는 데이터 중 전화번호 뒷자리 4자리를 "****"로 표시합니다.

Name	Tel
John	233-****
James	748-****
Donald	331-****
Lisa	744-****

 sample_grid_41.xfdl

24.11.2 예제에서 사용한 핵심 기능

maskeditformat

Grid Contents Editor에서 maskeditformat 속성값을 지정하면 원하는 부분을 감추어 표시할 수 있습니다. "{}"문자로 마스크 문자를 감싸면 해당 마스크 문자가 "*" 문자로 대체되어 출력됩니다. 입력되는 문자 일부를 감추어 표시하기 위해 사용합니다.

24.11.3 예제 구현 방법

1 화면 구성하기

화면에 Grid를 배치하고 Dataset 오브젝트를 생성합니다. Dataset 오브젝트의 데이터는 아래와 같이 작성합니다.

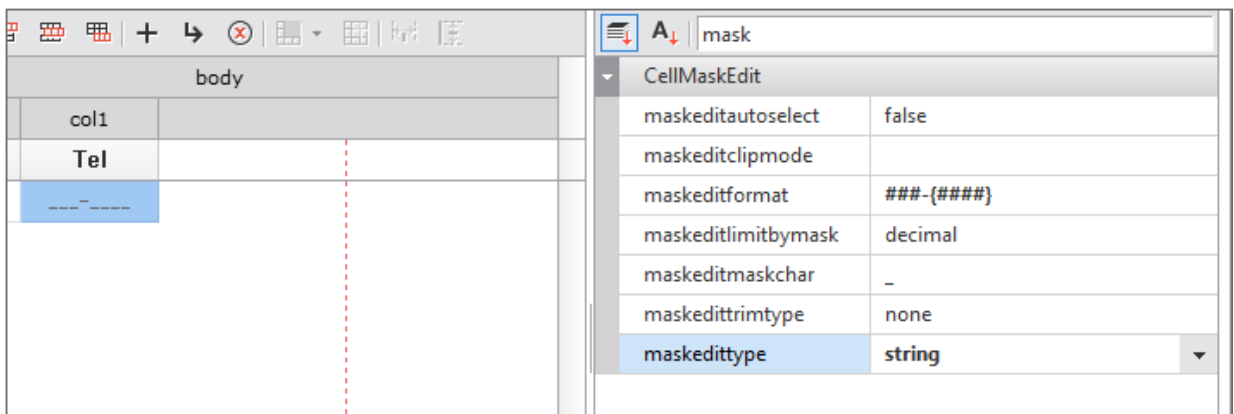
Rows			
No	Name	Tel	
1	John	233-2742	
2	James	748-2483	
3	Donald	331-3256	
4	Lisa	744-3223	

2 데이터 바인딩하기

데이터를 추가한 Dataset 오브젝트를 Grid 컴포넌트에 바인딩합니다. Invisible Objects 영역에 있는 Dataset을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

3 Grid Contents Editor 편집하기

Grid 컴포넌트를 더블 클릭하면 Grid Contents Editor가 실행됩니다. Tel 컬럼을 선택하고 속성창에서 displaytype 속성값은 "mask"로 지정합니다. 그리고 maskedittype 속성값을 "string"으로 지정하고 maskeditformat 속성값은 "###-{####}"으로 지정합니다.



전화번호는 7글자로 입력되어 있는데 그 중에서 앞자리 3글자와 "-" 문자를 제외한 뒷자리 4글자는 감추어 표시한다는 의미입니다.

4 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. 전화번호 뒷자리가 "*" 문자로 표시되는지 확인합니다.

24.12 선택한 Row 전체 컬럼을 입력 상태로 변환하기

Cell 오브젝트의 edittype 속성값을 지정한 경우에는 입력된 값을 편집할 수 있습니다. 하지만 셀을 선택했을때만 입력 상태로 변환되기 때문에 해당 항목을 입력할 수 있는 것인지 명확하게 알 수 없습니다. 그렇다고 Grid 컴포넌트 자체를 편집 상태로 보여주면 가독성이 줄어들 수 있습니다.

이번 예제에서는 특정 Row를 선택했을때 해당 Row 전체 컬럼을 입력 상태처럼 보이도록 변환해서 사용성을 높여주는 방법을 보여줍니다.

24.12.1 예제

특정 Row를 마우스로 클릭하거나 키보드로 이동해서 선택한 경우 해당 Row의 모든 컬럼이 입력 상태로 표시됩니다.

Name	Department	Vacation
John	Sales	1
James	IT	☐
Donald	Education	0
Lisa	Marketing	1

 sample_grid_42.xfdl

24.12.2 예제에서 사용한 핵심 기능

comp.parent

Grid 컴포넌트 Cell 오브젝트에서 expr 속성값을 처리할 때는 Form에서 생성한 함수나 컴포넌트에 직접 접근할 수가 없습니다. expr 속성값을 가지는 오브젝트에만 comp라는 이름으로 접근할 수 있습니다. 그래서 Form에 접근하기 위해서는 comp.parent로 접근합니다.

24.12.3 예제 구현 방법

① Form 화면 구성하기

Grid 컴포넌트를 배치하고 2개의 Dataset 오브젝트를 추가합니다. 하나는 전체 데이터를 담을 것이고, 나머지

하나는 Combo 컨트롤을 처리하기 위해 사용합니다. 첫번째 Dataset은 Grid 컴포넌트에 바인딩합니다.

Dataset Editor [Dataset00]					
▶ Const Columns					
▼ Columns					
No	id	type	size	prop	sun
1	Name	STRING	256		
2	Department	STRING	256		
3	Vacation	STRING	256		
▼ Rows					
No	Name	Department	Vacation		
1	John	01	1		
2	James	02	0		
3	Donald	03	0		
4	Lisa	04	1		

Dataset Editor [Dataset01]				
▶ Const Columns				
▼ Columns				
No	id	type	size	prop
1	code	STRING	256	
2	text	STRING	256	
▼ Rows				
No	code	text		
1	01	Sales		
2	02	IT		
3	03	Education		
4	04	Marketing		
5	05	HR		

2 Cell 속성값 지정하기

첫번째 Dataset을 바인딩한 후 Grid 컴포넌트를 더블클릭해 Grid Content Editor를 실행합니다. 각 컬럼별로 아래와 같이 지정합니다.

```
Name
displaytype: expr:comp.parent.iCurrow==currow?'editcontrol':'normal'
editttype: normal

Department
displaytype: expr:comp.parent.iCurrow == currow ? 'combocontrol' : 'combotext'
editttype: combo
combodataset: Dataset01
combocodecol: code
combodatacol: text

Vacation
displaytype: expr:comp.parent.iCurrow == currow ? 'checkboxcontrol' : 'normal'
editttype: checkbox
```

3 onrowposchanged 이벤트 함수 작성하기

사용자가 선택한 Grid 컴포넌트의 Row 위치가 바뀔 때마다 바인딩된 Dataset 오브젝트의 onrowposchanged 라는 이벤트가 발생합니다. 이때 변경된 위치 정보(newrow)를 확인할 수 있는데 이 정보를 iCurrow라는 변수값으로 담습니다.

```

this.Dataset00_onrowposchanged = function(obj:nexacro.NormalDataset,e:nexacro.
DSRowPosChangeEventInfo)
{
    this.iCurrow = e.newrow;
};

```

이렇게 하면 Cell 오브젝트의 displaytype 속성값에 지정한 expr이 어떻게 동작하는지 설명할 수 있습니다. expr 내에 사용한 currow는 모든 Row의 위치 정보입니다. 즉 사용자가 선택한 Row 위치와 각 Row 위치를 비교하는 것입니다. 조건이 맞는 경우에만 입력 상태처럼 보이도록 displaytype 속성값을 변경합니다.

```
comp.parent.iCurrow == currow?'editcontrol':'normal'
```

4 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 Grid의 선택한 Row를 변경해가면서 입력 상태로 변환되는지 확인합니다.

24.13 PopupDiv에서 선택한 값을 데이터로 입력하기

입력할 항목이 간단한 경우에는 edittype 속성값을 "combo"로 지정하고 목록에서 선택할 수 있지만 항목에 대한 설명이 있거나 복잡한 경우에는 새로운 창을 띄워서 선택하는 것이 편할 수 있습니다. PopupDiv 컴포넌트를 활용해 선택한 값을 입력하는 방법을 살펴봅니다.

24.13.1 예제

셀 영역에 버튼을 클릭하면 PopupDiv 컴포넌트에 배치한 Grid 컴포넌트가 표시되고 항목을 더블클릭해 선택하면 데이터가 입력됩니다.

name	item	
John	walnut	Q
James		Q
Donald		Q
Lisa	pear	Q
Leo		Q

code	item
01	apple
02	pear
03	walnut
04	acorn
05	strawberry
06	tangerine



24.13.2 예제에서 사용한 핵심 기능

screenToClientX , screenToClientY

마우스 이벤트에서 얻을 수 있는 좌표값 중에서 screenx, screeny 값은 사용하고 있는 모니터를 기준으로 표시되는 좌표값입니다. 모니터를 두개 사용하고 있다면 전체 모니터를 기준으로 좌표값이 표시됩니다. 때문에 실제 애플리케이션을 기준으로 표시되는 값을 얻으려면 screenToClientX 또는 screenToClientY 메소드를 사용합니다.

trackPopupByComponent

PopupDiv 컴포넌트는 화면에 표시되지 않다가 trackPopupByComponent 메소드 호출 시 지정된 위치에 보여집니다. 6번째 파라미터는 콜백 함수인데 closePopup 메소드 호출 시 실행되는 함수입니다. 예제에서는 Grid에서 항목 선택 시 PopupDiv 컴포넌트가 닫히면서 콜백 함수를 실행합니다.

closePopup

PopupDiv 컴포넌트를 닫습니다. 이때 파라미터를 지정하면 해당 값이 콜백 함수로 넘겨집니다. 여기에서는 Grid 컴포넌트에서 선택한 값을 넘겨줍니다.

24.13.3 예제 구현 방법

① Form 화면 구성하기

Grid 컴포넌트를 배치하고 Dataset 오브젝트를 추가합니다. Dataset 오브젝트에는 name 컬럼만 항목을 채우고 Item 컬럼은 비워놓습니다. Item 컬럼은 PopupDiv에 있는 Grid 컴포넌트에서 선택한 값을 채웁니다. 데이터를 설정하면 Grid 컴포넌트에 바인딩합니다.

Dataset Editor [Dataset01]			
Const Columns			
Columns			
No	id	type	size
1	name	STRING	256
2	Item	STRING	256
Rows			
No	name	Item	
1	John	[Undefined]	
2	James	[Undefined]	
3	Donald	[Undefined]	
4	Lisa	[Undefined]	
5	Leo	[Undefined]	

2 PopupDiv 화면 구성하기

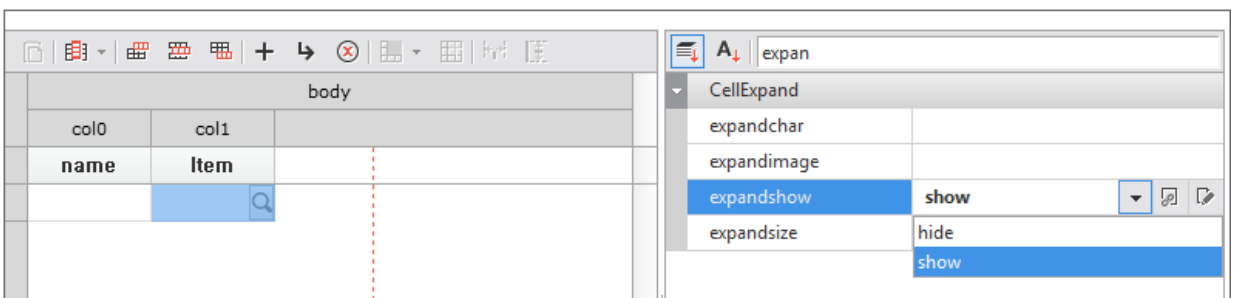
PopupDiv 컴포넌트는 띄울때 위치를 다시 지정해주기 때문에 화면 디자인 시 위치는 아무곳에나 배치해도 상관없습니다. 주로 다른 컴포넌트를 가리지 않도록 주의해 사용하지 않는 영역에 배치합니다. PopupDiv 컴포넌트 안에 Grid 컴포넌트를 배치합니다. 그리고 바인딩할 Dataset 오브젝트를 생성하고 데이터를 채운 후 바인딩해줍니다.

name	Item
John	
James	
Donald	
Lisa	
Leo	

code	item
01	apple
02	pear
03	walnut
04	acorn
05	strawberry
06	tangerine

3 expandshow 속성값 지정하기

셀 영역에 작은 버튼을 추가할 수 있습니다. 원가 확장 기능을 호출할때 주로 사용하는 버튼입니다. expandshow 속성값을 "show"로 변경하면 돋보기 모양의 이미지 버튼이 추가됩니다. 다른 속성값을 설정하면 이미지나 크기를 변경할 수 있습니다.



4 onexpandup 이벤트 함수 작성하기

expandshow 속성값을 변경해 표시된 버튼 클릭 시 실행할 이벤트 함수를 작성합니다. PopupDiv 컴포넌트를 띄운 후 다시 값을 설정해주어야 하기 때문에 column, row 정보를 임시로 저장해놓습니다. 그리고 클릭한 위치에 PopupDiv 컴포넌트를 띄워주기 위해서 screenToClientX, screenToClientY 메소드를 사용해 좌표값을 설정합니다.

```
this.col;
this.row;

this.Grid00_onexpandup = function(obj:nexacro.Grid,e:nexacro.GridMouseEventInfo)
```



```
{
    this.col = e.cell;
    this.row = e.row;
    var nX = system.screenToClientX(this, e.screenx);
    var nY = system.screenToClientY(this, e.screeny);
    this.PopupDiv00.trackPopupByComponent(this, nX, nY, null, null, "call_back");
};
```

5 oncelldblclick 이벤트 함수 작성하기

PopupDiv 컴포넌트 내 배치한 Grid 컴포넌트에서 원하는 항목을 찾아 더블클릭했을 때 이벤트를 처리합니다. 선택한 항목값을 가져와서(getColumn) 콜백함수에 전달합니다.

```
this.PopupDiv00_Grid00_oncelldblclick = function(obj:nexacro.Grid,e:nexacro.
GridClickEventInfo)
{
    var value = this.Dataset00.getColumn(e.row , 1);
    this.PopupDiv00.closePopup(value);
};
```

6 콜백 함수 작성하기

PopupDiv 컴포넌트의 closePopup 메소드 실행 시 콜백 함수가 호출됩니다. onexpandup 이벤트 함수를 처리하면서 임시로 저장해놓았던 column, row 정보와 PopupDiv 컴포넌트가 닫히면서 전달된 값을 컬럼값으로 지정해줍니다.

```
this.call_back = function (strId,str)
{
    this.Dataset01.setColumn( this.row, this.col , str)
};
```

7 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 원하는 위치의 extend 버튼을 클릭하고 PopupDiv 컴포넌트에 표시된 항목을 더블클릭합니다.

24.14 데이터가 없는 경우 Calendar 대신 대체 텍스트 표시하기

문자열만 표시하는 경우에는 데이터가 없을 때 아무것도 표시하지 않으면 되지만 특정 형식을 가지는 데이터인 경우에는 어떤 식으로 표시할지 모호할 수 있습니다. 특히 날짜 형식을 표시하는 경우에는 여러가지 요구사항이 있을 수 있습니다. 이런 경우 데이터를 어떻게 표시하는지 이번 예제에서 살펴봅니다.

24.14.1 예제

데이터가 null인 경우 대체 텍스트를 표시합니다.

text	date
TEST	2018-03-01 Thu
NULL TEST	0000-01-01 Sat
EMPTY TEST	0000-01-01 Sat

text	date
TEST	2018-03-01 Thu
NULL TEST	NULL
EMPTY TEST	- - 



sample_grid_45.xfdl

24.14.2 예제에서 사용한 핵심 기능

calendardisplaynulltext

calendardisplaynulltype 속성값을 "nulltext"로 설정한 경우 대체 텍스트를 지정하는 속성입니다. calendar displaynulltype 속성값을 "nulltext"로 설정하지 않으면 calendardisplaynulltext 속성값이 적용되지 않습니다.

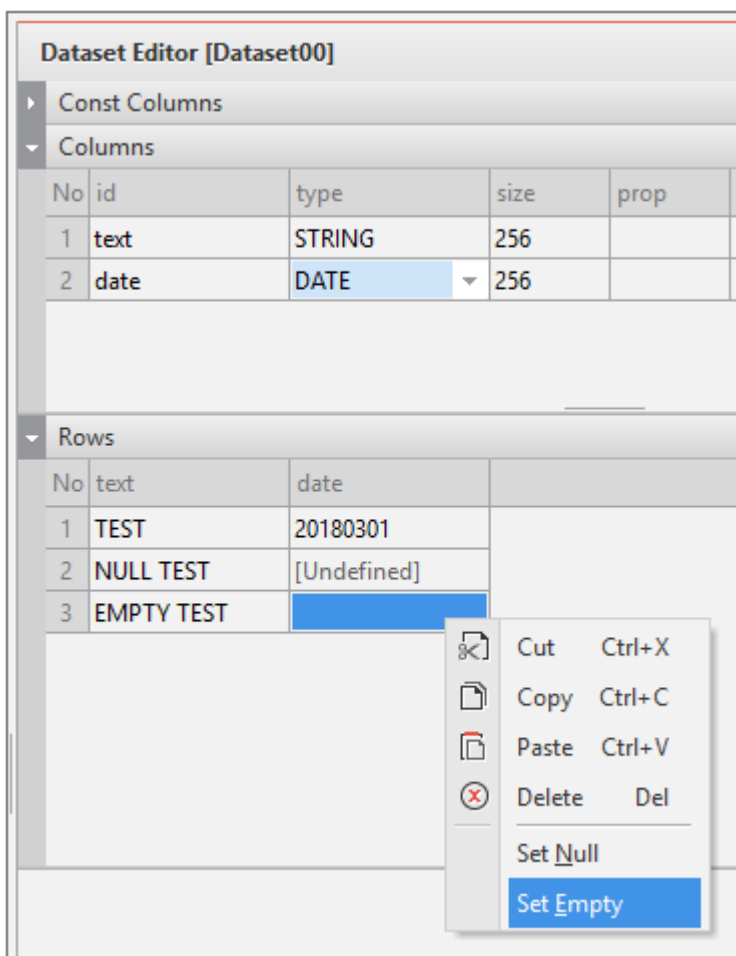
calendardisplaynulltype

속성값이 "default"인 경우에는 "0000/01/01" 형식으로 값을 표시합니다.

24.14.3 예제 구현 방법

① Form 화면 구성하기

Grid 컴포넌트 하나를 먼저 배치합니다. 그리고 바인딩할 Dataset 오브젝트를 아래와 같이 생성하고 바인딩합니다. date 컬럼의 type은 "DATE"로 지정하고 두번째 Row 값은 빈값으로 세번째 Row 값은 Empty 값으로 설정합니다.



Dataset Editor에서 Row의 컬럼을 선택하고 오른쪽 마우스를 클릭한 후 컨텍스트 메뉴에서 "Set Null" 또는 "Set Empty" 항목을 선택하면 NULL 값이나 빈값을 입력할 수 있습니다.

② date 컬럼 displaytype 속성값 설정하기

Grid 컴포넌트를 선택하고 더블클릭해서 Grid Contents Editor를 실행한 후 date 컬럼 Cell 오브젝트의 displ

aytype 속성값과 edittype 속성값을 "date"로 지정합니다.

3 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 빈 항목의 날짜값이 "0000-01-01" 형식으로 표시되는 것을 확인합니다.

text	date
TEST	2018-03-01 Thu
NULL TEST	0000-01-01 Sat
EMPTY TEST	0000-01-01 Sat

4 Grid 컴포넌트를 복사해 배치하기

Grid 컴포넌트를 복사해서 하나 더 생성합니다. 그리고 calendardisplaynulltype 속성값은 "nulltext"로 설정하고 calendardisplaynulltext 속성값은 "NULL"로 지정합니다.

The screenshot shows a grid component in the IDE. The grid has a header row with 'col0' and 'col1'. Below the header, there are rows for 'text' and 'date'. The 'date' column is currently empty, showing '--'. To the right, the properties panel for the 'calendardis' component is visible. It shows the 'CellCalendar' section with two properties: 'calendardisplaynulltext' set to 'NULL' and 'calendardisplaynulltype' set to 'nulltext'.

5 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 빈 항목의 날짜값이 어떻게 표시되는지 확인합니다. 그리고 편집모드에서 어떻게 표시되는지도 확인해봅니다.

24.15 조건에 따라 displaytype 속성값 변경하기

Cell 오브젝트의 특정 속성값을 지정할 때는 setCellProperty 메소드를 사용하는데 이때 expr 속성을 같이 사용하면 조건에 따라 속성값을 지정할 수 있습니다.

24.15.1 예제

버튼을 클릭하면 Column2 항목이 "Y"인 Row의 Column0 항목의 displaytype이 변경됩니다.

Column0	Column1	Column2
☒	test1	Y
☒	test2	Y
1	test3	N
☒	test4	Y
1	test5	N
1	test6	N
☒	test7	Y

Button00



sample_grid_46.xfdl

24.15.2 예제에서 사용한 핵심 기능

setCellProperty

네 번째 파라미터로 설정할 속성값을 지정하는데 이때 expr 속성을 사용할 수 있습니다. 예를 들어 `expr:Column2=='Y'` 와 같은 조건을 추가하면 Column2의 값에 따라 다른 속성값을 지정할 수 있습니다.

24.15.3 예제 구현 방법

① Form 화면 구성하기

Grid 컴포넌트와 Button 컴포넌트를 배치합니다. Dataset 오브젝트를 하나 추가하고 아래와 같이 데이터값을 설정합니다. 그리고 Dataset 오브젝트를 Grid 컴포넌트에 바인딩해줍니다.

Dataset Editor [Dataset00]				
▶ Const Columns				
▼ Columns				
No	id	type	size	prop
1	Column0	STRING	256	
2	Column1	STRING	256	
3	Column2	STRING	256	
Rows				
No	Column0	Column1	Column2	
1	1	test1	Y	
2	1	test2	Y	
3	1	test3	N	
4	1	test4	Y	
5	1	test5	N	
6	1	test6	N	
7	1	test7	Y	

2 Button 컴포넌트의 onclick 이벤트 함수 작성하기

Button 컴포넌트를 클릭했을 때 Cell 오브젝트의 속성을 변경합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Grid00.setCellProperty("body", 0, "displaytype",
        "expr:Column2=='Y'?checkboxcontrol:'normal');
    this.Grid00.setCellProperty("body", 0, "editttype",
        "expr:Column2=='Y'?checkbox:'none')
};
```

3 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭하고 첫번째 컬럼의 표시 형식이 변경되는 것을 확인합니다.

24.16 컬럼값이 "Y", "N"인 경우 checkboxcontrol 사용하기

Grid 컴포넌트 내에서 체크박스를 사용할때는 데이터값이 1, 0으로 설정되어야 합니다. 하지만 원본 데이터는 다양한 형태로 들어올 수 있습니다. 이런 경우 어떻게 체크박스의 표시를 제어할 수 있는지 살펴봅니다.

24.16.1 예제

원본 데이터는 "Y", "N"으로 체크되어 있지만, 체크박스가 표시됩니다. 체크박스의 체크 상태를 바꾸면 원본 데이터의 값이 "Y" 또는 "N"으로 입력됩니다.

Checkbox	Original Data
☒	Y
☐	N
☐	N
☒	Y

 [sample_grid_50.xfdl](#)

24.16.2 예제에서 사용한 핵심 기능

oncolumnchanged

Dataset 오브젝트의 컬럼 값이 변경되는 경우 발생하는 이벤트입니다. 예제에서는 체크박스의 상태가 변경될 때 1 또는 0이라는 값이 설정되는데 이를 강제적으로 "Y" 또는 "N"으로 변경합니다.

24.16.3 예제 구현 방법

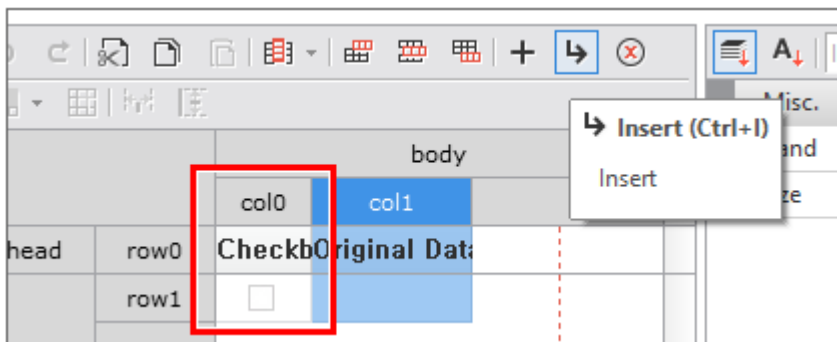
① Form 화면 구성하기

Grid 컴포넌트를 배치합니다. Dataset 오브젝트를 하나 추가하고 아래와 같이 데이터값을 설정합니다. 그리고 Dataset 오브젝트를 Grid 컴포넌트에 바인딩해줍니다.

Dataset Editor [Dataset00]			
▶ Const Columns			
▼ Columns			
No	id	type	size
1	Column0	STRING	256
▼ Rows			
No	Column0		
1	Y		
2	N		
3	N		
4	Y		

2 Grid Contents Editor 실행하기

Grid 컴포넌트를 더블클릭해 Grid Contents Editor를 실행합니다. 컬럼을 하나 추가합니다. 이때 [Add]가 아니라 [Insert]를 사용하면 선택된 컬럼 앞쪽에 새로운 컬럼을 추가할 수 있습니다.



head 영역의 셀을 선택하고 text 속성값을 수정해줍니다. 그리고 추가한 첫번째 body 셀을 선택하고 아래와 같이 설정합니다. 처음 바인딩한 컬럼과 마찬가지로 Column0의 데이터를 사용합니다. 하지만 expr 속성에서 데이터에 따라 값을 1 또는 0으로 변환해줍니다.

body		cssclass	
col0	col1	id	body0
CheckbOriginal Data		▼ Action	
☐		autosizecol	default
		autosizerow	default
		controlautosizingtype	both
		displaytype	checkboxcontrol
		edittype	checkbox
		expr	expr:Column0=='Y'?1:'0'
		suppress	0
		suppressalign	middle
		text	bind:Column0

3 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 Grid 컴포넌트의 데이터가 원하는 형태로 표시되는지 확인합니다. 하지만 체크박스의 상태를 변경하면 두번째 컬럼이 "Y"가 아니라 1로 표시되는 것을 확인할 수 있습니다. 그리고 체크박스의 상태도 변경되지 않습니다.

Checkbox	Original Data
☒	Y
☐	1
☐	N
☒	Y

4 oncolumnchanged 이벤트 함수 생성하기

체크박스 영역의 상태를 변경했을때 정상적으로 데이터를 처리하기 위해 oncolumnchanged 이벤트를 추가하고 이벤트 함수를 작성합니다. enableevent 속성값을 제어하는 것은 대용량의 데이터 처리시에 권장하고 있습니다. 아래 예제처럼 1건의 데이터를 변경하는 경우에는 큰 차이가 없을 수 있습니다.

```
this.Dataset00_onscolumnchanged = function(obj:nexacro.NormalDataset,e:nexacro.DSColChangeEventInfo)
{
    if(e.columnid == "Column0")
    {
        obj.set_enableevent(false);
        if(e.newvalue == '1')
        {
            obj.setColumn(e.row,"Column0",'Y');
        } else if(e.newvalue == '0')
        {
            obj.setColumn(e.row,"Column0",'N');
        }
        obj.set_enableevent(true);
    }
};
```

5 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 Grid 컴포넌트의 데이터가 원하는 형태로 표시되고 변경되는지 확인합니다.

24.17 URL 링크 클릭 시 해당 사이트로 이동하기

바인딩된 데이터에 URL 링크가 포함된 경우 Grid 컴포넌트 클릭 시 해당 사이트로 이동하는 예제입니다.

24.17.1 예제

화면에 보이지 않는 bLink 컬럼값이 "Y"인 경우에는 url 컬럼에 밑줄이 표시되고 마우스 커서 모양이 변경됩니다. 해당 셀 클릭 시 지정된 사이트로 이동합니다.

site	url
codeschool	http://codeschool.com
codecademy	https://www.codecademy.com
khanacademy	-
udacity	https://www.udacity.com



24.17.2 예제에서 사용한 핵심 기능

displaytype

displaytype 속성값을 "decoratetext"로 지정하면 줄바꿈이나 Decorate 태그를 사용할 수 있습니다. 지정된 태그가 없는 경우에는 텍스트를 그대로 표시합니다.

execBrowser

지정된 URL 경로를 웹브라우저에서 실행합니다. 웹브라우저에서 메소드가 실행된 경우에는 해당 웹브라우저의 새탭 또는 새창으로 실행합니다.

24.17.3 예제 구현 방법

① Form 화면 구성하기

Grid 컴포넌트를 배치합니다. Dataset 오브젝트를 하나 추가하고 아래와 같이 데이터값을 설정합니다. 그리고

Dataset 오브젝트를 Grid 컴포넌트에 바인딩해줍니다.

Const Columns					
Columns					
No	id	type	size	prop	su
1	site	STRING	256		
2	url	STRING	256		
3	bLink	STRING	256		
Rows					
No	site	url	bLink		
1	codeschool	http://codescho...	Y		
2	codecademy	https://www.co...	Y		
3	khanacademy	-	N		
4	udacity	https://www.ud...	Y		

2 Grid Content Editor 실행하기

bLink 컬럼은 사용하지 않으니 삭제합니다. url 컬럼 항목을 선택하고 displaytype 속성값을 "decorationtext"로 지정하고 expr 속성값을 아래와 같이 지정합니다. bLink 컬럼값이 "Y"인 경우에는 Decorate 태그를 추가하고 그렇지 않은 경우에는 그냥 url 컬럼값을 사용합니다.

```
bLink=='Y'?"<u v='true'>" + url + "</u>":url
```

col1		displaytype	decorationtext
url		edittype	none
		expr	expr:bli
			0
			first
			bind:url

cssclass 속성값도 아래와 같이 설정합니다.

```
cssclass: expr:'bLink=='Y'?'sample_grid_53_cursor':''
```

3 xcss 파일을 열어서 아래 코드를 추가합니다. Advanced 탭에서 선택자를 추가하거나 Text 탭에서 직접 코드 값을 입력할 수 있습니다.

```
.Grid .body .row .cell.sample_grid_53_cursor[status=mouseover]
{
    cursor : pointer;
}
```

④ oncellick 이벤트 함수 작성하기

셀 클릭 시 해당 사이트로 이동하는 기능을 추가합니다. bLink 컬럼값이 "Y"인 경우에만 이동하도록 조건을 추가합니다.

```
this.Grid00_oncellick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    if( e.cell == 1 ){
        if( this.Dataset00.getColumn(e.row, "bLink") == "Y")
            system.execBrowser(this.Dataset00.getColumn(e.row, 1));
    }
};
```

⑤ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 URL 클릭 시 해당 사이트로 이동하는지 확인합니다.

24.18 expr 내에서 사용할 수 있는 속성

expr 내에서 사용할 수 있는 속성을 정리해봅니다.

24.18.1 예제

expr 속성값에 지정된 항목을 Grid 컴포넌트에 표시합니다. 선택된 cell을 변경하면 표시되는 항목이 바뀝니다.

Column0	Column1	Column2
this.col: 0 dataset.rowcount: 5 dataset.rowposition: 1 comp.currentcell: 1 currow: 0	this.col: 1 dataset.rowcount: 5 dataset.rowposition: 1 comp.currentcell: 1 currow: 0	this.col: 2 dataset.rowcount: 5 dataset.rowposition: 1 comp.currentcell: 1 currow: 0
this.col: 0 dataset.rowcount: 5 dataset.rowposition: 1 comp.currentcell: 1 currow: 1	this.col: 1 dataset.rowcount: 5 dataset.rowposition: 1 comp.currentcell: 1 currow: 1	this.col: 2 dataset.rowcount: 5 dataset.rowposition: 1 comp.currentcell: 1 currow: 1
this.col: 0 dataset.rowcount: 5 dataset.rowposition: 1 comp.currentcell: 1 currow: 2	this.col: 1 dataset.rowcount: 5 dataset.rowposition: 1 comp.currentcell: 1 currow: 2	this.col: 2 dataset.rowcount: 5 dataset.rowposition: 1 comp.currentcell: 1 currow: 2



24.18.2 예제에서 사용한 핵심 기능

currentcell

선택된 Cell 인덱스를 표시합니다. Row 단위로 선택하는 경우에는 최초 선택 시 인덱스 값을 표시하기 때문에 예제에서는 Grid 컴포넌트의 selecttype 속성값을 "cell"로 지정합니다.

24.18.3 예제 구현 방법

1 Form 화면 구성하기

Grid 컴포넌트를 배치합니다. Dataset 오브젝트를 하나 추가하고 아래와 같이 데이터값을 설정합니다. 그리고 Dataset 오브젝트를 Grid 컴포넌트에 바인딩해줍니다.

Columns				
No	id	type	size	prop
1	Column0	STRING	256	
2	Column1	STRING	256	
3	Column2	STRING	256	
Rows				
No	Column0	Column1	Column2	
1	[Undefined]	[Undefined]	[Undefined]	
2	[Undefined]	[Undefined]	[Undefined]	
3	[Undefined]	[Undefined]	[Undefined]	
4	[Undefined]	[Undefined]	[Undefined]	
5	[Undefined]	[Undefined]	[Undefined]	

② expr 속성 설정하기

Grid 컴포넌트를 더블 클릭해 Grid Contents Editor를 실행하고 각 컬럼의 expr 속성값을 아래와 같이 지정합니다.

body			cssclass	
col0	col1	col2	id	body0
Column0	Column1	Column2	Action	
			autosizecol	default
			autosizerow	default
			controlautosizingtype	both
			displaytype	normal
			edittype	none
			expr	expr:"this.col: "+
			suppress	0

```
"this.col: "+this.col+"\n"+"dataset.rowcount: "+dataset.rowcount+"\n"+"dataset.
rowposition: "+dataset.rowposition+"\n"+"comp.currentcell: "+comp.currentcell+"\n"+"
currow: "+currow
```

③ Grid 컴포넌트 속성 설정하기

데이터를 모두 표시하기 위해 autofittype 속성값은 "col", autosizingtype 속성값은 "row"로 지정합니다. 그리고 currentcell 속성값이 잘 표현될 수 있도록 selecttype 속성값을 "cell"로 수정합니다.

④ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 Grid 컴포넌트에 표시되는 데이터를 확인합니다. col, rowcount, currow 속성값은 고정되어 표시되고 rowposition, currentrow 속성값은 선택된 cell에 따라 값이 바뀝니다.

24.19 스크립트로 suppress 속성 적용하기

스크립트 내에서 동적으로 suppress 속성을 적용할 수 있습니다.

24.19.1 예제

Column0, Column1 head 영역을 클릭하면 suppress가 동작합니다. 버튼을 클릭하면 초기 상태로 돌아갑니다.

Column0	Column1	Column2
1	1-1	A
		B
	1-2	C
	1-3	D
2	2-1	E
		F
	2-2	G
	2-3	H
		I
		J

Button00

 [sample_grid_58.xfdl](#)

24.19.2 예제에서 사용한 핵심 기능

suppress

Cell 오브젝트의 속성값을 지정하는 경우에는 setCellProperty 메소드를 사용합니다. 예제에서는 body 영역의 Column0, Column1에 해당하는 suppress 속성값을 변경합니다.

24.19.3 예제 구현 방법

① Form 화면 구성하기

Grid 컴포넌트를 배치합니다. Dataset 오브젝트를 하나 추가하고 아래와 같이 데이터값을 설정합니다. 그리고 Dataset 오브젝트를 Grid 컴포넌트에 바인딩해줍니다.

Columns				
No	id	type	size	prop
1	Column0	STRING	256	
2	Column1	STRING	256	
3	Column2	STRING	256	
Rows				
No	Column0	Column1	Column2	
1	1	1-1	A	
2	1	1-1	B	
3	1	1-2	C	
4	1	1-3	D	
5	2	2-1	E	
6	2	2-1	F	
7	2	2-2	G	
8	2	2-3	H	
9	2	2-3	I	
10	2	2-3	J	

② onheadclick 이벤트 함수 작성하기

Grid 컴포넌트의 head 영역 클릭 시 해당 컬럼의 suppress 속성값을 변경합니다. 첫번째 컬럼을 클릭한 경우에는 첫번째 컬럼만, 두번째 컬럼을 클릭한 경우에는 첫번째와 두번째 컬럼의 suppress 속성값을 변경합니다.

```
this.Grid00_onheadclick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    var sCol = e.col;
    if(sCol==0)
    {
        this.Grid00.setCellProperty("Body", 0, "suppress", 1);
    } else if(sCol==1)    {
        this.Grid00.setCellProperty("Body", 0, "suppress", 1);
        this.Grid00.setCellProperty("Body", 1, "suppress", 2);
    }
};
```

③ onclick 이벤트 함수 작성하기

버튼 클릭 시 suppress 상태를 초기화합니다.


```

this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Grid00.setCellProperty("Body", 0, "suppress", 0);
    this.Grid00.setCellProperty("Body", 1, "suppress", 0);
};

```

④ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 Grid 컴포넌트의 head 영역을 클릭해 suppress 상태가 변경되는 것을 확인합니다.

24.20 데이터가 없는 영역을 채우기

데이터 숫자가 동적으로 변경되는 경우에는 Grid 컴포넌트의 크기가 고정된 경우 데이터가 없는 영역은 아무것도 표시되지 않습니다. 이런 경우 빈 Row를 보여주는 방법을 설명합니다.

24.20.1 예제

Combo 컴포넌트의 항목 선택에 따라 7번째 Row 아래의 표시 방법이 변경됩니다.

index	Column0	Column1	Column2
TEST	☐	1	1
TEST	☒	2	2
TEST	☒	3	3
TEST	☐	4	4
TEST	☐	5	5
TEST	☒	6	6
TEST	☐		
TEST	☐		
TEST	☐		
TEST	☐		

allow ▼

24.20.2 예제에서 사용한 핵심 기능

fillareatype

Grid 컴포넌트에서 Row가 표시되지 않는 영역을 채우는 방식을 지정합니다. 예제에서는 Combo 컴포넌트에서 속성값 선택 시 반영하도록 했습니다.

24.20.3 예제 구현 방법

① Form 화면 구성하기

Grid 컴포넌트를 배치합니다. Dataset 오브젝트를 하나 추가하고 아래와 같이 데이터값을 설정합니다. 그리고 Dataset 오브젝트를 Grid 컴포넌트에 바인딩해줍니다.

Columns				
No	id	type	size	prop
1	Column0	STRING	256	
2	Column1	STRING	256	
3	Column2	STRING	256	

Rows			
No	Column0	Column1	Column2
1	0	1	1
2	1	2	2
3	1	3	3
4	0	4	4
5	0	5	5
6	1	6	6

② 컬럼 추가하기

fillareatype 속성값을 "datarow" 또는 "allrow"로 설정했을때 보여지는 상수 영역을 표시하기 위해 컬럼 하나를 추가합니다. Grid 컴포넌트를 더블클릭해 Grid Contents Editor를 실행하고 컬럼을 추가합니다.

body			
	col0	col1	
row0	index	Column0	Column1
row1	TEST	☐	

추가한 컬럼의 text 속성값은 "TEST"라는 고정된 문자열을 지정합니다. 이렇게 하면 첫번째 컬럼은 항상 "TEST"라는 문자열로 표시됩니다. Column0 항목의 displaytype 속성값은 "checkboxcontrol"으로 지정합니다.

3 Combo 컴포넌트 배치하고 설정하기

fillareatype 속성값을 Combo 컴포넌트에서 설정합니다. Combo 컴포넌트를 배치하고 속성값을 innerDataset 항목값으로 지정합니다.

InnerDataset		
No	codecolumn	datacolumn
1	none	none
2	linerow	linerow
3	datarow	datarow
4	controlrow	controlrow
5	allrow	allrow

onitemchanged 이벤트 함수에 선택된 Combo 컴포넌트의 값을 fillareatype 속성값으로 지정하는 코드를 추가합니다.

```
this.Combo00_onitemchanged = function(obj:nexacro.Combo,e:nexacro.ItemChangeEventInfo)
{
    this.Grid00.set_fillareatype(e.posttext);
};
```

4 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 Grid 컴포넌트의 head 영역을 클릭해 suppress 상태가 변경되는 것을 확인합니다.

24.21 특정 Row를 깜빡이게 표시하기

타이머와 cssclass 속성값을 사용해 특정 Row를 깜빡이는 것처럼 표시합니다. blinkCell, blinkCellByInterval 메소드를 사용하는 방법도 같이 살펴봅니다.

24.21.1 예제

Grid 컴포넌트에 표시되는 두번째 Row의 flag 컬럼값이 바뀌면서 배경색이 깜빡이는 것처럼 표시됩니다. [blinkCell] 버튼 클릭 시에는 blinkCell, blinkCellByInterval 메소드를 실행해 배경색이 깜빡이는 것처럼 표시합니다.

label	level	flag	blinkCell
A	0	0	blinkCell
B	1	0	
C	2	2	
D	2	3	



24.21.2 예제에서 사용한 핵심 기능

cellexprupdatecondition

expr값이 변경되었을때 값을 갱신하는 범위를 지정합니다. 기본값은 "all"입니다. 특정 Cell에 해당하는 expr 값이 바뀌면 다른 Row, Cell을 같이 갱신합니다. 하지만 예제처럼 특정 영역의 배경색만 바꾸고 나머지 영역에는 영향을 미치지 않는다면 속성값을 "none"으로 설정합니다. 설정에 따라 애플리케이션의 성능에 영향을 미칠 수 있습니다.

blinkCell (17.0.0.600 버전부터 지원)

특정 Row의 지정된 컬럼 항목에 깜빡임 효과를 처리합니다. 지정된 시간동안 blinked userstatus가 적용되고 userstatus가 해제되는 것을 반복합니다.

blinkCellByInterval (17.0.0.1000 버전부터 지원)

특정 Row의 지정된 컬럼 항목에 깜빡임 효과를 처리합니다. 500밀리초동안 blinked userstatus가 적용되고 지정된 시간동안 userstatus가 해제되는 것을 반복합니다.

24.21.3 예제 구현 방법

① Form 화면 구성하기

Grid 컴포넌트와 Button 컴포넌트를 배치합니다. Dataset 오브젝트를 하나 추가하고 아래와 같이 데이터값을

설정합니다. 그리고 Dataset 오브젝트를 Grid 컴포넌트에 바인딩해줍니다.

Columns					
No	id	type	size	prop	sumtext
1	label	STRING	256		
2	level	STRING	256		
3	flag	STRING	256		
Rows					
No	label	level	flag		
1	A	0	0		
2	B	1	0		
3	C	2	0		
4	D	2	0		

2. cssclass 속성값 설정하기

cssclass 속성값을 아래와 같이 지정합니다. 이때 body row에 해당하는 모든 컬럼을 다 선택해주면 한번에 속성값을 지정할 수 있습니다.

```
expr:comp.parent.setblink(currow)
```

body			
col0	col1	col	
label	level	fla	

Information	
cssclass	expr:comp.parent.setblink(currow)
id	
Action	
autosizecol	default
autosizerow	default

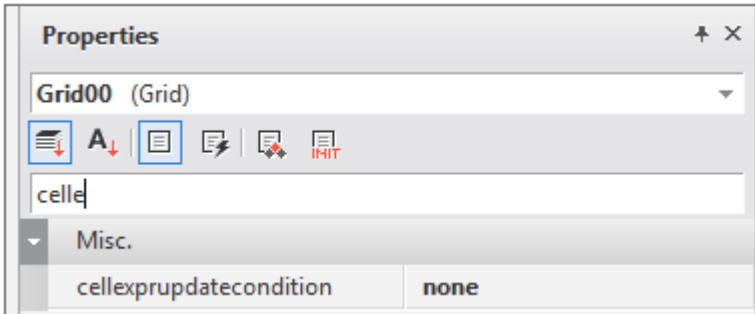
속성창에서 값은 한번만 지정해주지만 실제 값은 각 컬럼 속성으로 들어갑니다.

```
<Band id="body">
  <Cell text="bind:label" cssclass="expr:comp.parent.setblink(currow)"/>
  <Cell col="1" text="bind:level" cssclass="expr:comp.parent.setblink(currow)"/>
  <Cell col="2" text="bind:flag" cssclass="expr:comp.parent.setblink(currow)"/>
</Band>
```

3. cellexprupdatecondition 속성값 설정하기

Grid 컴포넌트를 선택하고 cellexprupdatecondition 속성값을 설정합니다. Grid 컴포넌트같은 경우에는 사

용할 수 있는 속성이 많기 때문에 검색 기능을 사용하면 좀 더 빠르게 찾을 수 있습니다. 해당 속성을 찾아서 "none"으로 값을 지정합니다.



④ onload, ontimer 이벤트 함수 작성하기

타이머 기능을 사용해 1초마다 배경색이 바뀌도록 설정합니다. 타이머 이벤트가 발생하면 flag 컬럼값을 1에서 0으로 또는 0에서 1로 변경합니다. 컬럼값이 바뀌면 setblink 함수에서 반환하는 값도 바뀌게 되고 각 Cell의 cssclass 속성값이 바뀌게 됩니다.

```
this.styleArr = ["sample_grid_62_blink2", "sample_grid_62_blink1", "sample_grid_62_blinkcell"];
this.idx = 0 ;
this.blinkRow = 0 ;

this.sample_grid_62_onload = function(obj:nexacro.Form,e:nexacro.LoadEventInfo)
{
    this.setTimer(1, 1000);
};

this.sample_grid_62_ontimer = function(obj:nexacro.Form,e:nexacro.TimerEventInfo)
{
    var v = this.Dataset00.getColumn(e.timerid,2)^1;
    this.Dataset00.setColumn(e.timerid, 2, v);

    this.idx = v;
    this.blinkRow = e.timerid;
};
```



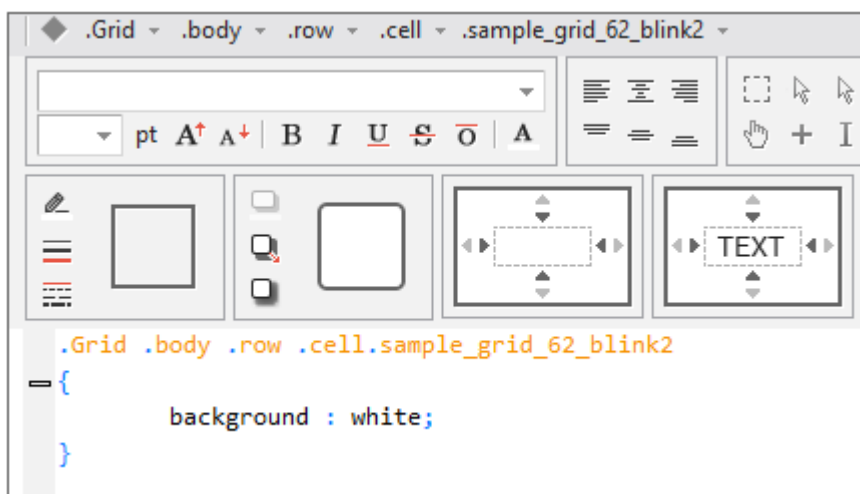
var v = this.Dataset00.getColumn(e.timerid,2)^1 에서 사용하는 연산자 "^"는 '비트단위 배타적 논리합' 연산자입니다. 비트단위 연산을 위해 사용하는 연산자인데 예제에서는 컬럼값에 따라 0 또는 1을 처리하기 위해 사용했습니다. 해당 연산자 대신 IF 문을 사용하거나 SWITCH 문을 사용해도 무관합니다.

setblink 함수는 앞에서 cssclass 속성값에 expr로 작성한 함수입니다. 세번째와 네번째 Row인 경우에는 blinkCell, blinkCellByInterval 메소드를 실행하기 위해 cssclass 속성값이 "sample_grid_62_blinkcell"로 지정되도록 합니다.

```
this.setblink = function (nRow)
{
    if(nRow >= 2)
    {
        return this.styleArr[2];
    }
    else if(this.blinkRow == nRow)
    {
        return this.styleArr[this.idx ];
    }
    return "";
};
```

5 xcss 파일 편집하기

xcss 파일을 열어서 아래 코드를 추가합니다. Advanced 탭에서 선택자를 추가하거나 Text 탭에서 직접 코드 값을 입력할 수 있습니다.



```
.Grid .body .row .cell.sample_grid_62_blink1
{
    background : gold;
}

.Grid .body .row .cell.sample_grid_62_blink2
{
```

```

    background : white;
}

.Grid .body .row .cell.sample_grid_62_blinkcell[userstatus=blinked]
{
    background : gold;
    color : #666666;
}

```

6 onclick 이벤트 함수 작성하기

버튼 클릭 시 blinkCell, blinkCellByInterval 메소드를 실행합니다.

```

this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Grid00.blinkCell(2, "label, level, flag", 5000, 5);
    this.Grid00.blinkCellByInterval(3, "label, level, flag", 500, 5);
};

```

blinkCell 메소드는 Row 인덱스값이 2인 세번째 Row에 깜빡임 효과를 적용합니다. 대상 컬럼은 3개 컬럼 모두이며 5초동안 5번 깜빡입니다. blinkCellByInterval 메소드는 Row 인덱스값이 3인 네번째 Row에 깜빡임 효과를 적용합니다. 대상 컬럼은 3개 컬럼 모두이며 1초 간격으로 깜빡임 효과를 적용합니다.



깜빡이는 횟수는 같지만, blinkCell 메소드는 [normal > blinked > normal...]순으로 동작하며 blinkCellByInterval 메소드는 [blinked > normal > blinked...]순으로 동작하기 때문에 번갈아가면서 2개의 Row가 깜빡이는 것처럼 보여지게 됩니다.

7 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 Grid 컴포넌트의 두번째 Row 배경색이 바뀌는 것을 확인합니다. 버튼을 클릭해서 세번째, 네번째 Row 배경색이 바뀌는 것도 확인합니다.

24.22 데이터 변경 시 Cell 글자색을 바꾸어 표시하기

사용자가 Grid 컴포넌트의 데이터를 바꾸었을때 데이터가 바뀐 것인지 쉽게 알 수 있도록 Cell 스타일을 변경하는 예제입니다.

24.22.1 예제

세 번째 컬럼의 체크박스 상태를 변경하면 첫 번째 컬럼의 글자색을 붉은색으로 표시합니다.

Column0	Column1	Column2
A	TEST	☒
B	TEST	☒
C	TEST	☒



24.22.2 예제에서 사용한 핵심 기능

getOrgColumn

초기 데이터값을 반환합니다. Cell 오브젝트의 edittype 속성값이 지정되어 있고 사용자가 데이터를 수정하면 getOrgColumn 메소드로 반환된 값과 현재 상태의 값과 다를 수 있습니다.

24.22.3 예제 구현 방법

1 Form 화면 구성하기

Grid 컴포넌트를 배치합니다. Dataset 오브젝트를 하나 추가하고 아래와 같이 데이터값을 설정합니다. 그리고 Dataset 오브젝트를 Grid 컴포넌트에 바인딩해줍니다.

Columns					
No	id	type	size	prop	sumtext
1	Column0	STRING	256		
2	Column1	STRING	256		
3	Column2	STRING	256		
Rows					
No	Column0	Column1	Column2		
1	A	TEST	0		
2	B	TEST	1		
3	C	TEST	0		

2 Grid Contents Editor 실행하기

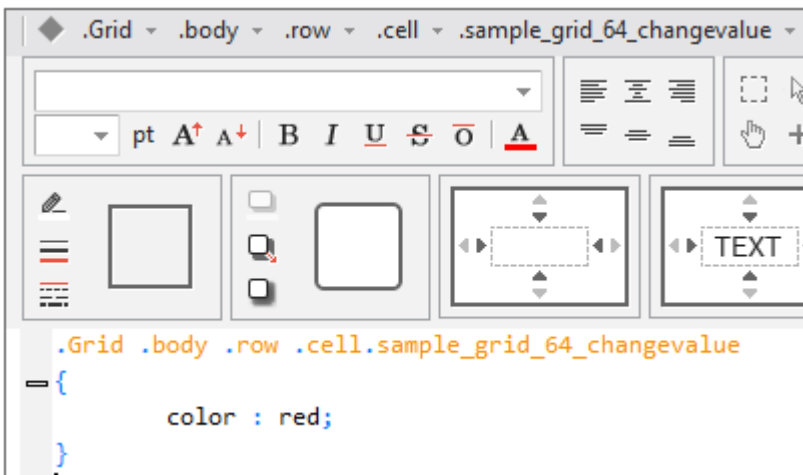
Column2 Cell의 edittype, displaytype 속성값을 수정합니다. edittype은 "checkbox", displaytype은 "checkboxcontrol"입니다. 그리고 Column0 Cell의 cssclass를 아래와 같이 설정합니다.

```
expr:Column2==dataset.getOrgColumn(currow,'Column2')?'':'sample_grid_64_changevalue'
```

Column2의 값이 getOrgColumn 메소드 결과값과 같으면 변경이 없다고 판단해서 cssclass를 지정하지 않고 값이 다르면 cssclass 속성값을 "sample_grid_64_changevalue"로 지정합니다. 이렇게 하면 값이 변경된 경우에만 Column0 Cell의 글자색이 붉은색으로 표시됩니다.

3 xcss 파일 편집하기

xcss 파일을 열어서 아래 코드를 추가합니다. Advanced 탭에서 선택자를 추가하거나 Text 탭에서 직접 코드를 입력할 수 있습니다.



```
.Grid .body .row .cell.sample_grid_64_changevalue
{
    color : red;
}
```

4 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 체크박스의 상태를 변경했을때 Column0의 글자색이 변경되는 것을 확인합니다.

24.23 행번호 표시하기

currow 속성값을 사용해 행번호를 표시하는 방법을 살펴봅니다.

24.23.1 예제

No 컬럼의 행번호가 자동으로 표시됩니다.

No	Column0	Column1
1	apple	A
2	car	A
3	water	B
4	computer	C
5	mouse	A



24.23.2 예제에서 사용한 핵심 기능

currow

Cell 오브젝트의 expr 속성값으로 지정하게 되면 현재 Row index를 반환합니다. 인덱스 값은 0부터 시작하기 때문에 행번호를 표시하기 위해서는 currow+1 계산식을 사용합니다.

24.23.3 예제 구현 방법

① Form 화면 구성하기

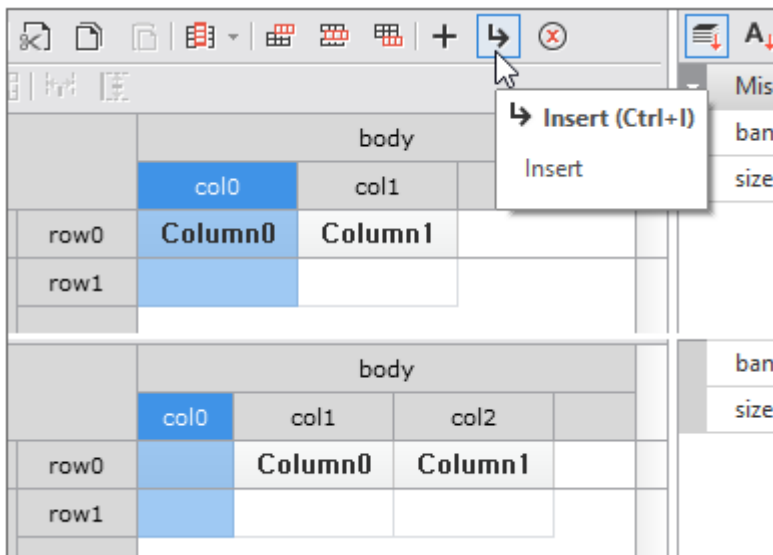
Grid 컴포넌트를 배치합니다. Dataset 오브젝트를 하나 추가하고 아래와 같이 데이터값을 설정합니다. 그리고 Dataset 오브젝트를 Grid 컴포넌트에 바인딩해줍니다.

Columns				
No	id	type	size	prop
1	Column0	STRING	256	
2	Column1	STRING	256	

Rows			
No	Column0	Column1	
1	apple	A	
2	car	A	
3	water	B	
4	computer	C	
5	mouse	A	

2 Grid Contents Editor 실행하기

Column0을 선택하고 [Insert] 메뉴를 선택합니다. 선택한 컬럼 앞에 새로운 컬럼을 추가합니다. 추가된 컬럼 Head row의 text 속성값을 "No"로 입력하고 Body row의 expr 속성값에 currow+1을 입력합니다.



The screenshot shows the Grid Contents Editor interface. A tooltip menu is open over the 'col0' header, showing the 'Insert (Ctrl+I)' option. The grid structure is as follows:

	col0	col1	
row0	Column0	Column1	
row1			

	col0	col1	col2	
row0		Column0	Column1	
row1				

3 QuickView로 확인하기

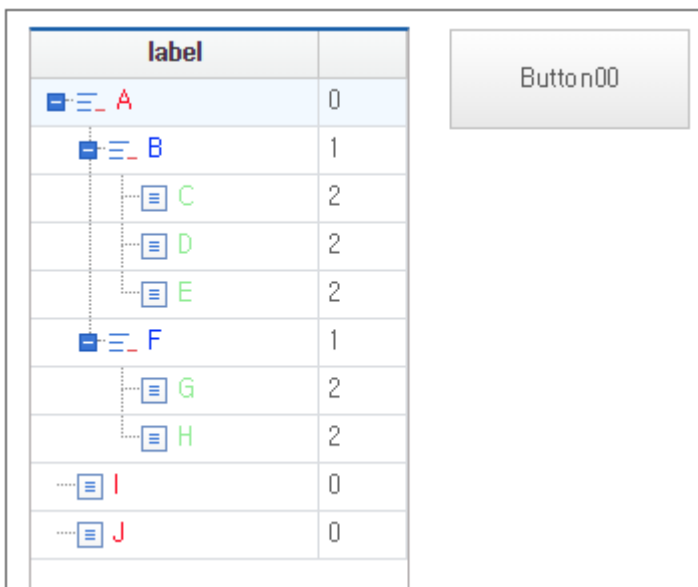
QuickView(Ctrl + F6)로 실행한 후 행번호가 표시되는지 확인합니다.

24.24 트리 레벨에 따라 글자색 바꾸어 표시하기

Grid를 Tree 형태로 표현할때 특정 레벨의 글자색이나 다른 스타일 속성을 바꾸어 표시할 수 있습니다.

24.24.1 예제

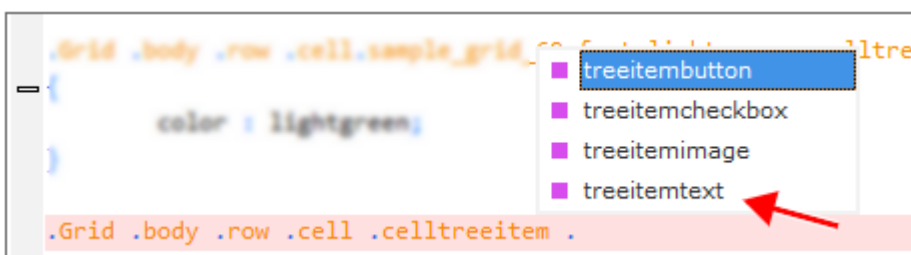
트리 레벨에 따라 다른 색으로 글자를 표시합니다. 버튼 클릭 시 다른 색으로 변경해 표시합니다.



24.24.2 예제에서 사용한 핵심 기능

treeitemtext

CellTreeItemControl 오브젝트 하위의 Control입니다. 텍스트가 표시되는 영역입니다. XCSS 편집기에서는 아래와 같이 접근할 수 있습니다.





treeitemtext 전체에 스타일을 적용하는 경우에는 아래와 같이 GridCellControl 항목으로 직접 지정할 수 있습니다. 하지만 Class 선택자를 지정할 수는 없습니다.

```
.GridCellControl .celltreeitem .treeitemtext /** 0 **/
{
    color : red;
}

.GridCellControl.sample_grid_60_font_red .celltreeitem .treeitemtext /** X **/
{
    color : red;
}
```

24.24.3 예제 구현 방법

① Form 화면 구성하기

Grid 컴포넌트를 배치합니다. Dataset 오브젝트를 하나 추가하고 아래와 같이 데이터값을 설정합니다. 그리고 Dataset 오브젝트를 Grid 컴포넌트에 바인딩해줍니다.

Columns				
No	id	type	size	prop
1	label	STRING	256	
2	level	STRING	256	
Rows				
No	label	level		
1	A	0		
2	B	1		
3	C	2		
4	D	2		
5	E	2		
6	F	1		
7	G	2		
8	H	2		
9	I	0		
10	J	0		

Grid 컴포넌트의 속성 중 트리 형태를 표현하기 위한 속성값을 아래와 같이 수정합니다.

```
treeinitstatus="expand,all"
autofitttype="col"
treeusecheckbox="false"
treeuseimage="true"
treeuseline="true"
```

2 Grid Contents Editor 실행하기

첫번째 컬럼의 Cell 속성을 아래와 같이 수정합니다.

```
displaytype="treeitemcontrol"
treelevel="bind:level"
editttype="tree"
```

3 xcss 파일 편집하기

xcss 파일을 열어서 아래 코드를 추가합니다. Advanced 탭에서 선택자를 추가하거나 Text 탭에서 직접 코드 값을 입력할 수 있습니다.



```

.Grid .body .row .cell.sample_grid_60_font_red .celltreeitem .treeitemtext
{
    color : red;
}

.Grid .body .row .cell.sample_grid_60_font_blue .celltreeitem .treeitemtext
{
    color : blue;
}

.Grid .body .row .cell.sample_grid_60_font_lightgreen .celltreeitem .treeitemtext
{
    color : lightgreen;
}

```

4 Grid Contents Editor 실행하기

다시 Grid Contents Editor를 실행하고 첫번째 컬럼의 Cell 속성 중에서 cssclass 속성값을 아래와 같이 수정합니다. level 컬럼값에 따라 다른 스타일 속성이 적용되도록 지정했습니다.

```

expr:level == 0?'sample_grid_60_font_red':level==1?'sample_grid_60_font_blue':'sample_
grid_60_font_lightgreen'

```

5 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 트리 레벨에 따라 첫번째 컬럼 글자색이 변경된 것을 확인합니다.

6 버튼 onclick 이벤트 함수 추가하기

화면에 Button 컴포넌트를 추가하고 onclick 이벤트 함수를 아래와 같이 생성합니다. cssclass 속성값을 스크립트에서 변경해서 현재 설정된 글자색을 바꿀 수 있습니다.

```

this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Grid00.setCellProperty("body", 0, "cssclass",
        "expr:level == 0?'sample_grid_60_font_lightgreen':level==1?'sample_grid_60_font_
red':'sample_grid_60_font_blue");
};

```

7 QuickView로 확인하기

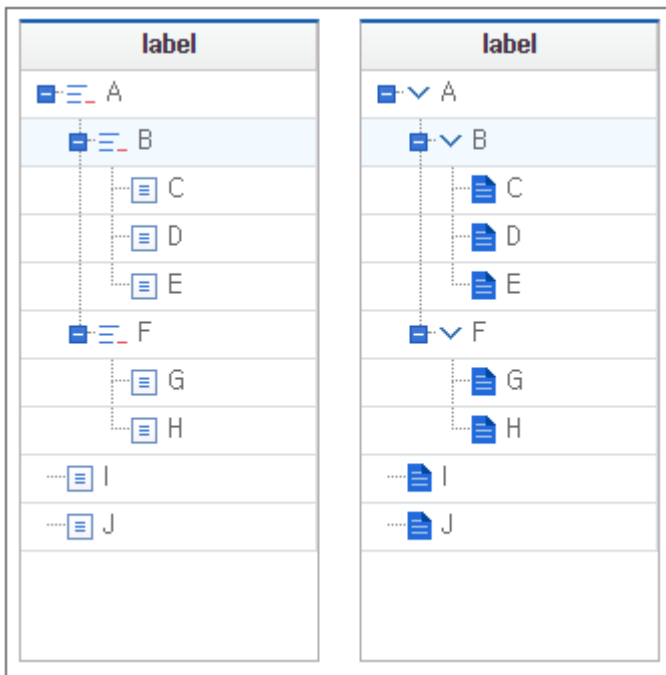
QuickView(Ctrl + F6)로 실행한 후 버튼 클릭 시 글자색이 변경된 것을 확인합니다.

24.25 아이콘 이미지 바꾸어 표시하기

Grid를 Tree 형태로 표현할때 아이콘 이미지를 바꾸어 표시할 수 있습니다.

24.25.1 예제

오른쪽에 있는 Grid 컴포넌트의 아이콘만 다르게 표시합니다.



 [sample_grid_61.xfdl](#)

24.25.2 예제에서 사용한 핵심 기능

-nexa-icon

TreeltemlconControl 오브젝트에서 사용할 아이콘 이미지 파일을 지정합니다. 하위 레벨을 가지고 있는 경우에는 접히거나 펴진 상태의 아이콘 이미지를 지정하며 마지막 트리인 경우에는 항목을 표시하는 아이콘 이미지를 지정합니다.

24.25.3 예제 구현 방법

① Form 화면 구성하기

Grid 컴포넌트를 배치합니다. Dataset 오브젝트를 하나 추가하고 아래와 같이 데이터값을 설정합니다. 그리고 Dataset 오브젝트를 Grid 컴포넌트에 바인딩해줍니다.

Columns				
No	id	type	size	prop
1	label	STRING	256	
2	level	STRING	256	

Rows		
No	label	level
1	A	0
2	B	1
3	C	2
4	D	2
5	E	2
6	F	1
7	G	2
8	H	2
9	I	0
10	J	0

Grid 컴포넌트의 속성 중 트리 형태를 표현하기 위한 속성값을 아래와 같이 수정합니다.

```
treeinitstatus="expand,all"
autofitttype="col"
treeusecheckbox="false"
treeuseimage="true"
treeuseline="true"
```

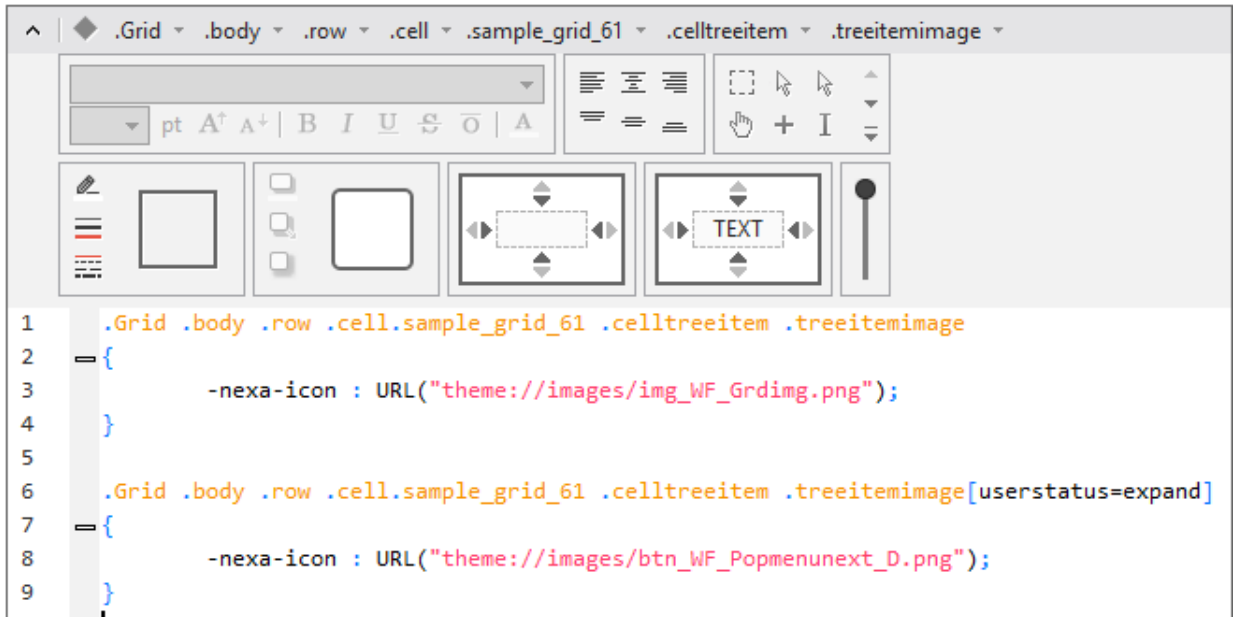
② Grid Contents Editor 실행하기

첫번째 컬럼의 Cell 속성을 아래와 같이 수정합니다.

```
displaytype="treeitemcontrol"
treelevel="bind:level"
editttype="tree"
```

3 xcss 파일 편집하기

xcss 파일을 열어서 아래 코드를 추가합니다. Advanced 탭에서 선택자를 추가하거나 Text 탭에서 직접 코드 값을 입력할 수 있습니다.



```

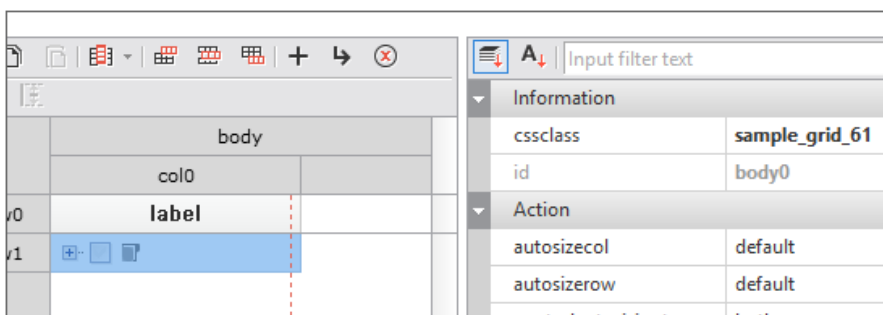
.Grid .body .row .cell.sample_grid_61 .celltreeitem .treeitemimage
{
    -nexa-icon : URL("theme://images/img_WF_Grdimg.png");
}

.Grid .body .row .cell.sample_grid_61 .celltreeitem .treeitemimage[userstatus=expand]
{
    -nexa-icon : URL("theme://images/btn_WF_Popmenunext_D.png");
}

```

4 Grid 컴포넌트 복사하기

이전 단계에서 만든 Grid 컴포넌트를 복사해 같은 속성을 가지는 Grid 컴포넌트를 생성합니다. 그리고 Grid Contents Editor를 실행하고 첫번째 컬럼의 Cell 속성 중에서 cssclass 속성값을 아래와 같이 수정합니다.



5 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 오른쪽에 생성한 Grid 컴포넌트의 트리 항목 아이콘이 변경된 것을 확인합니다.

24.26 선택한 Cell 속성 확인하기

Grid 컴포넌트의 특정 영역을 선택했을때 해당 영역이 head, body, summary 영역 중 어느 영역인지 확인할 수 있습니다.

24.26.1 예제

onlbuttondown 이벤트 발생 시 하단 TextArea 영역에 관련 정보를 표시합니다.

left	A	B	C	
			D	
1		1	1	
			1	
2		2	2	
			2	
3		3	3	
			3	

Body Band
 row: 2
 subrow: 1

 sample_grid_70.xfdl

24.26.2 예제에서 사용한 핵심 기능

row

예제에서는 GridMouseEventInfo 오브젝트의 속성값을 사용하지만 Grid 컴포넌트에 연결된 다른 이벤트 오브젝트에서도 row 속성값 정보를 제공합니다.

24.26.3 예제 구현 방법

① Form 화면 구성하기

Grid 컴포넌트를 배치합니다. Dataset 오브젝트를 하나 추가하고 아래와 같이 데이터값을 설정합니다. 그리고 Dataset 오브젝트를 Grid 컴포넌트에 바인딩해줍니다.

Columns				
No	id	type	size	prop
1	A	STRING	256	
2	B	STRING	256	
3	C	STRING	256	
4	D	STRING	256	
Rows				
No	A	B	C	D
1	1	1	1	1
2	2	2	2	2
3	3	3	3	3
4	4	4	4	4

② Grid Contents Editor 실행하기

다양한 Cell 속성을 확인하기 위해 몇 가지를 수정합니다. Left Column을 추가하고 Head, Body Row를 하나씩 더 추가합니다. Summary Row도 추가합니다. A, B 컬럼을 표시하는 항목은 head, body 영역의 row를 merge해서 표시하도록 합니다.

The screenshot shows a grid component with a toolbar at the top. The grid is divided into several sections: a 'left' column, a 'body' section, and a 'summary' row. Annotations with blue boxes and arrows point to specific areas:

- Add Left Column:** Points to the 'left' column header.
- Add Head Row:** Points to the 'head' row header.
- Add Body Row:** Points to a row in the 'body' section.
- Add Summary Row:** Points to the 'summary' row.

		left	body			
		col0	col1	col2	col3	
head	row0	left	A	B	C	
	row1				D	
body	row2					
	row3					
summary	row4					

3 onmousedown 이벤트 함수 작성하기

Grid 컴포넌트의 각 영역을 클릭했을 때 Cell 속성 정보를 확인할 수 있도록 onmousedown 이벤트 함수를 작성합니다.

```
this.Grid00_onmousedown = function(obj:nexacro.Grid,e:nexacro.GridMouseEventInfo)
{
    var strRow;
    switch(e.row) {
        case -1:
            strRow = 'Head Band';
            break;
        case -2:
            strRow = 'Summary Band';
            break;
        case -9:
            strRow = 'Blank Body';
            break;
        default:
            strRow = 'Body Band';
    }
    this.TextArea00.set_value(strRow+'\n'+row: '+e.row+'\n'+subrow: '+e.subrow)
```

};

④ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 Grid 각 영역을 클릭하면서 정보를 확인합니다.

24.27 ComboControl 내에서 선택한 항목의 스타일 바꾸어 표시하기

Grid 컴포넌트 내에서 다양한 형식으로 표시되는 컨트롤 내 스타일을 변경해 표시할 수 있습니다.

24.27.1 예제

오른쪽 Grid 컴포넌트에는 기본 테마 설정에 따라 목록을 표시하고 왼쪽 Grid 컴포넌트에는 지정된 스타일에 따라 목록 내 선택된 항목 텍스트를 빨간색으로 표시합니다.

Column0	Column1
1	1
1	1
2	2
3	3
2	2
3	3

 [sample_grid_71.xfdl](#)

24.27.2 예제에서 사용한 핵심 기능

combodataset

GridCellControl 오브젝트의 속성입니다. combocodecol, combodatacol 속성과 같이 사용합니다.

24.27.3 예제 구현 방법

1 Form 화면 구성하기

Grid 컴포넌트를 배치합니다. Dataset 오브젝트를 하나 추가하고 아래와 같이 데이터값을 설정합니다. 그리고 Dataset 오브젝트를 Grid 컴포넌트에 바인딩해줍니다.

Columns		
No	id	type
1	Column0	STRING
Rows		
No	Column0	
1	1	
2	2	
3	1	
4	2	
5	3	

그리고 code, data 컬럼을 가지는 Dataset 오브젝트를 하나 더 생성합니다.

Columns				
No	id	type	size	pro
1	code	STRING	256	
2	data	STRING	256	
Rows				
No	code	data		
1	1	1		
2	2	2		
3	3	3		

2 Grid Contents Editor 실행하기

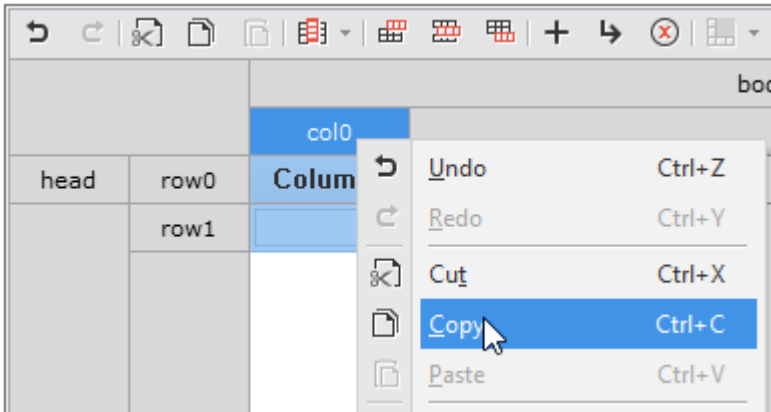
Cell 오브젝트의 속성을 아래와 같이 지정합니다.

```
displaytype: combocontrol
edittype: combo
combodataset: Dataset01
```



```
combodatacol: data
combocodecol: code
```

속성을 지정한 다음 해당 컬럼을 복사해서 붙여넣습니다. 붙여넣기를 할 때는 선택한 컬럼 앞에 추가하거나(Insert) 뒤에 추가(Append)할 수 있습니다.

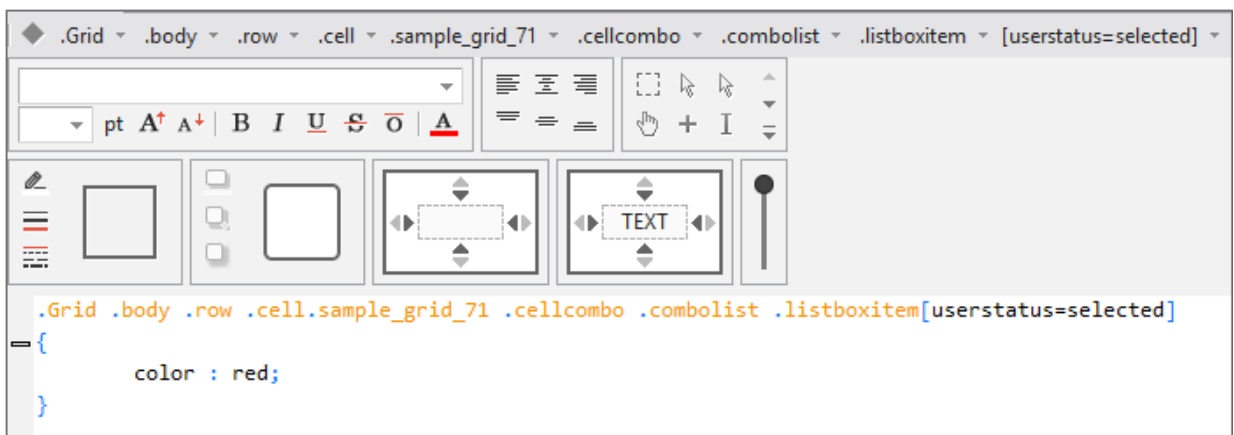


추가한 Cell 오브젝트의 class 속성값을 지정합니다.

body			Information	
col0	col1		cssclass	sample_grid_71
Column0	Column1		id	body0
			Action	
			autosizecol	default

3 xcss 파일 편집하기

xcss 파일을 열어서 아래 코드를 추가합니다. Advanced 탭에서 선택자를 추가하거나 Text 탭에서 직접 코드 값을 입력할 수 있습니다.



```
.Grid .body .row .cell.sample_grid_71 .cellcombo .combolist .listboxitem[userstatus=
selected]
{
    color : red;
}
```

4 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 Grid 각 영역을 클릭하면서 정보를 확인합니다.

24.28 Grid에서 특정 텍스트만 입력하기

Cell 편집 상태인 경우 입력되는 값을 확인하고 입력값을 제한할 수 있습니다.

24.28.1 예제

Column1 항목은 한글만 입력할 수 있습니다. 다른 텍스트를 입력하려고 시도하면 입력값을 빈값으로 대체합니다.

Column0	Column1
apple	사과
car	한글한글
water	물

 sample_grid_68.xfdl

24.28.2 예제에서 사용한 핵심 기능

getEditValue, setEditValue (17.0.0.1000 버전부터 지원)

Cell 편집 시 편집중인 값을 가져오거나 설정할 수 있습니다.

24.28.3 예제 구현 방법

① Form 화면 구성하기

Grid 컴포넌트를 배치합니다. Dataset 오브젝트를 하나 추가하고 아래와 같이 데이터값을 설정합니다. 그리고 Dataset 오브젝트를 Grid 컴포넌트에 바인딩해줍니다.

Columns				
No	id	type	size	prop
1	Column0	STRING	256	
2	Column1	STRING	256	

Rows		
No	Column0	Column1
1	apple	사과
2	car	자동차
3	water	물

② Grid Contents Editor 실행하기

Column1 Cell 오브젝트의 edittype 속성값을 "normal"로 수정합니다.

body		
col0	col1	
Column0	Column1	

cssclass	
id	body1
Action	
autosizecol	default
autosizerow	default
controlautosizing	both
displaytype	normal
edittype	normal
expr	

③ oninput 이벤트 함수 작성하기

Grid 컴포넌트의 oninput 이벤트 함수 내에서 입력값을 확인하고 한글인지 아닌지 판단한 후에 한글인 경우에는 그대로 입력하고 그렇지 않은 경우에는 빈값("")을 반환합니다.

```
this.Grid00_oninput = function(obj:nexacro.Grid,e:nexacro.InputEventInfo)
{
    var strControlValue = obj.getEditValue();
    if(strControlValue
```

```

{
    obj.setEditValue(strControlValue.replace(/[\^ㄱ-ㅎㅌ-ㅣ가-힣]/g, ""));
}
};

```

4 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 두번째 컬럼을 편집상태로 만든 후 텍스트를 입력해봅니다.

24.29 가로값이 동일한 셀을 하나의 셀로 표시하기

suppresshorzcell 속성은 Left, Right 컬럼에서 타이틀을 표시하는 경우 가로 방향으로 셀이 병합되는 효과를 처리할 수 있습니다. Grid 컴포넌트 속성이지만 Cell 오브젝트의 suppress 속성의 영향을 받습니다.

24.29.1 예제

L1~L3 컬럼은 suppresshorzcell 속성과 suppress 속성이 같이 동작합니다. B1~B3 컬럼은 Body 컬럼이기 때문에 suppresshorzcell 속성이 적용되지 않고 suppress 속성값만 적용됩니다. 1~3 컬럼은 suppress 속성을 적용하지 않은 원래 데이터를 표시합니다.

L1	L2	L3	B1	B2	B3	1	2	3
a			a	a	a	a	a	a
	b			b	b	a	b	b
				a	a	a	a	a
b	b		b	b	b	b	b	b
		a			a	b	b	a
	a			a		b	a	a

Button00



sample_grid_75.xfdl

24.29.2 예제에서 사용한 핵심 기능

suppresshorzcell 속성 (17.0.0.1000 버전부터 지원)

Left, Right 컬럼 내에서 가로값이 동일한 경우 하나의 셀로 표시할 수 있습니다.

24.29.3 예제 구현 방법

① 화면 구성하기

Grid 컴포넌트와 Button 컴포넌트를 화면에 배치하고 Dataset 오브젝트를 생성합니다. Dataset 오브젝트의 값은 아래와 같이 설정하고 Grid 컴포넌트에 바인딩해줍니다.

Columns			
No	id	type	size
1	Column0	STRING	256
2	Column1	STRING	256
3	Column2	STRING	256

Rows				
No	Column0	Column1	Column2	
1	a	a	a	
2	a	b	b	
3	a	a	a	
4	b	b	b	
5	b	b	a	
6	b	a	a	

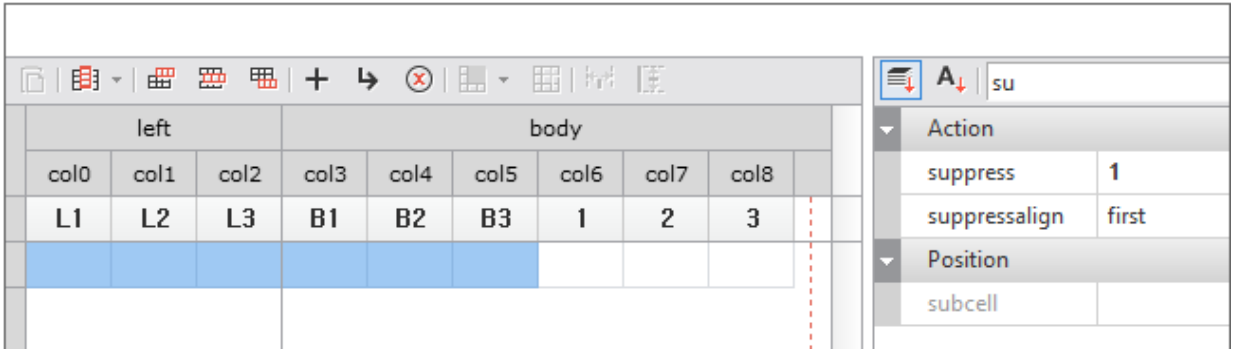
② Grid Contents Editor 실행하기

Left 컬럼과 Bobby 컬럼을 3개씩 추가합니다. 추가된 컬럼은 순서에 따라 text 속성값으로 Column1~Column 3을 지정해줍니다.

body			
5	col6	col7	col8
8	1	2	3

expr	
suppress	0
suppressalign	first
text	bind:Column2
tooltiptext	Column0
tooltiptype	Column1
Misc.	Column2

각 컬럼 Head text 속성값을 구분을 위해 L1~L3, B1~B3, 1~3으로 지정합니다. 그리고 L1~L3, B1~B3 컬럼의 Body 셀을 모두 선택하고 suppress 속성값을 지정합니다. 여러 셀을 선택한 상태에서 특정 속성값을 수정하면 모든 셀에 같은 속성값이 반영됩니다.



3 suppresshorzcell 속성값 설정

Grid Contents Editor 창을 닫고 Grid 컴포넌트를 선택한 상태에서 suppresshorzcell 속성값을 "left"로 지정합니다.

4 onclick 이벤트 함수 작성하기

버튼 클릭 시 L2, L3 컬럼 Cell의 suppress 속성값을 0으로 변경합니다.

```

this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Grid00.setCellProperty("body", 1, "suppress", "0");
    this.Grid00.setCellProperty("body", 2, "suppress", "0");
};
  
```

5 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

L1	L2	L3	B1	B2	B3	1	2	3
a			a	a	a	a	a	a
	b			b	b	a	b	b
				a	a	a	a	a
b	b		b	b	b	1	2	b
		a			a	b	b	a
	a			a		b	a	a

24.30 원하는 영역의 셀을 하나의 셀로 표시하기

데이터와 상관없이 지정된 영역의 셀을 하나의 셀로 표시할 수 있습니다. 동적으로 구성된 데이터보다는 고정된 데이터를 표시하면서 셀을 합칠 필요가 있는 경우 사용할 수 있습니다. 아래 예제는 식당 가격표에서 같은 금액대의 셀을 하나의 셀로 표시합니다.

HTML Table 태그에서 `rowspan`, `colspan` 기능을 동적으로 처리하는 것이라고 설명할 수 있습니다. `rowspan`, `colspan` 기능을 `mergeCell` 메소드 하나에서 모두 처리할 수 있습니다.



Grid Contents Editor 에서 셀 병합 기능을 사용했거나 `suppresshorzcell` 속성과 영역이 중첩되는 경우에는 유의해서 사용해야 합니다. 자세한 유의사항은 도움말에서 `mergeCell` 항목을 참고하세요.

24.30.1 예제

`mergeCell` 버튼 클릭 시 지정된 영역의 셀을 하나의 셀로 표시합니다. `splitCell` 버튼을 클릭하면 `adult`에 해당하는 Row 정보만 원래의 셀로 분리해 표시합니다.

type	lunch	dinner	weekend
adult	14,900	23,900	
student	10,900		20,000
infant	7,500		20,000

mergeCell
splitCell

 [sample_grid_77.xfdl](#)

24.30.2 예제에서 사용한 핵심 기능

mergeCell (17.0.0.1400 버전부터 지원)

`nStartCol`, `nEndCol`, `nStartRow`, `nEndRow` 4개의 파라미터로 병합할 셀 영역을 지정하고 셀을 병합합니다. 같은 Band 영역만 병합할 수 있으며 병합에 실패하거나 해당 영역이 이미 병합되어 있는 경우에는 `false` 값을 반환합니다.



mergeCell 메소드 실행 시 병합된 셀의 데이터는 첫번째 인덱스 셀을 기준으로 변경됩니다. Dataset 오브젝트가 바인딩된 경우에는 Dataset 오브젝트의 데이터도 변경됩니다.

splitCell (17.0.0.1400 버전부터 지원)

StartCol, nEndCol, nStartRow, nEndRow 4개의 파라미터로 병합을 해제할 셀 영역을 지정하고 셀병합을 해제합니다.

24.30.3 예제 구현 방법

① 화면에 Grid 컴포넌트를 배치합니다.

② Dataset 오브젝트를 Invisible Object 영역으로 배치하고 아래와 같이 데이터를 입력합니다.

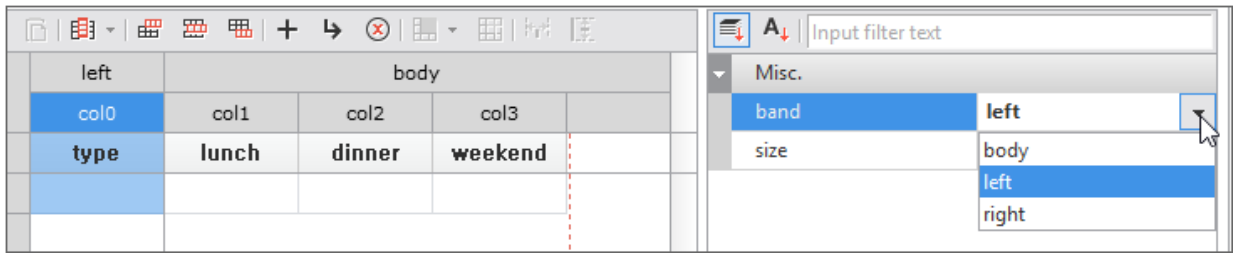
type 컬럼값이 adult 인 항목은 dinner, weekend 가격이 같고, type 컬럼값이 student 인 항목은 lunch, dinner 가격이 같습니다. 가격이 같은 셀을 병합할 겁니다.

Const Columns				
Columns				
No	id	type	size	prop
1	type	STRING	256	
2	lunch	INT	256	
3	dinner	INT	256	
4	weekend	INT	256	
Rows				
No	type	lunch	dinner	weekend
1	adult	14900	23900	23900
2	student	10900	10900	20000
3	infant	7500	7500	20000

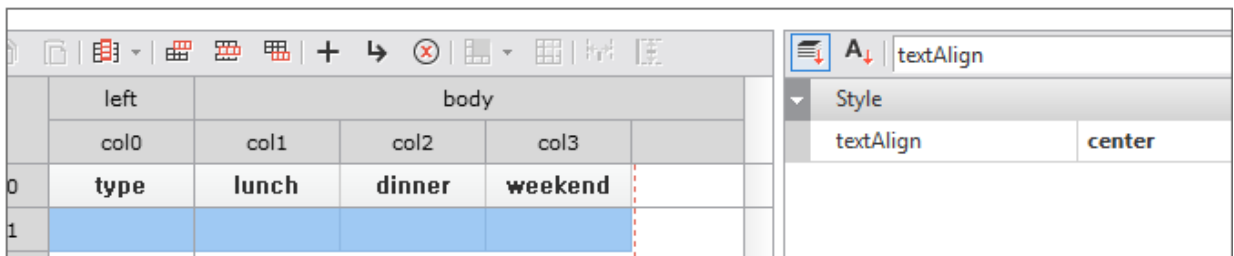
③ Grid 컴포넌트에 Dataset 오브젝트를 바인딩합니다.

④ Grid 컴포넌트를 더블 클릭해 Grid Contents Editor를 띄웁니다.

⑤ type 컬럼을 선택하고 band 속성값을 "left"로 수정합니다.



- ⑥ type 컬럼 항목의 body row를 선택하고 font-weight 속성값을 "bold"로 수정합니다.
- ⑦ body row 전체를 선택하고 textAlign 속성값을 "center"로 수정합니다.



- ⑧ Button 컴포넌트 2개를 Grid 컴포넌트 아래에 배치합니다.
Button 컴포넌트의 text 속성값을 "mergeCell", "splitCell"로 수정합니다.
- ⑨ Button 컴포넌트에 "Button_onclick"이라는 이름으로 onclick 이벤트 함수를 추가합니다.
Button 컴포넌트의 text 속성값에 따라 mergeCell 또는 splitCell 메소드를 실행합니다.

```
this.Button_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    if(obj.text == 'mergeCell')
    {
        this.Grid00.mergeCell(2, 3, 0, 0); // 23900
        this.Grid00.mergeCell(1, 2, 1, 1); // 10900
        this.Grid00.mergeCell(1, 2, 2, 2); // 7500
        trace(this.Grid00.mergeCell(0, 1, 0, 0)); // adult, 14900
    }
    else if(obj.text == 'splitCell')
    {
        this.Grid00.splitCell(2, 3, 0, 0);
    }
};
```

파라미터는 Column 인덱스와 Row 인덱스를 지정합니다. type 컬럼값이 "adult"인 항목의 dinner, weeken

d 컬럼의 셀을 하나의 셀로 합치려면 Column 인덱스는 2부터 3까지, Row 인덱스는 0부터 0까지입니다. 그래서 파라미터값은 (2, 3, 0, 0)이 됩니다.

	0	1	2	3
type	lunch	dinner	weekend	
adult	14,900	23,900	23,900	0
student	10,900	10,900	20,000	1
infant	7,500	7,500	20,000	2

Column 2, 3
Row 0, 0

mergeCell 버튼 클릭 시 네 번째로 처리하는 스크립트는 파라미터값이 (0, 1, 0, 0)인데, type 컬럼은 band 속성값을 "left"로 수정해서 left 밴드와 body 밴드를 합치려고 시도합니다. 하지만 서로 다른 밴드는 합칠 수 없어서 false 값을 반환합니다.

adult	14,900	23,900
student	10,900	20,000
infant	7,500	20,000

Column 0, 1
Row 0, 0

mergeCell splitCell

DevTools - 127.0.0.1:4098/quickview.html?scre

Elements Memory Console

top

false

- ⑩ QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. 버튼을 클릭해서 지정된 영역이 하나의 셀로 합쳐지고 합쳐진 셀이 해제되는지 확인합니다.

24.31 선택한 영역의 셀을 하나의 셀로 표시하기

selecttype 속성값이 "area"인 경우 선택한 영역의 여러 셀을 하나의 셀로 표시하는 방법을 살펴봅니다.

24.31.1 예제

하나의 셀로 표시하기 원하는 셀을 선택하고 mergeCell 버튼 클릭 시 선택한 영역의 셀을 하나의 셀로 표시합니다. splitCell 버튼을 클릭하면 선택한 영역의 병합된 셀을 해제합니다.

type	lunch	dinner	weekend
adult	14,900	23,900	23,900
student	10,900	10,900	20,000
infant	7,500	7,500	20,000

mergeCell

splitCell

 sample_grid_78.xfdl

24.31.2 예제에서 사용한 핵심 기능

selectstartcol, selectendcol, selectstartrow, selectendrow

선택한 영역의 Column, Row 인덱스 정보를 확인할 수 있습니다. 해당 속성값은 배열 형태입니다. selecttype 속성값이 "area"인 경우에는 배열의 길이가 1이고, "multiarea"인 경우에는 선택한 영역에 따라 배열의 길이가 1 이상이 될 수 있습니다.

24.31.3 예제 구현 방법

- ① 화면에 Grid 컴포넌트를 배치합니다.
- ② Dataset 오브젝트를 Invisible Object 영역으로 배치하고 아래와 같이 데이터를 입력합니다.

Const Columns				
Columns				
No	id	type	size	prop
1	type	STRING	256	
2	lunch	INT	256	
3	dinner	INT	256	
4	weekend	INT	256	

Rows				
No	type	lunch	dinner	weekend
1	adult	14900	23900	23900
2	student	10900	10900	20000
3	infant	7500	7500	20000

- ③ Grid 컴포넌트에 Dataset 오브젝트를 바인딩합니다.
- ④ Grid 컴포넌트를 더블 클릭해 Grid Contents Editor를 띄웁니다.
- ⑤ type 컬럼을 선택하고 band 속성값을 "left"로 수정합니다.

left				
body				
col0	col1	col2	col3	
type	lunch	dinner	weekend	

Misc.	
band	left
size	body
	left
	right

- ⑥ type 컬럼 항목의 body row를 선택하고 font-weight 속성값을 "bold"로 수정합니다.
- ⑦ body row 전체를 선택하고 textAlign 속성값을 "center"로 수정합니다.

left				
body				
col0	col1	col2	col3	
0 type	lunch	dinner	weekend	
1				

Style	
textAlign	center

- ⑧ Grid 컴포넌트를 선택하고 selecttype 속성값을 "area"로 수정합니다.

- 9 Button 컴포넌트 2개를 Grid 컴포넌트 아래에 배치합니다.

Button 컴포넌트의 text 속성값을 "mergeCell", "splitCell"로 수정합니다.

- 10 Button 컴포넌트에 "Button_onclick"이라는 이름으로 onclick 이벤트 함수를 추가합니다.

Button 컴포넌트의 text 속성값에 따라 mergeCell 또는 splitCell 메소드를 실행합니다.

```
this.Button_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    if(obj.text == 'mergeCell')
    {
        this.Grid00.mergeCell(this.Grid00.selectstartcol[0],
            this.Grid00.selectendcol[0],
            this.Grid00.selectstartrow[0],
            this.Grid00.selectendrow[0]);
    }
    else if(obj.text == 'splitCell')
    {
        this.Grid00.splitCell(this.Grid00.selectstartcol[0],
            this.Grid00.selectendcol[0],
            this.Grid00.selectstartrow[0],
            this.Grid00.selectendrow[0]);
    }
};
```

파라미터는 선택한 영역의 Column 인덱스와 Row 인덱스를 지정합니다.

	0	1	2	3	
	type	lunch	dinner	weekend	
adult	14,900	23,900	23,900		0
student	10,900	10,900	20,000		1
infant	7,500	7,500	20,000		2

selectstartcol[0]: 2
 selectendcol[0]: 3
 selectstartrow[0]: 0
 selectendrow[0]: 0

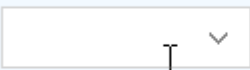
- 11 QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. 버튼을 클릭해서 선택한 영역이 하나의 셀로 합쳐지고 합쳐진 셀이 해제되는지 확인합니다.

24.32 셀 실제 속성값 가져오기

셀 속성값을 `expr`로 설정한 경우 실제 셀 속성값을 가져올 수 있습니다.

24.32.1 예제

Column0 컬럼의 셀을 클릭하면 실제 컨트롤 이름과 `expr`로 설정한 스크립트를 TextArea 컴포넌트에 표시합니다.

Column0	Column1
	buttoncontrol
☐	checkboxcontrol
	combocontrol
0000-01-01 M 	calendarcontrol

combocontrol
 expr:Column1



sample_grid_82.xfdl

24.32.2 예제에서 사용한 핵심 기능

getCellPropertyValue (17.0.0.1201 버전부터 지원)

바인딩되거나 `expr`로 설정된 경우 실제 값을 반환합니다. 예제에서는 `displaytype` 속성값을 `expr`로 지정하고 실제 `displaytype` 속성으로 적용된 값을 확인합니다.

24.32.3 예제 구현 방법

- ① 화면에 Grid 컴포넌트와 TextArea 컴포넌트를 배치합니다.
- ② Dataset 오브젝트를 추가하고 아래와 같이 데이터를 설정합니다.

Columns				
No	id	type	size	prop
1	Column0	STRING	256	
2	Column1	STRING	256	

Rows			
No	Column0	Column1	
1	[Undefined]	buttoncontrol	
2	[Undefined]	checkboxcontrol	
3	[Undefined]	combocontrol	
4	[Undefined]	calendarcontrol	

- ③ Grid 컴포넌트에 Dataset 오브젝트를 바인딩합니다.
- ④ Grid 컴포넌트를 더블 클릭해 Grid Contents Editor를 띄웁니다.
Column0 항목의 displaytype 속성값을 아래와 같이 지정합니다.

body		
col0	col1	
Column0	Column1	

cssclass	
id	body0
Action	
autosizecol	default
autosizerow	default
controlautosizingtype	both
displaytype	expr:Column1
edittype	none
expr	

- ⑤ Grid 컴포넌트의 oncellick 이벤트 함수를 아래와 같이 추가합니다.

```
this.Grid00_oncellick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    this.TextArea00.set_value(obj.getCellPropertyValue(e.row, e.cell, "displaytype")+"\n")
}
```

```
+obj.getCellProperty("body", e.cell, "displaytype"));
};
```

- ⑥ QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. 셀 클릭 시 실제 displaytype 속성에 적용된 값을 확인할 수 있습니다.

24.33 cssclass 속성값을 여러 개 지정하기

xcss 파일에서 지정할 수 있는 속성값이 여러가지입니다. 모든 케이스를 하나의 class로 작성하지 않고 중첩해서 적용할 수 있습니다. 예를 들어 배경색을 지정하는 class를 하나 만들고 상황에 따라 글자색을 변경하는 class를 여러개 만들 수 있습니다.

24.33.1 예제

cssclass 속성값에 따라 셀마다 다른 색상 스타일이 적용됩니다.

Column0	Column1	Column2
1	one	I
2	two	II
3	three	III
4	four	IV
5	five	V



sample_grid_84.xfdl

24.33.2 예제에서 사용한 핵심 기능

cssclass

classname을 여러 개 지정할 수 있습니다. 이때 중복된 스타일 속성이 지정되는 경우 XCSS 파일 상에서 마지막에 등록된 항목이 적용됩니다. cssclass 속성값으로 지정하는 순서가 아니라 XCSS 파일 상에서의 순서입니다.

24.33.3 예제 구현 방법

- ① 화면에 Grid 컴포넌트를 배치합니다.
- ② Dataset 오브젝트를 추가하고 3개의 컬럼을 생성한 후 데이터를 추가합니다.

Columns					
No	id	type	size	prop	sumt
1	Column0	STRING	256		
2	Column1	STRING	256		
3	Column2	STRING	256		
Rows					
No	Column0	Column1	Column2		
1	1	one	I		
2	2	two	II		
3	3	three	III		
4	4	four	IV		
5	5	five	V		

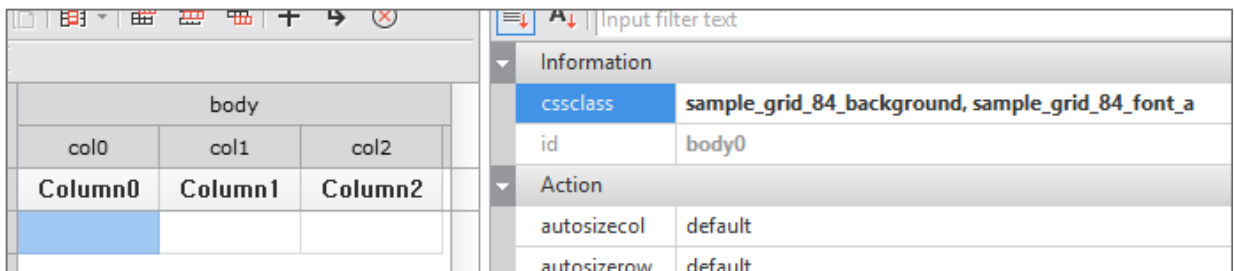
- ③ Dataset 오브젝트를 선택하고 Grid 컴포넌트로 드래그앤드롭해서 binddataset 속성으로 등록합니다.
- ④ XCSS 파일을 열어서 아래와 같이 코드를 추가합니다.

sample_grid_84_background 에서는 배경색과 글자색을 둘 다 지정하고 sample_grid_84_font_a, sample_grid_84_font_b 에서는 짝수번째 Row에 대한 배경색과 글자색을 지정합니다.

```
.Grid .body .row .cell.sample_grid_84_background
{
    -nexa-background-odd : silver;
    background : silver;
    color : white;
    -nexa-color-odd : white;
}
.Grid .body .row .cell.sample_grid_84_font_a
{
    background : blue;
    color : red;
}
```

```
.Grid .body .row .cell.sample_grid_84_font_b
{
    background : red;
    color : pink;
}
```

- 5 Grid 컴포넌트를 더블 클릭해서 Grid Contents Editor를 실행하고 Column0, Column1 Body 셀의 cssclass 속성값을 아래와 같이 지정합니다.



```
<Band id="body">
  <Cell text="bind:Column0" cssclass="sample_grid_84_background, sample_grid_84_font_a"/>
</Band>
  <Cell col="1" text="bind:Column1" cssclass="sample_grid_84_background, sample_grid_84_font_b"/>
  <Cell col="2" text="bind:Column2"/>
</Band>
```

- 6 QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. selected, mouseover 상태인 경우에는 기본 테마에 설정된 스타일이 적용됩니다.

Column0	Column1	Column2
1	one	I
2	two	II
3	three	III
4	four	IV
5	five	V

sample_grid_84_font_b: color

sample_grid_84_background

sample_grid_84_font_b

sample_grid_84_font_a

sample_grid_84_font_a: color

24.34 편집중인 값 가져오기

edittype 속성값에 따라 사용자가 입력한 값을 확인한 상태에서만 값을 확인할 수 있습니다. 예를 들어 edittype 속성값이 "date"인 경우에 사용자가 직접 날짜값을 입력하는 경우가 그렇습니다. 그래서 편집중인 상태에서도 oninput 이벤트에서 값을 확인할 수 있는 getEditingValue, getEditingText 메소드가 추가됐습니다.

24.34.1 예제

edittype 속성값에 따라 각 메소드가 값을 어떤 식으로 처리하는지 확인할 수 있습니다.

text	mask	date
TEST	1234-56	2021-10-14
	----'---	0000-01-01 Mon
	----'---	0000-01-01 Mon
	----'---	0000-01-01 Mon

```

getCurEditType: date
getEditValue: 20190218
getEditText: 2021-10-14
getEditingValue: 20211014
getEditingText: 2021-10-14

```



sample_grid_86.xfdl

24.34.2 예제에서 사용한 핵심 기능

getEditingValue, getEditingText (17.0.0.1700 버전부터 지원)

편집중인 값을 확인하는 메소드입니다. edittype 속성값이 "date", "mask"인 경우에는 mask 속성값에 따라 getEditingValue, getEditingText 메소드의 결과값이 다를 수 있습니다.

setEditingValue (17.0.0.1700 버전부터 지원)

편집중인 값을 확인하고 수정하려는 경우 사용하는 메소드입니다.

24.34.3 예제 구현 방법

- 1 화면에 Grid 컴포넌트와 TextArea 컴포넌트를 배치합니다.
- 2 Dataset 오브젝트를 추가하고 아래와 같이 데이터를 설정합니다.

Columns					
No	id	type	size	prop	sumtext
1	text	STRING	256		
2	mask	STRING	256		
3	date	DATE	256		

Rows			
No	text	mask	date
1	TEST	123456	20190218
2	[Undefined]	[Undefined]	[Undefined]
3	[Undefined]	[Undefined]	[Undefined]
4	[Undefined]	[Undefined]	[Undefined]

- 3 Dataset 오브젝트를 선택하고 Grid 컴포넌트로 드래그앤드롭해서 binddataset 속성으로 등록합니다.
- 4 Grid 컴포넌트를 더블클릭해서 Grid Content Editor를 실행하고 아래와 같이 속성값을 지정합니다.

The screenshot shows the Grid Content Editor interface. The main area displays a grid structure with columns (col0, col1, col2) and rows (row0, row1). The columns are defined as text, mask, and date. The rows are defined with specific data values. The right panel shows the properties of the selected grid component, including displaytype, edittype, and calendartype.

Column definitions:

- col0: text
- col1: mask
- col2: date

Row definitions:

- row0: TEST, 123456, 20190218
- row1: [Undefined], [Undefined], [Undefined]

Properties panel (Action):

- displaytype: calendarcontrol
- edittype: date
- CellCalendar: spin

Annotations for the first row (row0):

- edittype="text", displaytype="text" (for col0)
- edittype="mask", maskeditformat="@@@@-@@", displaytype="mask", maskedittype="string" (for col1)

- 5 Grid 컴포넌트의 oninput 이벤트를 함수를 아래와 같이 추가합니다.

```
this.Grid00_oninput = function(obj:nexacro.Grid,e:nexacro.InputEventInfo)
{
    this.TextArea00.set_value("getCurEditType: "+obj.getCurEditType()+"\n")
}
```

```

+"getEditValue: "+obj.getEditValue()+"\n"
+"getEditText: "+obj.getEditText()+"\n"
+"getEditingValue: "+obj.getEditingValue()+"\n"
+"getEditingText: "+obj.getEditingText());
};

```

- 6 QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. 각 컬럼별로 편집 상태에서 값을 변경하면서 TextArea 컴포넌트에 표시되는 값을 확인합니다.

24.35 자식 노드 한 번에 펼치기

displaytype 속성값이 "treeitemcontrol"인 경우 상위 노드를 선택하면 하위 노드가 펼쳐지거나 접힙니다. 이때 하위 노드의 상태는 이전에 상태를 가지고 있습니다. 레벨이 깊은 하위 노드에 접근하려면 여러 번 펼치는 작업이 필요합니다. 이번 예제에서는 상위 노드에서 자식 노드를 한 번에 펼치는 방법을 살펴봅니다.

24.35.1 예제

Menu 1 또는 Menu 2 항목 선택 시 자식 노드가 한 번에 펼쳐집니다.

Column 1
Menu 1
Menu 1-1
Menu 1-1-1
Menu 2
Menu 2-1
Menu 2-1-1
Menu 3
Menu 3-1
Menu 3-1-1

24.35.2 예제에서 사용한 핵심 기능

TreeltemIconControl

Grid 컴포넌트를 트리 형식으로 표시할 때 펼침, 접힘 아이콘에 해당하는 컨트롤 오브젝트입니다. 이미지와 버튼 2가지가 표시되고 있어서, 오브젝트 이름과 id를 조건으로 선택 여부를 확인합니다.

isTreeLeafRow

특정 Row가 자식이 없는 Row인지 확인합니다. 자식이 없으면 true를 반환합니다.

24.35.3 예제 구현 방법

- ① 화면에 Grid 컴포넌트를 배치합니다.
- ② Dataset 오브젝트를 추가하고 아래와 같이 데이터를 설정합니다.

Columns							
No	id	type	size	prop	sumtext	description	
1	Level	STRING	256				
2	MenuName	STRING	256				
3	MenuID	STRING	256				
4	Status	STRING	256				

Rows				
No	Level	MenuName	MenuID	Status
1	0	Menu 1	1	[Undefined]
2	1	Menu 1-1	2	[Undefined]
3	2	Menu 1-1-1	3	[Undefined]
4	0	Menu 2	5	1
5	1	Menu 2-1	6	[Undefined]
6	2	Menu 2-1-1	7	[Undefined]
7	0	Menu 3	8	[Undefined]
8	1	Menu 3-1	9	[Undefined]
9	2	Menu 3-1-1	10	[Undefined]

- ③ Dataset 오브젝트를 선택하고 Grid 컴포넌트로 드래그앤드롭해서 binddataset 속성으로 등록합니다.
- ④ Grid 컴포넌트의 treeinitstatus 속성값은 "collapse,null"로 지정합니다.

- 5 Grid 컴포넌트를 더블클릭해서 Grid Content Editor를 실행하고 아래와 같이 속성값을 지정합니다.

```
displaytype: treeitemcontrol
editttype: tree
text: bind:MenuName
treelevel: bind:Level
treestate: bind:Status
```

- 6 Grid 컴포넌트의 oncellick 이벤트 함수를 아래와 같이 추가합니다.

이벤트가 발생한 위치가 트리 아이템 버튼인지 확인하고 각 Row의 Status 값을 확인한 후 펼치거나 접을 때 상태값을 수정합니다. 작업 전에 Dataset 오브젝트의 enableevent 속성값을 false로 지정하고 작업이 끝나면 true로 지정합니다.

```
this.Grid00_oncellick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    if(e.fromreferenceobject == "[object TreeItemIconControl]" && e.fromreferenceobject.id
    == "treeitembutton")
    {
        this.Dataset00.set_enableevent(false);
        if(this.Dataset00.getColumn(e.row,"Status") == null)
        {
            this.Dataset00.setColumn(e.row,"Status",1);
            for(var i=1; i < this.Dataset00.rowcount; i++)
            {
                if(obj.isTreeLeafRow(e.row+i) == false)
                {
                    this.Dataset00.setColumn(e.row+i,"Status",1);
                } else {
                    break;
                }
            }
        } else {
            this.Dataset00.setColumn(e.row,"Status",null);
            for(var i=1; i < this.Dataset00.rowcount; i++)
            {
                if(obj.isTreeRootRow(e.row+i) == false)
                {
                    this.Dataset00.setColumn(e.row+i,"Status",null);
                } else {
                    break;
                }
            }
        }
    }
}
```

```

        }
    }
}
this.Dataset00.set_enableevent(true);
}
};

```

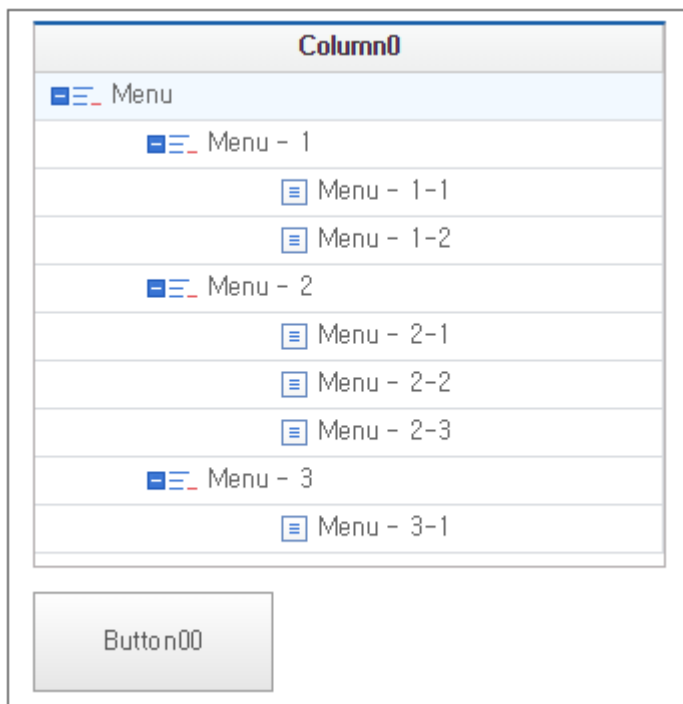
- ⑦ QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. 펼치기 작업 시 자식 노드가 모두 펼쳐지는지 확인합니다.

24.36 트리 형식에서 padding 속성 적용하기

Grid 컴포넌트가 트리 형식으로 표현될 때 레벨에 따라 원하는 너비로 들여쓰기를 하고자 하는 경우 사용할 수 있습니다.

24.36.1 예제

버튼을 클릭하면 padding 스타일 속성이 적용됩니다.





24.36.2 예제에서 사용한 핵심 기능

setCellProperty

Cell 오브젝트의 특정 속성값을 지정합니다. 예제에서는 cssclass 속성값을 버튼 클릭 시 지정합니다.

24.36.3 예제 구현 방법

- ① 화면에 Grid 컴포넌트를 배치합니다.
- ② Dataset 오브젝트를 추가하고 아래와 같이 데이터를 설정합니다.

Columns				
No	id	type	size	prop
1	level	STRING	256	
2	Column0	STRING	256	

Rows		
No	level	Column0
1	0	Menu
2	1	Menu - 1
3	2	Menu - 1-1
4	2	Menu - 1-2
5	1	Menu - 2
6	2	Menu - 2-1
7	2	Menu - 2-2
8	2	Menu - 2-3
9	1	Menu - 3
10	2	Menu - 3-1

- ③ Dataset 오브젝트를 선택하고 Grid 컴포넌트로 드래그앤드롭해서 binddataset 속성으로 등록합니다.
- ④ Grid 컴포넌트의 treeinitstatus 속성값은 "expand,all"로 지정합니다. treeusecheckbox, treeuseline 속성값은 false로 지정합니다.

- 5 Grid 컴포넌트를 더블클릭해서 Grid Content Editor를 실행하고 아래와 같이 속성값을 지정합니다.

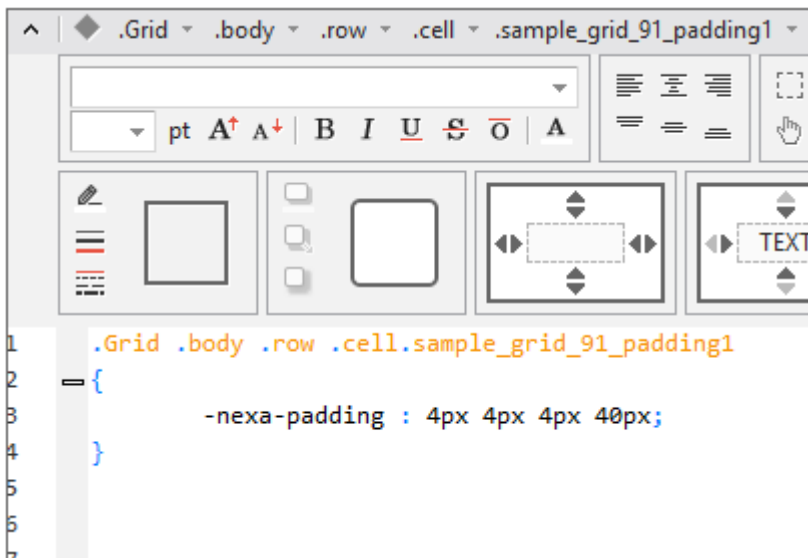
```
displaytype: treeitemcontrol
editttype: tree
text: bind:Column0
treelevel: bind:level
```

- 6 Button 컴포넌트의 onclick 이벤트 함수를 아래와 같이 추가합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Grid00.setCellProperty("body", 0, "cssclass", "expr:level==1?'sample_grid_91_
padding1':level==2?'sample_grid_91_padding2':''");
};
```

- 7 xcss 파일을 열어서 아래 코드를 추가합니다.

Advanced 탭에서 선택자를 추가하거나 Text 탭에서 직접 코드값을 입력할 수 있습니다.



```
.Grid .body .row .cell .sample_grid_91_padding1
{
    -nexa-padding : 4px 4px 4px 40px;
}

.Grid .body .row .cell .sample_grid_91_padding2
{
    -nexa-padding : 4px 4px 4px 80px;
```

}

- ⑧ QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. 버튼 클릭 시 padding 스타일 속성이 적용되는지 확인합니다.

24.37 체크박스 병합해서 표시하기

suppress 속성을 사용해 특정 그룹 단위로 체크박스 값을 처리하는 예제입니다.

24.37.1 예제

Column1을 기준으로 병합해서 표시합니다. 체크박스 선택 시 병합된 그룹의 데이터는 같은 값으로 변경합니다.

Column0	Column1	CheckBox
☒	Apple	1
		1
☐	Banana	0
		0
		0
☒	Orange	1
		1
		1
		1

 sample_grid_92.xfdl

24.37.2 예제에서 사용한 핵심 기능

suppress

17.0.0.1700 이후 버전에서는 Cell 오브젝트의 displaytype 속성값이 "progressbarcontrol", "treeitemcontrol"인 경우를 제외한 설정에서는 suppress 속성을 적용할 수 있습니다. "checkboxcontrol"의 경우에는 데

이터 값을 기준으로 병합을 처리합니다.

24.37.3 예제 구현 방법

- ① 화면에 Grid 컴포넌트를 배치합니다.
- ② Dataset 오브젝트를 추가하고 아래와 같이 데이터를 설정합니다.

Columns				
No	id	type	size	prop
1	Column0	STRING	256	
2	Column1	STRING	256	

Rows		
No	Column0	Column1
1	0	Apple
2	0	Apple
3	0	Banana
4	0	Banana
5	0	Banana
6	1	Orange
7	1	Orange
8	1	Orange
9	1	Orange

- ③ Dataset 오브젝트를 선택하고 Grid 컴포넌트로 드래그앤드롭해서 binddataset 속성으로 등록합니다.
- ④ Grid 컴포넌트를 더블클릭해서 Grid Content Editor를 실행하고 컬럼을 하나 더 추가합니다. 추가한 컬럼에서는 사용자가 체크박스 선택 시 실제 값이 어떻게 바뀌는지 확인합니다.

body				
	col0	col1	col2	
w0	Column0	Column1	CheckBox	
w1	☐			

controlautosizingtype	both
displaytype	checkboxcontrol
edittype	checkbox
expr	
subsumtext	
suppress	2
suppressalign	first
text	bind:Column0
tooltiptext	

각 컬럼별 속성값을 아래와 같이 지정합니다. suppress 우선순위를 Column1에 두었기 때문에 Column1에 표시되는 값을 기준으로 체크박스가 병합되어 표시됩니다.

```

== Column0
displaytype: checkboxcontrol
editttype: checkbox
text: bind:Column0
suppress: 2

== Column1
text: bind:Column1
suppress: 1

== CheckBox
text: bind:Column0

```

- ⑤ Dataset 오브젝트의 oncolumnchanged 이벤트 함수를 아래와 같이 추가합니다.

사용자가 체크박스를 선택해 값을 변경하면 Column0 컬럼의 값이 변경되는데 같은 그룹 내에 있는 Column0 컬럼값을 모두 같은 값으로 변경합니다.

```

this.Dataset00_oncolumnchanged = function(obj:nexacro.NormalDataset,e:nexacro.
DSColChangeEventInfo)
{
    if( e.columnid == "Column0" ){
        obj.set_enableevent(false);
        for(var i=0; i<obj.rowcount; i++)
        {
            if( obj.getColumn(e.row,"Column1") == obj.getColumn(i,"Column1"))
                obj.setColumn(i,e.columnid, e.newvalue);
        }
        obj.set_enableevent(true);
    }
};

```

- ⑥ QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. 체크박스가 Column1을 기준으로 병합되어 표시되는지 확인하고 체크박스 선택 시 같은 값으로 CheckBox 컬럼에 표시되는지 확인합니다.

24.38 셀에 입력한 문자열 길이 표시하기

TextArea 형태의 셀 사용 시 입력한 문자열 길이를 바로 표시하는 기능을 살펴봅니다.

24.38.1 예제

텍스트 입력 중인 셀 영역 옆에 입력할 수 있는 글자수와 입력한 글자수를 톨팁 형태로 표시합니다.

Column0	Column1	Column2
TEST	TEST SAMPL I	10 / 20



24.38.2 예제에서 사용한 핵심 기능

getEditingText

셀 편집 시 입력 중인 텍스트 값을 반환합니다. 편집 시에만 반환하기 때문에 이미 입력된 텍스트 길이를 가져 오기 위해서는 getCellText 메소드를 사용해야 합니다.

getCellText

지정한 위치의 셀에 있는 텍스트 값을 반환합니다. 예제에서는 편집 중에는 getEditingText 메소드를 사용하고 셀 위치를 변경했을 때 바로 텍스트 길이를 표시하기 위해 getCellText 메소드를 사용합니다.

24.38.3 예제 구현 방법

- 1 화면에 Grid 컴포넌트를 배치합니다.
- 2 Dataset 오브젝트를 추가하고 아래와 같이 데이터를 설정합니다.

예제에서는 빈 영역에 값을 입력하는 것을 보여주려고 값을 설정하지 않은 Row를 3개 추가합니다.

Columns						
No	id	type	size	prop	sumtext	di
1	Column0	STRING	256			
2	Column1	STRING	256			
3	Column2	STRING	256			

Rows			
No	Column0	Column1	Column2
1	[Undefined]	[Undefined]	[Undefined]
2	[Undefined]	[Undefined]	[Undefined]
3	[Undefined]	[Undefined]	[Undefined]

③ Dataset 오브젝트를 선택하고 Grid 컴포넌트로 드래그앤드롭해서 binddataset 속성으로 등록합니다.

④ Grid 컴포넌트를 더블클릭해서 Grid Content Editor를 실행하고 셀 속성을 아래와 같이 설정합니다.

```
displaytype: textareacontrol
editttype: textarea
```

⑤ 첫 번째와 두 번째 컬럼의 textareamaxlength 속성값을 10, 20으로 설정합니다.

세 번째 컬럼은 기본값(-1)을 유지합니다.

⑥ Grid 컴포넌트의 autoenter 속성값을 "select"로 변경합니다.

⑦ Grid 컴포넌트 아래에 Static 컴포넌트를 추가하고 속성값을 아래와 같이 설정합니다.

```
text: "0 / 10"
visible: false
border: 1px solid black
textAlign: center
```

⑧ Grid 컴포넌트의 oncellposchanged 이벤트를 함수를 아래와 같이 추가합니다.

TextArea 형태의 셀 편집을 위해 셀 위치가 변경될 때 Static 컴포넌트의 위치를 셀 옆에 위치하도록 처리합니다. textareamaxlength 속성값을 설정하지 않은 경우에는 길이 표시를 하지 않도록 처리합니다.

```
var maxlength = 0;

this.Grid00_0ncellposchanged = function(obj:nexacro.Grid,e:nexacro.GridSelectEventInfo)
```

```

{
  if(this.Grid00.getCellProperty("body", e.cell, "edittype") == "textarea")
  {
    maxlength = this.Grid00.getCellProperty("body", e.cell, "textareamaxlength");
    if(maxlength == -1)
    {
      this.Static00.set_visible(false);
    }
    else
    {
      var objRet = this.Grid00.getCellRect(e.row, e.cell);
      var vL = this.Grid00.getOffsetLeft() + objRet.left + objRet.width;
      var vT = this.Grid00.getOffsetTop() + objRet.top;

      this.Static00.move(vL, vT + 1);
      this.fn_checklength(e.row, e.cell);
      this.Static00.set_visible(true);
    }
  }
  else
    this.Static00.set_visible(false);
};

```

- 9 Grid 컴포넌트의 oninput 이벤트 함수를 아래와 같이 추가합니다.

문자열 입력 시 글자수를 체크하고 Static 컴포넌트에 표시합니다.

```

this.Grid00_oninput = function(obj:nexacro.Grid,e:nexacro.InputEventInfo)
{
  this.fn_checklength();
};

this.fn_checklength = function(row, cell)
{
  var length = 0;
  if(this.Grid00.getEditingText())
    length = this.Grid00.getEditingText().length;
  else if(this.Grid00.getCellText(row, cell))
    length = this.Grid00.getCellText(row, cell).length;
  this.Static00.set_text(length + " / "+maxlength);
}

```


10 QuickView(Ctrl + F6)를 실행하여 셀에 텍스트를 입력하고 입력한 값만큼 글자수를 표시하는지 확인합니다.

24.39 트리 형식에 마스크 문자 적용하기

셀에서 처리할 수 있는 displaytype 속성값은 하나이기 때문에 트리 형식과 마스크 문자를 동시에 적용할 수는 없습니다. 별도 함수를 사용해 마스크 문자를 적용하는 방법을 살펴봅니다.

24.39.1 예제

트리 형식의 셀에 마스크 문자를 적용해 표현합니다.

code
1111-1111-*****
2222-2222-*****
5555-5555-*****
6666-6666-*****
7777-7777-*****

 sample_grid_104.xfdl

24.39.2 예제에서 사용한 핵심 기능

getDisplayText

화면에 보여지는 문자열값을 반환합니다. MaskEdit 컴포넌트는 format 속성 등을 적용해서 최종적으로 화면에 보여지는 문자열값을 반환합니다.

24.39.3 예제 구현 방법

- 1 화면에 Grid 컴포넌트를 배치합니다.
- 2 Dataset 오브젝트를 추가하고 아래와 같이 데이터를 설정합니다.

Columns				
No	id	type	size	prop
1	Column0	STRING	256	
2	code	STRING	256	

Rows		
No	Column0	code
1	0	111111111111
2	0	222222222222
3	1	333333333333
4	1	444444444444
5	0	555555555555
6	1	666666666666
7	0	777777777777

- 3 Dataset 오브젝트를 선택하고 Grid 컴포넌트로 드래그앤드롭해서 binddataset 속성으로 등록합니다.
- 4 Grid 컴포넌트를 더블클릭해서 Grid Content Editor를 실행하고 셀 속성을 아래와 같이 설정합니다.

```
displaytype: treeitemcontrol
editttype: tree
expr: comp.parent.fn_getMaskString(currow)
```

- 5 MaskEdit 컴포넌트를 추가하고 아래와 같이 속성을 설정합니다.

MaskEdit 컴포넌트는 화면에 표시되지 않고 마스크 문자를 처리하기 위한 용도로만 사용합니다.

```
format: @@@@-@@@@-{@@@@}
type: String
visible: false
```

- 6 Grid 컴포넌트의 셀 expr 속성값으로 설정한 fn_getMaskString 함수를 아래와 같이 작성합니다.

code 컬럼값을 가져와서 MaskEdit 컴포넌트 value 속성값으로 설정하고 화면에 보여지는 문자열을 가져와서

반환합니다. 반환한 문자열은 Grid 컴포넌트에 표시합니다.

```
this.fn_getMaskString = function(nRow)
{
    var nStr = this.Dataset00.getColumn(nRow, "code");
    this.MaskEdit00.set_value(nStr);
    var rtn = this.MaskEdit00.getDisplayText();
    return rtn;
};
```

- ⑦ QuickView(Ctrl + F6)를 실행하여 원하는 마스크 문자가 적용됐는지 확인합니다.

24.40 병합된 셀의 체크박스 체크하기

두 개 이상의 셀을 병합해서 하나의 셀처럼 보이도록 할 수 있습니다. 하지만, 병합된 셀을 자식 셀을 유지하고 있더라도 하나의 셀처럼 동작하기 때문에 바로 편집을 할 수 없습니다. 예제에서는 병합된 셀에 포함된 체크박스를 다루는 방법을 살펴봅니다.

24.40.1 예제

같은 형태로 병합된 3개의 Grid 컴포넌트지만, 체크박스 처리 방식이 각각 다릅니다. 첫 번째 Grid 컴포넌트는 체크박스 영역을 선택해도 반응을 하지 않으며 두 번째 Grid 컴포넌트는 셀 영역 어느 곳이든 클릭하면 체크박스가 선택되고 세 번째 Grid 컴포넌트는 CheckBox 컨트롤 영역을 클릭하면 체크박스가 선택됩니다.

GRID_01	GRID_02	GRID_03
☒ test1	☒ test1	☒ test1
☒ test2	☒ test2	☒ test2
☐ test3	☐ test3	☐ test3
☒ test4	☒ test4	☒ test4

24.40.2 예제에서 사용한 핵심 기능

fromreferenceobject

이벤트가 발생한 시작점이 되는 오브젝트를 반환합니다. 하위 컨트롤이 있는 경우에는 하위 컨트롤 오브젝트를 반환합니다. 예제에서는 셀을 병합했지만, 자식 셀을 유지하도록 했기 때문에 fromreferenceobject 속성은 하위 컨트롤 오브젝트를 반환합니다.

24.40.3 예제 구현 방법

- ① 화면에 Grid 컴포넌트를 배치합니다.
- ② Dataset 오브젝트를 추가하고 아래와 같이 데이터를 설정합니다.

Columns				
No	id	type	size	prop
1	Column0	STRING	256	
2	Column1	STRING	256	

Rows		
No	Column0	Column1
1	1	test1
2	0	test2
3	0	test3
4	1	test4

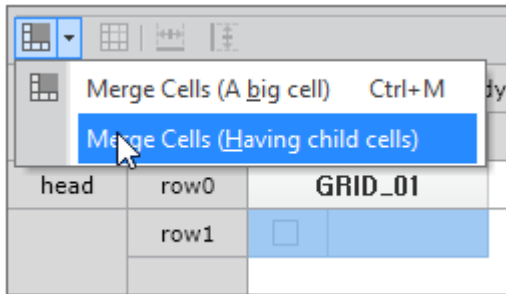
- ③ Dataset 오브젝트를 선택하고 Grid 컴포넌트로 드래그앤드롭해서 binddataset 속성으로 등록합니다.
- ④ Grid 컴포넌트를 더블클릭해서 Grid Content Editor를 실행하고 셀 속성을 아래와 같이 설정합니다.

body	
col0	col1
GRID_01	
size: 40	size: 80

cssclass	
id	body0
Action	
autosizecol	default
autosizerow	default
controlautosi	both
displaytype	checkboxcontrol
edittype	checkbox

- ⑤ 두 개의 셀을 선택하고 병합해줍니다.

병합 옵션은 "Merge Cells (Having child Cells)"를 선택합니다. 체크박스 컨트롤과 텍스트가 그대로 보이도록 합니다.



- ⑥ Grid 컴포넌트를 복사해서 2개 더 같은 컴포넌트를 만듭니다.

구분을 위해 헤더 텍스트만 변경해줍니다.

- ⑦ 새로 만든 두 번째, 세 번째 Grid 컴포넌트의 oncellick 이벤트 함수를 아래와 같이 추가합니다.

두 개의 이벤트 함수는 비슷해보이지만, 세 번째 Grid 컴포넌트는 GridClickEventInfo 오브젝트의 fromreferenceobject 속성값을 확인하고 "CheckBoxControl"인 경우에만 해당 컬럼값을 업데이트합니다.

```
this.Grid02_oncellick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    if(this.Dataset00.getColumn(e.row, "Column0") == 1)
        this.Dataset00.setColumn(e.row, "Column0", 0);
    else
        this.Dataset00.setColumn(e.row, "Column0", 1);
};

this.Grid03_oncellick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    if(e.fromreferenceobject == "[object CheckBoxControl]")
    {
        if(this.Dataset00.getColumn(e.row, "Column0") == 1)
            this.Dataset00.setColumn(e.row, "Column0", 0);
        else
            this.Dataset00.setColumn(e.row, "Column0", 1);
    }
};
```

- ⑧ QuickView(Ctrl + F6)를 실행하여 각 Grid 컴포넌트의 체크박스 선택 시 동작을 확인합니다.

25.

Grid 응용

25.1 Grid에서 엑셀로 복사/붙여넣기

컴퓨터를 사용할 때 시스템 클립보드를 통한 복사(**Ctrl**+**C**) / 붙여넣기(**Ctrl**+**V**) 기능은 사용의 편의성으로 인해 자주 사용됩니다. 마찬가지로 넥사크로플랫폼에서도 기능을 구현하면 복사/붙여넣기를 사용할 수 있습니다.

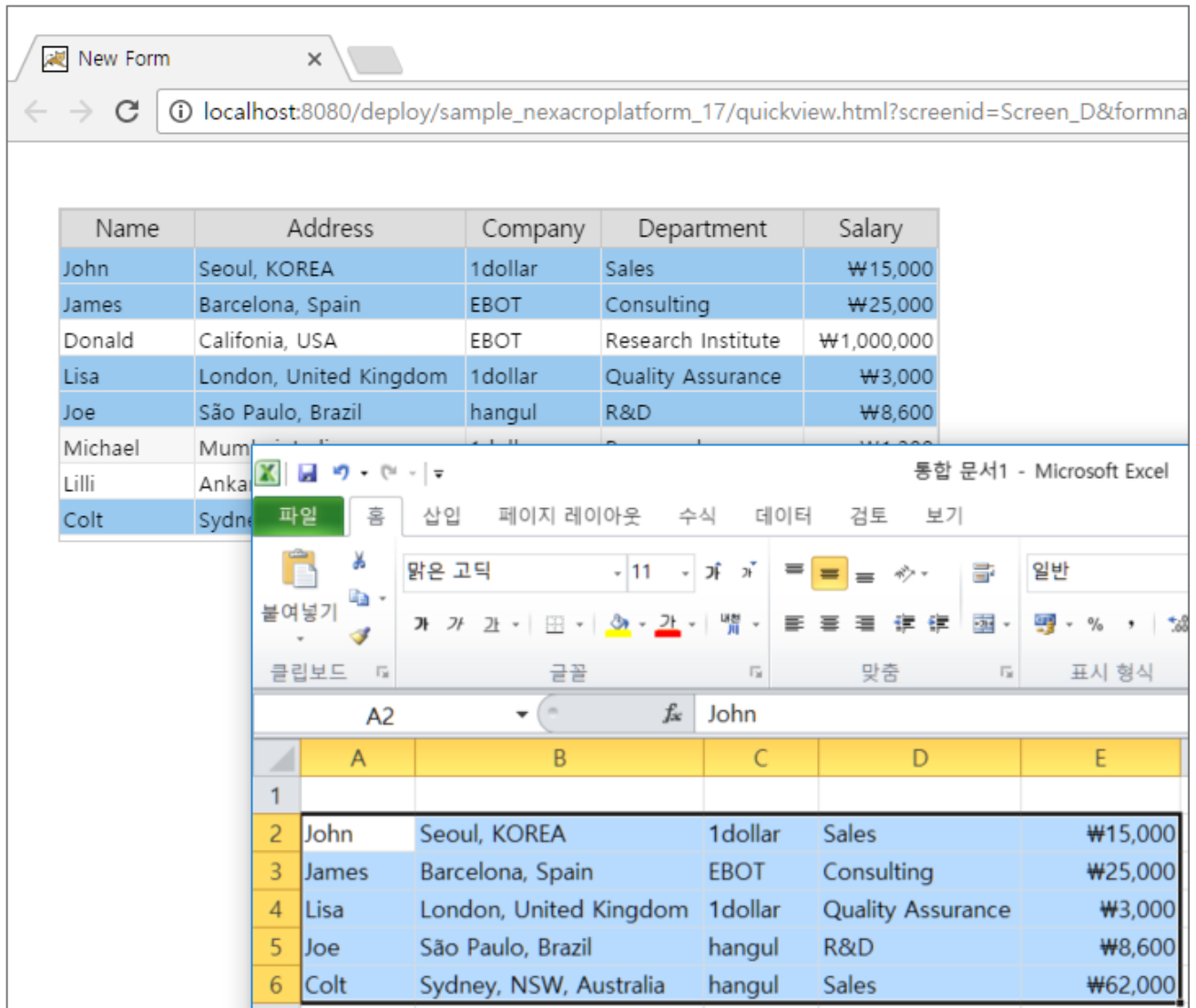
본 장에서는 Grid의 데이터를 시스템에 있는 엑셀, 메모장을 비롯한 애플리케이션으로 복사/붙여넣기 하는 기능을 구현하고 그 방법을 설명합니다.

25.1.1 예제

다음은 Grid의 데이터를 엑셀로 복사/붙여넣기 하는 예제입니다.

Grid의 Row를 마우스 드래그나 키보드를 사용해 선택한 후 **Ctrl** + **C**를 누르면 복사됩니다. 그런 후 시스템에 있는 엑셀에서 **Ctrl** + **V**를 눌러 붙여넣기 합니다.

Grid의 selecttype 속성이 multirow로 설정되어 있기 때문에 Row 단위로만 선택할 수 있습니다.



The screenshot shows a web browser window with a 'New Form' tab. The address bar displays 'localhost:8080/deploy/sample_nexacroplatform_17/quickview.html?screenid=Screen_D&formna'. The main content area contains a table with the following data:

Name	Address	Company	Department	Salary
John	Seoul, KOREA	1dollar	Sales	₩15,000
James	Barcelona, Spain	EBOT	Consulting	₩25,000
Donald	Califonia, USA	EBOT	Research Institute	₩1,000,000
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
Joe	São Paulo, Brazil	hangul	R&D	₩8,600
Michael	Mum			
Lilli	Anka			
Colt	Sydne			

An Excel spreadsheet titled '통합 문서1 - Microsoft Excel' is overlaid on the table. It shows the same data in a more detailed format, with columns A through E and rows 1 through 6. The data is as follows:

	A	B	C	D	E
1					
2	John	Seoul, KOREA	1dollar	Sales	₩15,000
3	James	Barcelona, Spain	EBOT	Consulting	₩25,000
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
5	Joe	São Paulo, Brazil	hangul	R&D	₩8,600
6	Colt	Sydney, NSW, Australia	hangul	Sales	₩62,000

 sample_grid_30.xfdl

25.1.2 예제에서 사용한 핵심 기능

navigatorname

브라우저의 이름 정보를 갖는 읽기 전용 속성입니다.

넥사크로플랫폼 런타임	nexacro
인터넷 익스플로러	IE
크롬	Chrome
파이어폭스	Gecko

setEventHandler

Grid의 이벤트 함수를 변경하는 메소드입니다.

selectstartrow

Grid에서 선택된 영역의 시작 셀의 Row 인덱스를 갖는 읽기전용 속성입니다.

selectendrow

Grid 에서 선택된 영역의 마지막 셀의 Row 인덱스를 갖는 읽기전용 속성입니다.

selectstartcol

Grid 에서 선택된 영역의 시작 셀의 Column 인덱스를 갖는 읽기전용 속성입니다.

selectendcol

Grid 에서 선택된 영역의 마지막 셀의 Column 인덱스를 갖는 읽기전용 속성입니다.

getFormatColCount

Grid 에 표시되고 있는 포맷에 정의된 Column 의 갯수를 반환하는 메소드입니다

getCellText

인수로 전달된 위치의 셀에 표시되는 텍스트 값을 반환하는 메소드입니다.

setClipboard

시스템의 클립보드에 지정된 형식으로 데이터를 저장하는 메소드입니다.

clearClipboard

시스템의 클립보드에 저장된 내용을 삭제하는 메소드입니다.

keycode

키보드로 입력된 키에 해당하는 코드값을 갖는 속성입니다

ctrlkey

이벤트 발생시 **Ctrl** 키가 눌렸는지 여부를 확인하는 속성입니다.

shiftkey

이벤트 발생시 **Shift** 키가 눌렸는지 여부를 확인하는 속성입니다.

altkey

이벤트 발생시 **Alt** 키가 눌렸는지 여부를 확인하는 속성입니다.

25.1.3 예제 구현 방법

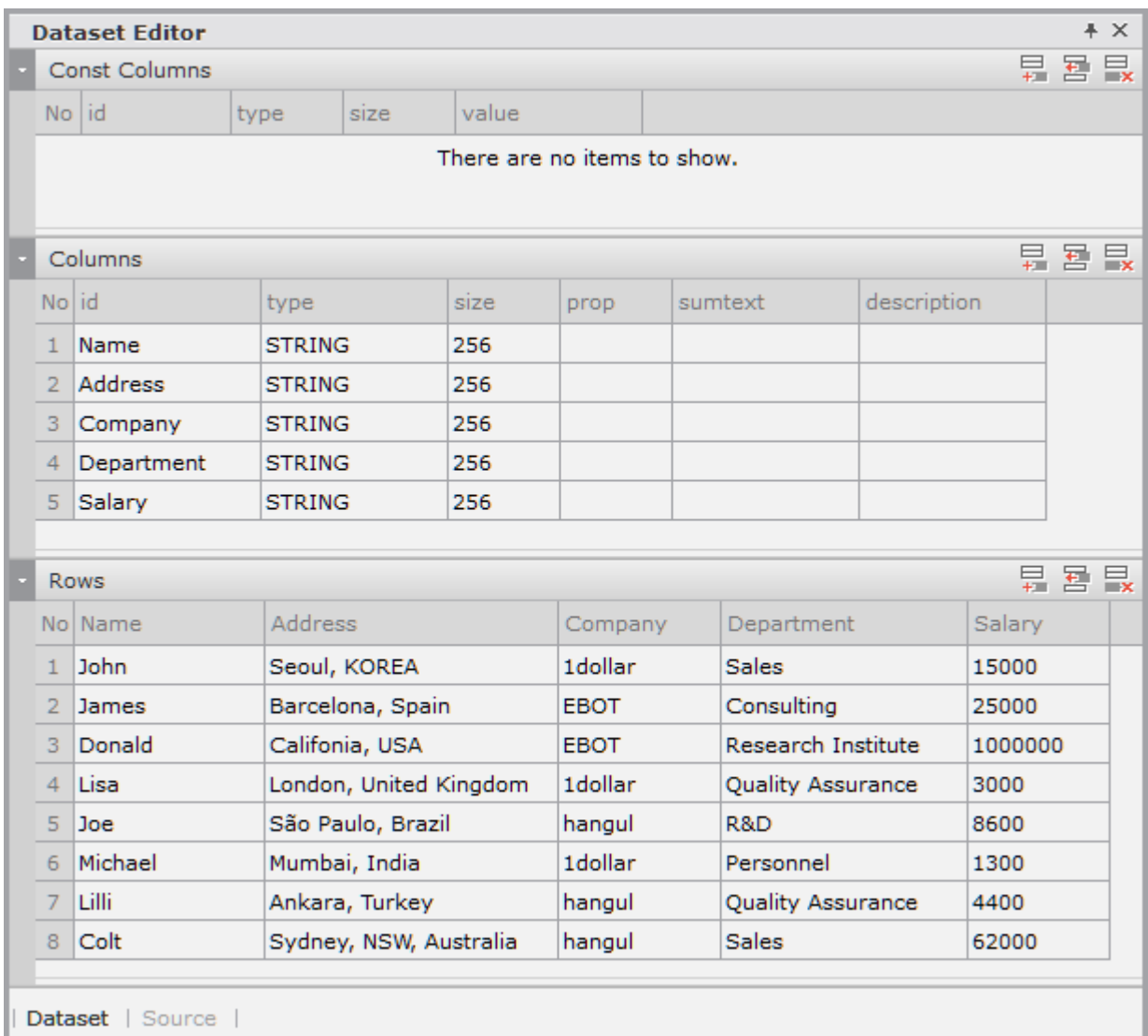
1 컴포넌트 생성하기

툴바에서 그리드  를 선택한 후, Form 위에 적당한 크기로 드래그하여 생성합니다. 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

2 Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.



The screenshot shows the 'Dataset Editor' window. It has three main sections: 'Const Columns', 'Columns', and 'Rows'.

Const Columns: A table with 5 columns: No, id, type, size, value. It contains the message 'There are no items to show.'

Columns: A table with 8 columns: No, id, type, size, prop, sumtext, description. It contains 5 rows of column definitions.

No	id	type	size	prop	sumtext	description
1	Name	STRING	256			
2	Address	STRING	256			
3	Company	STRING	256			
4	Department	STRING	256			
5	Salary	STRING	256			

Rows: A table with 7 columns: No, Name, Address, Company, Department, Salary. It contains 8 rows of data.

No	Name	Address	Company	Department	Salary
1	John	Seoul, KOREA	1dollar	Sales	15000
2	James	Barcelona, Spain	EBOT	Consulting	25000
3	Donald	Califonia, USA	EBOT	Research Institute	1000000
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000
5	Joe	São Paulo, Brazil	hangul	R&D	8600
6	Michael	Mumbai, India	1dollar	Personnel	1300
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000

At the bottom, there is a status bar with 'Dataset | Source |'.

③ Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

④ Grid 속성 설정하기

Grid를 다중 선택이 가능하도록 selecttype 속성을 'multirow'로 설정합니다.

⑤ Form 이벤트 함수 작성하기

Form을 선택한 후 onload 이벤트 함수를 다음과 같이 작성합니다.

```
this.Form_onload = function(obj:nexacro.Form,e:nexacro.LoadEventInfo)
{
    this.copyPaste.addGrid(this.Grid00);
};
```

⑥ 키 입력과 복사/붙여넣기 처리를 위한 copyPaste 오브젝트 작성하기

키 입력을 처리하고 Grid의 데이터를 브라우저에 따라 시스템의 클립보드 혹은 Textarea 요소로 복사하는 기능을 스크립트로 구현합니다.

기능 구현을 위해 copyPaste 오브젝트를 작성합니다. copyPaste 오브젝트는 키 입력 처리를 위한 addGrid, 선택된 Grid 데이터를 추출하기 위한 makeData, Grid 데이터를 복사하기 위한 copyGridCellData1, copyGridCellData2의 총 4개 메소드로 구성됩니다.

각 메소드에 대한 자세한 설명은 아래 코드의 주석을 참조하시기 바랍니다.

```
this.copyPaste = {

    //브라우저를 체크하여 브라우저에 맞는 onkeydown 이벤트 함수를 설정합니다.
    //브라우저에 따라 클립보드를 이용할지(copyGridCellData1) Textarea를 이용할지(
    copyGridCellData2) 결정합니다.
    addGrid: function(grid)
    {
        //브라우저가 넥사크로플랫폼 Runtime 혹은 IE인 경우
        if (system.navigatorname == "nexacro" || system.navigatorname == "IE" )
        {
            //grid에 onkeydown 이벤트 함수 등록
            grid.setEventHandler("onkeydown", this.copyGridCellData1(this), this);
        }
    }
};
```

```

//그 이외의 브라우저인 경우
else
{
    //HTML DOM의 Textarea 요소를 생성하고 body 요소에 추가합니다.
    var objTA = document.createElement("textarea");
    document.body.appendChild(objTA);

    //grid에 onkeydown 이벤트 함수 등록
    grid.setEventHandler("onkeydown", this.copyGridCellData2(this, objTA), this);
}

},

//Grid에서 선택된 영역의 데이터를 변수에 저장합니다.
makeData: function(obj)
{
    var sBody = "";
    var sSpr = "    "; //Tab 값 입력
    var nStartrow = obj.selectstartrow;
    var nEndrow = obj.selectendrow;
    var nStartCol = obj.selectstartcol;
    var nEndCol = obj.selectendcol;

    //selecttype이 row나 multirow인 경우
    //selectstartcol, selectendcol 값은 -1이 된다.
    //따라서 컬럼의 갯수를 계산하여 nStartCol과 nEndCol을 설정한다.
    var nLoop = nStartrow.length;

    if(nStartCol == -1 && nEndCol == -1)
    {
        nStartCol = 0;
        nEndCol = obj.getFormatColCount()-1;
    }

    //copy cell data from selected cell
    for(var k=0; k < nLoop; k++)
    {
        for (var i = nStartrow[k]; i <= nEndrow[k]; i++)
        {
            for (var j = nStartCol; j <= nEndCol; j++)

```

```

        {
            if (j < nEndCol)
            {
                sBody += obj.getCellText(i, j) + sSpr;
            }
            else
            {
                sBody += obj.getCellText(i, j) + "\r\n";
            }
        }
    }

    return sBody;
},

//IE 혹은 Runtime인 경우
//Ctrl + c가 입력되면 클립보드로 Grid의 선택된 데이터를 복사합니다.
//키 값은 ctrl=17, shift=16, alt=18, c=67 입니다.
copyGridCellData1: function(pThis)
{
    return function(obj, e)
    {
        //Ctrl 키가 눌린 경우
        if (e.ctrlkey && !e.shiftkey && !e.altkey)
        {
            //c 키가 눌린 경우
            if (e.keycode == 67)    //'c'
            {
                var rtnStr = pThis.makeData(obj);

                system.clearClipboard();
                system.setClipboard("CF_TEXT", rtnStr);
            }
        }
    }
},
},

```

```
//IE 혹은 Runtime 이외의 브라우저인 경우
//Ctrl + c가 입력되면 Textarea로 Grid의 선택된 데이터를 복사합니다.
//키 값은 ctrl=17, shift=16, alt=18, c=67 입니다.
copyGridCellData2: function(pThis, objTA)
{
    return function(obj, e)
    {
        //Ctrl 키가 눌린 경우
        if (e.ctrlkey && !e.shiftkey && !e.altkey)
        {
            //c 키가 눌린 경우
            if (e.keycode == 67)
            {
                var rtnStr = pThis.makeData(obj);

                objTA.value = rtnStr;
                objTA.select();
            }
        }
    }
}

}//end of copyGridCellData2: function(pThis, objTA)
```

```
}//end of this.copyPaste
```

7 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Grid를 마우스와 키보드를 사용하여 선택한 후 **Ctrl** + **C**로 복사하고 시스템의 엑셀에 **Ctrl** + **V**로 붙여넣습니다. 선택한 항목들이 모두 엑셀에 정상적으로 복사됐는지 확인합니다.

25.2 합계/소계 구하기

Grid에서는 데이터를 표 형태로 표현하는 기능뿐 아니라 필요한 경우 데이터를 자동으로 연산하는 기능을 사용할 수 있습니다. 기본적으로 지원하는 연산으로는 합계, 평균, 개수, 최대/최소값 등이 있습니다. 이 중 가장 빈번하게 사용하는 연산이 합계입니다. 합계는 컬럼의 총 합을 뜻하며 같은 값을 갖는 컬럼을 그룹별로 묶어 합을 구하는 것을 소개

라 합니다.

이번 장에서는 컬럼의 합계(total)와 소계(subtotal)를 구하는 방법에 대해 설명합니다.

25.2.1 예제

다음 Grid는 Salary 컬럼의 합계를 구하고 Company 컬럼으로 그룹을 지어 그룹별 소계를 계산한 예제입니다.

합계는 Grid의 가장 하단에 표시되고 소계는 각 그룹의 아래에 새로운 Row가 생성되어 표시됩니다.

Name	Address	Company	Department	Salary	
John	Seoul, KOREA	1dollar	Sales	\$15,000.00	1dollar group
Lisa	London, United Kingdom		Quality Assurance	\$3,000.00	
Michael	Mumbai, India		Personnel	\$1,300.00	
				\$19,300.00	
James	Barcelona, Spain	EBOT	Consulting	\$25,000.00	EBOT group
Donald	Califonia, USA		Research Institute	\$1,000,000.00	
				\$1,025,000.00	
Joe	São Paulo, Brazil	hangul	R&D	\$8,600.00	hangul group
Lilli	Ankara, Turkey		Quality Assurance	\$4,400.00	
Colt	Sydney, NSW, Australia		Sales	\$62,000.00	
				\$75,000.00	
TOTAL				\$1,119,300.00	



sample_grid_31.xfdl

25.2.2 예제에서 사용한 핵심 기능

expr

Grid에서 Cell의 text 속성 값에 적용할 수식을 설정하는 속성입니다.

getSum

Dataset에서 지정한 범위의 컬럼 값 혹은 계산식의 합계를 구하는 메소드입니다.

suppress

Grid에서 Cell에 suppress 기능을 설정하는 속성입니다. suppress 기능은 한 컬럼에 같은 값을 가진 셀이 여러 개 반복적으로 나열되어 있을 경우 하나의 Cell로 합쳐 보여줍니다.

keystring

Dataset에 로드된 데이터를 그룹/정렬할 기준이 되는 조건식을 설정하는 속성입니다. 그룹 타입을 생략하면 G 옵션이 적용됩니다.

```
this.Dataset00.set_keystring( "column0" );
this.Dataset00.set_keystring( "G:column0" );
this.Dataset00.set_keystring( "G:+column0-column1" );
this.Dataset00.set_keystring( "G:+column0,S:-column1" );
```

prop

Dataset의 keystring 속성에 의해 그룹핑된 논리적 레코드에 표시되는 값의 유형을 설정하는 속성입니다.

연산	설명
NONE	(default) 아무 값도 표시하지 않습니다.
AVG	그룹의 평균값을 계산합니다.
COUNT	그룹의 개수를 계산합니다.
KEY	그룹의 키 값을 표시합니다.
MAX	그룹의 최대 값을 표시합니다.
MIN	그룹의 최소 값을 표시합니다.
SUM	그룹의 합계를 계산합니다.
TEXT	sumtext 속성값을 표시합니다. sumtext 속성은 Grid의 Summary 영역에 표시할 문자를 설정하는 속성입니다.

displaytype

Grid의 Cell에서 데이터가 표시되는 형식을 설정하는 속성입니다.

25.2.3 예제 구현 방법

① 컴포넌트 생성하기

툴바에서 그리드  를 선택한 후, Form 위에 적당한 크기로 드래그하여 생성합니다. 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

② Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset을 더블 클

릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.

Dataset Editor [Dataset00]

Const Columns

No	id	type	size	value
There are no items to show.				

Columns

No	id	type	size	prop	sumtext	description
1	Name	STRING	256			
2	Address	STRING	256			
3	Company	STRING	256			
4	Department	STRING	256			
5	Salary	INT	256	SUM		

Rows

No	Name	Address	Company	Department	Salary
1	John	Seoul, KOREA	1dollar	Sales	15000
2	James	Barcelona, Spain	EBOT	Consulting	25000
3	Donald	Califonia, USA	EBOT	Research Institute	1000000
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000
5	Joe	São Paulo, Brazil	hangul	R&D	8600
6	Michael	Mumbai, India	1dollar	Personnel	1300
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000

Dataset | Source |

3 Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

4 Grid 설정하기

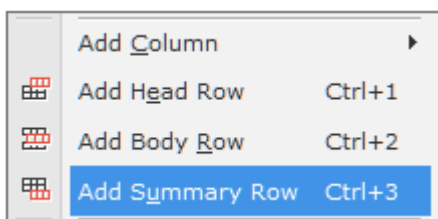
Salary 컬럼의 표시 형식을 화폐로 설정합니다. 다음과 같이 Grid Contents Editor를 열어 Salary 컬럼의 displaytype 속성을 'currency'로 설정합니다.

col4				cssclass	
Salary				id	body4
				Action	
				autosizecol	default
				autosizerow	default
				displaytype	currency

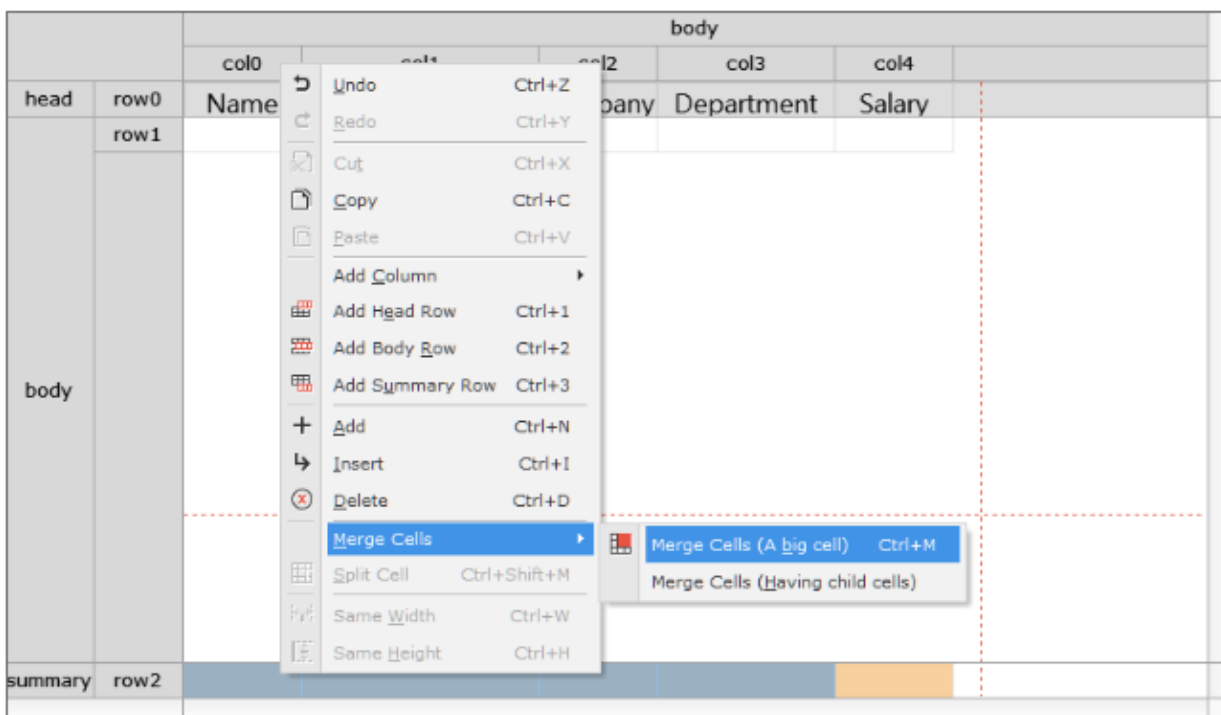
5 합계 구하기

합계를 계산해 표시하려면 우선 Grid의 어느 부분에 표시할지 정한 후 expr이나 스크립트를 사용해 합계를 계산후 해당 위치에 결과를 표시합니다. 이 샘플에서는 서머리 밴드에 합계를 표시합니다.

1. 서머리 밴드에 새로운 로우를 추가합니다. Grid를 더블클릭하여 Grid Contents Editor를 열고 마우스 오른쪽 버튼을 눌러 컨텍스트 메뉴에서 [Add Summary Row]를 선택합니다.



2. Grid상에 TOTAL로 표시하기 위해 새로 추가한 로우의 col0~col3까지를 하나의 Cell로 합칩니다. 다음 그림과 같이 col0~col3까지 Cell을 선택한 후 마우스 오른쪽 버튼을 눌러 컨텍스트 메뉴에서 [Merge Cells] 메뉴를 선택합니다. Cell이 하나로 합쳐지면 합쳐진 Cell의 text 속성을 'TOTAL'로 설정합니다.



QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Grid의 데이터가 Company 컬럼을 기준으로 그룹이 지어졌는지, 각 그룹별로 소계가 표시되는지, Grid 하단에 합계가 표시되는지 확인합니다.

25.3 소계 이름과 배경색 설정하기

앞의 [합계/소계 구하기](#)에서 합계와 소계를 구하는 방법에 대해 알아보았습니다. 합계의 경우 기본 테마에서 Summary 밴드 영역에 대해 정의된 스타일이 있어 한눈에 알아볼 수 있게 되어 있습니다. 그러나 소계의 경우에는 행 사이에 있고 소계라는 어떤 표시도 없기 때문에 사용자가 한눈에 알아볼 수 없습니다. 이는 기본 테마에서 소계에 대해 정의된 스타일이 없기 때문입니다. 소계는 Dataset 연산으로 생성된 논리적인 결과물이라 어느 위치에 어떤 형태로 출력될지 예측할 수 없기 때문에 무조건 일괄 적용되는 테마의 스타일로는 정의하기 힘듭니다. 따라서 개발자가 Dataset의 내용을 분석하여 화면을 디자인하는 시점에 스타일을 설정해야 합니다.

이번 장에서는 소계에 이름과 배경을 설정하는 방법에 관해 설명합니다. 사용할 예제는 앞 장에서 작성한 [예제](#)를 그대로 사용합니다. 예제 샘플이 없다면 앞 장을 참조하여 합계/소계가 적용된 Grid 예제를 먼저 작성하십시오.

25.3.1 예제

다음은 Salary Column의 합계를 구하고 Company Column으로 그룹을 지어 그룹별 소계를 계산한 Grid 예제입니다. 합계는 Grid의 가장 하단에 표시되고 소계는 각 그룹의 아래에 새로운 Row로 생성되어 표시됩니다.

Name	Address	Company	Department	Salary
John	Seoul, KOREA	1dollar	Sales	\$15,000.00
Lisa	London, United Kingdom		Quality Assurance	\$3,000.00
Michael	Mumbai, India		Personnel	\$1,300.00
		SUBTOTAL		\$19,300.00
James	Barcelona, Spain	EBOT	Consulting	\$25,000.00
Donald	California, USA		Research Institute	\$1,000,000.00
		SUBTOTAL		\$1,025,000.00
Joe	São Paulo, Brazil	hangul	R&D	\$8,600.00
Lilli	Ankara, Turkey		Quality Assurance	\$4,400.00
Colt	Sydney, NSW, Australia		Sales	\$62,000.00
		SUBTOTAL		\$75,000.00
TOTAL				\$1,119,300.00



소계의 이름은 Cell의 `expr` 속성을 사용하여 설정합니다. `expr` 스크립트로 해당 Row가 그룹화되지 않은 일반 Row인지 그룹화되어 생성된 소계 Row인지 판단하여 그에 맞는 결과를 출력합니다. 소계 Row인지는 Row의 레벨 값으로 확인할 수 있습니다. 일반 Row는 0 값을 그룹화된 Row는 1 이상의 값을 갖습니다.

소계의 배경색은 Cell의 `cssclass` 속성과 스타일을 정의한 XCSS 파일을 사용해 설정합니다. `expr` 스크립트로 해당 Row가 일반 Row인지 소계 Row인지 판단하여 소계 Row일 경우 XCSS에 정의된 클래스 선택자를 `cssclass` 속성에 설정합니다.



XCSS(넥사크로 스타일 시트)는 넥사크로플랫폼에서 사용할 수 있는 스타일 시트입니다. 자세한 설명은 스타일 활용 (XCSS) 장을 참조하십시오.

25.3.2 예제에서 사용한 핵심 기능

`getRowLevel`

Dataset을 그룹화했을 경우 지정한 Row의 레벨 값을 반환하는 메소드입니다. 일반 Row의 레벨 값은 0이며, 그룹화한 Row는 그룹화 정도에 따라 1 이상의 값을 갖습니다.

`cssclass`

XCSS 파일에 정의한 클래스 선택자의 스타일을 Cell에 적용하는 속성입니다. 설정값으로는 클래스 선택자 명을 입력합니다.

`expr`

수식을 입력하여 그 결과 값을 Cell의 `text` 속성에 적용하는 속성입니다.

25.3.3 예제 구현 방법

① Grid 생성, 합계/소계 하기

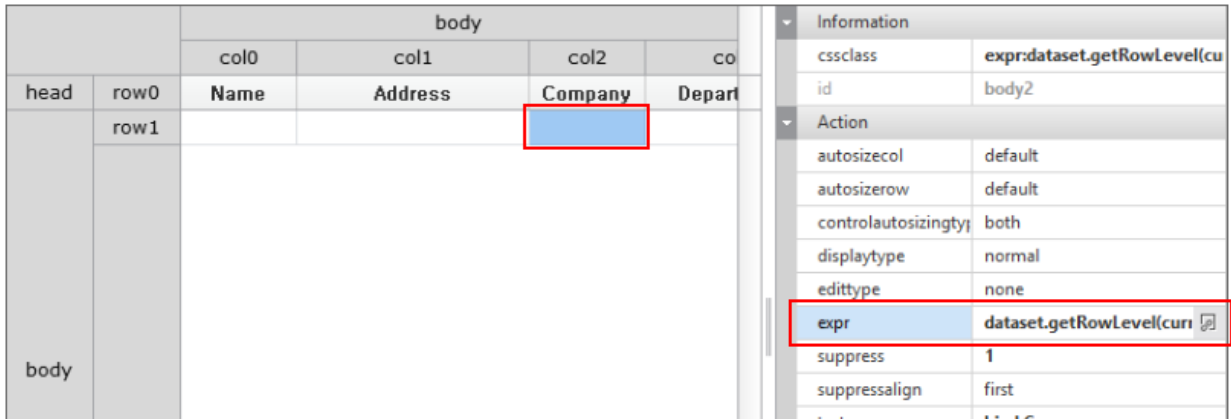
[합계/소계 구하기](#)를 참조하여 합계/소계를 포함한 Grid를 생성합니다.

② 소계 이름 설정하기

Company Column을 기준으로 그룹화 했으므로 Company 열에 해당하는 소계 Row에 소계 이름을 출력합니다.

1. 디자인 화면의 Grid를 더블 클릭하여 Grid Contents Editor를 엽니다.

2. Company Column에서 데이터가 출력되는 Body 밴드 영역의 row1 Cell을 선택 후 expr 속성을 설정합니다.



속성	값
expr	dataset.getRowLevel(curren)=='SUBTOTAL':Company

expr 수식은 현재 Grid에 바인딩 된 데이터셋에서 현재 Row의 레벨 값이 1인 경우에는 현재 Row에 "SUBTOTAL"이라고 문자열을 출력하고 아닌 경우에는 원래 출력해야 하는 Company Column의 Row값을 출력하라는 의미입니다.

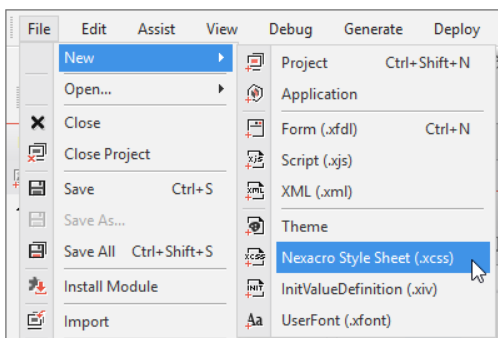
현재 Row가 소계 Row인지는 Row의 레벨 값으로 알 수 있습니다. 레벨 값은 일반 Row이면 0 값을, 그룹화된 Row이면 1 이상의 값을 갖습니다. 본 예제의 데이터셋은 Company Column으로 한 번만 그룹화했기에 1 값을 갖게 됩니다. 현재 Row의 인덱스는 currow 변수로, Row의 레벨 값은 getRowLevel 메소드로 구할 수 있습니다.



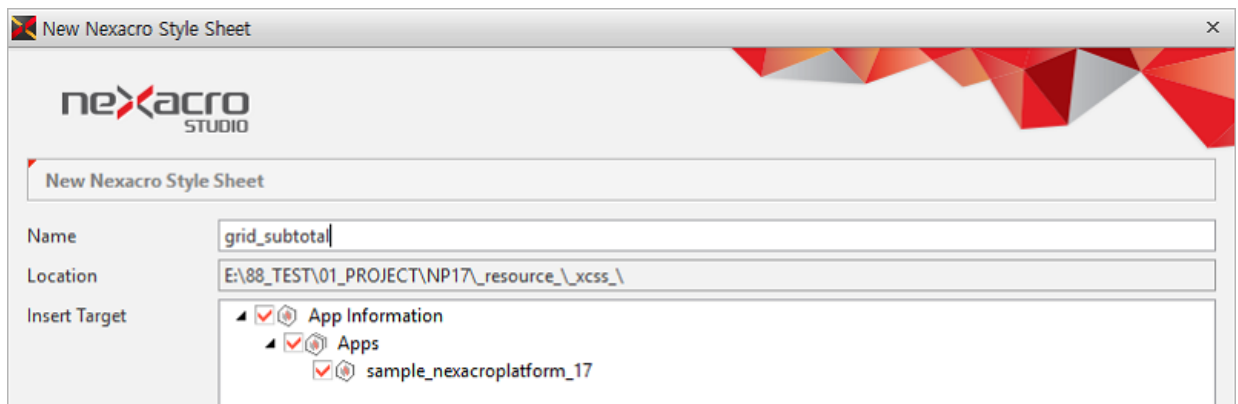
수식에서 사용된 currow 변수는 expr 수식에서만 사용할 수 있는 특수한 변수입니다. expr에서 사용할 수 있는 변수 및 함수에 대한 내용은 레퍼런스가이드의 GridCellControl 객체를 참조하십시오.

① XCSS 파일 생성, class 선택자 정의하기

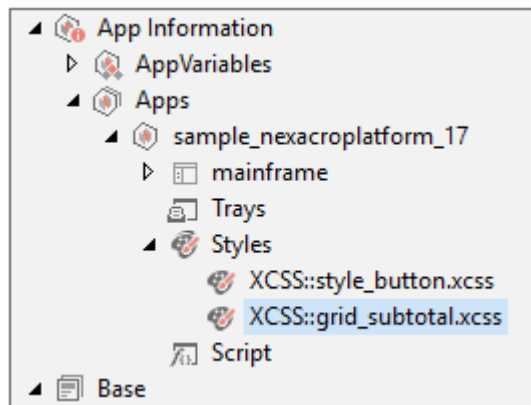
3. Menu에서 [File > New > Nexacro Style Sheet(.xcss)]를 선택하여 새로운 XCSS 파일을 생성합니다.



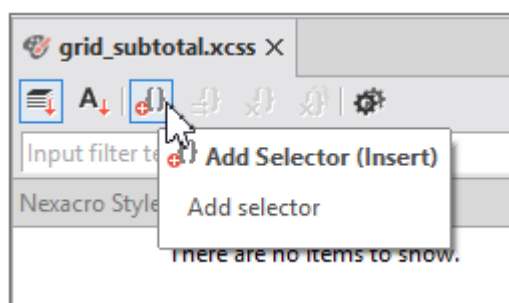
New Style Sheet 창에서 XCSS 이름과 경로, 적용 대상을 다음과 같이 설정합니다.



Finish 버튼을 클릭하면 다음과 같이 Project Explorer에 생성된 XCSS 파일이 나타나고 자동으로 Xcss Editor로 오픈됩니다.

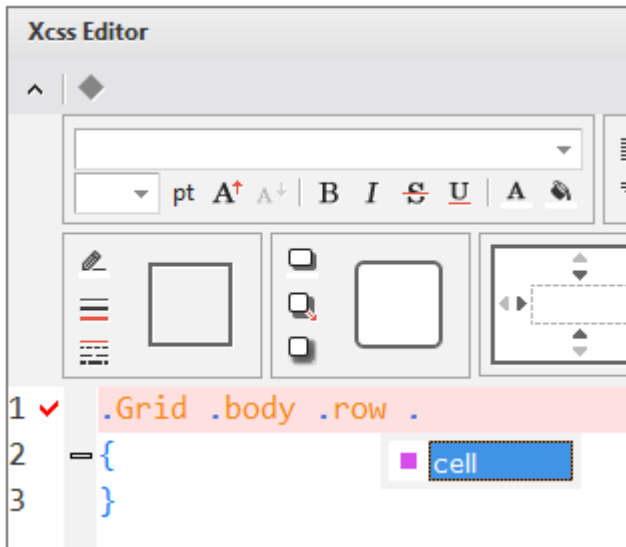


4. Xcss Editor에서 Add Selector 버튼을 클릭하여 새로운 선택자를 추가합니다.



5. Add Selector 창에서 스타일을 정의할 Grid 컴포넌트를 선택하고 Add 버튼을 클릭합니다. 그러면 아무 내용도 없는 Grid 선택자가 새로 생성됩니다.

스타일을 적용할 대상은 Grid의 자식인 Cell이므로 Cell에 적용되는 선택자 형식은 .Grid .body .row .cell 입니다. Xcss Editor에서 .Grid 선택자 뒤로 한 칸을 띄운 후 . 을 입력하면 자동으로 입력 가능한 자식 선택자가 콘텍스트 메뉴로 출력됩니다. body, row, cell을 차례로 선택해줍니다.



6. .Grid .body .row .cell 뒤에 subtotal이라는 이름으로 클래스 선택자를 추가합니다. 여기서 정한 클래스 선택자 이름이 폼 디자인 화면에서 컴포넌트의 cssclass 속성에 설정할 때 사용됩니다.

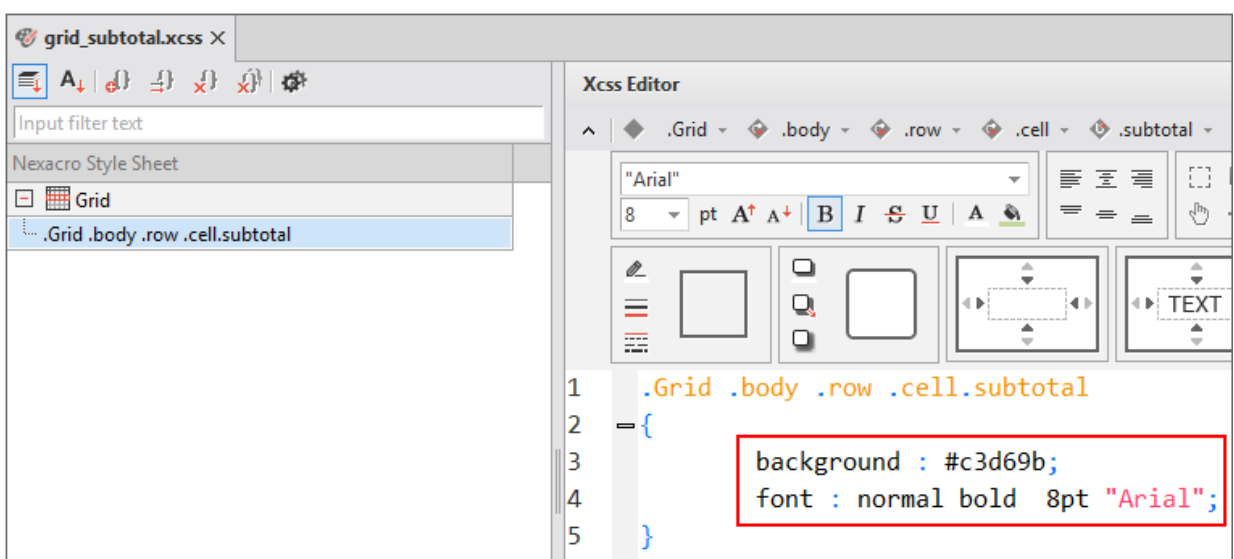
.Grid .body .row .cell.subtotal



클래스 선택자는 앞에 선언한 .Grid 선택자와 그 자식 선택자의 선언과 달리 한 칸을 띄우지 않고 붙여서 사용합니다.

7. Xcss Editor를 사용하여 소계에 적용할 Cell의 스타일을 설정합니다.

다음과 같이 background 속성과 font 속성을 설정한 후 XCSS 파일을 저장합니다. Xcss Editor를 사용하지 않고 사용자가 직접 입력할 수도 있습니다.



② 소계 Row 배경색 설정하기

소계 Row에 배경색을 설정하려면 `cssclass` 속성을 사용합니다. `cssclass`에 값이 설정되면 XCSS 파일에서 설정된 값에 해당하는 클래스 선택자를 찾아 정의된 스타일을 컴포넌트에 적용합니다.

1. Grid를 더블 클릭하여 Grid Contents Editor를 오픈한 후 Body 밴드의 Cell을 모두 선택합니다. 하나 이상의 Cell을 다중 선택 후 특정 속성을 설정하면 선택된 모든 Cell의 속성을 동일한 값으로 한번에 설정할 수 있습니다.

	body				
	col0	col1	col2	col3	col4
row0	Name	Address	Company	Department	Salary
row1					

2. `cssclass` 속성을 다음과 같이 설정합니다.

Information	
cssclass	expr:dataset.getRowLe  
id	
Action	
subtotal	default

속성	값
cssclass	expr:dataset.getRowLevel(currow)==1?'subtotal':''

설정한 `expr` 수식은 현재 Grid에 바인딩 된 데이터셋에서 현재 Row의 레벨 값이 1인 경우에는 `cssclass` 속성에 "subtotal"을 설정하고 아닌 경우에는 설정하지 않는다는 의미입니다. `cssclass` 속성에 설정되는 "subtotal" 값은 `grid_subtotal.xcss`에 정의된 클래스 선택자 이름입니다.

③ QuickView로 확인하기

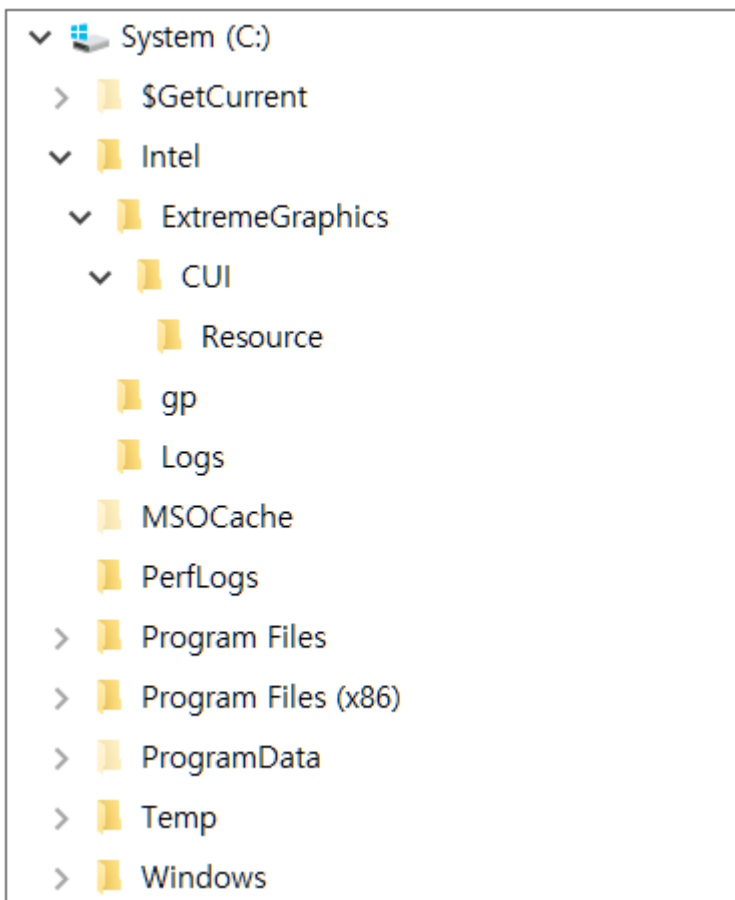
QuickView(Ctrl + F6)를 실행하여 소계 Row에 이름과 배경색이 설정되었는지 확인합니다. `cssclass`가 설정되는 즉시 디자인 화면에서도 확인할 수 있습니다.

25.4 트리 형태로 표현하기

트리는 영어 Tree의 국어 표현으로 나무를 뜻 합니다. 나무는 하나의 가지에서 여러 가지가 뿔어 나오는 모양을 띄는데 데이터간의 연관성을 표현할 때 이 나무와 같은 형태로 표현하는 방법을 트리라 합니다.

Grid는 많은 양의 데이터를 표 형태로 정리하여 보여주는 기능을 합니다. 그러나 어떤 데이터들은 표 형태로 표현하는 것보다 트리 형태로 표현하는 것이 좋은 경우가 있습니다. 예를 들면, 조직 구조 같은 경우가 그렇습니다. 그룹이 있고 그룹 아래 여러 팀이 있고 다시 그 팀 아래 여러 팀원이 존재할 수 있습니다. 이런 경우에는 그룹, 팀, 팀원을 표 형태로 표현하는 것보다 트리 형태로 표현하는 것이 그 관계를 더욱 효과적으로 보여줄 수 있습니다.

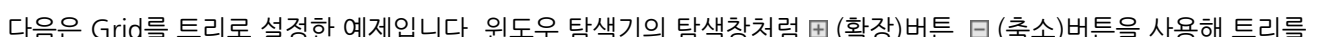
다음은 트리의 쓰임새를 잘 보여주고 있는 윈도우 탐색기의 탐색창입니다. 여기서는 디렉토리의 구조를 보여주기 위한 방법으로 트리를 활용하고 있습니다.



이처럼 트리는 데이터 연관성이 한눈에 들어와 효과적이지만 서로 연관성이 없는 데이터의 경우는 트리 형태가 적합하지 않습니다. 데이터간에 서로 연관성이 있고 부모, 자식 관계로 정리할 수 있는 경우가 트리 구조로 사용하기 좋습니다.

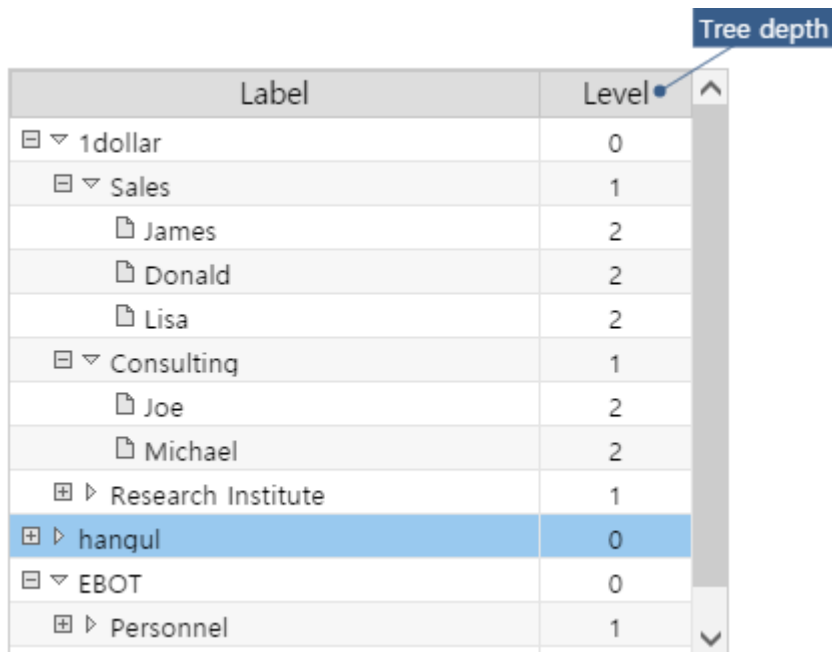
이번 장에서는 Grid를 트리 형태로 표현하는 방법에 대해 설명합니다.

25.4.1 예제

다음은 Grid를 트리로 설정한 예제입니다. 윈도우 탐색기의 탐색창처럼  (확장)버튼,  (축소)버튼을 사용해 트리를 동작시킬 수 있습니다.

Level 값은 트리의 depth를 설정하기 위한 값으로 depth가 낮을수록 상위 노드가 됩니다. 이 예제에서는 0~2까지의

레벨값을 갖는데 0은 회사, 1은 팀, 2는 팀원을 의미합니다.



Label	Level
▢ ▾ 1dollar	0
▢ ▾ Sales	1
▢ James	2
▢ Donald	2
▢ Lisa	2
▢ ▾ Consulting	1
▢ Joe	2
▢ Michael	2
▢ ▸ Research Institute	1
▢ ▸ hanqul	0
▢ ▾ EBOT	0
▢ ▸ Personnel	1

 sample_grid_32.xfdl

25.4.2 예제에서 사용한 핵심 기능

displaytype

Grid의 Cell에서 데이터가 표시되는 형식을 설정하는 속성입니다. 아무 값도 설정하지 않으면 normal이 기본값으로 설정됩니다.

edittype

Grid의 Cell에서 편집 창의 형식을 설정하는 속성입니다. 아무 값도 설정하지 않으면 none이 기본값으로 설정됩니다.

treelevel

Cell이 트리 형식일 때 트리의 레벨값으로 사용할 컬럼을 설정하는 속성입니다.

textAlign

Cell에 표시되는 데이터의 가로 정렬 방식을 설정하는 속성입니다.

treeusecheckbox

Cell이 트리 형식일 때 체크박스를 보여줄지 설정하는 속성입니다. 체크박스를 사용하려면 Cell의 treecheck 속성도 같이 설정해야 합니다.

25.4.3 예제 구현 방법

1 컴포넌트 생성하기

툴바에서 그리드 를 선택한 후, Form 위에 적당한 크기로 드래그하여 생성합니다. 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

2 Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset 을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.

Dataset Editor [Dataset00]

Const Columns

No

id

type

size

value

There are no items to show.

Columns

No

id

type

size

prop

sumtext

description

1

Label

STRING

256

2

Level

STRING

256

Rows

No

Label

Level

1

Idollar

0

2

Sales

1

3

James

2

4

Donald

2

5

Lisa

2

6

Consulting

1

7

Joe

2

8

Michael

2

9

Research Institute

1

10

Lilli

2

11

Colt

2

12

hangul

0

13

R&D

1

14

Joe

2

15

Michael

2

16

Quality Assurance

1

17

Colt

2

18

James

2

19

Lilli

2

20

Sales

1

21

Donald

2

22

EBOT

0

23

Personnel

1

24

Donald

2

25

James

2

26

Sales

1

27

Lilli

2

28

Lisa

2

Dataset | Source |

③ Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

④ Grid 속성 설정하기

체크박스를 사용하지 않도록 Grid의 treeusecheckbox 속성을 다음과 같이 설정합니다. 아무 설정도 하지 않으면 'true'로 적용되어 체크박스가 보여집니다.

속성	값
treeusecheckbox	false



treeusecheckbox 속성은 트리에 체크박스가 보여질지 여부만 설정하며 체크박스가 실제 동작하기 위해서는 셀의 treecheck 속성을 같이 설정해야 합니다.

⑤ Cell 속성 설정하기

Grid Contents Editor를 열어서 Label 컬럼의 Cell 속성을 다음과 같이 설정합니다.

속성	값
displaytype	treeitemcontrol
edittype	tree
treelevel	bind:Level
textAlign	left

⑥ QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Level 값(0~2)에 따라 트리가 알맞게 보여지는지 확인합니다. 값이 작을수록 트리의 상위에 위치합니다. 확장/축소 버튼을 눌러 트리의 동작을 확인합니다.

25.5 트리에서 편집하기

Grid를 트리 형태로 만들었을 경우에는 edittype 속성을 tree로 설정했기 때문에 편집할 수 없는 상태가 됩니다. 트리 형태의 Grid Cell을 편집하려면 Cell의 이벤트를 사용하여 해당 컬럼을 편집 가능하도록 해줘야 합니다.

이번 장에서는 Grid를 트리 형태로 만들었을 경우에 Grid Cell을 편집하는 방법에 대해 설명합니다.

25.5.1 예제

다음은 트리 형태로 설정한 Grid 예제입니다. 그러나 보통 트리와는 다르게 편집이 가능하도록 되어 있어 Cell을 더블 클릭하면 내용을 수정할 수 있습니다.

Label	Level	
1dollar	0	
Sales	1	
James	2	
Donald	2	
Lisa	2	
Consulting	1	
Joe	2	
Michael	2	
Research Institute	1	
hanqul	0	
EBOT	0	
Personnel	1	



sample_grid_33.xfdl

25.5.2 예제에서 사용한 핵심 기능

edittype

Grid의 Cell에서 편집 창의 형식을 설정하는 속성입니다. 아무 값도 설정하지 않으면 none이 기본값으로 설정됩니다.

setCellProperty

Cell의 특정 속성값을 설정하는 메소드입니다.

editautoselect

Cell의 편집 영역을 활성화했을 때 텍스트를 자동으로 전체 선택할지 설정하는 속성입니다.

autoenter

edittype 속성이 설정된 경우 해당 Cell 선택 시 자동으로 편집 모드로 변경하도록 설정하는 속성입니다.

setCellPos

특정 Cell에 포커스를 이동하는 메소드입니다. 예제에서는 Cell 더블 클릭 시 바로 편집할 수 있도록 포커스를 다시 지정해주는 역할을 합니다.

25.5.3 예제 구현 방법

① 컴포넌트 생성하기

툴바에서 그리드  를 선택한 후, Form 위에 적당한 크기로 드래그하여 생성합니다. 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

② Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.

Dataset Editor [Dataset00]

Const Columns

No	id	type	size	value
There are no items to show.				

Columns

No	id	type	size	prop	sumtext	description
1	Label	STRING	256			
2	Level	STRING	256			

Rows

No	Label	Level
1	Idollar	0
2	Sales	1
3	James	2
4	Donald	2
5	Lisa	2
6	Consulting	1
7	Joe	2
8	Michael	2
9	Research Institute	1
10	Lilli	2
11	Colt	2
12	hangul	0
13	R&D	1
14	Joe	2
15	Michael	2
16	Quality Assurance	1
17	Colt	2
18	James	2
19	Lilli	2
20	Sales	1
21	Donald	2
22	EBOT	0
23	Personnel	1
24	Donald	2
25	James	2
26	Sales	1
27	Lilli	2
28	Lisa	2

Dataset | Source

③ Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

④ Grid 속성 설정하기

체크박스를 사용하지 않도록 Grid의 treeusecheckbox 속성을 다음과 같이 설정합니다. 아무 설정도 하지 않으면 'true'로 적용되어 체크박스가 보여집니다. 그리고 해당 Cell에 포커스가 갔을 때 바로 편집 모드로 변경되도록 autoenter 속성을 지정합니다.

속성	값
treeusecheckbox	false
autoenter	select



treeusecheckbox 속성은 트리에 체크박스가 보여질지 여부만 설정하며 체크박스가 실제 동작하기 위해서는 셀의 treecheck 속성을 같이 설정해야 합니다.

⑤ Cell 속성 설정하기

Grid Contents Editor를 열어서 Label 컬럼의 Cell 속성을 다음과 같이 설정합니다.

속성	값
displaytype	treeitemcontrol
edittype	tree
treelevel	bind:Level
textAlign	left
editautoselect	true

⑥ Cell 편집 모드 전환 이벤트 함수 작성하기

Cell을 더블 클릭했을 경우 편집 모드로 전환되도록 처리하는 이벤트 함수를 작성합니다.

```
this.Grid00_oncelldblclick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    //if tree cell column
    if(e.col == 0)
    {
        var nCellIdx = this.Grid00.currentcell;
        this.Grid00.setCellProperty("body", nCellIdx, "edittype", "normal");
    }
}
```

```

        this.Grid00.setCellPos(nCellIdx);
    }

};

```

Cell의 내용을 변경한 후 선택 영역을 변경하면 다시 트리 형태로 보이도록 처리하는 이벤트 함수를 작성합니다.

```

this.Grid00_onselechanged = function(obj:nexacro.Grid,e:nexacro.GridSelectEventInfo)
{
    var nCellIdx = this.Grid00.currentcell;
    this.Grid00.setCellProperty("body", nCellIdx, "editttype", "tree");
}
};

```

⑦ QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

트리 Cell을 더블 클릭하여 편집 모드로 변경되는지 확인합니다. 편집 후 다른 셀을 클릭하거나 **Enter** 키를 입력하면 변경된 내용이 반영되고 다시 트리 형태로 변경되는지 확인합니다.

25.6 헤더 이벤트를 이용한 정렬하기

정렬은 데이터를 조건에 따라 일정한 순서가 되도록 배열하는 작업을 의미합니다. 특히 많은 양의 데이터를 그리드로 보여줄 때 사용자가 원하는 데이터를 쉽게 찾을 수 있도록 하려면 정렬은 필수입니다.

그리드에서 정렬 기능은 데이터셋의 keystack 메소드를 사용해 구현합니다. 자세한 사용법은 관련 요소를 참조하시기 바랍니다.

25.6.1 예제

다음은 keystack 메소드를 사용해 정렬을 수행하는 Grid 예제입니다.

각 헤더 Cell을 클릭하면 헤더 Cell에 오름차순(▲), 내림차순(▼)이 표시되며 해당 컬럼이 정렬됩니다.

Name	Address	Company▲	Department	Salary
John	Seoul, KOREA	1dollar	Sales	₩15,000
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
Michael	Mumbai, India	1dollar	Personnel	₩1,300
James	Barcelona, Spain	EBOT	Consulting	₩25,000
Donald	Califonia, USA	EBOT	Research Institute	₩1,000,000
Joe	São Paulo, Brazil	hangul	R&D	₩8,600
Lilli	Ankara, Turkey	hangul	Quality Assurance	₩4,400
Colt	Sydney, NSW, Australia	hangul	Sales	₩62,000

SORT



25.6.2 예제에서 사용한 핵심 기능

keystring

Dataset에 로드된 데이터를 그룹/정렬할 기준이 되는 조건식을 설정하는 속성입니다. 인수를 설정할 때 그룹 타입을 명시해야 하는데 'S'는 정렬, 'G'는 그룹을 의미합니다. 그룹 타입을 생략하면 'G'를 기본 값으로 적용합니다.

```
this.Dataset00.set_keystring( "S:column0" );
this.Dataset00.set_keystring( "S:column0+column0" );
this.Dataset00.set_keystring( "G:+column0,S:-column1" );
```

objects

폼에 등록된 모든 오브젝트(컴포넌트) 목록을 갖는 속성입니다. 배열 타입으로 다음과 같이 각 오브젝트에 접근할 수 있습니다.

```
var object1 = this.objects[2];
var object2 = this.objects["Grid00"];
```

getCellCount

Grid에서 특정 밴드에 정의된 Cell의 개수를 반환하는 메소드입니다.

```
var nCount = this.Grid00.getCellCount( "head" );
```

getCellText

지정된 위치의 Cell Text 속성값을 반환하는 메소드입니다.

```
/* Body 밴드의 첫번째 Row에서 두번째 Cell의 Text 값을 반환 */
var strText = this.Grid00.getCellText(0, 1);
```

getCellProperty

지정한 Cell의 특정 속성값을 반환하는 메소드입니다.

setCellProperty

지정한 Cell의 특정 속성값을 설정하는 메소드입니다.

split

문자열을 여러 개의 부분 문자열로 분할하여 배열로 반환하는 메소드입니다.

substr

문자열에서 지정한 위치부터 지정한 길이만큼의 문자열로 잘라 반환하는 메소드입니다.

25.6.3 예제 구현 방법

① 화면 구성하기

툴바에서 Grid  컴포넌트를 예제의 그림과 같이 적절히 배치합니다.

② Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.

Dataset Editor

Const Columns

No	id	type	size	value	
There are no items to show.					

Columns

No	id	type	size	prop	sumtext	description	
1	Name	STRING	256				
2	Address	STRING	256				
3	Company	STRING	256				
4	Department	STRING	256				
5	Salary	STRING	256				

Rows

No	Name	Address	Company	Department	Salary	
1	John	Seoul, KOREA	1dollar	Sales	15000	
2	James	Barcelona, Spain	EBOT	Consulting	25000	
3	Donald	Califonia, USA	EBOT	Research Institute	1000000	
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000	
5	Joe	São Paulo, Brazil	hangul	R&D	8600	
6	Michael	Mumbai, India	1dollar	Personnel	1300	
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400	
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000	

Dataset

|

Source

|

③ Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

④ 정렬 기능 구현

Form에 있는 Grid를 선택한 후 onheadclick 이벤트 함수를 다음과 같이 추가합니다.

```
this.Grid00_onheadclick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    var objDs = this.objects[obj.binddataset];

    for (var i = 0; i < obj.getCellCount("head"); i++)
    {
```

```

var sHeadText = obj.getCellText(-1, i);
var nLen      = sHeadText.length - 1;

/* In case of the clicked head cell */
if (i == e.cell)
{
    var sColId = (obj.getCellProperty("body", e.col, "text")).toString().split(":");
};

/* If the previous sort in ascending order, it will be sorted in descending
order */
if (sHeadText.substr(nLen) == "▲")
{
    obj.setCellProperty( "head", i, "text", sHeadText.substr(0, nLen)+ "▼");
    objDs.set_keystring("S:-" + sColId[1]);
}
/* If the previous sort in descending order, it will be sorted in ascending
order */
else if (sHeadText.substr(nLen) == "▼")
{
    obj.setCellProperty( "head", i, "text", sHeadText.substr(0, nLen)+ "▲");
    objDs.set_keystring("S:+" + sColId[1]);
}
/* If first sort, it will sort in ascending order */
else
{
    obj.setCellProperty( "head", i, "text", sHeadText+"▲");
    objDs.set_keystring("S:+" + sColId[1]);
}
}

/* In case of it's not the clicked head cell */
else
{

    /* Remove the arrow indication is displayed in the head cell which are
arranged previously */
    if (sHeadText.substr(nLen) == "▲" || sHeadText.substr(nLen) == "▼")
    {
        obj.setCellProperty( "head", i, "text", sHeadText.substr(0, nLen));
    }
}

```

```

    }
}
};

```

5 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

위치를 변경하려는 Column의 헤더를 마우스로 드래그하여 원하는 위치에 드롭합니다.

25.7 전체 선택/해제 하기

Grid 사용시 필요한 데이터만을 선택하여 처리해야 하는 경우가 있습니다. 이럴 때 Grid의 체크박스 기능을 사용하면 편리하게 데이터를 선택할 수 있습니다.

Grid에서 데이터 선택은 체크박스의 기본 기능이므로 별다른 처리가 필요 없지만 데이터의 전체 선택/해제의 경우에는 스크립트로 기능을 구현해야 합니다.

이번 장에서는 Grid 체크박스의 전체 선택/해제 하는 방법에 대해 설명합니다.

25.7.1 예제

다음은 체크박스로 설정한 Grid 예제입니다.

헤드 컬럼의 체크박스를 클릭하면 전체 선택/해제 할 수 있습니다.

☒	Name	Address	Company	Department	Salary
☒	John	Seoul, KOREA	1dollar	Sales	₩15,000
☒	James	Barcelona, Spain	EBOT	Consulting	₩25,000
☒	Donald	Califonia, USA	EBOT	Research Institute	₩1,000,000
☒	Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
☒	Joe	São Paulo, Brazil	hangul	R&D	₩8,600
☒	Michael	Mumbai, India	1dollar	Personnel	₩1,300
☒	Lilli	Ankara, Turkey	hangul	Quality Assurance	₩4,400
☒	Colt	Sydney, NSW, Australia	hangul	Sales	₩62,000



25.7.2 예제에서 사용한 핵심 기능

displaytype

Grid의 Cell에서 데이터가 표시되는 형식을 설정하는 속성입니다. 아무 값도 설정하지 않으면 'normal'이 기본값으로 설정됩니다.

edittype

Grid의 Cell에서 편집 창의 형식을 설정하는 속성입니다. 아무 값도 설정하지 않으면 'none'이 기본값으로 설정됩니다.

onheadclick

Grid에서 Head 영역의 Cell을 클릭했을 때 발생하는 이벤트입니다.

getRowCount

Dataset에서 Row의 개수를 구하는 메소드입니다.

setColumn

Dataset에서 지정한 Row의 Column에 해당하는 값을 변경하는 메소드입니다.

getCellProperty

Cell의 특정 속성 값을 반환하는 메소드입니다.

setCellProperty

Cell의 특정 속성 값을 설정하는 메소드입니다.

25.7.3 예제 구현 방법

① 화면 구성하기

툴바에서 Grid  컴포넌트를 예제의 그림과 같이 적절히 배치합니다.

② Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.

2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.

Dataset Editor [Dataset00]

Const Columns

No	id	type	size	value
There are no items to show.				

Columns

No	id	type	size	prop	sumtext	description
1	Name	STRING	256			
2	Address	STRING	256			
3	Company	STRING	256			
4	Department	STRING	256			
5	Salary	STRING	256			
6	Check	STRING	256			

Rows

No	Name	Address	Company	Department	Salary	Check
1	John	Seoul, KOREA	1dollar	Sales	15000	0
2	James	Barcelona, Spain	EBOT	Consulting	25000	0
3	Donald	Califonia, USA	EBOT	Research Institute	1000000	0
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000	0
5	Joe	São Paulo, Brazil	hangul	R&D	8600	0
6	Michael	Mumbai, India	1dollar	Personnel	1300	0
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400	0
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000	0

Dataset | Source |

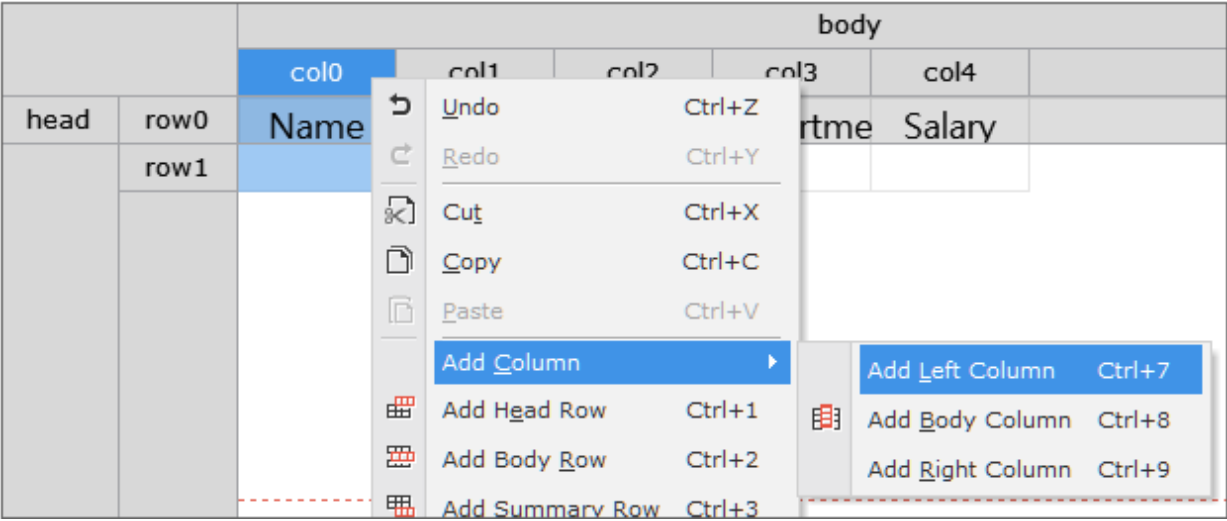
3 Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

4 Grid 설정하기

1. Grid Contents Editor를 열어 Check 컬럼을 삭제합니다.
2. Grid의 가장 왼쪽에 체크박스를 표시할 새로운 컬럼을 생성합니다. 컬럼의 생성은 다음과 같이 Add Left Column 메뉴를 사용합니다.



3. 새로 생성한 컬럼의 헤드 셀의 속성을 다음과 같이 설정합니다.

속성	값
displaytype	checkboxcontrol
edittype	checkbox

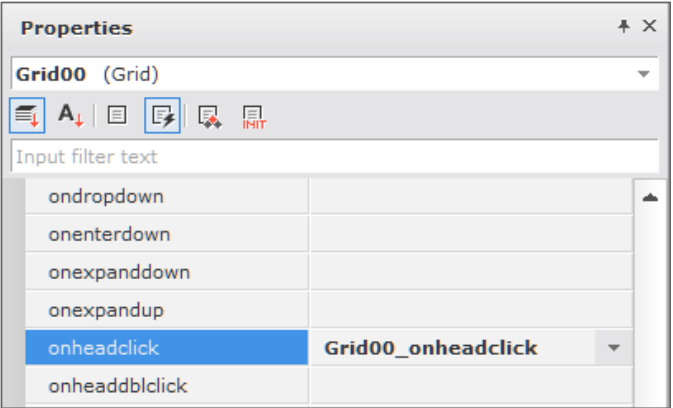
4. 새로 생성한 컬럼의 바디 셀의 속성을 다음과 같이 설정합니다.

속성	값
displaytype	checkboxcontrol
edittype	checkbox
text	bind:Check



CheckBox 컴포넌트는 생성하면 바로 동작시킬 수 있지만 Grid의 체크박스는 반드시 Dataset을 바인딩해줘야 동작시킬 수 있습니다.

1 Grid 이벤트 함수 작성하기



Grid을 선택한 후 onclick 이벤트 함수를 생성하고 다음과 같이 스크립트를 작성합니다.

```
this.Grid00_onheadclick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    //체크박스로 설정한 첫 번째 head 컬럼을 클릭했을 경우에 gf_setCheckAll 함수를 호출
    if(e.cell == 0)
    {
        this.gf_setCheckAll(obj, e);
    }
};
```

체크박스 전체 선택/해제를 처리하는 함수를 다음과 같이 작성합니다.

```
this.gv_isCheckAll = 0;
this.gf_setCheckAll = function(obj:Grid, e:GridClickEventInfo)
{
    var strColID = obj.getCellProperty("body", e.cell, "text").replace(/bind:/i, "");

    //체크박스가 체크되어 있으면 체크 해제
    //체크되어 있지 않으면 체크
    this.gv_isCheckAll = (this.gv_isCheckAll ? 0 : 1);

    //Dataset의 이벤트를 끄
    //this.Dataset00.enableevent = false;

    //check to body column
    for(var i=0; i< this.Dataset00.getRowCount(); i++)
    {
        this.Dataset00.setColumn(i, strColID, this.gv_isCheckAll);
    }

    //check to head column
    obj.setCellProperty("Head", 0, "text", this.gv_isCheckAll);

    //Dataset의 이벤트를 켜
    //this.Dataset00.enableevent = true;
}
```



Dataset은 데이터를 변경할 때 마다 이벤트가 발생하는데 잦은 이벤트 발생은 처리 속도 지연이나 화면 깜빡임 등의 증상으로 나타날 수 있습니다. 따라서 데이터가 많은 경우에는 변경 전에 이벤트를 꺾다가 변경 후에 다시 이벤트를 키도록 처리하면 그런 증상을 완화할 수 있습니다. Dataset 이벤트의 설정/해제는 `enableevent` 속성을 사용합니다.



`replace` 함수 안에 정규식 `i` 옵션을 추가해 대소문자 상관없이 치환하도록 설정했습니다.

2 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

체크박스로 설정된 Grid의 헤드 셀을 클릭하여 전체 선택/해제가 동작하는지 확인합니다.

25.8 Grid간의 복사/붙여넣기

컴퓨터를 사용할 때 복사(**Ctrl+C**) / 붙여넣기(**Ctrl+V**) 기능은 자주 사용됩니다. 마찬가지로 넥사크로플랫폼에서도 기능을 구현하면 복사/붙여넣기를 사용할 수 있습니다.

일반적으로 컴퓨터 사용시의 복사/붙여넣기는 클립보드라는 OS에서 제공하는 임시 메모리 공간을 사용합니다. 그러나 웹 환경에서는 보안상의 문제로 OS에서 제공하는 클립보드를 이용하는데 제한이 있습니다. 따라서 개발자는 어떤 환경인지를 먼저 파악한 후 복사/붙여넣기 기능을 구현해야 합니다. 일례로 넥사크로플랫폼 런타임이나 인터넷익스플로러의 경우에는 클립보드를 사용할 수 있습니다. 그러나 그 외의 브라우저에서는 클립보드로의 접근이 허용되지 않기 때문에 스크립트에서 변수나 컴포넌트같은 별도의 임시 저장 공간을 마련해야 합니다.

본 장에서는 Grid 간의 데이터를 복사/붙여넣기 하는 기능을 구현하고 그 방법에 대해 설명합니다.

25.8.1 예제

다음은 Grid에서 Grid로 Row를 복사/붙여넣기 하는 예를 보여줍니다.

Name	Address	Company	Department	Salary
John	Seoul, KOREA	1dollar	Sales	₩15,000
James	Barcelona, Spain	EBOT	Consulting	₩25,000
Donald	Califonia, USA	EBOT	Research Institute	₩1,000,000
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
Joe	São Paulo, Brazil	hangul	R&D	₩8,600
Michael	Mumbai, India	1dollar	Personnel	₩1,300
Lilli	Ankara, Turkey	hangul	Quality Assurance	₩4,400
Colt	Sydney, NSW, Australia	hangul	Sales	₩62,000

COPY
(Ctrl + c)



Name	Address	Company	Department	Salary
	DELETE (Delete key)			
John	Seoul, KOREA	1dollar	Sales	₩15,000
James	Barcelona, Spain	EBOT	Consulting	₩25,000
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
Joe	São Paulo, Brazil	hangul	R&D	₩8,600
Colt	Sydney, NSW, Australia	hangul	Sales	₩62,000

PASTE
(Ctrl + v)



복사/붙여넣기 동작은 윈도우에서의 동작과 동일합니다. 우선 위 Grid에서 복사하려는 Row를 선택한 후 Ctrl + c를 입력합니다. 그런 후 다른 Grid에서 붙여넣고 싶은 위치를 클릭한 후 Ctrl + v로 붙여넣습니다.

삭제 동작은 먼저 삭제하려는 Row를 선택한 후 Delete 키를 입력합니다.

본 예제에서는 Row 단위로 처리했지만 기능 구현에 따라 Col 단위나 영역으로 동작하도록 할 수도 있습니다.

25.8.2 예제에서 사용한 핵심 기능

selectstartrow

Grid에서 선택된 영역의 시작 셀의 Row 인덱스를 갖는 읽기전용 속성입니다.

selectendrow

Grid에서 선택된 영역의 마지막 셀의 Row 인덱스를 갖는 읽기전용 속성입니다.

selectstartcol

Grid에서 선택된 영역의 시작 셀의 Column 인덱스를 갖는 읽기전용 속성입니다.

selectendcol

Grid 에서 선택된 영역의 마지막 셀의 Column 인덱스를 갖는 읽기전용 속성입니다.

getCellValue

해당 위치 Cell의 text 속성에 설정된 값을 반환하는 메소드입니다.

getCellCount

Grid에서 특정 밴드에 정의된 Cell의 개수를 반환하는 메소드입니다.

getCellPos

Cell의 인덱스 값을 반환하는 메소드입니다. 각 밴드에 정의된 Cell은 좌상단부터 우하단까지 순차적으로 Cell 인덱스를 갖고 있기 때문에 이를 위치 값으로 활용할 수 있습니다.

getCellProperty

지정한 Cell의 특정 속성값을 반환하는 메소드입니다.

getRowCount

Dataset에서 Row의 개수를 구하는 메소드입니다.

deleteRow

Dataset에서 지정한 Row를 삭제하는 메소드입니다.

rowposition

Dataset에서 현재 선택된 Row의 인덱스 값을 갖는 속성입니다.

setColumn

Dataset에서 지정한 Row의 Column에 해당하는 값을 변경하는 메소드입니다.

keycode

키보드로 입력된 키에 해당하는 코드값을 갖는 속성입니다

ctrlkey

이벤트 발생시 **Ctrl** 키가 눌렸는지 여부를 확인하는 속성입니다.

split

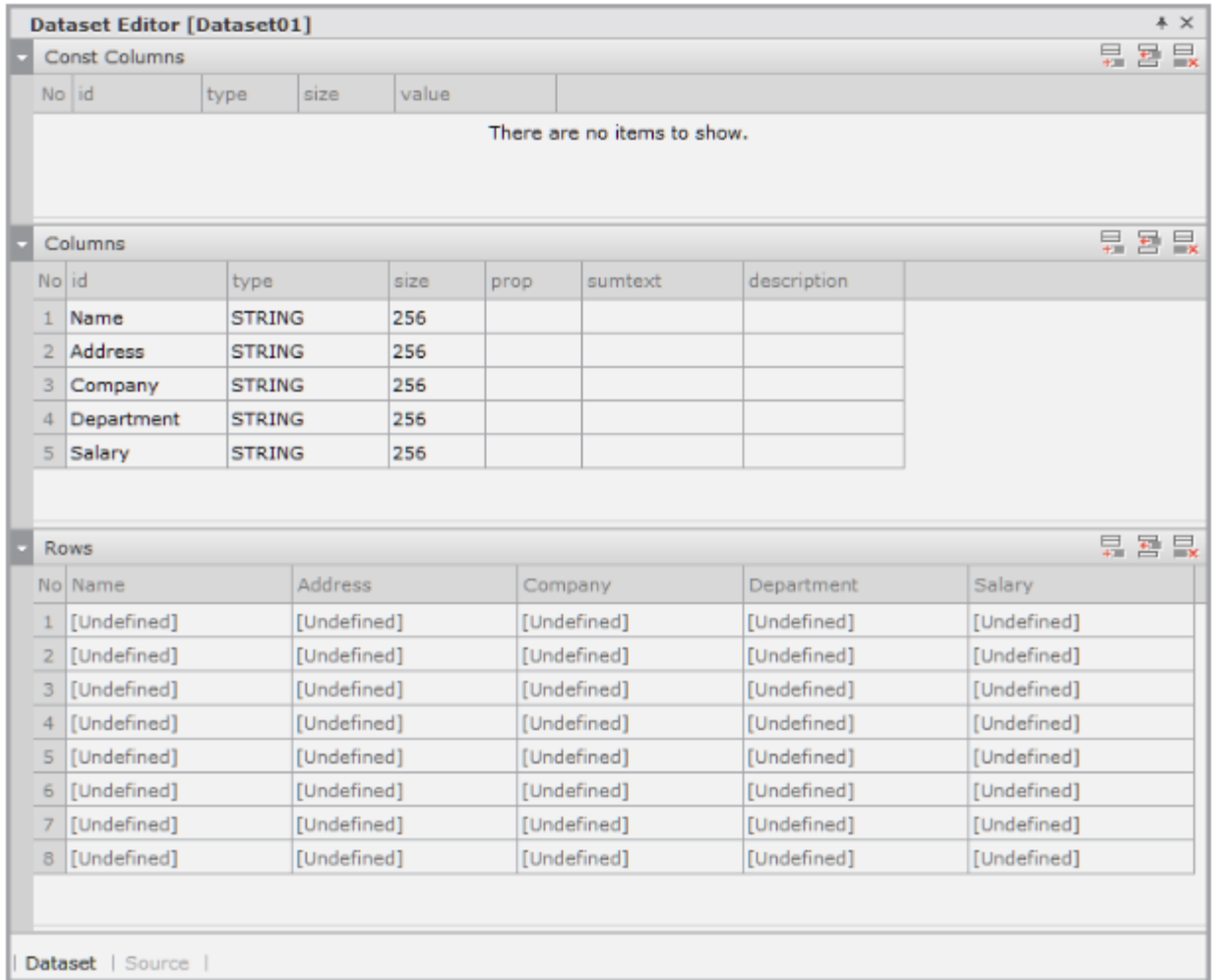
문자열을 여러 개의 부분 문자열로 분할하여 배열로 반환하는 메소드입니다.

substr

문자열에서 지정한 위치부터 지정한 길이만큼의 문자열로 잘라 반환하는 메소드입니다.

- Dataset01의 아이템 목록 작성

Column만 만들고 데이터는 입력하지 않습니다.



3 Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다. Invisible Objects 영역에 있는 Dataset을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하여 바인딩합니다.

Grid00에는 Dataset00을 Grid01에는 Dataset01을 바인딩합니다.

4 Grid 이벤트 함수 작성하기

Grid의 onkeydown 이벤트 함수를 다음과 같이 작성합니다. 두 Grid에서 공통적으로 사용할 것이므로 두 Grid 모두 Grid_onkeydown 이벤트 함수를 연결합니다.

스크립트에 대한 자세한 설명은 주석을 참조하시기 바랍니다.

```

var Buff = null;

this.Grid_onkeydown = function(obj:nexacro.Grid,e:nexacro.KeyEventInfo)
{
    //Ctrl 키가 입력됐는지 확인
    if (e.ctrlkey)
    {
        //Ctrl 키가 입력된 상태에서 c 키가 입력됐는지 확인
        if (e.keycode == 67)
        {
            //현재 Grid에서 선택되어 있는 column, row 정보를 얻어옴
            var nStartrow = obj.selectstartrow;
            var nEndrow = obj.selectendrow;
            var nStartCol = obj.selectstartcol;
            var nEndCol = obj.selectendcol;

            //임시로 데이터를 저장할 변수
            var sClip = "";
            var sSpr = " ";

            /*
            multirow로 선택하는 경우에 몇 번 반복문을 수행해야 할지 회수를 설정
            selectstartrow, selectendrow에는 선택된 row 정보가 배열 형태로 저장되어 있음
            */
            var nLoopCnt = nStartrow.length;

            /*
            row 단위 선택인 경우 column 정보는 -1로 나옴
            따라서 실제 column의 시작, 끝 index 값으로 설정
            */
            if(nStartCol == -1 && nEndCol == -1)
            {
                var objDs = this.objects[obj.binddataset];

                nStartCol = 0;
                nEndCol = objDs.colcount-1;
            }

            //선택한 Row의 데이터를 추출
            for(var k=0; k < nLoopCnt; k++)
            {

```

```

        for (var i = nStartrow[k]; i <= nEndrow[k]; i++)
        {
            for (var j = nStartCol; j <= nEndCol; j++)
            {
                if (j < nEndCol)
                {
                    sClip += obj.getCellValue(i, j) + sSpr;
                }
                else
                {
                    sClip = sClip + obj.getCellValue(i, j) + "\n";
                }
            }
        }

    }

    //추출한 데이터를 Textarea에 복사
    Buff = sClip.substr(0, sClip.length-1);
}

//Ctrl 키가 입력된 상태에서 v 키가 입력됐는지 확인
else if (e.keycode == 86)
{
    var nIndex = 0;
    var objDs = this.objects[obj.binddataset];
    var nCellCnt = obj.getCellCount("body");
    var nCellPos = obj.getCellPos();
    var nRowPos = objDs.rowposition;

    /*
    Buff 변수에 저장되어 있는 값을 변수로 받아온 후 개행문자를 기준으로
    잘라 배열 형태로 저장
    */
    var sClip = Buff;
    var arrRow = sClip.split("\n");
    var arrCol = new Array();

    for (var i = 0; i < arrRow.length; i++)
    {
        /*

```



```

Row를 탭문자를 기준으로 잘라 배열 형태로 저장
*/
arrCol = arrRow[i].split("    "); //tab

/*
현재 Row 위치를 기준으로 붙여 넣으려는 Row가 현재 Row보다 큰 경우
빈 Row를 추가
*/
if (objDs.getRowCount() <= nRowPos)
{
    objDs.addRow();
}

/*
현재 선택한 Row 위치에 Row를 붙여 넣음
*/
for (var j = 0; j < objDs.getColCount(); j++)
{
    objDs.setColumn(nRowPos, j, arrCol[nIndex++]);
}

/*
붙여 넣기할 다음 Row로 위치를 이동
*/
nRowPos++;
objDs.set_rowposition(nRowPos);
nIndex = 0;
}
}
}

//Delete 키가 입력됐는지 확인
if (e.keycode == 46)
{
    var objDs = this.objects[obj.binddataset];

    //현재 Grid에서 선택되어 있는 row 정보를 얻어옴
    var nStartrow = obj.selectstartrow;
    var nEndrow = obj.selectendrow;

    /*

```

```

multirow로 선택하는 경우에 몇 번 반복문을 수행해야 할지 회수를 설정
selectstartrow, selectendrow에는 선택된 row 정보가 배열 형태로 저장되어 있음
*/
var nLoopCnt = nStartrow.length;

//선택한 Row를 삭제
for(var k=nLoopCnt; k >= 0; k--)
{
    for (var i = nEndrow[k]; i >= nStartrow[k]; i--)
    {
        objDs.deleteRow(i);
    }
}
}
}

```

5 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Grid를 마우스와 키보드를 사용하여 선택한 후 **Ctrl** + **c**로 복사하고 다른 Grid에 **Ctrl** + **v**로 붙여넣습니다.

Row를 선택한 후 Delete 키를 눌러 삭제가 되는지 확인합니다.

25.9 페이지 기능 구현하기

Grid에서 데이터가 많아 한번에 다 표시할 수 없는 경우에 몇 개 단위로 페이지로 나눠 표시하면 데이터를 한눈에 볼 수 있어 편리합니다.

25.9.1 예제

다음은 많은 양의 데이터를 효과적으로 볼 수 있도록 페이지 기능을 추가한 Grid 예제입니다.

상단에 Inquire 버튼을 누르면 Grid에 데이터를 로딩하고 한번에 10개씩 볼 수 있도록 페이지를 나눠 표시합니다. 하단의 페이지 네비게이션 바를 살펴보면 앞 페이지로 이동하는 Prev 버튼, 뒤 페이지로 이동하는 Next 버튼 그리고 페이지로 직접 이동하는 숫자 버튼이 있습니다.

Inquire 버튼은 한번 동작시키면 데이터를 로딩한 후 사라집니다. 이는 데이터 로딩을 반복해서 수행하지 못 하도록 버튼을 안 보이게 처리했기 때문입니다.

No	Book Title	Publisher	Author
431	PITY THE BEAUTIFUL: POEMS.	GRAYWOLF	GIOIA, DANA.
432	PLACE.	ECCO	GRAHAM, JORIE, 1951-
433	PLAYDATE: A NOVEL.	ATRIA BOOKS	MILLAR, LOUISE.
434	POINT OF VIEW.	PICADOR	JAMES, CLIVE.
435	POSEIDON'S ARROW.	PUTNAM	CUSSLER, CLIVE.
436	POTBOILER.	PUTNAM	KELLERMAN, JESSE.
437	POTTER'S HAND.	ATLANTIC	WILSON, A.N.
438	PRE-RAPHAELITE DRAWING.	THAMES & HUDSON	CRUISE, COLIN.
439	PRICE OF INEQUALITY.	W.W. NORTON	STIGLITZ, JOSEPH E.
440	PRINCE.	FABER AND FABER	THORNE, MATT.

Prev 41 42 43 44 45 46 47 48 49 50 Next



25.9.2 예제에서 사용한 핵심 기능

visible

컴포넌트를 화면에 표시할지 설정하는 속성입니다. false로 설정하면 화면에 표시되지 않습니다.

filter

Dataset에 로드한 데이터를 조건에 맞는 데이터만 볼 수 있게 필터링하는 메소드입니다.

clearData

Dataset의 컬럼 정보를 제외하고 모든 데이터(Row)를 삭제하는 메소드입니다.

copyData

현재 Dataset에 인수로 지정한 Dataset의 데이터(Row)를 복사하는 메소드입니다.

getRowCount

Dataset의 Row 개수를 구하는 메소드입니다.

25.9.3 예제 구현 방법

1 컴포넌트 생성하기

그리드  컴포넌트를 하나 생성합니다. 그리고 데이터 조회에 사용할 버튼  을 생성합니다.

2 Page 네비게이션 바 만들기

다음과 같이 페이지 전환에 사용할 네비게이션 바를 만듭니다.



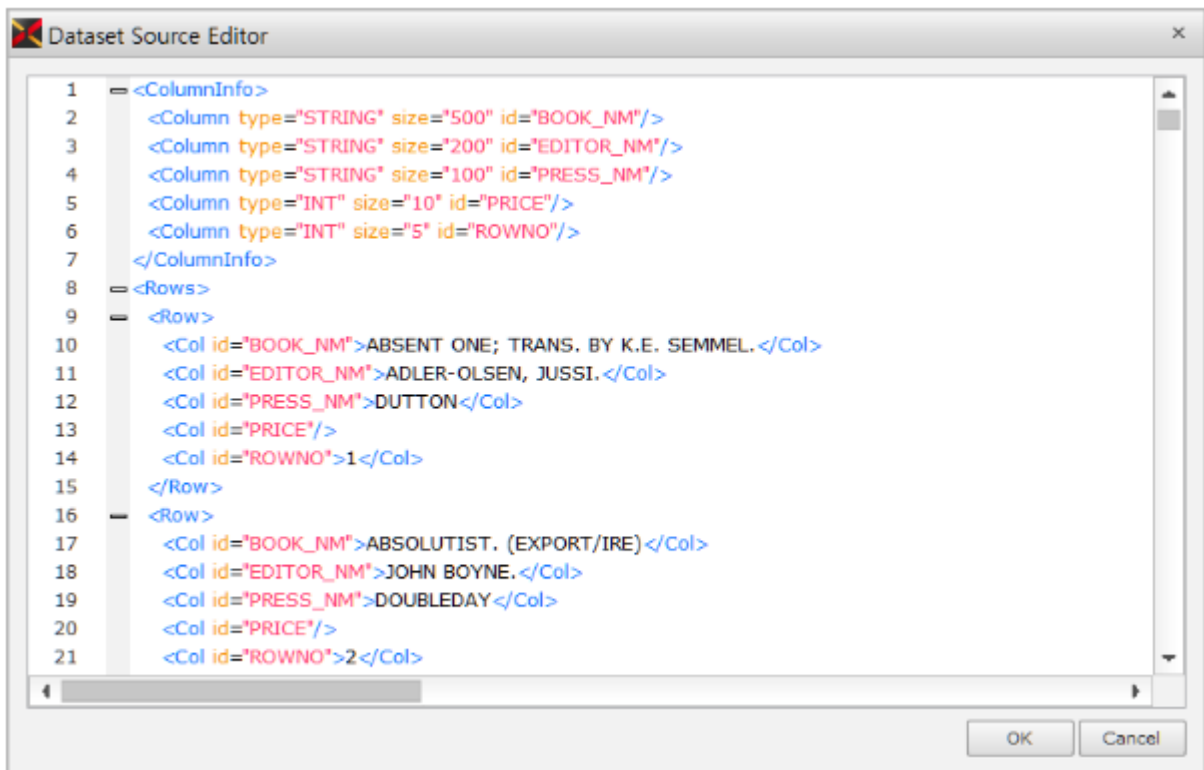
네비게이션의 바의 베이스로 사용할 Div  컴포넌트를 하나 생성하고 Div 내부에 페이지 선택 버튼으로 사용할 Static  컴포넌트를 12개 생성합니다. 생성한 Static 컴포넌트는 앞에서부터 다음 표와 같이 설정합니다.

ID	속성	값
stt_prev	text	Prev
	visible	false
stt_0	text	1
	visible	false
stt_1	text	2
	visible	false
stt_2	text	3
	visible	false
stt_3	text	4
	visible	false
stt_4	text	5
	visible	false
stt_5	text	6
	visible	false
stt_6	text	7
	visible	false
stt_7	text	8
	visible	false
stt_8	text	9
	visible	false
stt_9	text	10
	visible	false
stt_next	text	Next
	visible	false

3 Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 2개 생성한 후 ID를 각각 ds_server, ds_client 로 설정합니다. ds_server는 서버로부터 transaction을 통해 내려받은 데이터라 가정하며 ds_client는 Grid에 바인딩해 UI에서 사용합니다. 본 예제는 서버와의 트랜잭션을 가정합니다.

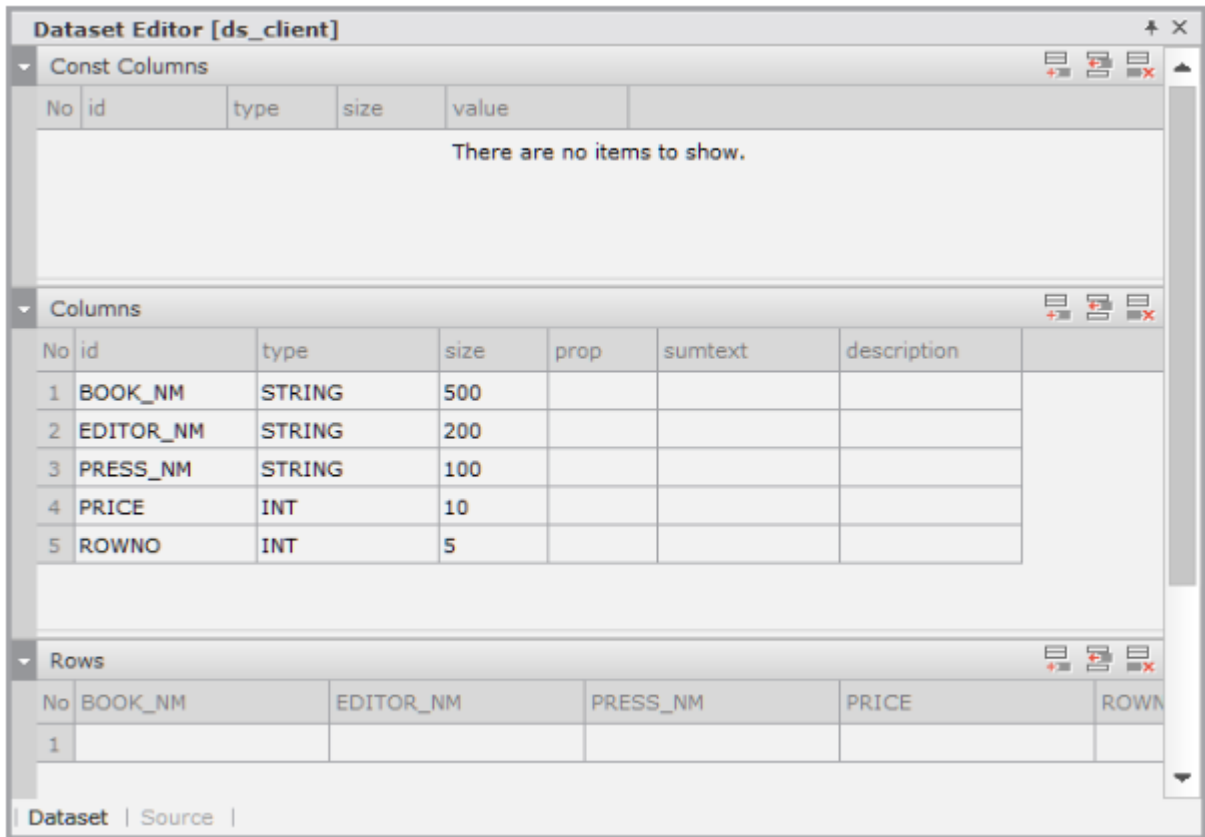
1. ds_server에 데이터를 입력하는데 입력해야 할 데이터의 양이 많으므로 XML 형태의 데이터를 직접 입력할 수 있는 Dataset Source Editor를 사용합니다. Dataset Source Editor는 Dataset을 더블 클릭하여 Dataset Editor를 띄운 후 하단의 Source 탭을 클릭하면 나옵니다.



다음 링크의 XML 데이터를 Dataset Source Editor를 사용하여 한번에 복사/붙여넣기 합니다.

 [sample_grid_38.xml](#)

2. ds_client에 데이터를 입력합니다. Dataset을 더블 클릭하여 Dataset Editor를 띄운 후 다음과 같이 데이터는 입력하지 않고 컬럼만 입력합니다.



4 Dataset 바인딩

ds_client를 Grid 컴포넌트로 드래그 앤 드롭하여 바인딩합니다. 데이터가 없으므로 컬럼명만 표시됩니다.

5 Grid 속성 설정하기

Grid Contents Editor를 열어 헤드 컬럼과 보디 컬럼을 다음과 같이 설정합니다.

컬럼	(head) row0의 text	(body) row1의 text
col0	No	bind:ROWNO
col1	Book Title	bind:BOOK_NM
col2	Publisher	bind:PRESS_NM
col3	Author	bind:EDITOR_NM

6 Inquire 버튼의 클릭 이벤트 함수 작성하기

Inquire 버튼의 onclick 이벤트 함수를 생성하고 다음과 같이 작성합니다.

```
/*
list inquire
*/
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
```

```

{
    obj.set_visible(false);
    this.div_page.set_visible(true);

    /* transaction call */

    this.fn_callback("tr1", 0, "success");
};

```

예제 코드 중간의 transaction call이라 쓰여진 주석 부분에 트랜잭션 함수를 호출하는 코드를 작성해 넣으면 이 예제를 실제 상황에서도 사용할 수도 있습니다.

7 Page 네비게이션 버튼의 클릭 이벤트 함수 작성하기

Div와 Static 컴포넌트로 만든 네비게이션 바에서 각 컴포넌트의 onclick 이벤트 함수를 작성합니다.

다음 코드를 보면 세 개의 onclick 이벤트 함수가 있습니다. div_page_stt onclick은 1~10까지의 숫자 버튼을 모두 연결하고 div_page_stt_prev onclick, div_page_stt_next onclick은 각각 Prev, Next 버튼과 연결합니다.

```

/*
click event function for page number(1~10)
*/
this.div_page_stt onclick = function(obj:Static, e:nexacro.ClickEventInfo)
{
    var iNo = new Number(obj.text);
    var iFirstNum = (iNo-1) * this.gv_iMaxPageNum + 1;
    var iLastNum = iFirstNum + this.gv_iMaxPageNum;

    this.fn_SetStyle();
    obj.set_color("red");

    this.ds_client.filter("");
    this.ds_client.filter("ROWNO >= " + iFirstNum + " && ROWNO < " + iLastNum);
}

/*
click event function for prev
*/
this.div_page_stt_prev onclick = function(obj:Static, e:nexacro.ClickEventInfo)
{

```

```

    this.gv_iStartPageNo = this.gv_iStartPageNo - 1;

    /* transaction function call */

    this.fn_callback("tr1", 0, "success");
}

/*
click event function for next
*/
this.div_page_stt_next_onclick = function(obj:Static, e:nexacro.ClickEventInfo)
{

    this.gv_iStartPageNo = this.gv_iStartPageNo + 1;

    /* transaction function call */

    this.fn_callback("tr1", 0, "success");
}

```

8 전역변수 선언하기

스크립트의 가장 상단에 다음과 같이 변수를 선언합니다. 다음 변수들은 초기 설정값으로 사용하며 이 값을 변경시키면 그에 맞게 화면이 재구성됩니다.

```

this.gv_iStartPageNo    = 0;        /* current page number */
this.gv_iUnitSelRowNo   = 100;     /* number of rows to retrieve when server is called once */
this.gv_iStartSelRowNo  = 1;        /* row number to start when calling server */
this.gv_iMaxPageNum     = 10;       /* maximum page number to display on screen */
this.gv_iServerRowCount = 0;        /* number of rows retrieved from the server */

```

9 Page 네비게이션 설정 함수 작성하기

다음은 사용자의 조작에 의해 Page 네비게이션의 버튼을 설정하는 함수입니다.

```

/*
page number setting
*/
this.fn_SetPageNo = function()
{

```



```

var sPage = this.gv_iStartPageNo * this.gv_iMaxPageNum;

/* prev button setting */
if (this.gv_iStartPageNo < 1)
{
    this.div_page.form.stt_prev.set_visible(false);
}
else
{
    this.div_page.form.stt_prev.set_visible(true);
}

/* next button setting */
if (this.gv_iServerRowCount < this.gv_iUnitSelRowNo)
{
    this.div_page.form.stt_next.set_visible(false);
}
else
{
    this.div_page.form.stt_next.set_visible(true);
}

/* page button setting */
for (var i = 0; i < this.gv_iMaxPageNum; i++)
{
    this.div_page.form.components["stt_"+i].set_visible(true);

    this.div_page.form.components["stt_"+i].set_text(sPage+i+1);

    if (this.gv_iServerRowCount < (i * this.gv_iMaxPageNum))
    {
        this.div_page.form.components["stt_"+i].set_visible(false);
    }
    else
    {
        this.div_page.form.components["stt_"+i].set_visible(true);
    }
}
}

/*

```

```

style settings
*/
this.fn_SetStyle = function()
{
    for (var i = 0; i < this.gv_iMaxPageNum; i++)
    {
        this.div_page.form.components["stt_"+i].set_color("blue");
    }
}

```

10 콜백 함수 작성하기

서버로부터 받은 데이터를 처리하기 위한 콜백 함수를 작성합니다.

트랜잭션이 완료된 후 후처리를 위해 호출되는 콜백 함수지만 예제에서는 실제 트랜잭션을 사용하지 않기 때문에 트랜잭션 함수를 호출할 위치에서 임의로 호출해 사용합니다.

26~28번 줄의 코드는 ds_server의 데이터를 ds_client로 복사하는 코드입니다. 이는 서버의 데이터를 사전에 ds_server에 저장해 놓고 트랜잭션을 수행했음을 가정했기 때문에 필요한 부분으로 실제 트랜잭션 상황에서는 필요없는 코드입니다.

```

1  /*
2  callback function
3  */
4  this.fn_callback = function(sTrId, ErrorCode, ErrorMsg)
5  {
6      if (sTrId == "tr1")
7      {
8          if (ErrorCode < 0)
9          {
10             trace("Error");
11         }
12         else
13         {
14             var iLastRow = this.gv_iStartPageNo * this.gv_iUnitSelRowNo;
15             this.ds_server.filter("ROWNO >= "+(iLastRow + this.gv_iStartSelRowNo)+" &&
ROWNO <= "+(iLastRow + this.gv_iUnitSelRowNo));
16
17             if (this.ds_server.getRowCount() == 0)
18             {
19                 alert("No more data to view.");
20                 this.ds_server.filter("");

```

```

21         return;
22     }
23     else
24     {
25         /* if transaction call using, delete the three lines below */
26         this.ds_client.clearData();
27         this.ds_client.copyData(this.ds_server, true);
28         this.ds_client.filter("ROWNO >= "+(iLastRow + this.gv_iStartSelRowNo)+
" && ROWNO <= "+(iLastRow + this.gv_iMaxPageNum));
29
30         this.gv_iServerRowCount = this.ds_server.getRowCount();
31         this.fn_SetStyle();
32         this.fn_SetPageNo();
33
34         this.div_page.form.stt_0.set_color("red");
35     }
36 }
37 }
38 }

```

11 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Page 네비게이션의 Prev, 숫자, Next 버튼을 눌러 페이지 이동에 따라 데이터가 정상적으로 표현되는지 확인합니다.

25.10 Grid에서 간단한 설문조사 구현하기

설문조사 항목 중 주관식이나 특정 답변을 선택하는 것 외에 많이 사용하는 것이 항목에 대한 점수를 매기는 방식입니다. 예를 들어 식당을 방문하고 나서 서비스나 가격, 맛이 어떠했는지 1점부터 5점까지 점수 중에서 선택하는 설문을 진행할 수 있습니다.

Grid 컴포넌트에서 이런 방식을 구현하려면 edittype 속성값을 'text'로 지정해서 직접 값을 기입하거나 "combo"로 지정해서 값을 선택하도록 할 수 있습니다. 하지만 직접 값을 입력하는 경우에는 사용자가 어떤 값을 입력해야 하는지 확인하기 어렵고 콤보박스 방식은 combolist를 열어서 항목을 선택하는 불편함이 있습니다. 예제에서는 edittype 속성값을 "checkbox"로 지정해서 해당하는 점수를 체크하는 것만으로 설문조사에 참여할 수 있도록 구현합니다.



설문의 형태에 따라 Grid 컴포넌트를 사용하지 않을 수도 있습니다. 점수를 체크하는 방식에서는 Radio 컴포넌트를 배치해 사용하는 것이 좀 더 쉬운 방법입니다. 하지만 아래 예처럼 정해진 항목이 반복되는 경우에는 Grid 컴포넌트를 사용하는 것이 시각적으로 좀 더 사용자에게 익숙할 수 있습니다.

25.10.1 예제

실제 데이터는 Restaurant, Service, Value 3개의 값만을 가지고 있습니다. 식당의 서비스와 가격에 대한 평가는 1~3점까지 점수를 체크할 수 있습니다. 위에 있는 Grid에서 해당하는 점수를 체크하면 Dataset에 반영되고 바인딩된 다른 Grid 컴포넌트에 반영됩니다 (아래쪽에 있는 Grid는 데이터가 변경되는 것을 확인할 수 있도록 배치했습니다).

Restaurant	Service			Value		
	1	2	3	1	2	3
Burgerking	☐	☒	☐	☐	☐	☒
McDonald's	☒	☐	☐	☐	☒	☐
SHAKE SHACK	☐	☒	☐	☐	☒	☐

Restaurant	Service	Value
Burgerking	2	3
McDonald's	1	2
SHAKE SHACK	2	2



sample_grid_40.xfdl

25.10.2 예제에서 사용한 핵심 기능

setColumn

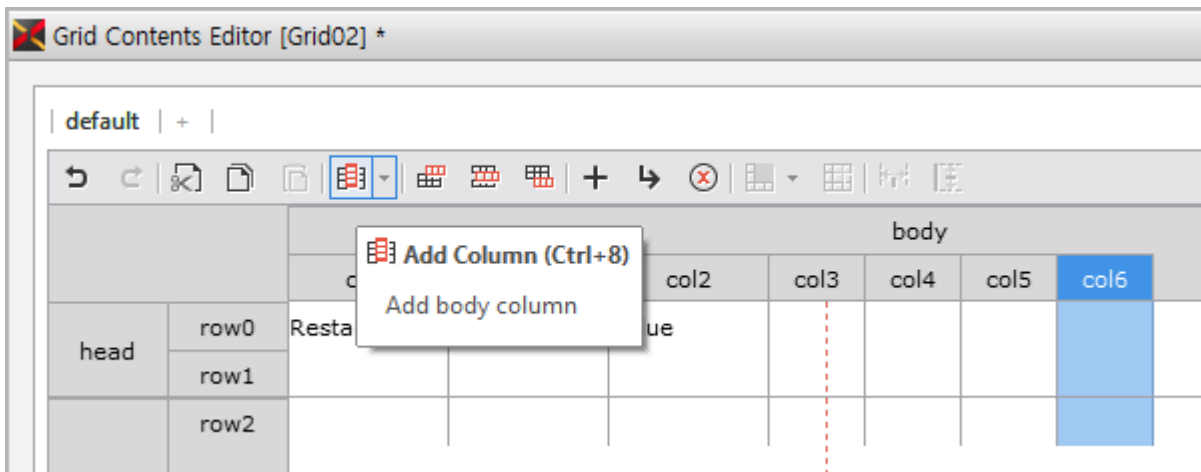
Dataset 오브젝트에 Row, Column을 지정하고 값을 변경합니다. Row는 인덱스로만 지정하고 Column 항목은 인덱스 또는 컬럼명으로 지정할 수 있습니다. 예제에서는 특정 셀 클릭 시 셀 위치에 따라 변경할 컬럼명을 지정합니다.

currow

GridCellControl 오브젝트의 속성값 지정 시 수식(expr)을 지원하는 경우 사용할 수 있는 변수 중 하나입니다. 현재 Row의 위치를 인덱스값으로 반환합니다. GridCellControl 오브젝트 도움말 항목에서 지원하는 변수 목록을 확인할 수 있습니다.

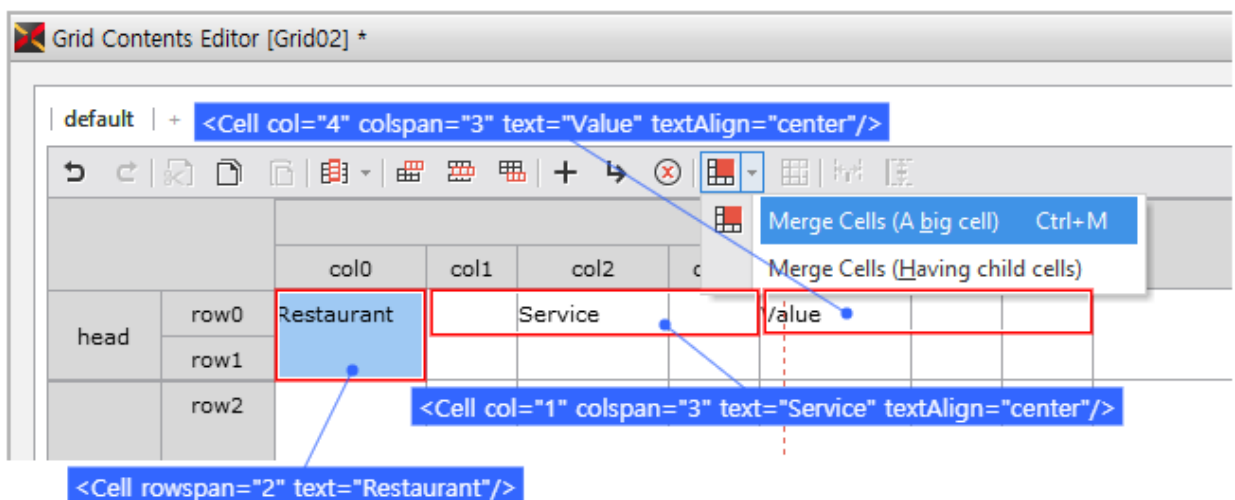
4 Grid Contents Editor - Add Column

4개의 컬럼을 추가합니다. Add Column 메뉴는 마지막 컬럼 뒤에 컬럼을 추가하는 방식인데 특정 위치에 컬럼을 추가하고 싶다면 Insert 메뉴를 사용합니다. 예제에서는 Restaurant 컬럼을 제외하고는 모두 수정할 것이라 어떤 방법을 사용하든 상관없습니다.



5 Grid Contents Editor - Merge Cells

아래 그림처럼 Head Row 항목을 병합합니다. 화면에 보여주는 용도이기 때문에 개별 Child Cell을 유지할 필요는 없습니다. 아래 그림에서 보이는 코드는 병합 후 생성되는 코드입니다.



병합 후 Head row1 항목에는 Service, Value 항목 아래에 1, 2, 3 점수값을 text 속성값으로 입력합니다.

6 Grid Contents Editor - Cell 속성값 지정

각 점수에 해당하는 Cell 속성값을 지정합니다. displaytype 속성값은 "checkboxcontrol", edittype 속성값은 "checkbox"로 지정합니다. 그리고 text 속성값은 Dataset의 컬럼값을 그대로 보여주지 않고 약간의 가공작업을 추가합니다.

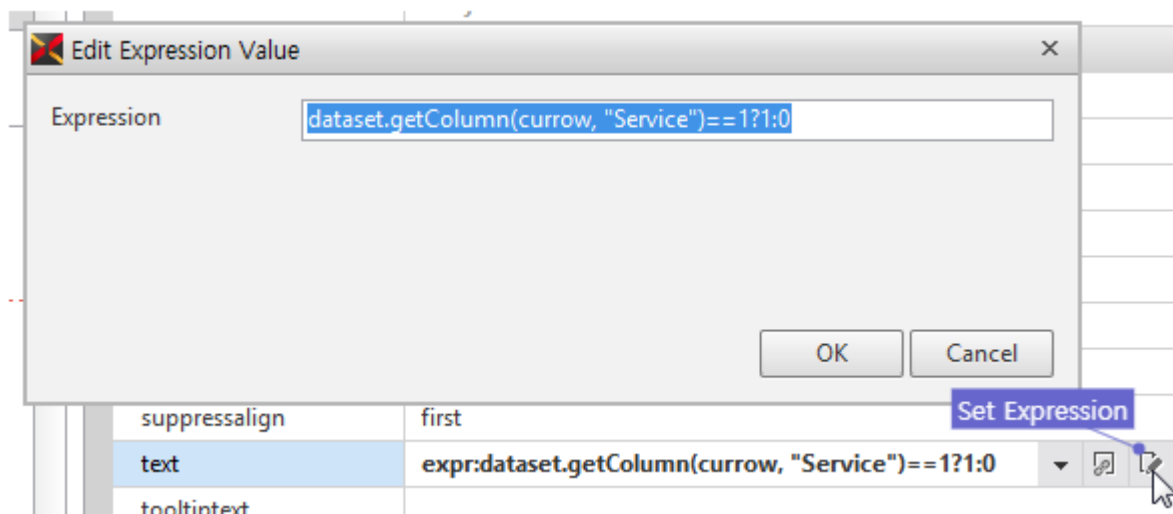
body			
col0	col1	col2	col3
Restaurant	Service		
	1	2	3
	☒	☐	☐

cssclass	
id	body1
Action	
autosizecol	default
autosizerow	default
controlautosizingtype	both
displaytype	checkboxcontrol
edittype	checkbox
expr	
suppress	0
suppressalign	first
text	expr:dataset.getColumn(currow, "Service")==1?1:0

Dataset에는 Service, Value 항목 점수값만 들어있기 때문에 어떤 체크박스를 체크된 상태로 표시해야 할지 판단해야 합니다. 같은 점수에 해당하는 Cell의 체크박스만 체크된 상태로 표시하기 위해 수식(expr)을 사용합니다.

7 Grid Contents Editor - 조건식 사용

text 속성값을 직접 입력하거나 [Edit Expression Value] 창을 실행한 후 입력합니다. text 속성값을 직접 입력하는 경우에는 "expr:"이라는 텍스트를 입력해주어야 하며 속성창에서 오른쪽 끝에 있는 버튼(Set Expression) 클릭 후 [Edit Expression Value] 창을 실행한 경우에는 수식만 입력하면 됩니다.



수식은 한 줄에 원하는 식을 담아야 하므로 if 문 대신 3항 연산자를 사용합니다. 3항 연산자는 조건식을 실행하고 해당 조건식이 참인 경우와 거짓인 경우 value값을 반환합니다. 예제에서는 getColumn 메소드를 사용해 현재 Row의 Service 컬럼값이 1인지 확인합니다. 1이 맞다면 1을 아니면 0을 반환합니다. 그래서 조건이 맞는 경우에는 체크박스가 체크되어 표시되고 맞지 않는다면 체크되지 않는 상태로 표시됩니다.

2점과 3점을 표시하는 Cell에서 마찬가지로 컬럼값이 2 또는 3인지 확인하고 1 또는 0 값을 반환합니다.

```

expr:dataset.getColumn(currow, "Service")==1?1:0
expr:dataset.getColumn(currow, "Service")==2?1:0
expr:dataset.getColumn(currow, "Service")==3?1:0

```

Value 컬럼도 마찬가지로입니다. 컬럼값만 다르고 나머지는 같은 식을 사용합니다.

```

expr:dataset.getColumn(currow, "Value")==1?1:0
expr:dataset.getColumn(currow, "Value")==2?1:0
expr:dataset.getColumn(currow, "Value")==3?1:0

```

이제 Dataset에 있는 점수값을 Grid 컴포넌트에 체크박스 형식으로 표시할 수 있습니다.

8 oncellick 이벤트 추가하기

화면에 점수를 보여주는 것까지는 처리했는데 체크박스를 클릭하면 아무런 변화가 없습니다. 체크박스와 Dataset 컬럼값이 직접 바인딩된 상태가 아니기 때문입니다. 체크박스가 가지고 있는 값은 1 또는 0인데 Dataset에 들어가야 하는 값은 1점부터 3점까지이기 때문에 맞지도 않습니다.

Grid 컴포넌트에 oncellick 이벤트를 추가하고 Cell 클릭 시 Column, Row 정보로 Dataset 컬럼값을 업데이트합니다. 예제에서는 Column 정보로 어떤 항목의 점수인지 확인할 수 있기 때문에 아래와 같이 값을 지정했습니다.

```

this.Grid00_onsellick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    if(e.col>0 && e.col<4)
    {
        this.Dataset00.setColumn(e.row, "Service", e.col);
    } else
    if(e.col>=4 && e.col<7)
    {
        this.Dataset00.setColumn(e.row, "Value", e.col-3);
    }
};

```

9 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 위쪽 Grid 컴포넌트에 점수가 표시되는지, 빈 체크박스 Cell을 클릭했을 때 데이터가 반영되는지 확인합니다.

25.11 마우스 휠 동작을 이용한 페이징 처리

데이터가 많은 경우 한번에 모든 데이터를 다 처리하지 않고 화면에 보여지는 데이터만 처리하는 방법 중 하나입니다. 최초 로딩시에는 지정된 만큼 데이터를 보여주고 마우스 휠을 움직이면 보여줄 데이터를 추가해줍니다.

25.11.1 예제

버튼을 클릭하면 10개의 데이터를 보여주고 마우스 휠을 움직이면 10개 단위로 데이터를 추가해서 표시합니다.

COL0	COL1
105	data105
106	data106
107	data107
108	data108
109	data109
110	data110

Button00

 sample_grid_47.xfdl

25.11.2 예제에서 사용한 핵심 기능

filter

Dataset 오브젝트의 데이터 중에서 조건에 만족하는 데이터만 사용하도록 합니다. 데이터 자체를 삭제하거나 조정하는 것은 아닙니다.

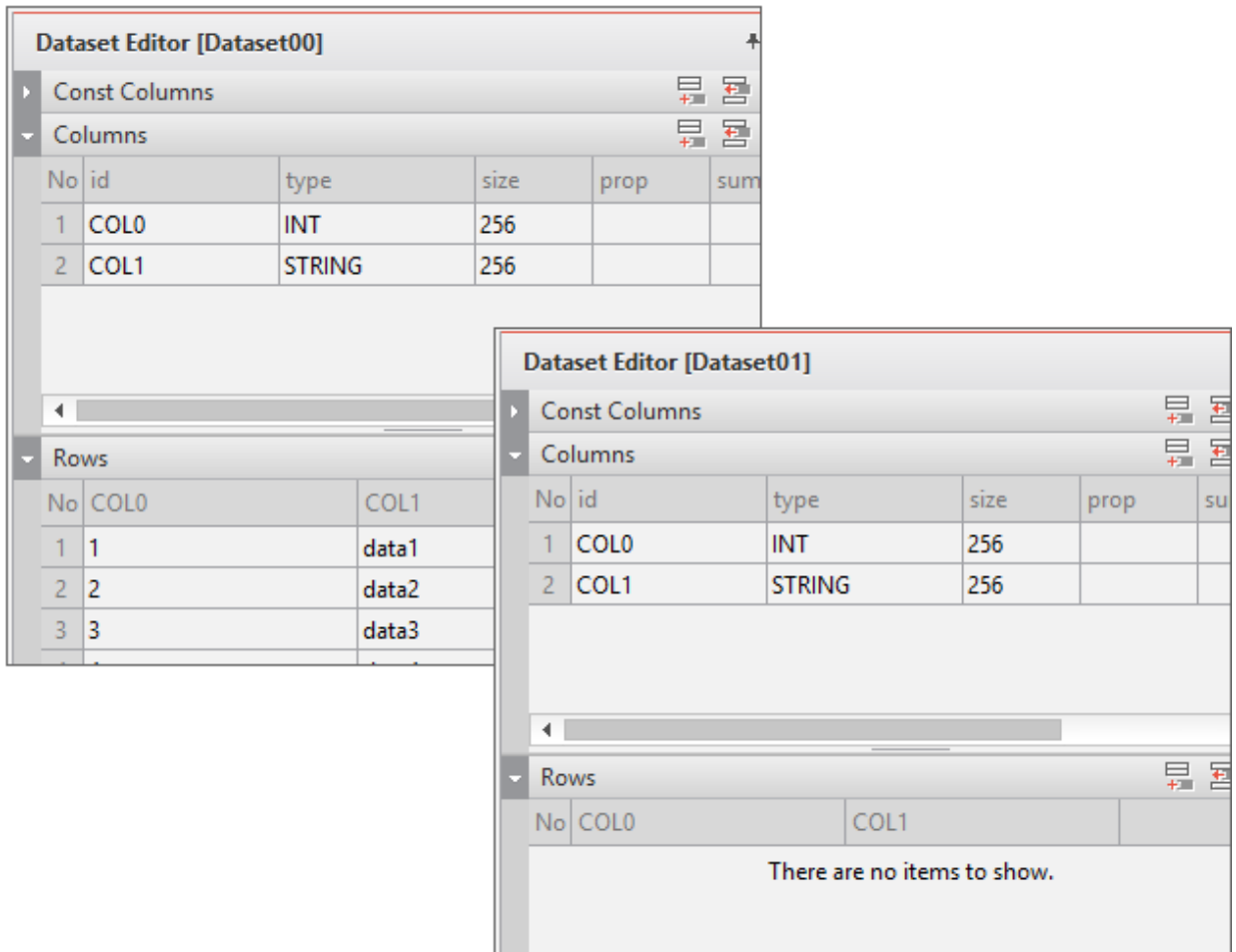
onvscroll

수직 스크롤바가 움직일때 발생하는 이벤트입니다. ScrollEventInfo 오브젝트의 type 속성값에 따라 스크롤바의 현재 상태를 알 수 있습니다. 예제에서는 type 속성값이 "wheelastover"인 경우에 데이터를 추가합니다.

25.11.3 예제 구현 방법

① 화면 구성하기

Grid 컴포넌트와 Button 컴포넌트를 배치합니다. Dataset 오브젝트는 2개를 생성하는데 하나는 데이터를 채우고 하나는 컬럼 정보만 생성합니다. 데이터가 있는 Dataset 오브젝트가 원본이고 컬럼 정보만 가지고 있는 Dataset 오브젝트는 Grid 컴포넌트에 데이터를 보여주기 위한 용도로 사용합니다.



② Dataset 바인딩

데이터가 없는 Dataset01 오브젝트를 Grid 컴포넌트에 바인딩합니다.

③ Button 컴포넌트 onclick 이벤트 함수 작성하기

fn_retrieve 함수를 호출합니다. fn_retrieve 함수에서는 전체 데이터를 확인하고 얼마만큼의 데이터를 Grid 컴포넌트에 바인딩된 Dataset에 추가할지 판단하고 처리합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
```

```

    this.fn_retrieve(false);
};

```

한 페이지를 10개 데이터 단위로 처리합니다. 버튼 클릭 시에는 10개의 데이터를 Dataset00에서 가져와서 Dataset01에 추가합니다(appendData).

```

this.fv_nTotCount = 0;
this.fv_nNowPage = 0;
this.fv_nPageSize = 10;
this.fv_bNext = true;
this.fn_retrieve = function(bNext)
{
    if (bNext == false)
    {
        this.fv_nTotCount = this.Dataset00.getRowCount();
    }

    if (this.Dataset01.rowcount >= this.fv_nTotCount)
    {
        this.fv_bNext = false;
        return;
    }

    var nStrat = this.fv_nNowPage * this.fv_nPageSize + 1;
    var nEnd = this.fv_nNowPage * this.fv_nPageSize + this.fv_nPageSize;
    this.Dataset00.filter("COL0 >= " + nStrat + " && COL0 <= " + nEnd);
    this.Dataset01.appendData(this.Dataset00);
    this.Dataset01.set_rowposition(this.Dataset01.rowcount - 1);
    this.fv_nNowPage++;
}

```

4. 마우스 휠 이벤트 처리하기

onvscroll 이벤트 발생 시 ScrollEventInfo의 type 속성값을 확인하고 "wheelastover"인 경우 데이터를 추가하기 위해 fn_retrieve 함수를 호출합니다.

```

this.Grid00_onvscroll = function(obj:nexacro.Grid,e:nexacro.ScrollEventInfo)
{
    if (e.type == "wheelastover")
    {
        if (this.fv_bNext)

```

```

    {
        this.fn_retrieve(true);
    }
}
};

```

5 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 초기값을 설정하고 마우스 휠을 움직여 데이터가 추가되는 것을 확인합니다.

25.12 1건만 있는 소계 삭제하기

Dataset 오브젝트에서 keystack 속성값을 지정하면 그룹핑 설정을 할 수 있고 소계를 출력할 수 있습니다. 하지만 기본 설정에서는 그룹핑된 항목이 1건이라도 소계를 출력합니다. 불필요하게 소계 항목을 출력하고 있는 것입니다. 이번 예제에서는 그룹핑된 항목이 1건인 경우 삭제하는 방법을 살펴봅니다.

25.12.1 예제

그룹핑된 항목이 1건인 소계도 보여주는 상태에서 버튼을 클릭하면 그룹핑된 항목이 1건인 소계 Row를 삭제합니다.

Column0	Column1	rowcnt	Button00
A01	10		
	20		
	30	2	
A02	15		
A03	12		
	25		
	37	2	
A04	10		



sample_grid_48.xfdl

25.12.2 예제에서 사용한 핵심 기능

getRowType

지정된 Row의 타입 정보를 반환합니다. 그룹핑된 정보의 경우에는 Dataset.ROWTYPE_GROUP 정보를 반환합니다. keystack 속성값으로 지정한 항목이 2개 이상인 경우에 각 컬럼 정보를 구분해야 한다면 getRowLevel 메소드를 사용합니다.

getGroupRangeCount

그룹핑된 정보의 경우에 참조하는 Row 정보를 반환합니다. 예제에서는 1건만 있는 소계를 삭제하기 때문에 반환값이 2보다 작은 경우에 조건식을 처리합니다.

25.12.3 예제 구현 방법

① 화면 구성하기

Grid 컴포넌트와 Button 컴포넌트를 배치합니다. Dataset 오브젝트를 생성하고 아래와 같이 데이터를 설정합니다.

Dataset Editor [Dataset00]					
Const Columns					
Columns					
No	id	type	size	prop	sumtext
1	Column0	STRING	256		
2	Column1	INT	256	SUM	
3	rowcnt	INT	256	COUNT	
Rows					
No	Column0	Column1	rowcnt		
1	A01	10	[Undefined]		
2	A01	20	[Undefined]		
3	A02	15	[Undefined]		
4	A03	12	[Undefined]		
5	A03	25	[Undefined]		
6	A04	10	[Undefined]		

첫번째 컬럼을 그룹으로 묶을 겁니다. Dataset 오브젝트의 keystack 속성값을 "Column0"으로 지정합니다.

② suppress 설정하기

Grid 컴포넌트에 Dataset 오브젝트를 바인딩하고 Grid Contents Editor에서 Column0 셀 항목의 suppress 속성값을 1로 지정합니다.

body				edittype	none
col0	col1	col2		expr	
Column0	Column1	rowcnt		suppress	1
				suppressalign	first
				text	bind:Column0
				tooltiptext	

③ onclick 이벤트 함수 작성하기

버튼 클릭 시 그룹핑된 항목이 1건인 데이터를 삭제합니다. 조건에 맞는 Row를 삭제하기 때문에 for 반복문을 0부터 시작하는 것이 아니라 가장 큰 인덱스 값부터 시작해서 작은 인덱스 값으로 반복해 처리합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    for(var i=this.Dataset00.getRowCount()-1; i>=0 ;i-- )
    {
        if( this.Dataset00.getRowType(i) == Dataset.ROWTYPE_GROUP
            && this.Dataset00.getGroupRangeCount(i) < 2){
            this.Dataset00.deleteRow(i);
        }
    }
};
```

④ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 그룹핑된 항목이 1건인 데이터가 삭제되는지 확인합니다.

25.13 특정 위치의 부모 트리만 펼치기

접혀진 상태의 Grid 컴포넌트에서 특정 위치의 항목을 선택하면 해당 항목의 부모 트리만 펼치는 기능입니다.

25.13.1 예제

오른쪽에 있는 Grid 컴포넌트에서 원하는 항목을 선택하면 해당 항목이 속한 부모 트리만 펼쳐집니다.

Column0	Column0	Column1
+ 1	1	0
- 8	2	1
- 9	3	1
+ 10	4	2
- 11	5	2
+ 12	6	3
	7	3
	8	0
	9	1
	10	1
	11	2
	12	2
	13	3
	14	3



sample_grid_51.xfdl

25.13.2 예제에서 사용한 핵심 기능

treeinitstatus

Grid 컴포넌트가 트리 형태로 표시될때 초기 형태를 설정하는 속성입니다. 중간에 해당 속성을 변경하면 지정된 속성값에 따라 초기화된 상태로 표시됩니다. 예제에서는 항목 선택 시 먼저 모든 항목을 접은 상태에서 시작합니다.

setTreeStatus

지정된 Row의 상태를 접거나 펴진 상태로 지정합니다. 예제에서는 반복문을 사용해 선택된 항목 상위의 트리는 모두 펼치도록 했습니다.

25.13.3 예제 구현 방법

① 화면 구성하기

Grid 컴포넌트를 2개 배치합니다. Dataset 오브젝트를 생성하고 아래와 같이 데이터를 설정합니다. 그리고 두 개의 Grid 컴포넌트에 같은 Dataset 오브젝트를 바인딩합니다.

Dataset Editor [Dataset00]			
▶ Const Columns			
▼ Columns			
No	id	type	size
1	Column0	STRING	256
2	Column1	STRING	256
◀			
▼ Rows			
No	Column0	Column1	
1	1	0	
2	2	1	
3	3	1	
4	4	2	
5	5	2	
6	6	3	
7	7	3	
8	8	0	
9	9	1	
10	10	1	
11	11	2	
12	12	2	
13	13	3	
14	14	3	

② 첫번째 Grid 컴포넌트 편집하기

첫번째 Grid 컴포넌트를 트리 형태로 보여주기 위해 Grid Contents Editor를 실행합니다. Grid 컴포넌트를 더블클릭하면 Editor가 실행됩니다. Dataset 오브젝트를 바인딩하면서 생성된 Column1은 삭제해버립니다. Column0의 body 셀 영역을 선택하고 displaytype, edittype 속성값을 아래와 같이 변경합니다.

body	
col0	
Column0	

cssclass	
id	body0
Action	
autosizecol	default
autosizerow	default
controlautosizingtype	both
displaytype	treeitemcontrol
edittype	tree
expr	

그리고 treelevel 속성값을 Column1로 지정해줍니다. 속성창 위에 있는 검색창을 사용하면 속성값을 빠르게 찾을 수 있습니다.

A	tree
CellTreeItem	
treecheck	
treelevel	bind:Column1
treestartlevel	Column0
treestate	Column1

③ oncellick 이벤트 함수 작성하기

전체 데이터 목록에서 특정한 항목을 선택하면 이벤트를 처리하기 위해 oncellick 이벤트 함수를 작성합니다. 해당 항목의 부모 트리 항목은 findParentArr 함수에서 따로 처리해서 배열 형태로 받아옵니다. 해당 배열 내에 있는 항목은 setTreeStatus 메소드를 사용해 하나씩 펼쳐줍니다.

```

this.Grid01_onsellick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    this.Grid00.set_treeinitstatus("collapse,all");

    var path_Arr = this.findParentArr(this.Grid00, e.row);
    for(var i = path_Arr.length ; i > 0 ; i--) // Should be expanded in reverse.
    {
        this.Grid00.setTreeStatus(this.Grid00.getTreeRow(path_Arr[i]), true);
    }
    this.Dataset00.set_rowposition(path_Arr[0]);
};

```

4 findParentArr 함수 작성하기

선택한 Row의 인덱스값을 기준으로 부모 Row를 찾아갑니다. 예를 들어 12번 항목(인덱스는 11)을 선택했다면 부모 항목은 10번(인덱스는 9)이고 10번 항목의 부모 항목은 8번(인덱스는 7)입니다. 배열에는 11, 9, 7 값이 담기게 됩니다.

```
this.findParentArr = function (obj, idx)
{
    var arr = [];
    arr[arr.length] = idx;
    for(;;)
    {
        var _p = obj.getTreeParentRow(arr[arr.length-1]);
        if(_p < 0){return arr;}
        arr[arr.length] = _p;
    }
};
```



for(;;) 문은 계속 반복을 하도록 설정한 반복문입니다. 이런 경우에는 반복문을 멈추는 조건이 없기 때문에 중간에 반복문을 중단하는 문장(return arr)이 없다면 계속 반복문이 동작하게 됩니다. while(true)를 사용하는 것과 같은 방식입니다. getTreeParentRow 메소드는 Root row를 찾았을 때 -1을 반환하며 조건에 따라 결과값을 반환하게 됩니다.

5 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 오른쪽 Grid 항목을 선택해 왼쪽 Grid 컴포넌트의 트리 상태가 변하는 것을 확인합니다.

25.14 영수증 계산하기

합계나 소계를 구하는 것은 컬럼 속성을 "SUM"으로 지정하거나 getSum 메소드를 사용하는데 단가와 수량이 있는 경우에는 바로 합계를 구할 수 없습니다. 이런 경우에는 어떻게 연산을 할 수 있는지 살펴봅니다.

25.14.1 예제

단가와 수량을 계산한 합계값을 표시합니다.

item	price	amount	Total
apple	500	10	\$5,000.00
soju	1,050	20	\$21,000.00
water	500	5	\$2,500.00
paper	2,500	2	\$5,000.00
			\$33,500.00



25.14.2 예제에서 사용한 핵심 기능

displaytype

속성값을 "currency"로 지정하면 현재 locale 설정의 통화를 기준으로 표시합니다.

25.14.3 예제 구현 방법

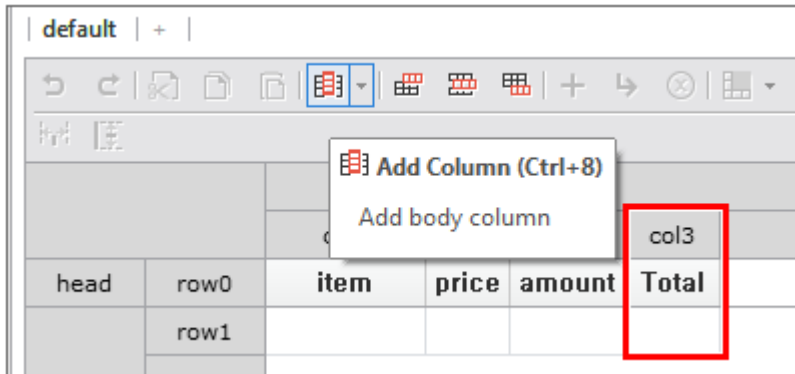
① 화면 구성하기

Grid 컴포넌트를 배치합니다. Dataset 오브젝트를 생성하고 아래와 같이 데이터를 설정합니다. 컬럼 type 설정 시 price, amount 항목은 "INT"로 지정합니다. 해당 type을 지정하지 않으면 getSum 메소드가 정상적으로 실행되지 않습니다. 생성된 Dataset 오브젝트를 Grid 컴포넌트에 바인딩합니다.

Columns					
No	id	type	size	prop	sumtext
1	item	STRING	256		
2	price	INT	256		
3	amount	INT	256		
Rows					
No	item	price	amount		
1	apple	500	10		
2	soju	1050	20		
3	water	500	5		
4	paper	2500	2		

② Total 컬럼 추가하기

Grid Contents Editor를 실행하고 컬럼을 추가합니다. 단가와 수량을 계산한 값을 표시하기 위한 컬럼입니다. [Add Column] 버튼을 클릭하면 컬럼이 추가됩니다. head 영역을 선택하고 text 속성값에 "Total"을 입력합니다.



그리고 Total 셀 항목을 선택하고 expr에 아래와 같이 입력합니다. price 컬럼값과 amount 컬럼값을 곱해서 아이템별 합계를 계산하고 화면에 표시합니다.

```
expr:price * amount
```

displaytype 속성값은 컬럼별로 아래와 같이 설정합니다.

```
<Band id="body">
  <Cell text="bind:item"/>
  <Cell col="1" text="bind:price" displaytype="number"/>
  <Cell col="2" text="bind:amount" displaytype="number"/>
  <Cell col="3" expr="expr:price * amount" displaytype="currency"/>
</Band>
```



expr 속성값 지정 시 직접 계산식을 지정하지 않고 별도 함수를 작성해서 처리할 수 있습니다. 계산식이 복잡할 경우에는 별도 함수를 작성하는 것이 좀 더 직관적입니다. 예를 들어 위와 같은 경우에는 아래와 같이 설정합니다.

```
...
<Cell col="3" expr="expr:comp.parent.fn_total(rowidx)" displaytype="currency"/>
...
```

그리고 fn_sum 이라는 함수를 다음과 같이 생성합니다.

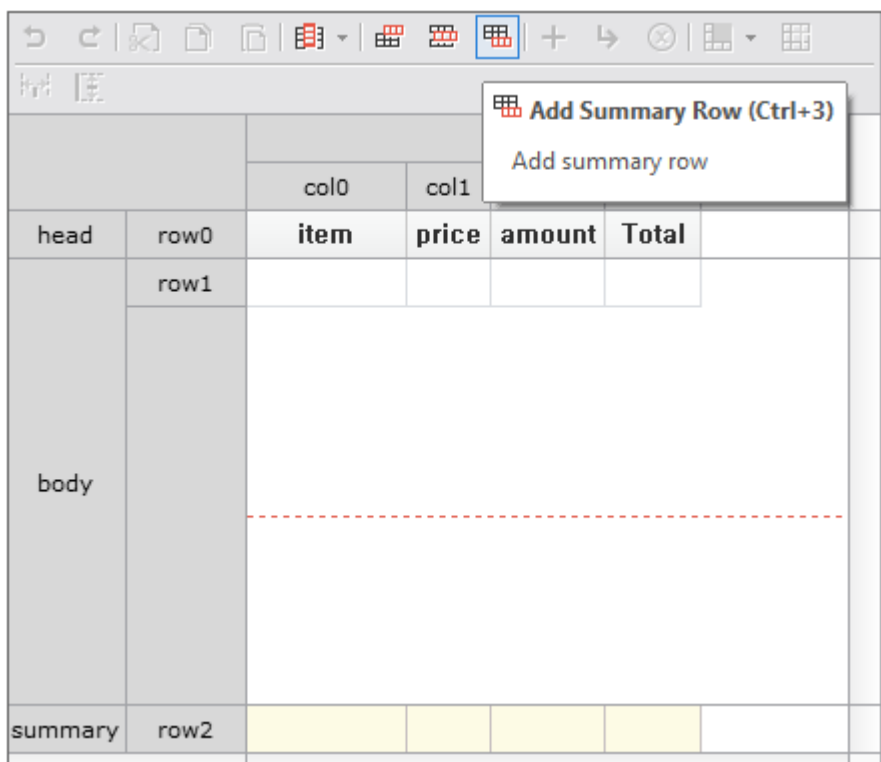
```

this.fn_total = function(row)
{
    var total= this.Dataset00.getColumn(row,"price") * this.Dataset00.getColumn(row,"
amount");
    return total;
};

```

③ Summary Row 추가하기

[Add Summary Row] 버튼을 클릭해 총합계를 표시할 Row를 추가합니다.



Total 컬럼에 해당하는 summary row 셀의 displaytype 속성값은 "currency"로 지정하고 expr 항목은 아래와 같이 지정합니다. getSum 메소드는 컬럼 ID를 지정하거나 컬럼 ID의 계산식을 파라미터로 지정할 수 있습니다.

```

expr:dataset.getSum('price*amount')

```

④ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 아이템별 합계와 전체 합계가 표시되는 것을 확인합니다.

25.15 그룹에 레벨 지정해서 subtotal 계산하기

그룹이 두 개 이상인 경우 subtotal 값을 계산해야 하는 경우에는 어떤 그룹을 먼저 묶어야 할지 지정해주어야 합니다. 이번 예제에서는 그룹이 두 개 이상인 경우 처리하는 방식을 살펴봅니다.

25.15.1 예제

왼쪽에 표시되는 Grid 컴포넌트는 C1, C2 2개 컬럼을 레벨을 지정했습니다. 오른쪽 Grid는 레벨을 지정하지 않고 그룹핑 조건으로만 C1, C2 컬럼을 묶어주었습니다.

C1	C2	C3	C4	C1	C2	C3	C4
A	apple	asia	10	A	apple	asia	10
		africa	20			africa	20
		sum apple	30				30
	sum A		30	B	car	asia	10
B	car	asia	10			africa	10
		africa	10				20
		sum car	20		water	asia	20
	water	asia	20			africa	10
		africa	10				30
		sum water	30				
	sum B		50				



sample_grid_56.xfdl

25.15.2 예제에서 사용한 핵심 기능

keystring

", "를 이용하면 그룹에 레벨을 줄 수 있습니다. 예제에서는 "G:+C1,+C2"으로 지정된 Dataset 오브젝트를 왼쪽 Grid 컴포넌트에 바인딩했는데 C1을 기준으로 소계를 계산하고, C1,C2를 기준으로 소계를 계산한다는 의미입니다. 오른쪽 Grid 컴포넌트에 바인딩된 Dataset 오브젝트의 keystring 속성값은 "G:+C1+C2"입니다. 때문에 C1, C2를 기준으로 소계를 계산해 표시합니다.

25.15.3 예제 구현 방법

1 화면 구성하기

Grid 컴포넌트를 2개 배치합니다. Dataset 오브젝트는 2개를 생성하는데 데이터는 똑같이 입력합니다.

Dataset Editor [Dataset01]						
Const Columns						
Columns						
No	id	type	size	prop	sumtext	description
1	C1	STRING	256			
2	C2	STRING	256			
3	C3	STRING	256			
4	C4	INT	256	SUM		
Rows						
No	C1	C2	C3	C4		
1	A	apple	asia	10		
2	A	apple	africa	20		
3	B	car	asia	10		
4	B	car	africa	10		
5	B	water	asia	20		
6	B	water	africa	10		

Dataset 오브젝트의 keystring 속성값을 아래와 같이 지정합니다. ","가 중간에 들어가는지 아닌지의 차이입니다. 왼쪽 Grid 컴포넌트에는 Dataset00 오브젝트를 바인딩하고, 오른쪽 Grid 컴포넌트에는 Dataset01 오브젝트를 바인딩합니다.

Dataset00: G:+C1,+C2

Dataset01: G:+C1+C2

2 suppress 속성값 설정하기

Grid 컴포넌트를 더블클릭해서 Grid Contents Editor를 실행하고 suppress 속성값을 지정합니다. C1 컬럼은 1, C2 컬럼은 2로 지정합니다.

3 subtotal 텍스트 설정하기

왼쪽 Grid 컴포넌트에는 subtotal 앞에 문구를 표시하도록 합니다. expr 항목을 아래와 같이 지정합니다.

```
expr:dataset.getRowLevel(rowidx)==2?'sum ' + dataset.getColumn(rowidx-2,'C1'):C2
expr:dataset.getRowLevel(rowidx)==1?'sum ' + dataset.getColumn(rowidx-1,'C2'):C3
```

4 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 Grid 컴포넌트에 표시되는 항목을 확인합니다.

25.16 동적으로 Grid 컴포넌트 생성하고 데이터 바인딩하기

Grid 컴포넌트는 정해놓은 데이터 형식에 따라 데이터를 보여주는 용도로 사용됩니다. 하지만 상황에 따라 Grid 컴포넌트를 스크립트에서 동적으로 생성하고 데이터를 설정해주어야 하는 경우도 있습니다. 이번 예제에서는 동적으로 Grid 컴포넌트를 생성하고 포맷을 지정하는 방법을 살펴봅니다.

25.16.1 예제

버튼을 클릭하면 하단에 동적으로 생성된 Grid 컴포넌트를 표시합니다.

Button00

no	name	age
0	employ_0	10
1	employ_1	11
2	employ_2	12
3	employ_3	13
4	employ_4	14
total		60



sample_grid_63.xfdl

25.16.2 예제에서 사용한 핵심 기능

appendContentsRow

"head", "body", "summary" Row를 추가하는 메소드입니다. Grid Contents Editor에서 [Add Head Row], [Add Body Row], [Add Summary Row] 항목과 같은 기능을 처리합니다.

appendContentsCol

컬럼을 추가하는 메소드입니다. "left", "body", "right" 밴드 영역을 선택할 수 있습니다. 지정된 밴드 영역이 없으면 "body" 밴드 영역에 추가됩니다. 예제에서는 3개의 컬럼이 필요한데, appendContentsCol 메소드를 두 번 실행했습니다. 첫 번째 컬럼은 appendContentsRow 메소드가 실행되면서 이미 추가된 상태이기 때문에 두 번만 appendContentsCol 메소드를 실행합니다.

25.16.3 예제 구현 방법

① 화면 구성하기

화면에는 버튼 하나만 배치합니다. 버튼 클릭 시 Grid 컴포넌트를 생성하고 배치합니다.

② onclick 이벤트 함수 작성하기

버튼 클릭 시 Grid를 배치하고 생성합니다. 전체 코드를 부분별로 나누어 설명합니다. 가장 먼저 Grid 컴포넌트를 생성하고 form 오브젝트의 자식으로 추가합니다.

```
this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var objGrid = new Grid();
    objGrid.init("GridNm", 30, 80, 350, 200,null,null);
    this.addChild("Grid00", objGrid);
    objGrid.show();
    ...
}
```

Dataset 오브젝트를 생성하고 Columns, Rows 정보를 설정합니다.

```
var objDs = new Dataset();
objDs.set_name("ds_test");
objDs.addColumn("no", "string");
objDs.addColumn("name", "string");
objDs.addColumn("age", "int");
```

```
for (var i = 0; i < 5; i++)
{
    var nRow = objDs.addRow();

    objDs.setColumn(nRow, "no", i);
    objDs.setColumn(nRow, "name", 'employ_' + i);
    objDs.setColumn(nRow, "age", (10 + i));
}
```

바인딩 정보를 설정합니다. 바인딩 정보 설정 후 Grid 컴포넌트의 createFormat 메소드를 호출하면 Dataset 오브젝트의 정보를 바탕으로 포맷 정보를 자동으로 설정합니다. 하지만 예제에서는 몇 가지 조정하는 부분이 있어서 createFormat 메소드를 사용하지 않습니다.

```
objGrid.set_binddataset(objDs);
```

Row 밴드 정보를 설정하고 컬럼을 추가합니다. appendContentsRow 메소드 실행 시 첫 번째 컬럼은 자동으로 생성되기 때문에 appendContentsCol 메소드는 두 번만 실행합니다.

```
objGrid.appendContentsRow("head");
objGrid.appendContentsRow("body");
objGrid.appendContentsRow("summ");

objGrid.appendContentsCol();
objGrid.appendContentsCol();

objGrid.setFormatColProperty(0, "size", 100);
objGrid.setFormatColProperty(1, "size", 100);
objGrid.setFormatColProperty(2, "size", 100);
```

화면에 표시되는 head 영역의 문자열과 body 영역에 표시될 Cell 속성을 지정합니다.

```
for (var r = 0; r < objDs.getColCount(); r++)
{
    var colinfo = objDs.getColumnInfo(r);
    objGrid.setCellProperty("head", r, "text", colinfo.name);
    objGrid.setCellProperty("body", r, "text", "bind:" + colinfo.name);
    if (r == 2)
    {
        objGrid.setCellProperty("summ", r, "text", "expr:dataset.getSum('"+colinfo.name+
            "')");
    }
}
```

```

    }
    else if(r == 1)
    {
        objGrid.setCellProperty("sum", r, "text", "total");
    }
}

```

③ QuickView로 확인하기

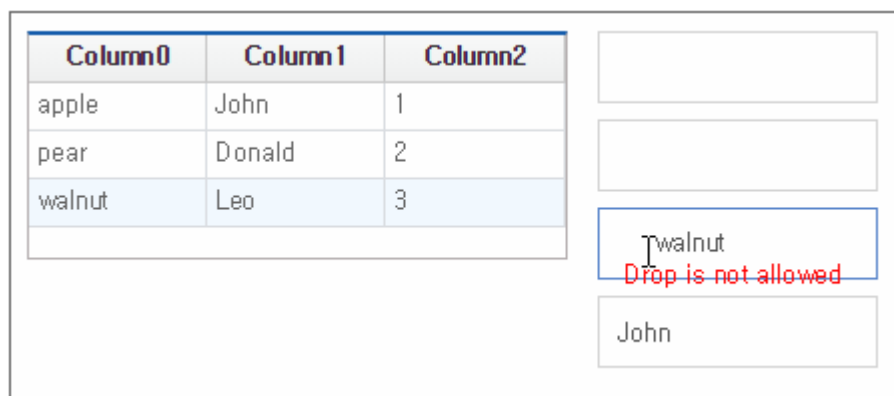
QuickView(Ctrl + F6)로 실행한 후 버튼 클릭 시 생성되는 Grid 컴포넌트를 확인합니다.

25.17 드래그앤드롭으로 Grid 컴포넌트의 문자열 옮기기

드래그 앤 드롭 기능을 활용해 Grid 컴포넌트에 보여지는 데이터를 다른 컴포넌트로 옮기는 예제입니다.

25.17.1 예제

Grid 컴포넌트의 특정 Cell을 드래그해서 Edit 컴포넌트로 옮길 수 있습니다. Edit 컴포넌트 중 2, 4번째 컴포넌트는 드래그한 문자열을 옮길 수 없도록 제한해놓았습니다.



25.17.2 예제에서 사용한 핵심 기능

getCellValue

Grid 컴포넌트에 보여지는 문자열을 반환합니다. getCellProperty 메소드를 사용할 경우에는 문자열로 text 속성값을 지정한 경우에는 원하는 결과를 얻을 수 있지만, 바인딩된 컬럼값을 사용하거나 expr 속성을 지정한 경우에는 원하는 문자열을 얻을 수 없습니다.

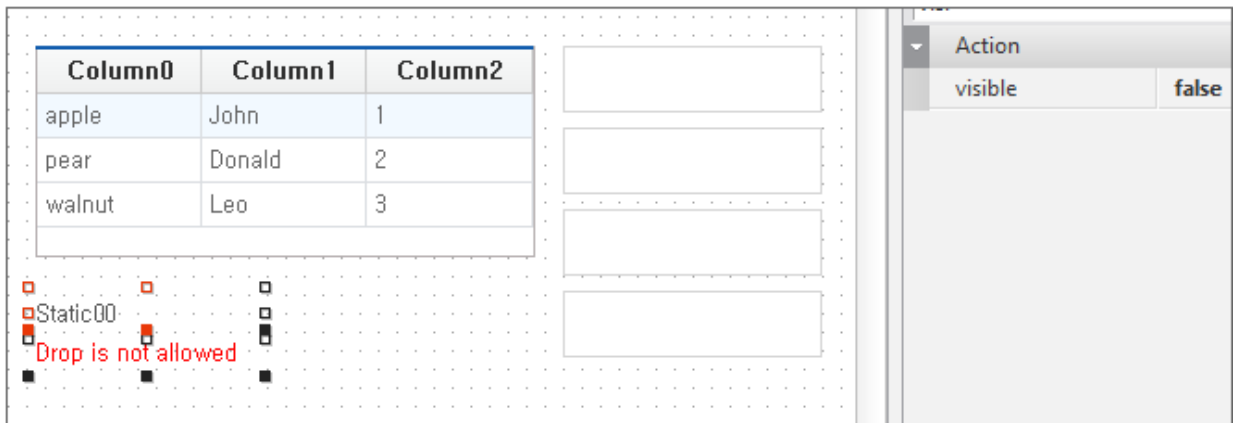
getCellRect

선택한 Cell 영역의 좌표값을 반환합니다. 예제에서는 드래그 시작 시점에 Static 컴포넌트가 해당 위치에 보여지도록 설정합니다.

25.17.3 예제 구현 방법

① 화면 구성하기

Grid 컴포넌트와 Edit 컴포넌트를 배치합니다. Static 컴포넌트 2개를 배치하고 visible 속성값을 false로 지정합니다. Static 컴포넌트는 드래그앤드롭 상태에 따라 visible 속성을 변경합니다. Dataset 오브젝트를 생성하고 Grid 컴포넌트에 바인딩합니다.



② onload 이벤트 함수 작성하기

예제에서는 Edit 컴포넌트 중 2개는 드래그한 문자열을 옮길 수 있고 나머지 2개는 옮길 수 없도록 지정합니다. 문자열을 옮길 수 있는 컴포넌트와 그렇지 않은 컴포넌트를 onload 이벤트 함수에서 배열로 지정합니다.

```
var arr1 = [];
var arr2 = [];

this.sample_grid_65_onload = function(obj:nexacro.Form,e:nexacro.LoadEventInfo)
{
```

```

// Drop unavailable
arr1[0] = this.Edit00;
arr1[1] = this.Edit02;

// Drop possible
arr2[0] = this.Edit01;
arr2[1] = this.Edit03;
};

```

③ ondrag 이벤트 함수 작성하기

ondrag 이벤트 함수는 Grid 컴포넌트에서 작성합니다. Grid 영역에서 드래그 동작이 시작되는 시점에 이벤트가 발생합니다. GridDragEventInfo 오브젝트의 row 속성값이 -1 또는 -2인 경우에는 head, summary Row 밴드에서 드래그 동작이 발생한 것이므로 이벤트를 무효화하고 그렇지 않은 경우에는 선택한 Cell 영역의 위치 값을 확인하고 Static 컴포넌트를 그 위에 표시합니다.

```

this.Grid00_ondrag = function(obj:nexacro.Grid, e:nexacro.GridDragEventInfo)
{
    if(e.row == -1 || e.row == -2)
    {
        return false;
    }

    sTextDragData = obj.getCellValue(e.row, e.cell);

    var cell_rect = obj.getCellRect(e.row, e.cell);
    this.Static00.set_left(cell_rect.left);
    this.Static00.set_top(cell_rect.top);
    this.Static00.set_width(cell_rect.width);
    this.Static00.set_height(cell_rect.height);
    this.Static00.set_text(sTextDragData);
    this.Static00.set_visible(true);
    return true;
};

```

④ ondragmove 이벤트 함수 작성하기

ondragmove 이벤트 함수에서는 마우스 움직임에 따라 Static 컴포넌트를 이동합니다. 문자열을 옮길 Edit 컴포넌트 위로 올라가면 옮길 수 없는 컴포넌트인지 확인하고 Static 컴포넌트의 visible 속성값을 조정합니다.

```

this.DragTextMove = function(obj, e)
{
    if( this.Static00.visible == true )
    {
        for(var i = 0; i < arr1.length; i++)
        {
            if(arr1[i].name == e.fromobject.name)
            {
                this.equal_Obj(obj, e);
                break;
            } else {
                this.different_Obj(obj, e);
            }
        }
        this.Static00.move(e.clientx+5, e.clienty-15);
    }
};

this.equal_Obj = function(obj, e)
{
    this.Static01.set_visible(true);
    this.Static01.move(e.clientx-10, e.clienty+5);
};

this.different_Obj = function(obj, e)
{
    this.Static01.set_visible(false);
};

```

5 ondrop 이벤트 함수 작성하기

ondrop 이벤트가 발생하면 Edit 컴포넌트가 문자열을 옮길 수 있는지 없는지 판단하고 옮길 수 있는 컴포넌트라면 value 속성값을 설정해줍니다. 그리고 Static 컴포넌트는 모두 보이지 않게 합니다.

```

this.sample_grid_65_ondrop = function(obj:nexacro.Form,e:nexacro.DragEventInfo)
{
    for(var i = 0; i < arr1.length; i++ )
    {
        if(e.fromobject != undefined && e.fromobject.name == arr1[i].name)
        {
            this.Static00.set_visible(false);

```

```

        this.Static01.set_visible(false);
        return false;
    }
}

for(var i = 0; i < arr2.length; i++ )
{
    if(e.fromobject != undefined && e.fromobject.name == arr2[i].name)
    {
        e.fromobject.set_value(sTextDragData);
        this.Static00.set_visible(false);
    }
}
this.Static00.set_visible(false);
};

```

⑥ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 Grid 컴포넌트의 문자열을 Edit 컴포넌트로 드래그앤드롭 동작으로 이동합니다.

25.18 두 개의 Grid 컴포넌트의 스크롤을 동시에 움직이기

Grid 컴포넌트의 스크롤 이동 시 발생하는 이벤트 함수 내에서 다른 Grid 컴포넌트의 스크롤 위치를 변경할 수 있습니다. 두 개의 Grid 컴포넌트를 하나처럼 보이게 할 수 있습니다.

25.18.1 예제

왼쪽에 배치한 Grid 컴포넌트는 스크롤바가 보이지 않도록 설정했습니다. 오른쪽 Grid 컴포넌트의 스크롤바 이동 시 같이 움직입니다.

Column0	Column1	Column0	Column1
2	B	2	duo
3	C	3	tria
4	D	4	quatuor
5	E	5	quinque
6	F	6	sex
7	G	7	septem
8	H	8	octo



sample_grid_69.xfdl

25.18.2 예제에서 사용한 핵심 기능

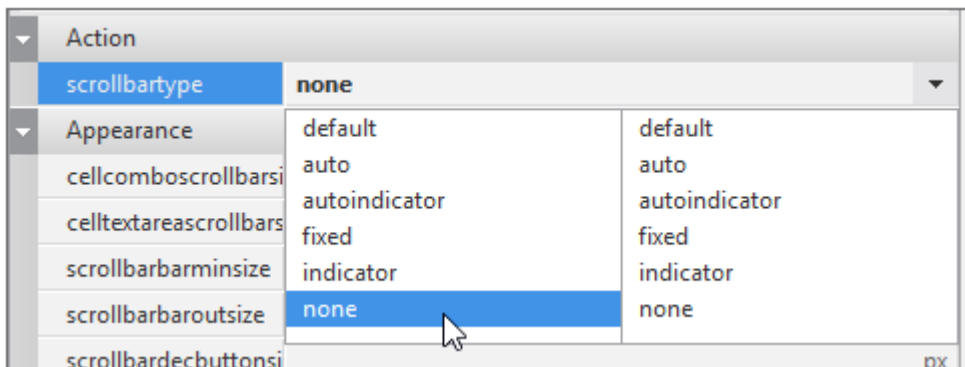
scrollTo

가로, 세로 스크롤바의 위치를 지정한 위치로 이동합니다.

25.18.3 예제 구현 방법

① 화면 구성하기

Grid 컴포넌트를 예제 화면과 같이 배치합니다. 왼쪽에 배치하는 Grid 컴포넌트는 scrollbar type 속성값을 "none"으로 지정합니다. 가로, 세로 스크롤바가 모두 보이지 않도록 하나의 값("none")만 설정합니다.



예제에서는 Dataset 오브젝트를 두 개 생성했지만, 같은 Dataset 오브젝트를 바인딩해서 사용해도 상관없습니다. 각 Dataset 오브젝트의 데이터 갯수만 일치하도록 합니다.

② onvscroll 이벤트 함수 작성하기

오른쪽 Grid 컴포넌트의 onvscroll 이벤트 함수를 작성합니다. 스크롤이 동작할때마다 이벤트가 발생하며 왼쪽 Grid 컴포넌트의 스크롤 위치를 변경해줍니다.

```
this.Grid01_onvscroll = function(obj:nexacro.Grid,e:nexacro.ScrollEventInfo)
{
    this.Grid00.scrollTo(0,e.pos);
};
```

3 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 오른쪽 Grid 컴포넌트의 스크롤바를 이동했을때 왼쪽 Grid 컴포넌트가 스크롤되는지 확인해봅니다.

25.19 체크된 Row 숫자 반환하기

CheckBoxControl 오브젝트의 text 속성값을 expr로 처리합니다. 체크된 상태를 별도의 변수로 저장하고 있다가 버튼 클릭 시 결과를 반환합니다.

25.19.1 예제

Grid 컴포넌트에서 체크박스를 선택한 결과를 Button 컴포넌트 클릭 시 반환합니다.

Column0	Column1	Column2
☒		
☒		
☐		
☒		
☐		

Button00

No. of selected rows : 3



25.19.2 예제에서 사용한 핵심 기능

redrawExprCell

Cell 오브젝트에 설정한 expr 값의 갱신 범위를 지정합니다. head, body, summary 중 하나를 설정할 수 있으며, 범위를 지정하지 않으면 전체 밴드 영역의 expr 값을 갱신합니다.



redrawExprCell 메소드는 17.0.0.400 이후 버전부터 사용할 수 있습니다.

25.19.3 예제 구현 방법

① 화면 구성하기

Grid 컴포넌트와 Button 컴포넌트를 예제 화면과 같이 배치합니다. Dataset 오브젝트를 아래와 같이 생성하고 바인딩해줍니다.

Columns			
No	id	type	size
1	Column0	STRING	256
2	Column1	STRING	256
3	Column2	STRING	256

Rows		
No	Column0	Column1
1	[Undefined]	[Undefined]
2	[Undefined]	[Undefined]
3	[Undefined]	[Undefined]
4	[Undefined]	[Undefined]
5	[Undefined]	[Undefined]

② Grid Contents Editor 실행하기

Column0 컬럼 Cell 오브젝트의 속성을 아래와 같이 수정합니다. 체크박스 체크 여부는 ck_func 라는 함수에서 처리합니다.

```
displaytype: checkboxcontrol
editttype: checkbox
text: expr:comp.parent.ck_func(currow)
```

3 ck_func 함수 작성하기

ck_func 함수에서는 배열 변수에 해당하는 row를 인덱스값으로 지정해서 0 또는 1의 값을 설정하고 저장한 다음 값을 반환합니다. 실제 배열 변수에 있는 값은 Cell 영역을 클릭 시 값이 토글 형태로 바뀝니다. 값이 바뀌면 redrawExprCell 메소드를 호출해서 expr로 설정된 값을 처리하도록 합니다.

```
this.ck_arr = [];

this.ck_func = function(nRow)
{
    return this.ck_arr[nRow] == null ? 0 : this.ck_arr[nRow];
}

this.Grid00_onclick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    if(e.cell == 0 )
    {
        this.ck_arr[e.row] = this.ck_arr[e.row] == null ? 1 : this.ck_arr[e.row]^1;
        this.Grid00.redrawExprCell('body');
    }
};
```



^ 기호는 배타적 논리합 연산자입니다. 비트 단위 연산 시 사용합니다. 예제에서는 토글 형태로 값이 0과 1로 변경되는 연산 처리에 사용했습니다.

4 onclick 이벤트 함수 작성하기

Button 컴포넌트 클릭 시 배열 변수에 담긴 값을 확인하고 선택된 값이 몇 개인지 TextArea 컴포넌트에 표시합니다.

```
this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var cnt = 0;
    for(var i = 0 ; i < this.ck_arr.length ; i++)
    {
```

```

        if(this.ck_arr[i])
        {
            cnt = cnt + 1;
        }
    }

    this.TextArea00.set_value("No. of selected rows : " + cnt);

    if(cnt == 0){
        this.TextArea00.set_value("There is no selected row " );
    }

};

```

5 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 체크박스의 상태를 바꾼 다음 Button 컴포넌트를 클릭해 TextArea 컴포넌트에 값이 표시되는 것을 확인합니다.

25.20 Like 조건으로 검색하기

filter 메소드에 조건식을 사용해 입력된 문자열과 일부만 일치하는 검색 결과를 표시할 수 있습니다.

25.20.1 예제

검색어를 입력하고 [%Like%] 버튼을 클릭하면 해당 검색어가 포함된 모든 결과를 표시하고 [Like%] 버튼을 클릭하면 해당 검색어로 시작하는 결과만 표시합니다. [Init] 버튼 클릭 시 검색결과를 초기화합니다.

Name
donald
lisa
michael
lili
colt

%Like%
Like%
Init



25.20.2 예제에서 사용한 핵심 기능

filter

파라미터 없이 filter 메소드를 실행하는 경우에는 filterstr 속성값에 지정된 값으로 처리됩니다. filterstr 속성값이 없는 상태에서 파라미터 없이 filter 메소드를 실행하면 undefined 값이 전달되고 false로 처리되어 아무런 값도 표시하지 않습니다.

25.20.3 예제 구현 방법

① 화면 구성하기

Grid 컴포넌트와 Button 컴포넌트를 예제 화면과 같이 배치합니다. Dataset 오브젝트를 아래와 같이 생성하고 바인딩해줍니다. 예제에서는 대소문자 구분없이 검색하는 부분을 제외했습니다.

Columns		
No	id	type
1	Name	STRING
Rows		
No	Name	
1	john	
2	james	
3	donald	
4	lisa	
5	joe	
6	michael	
7	lilli	
8	colt	

② onclick 이벤트 함수 작성하기

[%Like%] 버튼 클릭 시 입력된 값이 포함된 모든 결과값을 반환합니다. indexOf 메소드를 사용해 검색어가 포함된 경우 0 이상의 값을 반환하는 모든 데이터를 표시하도록 합니다.

```

this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Dataset00.filter("String(Name).indexOf('"+this.Edit00.value+"')>=0");
};

```

[Like%] 버튼 클릭 시 입력된 값으로 시작되는 결과값을 반환합니다. indexOf 메소드 실행 시 검색어와 일치하는 인덱스가 0인 데이터만 표시하도록 합니다.

```

this.Button01_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Dataset00.filter("String(Name).indexOf('"+this.Edit00.value+"')==0");
};

```

[Init] 버튼 클릭 시 값을 초기화합니다. filterstr 속성값만 지정하더라도 onrowsetchanged 이벤트가 발생하며 초기화가 처리됩니다.

```

this.Button02_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Dataset00.set_filterstr();
    this.Dataset00.filter();
};

```

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 검색어를 입력하고 검색 결과를 확인합니다.

25.21 중복된 항목 제거하기

filter 메소드에 조건식을 사용해 중복된 데이터를 제거합니다. SQL문에서 DISTINCT 키워드와 비슷한 동작을 구현할 수 있습니다.

25.21.1 예제

검색어를 입력하고 [Column0] 버튼을 클릭하면 Column0 컬럼의 데이터 중 중복된 항목을 제거하고 남은 결과를 표시하고 [Column1 && Column2] 버튼을 클릭하면 Column0 컬럼의 데이터 중 중복된 항목을 제거하고 남은 결

과만 표시합니다. [Init] 버튼 클릭 시 검색결과를 초기화합니다.

Column0	Column1	Column2
1	1	a
2	1	b
4	2	d
5	2	e
6	3	f
7	3	g
5	4	h

Column0

Column1 && Column2

Init

 sample_grid_74.xfdl

25.21.2 예제에서 사용한 핵심 기능

findRowExpr

조건에 해당하는 결과 중 첫번째 Row 인덱스를 반환합니다. 조건은 맞지만 첫번째 Row 인덱스가 아니라면 중복된 결과로 판단하고 필터링합니다.

25.21.3 예제 구현 방법

① 화면 구성하기

Grid 컴포넌트와 Button 컴포넌트를 예제 화면과 같이 배치합니다. Dataset 오브젝트를 아래와 같이 생성하고 바인딩해줍니다.

Columns				
No	id	type	size	prop
1	Column0	STRING	256	
2	Column1	STRING	256	
3	Column2	STRING	256	
Rows				
No	Column0	Column1	Column2	
1	1	1	a	
2	2	1	b	
3	1	1	a	
4	4	2	d	
5	5	2	e	
6	6	3	f	
7	7	3	g	
8	5	4	h	
9	9	3	f	
10	4	1	a	

② onclick 이벤트 함수 작성하기

[Column0] 버튼 클릭 시 Column0 컬럼 내에서 중복된 항목을 제거합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var sExpr    = "rowidx==dataset.findRowExpr(\"Column0=='\" + Column0 + \"'\")";
    this.Dataset00.filter(sExpr);
};
```

[Column1 && Column2] 버튼 클릭 시에는 Column1, Column2 컬럼이 모두 중복된 항목을 제거합니다. 예제에서는 다른 컬럼을 설정한 경우에도 처리할 수 있도록 컬럼 정보를 파라미터로 받아서 처리합니다.

```
this.Button01 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.gf_distincDsFilter(this.Dataset00, "Column1,Column2");
};

this.gf_distincDsFilter = function(objDs, sColID)
{
    var arrArgs    = sColID.split(",");
    var sFilterExpr    = "";
```



```

var sFindRowExpr    = "";

for (var i=0; i<arrArgs.length; i++)
{
    if (objDs.getColumnInfo(arrArgs[i]) == null) continue;

    sFindRowExpr    += (sFindRowExpr) ? " && " : "";
    sFindRowExpr    += " " + arrArgs[i] + "=='\" + " + arrArgs[i] + " + \"'";
}

if (sFindRowExpr) {
    sFilterExpr     = "rowidx==dataset.findRowExpr(\"\" + sFindRowExpr + "\")";
}
objDs.filter(sFilterExpr);
};

```

[Init] 버튼 클릭 시 값을 초기화합니다.

```

this.Button02_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Dataset00.filter("");
};

```

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 중복된 항목이 제거되는지 확인합니다.

25.22 Radio 버튼 표현하기

Cell 오브젝트의 displaytype으로 Radio 컨트롤은 제공하지 않습니다. 하지만 필요에 따라 제시된 목록 중에서 하나만 선택해야 하는 경우 CheckBox 보다는 Radio 컨트롤이 사용자에게 더 익숙할 수 있습니다.

25.22.1 예제

첫번째 컬럼에 Radio 컨트롤을 표시합니다. 해당 항목을 선택했을때 기존에 선택된 항목이 있다면 선택을 해제하고 하나의 항목만 선택하도록 합니다.

	Column1	Column2
☐	A	1
☒	B	2
☐	C	3
☐	D	4



25.22.2 예제에서 사용한 핵심 기능

displaytype

바인딩된 데이터가 화면에 표시되는 형식을 설정합니다. "imagecontrol"을 속성값으로 지정한 경우에는 text 속성값에 이미지 파일이 있는 경로를 지정하거나 BLOB 값을 설정해주어야 합니다. 예제에서는 Column0 값에 따라 이미지 경로를 다르게 해서 체크된 상태를 보여줄 수 있도록 했습니다.

25.22.3 예제 구현 방법

① 화면 구성하기

Grid 컴포넌트를 예제 화면과 같이 배치합니다. Dataset 오브젝트를 아래와 같이 생성하고 바인딩해줍니다.

Columns				
No	id	type	size	prop
1	Column0	STRING	256	
2	Column1	STRING	256	
3	Column2	STRING	256	
Rows				
No	Column0	Column1	Column2	
1	0	A	1	
2	0	B	2	
3	0	C	3	
4	0	D	4	

2 Grid Contents Editor 실행하기

Column0 컬럼은 Radio 컨트롤 이미지만 표시할겁니다. size를 30 정도로 수정하고 head cell의 text 속성은 지웁니다. 그리고 body cell 속성에서 displaytype 속성값은 "imagecontrol", text 속성값은 이미지 경로로 수정합니다.

body				cssclass	
col0	col1	col2		id	body0
	Column1	Column2		Action	
				autosizecol	default
				autosizerow	default
				controlautosizing	both
				displaytype	imagecontrol
				edittype	none
				expr	
				suppress	0
				suppressalign	first
				text	expr:Column0==1?"theme://images/rdo_WF_Radio_S.png":

설정한 text 속성값은 아래와 같습니다.

```
expr:Column0==1?"theme://images/rdo_WF_Radio_S.png":"theme://images/rdo_WF_Radio_0.png"
```

3 onclick 이벤트 함수 작성하기

Radio 컨트롤이 표시되는 셀 클릭 시 Column0 값을 업데이트합니다. 클릭한 셀에 해당하는 데이터는 1로 나머지는 0으로 설정합니다.

```
this.Grid00_onclick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    if (e.cell == 0)
    {
        this.Dataset00.set_enableevent(false);
        for(var i=0; i<this.Dataset00.rowcount; i++)
        {
            if (e.row == i)
            {
                this.Dataset00.setColumn(e.row,"Column0","1");
            }else{
                this.Dataset00.setColumn(i,"Column0","0");
            }
        }
        this.Dataset00.set_enableevent(true);
    }
}
```

```
}
};
```

④ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 Radio 컨트롤이 표시되는 셀을 클릭해봅니다.

25.23 세계 시간 표시하기

간단한 스크립트로 세계 시간을 Grid 컴포넌트에 표시하는 방법을 살펴봅니다. 타이머를 사용하고 갱신되는 시점에 시간 정보를 가져오기 때문에 아주 정확한 시간은 아닙니다.

25.23.1 예제

화면 로딩 시 타이머 이벤트를 처리하며 1초마다 시간값을 새로고쳐서 표시합니다.

timezone	timevalue
KST (한국 표준시)	2019-01-08 10:33:23
PST (태평양 표준시)	2019-01-07 17:33:23
PDT (태평양 표준시/일광절약시간제)	2019-01-07 18:33:23
EST (동부 표준시)	2019-01-07 20:33:23
EDT (동부 표준시/일광절약시간제)	2019-01-07 21:33:23
CET (중앙유럽 표준시)	2019-01-08 02:33:23
CEST (중앙유럽 표준시/일광절약시간제)	2019-01-08 03:33:23
CST (중국 표준시)	2019-01-08 09:33:23
UTC (협정 세계시)	2019-01-08 01:33:23



25.23.2 예제에서 사용한 핵심 기능

getTimezoneOffset

Date 오브젝트와 UTC(협정세계시) 사이의 시간차를 반환합니다. 예제에서는 Date 오브젝트에서 현재 시간 정보를 가져와서 UTC 시간으로 맞추기 위한 용도로 getTimezoneOffset 메소드를 사용합니다.

25.23.3 예제 구현 방법

- 1 화면에 Grid 컴포넌트를 배치합니다.
- 2 Dataset 오브젝트를 Invisible Object 영역으로 배치하고 아래와 같이 데이터를 입력합니다. timezone 컬럼 값은 Grid 컴포넌트에 표시하는 항목이고 timevalue 컬럼 값은 offset 컬럼 값을 가지고 계산한 결과를 보여줄 겁니다.

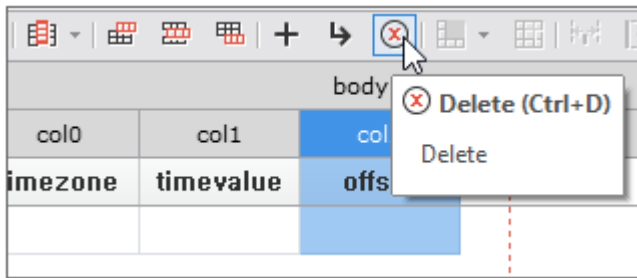
Columns				
No	id	type	size	prop
1	timezone	STRING	256	
2	timevalue	STRING	256	
3	offset	INT	256	

Rows			
No	timezone	timevalue	offset
1	KST (한국 표준시)	[Undefined]	9
2	PST (태평양 표준시)	[Undefined]	-8
3	PDT (태평양 표준시/일광...	[Undefined]	-7
4	EST (동부 표준시)	[Undefined]	-5
5	EDT (동부 표준시/일광...	[Undefined]	-4
6	CET (중앙유럽 표준시)	[Undefined]	1
7	CEST (중앙유럽 표준시/...	[Undefined]	2
8	CST (중국 표준시)	[Undefined]	8
9	UTC (협정 세계시)	[Undefined]	0

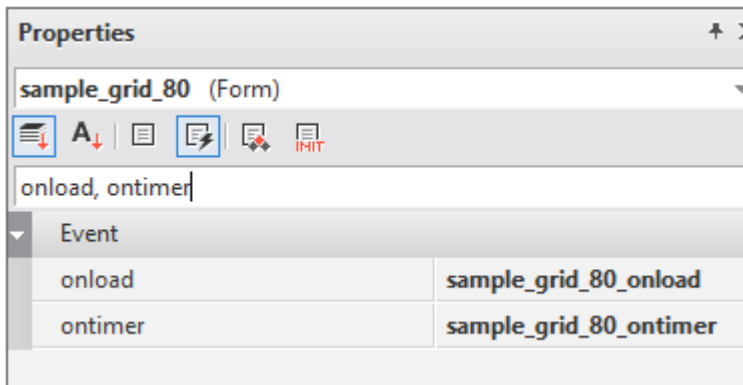
offset 컬럼 값을 원하는 값으로 수정하면 다른 시간대를 확인할 수 있습니다.

- 3 Grid 컴포넌트에 Dataset 오브젝트를 바인딩합니다.
- 4 Grid 컴포넌트를 더블 클릭해 Grid Contents Editor를 띄웁니다.

- ⑤ offset 컬럼값은 계산을 위해 사용하고 화면에 보여주지 않습니다. 해당 컬럼을 삭제합니다.



- ⑥ Form 오브젝트에 onload, ontimer 이벤트를 등록합니다.



onload 이벤트 함수에서는 타이머를 설정하고 ontimer 이벤트 함수에서는 1초마다 시간 정보를 받아서 Grid 컴포넌트의 timevalue 컬럼값에 값을 설정합니다.

```
this.sample_grid_80_onload = function(obj:nexacro.Form,e:nexacro.LoadEventInfo)
{
    this.setTimer(0, 1000);
};

this.sample_grid_80_ontimer = function(obj:nexacro.Form,e:nexacro.TimerEventInfo)
{
    for(var i=0;i<this.Dataset00.getRowCount();i++)
    {
        this.Dataset00.setColumn(i, 1,
            this.getWorldTime(this.Dataset00.getColumn(i, 2)));
    }
};
```

- ⑦ 시간값을 계산해서 반환할 getWorldTime 함수를 아래와 같이 작성합니다. 현재 시간을 1/1000 초 기준으로 계산(getTime)하고 이 값을 UTC 시간으로 보정한 후(getTimezoneOffset) 각 지역 시간대에 맞게 offSet 값을 더해서 해당 시간대의 시간 정보를 생성합니다.

```

this.getWorldTime = function(offSet)
{
    var objDate = new Date();
    var nTime = objDate.getTime() + (objDate.getTimezoneOffset() * 60000) + (offSet * 3600000);
    objDate.setTime(nTime);

    var s =
    this.fn_getDate(objDate.getFullYear(), 4) + '-' +
    this.fn_getDate(objDate.getMonth() + 1, 2) + '-' +
    this.fn_getDate(objDate.getDate(), 2) + ' ' +

    this.fn_getDate(objDate.getHours(), 2) + ':' +
    this.fn_getDate(objDate.getMinutes(), 2) + ':' +
    this.fn_getDate(objDate.getSeconds(), 2);

    return s;
}

```

fn_getDate 함수는 시간 값이 지정된 자리수보다 작은 경우 앞에 "0"을 채워서 표시합니다.

```

this.fn_getDate = function(n, digits)
{
    var zero = '';
    n = n.toString();

    if (n.length < digits) {
        for (i = 0; i < digits - n.length; i++)
            zero += '0';
    }
    return zero + n;
}

```

- ⑧ QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. Grid 컴포넌트에 세계 시간이 표시되는지 확인합니다.

25.24 PopupDiv 컴포넌트를 사용해 선택한 셀 테두리선 표현하기

PopupDiv 컴포넌트를 사용해 사용자가 선택한 셀 주변에 원하는 색상의 테두리선을 표현해봅니다.

25.24.1 예제

특정 셀을 클릭하면 해당 셀 주변에 빨간 테두리선을 표시합니다. Grid 컴포넌트 외 다른 영역을 클릭하거나 다른 셀을 클릭하면 이전 테두리선은 사라집니다.

Column0	Column1
1	A
2	B
3	C
4	D
5	E

 sample_grid_81.xfdl

25.24.2 예제에서 사용한 핵심 기능

getCellRect

선택한 셀 좌표값을 반환합니다. 해당 좌표값에 따라 PopupDiv 컴포넌트를 띄워줍니다.

25.24.3 예제 구현 방법

- 1 화면에 Grid 컴포넌트를 배치합니다.
- 2 Dataset 오브젝트를 Invisible Object 영역으로 배치하고 데이터를 입력합니다. 예제에서는 데이터가 큰 의미가 없기 때문에 2개의 컬럼을 만들고 1-5, A-E 까지의 텍스트를 입력했습니다.

- ③ Grid 컴포넌트에 Dataset 오브젝트를 바인딩합니다.
- ④ PopupDiv 컴포넌트를 배치합니다. visible 속성값은 false로 설정하고 border 속성은 원하는 스타일로 지정합니다. 예제에서는 아래와 같이 지정했습니다.

1px solid coral

- ⑤ Grid 컴포넌트에 oncellclick 이벤트 함수를 등록합니다. 셀 영역 클릭 시 선택한 셀 좌표값을 확인하고 해당 좌표값을 파라미터로 PopupDiv 컴포넌트의 trackPopupByComponent 메소드를 실행합니다.

```
this.Grid00_onclick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    var aryRect = this.Grid00.getCellRect(e.row, e.cell);

    var nX = aryRect.left;
    var nW = aryRect.width;
    var nH = aryRect.height;
    var nY = nY = aryRect.top + 2;

    this.PopupDiv00.trackPopupByComponent(this.Grid00, nX, nY, nW, nH, "", false);
};
```

- ⑥ QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. Grid 컴포넌트의 셀 클릭 시 테두리선이 표시되는지 확인합니다.

25.25 트리 구조에서 자식 정보 확인하기

트리 구조로 표현된 Grid 컴포넌트에서 선택한 셀의 자식 정보를 확인합니다.

25.25.1 예제

특정 셀을 클릭했을 때 하위 트리 정보가 있는 경우 TextArea 컴포넌트에 표시합니다. 해당 정보가 없으면 자식 정보가 없다고 표시합니다.

label	level
A	0
B	1
C	2
D	2
E	2
F	1
G	2
H	2
I	0
J	0

ChildCount = 2
Row = 1, RowText = B
Row = 5, RowText = F



sample_grid_79.xfdl

25.25.2 예제에서 사용한 핵심 기능

getTreeChildCount

특정 Row 아래의 자식 Row 정보를 반환합니다. 트리 구조인 경우에 하위 정보가 있는지 확인하는 용도로 사용합니다.

treeuseline

트리 형태인 경우 연결선을 표시합니다. 예제에서는 연결선을 보이지 않도록 false로 값을 설정했습니다. 연결선이 없더라도 레벨에 따라 들여쓰기된 상태로 표시됩니다.

treeusebutton

자식 Row 정보가 있는 경우에는 접거나 펴는 기능을 처리할 수 있습니다. 예제에서는 트리를 접을 수 없도록 treeusebutton 속성값을 "noclick"으로 지정했습니다. 이렇게 하면 버튼이 보이지도 않고 버튼 영역을 클릭해도 이벤트가 발생하지 않습니다.

25.25.3 예제 구현 방법

- 1 화면에 Grid 컴포넌트와 TextArea 컴포넌트를 배치합니다.
- 2 Dataset 오브젝트를 추가하고 아래와 같이 데이터를 설정합니다.

Columns				
No	id	type	size	prop
1	label	STRING	256	
2	level	STRING	256	

Rows		
No	label	level
1	A	0
2	B	1
3	C	2
4	D	2
5	E	2
6	F	1
7	G	2
8	H	2
9	I	0
10	J	0

- Dataset 오브젝트를 Grid 컴포넌트로 드래그앤드롭해서 `binddataset` 속성으로 지정합니다.
- Grid 컴포넌트를 더블클릭 해 Grid Contents Editor를 실행하고 첫 번째 컬럼 셀 속성값을 아래와 같이 수정합니다.

The screenshot shows the Grid Contents Editor interface. On the left, a table structure is visible with columns 'col0' and 'col1', and rows 'label' and 'level'. On the right, the 'Action' group is expanded, showing the following properties:

- displaytype**: treeitemcontrol
- edittype**: tree

The 'CellMaskEdit' group is also expanded, showing:

- maskedittype**: number

The 'CellTreeltem' group is expanded, showing:

- treelevel**: bind:level

- Grid 컴포넌트의 속성 중 `CellTreeltem` 그룹 아래의 속성 일부를 수정합니다.

트리 구조를 접을 수 없도록 `treeusebutton` 속성값을 "noclick"으로 지정하고 최초 상태가 모두 펼쳐진 상태로 표시되도록 `treeinitstatus` 속성값을 "expand,all"으로 지정합니다.

CellTreeltem	
treeasynctoggle	false
treeinitstatus	expand,all
treeusebutton	noclick
treeusecheckbox	false
treeuseexpandkey	false
treeuseimage	false
treeuseline	false

- ⑥ Grid 컴포넌트의 oncellclick 이벤트를 함수를 아래와 같이 추가합니다.

getTreeChildCount 메소드로 자식 정보가 있는지 확인하고 자식 정보가 있다면 getTreeChildRow 메소드를 사용해 해당 Row의 인덱스 정보를 확인하고 정보를 TextArea 컴포넌트에 표시합니다.

```
this.Grid00_onclick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    var numChildcount = this.Grid00.getTreeChildCount(this.Dataset00.rowposition);
    var strChildInfo = "ChildCount = "+numChildcount+"\n";

    for(var i=0; i<numChildcount; i++)
    {
        var numChildRow = this.Grid00.getTreeChildRow(this.Dataset00.rowposition, i, false
    );
        strChildInfo += "Row = "+numChildRow+", RowText = "+this.Grid00.getCellText(
numChildRow, 0)+"\n";
    }
    this.TextArea00.set_value(strChildInfo);
};
```

- ⑦ QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. Grid 컴포넌트의 셀 클릭 시 자식 Row 정보가 표시되는지 확인합니다.

25.26 데이터 정렬 시 null 값을 가장 위에 표시하기

keystring 속성을 사용하면 데이터 정렬을 처리할 수 있습니다. 하지만, 경우에 따라 null 값은 정렬 방향과 상관없이 무조건 위 또는 아래에 배치해야 하는 경우가 있습니다. 기본 옵션에서는 제공하지 않는 기능이라 약간의 트릭을 사용

해야 합니다.

25.26.1 예제

"null check" 버튼 클릭 시 null 값을 가장 위로 올리고 내림차순으로 데이터를 정렬합니다.



17.1.0.100 이후 버전에서는 문자열 정렬 방식이 변경되었습니다. 때문에 첫 번째 Button 컴포넌트 클릭 시 null 값이나 빈 값을 가장 위에 표시합니다. 해당 예제는 17.0 버전을 사용하거나 datatyperule 속성 값을 "1.0"으로 지정해 사용하는 경우 참고해주세요.

[데이터 정렬 방식 변경](#)

Column0	Column1
	B
	D
8	J
7	I
6	H
5	F
4	E
3	G
2	C
1	A

S:- Column0

null check



[sample_grid_87.xfdl](#)

25.26.2 예제에서 사용한 핵심 기능

copyRow

Dataset 오브젝트 간 특정 Row 데이터를 복사합니다. 예제처럼 임시로 값을 옮겨놓고 가공하는 경우에 사용할 수 있습니다.

25.26.3 예제 구현 방법

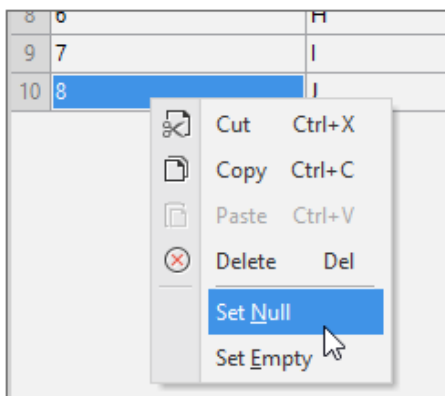
- ① 화면에 Grid 컴포넌트와 Button 컴포넌트를 배치합니다.
- ② Dataset 오브젝트를 추가하고 아래와 같이 데이터를 설정합니다. 임시로 사용할 Dataset 오브젝트는 id 속성 값을 "dsTemp"로 지정하고 Columns 만 설정합니다.

Dataset Editor [Dataset00]

Const Columns				
Columns				
No	id	type	size	prop
1	Column0	STRING	256	
2	Column1	STRING	256	

Rows		
No	Column0	Column1
1	1	A
2	[Undefined]	B
3	2	C
4	[Undefined]	D
5	4	E
6	5	F
7	3	G
8	6	H
9	7	I
10	8	J

null 값을 예외 처리하는 예제이기 때문에 Column0 항목에 null 값을 설정해주어야 합니다. 값을 지정하지 않으면 null 값이 설정되고 컨텍스트 메뉴에서 [Set Null] 항목을 선택해도 null 값을 설정할 수 있습니다.



- ③ 첫 번째 Button 컴포넌트 클릭 시에는 내림차순으로 정렬만 합니다. 이 경우에는 null 값은 마지막에 배치됩니다.

```
this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Dataset00.set_keystring("S:-Column0");
};
```

- ④ 두 번째 Button 컴포넌트 클릭 시에는 null 값을 체크해서 임시 Dataset 오브젝트로 복사한 후 나중에 다시 값을 가져옵니다.

```
this.Button01_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    for(var i=0 ; i < this.Dataset00.rowcount; i++){
        if(this.Dataset00.getColumn(i,"Column0") == null){
            var nRow = this.dsTemp.addRow();
            this.dsTemp.copyRow(nRow,this.Dataset00,i);
        }
    }

    for(var k=this.Dataset00.rowcount-1 ; k >= 0; k--){
        if(this.Dataset00.getColumn(k,"Column0") == null){
            this.Dataset00.deleteRow(k);
        }
    }

    this.Dataset00.set_keystring( "S:-Column0" );

    for(var l=0; l < this.dsTemp.rowcount; l++){
        this.Dataset00.insertRow(l);
        this.Dataset00.copyRow(l,this.dsTemp,l);
    }
};
```

- ⑤ QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. null 값이 어느 부분에 위치하면서 정렬되는지 확인합니다.

25.27 데이터 너비에 맞추어 컬럼 너비 조절하기

Grid 컴포넌트의 `autosizingtype` 속성값을 지정하지 않고 동적으로 데이터 너비에 맞추어 컬럼 너비를 조절하는 방법을 살펴봅니다. Grid 컴포넌트에서 지원하는 기능은 아니고 약간의 트릭을 사용합니다.

25.27.1 예제

Grid 컴포넌트 컬럼 헤더 영역을 클릭하면 해당 컬럼 너비가 데이터 너비에 맞추어 조절됩니다.

Column0	Column1	Column2	Column3
ABCDEFGHIJK	ABC	ABC	ABCD
ABC	ABCD	ABC	AB
ABCD	ABC	ABCDEFGHIJ	ABCD
ABCDEFGHIJK	ABC	ABC	ABCDEFGHIJKLM
ABCDEFGHIJK	AB	ABC	



sample_grid_90.xfdl

25.27.2 예제에서 사용한 핵심 기능

`getOffsetWidth`

동적으로 계산된 컴포넌트의 너비값을 반환합니다.

`setFormatColProperty`

컬럼 속성값을 설정합니다. 예제에서는 너비값을 재정의합니다.

25.27.3 예제 구현 방법

- 1 화면에 Grid 컴포넌트와 Static 컴포넌트를 배치합니다.
- 2 Dataset 오브젝트를 추가하고 아래와 같이 데이터를 설정합니다. 데이터는 적당한 길이로 기본 Grid 컴포넌트

의 컬럼 너비에서 보이지 않을 정도로만 지정합니다.

Columns							
No	id	type	size	prop	sumtext	description	
1	Column0	STRING	256				
2	Column1	STRING	256				
3	Column2	STRING	256				
4	Column3	STRING	256				

Rows				
No	Column0	Column1	Column2	Column3
1	ABCDEFGHUIJK	ABC	ABC	ABCD
2	ABC	ABCD	ABC	AB
3	ABCD	ABC	ABCDEFGHUIJK	ABCD
4	ABCDEFGHUIJK	ABC	ABC	ABCDEFGHUIJKLM
5	ABCDEFGHUIJK	AB	ABC	

- ③ 데이터가 입력되었다면 Grid 컴포넌트와 바인딩해줍니다.
- ④ Static 컴포넌트의 visible 속성값을 false로 지정하고 fittocontents 속성값은 "width"로 지정합니다. Static 컴포넌트는 너비 조정을 위한 보조 용도로 사용합니다.
- ⑤ Grid 컴포넌트의 onheadclick 이벤트 함수를 아래와 같이 작성합니다.

컬럼 헤더 영역을 클릭하게 되면 해당 컬럼 위치의 데이터 중에서 가장 긴 문자열을 확인하고 해당 문자열을 Static 컴포넌트의 text 속성값으로 설정합니다. 그리고 resetScroll 메소드를 실행하게 되면 fittocontents 속성 설정에 따라 Static 컴포넌트의 너비가 조정됩니다. 그러면 해당 너비값을 기준으로 컬럼 너비값을 조정합니다.

```

this.Grid00_onheadclick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    var nMaxVal = 0;
    var nCurrVal = 0;
    var sMaxText = null;
    for(var i=0; i<this.Dataset00.rowcount; i++) {
        nCurrVal = this.Dataset00.getColumn(i, e.col).length;
        if(nCurrVal > nMaxVal) {
            nMaxVal = nCurrVal;
            sMaxText = this.Dataset00.getColumn(i, e.col);
        }
    }
    this.staTemp.set_text(sMaxText);
}

```

```

this.resetScroll();
this.Grid00.setFormatColProperty(e.col, "size", this.staTemp.getOffsetWidth()+10);
};

```

- ⑥ QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. 컬럼 헤더 영역을 클릭하면 컬럼 너비가 조절되는지 확인합니다.

25.28 특정 Grid 컴포넌트만 합계 Row 추가하기

Grid 컴포넌트에 사용자 속성을 추가하고 사용자 속성값에 따라 특정 기능을 추가하는 방법을 살펴봅니다.

25.28.1 예제

버튼 클릭 시 첫 번째, 두 번째 Grid 컴포넌트 마지막 Row에 합계값을 보여줍니다.

Add sum row

Column0	Column1	Column2
1	1	2
2	3	3
3	4	5
6	8	10

Column0	Column1	Column2
4	5	3
2	6	3
9	7	5
15	18	11

Column0	Column1	Column2
41	5	31
2	61	3
9	7	5

 sample_grid_93.xfdl

25.28.2 예제에서 사용한 핵심 기능

getSum

Dataset 오브젝트에서 필터링되지 않은 Row 영역의 합계값을 반환합니다.

getCellCount

Grid 컴포넌트의 특정 밴드(예제에서는 body 밴드)의 Cell 개수를 반환합니다. 예제에서는 body 밴드에 하나의 Row만 있기 때문에 Cell 개수가 컬럼 개수와 같습니다. getCellCount 대신 바인딩된 Dataset 오브젝트의 getColCount 메소드를 사용해도 같은 결과를 반환합니다.

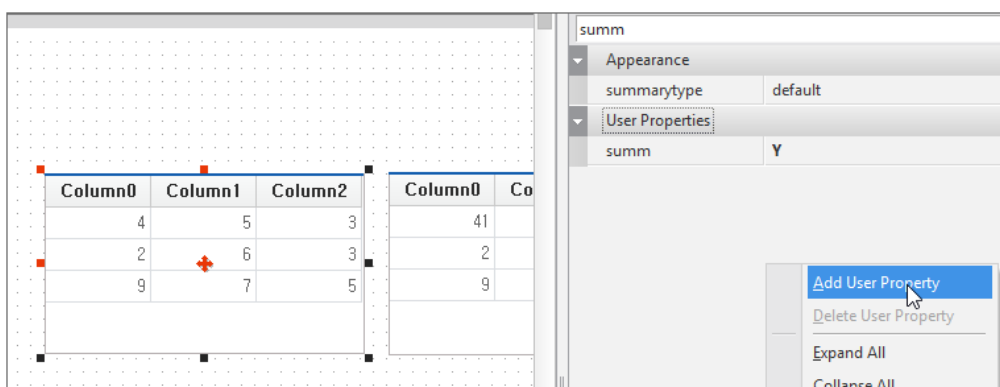
25.28.3 예제 구현 방법

- 1 화면에 Grid 컴포넌트와 Button 컴포넌트를 배치합니다.
- 2 Dataset 오브젝트를 추가하고 아래와 같이 데이터를 설정합니다. 합계값을 계산해야 하기 때문에 type 항목은 "INT"로 지정합니다. 3개의 Grid 컴포넌트에 각각 다른 Dataset 오브젝트를 만들어서 바인딩합니다.

Columns					
No	id	type	size	prop	sumtext
1	Column0	INT	256		
2	Column1	INT	256		
3	Column2	INT	256		

Rows			
No	Column0	Column1	Column2
1	1	1	2
2	2	3	3
3	3	4	5

- 3 데이터가 입력되었다면 Grid 컴포넌트와 바인딩해줍니다.
- 4 첫 번째와 두 번째 Grid 컴포넌트를 선택하고 속성창에서 오른쪽 마우스 클릭 후 [Add User Property] 항목을 선택해 "summ"이라는 이름의 속성을 추가하고 값은 "Y"로 설정합니다.



- 5 Button 컴포넌트의 onclick 이벤트 함수를 아래와 같이 작성합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.fn_setSumm(this);
};
```

합계값을 처리하는 것은 fn_setSumm 함수에서 처리합니다. Form에 속한 자식 중에서 Grid 컴포넌트를 찾아내고 Grid 컴포넌트의 사용자 속성 summ의 속성값이 "Y"인 항목을 찾습니다.

Grid 컴포넌트에 바인딩된 Dataset 오브젝트를 찾은 다음 Row를 추가하고 합계값을 설정합니다.

```
this.fn_setSumm = function(obj)
{
    var colNm;
    var objGrd;
    var objDs;
    for(var i=0; i<obj.all.length; i++){
        if( obj.all[i].toString() == "[object Grid]" ){
            objGrd = obj.all[i];
            if( objGrd.summ == "Y"){
                objDs = obj.all[objGrd.binddataset];
                var idx = objDs.addRow();
                for(var j=0; j<objGrd.getCellCount("body"); j++){
                    colNm = objGrd.getCellProperty("body", j, "text").split(":");
                    objDs.setColumn(idx, colNm[1], objDs.getSum(colNm[1], 0, objDs.
rowcount-1));
                }
            }
        }
    }
}
```

세 번째 Grid 컴포넌트는 summ이라는 속성이 없기 때문에 objGrd.summ 값이 null로 처리되고 해당 조건문에서 false로 처리됩니다.

- 6 QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. 버튼 클릭 시 첫 번째와 두 번째 Grid 컴포넌트에 Row가 추가되고 합계값이 표시되는지 확인합니다.

25.29 선택한 영역의 합계 보여주기

Grid 컴포넌트의 selecttype 속성값을 "area"로 지정한 경우 선택한 영역의 합계를 보여줍니다.

25.29.1 예제

SHIFT 키를 같이 사용해 특정 영역의 셀을 선택하면 선택한 셀의 합계값을 보여줍니다.

Column0	Column1
2,500	2,500
3,600	3,600
500	500
2,400	2,400
12,500	12,500
35,000	35,000

 sample_grid_98.xfdl

25.29.2 예제에서 사용한 핵심 기능

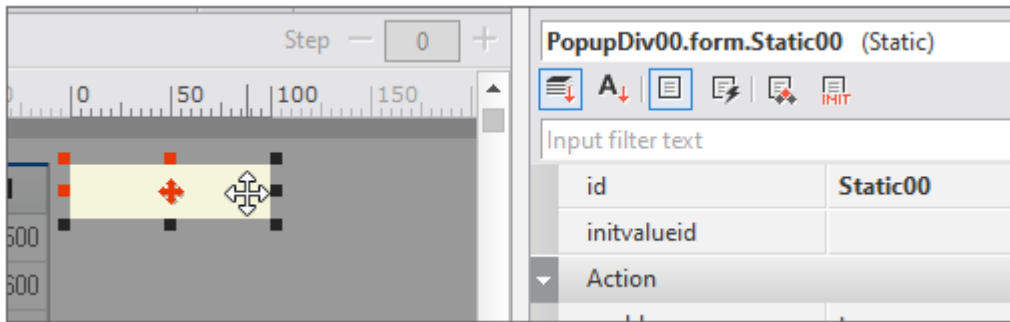
getSum

getSum 메소드의 첫 번째 파라미터는 컬럼 ID를 지정할 수도 있지만, 계산식을 설정할 수 있습니다. 예제에서는 선택한 컬럼이 2개인 경우 2개 컬럼의 값을 더하는 식을 첫 번째 파라미터로 지정합니다.

25.29.3 예제 구현 방법

- 1 화면에 Grid 컴포넌트와 PopupDiv 컴포넌트를 배치합니다.

PopupDiv 컴포넌트 안에는 Static 컴포넌트를 배치합니다. Static 컴포넌트에서 합계값을 표시합니다.



- ② Dataset 오브젝트를 추가하고 아래와 같이 데이터를 설정합니다. 합계값을 계산해야 하기 때문에 type 항목은 "INT"로 지정합니다.

Columns			
No	id	type	size
1	Column0	INT	256
2	Column1	INT	256

Rows		
No	Column0	Column1
1	2500	2500
2	3600	3600
3	500	500
4	2400	2400
5	12500	12500
6	35000	35000

- ③ 데이터가 입력되었다면 Grid 컴포넌트와 바인딩해줍니다.
- ④ Grid 컴포넌트의 selecttype 속성값을 "area"로 설정합니다.
- ⑤ Grid 컴포넌트의 onkeydown 이벤트를 함수를 아래와 같이 작성합니다.

Grid 컴포넌트에서 SHIFT 키를 눌러 여러 영역을 선택하는 경우 특정한 변수값을 설정합니다.

```

this.nShift = 0;
this.Grid00_onkeydown = function(obj:nexacro.Grid,e:nexacro.KeyEventInfo)
{
    if(e.keycode == 16) {
        this.nShift = 1;
    } else {
        this.nShift = 0;
    }
}

```

```

    }
};

```

- ⑥ Grid 컴포넌트의 onbuttonup 이벤트 함수를 아래와 같이 작성합니다.

Grid 컴포넌트의 특정 셀에서 SHIFT 키를 누른 상태에서 마우스 버튼을 놓았을 때 합계값을 처리합니다.

```

this.Grid00_onbuttonup = function(obj:nexacro.Grid,e:nexacro.GridMouseEventInfo)
{
    if(this.nShift == 1) {
        var strColExpr;
        if(obj.selectstartcol==0) {
            if(obj.selectendcol==1) {
                strColExpr = "Column0+Column1";
            } else {
                strColExpr = "Column0";
            }
        } else {
            strColExpr = "Column1";
        }

        var nTot = this.Dataset00.getSum(strColExpr, obj.selectstartrow, Number(obj.
selectendrow)+1);

        this.PopupDiv00.form.Static00.set_text("SUM : "+nTot);
        this.PopupDiv00.form.Static00.set_fittocontents('width');
        this.PopupDiv00.form.resetScroll();
        this.PopupDiv00.trackPopupByComponent(obj,e.canvasx,e.canvasy);
        this.nShift = 0;
    }
};

```

- ⑦ PopupDiv 컴포넌트의 oncloseup 이벤트를 함수를 아래와 같이 작성합니다.

```

this.PopupDiv00_oncloseup = function(obj:nexacro.PopupDiv,e:nexacro.EventInfo)
{
    obj.form.Static00.set_text("");
};

```

- ⑧ QuickView(Ctrl + F6)를 실행하여 특정 영역 선택 시 해당하는 값의 합계가 표시되는지 확인합니다.

25.30 클릭한 셀 표시하기

Grid 컴포넌트의 `selecttype` 속성값을 "row"로 지정한 경우 선택한 Row는 표시되는데 클릭한 셀은 따로 표시하지 않습니다. 클릭한 셀을 다른 형식으로 표시하는 예제입니다.

25.30.1 예제

클릭한 셀은 다른 배경색으로 표시합니다.

Column0	Column1	Column2



25.30.2 예제에서 사용한 핵심 기능

`getBindCellIndex`

Dataset 오브젝트의 특정 컬럼과 바인드된 첫 번째 Cell 인덱스를 반환합니다. Grid 컴포넌트에서 클릭한 셀의 인덱스를 확인하기 위한 용도로 사용했습니다.

`getCellPos`

선택한 셀의 인덱스값을 반환합니다. 예제에서는 `getBindCellIndex` 메소드에서 반환한 값과 `getCellPos` 메소드에서 반환한 값을 비교합니다.

25.30.3 예제 구현 방법

- 1 화면에 Grid 컴포넌트를 배치합니다.

- ② Dataset 오브젝트를 추가하고 아래와 같이 데이터를 설정합니다. Row는 값은 지정하지 않고 5개 정도만 추가합니다.

Columns						
No	id	type	size	prop	sumtext	
1	Column0	STRING	256			
2	Column1	STRING	256			
3	Column2	STRING	256			

Rows				
No	Column0	Column1	Column2	
1	[Undefined]	[Undefined]	[Undefined]	
2	[Undefined]	[Undefined]	[Undefined]	
3	[Undefined]	[Undefined]	[Undefined]	
4	[Undefined]	[Undefined]	[Undefined]	
5	[Undefined]	[Undefined]	[Undefined]	

- ③ 데이터가 입력되었다면 Grid 컴포넌트와 바인딩해줍니다.
- ④ Grid 컴포넌트를 더블 클릭하고 Grid Contents Editor를 실행합니다.
- ⑤ body 밴드의 각 셀을 선택하고 cssclass 속성값을 아래와 같이 지정합니다.
- getBindCellIndex 메소드의 컬럼 아이디 문자열은 각 컬럼에 맞게 지정합니다.

```
expr:comp.getBindCellIndex("body","Column0")==comp.getCellPos()?'sample_grid_99_cellclick':''
```

- ⑥ Grid 컴포넌트의 oncellclick 이벤트 함수를 아래와 같이 작성합니다.

```
this.Grid00_oncellclick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    obj.redrawExprCell("body");
};
```

- ⑦ xcss 파일을 열어서 아래 코드를 추가합니다.

Advanced 탭에서 선택자를 추가하거나 Text 탭에서 직접 코드값을 입력할 수 있습니다.

```
.Grid .body .row .cell.sample_grid_99_cellclick[userstatus=selected]
{
    -nexa-background-odd : lawngreen;
    background : lawngreen;
}
```

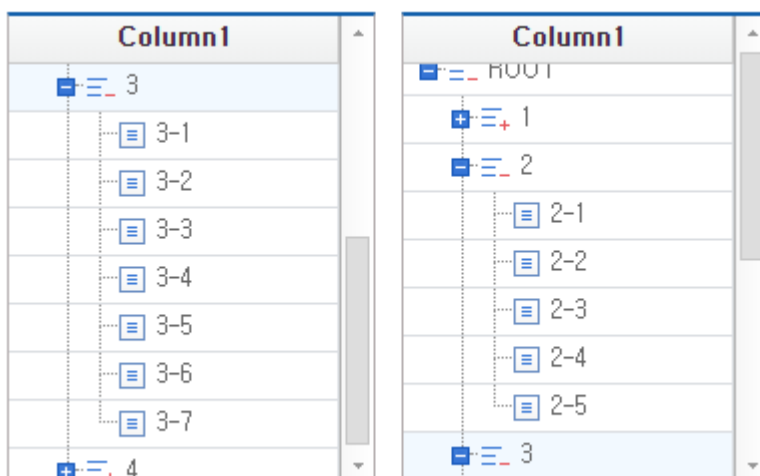
- ⑧ QuickView(Ctrl + F6)를 실행하여 셀 클릭 시 해당 Row가 선택되고 클릭한 셀의 배경색이 변경되는 것을 확인합니다.

25.31 트리를 펼치거나 접을 때 하위 레벨 데이터 보이게 하기

트리 형태일 때 데이터가 적으면 별 문제가 없지만 Grid 컴포넌트 영역보다 데이터가 많은 경우에는 트리를 펼쳤을 때 하위 레벨 데이터가 바로 보이지 않는 문제가 있습니다. 예제에서는 선택한 하위 레벨 데이터가 좀 더 잘 보일 수 있는 방법을 살펴봅니다.

25.31.1 예제

"3" 항목을 펼쳤을 때 오른쪽 Grid는 스크롤 위치 조정을 하지 않아서 하위 레벨 데이터가 보이지 않습니다. 왼쪽 Grid에서는 선택한 항목을 위에 보이도록 위치를 조정합니다.





25.31.2 예제에서 사용한 핵심 기능

getRealRowSize

특정 Row의 높이를 반환합니다. Grid 컴포넌트 설정 시 지정한 Row의 높이값은 다른 속성 설정이나 데이터에 따라 실제 보여지는 높이가 달라질 수 있어서 getRealRowSize 메소드를 사용해 실제 높이값을 확인합니다.

getTreeRow

트리 구조에서 Row 위치를 반환합니다. 예제에서 "2" 항목을 클릭하는 경우 선택한 Row의 index는 6이지만, 펼쳐지지 않은 트리 아이টে를 제외하고 보여지는 Row의 index는 2가 됩니다. getTreeRow 메소드는 펼쳐지지 않은 트리 아이টে를 제외하고 화면에 보여지는 Row index를 반환합니다.

25.31.3 예제 구현 방법

- ① 화면에 Grid 컴포넌트를 배치합니다.
- ② Dataset 오브젝트를 추가하고 아래와 같이 데이터를 설정합니다.

```
<ColumnInfo>
  <Column id="Column0" type="STRING" size="256"/>
  <Column id="Column1" type="STRING" size="256"/>
</ColumnInfo>
<Rows>
  <Row>
    <Col id="Column0">0</Col>
    <Col id="Column1">ROOT</Col>
  </Row>
  <Row>
    <Col id="Column0">1</Col>
    <Col id="Column1">1</Col>
  </Row>
  <Row>
    <Col id="Column0">2</Col>
    <Col id="Column1">1-2</Col>
  </Row>
```

```

<Row>
  <Col id="Column0">2</Col>
  <Col id="Column1">1-3</Col>
</Row>
<Row>
  <Col id="Column0">2</Col>
  <Col id="Column1">1-4</Col>
</Row>
<Row>
  <Col id="Column0">2</Col>
  <Col id="Column1">1-5</Col>
</Row>
<Row>
  <Col id="Column0">1</Col>
  <Col id="Column1">2</Col>
</Row>
<Row>
  <Col id="Column0">2</Col>
  <Col id="Column1">2-1</Col>
</Row>
<Row>
  <Col id="Column0">2</Col>
  <Col id="Column1">2-2</Col>
</Row>
<Row>
  <Col id="Column0">2</Col>
  <Col id="Column1">2-3</Col>
</Row>
<Row>
  <Col id="Column0">2</Col>
  <Col id="Column1">2-4</Col>
</Row>
<Row>
  <Col id="Column0">2</Col>
  <Col id="Column1">2-5</Col>
</Row>
<Row>
  <Col id="Column0">1</Col>
  <Col id="Column1">3</Col>
</Row>
<Row>

```

```

    <Col id="Column0">2</Col>
    <Col id="Column1">3-1</Col>
</Row>
<Row>
    <Col id="Column0">2</Col>
    <Col id="Column1">3-2</Col>
</Row>
<Row>
    <Col id="Column0">2</Col>
    <Col id="Column1">3-3</Col>
</Row>
<Row>
    <Col id="Column0">2</Col>
    <Col id="Column1">3-4</Col>
</Row>
<Row>
    <Col id="Column0">2</Col>
    <Col id="Column1">3-5</Col>
</Row>
<Row>
    <Col id="Column0">2</Col>
    <Col id="Column1">3-6</Col>
</Row>
<Row>
    <Col id="Column0">2</Col>
    <Col id="Column1">3-7</Col>
</Row>
<Row>
    <Col id="Column0">1</Col>
    <Col id="Column1">4</Col>
</Row>
<Row>
    <Col id="Column0">2</Col>
    <Col id="Column1">4-1</Col>
</Row>
<Row>
    <Col id="Column0">2</Col>
    <Col id="Column1">4-2</Col>
</Row>
<Row>
    <Col id="Column0">2</Col>

```

```

    <Col id="Column1">4-3</Col>
  </Row>
</Row>
  <Col id="Column0">2</Col>
  <Col id="Column1">4-4</Col>
</Row>
</Row>
  <Col id="Column0">2</Col>
  <Col id="Column1">4-5</Col>
</Row>
</Row>
  <Col id="Column0">2</Col>
  <Col id="Column1">4-6</Col>
</Row>
</Row>
  <Col id="Column0">2</Col>
  <Col id="Column1">4-7</Col>
</Row>
</Row>
  <Col id="Column0">2</Col>
  <Col id="Column1">4-8</Col>
</Row>
</Row>
  <Col id="Column0">2</Col>
  <Col id="Column1">4-9</Col>
</Row>
</Rows>

```

- ③ 데이터가 입력되었다면 Grid 컴포넌트와 바인딩해줍니다.
- ④ Grid 컴포넌트를 더블 클릭하고 Grid Contents Editor를 실행합니다.
- ⑤ body 밴드의 셀을 선택하고 아래와 같이 속성값을 설정합니다.

```

displaytype: treeitemcontrol
edittype: tree
treelevel: bind:Column0

```

- ⑥ Grid 컴포넌트와 Dataset 오브젝트를 복사해서 새로운 Grid 컴포넌트와 Dataset 오브젝트를 만듭니다.

- ⑦ 복사해서 만든 Grid 컴포넌트의 binddataset 속성값을 복사해서 만든 Dataset 오브젝트로 변경합니다.
- ⑧ 원래 있던 Grid 컴포넌트의 oncellclick 이벤트 함수를 아래와 같이 작성합니다.

```
this.Grid00_onclick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    if(obj.getBindDataset().getColumn(e.row, "Column0" ) == '1')
    {
        var nRowSize = obj.getRealRowSize(0);
        var nPos = obj.getTreeRow(e.row) * nRowSize;
        trace(e.row);
        trace(obj.getTreeRow(e.row));
        obj.vscrollbar.set_pos(nPos);
    }
};
```

- ⑨ QuickView(Ctrl + F6)를 실행하여 트리 구조의 아이템을 클릭해서 두 개 Grid 컴포넌트의 스크롤 위치를 비교해봅니다.

25.32 Grid 컴포넌트로 달력 형태의 일정표 만들기

일정이 있는 날짜를 달력에 표시하고 해당 날짜 선택 시 상세 일정을 확인할 수 있는 일정표를 만드는 예제입니다. Grid 컴포넌트로 달력을 표현하고 ListBox 컴포넌트에 선택한 날짜의 상세 일정을 표시합니다.

예제에서는 2020년 9월과 10월 일정을 Dataset 오브젝트에 직접 추가했습니다.

25.32.1 예제

일정이 있는 날짜에는 밑줄이 표시되고 해당 날짜를 클릭하면 상세 일정 목록을 표시합니다.

<<	<	2020	>	9	>	>>
Sun	Mon	Tue	Wed	Thu	Fri	Sat
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30			

Everton FC vs West Bromwich Albion

Leeds United vs Fulham FC

Leicester City vs Burnley FC

Manchester United vs Crystal Palace



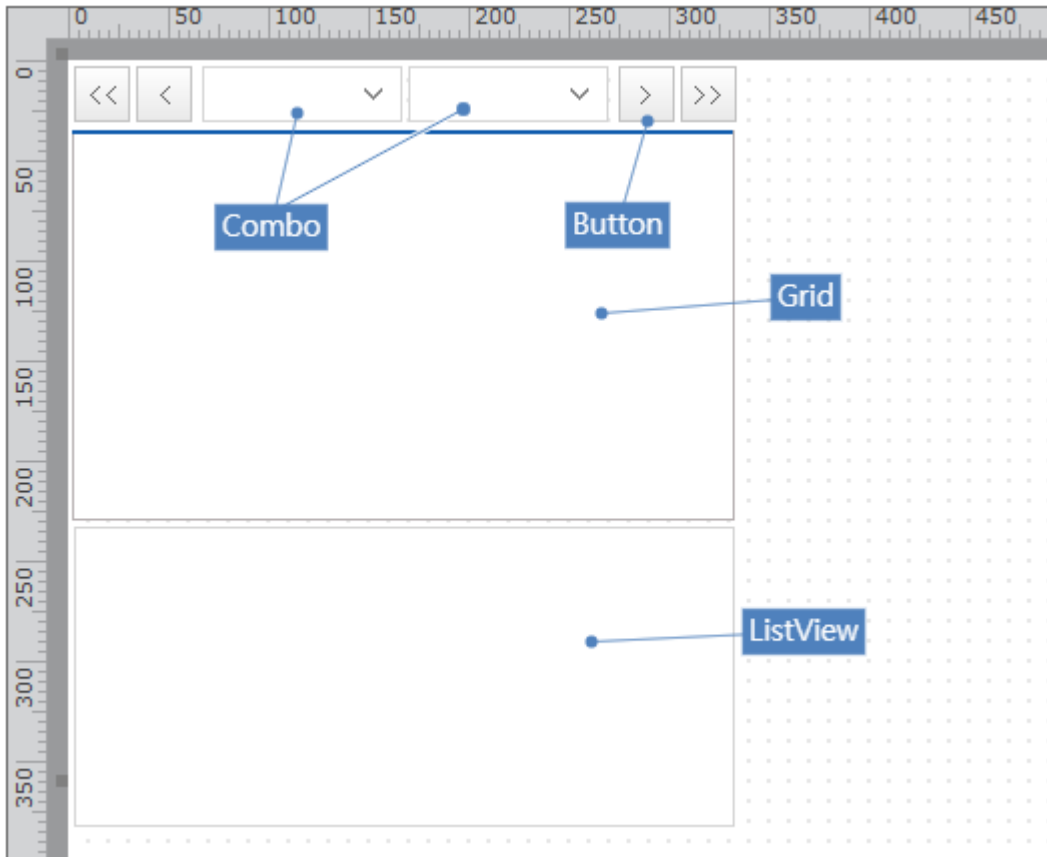
25.32.2 예제에서 사용한 핵심 기능

getCaseCountNF

조건표현식에 만족하는 Row 전체 갯수를 반환합니다. 예제에서는 해당 날짜에 해당하는 일정이 데이터에 있는지 확인하기 위한 용도로 사용했습니다. ListBox 컴포넌트에 데이터를 보여주기 위해 필터링을 한 상태라 getCaseCount 메소드가 아닌 getCaseCountNF 메소드를 사용했습니다.

25.32.3 예제 구현 방법

- 1 화면에 아래와 같이 컴포넌트를 배치합니다.



- ② Dataset 오브젝트를 추가하고 아래와 같이 데이터를 설정합니다.

7개의 컬럼은 일요일부터 토요일까지를 처리합니다. Row는 5개를 추가해 7*5 형태의 Grid 컴포넌트를 표현할 수 있도록 합니다.

Columns					
No	id	type	size	prop	su
1	cal0	STRING	256		
2	cal1	STRING	256		
3	cal2	STRING	256		
4	cal3	STRING	256		
5	cal4	STRING	256		
6	cal5	STRING	256		
7	cal6	STRING	256		

Rows			
No	cal0	cal1	cal2
1	[Undefined]	[Undefined]	[Undefined]
2	[Undefined]	[Undefined]	[Undefined]
3	[Undefined]	[Undefined]	[Undefined]
4	[Undefined]	[Undefined]	[Undefined]
5	[Undefined]	[Undefined]	[Undefined]

- ③ 데이터가 입력되었다면 Grid 컴포넌트와 바인딩해줍니다.
- ④ Grid 컴포넌트를 더블 클릭하고 Grid Contents Editor를 실행합니다.
- ⑤ head 밴드의 셀을 선택하고 요일명으로 text 속성값을 변경합니다.

		body		
		col0	col1	col2
head	row0	Sun	Mon	Tue
	row1			

- ⑥ Body 밴드의 셀을 선택하고 displaytype 속성값을 "decoratetext"로 변경합니다.
- ⑦ 각 요일별 expr 속성값을 아래와 같이 변경합니다.

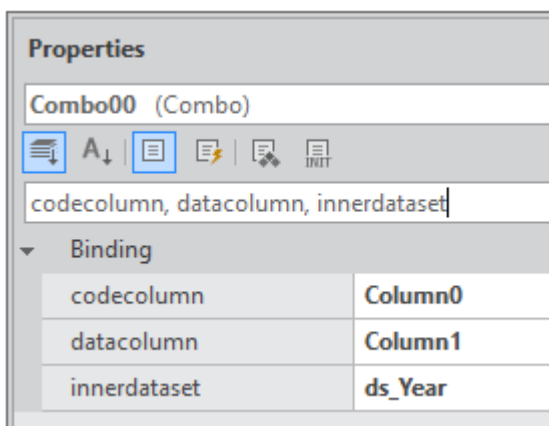
일요일과 토요일은 빨간색과 파란색으로 표시하고 일정이 있는 날짜는 fn_checkTable 함수에서 체크해서 밑줄을 표시합니다.

```
Sun: comp.parent.fn_checkTable(cal0)>0?"<fc v='red'><u v='true'>"+cal0+"</u></fc>":cal0?"<fc v='red'>"+cal0+"</fc>":cal0
Mon~Fri: comp.parent.fn_checkTable(cal1)>0?"<u v='true'>"+cal1+"</u>":cal1
Sat: comp.parent.fn_checkTable(cal6)>0?"<fc v='blue'><u v='true'>"+cal6+"</u></fc>":cal6?"<fc v='blue'>"+cal6+"</fc>":cal6
```

```
this.fn_checkTable = function(strDay)
{
    return this.Dataset00.getCaseCountNF("date == '"+this.fn_getDateString(strDay)+"'");
}

this.fn_getDateString = function(strDay)
{
    return new nexacro.Date(this.Combo00.value ,this.Combo01.value ,strDay).toString();
}
```

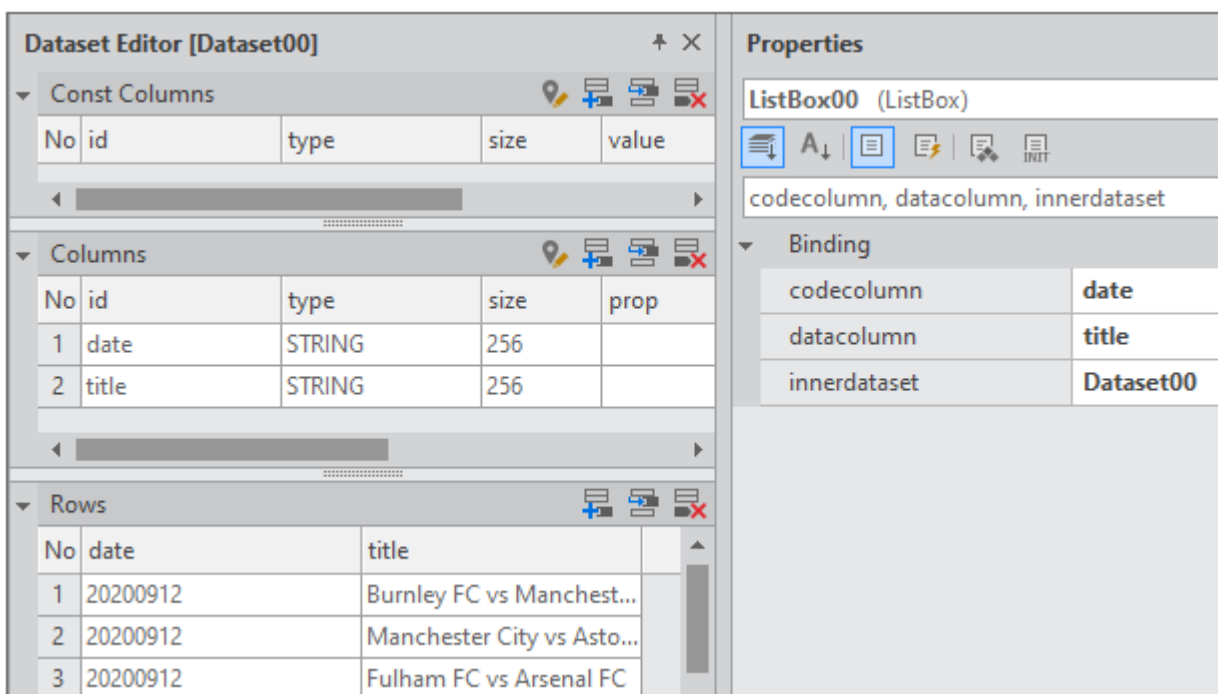
- ⑧ 2개의 컬럼을 가지는 Dataset 오브젝트를 2개 추가합니다.
- ⑨ Combo 컴포넌트에 2개의 컬럼을 가지는 Dataset 오브젝트를 innerdataset으로 바인딩합니다.



- ⑩ date, title 컬럼을 가지는 Dataset 오브젝트를 추가합니다.

데이터는 date 컬럼에 "YYYYMMDD" 형태로 날짜를 입력하고 title 컬럼에는 ListBox 컴포넌트에 표시할 텍스트를 입력합니다.

- ⑪ ListBox 컴포넌트에 Dataset 오브젝트를 innerdataset으로 바인딩합니다.



- ⑫ Form 오브젝트의 onload 이벤트를 함수를 아래와 같이 추가합니다.

예제에서는 2020년 9월과 10월 데이터만 만들어놓아서 2020년 9월 1일로 날짜를 설정했습니다. 그리고 해당 년도와 월을 기준으로 Combo 컴포넌트에 표시될 데이터를 설정하고 Grid 달력에 표시할 날짜를 처리합니다.

```

this.sample_grid_103_onload = function(obj:nexacro.Form,e:nexacro.LoadEventInfo)
{
    var date = new nexacro.Date(2020,8,1);
    this.fn_setYear(date.getFullYear());
    this.fn_setMonth(date.getMonth());
    this.fn_setCalendar();
};

```

- ⑬ 년도를 표시하는 Dataset 오브젝트의 데이터를 처리하는 fn_setYear 함수를 아래와 같이 추가합니다.

2020년을 기준으로 이전 100년(1920년), 이후 100년(2120년)을 Dataset 오브젝트에 추가하고 Combo 컴포넌트에 보여질 수 있도록 합니다.

```

this.minYear = -100;
this.maxYear = 100;

this.fn_setYear = function(numCurrentYear)
{
    this.ds_Year.set_enableevent(false);
    for(var i=numCurrentYear+this.minYear ; i<=numCurrentYear+this.maxYear ; i++)
    {
        var nRow = this.ds_Year.addRow();
        this.ds_Year.setColumn(nRow, 0, i);
        this.ds_Year.setColumn(nRow, 1, i);
    }
    this.ds_Year.set_enableevent(true);
    this.Combo00.set_value(numCurrentYear);
}

```

- ⑭ Combo 컴포넌트의 type 속성값은 "search"로 변경합니다.

- ⑮ 년도를 표시할 Combo 컴포넌트의 displayrowcount 속성값을 5로 설정합니다.

- ⑯ 월을 표시하는 Dataset 오브젝트의 데이터를 처리하는 fn_setMonth 함수를 아래와 같이 추가합니다.

```

this.fn_setMonth = function(numCurrentMonth)
{
    this.ds_Month.set_enableevent(false);
    for(var i=0 ; i<12 ; i++)
    {

```

```

        var nRow = this.ds_Month.addRow();
        this.ds_Month.setColumn(nRow, 0, i);
        this.ds_Month.setColumn(nRow, 1, i+1);
    }
    this.ds_Month.set_enableevent(true);
    this.Combo01.set_value(numCurrentMonth);
}

```

- ①7 Combo 컴포넌트의 onitemchanged 이벤트 함수를 아래와 같이 추가합니다.

년도나 월 선택값을 변경하면 Grid 컴포넌트에 표시하는 날짜를 업데이트합니다.

```

this.Combo_onitemchanged = function(obj:Combo, e:nexacro.ItemChangeEventInfo)
{
    this.fn_setCalendar();
}

```

- ①8 Grid 컴포넌트에 날짜를 처리하는 fn_setCalendar 함수를 아래와 같이 추가합니다.

선택한 년도와 월에 해당하는 날짜값을 배열로 생성하고 해당 배열값 순서에 따라 Grid 컴포넌트에 바인딩된 Dataset 오브젝트에 데이터를 설정합니다.

```

this.fn_setCalendar = function()
{
    var y = nexacro.toNumber(this.Combo00.value);
    var m = nexacro.toNumber(this.Combo01.value);

    var arr = this.getCalendarArr(y,m);

    this.ds_calendar.set_enableevent(false);
    this.ds_calendar.clearData();
    for(var i = 0 ; i < arr.length ; i++){
        this.ds_calendar.addRow();
        for(var y = 0 ; y < arr[i].length; y++)
        {
            if(arr[i][y] != null){ this.ds_calendar.setColumn(i,y, arr[i][y]); }
        }
    }
    this.ds_calendar.set_enableevent(true);
    this.Dataset00.filter("date == '00000000'");
    this.Grid00.clearSelect();
}

```

```

}

this.getCalendarArr = function(y,m,d)
{
    var date = new nexacro.Date();
    date.setFullYear(y, m , 1);
    return this.get_DateArr(date, new Array(), m);
}

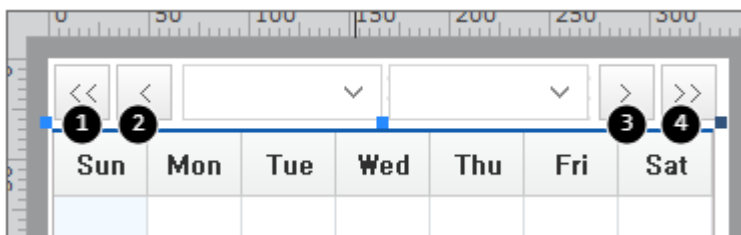
this.get_DateArr = function(date, arr , month)
{
    if(month != date.getMonth())
        return arr;

    var idx = date.getDay();
    if(arr.length == 0 ) {
        arr[0] = new Array();
    } else {
        if(idx == 0) {
            arr[arr.length] = new Array();
        }
    }
    arr[arr.length-1][idx] = date.getDate();
    date.addDate(1);
    return this.get_DateArr(date, arr, month);
}

```

- 19 상단에 배치한 Button 컴포넌트에 User Properties를 추가합니다.

User Properties를 추가해 onclick 이벤트를 하나로 처리합니다.



User Properties	1	2	3	4
dataType	year	month	month	year
dataValue	-1	-1	1	1

- 20 Button 컴포넌트의 onclick 이벤트 함수를 아래와 같이 추가합니다.

버튼 클릭 시 년도와 월을 변경합니다.

```
this.Button_onclick = function(obj:Button, e:nexacro.ClickEventInfo)
{
    if(obj.dataType == "month")
    {
        var numMonth = nexacro.toNumber(this.Combo01.value)+nexacro.toNumber(obj.dataValue);
        var value = (numMonth%12)!=0 ? (numMonth%12) : 0;
        trace(value);
        this.Combo01.set_value(value);
    }
    else if(obj.dataType == "year")
    {
        var numYear = nexacro.toNumber(this.Combo00.value)+nexacro.toNumber(obj.dataValue);
        this.Combo00.set_value(numYear);
    }
    this.fn_setCalendar();
}
```

- 21 Grid 컴포넌트의 oncellick 이벤트 함수를 아래와 같이 추가합니다.

선택한 날짜에 따라 ListBox 컴포넌트에 바인딩된 Dataset 오브젝트의 데이터를 필터링합니다.

```
this.Grid00_oncellick = function(obj:Grid, e:nexacro.GridClickEventInfo)
{
    this.Dataset00.filter("date == '"+this.fn_getDateString(this.ds_calendar.getColumn(e.row,e.cell))+"'");
}
```

- 22 QuickView(Ctrl + F6)를 실행하여 2020년 9월 달력이 표시되는지 확인하고 밑줄로 표시된 날짜의 상세 데이터를 확인합니다.

26.

GroupBox

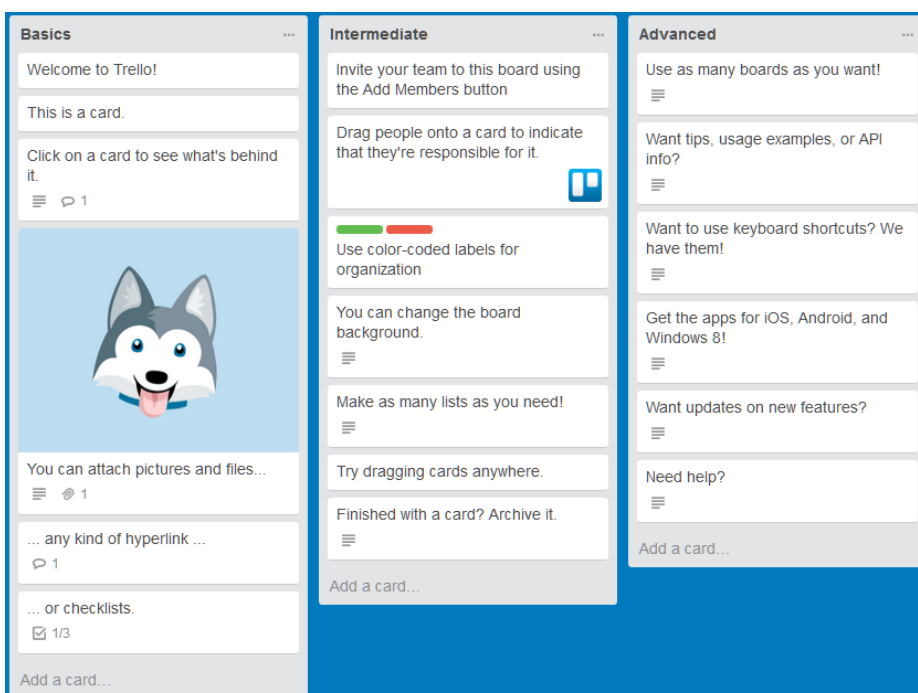
26.1 GroupBox 소개

GroupBox는 시각적으로 영역을 구분하는 데 사용하는 컴포넌트입니다. Div 컴포넌트처럼 그 하위에 UI 요소들을 갖는 것이 아니라 단지 GroupBox 영역에 있는 UI 요소들이 하나의 그룹인 것처럼 보이는 효과를 줍니다.

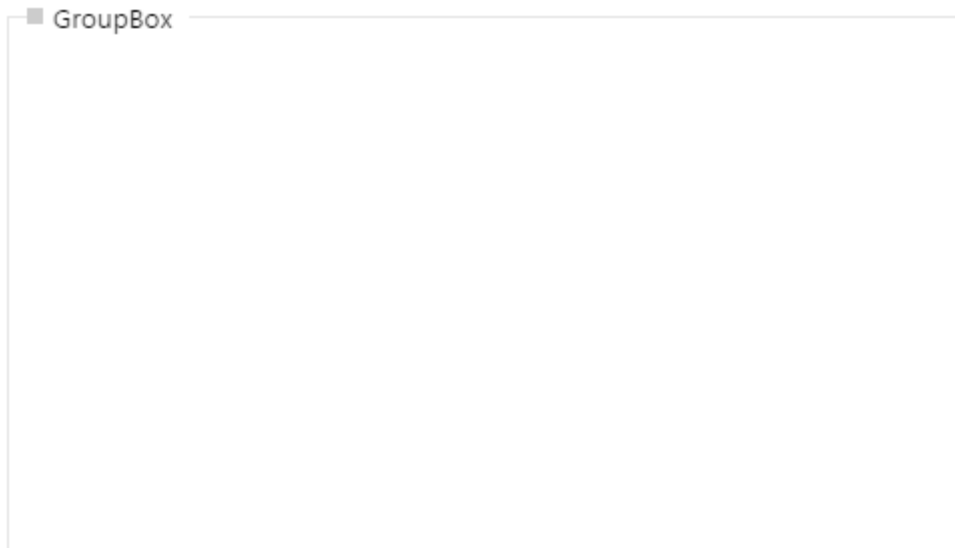
GroupBox는 테두리 영역에 타이틀을 출력할 수 있는데 그룹 지어진 UI 요소들의 목적을 타이틀로 표현함으로써 그 의미를 명확히 합니다.

26.1.1 사용 예

다음은 Trello라는 일정관리 애플리케이션의 화면입니다. Basics, Intermediate, Advanced라는 타이틀을 갖는 세 영역으로 나누고 관련된 UI 요소들을 배치하고 있습니다. 이런 경우 그룹박스를 사용하면 각 영역 간 구분을 손쉽게 구현할 수 있습니다.



26.1.2 GroupBox 만들기



1 GroupBox 컴포넌트 생성하기

툴바에서 GroupBox  를 선택한 후, Form 위에 적당한 크기로 드래그하여 생성합니다. 컴포넌트를 선택 한 상태에서 폼 위에 클릭하면 기본 크기로 생성됩니다.

Properties 창에서 text 속성을 "GroupBox"로 설정합니다.

2 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행합니다. GroupBox는 시각적인 효과를 주는데 목적이 있는 컴포넌트라 어떤 동작을 수행하지 않습니다.

26.2 UI 요소 그룹짓기

GroupBox는 화면의 여러 UI 요소들을 하나의 그룹처럼 보이도록 합니다. 실제로 UI 요소들과 연계되진 않고 시각적인 효과만을 줍니다.

26.2.1 예제

다음은 여러 UI 요소들을 하나의 그룹처럼 보이기 위해 GroupBox를 사용해 감싼 모습을 보여줍니다. GroupBox는 UI 요소들에 어떤 영향도 주지 않습니다.



26.2.2 예제에서 사용한 핵심 기능

text

GroupBox의 타이틀을 설정하는 속성입니다.

26.2.3 예제 구현 방법

① GroupBox 컴포넌트 생성하기

툴바에서 GroupBox  컴포넌트를 예제의 그림과 같이 적절히 배치합니다. 컴포넌트를 선택 한 상태에서 폼 위에 클릭하면 기본 크기로 생성됩니다.

② 타이틀 설정하기

속성	값
text	GroupBox

3 UI 요소(컴포넌트) 배치하기

예제와 같이 적절히 컴포넌트를 배치합니다.

4 QuickView로 확인하기

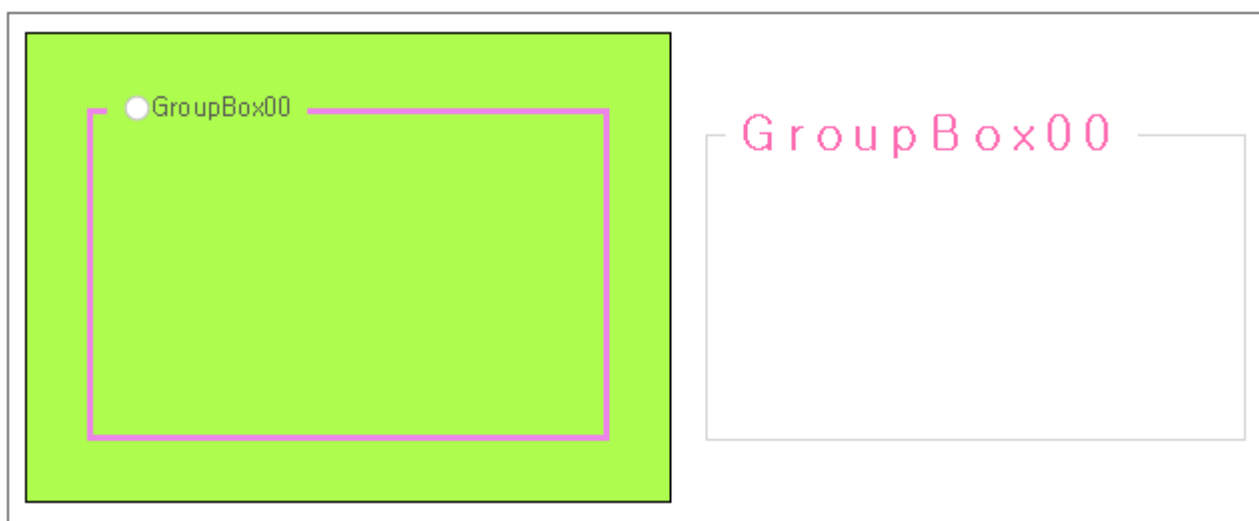
QuickView(Ctrl + F6)로 실행합니다. 타이틀이 설정된 GroupBox와 각 컴포넌트가 정상적으로 보이는지 확인합니다.

26.3 GroupBox 스타일 적용하기

GroupBox 컴포넌트는 가상의 사각형 영역 내에 박스 형태의 groupboxcontents가 배치되어 있고 텍스트를 표시하는 groupboxtitle가 배치되어 있는 구조입니다. 때문에 눈에 보이는 스타일을 지정하기 위해서는 groupboxcontents 또는 groupboxtitle의 스타일을 지정해주어야 합니다.

26.3.1 예제

왼쪽에 배치된 GroupBox는 스타일을 적용한 상태이고 오른쪽은 기본 상태에서 font 속성만 수정한 GroupBox를 표시합니다.



26.3.2 예제에서 사용한 핵심 기능

font

GroupBox 컴포넌트의 font 속성 설정 시에는 컴포넌트 하위의 groupboxtitle에도 적용됩니다. 때문에 컴포넌트에서 font 속성 변경 시 groupboxtitle의 텍스트에 반영됩니다.

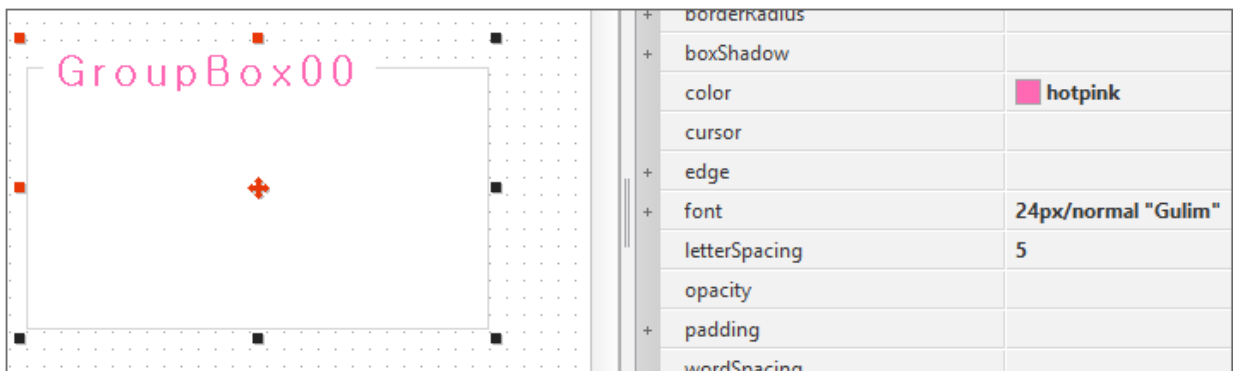
26.3.3 예제 구현 방법

1 GroupBox 컴포넌트 생성하기

화면에 GroupBox 컴포넌트 2개를 배치합니다. 왼쪽에 배치하는 GroupBox 컴포넌트는 Div 컴포넌트를 먼저 배치하고 그 안에 GroupBox 컴포넌트를 배치합니다. GroupBox 컴포넌트는 박스 형태로 구성되어있지만 타이틀을 표시하는 텍스트가 박스 영역과 바깥 영역 중간에 위치하기 때문에 보통은 Form과 박스영역의 배경색을 같게 하고 텍스트 영역의 배경색은 투명으로 처리합니다.

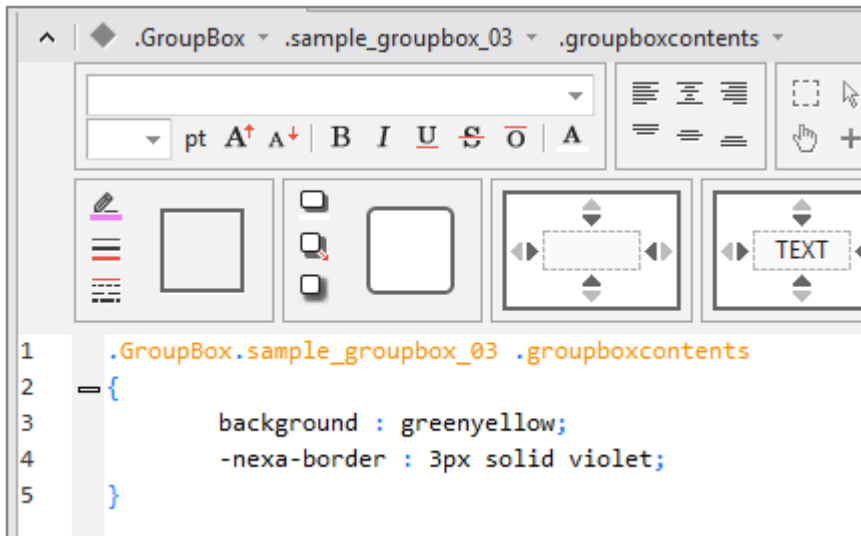
2 스타일 속성 지정하기

왼쪽 GroupBox 컴포넌트의 cssclass 속성값을 "sample_groupbox_03"로 지정합니다. 오른쪽 GroupBox 컴포넌트는 Font 관련 속성값을 적절하게 수정합니다.



3 xcss 파일 편집하기

xcss 파일을 열어서 아래 코드를 추가합니다. Advanced 탭에서 선택자를 추가하거나 Text 탭에서 직접 코드 값을 입력할 수 있습니다.



```
.GroupBox.sample_groupbox_03 .groupboxcontents
{
    background : greenyellow;
    -nexa-border : 3px solid violet;
}

.GroupBox.sample_groupbox_03 .groupboxtitle
{
    background : greenyellow;
    -nexa-icon : URL("theme://images/rdo_WF_Radio.png");
}
```

④ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행합니다. 지정된 스타일이 처리되었는지 확인합니다.

27.

ImageViewer

27.1 ImageViewer 소개

ImageViewer 컴포넌트는 이미지 콘텐츠를 화면에 보이도록 처리합니다. 웹에서 이미지를 처리하는 것은 이미지를 보여주고 정렬하는 수준이라면 ImageViewer 컴포넌트는 해당 영역에 텍스트를 추가하고 편집할 수 있습니다.

27.1.1 사용 예

이미지는 단독으로 쓰이기도 하지만 주로 텍스트를 보조하기 위한 수단으로 사용됩니다. 텍스트를 사용하는 단락과 어울리게 이미지를 배치하고 활용하는 방법이 필요합니다. 또한, 이미지를 적당한 크기로 배치하는 것도 중요합니다.



“Mr. Dylan, I have just one question,”
Archy Techt from New York [wrote](#). “How
does it feel?”

“I just walked over to Carl Schurz Park,
and I swear I saw Einstein there,
disguised as Robin Hood, with his
memories in a trunk. He broke out his
electric violin and we all celebrated this
great news,” Bill from Beijing
[commented](#).

image source: <http://www.nytimes.com/2016/10/15>

“Doesn’t take a weatherman to know

27.1.2 ImageViewer 만들기

ImageViewer 컴포넌트는 이미지를 담기 위한 용도로 사용합니다. 여기에 텍스트를 추가해서 이미지를 설명할 수 있으며 경계선을 추가해서 액자와 같은 효과를 줄 수 있습니다.

1 ImageViewer 컴포넌트 생성하기

툴바에서 ImageViewer 항목을 선택한 후, Form 위에 드래그 앤 드롭해 생성합니다. 빈 흰색 상자가 화면에 표시됩니다.

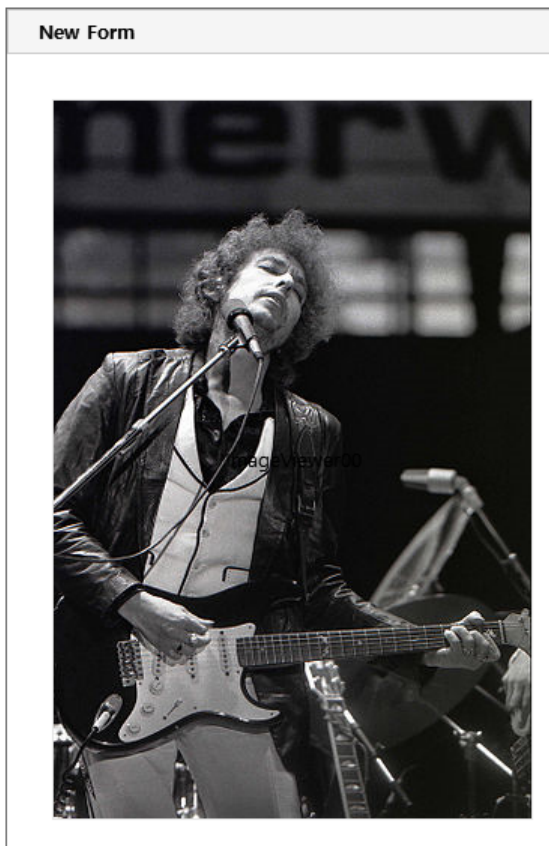
2 image 속성값 설정하기

image 속성값에는 이미지 파일이 있는 URL을 지정할 수 있습니다. 공개된 이미지 파일 중 아래 이미지 주소를 입력합니다. 이미지 파일 크기가 320x480이므로 ImageViewer 컴포넌트의 width, height 속성값도 320, 480으로 지정합니다.

```
https://upload.wikimedia.org/wikipedia/commons/thumb/a/a7/Bob_Dylan_1978.jpg/320px-Bob_Dylan_1978.jpg
```

3 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행합니다. 화면에 지정된 이미지가 나타나는 것을 확인할 수 있습니다.





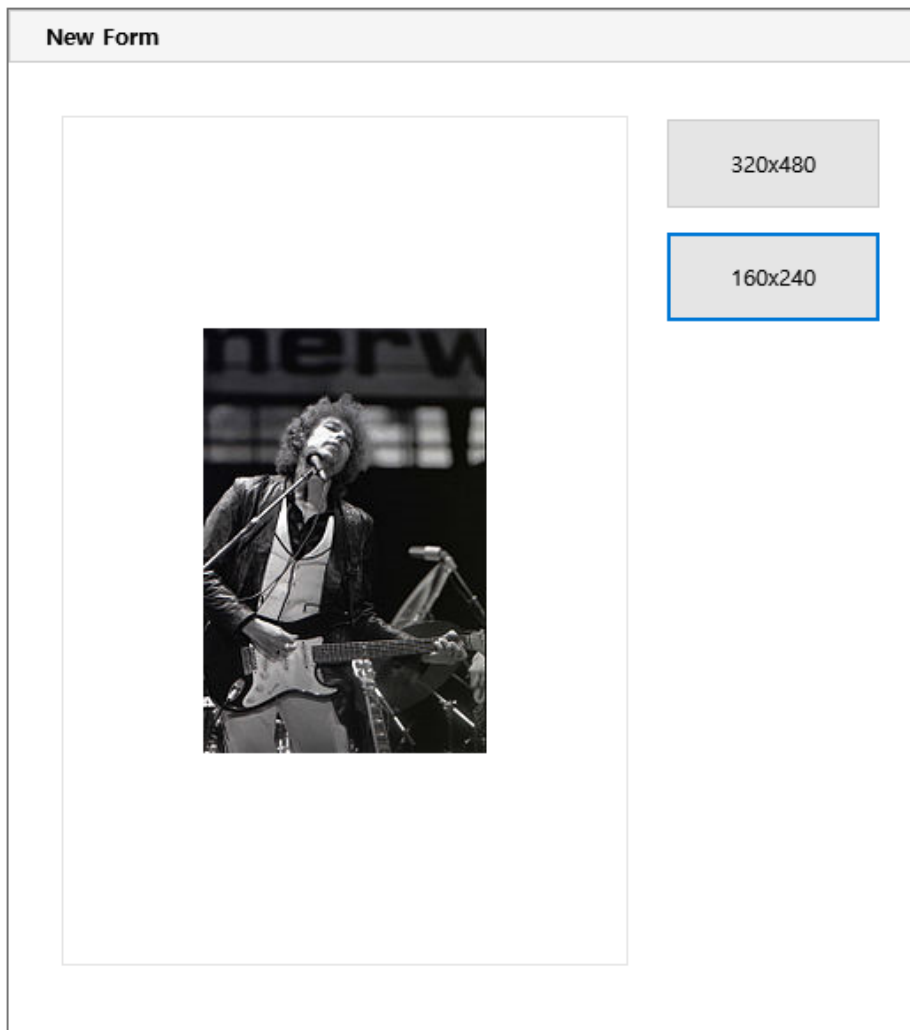
ImageViewer 컴포넌트를 생성하면 text 속성값이 default 값으로 설정됩니다. text가 이미지 위에 보이지 않게 하려면 해당 속성값을 지워야 합니다.

27.2 이미지 폴더에 있는 이미지 보여주기

애플리케이션에서 자주 사용하는 이미지는 이미지 폴더에 따로 저장해놓고 필요한 이미지를 사용할 수 있습니다. 넥사크로 스튜디오에서는 서비스 폴더를 추가해서 이미지 폴더에 손쉽게 접근할 수 있습니다.

27.2.1 예제

버튼 클릭 시 지정된 이미지를 로드해 화면에 표시합니다. 이미지 파일은 미리 올려놓은 파일을 사용합니다.





27.2.2 예제에서 사용한 핵심 기능

image

표시될 이미지 파일의 경로를 지정합니다. 이미지를 지정하는 방식은 여러 방식을 사용할 수 있습니다. 외부 경로에 있는 파일이라면 URL을 직접 지정하고 애플리케이션과 같은 프로젝트에 포함된 경우라면 TypeDefinition에 지정된 이미지를 사용합니다. 이번 예제에서는 TypeDefinition에 지정된 이미지를 사용합니다.

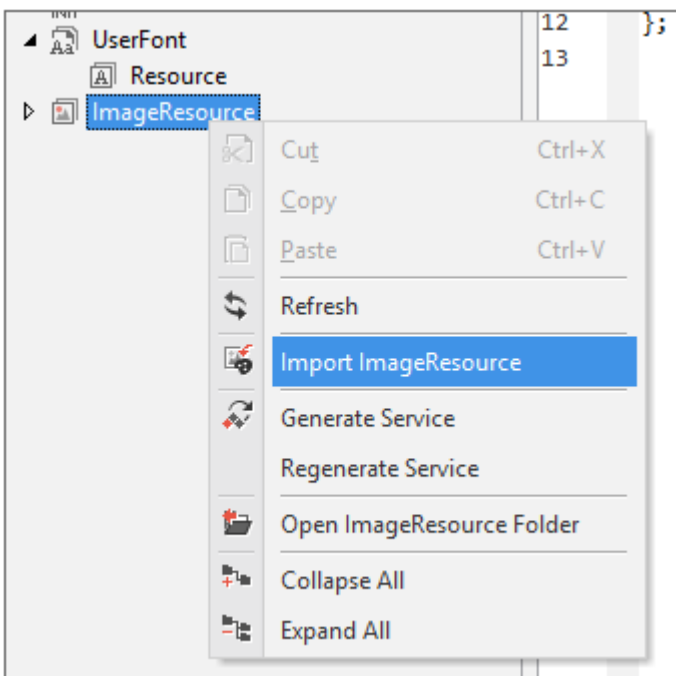
27.2.3 예제 구현 방법

① 화면 구성하기

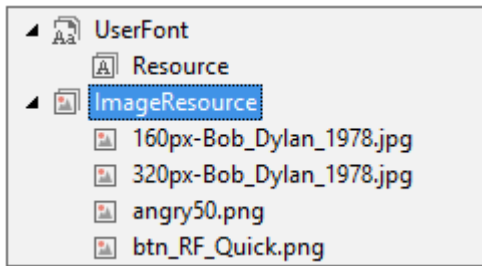
ImageViewer, Button 컴포넌트를 예제 화면과 같이 배치합니다. ImageViewer 컴포넌트의 width, height 속성값은 320, 480으로 지정합니다. 그리고 화면에 text가 표시되지 않도록 default 속성값은 삭제합니다.

② ImageResource 추가하기

Resource Explorer에서 ImageResource 항목을 선택하고 컨텍스트 메뉴에서 [Import ImageResource] 항목을 선택합니다. 파일 대화 상자를 열어 이미지 파일을 선택하면 ImageResource를 추가할 수 있습니다.



추가된 이미지 파일은 Resource Explorer에서 확인할 수 있습니다.



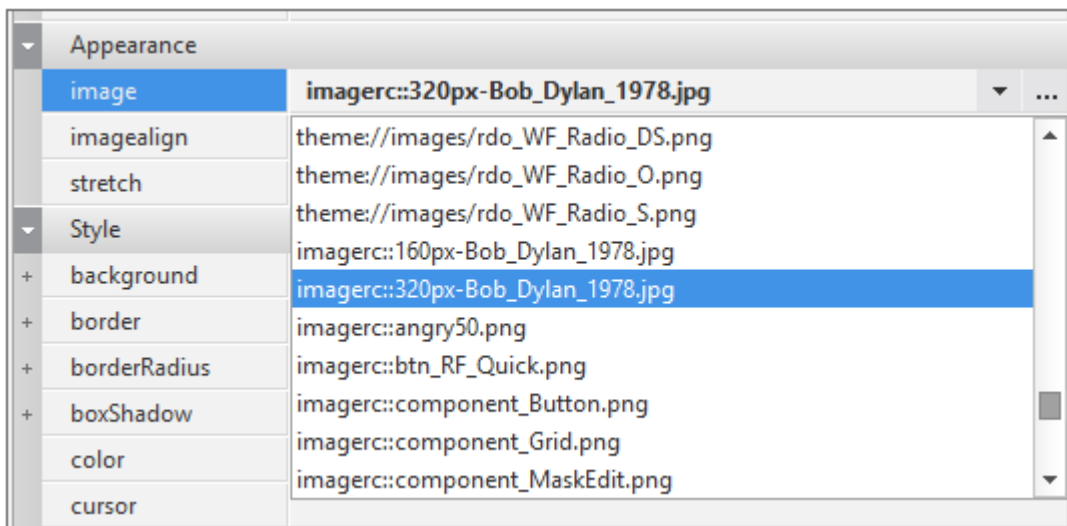
참고한 이미지 파일 URL은 아래와 같습니다.

https://upload.wikimedia.org/wikipedia/commons/thumb/a/a7/Bob_Dylan_1978.jpg/160px-Bob_Dylan_1978.jpg

https://upload.wikimedia.org/wikipedia/commons/thumb/a/a7/Bob_Dylan_1978.jpg/320px-Bob_Dylan_1978.jpg

3 Image 속성값 지정하기

애플리케이션이 실행됐을 때 보여줄 이미지를 지정합니다. ImageViewer 컴포넌트를 선택하고 속성창에서 image 속성값 항목 옆에 있는 목록을 펼쳐보면 앞 단계에서 ImageResource로 등록된 이미지 파일을 확인할 수 있습니다.



4 Button 컴포넌트 이벤트 함수 작성하기

Button 컴포넌트를 선택한 후 onclick 이벤트 함수를 작성합니다. 각 버튼에 지정된 이미지 파일을 image 속성값으로 지정합니다.

```
this.btn320_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.imageviewerMain.set_image("imagerc::320px-Bob_Dylan_1978.jpg");
};
```

```

this.btn160_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.imageviewerMain.set_image("imagerc::160px-Bob_Dylan_1978.jpg");
};

```

5 QuickView로 확인하기

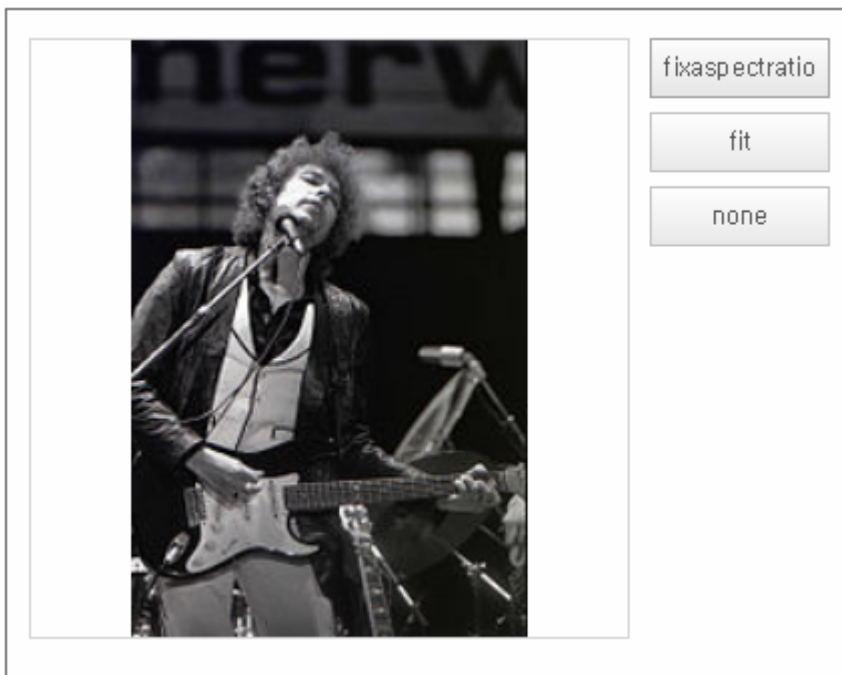
QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 이미지를 바꾸어가면서 확인해봅니다.

27.3 이미지 크기 조정하기

이미지를 화면에 표시할때 지정된 크기로 보여주어야 하는 경우 자동으로 크기를 조정하는 방식을 살펴봅니다.

27.3.1 예제

버튼 클릭 시 ImageViewer 컴포넌트의 크기에 따라 자동으로 이미지 크기를 조정합니다.



27.3.2 예제에서 사용한 핵심 기능

stretch

이미지 크기를 늘리거나 줄여주는 기능을 처리합니다. 3가지 파라미터를 지정할 수 있습니다. 속성값을 "fit"으로 지정하면 가로, 세로 크기를 ImageViewer 컴포넌트의 크기에 맞춥니다. 이 과정에서 이미지의 가로, 세로 비율이 변경되면 이미지가 일그러진 상태로 보일 수 있습니다. "fixaspectratio"으로 지정한 경우에는 가로, 세로 비율은 유지되지만 공백이 생길 수 있습니다.

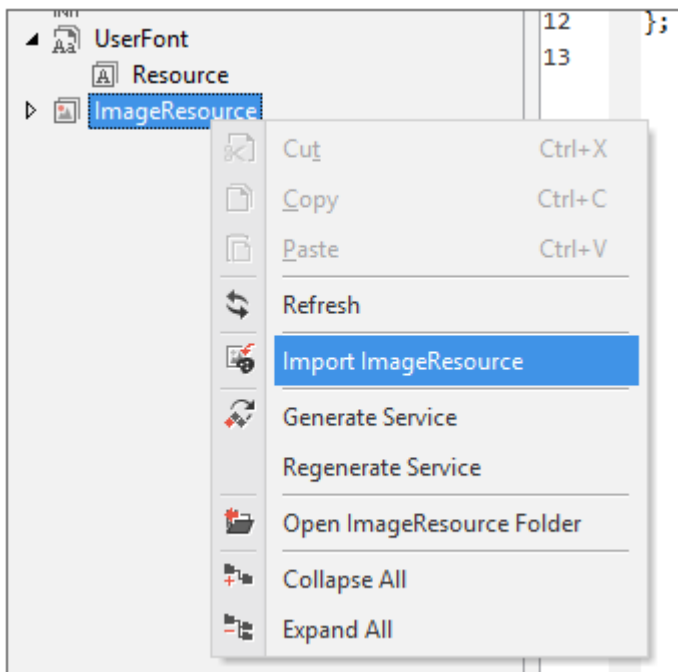
27.3.3 예제 구현 방법

① 화면 구성하기

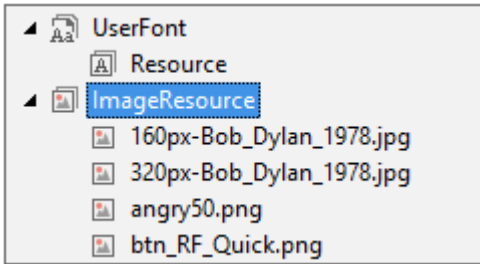
ImageViewer, Button 컴포넌트를 예제 화면과 같이 배치합니다. ImageViewer 컴포넌트의 width, height 속성값은 샘플에서 사용할 이미지 크기보다 살짝 크게 지정합니다.

② ImageResource 추가하기

Resource Explorer에서 ImageResource 항목을 선택하고 컨텍스트 메뉴에서 [Import ImageResource] 항목을 선택합니다. 파일 대화 상자를 열어 이미지 파일을 선택하면 ImageResource를 추가할 수 있습니다.



추가된 이미지 파일은 Resource Explorer에서 확인할 수 있습니다.

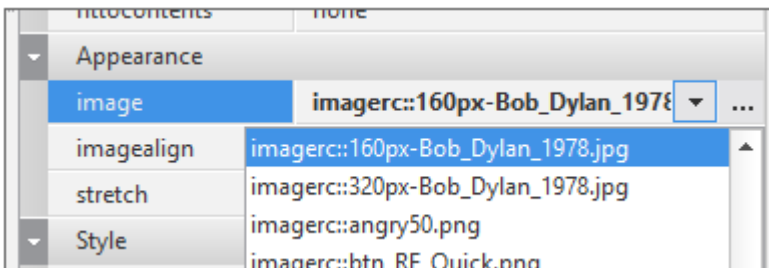


참고한 이미지 파일 URL은 아래와 같습니다.

https://upload.wikimedia.org/wikipedia/commons/thumb/a/a7/Bob_Dylan_1978.jpg/160px-Bob_Dylan_1978.jpg

3 Image 속성값 지정하기

애플리케이션이 실행됐을 때 보여줄 이미지를 지정합니다. ImageViewer 컴포넌트를 선택하고 속성창에서 image 속성값 항목 옆에 있는 목록을 펼쳐보면 앞 단계에서 ImageResource로 등록된 이미지 파일을 확인할 수 있습니다.



4 Button 컴포넌트 이벤트 함수 작성하기

Button 컴포넌트를 선택한 후 onclick 이벤트 함수를 작성합니다. 각 버튼에 지정된 stretch 속성값을 지정합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.imageviewerMain.set_stretch('fixaspectratio');
};

this.Button01 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.imageviewerMain.set_stretch('fit');
};

this.Button02 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
```

```
this.imageviewerMain.set_stretch('none');
};
```

5 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 이미지 크기를 변경해봅니다.

27.4 Base64로 인코딩된 이미지 보여주기

이미지 파일을 변환한 Base 64 코드값을 다시 이미지로 보여주는 방법을 살펴봅니다.

27.4.1 예제

Edit 컴포넌트에 Base 64 코드값을 입력하고 Button 컴포넌트를 클릭하면 아래에 변환된 이미지와 이미지 크기 정보를 보여줍니다.



이미지 파일을 변환해 Base 64 코드값을 제공하는 온라인 사이트를 활용해 테스트해볼 수 있습니다.
아래 주소에서 이미지 파일을 업로드하면 Base 64 코드값을 확인할 수 있습니다.
<https://www.base64-image.de/>

639 x 143

492	1066	5084	2262
Downloads	Hours Working	Lines Written	Another Number



sample_imageviewer_03.xfdl

27.4.2 예제에서 사용한 핵심 기능

imagewidth, imageheight

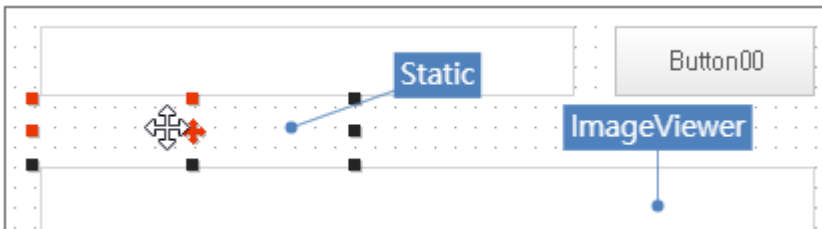
화면에 표시할 이미지의 너비, 높이값을 반환하거나 설정할 수 있습니다.



넥사크로 브라우저를 사용하는 경우에는 `system.saveToImageBase64String` 메소드를 사용해 화면에 보여지는 컴포넌트 또는 오브젝트를 Base64로 인코딩된 문자열을 얻을 수 있습니다.

27.4.3 예제 구현 방법

- ① Edit 컴포넌트와 Button 컴포넌트를 화면에 배치합니다.
- ② Edit 컴포넌트 아래에 Static 컴포넌트를 배치하고 그 아래에 ImageViewer 컴포넌트를 배치합니다. Static 컴포넌트에는 표시할 이미지의 크기 정보를 보여줄겁니다.



- ③ ImageViewer 컴포넌트의 `fittedContents` 속성값은 "both"로 지정합니다. ImageViewer 컴포넌트의 크기와 상관없이 불러오는 이미지 파일 크기를 기준으로 표시됩니다.
- ④ Button 컴포넌트에 `onclick` 이벤트를 아래와 같이 추가합니다. Edit 컴포넌트에 입력된 텍스트를 그대로 `image` 속성값으로 지정합니다.

```
this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.ImageViewer00.set_image(this.Edit00.value);
};
```

- ⑤ ImageViewer 컴포넌트에 `onload` 이벤트를 아래와 같이 추가합니다. 이미지를 불러오면서 크기 정보를 표시하고 `resetScroll` 메소드를 호출해 화면 상태를 갱신합니다.

```

this.ImageView00_onload = function(obj:nexacro.ImageView,e:nexacro.LoadEventInfo)
{
    this.Static00.set_text(this.ImageView00.imagewidth + " x " + this.ImageView00.
imageheight);
    this.resetScroll();
};

```

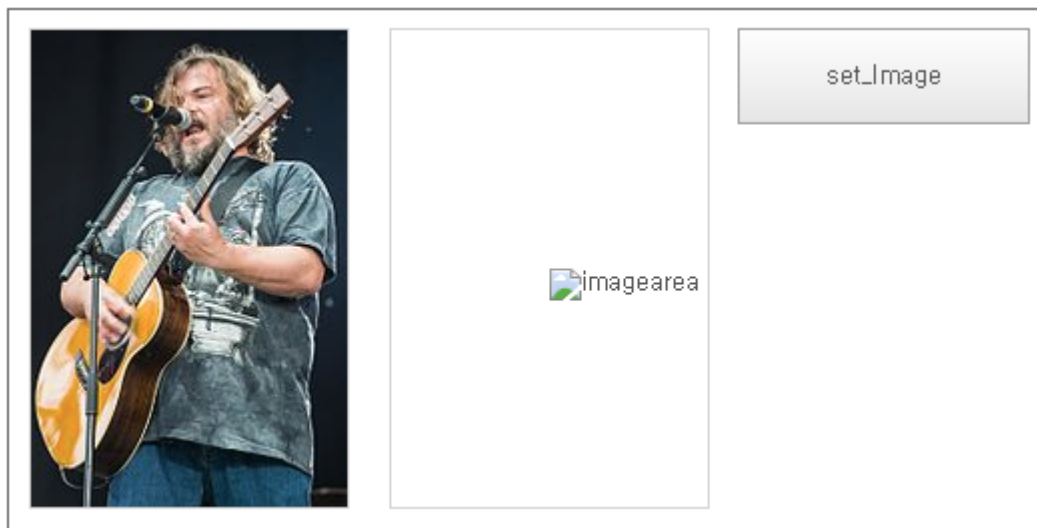
- ⑥ QuickView(Ctrl + F6)로 실행한 후 Base64 인코딩된 문자열을 입력하고 버튼을 클릭해 이미지를 확인합니다.

27.5 이미지를 찾지 못할 때 대체 이미지 보여주기

이미지 파일명이 잘못되었거나 서버에 문제가 있을 때 대체 이미지를 보여주는 방식입니다. 특정 이미지 파일에 접근하지 못하는 경우에 유용할 수 있습니다.

27.5.1 예제

버튼을 클릭하면 두 개의 ImageViewer 컴포넌트에 image 속성값을 접속할 수 없는 URL로 지정합니다. 하나는 대체 이미지를 표시하고 하나는 깨진 이미지를 표시합니다.



27.5.2 예제에서 사용한 핵심 기능

imagewidth, imageheight

화면에 표시할 이미지의 너비, 높이값을 반환합니다. 이미지를 가져오지 못한 경우에는 0을 반환합니다.

27.5.3 예제 구현 방법

- ① ImageViewer 컴포넌트와 Button 컴포넌트를 화면에 배치합니다.
- ② Button 컴포넌트에 onclick 이벤트를 아래와 같이 추가합니다. 실제 없는 URL 값을 image 속성값으로 지정합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.ImageView00.set_image("https://dummy/Jack_Black_2007.jpg");
    this.ImageView01.set_image("https://dummy/Jack_Black_2007.jpg");
};
```

- ③ 첫 번째 ImageViewer 컴포넌트에 onload 이벤트를 아래와 같이 추가합니다.

```
// https://commons.wikimedia.org/wiki/File:2016_RiP_Tenacious_D_-_Jack_Black_-_by_2eight_-_8SC8891.jpg
this.ImageView00 onload = function(obj:nexacro.ImageView,e:nexacro.LoadEventInfo)
{
    if(obj.imageheight == 0 && obj.imagewidth == 0)
    {
        this.ImageView00.set_image("https://upload.wikimedia.org/wikipedia/commons/thumb/4/48/2016_RiP_Tenacious_D_-_Jack_Black_-_by_2eight_-_8SC8891.jpg/160px-2016_RiP_Tenacious_D_-_Jack_Black_-_by_2eight_-_8SC8891.jpg");
    }
};
```

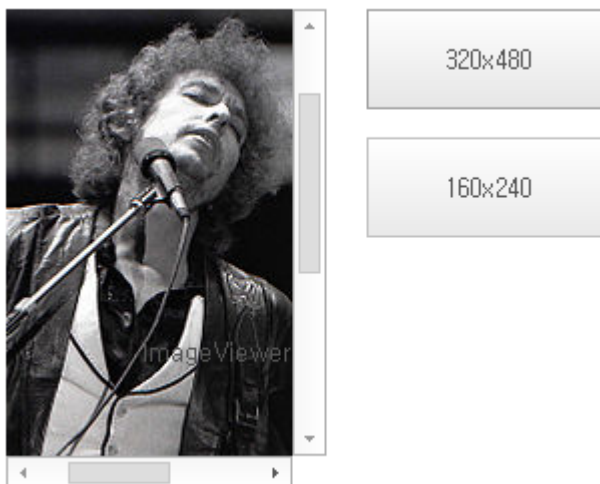
- ④ QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해서 이미지 표시 여부를 확인합니다.

27.6 이미지가 ImageViewer보다 클 경우 스크롤 처리하기

ImageViewer 컴포넌트는 자체 스크롤바를 가지고 있지 않아서 이미지 크기가 큰 경우에는 전체 이미지를 확인할 수 없습니다. ImageViewer 컴포넌트를 감싸는 Div 컴포넌트를 사용해 스크롤 기능을 처리하는 방법을 살펴봅니다.

27.6.1 예제

320x480 버튼 클릭 시 화면에 보여지는 ImageViewer보다 큰 크기의 이미지를 로딩하는데, 이때 스크롤바가 활성화되며 이미지 전체를 탐색할 수 있습니다.



 [sample_imageviewer_05.xfdl](#)

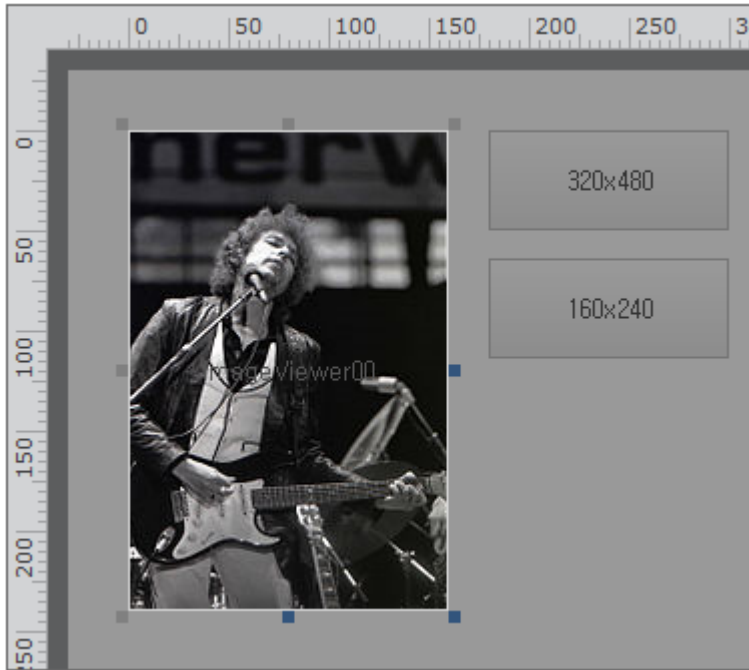
27.6.2 예제에서 사용한 핵심 기능

imagewidth, imageheight

이미지 로딩 시 화면에 표시할 이미지의 너비, 높이값을 반환합니다. 이미지 크기에 따라 ImageViewer 컴포넌트의 크기를 설정합니다.

27.6.3 예제 구현 방법

- 1 Div 컴포넌트를 배치하고 그 안에 ImageViewer 컴포넌트를 배치합니다. 그리고 Button 컴포넌트를 화면에 배치합니다.



- 2 Button 컴포넌트에 onclick 이벤트를 아래와 같이 추가합니다. 실제 없는 URL 값을 image 속성값으로 지정합니다.

```
this.btn320_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Div00.form.imageviewerMain.set_image("imagerc::320px-Bob_Dylan_1978.jpg");
};

this.btn160_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Div00.form.imageviewerMain.set_image("imagerc::160px-Bob_Dylan_1978.jpg");
};
```

- 3 ImageViewer 컴포넌트에 onload 이벤트를 아래와 같이 추가합니다.

이미지가 정상적으로 로딩되면 ImageViewer 컴포넌트의 높이와 너비를 이미지 크기에 맞게 설정합니다. 실제 표시되는 것은 ImageViewer 컴포넌트의 스크롤바가 아니라 Div 컴포넌트의 스크롤바가 보여집니다.

```
this.Div00_imageviewerMain_onload = function(obj:nexacro.ImageViewer,e:nexacro.  
LoadEventInfo)  
{  
    var nHeight = obj.imageheight;  
    var nWidth = obj.imagewidth;  
  
    obj.set_height(nHeight);  
    obj.set_width(nWidth);  
  
    this.Div00.form.resetScroll();  
};
```

- ④ QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해서 스크롤바 표시 여부를 확인합니다.

28.

ListBox

28.1 ListBox 소개

Listbox는 여러 아이템 목록을 나열해 놓고 그중 하나 혹은 여러 개의 아이템을 선택 및 출력하기 위한 용도로 사용하는 컴포넌트입니다.

Combo와 유사하나 아이템 목록이 항상 열려있다는 점, 여러 아이템을 선택할 수 있다는 점이 다릅니다. 여러 개의 아이템을 선택할 때에는 **Ctrl** 키 또는 **Shift** 키를 누른 상태에서 아이템을 선택합니다.

아이템 목록이 Listbox에 표시할 수 있는 개수보다 많아지면 자동으로 스크롤바가 생깁니다. 가능하면 스크롤바가 생기지 않도록 Listbox 크기를 충분히 조절하는 것이 사용자가 아이템 목록을 보고 선택하기에 좋습니다.



28.1.1 사용 예

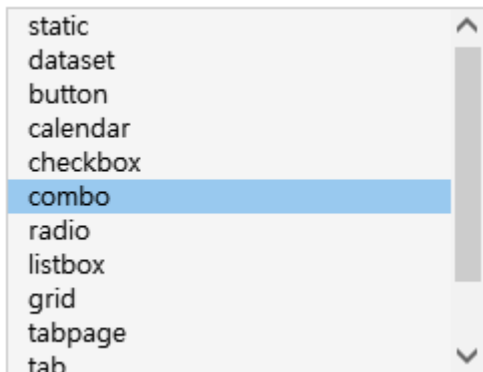
다음은 어느 웹 메일의 주소록 예시를 보여줍니다. 사용자는 화면에 나열된 주소 목록을 보며 메일을 보내려는 상대를 여럿 지정할 수 있습니다.

이렇게 나열된 목록을 보며 다수를 선택하는 기능이 필요한 경우에 Listbox는 매우 유용합니다.

☐	butterfly@altavista.com	butterfly@altavista.com	⋮
☐	gilaim@bluehouse.gov	gilaim@bluehouse.gov	⋮
☒	jane@gmail.com	jane@gmail.com	⋮
☒	john.doe@teramail.com	john.doe@teramail.com	⋮
☐	listbox@ui.com	listbox@ui.com	⋮
☐	pamela@yahoo.com	pamela@yahoo.com	⋮
☒	teamempire@hanmail.net	teamempire@hanmail.net	✎ ☆ ⋮
☐	zimmerman@mmail.com	zimmerman@mmail.com	⋮

28.1.2 ListBox 만들기

ListBox를 사용하려면 Dataset을 바인딩해 아이템 목록을 구성해야 합니다. 그래야 예제와 같이 아이템 목록이 나오며 사용자가 선택할 수 있는 환경이 만들어집니다.



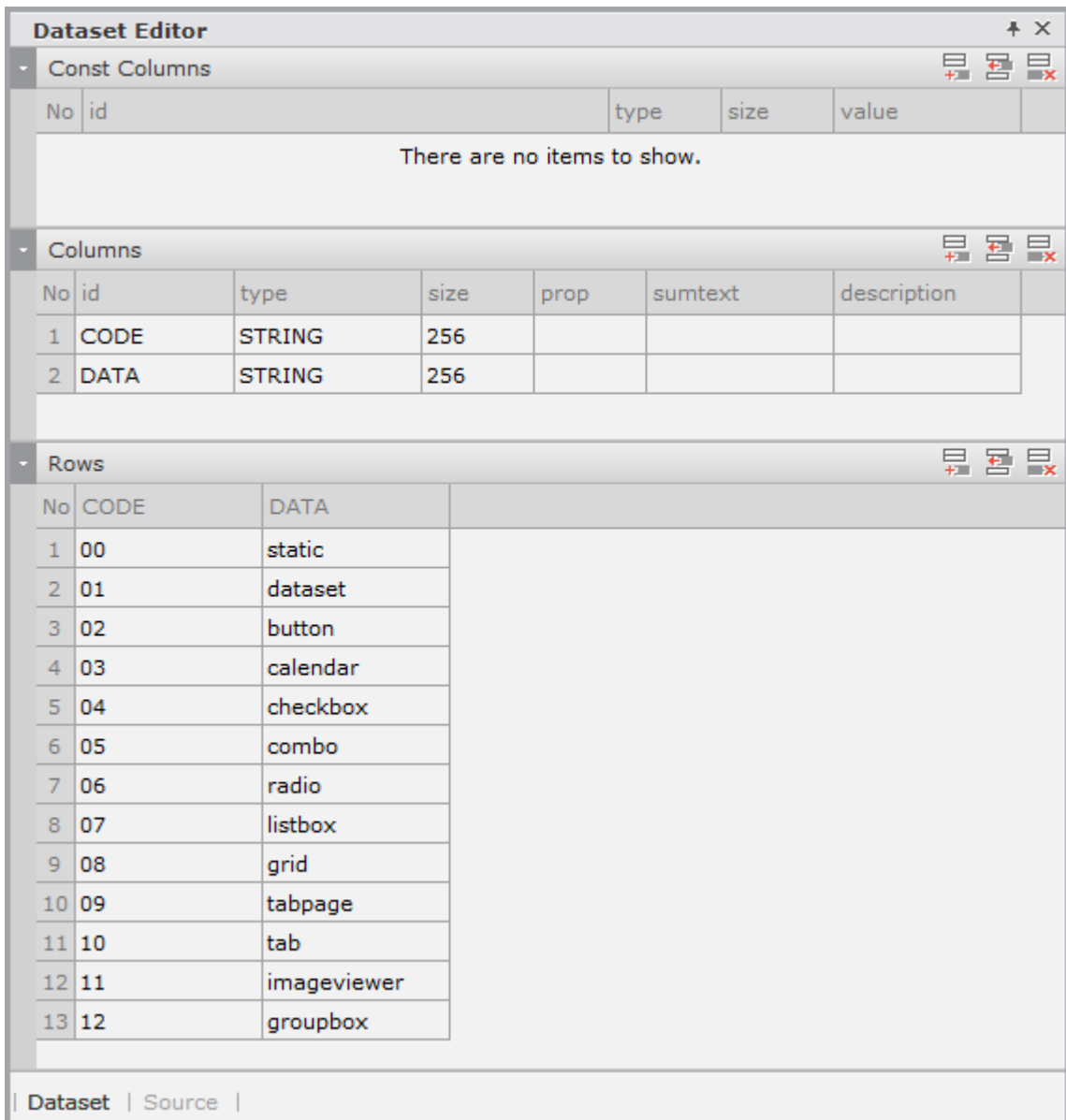
1 ListBox 생성하기

툴바에서 ListBox 를 선택한 후, 폼 위에서 적당한 크기로 드래그하여 생성합니다. 컴포넌트를 선택한 상태에서 폼 위에 클릭하면 기본 크기로 생성됩니다.

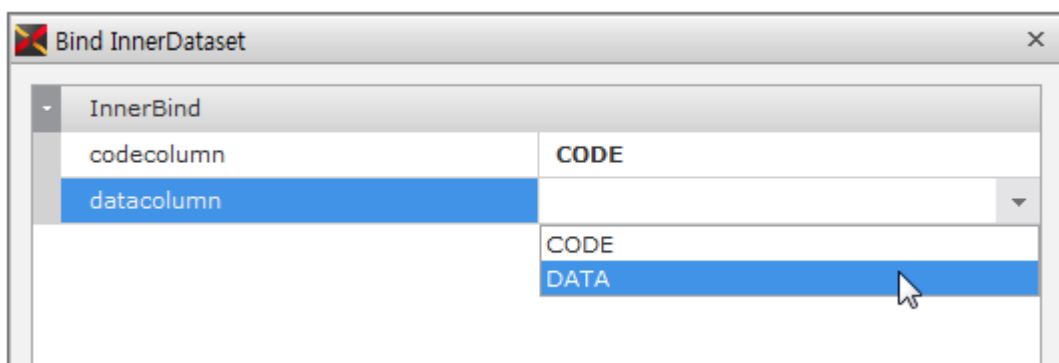
2 ListBox 아이템 목록 구성하기

Dataset으로 아이템 목록을 만들고 ListBox에 바인딩합니다.

1. 툴바에서 Dataset 을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. Invisible Objects 영역에 있는 Dataset00을 더블 클릭하여 Dataset 편집기를 실행합니다. 컬럼과 로우에 다음과 같이 입력하고 Dataset 편집기를 닫습니다.



3. Invisible Objects 영역에 있는 Dataset00을 폼에 배치한 ListBox 컴포넌트로 드래그 앤 드롭하여 메뉴에서 [Bind InnerDataset "Dataset00"]을 선택합니다.
4. Bind InnerDataset 창이 뜨면 codecolumn에 'CODE'를 datacolumn에 'DATA'를 입력 혹은 선택합니다. [OK] 버튼을 클릭하면 바인딩이 완료됩니다.



3 QuickView로 확인하기

ListBox에 아이템 목록이 정상적으로 출력되는지 QuickView(**Ctrl** + **F6**)로 확인합니다.

28.2 다중선택

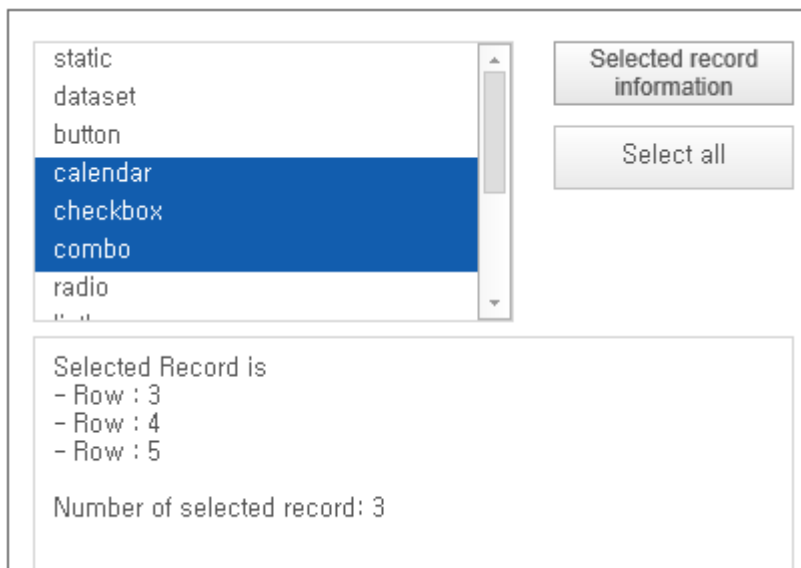
ListBox는 기본적으로 단일 선택 모드를 지원합니다. 다중 선택 모드를 사용하려면 ListBox의 `multiselect` 속성을 설정해야 합니다.

본 예제에서는 다중 선택이 가능한 ListBox를 만들고 선택한 아이템의 정보를 추출하는 방법에 대해 설명합니다.

28.2.1 예제

예제는 선택한 아이템의 정보를 추출하는 기능과 아이템을 모두 선택하는 두 가지 기능을 갖습니다.

- 선택한 아이템의 정보를 추출하는 기능: ListBox에서 아이템을 선택한 후 Selected record 버튼을 클릭하면 선택한 아이템의 로우 인덱스와 선택한 아이템의 개수가 TextArea 컴포넌트에 출력됩니다.
- 모든 아이템을 선택하는 기능: Select all 버튼을 클릭하면 ListBox의 모든 아이템이 선택됩니다.



sample_listbox_03.xfdl

28.2.2 예제에서 사용한 핵심 기능

multiselect

ListBox에서 여러 개의 아이템을 선택할 수 있게 설정하는 속성입니다.

getSelectedCount

ListBox에서 선택 상태인 아이템의 개수를 반환하는 메소드입니다.

getSelectedItems

ListBox에서 선택 상태인 아이템의 인덱스를 배열로 반환하는 메소드입니다. 배열의 인덱스 값은 자동으로 정렬됩니다.

getCount

ListBox에 표시되는 전체 아이템의 개수를 반환하는 메소드입니다. innerdataset 속성에 설정된 Dataset이 필터링되어 있다면 필터링된 결과만 아이템 개수에 포함하여 반환합니다.

setSelect

ListBox에서 특정 아이템의 선택 상태를 설정하는 메소드입니다.

28.2.3 예제 구현 방법

① 화면 구성하기

ListBox , Button  컴포넌트를 예제의 그림과 같이 적절히 배치합니다.

② ListBox 아이템 목록 구성하기

Dataset으로 아이템 목록을 만들고 ListBox에 바인딩합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. Invisible Objects 영역에 있는 Dataset00을 더블 클릭하여 Dataset 편집기를 실행합니다. 컬럼과 로우에 다음과 같이 입력하고 Dataset 편집기를 닫습니다.

Dataset Editor

Const Columns

No	id	type	size	value
There are no items to show.				

Columns

No	id	type	size	prop	sumtext	description
1	CODE	STRING	256			
2	DATA	STRING	256			

Rows

No	CODE	DATA
1	00	static
2	01	dataset
3	02	button
4	03	calendar
5	04	checkbox
6	05	combo
7	06	radio
8	07	listbox
9	08	grid
10	09	tabpage
11	10	tab
12	11	imageviewer
13	12	groupbox

Dataset | Source |

3. Invisible Objects 영역에 있는 Dataset00을 폼에 배치한 ListBox 컴포넌트로 드래그 앤 드롭하여 메뉴에서 [Bind InnerDataset "Dataset00"]을 선택합니다.
4. Bind InnerDataset 창이 뜨면 codecolumn에 'CODE'를 datacolumn에 'DATA'를 입력 혹은 선택합니다. [OK] 버튼을 클릭하면 바인딩이 완료됩니다.

Bind InnerDataset

InnerBind

codecolumn	CODE
datacolumn	CODE DATA

3 ListBox 설정하기

ListBox의 multiselect 속성을 'true'로 설정합니다.

4 선택한 아이템의 정보를 추출하는 기능 구현하기

ListBox에서 선택된 레코드(아이템)와 개수를 구해서 TextArea 컴포넌트의 value 속성으로 설정합니다. Selected record 버튼의 onclick 이벤트를 함수를 작성합니다.

```
this.Button00.onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var sMsg = "Selected record is\n";
    var arrSelectedItems = this.ListBox00.getSelectedItems();

    for(var i=0;i<arrSelectedItems.length;i++)
    {
        sMsg += "- Row : "+ arrSelectedItems[i] +"\n";
    }

    sMsg += "\nNumber of selected record: " + this.ListBox00.getSelectedCount();

    this.TextArea00.set_value(sMsg);

};
```

5 모든 아이템을 선택하는 기능 구현하기

모든 아이템을 선택상태로 전환합니다. Select all 버튼의 onclick 이벤트를 함수를 작성합니다.

```
this.Button01.onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    for(i=0;i<this.ListBox00.getCount();i++)
    {
        this.ListBox00.setSelect(i, true);
    }
};
```

6 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 ListBox의 아이템을 선택하고 버튼을 차례로 눌러 정확히 동작하는지 확인합니다.

29.

ListView

29.1 ListView 소개

ListView 컴포넌트는 Grid 컴포넌트처럼 바인딩 된 Dataset 오브젝트의 데이터를 표현할 수 있습니다. Grid 컴포넌트는 Row, Column 기준으로 데이터를 표현하지만, ListView 컴포넌트에서는 Body 또는 Detail 밴드 영역 내에서 자유롭게 데이터를 배치해서 원하는 형식으로 표현할 수 있습니다.

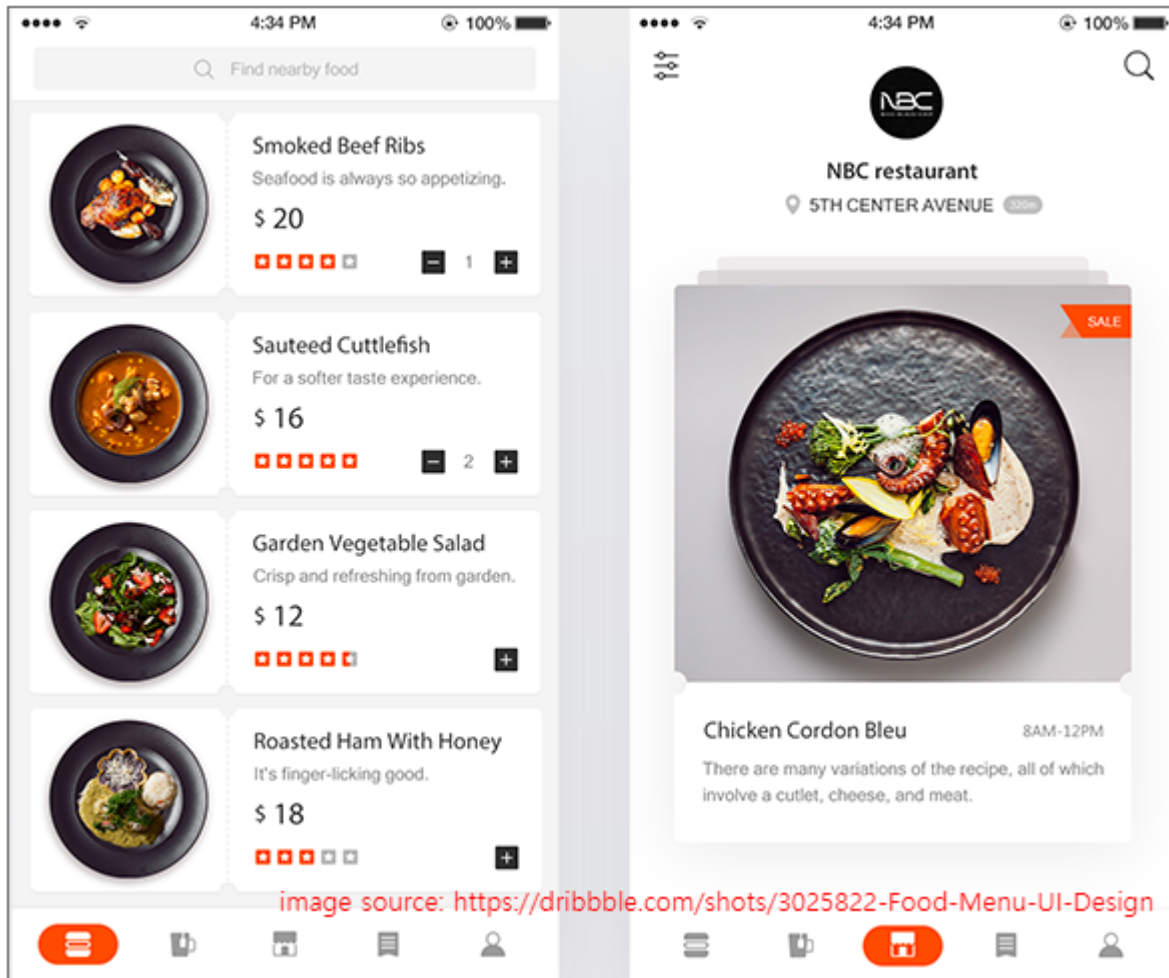
Body 밴드는 기본적으로 표시하고 Detail 밴드는 사용자가 버튼을 클릭하거나 다른 이벤트를 통해 표시하도록 설정할 수 있습니다.



ListView 컴포넌트는 17.0.0.1100 버전부터 지원합니다.

29.1.1 사용 예

아래 이미지는 모바일 기기에서 사용하는 음식점 메뉴 소개 디자인입니다. 단순히 목록을 나열하지 않고 왼쪽에 이미지를 배치하고 오른쪽에는 이미지(메뉴)에 대한 설명을 배치합니다. Grid 컴포넌트 같은 경우에는 특정 데이터를 강조하기가 쉽지 않은데, ListView 컴포넌트는 개별 데이터를 각각의 Cell로 처리하기 때문에 글자색을 다르게 하거나 글자 크기를 원하는 형태로 바꾸어 표시할 수 있습니다.



29.1.2 ListView 만들기

1 ListView 생성하기

툴바에서 ListView를 선택한 후, 폼 위에서 적당한 크기로 드래그하여 생성합니다. 컴포넌트를 선택한 상태에서 폼 위에 클릭하면 기본 크기로 생성됩니다.



2 ListView 아이템 목록 구성하기

Dataset 오브젝트에 화면에 표시할 데이터를 생성하고 ListView 컴포넌트에 바인딩합니다. Dataset 오브젝트는 아래와 같이 설정합니다.

Columns						
No	id	type	size	prop	sumtext	c
1	No	STRING	256			
2	Title	STRING	256			
3	Detail	STRING	256			

Rows				
No	No	Title	Detail	
1	1	ABC	ABCDEFGG	
2	2	DEF	DFEABCD	

3 Dataset 바인딩하기

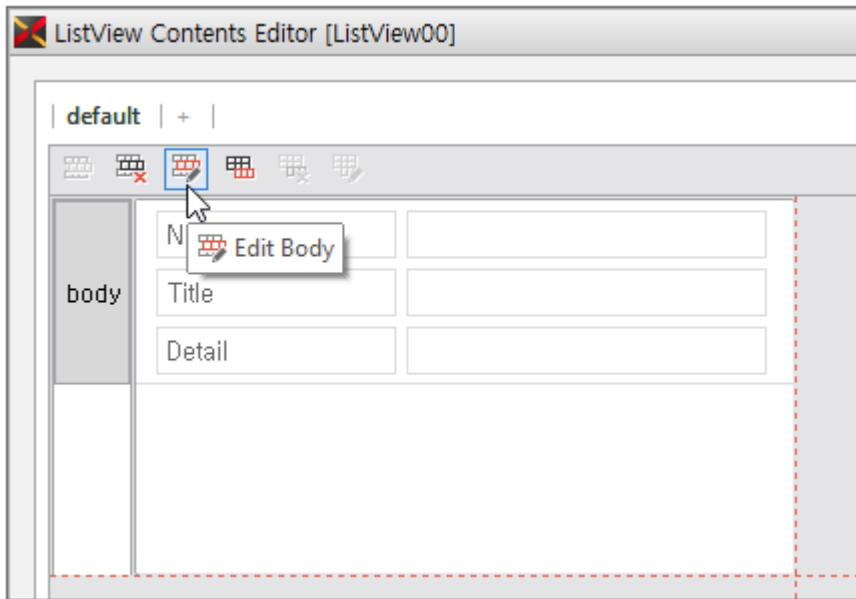
생성된 Dataset 오브젝트를 선택하고 마우스로 드래그해서 ListView 컴포넌트로 가져옵니다. 바인딩이 완료 되면 아래와 같이 기본 레이아웃이 설정됩니다.

No	1
Title	ABC
Detail	ABCDEFGG
No	2
Title	DEF
Detail	DFEABCD

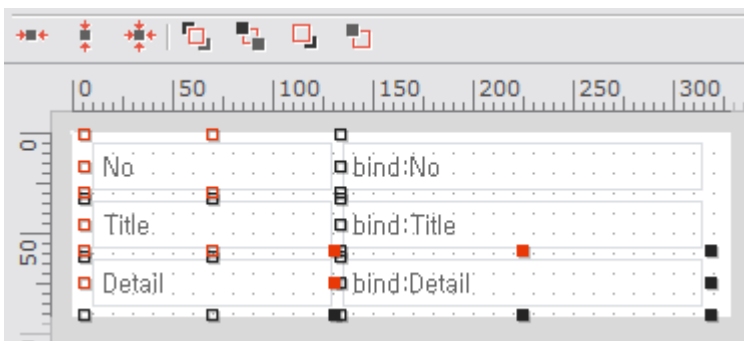
그대로 표시하더라도 사용자에게 정보를 전달할 수 있지만, 좀 더 깔끔하게 데이터를 정리해봅니다.

4 ListView Contents Editor 실행하기

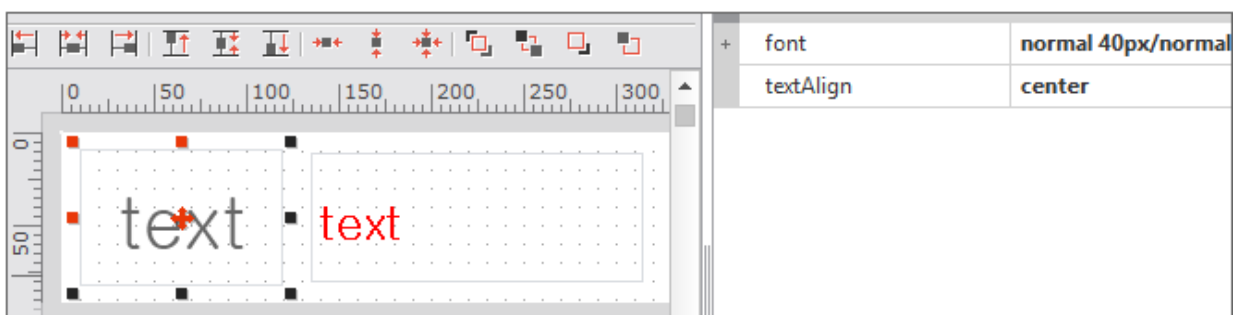
ListView 컴포넌트를 더블클릭하면 ListView Contents Editor를 실행합니다. "No" 항목을 글자 크기를 키워서 왼쪽에 배치하고 오른쪽에는 Title을 붉은색으로 표시합니다. 상단 툴바에서 [Edit Body]를 선택해서 Body 밴드 편집 모드로 전환합니다.



필요 없는 셀들은 모두 삭제합니다. "bind:No", "bind:Detail" 항목 외에는 모두 삭제합니다. Form에서 컴포넌트 편집하는 것과 같은 방식으로 셀을 선택하고 삭제합니다.



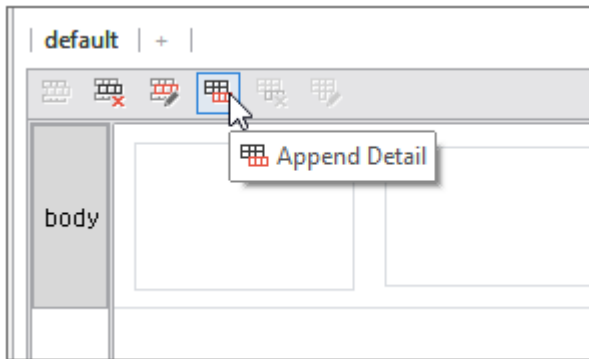
"No" 항목을 표시하는 부분은 font 속성값에서 font-size를 40, textAlign 속성값을 "center"로 설정합니다. "Title" 항목을 표시하는 부분은 font 속성값에서 font-size를 25, color 속성값을 "red"로 설정합니다. 그리고 셀의 크기와 위치를 적절하게 조정합니다.



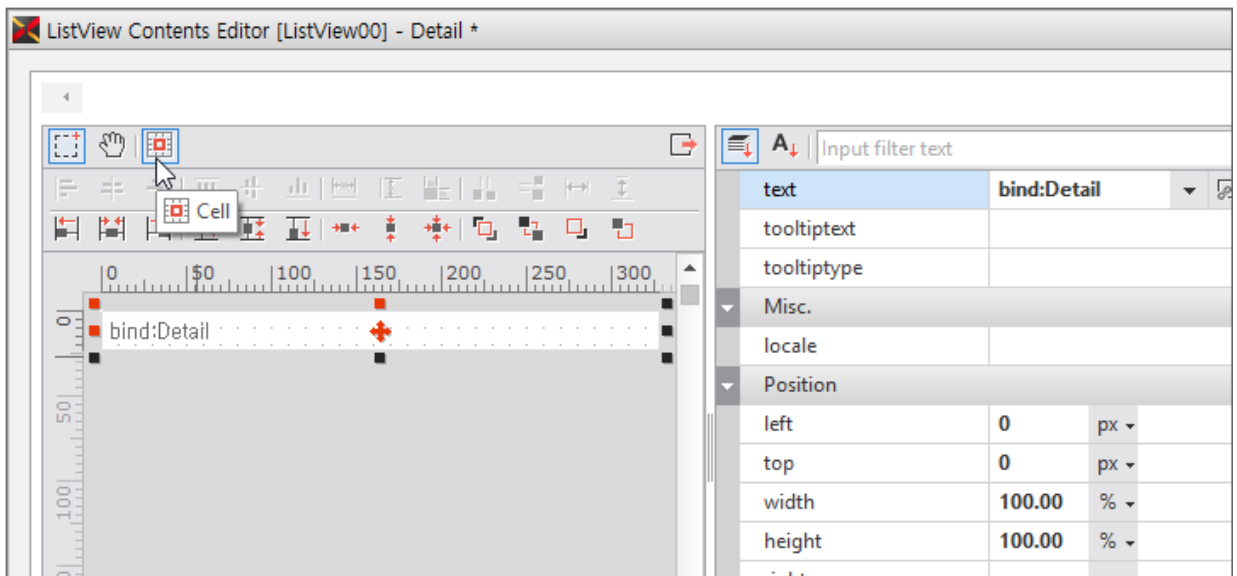
셀 편집이 끝나면 Enter 키를 입력하거나 [Exit Editor] 메뉴를 선택해 기본 편집 모드로 전환합니다.

5 Detail 밴드 추가하기

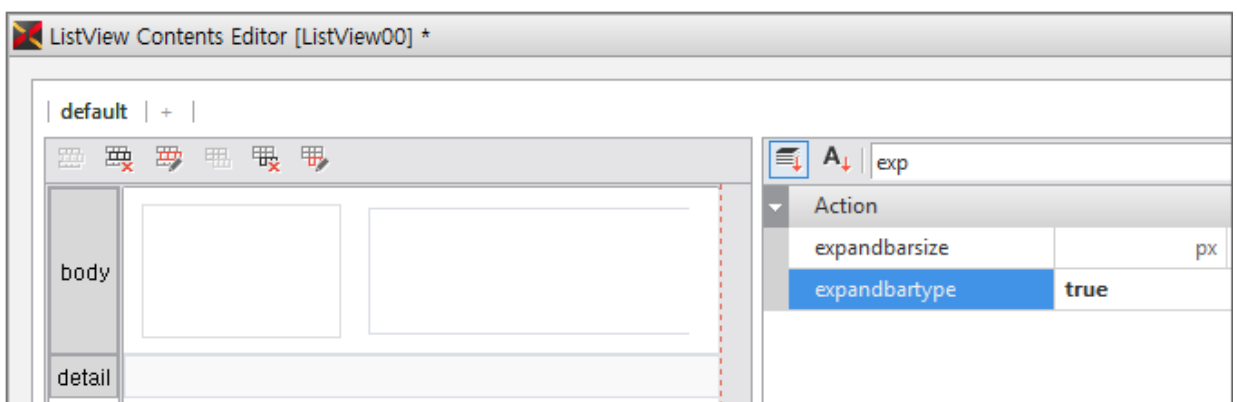
상단 툴바에서 [Append Detail] 항목을 선택하고 Detail 밴드를 추가합니다. [Edit Detail] 항목을 선택한 후 Detail 밴드에 표시될 내용을 편집합니다.



[Cell] 항목을 선택해서 셀을 추가하고 text 속성값과 Position 관련 속성을 수정합니다. 셀 편집이 끝나면 Enter 키를 입력하거나 [Exit Editor] 메뉴를 선택해 기본 편집 모드로 전환합니다.



Body 밴드를 선택하고 속성창에서 expandbartype 속성값을 true로 설정합니다.



6 QuickView로 확인하기

QuickView를 실행하고 설정한 값이 어떻게 표시되는지 확인합니다. 오른쪽에 표시되는 버튼을 클릭하면 Detail 밴드 영역을 보였다가 보이지 않도록 설정할 수 있습니다.



29.2 기본, 상세 데이터 표현하기

ListView 컴포넌트의 기본 기능인 기본, 상세 데이터를 표현하는 방식과 세부 속성에 따른 차이를 살펴봅니다. 대략적인 화면은 아래와 같이 구성합니다.



예제에서 사용한 이미지와 텍스트는 아래 자료에서 가져왔습니다.

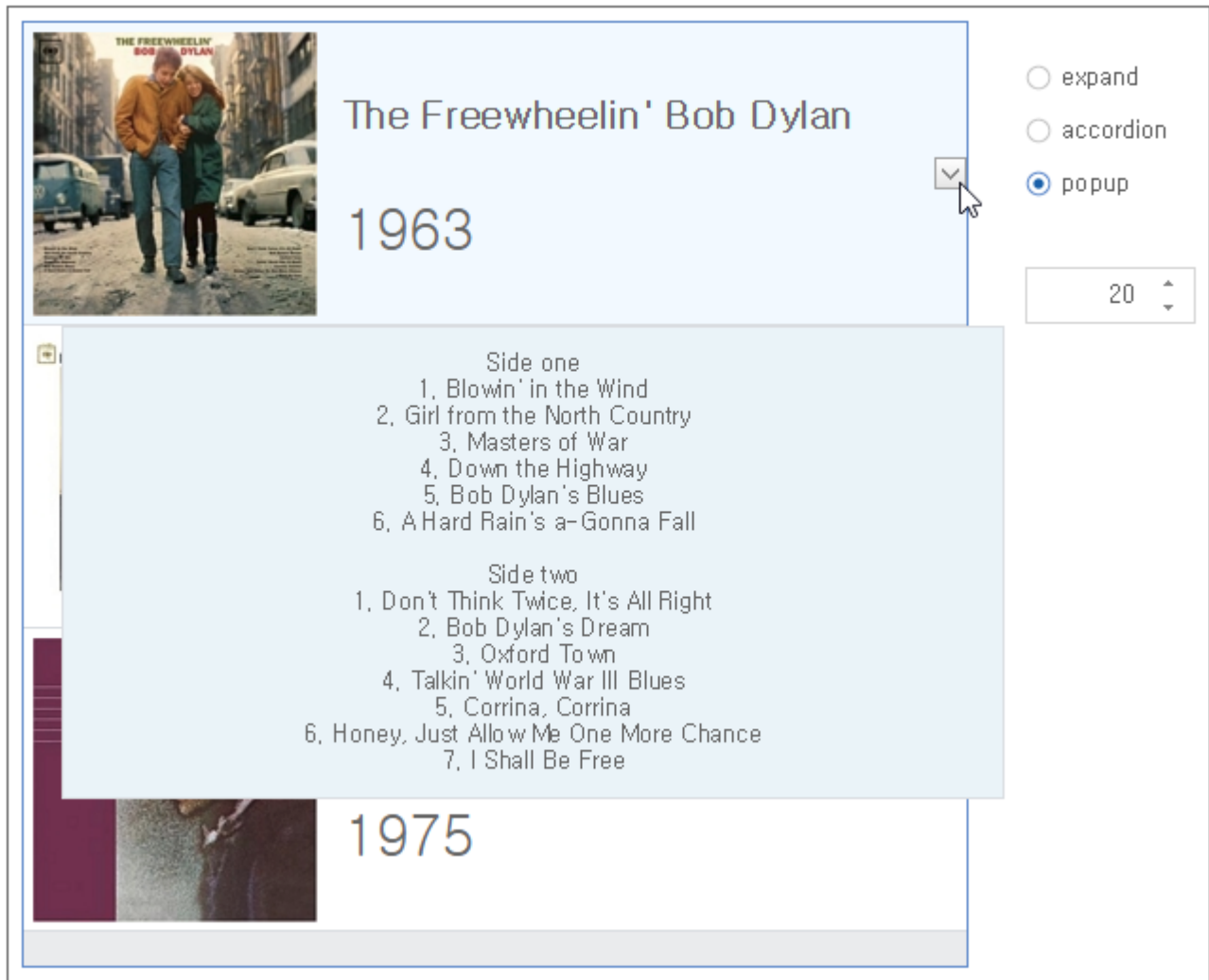
https://en.wikipedia.org/wiki/The_Freewheelin%27_Bob_Dylan

https://en.wikipedia.org/wiki/Highway_61_Revisited

https://en.wikipedia.org/wiki/Blood_on_the_Tracks

29.2.1 예제

앨범 정보를 표시합니다. 오른쪽 버튼을 클릭하면 해당 앨범 수록곡을 표시합니다. Radio 컴포넌트를 선택하면 band expandtype 속성값을 변경하고 Spin 컴포넌트의 숫자를 변경하면 bandindentsize 속성값으로 반영됩니다.



29.2.2 예제에서 사용한 핵심 기능

bandexpandtype

Body 밴드 설정에서 expandbartye 속성값을 true로 지정한 경우 목록 오른쪽에 Detail 밴드 영역을 제어할 수 있는 버튼이 표시됩니다. 이때 Detail 밴드 영역을 어떻게 표시할지 bandexpandtype 속성에서 지정할 수 있습니다. 속성값을 "expand", "accordion"으로 설정한 경우 비슷하게 동작하는 것처럼 보이는데 "accordion"으로 설정한 경우에는 한번에 하나의 Detail 밴드만 표시합니다. 예를 들어 첫 번째 아이템의 Detail 밴드가

열린 상태에서 두 번째 아이템의 Detail 밴드를 펼치면 첫 번째 아이템의 Detail 밴드는 자동으로 닫힙니다.

bandindentsize

Detail 밴드가 펼쳐질 때 지정된 속성값만큼 왼쪽 여백이 생깁니다. bandexpandtype 속성값이 "popup"인 경우에는 ListView 컴포넌트의 영역을 벗어나 Detail 밴드가 표시될 수 있습니다.

29.2.3 예제 구현 방법

① 화면 구성하기

ListView 컴포넌트와 Radio, Spin 컴포넌트를 예제 화면과 같이 배치합니다. ListView 컴포넌트의 width, height 속성값은 500으로 지정합니다. Dataset 오브젝트를 생성하고 아래와 같이 설정합니다. Dataset 오브젝트를 화면에서 드래그해서 ListView 컴포넌트에 바인딩합니다.

Columns					
No	id	type	size	prop	st
1	title	STRING	256		
2	release	STRING	256		
3	cover	STRING	256		
4	songs	STRING	256		

Rows			
No	title	release	cover
1	The Freewheelin' Bob D...	1963	imagerc::
2	Highway 61 Revisited	1965	imagerc::
3	Blood on the Tracks	1975	imagerc::

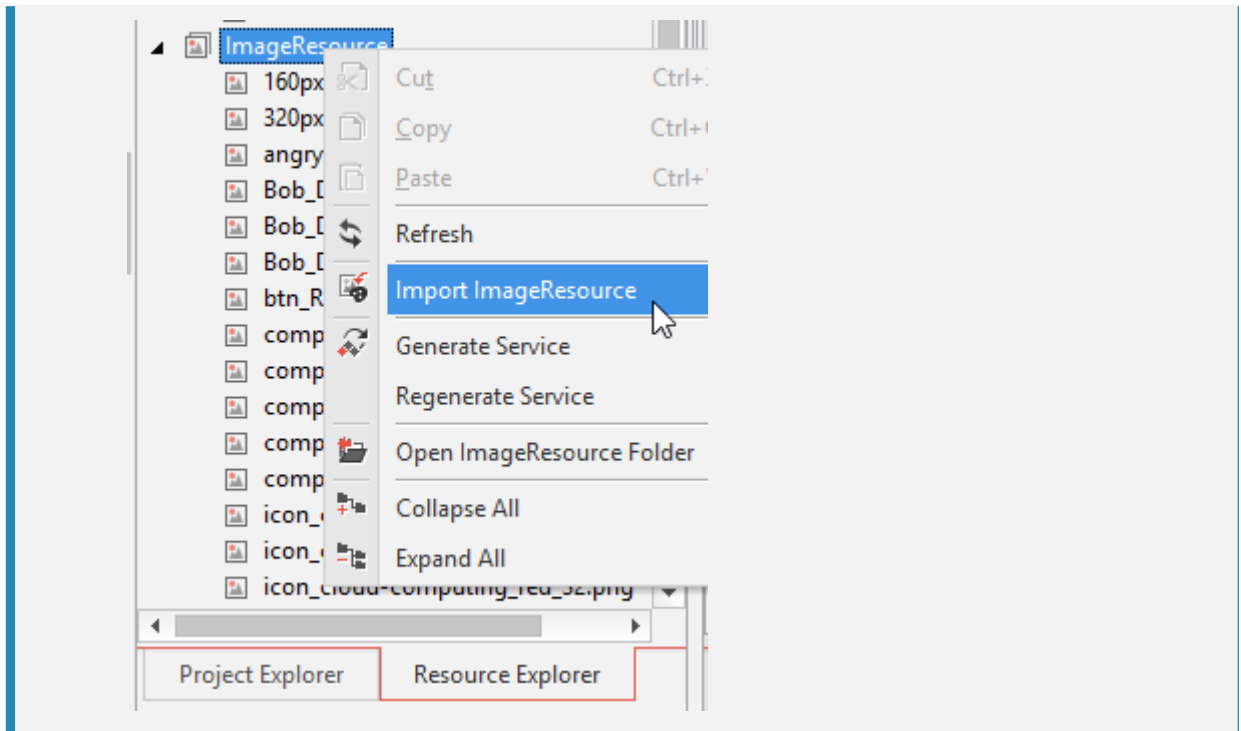


수록곡 데이터가 포함되어 있어 위의 이미지에서는 전체 데이터를 다 보여주지 않습니다. 해당 샘플 코드를 참고하세요.



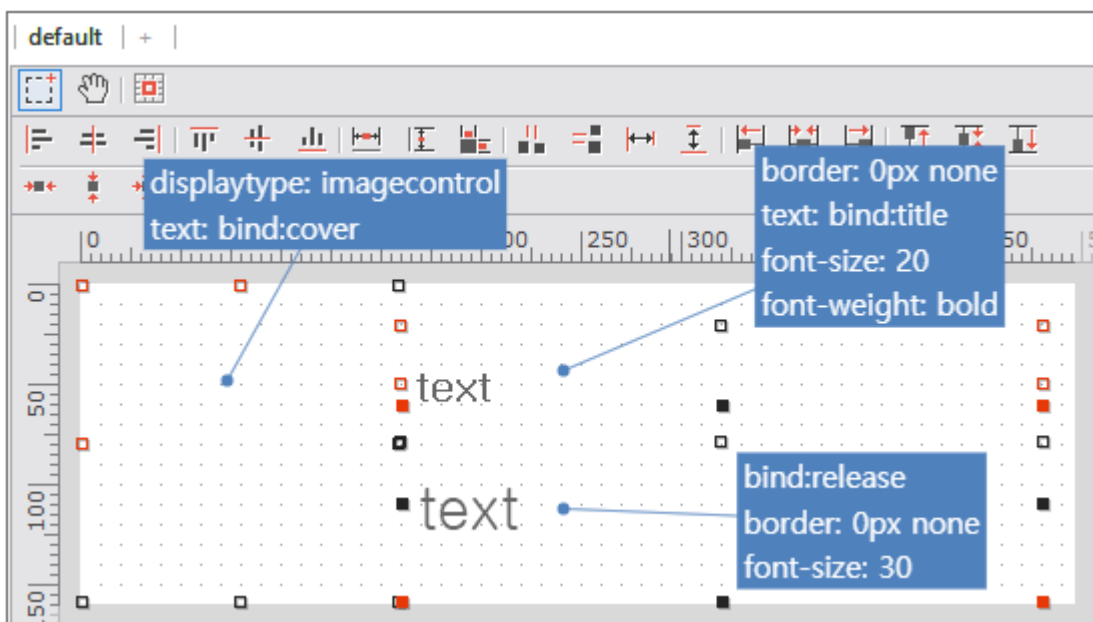
cover 컬럼값에는 앨범 커버 이미지 경로를 지정합니다. 이미지 파일은 Resource Explorer에서 [Import ImageResource] 기능을 사용해 프로젝트에 추가합니다. 추가된 이미지는 아래와 같은 경로를 가집니다.

imagerc::Bob_Dylan_-_The_Freewheelin'_Bob_Dylan.jpg



2 ListView Contents Editor 실행하기

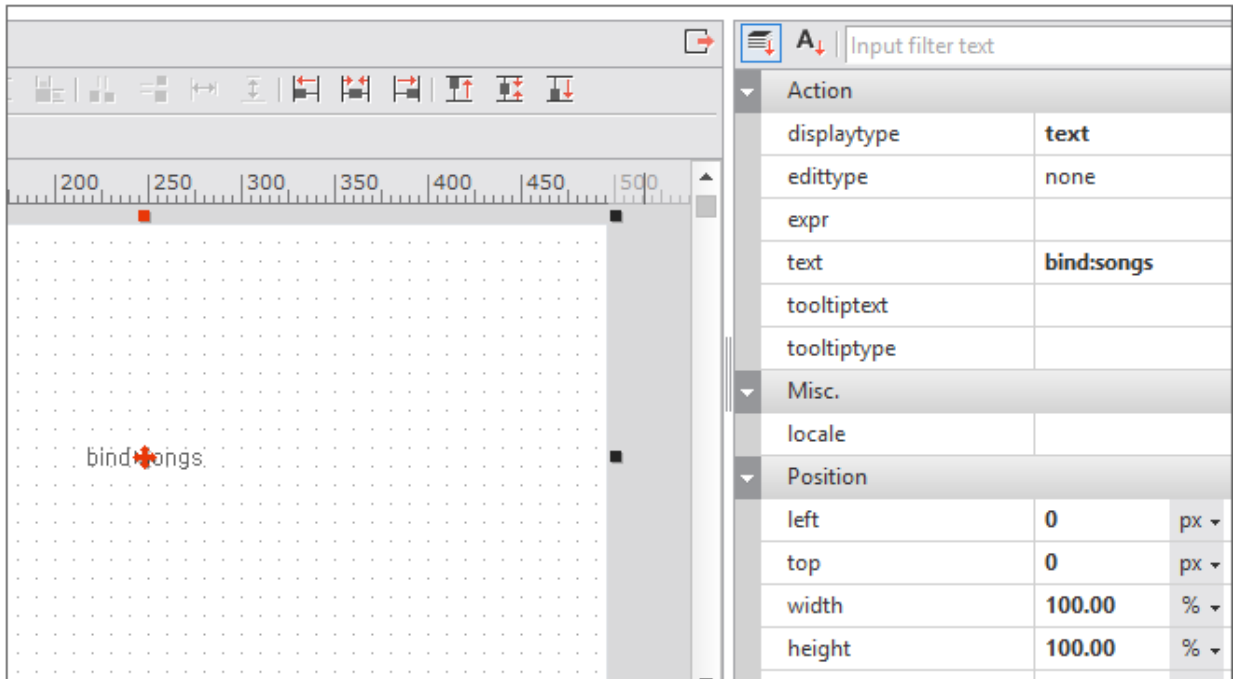
Dataset 오브젝트의 값에 따라 컬럼 id, 컬럼 데이터를 ListView 컴포넌트의 셀로 자동 설정합니다. 하지만, 예제에서는 다른 식으로 화면을 구성할 것이기 때문에 Body 밴드 편집 모드로 전환하고 설정된 셀을 모두 삭제합니다. 그리고 아래와 같이 3개의 셀을 배치합니다.



Body 밴드의 expandbartype 속성값은 true로 설정하고 height 속성값은 160으로 지정합니다.

3 Detail 밴드 추가하고 설정하기

ListView Contents Editor 상단 툴바에서 [Append Detail] 항목을 선택하고 Detail 밴드를 추가합니다. Detail 밴드의 height 속성값은 250으로 지정하고 Detail 밴드 편집 모드로 전환합니다. 셀을 하나 추가하고 셀 속성을 아래와 같이 지정합니다.



ListView Contents Editor 창을 닫고 ListView 컴포넌트의 bandinitstatus 속성값을 "collapse"로 설정합니다. QuickView(Ctrl + F6)로 실행한 후 화면에 앨범 정보가 표시되고 오른쪽 버튼 클릭 시 수록곡 정보가 표시되는지 확인합니다.

④ bandexpandtype 속성값 바꾸어보기

예제에서 bandexpandtype 속성값을 바꾸어볼 수 있도록 Radio 컴포넌트에서 bandexpandtype 속성값을 설정할 수 있도록 합니다. innerdataset 항목값으로 "expand", "accordion", "popup"을 설정하고 onitemchanged 이벤트 함수를 아래와 같이 작성합니다.

```
this.Radio00_onitemchanged = function(obj:nexacro.Radio,e:nexacro.ItemChangeEventInfo)
{
    this.ListView00.set_bandexpandtype(e.postvalue);
};
```

⑤ bandindentsize 속성값 바꾸어보기

Spin 컴포넌트에서 설정한 값이 bandindentsize 속성값으로 지정될 수 있도록 onchanged 이벤트 함수를 아래와 같이 작성합니다.

```

this.Spin00_onchanged = function(obj:nexacro.Spin,e:nexacro.ChangeEventInfo)
{
    this.ListView00.set_bandindentsize(e.postvalue);
};

```

⑥ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 bandexpandtype, bandindentsize 속성값을 바꾸고 화면이 어떻게 표시되는지 확인합니다.

29.3 너비에 따라 다른 형식으로 데이터 표현하기

같은 데이터라도 사용하는 디바이스에 따라 다른 형식으로 표현할 수 있습니다. 이미지와 텍스트를 표현할 때 이미지와 텍스트를 나란히 하나의 컬럼에 표현할 수도 있고 너비가 좀 더 커지면 두개 이상의 컬럼에 표현할 수도 있습니다. 너비가 텍스트를 같이 표시하기에 좁은 경우에는 이미지 위에 텍스트를 표현하는 형식도 선택할 수 있습니다.

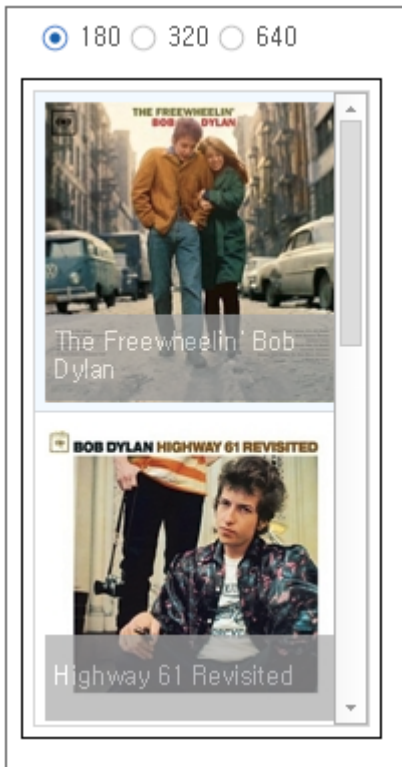


예제에서 사용한 이미지와 텍스트는 아래 자료에서 가져왔습니다.

https://en.wikipedia.org/wiki/The_Freewheelin%27_Bob_Dylan
https://en.wikipedia.org/wiki/Highway_61_Revisited
https://en.wikipedia.org/wiki/Blood_on_the_Tracks
https://en.wikipedia.org/wiki/Bringing_It_All_Back_Home
https://en.wikipedia.org/wiki/Blonde_on_Blonde

29.3.1 예제

Radio 컴포넌트에서 너비값을 선택하면 해당 너비에 적절한 형식으로 데이터를 표현합니다.



29.3.2 예제에서 사용한 핵심 기능

formatid

조건에 따라 다른 형식으로 데이터를 표현하고자 할때 formatid 속성값을 변경해 적용할 수 있습니다. Grid 컴포넌트의 경우에는 컬럼의 순서를 바꾸거나 표현하는 컬럼 개수를 바꾸는 등의 제한적인 활용만 할 수 있는데 ListView 컴포넌트에서는 좀 더 자유로운 형식으로 배치할 수 있습니다.

29.3.3 예제 구현 방법

① 화면 구성하기

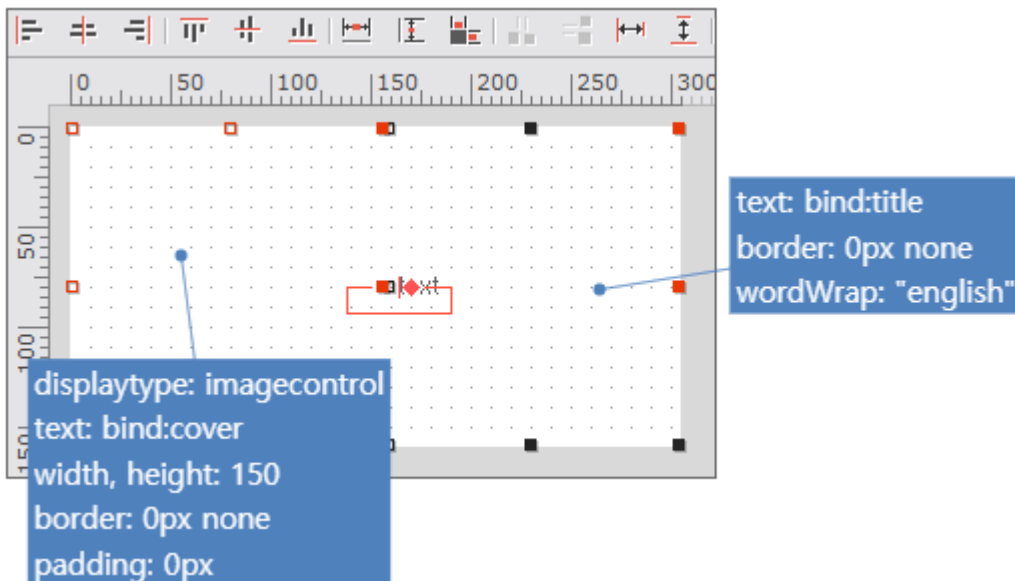
ListView 컴포넌트와 Radio 컴포넌트를 예제 화면과 같이 배치합니다. ListView 컴포넌트를 배치하기 전에 Div 컴포넌트를 배치하고 그 안에 ListView 컴포넌트를 배치합니다. Div 컴포넌트의 width, height 속성값은 320으로 지정하고 ListView 컴포넌트는 left, top, right, bottom 속성값을 지정해 Div 컴포넌트의 너비값을 변경할때 같이 너비값이 변경될 수 있도록 합니다. Dataset 오브젝트를 생성하고 아래와 같이 설정합니다. Dataset 오브젝트를 화면에서 드래그해서 ListView 컴포넌트에 바인딩합니다.

Columns				
No	id	type	size	prop
1	title	STRING	256	
2	cover	STRING	256	

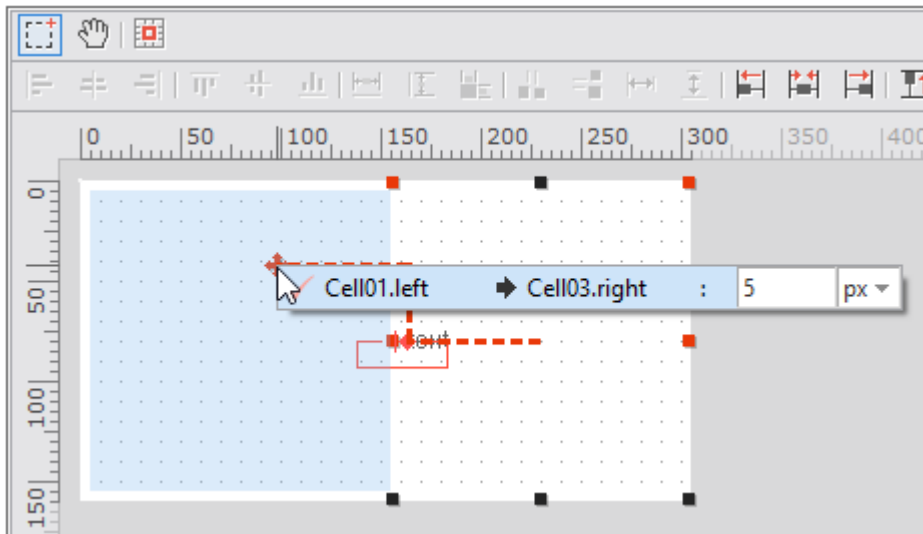
Rows		
No	title	cover
1	The Freewheelin' Bob D...	imagerc::Bob_Dylan_-_T...
2	Highway 61 Revisited	imagerc::Bob_Dylan_-_...
3	Blood on the Tracks	imagerc::Bob_Dylan_-_B...
4	Blonde on Blonde	imagerc::Bob_Dylan_-_B...
5	Bringing It All Back Home	imagerc::Bob_Dylan_-_B...

2 ListView Contents Editor 실행하기

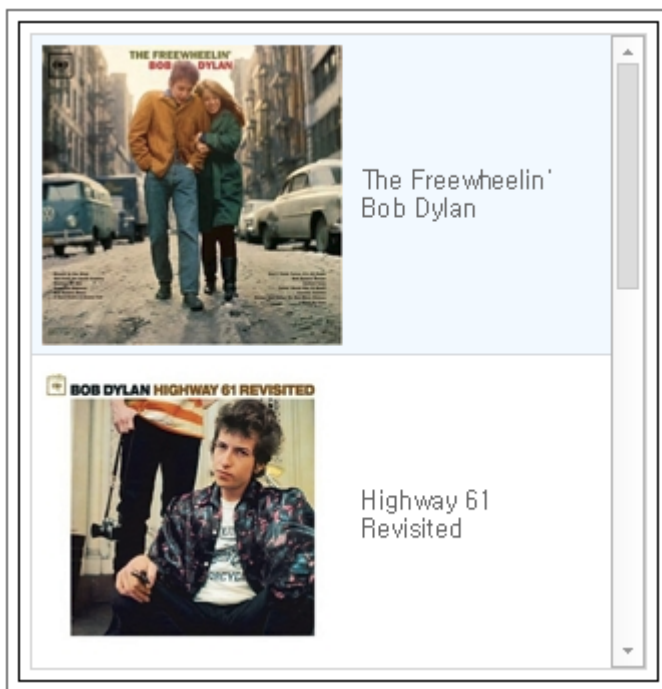
Dataset 오브젝트의 값에 따라 컬럼 id, 컬럼 데이터를 ListView 컴포넌트의 셀로 자동 설정합니다. 하지만, 예제에서는 다른 식으로 화면을 구성할 것이기 때문에 Body 밴드 편집 모드로 전환하고 컬럼 id를 나타내는 셀은 삭제하고 나머지 2개의 컬럼 데이터를 표현하는 셀만 사용합니다.



앨범 제목을 표시하는 셀은 텍스트가 길어질 수 있어서 wordWrap 속성을 적용했습니다. 그리고 앨범 이미지 바로 옆에 위치할 수 있도록 PositionBase 속성을 적용합니다.



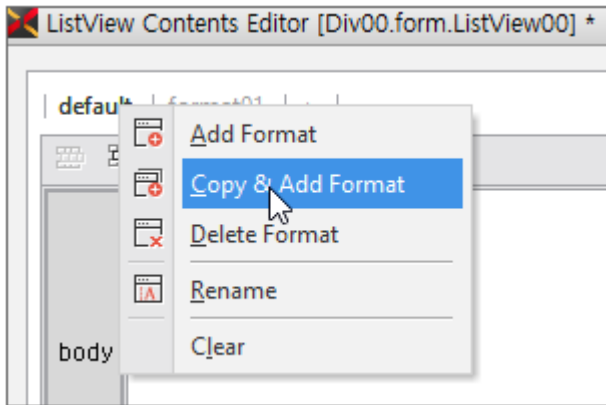
적용된 결과는 아래와 같습니다. 하나의 컬럼에 데이터를 표시합니다.



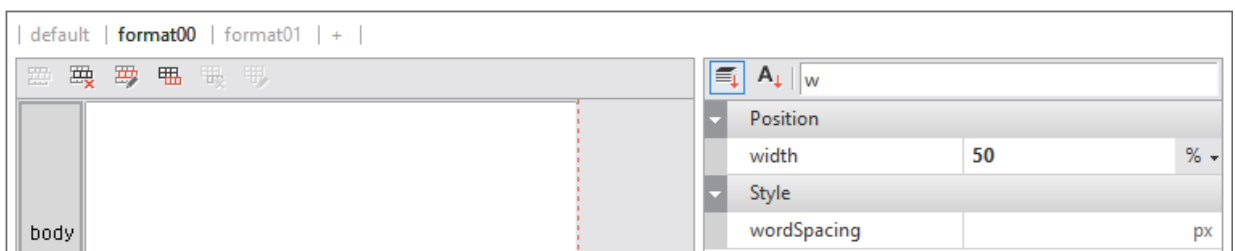
3 두 개의 컬럼에 데이터 표시하기

ListView 컴포넌트에서는 컬럼이라는 개념은 없습니다. ListView 컴포넌트의 너비와 Body 밴드의 너비에 따라 자동으로 하나의 Row에 몇 개의 데이터를 표시할지 결정합니다. 예를 들어 Body 밴드의 너비를 50%로 지정하면 2개의 데이터를 하나의 Row에 표시합니다.

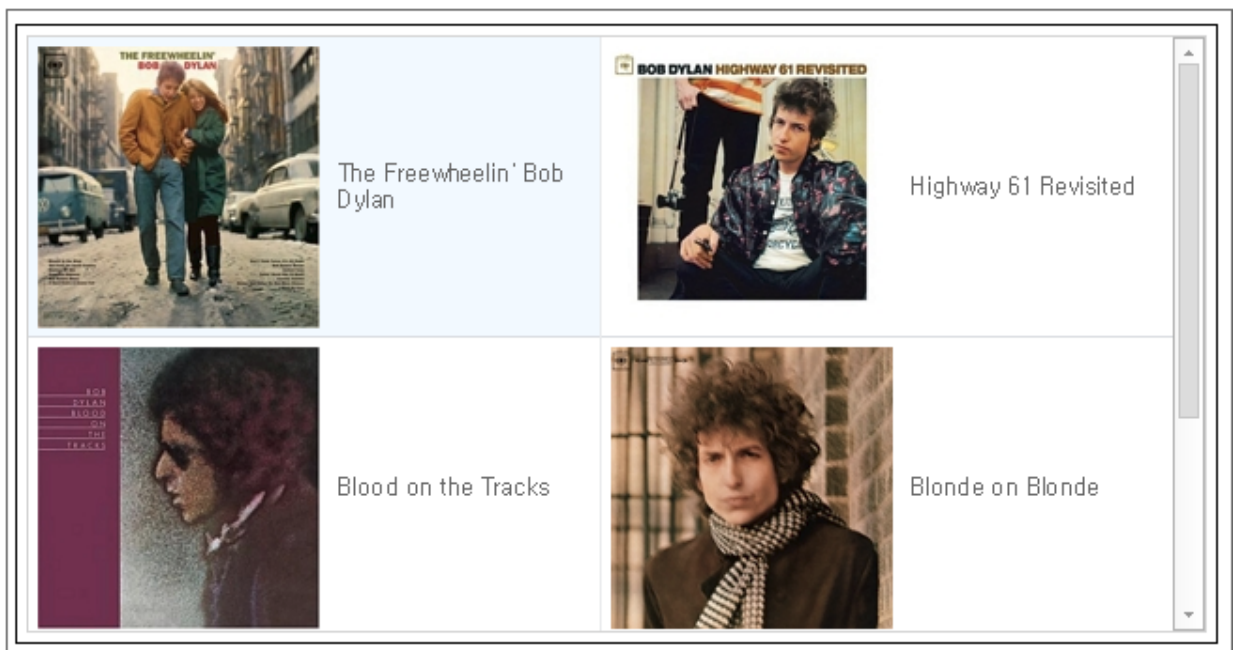
ListView Contents Editor를 실행하고 default 포맷 항목 위에서 오른쪽 마우스 버튼을 클릭하고 [Copy & Add Format] 항목을 선택합니다. Format id는 "format00"으로 수정합니다.



Body 밴드를 선택하고 width 속성값을 50%로 수정합니다.

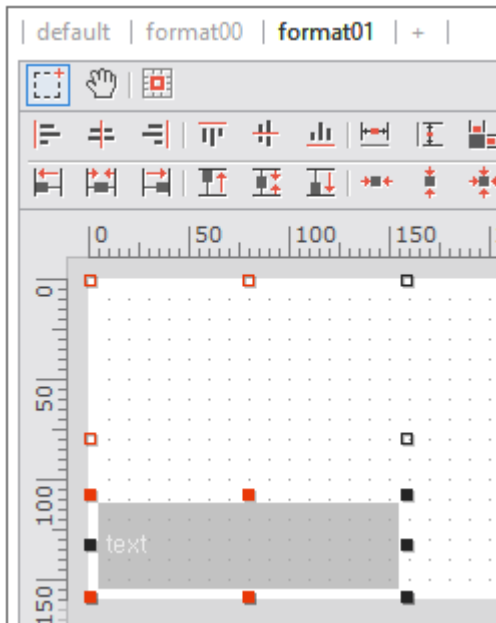


적용된 결과는 아래와 같습니다. 두 개의 컬럼에 데이터를 표시합니다. width 속성값을 30%로 설정하면 세 개의 컬럼에 데이터를 표시하게 됩니다.

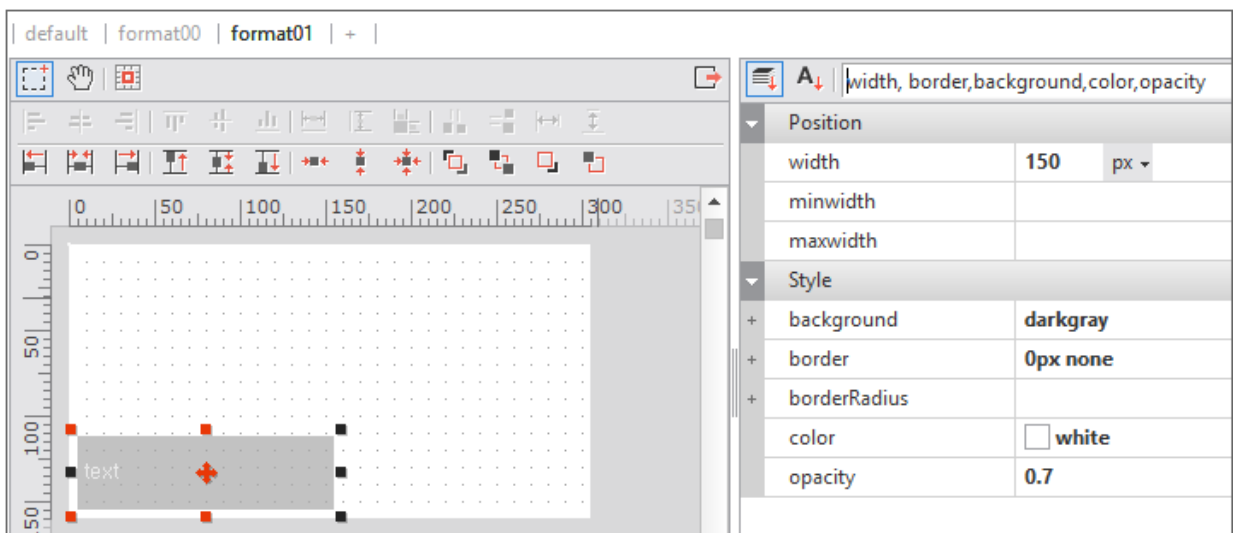


4 이미지 위에 텍스트 표시하기

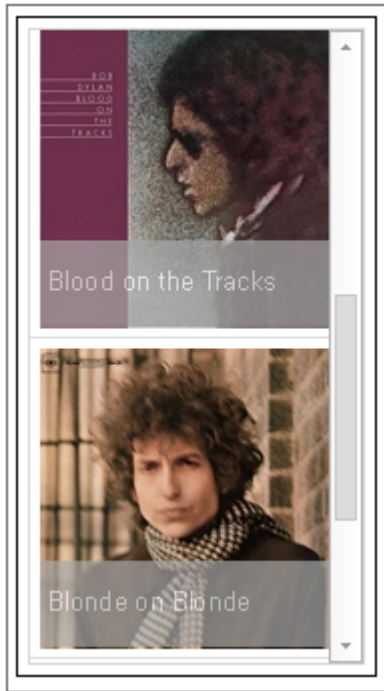
ListView Contents Editor를 실행하고 default 포맷 항목 위에서 오른쪽 마우스 버튼을 클릭하고 [Copy & Add Format] 항목을 선택합니다. Format id는 "format01"으로 수정합니다. Body 밴드 편집 모드로 들어가서 앨범 타이틀을 표시하던 셀의 위치를 앨범 커버를 표시하는 셀 위로 이동합니다.



width 속성값을 앨범 커버를 표시하는 셀과 같은 150으로 설정하고 background, color, opacity 속성값을 수정해줍니다.



적용된 결과는 아래와 같습니다. 앨범 타이틀 텍스트를 앨범 이미지 위에 살짝 투명한 배경을 가지고 표현합니다.



5. 너비에 따라 formatid 다르게 적용하기

Radio 컴포넌트에서 div 컴포넌트의 너비값을 설정할 수 있도록 합니다. innerdataset 항목값으로 "180", "320", "640"을 설정하고 onitemchanged 이벤트를 함수를 아래와 같이 작성합니다.

```
this.Radio00_onitemchanged = function(obj:nexacro.Radio,e:nexacro.ItemChangeEventInfo)
{
    this.Div00.set_width(e.postvalue);
    if(e.postvalue == 640)
    {
        this.Div00.form.ListView00.set_formatid("format00");
    }
    else if(e.postvalue == 180)
    {
        this.Div00.form.ListView00.set_formatid("format01");
    }
    else
    {
        this.Div00.form.ListView00.set_formatid("default");
    }
};
```

6. QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 너비 속성값에 따라 formatid가 변경되면서 데이터가 표현되는 형식이 바

끼는 것을 확인합니다.

29.4 버튼 클릭 이벤트 처리하기

ListView 컴포넌트에서 표시되는 각 항목은 셀로 처리됩니다. 이미지 버튼을 배치하고 클릭했을 때 원하는 동작을 어떻게 처리하는지 살펴봅니다.

29.4.1 예제

각 항목 아래 [-], [+] 버튼 클릭 시 숫자가 변경되며 오른쪽 Grid 컴포넌트에 합계 금액이 표시됩니다.



The Freewheelin' Bob Dylan

\$7.98

- 1 +



Highway 61 Revisited

\$7.29

- 2 +

title	price	amount	Total
The Freewheelin' Bob Dylan	7,98	1	\$7,98
Highway 61 Revisited	7,29	2	\$14,58
Blood on the Tracks	9,98	1	\$9,98
Blonde on Blonde	9,98	0	\$0,00
Bringing It All Back Home	7,29	1	\$7,29
			\$39,83

 [sample_listview_03.xfdl](#)

29.4.2 예제에서 사용한 핵심 기능

cellid

ListView 컴포넌트의 EventInfo 오브젝트가 가지고 있는 속성 중 하나입니다. cellid는 bandid에 따라 같은 속성값을 가질 수 있습니다. 예를 들어 Body 밴드에 "Cell00"이라는 셀이 있어도 Detail 밴드에 "Cell00"이라는 셀이 있을 수 있습니다. 그렇기 때문에 Detail 밴드를 가지고 있는 경우에는 cellid와 bandid를 같이 확인해

야 정확하게 어떤 셀에서 이벤트가 발생한 것인지 확인할 수 있습니다.

29.4.3 예제 구현 방법

1 화면 구성하기

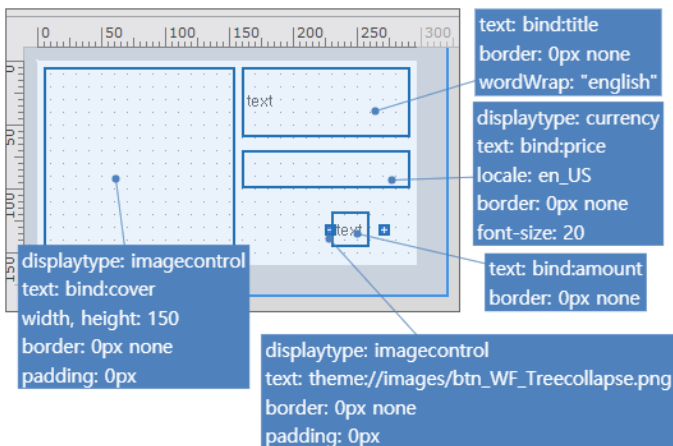
ListView 컴포넌트와 Grid 컴포넌트를 예제 화면과 같이 배치합니다. Dataset 오브젝트를 생성하고 아래와 같이 설정합니다. Dataset 오브젝트를 화면에서 드래그해서 ListView 컴포넌트와 Grid 컴포넌트에 바인딩합니다.

Columns					
No	id	type	size	prop	sumtext
1	title	STRING	256		
2	cover	STRING	256		
3	price	FLOAT	256		
4	amount	INT	256		

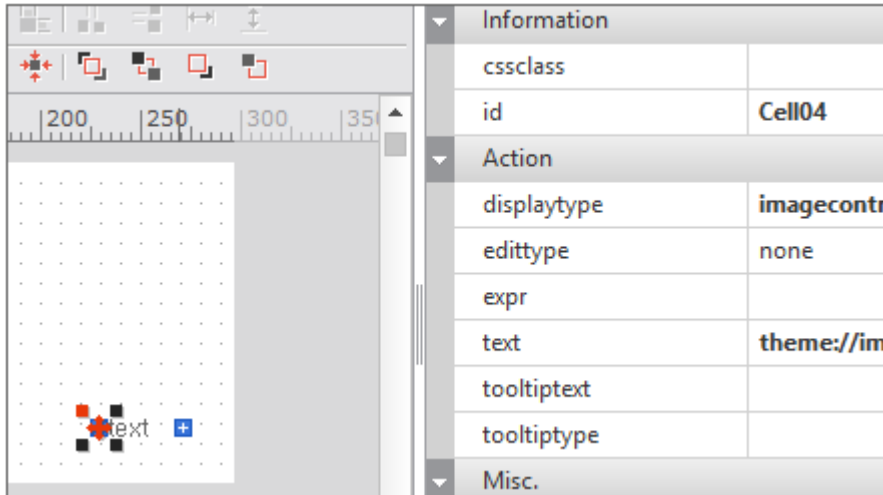
Rows					
No	title	cover	price	amount	
1	The Freewheelin' Bob D...	imagerc::Bob_Dylan_-_T...	7.98	0	
2	Highway 61 Revisited	imagerc::Bob_Dylan_-_...	7.29	0	
3	Blood on the Tracks	imagerc::Bob_Dylan_-_B...	9.98	0	
4	Blonde on Blonde	imagerc::Bob_Dylan_-_B...	9.98	0	
5	Bringing It All Back Home	imagerc::Bob_Dylan_-_B...	7.29	0	

2 ListView Contents Editor 실행하기

Dataset 오브젝트의 값에 따라 컬럼 id, 컬럼 데이터를 ListView 컴포넌트의 셀로 자동 설정합니다. 하지만, 예제에서는 다른 식으로 화면을 구성할 것이기 때문에 Body 밴드 편집 모드로 전환하고 컬럼 id를 나타내는 셀은 삭제하고 나머지 4개의 컬럼 데이터를 표현하는 셀만 사용합니다.



4개의 컬럼 데이터 외에 버튼 기능을 할 셀 2개를 추가합니다. displaytype 속성값은 "imagecontrol"으로 지정하고 text 속성값에 표시할 이미지 경로를 지정합니다. 예제에서는 테마에 있는 이미지를 사용했습니다. 해당 셀을 사용자가 클릭했을때 이벤트를 처리하기 위해서는 셀 id 값을 알아야 합니다. 셀을 선택하고 속성창에서 id 속성값을 확인하거나 수정할 수 있습니다.



3 oncellick 이벤트 함수 작성하기

ListView 컴포넌트의 oncellick 이벤트 함수를 작성합니다. 사용자가 [-] 또는 [+] 버튼 클릭 시 Dataset 오브젝트의 값을 업데이트합니다.

```
this.ListView00_onsellick = function(obj:nexacro.ListView,e:nexacro.
ListViewClickEventInfo)
{
    var cur_amount = this.Dataset00.getColumn(e.row, "amount");
    if(e.cellid == 'Cell04')
    {
        if(cur_amount != 0)
        {
            this.Dataset00.setColumn(e.row, "amount", --cur_amount);
        }
    }
    else if(e.cellid == 'Cell02')
    {
        this.Dataset00.setColumn(e.row, "amount", ++cur_amount);
    }
};
```

4 합계 표시하는 Grid 컴포넌트 설정하기



Grid 컴포넌트에 합계를 표시하는 방법은 아래 예제를 참고하세요.
[영수증 계산하기](#)

5 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 장바구니에 담은 앨범 개수를 변경합니다. 변경된 값이 Grid 컴포넌트에 반영되는지 확인합니다.

29.5 body 밴드 영역 클릭 시 상세 데이터 표현하기

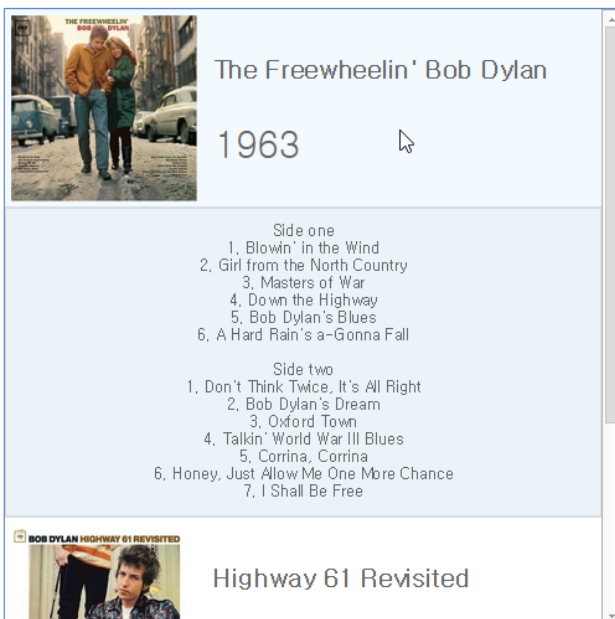
ListView 컴포넌트의 expandbartype 속성을 설정하지 않고 body, detail 영역 클릭 시 상세 데이터를 펼치거나 접을 수 있는 기능을 살펴봅니다.



[기본, 상세 데이터 표현하기](#)에서 사용한 예제를 일부 수정해 사용했습니다.

29.5.1 예제

앨범 정보를 표시합니다. body 영역을 클릭하면 해당 앨범 수록곡(상세 데이터)을 표시합니다. body 영역을 다시 클릭하거나 detail 영역을 클릭하면 상세 데이터 표시 영역을 감춥니다.





29.5.2 예제에서 사용한 핵심 기능

getBandExpandStatus (17.0.0.1800 지원)

특정 아이템의 Detail 밴드 표시 여부를 반환합니다. 아이템의 인덱스(Dataset 오브젝트의 rowposition)값을 지정하면 해당 Detail 밴드가 표시되고 있는지 여부를 반환합니다.

showDetailBand, hideDetailBand

Detail 밴드를 펼치거나 접을 때 사용하는 메소드입니다.

29.5.3 예제 구현 방법

- ① ListView 컴포넌트를 화면에 배치합니다. ListView 컴포넌트의 width, height 속성값은 500으로 지정합니다.
- ② Dataset 오브젝트를 생성하고 아래와 같이 설정합니다. Dataset 오브젝트를 화면에서 드래그해서 ListView 컴포넌트에 바인딩합니다.

Columns					
No	id	type	size	prop	st
1	title	STRING	256		
2	release	STRING	256		
3	cover	STRING	256		
4	songs	STRING	256		
Rows					
No	title	release	cover		
1	The Freewheelin' Bob D...	1963	imagerc::		
2	Highway 61 Revisited	1965	imagerc::		
3	Blood on the Tracks	1975	imagerc::		

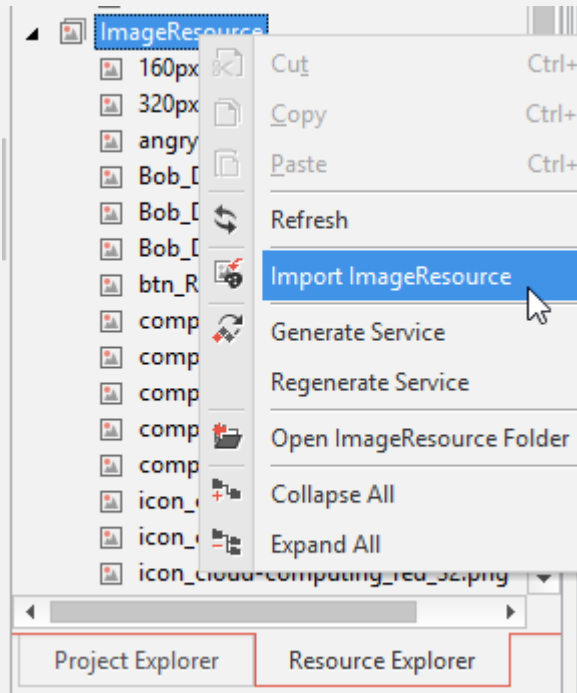


수록곡 데이터가 포함되어 있어 위의 이미지에서는 전체 데이터를 다 보여주지 않습니다. 해당 샘플 코드를 참고하세요.



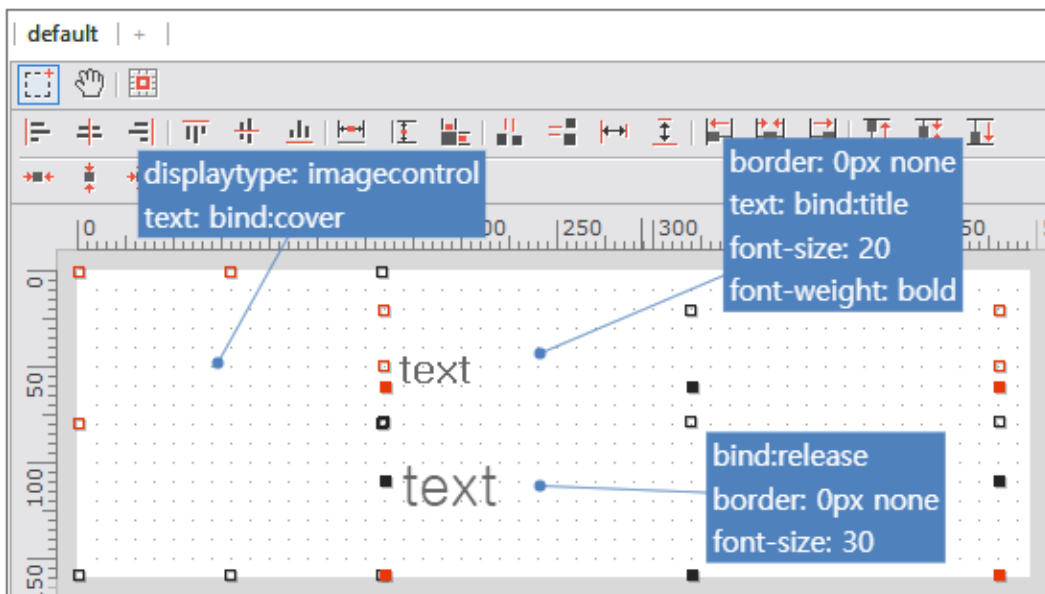
cover 컬럼값에는 앨범 커버 이미지 경로를 지정합니다. 이미지 파일은 Resource Explorer에서 [Import ImageResource] 기능을 사용해 프로젝트에 추가합니다. 추가된 이미지는 아래와 같은 경로를 가집니다.

imagerc::Bob_Dylan_-_The_Freewheelin'_Bob_Dylan.jpg



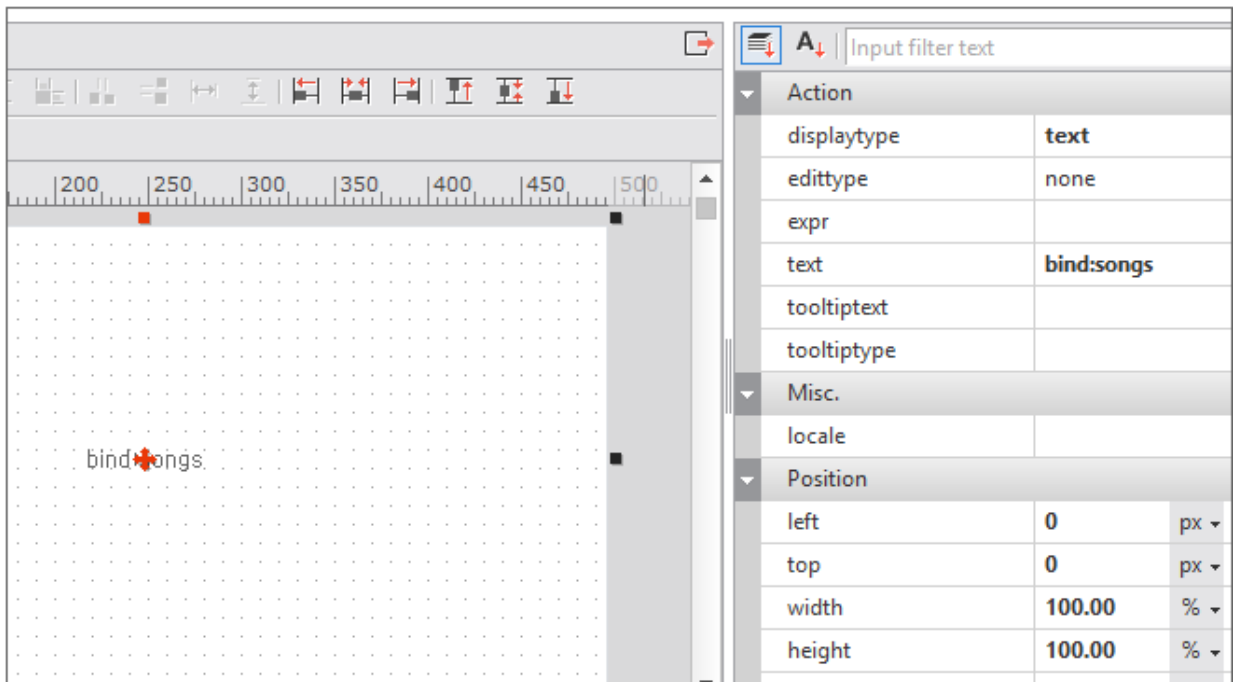
- 3 ListView Contents Editor 실행하고 셀을 배치하고 필요한 속성값을 설정합니다.

Dataset 오브젝트의 값에 따라 컬럼 id, 컬럼 데이터를 ListView 컴포넌트의 셀로 자동 설정합니다. 하지만, 예제에서는 다른 식으로 화면을 구성할 것이기 때문에 Body 밴드 편집 모드로 전환하고 설정된 셀을 모두 삭제합니다. 그리고 height 속성값은 160으로 지정한 후 아래와 같이 3개의 셀을 배치합니다.



④ Detail 밴드 추가하고 필요한 속성값을 설정합니다.

ListView Contents Editor 상단 툴바에서 [Append Detail] 항목을 선택하고 Detail 밴드를 추가합니다. Detail 밴드의 height 속성값은 250으로 지정하고 Detail 밴드 편집 모드로 전환합니다. 셀을 하나 추가하고 셀 속성을 아래와 같이 지정합니다.



⑤ ListView Contents Editor 창을 닫고 ListView 컴포넌트의 bandinitstatus 속성값을 "collapse"로 설정합니다.

⑥ ListView 컴포넌트의 oncellclick 이벤트 함수를 아래와 같이 추가합니다. body, detail 밴드 영역 클릭 시 해당 이벤트가 발생합니다. detail 밴드 영역이 펼쳐져 있는지 확인하고 상태에 따라 showDetailBand 또는 hideDetailBand 메소드를 실행합니다.

```
this.ListView00_oncellclick = function(obj:nexacro.ListView,e:nexacro.
ListViewClickEventInfo)
{
    if(obj.getBandExpandStatus(e.row))
    {
        obj.hideDetailBand(e.row);
    }
    else
    {
        obj.showDetailBand(e.row);
    }
};
```

- 7 QuickView(Ctrl + F6)로 실행한 후 body 밴드 영역을 클릭해서 detail 밴드가 펼쳐지는지 확인합니다.

30.

MaskEdit

30.1 MaskEdit 소개

MaskEdit은 정해진 형식으로 입/출력할 때 사용하는 Edit 계열의 컴포넌트입니다. 기본적인 모양이나 사용법은 Edit 컴포넌트와 유사하며 형식이 정해져 있는 주민등록번호, 전화번호, 날짜 등과 같은 정보를 다룰 때 특히 유용합니다.

MaskEdit을 사용하면 정해진 형식에서의 사용을 강제할 수 있어 입/출력 오류를 줄일 수 있고, 복잡한 문자열 처리 과정을 간단한 마스킹 처리로 대체할 수 있어 편리합니다.

MaskEdit은 type 속성에 따라 number 형식과 string 형식으로 설정할 수 있습니다. number 형식은 숫자 형식으로 입력받아 처리할 경우에 사용하고 string 형식은 문자열 형식으로 입력받아 처리할 수 있습니다.

또한, format 속성의 설정에 따라 입력 문자, 길이, 표시 형식 등을 필요에 따라 제한할 수 있습니다.

30.1.1 사용 예

아래는 어느 포털사이트의 개인 정보 조회 화면입니다. 개인 정보 보호를 위해 아이디, 도메인 주소, 전화번호의 일부를 별표(*)로 표시하거나, 번호의 앞에 지역 번호를 붙이는 등 다양한 형식에서의 입, 출력이 가능합니다.

연락처	
기본 이메일	do*****@n*****,com
본인확인 이메일	do*****@g*****,com
휴대전화	+82 16-2****-****8

30.1.2 MaskEdit 만들기

-1,000,000,000,000

1 MaskEdit 컴포넌트 생성하기

툴바에서 MaskEdit  을 선택한 후, 폼 위에 적당한 크기로 드래그하여 생성합니다. 컴포넌트를 선택 한 상태에서 폼 위에 클릭하면 기본 크기로 생성됩니다.

2 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행합니다. MaskEdit 생성 후 type 속성을 별도로 설정하지 않으면 'number'로 자동 설정되어 숫자와 부호만 입력할 수 있습니다.

30.2 정해진 형식으로 입력받기

format 속성과 type 속성의 설정에 따라 사용자를 정해진 특정 형식으로 입력하도록 유도할 수 있습니다.

30.2.1 예제

다음은 정해놓은 형식으로 입력할 수 있도록 하는 예제입니다.

String :	abcd - 12345
Number :	1,234,567.890



sample_maskedit_02.xfdl

String 입력은 알파벳 4자리와 숫자 5자리를 입력받을 수 있습니다. MaskEdit 중간에 있는 공백과 하이픈은 마스크 설정에 정의되어 있어 사용자가 임의로 수정하거나 지울 수 없습니다. 마스크 설정은 다음과 같습니다.

aaaa - #####

Number 입력은 자리수 제한없는 정수와 소수점 3자리까지 입력받을 수 있습니다. 또 보기 편하도록 정수부 3자리마

다 쉼표(,)를 표시해 줍니다. 마스크 설정은 다음과 같습니다.

#,#.###

30.2.2 예제에서 사용한 핵심 기능

type

MaskEdit에 입력되는 문자 형식을 설정하는 속성입니다. 'number'와 'string' 중 하나로 설정할 수 있는데 아무 설정도 하지 않을 경우 'number'로 동작합니다.

format

입력받은 문자에 적용할 마스크 형식을 설정하는 속성입니다. 마스크 형식을 설정할 때는 다음과 같은 기호를 조합해 사용합니다. 마스크 기호는 type 속성의 설정에 따라 달리 동작할 수 있으므로 유의하십시오.

```
this.MaskEdit00.set_format( "####" );
this.MaskEdit00.set_format( "###,000.00##" );
this.MaskEdit00.set_format( "999,000.0099" );
```

30.2.3 예제 구현 방법

① 화면 구성하기

MaskEdit , Static  컴포넌트를 예제의 그림과 같이 적절히 배치합니다.

② String 형식 MaskEdit 설정하기

문자 4자리, 하이픈(-), 숫자 5자리를 입력받도록 속성을 설정합니다.

속성	값
type	string
format	aaaa - #####

③ Number 형식 MaskEdit 설정하기

소수점 3자리까지 표시할 수 있는 숫자를 입력받고 정수부 세 자리마다 쉼표(,)를 표시하도록 속성을 설정합니다.

속성	값
----	---

속성	값
type	number
format	#,###.###

④ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 문자열을 입력하여 예제와 같이 입, 출력되는지 확인합니다.

30.3 주민등록번호 안 보이게 처리하기

주민등록번호와 같은 개인 정보는 타인이 볼 수 없게 보호 처리가 필요합니다. MaskEdit의 format 속성을 사용하면 타인이 알아볼 수 없도록 손쉽게 보호 처리가 가능합니다.

30.3.1 예제

주민등록번호를 안 보이게 처리할 때 모든 숫자를 별표 처리하면 올바르게 입력했는지 확인할 수 없습니다. 숫자의 일부분을 사용자가 확인할 수 있도록 표시해 주면 입력 오류를 줄일 수 있습니다.

911222 - 2*****



sample_maskedit_03.xfdl

30.3.2 예제에서 사용한 핵심 기능

type

MaskEdit에 입력되는 문자 형식을 설정하는 속성입니다. 'number'와 'string' 중 하나로 설정할 수 있는데 아무 설정도 하지 않으면 'number'로 동작합니다.

format

입력받은 문자에 적용할 마스크 형식을 설정하는 속성입니다. 마스크 형식을 설정할 때는 다음과 같은 기호를 조합해 사용합니다. 마스크 기호는 type 속성의 설정에 따라 달리 동작할 수 있으므로 유의하십시오.


```

this.MaskEdit00.set_format( "####" );
this.MaskEdit00.set_format( "###,000.00##" );
this.MaskEdit00.set_format( "999,000.0099" );

```

30.3.3 예제 구현 방법

① 화면 구성하기

MaskEdit 를 예제의 그림과 같이 적절히 배치합니다.

② MaskEdit 설정하기

앞의 생년월일 숫자 6자리, 하이픈(-) 그리고 뒤의 표시되는 숫자 1자리, 표시되지 않는 숫자 6자리를 입력받기 위해 다음과 같이 설정합니다. 표시되지 않는 숫자는 중괄호({})로 감싸서 입력합니다.

속성	값
type	string
format	##### - #{#####}

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 문자열을 입력하여 예제와 같이 입, 출력되는지 확인합니다.

31.

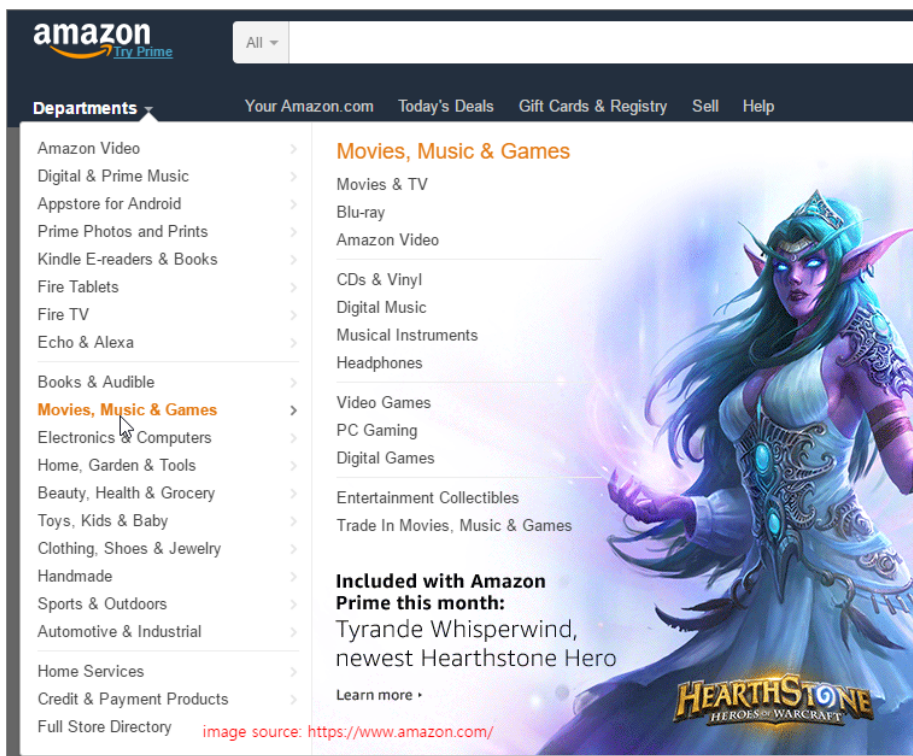
Menu

31.1 Menu 소개

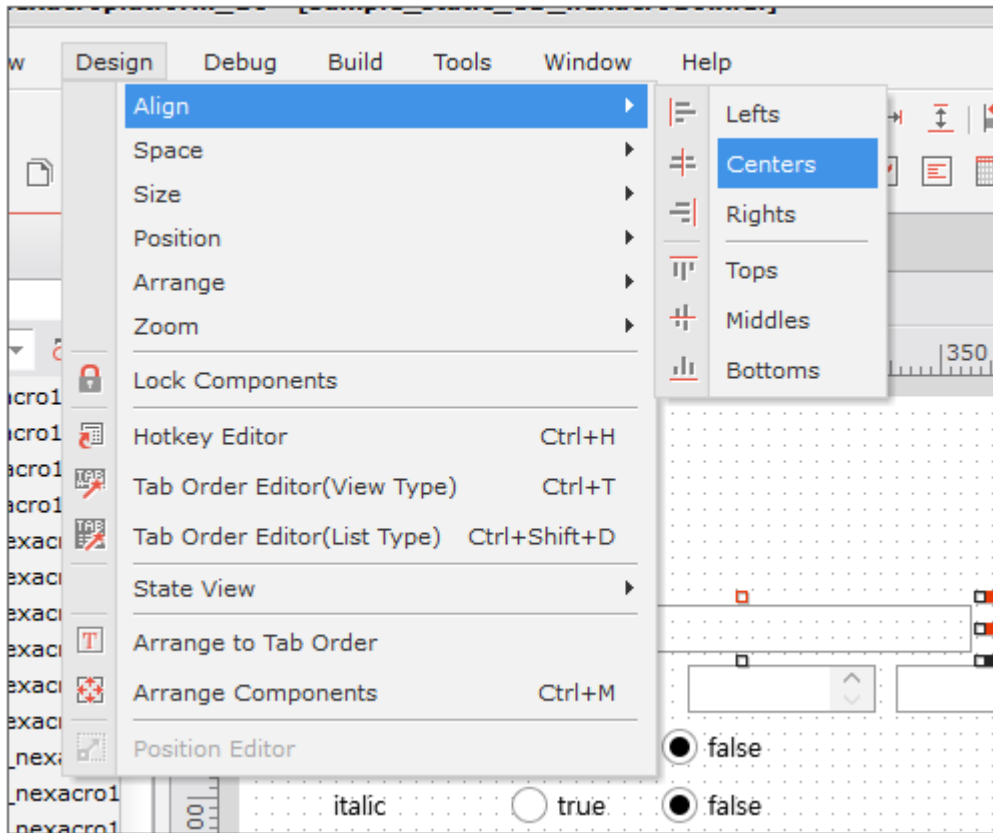
Menu 컴포넌트는 다양한 정보를 가지고 있는 사이트에서 정보를 쉽게 찾아갈 수 있도록 안내를 제공하거나 문서도 구에서 원하는 기능을 바로 실행할 수 있도록 기능 목록을 제공합니다. 최근에는 Menu 컴포넌트를 변형된 형태로 사용하는 경우가 많지만, 정보를 찾아가는 방식 자체는 크게 다르지 않습니다.

31.1.1 사용 예

다양한 정보를 가지는 웹사이트에서 정보를 찾는 방법은 검색창을 사용하거나 메뉴를 사용하는 방식입니다. 특정 제품을 찾는 경우에는 검색창이 빠르겠지만 어떤 제품군 내 상품 목록을 보려하는 경우에는 메뉴에서 원하는 항목을 찾는 방식이 보편적입니다.



넥사크로 스튜디오에서도 원하는 기능을 처리하기 위한 방법 중 하나로 Menu 기능을 사용합니다. 넥사크로 스튜디오와 같은 저작 도구는 메뉴 외에도 단축키, 툴바, 바로가기 등을 사용해 원하는 기능을 처리할 수 있습니다.



31.1.2 Menu 만들기

Menu 컴포넌트는 초기 상태에서는 아무런 동작도 하지 않습니다. Menu의 목적이 정보를 찾아가거나 특정 기능을 수행하는 것이기 때문에 어떤 작업을 처리할지 지정해주어야 합니다.

① Menu 컴포넌트 생성하기

툴바에서 Menu 항목을 선택한 후, Form 위에 드래그 앤 드롭해 생성합니다. 빈 회색 막대가 화면에 표시됩니다.

② innerdataset 설정하기

속성창에서 innerdataset 항목을 찾아 innerdataset 편집창을 실행합니다. 화면에 A, B, C 3개의 메뉴 항목을 표시하고 A 항목을 선택하면 A-1, A-2 2개의 메뉴 항목이 표시되도록 하겠습니다. innerdataset 편집창을 실행하면 입력할 항목이 많은데 이 중에서 captioncolumn, levelcolumn 2개 속성값만 지정합니다.

InnerDataset									
No	captioncolumn	c...	e...	ho...	ico...	id...	levelcolumn	userdatacolumn	
1	A	[...]	[...]	[U...]	[U...]	[U...]	0	[Undefined]	
2	A-1	[...]	[...]	[U...]	[U...]	[U...]	1	[Undefined]	
3	A-2	[...]	[...]	[U...]	[U...]	[U...]	1	[Undefined]	
4	B	[...]	[...]	[U...]	[U...]	[U...]	0	[Undefined]	
5	C	[...]	[...]	[U...]	[U...]	[U...]	0	[Undefined]	

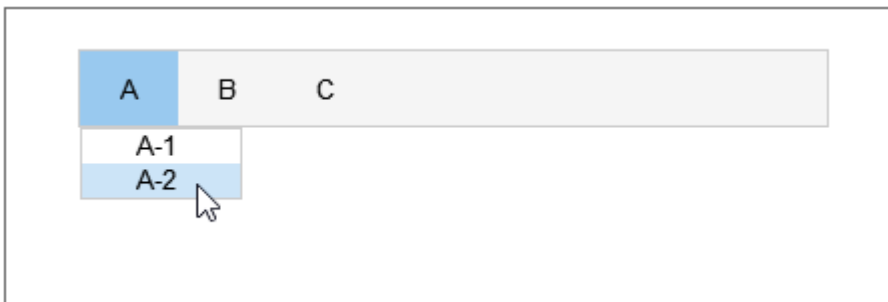
captioncolumn 속성값은 화면에 표시되는 메뉴 항목입니다. levelcolumn 속성값이 0인 경우에는 Menu 컴포넌트가 화면에 보일 때 대메뉴로 보이는 항목입니다. levelcolumn 속성값을 1이상의 값으로 지정하면 속성값이 0인 항목을 선택했을 때 세부 메뉴 항목으로 보입니다.



Menu 컴포넌트는 메뉴 레벨 구조를 순서대로 처리하기 때문에 "A" 항목을 선택했을 때 "A-1"이 표시되도록 하려면 innerdataset 속성값에서 "A" 항목 아래 "A-1" 항목이 지정되어야 합니다.

3 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행합니다. "A" 항목을 선택하면 하위 메뉴 항목을 확인할 수 있습니다.

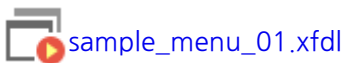


31.2 메뉴 구성하기

Menu 컴포넌트는 정보를 좀 더 쉽고 직관적으로 찾을 수 있도록 몇 가지 추가적인 옵션을 제공합니다. 간단한 메뉴라면 captioncolumn, levelcolumn 속성값만 지정해서 기본 메뉴를 제공할 수 있습니다.

31.2.1 예제

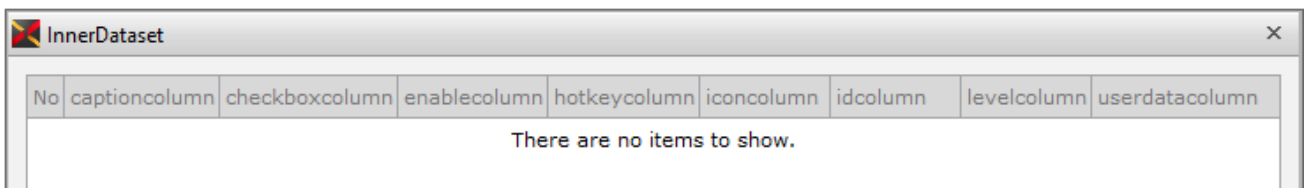
주류와 음료를 판매하는 쇼핑몰에서 제공하는 메뉴입니다. 사용자가 19세 미만이면 주류는 선택할 수 없고 19세 이상이면 모든 메뉴를 선택할 수 있습니다. "Age" 항목에서 나이를 선택하고 선택된 항목은 체크박스로 표시합니다.



31.2.2 예제에서 사용한 핵심 기능

innerdataset

Menu 컴포넌트에서 별도 Dataset 오브젝트를 사용하지 않고 innerdataset 편집창을 사용한다면 자동으로 컬럼항목을 생성하고 Menu 컴포넌트의 관련 속성값(captioncolumn, checkboxcolumn 등)도 지정합니다.



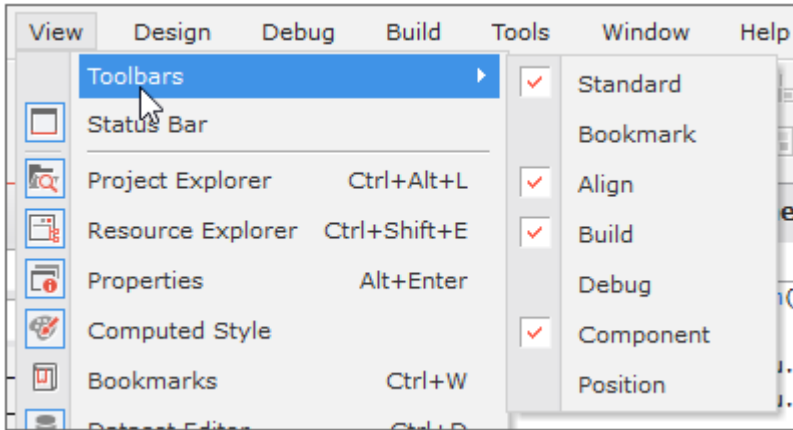
innerdataset을 직접 지정해 사용하는 경우에는 입력한 데이터를 스크립트에서 수정할 수 없습니다. 그래서 이번 예제에서는 같은 형식으로 Dataset 오브젝트를 만들어 사용했습니다.



this.menuOption._innerdataset 와 같은 방식으로 접근해 Dataset 오브젝트처럼 사용할 수는 있지만, 공식적으로 허용된 방식이 아니므로 이번 예제에서는 적용하지 않았습니다.

checkboxcolumn

해당 컬럼값이 true인 경우에는 메뉴 항목 앞에 체크박스를 표시합니다. 선택된 상태를 지속적으로 표기하기 위해 사용합니다. 넥사크로 스튜디오에서 메뉴[View > Toolbars] 항목을 보면 사용하는 것으로 선택한 툴바 앞에 체크박스가 표시되어 있습니다.



enablecolumn

해당 컬럼값이 false인 경우에는 메뉴 항목을 사용할 수 없습니다. 하위 메뉴가 있는 경우에도 하위 메뉴가 펼쳐지지 않고 메뉴를 선택하더라도 이벤트가 발생하지 않습니다.

idcolumn, userdatacolumn

화면에 표시되지는 않지만 부가적인 정보 제공이 필요할 때 사용합니다. onmenuclick 이벤트 발생 시 idcolumn과 userdatacolumn에 해당하는 값은 아래와 같이 확인할 수 있습니다.

```
this.menuOption_onmenuclick = function(obj:nexacro.Menu,e:nexacro.MenuClickEventInfo)
{
    trace(e.id); // idcolumn
    trace(e.userdata); // userdatacolumn
};
```

findRow

Menu 컴포넌트는 선택된 항목에 대한 Dataset 오브젝트의 Row 정보를 제공하지 않아서 컬럼값과 실제 데이터값을 가지고 해당하는 Row 정보를 찾습니다. findRow 메소드는 컬럼값, 데이터값을 가지고 Row 정보를 찾을 수 있습니다. 같은 값이 여러 개 있는 경우에는 첫 번째 값만을 반환하며 시작 인덱스값을 추가로 지정해 찾는 범위를 조정할 수 있습니다.

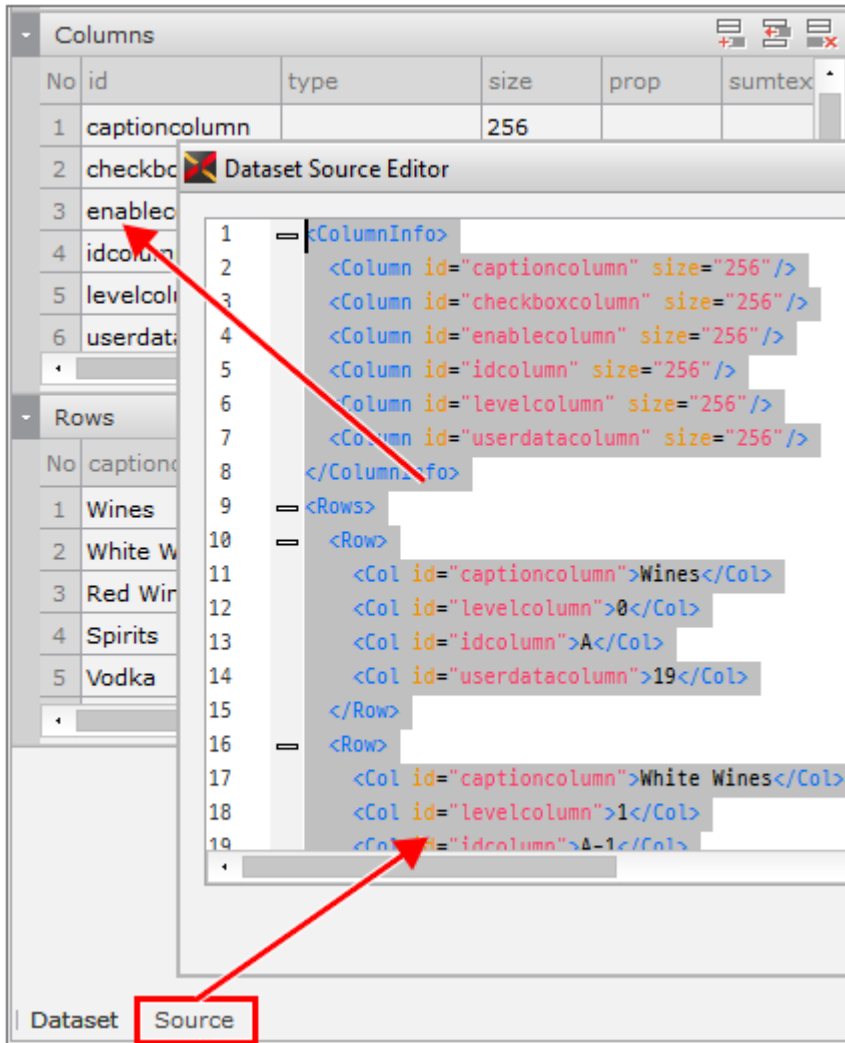
31.2.3 예제 구현 방법

① 화면 구성하기

Menu 컴포넌트를 예제 화면과 같이 배치합니다. innerdataset 속성값을 지정하기 전에는 회색막대만 보일 겁니다.

② Dataset 오브젝트 생성하기

툴바에서 Dataset 오브젝트를 "invisible object" 영역에 추가합니다. Dataset 오브젝트를 더블클릭하고 [Dataset Editor]에 데이터를 추가합니다. [Dataset Editor] 하단탭에서 "Source"를 선택하면 [Dataset Source Editor]가 실행되어 직접 XML 형식의 데이터를 입력할 수 있습니다.



```
<ColumnInfo>
  <Column id="captioncolumn" size="256"/>
  <Column id="checkboxcolumn" size="256"/>
  <Column id="enablecolumn" size="256"/>
  <Column id="idcolumn" size="256"/>
  <Column id="levelcolumn" size="256"/>
  <Column id="userdatacolumn" size="256"/>
</ColumnInfo>
<Rows>
  <Row>
    <Col id="captioncolumn">Wines</Col>
    <Col id="levelcolumn">0</Col>
    <Col id="idcolumn">A</Col>
```

```

    <Col id="userdatacolumn">19</Col>
  </Row>
  <Row>
    <Col id="captioncolumn">White Wines</Col>
    <Col id="levelcolumn">1</Col>
    <Col id="idcolumn">A-1</Col>
  </Row>
  <Row>
    <Col id="captioncolumn">Red Wines</Col>
    <Col id="levelcolumn">1</Col>
    <Col id="idcolumn">A-2</Col>
  </Row>
  <Row>
    <Col id="captioncolumn">Spirits</Col>
    <Col id="levelcolumn">0</Col>
    <Col id="idcolumn">B</Col>
    <Col id="userdatacolumn">19</Col>
  </Row>
  <Row>
    <Col id="captioncolumn">Vodka</Col>
    <Col id="levelcolumn">1</Col>
    <Col id="idcolumn">B-1</Col>
  </Row>
  <Row>
    <Col id="captioncolumn">Plain</Col>
    <Col id="levelcolumn">2</Col>
    <Col id="idcolumn">B-1-1</Col>
  </Row>
  <Row>
    <Col id="captioncolumn">Flavoured</Col>
    <Col id="levelcolumn">2</Col>
    <Col id="idcolumn">B-1-2</Col>
  </Row>
  <Row>
    <Col id="captioncolumn">Cognac</Col>
    <Col id="levelcolumn">1</Col>
    <Col id="idcolumn">B-2</Col>
  </Row>
  <Row>
    <Col id="captioncolumn">Rum</Col>
    <Col id="levelcolumn">1</Col>

```



```

    <Col id="idcolumn">B-3</Col>
</Row>
<Row>
    <Col id="captioncolumn">Tequila</Col>
    <Col id="levelcolumn">1</Col>
    <Col id="idcolumn">B-4</Col>
</Row>
<Row>
    <Col id="captioncolumn">Soft Drink</Col>
    <Col id="levelcolumn">0</Col>
    <Col id="idcolumn">C</Col>
</Row>
<Row>
    <Col id="captioncolumn">Coke</Col>
    <Col id="levelcolumn">1</Col>
    <Col id="idcolumn">C-1</Col>
</Row>
<Row>
    <Col id="captioncolumn">Juice</Col>
    <Col id="levelcolumn">1</Col>
    <Col id="idcolumn">C-2</Col>
</Row>
<Row>
    <Col id="captioncolumn">Age</Col>
    <Col id="idcolumn">D</Col>
    <Col id="levelcolumn">0</Col>
</Row>
<Row>
    <Col id="captioncolumn">Under 19</Col>
    <Col id="idcolumn">D-1</Col>
    <Col id="levelcolumn">1</Col>
</Row>
<Row>
    <Col id="captioncolumn">19+</Col>
    <Col id="checkboxcolumn">true</Col>
    <Col id="idcolumn">D-2</Col>
    <Col id="levelcolumn">1</Col>
</Row>
</Rows>

```

3 innerdataset 속성값 설정하기

생성한 Dataset 오브젝트를 innerdataset 속성값으로 지정하고 captioncolumn, checkboxcolumn, enablecolumn, idcolumn, levelcolumn, userdatacolumn 속성값을 지정합니다.



Dataset 오브젝트의 컬럼값이 Menu 컴포넌트의 속성값과 같을 필요는 없습니다. 다만 개발 편의를 위해 속성값과 컬럼값을 같게 설정했습니다. 실제 개발 단계에서는 내부 코딩 규정에 따라 항목값을 처리할 수 있습니다.

4 Menu 컴포넌트 이벤트 함수 작성하기

Menu 컴포넌트를 선택한 후 onmenuclick 이벤트 함수를 작성합니다. 이벤트 함수 내에서는 "Age" 하위 메뉴 선택에 따라 19세 미만이면 사용할 수 없는 메뉴를 설정합니다.

```
this.menuOption_onmenuclick = function(obj:nexacro.Menu,e:nexacro.MenuClickEventInfo)
{
    var d1ColumnRow = this.datasetMenu.findRow("idcolumn", "D-1");
    var d2ColumnRow = this.datasetMenu.findRow("idcolumn", "D-2");

    if(e.id == "D-1") {
        this.datasetMenu.setColumn(d1ColumnRow, "checkboxcolumn", true);
        this.datasetMenu.setColumn(d2ColumnRow, "checkboxcolumn", false);

        var currentAgeColumnRow = -1;
        do {
            currentAgeColumnRow++;
            currentAgeColumnRow = this.datasetMenu.findRow("userdatacolumn", "19",
currentAgeColumnRow);
            this.datasetMenu.setColumn(currentAgeColumnRow, "enablecolumn", false);
        } while (currentAgeColumnRow != -1);
    } else if(e.id == "D-2") {
        this.datasetMenu.setColumn(d1ColumnRow, "checkboxcolumn", false);
        this.datasetMenu.setColumn(d2ColumnRow, "checkboxcolumn", true);

        var currentAgeColumnRow = -1;
        do {
            currentAgeColumnRow++;
            currentAgeColumnRow = this.datasetMenu.findRow("userdatacolumn", "19",
currentAgeColumnRow);
            this.datasetMenu.setColumn(currentAgeColumnRow, "enablecolumn", true);
        } while (currentAgeColumnRow != -1);
    }
}
```

```
};
```

예제에서 "Age" 하위 메뉴 2개는 변하지 않는 항목이기 때문에 해당 항목의 Row 정보를 찾아 d1ColumnRow, d2ColumnRow 변수에 저장해놓았습니다. 그리고 조건에 따라 해당하는 Row의 checkboxcolumn 항목 값을 수정하도록 했습니다.

```
this.datasetMenu.setColumn(d1ColumnRow, "checkboxcolumn", true);
this.datasetMenu.setColumn(d2ColumnRow, "checkboxcolumn", false);
```

주류 메뉴는 사용하는 데이터에 따라 늘어날 수 있으므로 userdatacolumn 항목에 "19"라는 정보를 입력하고 해당값을 가진 데이터는 모두 주류 메뉴로 처리하도록 했습니다. 그러기 위해 do~while 문을 사용했습니다. 예제에서는 2개의 메뉴를 주류 메뉴로 지정했는데 해당하는 Row 정보를 currentAgeColumnRow 라는 변수에 담아 처리합니다.

첫 번째 do~while 문이 실행되는 동안에는 아래와 같이 값이 처리됩니다. findRow 메소드는 시작 인덱스를 기준으로 지정된 컬럼값과 가장 먼저 일치하는 Row를 찾습니다. 시작 인덱스(currentAgeColumnRow)값은 0부터 시작하는데 findRow 메소드를 실행해서 0번째 인덱스에서 같은 값을 찾았기 때문에 currentAgeColumnRow은 여전히 0이 됩니다. 그리고 enablecolumn 컬럼값을 false로 업데이트합니다.

```
var currentAgeColumnRow = -1; // (do 구문에서 0번째 Row부터 시작하기 위해 초기값을 -1로 지정)
do {
    currentAgeColumnRow++; // 0 (이전에 찾은 값 또는 초기값 다음 Row부터 findRow 메소드를 처리하기 위해 ++ 연산자 사용)
    currentAgeColumnRow = this.datasetMenu.findRow("userdatacolumn", "19", currentAgeColumnRow); // 0 (조건에 맞는 Row 찾기)
    this.datasetMenu.setColumn(currentAgeColumnRow, "enablecolumn", false); // true (조건에 맞는 Row의 enablecolumn 값을 false로 수정)
} while (currentAgeColumnRow != -1); // true (do 구문에서 조건에 맞는 Row를 찾았다면 한 번 더 반복)
```

No	captioncolu...	checkboxcolumn	enablecolumn	idcolumn	levelcolumn	userdatacolumn
1	Wines	[Undefined]	[Undefined]	A	0	19
2	White Wines	[Undefined]	[Undefined]	A-1	1	[Undefined]
3	Red Wines	[Undefined]	[Undefined]	A-2	1	[Undefined]

currentAgeColumnRow -> 0

this.Dataset00.setColumn(0, "enablecolumn", false)

this.Dataset00.findRow("userdatacolumn", "19", 0) -> 0



++ 연산자는 변수값을 1씩 증가시키는 연산자입니다.
예를 들어 초기값이 0인 변수 i가 있을 때 i++ 라는 연산을 실행하면 i값은 1이 됩니다.



++ 연산자는 위치에 따라 처리 방식이 달라집니다. ++ 연산자가 변수 뒤에 붙으면 변수값을 나중에 증가시키고 ++ 연산자가 변수 앞에 붙으면 변수값을 먼저 증가시킵니다. 아래 코드를 참고하세요.

```
// i++
var i=0;
trace(i++); //0
trace(i); //1

//++i
var i=0;
trace(++i); //1
trace(i); //1
```

do~while 문은 while 구문 내의 값이 false이면 반복을 멈추는데 아직 true이므로 한 번 더 반복합니다.

```
do {
    currentAgeColumnRow++; // 1 (이전에 찾은 값이 0이었기 때문에 ++ 연산 후 다음 Row인 1부터 시작)
    currentAgeColumnRow = this.datasetMenu.findRow("userdatacolumn", "19",
currentAgeColumnRow); // 3 (Row 값이 1,2인 경우에는 userdatacolumn 값이 없고 Row 값이 3일
때 찾을)
    this.datasetMenu.setColumn(currentAgeColumnRow, "enablecolumn", false); // true
} while (currentAgeColumnRow != -1); // true
```

No	captioncolu...	checkboxcolumn	enablecolumn	idcolumn	levelcolumn	userdatacolumn
1	Wines	[Undefined]	[Undefined]	A	0	19
2	White Wines	[Undefined]	[Undefined]	A-1	1	[Undefined]
3	Red	currentAgeColumnRow -> 1	[Undefined]	A-2	1	[Undefined]
4	Spirits	[Undefined]	[Undefined]	B	0	19
5	Vodka	[Undefined]	[Undefined]	B-1	1	[Undefined]
6	Plain	[Undefined]	[Undefined]	B-1-1	2	[Undefined]

this.Dataset00.findRow("userdatacolumn", "19", 1) -> 3

this.Dataset00.setColumn(3, "enablecolumn", false)

아직 while 구문의 값이 true이기 때문에 한 번 더 반복합니다. 하지만 이번에는 해당하는 값을 찾지 못하고 반복을 멈추게 됩니다. 하위메뉴의 enablecolumn 값은 변경하지 않았는데 상위메뉴의 enablecolumn 값이 false인 경우에는 하위메뉴 자체에 접근할 수 없으므로 상위메뉴만 수정했습니다.

```
do {
    currentAgeColumnRow++; // 4
    currentAgeColumnRow = this.datasetMenu.findRow("userdatacolumn", "19",
currentAgeColumnRow); // -1 (Row 값이 4,5인 경우 userdatacolumn값을 찾지 못하고 -1을 반환)
    this.datasetMenu.setColumn(currentAgeColumnRow, "enablecolumn", false); // false
} while (currentAgeColumnRow != -1); // false (currentAgeColumnRow 값이 -1이 아니라는
조건이 성립되지 못해 반복을 멈추고 다음으로 진행)
```

5 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 "Age" 하위 메뉴를 변경한 후 나머지 메뉴의 활성화 여부를 확인해봅니다. "Under 19" 메뉴를 선택한 경우에는 주류 관련 메뉴를 선택할 수 없습니다.

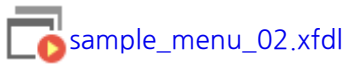
31.3 단축키로 메뉴 실행하기

넥사크로 스튜디오에서 CTRL+N 키 입력 시 Form Wizard가 실행되는 것처럼 Menu 컴포넌트에서 각 메뉴 아이템에 단축키를 설정하고 단축키 입력 시 원하는 동작을 실행할 수 있습니다.

31.3.1 예제

"SHOW BUTTON", "HIDE BUTTON" 메뉴 항목 클릭 시 Button 컴포넌트가 보였다가 사라집니다. 메뉴를 펼쳐서 선택하지 않아도 지정된 단축키 입력 시 같은 동작을 처리합니다. ALT+1 단축키를 입력하면 Button 컴포넌트가 보이고 ALT+2 단축키를 입력하면 Button 컴포넌트가 사라집니다.





31.3.2 예제에서 사용한 핵심 기능

autohotkey

hotkeycolumn 속성으로 설정한 단축키 입력 시 onmenuclick 이벤트가 발생할지 여부를 설정합니다. autohotkey 속성값이 false라면 단축키를 입력해도 이벤트가 발생하지 않습니다.

hotkeycolumn

단축키를 설정한 컬럼을 지정합니다. 컬럼 데이터는 "ALT+1"처럼 문자열로 입력합니다. 속성값을 지정하면 메뉴 아이템 텍스트와 함께 단축키 텍스트를 같이 표시합니다.

31.3.3 예제 구현 방법

- ① Menu 컴포넌트와 Button 컴포넌트를 배치합니다.
- ② Menu 컴포넌트의 autohotkey 속성값을 true로 설정합니다.
- ③ innerdataset 속성값을 아래와 같이 설정합니다.

InnerDataset								
No	captioncolumn	ch...	ena...	hotkeycolumn	ico...	idcolumn	levelcolumn	userd...
1	BUTTON CONTROL	[Un...	[Un...	[Undefined]	[Un...	[Undefined]	0	[Unde...
2	SHOW BUTTON	[Un...	[Un...	ALT+1	[Un...	1	1	[Unde...
3	HIDE BUTTON	[Un...	[Un...	ALT+2	[Un...	2	1	[Unde...

- ④ captioncolumn, hotkeycolumn 등 속성값은 innerdataset 데이터를 설정하면 자동으로 채워집니다. checkboxcolumn 처럼 사용하지 않은 속성값은 지워줍니다.

Binding	
captioncolumn	captioncolumn
checkboxcolumn	
enablecolumn	
hotkeycolumn	hotkeycolumn
iconcolumn	
idcolumn	idcolumn
innerdataset	innerdataset
levelcolumn	levelcolumn
userdatacolumn	

- ⑤ Menu 컴포넌트의 onmenuclick 이벤트 함수를 아래와 같이 추가합니다.

단축키 입력 시 onmenuclick 이벤트가 발생하며 단축키에 해당하는 idcolumn 값이 MenuClickEventInfo 오브젝트의 id 값으로 전달됩니다.

```
this.Menu00_onmenuclick = function(obj:nexacro.Menu,e:nexacro.MenuClickEventInfo)
{
    if(e.id == "1")
        this.Button00.set_visible(true);
    else if(e.id == "2")
        this.Button00.set_visible(false);
};
```

- ⑥ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 메뉴 항목을 클릭해서 버튼 보여주기 여부를 처리할 수 있는지 확인합니다. 이번에는 메뉴를 건드리지 않고 단축키 (ALT+1, ALT+2)만 입력해서 버튼 보여주기 여부가 처리되는지 확인합니다.

32.

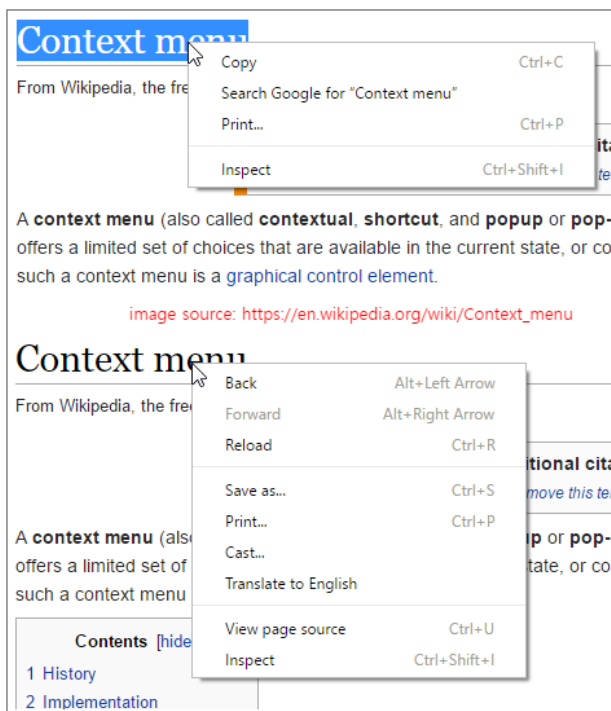
PopupMenu

32.1 PopupMenu 소개

PopupMenu 컴포넌트는 웹브라우저에서 오른쪽 마우스를 눌렀을 때 표시되는 컨텍스트 메뉴와 비슷한 기능을 수행합니다. 평소에는 화면에 보이지 않다가 필요할 때 나타납니다. Menu 컴포넌트에서도 대메뉴 선택 시 나타나는 하위 메뉴 영역은 PopupMenu 컴포넌트를 사용하고 있습니다.

32.1.1 사용 예

웹브라우저나 개발도구 등에서 특정 기능이 필요할 때 처리할 수 있도록 PopupMenu 컴포넌트를 사용합니다. 컨텍스트 메뉴, 팝업 메뉴, 팝업 등의 이름으로 불리기도 합니다. 사용자의 현재 상태에 따라 메뉴 항목을 다르게 표기할 수 있습니다. 예를 들어 웹브라우저에서 특정 문자열을 선택하고 메뉴를 호출하면 복사, 검색, 인쇄 등의 기능을 보여 주고 그렇지 않은 경우에는 페이지 이동, 소스보기 등의 기능을 표시합니다.



32.1.2 PopupMenu 만들기

PopupMenu 컴포넌트는 초기 상태에서는 아무런 동작도 하지 않습니다. 심지어는 화면에 보이지도 않습니다 (넥사 크로 스튜디오에서는 사각형 모양으로 보이는데 실제 실행해보면 아무것도 보이지 않습니다). 다른 컴포넌트와 다르게 실제 화면에 PopupMenu 컴포넌트를 보이게 하려면 한 가지 추가적인 단계가 필요합니다.

① PopupMenu 컴포넌트 생성하기

툴바에서 PopupMenu 항목을 선택한 후, Form 위에 드래그 앤 드롭해 생성합니다. 빈 흰색 상자가 화면에 표시됩니다.

② innerdataset 설정하기

속성창에서 innerdataset 항목을 찾아 innerDataset 편집창을 실행합니다. 화면에 A, B, C 3개의 메뉴 항목을 표시하고 A 항목을 선택하면 A-1, A-2 2개의 메뉴 항목이 표시되도록 하겠습니다. innerDataset 편집창을 실행하면 입력할 항목이 많은데 이 중에서 captioncolumn, levelcolumn 2개 속성값만 지정합니다.

InnerDataset									
No	captioncolumn	c...	e...	ho...	ico...	id...	levelcolumn	userdatacolumn	
1	A	[...]	[...]	[U...]	[U...]	[U...]	0	[Undefined]	
2	A-1	[...]	[...]	[U...]	[U...]	[U...]	1	[Undefined]	
3	A-2	[...]	[...]	[U...]	[U...]	[U...]	1	[Undefined]	
4	B	[...]	[...]	[U...]	[U...]	[U...]	0	[Undefined]	
5	C	[...]	[...]	[U...]	[U...]	[U...]	0	[Undefined]	

captioncolumn 속성값은 화면에 표시되는 메뉴 항목입니다. levelcolumn 속성값이 0인 경우에는 Menu 컴포넌트가 화면에 보이면서 먼저 보이는 항목입니다. levelcolumn 속성값이 1 이상인 경우에는 속성값이 0인 항목을 선택했을 때 세부 메뉴 항목으로 보이는 항목을 지정합니다.



PopupMenu 컴포넌트는 메뉴 레벨 구조를 순서대로 처리하기 때문에 "A" 항목을 선택했을 때 "A-1"이 표시되도록 하려면 innerdataset 속성값에서 "A" 항목 아래 "A-1" 항목이 지정되어야 합니다.

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행합니다. 화면에 아무것도 나타나지 않습니다.

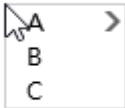
④ Form 오브젝트의 onrbuttondown 이벤트 추가하기

마우스 오른쪽 마우스를 클릭하면 PopupMenu 컴포넌트가 화면에 나타나도록 onrbuttondown 이벤트를 함수를 추가합니다. PopupMenu 컴포넌트의 trackPopupByComponent 메소드를 사용합니다.

```
this.Form_onrbuttondown = function(obj:nexacro.Form,e:nexacro.MouseEventInfo)
{
    this.PopupMenu00.trackPopupByComponent(this, e.clientx, e.clienty);
};
```

5 QuickView로 확인하기

다시 QuickView(Ctrl + F6)로 실행합니다. 화면에 아무것도 나타나지 않지만 오른쪽 마우스를 클릭하면 메뉴가 나타납니다.

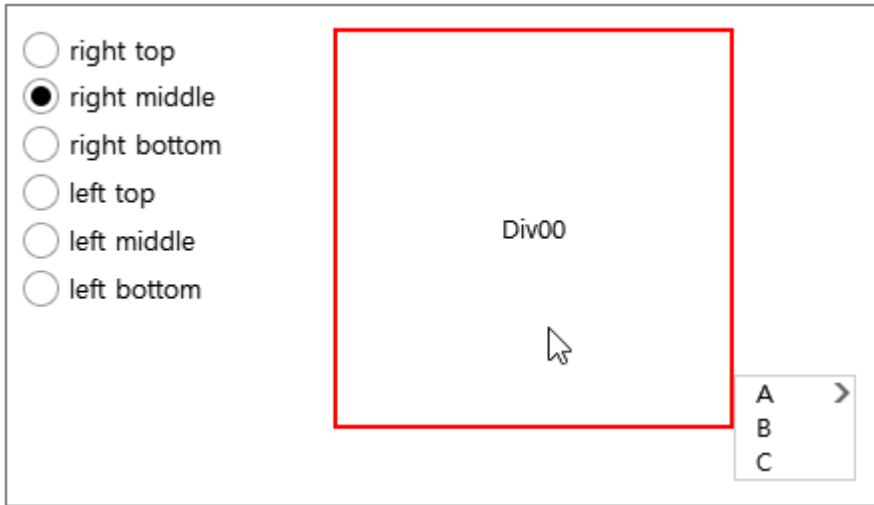


32.2 팝업 위치 변경하기

일반적인 팝업 메뉴는 마우스 클릭 시 마우스 포인터의 오른쪽 아래쪽에 팝업 메뉴가 위치하도록 처리합니다. 하지만 사용하는 화면의 기능에 따라 팝업 위치를 다른 곳에 배치해야 하는 경우도 있습니다. 그럴 때 팝업 위치를 어떻게 변경할 수 있는지 살펴봅니다.

32.2.1 예제

화면에 보이는 Div 컴포넌트 클릭 시 Div 컴포넌트의 오른쪽 하단 꼭지점을 기준으로 PopupMenu 컴포넌트가 배치됩니다. Radio 컴포넌트로 설정한 정렬 조건을 선택하면 해당 조건에 따라 PopupMenu 컴포넌트의 위치가 변경됩니다.



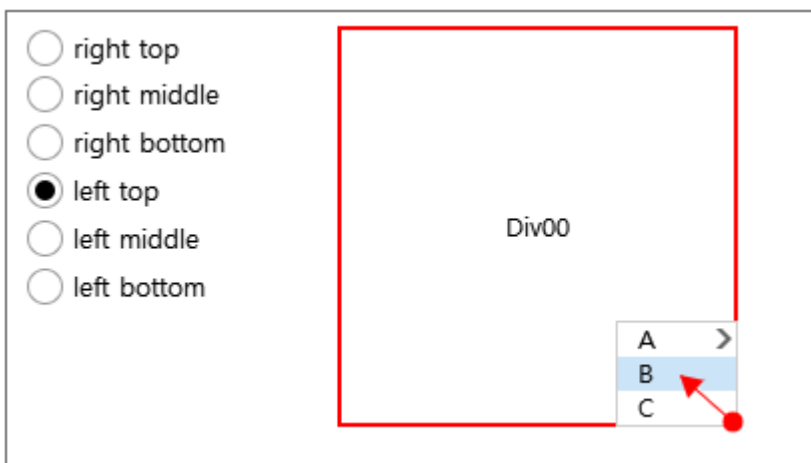
32.2.2 예제에서 사용한 핵심 기능

trackPopupByComponent (objComp, nXpos, nYpos...)

첫 번째 인자로 지정한 컴포넌트 또는 오브젝트를 기준으로 팝업 메뉴를 보여줄 위치를 지정합니다. 예제에서는 Div 컴포넌트를 인자로 지정했기 때문에 Div 컴포넌트의 top, left 좌표값을 (0, 0)으로 설정하고 지정된 좌표값에 팝업 메뉴를 표시합니다.

trackPopupByComponent (...strAlign...)

팝업 메뉴를 보여주는 좌표점에서 팝업 메뉴의 가로, 세로 정렬 방식을 지정합니다. default 값은 "right bottom"입니다. 이때 사용하는 방향은 좌표점을 기준으로 생각해야 합니다. 좌표점이 Div 컴포넌트의 오른쪽 아래 꼭지점이라면 default 정렬값을 적용해 해당 꼭지점 오른쪽 아래에 팝업 메뉴가 나타납니다. 반대로 정렬값을 "left top"이라고 설정한다면 아래 그림과 같이 Div 컴포넌트 안쪽에 팝업 메뉴가 나타납니다.



32.2.3 예제 구현 방법

① 화면 구성하기

Radio, Div, PopupMenu 컴포넌트를 예제 화면과 같이 배치합니다. Radio 컴포넌트의 innerdataset 속성값은 아래와 같이 지정합니다. Div 컴포넌트는 눈에 잘 보이도록 border 속성값을 "2px solid red"로 지정했습니다.

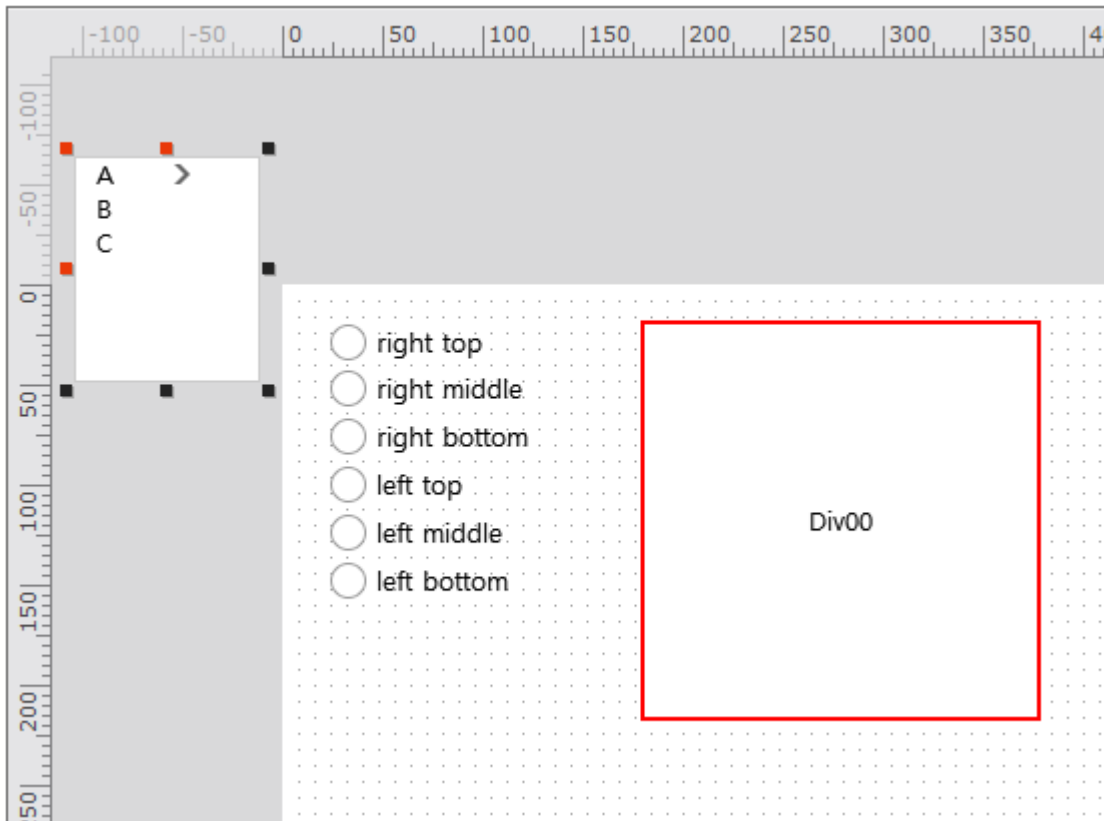
InnerDataset		
No	codecolumn	datacolumn
1	right top	right top
2	right middle	right middle
3	right bottom	right bottom
4	left top	left top
5	left middle	left middle
6	left bottom	left bottom

PopupMenu 컴포넌트의 innerdataset 속성값은 아래와 같이 지정합니다. 입력해야 하는 값은 captioncolumn, levelcolumn 값입니다.

InnerDataset								
No	captioncolumn	c...	e...	ho...	ico...	id...	levelcolumn	userdatacolumn
1	A	[...]	[...]	[U...]	[U...]	[U...]	0	[Undefined]
2	A-1	[...]	[...]	[U...]	[U...]	[U...]	1	[Undefined]
3	A-2	[...]	[...]	[U...]	[U...]	[U...]	1	[Undefined]
4	B	[...]	[...]	[U...]	[U...]	[U...]	0	[Undefined]
5	C	[...]	[...]	[U...]	[U...]	[U...]	0	[Undefined]



화면에 컴포넌트를 배치할 때 PopupMenu 컴포넌트의 위치는 신경쓰지 않아도 됩니다. 넥사크로 스튜디오에서 화면 편집 시 편의를 위해 보이는 것일 뿐 실제 실행 화면에서는 보이지 않습니다. Form 영역 내에 있는 것이 신경쓰인다면 아예 Form 영역 밖으로 내보내도 괜찮습니다.



2 Radio 컴포넌트 이벤트 함수 작성하기

Radio 컴포넌트를 선택한 후 onitemchanged 이벤트 함수를 작성합니다. Radio 컴포넌트에서 항목 선택 시 align이라는 변수에 정렬 조건값을 저장합니다. 저장된 값은 Div 컴포넌트의 onclick 이벤트에서 사용합니다.

```
var align;
this.radioAlign_onitemchanged = function(obj:nexacro.Radio,e:nexacro.ItemChangeEventInfo)
{
    align = obj.value;
};
```

3 Div 컴포넌트 이벤트 함수 작성하기

Div 컴포넌트를 선택한 후 onclick 이벤트 함수를 작성합니다. 팝업 메뉴가 표시되는 기준점은 Div 컴포넌트이고 Div 컴포넌트의 너비와 높이를 X, Y 좌표값으로 지정하면 Div 컴포넌트의 좌표값을 기준으로 X, Y 좌표값 만큼 이동해서 오른쪽 아래 꼭지점을 기준으로 팝업 메뉴가 표시됩니다. 여기에 정렬 조건을 선택한 경우 해당 조건이 반영될 수 있도록 조건식을 추가합니다.

```
this.divClick onclick = function(obj:nexacro.Div,e:nexacro.ClickEventInfo)
{
    var targetX = obj.width;
    var targetY = obj.height;
```

```

if(align) {
    this.popupmenuABC.trackPopupByComponent(obj, targetX, targetY, align);
} else {
    this.popupmenuABC.trackPopupByComponent(obj, targetX, targetY);
}
};

```

4 QuickView로 확인하기

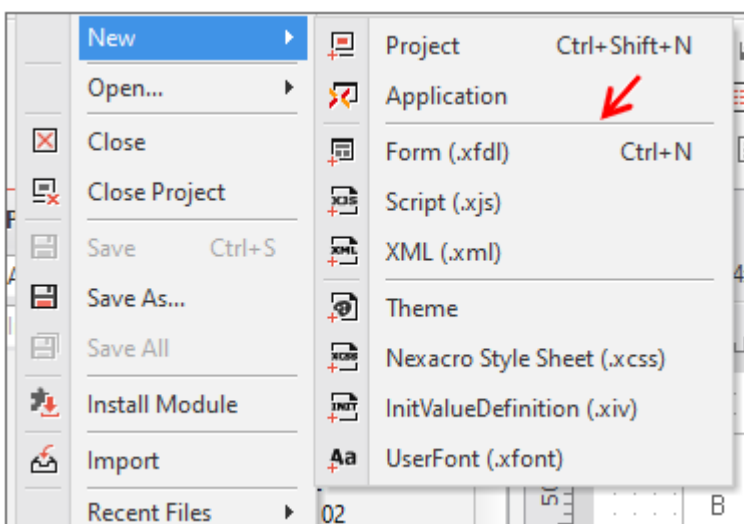
QuickView(Ctrl + F6)로 실행한 후 Div 컴포넌트 영역을 오른쪽 마우스로 클릭해 팝업 메뉴가 나타나는 것을 확인합니다.



웹브라우저에서 실행하는 경우 onbuttondown 이벤트를 사용한다면 오른쪽 마우스 클릭 시 웹 브라우저에서 지정된 컨텍스트 메뉴가 나타나면서 PopupMenu 컴포넌트를 가립니다. 웹브라우저 환경에서 동작하는 애플리케이션이라면 PopupMenu를 실행하는 이벤트로 onbuttondown를 사용하는 것은 문제가 생길 수 있습니다.

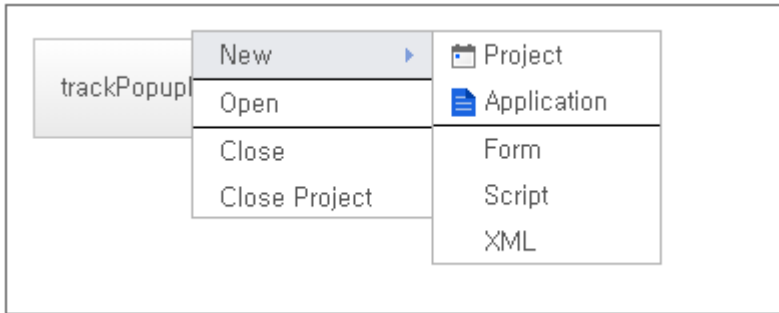
32.3 구분선 추가하기

넥사크로 스튜디오 메뉴를 열어보면 특정 메뉴 항목들을 묶어서 표시하고 중간에 구분선을 표시합니다. 구분선 자체는 기능을 가지고 있지 않지만, 메뉴 항목이 많은 경우 사용자가 쉽게 인지할 수 있게 구분해줍니다.



32.3.1 예제

버튼 클릭 시 팝업 메뉴가 표시되며 구분선이 표시된 상태를 확인할 수 있습니다. [New] 항목 선택 시 하위 메뉴에는 지정한 아이콘이 표시됩니다.



32.3.2 예제에서 사용한 핵심 기능

iconcolumn

메뉴 앞에 아이콘을 표시합니다. 첫 번째 레벨에서는 아이콘을 표시하지 않으며 두 번째 이하 레벨에서만 표시합니다. 예제에서는 테마에서 사용하는 파일을 사용했습니다.

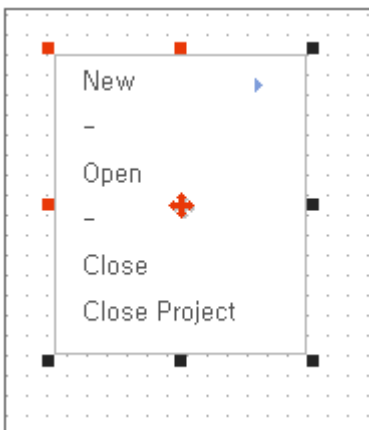
32.3.3 예제 구현 방법

- ① PopupMenu 컴포넌트를 화면에 배치합니다.
- ② PopupMenu 컴포넌트의 innerdataset 속성을 아래와 같이 지정합니다.

captioncolumn 항목은 화면에 표시되는 메뉴명입니다. 이 항목에 "-"를 입력하면 구분선으로 처리합니다. 예제에서는 4, 8, 10번째 Row에 "-"를 입력했습니다.

InnerDataset								
No	captioncolumn	ch...	ena...	ho...	iconcolumn	idcol...	levelcolumn	userdatac
1	New	[U...	[Und...	[U...	[Undefined]	[Und...	0	[Undefined]
2	Project	[U...	[Und...	[U...	url('theme://images/btn_WF_Calendar.png')	[Und...	1	[Undefined]
3	Application	[U...	[Und...	[U...	url('theme://images/img_WF_Grdimg.png')	[Und...	1	[Undefined]
4	-	[U...	[Und...	[U...	[Undefined]	[Und...	1	[Undefined]
5	Form	[U...	[Und...	[U...	[Undefined]	[Und...	1	[Undefined]
6	Script	[U...	[Und...	[U...	[Undefined]	[Und...	1	[Undefined]
7	XML	[U...	[Und...	[U...	[Undefined]	[Und...	1	[Undefined]
8	-	[U...	[Und...	[U...	[Undefined]	[Und...	0	[Undefined]
9	Open	[U...	[Und...	[U...	[Undefined]	[Und...	0	[Undefined]
10	-	[U...	[Und...	[U...	[Undefined]	[Und...	0	[Undefined]
11	Close	[U...	[Und...	[U...	[Undefined]	[Und...	0	[Undefined]
12	Close Project	[U...	[Und...	[U...	[Undefined]	[Und...	0	[Undefined]

넥사크로 스튜디오 Form 화면에서는 "-" 입력값이 그대로 표시되지만, 실제 애플리케이션 실행 시에는 구분선으로 표시됩니다.



- ③ 화면에 Button 컴포넌트를 배치하고 onclick 이벤트 함수를 아래와 같이 작성합니다.

버튼 클릭 시 팝업 메뉴가 표시되도록 합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.popupmenu00.trackPopupByComponent(this, e.clientx, e.clienty);
};
```

- ④ QuickView(Ctrl + F6)로 실행한 후 버튼 클릭 시 표시되는 팝업 메뉴를 확인합니다.

ProgressBar

33.1 ProgressBar 소개

ProgressBar는 작업의 진행 상태를 사용자에게 알려줄 때 사용하는 컴포넌트입니다. 파일의 업/다운로드나 로딩이 오래 걸리는 페이지 오픈 시 작업이 완료되려면 얼마나 걸릴지, 정상적으로 진행되고 있는 건지 사용자가 시각적으로 확인할 수 있게 진행 정보를 제공합니다.

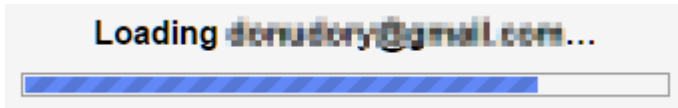
진행 상태는 긴 막대에 배열된 사각형의 빈 영역을 채워 전체 작업 대비 현재 작업의 진행률을 표시합니다. 속성의 설정에 따라 ProgressBar의 모양이나 속도, 방향 등을 설정할 수 있습니다.

33.1.1 사용 예

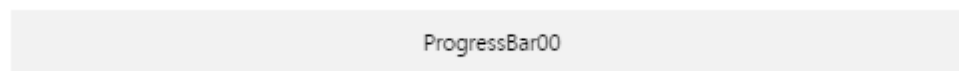
다음은 ProgressBar가 사용되는 실제 예입니다. 파일을 다운로드하는 상황에서 ProgressBar와 속도, 용량, 시간을 함께 제공하여 사용자가 진행 상황을 확인하며 완료되기까지 얼마나 기다려야 할지 예측할 수 있게 합니다.



ProgressBar는 페이지 로딩 시에도 사용할 수 있습니다. 예를 들어, 웹 메일 페이지를 오픈 시 시간이 오래 걸릴 수 있는데 사용자로서는 페이지가 정상적으로 오픈되고 있는지 확인할 방법이 없습니다. 최악의 경우 서비스에 문제가 생겨도 이를 인지할 수 없어 빈 화면만 바라보며 하염없이 기다리는 경우가 생깁니다. 그러나 페이지 로딩 상태를 ProgressBar로 표시해 주면 사용자는 진행 상태를 보며 문제가 있는지 판단할 수 있습니다.



33.1.2 ProgressBar 만들기



① ProgressBar 컴포넌트 생성하기

툴바에서 ProgressBar  를 선택한 후, 폼 위에 적당한 크기로 드래그하여 생성합니다.

ProgressBar는 기본 생성만으로 동작하지 않습니다. 시간, 파일의 용량 등과 같이 진행의 척도를 가늠할 수 있는 정보를 바탕으로 ProgressBar의 막대 위치를 알맞게 변경하는 작업이 필요합니다. 이는 스크립트로 작성해야 하는데 자세한 정보는 다음 장에서 다룹니다.

② QuickView로 확인하기

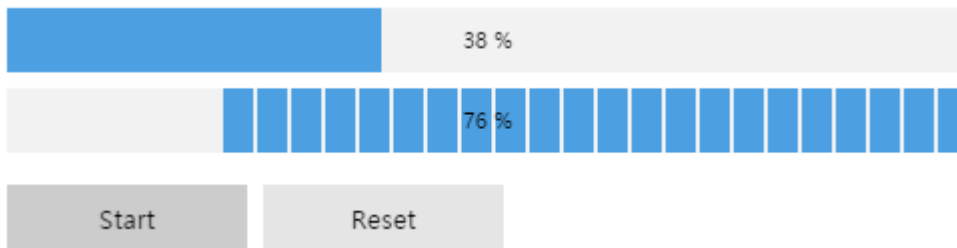
QuickView(Ctrl + F6)로 실행합니다.

33.2 진행 상태 표시하기

ProgressBar는 어떤 작업의 진행 상태를 사용자에게 제공하는 용도로 사용됩니다. 따라서 진행 상태를 표시하기 위한 정보가 있어야 하며 그런 정보는 서버나 서비스를 통해 받을 수 있습니다.

33.2.1 예제

다음은 ProgressBar의 예입니다. 예제의 단순성을 위해 작업 진행 상태 정보를 타이머로 대신했기 때문에 시간의 흐름에 따라 일정한 속도로 동작합니다. 그리고 속성을 다르게 설정한 두 개의 ProgressBar를 두어 속성의 설정에 따라 모양과 동작이 어떻게 변하는지 보여줍니다.



33.2.2 예제에서 사용한 핵심 기능

max

ProgressBar의 pos 속성이 가질 수 있는 최대값을 설정하는 속성입니다.

pos

ProgressBar에 표시되는 진행바의 위치를 설정하는 속성입니다.

step

ProgressBar의 stepIt 메소드 수행 시 pos 속성에 더해지는 값을 설정하는 속성입니다.

smooth

ProgressBar의 진행바를 공백없이 연속된 형태로 표시할지 설정하는 속성입니다.

stepIt

진행표시줄의 진행바 위치를 step 속성값만큼 변경하는 메소드입니다.

setTimer

주기적으로 ontimer 이벤트를 발생시키는 타이머를 생성하는 메소드입니다.

killTimer

setTimer 메소드로 생성한 타이머를 삭제하는 메소드입니다.

33.2.3 예제 구현 방법

① ProgressBar 컴포넌트 생성하기

툴바에서 ProgressBar , Button  컴포넌트를 생성하여 예제 화면과 같이 폼에 배치합니다.

② 컴포넌트 속성 설정하기

생성한 ProgressBar00, ProgressBar01의 속성을 각각 다음과 같이 설정합니다.

- ProgressBar00의 설정

속성	값
direction	forward
smooth	true
step	1

- ProgressBar01의 설정

속성	값
direction	backward
smooth	false
step	2

생성한 Button00, Button01의 속성을 각각 다음과 같이 설정합니다.

- Button00의 설정

속성	값
text	Start

- Button01의 설정

속성	값
text	Reset

③ Start 버튼 기능 구현하기

- Start 버튼의 onclick 이벤트 함수

Start 버튼의 onclick 이벤트 함수를 등록하고 다음과 같이 스크립트를 작성합니다.

100밀리초(0.1초)마다 ontimer 이벤트를 발생시키는 타이머를 생성합니다. 타이머 ID는 0으로 설정합니다.

```
this.Button00.onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.setTimer(0, 100);
};
```

- 폼의 ontimer 이벤트 함수

폼에 타이머에 의해 호출되는 ontimer 이벤트 함수를 추가하고 다음과 같이 스크립트를 작성합니다.

```
this.Form_ontimer = function(obj:nexacro.Form,e:nexacro.TimerEventInfo)
{
    //각 진행표시줄에 표시할 진행률을 계산합니다.
    var varProgressValue00 = parseInt((this.ProgressBar00.pos / this.ProgressBar00.max) *
100);
    var varProgressValue01 = parseInt((this.ProgressBar01.pos / this.ProgressBar01.max) *
100);

    //ID가 0인 타이머
    if(e.timerid == 0)
    {
        //진행이 완료되면 타이머를 종료시킵니다.
        if(this.ProgressBar00.pos == this.ProgressBar01.max && this.ProgressBar01.pos ==
this.ProgressBar01.max)
        {
            this.killTimer(0);
        }

        //각 진행바를 step 속성값만큼 변경합니다.
        this.ProgressBar00.stepIt();
        this.ProgressBar00.set_text(varProgressValue00 + " %");

        this.ProgressBar01.stepIt();
        this.ProgressBar01.set_text(varProgressValue01 + " %");
    }
};
```

④ Reset 버튼 기능 구현하기

- Reset 버튼의 onclick 이벤트 함수

Reset 버튼의 onclick 이벤트 함수를 등록하고 다음과 같이 스크립트를 작성합니다.

각 ProgressBar의 막대 위치 정보를 초기화하고 타이머를 종료시킵니다.

```
this.Button01 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.ProgressBar00.set_pos(0);
    this.ProgressBar00.set_text("ProgressBar00");
```

```
this.ProgressBar01.set_pos(0);  
this.ProgressBar01.set_text("ProgressBar01");  
  
this.killTimer(0);  
};
```

5 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Start 버튼을 눌러 각 진행표시줄이 시간에 따라 변화하는지 확인합니다. ProgressBar00은 왼쪽에서 오른쪽으로, ProgressBar01은 오른쪽에서 왼쪽으로 진행됩니다. 진행바가 끝까지 진행되면 자동으로 멈춥니다.

Reset 버튼을 누르면 진행표시줄이 모두 초기화됩니다.

34.

Radio

34.1 Radio 소개

Radio 컴포넌트는 선택할 수 있는 항목 중 하나만 선택할 수 있는 컴포넌트입니다. 자동차에 내장된 라디오는 특정 방송을 쉽게 선택할 수 있도록 버튼을 제공했는데 두 개 이상의 버튼이 눌러지지 않도록 버튼을 누르면 이미 눌린 버튼은 해제되도록 구성됐습니다. 이런 물리적인 형태를 사용자 인터페이스로 옮겨놓은 것이 Radio 컴포넌트입니다.

CheckBox 컴포넌트는 두 개 이상의 항목을 선택할 수 있지만, Radio 컴포넌트는 한 개만 선택할 수 있습니다.



HTML Input 요소에서 type 속성값을 'radio'로 설정하면 Radio 버튼 기능을 사용할 수 있습니다. 화면에 표시되는 모습은 비슷하지만 넥사크로플랫폼에서 사용하는 Radio 컴포넌트는 HTML Input 요소에서 제공하지 않는 추가적인 기능을 지원합니다.

34.1.1 사용 예

항목을 선택하는 경우에 선택할 수 있는 항목이 하나인지 아니면 두 개 이상인지에 따라 Radio 컴포넌트와 CheckBox 컴포넌트를 사용합니다.

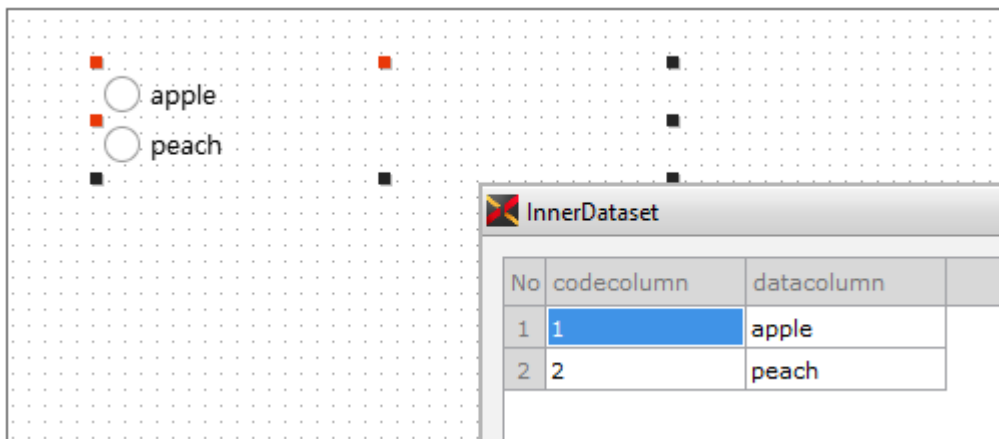
The screenshot shows a settings interface with two sections: 'On startup' and 'Appearance'. In the 'On startup' section, there are three radio buttons: 'Open the New Tab page' (selected), 'Continue where you left off', and 'Open a specific page or set of pages. Set pages'. In the 'Appearance' section, there are two buttons: 'Get themes' and 'Reset to default theme'. Below these are two checked checkboxes: 'Show Home button' and 'Always show the bookmarks bar'. The 'New Tab page' text is followed by a 'Change' link.

34.1.2 Radio 만들기

1 Radio 컴포넌트 생성하기

툴바에서 Radio를 선택한 후, Form 위에 드래그 앤 드롭해 생성합니다. 드래그 앤 드롭으로 기본 모습이 만들어지는 다른 컴포넌트와 달리 Radio 컴포넌트는 데이터를 설정해주어야 모습을 보입니다.

화면에 배치된 Radio 컴포넌트를 선택하고 속성창에서 innerdataset 항목에 있는 버튼을 클릭합니다. innerdataset 편집창이 열리면 codecolumn 항목에는 항목 선택 시 처리할 코드값을 입력하고 datacolumn 항목에는 화면에 보일 텍스트를 입력합니다.



컴포넌트를 배치하고 innerdataset 속성값만 설정했다면 항목이 선택되지 않은 상태로 초기화됩니다.

2 QuickView로 확인하기

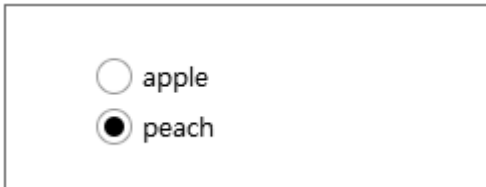
QuickView(Ctrl + F6)로 실행합니다. 컴포넌트를 배치하고 innerdataset 속성값만 설정했다면 항목이 선택되지 않은 상태로 초기화됩니다.

34.2 기본값 설정하기

Radio 컴포넌트의 기능은 사용자가 원하는 항목을 선택하게 하는 것이지만 많은 사용자가 선택하는 항목이 있다면 기본값으로 미리 선택해놓기도 합니다.

34.2.1 예제

실행 시 두 개 항목 중 지정된 항목을 선택된 상태로 표시합니다. 기본값 설정 상태는 넥사크로 스튜디오에서도 확인할 수 있습니다.



34.2.2 예제에서 사용한 핵심 기능

index

Radio 컴포넌트의 항목 중 선택된 아이템의 인덱스를 갖는 속성입니다. 첫 번째 데이터는 0부터 시작합니다. default 값은 -1입니다. 속성값이 -1인 경우는 선택된 아이템이 없는 상태를 의미합니다.

34.2.3 예제 구현 방법

① 화면 구성하기

Radio 컴포넌트를 예제 화면과 같이 배치합니다. innerdataset 속성값은 [Radio 만들기](#) 항목에서 작성한 값을 사용합니다.

② index 속성값 설정하기

index 속성값을 설정합니다. 첫 번째 값을 기본값으로 설정하고자 한다면 "0"을 입력하고 두 번째 값을 기본값으로 설정하고자 한다면 "1"을 입력합니다.

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 선택된 값을 확인합니다.

34.3 가로 방향으로 나열하기

Radio 컴포넌트의 default 설정은 세로 방향으로 아이템을 나열합니다. 하지만 화면을 구성하는 형식에 따라 Radio 컴포넌트의 아이템을 세로 또는 가로로 나열하는 요구사항이 있을 수 있습니다. 이번 예제에서는 Radio 컴포넌트의 아이템이 화면에 나열되는 여러 옵션을 살펴봅니다.



default 설정에서 direction 속성값만 "vertical"로 변경하면 가로 방향으로 아이템을 나열합니다. 간단하게 방향만 변경하는 요구사항이라면 해당 속성값만 변경해서 처리할 수 있습니다.

34.3.1 예제

Radio 컴포넌트에 10개의 데이터를 설정하고 방향과 화면에 표시되는 column과 row의 숫자를 변경할 수 있습니다.

columncount: direction: ☒ vertical ☐ horizontal [change direction](#)

rowcount:

☒ 1. apple ☐ 4. acorn

☐ 2. pear ☐ 5. strawberry

☐ 3. walnut



sample_radio_02.xfdl

34.3.2 예제에서 사용한 핵심 기능

direction

화면에서 아이템(데이터)이 표시되는 방향을 지정합니다. default 값은 "horizontal"입니다. direction 속성에 대한 오해 중 하나가 innerdataset을 설정하고 화면에 표시했을 때 세로 방향으로 데이터가 보이는데 왜 default 값은 "horizontal"이냐는 것입니다. 해당 속성의 정의가 Radio 컴포넌트의 방향이 아니라 아이템이 표시되는 방향을 지정하기 때문입니다. 아래 예제를 보면 좀 더 이해가 빠를 겁니다. columncount 속성값은 3

이고 direction 속성값만 변경했습니다.

Left group (selected: 1. apple):

- 1. apple
- 2. pear
- 3. walnut
- 4. acorn
- 5. strawberry
- 6. pineapple
- 7. grape
- 8. peach
- 9. kiwi
- 10. tomato

Right group (selected: 1. apple):

- 1. apple
- 2. pear
- 3. walnut
- 4. acorn
- 5. strawberry
- 6. pineapple
- 7. grape
- 8. peach
- 9. kiwi
- 10. tomato

columncount, rowcount

columncount는 한 행에 표시되는 아이템의 개수고 rowcount는 한 열에 표시되는 아이템의 개수입니다.

34.3.3 예제 구현 방법

① 화면 구성하기

Edit, Static, Button, Radio 컴포넌트를 예제 화면과 같이 배치합니다. Radio 컴포넌트의 innerdataset 속성값은 각각 아래와 같습니다.

컴포넌트	속성	값
Edit(columncount)	id	editColumnCount
Edit(rowcount)	id	editRowCount
Radio(direction)	id	radioDirection
	index	0
	codecolumn	codecolumn
	datacolumn	datacolumn
	innerdataset	innerdataset
Radio(fruit list)	id	radioValue
	index	0
	codecolumn	codecolumn
	datacolumn	datacolumn
	innerdataset	innerdataset

과일 목록을 출력하는 Radio 컴포넌트(fruit list)의 innerdataset 속성값은 각각 아래와 같습니다.

InnerDataset		
No	codecolumn	datacolumn
1	1	1. apple
2	2	2. pear
3	3	3. walnut
4	4	4. acorn
5	5	5. strawberry

direction을 설정하는 Radio 컴포넌트(direction)의 innerdataset 속성값은 각각 아래와 같습니다.

InnerDataset		
No	codecolumn	datacolumn
1	vertical	vertical
2	horizontal	horizontal

② Button 컴포넌트 이벤트 함수 작성하기

Button 컴포넌트를 선택한 후 onclick 이벤트 함수를 작성합니다. Button 컴포넌트 클릭 시 Edit 컴포넌트에 입력한 속성값과 Radio 컴포넌트에서 선택된 값을 Radio 컴포넌트에 적용합니다.

```
this.btnChange_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    if(this.radioDirection.value != null)
    {
        this.radioValue.set_direction(this.radioDirection.value);
    }
    if(this.editRowCount.value != null)
    {
        this.radioValue.set_rowcount(this.editRowCount.value);
    }
    if(this.editColumnCount.value != null)
    {
        this.radioValue.set_columncount(this.editColumnCount.value);
    }
};
```

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 선택된 값을 확인합니다. 입력값에 따라 Radio 컴포넌트의 아이템이 다르

게 표시됩니다.

34.4 innerdataset 항목 추가하기

innerdataset 속성값으로 지정된 데이터를 추가하는 방법을 살펴봅니다.

34.4.1 예제

최초 실행 시 3개의 데이터가 지정되어 있습니다. Edit 컴포넌트에 값을 입력하고 버튼 클릭 시 innerdataset 항목으로 반영됩니다.



 [sample_radio_03.xfdl](#)

34.4.2 예제에서 사용한 핵심 기능

getInnerDataset

innerdataset 속성에 설정된 Dataset 오브젝트를 반환합니다. Dataset 오브젝트의 속성이나 메소드는 그대로 사용합니다.

34.4.3 예제 구현 방법

① 화면 구성하기

Radio 컴포넌트를 예제 화면과 같이 배치합니다. innerdataset 속성값은 아래와 같이 지정합니다.

InnerDataset		
No	codecolumn	datacolumn
1	apple	apple
2	banana	banana
3	cocoa	cocoa

innerdataset 데이터가 추가되면 radio 컴포넌트의 크기가 같이 커져야 합니다. 그렇지 않으면 추가된 데이터가 보이지 않을 수 있습니다. radio 컴포넌트를 선택하고 fittocontents 속성값을 "height"로 변경합니다.

2 onclick 이벤트 함수 작성하기

Edit 컴포넌트와 Button 컴포넌트를 배치합니다. 입력된 문자열을 innerdataset 데이터로 추가합니다. 데이터를 추가할 인덱스 파라미터는 현재 innerdataset의 rowcount를 지정합니다. 데이터가 추가되면 fittocontents 속성으로 지정된 radio 컴포넌트의 크기 변경을 처리하기 위해 resetScroll 메소드를 실행합니다.

```
this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var rtn = this.Radio00.getInnerDataset();
    var nindex = rtn.rowcount;
    rtn.insertRow(nindex);
    rtn.setColumn(nindex, "codecolumn", this.Edit00.value);
    rtn.setColumn(nindex, "datacolumn", this.Edit00.value);
    this.resetScroll();
};
```

3 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 값을 입력해 innerdataset을 추가합니다.

34.5 fittocontents 속성 조합 시 표시 방식



17.0.2102 AS 버전에서 수정된 변경 건이 복원되면서 관련 콘텐츠 게시를 중단합니다.
아래 릴리스 노트를 참고해주세요.

http://docs.tobesoft.com/product_information_nexacro_17_ko#4bf730e5d6fa4538

35.

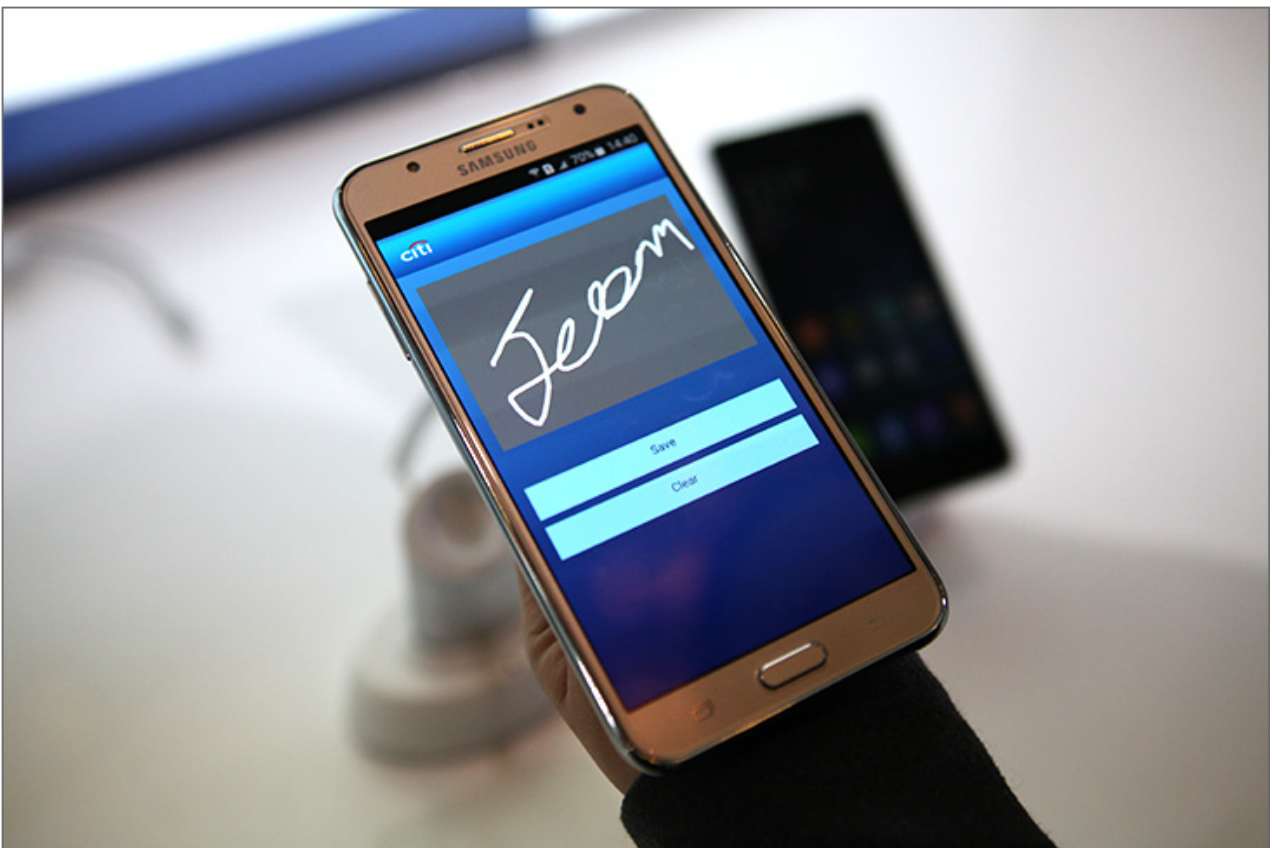
Sketch

35.1 Sketch 소개

Sketch 컴포넌트는 이름 그대로 백지에 무언가 그리는 기능을 제공하는 컴포넌트입니다. 전문적으로 그림을 그리기 위한 도구는 아니고 간단한 서명하기 등의 기능이 필요할 때 사용할 수 있습니다.

35.1.1 사용 예

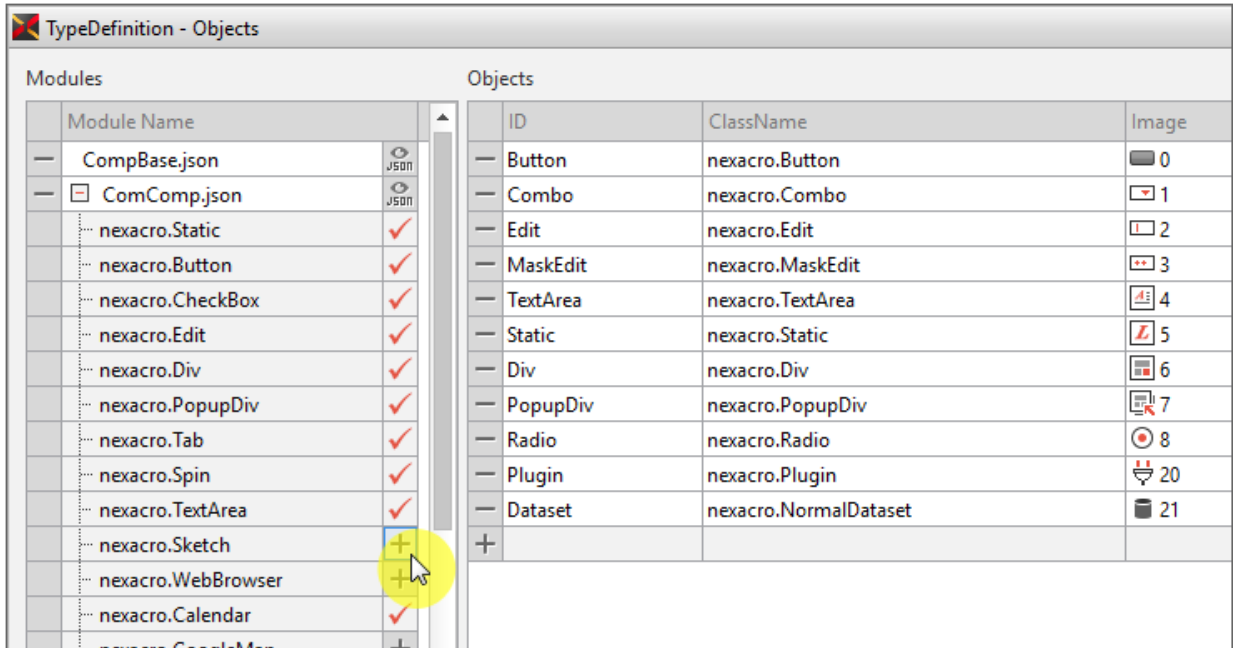
일반 소매점에서 카드 결제 후 서명을 하는 단말기에 사용되는 컴포넌트도 Sketch 컴포넌트와 비슷한 기능을 처리합니다. 서명을 받고 이를 이미지로 저장해 카드 회사로 결제 정보와 함께 전송하는 것입니다.



35.1.2 Sketch 만들기

1 Sketch 컴포넌트 생성하기

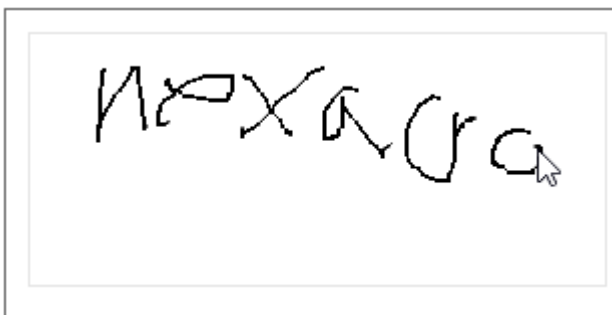
기본 TypeDefinition에는 Sketch 컴포넌트가 등록되어 있지 않습니다. Project Explorer에서 [TypeDefinition > Objects] 항목을 선택하고 에디터를 실행하면 Modules 항목에서 추가되지 않은 오브젝트 목록을 확인할 수 있습니다. 목록에서 "nexacro.Sketch" 항목 옆에 [+] 버튼을 클릭해 오브젝트를 추가합니다.



툴바에서 Sketch를 선택한 후, Form 위에 드래그 앤 드롭해 생성합니다.

2 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행합니다. default 설정값이 바로 Sketch 컴포넌트 위에 그림을 그릴 수 있는 것이어서 해당 영역 위에 마우스 왼쪽 버튼 클릭 후 드래그를 하면 그림이 그려집니다.



35.2 Sketch 옵션 추가하기

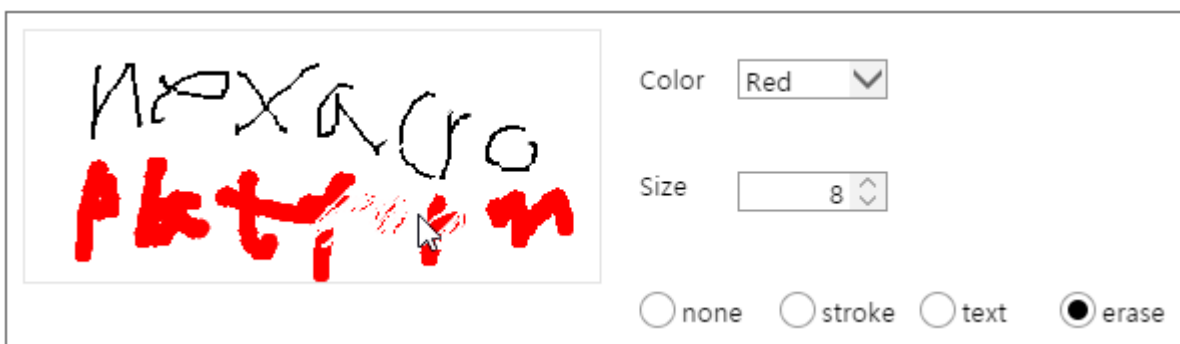
그리기와 지우기 두 가지 기본 기능을 제공하며 Text를 추가할 수 있습니다. 그리기 작업 시에는 브러시의 색상, 크기, 모양 등을 설정할 수 있습니다. 이번 예제에서는 간단한 옵션을 추가해봅니다.

35.2.1 예제



17.0.0.100 버전에서 작성된 이미지 파일을 웹브라우저에서 내려받지 못하는 오류가 있어 예제가 정상적으로 동작하지 않습니다.

Sketch 컴포넌트 영역이 표시되고 Color, Size 옵션을 선택할 수 있습니다. 그리고 Radio 버튼을 선택하면 편집 시 텍스트를 추가하거나 그려진 이미지를 지울 수 있습니다.



웹브라우저에서 실행한 경우에는 작성된 이미지를 파일로 저장할 수 있습니다. 웹브라우저 외 런타임 브라우저를 사용하는 경우에는 별도 메소드를 사용해 저장 기능을 구현할 수 있습니다.



 [sample_sketch_01.xfdl](#)

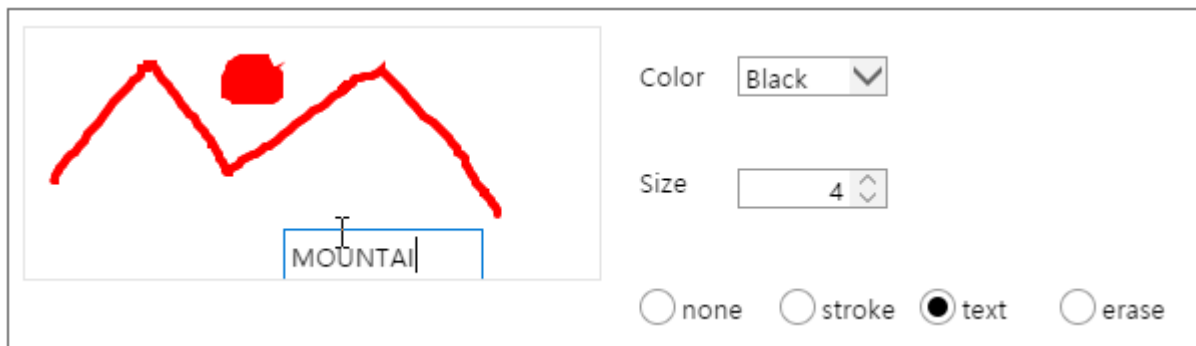
35.2.2 예제에서 사용한 핵심 기능

setBrushColor, setBrushSize

Sketch 영역에 그려지는 선의 색상이나 크기를 지정합니다.

editmode

네 가지 옵션 중 하나를 선택할 수 있습니다. default 값은 "stroke"입니다. "text"를 선택한 경우에는 선택한 지점에 문자열을 추가할 수 있습니다. 문자열은 입력이 완료되면 이미지로 변환되며 수정할 수 없습니다.



35.2.3 예제 구현 방법

① 화면 구성하기

Sketch 컴포넌트를 배치하고 옵션 설정을 위해 Static 컴포넌트와 Combo, Spin, Radio 컴포넌트를 배치합니다. Combo 컴포넌트의 innerdataset은 "Black, Blue, Red, Yellow" 4가지 색상을 지정하고 Radio 컴포넌트의 innerdataset은 "none, stroke, text, erase" 4가지 옵션을 지정합니다.

Sketch 컴포넌트에서 사용하는 브러시 크기는 0보다 큰 값으로 지정해주어야 합니다. Spin 컴포넌트의 min 속성값을 1로 지정합니다.

② onitemchanged, onchanged 이벤트 추가하기

옵션값을 수정할 때 수정된 값을 Sketch 컴포넌트에 반영할 수 있도록 해당하는 이벤트를 추가합니다.

```
this.cmbColor_onitemchanged = function(obj:nexacro.Combo,e:nexacro.ItemChangeEventInfo)
{
    this.sketchMain.setBrushColor(obj.value);
};

this.spinSize_onchanged = function(obj:nexacro.Spin,e:nexacro.ChangeEventInfo)
{

```

```
        this.sketchMain.setBrushSize(obj.value);
    };

    this.Radio000_onitemchanged = function(obj:nexacro.Radio,e:nexacro.ItemChangeEventInfo)
    {
        this.sketchMain.set_editmode(obj.value);
    };
};
```

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 옵션값을 변경하면서 Sketch 컴포넌트에 그림을 그리거나 문자열을 추가합니다.

36.

Spin

36.1 Spin 소개

Spin은 일정 범위 내의 숫자 값을 입력받기 위해 사용하는 컴포넌트입니다.



일정 범위 내의 숫자값을 입력하기 위해 버튼으로 값을 증가시키거나 감소시킬 수 있습니다. 이때, 값이 변경되는 단위를 조정할 수 있어 특정 단위로만 값을 증감시켜야 하는 경우에 사용하기 좋습니다.

max, min 속성을 정의하면 최댓값과 최솟값을 설정할 수 있고 increment 속성으로 한번 클릭 시 증감되는 값을 설정할 수 있습니다. type 속성을 noneditable로 설정하여 버튼으로만 조작하게 할 수도 있는데 이는 사용자가 잘못된 값을 입력하는 것을 막는 방법으로 쓰입니다. 그리고 spinonly로 설정하면 편집창없이 스펀 업/다운 버튼만 사용할 수도 있습니다.

값의 입력은 스펀 업/다운 버튼을 사용하는 방법 이외에 사용자가 직접 편집창을 통해 입력할 수도 있고 키보드의 방향키인 ↑ 혹은 ↓로도 값을 증감시킬 수 있습니다.

36.1.1 사용 예

다음은 어느 피자 사이트의 주문화면입니다. 각 메뉴가 사진으로 나와 있고 그 밑에 수량을 입력받는 방법으로 Spin 컴포넌트를 사용하고 있습니다.

일반적으로 편집창을 통한 입력은 사용자의 실수를 막기 위해 복잡한 스크립트로 처리해야 합니다. 그러나 Spin 컴포넌트를 사용함으로써 사용자가 잘못된 값을 입력하는 것을 사전에 방지하는 효과를 얻을 수 있습니다.



36.1.2 Spin 만들기

1 Spin 컴포넌트 생성하기

툴바에서 Spin  을 선택한 후, Form 위에 적당한 크기로 드래그하여 생성합니다. 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

2 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행합니다.

스핀 업/다운 버튼을 통해 0~10000 범위까지 값을 증감시킬 수 있습니다. increment 속성의 기본값이 1로 설정되어 있어 값이 1 단위로 변경됩니다.

36.2 Spin 사용하기

Spin 컴포넌트는 입력 값의 범위, 증감 단위, 입력 방법, 모양을 속성의 설정을 통해 변경할 수 있습니다.

36.2.1 예제

max, min 속성을 사용하면 입력 값의 범위를 한정할 수 있고, increment 속성은 증감 단위를, type 속성은 입력 방법을 설정할 수 있습니다.

다음은 각 속성을 설정한 Spin 컴포넌트 예제입니다.



36.2.2 예제에서 사용한 핵심 기능

max

value 속성이 가질 수 있는 최댓값을 설정하는 속성입니다.

min

value 속성이 가질 수 있는 최솟값을 설정하는 속성입니다.

increment

스핀 업/다운 버튼 클릭 시 value 속성값을 증가/감소시킬 단위를 설정하는 속성입니다.

type

Spin 컴포넌트가 화면에 표시되는 형식을 설정하는 속성입니다.

enumType ::= 'normal' 'noneditable' 'spinonly'	
normal	(default) 편집 영역과 스핀 버튼이 있는 모양으로 표시됩니다. 편집 영역에서 값을 직접 입력할 수 있습니다.
noneditable	편집 영역과 스핀 버튼이 있는 모양으로 표시됩니다. 편집 영역은 읽기 전용으로 설정되어 값을 직접 입력할 수 없습니다.
spinonly	편집 영역 없이 스핀 업/다운 버튼만 있는 모양으로 표시됩니다.

```
this.Spin00.set_type( "normal" );
```

displaycomma

Spin의 편집 창에서 보일 숫자가 천 단위마다 콤마를 표시할지 설정하는 속성입니다.

bDisplayComma ::= 'true' 'false'	
true	value 속성값이 1000 이상일 경우 천 단위마다 콤마를 표시합니다.
false	(default) 콤마를 표시하지 않습니다.

```
this.Spin00.set_displaycomma( true );
```

36.2.3 예제 구현 방법

① Spin 컴포넌트 생성하기

툴바에서 Spin  을 선택한 후, Form 위에 적당한 크기로 드래그하여 생성합니다. 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

② Spin 속성 설정하기

Spin 컴포넌트를 선택한 후 Properties 창에서 각 속성을 아래와 같이 설정합니다.

속성	값
max	100000000
min	-100000000
increment	1000
type	noneditable
displaycomma	true
value	0

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행합니다.

편집창이 비활성화 돼 있고 초기값이 0으로 나오는지 확인합니다. 그리고 스펀 버튼을 눌러 값이 1000 단위로 증가 혹은 감소하는지 확인합니다.

37.

Static

37.1 Static 소개

Static 컴포넌트는 화면에 텍스트 정보를 출력하는 수단 중 하나입니다. 텍스트를 편집할 수 있는 컴포넌트와 달리 화면에 보여주는 기능만 제공합니다. 스크립트로 text 속성값을 변경하거나 Dataset 오브젝트에 바인딩된 상태에서 값이 변경될 수는 있지만, 사용자가 임의로 값을 수정하는 기능은 제공하지 않습니다.



Static 컴포넌트는 주어진 크기를 동적으로 변경하지 않습니다. 그래서 화면에 보여줄 텍스트 정보와 Static 컴포넌트의 크기를 어느 정도 맞추어야 하는 한계가 있습니다.

37.1.1 사용 예

텍스트를 표시하는 항목 대부분은 Static 컴포넌트를 사용합니다.

Few understand this better than Facebook CEO Mark Zuckerberg. We just had a pretty great conversation with some young entrepreneurs from across the globe and now I'm joining him one-on-one to discuss how we can enable budding entrepreneurs - especially women, youth, and minorities - to connect with new audiences, build new partnerships, and unleash their full potential.

Tune in!

image source: <https://www.facebook.com/potus>



37.1.2 Static 만들기

① Static 컴포넌트 생성하기

툴바에서 Static 항목을 선택한 후, Form 위에 드래그 앤 드롭해 생성합니다.

② QuickView로 확인하기

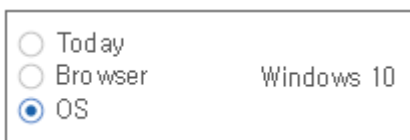
QuickView(Ctrl + F6)로 실행합니다.

37.2 텍스트를 동적으로 표현하기

Static 컴포넌트를 화면에 배치하고 text 속성값에 원하는 텍스트를 입력하면 더는 추가적인 작업은 필요하지 않습니다. 하지만 간혹 동적으로 생성된 텍스트를 표시해야 하는 경우도 있습니다. 예를 들어 오늘 날짜를 화면에 표시해야 하는 경우라면 text 속성값을 사전에 지정할 수는 없습니다. 이런 경우에는 expr 속성값을 이용할 수 있습니다.

37.2.1 예제

expr 속성값에 오늘 날짜를 가져오는 함수를 지정하고 Static 컴포넌트가 생성되면서 해당 함수의 반환값을 화면에 표시합니다. Radio 컴포넌트 항목 중 "Browser" 또는 "OS"를 선택하면 expr 속성값에 지정된 함수가 변경되면서 현재 실행 중인 브라우저 정보를 표시합니다.




37.2.2 예제에서 사용한 핵심 기능

navigatorname

애플리케이션이 실행되는 환경 정보를 반환합니다. 넥사크로 브라우저를 실행한 경우에는 "nexacro"로 표시하며, 그 외의 경우에는 브라우저명을 반환합니다.

osversion

애플리케이션이 실행되는 운영체제 정보를 반환합니다.

expr

속성값으로 설정된 코드를 실행하고 그 결과값을 표시합니다. 함수가 아니라 수식이나 특정 변수값을 사용할 수도 있습니다.



"comp.parent.[함수명]" 대신 "expr:comp.parnet.[함수명]"을 사용할 수 있습니다. "expr:" 부분은 내부적으로 무시하고 처리하기 때문에 같은 결과값으로 처리됩니다.



expr 속성값에 함수를 지정하는 경우에는 "comp.parent.[함수명]"과 같은 형식을 사용합니다. 내부적으로 처리 시 컴포넌트(comp) 위치에서 해당 함수를 호출할 경로를 지정해주어야 합니다. 그래서 컴포넌트 위치에서 parent 경로에 지정된 함수를 호출하기 위해 "comp.parent.[함수명]"과 같은 형식을 사용합니다.

예제에서 comp는 Static 컴포넌트를 가리키며 comp.parent는 Form 오브젝트를 가리킵니다. 함수는 Form 오브젝트 내 작성되어있기 때문에 "comp.parent.[함수명]" 이런 식으로 호출하게 됩니다.

Div 컴포넌트 내 Static 컴포넌트에서 expr 속성값을 함수로 지정한다면 컴포넌트 위치에서 parent 경로에 Div 컴포넌트가 있고 한 번 더 parent 경로를 찾아가야 함수를 찾을 수 있습니다. 그래서 "comp.parent.parent.parent.[함수명]"과 같은 형식을 사용해야 합니다.



expr 속성값을 처리하면서 임시로 함수를 생성하는데 이때 전달되는 파라미터명이 "comp"입니다. "comp"는 expr 속성을 사용하는 컴포넌트를 가리키고 있습니다. 그래서 "comp.parent.[함수명]"과 같은 형식을 사용합니다. 아래 코드를 참고하세요.

```
_fn_ = function(comp) { try { return comp.parent.fn_date(); } catch(e) { return undefined; } };
```



expr 속성을 과도하게 사용할 경우에는 애플리케이션 성능에 영향을 미칠 수 있습니다. 이번 예제는 사용자의 이해를 돕기 위해 작성된 예제이며 최선의 코드는 아닙니다.

37.2.3 예제 구현 방법

① 화면 구성하기

Radio, Static 컴포넌트를 예제 화면과 같이 배치합니다. Radio 컴포넌트의 innerdataset 속성값은 아래와 같이 지정하고 실행 시 첫 번째 아이템인 "today"가 선택된 상태로 보이도록 index 속성값은 0으로 지정합니다.

Static 컴포넌트의 expr 속성값은 "comp.parent.fn_dateToStr()"로 지정합니다.

InnerDataset		
No	codecolumn	datacolumn
1	Today	Today
2	Browser	Browser
3	OS	OS

Action	
expr	comp.parent.fn_dateToStr()

innerdataset 직접 지정하기

② expr 속성값에 지정할 함수 작성하기

간단하게 오늘 날짜값과 브라우저, 운영체제 정보를 반환하는 함수를 작성합니다.

```

this.fn_dateToStr = function()
{
    var objDate = new Date();
    var sRtn = objDate.getFullYear()+"/"
        + (objDate.getMonth()+1).toString().padLeft(2, "0")+"/"
        + objDate.getDate().toString().padLeft(2, "0");
    return sRtn;
};

this.fn_checkBrowser = function()
{
    return system.navigatorname + " " + system.navigatorversion;
}

this.fn_checkOS = function()
{
    return system.osversion;
}

```

③ Radio 컴포넌트 이벤트 함수 작성하기

Radio 컴포넌트를 선택한 후 onitemchanged 이벤트 함수를 작성합니다. Radio 컴포넌트에서 항목 선택 시

Static 컴포넌트의 expr 속성값을 변경합니다.

```
this.radioExpr_onitemchanged = function(obj:nexacro.Radio,e:nexacro.ItemChangeEventInfo)
{
    var todayExpr = "comp.parent.fn_dateToStr()";
    var browserExpr = "comp.parent.fn_checkBrowser()";
    var osExpr = "comp.parent.fn_checkOS()";
    if(obj.value == "Today")
        this.staticExpr.set_expr(todayExpr);
    else if(obj.value == "Browser")
        this.staticExpr.set_expr(browserExpr);
    else if(obj.value == "OS")
        this.staticExpr.set_expr(osExpr);
};
```



예제에서는 expr를 지정하는 것보다 직접 text 속성값을 지정하는 것이 더 빠를 수 있습니다.
this.staticExpr.set_text(this.fn_dateToStr());



expr 속성값이 지정된 경우에는 text 속성값을 지정해도 적용되지 않습니다.
expr 속성값으로 지정한 값을 text 속성값으로 대체하려면 아래와 같이 expr 속성값을 무효로 만들어야 합니다.
this.staticExpr.set_expr(null);

4 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 선택항목을 변경하면서 확인해봅니다.

37.3 자동줄넘김(wordwrap) 기능

Static 컴포넌트의 text 속성값은 사전에 지정된 값을 보여주기도 하지만 데이터베이스에서 가져온 데이터를 보여주거나 특정 결과값을 보여주는 경우에는 Static 컴포넌트에서 보여주어야 할 텍스트의 너비를 사전에 예측하지 못할 수 있습니다. Static 컴포넌트는 기본적으로 표현할 텍스트를 한 줄로 보여주는데 자동으로 줄바꿈 처리가 필요한 경우에는 wordwrap 속성을 지정합니다.

37.3.1 예제

최초 실행 시에는 wordwrap 속성을 적용하지 않고 Static 컴포넌트 오른쪽에 있는 Radio 컴포넌트의 아이템 선택 시 선택한 wordwrap 속성값을 적용합니다.

Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

☐ none
 ☐ char
 ☒ english



sample_static_02.xfdl

37.3.2 예제에서 사용한 핵심 기능

wordwrap

컴포넌트의 영역을 벗어나는 입력값을 줄바꿈해서 보여줄지 지정하는 속성입니다. 'none', 'char', 'english' 속성 중 하나를 선택할 수 있습니다. 'english'를 선택하면 단어 단위로 줄바꿈을 처리하고 'char'를 선택하면 단어 단위와 상관없이 길이에 따라 줄바꿈을 처리합니다.



CJK(Chinese/Japanese/Korean) 문자는 'english', 'char' 속성값 선택에 따른 차이가 없습니다(아예 차이가 없는 것은 아닙니다. 공백 문자 처리 방식이 다르므로 'english' 속성값을 선택하는 것이 깔끔해 보일 수 있습니다).



각 속성값은 HTML5 Element에서 아래와 같이 처리됩니다 (CSS3, HTML5를 지원하는 브라우저인 경우).

HTML5 Element	none	char	english
white-space	pre	pre-wrap	pre-wrap
overflow-wrap	normal	break-word	break-word
word-break	-	break-all	-
wrap	hard	hard	hard



웹브라우저와 속성 조합 조건에 따라 처리 방식은 조금씩 달라질 수 있습니다. 예를 들어 IE 브라우저에서는 -ms-word-break, -ms-word-wrap 속성으로 처리될 수 있습니다.



<https://developer.mozilla.org/ko/docs/Web/CSS/white-space>
<https://developer.mozilla.org/en-US/docs/Web/CSS/overflow-wrap>
<https://developer.mozilla.org/en-US/docs/Web/CSS/word-break>
<https://developer.mozilla.org/en-US/docs/Web/HTML/Element/textarea>

wrap 속성은 TextArea 컴포넌트에만 해당하는 처리방식입니다.

37.3.3 예제 구현 방법

① 화면 구성하기

Static, Radio 컴포넌트를 예제 화면과 같이 배치합니다. Radio 컴포넌트의 innerdataset 속성값은 아래와 같이 지정합니다. Static 컴포넌트의 text 속성값은 원하는 텍스트를 입력합니다. 예제에서는 Lorem ipsum 텍스트를 사용했습니다.

InnerDataset			
No	codecolumn	datacolumn	
1	none	none	
2	char	char	
3	english	english	

② Radio 컴포넌트 이벤트 함수 작성하기

Radio 컴포넌트를 선택한 후 onitemchanged 이벤트 함수를 작성합니다. Radio 컴포넌트에서 항목 선택 시 Static 컴포넌트의 wordWrap 속성값으로 지정합니다.

```
this.radioWordwrap_onitemchanged = function(obj:nexacro.Radio,e:nexacro.ItemChangeEventInfo)
{
    this.staticWordWrap.set_wordWrap(this.radioWordwrap.value);
};
```

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 속성값을 변경하면서 확인해봅니다. TextArea 컴포넌트와 달리 텍스트가 주어진 컴포넌트 크기보다 많은 경우 스크롤바가 적용되지 않기 때문에 화면 구성 시 적절한 크기를 설정해주어야 합니다.

37.4 텍스트 꾸미기

Static 컴포넌트는 텍스트를 그대로 보여주는 것이 주요 기능입니다. 하지만 필요에 따라 특정 글자를 강조할 필요가 있습니다. 그런 경우에는 usedecorate 속성값을 지정할 수 있습니다.

37.4.1 예제

text 항목에 텍스트를 입력하고 꾸미기 옵션을 선택한 후 "input text" 버튼 클릭 시 상단에 지정한 꾸미기 옵션에 따라 텍스트가 표시됩니다. 하단 TextArea 컴포넌트에는 text 속성값에 지정한 태그를 표시합니다.



font color와 font family에 들어가는 값은 대표적인 값만 예제로 추가했습니다.

My name is **Ernest Miller Hemingway**

text

font size / color / family

bold
☒ true
☐ false

italic
☐ true
☒ false

underline
☒ true
☐ false

strike
☐ true
☒ false

```

My name is <fs v='12'><fc v='green'><ff v='Comic Sans MS'><b v='true'><i v='false'>
<u v='true'><s v='false'><fs v='12'><fc v='green'><ff v='Comic Sans MS'><b v='true'
><i v='false'><u v='true'><s v='false'><fs v='12'><fc v='green'><ff v='Georgia'><b v

```



sample_static_03.xfdl

37.4.2 예제에서 사용한 핵심 기능

usedecorate

Static 컴포넌트에 표시되는 텍스트에 특정 태그 기반 효과를 적용할지를 지정합니다. 해당 속성은 적용 여부를 결정하며 실제 사용하는 태그는 text 속성값 또는 expr 속성값으로 지정해주어야 합니다.

```
fs (font-size)
fc (color)
ff (font-family)
b (bold)
i (italic)
u (underline)
s (strike)
```



태그는 아래와 같이 중첩해서 사용할 수 있습니다. 텍스트 앞에 붙는 태그와 뒤에 붙는 태그의 순서가 일치하지 않으면 텍스트를 표기하지 못합니다.

```
<u v='true'><i v='true'>Static00</i></u>
```



태그를 사용한 경우에 usedecorate 속성값을 false로 설정하면 태그가 그대로 표시됩니다.

37.4.3 예제 구현 방법

① 화면 구성하기

예제에서 표기한 것처럼 Static, Edit, Spin, Combo, Radio, TextArea 컴포넌트를 배치합니다. font color, font family 항목을 처리하는 Combo 컴포넌트의 innerdataset 속성값은 아래와 같이 지정합니다. bold, italic, underline, strike 항목을 처리하는 Radio 컴포넌트의 innerdataset 속성값은 "true", "false" 두 가지 값을 지정하고 direction 속성값은 "vertical"로 지정합니다.

InnerDataset			InnerDataset		
No	codecolumn	datacolumn	No	codecolumn	datacolumn
1	red	red	1	Arial	Arial
2	orange	orange	2	Comic Sans MS	Comic Sans MS
3	yellow	yellow	3	Courier New	Courier New
4	green	green	4	Georgia	Georgia
5	blue	blue			
6	indigo	indigo			
7	purple	purple			
8	black	black			



innerdataset 직접 지정하기

2 태그를 저장할 변수를 초기화하는 함수 작성하기

꾸미기 옵션값에 따라 태그가 만들어지고 사용자가 입력한 값과 합쳐져 value 속성값에 적용할 태그를 만들어야 합니다. 다음 순서에서 작성하는 태그 처리 함수에서 처리된 값을 저장하고 value 속성값에 적용할 변수를 초기화하는 함수를 작성합니다.

```

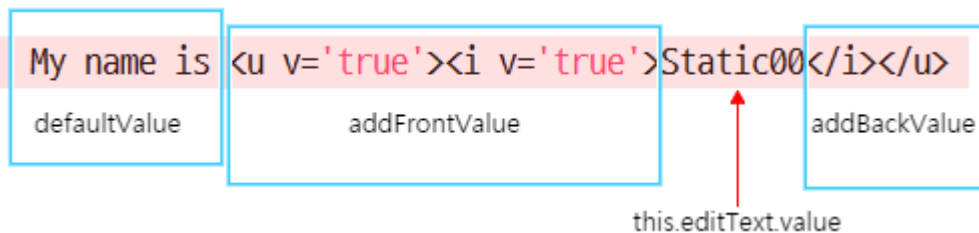
this.defaultValue = "My name is ";
this.addFrontValue = "";
this.addBackValue = "";

this.fn_initValue = function()
{
    this.addFrontValue = "";
    this.addBackValue = "";
}

```

defaultValue 변수는 "My name is "라는 값을 가집니다. defaultValue 변수는 계속 같은 값을 유지합니다. addFrontValue, addBackValue 변수는 사용자가 선택한 옵션에 따라 생성된 태그값을 가지는 변수입니다. 그래서 사용자가 버튼을 클릭할 때마다 값을 초기화하고 새로운 태그값을 저장하도록 준비합니다.

아래 그림은 각 변수값이 어떻게 적용되는지 보여주고 있습니다.



3 태그 처리 함수 작성하기

꾸미기 옵션에서 선택한 값을 태그로 변환하는 작업이 필요한데 태그와 태그에 포함할 value 값만 다르고 나머지 규칙은 같습니다. 태그값과 value값을 받아서 전체 태그를 만드는 함수를 작성합니다.

```

this.addValue = function(tag:String, value:String) {
    if(value) {
        this.addFrontValue += "<" + tag + " v='" + value + "'>";
        this.addBackValue = "</" + tag + ">" + this.addBackValue ;
    }
}

```

};

④ Button 컴포넌트 이벤트 함수 작성하기

Button 컴포넌트를 클릭 시 처리할 onclick 이벤트 함수를 작성합니다. 각 꾸미기 옵션을 변경할 때 이벤트를 처리할 수도 있으나 이번 예제에서는 버튼 클릭 이벤트에서만 처리하는 것으로 작성했습니다. 버튼을 클릭하면 addFrontValue, addBackValue 변수값을 초기화하기 위해 fn_initValue 함수를 호출합니다.

꾸미기 옵션값을 addValue 함수에서 처리한 후 변수에 담고 마지막에 화면에 표시할 태그를 작성합니다. 작성된 태그를 확인할 수 있도록 TextArea 컴포넌트에 Static 컴포넌트의 text 속성값을 표시합니다.

```
this.btnText onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.fn_initValue();
    this.addValue("fs", this.spinFontSize.value);
    this.addValue("fc", this.comboFontColor.value);
    this.addValue("ff", this.comboFontFamily.value);
    this.addValue("b", this.radioBold.value);
    this.addValue("i", this.radioItalic.value);
    this.addValue("u", this.radioUnderline.value);
    this.addValue("s", this.radioStrike.value);
    this.staticDecorate.set_text(this.defaultValue + this.addFrontValue + this.editText.value + this.addBackValue);
    this.textareaDecorate.set_value(this.staticDecorate.text);
};
```

⑤ QuickView로 확인하기

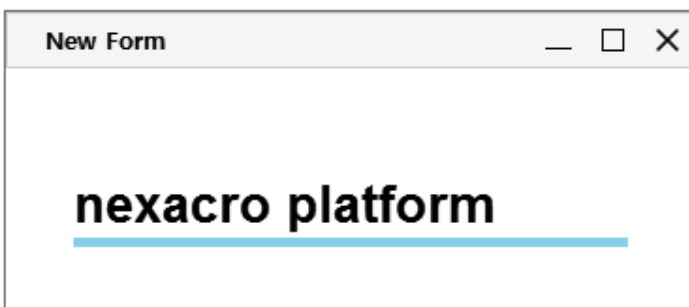
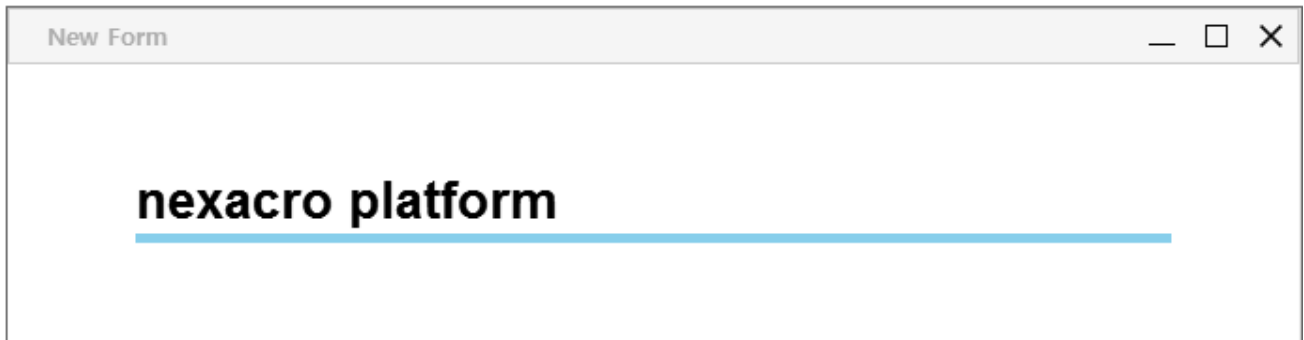
QuickView(Ctrl + F6)로 실행한 후 선택항목을 변경하면서 확인해봅니다.

37.5 가로바 만들기

화면을 배치할 때 제목 영역을 강조하거나 제목과 본문을 구분하기 위해 가로바를 사용합니다. 가로바를 만드는 방법은 여러 가지가 있지만, 이번 예제에는 Static 컴포넌트의 배경색을 지정하는 방식으로 가로바를 만들어봅니다. 같은 방식으로 세로 형태의 바도 만들 수 있습니다.

37.5.1 예제

화면 상단에 제목을 표시하고 제목 아래 영역에 가로바를 표시합니다. 제목과 가로바는 position 속성값을 "%"로 지정해 화면 크기가 달라지더라도 형태를 유지할 수 있습니다.



37.5.2 예제에서 사용한 핵심 기능

background

컴포넌트의 배경 속성을 지정합니다. 이번 예제에서는 배경색을 지정하지만 색상값을 직접 지정하지 않고 이미지를 배경으로 지정하거나 그라데이션 효과를 지정할 수 있습니다.



작성된 페이지가 많은 경우에는 직접 스타일 관련 속성을 지정하지 않습니다. 테마에서 전체적인 스타일을 잡아주고 특정한 상황에 맞는 스타일 속성은 cssclass 속성값을 지정해 적용할 수 있습니다.



직접 스타일 관련 속성을 지정한 경우에는 나중에 속성값을 변경해야 할 때 전체 소스를 수정해주어야 하는 불편함이 있습니다. 테마나 스타일 파일을 따로 생성해 관리하면 해당 파일만 수정해서 전체 애플리케이션에 반영할 수 있습니다.

37.5.3 예제 구현 방법

1 화면 구성하기

제목을 구성할 Static 컴포넌트를 예제 화면과 같이 배치합니다. 제목처럼 보이도록 font 속성값을 아래와 같이 지정합니다. 그리고 화면 왼쪽에 고정될 수 있도록 left 속성값을 "10 %"로 지정합니다.

font: normal bold 20pt Arial

font 속성값은 넥사크로 스튜디오 속성창에서 font 항목을 확장하면 세부항목을 지정할 수 있습니다. 예제에서는 font-weight, font-size만 수정하고 나머지 항목은 default 값을 사용합니다.

font	normal bold 20pt "Arial"
font-style	normal
font-weight	bold
font-size	20 pt
font-family	"Arial"

2 가로바 만들기

제목으로 지정한 Static 컴포넌트 아래 Static 컴포넌트를 하나 더 배치합니다. left, right 속성값은 "10 %"로 지정합니다. 이렇게 지정하면 왼쪽 면은 제목과 정렬된 상태를 유지하고 오른쪽 면은 화면 크기가 변경되더라도 10%의 간격을 떨어뜨릴 수 있습니다. height 속성값은 적당하게 지정합니다.



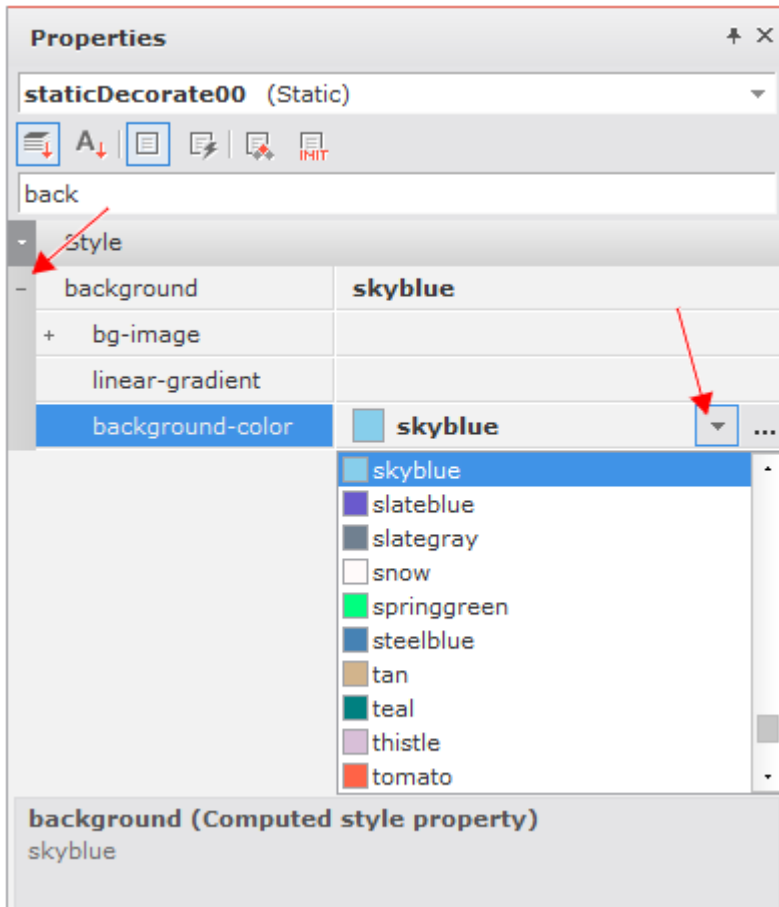
컴포넌트를 처음 배치했을 때 지정되는 Position 속성값은 left, top, width, height입니다. right 속성값을 지정하면 width 속성값에 지정한 값이 사라지고 left, right 두 개의 속성값을 기준으로 width를 계산해 처리합니다. 속성값이 지정되지 않은 항목은 비활성화된 것처럼 보이지만 해당 속성값을 지정하면 다시 활성화됩니다.

left	50	px	left	50	px
right		px	right	50	px
top	50	px	top	50	px
bottom		px	bottom		px
width	120	px	width		px
height	40	px	height	40	px

3 배경색 지정하기

가로바로 지정한 Static 컴포넌트의 background 속성값을 지정합니다. 속성창에서 background 속성값을 선택하면 내용을 확장할 수 있는 아이콘이 표시됩니다. 해당 아이콘을 클릭하면 background 속성값에서 지정할

수 있는 항목을 확인할 수 있습니다. 이번 예제에서는 배경색상값만 지정합니다. background-color 항목을 선택하고 목록에서 원하는 색을 선택합니다.



4 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 브라우저의 크기를 바꾸어가면서 가로바의 크기가 제대로 변경되는지 확인합니다.

37.6 글자 길이에 맞추어 배치하기

컴포넌트를 배치할 때 처음 정했던 텍스트가 운영상의 이유로 변경될 수 있습니다. 또는 다국어를 지원한다면 한국어와 영어를 변경할 때 텍스트 길이가 달라지기도 합니다. 텍스트가 변경되면 그때마다 컴포넌트의 배치를 바꾸어주고, 다국어를 지원해야 하는 경우에는 가장 긴 언어 설정의 길이를 찾아 맞추어 주기도 합니다.

넥사크로 17에서는 value 속성값에 맞게 컴포넌트의 크기를 조정해주는 Fit To Contents 속성을 지원합니다. 텍스트가 바뀌거나 다국어를 사용할 때 적용되는 값에 따라 컴포넌트의 크기를 조정하고 배치할 수 있습니다.

37.6.1 예제

예제에서는 한국어, 영어 2가지 언어를 선택할 수 있습니다. 선택한 언어가 Static 컴포넌트에 적용되어 표시됩니다. "Fit To Contents" 항목을 선택하기 전에는 텍스트 길이가 늘어나도 Static 컴포넌트는 넥사크로 스튜디오에서 디자인한 크기 그대로 보이며 텍스트가 길어지면 나머지는 잘려서 보이지 않습니다.

"Fit To Contents" 항목을 선택해주면 적용된 텍스트 길이에 따라 Static 컴포넌트의 크기가 달라집니다. "Fit To Contents" 항목을 선택했을 때 첫번째 "1. 이름" 항목은 언어를 영어로 선택하면 Static 영역이 Edit 컴포넌트 아래로 숨겨져 보이지 않습니다. Static 컴포넌트의 크기는 커졌지만, Edit 컴포넌트의 위치는 그대로이기 때문입니다. "2. 직장" 항목은 영어 선택 시 Edit 컴포넌트의 위치도 Static 컴포넌트가 커진 만큼 이동하도록 설정했습니다.

 sample_static_05.xfdl

37.6.2 예제에서 사용한 핵심 기능

fittococontents

컴포넌트의 콘텐츠에 따라 컴포넌트의 크기를 자동으로 설정합니다. 기본값은 "none"입니다. 넥사크로 스튜디오에서 지정된 크기 속성에 따라 표시됩니다. 설정할 수 있는 속성은 "width", "height", "both" 3가지입니다. 한 줄로 표시되어야 하는 경우에는 "width"로 설정하고 게시판에 등록한 글을 보여주는 경우에는 "height"로 설정해 높이를 자동 조정하도록 합니다.



속성값을 변경한 뒤에는 resetScroll 메소드를 호출해주어야 화면에 적용될 수 있습니다.

left

크기나 위치를 설정하는 속성은 픽셀 또는 비율로 크기를 지정할 수 있었습니다. 넥사크로 17에서는 Position Base를 지정하는 옵션이 추가되었습니다. 예제처럼 주변 컴포넌트의 크기나 위치가 가변적으로 변하는 경우 모든 컴포넌트의 위치나 크기를 하나하나 지정하는 것이 아니라 최초 디자인 작업 시 기준 컴포넌트를 정해주면 연쇄적으로 변경된 크기나 위치값을 적용할 수 있습니다.



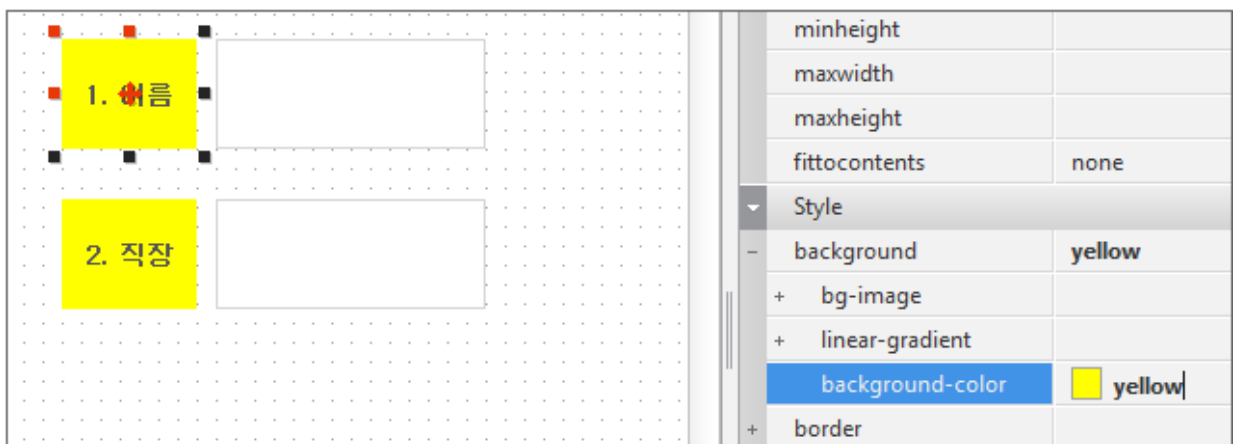
```
this.Static00.set_left("Button00:10px");
```

위와 같이 적용하면 Static00 컴포넌트의 위치는 Button00 컴포넌트 오른쪽 측면을 기준으로 10픽셀 떨어진 곳부터 시작한다는 의미입니다. Button00 컴포넌트의 위치가 변경되더라도, 오른쪽 측면으로 10픽셀의 간격을 항상 유지할 수 있습니다.

37.6.3 예제 구현 방법

① 화면 구성하기

Static 컴포넌트와 Edit 컴포넌트를 배치합니다. Static 컴포넌트의 value 속성값은 한글로 먼저 입력합니다. 컴포넌트의 크기 변화를 한눈에 확인하기 위해 Static 컴포넌트의 background 색을 지정합니다.



② 언어 변경하기

언어를 선택할 수 있도록 Radio 컴포넌트를 배치 합니다. codecolumn 항목은 "KOR", "ENG", datacolumn 항목은 "한국어", "영어"로 지정합니다.

onitemchanged 이벤트 함수를 아래와 같이 지정합니다. 실제 운영 시스템에서 다국어를 설정할때는 별도 Dataset을 구성하는데 이번 예제에서는 항목이 적기 때문에 선택된 아이템에 따라 text 속성값을 직접 지정합니다.

onitemchanged 이벤트 발생 시 ItemChangeEventInfo 오브젝트에 관련 정보를 담습니다. 선택한 codecolumn 항목의 값은 postvalue 속성값으로, datacolumn 항목의 값은 posttext 속성값으로 확인할 수 있습니다.

예제에서는 postvalue 속성값을 비교합니다.

```
this.radioLang_onitemchanged = function(obj:nexacro.Radio,e:nexacro.ItemChangeEventInfo)
{
    if(e.postvalue == "KOR")    {
        this.staticName.set_text("1. 이름");
        this.staticWorkplace.set_text("2. 직장");
    } else if(e.postvalue == "ENG")    {
        this.staticName.set_text("1. First name");
        this.staticWorkplace.set_text("2. place of work");
    }
    this.resetScroll();
};
```

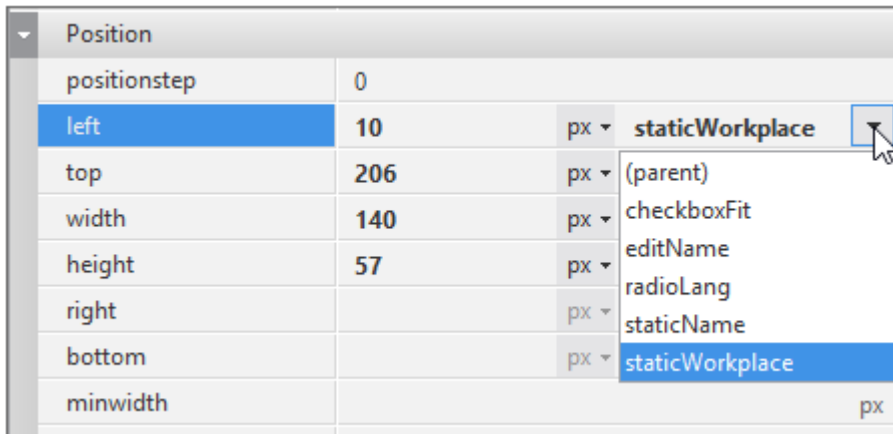
③ Fit To Contents 적용하기

Static 컴포넌트의 fittocontents 속성값을 설정할 수 있도록 checkbox 컴포넌트를 배치합니다. onchanged 이벤트 함수에서 체크 여부에 따라 fittocontents 속성값을 설정합니다.

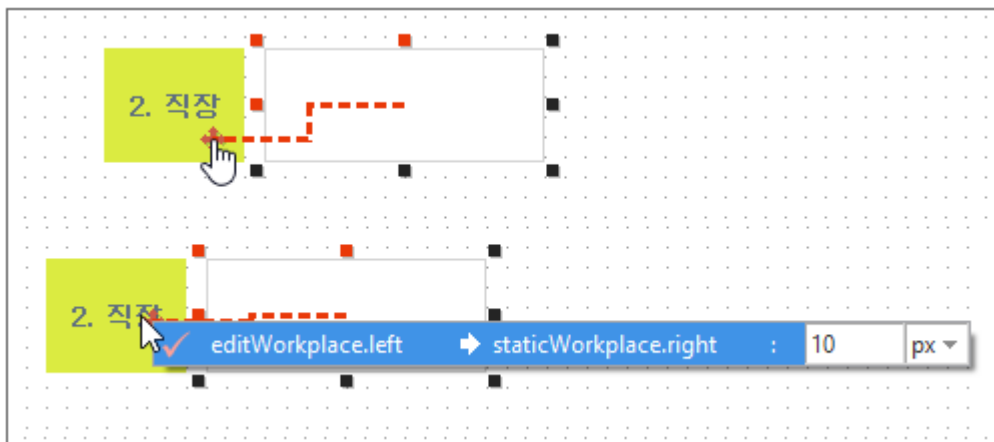
```
this.checkboxFit_onchanged = function(obj:nexacro.CheckBox,e:nexacro.
CheckBoxChangedEventInfo)
{
    if(e.postvalue == true)    {
        this.staticName.set_fittocontents('width');
        this.staticWorkplace.set_fittocontents('width');
    } else if(e.postvalue == false)    {
        this.staticName.set_fittocontents('none');
        this.staticWorkplace.set_fittocontents('none');
    }
    this.resetScroll();
};
```

④ PositionBase 적용하기

PositionBase는 속성창에서 직접 적용하거나 넥사크로 스튜디오 디자인 창에서 마우스로 컨트롤할 수 있습니다. 속성창에서 직접 적용하는 경우에는 적용할 속성 항목을 선택하고 기준이 되는 컴포넌트와 기준 컴포넌트와의 간격을 지정해줍니다.



디자인 창에서 지정하는 경우에는 컴포넌트를 선택하고 가운데 표시되는 십자화살표 모양을 드래그해서 기준이 되는 컴포넌트 위로 가져가 마우스 버튼을 놓으면 어떤 속성 항목을 설정할지 표시하는 작은창이 나타납니다. 연결되는 방향에 따라 가장 인접한 속성 항목을 자동으로 설정합니다.



5 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 언어와 Fit To Contents 설정을 변경해가면서 컴포넌트의 크기와 배치가 어떻게 바뀌는지 확인합니다.

37.7 여러줄로 문자열 표시하기

Static 컴포넌트 내에는 여러 줄로 문자열을 입력할 수 있습니다. 속성창에서 Ctrl+ENTER 키를 입력해 줄을 바꿀 수 있습니다. 하지만 스크립트 내에서 문자열을 조합하는 경우에는 이런 방법을 사용할 수 없습니다. 이번 예제에서는 스크립트 내에서 여러줄로 문자열을 표시하는 방법을 알아봅니다.

37.7.1 예제

Edit 컴포넌트에 입력된 2개의 문자열을 2줄로 Static 컴포넌트에 표시합니다.



37.7.2 예제에서 사용한 핵심 기능

fromCharCode

String.fromCharCode(10)를 호출했을때 줄바꿈 문자를 반환합니다.

37.7.3 예제 구현 방법

① 화면 구성하기

Edit 컴포넌트를 2개 배치하고 Button 컴포넌트를 추가합니다. 결과값을 보여줄 Static 컴포넌트를 적당한 크기로 배치합니다.

② Button 컴포넌트 onclick 이벤트 추가하기

아래와 같이 Button 컴포넌트의 이벤트를 추가합니다. 입력된 값을 더해서 Static 컴포넌트에 표시해주는 기능을 처리합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Static00.set_text(this.Edit00.value
```

```
+String.fromCharCode(10)
+this.Edit01.value);
};
```

③ QuickView로 확인하기

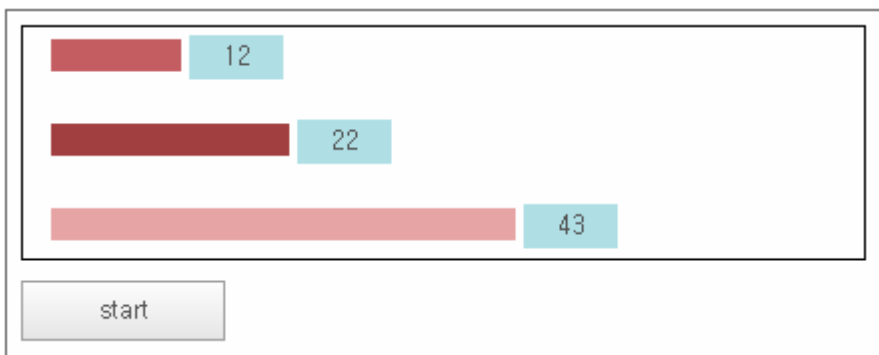
QuickView(Ctrl + F6)로 실행한 후 Edit 컴포넌트에 입력한 2개의 문자열이 여러줄로 표시되는지 확인합니다.

37.8 Bar 차트 만들기

Static 컴포넌트는 문자열을 표현하는 용도로도 사용하지만 간단한 사각형을 표현하는 용도로도 사용합니다. 이미 [가 로바 만들기](#) 예제에서도 다루었는데, 이를 활용하면 간단한 Bar 차트도 만들어볼 수 있습니다.

37.8.1 예제

버튼을 클릭하면 Bar 차트가 애니메이션 효과를 적용하면서 표시됩니다.



 [sample_static_07.xfdl](#)

37.8.2 예제에서 사용한 핵심 기능

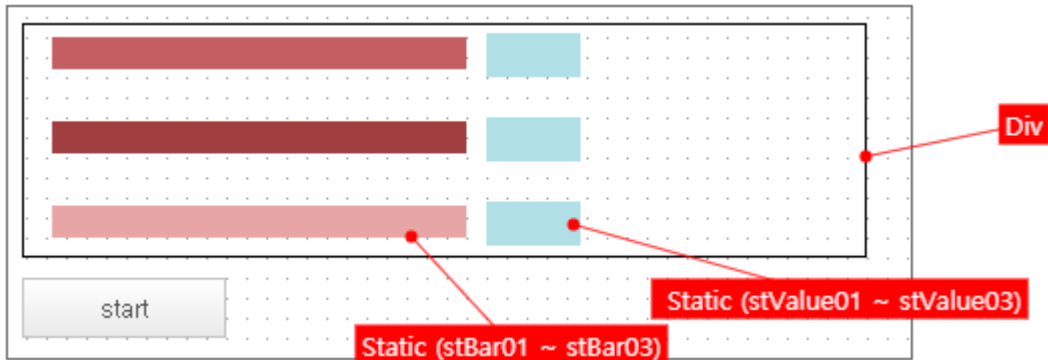
components

Form 오브젝트 내 컴포넌트 이름이나 인덱스를 알고 있다면 컴포넌트를 동적으로 접근할 수 있습니다. 예를 들어 Button00이라는 컴포넌트에 접근할 때 `this.components["Button00"]`으로 접근할 수 있습니다.

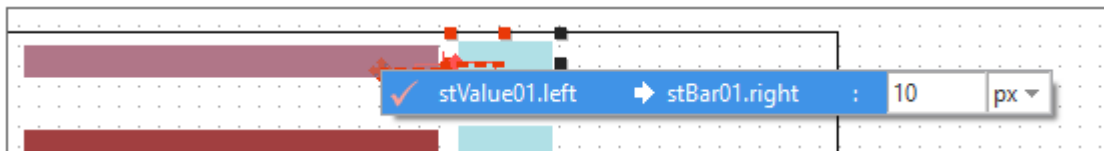
37.8.3 예제 구현 방법

1 화면 구성하기

아래 그림과 같이 컴포넌트를 배치합니다.



stValue로 시작하는 Static 컴포넌트는 stBar로 시작하는 컴포넌트를 기준으로 위치를 지정합니다. 이렇게 하면 데이터에 따라 Bar 차트를 움직일 때 stBar로 시작하는 컴포넌트만 이동하고 stValue로 시작하는 컴포넌트는 이를 따라가도록 합니다.



2 Dataset 생성하기

예제에서는 Bar 차트에서 사용하는 데이터를 Dataset에서 가져옵니다. Dataset을 생성하고 아래와 같이 데이터를 입력합니다.

Dataset Editor [ds_Chart01]				
Const Columns				
Columns				
No	id	type	size	prop
1	Lable	STRING	256	
2	Value	STRING	256	
Rows				
No	Lable	Value		
1	item1	12		
2	item2	22		
3	item3	43		

3 타이머 이벤트 시작하기

버튼 클릭 시 setTimer 메소드를 실행합니다. form 오브젝트의 ontimer 이벤트 함수도 같이 만들어줍니다. 타이머 이벤트가 발생할때마다 fn_Chart01 함수를 실행합니다. 해당 함수에서 Static 컴포넌트의 너비를 적당하게 조정해줍니다. fn_Chart01 함수 실행 후에는 resetScroll 메소드를 실행하는데 stValue로 시작하는 컴포넌트의 위치를 조정해주기 위한 용도입니다.

```

this.nTimerIdx = 0;
this.nTimerMax = 20;

this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.setTimer(1, 50);
};

this.BarChart_ontimer = function(obj:nexacro.Form,e:nexacro.TimerEventInfo)
{
    if(e.timerid==1)
    {
        this.nTimerIdx++;
        this.fn_Chart01();
        this.divChart1.form.resetScroll();
        if(this.nTimerIdx==this.nTimerMax)
        {
            this.nTimerIdx = 0;
            this.killTimer(1);
        }
    }
};

```

4 fn_Chart01 함수 구현하기 (차트의 너비 변경하기)

Dataset 오브젝트에 row 만큼 for 반복문을 실행합니다. Static 컴포넌트의 left, top, height 값은 그대로 유지하고 width 값만 바꾸어 move 메소드를 실행합니다. 아래 예제에서는 move 메소드를 사용하지 않고 width 속성값만 바꾸어도 같은 결과가 나옵니다.

```

this.fn_Chart01 = function ()
{
    var i;
    var nLeft;
    var nTop;

```

```

var nWidth;
var nHeight;
var objBar;
var objValue;
var nValue;

for(i=0;i<this.ds_Chart01.rowcount;i++)
{
    objBar = this.divChart1.form.components["stBar0"+(i+1)];
    objValue = this.divChart1.form.components["stValue0"+(i+1)];

    nValue = Math.round(this.ds_Chart01.getColumn(i, "Value")*5.4);
    objValue.set_text(this.ds_Chart01.getColumn(i, "Value"));

    nLeft = objBar.left;
    nTop = objBar.top;
    nWidth = this.fn_move(this.nTimerIdx, 0, nValue, this.nTimerMax);
    nHeight = objBar.height;

    objBar.move(nLeft, nTop, nWidth, nHeight);
}
};

```

fn_move 함수는 Bar 차트를 타이머 이벤트가 발생했을때 얼마나 크기를 변경할 것인지를 계산합니다. 전체 타이머 동작 횟수와 value 값, 현재 타이머 회차를 가지고 값을 계산합니다.

```

this.fn_move = function(t, b, c, d)
{
    t /= d/2;
    if (t < 1) return Math.round(c/2*t*t + b);
    t--;
    return Math.round(-c/2 * (t*(t-2) - 1) + b);
};

```

5 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 Bar 차트가 움직이는 것을 확인합니다.

37.9 아코디언 UI 만들기

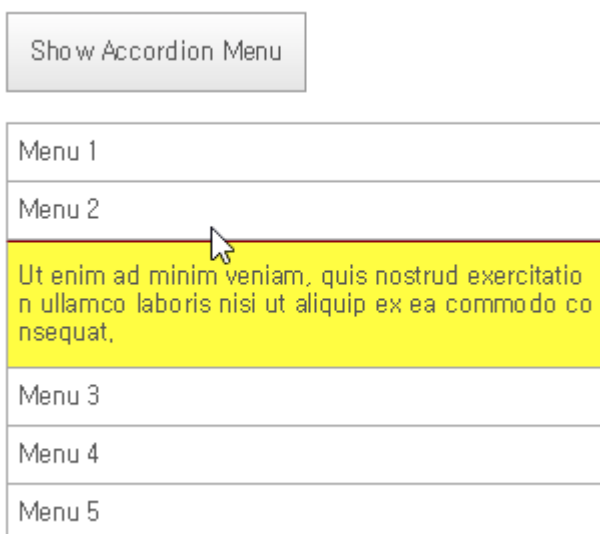
상세 항목은 숨겨놓고 제목 선택 시 상세 항목을 보여주는 형식을 아코디언 UI 라고 합니다. FAQ 같은 화면 구성 시 주로 사용합니다. 예제에서는 PositionBase 기능과 fittocontents 속성을 활용해 아코디언 UI 형식을 만들어봅니다.

PositionBase

http://docs.tobesoft.com/development_tools_guide_nexacro_17_ko#ef9f3d73e4065584

37.9.1 예제

버튼을 클릭하면 기본 메뉴 타이틀이 표시되고 메뉴 타이틀 클릭 시 세부 설명이 표시됩니다.



 [sample_static_08.xfdl](#)

37.9.2 예제에서 사용한 핵심 기능

fittocontents

컴포넌트 콘텐츠의 크기에 따라 콘텐츠의 크기를 조정합니다. 예제에서는 Dataset 오브젝트의 데이터에 따라 Static 컴포넌트의 높이를 조정합니다.

37.9.3 예제 구현 방법

- 1 Dataset 오브젝트를 추가하고 2개의 컬럼을 설정합니다. 첫 번째 컬럼은 메뉴 타이틀이 되고, 두 번째 컬럼은 상세 항목이 됩니다. 상세 항목에는 적당한 분량의 텍스트를 설정합니다.

Const Columns				
Columns				
No	id	type	size	prop
1	Column0	STRING	256	
2	Column1	STRING	256	
Rows				
No	Column0	Column1		
1	Menu 1	Lorem ipsum dolor sit a...		
2	Menu 2	Ut enim ad minim venia...		
3	Menu 3	Duis aute irure dolor in r...		
4	Menu 4	Excepteur sint occaecat ...		
5	Menu 5	Lorem ipsum dolor sit a...		

- 2 Button 컴포넌트를 화면에 배치하고 아래와 같이 onclick 이벤트 함수를 작성합니다.

Dataset 오브젝트의 rowcount 만큼 Static 컴포넌트를 생성하고 배치합니다. 첫 번째 Static 컴포넌트는 지정된 top 위치에 배치되고 두 번째 이후의 Static 컴포넌트는 바로 전에 만든 Static 컴포넌트를 기준으로 위치가 지정됩니다.

메뉴 타이틀 클릭 시 보여지는 상세 항목의 Static 컴포넌트도 같이 생성합니다. 각 Row에 따라 Static 컴포넌트의 id 속성값을 지정하고 height 속성값은 0으로 지정합니다.

```

this.preObj = "";

this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var nTop = 100;

    for(var i = 0; i < this.Dataset00.rowcount; i++)
    {
        var objStatic = new Static();
    }
}

```



```

objStatic.init("Static"+i, 30, 70, 300, 30);
this.addChild("Static"+i, objStatic);
objStatic.set_border("1px solid darkgray");
objStatic.set_padding("0px 0px 0px 5px");
objStatic.show();

objStatic.addEventHandler("onclick", this.StaticAll_onclick, this);
objStatic.set_text(this.Dataset00.getColumn(i, "Column0"));

if(i != 0)
{
    var imgID = "Static"+(i-1)+"_1";
    objStatic.set_top(imgID+":-1");
}

var objStatic1 = new Static();
objStatic1.init("Static"+i + "_1", 30, nTop+60, 300, 0, null, null);
this.addChild("Static"+i + "_1", objStatic1);
objStatic1.set_border("1px solid darkred, 1px solid darkgray, 1px solid darkgray"
);

objStatic1.set_background("yellow");
objStatic1.set_wordWrap("char");
objStatic1.show();

objStatic1.set_text(this.Dataset00.getColumn(i, "Column1"));
objStatic1.set_top(objStatic.id+":0px");
nTop += 40;

}
};

```

메뉴 타이틀을 구성하는 Static 컴포넌트는 모두 StaticAll_onclick 이벤트 함수와 연결됩니다. 아래와 같이 이벤트 함수 코드를 작성합니다. 메뉴 타이틀 클릭 시 Static 컴포넌트의 id 속성값을 확인하고 해당하는 상세 항목의 Static 컴포넌트 크기를 조정합니다. 이전에 선택한 항목이 있다면 해당 Static 컴포넌트는 초기화합니다.

```

this.StaticAll_onclick = function(obj:nexacro.Static,e:nexacro.ClickEventInfo)
{
    if(this.preObj != "")
    {
        var objID = this.preObj + "_1";
        this.components[objID].set_fittocontents("none");
    }
}

```

```

        this.components[objID].set_height(0);
    }

    if(obj.id == this.preObj)
    {
        this.preObj = "";
        this.resetScroll();
        return;
    }
    else
    {
        var objID1 = obj.id + "_1";
        this.components[objID1].set_fittocontents("height");
        this.components[objID1].set_padding("10px 5px");
        this.resetScroll();
    }
    this.preObj = obj.id;
};

```

- ③ QuickView(Ctrl + F6)로 실행한 후 메뉴 타이틀 항목을 클릭해 상세 항목이 표시되는지 확인합니다.

37.10 특정 구분자를 줄바꿈 문자로 바꾸기

저장된 데이터는 콤마(,)로 구분하지만, 화면에 표시할 때는 줄을 바꾸어 표시해야 하는 경우 적용할 수 있습니다.

37.10.1 예제

콤마(,)로 구분된 텍스트를 선택하면 Static 컴포넌트에 줄바꿈 문자를 추가해서 여러 줄로 표시합니다.

Column0
ABC,A,B,C,D,E
Lorem ipsum dolor sit amet, conse
1,2,3,4,5
Lorem ipsum dolor sit amet consectetur adipiscing elit sed do eiusmod tempor incididunt ut labore et dolore magna aliqua,



37.10.2 예제에서 사용한 핵심 기능

expr

Static 컴포넌트에 표시되는 텍스트를 동적으로 설정할 수 있습니다. 예제에서는 외부 함수를 사용해 설정한 값을 표시하도록 했습니다.

37.10.3 예제 구현 방법

- 1 Dataset 오브젝트를 추가하고 1개의 컬럼을 설정합니다. 데이터는 콤마(,)가 들어간 항목으로 적절하게 설정합니다.

Columns				
No	id	type	size	prop
1	Column0	STRING	256	
Rows				
No	Column0			
1	ABC,A,B,C,D,E			
2	Lorem ipsum dolor sit amet, consectetur adipiscing...			
3	1,2,3,4,5			

- 2 Grid 컴포넌트와 Static 컴포넌트를 배치하고 Dataset 오브젝트와 바인딩합니다. Static 컴포넌트는 text 속성과 연결해줍니다.
- 3 Static 컴포넌트의 expr 속성값에 아래와 같이 설정합니다.

```
comp.parent.setWordWrap(comp.text)
```

- 4 setWordWrap 함수를 아래와 같이 작성합니다.

text 속성값에서 콤마(,)를 찾아서 줄바꿈 문자로 변경합니다. 줄바꿈 처리 후 공백문자가 포함된 경우에는 공백문자를 제거해줍니다.

```
this.setWordWrap = function(sValue)
{
    return nexacro.replaceAll(sValue, ",", "\n").replace(/^\s+/gm, '');
}
```

- ⑤ QuickView(Ctrl + F6)로 실행한 후 Grid 컴포넌트의 항목을 선택하고 Static 컴포넌트에 어떻게 보여지는지 확인합니다.

38.

TextArea

38.1 TextArea 소개

TextArea 컴포넌트는 텍스트 정보를 입력하는 수단 중 하나입니다. Edit 컴포넌트는 1줄로 된 짧은 항목을 입력하는 용도로 사용한다면 TextArea 컴포넌트는 1줄 이상의 문장 형태의 정보를 입력합니다. 이메일을 작성할 때 제목 입력은 Edit 컴포넌트를 사용하고 본문은 TextArea 컴포넌트를 사용해 입력할 수 있습니다.



위의 예에서 이메일 본문 작성 시 TextArea 컴포넌트를 사용한다고 했지만, 보통은 WYSIWYG HTML Editor 또는 Rich Text Editor라는 별도 모듈을 사용합니다. TextArea 컴포넌트는 1줄 이상의 텍스트 입력 기능만 제공하고 입력된 문장을 꾸미는 기능은 지원하지 않습니다.

38.1.1 사용 예

웹에서 가장 흔하게 볼 수 있는 것이 댓글 입력창입니다. PC나 모바일 기기에서 짧은 의견을 남기는 기능을 제공합니다.

Few understand this better than Facebook CEO Mark Zuckerberg. We just had a pretty great conversation with some young entrepreneurs from across the globe and now I'm joining him one-on-one to discuss how we can enable budding entrepreneurs - especially women, youth, and minorities - to connect with new audiences, build new partnerships, and unleash their full potential.

Tune in!

image source: <https://www.facebook.com/potus>



554K Views

Like Comment Share

Anton Krasovsky, Dave Morin and 33K others

Top Comments ▾

7,316 shares



Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.



38.1.2 TextArea 만들기

1 TextArea 컴포넌트 생성하기

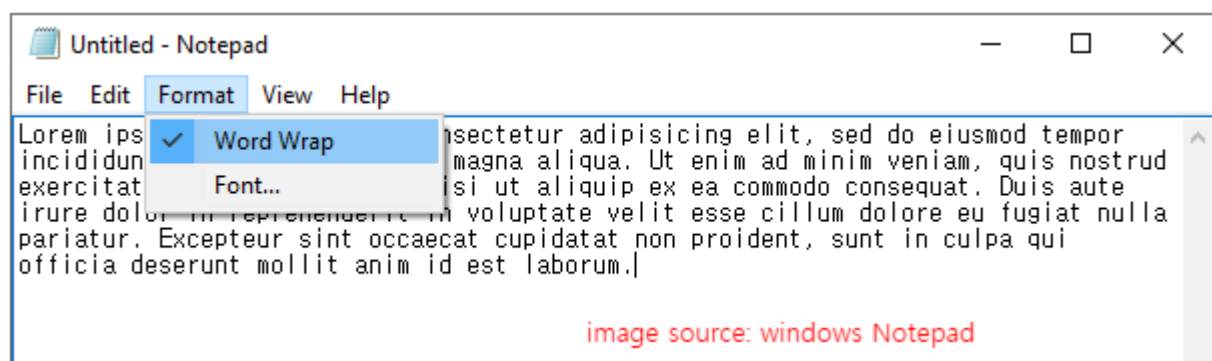
툴바에서 TextArea를 선택한 후, Form 위에 드래그 앤 드롭해 생성합니다.

2 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행합니다. 1줄 이상의 텍스트가 입력되는지 확인해봅니다.

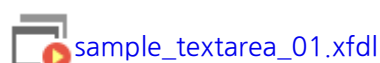
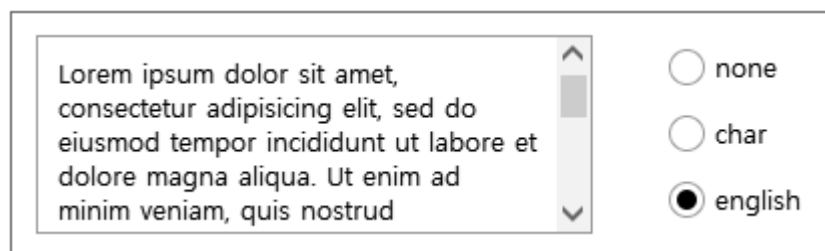
38.2 자동줄넘김(wordwrap) 기능

TextArea 컴포넌트를 아무 설정 없이 화면에 배치한 경우에는 텍스트를 입력했을 때 엔터키를 입력하지 않으면 가로 스크롤바가 생기면서 한 줄로 입력됩니다. 윈도우 운영체제에 설치된 메모장 앱도 자동줄넘김 설정을 하지 않으면 한 줄로 텍스트가 입력됩니다.



38.2.1 예제

최초 실행 시에는 wordwrap 속성을 적용하지 않고 TextArea 컴포넌트 오른쪽에 있는 Radio 컴포넌트 선택 시 해당 속성을 적용합니다.



38.2.2 예제에서 사용한 핵심 기능

wordwrap

컴포넌트의 영역을 벗어나는 입력값을 줄바꿈해서 보여줄지 지정하는 속성입니다. 'none', 'char', 'english' 속성 중 하나를 선택할 수 있습니다. 'english'를 선택하면 단어 단위로 줄바꿈을 처리하고 'char'를 선택하면 단어 단위와 상관없이 길이에 따라 줄바꿈을 처리합니다.



CJK(Chinese/Japanese/Korean) 문자는 'english', 'char' 속성값 선택에 따른 차이가 없습니다(아예 차이가 없는 것은 아닙니다. 공백 문자 처리 방식이 다르므로 'english' 속성값을 선택하는 것이 깔끔해 보일 수 있습니다).



각 속성값은 HTML5 Element에서 아래와 같이 처리됩니다 (CSS3, HTML5를 지원하는 브라우저인 경우).

HTML5 Element	none	char	english
white-space	pre	pre-wrap	pre-wrap
overflow-wrap	normal	break-word	break-word
word-break	-	break-all	-
wrap	hard	hard	hard



웹브라우저와 속성 조합 조건에 따라 처리 방식은 조금씩 달라질 수 있습니다. 예를 들어 IE 브라우저에서는 -ms-word-break, -ms-word-wrap 속성으로 처리될 수 있습니다.



<https://developer.mozilla.org/ko/docs/Web/CSS/white-space>
<https://developer.mozilla.org/en-US/docs/Web/CSS/overflow-wrap>
<https://developer.mozilla.org/en-US/docs/Web/CSS/word-break>
<https://developer.mozilla.org/en-US/docs/Web/HTML/Element/textarea>

wrap 속성은 TextArea 컴포넌트에만 해당하는 처리방식입니다.

38.2.3 예제 구현 방법

① 화면 구성하기

TextArea, Radio 컴포넌트를 예제 화면과 같이 배치합니다. Radio 컴포넌트의 innerdataset 속성값은 아래와 같이 지정합니다. TextArea 컴포넌트의 value 속성값은 원하는 텍스트를 입력합니다. 예제에서는 Lorem ipsum 텍스트를 사용했습니다.

InnerDataset		
No	codecolumn	datacolumn
1	none	none
2	char	char
3	english	english



innerdataset 직접 지정하기



value 속성값을 미리 설정하지 않고 예제 실행 후 직접 문자열을 입력해 테스트할 수도 있습니다.

2 Radio 컴포넌트 이벤트 함수 작성하기

Radio 컴포넌트를 선택한 후 onitemchanged 이벤트 함수를 작성합니다. Radio 컴포넌트에서 항목 선택 시 TextArea 컴포넌트의 wordWrap 속성값으로 지정합니다.

```
this.radioWordwrap_onitemchanged = function(obj:nexacro.Radio,e:nexacro.ItemChangeEventInfo)
{
    this.textareaWordwarp.set_wordWrap(this.radioWordwrap.value);
};
```

3 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 속성값을 변경하면서 확인해봅니다. 다른 텍스트값을 TextArea 컴포넌트에 입력하고 테스트해볼 수도 있습니다.

39.

VideoPlayer

39.1 VideoPlayer 소개

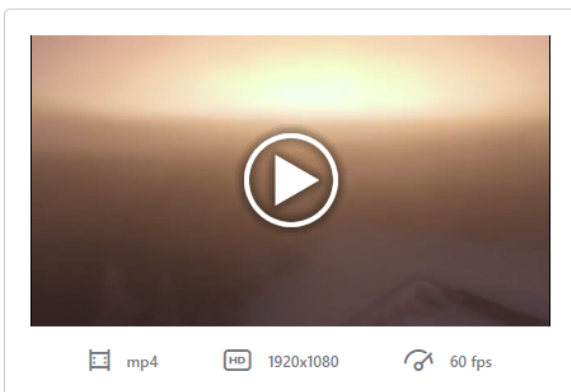
다양한 종류의 콘텐츠로 구성된 웹 환경에서 미디어 플레이어는 필수 요건이 되었습니다. 기존에 개인이 휴대용 장치에 넣어 다니며 사용하던 비디오, 오디오 파일을 웹을 통해 저장 공간의 제약 없이 전문적으로 제공하는 서비스가 주목받고 있습니다. 이런 추세에 맞춰 넥사크로플랫폼에서는 미디어를 재생하고 제어할 수 있는 VideoPlayer 컴포넌트를 제공하고 있습니다.

VideoPlayer는 비디오, 오디오 파일 및 스트리밍을 재생할 때 사용하는 컴포넌트입니다. 간단한 속성의 설정만으로 미디어 재생에 필요한 재생/정지 버튼, 진행표시줄, 전체 화면, 볼륨 조절 등의 기능을 갖춘 UI 컨트롤을 기본으로 제공합니다. 따라서 개발자는 제어 기능을 별도로 구현할 필요 없이 미디어를 이용한 응용 애플리케이션을 빨리 개발할 수 있습니다. 만일 기본 컨트롤 기능을 사용하지 않고 직접 컨트롤 기능을 구현하고 싶다면 제어에 필요한 다양한 속성 및 메소드를 이용해 스크립트로 구현하는 것도 가능합니다.

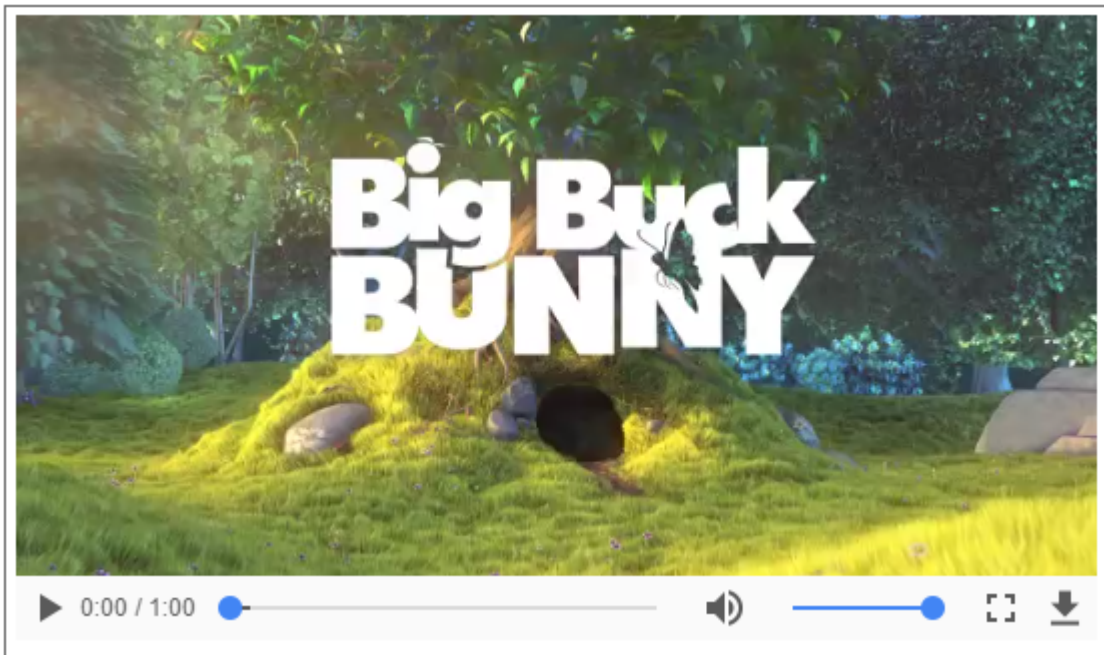
VideoPlayer는 webm, ogg, mp4(h.264) mp3, wav, flac 등의 다양한 비디오, 오디오 형식을 지원합니다. 자세한 내용은 도움말을 참고하세요.

39.1.1 사용 예

다음은 미디어 서비스를 제공하는 어느 사이트의 미디어 플레이어 예를 보여줍니다. 사용자는 원하는 미디어를 재생하는 것은 물론 컨트롤을 사용하여 해상도, 프레임까지 조절하며 미디어를 즐길 수 있습니다.



39.1.2 VideoPlayer 만들기



 [sample_videoplayer_01.xfdl](#)

① VideoPlayer 컴포넌트 생성하기

툴바에서 VideoPlayer를 선택한 후, Form 위에 적당한 크기로 드래그하여 생성합니다.



만일 툼바에서 VideoPlayer 아이콘이 보이지 않거나 컴포넌트를 사용할 수 없을 때는 [비활성화되어 있는 컴포넌트 사용하기](#)를 참조하여 VideoPlayer 컴포넌트를 사용할 수 있도록 합니다.

② VideoPlayer 속성 설정하기

VideoPlayer를 사용하려면 먼저 url 속성에 미디어의 위치를 설정해야 합니다. 아래 표의 값은 샘플에서 사용하는 값이므로 개발 시에는 파일이 실제 있는 URL을 설정해 주십시오.

VideoPlayer를 재생하거나 조절하려면 UI 컨트롤이 있어야 하는데 이는 showcontrolbar 속성을 true로 설정해야 사용할 수 있습니다. 컨트롤에는 기본적으로 재생, 정지 버튼, 진행표시줄, 볼륨 조절, 전체화면, 파일 다운로드 기능이 있습니다.

속성	값
url	https://github.com/mediaelement/mediaelement-files/blob/master/big_buck_bunny.mp4?raw=true
showcontrolbar	true

3 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

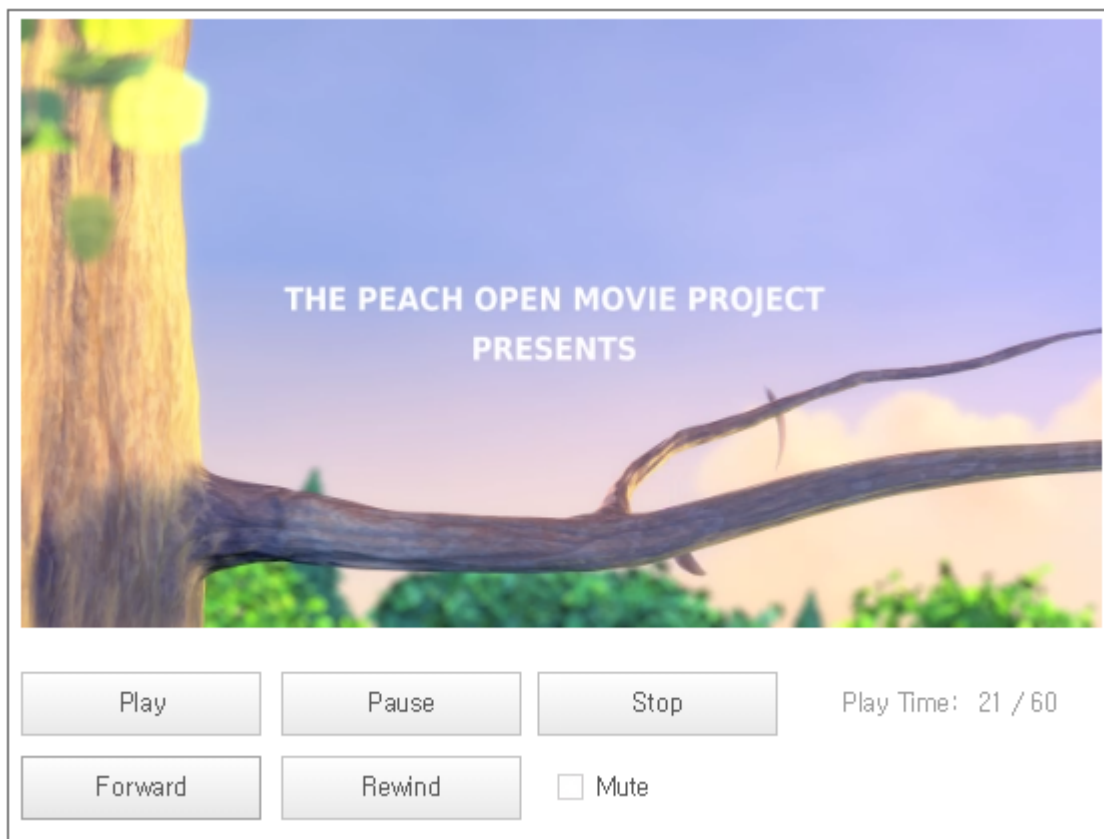
컨트롤에서 플레이 버튼을 눌러 비디오를 재생시킵니다. 정상적으로 비디오가 재생되는지 확인하고 각 기능을 사용해 봅니다.

39.2 컨트롤 기능 만들기

VideoPlayer 컴포넌트에서 제공하는 여러 기능을 사용하면 컨트롤을 직접 구현할 수 있습니다. 이번 장에서는 Video Player를 제어할 때 필요한 속성과 메소드, 이벤트를 사용해 컨트롤 기능을 만들어 봅니다.

39.2.1 예제

다음은 VideoPlayer 컴포넌트의 여러 속성, 메소드, 이벤트를 사용하여 콘텐츠를 재생하는 플레이어 예제입니다.



sample_videoplayer_02.xfdl

- Play 버튼은 파일을 재생합니다. 재생이 시작되면 우측의 Play Time에 재생 시간이 초 단위로 표시됩니다.
- Pause, Stop 버튼은 각각 재생을 일시 중지시키는 기능과 멈추는 기능을 합니다.
- Forward 버튼은 영상을 2초만큼 앞으로, Rewind 버튼은 영상을 2초만큼 뒤로 이동합니다.
- Mute 체크박스를 체크하면 음소거 됩니다.

39.2.2 예제에서 사용한 핵심 기능

play

영상 파일을 재생하는 메소드입니다. 영상을 재생하면 oncurrenttimechanged 이벤트가 발생합니다.

pause

재생중인 영상을 일시 정지하는 메소드입니다.

stop

영상 재생을 중지시키고 재생 위치를 처음으로 되돌리는 메소드입니다.

currenttime

영상의 재생 위치(시간)를 설정하는 속성입니다. 단위는 ms(밀리초)입니다.

duration

영상 파일의 총 재생 시간을 갖는 읽기 전용 속성입니다.

mute

영상 재생시 음소거를 설정하는 속성입니다. true로 설정하면 음소거됩니다.

oncurrenttimechanged

영상의 재생 위치가 변경될 때 발생하는 이벤트입니다. 영상이 재생되면 oncurrenttimechanged 이벤트가 지속적으로 발생합니다.

onplaystatuschanged

영상의 재생 상태가 변경될 때 발생하는 이벤트입니다.

newstate

영상의 재생 상태 정보를 갖는 속성입니다. 웹 브라우저에서는 상태에 따라 stop, pause, play, buffer, ended의 다섯 가지 값을 갖습니다.

39.2.3 예제 구현 방법

① 컴포넌트 생성하기

툴바에서 VideoPlayer 1개, Button 5개, CheckBox 1개, Static 2개를 위의 예제와 같이 Form 위에 적당한 크기로 드래그하여 생성, 배치합니다.

2 컴포넌트 속성 설정하기

생성한 컴포넌트의 속성을 다음과 같이 설정합니다.

컴포넌트	속성	값
Button00 (Play)	id	btn_play
	text	Play
Button01 (Pause)	id	btn_pause
	text	Pause
Button02 (Stop)	id	btn_stop
	text	Stop
Button03 (Forward)	id	btn_forward
	text	Forward
Button04 (Rewind)	id	btn_rewind
	text	Rewind
CheckBox00 (Mute)	id	chk_mute
	text	Mute
Static00	id	stt_playtime
	text	
Static01	id	stt_playtime_title
	text	Play Time:

3 이벤트 함수 작성하기

- Play 버튼의 onclick 이벤트 함수

Play 버튼을 선택한 후 onclick 이벤트 함수를 다음과 같이 추가합니다.

```
this.btn_play_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.VideoPlayer00.set_url("https://github.com/mediaelement/mediaelement-files/blob/master/big_buck_bunny.mp4?raw=true");
    this.VideoPlayer00.play();
};
```

VideoPlayer00 컴포넌트의 url 속성에 영상 파일의 URL을 설정합니다. play 메소드를 호출하면 url 속성에 설정한 파일이 재생됩니다.

- Pause 버튼의 onclick 이벤트 함수

Pause 버튼을 선택한 후 onclick 이벤트 함수를 다음과 같이 추가합니다.

```
this.btn_pause_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.VideoPlayer00.pause();
};
```

- Stop 버튼의 onclick 이벤트 함수

Stop 버튼을 선택한 후 onclick 이벤트 함수를 다음과 같이 추가합니다.

```
this.btn_stop_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.VideoPlayer00.stop();
};
```

- Forward 버튼의 onclick 이벤트 함수

Forward 버튼을 선택한 후 onclick 이벤트 함수를 다음과 같이 추가합니다.

```
this.btn_forward_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.VideoPlayer00.set_currenttime(this.VideoPlayer00.currenttime + 2000);
};
```

VideoPlayer00의 currenttime 속성은 영상의 재생 위치를 밀리 초(ms) 단위로 설정합니다. 다음 코드는 현재 위치에서 2초만큼 앞으로 재생 위치를 설정합니다.

- Rewind 버튼의 onclick 이벤트 함수

Rewind 버튼을 선택한 후 onclick 이벤트 함수를 다음과 같이 추가합니다.

```
this.btn_rewind_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.VideoPlayer00.set_currenttime(this.VideoPlayer00.currenttime - 2000);
};
```

VideoPlayer00의 currenttime 속성은 영상의 재생 위치를 밀리 초(ms) 단위로 설정합니다. 다음 코드는 현재 위치에서 2초만큼 뒤로 재생 위치를 설정합니다.

- Mute 체크박스의 onchange 이벤트 함수

Mute 체크박스를 선택한 후 onchange 이벤트 함수를 다음과 같이 추가합니다.

```

this.chk_mute_onchanged = function(obj:nexacro.CheckBox,e:nexacro.CheckBoxChangedEventInfo
)
{
    if(e.postvalue)
    {
        this.VideoPlayer00.set_mute(true);
    }
    else
    {
        this.VideoPlayer00.set_mute(false);
    }
};

```

e.postvalue는 변경된 체크박스의 값을 갖습니다. 체크박스가 체크되면 VideoPlayer00의 mute 속성을 true로 설정하고, 체크박스가 해제되면 VideoPlayer00의 mute 속성을 false로 설정합니다.

- VideoPlayer의 oncurrenttimechanged 이벤트 함수

VideoPlayer를 선택한 후 oncurrenttimechanged 이벤트 함수를 다음과 같이 추가합니다.

```

this.VideoPlayer00_oncurrenttimechanged = function(obj:nexacro.VideoPlayer,e:nexacro.
VideoCurrentTimeChangedEventInfo)
{
    var strPlayTime = nexacro.round(this.VideoPlayer00.currenttime / 1000) + " / " +
nexacro.round(this.VideoPlayer00.duration / 1000);

    this.stt_playtime.set_text(strPlayTime);
};

```

oncurrenttimechanged는 VideoPlayer에서 재생 위치가 변경되면 발생하는 이벤트입니다. 영상의 총 재생 시간과 현재 시간을 구해 화면의 Static 컴포넌트에 표시합니다.

재생 시간은 밀리초 단위이므로 1000으로 나누어 초 단위로 환산합니다. 환산한 시간은 소수점을 포함하므로 사용자가 보기편하도록 round 메소드를 사용해 반올림한 정수로 계산합니다. 계산된 결과는 (현재 재생 시간) / (총 재생 시간) 형식으로 표시됩니다.

- VideoPlayer의 onplaystatuschanged 이벤트 함수

VideoPlayer를 선택한 후 onplaystatuschanged 이벤트 함수를 다음과 같이 추가합니다.


```

this.VideoPlayer00_onplaystatuschanged = function(obj:nexacro.VideoPlayer,e:nexacro.
VideoPlayStateChangedEventInfo)
{
    switch(e.newstate)
    {
        case "stop":
            this.stt_playtime.set_text(" ");
        case "pause":
        case "play":
        case "buffer":
        case "ended":
        default:
            trace("state: "+ e.newstate);
    }
};

```

onplaystatuschanged는 VideoPlayer의 재생 상태가 변경될 때 발생하는 이벤트입니다. e.newstate의 영상 재생 상태 정보에 따라 분기하여 필요한 처리를 할 수 있습니다.

④ QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Play 버튼을 누르면 영상 파일이 재생되며 재생시간이 표시됩니다. 각 버튼을 눌러 동작에 이상이 없는지 확인합니다.

40.

WebBrowser

40.1 WebBrowser 소개

WebBrowser는 넥사크로플랫폼 애플리케이션 화면 내에 다른 웹 페이지 화면이나 콘텐츠를 보여줄 때 사용하는 컴포넌트입니다. WebBrowser 컴포넌트마다 각각 웹 페이지 화면을 출력할 수 있기 때문에 여러 개의 WebBrowser 컴포넌트를 배치하면 그 숫자만큼의 웹 페이지 화면을 동시에 보여주는 것이 가능합니다. 또한, 웹 페이지뿐 아니라 동영상, 이미지 등의 콘텐츠 역시 가능합니다.

WebBrowser 컴포넌트는 넥사크로플랫폼 애플리케이션과 웹 페이지 간의 상호작용을 위한 메소드와 이벤트를 제공합니다. 웹 페이지간의 상호작용이란 웹 페이지에 있는 엘리먼트 속성 혹은 스크립트 변수값 등을 넥사크로플랫폼 애플리케이션쪽으로 전달한다거나 반대로 넥사크로플랫폼 애플리케이션 쪽에서 웹페이지에 선언된 스크립트 함수를 호출하는 등의 두 화면 간의 데이터 교환 기능을 의미합니다. 개발자는 `getProperty`, `setProperty`, `callMethod` 메소드를 사용하여 웹 페이지의 엘리먼트나 속성값을 읽어오거나 설정할 수 있고, 함수를 호출할 수 있습니다. 또한, 웹 페이지 쪽에서 넥사크로플랫폼 애플리케이션 쪽으로 `usernotify` 이벤트를 발생시켜 사용자 데이터를 전달하거나 함수 호출을 하는 것도 가능합니다.

넥사크로플랫폼 애플리케이션에서 웹 페이지로 접근할 때에는 자바스크립트의 `window` 객체와 `document` 객체를 사용합니다. `window`는 웹 브라우저 윈도우를 나타내는 최상위 객체로 `document` 객체를 비롯한 모든 전역 객체와 함수는 `window` 객체에 속합니다. `document`는 웹 브라우저 윈도우에 로딩된 html 문서의 정보(DOM)를 담고 있는 객체로 웹 페이지 자체를 나타냅니다. `document` 객체는 `window` 객체의 하위 객체이기 때문에 원래는 `window.document` 형식으로 접근해야 하지만 `window`는 생략하여 사용할 수 있으므로 그냥 `document`만 사용해도 객체에 접근할 수 있습니다.

WebBrowser 컴포넌트에서 도메인이 다른 외부의 웹 페이지를 불러오는 경우 크로스 도메인 문제가 발생할 수 있습니다. 이를 해결하는 가장 간단한 방법은 두 파일의 도메인이 같도록 해주는 것입니다. 그러나 여의치 않을 때에는 CORS를 비롯하여 크로스 도메인을 피할 수 있는 방법을 사용해야 합니다.



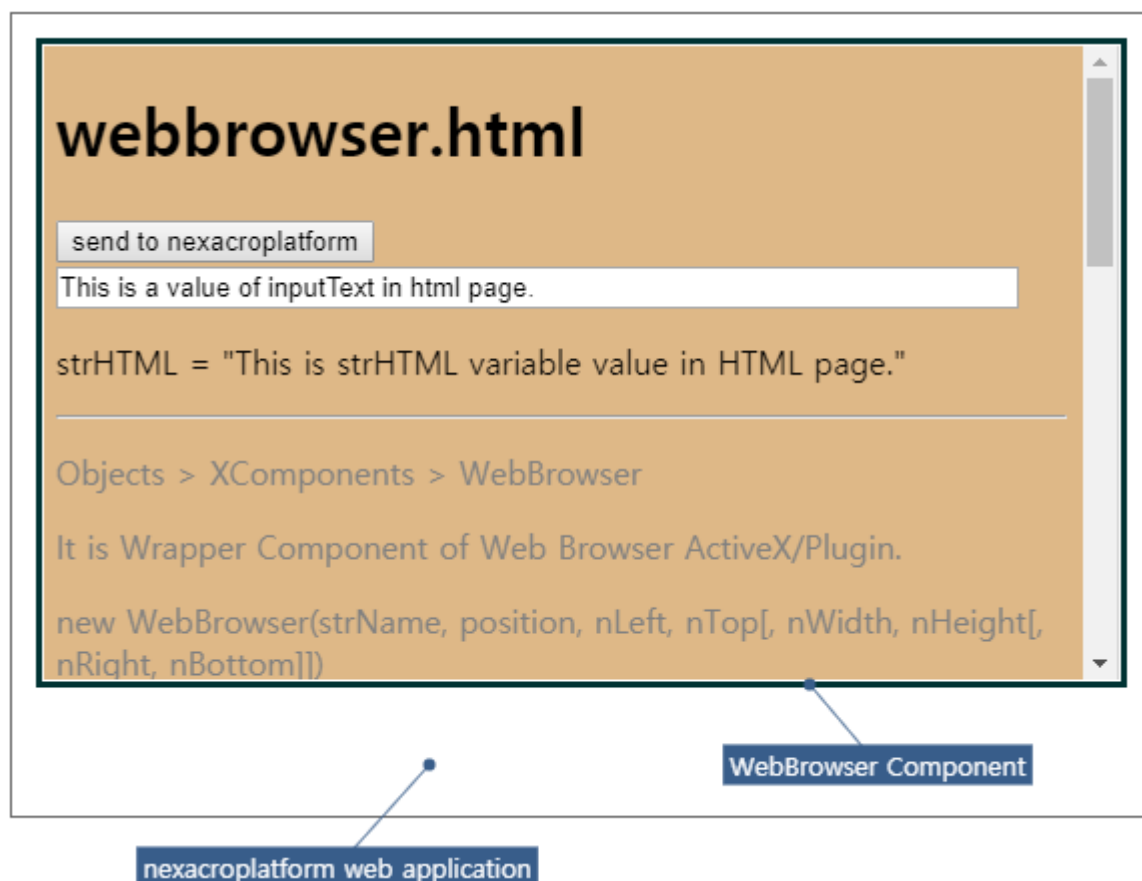
크로스 도메인 환경에서는 **동일 출처 정책(Same-origin policy)**으로 인해 사용에 제한이 있습니다. 다시 말해 호스트 명, 프로토콜, 포트 등이 다른 도메인 간에는 AJAX 호출을 할 수 없기 때문에 **CORS(Cross-Origin Resource Sharing)**를 적용하거나 JSONP 등의 방식으로 회피해야 합니다.

40.1.1 사용 예

WebBrowser 컴포넌트와 비슷한 예로 iframe이 있습니다. iframe은 frame과 유사하지만 좀 더 손쉽게 사용할 수 있게 만들어진 내부 프레임 요소로 한 웹 페이지 내에 다른 웹 페이지 혹은 콘텐츠를 출력하는 기능을 합니다.

40.1.2 WebBrowser 만들기

넥사크로플랫폼 애플리케이션 화면 내에 외부의 웹 페이지 화면을 보여줄 수 있습니다.



① WebBrowser 컴포넌트 생성하기

툴바에서 WebBrowser  를 선택한 후, Form 위에 적당한 크기로 드래그하여 생성합니다. 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

② url 속성 설정하기

폼에 생성한 WebBrowser를 선택한 후 Properties 창에서 url 속성을 설정합니다.

Property	Value
url	http://[서버 URL]/[파일 이름]

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행합니다.

url 속성을 설정했으면 WebBrowser 컴포넌트에 해당 URL 페이지가 표시됩니다.

40.2 엘리먼트 속성값 가져오기

WebBrowser 컴포넌트의 `getProperty`와 `callMethod` 메소드를 사용하여 로딩된 웹 페이지에 정의된 엘리먼트의 속성값을 넥사크로플랫폼 웹 애플리케이션에서 가져오는 방법에 대해 설명합니다.

40.2.1 예제

다음은 넥사크로플랫폼 화면에서 WebBrowser 컴포넌트를 사용해 웹 페이지를 로딩한 예제입니다.

Get Input Value 버튼을 클릭하면 WebBrowser 컴포넌트에 로딩한 HTML 페이지의 input 엘리먼트의 value 값을 넥사크로플랫폼 애플리케이션 쪽으로 가져와 Static 컴포넌트에 설정하여 화면에 출력합니다.





40.2.2 예제에서 사용한 핵심 기능

getProperty

WebBrowser 컴포넌트의 document 객체에 정의된 속성값을 반환하는 메소드입니다.

callMethod

WebBrowser 컴포넌트의 document 객체에 정의된 메소드를 호출하는 메소드입니다.

document (Javascript)

웹 페이지 정보(DOM)를 담고 있는 자바스크립트 객체입니다. document 객체를 사용해 html 문서의 요소에 접근할 수 있습니다.

getElementById (Javascript)

ID에 해당하는 엘리먼트를 반환하는 자바스크립트 메소드입니다.

40.2.3 예제 구현 방법

① WebBrowser 컴포넌트 생성하기

툴바에서 WebBrowser , Button , Static  컴포넌트를 생성하여 예제 화면과 같이 Form에 배치합니다.

① html 파일 생성하기

http://demo.nexacro.com/developer_guide/17/Service/webbrowser.html 파일의 소스 코드를 복사해서 webbrowser.html 파일을 생성합니다.

② url 속성 설정하기

WebBrowser 컴포넌트의 url 속성에 앞의 과정에서 생성한 webbrowser.html 문서의 URL을 설정합니다.

Property	Value
url	http://[서버 URL]/[파일 이름]



서버 URL이 고정된 경우에는 직접 url 속성값을 지정합니다. 예제에서는 로컬 테스트를 위해 onload 이벤트 함수 내에서 URL값을 확인하고 값을 설정해줍니다.

```
this.sample_webbrowser_02 onload = function(obj:nexacro.Form,e:nexacro.LoadEventInfo)
{
    var domain = document.location.href;
    var arrdomain = domain.split('/');
    var currentfile = arrdomain.pop();
    domain = domain.replace(currentfile, "Service/webbrowser.html")
    this.WebBrowser00.set_url(domain);
};
```

3 Get input value 버튼 기능 구현하기

넥사크로플랫폼에서 html 페이지에 있는 input 엘리먼트의 value 속성값을 가져옵니다.

```
1  this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
2  {
3      var _doc = this.WebBrowser00.getProperty("document");
4
5      var dom = _doc.callMethod("getElementById", "inputText");
6
7      var str = dom.getProperty("value");
8
9      this.Static00.set_text(str);
10
11     if(_doc && dom)
12     {
13         _doc.destroy();
14         _doc = null;
15
16         dom.destroy();
17         dom = null;
18     }
19 };
```

- 3번 줄: html 페이지의 document 객체가 가진 DOM 정보를 Plugin 객체로 반환합니다.
- 5번 줄: html 페이지에서 getElementById 메소드를 호출하여 id가 inputText인 엘리먼트의 DOM 정보를 반환합니다.

- 7번 줄: id가 inputText인 input 엘리먼트의 속성중 value 속성의 값을 반환합니다.
- 9번 줄: value 속성값을 Static00의 text 속성에 설정합니다.
- 11~18번 줄: 사용을 완료한 Plugin 객체의 자원을 해제합니다. 해제하지 않을 시 메모리 누수가 발생할 수 있습니다.

다음은 WebBrowser 컴포넌트에 로딩한 html 페이지의 코드 중 일부분으로 id가 inputText인 input 엘리먼트의 value 속성값을 확인할 수 있습니다.

```
<input id="inputButton" type="button" value="send to nexacroplatform" onclick="
sendToNexacro()">
<input id="inputText" type="text" value="This is value of inputText in html page." size=
"64"></p>
```

4 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Get input value 버튼을 누르면 WebBrowser 컴포넌트에 로딩된 웹 페이지의 input 엘리먼트의 설정된 "This is value of inputText in html page." 텍스트가 화면 하단의 Static 컴포넌트에 설정됩니다.

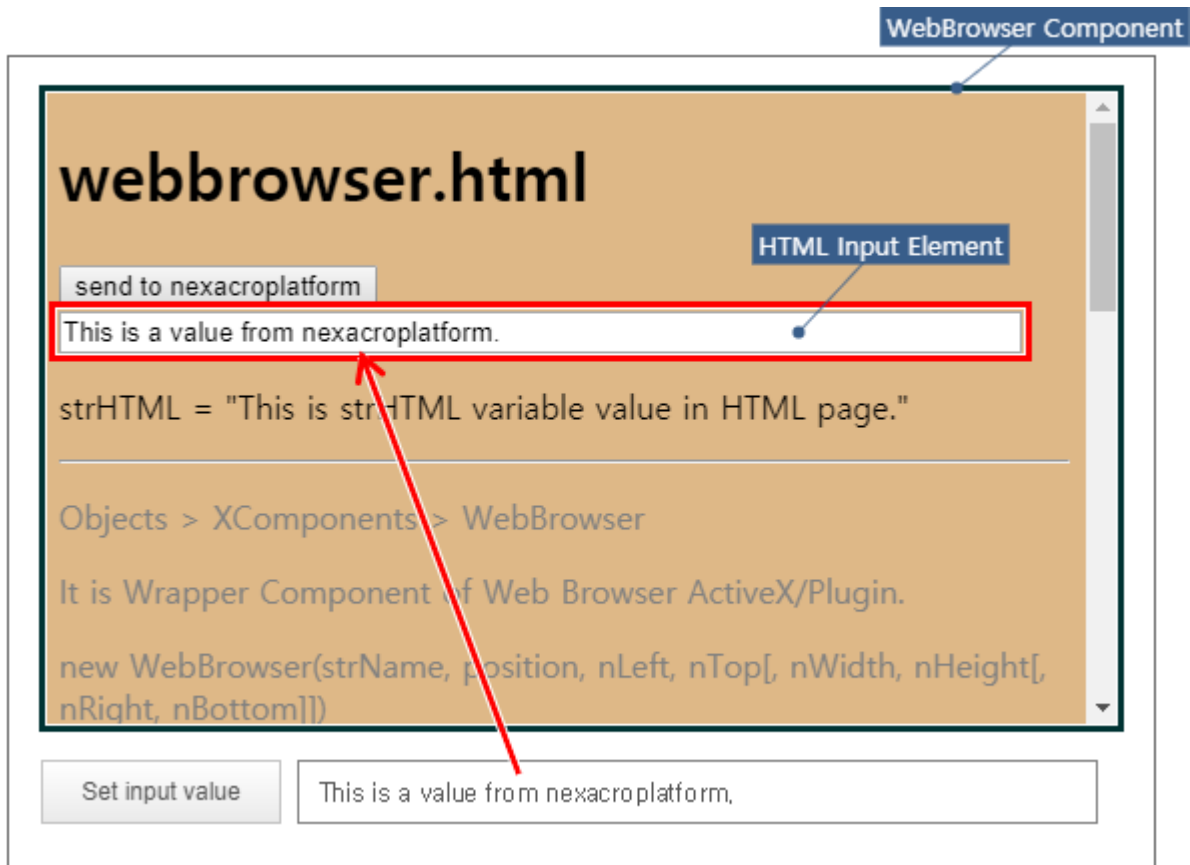
40.3 엘리먼트 속성값 설정하기

WebBrowser 컴포넌트의 setProperty 메소드를 사용하여 로딩된 웹 페이지에 정의되어 있는 엘리먼트의 속성값을 설정하는 방법에 대해 설명합니다.

40.3.1 예제

다음은 넥사크로플랫폼 화면에서 WebBrowser 컴포넌트를 사용해 웹 페이지를 로딩한 예제입니다.

Set Input Value 버튼을 클릭하면 넥사크로플랫폼 애플리케이션 화면의 Edit 컴포넌트에 설정한 문자열을 WebBrowser 컴포넌트에 로딩한 웹 페이지의 input 엘리먼트의 value 값으로 설정합니다.



 sample_webbrowser_03.xfdl

40.3.2 예제에서 사용한 핵심 기능

setProperty

WebBrowser 컴포넌트의 document 객체에 정의된 속성에 값을 설정하는 메소드입니다.

getProperty

WebBrowser 컴포넌트의 document 객체에 정의된 속성값을 반환하는 메소드입니다.

callMethod

WebBrowser 컴포넌트의 document 객체에 정의된 메소드를 호출하는 메소드입니다.

document (Javascript)

웹 페이지 정보(DOM)를 담고 있는 자바스크립트 객체입니다. document 객체를 사용해 html 문서의 요소에 접근할 수 있습니다.

getElementById (Javascript)

ID에 해당하는 엘리먼트를 반환하는 자바스크립트 메소드입니다.

40.3.3 예제 구현 방법

① WebBrowser 컴포넌트 생성하기

툴바에서 WebBrowser , Button , Edit  컴포넌트를 생성하여 예제 화면과 같이 Form에 배치합니다.

① html 파일 생성하기

http://demo.nexacro.com/developer_guide/17/Service/webbrowser.html 파일의 소스 코드를 복사해서 webbrowser.html 파일을 생성합니다.

② url 속성 설정하기

WebBrowser 컴포넌트의 url 속성에 앞의 과정에서 생성한 webbrowser.html 문서의 URL을 설정합니다.

Property	Value
url	http://[서버 URL]/[파일 이름]

③ Set input value 버튼 기능 구현하기

넥사크로플랫폼 애플리케이션의 Edit 컴포넌트에 설정한 값을 WebBrowser 컴포넌트에 로딩한 html 페이지에 있는 input 엘리먼트의 value 속성값으로 설정합니다. 아래 코드는 다음과 같이 동작합니다.

```

1  this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
2  {
3      var _doc = this.WebBrowser00.getProperty("document");
4
5      var dom = _doc.callMethod("getElementById", "inputText");
6
7      dom.setProperty("value", this.Edit00.text);
8
9      if(_doc && dom)
10     {
11         _doc.destroy();
12         _doc = null;
13
14         dom.destroy();
15         dom = null;

```

```
16     }
17 };
```

- 3번 줄: html 페이지의 document 객체가 가진 DOM 정보를 Plugin 객체로 반환합니다.
- 5번 줄: html 페이지에서 getElementById 메소드를 호출하여 id가 inputText인 엘리먼트의 DOM 정보를 반환합니다.
- 7번 줄: Edit00 컴포넌트의 text 속성값을 id가 inputText인 엘리먼트의 value 속성값으로 설정합니다.
- 9~16번 줄: 사용을 완료한 Plugin 객체의 자원을 해제합니다. 해제하지 않을 시 메모리 누수가 발생할 수 있습니다.

4 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Set input value 버튼을 누르면 화면의 Edit 컴포넌트의 "This is a value from nexacroplatform." 텍스트가 WebBrowser 컴포넌트에 로딩된 웹 페이지의 Input 엘리먼트에 설정됩니다.

40.4 스크립트의 변수값 가져오기

WebBrowser 컴포넌트의 getProperty 메소드를 사용하여 로딩된 웹 페이지에 정의된 javascript의 변수 값을 넥사크로플랫폼 웹 애플리케이션에서 가져오는 방법에 대해 설명합니다.

40.4.1 예제

다음은 넥사크로플랫폼 화면에서 WebBrowser 컴포넌트를 사용해 웹 페이지를 로딩한 예제입니다.

Get Variable 버튼을 누르면 WebBrowser 컴포넌트에 로딩된 웹 페이지의 strHTML 변수값인 "This is strHTML variable value in HTML page."를 넥사크로플랫폼 애플리케이션 쪽의 Static 컴포넌트에 설정합니다.



 sample_webbrowser_04.xfdl

다음은 웹 페이지에 선언된 strHTML 변수입니다.

```

<script type="text/javascript">
.
.
strHTML = "This is strHTML variable value in HTML page.";
</script>

```

40.4.2 예제에서 사용한 핵심 기능

getProperty

WebBrowser 컴포넌트의 document 객체에 정의된 속성값을 반환하는 메소드입니다.

window (Javascript)

웹 브라우저 윈도우를 나타내는 최상위 자바스크립트 객체입니다. document 객체를 비롯하여 모든 전역 객체와 함수는 window 객체에 속하기 때문에 window 객체를 통해 접근할 수 있습니다.

40.4.3 예제 구현 방법

① WebBrowser 컴포넌트 생성하기

툴바에서 WebBrowser , Button , Static  컴포넌트를 생성하여 예제 화면과 같이 Form에 배치합니다.

① html 파일 생성하기

http://demo.nexacro.com/developer_guide/17/Service/webbrowser.html 파일의 소스 코드를 복사해서 webbrowser.html 파일을 생성합니다.

② url 속성 설정하기

WebBrowser 컴포넌트의 url 속성에 앞의 과정에서 생성한 webbrowser.html 문서의 URL을 설정합니다.

Property	Value
url	http://[서버 URL]/[파일 이름]

③ Get Variable 버튼 기능 구현하기

넥사크로플랫폼 애플리케이션에서 html 페이지에 있는 자바스크립트 strHTML 변수값을 가져옵니다.

```

1  this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
2  {
3      var _win = this.WebBrowser00.getProperty("window");
4
5      var str = _win.getProperty("strHTML");
6
7      this.Static00.set_text(str);
8
9      if(_win)
10     {
11         _win.destroy();
12         _win = null;
13     }
14 };

```

- 3번 줄: html 페이지의 window 객체 정보를 담은 Plugin 객체를 반환합니다.
- 5번 줄: html 페이지의 strHTML 변수값을 str 변수에 저장합니다.
- 7번 줄: str 변수값을 Static00의 text 속성에 설정합니다.
- 9~13번 줄: 사용을 완료한 Plugin 객체의 자원을 해제합니다. 해제하지 않을 시 메모리 누수가 발생할 수 있습니다.

4 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

Get Variable 버튼을 누르면 WebBrowser 컴포넌트에 로딩된 웹 페이지의 strHTML 변수값인 "This is strHTML variable value in HTML page."를 넥사크로플랫폼 애플리케이션쪽의 Static 컴포넌트에 설정합니다.

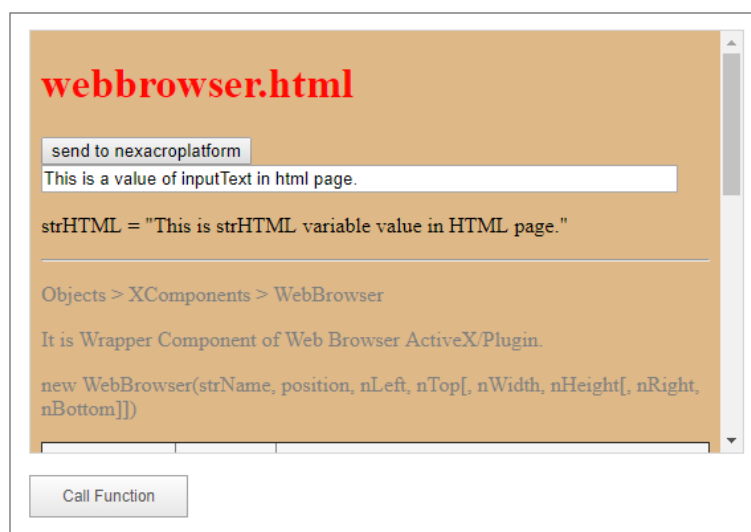
40.5 스크립트의 함수 호출하기

WebBrowser 컴포넌트의 callMethod 메소드를 사용하여 로딩된 웹 페이지에 정의된 javascript의 함수를 호출하는 방법에 대해 설명합니다. callMethod 메소드로 함수 호출 시 복수의 인수도 전달할 수 있습니다.

40.5.1 예제

다음은 넥사크로플랫폼 화면에서 WebBrowser 컴포넌트를 사용해 웹 페이지를 로딩한 예제입니다.

Call Function 버튼을 누르면 WebBrowser 컴포넌트에 로딩된 웹 페이지에 선언된 colorFunction 함수를 호출합니다. colorFunction 함수는 문자열을 인수로 하여 타이틀 문자열의 색상을 바꾸는 함수를 호출하는데 정상적으로 함수 호출이 이뤄지면 "webbrowser.html" 문자열 색상이 빨간색으로 변경됩니다.





다음은 넥사크로플랫폼 애플리케이션에 의해 호출되는 웹 페이지에 선언된 colorFunction 함수입니다.

```
function colorFunction(str)
{
    var doc = document.getElementById('title_test');
    doc.style.color = str;
}
```

40.5.2 예제에서 사용한 핵심 기능

getProperty

WebBrowser 컴포넌트의 document 객체에 정의된 속성값을 반환하는 메소드입니다.

callMethod

WebBrowser 컴포넌트의 document 객체에 정의된 메소드를 호출하는 메소드입니다.

window (Javascript)

웹 브라우저 윈도우를 나타내는 최상위 자바스크립트 객체입니다. document 객체를 비롯하여 모든 전역 객체와 함수는 window 객체에 속하기 때문에 window 객체를 통해 접근할 수 있습니다.

40.5.3 예제 구현 방법

① WebBrowser 컴포넌트 생성하기

툴바에서 WebBrowser , Button , Static  컴포넌트를 생성하여 예제 화면과 같이 Form에 배치합니다.

① html 파일 생성하기

http://demo.nexacro.com/developer_guide/17/Service/webbrowser.html 파일의 소스 코드를 복사해서 webbrowser.html 파일을 생성합니다.

② url 속성 설정하기

WebBrowser 컴포넌트의 url 속성에 앞의 과정에서 생성한 webbrowser.html 문서의 URL을 설정합니다.

Property	Value
url	http://[서버 URL]/[파일 이름]

③ Get Variable 버튼 기능 구현하기

넥사크로플랫폼 애플리케이션에서 html 페이지에 있는 자바스크립트 strHTML 변수값을 가져옵니다.

```

1  this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
2  {
3      var _win = this.WebBrowser00.getProperty("window");
4
5      _win.callMethod("colorFunction", "red");
6
7      if(_win)
8      {
9          _win.destroy();
10         _win = null;
11     }
12 };

```

- 3번 줄: html 페이지의 window 객체 정보를 담은 Plugin 객체를 반환합니다.
- 5번 줄: html 페이지의 "red" 문자열을 인수로 전달하고 alertFunction 함수를 호출합니다.
- 7~11번 줄: 사용을 완료한 Plugin 객체의 자원을 해제합니다. 해제하지 않을 시 메모리 누수가 발생할 수 있습니다.

④ QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. Call Function 버튼을 누르면 문자열 색상이 변경됩니다.

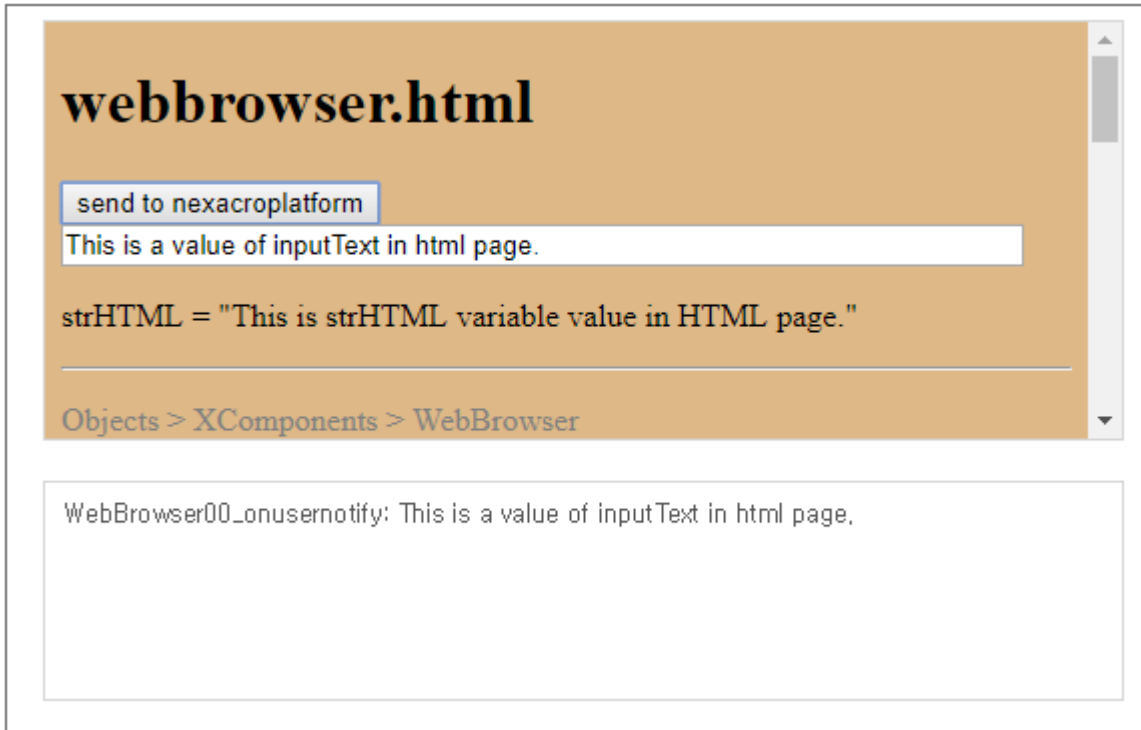
40.6 usernotify 이벤트를 이용한 사용자 데이터 전달하기

웹 페이지에서 usernotify 이벤트를 통해 넥사크로플랫폼 애플리케이션으로 사용자 데이터를 전달하는 방법에 관해 설명합니다.

40.6.1 예제

다음은 넥사크로플랫폼 화면에서 WebBrowser 컴포넌트를 사용해 웹 페이지를 로딩한 예제입니다.

WebBrowser 컴포넌트에 로딩된 웹 페이지의 [send to nexacroplatform] 버튼을 누르면 TextArea 컴포넌트에 웹 페이지로부터 넥사크로플랫폼 애플리케이션 쪽으로 usernotify 이벤트의 사용자 데이터를 통해 전달된 "This is a value of inputText in html page." 문자열이 출력됩니다.



sample_webbrowser_06.xfdl

40.6.2 예제에서 사용한 핵심 기능

onusernotify

WebBrowser 컴포넌트에 로딩된 웹 페이지에서 넥사크로플랫폼 애플리케이션 쪽으로 정보를 전달할 때 발생하는 이벤트입니다.

userdata

WebBrowser 컴포넌트에 로딩된 웹 페이지에서 넥사크로플랫폼 애플리케이션으로 전달되는 사용자 데이터를 갖는 WebUserNotifyEventInfo 객체의 읽기 전용 속성입니다.

40.6.3 예제 구현 방법

① WebBrowser 컴포넌트 생성하기

툴바에서 WebBrowser  컴포넌트와 TextArea 컴포넌트를 생성하여 예제 화면과 같이 Form에 배치합니다.

① html 파일 생성하기

http://demo.nexacro.com/developer_guide/17/Service/webbrowser.html 파일의 소스 코드를 복사해서 webbrowser.html 파일을 생성합니다.

② url 속성 설정하기

WebBrowser 컴포넌트의 url 속성에 앞의 과정에서 생성한 webbrowser.html 문서의 URL을 설정합니다.

Property	Value
url	http://[서버 URL]/[파일 이름]

③ WebBrowser00 컴포넌트의 이벤트 함수 추가 및 작성하기

WebBrowser 컴포넌트에 onusernotify 이벤트 함수를 추가하고 다음과 같이 작성합니다.

```

1  this.WebBrowser00_onusernotify = function(obj:nexacro.WebBrowser,e:nexacro.
WebUserNotifyEventInfo)
2  {
3      if(e.userdata)
4      {
5          var str = this.WebBrowser00.id + "_onusernotify: " + e.userdata;
6          this.TextArea00.set_value(str);
7      }
8  };

```

- 3번 줄: WebBrowser 컴포넌트에 로딩된 웹 페이지에서 넥사크로플랫폼 애플리케이션으로 전달되는 사용자 데이터가 있는지 확인한다.
- 5~7번 줄: 사용자 데이터를 alert 팝업으로 출력한다.

④ 웹 페이지에 이벤트 함수 작성하기

웹 페이지(webbrowser.html)에 넥사크로플랫폼 애플리케이션 쪽으로 usernotify 이벤트를 발생시킬 수 있는 이벤트 함수를 작성합니다.

```

1 <script type="text/javascript">
2   if (! window.NEXACROHTML)
3   {
4       window.NEXACROHTML = {};
5   }
6
7   window.NEXACROHTML.FireUserNotify = function(userdata)
8   {
9       if (window.NEXACROWEBBROWSER)
10      {
11          window.NEXACROWEBBROWSER.on_fire_onusernotify(window.NEXACROWEBBROWSER,
12          userdata);
13      }
14      else
15      {
16          window.document.title = userdata;
17      }
18  }
19 </script>

```

- 2~5번 줄: 전역 객체인 NEXACROHTML을 선언합니다.
- 7번 줄: NEXACROHTML 객체에 FireUserNotify 이벤트 함수를 선언합니다.
- 9~16번 줄: 브라우저의 종류가 넥사크로브라우저이면 userdata를 인수로 한 NEXACROWEBBROWSER 객체의 on_fire_onusernotify 이벤트 함수를 호출하고 그 외의 브라우저면 DOM 객체의 타이틀로 userdata를 설정합니다.

5 QuickView로 확인하기

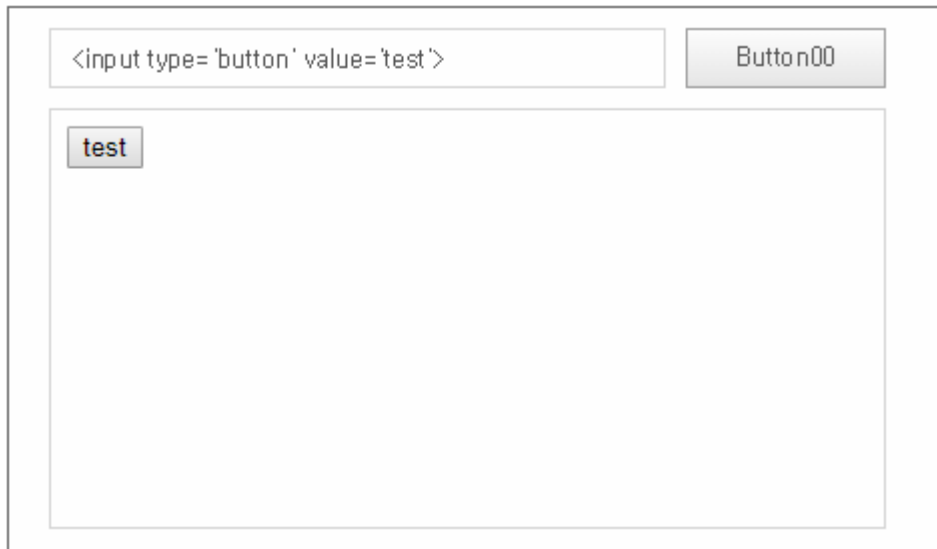
QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. WebBrowser 컴포넌트에 로딩된 웹 페이지의 send to nexacroplatform 버튼을 누르면 TextArea 컴포넌트에 웹 페이지로부터 전달된 "This is a value of inputText in html page." 문자열이 출력됩니다.

40.7 innerHTML을 사용해 데이터 표현하기

setProperty 메소드를 사용해 지정된 값을 WebBrowser 컴포넌트에 표시하는 방법을 살펴봅니다.

40.7.1 예제

Edit 컴포넌트에 값(텍스트 또는 HTML 태그)을 입력하고 버튼 클릭 시 해당 값이 WebBrowser 컴포넌트에 표시됩니다.



40.7.2 예제에서 사용한 핵심 기능

setProperty

document 객체에 정의된 속성의 값을 설정할 수 있습니다. 예제에서는 innerHTML 속성의 값을 설정합니다. innerHTML 속성값으로 지정된 값은 지정된 위치의 태그 아래 콘텐츠로 추가됩니다.

40.7.3 예제 구현 방법

① 화면 배치하기

화면에 WebBrowser 컴포넌트를 배치하고 상단에 Edit 컴포넌트와 Button 컴포넌트를 배치합니다. WebBrowser 컴포넌트의 url 속성값은 설정하지 않습니다.

② onclick 이벤트 함수 작성하기

버튼 클릭 시 document 객체를 가져와서 innerHTML 속성값을 설정해줍니다.

```

this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var vHtml = this.WebBrowser00.getProperty("document").getProperty("body");
    vHtml.setProperty("innerHTML", this.Edit00.value);
};

```

3 QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다. Edit 컴포넌트에 HTML 태그를 입력하면 해당 태그가 실제 콘텐츠처럼 동작합니다.

40.8 외부 웹에디터 사용하기

외부 웹에디터 중 CKEditor를 사용하는 방법을 살펴봅니다. 전체 API를 모두 활용하지는 않고 간단하게 콘텐츠를 설정하고 가져오는 방법만 살펴봅니다.

40.8.1 예제

WebBrowser 컴포넌트에 지정된 HTML 파일을 로딩해서 웹에디터를 표시합니다. [setData] 버튼 클릭 시 특정 HTML 코드 문자열을 전달해서 웹에디터에 표시하고, 웹에디터의 콘텐츠를 수정한 후 [getData] 버튼 클릭 시 하단 TextArea 컴포넌트에 HTML 코드 문자열을 가져와서 표시합니다.



예제에서는 CKEditor 4 버전을 사용했습니다. CKEditor는 사용 방식에 따라 무료 또는 유료로 사용됩니다.

<https://ckeditor.com/ckeditor-4/>



CKEditor 5 버전은 IE11을 지원하지 않습니다.

<https://docs.ckeditor.com/ckeditor5/latest/builds/guides/support/browser-compatibility.html>



sample_webbrowser_08.xfdl

40.8.2 예제에서 사용한 핵심 기능

callMethod

window 객체에 정의된 function을 호출합니다. 반환되는 값이 있는 경우 반환값을 받아서 처리할 수 있습니다.

40.8.3 예제 구현 방법

1 화면 배치하기

화면에 WebBrowser 컴포넌트, Button 컴포넌트, TextArea 컴포넌트를 배치합니다. WebBrowser 컴포넌트의 url 속성값은 설정하지 않습니다.

2 onload 이벤트 함수 작성하기

form 오브젝트의 onload 이벤트 함수 내에서 웹에디터를 호출하는 HTML 파일을 로딩하는 스크립트를 실행합니다.

```

this.sample_webbrowser_08_onload = function(obj:nexacro.Form,e:nexacro.LoadEventInfo)
{
    var domain = document.location.href;
    var arrdomain = domain.split('/');
    var currentfile = arrdomain.pop();
    domain = domain.replace(currentfile, "Service/sample_webbrowser_08.html")
    this.WebBrowser00.set_url(domain);
};

```

HTML 파일은 아래와 같습니다. HTML 코드 샘플은 CKEditor 예제를 참고했습니다.



<https://docs.ckeditor.com/ckeditor5/latest/builds/guides/integration/basic-api.html>

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>CKEditor 4 - basic editor</title>
    <script src="https://cdn.ckeditor.com/4.10.0/basic/ckeditor.js"></script>
</head>
<body>
    <textarea name="content" id="editor">
    </textarea>
    <script>
        CKEDITOR.replace( 'editor' );

        function setData(str)
        {
            CKEDITOR.instances.editor.setData(str);
        }

        function getData()
        {
            return CKEDITOR.instances.editor.getData();
        }
    </script>
</body>
</html>

```

③ onloadcompleted 이벤트 함수 작성하기

callMethod 메소드를 실행하기 위해 window 오브젝트를 가져와서 변수로 담아놓습니다.

```
this.WebBrowser00 onloadcompleted = function(obj:nexacro.WebBrowser,e:nexacro.
WebLoadCompEventInfo)
{
    _win = this.WebBrowser00.getProperty('window');
};
```

④ onclick 이벤트 함수 작성하기

버튼 클릭 시 HTML 코드 문자열을 웹에디터로 보내거나 웹에디터에서 편집한 HTML 코드를 반환합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    _win.callMethod("setData", "<p><strong>nexacro platform</strong></p><p><em>Nexacro
Platform programming is divided into two parts</em></p><p>one to define user-visible
screens visible and a second used for the scripts that implement business logic.
Additionally, Nexacro Platform provides capabilities for applying and managing styles and
themes in screens.</p>");
};

this.Button01 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.TextArea00.set_value(_win.callMethod("getData"));
};
```

⑤ QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

40.9 MS 오피스 뷰어 사용하기

문서 파일을 내려받지 않고 웹브라우저에서 바로 온라인 뷰어를 실행하는 방법입니다.

40.9.1 예제

Grid 컴포넌트에서 원하는 파일 URL을 선택하면 WebBrowser 컴포넌트에 표시합니다.



마이크로소프트에서 제공하는 서비스를 사용하는 방식입니다. 사정에 따라 서비스가 동작하지 않을 수도 있습니다.

<https://support.office.com/en-us/article/view-office-documents-online-1cc2ea26-0f7b-41f7-8e0e-6461a104544e>

Column0					
http://file-examples.com/wp-content/uploads/2017/02/file-sample_100kB.docx					
http://file-examples.com/wp-content/uploads/2017/02/file_example_XLSX_10.xlsx					
http://file-examples.com/wp-content/uploads/2017/08/file_example_PPT_250kB.ppt					
http://file-examples.com/wp-content/uploads/2017/10/file-sample_100kB.odt					

	G	H	I	J	K
6	58	16/08/2016	2468		
7	24	21/05/2015	2554		
8	56	15/10/2017	3598		
9	27	16/08/2016	2456		
10	40	21/05/2015	6548		
11					
12					
13					
14					
15					
16					
17					
18					



sample_webbrowser_09.xfdl

40.9.2 예제에서 사용한 핵심 기능

url

url 속성값에 로컬에 위치한 파일 경로도 지정할 수 있지만, 예제에서는 온라인 상에 등록된 문서 파일만 사용할 수 있습니다.

40.9.3 예제 구현 방법

- ① 화면에 Grid 컴포넌트와 WebBrowser 컴포넌트를 배치합니다.
- ② Dataset 오브젝트를 생성하고 컬럼 한개를 추가한 후 문서 파일의 URL 값을 컬럼 데이터로 추가합니다.



예제에서는 샘플로 제공되는 문서 파일을 사용했습니다.

<http://file-examples.com/index.php/sample-documents-download/>

- ③ Dataset 오브젝트를 선택하고 드래그앤드롭으로 Grid 컴포넌트에 binddataset 속성으로 지정합니다.
- ④ Grid 컴포넌트의 oncelldbclick 이벤트 함수를 아래와 같이 작성합니다.

해당 셀을 더블클릭했을때 URL 값을 가져와 인코딩 후 온라인 서비스를 호출합니다.

```
this.Grid00_oncelldbclick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    this.WebBrowser00.set_url("https://view.officeapps.live.com/op/embed.aspx?src="+
    encodeURIComponent(this.Dataset00.getColumn(e.row,0)));
};
```

- ⑤ QuickView(Ctrl + F6)를 실행하여 결과를 확인합니다.

파트 IV.

Container Component

41.

Div

41.1 Div 소개

Div 컴포넌트는 하나의 화면을 여러 개로 분할해서 처리해야 하는 경우 이런 레이아웃을 지원하는 컴포넌트입니다. 예를 들어 하나의 화면 가운데를 가로 방향으로 잘라서 위에는 차트를 보여주고 아래는 그리드를 보여주고자 할 때 사용할 수 있습니다. 물론 하나의 화면에 컴포넌트를 배치해서 분할된 화면처럼 구성할 수 있지만 Div 컴포넌트는 이런 기능을 처리하기 위해 몇 가지 기능을 제공합니다.

구분되는 화면의 경계선을 표시할 수 있습니다.

하나의 컴포넌트만 들어간다면 모르겠지만 두 개 이상의 컴포넌트를 묶어서 구분하는 경우에는 경계선을 표시하기가 모호합니다. 이런 경우 Div 컴포넌트를 사용하면 쉽게 경계를 구분할 수 있습니다.

Form을 재활용할 수 있습니다.

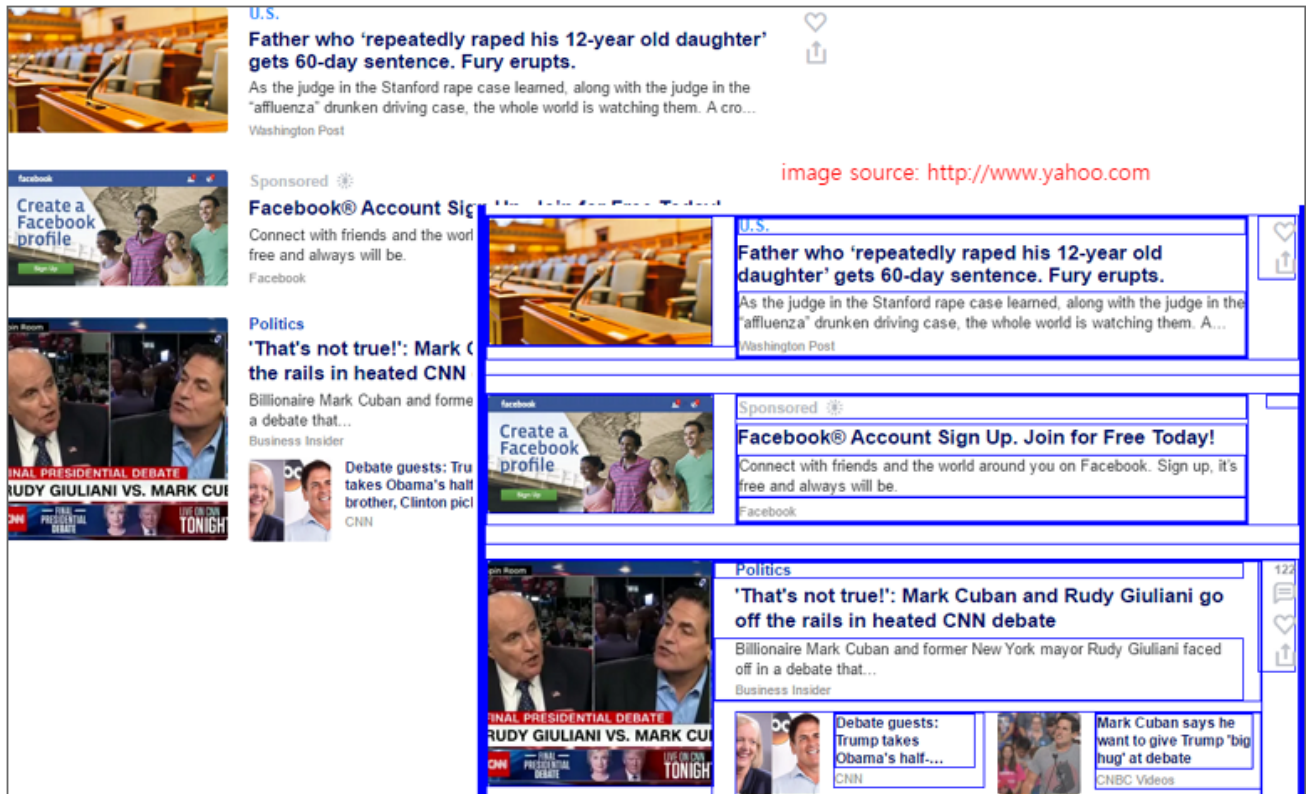
Div 컴포넌트는 이미 생성된 Form을 연결할 수 있습니다. 차트 기능을 제공하는 Form을 만들고 모든 화면에 적용하고 싶다면 Div 컴포넌트를 배치하고 해당하는 Form만 연결합니다. 매번 새로운 화면을 만들지 않아도 됩니다.

영역의 크기를 조절할 수 있습니다.

Div 컴포넌트 내 배치된 컴포넌트는 Div 컴포넌트를 기준으로 위치값을 지정합니다. 때문에 Div 컴포넌트의 크기가 변경되면 연결된 컴포넌트의 위치도 같이 변경됩니다.

41.1.1 사용 예

웹사이트에 보이는 화면의 구성요소(이미지, 텍스트 등)를 배치할 때 아무것도 없이 정확한 위치에 배치되는 것처럼 보이지만 실제로는 수많은 div 태그를 사용합니다. div 태그는 넥사크로플랫폼에서 다루는 Div 컴포넌트와는 다르지만, 화면에 보이는 요소를 감싸는 용도로 사용한다는 점에서 비슷한 특성이 있습니다.



41.1.2 Div 만들기

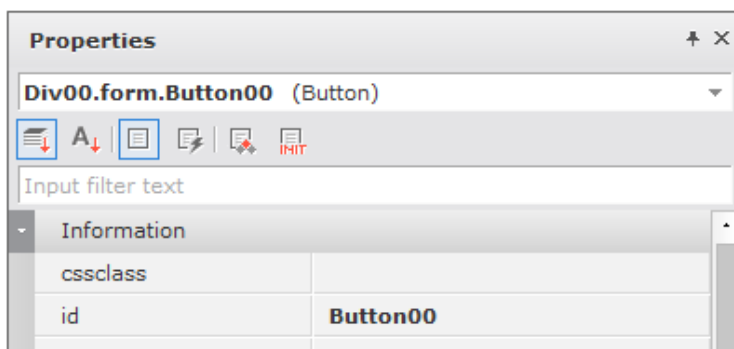
Div 컴포넌트만 단독으로 사용하는 일은 거의 없습니다. 대부분은 Form을 연결하거나 내부에 컴포넌트를 포함하는 형식으로 구현합니다. 이번 예제에서는 Div 컴포넌트를 배치하고 Button 컴포넌트 하나를 추가합니다.

1 Div 컴포넌트 생성하기

툴바에서 Div 항목을 선택한 후, Form 위에 드래그 앤 드롭해 생성합니다. 빈 흰색 상자가 화면에 표시됩니다.

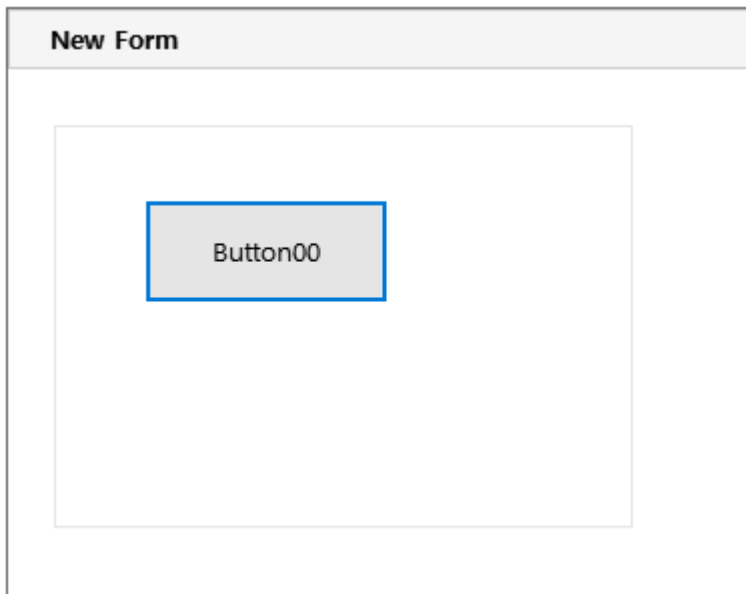
2 Button 컴포넌트 추가하기

툴바에서 Button 항목을 선택한 후, Div 영역 위에 드래그 앤 드롭해 생성합니다. 정확하게 Div 영역 내에서 생성되어야 합니다. Div 컴포넌트 내 생성되었다면 속성창에 아래와 같이 표시됩니다.



3 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행합니다. Default 테마에서 지정된 Div 컴포넌트의 border 속성때문에 경계선이 표시됩니다. border 속성을 보이지 않게 지정한다면 경계선이 보이지 않습니다. 하지만 Div 컴포넌트 내 Button 컴포넌트가 생성된 것은 변함없습니다.



default 테마에서 Div 컴포넌트의 border 속성값은 "1px solid #e5e5e5"입니다. border 속성값을 "0px none"으로 수정하면 경계선이 보이지 않습니다.

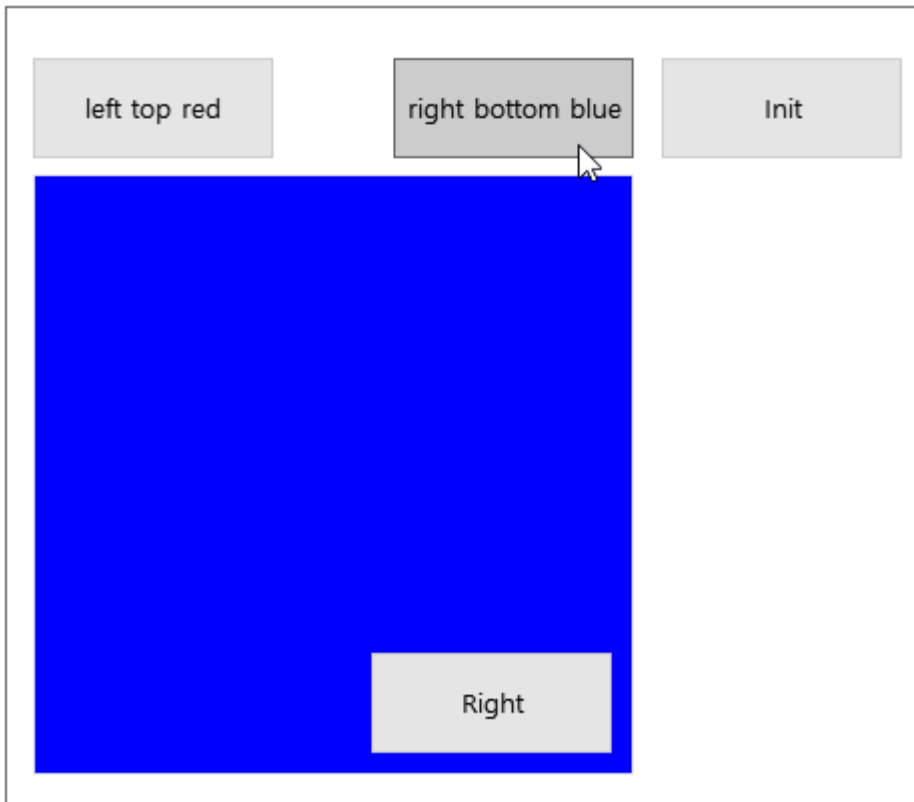
Style	
+	background
-	border
+	border-top
+	border-right
+	border-bottom
+	border-left
	border-radius

41.2 다른 화면 보여주기

선택한 메뉴에 따라 다른 화면을 보여주어야 한다면 어떻게 구현할 수 있을까요? 전체 페이지 URL을 변경할 수도 있지만, 콘텐츠 영역만 원하는 화면으로 바꾸어 보여주는 방법도 있습니다. Div 컴포넌트에 연결된 Form만 변경하는 방법을 사용하면 자연스럽게 원하는 콘텐츠를 사용자에게 보여줄 수 있습니다.

41.2.1 예제

버튼을 클릭하면 해당하는 화면이 Div 컴포넌트 영역에 표시됩니다. 다른 콘텐츠를 보여주기 위해 이벤트를 처리하는 메인 Form과 콘텐츠를 담고 있는 2개의 Form을 생성합니다. 콘텐츠를 담고 있는 두 개의 Form은 Button 컴포넌트의 위치를 각각 왼쪽 상단과 오른쪽 하단에 배치하고 Form 배경색을 다르게 표시해 콘텐츠가 바뀌었다는 것을 알 수 있도록 합니다. "Init" 버튼을 클릭하면 아무것도 지정되지 않은 상태로 변경합니다.



41.2.2 예제에서 사용한 핵심 기능

url

Div 컴포넌트에 연결되는 Form 경로를 지정합니다. 절대경로나 상대경로, 서비스 경로를 지정할 수 있습니다. 예제에서는 3개의 Form이 같은 서비스 경로에 있는 것으로 가정합니다.



url 경로를 지정하는 경우 Form 확장자는 "xfdl" 또는 "xfdl.js"로 지정해주어야 합니다. 확장자를 지정 해주지 않으면 해당 Form을 찾지 못합니다. "xfdl"로 지정한 경우에는 ".js"를 내부적으로 붙여서 처리합니다.

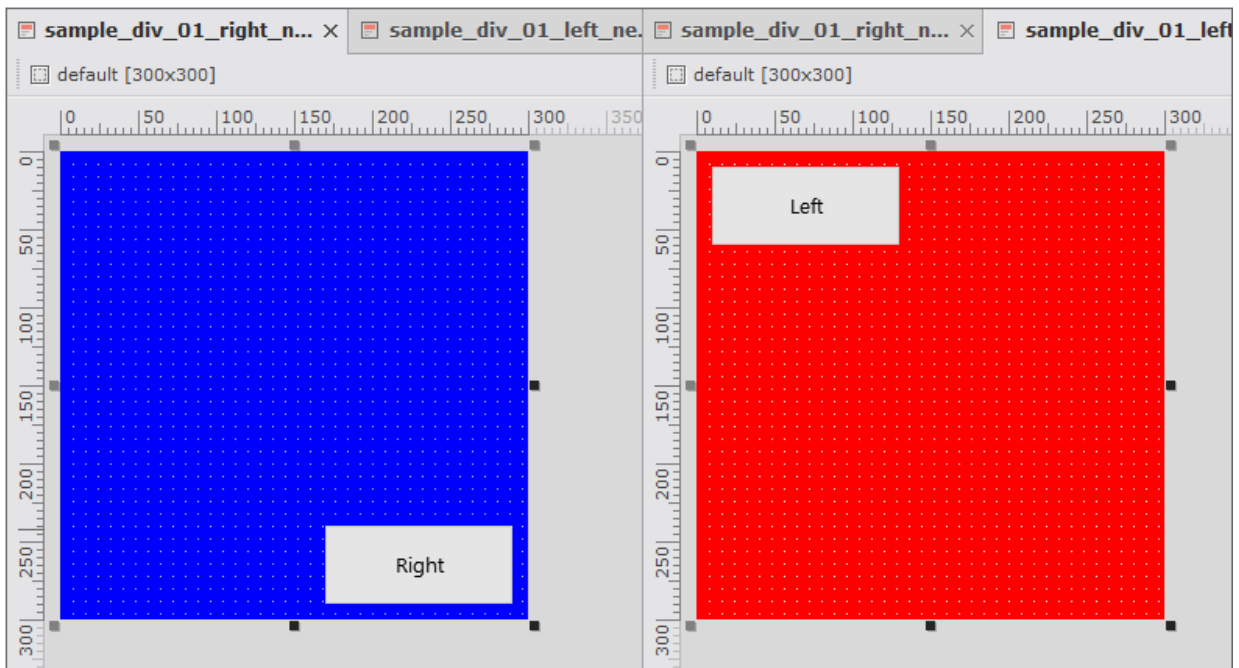
41.2.3 예제 구현 방법

① 메인 화면 구성하기

Div, Button 컴포넌트를 예제 화면과 같이 배치합니다. Div 컴포넌트의 text 속성값은 "divMain"으로 지정해서 초기화했을 때 상태를 확인할 수 있도록 합니다. width, height 속성값은 300으로 지정합니다. 다음 단계에서 만드는 콘텐츠를 담은 Form 오브젝트의 width, height 속성값도 300으로 지정할 겁니다.

② 콘텐츠 화면 구성하기

Form을 새로 만들고 Button 컴포넌트를 배치합니다. Form의 width, height 속성값은 300으로 지정합니다. 메인 화면에서 "left top red" 버튼 클릭 시 보일 화면은 Button 컴포넌트를 왼쪽 상단에 배치하고 Form의 background 속성값을 "red"로 지정합니다. 메인 화면에서 "right bottom blue" 버튼 클릭 시 보일 화면은 Button 컴포넌트를 오른쪽 하단에 배치하고 Form의 background 속성값을 "blue"로 지정합니다.



③ Button 컴포넌트 이벤트 함수 작성하기

3개 버튼 클릭 시 처리할 이벤트 함수를 작성합니다. 각 버튼에 맞는 파일 경로를 지정하고 초기화를 선택한 경우에는 null값을 지정합니다.

```
this.btnLeft_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.divMain.set_url("Sample::sample_div_01_left.xfdl");
};
```

```

this.btnRight_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.divMain.set_url("Sample::sample_div_01_right.xfdl");
};

this.btnInit_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.divMain.set_url(null);
};

```

④ 부모 컴포넌트 속성 접근하기

sample_div_01_left.xfdl, sample_div_01_right.xfdl 에 포함된 Button 컴포넌트 클릭 시 sample_div_01.xfdl 의 Form 오브젝트 id 속성값에 접근해서 alert 메소드로 표시하는 기능을 추가합니다.

```

this.btnLeft_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.alert(this.parent.parent.name);
};

```

```

this.btnRight_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.alert(this.parent.parent.name);
};

```

⑤ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭하고 변경된 콘텐츠를 확인합니다.



sample_div_01.xfdl, sample_div_01_left.xfdl, sample_div_01_right.xfdl 3개 파일이 모두 Generate 되어야 동작합니다. 작성된 코드가 저장되었는지 확인해주세요.

41.3 Div Contents 접근하기

Form 안에 배치된 컴포넌트나 오브젝트는 다 같은 형제입니다. 서로 이름을 부르면 대답할 수 있습니다. 하지만 Div 컴포넌트 안에 포함된 컴포넌트는 바로 이름을 부를 수 없습니다. 항상 Div 컴포넌트를 지나가야만 합니다.

41.3.1 예제

Edit 컴포넌트에 텍스트를 입력하고 버튼을 클릭하면 입력한 텍스트가 Div 컴포넌트 내 Static 컴포넌트의 text 속성값으로 입력됩니다.

 [sample_div_02.xfdl](#)

41.3.2 예제에서 사용한 핵심 기능

form

Div 컴포넌트 내부에 정의된 컴포넌트에 접근하기 위한 속성입니다. url 속성값에 Form을 연결한 경우에도 같은 방식으로 접근합니다.

```
this.[Div Component].form.[Component];
```

41.3.3 예제 구현 방법

① 메인 화면 구성하기

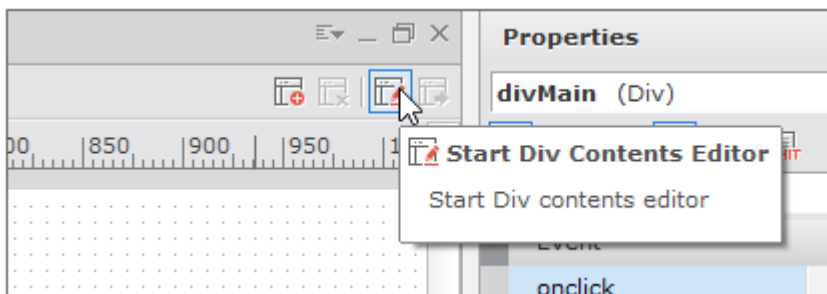
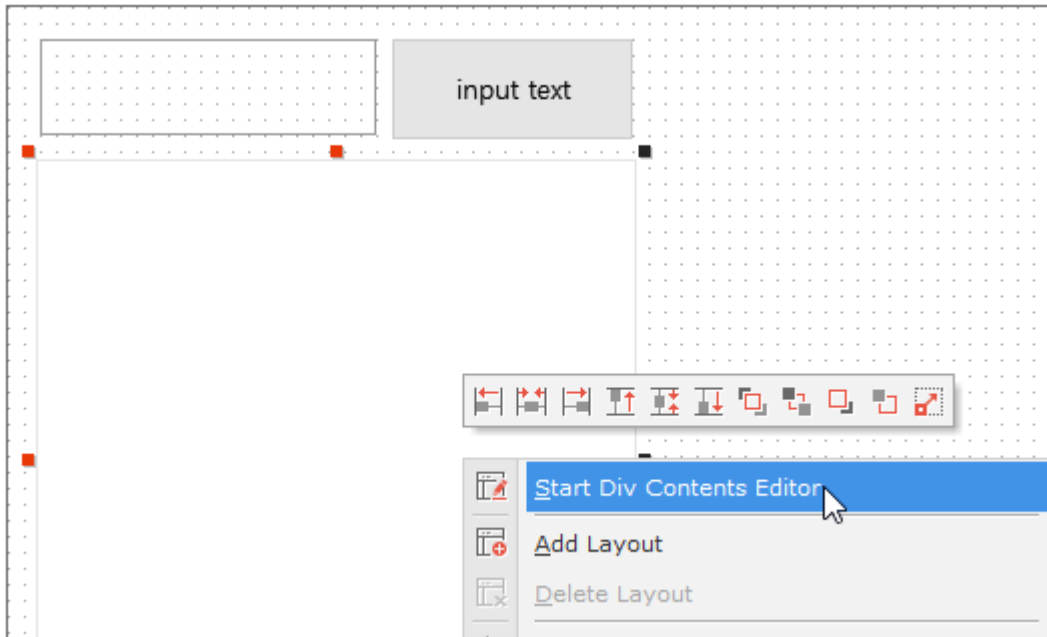
Edit, Button, Div 컴포넌트를 예제 화면과 같이 배치합니다. Div 컴포넌트가 잘 보이도록 border 속성값을 아

래와 같이 지정합니다.

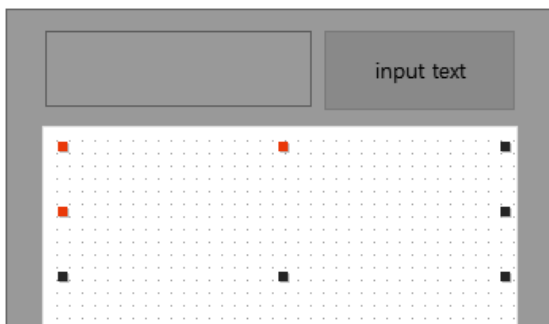
1px solid #e5e5e5

2 콘텐츠 화면 구성하기

Div 컴포넌트를 선택하고 컨텍스트 메뉴에서 [Start Div Contents Editor] 항목을 선택하거나 디자인창 오른쪽 상단 툴바에서 [Start Div Contents Editor] 항목을 선택합니다.



Div Content 편집 상태에서 Static 컴포넌트를 배치합니다. 편집 상태에 들어가지 않고 Div 영역 위에 Static 컴포넌트를 올려놓을 수도 있지만 편집 상태에서는 좀 더 명확하게 컴포넌트를 배치할 수 있습니다.



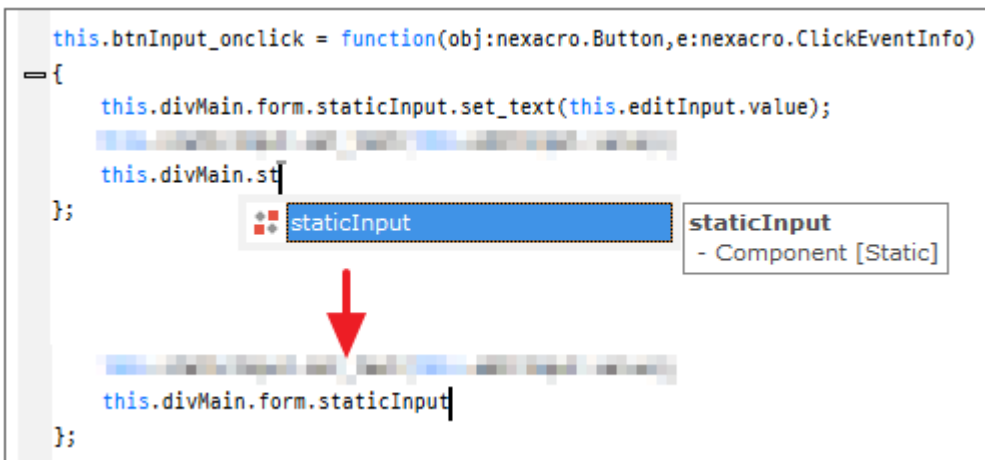
편집이 끝나면 컨텍스트 메뉴에서 [End Div Contents Editor] 항목을 선택하거나 디자인창 오른쪽 상단 툴바에서 [End Div Contents Editor] 항목을 선택합니다.

3 Button 컴포넌트 이벤트 함수 작성하기

입력된 값을 Div 컴포넌트 내 Static 컴포넌트의 text 속성값으로 전달합니다.

```
this.btnInput_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.divMain.form.staticInput.set_text(this.editInput.value);
};
```

넥사크로 스튜디오 스크립트 창에서 코드 힌트 기능을 사용하는 경우에는 "form" 항목을 입력하지 않아도 자동으로 입력해줍니다.



코드 힌트 기능을 사용하지 않고 직접 텍스트를 입력하는 경우에는 동작하지 않습니다.

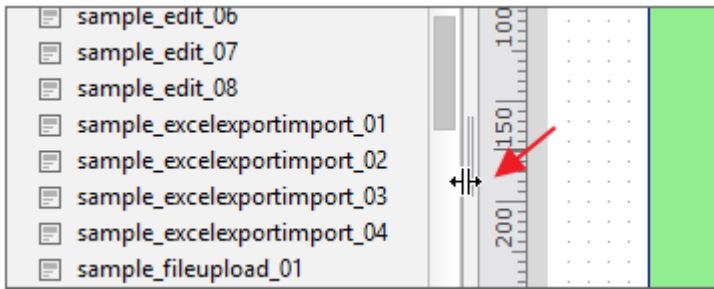
4 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭하고 변경된 콘텐츠를 확인합니다.

41.4 간단한 splitter 구현하기

프레임이나 윈도우 화면이 분리된 경우 구분된 화면의 크기를 조정하기 위해 사용하는 것이 스플리터(splitter)입니다. 넥사크로 스튜디오에서도 각 화면 영역의 크기를 조정하기 위해 사용하고 있습니다. 아래 그림처럼 해당 영역에 마우스 포인터를 가져가면 커서 모양이 변합니다. 이때 마우스 왼쪽 버튼을 누르고 드래그해서 원하는 방향으로 이동

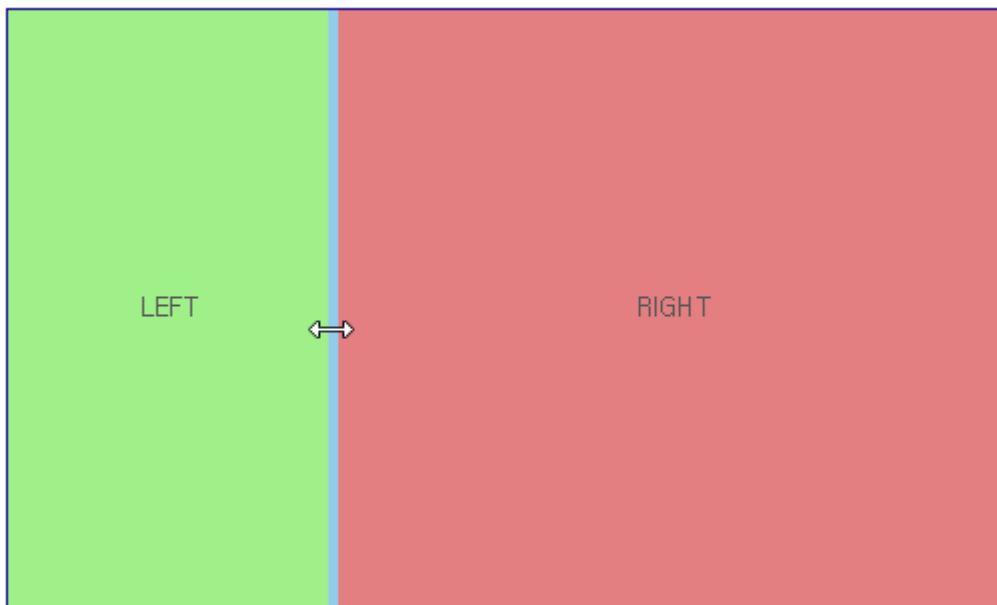
하면 화면 크기를 조정할 수 있습니다.



스플리터(splitter)라는 표현을 주로 사용하고 있지만, resize bar나 다른 표현을 사용하는 경우도 있습니다. 분할된 화면의 크기를 조정하는 기능이 좀 더 정확한 표현이겠네요.

41.4.1 예제

가운데 표시되는 하늘색 막대(스플리터)에 마우스를 가져가면 커서 모양이 바뀝니다. 마우스 왼쪽 버튼을 누르고 마우스 드래그 동작으로 LEFT, RIGHT 영역의 너비를 변경할 수 있습니다.



sample_div_03.xfdl

41.4.2 예제에서 사용한 핵심 기능

ondrag

컴포넌트 위에서 마우스 왼쪽 버튼을 누른 상태에서 드래그 동작을 실행했을 때 발생하는 이벤트입니다. 이때 true 값을 반환해야 상위 컴포넌트로 이벤트가 전파됩니다. 드래그 동작은 드래그를 시작한 컴포넌트의 범위를 넘어서기 때문에 움직이는 좌표값은 상위 컴포넌트에서 처리할 수 있습니다. 예제에서는 상위 컴포넌트의 ondragmove 이벤트에서 스플리터 영역의 움직임을 처리합니다.

cursor

예제에서는 LEFT, RIGHT 영역의 크기만 조정하기 때문에 cursor 속성값을 "e-resize"로 지정했습니다. 스플리터를 움직이는 방향에 따라 속성값을 선택할 수 있습니다.

41.4.3 예제 구현 방법

① 메인 화면 구성하기

화면 구성은 Div 컴포넌트 안에 3개의 Div 컴포넌트를 배치합니다. 가운데 배치되는 Div 컴포넌트가 스플리터 역할을 담당합니다. 3개의 컴포넌트의 top 속성값은 0px, height 속성값은 100%로 지정합니다. 그리고 LEFT Div 컴포넌트의 left 속성값은 0px, RIGHT Div 컴포넌트의 right 속성값도 0px로 지정합니다. 스플리터 Div 컴포넌트의 left 속성값은 적절하게 지정하고 width 속성값은 5px로 지정합니다.

LEFT, RIGHT Div 컴포넌트는 width 속성값을 따로 지정하지 않고 스플리터 Div 컴포넌트의 움직임에 따라 right, left 속성값이 변경될 수 있도록 스플리터 Div 컴포넌트를 기준 컴포넌트로 지정합니다. LEFT Div 컴포넌트의 right 속성값은 "Div01:0", RIGHT 컴포넌트의 left 속성값은 "Div01:0"으로 지정합니다.



기본 컴포넌트는 아래의 방법으로 지정할 수 있습니다.

- PositionBase 트래커 사용
- Position Editor 사용
- 컴포넌트를 선택하고 속성창에서 기본 컴포넌트 지정

② ondrag 이벤트 함수

스플리터 Div 컴포넌트의 ondrag 이벤트를 함수를 추가합니다. 이벤트 함수 내에서는 true 값을 반환해 이벤트를 상위 컴포넌트로 전파합니다.

```
this.Div00_Div01_ondrag = function(obj:nexacro.Div,e:nexacro.DragEventInfo)
{
    return true;
};
```

3 ondragmove 이벤트 함수

바깥쪽을 감싸고 있는 Div 컴포넌트의 ondragmove 이벤트 함수를 추가합니다. 이벤트 함수 내에서는 드래그하는 위치를 확인하고 해당 위치로 스플리터 Div 컴포넌트를 이동합니다.

```
this.Div00_ondragmove = function(obj:nexacro.Div,e:nexacro.DragEventInfo)
{
    this.Div00.form.Div01.set_left(e.clientx);
    this.Div00.form.resetScroll();
};
```



예제에서는 마우스 포인터가 움직이는 동안 쉬지않고 화면을 갱신하는 작업(resetScroll)을 처리합니다. resetScroll 메소드를 실행하지 않으면 기본컴포넌트로 연결된 LEFT, RIGHT Div 컴포넌트의 크기가 변하지 않습니다. 하지만, 화면이 복잡한 경우에는 계속 화면을 갱신하는 작업 때문에 메모리가 증가하거나 속도가 느려질 수 있습니다. 그런 경우에는 드래그 동작이 끝났을 때만(ondrop) 화면을 갱신하는 것을 권장합니다.

4 QuickView로 확인하기

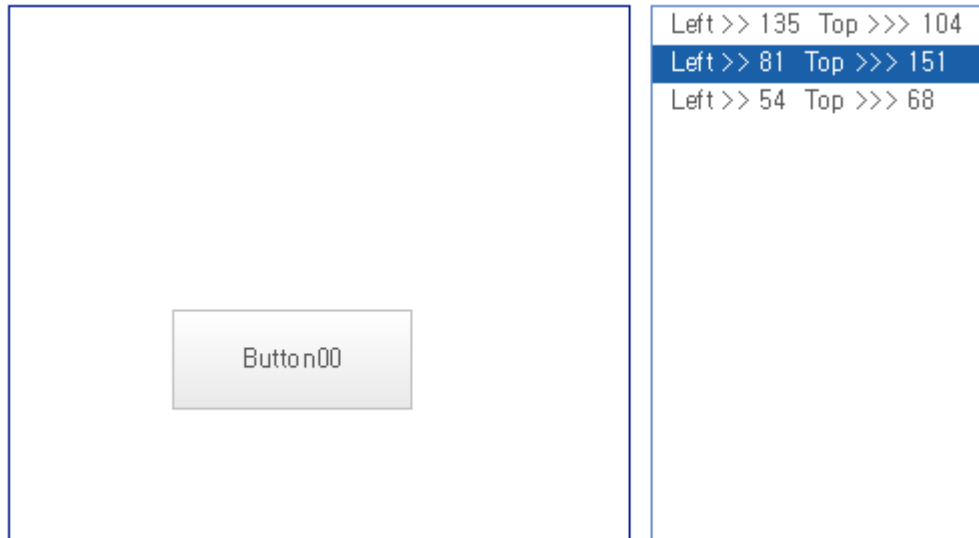
QuickView(Ctrl + F6)로 실행한 후 스플리터 Div 영역을 마우스 왼쪽 버튼으로 누른 상태에서 드래그해서 움직여봅니다.

41.5 마지막 좌표값 처리하기

Div 컴포넌트 내에 Button 컴포넌트를 마우스 드래그 동작으로 움직일 수 있도록 하고 마지막 좌표값을 저장해놓고 이전 좌표값으로 이동하는 방법을 살펴봅니다.

41.5.1 예제

Button 컴포넌트를 마우스로 드래그해서 움직일 수 있습니다. 마우스 버튼을 놓을 때 좌표값을 ListBox 컴포넌트에서 저장하고 ListBox 컴포넌트의 목록을 선택하면 이전 좌표값으로 이동합니다.



41.5.2 예제에서 사용한 핵심 기능

getOffsetWidth, getOffsetHeight

컴포넌트의 너비, 높이를 픽셀 단위 숫자로 반환합니다.

getInnerDataset

컴포넌트의 innerdataset 속성에 설정된 Dataset 오브젝트에 접근하기 위한 용도로 사용합니다.

clientx, clienty

컴포넌트의 클라이언트 영역을 기준으로 좌표값을 반환합니다. 예제에서는 마우스 드래그를 시작할 때 Button 컴포넌트의 clientx, clienty를 임시로 저장해 Div 컴포넌트의 드래그 이벤트에서 발생하는 좌표값을 보정합니다.

41.5.3 예제 구현 방법

- ① Div 컴포넌트를 배치하고 그 안에 Button 컴포넌트를 배치합니다.
- ② Div 컴포넌트 옆에 ListBox 컴포넌트를 배치합니다.
- ③ ListBox 컴포넌트의 innerdataset을 설정합니다. 속성창에서 [...] 버튼을 클릭하고 InnerDataset 설정창에서 Row를 추가하지 않고 [OK] 버튼만 클릭합니다.

- ④ ListBox 컴포넌트의 `codecolumn`, `datacolumn` 속성값을 각각 `"codecolumn"`, `"datacolumn"`으로 지정합니다.
- ⑤ Form 오브젝트의 `onload` 이벤트를 함수를 아래와 같이 작성합니다.

```
var btnOrgW;
var btnOrgH;
this.sample_div_04_onload = function(obj:nexacro.Form,e:nexacro.LoadEventInfo)
{
    var btnOrgW = this.Div00.form.Button00.getOffsetWidth();
    var btnOrgH = this.Div00.form.Button00.getOffsetHeight();
};
```

- ⑥ Button 컴포넌트의 `onmousedown` 이벤트를 함수를 아래와 같이 작성합니다.
- 드래그 동작이 시작하는 시점에 Button 컴포넌트의 좌표값을 확인합니다.

```
var nButtonX;
var nButtonY;
this.Div00_Button00_onmousedown = function(obj:nexacro.Button,e:nexacro.MouseEventInfo)
{
    nButtonX = e.clientx;
    nButtonY = e.clienty;
};
```

- ⑦ Div 컴포넌트의 `ondrag`, `ondragmove`, `ondrop` 이벤트를 함수를 아래와 같이 작성합니다.
- `ondrop` 이벤트 함수에서는 마지막 좌표값을 ListBox 컴포넌트의 `innerdataset` 오브젝트에 추가합니다.

```
this.Div00_ondrag = function(obj:nexacro.Div,e:nexacro.DragEventInfo)
{
    return true;
};

this.Div00_ondragmove = function(obj:nexacro.Div,e:nexacro.DragEventInfo)
{
    var nLeft = e.clientx-nButtonX;
    var nTop = e.clienty-nButtonY;
    this.Div00.form.Button00.move(nLeft,nTop, btnOrgW, btnOrgH);
};
```



```

this.Div00_ondrop = function(obj:nexacro.Div,e:nexacro.DragEventInfo)
{
    var tempDS = this.ListBox00.getInnerDataset();
    var nIndex = tempDS.rowcount;
    tempDS.insertRow(nIndex);
    tempDS.setColumn(nIndex, "codecolumn", (e.clientx-nButtonX)+","+(e.clienty-nButtonY));
    tempDS.setColumn(nIndex, "datacolumn", "Left >> " + (e.clientx-nButtonX) + " Top >>>
" + (e.clienty-nButtonY));
    this.resetScroll();
};

```

- ⑧ ListBox 컴포넌트의 onitemchanged, onkillfocus 이벤트 함수를 아래와 같이 작성합니다.

특정 좌표값을 선택하면 해당 위치로 Button 컴포넌트를 움직입니다. ListBox 컴포넌트에서 포커스를 벗어나게 되면 선택 상태를 초기화합니다.

```

this.ListBox00_onitemchanged = function(obj:nexacro.ListBox,e:nexacro.ItemChangeEventInfo)
{
    if(e.postindex!==-1)
    {
        var tempArray = e.postvalue.split(',');
        var nLeft = tempArray[0];
        var nTop = tempArray[1];
        this.Div00.form.Button00.move(nLeft,nTop, btn0rgW, btn0rgH);
    }
};

this.ListBox00_onkillfocus = function(obj:nexacro.ListBox,e:nexacro.KillFocusEventInfo)
{
    obj.setSelect(-1);
};

```

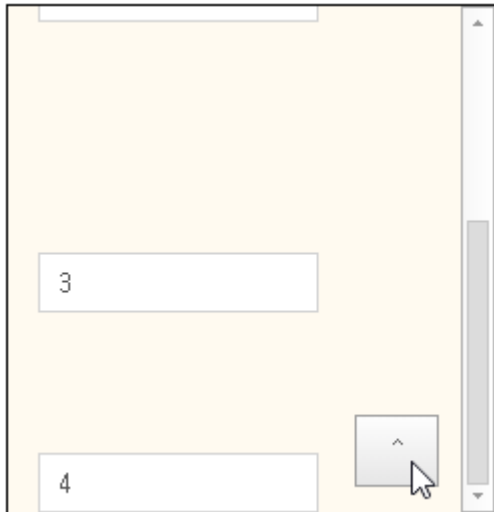
- ⑨ QuickView(Ctrl + F6)로 실행한 후 Button 컴포넌트를 드래그앤드롭 동작으로 움직이고 ListBox 컴포넌트에 기록된 이전 좌표값을 선택해 Button 컴포넌트를 이전 위치로 이동해봅니다.

41.6 맨 위로 가기 버튼 만들기

스크롤이 길어지는 콘텐츠의 경우 맨 위로 바로 가기 버튼을 보여주는 경우가 있습니다. 스크롤바를 움직이지 않고 맨 위로 이동하고자 할 때 편하게 사용할 수 있습니다.

41.6.1 예제

Div 컴포넌트의 스크롤바를 아래로 움직이면 위로 가기 버튼이 표시되고 버튼을 클릭하면 맨 위로 이동합니다.



41.6.2 예제에서 사용한 핵심 기능

pos

스크롤바의 위치값을 갖는 속성입니다. 예제에서는 Div 컴포넌트 내 innerform 오브젝트의 vscrollbar 컨트롤의 속성값을 설정합니다.

onvscroll

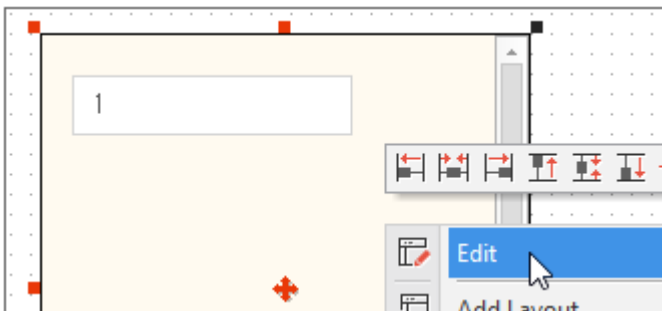
스크롤바가 움직일때 발생하는 이벤트입니다. 예제에서는 위치가 맨 위가 아닌 경우 버튼을 보이게 합니다.

41.6.3 예제 구현 방법

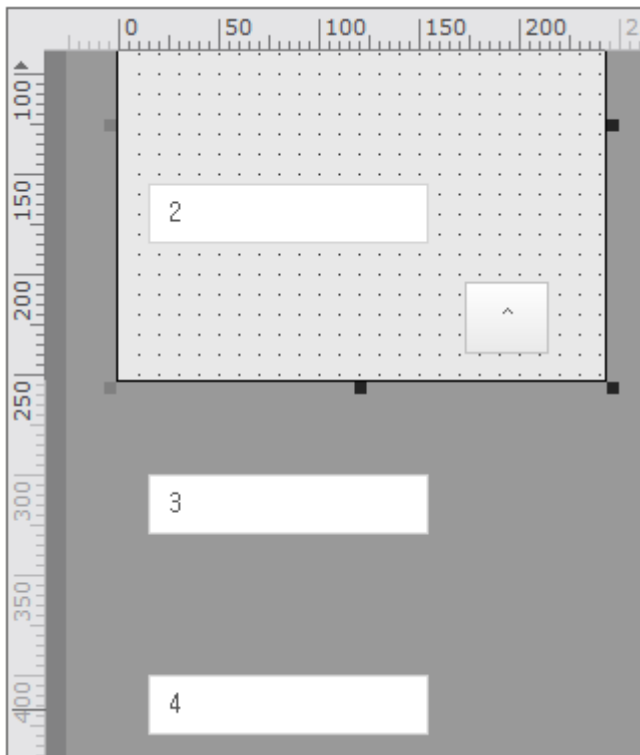
① Div 컴포넌트를 배치하고 그 안에 Edit, Button 컴포넌트를 배치합니다.

② Div 컴포넌트를 더블클릭합니다.

Div 컴포넌트를 더블클릭하거나 컨텍스트 메뉴에서 [Edit] 항목을 선택하면 Div 컴포넌트 내 배치된 컴포넌트의 위치를 편집할 수 있습니다. 예제에서는 일부 Edit 컴포넌트는 Div 컴포넌트 영역 밖에 배치합니다.



세 번째와 네 번째 Edit 컴포넌트는 Div 컴포넌트 영역 밖에 배치합니다.



③ Div 컴포넌트 내 Edit 컴포넌트를 배치했으면 ESC 키를 누르거나 컨텍스트 메뉴에서 [Exit]를 선택합니다.

④ Div 컴포넌트의 onvscroll 이벤트를 함수를 아래와 같이 작성합니다.

```

var nTop = this.Div00.form.Button00.top;

this.Div00_onvscroll = function(obj:nexacro.Div,e:nexacro.ScrollEventInfo)
{
    if( e.pos > 0 ) {
        this.Div00.form.Button00.set_visible(true);
        this.Div00.form.Button00.set_top(nexacro.toNumber(nTop)+ nexacro.toNumber(e.pos));
    } else {
        this.Div00.form.Button00.set_visible(false);
    }
};

```

- ⑤ Button 컴포넌트의 onclick 이벤트 함수를 아래와 같이 작성합니다.

```

this.Div00_Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Div00.form.vscrollbar.set_pos(0);
};

```

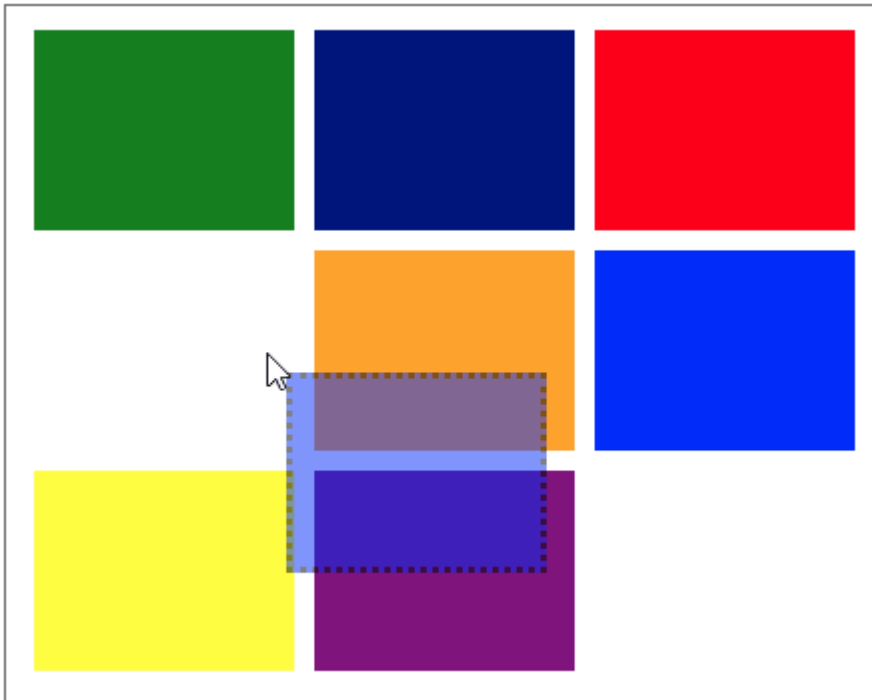
- ⑥ QuickView(Ctrl + F6)로 실행한 후 스크롤바를 아래로 움직여서 버튼이 표시되는지 확인하고 버튼 클릭 시에 맨 위로 이동하는지 확인합니다.

41.7 포틀릿 화면 구현하기

포틀릿(Portlet)은 작은 창을 의미합니다. 여러 정보를 박스 형태로 보여주는 용도로 사용합니다. 예제에서는 Div 컴포넌트를 포틀릿 형태로 배치하고 각 창을 이동하는 방식을 살펴봅니다.

41.7.1 예제

Div 컴포넌트를 마우스 드래그 동작으로 다른 Div 컴포넌트 위치로 가져가서 두 개의 Div 컴포넌트 위치를 바꿉니다.



41.7.2 예제에서 사용한 핵심 기능

fromobject

이벤트가 발생한 오브젝트를 가리킵니다. 예제에서는 Div 컴포넌트의 InnerForm 오브젝트가 fromobject가 되며 Div 컴포넌트를 찾기 위해서는 fromobject의 parent를 찾습니다.

userdata

DragEventInfo 오브젝트에서 userdata 속성값을 지정하면 연결된 드래그 동작에서 해당 속성값을 가져올 수 있습니다. 예제에서는 ondrag 이벤트 함수에서 속성값을 지정하고 ondrop 이벤트 함수에서 해당 값을 사용합니다.

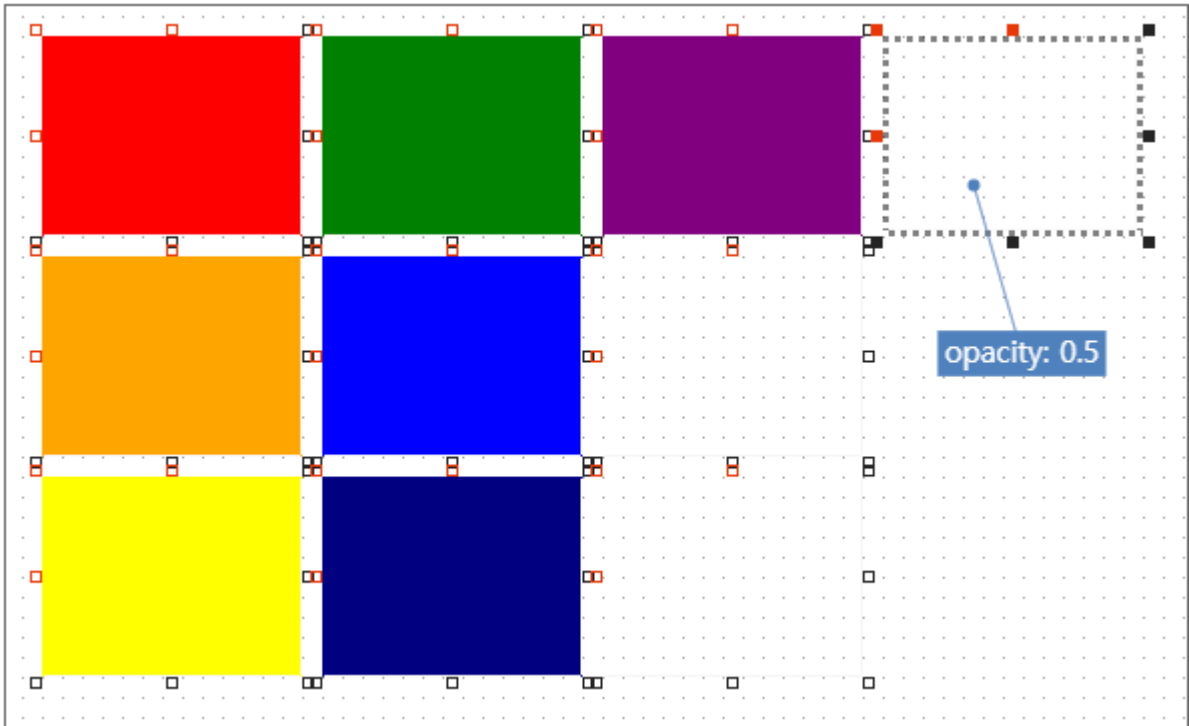
41.7.3 예제 구현 방법

- 1 Div 컴포넌트를 아래와 같이 배치합니다.

각 Div 컴포넌트는 다른 색으로 background 속성값을 지정합니다. 아래 예제에서 오른쪽 아래의 2개 Div 컴포넌트는 배경색을 지정하지 않았습니다.

오른쪽에 배치한 Div 컴포넌트는 드래그 동작 시 움직임을 표현하기 위한 임시 Div 컴포넌트입니다. visible 속

성값을 false로 지정하고 border 속성을 적절하게 지정합니다. 이동하는 Div 컴포넌트의 색상을 연하게 표현하기 위해 opacity 속성값은 0.5로 지정합니다.

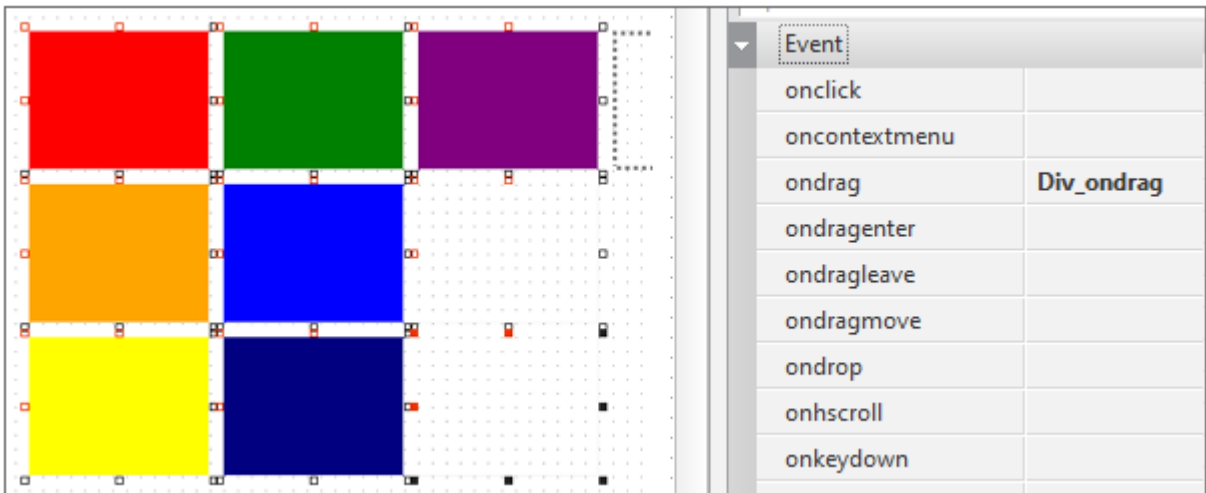


- ② Div 컴포넌트에 ondrag 이벤트 함수를 아래와 같이 추가합니다.

공통으로 하나의 함수를 사용하기 때문에 스크립트 창에서 함수를 미리 만들어놓고 컴포넌트 이벤트 속성으로 지정합니다.

```
this.Div_ondrag = function(obj:nexacro.Div,e:nexacro.DragEventInfo)
{
    e.userdata = obj;
    this.fn_makeDragObj(obj);
    return true;
};

this.fn_makeDragObj = function (obj)
{
    this.divMove.set_background(obj.background);
    this.divMove.set_visible(true);
};
```



- 3 Form 오브젝트의 ondragmove, ondrop 이벤트를 함수를 작성합니다.

```
this.form_ondragmove = function(obj:nexacro.Form,e:nexacro.DragEventInfo)
{
    this.divMove.move(e.clientX + 10, e.clientY + 10);
};
```

Div 컴포넌트 이동 시 애니메이션 처리를 위해 Animation 오브젝트를 생성합니다.

```
this.fv_objFrom;
this.fv_objTo;

var aniObj = new nexacro.Animation("Ani00", this);
this.addChild("Ani00", aniObj);

this.form_ondrop = function(obj:nexacro.Form,e:nexacro.DragEventInfo)
{
    this.divMove.set_visible(false);
    this.fv_objFrom = e.userData;
    this.fv_objTo = e.fromobject.parent;
    if (this.fv_objTo != "[object Div]")
    {
        return;
    }

    this.Ani00.addTarget("AniItem00", this.fv_objFrom, "left:"+this.fv_objTo.left+", top:"
+this.fv_objTo.top);
    this.Ani00.addTarget("AniItem01", this.fv_objTo, "left:"+this.fv_objFrom.left+", top:"
```

```
+this.fv_objFrom.top);  
    this.Ani00.play();  
    this.Ani00.removeTarget("AniItem00");  
    this.Ani00.removeTarget("AniItem01");  
};
```

- ④ QuickView(Ctrl + F6)로 실행한 후 배경색이 설정된 Div 컴포넌트를 마우스 드래그로 움직여서 다른 Div 컴포넌트 또는 빈 Div 컴포넌트 영역으로 이동합니다.

42.

PopupDiv

42.1 PopupDiv 소개

PopupDiv는 하나의 화면을 여러 개의 부분 화면으로 나눠 구성할 때 사용하는 컴포넌트입니다. 특정 메시지나 추가적인 정보와 같이 필요할 때 한 번씩 확인하는 경우에 사용하면 유용합니다.

기본적으로는 Div와 동일하나 팝업으로 동작하기 때문에 화면상에서 다른 UI 요소들보다 상위에 위치합니다. 그리고 포커스를 잃으면 자동으로 사라지기 때문에 여러 개를 동시에 화면에 출력할 수 없다는 특징이 있습니다. 화면에 PopupDiv를 여러 개 생성해 배치해도 한 번에 보여줄 수 있는 건 한 개뿐입니다.

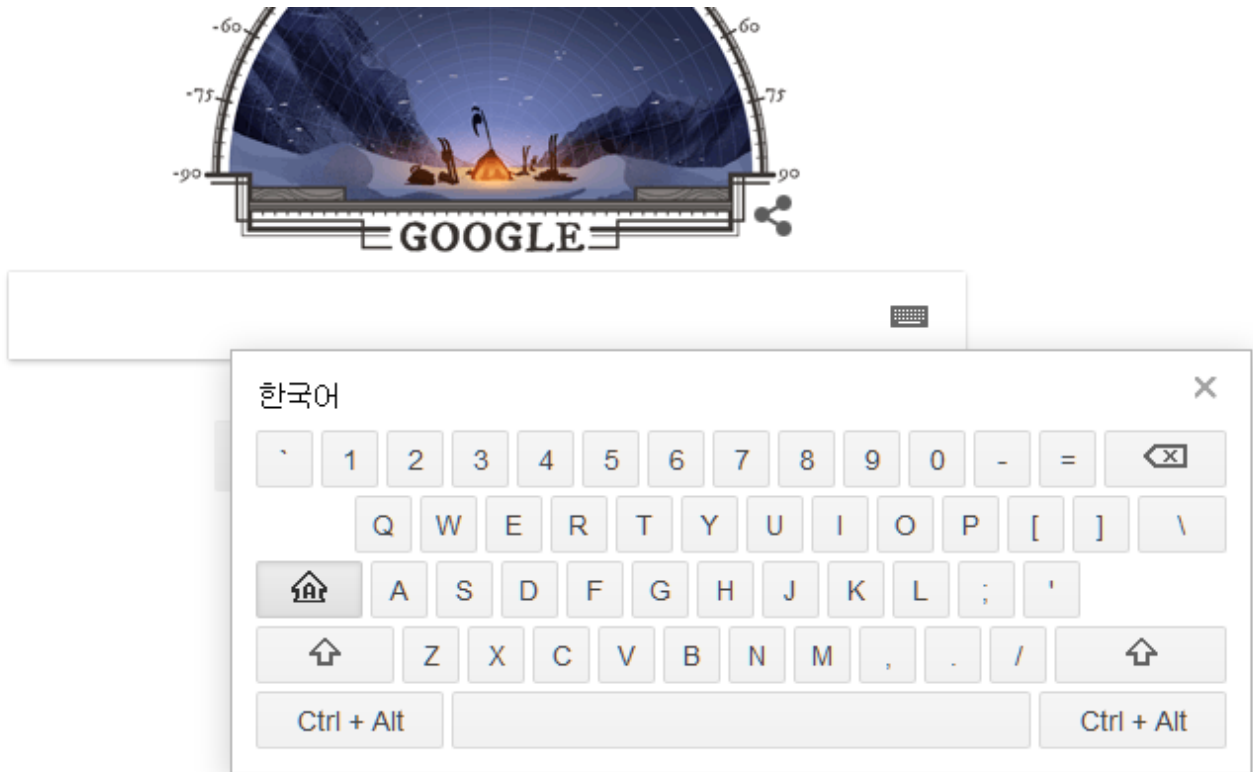
PopupDiv는 화면의 구성을 수월하게 하고 화면을 분할해서 폼에 대한 재사용성을 높입니다. 그러나 중첩해서 사용하는 경우 단계가 깊어질수록 초기화 시간과 화면 로딩이 느려집니다. 예를 들어, 1개 화면에 2~3단계로 구성된 PopupDiv를 여러 개 사용하는 것이 4~5단계로 구성된 PopupDiv를 1개 사용하는 것보다 빠릅니다. 따라서 PopupDiv는 보통 2~3단계 정도로만 중첩하여 구성하는 것이 좋습니다.

PopupDiv는 화면 디자인 시 미리 생성해 놓고 배치할 수도 있고 실행중에 스크립트로 생성해 보여줄 수도 있습니다. 다른 컴포넌트와 달리 폼에 생성해서 배치해도 화면에서 보이지 않습니다. 실제로 화면에 출력하기 위해서는 trackPopup, trackPopupComponent 메소드를 사용해서 화면에 배치하는 작업을 별도로 해줘야 합니다.

PopupDiv는 팝업으로 동작하지만 실제 팝업 창과는 다르게 UI 영역 내에서만 동작합니다. 폼의 영역 내에서만 보이며 그 영역을 벗어나는 부분은 잘려서 보이게 되므로 사용 시 유의해야 합니다.

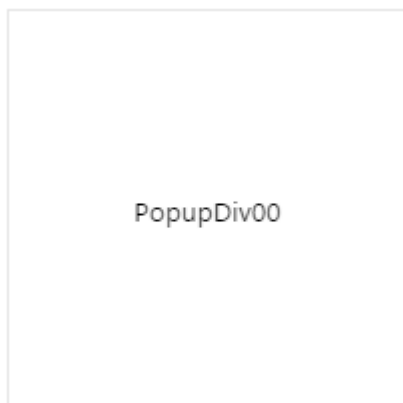
42.1.1 사용 예

다음은 대표적인 검색 엔진인 구글의 검색 페이지입니다. 구글의 검색 페이지에서는 사용자 입력을 돕기 위해 키보드 형태의 입력 도구를 지원하는데 이 키보드와 같은 UI를 PopupDiv로 만들 수 있습니다.



42.1.2 PopupDiv 만들기

다음은 마우스로 폼을 클릭하면 PopupDiv가 나타나는 예제입니다. PopupDiv를 생성해 놓은 상태에서 trackPopup, trackPopupByComponent 메소드를 사용해 화면에 출력합니다.



① PopupDiv 컴포넌트 생성하기

툴바에서 PopupDiv  를 선택한 후, Form 위에 적당한 크기로 드래그하여 생성합니다. 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

2 PopupDiv 화면에 출력하기

품을 클릭하면 PopupDiv가 화면에 출력되도록 onclick 이벤트 핸들러를 생성한 후 다음과 같이 trackPopup 메소드를 호출합니다.

```
this.Form_onclick = function(obj:nexacro.Form,e:nexacro.ClickEventInfo)
{
    this.PopupDiv00.trackPopup(40, 32, 200, 200);
};
```

3 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행합니다.

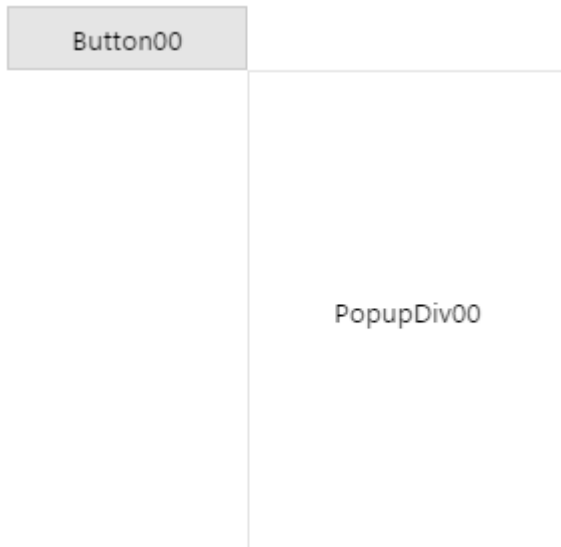
품을 클릭하면 PopupDiv가 화면에 나타나며 다시 클릭하면 사라집니다.

42.2 PopupDiv 배치하기

생성한 PopupDiv 컴포넌트를 화면에 출력하려면 trackPopup, trackPopupByComponent 메소드를 사용해서 좌표와 크기를 설정해야 합니다. 두 메소드는 같은 기능을 하는데 trackPopup 메소드는 UI 화면의 좌상단을 기준으로 좌표를 결정하고 trackPopupByComponent 메소드는 인수한 컴포넌트를 기준으로 좌표를 결정한다는 차이가 있습니다. 두 메소드중 어떤 것을 사용해도 상관없으며 상황에 맞게 메소드를 사용하면 됩니다.

42.2.1 예제

다음은 trackPopupByComponent 메소드를 사용하여 PopupDiv 컴포넌트를 화면에 배치한 예제입니다. trackPopupByComponent 메소드는 다른 컴포넌트를 기준으로 좌표를 설정하는 메소드로 예제에서는 Button00을 기준으로 PopupDiv00을 화면에 배치하였습니다.



sample_popupdiv_02.xfdl

42.2.2 예제에서 사용한 핵심 기능

trackPopupByComponent

인수로 전달된 컴포넌트를 기준으로 PopupDiv를 특정 위치에 배치하는 메소드입니다.

```
this.PopupDiv00.trackPopupByComponent( this.Button00, 10, 10 );
this.PopupDiv00.trackPopupByComponent( this.Button00, 10, 10, 200, 200, "fn_Callback" );
```

getOffsetHeight

컴포넌트의 height 속성값을 픽셀 단위로 반환하는 메소드입니다.

getOffsetWidth

컴포넌트의 width 속성값을 픽셀 단위로 반환하는 메소드입니다.

42.2.3 예제 구현 방법

① PopupDiv 컴포넌트 생성하기

툴바에서 Button  과 PopupDiv  를 선택한 후, Form 위에 적당한 크기로 드래그하여 생성합니다. 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

PopupDiv는 trackPopupByComponent 메소드에 의해 애플리케이션 실행 시 재배치 되므로 넥사크로스튜

디오의 디자인 화면에서는 배치에 신경쓰지 않아도 됩니다.

② Button 이벤트 핸들러 작성하기

버튼 클릭 시 PopupDiv가 정해진 위치에 배치되도록 trackPopupByComponent 메소드를 호출합니다.

- Button00 버튼의 onclick 이벤트 핸들러

```
this.Button00.onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.PopupDiv00.trackPopupByComponent(obj, obj.getOffsetWidth(), obj.getOffsetHeight
    (), 160, 240);
};
```

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행합니다.

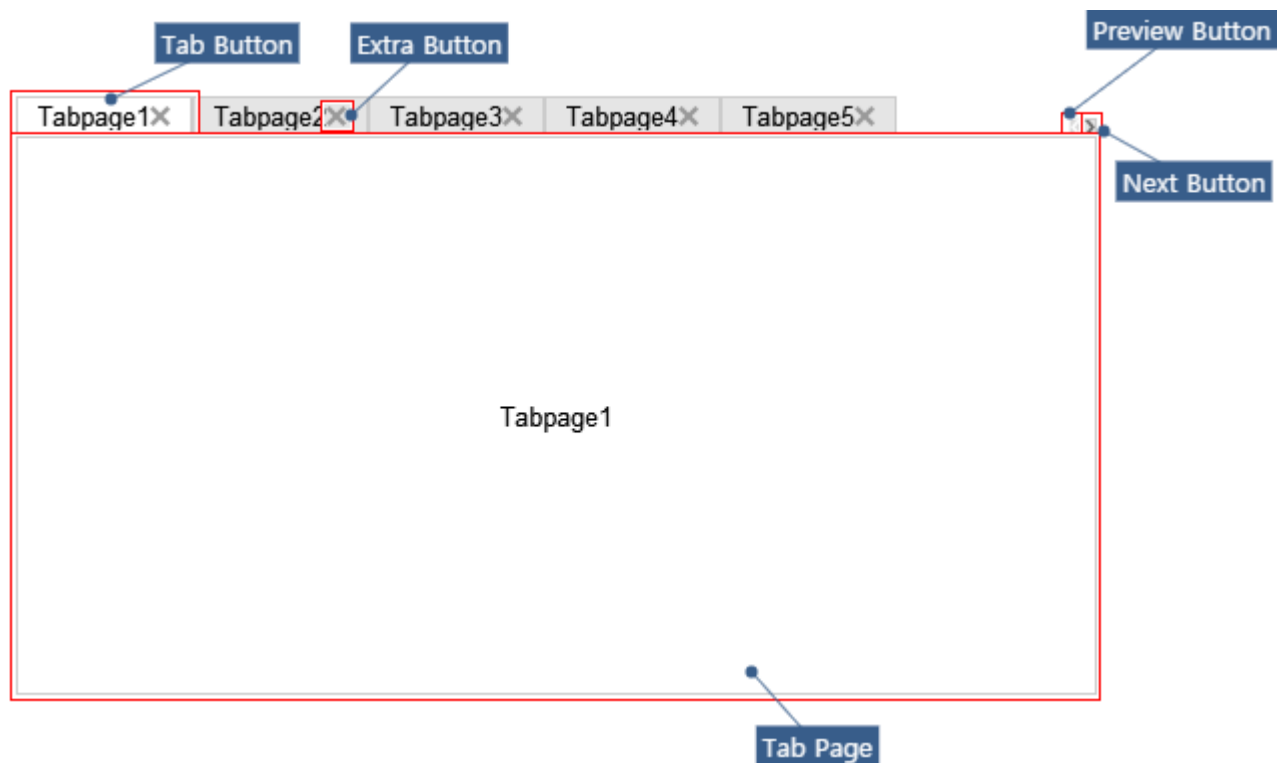
Button00을 클릭하여 PopupDiv가 설정한 위치에 출력되는지 확인합니다.

43.

Tab

43.1 Tab 소개

Tab은 좁은 화면에 많은 수의 컴포넌트를 배치할 필요가 있을 때 사용하는 컴포넌트입니다. 색인표라는 Tab의 사전적 의미처럼 사용자는 탭 페이지라는 색인표를 보며 필요한 정보를 효율적으로 탐색할 수 있습니다.



Tab의 구성 요소는 다음과 같습니다.

구성 요소	설명
탭 버튼(Tab Button)	탭 페이지와 1대1로 매핑되어 탭 페이지를 전환하는 데 사용하는 버튼입니다.
엑스트라 버튼(Extra Button)	이벤트 처리를 통해 기능을 수행할 수 있는 버튼입니다. 일반적으로는 탭 페이지를 화면에서 제거하는 용도로 사용합니다.
프리뷰 버튼(Preview Button)	탭 버튼의 수가 많아 Tab 컴포넌트의 너비를 넘어갔을 때 생기는 버튼으로

43.1.2 Tab 만들기

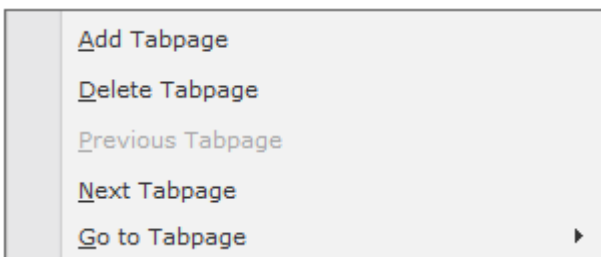


1 Tab 컴포넌트 생성하기

툴바에서 Tab  을 선택한 후, 폼 위에 적당한 크기로 드래그하여 생성합니다. 그냥 폼에 대고 클릭하면 기본 크기로 생성되며 2개의 탭 페이지가 자동 생성됩니다.

2 탭 페이지 생성하기

생성된 Tab을 선택한 후 마우스 오른쪽 버튼을 누르면 탭 페이지 컨텍스트 메뉴가 뜹니다. [Add Tabpage]를 선택하면 마지막 탭 페이지 뒤로 새로운 탭 페이지가 추가됩니다.



탭 페이지를 삭제하려면 [Delete Tabpage]를 선택합니다. 그러면 현재 활성화되어 있는 탭 페이지가 삭제됩니다.

탭 페이지 간의 전환은 [Previous Tabpage], [Next Tabpage], [Go to Tabpage] 메뉴를 사용합니다.

3 엑스트라 버튼 생성하기

Tab 컴포넌트의 showextrabutton 속성을 'true'로 설정하면 탭 버튼 위에 엑스트라 버튼이 생성됩니다. 생성

만으로는 어떤 동작도 하지 않으므로 onextrabuttonclick 이벤트를 통해 버튼의 기능을 정의해줘야 합니다.

4 프리뷰/넥스트 버튼 생성하기

프리뷰, 넥스트 버튼은 Tab 컴포넌트의 너비 이상으로 탭 버튼의 수가 증가하면 자동으로 생성됩니다. 탭 버튼의 수가 늘어나도록 탭 페이지를 적당히 생성합니다.

5 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 탭 페이지를 전환하여 동작에 이상이 없는지 확인합니다.

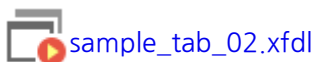
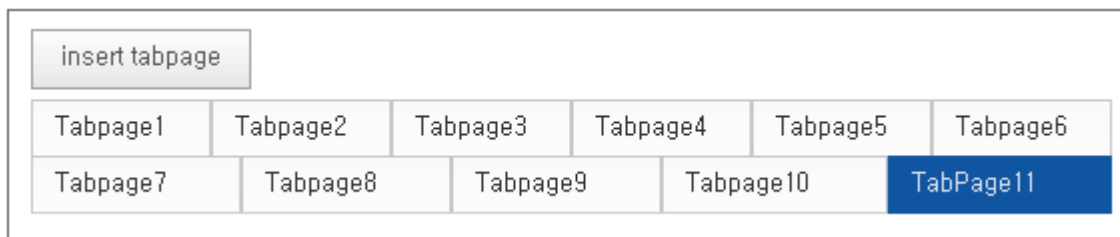
Tab은 별도로 탭 페이지를 추가하지 않으면 기본적으로 2개의 탭 페이지를 가집니다.

43.2 멀티 라인 Tab 구성하기

표시할 정보가 많고 어떤 정보들이 있는지 한눈에 보여줄 필요가 있을 때 Tab의 멀티 라인 기능을 사용하면 편리합니다. 멀티 라인 기능은 다수의 탭 페이지로 구성된 Tab에서 탭 버튼을 여러 줄로 배치하여 어떤 탭 페이지가 있는지 사용자가 한눈에 확인할 수 있게 합니다.

43.2.1 예제

multiline 속성을 설정하고 탭 버튼의 너비가 Tab의 너비보다 크도록 탭 페이지를 추가하면 멀티 라인 Tab으로 구성됩니다. 버튼을 클릭하면 TabPage가 추가됩니다.



43.2.2 예제에서 사용한 핵심 기능

multiline

Tab 컴포넌트에 일정 수 이상으로 탭 페이지를 추가했을 때 탭 버튼을 여러 줄로 배치할지 설정하는 속성입니다.

43.2.3 예제 구현 방법

① Tab 컴포넌트 생성하기

툴바에서 Tab  을 선택한 후, 폼 위에 적당한 크기로 드래그하여 생성합니다.

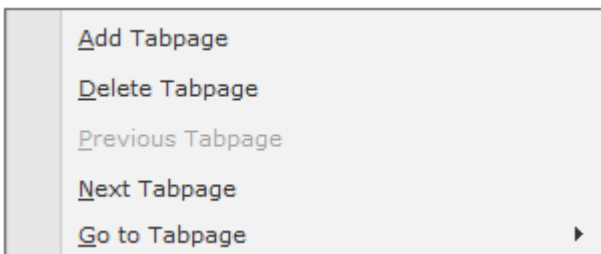
② multiline 속성 설정하기

Tab 컴포넌트를 선택한 후 Properties 창에서 multiline 속성을 'true'로 설정합니다.

③ TabPage 추가하기

기본 생성된 Tab은 2개의 탭 페이지를 갖습니다. 멀티 라인 Tab을 구성하기 위해 탭 페이지를 추가합니다.

Tab을 선택한 후 마우스 커서를 탭 페이지 영역에 놓고 마우스 오른쪽 버튼을 클릭하면 아래 그림과 같이 컨텍스트 메뉴가 나옵니다. [Add Tabpage]를 클릭하여 탭 버튼이 여러 줄이 될 때까지 새로운 탭 페이지를 생성합니다.



Tab 관련 메뉴의 기능은 다음과 같습니다.

- [Add Tabpage] 새로운 탭 페이지를 생성합니다.
- [Delete Tabpage] 현재 선택된 탭 페이지를 제거합니다.
- [Previous Tabpage] 현재 선택된 탭 페이지 바로 앞의 인덱스를 가진 탭 페이지를 선택합니다.
- [Next Tabpage] 현재 선택된 탭 페이지 바로 뒤의 인덱스를 가진 탭 페이지를 선택합니다.
- [Go to Tabpage] 특정 탭 페이지를 지정하여 선택합니다.

④ onclick 이벤트 함수 작성하기

Tab 컴포넌트 위에 Button 컴포넌트를 배치하고 onclick 이벤트를 추가합니다.

```
this.btnInsert_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var i = this.Tab00.getTabPageCount()+1;
    this.Tab00.insertTabPage("TabPage"+i, -1);
};
```

5 QuickView로 확인하기

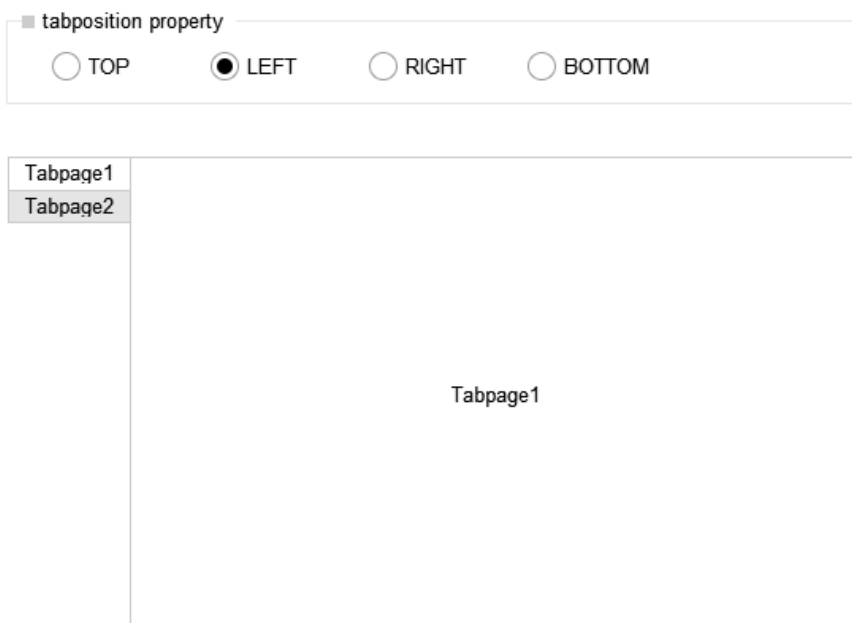
QuickView(Ctrl + F6)로 실행합니다. TagButton이 여러 줄로 구성되어 보이는지 확인합니다.

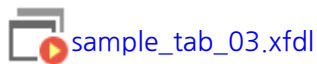
43.3 탭 버튼 위치 조정하기

화면 구성에 따라 탭 버튼의 위치를 왼쪽이나 오른쪽 혹은 아래쪽으로 위치시켜야 할 때가 있습니다. 이때, Tab 컴포넌트의 tabposition 속성을 사용하면 탭 버튼의 위치를 변경할 수 있습니다.

43.3.1 예제

tabposition 속성을 설정하여 탭 버튼의 위치를 설정합니다. 탭 버튼의 위치는 왼쪽, 오른쪽, 위, 아래로 설정할 수 있습니다.





43.3.2 예제에서 사용한 핵심 기능

tabposition

Tab 컴포넌트에서 탭 버튼이 표시되는 위치를 설정하는 속성입니다.

43.3.3 예제 구현 방법

① Tab 컴포넌트 생성하기

툴바에서 Tab , Radio  컴포넌트를 생성하여 예제 화면과 같이 폼에 배치합니다.

② Radio 컴포넌트 설정하기

Tab 컴포넌트의 tabposition 속성을 설정하기 위해 라디오 버튼을 사용합니다. tabposition 속성은 'top', 'left', 'right', 'bottom'의 총 4가지 값으로 설정할 수 있습니다.

1. 라디오 메뉴 구성

4개의 라디오 버튼을 생성하기 위해 Radio 컴포넌트를 선택한 후 속성창의 innerdataset 속성에서 [...] 버튼을 눌러 innerDataset을 다음과 같이 설정합니다.

No	codecolumn	datacolumn
1	top	TOP
2	left	LEFT
3	right	RIGHT
4	bottom	BOTTOM

2. Radio 속성 설정

속성	값
columncount	4
index	0

3. onitemchanged 이벤트 함수 등록하기

라디오 버튼의 선택에 따라 Tab의 tabposition 속성을 설정하기 위해 Radio 컴포넌트에 onitemchanged 이

벤트 함수를 작성하여 등록합니다.

```
this.Radio00_onitemchanged = function(obj:nexacro.Radio,e:nexacro.ItemChangeEventInfo)
{
    this.Tab00.set_tabposition(e.postvalue);
};
```

3 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행합니다. 라디오 버튼을 클릭하여 탭 버튼의 위치가 바뀌는지 확인합니다.

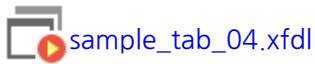
43.4 탭 버튼 비활성화

상황에 따라 사용자에게 탭 페이지를 선별해서 제공해야 할 경우가 있습니다. 이때, 탭 버튼을 비활성화하면 사용자의 불필요한 접근을 막을 수 있습니다.

43.4.1 예제

총 4개의 탭 페이지 중 2, 3번을 비활성화한 예제입니다. 화면에는 보이거나 비활성화되어 선택할 수 없습니다.





43.4.2 예제에서 사용한 핵심 기능

enable

탭 페이지의 사용 가능 여부를 설정하는 속성입니다.

postindex

새로 변경된 탭 페이지의 index 값을 갖는 TabIndexChangeEventInfo 이벤트 정보 오브젝트의 속성입니다.

43.4.3 예제 구현 방법

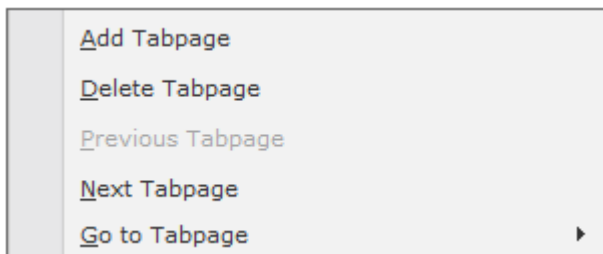
① Tab 컴포넌트 생성하기

툴바에서 Tab  컴포넌트를 생성하여 예제 화면과 같이 Form에 배치합니다.

② 탭 페이지 생성하기

기본 생성된 Tab은 2개의 탭 페이지를 갖습니다. 4개의 탭 페이지를 가질 수 있도록 추가합니다.

Tab을 선택한 후 마우스 오른쪽 버튼을 클릭하면 탭 페이지 관련 컨텍스트 메뉴가 뜹니다. [Add Tabpage]를 선택하면 마지막 탭 페이지 뒤로 새로운 탭 페이지가 추가됩니다.



탭 페이지를 삭제하려면 [Delete Tabpage]를 선택합니다. 그러면 현재 활성화되어 있는 탭 페이지가 삭제됩니다.

탭 페이지 간의 전환은 [Previous Tabpage], [Next Tabpage], [Go to Tabpage] 메뉴를 사용합니다.

③ 탭 페이지의 enable 속성 설정하기

비활성화 처리하려는 Tabpage2와 Tabpage3를 각각 선택하여 enable 속성을 'false'로 설정합니다.

탭 페이지 이름	enable 속성값
Tabpage1	true
Tabpage2	false
Tabpage3	false
Tabpage4	true

4 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행합니다. 탭 버튼 클릭하여 탭 페이지를 전환하며 설정한 데로 동작하는지 확인합니다.

파트 V.

Animation Objects

44.

Animation

44.1 Animation 소개

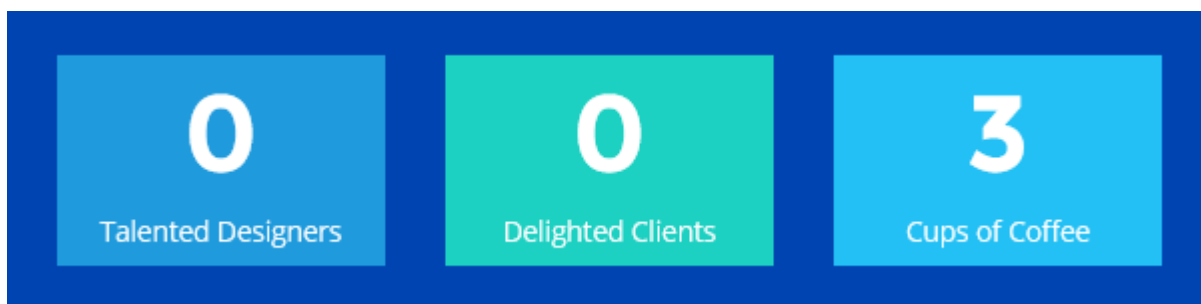
애니메이션 기능은 데이터를 돋보이게 하거나 특정 기능을 강조하기 위해 사용합니다. 예를 들어 Edit 컴포넌트에 잘못된 값을 입력했을 때 살짝 흔들리는 효과를 추가하거나 데이터가 업데이트되는 것을 강조할 수 있습니다.



Animation 오브젝트는 17.0.0.700 버전부터 지원합니다.

44.1.1 사용 예

화면에 표시되는 데이터를 강조하기 위해 카운터 효과를 내는 것도 애니메이션 기능을 활용하는 것입니다.



44.1.2 Animation 만들기

① Animation 오브젝트 생성하기

Animation 오브젝트는 넥사크로 스튜디오 툴바에 표시되지 않습니다. 스크립트에서 새로운 오브젝트를 생성 해주어야 합니다. form 오브젝트의 onload 이벤트 함수 내에서 Animation 오브젝트를 추가한다면 다음과 같이 이벤트 함수를 작성합니다.

```

this.form_onload = function(obj:nexacro.Form,e:nexacro.LoadEventInfo)
{
    var aniObj = new nexacro.Animation("Ani00", this);
    this.addChild("Ani00", aniObj);
    this.Ani00.set_loop(true);
}

```

화면에 보여지는 것이 아니기때문에 addChild 메소드 호출 후 show 메소드를 호출하지 않습니다.

2 Animation 오브젝트의 속성 지정하기

어떤 형식으로 애니메이션 효과를 처리할지 지정할 수 있는 속성이 있습니다. Animation 오브젝트를 생성한 이후 원하는 속성을 지정할 수 있습니다.

```

this.Ani00.set_loop(true);
this.Ani00.set_direction("alternate");

```

3 애니메이션 대상 선택하기

애니메이션 효과를 적용할 대상을 지정해주어야 합니다. addTarget 메소드를 사용해 애니메이션 효과를 적용할 컴포넌트와 해당 컴포넌트의 어떤 속성을 변경할지 지정해줍니다. 예를 들어 Button 컴포넌트의 left 속성값을 변경해 컴포넌트를 오른쪽으로 이동시키면서 width 속성값을 변경해 크기를 변경시킬 수 있습니다.

```

this.Ani00.addTarget("AniItem00", this.Button00, "width:200, left:300");

```

addTarget 메소드가 정상적으로 실행되면 AnimationItem 오브젝트가 Animation 오브젝트 아래 추가됩니다.

```

▼ this.Ani00.items: nexacro.Collection
  ▶ 0: nexacro.AnimationItem {name: "AniItem00", id: "AniItem00", parent:
  ▶ 1: nexacro.AnimationItem {name: "AniItem01", id: "AniItem01", parent:
  ▶ 2: nexacro.AnimationItem {name: "AniItem02", id: "AniItem02", parent:
  ▶ AniItem00: nexacro.AnimationItem {name: "AniItem00", id: "AniItem00",
  ▶ AniItem01: nexacro.AnimationItem {name: "AniItem01", id: "AniItem01",
  ▶ AniItem02: nexacro.AnimationItem {name: "AniItem02", id: "AniItem02",
    length: 3

```

4 애니메이션 효과 실행하기

원하는 애니메이션 효과의 형태와 대상을 지정한 다음에는 애니메이션 효과를 실행합니다. play 메소드를 사용해 애니메이션 효과를 실행합니다. loop 설정을 한 경우에는 애니메이션 효과를 멈추기 위해 stop 메소드나 pause 메소드를 사용할 수 있습니다.

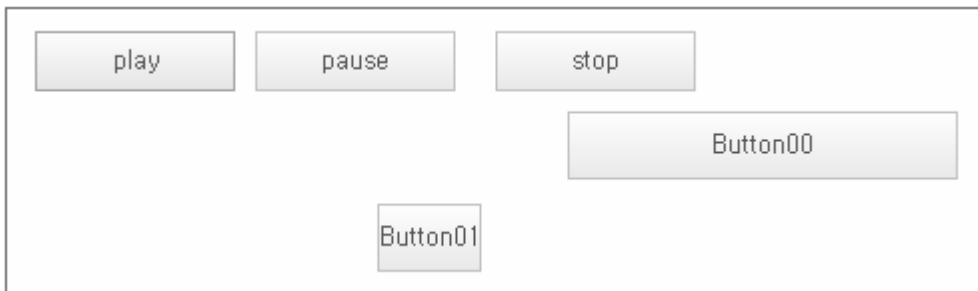
```
this.Animate.play();
```

44.2 크기과 위치 변경하기

addTarget 메소드 실행 시 props 파라미터로 입력할 수 있는 항목은 대상 컴포넌트에서 사용할 수 있는 모든 속성값입니다. 예제에서는 크기와 위치만 변경해봅니다.

44.2.1 예제

Button00, Button01 컴포넌트를 대상으로 애니메이션 효과를 적용합니다. [play] 버튼 클릭 시 애니메이션 효과가 실행되고 [pause] 버튼을 클릭하면 애니메이션 효과가 잠시 멈춥니다. [stop] 버튼을 클릭하면 초기 상태로 돌아옵니다.



44.2.2 예제에서 사용한 핵심 기능

loop

애니메이션 효과의 반복 실행 여부를 지정합니다. 위치를 변경하도록 지정한 경우에는 direction 속성값을 "alternate"으로 지정해야 반복되는 효과를 볼 수 있습니다.

44.2.3 예제 구현 방법

① 화면 구성하기

Button 컴포넌트를 화면에 배치합니다. 상단에 배치하는 Button 컴포넌트의 text 속성값은 "play", "pause", "stop"으로 지정합니다.

② Animation 오브젝트 생성하기

Form 오브젝트의 onload 이벤트 함수 내에서 Animation 오브젝트를 생성하고 addTarget 메소드를 실행해 애니메이션 효과를 적용할 대상 컴포넌트를 지정합니다.

```
this.sample_animation_01 onload = function(obj:nexacro.Form,e:nexacro.LoadEventInfo)
{
    var aniObj = new nexacro.Animation("Ani00", this);
    this.addChild("Ani00", aniObj);

    this.Ani00.set_loop(true);
    this.Ani00.set_direction("alternate");

    this.Ani00.addTarget("AniItem00", this.Button00, "width:200, left:300");
    this.Ani00.addTarget("AniItem01", this.Button01, "width:50, left:200");
};
```

③ 애니메이션 효과 실행, 일시중지, 중지 기능 처리하기

상단 버튼 클릭 시 애니메이션 효과를 제어하는 기능을 처리하도록 이벤트 함수를 작성합니다.

```
this.btnPlay_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Ani00.play();
};

this.btnPause__onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Ani00.pause();
};

this.btnStop__onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Ani00.stop();
};
```

};

4 QuickView로 확인하기

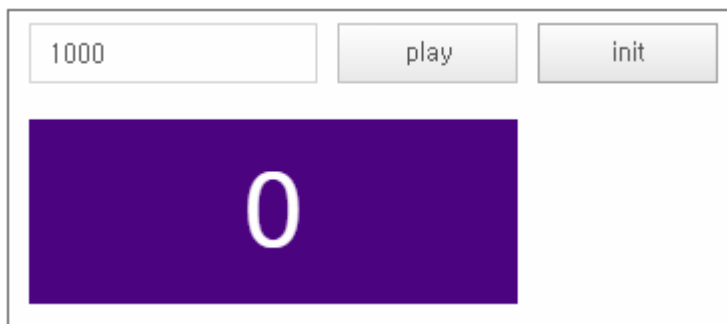
QuickView(Ctrl + F6)로 실행한 후 상단 버튼을 클릭해 애니메이션 효과가 적용되는 것을 확인합니다.

44.3 숫자 카운터 효과 만들기

Animation 오브젝트 사용 예에서 설명한 숫자 카운터 효과를 만들어봅니다.

44.3.1 예제

[play] 버튼 클릭 시 카운터가 동작하고 [init] 버튼을 클릭하면 초기 상태로 돌아갑니다. Edit 컴포넌트에 카운터 최대 값을 지정할 수 있습니다.



sample_animation_02.xfdl

44.3.2 예제에서 사용한 핵심 기능

easing

애니메이션 효과 처리 시 적용되는 Interpolation(선형보간법) 속성을 지정합니다. 예를 들어 1부터 100까지 속성값이 변경되는데 같은 속도로 카운터가 올라가는 것이 아니라 빠르기를 조정할 수 있습니다. default 값은 "easeoutelastic"인데 예제에서는 같은 속도로 처리될 수 있도록 "linear"로 지정했습니다.

duration

애니메이션 효과를 처리하는 시간을 지정합니다. default 값은 1000(1초)입니다. duration 속성값을 너무 짧게 지정하면 애니메이션 효과를 사용자가 느끼지 못할 수 있습니다.

onrun

애니메이션 효과를 처리하면서 발생하는 이벤트입니다. 일반적으로는 1초에 60번 발생하지만, 실행되는 환경에 따라 다를 수 있습니다. onupdate 이벤트와 비슷한데, onupdate 이벤트는 delay 속성값으로 지정된 시간에도 발생하지만 onrun 이벤트는 실제 애니메이션 효과가 동작하는 동안에만 발생합니다.

44.3.3 예제 구현 방법

① 화면 구성하기

Edit 컴포넌트와 Button 컴포넌트를 화면에 배치합니다. 상단에 배치하는 Button 컴포넌트의 text 속성값은 "play", "init"로 지정합니다. Static 컴포넌트를 배치하고 배경색(background)과 글자크기(font), 글자색(color) 속성을 지정합니다. 예제에서는 배경색은 "indigo", 글자색은 "white", 글자크기는 40으로 지정했습니다.

② Animation 오브젝트 생성하기

Form 오브젝트의 onload 이벤트 함수 내에서 Animation 오브젝트를 생성하고 addTarget 메소드를 실행해 애니메이션 효과를 적용할 대상 컴포넌트를 지정합니다. Animation 오브젝트의 easing 속성과 duration 속성값을 수정합니다. onrun 이벤트는 화면에 표시되는 숫자를 조정하기 위해 추가합니다.

```
this.sample_animation_02 onload = function(obj:nexacro.Form,e:nexacro.LoadEventInfo)
{
    var aniObj = new nexacro.Animation("Ani00", this);
    this.addChild("Ani00", aniObj);

    this.Ani00.set_easing("linear");
    this.Ani00.set_duration("5000");

    this.Ani00.addTarget("AniItem00", this.Static00, "text:100");
    this.Ani00.setEventHandler("onrun", this.Ani00_onrun, this);
};
```

③ onrun 이벤트 함수 작성하기

onrun 이벤트는 애니메이션 효과가 동작하는 동안 발생합니다. 숫자 관련 속성을 변경할때 발생하는 값이 정수가 아니기 때문에 onrun 이벤트 함수 내에서 값을 정수로 바꾸어주는 기능을 추가합니다.

```

this.Ani00_onrun = function(obj,e)
{
    this.Static00.set_text(nexacro.round(this.Static00.text));
};

```

4 버튼 클릭 시 이벤트 함수 작성하기

버튼 클릭 시 애니메이션 효과를 시작하고 초기값으로 바꾸어주는 이벤트 함수를 작성합니다. Edit 컴포넌트에 입력된 값이 있는 경우에는 AnimationItem 오브젝트에 설정된 props 값을 다시 설정하고 play 메소드를 실행합니다.

```

this.btnPlay_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    if(this.Edit00.value > 0)
    {
        this.Ani00.items.AniItem00.props = "text:"+this.Edit00.value;
    }
    this.Ani00.play();
};

this.btnInit_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Static00.set_text(0);
};

```

5 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 상단 버튼을 클릭해 애니메이션 효과가 적용되는 것을 확인합니다.

44.4 easing 효과 만들기

컴포넌트가 움직이는 효과를 만들때 easing 속성값을 "linear"으로 지정하면 처음부터 끝까지 같은 속도로 애니메이션 효과를 처리합니다. 하지만 상황에 따라 같은 속도가 유지되는 것은 자연스럽지 못할 수 있습니다. 예를 들어 자동차가 움직이는 효과를 만든다고 할때 실제 자동차는 천천히 출발해서 속도를 올려서 일정 속도에 도달한 후 다시 멈추기 위해 속도를 줄입니다. 사용자에게 자연스럽게 전달하려면 이런 속도의 변화를 애니메이션 효과에 담을 수 있어야 합니다.

44.4.1 예제

Button00, Button01, Button02, Button03 컴포넌트를 대상으로 애니메이션 효과를 적용합니다. [play] 버튼 클릭 시 애니메이션 효과가 실행되고 [pause] 버튼을 클릭하면 애니메이션 효과가 잠시 멈춥니다. [stop] 버튼을 클릭하면 초기 상태로 돌아갑니다. 하단의 Radio 버튼을 선택하면 easing 속성값과 Button 컴포넌트의 텍스트를 변경합니다.

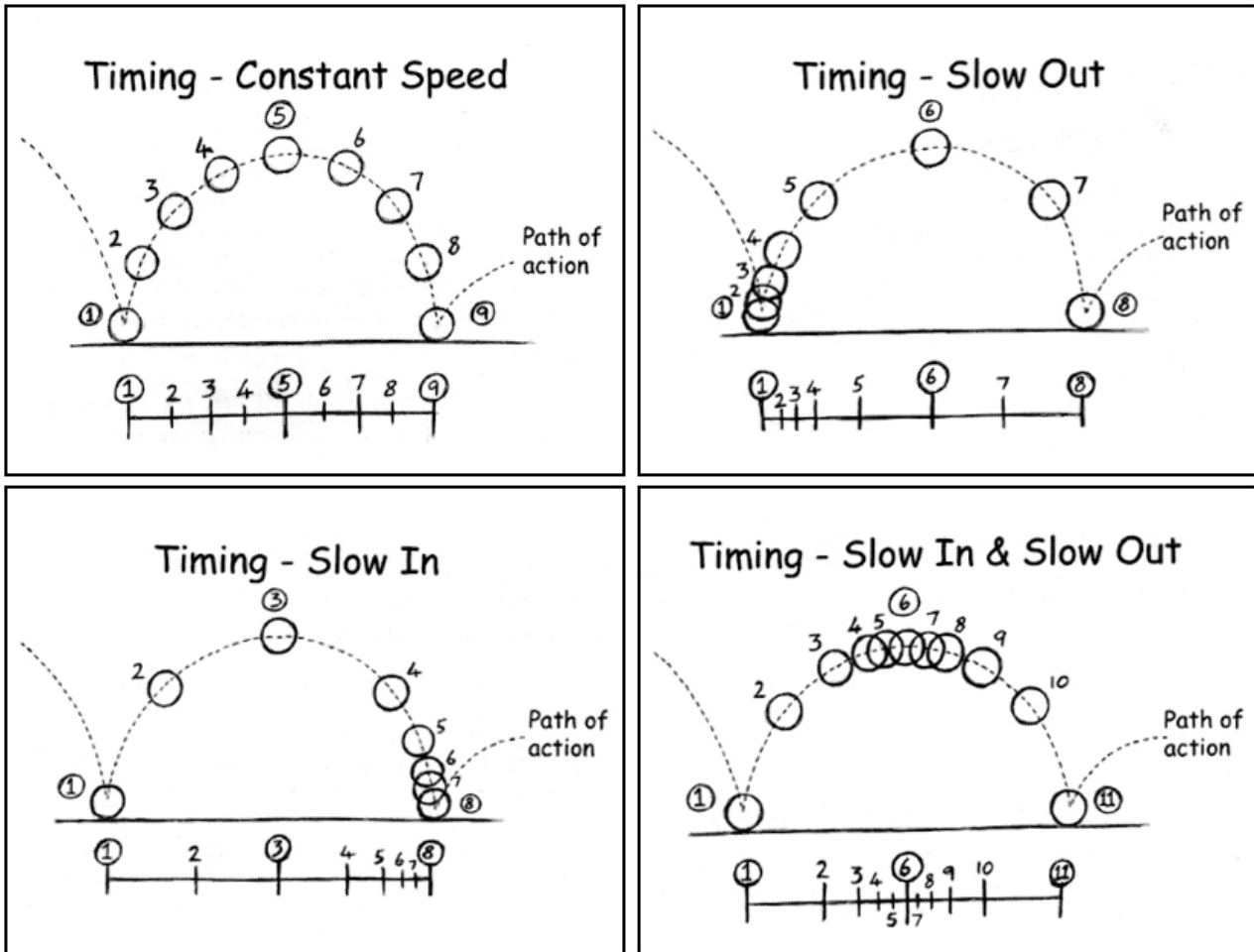


 sample_animation_03.xfdl

44.4.2 예제에서 사용한 핵심 기능

easing

애니메이션 효과가 처리되는 속도를 지정합니다. easing 속성값을 지정하는 값은 "linear"을 제외하고 3가지로 구분할 수 있습니다. 가장 앞 부분에는 "ease"라는 문자열이 붙습니다. 그 다음에는 "In", "Out", "InOut" 3가지 중 하나의 문자열이 붙습니다. 두 개의 문자열을 합치면 "ease in"이라는 문자열이 되는데, 이 표현은 전통적인 애니메이션 업계에서 사용하는 용어입니다. 물체의 움직임이 서서히 시작해서 점점 빨라지는 것을 의미합니다. "ease out"은 반대로 빠르게 시작해서 점점 속도를 줄이는 것이지요. "slow in", "slow out"이라고 표현하기도 합니다. "ease in out"은 마치 자동차처럼 점점 속도를 올려서 중간 지점에서 가장 빠른 속도를 내고 중간 지점을 지나면서 점점 느려집니다. 마지막에 붙는 문자열은 속도의 세세한 변화 시점을 정하는 것입니다.



easing 속성값으로 지정할 수 있는 파라미터에 대해서는 아래 링크에서 확인할 수 있습니다. 애플리케이션의 성격에 따라 적절한 값을 선택해서 사용합니다.

<https://easings.net/>

44.4.3 예제 구현 방법

① 화면 구성하기

Button 컴포넌트를 화면에 배치합니다. 상단에 배치하는 Button 컴포넌트의 text 속성값은 "play", "pause", "stop"으로 지정합니다. Button 컴포넌트의 애니메이션 효과가 시작하는 지점과 끝나는 지점에 Static 컴포넌트를 길게 배치하고 background 속성으로 배경색을 지정합니다. "easeInBack"이나 "easeInElastic"으로 easing 속성값을 지정하는 경우에는 해당 경계를 벗어나는 것을 확인할 수 있습니다.

② radion 컴포넌트 배치하고 데이터 설정하기

화면 하단에 radion 컴포넌트를 배치하고 innerdataset을 아래와 같이 지정합니다.

InnerDataset		
No	codecolumn	datacolumn
1	linear,easeInSine,easeOutSine,easeInoutSine	linear, easeInSine, easeOutS...
2	easeInOutSine,easeInOutQuad,easeInOutCubic,easeInOutQuart	easeInOutSine, easeInOutQ...
3	easeInOutSine,easeInOutQuint,easeInOutExpo,easeInOutCirc	easeInOutSine, easeInOutQ...
4	easeInOutSine,easeInOutBack,easeInOutElastic,easeInOutBounce	easeInOutSine, easeInOutB...

주의할 점은 codecolumn 항목에 들어가는 문자열은 구분자(,)와 각 속성값 사이에 공백문자가 들어가지 않도록 합니다. 아이템 변경 시 value값을 배열로 바꾸어 사용하는데 공백문자가 있으면 공백문자까지 배열값으로 인식되어질 수 있습니다.

```
linear,easeInSine,easeOutSine,easeInoutSine
easeInOutSine,easeInOutQuad,easeInOutCubic,easeInOutQuart
easeInOutSine,easeInOutQuint,easeInOutExpo,easeInOutCirc
easeInOutSine,easeInOutBack,easeInOutElastic,easeInOutBounce
```

3 Animation 오브젝트 생성하기

4개의 easing 속성값을 비교해야 하기 때문에 Animation 오브젝트를 4개 생성해서 사용합니다.

```
this.sample_animation_01_onload = function(obj:nexacro.Form,e:nexacro.LoadEventInfo)
{
    this.initAnimation("Ani00", "linear", this.Button00);
    this.initAnimation("Ani01", "easeInSine", this.Button01);
    this.initAnimation("Ani02", "easeOutSine", this.Button02);
    this.initAnimation("Ani03", "easeInOutSine", this.Button03);
};
```

Animation 오브젝트 id값과 easing 속성값, 버튼 오브젝트를 받아서 Animation 오브젝트의 상태를 지정합니다. 최초 실행 시에는 Animation 오브젝트를 생성하고 이후에는 easing 속성값과 Button 컴포넌트의 text 속성만 변경합니다.

```
this.initAnimation = function(sId, sEasing, objButton)
{
    if(!this.all[sId])
    {
        var aniObj = new nexacro.Animation(sId, this);
        this.addChild(sId, aniObj);
        this.all[sId].addTarget("AniItem00", objButton, "left:300");
        this.all[sId].set_direction("alternate");
    }
}
```

```

        this.all[sId].set_duration(3000);
        this.all[sId].set_direction("alternate");
        this.all[sId].set_loop(true);
    }
    this.all[sId].set_easing(sEasing);
    objButton.set_text(sEasing);
}

```

4 애니메이션 효과 실행, 일시중지, 중지 기능 처리하기

상단 버튼 클릭 시 애니메이션 효과를 제어하는 기능을 처리하도록 이벤트 함수를 작성합니다.

```

this.btnPlay_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Ani00.play();
};

this.btnPause__onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Ani00.pause();
};

this.btnStop__onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Ani00.stop();
};

```

5 Radion 컴포넌트 onitemchanged 이벤트 함수 작성하기

아이템을 변경했을때 변경된 상태를 Animation 오브젝트에 전달합니다. 선택된 아이템의 value값을 split 메소드를 사용해 배열로 변경하고 생성된 값을 initAnimation 함수에 전달합니다.

```

this.Radio00_onitemchanged = function(obj:nexacro.Radio,e:nexacro.ItemChangeEventInfo)
{
    var arr = e.postvalue.toString().split(",");
    this.initAnimation("Ani00", arr[0], this.Button00);
    this.initAnimation("Ani01", arr[1], this.Button01);
    this.initAnimation("Ani02", arr[2], this.Button02);
    this.initAnimation("Ani03", arr[3], this.Button03);
};

```



애니메이션이 동작하고 있는 중에는 변경된 상태가 반영되지 않습니다. stop 메소드를 실행한 상태에서만 변경된 속성값을 반영할 수 있습니다. 예제에서는 [stop] 버튼을 클릭해 애니메이션을 종료하고 다시 [play] 버튼을 클릭해야 변경된 easing 속성값이 반영됩니다.

6 QuickView로 확인하기

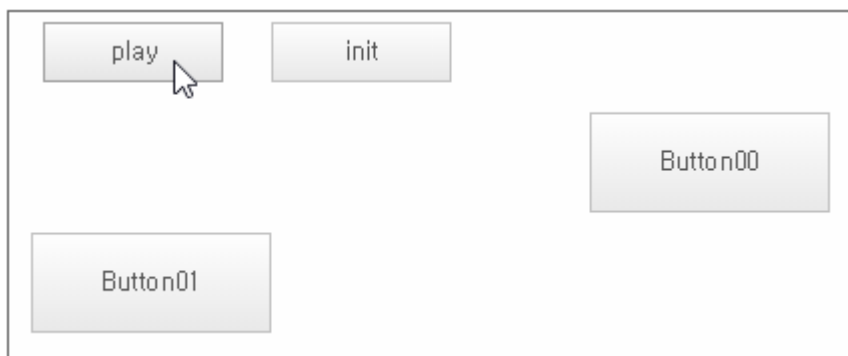
QuickView(Ctrl + F6)로 실행한 후 상단 버튼을 클릭해 애니메이션 효과가 적용되는 것을 확인합니다.

44.5 순서대로 애니메이션 처리하기

oncomplete 이벤트를 사용하면 애니메이션이 종료되는 시점을 체크할 수 있습니다. 두 가지 Animation 오브젝트를 생성하고 순서대로 처리하는 방식을 살펴봅니다.

44.5.1 예제

Button00, Button01 컴포넌트를 대상으로 애니메이션 효과를 적용합니다. [play] 버튼 클릭 시 Button00 컴포넌트에 적용된 애니메이션 효과가 실행되고 애니메이션 효과가 끝나면 Button01 컴포넌트에 적용된 애니메이션 효과가 실행됩니다.



sample_animation_04.xfdl

44.5.2 예제에서 사용한 핵심 기능

oncomplete

duration 속성값으로 지정된 시간이 지나면 발생하는 이벤트입니다. loop 속성값이 true인 경우에 loopcount 속성값을 지정하지 않으면 애니메이션이 멈추지 않기 때문에 oncomplete 이벤트가 발생하지 않습니다. 예제에서는 loopcount 속성값을 2로 지정해서 첫번째 애니메이션 효과가 두 번 동작하고 다음 애니메이션 효과가 동작합니다.

44.5.3 예제 구현 방법

① 화면 구성하기

Button 컴포넌트를 화면에 배치합니다. 상단에 배치하는 Button 컴포넌트의 text 속성값은 "play", "init"로 지정합니다.

② Animation 오브젝트 생성하기

Form 오브젝트의 onload 이벤트 함수 내에서 Animation 오브젝트를 생성하고 addTarget 메소드를 실행해 애니메이션 효과를 적용할 대상 컴포넌트를 지정합니다. 첫번째 Animation 오브젝트는 loop, loopcount 속성값을 지정하고 oncomplete 이벤트를 추가합니다.

```
this.sample_animation_01 onload = function(obj:nexacro.Form,e:nexacro.LoadEventInfo)
{
    var aniObj00 = new nexacro.Animation("Ani00", this);
    this.addChild("Ani00", aniObj00);
    this.Ani00.addTarget("AniItem00", this.Button00, "left:300");
    this.Ani00.setEventHandler("oncomplete", this.Ani00_oncomplete, this);
    this.Ani00.set_loop(true);
    this.Ani00.set_loopcount(2);

    var aniObj01 = new nexacro.Animation("Ani01", this);
    this.addChild("Ani01", aniObj01);
    this.Ani01.addTarget("AniItem00", this.Button01, "left:300");
};
```

③ 애니메이션 효과 실행, 초기화 처리하기

상단 버튼 클릭 시 애니메이션 효과를 제어하는 기능을 처리하도록 이벤트 함수를 작성합니다.

```

this.btnPlay_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Ani00.play();
};

this.btnInit_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Button00.set_left(20);
    this.Button01.set_left(20);
};

```

④ oncomplete 이벤트 함수 작성하기

첫번째 애니메이션 효과가 끝났을때 두번째 애니메이션 play 메소드를 실행합니다.

```

this.Ani00_oncomplete = function(obj:nexacro.Animation,e:nexacro.AnimationEventInfo)
{
    this.Ani01.play();
};

```

⑤ QuickView로 확인하기

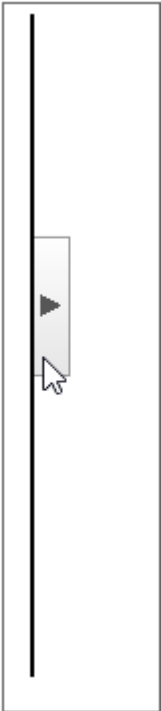
QuickView(Ctrl + F6)로 실행한 후 상단 버튼을 클릭해 애니메이션 효과가 적용되는 것을 확인합니다.

44.6 펼쳐지는 메뉴 효과 만들기

컨텐츠 화면을 넓게 사용하는 경우 메뉴 영역을 필요할때만 펼치는 경우가 있습니다. 애니메이션 효과를 적용해 서서히 펼쳐지고 접히는 메뉴 영역을 만들어봅니다.

44.6.1 예제

버튼을 클릭하면 메뉴가 펼쳐지고 다시 버튼을 클릭하면 메뉴가 서서히 사라집니다.



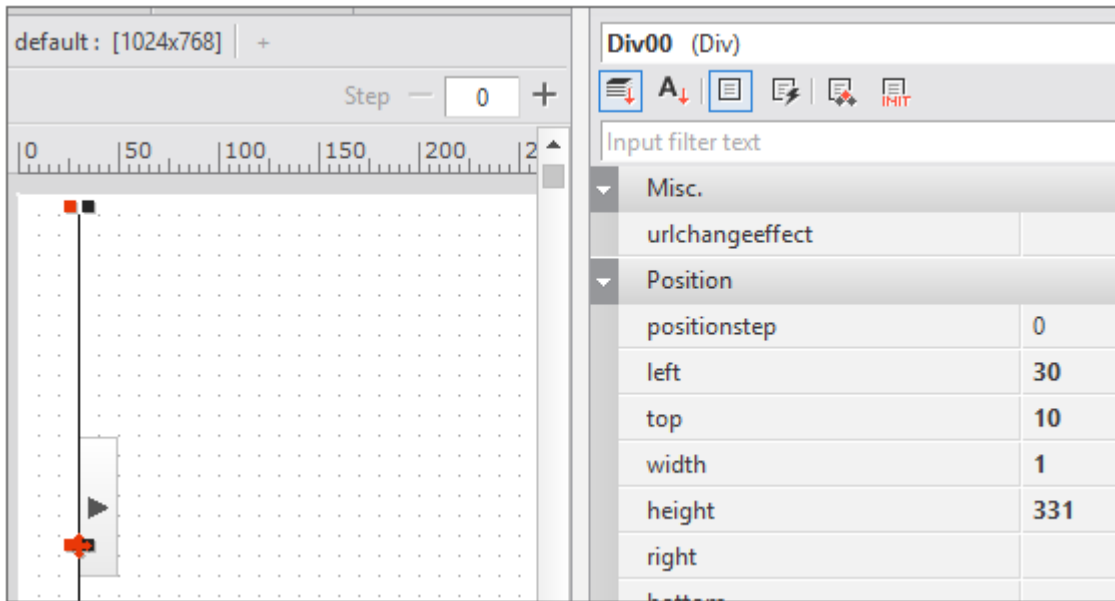
44.6.2 예제에서 사용한 핵심 기능

oncomplete

duration 속성값으로 지정된 시간이 지나면 발생하는 이벤트입니다. loop 속성값이 true인 경우에 loopcount 속성값을 지정하지 않으면 애니메이션이 멈추지 않기 때문에 oncomplete 이벤트가 발생하지 않습니다. 예제에서는 loopcount 속성값을 2로 지정해서 첫번째 애니메이션 효과가 두 번 동작하고 다음 애니메이션 효과가 동작합니다.

44.6.3 예제 구현 방법

- ① 화면에 Div 컴포넌트와 Button 컴포넌트를 배치합니다. Div 컴포넌트는 width 속성값을 1로 지정해서 접혀진 것처럼 보이도록 합니다.



- ② Form 오브젝트의 onload 이벤트를 함수를 아래와 같이 추가합니다.

Animation 오브젝트를 생성하고 Target과 이벤트리스너를 등록합니다. easing 속성값은 완만한 변화를 보여주는 "easeInSine"으로 등록합니다.

```
this.sample_animation_01 onload = function(obj:nexacro.Form,e:nexacro.LoadEventInfo)
{
    var aniObj00 = new nexacro.Animation("Ani00", this);
    this.addChild("Ani00", aniObj00);
    this.Ani00.addTarget("AniItem00", this.btnPlay, "width:20, left:329");
    this.Ani00.addTarget("AniItem01", this.Div00, "width:300, left:30");
    this.Ani00.setEventHandler("oncomplete", this.Ani00_oncomplete, this);
    this.Ani00.set_easing("easeInSine");

    var aniObj01 = new nexacro.Animation("Ani01", this);
    this.addChild("Ani01", aniObj01);
    this.Ani01.addTarget("AniItem00", this.btnPlay, "width:20, left:30");
    this.Ani01.addTarget("AniItem01", this.Div00, "width:1, left:30");
    this.Ani01.setEventHandler("oncomplete", this.Ani01_oncomplete, this);
    this.Ani01.set_easing("easeInSine");
};
```

- ③ 애니메이션 효과가 끝났을때 버튼 텍스트를 수정해주도록 oncomplete 이벤트 함수를 작성합니다.

```
this.Ani00_oncomplete = function(obj:nexacro.Animation,e:nexacro.AnimationEventInfo)
{
```



```

    this.btnPlay.set_text("◀");
};

this.Ani01_oncomplete = function(obj:nexacro.Animation,e:nexacro.AnimationEventInfo)
{
    this.btnPlay.set_text("▶");
};

```

- ④ Button 컴포넌트 클릭 시 애니메이션 효과가 실행되도록 onclick 이벤트 함수를 작성합니다.

```

this.btnPlay onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    if( obj.text == "▶")
    {
        this.Ani00.play();
    } else {
        this.Ani01.play();
    }
};

```

- ⑤ QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 애니메이션 효과가 적용되는 것을 확인합니다.

45.

AnimationTimeline

45.1 AnimationTimeline

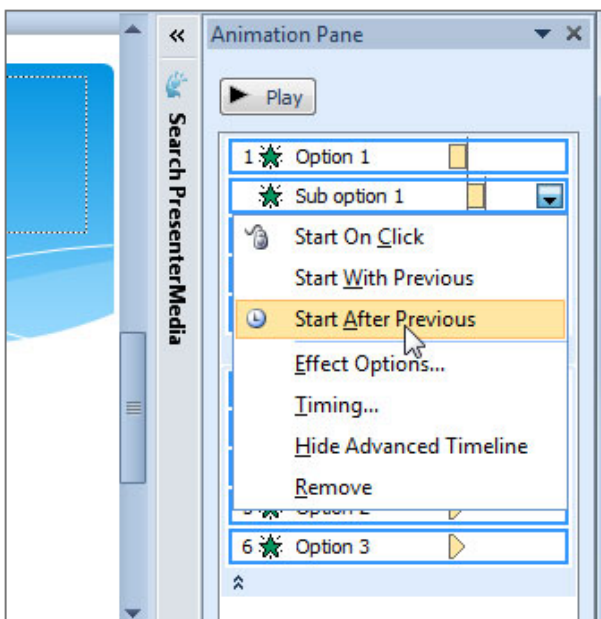
Animation 오브젝트는 애니메이션에서 하나의 컷을 만드는 것이라면 AnimationTimeline 오브젝트는 만들어진 컷을 연결하는 역할을 담당합니다. Animation 오브젝트가 많지 않다면 oncomplete 이벤트를 사용해 애니메이션의 동작 시점을 지정할 수 있지만, Animation 오브젝트가 많아진다면 이벤트 함수 내에서 관리하는것이 불편할 수 있습니다. AnimationTimeline 오브젝트를 사용하면 애니메이션 효과의 적용 시점을 좀 더 쉽게 관리할 수 있습니다.



AnimationTimeline 오브젝트는 17.0.0.700 버전부터 지원합니다.

45.1.1 사용 예

프리젠테이션 도구에서 애니메이션 적용 시 "이전 효과 다음에 시작(Start After Previous)"를 옵션으로 선택하는 것도 타임라인을 관리하는 하나의 방법입니다.



45.1.2 AnimationTimeline 만들기

① AnimationTimeline 오브젝트 생성하기

AnimationTimeline 오브젝트는 넥사크로 스튜디오 툴바에 표시되지 않습니다. 스크립트에서 새로운 오브젝트를 생성해주어야 합니다. form 오브젝트의 onload 이벤트 함수 내에서 AnimationTimeline 오브젝트를 추가한다면 다음과 같이 이벤트 함수를 작성합니다.

```
this.form_onload = function(obj:nexacro.Form,e:nexacro.LoadEventInfo)
{
    var anitimeObj = new nexacro.AnimationTimeline("Anitime00", this);
    this.addChild("Anitime00", anitimeObj);
}
```

화면에 보여지는 것이 아니기때문에 addChild 메소드 호출 후 show 메소드를 호출하지 않습니다.

② Animation 오브젝트 추가하기

애니메이션 효과를 적용할 Animation 오브젝트를 추가해주어야 합니다. addTarget 메소드를 사용해 추가합니다.

```
this.Anitime00.addTarget("AniTimeItem00", aniObj00);
```

③ 애니메이션 효과 실행하기

AnimationTimeline 오브젝트를 사용하는 경우에는 개별 Animation 오브젝트의 play 메소드를 사용하지 않고 AnimationTimeline 오브젝트의 play 메소드를 사용합니다.

```
this.Anitime00.play();
```

45.2 순서대로 애니메이션 처리하기

Animation 오브젝트에서는 oncomplete 이벤트를 사용했는데, 이번에는 AnimationTimeline 오브젝트를 사용해 애니메이션을 순서대로 처리합니다.

45.2.1 예제

Button00, Button01 컴포넌트를 대상으로 애니메이션 효과를 적용합니다. [play] 버튼 클릭 시 Button00 컴포넌트에 적용된 애니메이션 효과가 실행되고 애니메이션 효과가 끝나면 Button01 컴포넌트에 적용된 애니메이션 효과가 실행됩니다.



45.2.2 예제에서 사용한 핵심 기능

addTarget

Animation 오브젝트의 메소드 이름과 같지만, AnimationTimeline 오브젝트의 addTarget 메소드는 대상 Animation 오브젝트만 지정하고 props 속성값을 지정하지 않습니다. 세번째 파라미터로 nOffset를 지정할 수 있는데, 지정하지 않은 경우에는 0으로 처리합니다.

45.2.3 예제 구현 방법

① 화면 구성하기

Button 컴포넌트를 화면에 배치합니다. 상단에 배치하는 Button 컴포넌트의 text 속성값은 "play", "init"로 지정합니다.

② AnimationTimeline, Animation 오브젝트 생성하기

Form 오브젝트의 onload 이벤트 함수 내에서 AnimationTimeline, Animation 오브젝트를 생성하고 addTarget 메소드를 실행해 두 개의 Animation 오브젝트를 대상으로 지정합니다.

```

this.sample_animationtimeline_01_onload = function(obj:nexacro.Form,e:nexacro.
LoadEventInfo)
{
    var anitimeObj = new nexacro.AnimationTimeline("Anitime00", this);
    this.addChild("Anitime00", anitimeObj);

    var aniObj00 = new nexacro.Animation("Ani00", this);
    this.addChild("Ani00", aniObj00);
    this.Ani00.addTarget("AniItem00", this.Button00, "left:300");

    var aniObj01 = new nexacro.Animation("Ani01", this);
    this.addChild("Ani01", aniObj01);
    this.Ani01.addTarget("AniItem00", this.Button01, "left:300");

    this.Anitime00.addTarget("AniTimeItem00", aniObj00);
    this.Anitime00.addTarget("AniTimeItem01", aniObj01);
};

```

③ 애니메이션 효과 실행, 초기화 처리하기

상단 버튼 클릭 시 애니메이션 효과를 제어하는 기능을 처리하도록 이벤트 함수를 작성합니다. Animation 오브젝트의 play 메소드가 아니라 AnimationTimeline 오브젝트의 play 메소드를 실행합니다.

```

this.btnPlay_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Anitime00.play();
};

this.btnInit_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Button00.set_left(20);
    this.Button01.set_left(20);
};

```

④ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 상단 버튼을 클릭해 애니메이션 효과가 적용되는 것을 확인합니다.

파트 VI.

Data Objects

46.

Dataset

46.1 Dataset 소개

Dataset은 데이터베이스 자원을 효율적으로 활용하고자 도입된 개념입니다. 한정적인 데이터베이스 자원에 다수 사용자의 지속적인 연결로 인해 발생하는 오버헤드는 전체 응용 프로그램의 성능을 저하하는 주된 요인이었습니다. 이를 해결하고자 데이터베이스에 대한 지속적 연결없이 데이터 작업이 가능한 비연결지향 방식의 Dataset을 도입하게 되었습니다.

Dataset은 데이터를 다루는 기본 단위로 넥사크로플랫폼 내부적으로 데이터를 관리하거나 서버와 통신할 때 주고받는 데이터 형식으로 사용합니다. 데이터를 추가, 수정, 삭제하거나 연산하는 메소드를 제공하며 다른 Dataset과 병합, 복사하는 등의 기능을 제공합니다.

Dataset은 비연결형 방식으로 데이터베이스에 대한 부하를 줄여주고 시스템 가용성을 높여 더 많은 응용 프로그램 서비스를 제공할 수 있게 해줍니다. 또한, 코드량을 줄여주고 데이터 유지 관리를 편하게 해준다는 장점이 있습니다. 그러나 원본 데이터를 로컬의 Dataset으로 복사하여 사용하므로 대량의 데이터 작업에서는 클라이언트측 메모리 사용률이 높아져 응용 프로그램에서의 적절한 데이터 관리가 필요합니다.

일반적으로 Dataset은 독자적으로 사용하지 않고 데이터를 시각적으로 표현하는 컴포넌트에 바인딩하는 형태로 사용합니다. Grid, Combo, ListBox 등의 컴포넌트는 화면에 보여주기 위한 형태만 가지고 있을 뿐 실제로 의미있는 데이터는 Dataset이 갖고 있습니다.

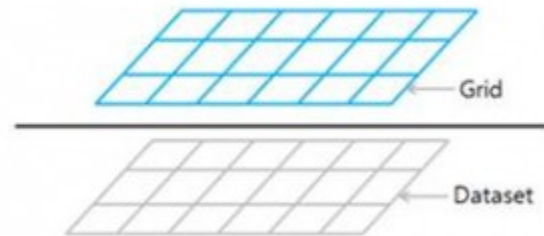
46.2 데이터 바인딩(Data Binding)

데이터 바인딩은 데이터 정보를 담고 있는 Dataset 오브젝트와 시각적으로 표현되는 컴포넌트 간에 데이터와 정보를 연결하고 상호 동기화하는 기법입니다. 바인딩 기법은 그 사용 방법에 따라 다음의 네 가지로 정도로 나뉘볼 수 있습니다.

- Full Binding

Grid는 그 형태가 Dataset과 논리적으로 같은 컴포넌트입니다. 따라서 Grid는 Dataset과 완전히 매핑되는데 이런 식으로 Dataset과 컴포넌트가 1대1로 완전히 매핑될 수 있는 바인딩을 Full binding이라고 합니다. 여러 컴포넌트 중 Grid만이 Full binding이 가능합니다.

EmployeeID	FullName	Department
AA-001	ShinSoo Choo	Sales
BB-001	Yuna Kim	IT
CC-001	Sumi Jo	Education
DD-001	JS Park	Marketing

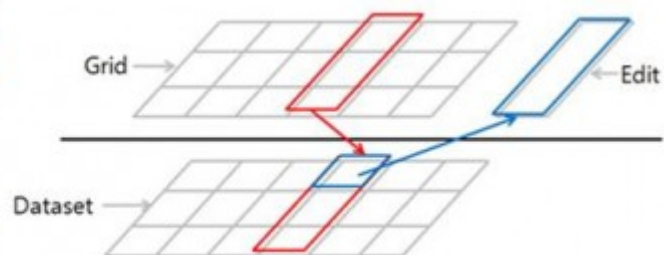


- Master & Detail Binding

Grid와 단일 데이터를 표시하는 컴포넌트 사이의 binding입니다. 아래 그림처럼 Grid의 레코드를 선택했을 때 Edit 컴포넌트에 선택된 레코드의 특정 Column을 입/출력하는 경우가 이에 해당합니다.

EmployeeID	FullName	Department
AA-001	ShinSoo Choo	Sales
BB-001	Yuna Kim	IT
CC-001	Sumi Jo	Education
DD-001	JS Park	Marketing

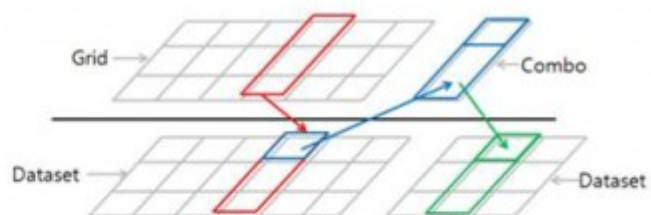
ShinSoo Choo



- Lookup Binding

단일 데이터를 입/출력하는 컴포넌트가 내부에 또 다른 Dataset을 가지는 형태의 바인딩입니다. 아래 그림처럼 Grid의 레코드를 선택했을 때 Combo 컴포넌트에 선택된 레코드의 특정 Column을 연결하고 Combo에 바인딩되어 있는 Dataset에서 그에 해당하는 값을 찾아 Combo에 출력하는 경우가 이에 해당합니다.

EmployeeID	FullName	Department
AA-001	ShinSoo Choo	Sales
BB-001	Yuna Kim	IT
CC-001	Sumi Jo	Education
DD-001	JS Park	Marketing



Sales

Sales

IT

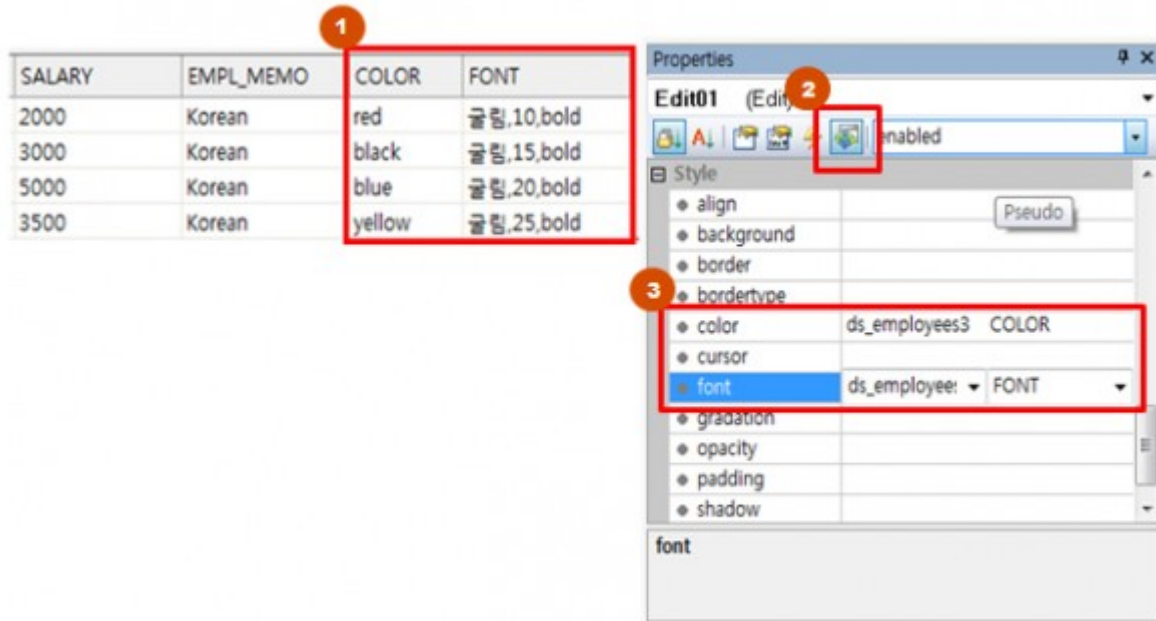
Education

Marketing

HR

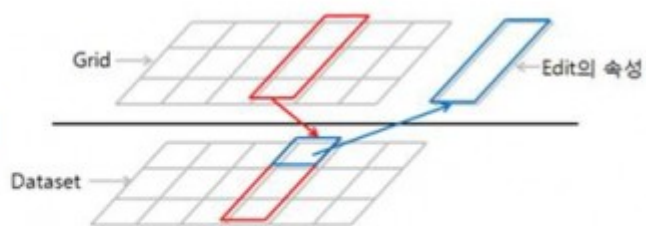
- Presentation Binding

컴포넌트에 출력하는 데이터에 스타일을 주는 용도의 바인딩 기법입니다. 아래 그림의 왼쪽과 같이 Dataset에 컴포넌트의 스타일 속성값을 입력해 놓고 이를 컴포넌트의 스타일 속성에 바인딩하는 경우가 이에 해당합니다.



EmployeeID	FullName	Department
AA-001	ShinSoo Choo	Sales
BB-001	Yuna Kim	IT
CC-001	Sumi Jo	Education
DD-001	JS Park	Marketing

Sumi Jo

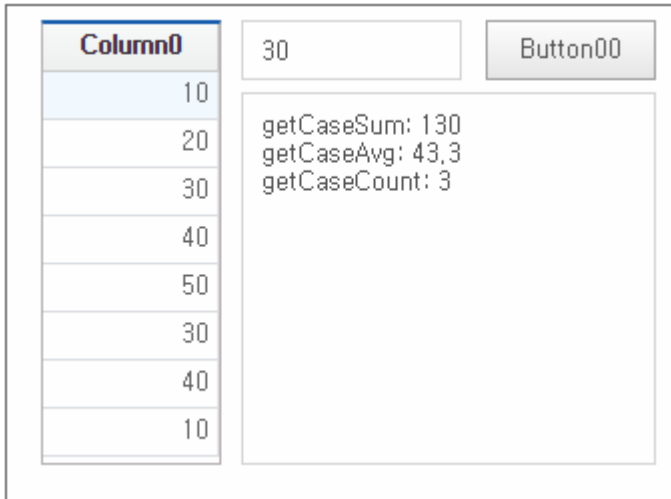


46.3 조건에 따라 데이터 평균, 합계 계산하기

getSum 메소드는 지정된 만큼의 합계를 구하는 것이라면 getCaseSum 메소드는 조건을 추가할 수 있습니다. 특정 조건에 해당하는 데이터만 계산하는 것입니다.

46.3.1 예제

Edit 컴포넌트에 숫자를 입력하고 Button 컴포넌트를 클릭하면 해당 숫자보다 큰 값의 합계, 평균, 개수를 표시합니다.



The screenshot shows a UI component with a table on the left and a summary box on the right. The table has a header 'Column0' and contains the following values: 10, 20, 30, 40, 50, 30, 40, 10. The summary box contains the following text: '30', 'Button00', 'getCaseSum: 130', 'getCaseAvg: 43.3', 'getCaseCount: 3'.

 [sample_dataset_01.xfdl](#)

46.3.2 예제에서 사용한 핵심 기능

getCaseSum

getSum 메소드와 지정하는 파라미터는 비슷합니다. getCase~로 시작하는 메소드는 첫번째 파라미터로 조건표현식을 추가할 수 있는 점이 다른 점입니다.

46.3.3 예제 구현 방법

① 화면 구성하기

Grid 컴포넌트를 배치합니다. Dataset 오브젝트를 생성하고 아래와 같이 데이터를 설정합니다. 생성된 Dataset 오브젝트를 Grid 컴포넌트에 바인딩합니다. Grid 컴포넌트는 데이터를 표시하는 도구로만 사용합니다. 데이터의 합계나 평균을 구하기 위해 type 속성값은 "INT"와 같은 숫자 형식으로 지정합니다.

Columns			
No	id	type	size
1	Column0	INT	256
Rows			
No	Column0		
1	10		
2	20		
3	30		
4	40		
5	50		
6	30		
7	40		
8	10		

조건식에 들어갈 값을 입력할 Edit 컴포넌트와 Button 컴포넌트, 결과를 표시할 TextArea 컴포넌트를 배치합니다.

② onclick 이벤트 함수 작성하기

Button 컴포넌트 클릭 시 연산 결과값을 TextArea 컴포넌트에 표시합니다. `getCaseAvg` 메소드는 소수점 처리를 위해 `nexacro.round` 메소드를 추가로 실행합니다.

```

this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var strCmpExpr = "Column0 > "+this.Edit00.value;
    var casesum = this.Dataset00.getCaseSum(strCmpExpr, "Column0");
    var caseavg = nexacro.round(this.Dataset00.getCaseAvg(strCmpExpr, "Column0"), 1);
    var casecount = this.Dataset00.getCaseCount(strCmpExpr, "Column0");
    this.TextArea00.set_value("getCaseSum: "+casesum+"\n"
        +"getCaseAvg: "+caseavg+"\n"
        +"getCaseCount: "+casecount);
};

```



컬럼 type 속성값이 "String"인 경우에는 연산을 처리하기 전에 숫자 형태로 바꾸어주어야 합니다. 위 예제에서 Column0 컬럼의 type 속성값이 "String"인 경우에는 `getCaseSum` 구문은 아래와 같이 변경할 수 있습니다.

```
var casesum = this.Dataset00.getCaseSum(strCmpExpr, "parseInt(Column0)");
```

3 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 데이터 범위 내 숫자를 입력하고 버튼 클릭 시 결과값을 확인합니다.

46.4 컬럼 정보 확인하고 type 속성값 변경하기

컬럼 정보를 확인하고 원하는 컬럼의 속성값을 변경할 수 있습니다. 예제에서는 type 속성값을 변경하는 방법을 보여줍니다.

46.4.1 예제

[getColumnInfo] 버튼을 클릭하면 Dataset 오브젝트의 컬럼 정보를 TextArea 컴포넌트에 표시합니다. 변경하고자 하는 컬럼 인덱스를 선택하고 type 속성값을 선택한 후 [set_type] 버튼을 클릭하면 해당 컬럼 type 속성값을 변경합니다. [getColumnInfo] 버튼을 클릭하면 변경된 정보를 확인할 수 있습니다.

getColumnInfo

2

INT

set_type

ID = Column0 SIZE = 256 type = STRING
ID = Column1 SIZE = 256 type = STRING
ID = Column2 SIZE = 256 type = INT
ID = Column3 SIZE = 256 type = STRING
ID = Column4 SIZE = 256 type = STRING
ID = Column5 SIZE = 256 type = STRING



sample_dataset_03.xfdl

46.4.2 예제에서 사용한 핵심 기능

getColumnInfo

ColumnInfo 오브젝트를 반환합니다. name, prop, size, sumtext, type 속성값을 확인할 수 있습니다. name 속성값은 readonly이므로 변경할 수 없습니다.

46.4.3 예제 구현 방법

1 화면 구성하기

Dataset 오브젝트를 생성하고 아래와 같이 데이터를 설정합니다. 컬럼 속성 정보만 확인하는 용도이므로 데이터를 추가하지는 않습니다. Button, Spin, Combo, TextArea 컴포넌트를 적절히 배치합니다.

Columns					
No	id	type	size	prop	sumtext
1	Column0	STRING	256		
2	Column1	STRING	256		
3	Column2	STRING	256		
4	Column3	STRING	256		
5	Column4	STRING	256		
6	Column5	STRING	256		
Rows					
No	Column0	Column1	Column2	Column3	
There are no items to show.					

Combo 컴포넌트의 innerdataset 속성값으로는 type 속성값에 사용할 수 있는 목록을 입력합니다.

No	codecolumn	datacolumn
1	STRING	STRING
2	INT	INT
3	FLOAT	FLOAT
4	BIGDECIMAL	BIGDECIMAL
5	DATE	DATE
6	TIME	TIME
7	DATETIME	DATETIME
8	BLOB	BLOB

② onload 이벤트 함수 작성하기

Spin 컴포넌트에서는 변경할 컬럼 인덱스를 지정합니다. max 속성값을 onload 이벤트 함수에서 지정합니다.

```
this.sample_dataset_03 onload = function(obj:nexacro.Form,e:nexacro.LoadEventInfo)
{
    this.Spin00.set_max(this.Dataset00.getColCount()-1);
};
```

③ onclick 이벤트 함수 작성하기

[getColumnInfo] 버튼 클릭 시에는 Dataset 오브젝트의 컬럼 수만큼 반복하면서 컬럼 정보를 확인합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var strText = new String();
    for (var i = 0; i < this.Dataset00.getColCount(); i++)
    {
        var objColInfo = this.Dataset00.getColumnInfo(i);
        strText += "ID = " + objColInfo.name;
        strText += " SIZE = " + objColInfo.size;
        strText += " type = " + objColInfo.type;
        strText += "\n";
    }
    this.TextArea00.set_value(strText);
};
```

[set_type] 버튼 클릭 시에는 지정된 컬럼 인덱스의 type 속성값을 변경합니다.

```
this.Button01 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var objColInfo = this.Dataset00.getColumnInfo(this.Spin00.value);
    objColInfo.set_type(this.Combo00.value);
};
```

④ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 컬럼 인덱스를 지정하고 type 속성 변경 후 결과값을 확인합니다.

46.5 삭제된 Row 정보 확인하기

Dataset 오브젝트에서 삭제된 정보는 바인딩된 컴포넌트(Grid 컴포넌트 또는 다른 컴포넌트)에서 확인할 수 없습니다. 하지만 Dataset 오브젝트에서는 삭제된 정보를 관리하고 있으며 특정 메소드를 통해 확인할 수 있습니다.

46.5.1 예제

[delete] 버튼을 클릭하면 Row 정보를 삭제합니다. [add] 버튼을 클릭하면 Row를 추가할 수 있습니다. 각 셀을 선택해서 내용을 수정할 수 있습니다. 이렇게 추가, 삭제, 수정된 정보는 [execute] 버튼을 클릭하면 아래에 있는 Grid 컴포넌트에 표시합니다.

Column0	Column1
CCC	4444
DDD	444
EEE	555
GGG	33333

delete

add

execute

COL_TYPE	Column0	Column1
4	CCC	333
2	GGG	33333
8	AAA	111
8	BBB	222

46.5.2 예제에서 사용한 핵심 기능

getRowType

지정된 Row에 해당하는 타입을 반환합니다. 삭제된 Row는 확인할 수 없습니다. 예제에서는 추가(2), 변경(4)에 해당하는 정보를 확인하고 처리합니다.

getOrgColumn

지정된 Row, Column에 해당하는 초기값을 반환합니다. 값이 변경된 경우 원래의 값을 확인할 수 있습니다.

getDeletedRowCount

삭제된 Row 갯수를 확인할 수 있습니다. 삭제된 Row 정보는 배열 형태로 처리됩니다. getDeletedRowset 메소드를 사용해서 배열 정보를 확인할 수 있습니다. getDeletedRowCount 메소드의 실행결과는 getDeletedRowset 메소드를 실행해서 얻은 배열의 length 속성값과 같습니다.

getDeletedColumn

삭제된 Row 정보에서 지정된 컬럼과 Row(인덱스)에 해당하는 값을 반환합니다.

46.5.3 예제 구현 방법

① 화면 구성하기

Grid 컴포넌트와 Button 컴포넌트를 예제와 같이 배치합니다. 그리고 2개의 Dataset 오브젝트를 생성합니다. 하나(Dataset00)는 아래와 같이 데이터를 지정하고 첫번째 Grid 컴포넌트에 바인딩합니다. 그리고 다른 Dataset(Dataset01)은 데이터 지정없이 두번째 Grid 컴포넌트에 바인딩합니다.

Columns				
No	id	type	size	prop
1	Column0	STRING	256	
2	Column1	STRING	256	
◀				
Rows				
No	Column0	Column1		
1	AAA	111		
2	BBB	222		
3	CCC	333		
4	DDD	444		
5	EEE	555		

② Grid Contents Editor 실행하기

데이터를 수정할 수 있도록 첫번째 Grid 컴포넌트의 edittype 속성값을 "normal"로 수정합니다.

③ onclick 이벤트 함수 작성하기

각 버튼 클릭 시 처리할 스크립트를 onclick 이벤트 함수 내에 작성합니다. [execute] 버튼 클릭 시 컬럼 정보를 추가하고 삭제되거나 수정, 추가된 데이터를 확인하기 위해 fn_orgDataset 함수를 실행합니다.

```

this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Dataset00.deleteRow(this.Dataset00.rowposition)
};

this.Button02 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Dataset00.addRow();
};

this.Button01 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Dataset01.addColumn("COL_TYPE", "STRING");
    for(var i=0;i<this.Dataset00.colcount;i++)
    {
        this.Dataset01.addColumn(this.Dataset00.getColID(i), "STRING");
    }
    this.fn_orgDataset();
};

```

```

this.fn_orgDataset = function()
{
    var sOrgCol = "";
    var sOrgVal = "";
    this.Dataset01.clearData();

    for (var i = 0; i < this.Dataset00.rowcount; i++)
    {
        var nRowType = this.Dataset00.getRowType(i);
        if (nRowType == Dataset.ROWTYPE_INSERT || nRowType == Dataset.ROWTYPE_UPDATE)
        {
            this.Dataset01.addRow();
        }
    }
}

```

```

        this.Dataset01.setColumn(this.Dataset01.rowposition, "COL_TYPE", nRowType);
        for (var j = 0; j < this.Dataset00.colcount; j++)
        {
            sOrgCol = this.Dataset00.getColID(j);
            sOrgVal = this.Dataset00.getOrgColumn(i, sOrgCol);

            this.Dataset01.setColumn(this.Dataset01.rowposition, sOrgCol, sOrgVal);
        }
    }
}

trace(this.Dataset00.getDeletedRowset().length);
for (var i = 0; i < this.Dataset00.getDeletedRowCount(); i++)
{
    this.Dataset01.addRow();
    this.Dataset01.setColumn(this.Dataset01.rowposition, "COL_TYPE", Dataset.ROWTYPE_
DELETE);
    for (var j = 0; j < this.Dataset00.colcount; j++)
    {
        sOrgCol = this.Dataset00.getColID(j);
        sOrgVal = this.Dataset00.getDeletedColumn(i, sOrgCol);
        this.Dataset01.setColumn(this.Dataset01.rowposition, sOrgCol, sOrgVal);
    }
}

this.Grid01.createFormat();
};

```



Row 타입을 표시하는 상수는 아래의 값으로 처리됩니다.

Dataset.ROWTYPE_INSERT: 2

Dataset.ROWTYPE_UPDATE: 4

Dataset.ROWTYPE_DELETE: 8

④ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 데이터를 삭제, 변경, 추가한 후 [execute] 버튼을 클릭해 Row 정보를 확인합니다.

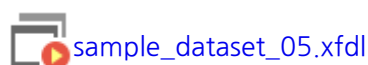
46.6 스크립트로 Dataset 생성하기

스크립트로 Dataset 오브젝트를 생성하고 Grid 컴포넌트에 바인딩할 수 있습니다. AppVariables로 Dataset 오브젝트를 추가하고 활용하는 방법도 같이 살펴봅니다.

46.6.1 예제

버튼을 클릭하면 AppVariables의 변수로 Dataset 오브젝트를 생성하고 Form 오브젝트의 자식으로 Dataset 오브젝트를 생성합니다. 생성된 Dataset 오브젝트는 Grid 컴포넌트에 바인딩되어서 표시됩니다.

The screenshot shows a user interface with two tables and a button. The first table on the left has columns 'COL0' and 'COL1' and contains the text 'global COL0' and 'global COL1'. The second table on the right also has columns 'COL0' and 'COL1' and contains the values '1', 'first', '2', and 'second'. To the right of the tables is a button labeled 'Button00'. Below the tables is a text area containing the text: 'global dataset rowcount : 1' and 'local dataset rowcount : 2'.



46.6.2 예제에서 사용한 핵심 기능

addVariable

AppVariables 영역에 변수를 추가합니다. Dataset 오브젝트를 생성하고 생성된 오브젝트를 변수로 지정할 수 있습니다. 설정된 변수는 `nexacro.getApplication().[변수명]` 으로 접근할 수 있습니다.

46.6.3 예제 구현 방법

① 화면 구성하기

Grid 컴포넌트와 Button 컴포넌트, TextArea 컴포넌트를 예제와 같이 배치합니다. Dataset 오브젝트는 스크립트에서 생성하고 처리하기 때문에 화면 구성 시 작업하지 않습니다.

2 onclick 이벤트 함수 작성하기

AppVariables에 추가되는 Dataset 오브젝트와 Form 오브젝트의 자식으로 추가되는 Dataset 오브젝트의 차이는 생성된 Dataset 오브젝트를 처리해주는 메소드(addVariable, addChild)만 다르고 나머지 과정은 같습니다.

```
this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var objApp = nexacro.getApplication();
    var objGdS = new Dataset;
    objApp.addVariable("gds_data", objGdS);
    objApp.gds_data.addColumn("COL0", "String");
    objApp.gds_data.addColumn("COL1", "String");
    objApp.gds_data.addRow();
    objApp.gds_data.setColumn(objApp.gds_data.rowposition, "COL0", "global COL0");
    objApp.gds_data.setColumn(objApp.gds_data.rowposition, "COL1", "global COL1");
    this.Grid00.set_binddataset("gds_data");
    this.Grid00.createFormat();

    var objLdS = new Dataset;
    objLdS.set_name("ds_data");
    this.addChild("ds_data", objLdS);
    this.ds_data.addColumn("COL0", "String");
    this.ds_data.addColumn("COL1", "String");
    this.ds_data.addRow();
    this.ds_data.setColumn(this.ds_data.rowposition, "COL0", "1");
    this.ds_data.setColumn(this.ds_data.rowposition, "COL1", "first");
    this.ds_data.addRow();
    this.ds_data.setColumn(this.ds_data.rowposition, "COL0", "2");
    this.ds_data.setColumn(this.ds_data.rowposition, "COL1", "second");
    this.Grid01.set_binddataset("ds_data");
    this.Grid01.createFormat();

    this.TextArea00.set_value('global dataset rowcount : ' + objApp.all["gds_data"].
rowcount+'\n'
        +'local dataset rowcount : ' + this.all["ds_data"].rowcount);
};
```

3 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭하고 Dataset이 정상적으로 생성되고 바인딩되는지 확인합니다.

46.7 변경된 컬럼정보 원복하기

Dataset 오브젝트의 값을 변경한 경우에는 변경된 상태 정보를 관리합니다. 때문에 필요한 경우 원본 데이터로 원복할 수 있습니다.

46.7.1 예제

Grid 컴포넌트에 바인딩된 Dataset 오브젝트의 값을 변경한 후 버튼을 클릭하면 원래 데이터로 원복합니다.

Column0	Column1	Column2	Column3	Column4	Column5
☐	1	12	3	5	4
☒	a	b	c	d	4
☐	data1	data2	data3	data4	1

Button00



46.7.2 예제에서 사용한 핵심 기능

getColCount

전체 컬럼 갯수를 반환합니다. 예제에서는 getRowType 메소드로 변경된 데이터가 있는지 확인하고 변경된 데이터가 있는 Row의 모든 컬럼의 getColumn 메소드 반환값과 getOrgColumn 메소드 반환값이 다른 경우 값을 원복합니다.

46.7.3 예제 구현 방법

① 화면 구성하기

Grid 컴포넌트와 Button 컴포넌트를 예제와 같이 배치합니다. Dataset 오브젝트를 아래와 같이 생성하고 Grid 컴포넌트에 바인딩합니다.

Columns					
No	id	type	size	prop	sumtext
1	Column0	STRING	256		
2	Column1	STRING	256		
3	Column2	STRING	256		
4	Column3	STRING	256		
5	Column4	STRING	256		
Rows					
No	Column0	Column1	Column2	Column3	Column4
1	0	1	2	3	5
2	0	a	b	c	d
3	0	data1	data2	data3	data4

② Grid Contents Editor 실행하기

Grid Contents Editor를 실행하고 첫번째 컬럼의 Body 셀을 선택해서 displaytype, edittype 속성값을 "checkboxcontrol", "checkbox"로 수정합니다. 나머지 컬럼의 Body 셀은 edittype 속성값을 "normal"로 수정합니다. 그리고 컬럼 하나를 추가하고 Body 셀의 text 속성값을 아래와 같이 수정합니다.

col5		coltype	none
Column5		expr	
		suppress	0
		suppressalign	first
		text	expr:dataset.getRowType(currow)

③ onclick 이벤트 함수 작성하기

값이 변경된 Row가 있는지 확인하고 변경된 값을 원복합니다. 삭제된 Row를 원복하는 코드는 포함하지 않았습니다.

```

this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var iRowCount = this.Dataset00.getRowCount();
    var iColCount = this.Dataset00.getColCount();

    for (var i = 0; i < iRowCount; i++)
    {
        if (this.Dataset00.getRowType(i) != "1")

```

```

    {
        for (var j = 0; j < iColCount; j++)
        {
            if (this.Dataset00.getColumn(i, j) != this.Dataset00.getOrgColumn(i, j))
            {
                this.Dataset00.setColumn(i, j, this.Dataset00.getOrgColumn(i, j));
            }
        }
    }
}

```

4 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭하고 값을 수정한 상태에서 버튼 클릭 시 원래의 값으로 원복되는지 확인합니다.

46.8 RowType 변경하기

Dataset 오브젝트의 RowType은 값이 변경되거나 삭제되는 등의 변경이 생겼을때 자동으로 업데이트됩니다. RowType에 따라 원본값도 같이 관리하게 됩니다. 이번 예제에서는 변경된 RowType을 강제로 수정해서 변경된 값을 확정하는 방법을 살펴봅니다.

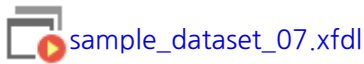
46.8.1 예제

Grid 컴포넌트에 바인딩된 Dataset 오브젝트의 값을 변경한 후 버튼을 클릭하면 RowType 값을 변경하고 변경된 상태를 TextArea 컴포넌트에 표시합니다. 아래 그림에서 원래의 값(1)을 12로 수정한 후 RowType을 ROWTYPE_NORMAL로 변경하면 원래의 값이 12로 변경되는 것을 확인할 수 있습니다.

Column0	Column1
12	a
2	b

Button00

index:0 getOrgColumn: 1
 index:0 getColumn: 12
 index:0 ROWTYPE_NORMAL_getOrgColumn: 12



46.8.2 예제에서 사용한 핵심 기능

updatecontrol

Dataset 오브젝트의 데이터가 변경됐을때 RowType을 자동으로 변경할지 설정하는 속성입니다. 기본값은 true로 설정되기 때문에 자동으로 값이 변경됩니다. setRowType 메소드를 사용해 값을 변경하고자 할 경우에는 먼저 updatecontrol 속성값을 false로 변경한 후에 setRowType 메소드를 사용해야 합니다.

46.8.3 예제 구현 방법

1 화면 구성하기

Grid 컴포넌트와 Button 컴포넌트, TextArea 컴포넌트를 예제와 같이 배치합니다. Dataset 오브젝트를 아래와 같이 생성하고 Grid 컴포넌트에 바인딩합니다.

Columns			
No	id	type	size
1	Column0	STRING	25
2	Column1	STRING	25
Rows			
No	Column0	Column1	
1	1	a	
2	2	b	

2 Grid Contents Editor 실행하기

Grid Contents Editor를 실행하고 첫번째 컬럼의 Body 셀을 선택해서 edittype 속성값을 "normal"로 수정합니다.

3 onclick 이벤트 함수 작성하기

값이 변경된 Row가 있는지 확인하고 변경된값(getColumn)과 원본값(getOrgColumn)을 출력합니다. 그리고 RowType을 변경한 후 원본값(ROWTYPE_NORMAL_getOrgColumn)을 출력합니다.


```

this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var rowcnt = this.Dataset00.getRowCount();
    for(var i=0;i<rowcnt;i++)
    {
        if(this.Dataset00.getRowType(i)==4)
        {
            this.fn_addvalue("index:"+i+" getOrgColumn: "+this.Dataset00.getOrgColumn(i,0
));
            this.fn_addvalue("index:"+i+" getColumn: "+this.Dataset00.getColumn(i,0));
            this.Dataset00.set_updatecontrol(false);
            this.Dataset00.setRowType(i,Dataset.ROWTYPE_NORMAL);
            this.Dataset00.set_updatecontrol(true);
            this.fn_addvalue("index:"+i+" ROWTYPE_NORMAL_getOrgColumn: "+this.Dataset00.
getOrgColumn(i,0));
        }
    }
}

this.fn_addvalue = function(str:String)
{
    if(this.TextArea00.value)
    {
        this.TextArea00.set_value(this.TextArea00.value+"\n"+str);
    } else {
        this.TextArea00.set_value(str);
    }
}

```

4 QuickView로 확인하기

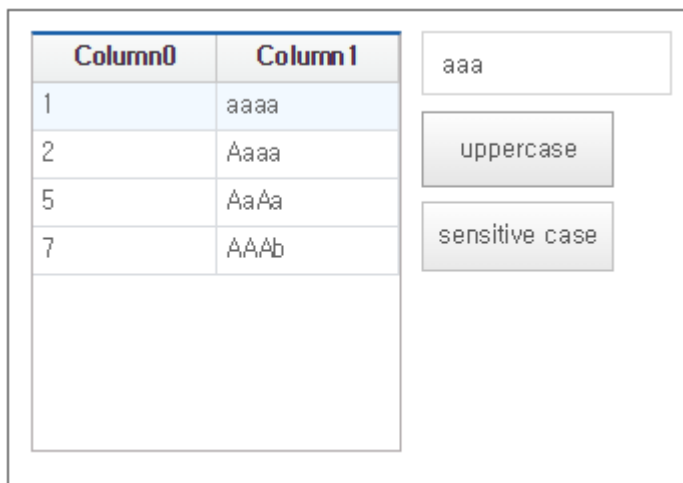
QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭하고 값을 수정한 상태에서 버튼 클릭 시 출력되는 값을 확인합니다.

46.9 대소문자 구분 없이 필터링하기

filter 메소드를 사용할때 영문 텍스트로 필터링하는 경우 대소문자 구분없이 처리하는 방식을 살펴봅니다.

46.9.1 예제

입력된 텍스트로 필터링을 처리합니다. [uppercase] 버튼 클릭 시에는 대소문자 구분없이 필터링하고, [sensitive case] 버튼 클릭 시에는 입력된 영문 텍스트 그대로 필터링합니다.



 sample_dataset_08.xfdl

46.9.2 예제에서 사용한 핵심 기능

filter

조건에 따라 필터링을 처리합니다. 조건이 없는 경우(빈문자열)는 조건이 해제되어 원본 데이터를 표시합니다.

46.9.3 예제 구현 방법

① 화면 구성하기

Grid 컴포넌트와 Button 컴포넌트, Edit 컴포넌트를 예제와 같이 배치합니다. Dataset 오브젝트를 아래와 같이 생성하고 Grid 컴포넌트에 바인딩합니다.

Columns			
No	id	type	size
1	Column0	STRING	255
2	Column1	STRING	255

Rows			
No	Column0	Column1	
1	1	aaaa	
2	2	Aaaa	
3	3	aabb	
4	4	aaBB	
5	5	AaAa	
6	6	AABB	
7	7	AAAb	

2 onclick 이벤트 함수 작성하기

대소문자 구분 없이 처리하기 위해서는 비교할 문자열 모두를 대문자 또는 소문자로 맞추어주어야 합니다. 예제에서는 toUpperCase를 사용해 Dataset 오브젝트 데이터와 Edit 컴포넌트에 입력된 텍스트를 모두 대문자로 변환해 비교합니다.

```

this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Dataset00.filter();
    var sFilter = "Column1.toString().toUpperCase().indexOf(' " + this.Edit00.value.
toUpperCase() + "') >= 0";
    this.Dataset00.filter(sFilter);
}

this.Button01 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Dataset00.filter();
    var sFilter = "Column1.toString().indexOf(' " + this.Edit00.value + "') >= 0";
    this.Dataset00.filter(sFilter);
};

```

3 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 필터링할 데이터를 입력하고 두 개의 버튼을 클릭해 결과를 비교합니다.

46.10 그룹화된 데이터와 관련된 계산값 구하기

keystring 속성을 지정해서 데이터를 그룹화한 경우 그룹화된 데이터에 대한 값을 확인합니다. ColumnInfo 오브젝트의 prop 속성값을 지정해서 원하는 값을 표시할 수 있지만, 선택한 그룹화된 데이터에 대한 여러 데이터를 한번에 확인하는 방법을 살펴봅니다.

46.10.1 예제

SUBTOTAL에 해당하는 Row를 클릭하면 해당 그룹화된 데이터와 관련된 평균값, 최대값 등을 표시합니다.

Column0	Column1	keystring: A getAvg: 17,5 getMax: 30 getMin: 10 getSum: 70 getCount: 4
A	10	
	20	
	30	
	10	
SUBTOTAL	70	
B	20	
	30	
	20	
SUBTOTAL	70	



46.10.2 예제에서 사용한 핵심 기능

getGroupRangeStart

그룹정보를 포함하고 있는 Row가 참조하는 시작 인덱스값을 반환합니다. 예제에서는 A, B 두 개의 값으로 그룹화된 데이터의 시작 인덱스값(A는 0, B는 5)을 반환합니다.

getGroupRangeCount

그룹정보를 포함하고 있는 Row가 참조하는 Row의 개수를 반환합니다. 예제에서는 A, B 두 개의 값으로 그룹화된 데이터의 Count값(A는 4, B는 3)을 반환합니다. getGroupRangeStart 메소드에서 얻은 값에 getGroupRangeCount 메소드에서 얻은 값을 더하면 마지막 인덱스값을 구할 수 있습니다.

46.10.3 예제 구현 방법

- 1 Grid 컴포넌트를 화면에 배치합니다.
- 2 Dataset 오브젝트를 아래와 같이 생성하고 Grid 컴포넌트에 바인딩합니다. Dataset 오브젝트의 keystack 속성값은 "Column0"으로 지정합니다.

Columns				
No	id	type	size	prop
1	Column0	STRING	256	
2	Column1	INT	256	

Rows		
No	Column0	Column1
1	A	10
2	A	20
3	A	30
4	B	20
5	B	30
6	B	20
7	A	10

- 3 Grid Contents Editor에서 첫 번째 컬럼의 expr 속성값을 아래와 같이 지정합니다.
그룹화된 데이터의 합계값을 표시하는 Row에 "SUBTOTAL"이라는 텍스트를 표시합니다.

body			Action	
col0	col1		expr	
Column0	Column1			<code>dataset.getRowLevel(currow)>0?"SUBTOTAL":Column0</code>
			CellExpand	
			expandchar	
			expandimage	
			expandshow	hide

- 4 Grid 컴포넌트 옆에 TextArea 컴포넌트를 배치합니다.
- 5 onclick 이벤트 함수를 아래와 같이 작성합니다.

getRowLevel 메소드로 선택한 ROW가 그룹정보를 포함하는 Row인지 확인합니다. getGroupRangeStart, getGroupRangeCount 메소드로 그룹화된 데이터의 시작 인덱스와 끝 인덱스를 확인한 후 원하는 메소드를 호출합니다.

```

this.Grid00_onclick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    var temp_str;
    var nStartIdx ;
    var nEndIdx ;
    if(this.Dataset00.getRowLevel(e.row)==0)
    {
        temp_str = "CLICK SUBTOTAL ROW";
    }
    else
    {
        nStartIdx = this.Dataset00.getGroupRangeStart(e.row);
        nEndIdx = this.Dataset00.getGroupRangeStart(e.row)+this.Dataset00.
getGroupRangeCount(e.row);
        temp_str = "keystring: "+this.Dataset00.getColumn(e.row, "Column0")+"\n";
        temp_str += "getAvg: "+nexacro.round(this.Dataset00.getAvg("Column1", nStartIdx,
nEndIdx),2)+"\n";
        temp_str += "getMax: "+this.Dataset00.getMax("Column1", nStartIdx, nEndIdx)+"\n";
        temp_str += "getMin: "+this.Dataset00.getMin("Column1", nStartIdx, nEndIdx)+"\n";
        temp_str += "getSum: "+this.Dataset00.getSum("Column1", nStartIdx, nEndIdx)+"\n";
        temp_str += "getCount: "+this.Dataset00.getCount("Column1", nStartIdx, nEndIdx);
    }
    this.TextArea00.set_value(temp_str);
};

```

- ⑥ QuickView(Ctrl + F6)로 실행한 후 SUBTOTAL에 해당하는 Row를 선택하면 해당 그룹화된 데이터에 대한 정보를 TextArea 컴포넌트에 출력합니다.

47.

ExcelExportObject / ExcelImportObject

47.1 nexacro-xeni 소개

넥사크로플랫폼에서는 엑셀의 익스포트와 임포트 처리를 위해 nexacro-xeni를 제공합니다. nexacro-xeni는 엑셀의 익스포트/임포트를 서버에서 처리한 후 그 결과를 돌려주는 기능을 수행하는 서버 애플리케이션입니다.

엑셀 익스포트/임포트 기능을 자체적으로 처리하지 않고 번거롭게 서버 애플리케이션을 통해 처리하는 이유는 엑셀 파일을 처리하려면 로컬 시스템의 자원을 사용해야 하기 때문입니다. 이는 심각한 보안 문제를 야기할 수 있어 브라우저에서 막고 있습니다. 따라서 넥사크로플랫폼에서는 nexacro-xeni를 사용한 방법을 제공합니다.



nexacro-xeni는 Apache POI를 기반으로 작성되었습니다. 따라서 Apache POI를 사용할 수도 있으나 그로 인해 발생하는 문제에 대해서는 책임지지 않습니다. 자세한 설명은 <https://poi.apache.org/> 를 참조하십시오.

nexacro-xeni는 다음의 문서를 익스포트/임포트할 수 있습니다.

파일형태	확장자	설명
Microsoft Excel 97-2003	xls	export / import 둘 다 지원
Microsoft Excel 2007/2010	xlsx	export / import 둘 다 지원
CSV format file	csv	import만 지원
HancomOffice Hancell 2010 / 2014	cell	지원 예정

익스포트/임포트 동작 시 서버에는 파일들이 생성됩니다. 이 파일들은 nexacro-xeni의 파일매니저를 통해 관리되기 때문에 기본 설정으로 사용하면 개발자가 신경쓸 필요는 없습니다. 파일매니저는 서버의 /WEB-INF/web.xml 파일에서 서블릿 컨텍스트 파라미터를 통해 동작을 설정할 수 있습니다. 다음은 서블릿 컨텍스트 파라미터로 설정할 수 있는 요소에 대한 설명입니다.

이름	기본값	설명
export-path	/export	내려받을 파일을 임시로 저장하는 경로를 지정합니다.
import-path	/import	그리드 컴포넌트로 가져올 파일을 임시로 저장하는 경로를 지정합니다. monitor-enabled 설정값이 true이면 데이터 처리 후 바로 삭제합니다.

이름	기본값	설명
monitor-enabled	true	파일 매니저 실행 여부를 지정합니다. false로 지정한 경우에는 파일 매니저가 동작하지 않고 임시 파일을 삭제하지 않습니다.
monitor-cycle-time	30	임시 저장된 내려받을 파일을 삭제하는 주기를 '분' 단위로 설정합니다. 해당 주기로 파일 매니저를 실행해 삭제할 파일을 체크하고 삭제 처리합니다.
file-stroage-time	10	임시로 저장한 내려받을 파일을 보관하는 시간을 '분' 단위로 설정합니다. 파일 매니저 실행 시 생성 후 지정된 시간이 지난 파일은 삭제합니다.



자세한 내용은 배포되는 nexacro-xeni.war 파일에 포함된 README.txt 파일을 참조하십시오.

47.2 ExcelExportObject 소개

ExcelExportObject는 Grid의 내용을 엑셀 파일로 익스포트하는데 사용하는 오브젝트입니다. 개발자는 이 오브젝트를 사용해 익스포트에 필요한 설정과 서버로의 요청을 수행할 수 있습니다.

익스포트 처리 결과는 이벤트를 통해 확인할 수 있습니다. 사용할 수 있는 이벤트로는 onsuccess, onerror, onprogress가 있습니다. onsuccess는 익스포트가 성공적으로 완료되었을 때 발생하는 이벤트이고, onerror는 익스포트중 오류가 생겼을 때 발생하는 이벤트입니다. 그리고 onprogress는 익스포트 진행 상태에 따라 발생하는 이벤트입니다. 이벤트는 객체 생성 시 이벤트와 이벤트 핸들러를 등록하여 사용할 수 있습니다.

ExcelExportObject는 다른 UI 컴포넌트와 달리 형태가 없는 오브젝트입니다. 따라서 스크립트로 직접 객체와 이벤트를 생성해줘야 합니다. 다음은 ExcelExportObject를 사용해 익스포트를 수행하는 절차에 대한 간단한 설명과 샘플을 보여줍니다.

① 객체의 생성

ExcelExportObject 객체를 생성합니다.

```
this.exportObj = new ExcelExportObject("Export00", this);
```

② 속성의 설정

저장할 엑셀 파일의 이름, 익스포트를 수행하는 서비스 URL 그리고 대상이 될 Grid와 범위 등을 필요에 따라 설정합니다.


```

this.exportObj.set_exportfilename("ExcelExportFile");
this.exportObj.set_exporttype(nexacro.ExportTypes.EXCEL2007);
this.exportObj.set_exporturl("http://localhost:8080/XExportImport");

```

③ 익스포트할 컴포넌트 추가

```

this.exportObj.addExportItem(nexacro.ExportItemTypes.GRID, this.Grid00, "Sheet1!A1");

```

④ 이벤트 핸들러 등록

필요에 따라 addEventHandler 메소드를 사용해 이벤트와 이벤트 핸들러를 등록합니다. 여기서 등록한 이벤트 핸들러는 직접 구현해야 합니다.

```

this.addEventHandler("onsuccess", this.Export00_onsuccess, this);
this.addEventHandler("onerror", this.Export00_onerror, this);

```

⑤ 익스포트 수행

모든 준비가 완료되면 익스포트를 수행합니다.

```

this.exportObj.exportData();

```

47.3 ExcellImportObject 소개

ExcellImportObject는 엑셀 파일의 내용을 Grid에 임포트하는데 사용하는 오브젝트입니다. 임포트에 필요한 설정과 서버로의 요청을 담당합니다. 개발자는 스크립트에서 이를 사용하여 엑셀을 임포트할 수 있습니다.

임포트 처리 결과는 이벤트를 통해 확인할 수 있습니다. 사용할 수 있는 이벤트로는 onsuccess, onerror가 있습니다. onsuccess는 임포트가 성공적으로 완료되었을 때 발생하는 이벤트이고, onerror는 임포트중 오류가 생겼을 때 발생하는 이벤트입니다. 이벤트는 객체 생성시 이벤트와 핸들러를 등록하여 사용할 수 있습니다.

① 객체의 생성

ExcellImportObject 객체를 생성합니다.

```
this.importObj = new ExcelImportObject("Import00", this);
```

2 속성의 설정

임포트할 파일의 형식, 임포트를 수행하는 서비스 URL 등을 필요에 따라 설정합니다.

```
this.importObj.set_importtype(nexacro.ImportTypes.EXCEL);
this.importObj.set_importurl("http://localhost:8080/deploy/XImport");
```

3 이벤트 핸들러 등록

필요에 따라 addEventHandler 메소드를 사용해 이벤트와 이벤트 핸들러를 등록합니다. 여기서 등록한 이벤트 핸들러는 직접 구현해야 합니다.

```
this.importObj.addEventHandler("onsuccess", this.Import00_onsuccess, this);
this.importObj.addEventHandler("onerror", this.Import00_onerror, this);
```

4 수행

모든 준비가 완료되면 임포트를 수행합니다.

```
this.importObj.importData("", "Sheet1!A1:F6", "Dataset00");
```

47.4 Grid를 엑셀 파일로 내보내기(익스포트)

ExcelExportObject를 사용하여 화면상의 Grid를 엑셀 파일로 익스포트할 수 있습니다.



제품 버전에 따라 엑셀 파일 내보내기 시 웹브라우저에서 새 창이 표시될 수 있습니다.
 IE (17.0.0.1600 미만 버전)
 Chrome (17.0.0.2300 미만 버전)

47.4.1 예제

다음은 ExcelExportObject를 사용하여 Grid의 내용을 엑셀 파일로 다운로드하는 예제입니다.

Export 버튼을 누르면 ExcelExportObject 객체를 생성하고 익스포트에 필요한 속성을 설정한 후 익스포트가 수행됩니다. 익스포트가 수행되면 파일 다운로드 다이얼로그가 뜨며 로컬PC로 저장할 수 있습니다.

Name	Address	Company	Department	Salary
John	Seoul, KOREA	1dollar	Sales	₩15,000
James	Barcelona, Spain	EBOT	Consulting	₩25,000
Donald	Califonia, USA	EBOT	Research Institute	₩1,000,000
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
Joe	São Paulo, Brazil	hangul	R&D	₩8,600
Michael	Mumbai, India	1dollar	Personnel	₩1,300
Lilli	Ankara, Turkey	hangul	Quality Assurance	₩4,400
Colt	Sydney, NSW, Australia	hangul	Sales	₩62,000

Export

 sample_excelexportimport_01.xfdl

다음은 익스포트를 수행한 후 로컬 PC에 저장된 엑셀 파일의 내용을 보여줍니다. Grid의 데이터가 그대로 엑셀 파일로 익스포트됨을 확인할 수 있습니다.

	A	B	C	D	E
1	Name	Address	Company	Department	Salary
2	John	Seoul, KOREA	1dollar	Sales	₩15,000
3	James	Barcelona, Spain	EBOT	Consulting	₩25,000
4	Donald	Califonia, USA	EBOT	Research Institute	₩1,000,000
5	Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
6	Joe	São Paulo, Brazil	hangul	R&D	₩8,600
7	Michael	Mumbai, India	1dollar	Personnel	₩1,300
8	Lilli	Ankara, Turkey	hangul	Quality Assurance	₩4,400
9	Colt	Sydney, NSW, Australia	hangul	Sales	₩62,000

47.4.2 예제에서 사용한 핵심 기능

exportfilename

익스포트 완료 후 저장될 파일명을 설정하는 속성입니다. 본 속성을 설정하지 않으면 임의의 파일명으로 설정됩니다.

exporturl

익스포트를 수행할 서비스의 URL을 설정하는 속성입니다. 본 속성을 설정하지 않으면 익스포트가 수행되지 않습니다.

addExportItem

익스포트할 아이템(컴포넌트)을 ExcelExportObject의 익스포트 목록에 추가하는 메소드입니다. 현재 Grid만 지원합니다.

addEventHandler

이벤트 발생 시 수행할 이벤트 핸들러를 추가하는 메소드입니다.

exportData

ExcelExportObject에 추가된 아이템을 익스포트하는 메소드입니다.

importData

이벤트 발생 시 수행할 이벤트 핸들러를 추가하는 메소드입니다.

47.4.3 예제 구현 방법



이 샘플은 demo.nexacro.com에서 동작하도록 만들어 졌습니다. 사용자 PC에서 샘플을 수행하려면 WAS와 nexacro-xeni가 설치되고 실행중인 상태여야 합니다. 그리고 샘플의 스크립트 코드에서 url 관련 설정을 사용자 환경에 맞게 수정해야 합니다. WAS와 사용자 PC의 도메인이 다른 경우 크로스 도메인 문제가 발생할 수 있습니다.

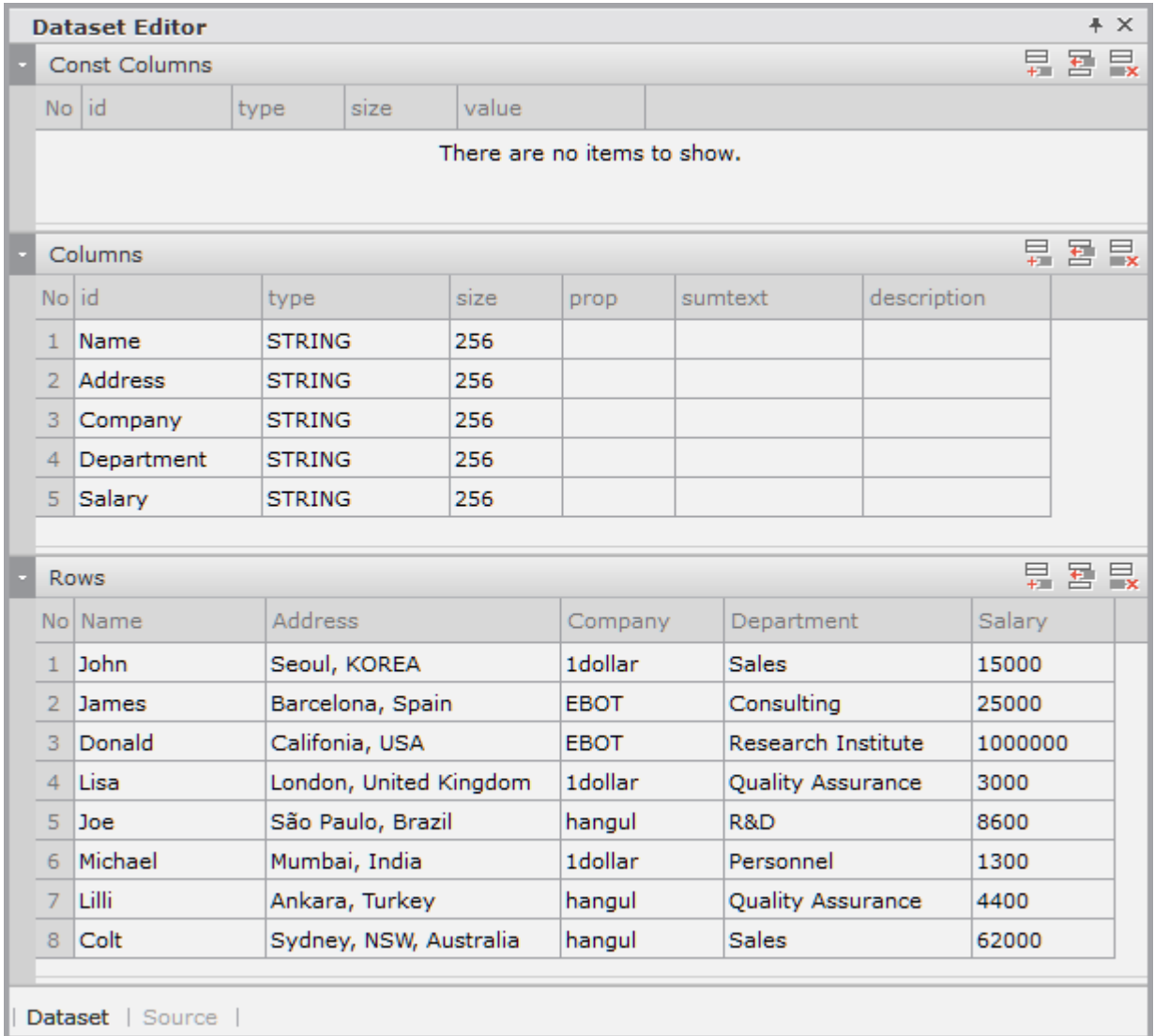
① 컴포넌트 생성하기

툴바에서 버튼 과 그리드 를 선택한 후, Form 위에 적당한 크기로 드래그하여 생성합니다. 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

② Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.



No	id	type	size	prop	sumtext	description
1	Name	STRING	256			
2	Address	STRING	256			
3	Company	STRING	256			
4	Department	STRING	256			
5	Salary	STRING	256			

No	Name	Address	Company	Department	Salary
1	John	Seoul, KOREA	1dollar	Sales	15000
2	James	Barcelona, Spain	EBOT	Consulting	25000
3	Donald	Califonia, USA	EBOT	Research Institute	1000000
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000
5	Joe	São Paulo, Brazil	hangul	R&D	8600
6	Michael	Mumbai, India	1dollar	Personnel	1300
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000

Dataset | Source |

③ Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

④ 익스포트 기능 구현하기

Form에 있는 Button을 선택한 후 onclick 이벤트 핸들러를 아래와 같이 추가합니다.

```

this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.exportObj = new ExcelExportObject("Export00", this);

    this.exportObj.set_exportfilename("ExcelExportFile");
    this.exportObj.set_exporturl("http://demo.nexacro.com/developer_guide/XExportImport");

    this.exportObj.addExportItem(nexacro.ExportItemTypes.GRID, this.Grid00, "Sheet1!A1");

    this.addEventHandler("onsuccess", this.Export00_onsuccess, this);
    this.addEventHandler("onerror", this.Export00_onerror, this);

    var intExportedItem = this.exportObj.exportData();
    trace("Number of Exported Item: " + intExportedItem);
};

```

5 이벤트 핸들러 구현하기

```

this.Export00_onsuccess = function(obj:ExcelExportObject, e:nexacro.ExcelExportEventInfo)
{
    trace("Export00_onsuccess");
}

this.Export00_onerror = function(obj:ExcelExportObject, e:nexacro.ExcelExportEventInfo)
{
    trace("Export00_onerror");
}

```

6 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행합니다.

Export 버튼을 눌러 화면의 Grid에 데이터가 엑셀 파일로 익스포트되는지 확인합니다.

47.5 엑셀 파일 읽어서 Grid로 보여주기(임포트)

ExcellImportObject를 사용하여 로컬 PC에 있는 엑셀 파일을 화면상의 Grid로 임포트할 수 있습니다.

47.5.1 예제

다음은 ExcellImportObject를 사용하여 로컬 PC에 있는 엑셀 파일을 화면상의 Grid로 임포트하는 예제입니다.

Import 버튼을 누르면 ExcellImportObject 객체를 생성하고 임포트에 필요한 속성을 설정한 후 임포트가 수행됩니다. 임포트가 수행되면 파일 열기 다이얼로그가 뜨며 로컬PC에서 엑셀 파일을 선택할 수 있습니다. 임포트 진행 중에는 진행을 알리는 이미지가 뜨는데 그때, **ESC**키를 누르면 임포트가 중단됩니다.

Name	Address	Company	Department	Salary
John	Seoul, KOREA	1dollar	Sales	₩15,000
James	Barcelona, Spain	EBOT	Consulting	₩25,000
Donald	Califonia, USA	EBOT	Research Institute	₩1,000,000
Lisa	London, United K	1dollar	Quality Assurance	₩3,000
Joe	São Paulo, Brazil	hangul	R&D	₩8,600
Michael	Mumbai, India	1dollar	Personnel	₩1,300

Import



sample_excelexportimport_02.xfdl

다음은 임포트를 수행 시에 사용한 로컬 PC의 엑셀 파일입니다. 엑셀 파일의 데이터와 임포트한 Grid의 데이터가 차이가 보임을 확인할 수 있는데 이는 임포트 시 셀의 범위를 A1~E7까지로 정했기 때문입니다.

	A	B	C	D	E
1	Name	Address	Company	Department	Salary
2	John	Seoul, KOREA	1dollar	Sales	₩15,000
3	James	Barcelona, Spain	EBOT	Consulting	₩25,000
4	Donald	Califonia, USA	EBOT	Research Institute	₩1,000,000
5	Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
6	Joe	São Paulo, Brazil	hangul	R&D	₩8,600
7	Michael	Mumbai, India	1dollar	Personnel	₩1,300
8	Lilli	Ankara, Turkey	hangul	Quality Assurance	₩4,400
9	Colt	Sydney, NSW, Australia	hangul	Sales	₩62,000



이 샘플은 기능을 보여주기 위한 용도로 제작되었기 때문에 구조가 복잡한 엑셀의 경우 임포트시 에러가 발생할 수 있습니다.

47.5.2 예제에서 사용한 핵심 기능

importtype

임포트할 파일의 형식을 설정하는 속성입니다. 설정은 다음 표의 상수값으로 할 수 있습니다. 한셀 2010 형식의 경우 현재 지원하지 않으니 엑셀 2007 파일로 변환하여 사용하시기 바랍니다.

nexacro.ImportTypes.EXCEL	엑셀 97 형식으로 임포트합니다.
nexacro.ImportTypes.EXCEL97	엑셀 97~2003 형식으로 임포트합니다.
nexacro.ImportTypes.EXCEL2007	엑셀 2007 형식으로 임포트합니다.
nexacro.ImportTypes.HANCELL2014	한셀 2014 형식으로 임포트합니다.

importurl

임포트를 수행할 서비스의 URL을 설정하는 속성입니다. 본 속성을 설정하지 않으면 임포트가 수행되지 않습니다.

addEventHandler

이벤트 발생 시 수행할 이벤트 핸들러를 추가하는 메소드입니다.

importData

엑셀 또는 한셀 파일을 Dataset으로 임포트하는 메소드입니다.

```

this.ExcelImportObject00.importData("http://localhost:8080/xeni/XImport/my.xlsx","Sheet00!D15:E28","Dataset03=output1");

this.ExcelImportObject00.importData("", "Sheet00!D15:E28", "Dataset03=output1", "filepassword=
strpassword");

this.ExcelImportObject00.importData("", "[Command=getsheetlist;Output=sheetlist][Command=
getsheetdata;Output=output1;Head=!A1:K1;Body=!A2:K15][Output=output2; Body=Sheet1!B7:Z207]", "
ds_list=sheetlist, ds_Data1=output1, ds_Data2=output2");

```


47.5.3 예제 구현 방법



이 샘플은 demo.nexacro.com에서 동작하도록 만들어 졌습니다. 사용자 PC에서 샘플을 수행하려면 WAS와 nexacro-xeni가 설치되고 실행중인 상태여야 합니다. 그리고 샘플의 스크립트 코드에서 url 관련 설정을 사용자 환경에 맞게 수정해야 합니다. WAS와 사용자 PC의 도메인이 다른 경우 크로스 도메인 문제가 발생할 수 있습니다.

1 컴포넌트 생성하기

툴바에서 버튼 과 그리드 를 선택한 후, Form 위에 적당한 크기로 드래그하여 생성합니다. 그냥 폼에 대고 클릭하면 기본 크기로 생성됩니다.

2 импорт 기능 구현하기

Form에 있는 Button을 선택한 후 onclick 이벤트 핸들러를 아래와 같이 추가합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.importObj = new ExcelImportObject("Import00", this);

    this.importObj.set_importtype(nexacro.ImportTypes.EXCEL);
    this.importObj.set_importurl("http://demo.nexacro.com/developer_guide/XImport");

    this.importObj.addEventHandler("onsuccess", this.Import00_onsuccess, this);
    this.importObj.addEventHandler("onerror", this.Import00_onerror, this);

    this.importObj.importData("", "Sheet1!A1:E7", "Dataset00");

};
```

3 이벤트 핸들러 구현하기

```
this.Import00_onsuccess = function(obj:ExcelImportObject, e:nexacro.ExcelImportEventInfo)
{
    trace("Import00_onsuccess");

    this.Grid00.createFormat();
}
```

```

this.Import00_onerror = function(obj:ExcelImportObject, e:nexacro.
ExcelImportErrorEventInfo)
{
    trace("Import00_onerror");
}

```

④ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행합니다.

Import 버튼을 누르면 파일 다이얼로그 창이 뜹니다. 엑셀 파일을 선택하면 임포트가 진행되고 화면의 Grid로 엑셀 내용이 임포트되는지 확인합니다.

47.6 Grid의 이미지를 엑셀 파일로 내보내기

ExcelExportObject를 사용하여 화면상의 Grid를 엑셀 파일로 익스포트할 수 있습니다. 이때 Grid에 포함된 이미지가 있다면 ExportItem 오브젝트의 exportimage 속성을 사용하여 이미지 또한 익스포트가 가능합니다.

47.6.1 예제

다음은 ExcelExportObject를 사용하여 Grid의 내용을 엑셀 파일로 다운로드하는 예제입니다.

Export image 버튼을 누르면 ExcelExportObject 객체를 생성하고 익스포트에 필요한 속성을 설정한 후 익스포트가 수행됩니다. 익스포트가 수행되면 파일 다운로드 다이얼로그가 뜨며 로컬PC로 저장할 수 있습니다.

Name	Address	Company	Department	Salary	Image
John	Seoul, KOREA	1dollar	Sales	₩15,000	★
James	Barcelona, Spain	EBOT	Consulting	₩25,000	🔍
Donald	Califonia, USA	EBOT	Research Institute	₩1,000,000	★
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000	★
Joe	São Paulo, Brazil	hangul	R&D	₩8,600	🔍
Michael	Mumbai, India	1dollar	Personnel	₩1,300	★
Lilli	Ankara, Turkey	hangul	Quality Assurance	₩4,400	🔍
Colt	Sydney, NSW, Australia	hangul	Sales	₩62,000	🔍

Export image



sample_excelexportimport_03.xfdl

다음은 익스포트를 수행한 후 로컬 PC에 저장된 엑셀 파일의 내용을 보여줍니다. Grid의 데이터와 이미지가 그대로 엑셀 파일로 익스포트됨을 확인할 수 있습니다.

	A	B	C	D	E	F
1	Name	Address	Company	Department	Salary	Image
2	John	Seoul, KOREA	1dollar	Sales	₩15,000	
3	James	Barcelona, Spain	EBOT	Consulting	₩25,000	
4	Donald	Califonia, USA	EBOT	Research Institute	₩1,000,000	
5	Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000	
6	Joe	São Paulo, Brazil	hangul	R&D	₩8,600	
7	Michael	Mumbai, India	1dollar	Personnel	₩1,300	
8	Lilli	Ankara, Turkey	hangul	Quality Assurance	₩4,400	
9	Colt	Sydney, NSW, Australia	hangul	Sales	₩62,000	

47.6.2 예제에서 사용한 핵심 기능

getExportItem

ExcelExportObject에서 지정한 타입의 아이템을 반환하는 메소드입니다.

setExportItem

익스포트할 아이템을 ExcelExportObject의 특정 위치에 추가하는 메소드입니다.

exportimage

Grid의 셀에 설정된 이미지의 익스포트 방식을 설정하는 속성입니다.

47.6.3 예제 구현 방법



이 샘플은 demo.nexacro.com에서 동작하도록 만들어 졌습니다. 사용자 PC에서 샘플을 수행하려면 WAS와 nexacro-xeni가 설치되고 실행중인 상태여야 합니다. 그리고 샘플의 스크립트 코드에서 url 관련 설정을 사용자 환경에 맞게 수정해야 합니다. WAS와 사용자 PC의 도메인이 다른 경우 크로스 도메인 문제가 발생할 수 있습니다.

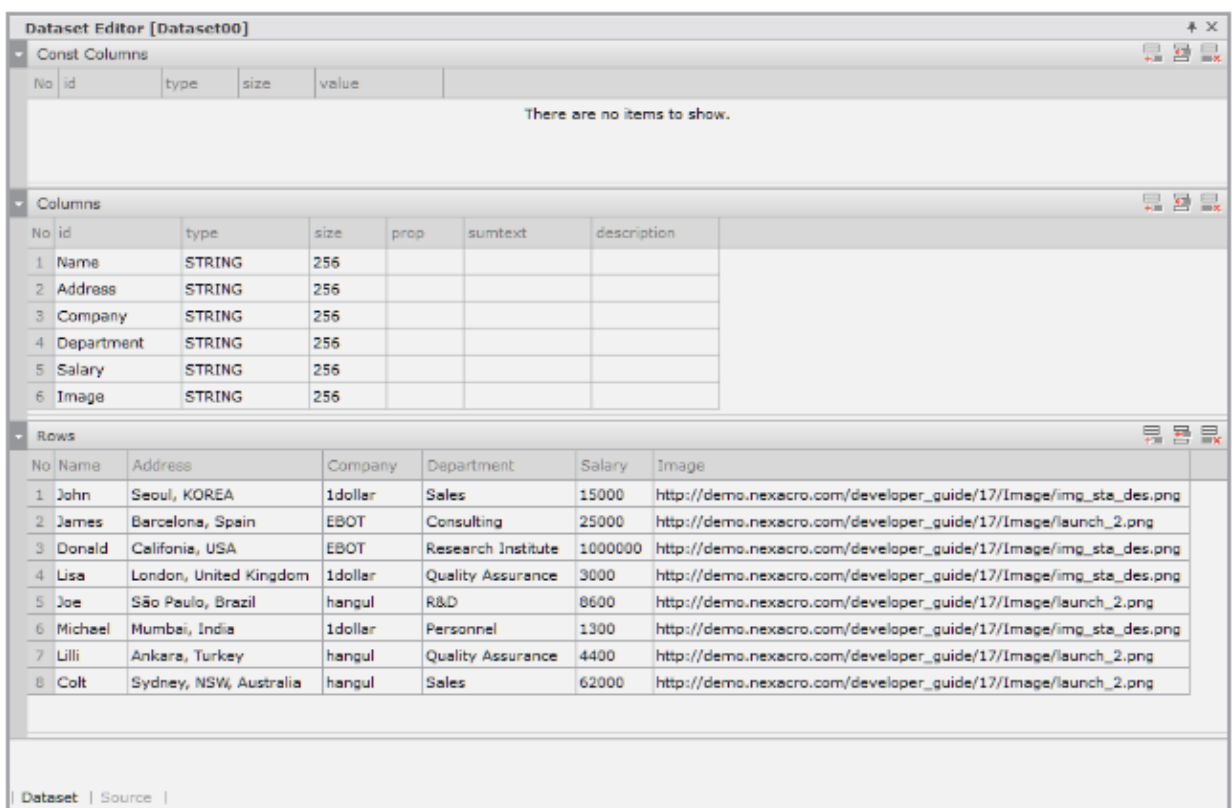
1 컴포넌트 생성하기

툴바에서 버튼 과 그리드 를 선택한 후, Form 위에 적당한 크기로 드래그하여 생성합니다.

2 Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset 을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.



Dataset Editor [Dataset00]

Const Columns

No	id	type	size	value
There are no items to show.				

Columns

No	id	type	size	prop	sumtext	description
1	Name	STRING	256			
2	Address	STRING	256			
3	Company	STRING	256			
4	Department	STRING	256			
5	Salary	STRING	256			
6	Image	STRING	256			

Rows

No	Name	Address	Company	Department	Salary	Image
1	John	Seoul, KOREA	1dollar	Sales	15000	http://demo.nexacro.com/developer_guide/17/Image/img_sta_des.png
2	James	Barcelona, Spain	EBOT	Consulting	25000	http://demo.nexacro.com/developer_guide/17/Image/launch_2.png
3	Donald	California, USA	EBOT	Research Institute	1000000	http://demo.nexacro.com/developer_guide/17/Image/img_sta_des.png
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000	http://demo.nexacro.com/developer_guide/17/Image/img_sta_des.png
5	Joe	São Paulo, Brazil	hangul	R&D	8600	http://demo.nexacro.com/developer_guide/17/Image/launch_2.png
6	Michael	Mumbai, India	1dollar	Personnel	1300	http://demo.nexacro.com/developer_guide/17/Image/img_sta_des.png
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400	http://demo.nexacro.com/developer_guide/17/Image/launch_2.png
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000	http://demo.nexacro.com/developer_guide/17/Image/launch_2.png

Dataset | Source |

3 Dataset 바인딩

Dataset을 Grid 컴포넌트에 바인딩합니다.

Invisible Objects 영역에 있는 Dataset을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

4 익스포트 기능 구현하기

Form에 있는 Button을 선택한 후 onclick 이벤트 핸들러를 아래와 같이 추가합니다.

```

this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.exportObj = new ExcelExportObject("Export00", this);

    this.exportObj.set_exportfilename("ExcelExportFile");
    this.exportObj.set_exporttype(nexacro.ExportTypes.EXCEL2007);
    this.exportObj.set_exporturl("http://localhost:8080/deploy/XExportImport");

    this.exportObj.addExportItem(nexacro.ExportItemTypes.GRID, this.Grid00, "Sheet1!A1");

    var objExportItem = this.exportObj.getExportItem(nexacro.ExportItemTypes.GRID, 0);
    objExportItem.set_exportimage("image");

    this.exportObj.setExportItem(nexacro.ExportItemTypes.GRID, 0, objExportItem);

    this.addEventHandler("onsuccess", this.Export00_onsuccess, this);
    this.addEventHandler("onerror", this.Export00_onerror, this);

    var intExportedItem = this.exportObj.exportData();
    trace("Number of Exported Item: " + intExportedItem);
};

```

5 이벤트 핸들러 구현하기

```

this.Export00_onsuccess = function(obj:ExcelExportObject, e:nexacro.ExcelExportEventInfo)
{
    trace("Export00_onsuccess");
}

this.Export00_onerror = function(obj:ExcelExportObject, e:nexacro.ExcelExportEventInfo)
{
    trace("Export00_onerror");
}

```

6 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행합니다.

Export 버튼을 눌러 화면의 Grid에 데이터와 이미지가 엑셀 파일로 익스포트되는지 확인합니다.

47.7 여러 Grid 데이터를 엑셀 파일로 보내내기

ExcelExportObject를 사용하여 여러 개의 Grid를 엑셀 파일로 익스포트할 때 하나 혹은 여러 시트(Sheet)로 지정하여 익스포트할 수 있습니다. 이는 ExcelExportObject에 익스포트할 컴포넌트를 추가하는 addExportItem 메소드에서 익스포트 할 위치를 지정할 수 있기 때문입니다.

익스포트할 위치는 addExportItem 메소드의 매개변수 중 strRange를 설정하여 정할 수 있습니다. strRange를 설정할 때는 "시트명!셀영역" 또는 "셀영역"의 형식을 사용합니다. 예를 들어, export2 시트의 C3 셀에 Grid를 익스포트하고 싶으면 "export2!C3" 으로 설정합니다.

47.7.1 예제

다음은 ExcelExportObject를 사용하여 화면상에 있는 2개의 Grid를 엑셀 파일로 익스포트하는 예제입니다.

익스포트를 수행하는 2개의 버튼이 있는데 **Export to 1 sheet** 버튼을 누르면 2개 Grid를 1개 시트로 모두 익스포트하고 **Export to 2 sheets** 버튼을 누르면 2개 Grid를 2개 시트로 각각 익스포트합니다.

Name	Address	Company	Department	Salary
John	Seoul, KOREA	1dollar	Sales	₩15,000
James	Barcelona, Spain	EBOT	Consulting	₩25,000
Donald	Califonia, USA	EBOT	Research Institute	₩1,000,000
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
Joe	São Paulo, Brazil	hangul	R&D	₩8,600
Michael	Mumbai, India	1dollar	Personnel	₩1,300
Lilli	Ankara, Turkey	hangul	Quality Assurance	₩4,400
Colt	Sydney, NSW, Australia	hangul	Sales	₩62,000

Name	Address	Company	Department	Salary
John	Seoul, KOREA	1dollar	Sales	₩15,000
James	Barcelona, Spain	EBOT	Consulting	₩25,000
Donald	Califonia, USA	EBOT	Research Institute	₩1,000,000
Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
Joe	São Paulo, Brazil	hangul	R&D	₩8,600
Michael	Mumbai, India	1dollar	Personnel	₩1,300
Lilli	Ankara, Turkey	hangul	Quality Assurance	₩4,400
Colt	Sydney, NSW, Australia	hangul	Sales	₩62,000

Export to 1 sheet

Export to 2 sheets



sample_excelexportimport_04.xfdl

다음은 수행된 결과로 생성된 엑셀 파일의 내용을 보여줍니다.

Export to 1 sheet 버튼을 누른 후 익스포트된 ExcelExportFile00.xls을 보면 Grid00을 Sheet1의 A1셀에 Grid01을 Sheet1의 A11셀에 익스포트한 결과를 확인할 수 있습니다.

	A	B	C	D	E
1	Name	Address	Company	Department	Salary
2	John	Seoul, KOREA	1dollar	Sales	₩15,000
3	James	Barcelona, Spain	EB0T	Consulting	₩25,000
4	Donald	Califonia, USA	EB0T	Research Institute	₩1,000,000
5	Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
6	Joe	São Paulo, Brazil	hangul	R&D	₩8,600
7	Michael	Mumbai, India	1dollar	Personnel	₩1,300
8	Lilli	Ankara, Turkey	hangul	Quality Assurance	₩4,400
9	Colt	Sydney, NSW, Australia	hangul	Sales	₩62,000
10					
11	Name	Address	Company	Department	Salary
12	John	Seoul, KOREA	1dollar	Sales	₩15,000
13	James	Barcelona, Spain	EB0T	Consulting	₩25,000
14	Donald	Califonia, USA	EB0T	Research Institute	₩1,000,000
15	Lisa	London, United Kingdom	1dollar	Quality Assurance	₩3,000
16	Joe	São Paulo, Brazil	hangul	R&D	₩8,600
17	Michael	Mumbai, India	1dollar	Personnel	₩1,300
18	Lilli	Ankara, Turkey	hangul	Quality Assurance	₩4,400
19	Colt	Sydney, NSW, Australia	hangul	Sales	₩62,000
20					
21					
22					

Export to 2 sheets 버튼을 누른 후 익스포트된 ExcelExportFile01.xls을 보면 Grid00을 Sheet1의 A1셀에 Grid01을 Sheet2의 A1셀에 익스포트되었음을 확인할 수 있습니다.

ExcelExportFile01:1					ExcelExportFile01:2				
	A	B	C	D		A	B	C	D
1	Name	Address	Company	Department	1	Name	Address	Company	Department
2	John	Seoul, KOREA	1dollar	Sales	2	John	Seoul, KOREA	1dollar	Sales
3	James	Barcelona, Spain	EBOT	Consulting	3	James	Barcelona, Spain	EBOT	Consulting
4	Donald	Califonia, USA	EBOT	Research Institute	4	Donald	Califonia, USA	EBOT	Research Institute
5	Lisa	London, United Kingdom	1dollar	Quality Assurance	5	Lisa	London, United Kingdom	1dollar	Quality Assurance
6	Joe	São Paulo, Brazil	hangul	R&D	6	Joe	São Paulo, Brazil	hangul	R&D
7	Michael	Mumbai, India	1dollar	Personnel	7	Michael	Mumbai, India	1dollar	Personnel
8	Lilli	Ankara, Turkey	hangul	Quality Assurance	8	Lilli	Ankara, Turkey	hangul	Quality Assurance
9	Colt	Sydney, NSW, Australia	hangul	Sales	9	Colt	Sydney, NSW, Australia	hangul	Sales
10					10				
11					11				

47.7.2 예제에서 사용한 핵심 기능

clearExportItems

지정하는 아이템을 ExcelExportObject에서 삭제하는 메소드입니다. 삭제되면 익스포트에서 제외됩니다.

addExportItem

익스포트할 아이템(컴포넌트)을 ExcelExportObject의 익스포트 목록에 추가하는 메소드입니다. 현재 Grid만 지원합니다.

47.7.3 예제 구현 방법



이 샘플은 demo.nexacro.com에서 동작하도록 만들어 졌습니다. 사용자 PC에서 샘플을 수행하려면 WAS와 nexacro-xeni가 설치되고 실행중인 상태여야 합니다. 그리고 샘플의 스크립트 코드에서 url 관련 설정을 사용자 환경에 맞게 수정해야 합니다. WAS와 사용자 PC의 도메인이 다른 경우 크로스 도메인 문제가 발생할 수 있습니다.

① 컴포넌트 생성하기

Button 컴포넌트와 Grid 컴포넌트를 각각 2개씩 생성하여 예제와 같이 배치합니다. 툴바에서 Button  과 Grid  를 선택한 후, Form 위에 적당한 크기로 드래그하여 생성합니다.

② Dataset 생성과 데이터 입력하기

Grid에 출력할 데이터 아이템 목록을 가진 Dataset을 생성합니다.

1. 툴바에서 Dataset  을 선택한 후, 폼 위 적당한 공간에 클릭하여 Dataset을 생성합니다.
2. 생성한 Dataset에 데이터 아이템 목록을 입력합니다. Invisible Objects 영역에 있는 Dataset을 더블 클릭하면 Dataset Editor가 뜨는데 Columns와 Rows에 아래 그림과 같이 입력합니다.

Dataset Editor

Const Columns

No	id	type	size	value
There are no items to show.				

Columns

No	id	type	size	prop	sumtext	description
1	Name	STRING	256			
2	Address	STRING	256			
3	Company	STRING	256			
4	Department	STRING	256			
5	Salary	STRING	256			

Rows

No	Name	Address	Company	Department	Salary
1	John	Seoul, KOREA	1dollar	Sales	15000
2	James	Barcelona, Spain	EBOT	Consulting	25000
3	Donald	Califonia, USA	EBOT	Research Institute	1000000
4	Lisa	London, United Kingdom	1dollar	Quality Assurance	3000
5	Joe	São Paulo, Brazil	hangul	R&D	8600
6	Michael	Mumbai, India	1dollar	Personnel	1300
7	Lilli	Ankara, Turkey	hangul	Quality Assurance	4400
8	Colt	Sydney, NSW, Australia	hangul	Sales	62000

Dataset | Source |

3 Dataset 바인딩

생성한 Dataset을 두 개의 Grid에 각각 바인딩합니다.

Invisible Objects 영역에 있는 Dataset을 Form에 생성한 Grid 컴포넌트로 드래그 앤 드롭하면 바인딩이 완료됩니다.

4 Export to 1 sheet 버튼 기능 구현하기(2개의 Grid를 1개 시트로 익스포트)

Form에 있는 Export to 1 sheet 버튼을 선택한 후 onclick 이벤트 핸들러를 아래와 같이 추가합니다.

```

this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.exportObj = new ExcelExportObject("Export00", this);

    this.exportObj.set_exporturl("http://localhost:8080/deploy/XExportImport");
    this.exportObj.set_exportfilename("ExcelExportFile00");
    this.exportObj.set_exporttype(nexacro.ExportTypes.EXCEL2007);

    this.exportObj.clearExportItems(nexacro.ExportItemTypes.GRID);

    this.exportObj.addExportItem(nexacro.ExportItemTypes.GRID, this.Grid00, "Sheet1!A1" );
    this.exportObj.addExportItem(nexacro.ExportItemTypes.GRID, this.Grid01, "Sheet1!A11"
);

    var nCount = this.exportObj.exportData();
};

```

5 Export to 2 sheets 버튼 기능 구현하기(2개의 Grid를 2개 시트로 익스포트)

Form에 있는 Export to 2 sheets 버튼을 선택한 후 onclick 이벤트 핸들러를 아래와 같이 추가합니다.

```

this.Button01_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.exportObj = new ExcelExportObject("Export01", this);

    this.exportObj.set_exporturl("http://localhost:8080/deploy/XExportImport");
    this.exportObj.set_exportfilename("ExcelExportFile01");
    this.exportObj.set_exporttype(nexacro.ExportTypes.EXCEL2007);

    this.exportObj.clearExportItems(nexacro.ExportItemTypes.GRID);

    this.exportObj.addExportItem(nexacro.ExportItemTypes.GRID, this.Grid00, "Sheet1!A1" );
    this.exportObj.addExportItem(nexacro.ExportItemTypes.GRID, this.Grid01, "Sheet2!A1" );

    var nCount = this.exportObj.exportData();
};

```

6 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행합니다.

Export to 1 sheet 버튼을 눌러 Grid 2개가 엑셀 파일에 1개의 Sheet1로 모두 익스포트되는지 확인합니다.

Export to 2 sheets 버튼을 눌러 Grid 2개가 엑셀 파일에 Sheet1, Shee2로 각각 익스포트되는지 확인합니다.

48.

DataObject

48.1 DataObject 오브젝트 소개

오브젝트 형태의 데이터를 직접 작성하거나 서비스를 호출해서 관리할 수 있습니다. Dataset 오브젝트와 바인딩해서 사용하는 경우 기존 Dataset 오브젝트를 사용하는 컴포넌트와 같이 사용할 수 있습니다.



DataObject 오브젝트는 17.1.2.100 버전부터 지원합니다.



Dataset 오브젝트와 DataObject 오브젝트는 단방향 바인딩만 지원합니다. Dataset 오브젝트에서 변경된 데이터가 DataObject 오브젝트로 반영되지는 않습니다.

48.2 간단한 데이터 표시하기

직원 목록을 JSON으로 받아와서 Grid 컴포넌트에 표시합니다.



데이터는 아래 URL에서 제공하는 샘플 데이터를 사용합니다.

http://demo.nexacro.com/developer_guide/17/FileSample/data_sample_dataobject_01.json



크롬 등 일부 브라우저에서는 CORS 정책에 따라 샘플 데이터 URL에 접근하지 못할 수 있습니다. 아래 첨부 파일을 내려받아 접근할 수 있는 서버에 놓고 확인해보세요.

[data_sample_dataobject_01.json](#)

48.2.1 예제

특정 서비스에서 제공하는 JSON 데이터를 가져와 Dataset 오브젝트에 바인딩하고 Grid 컴포넌트에 표시합니다.

id	employee_name	employee_salary	employee_age
1	Fletcher Connolly	320800	61
2	Lillie-May Fuller	345300	55
3	Skylar Tanner	442800	51
4	Connar Sanders	420000	33
5	Felix Collins	333800	31
6	Osman Weeks	320800	41
7	Aubree Salter	440800	33

 sample_dataobject_01.xfdl

48.2.2 예제에서 사용한 핵심 기능

binddataobject

Dataset 오브젝트와 바인딩할 DataObject 오브젝트를 선택합니다. 실제 적용되는 값은 dataobjectpath 설정에 따라 달라질 수 있습니다.

dataobjectpath

DataObject 오브젝트 데이터 중에서 실제 반영할 데이터를 필터링하기 위해 설정하는 값입니다. 배열 또는 오브젝트를 선택하면 해당하는 값의 목록을 반환합니다.

48.2.3 예제 구현 방법

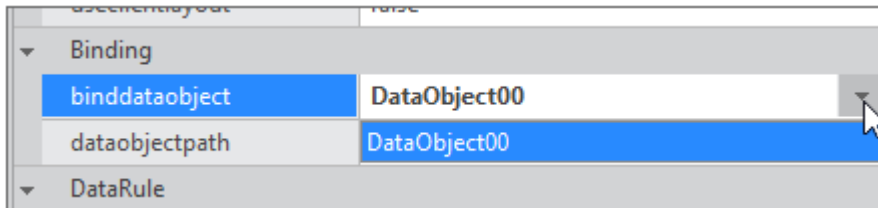
데이터를 가져올 url 설정하기

- 1 DataObject 오브젝트를 추가합니다.
DataObject 오브젝트는 Invisible Object로 추가됩니다.
- 2 DataObject 오브젝트의 url 속성값을 아래와 같이 설정합니다.

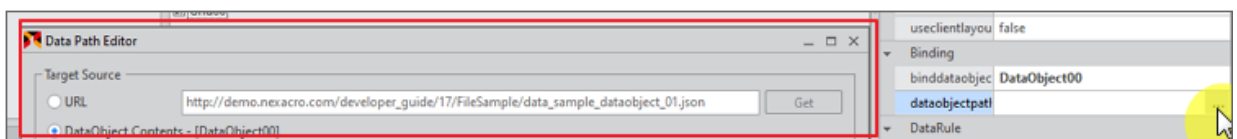
url: http://demo.nexacro.com/developer_guide/17/FileSample/data_sample_dataobject_01.json

Dataset 오브젝트와 DataObject 오브젝트 연결하기

- 1 Dataset 오브젝트를 추가합니다.
- 2 Dataset 오브젝트를 선택하고 속성창에서 binddataobject 속성값을 "DataObject00"으로 선택합니다.

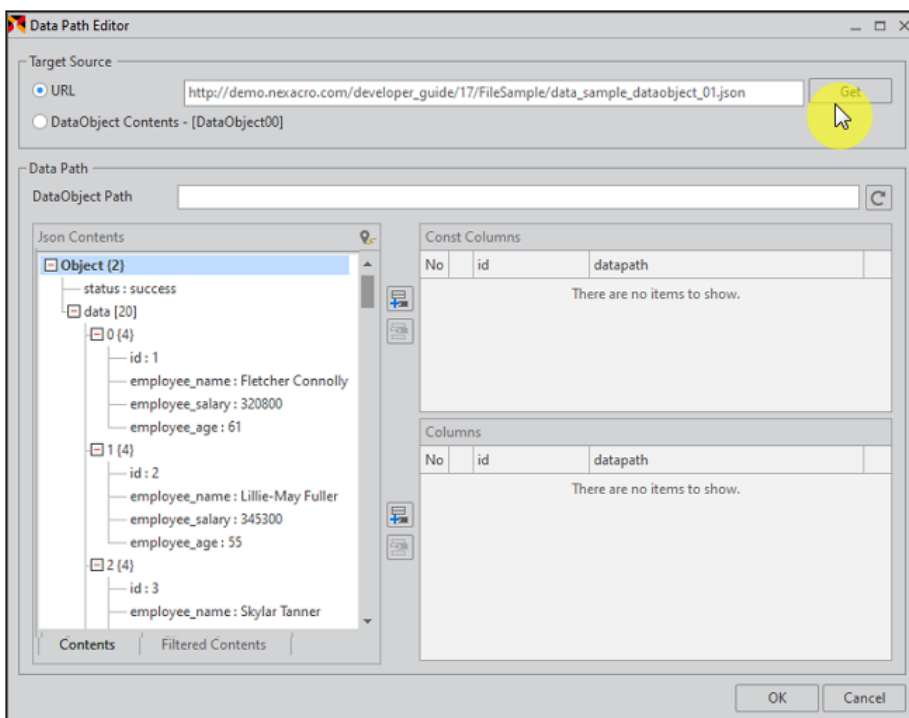


- 3 속성창에서 dataobjectpath 속성 옆에 있는 버튼을 클릭하고 [Data Path Editor]를 실행합니다.

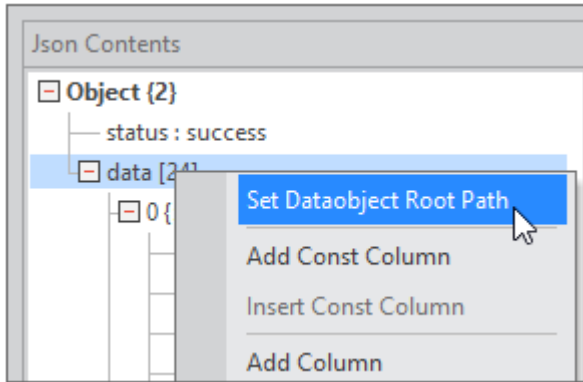


- 4 [Data Path Editor] 창에서 URL 항목을 체크하고 [Get] 버튼을 클릭합니다.

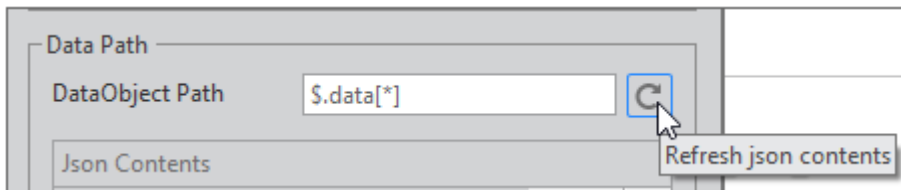
URL 값은 이전 단계에서 DataObject 오브젝트의 url 속성값으로 설정한 값이 보여집니다. 하단 Json Contents 영역에 서버에서 가져온 JSON 데이터를 보여줍니다.



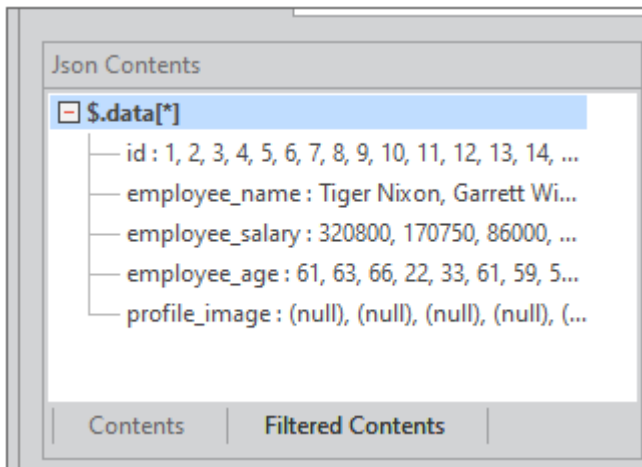
- 5 Json Contents 데이터에서 "data [24]"로 표시된 항목을 선택하고 오른쪽 마우스 클릭 후 [Set Dataobject Root Path] 항목을 선택합니다.



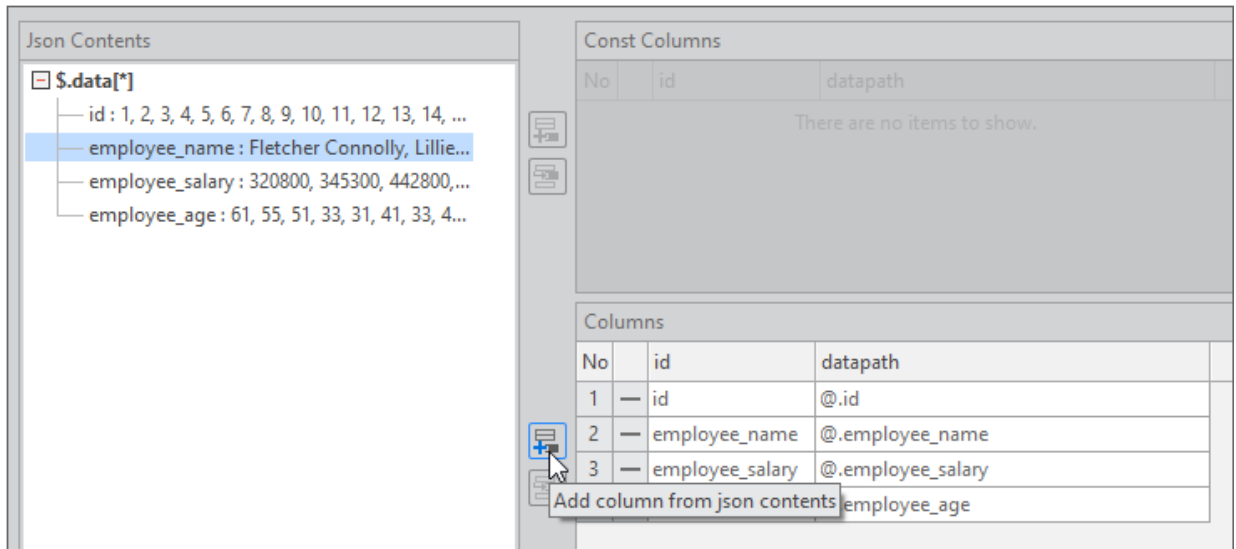
- 6 DataObject Path에 선택한 항목에 루트 경로가 새로 설정된 것을 확인하고 [Refresh json contents] 버튼을 클릭합니다.



- 7 새로 설정된 루트 경로 아래의 데이터가 [Filtered Contents] 탭에 표시됩니다.



- 8 Json Contents 데이터에서 id 항목을 선택하고 오른쪽 마우스 클릭 후 [Add Column] 항목을 선택합니다. 또는 Columns 항목 옆에 있는 버튼을 클릭해서 컬럼 항목을 추가할 수 있습니다.



- ⑨ 순서대로 id, employee_name, employee_salary, employee_age 항목을 컬럼으로 추가합니다.
- ⑩ [OK] 버튼을 클릭합니다. 속성창에서 dataobjectpath 값이 설정된 것을 확인할 수 있습니다.

Grid 컴포넌트에 데이터 표시하기

- ① Grid 컴포넌트를 화면에 배치합니다.
- ② Grid 컴포넌트와 Dataset 오브젝트를 바인딩합니다.
- ③ Button 컴포넌트를 추가하고 아래와 같이 onclick 이벤트 함수를 작성합니다.

```
this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.DataObject00.load();
};
```

- ④ QuickView(Ctrl + F6)로 실행한 후 버튼 클릭 시 정보가 조회되는 것을 확인합니다.

48.3 다국어로 Button 라벨 표시하기

DataObject 오브젝트의 데이터는 외부 서비스에서 받아올 수도 있지만, 직접 편집할 수도 있습니다. 이번 예제에서는 데이터를 직접 추가하고 추가한 데이터를 2개의 Dataset 오브젝트에 바인딩해서 다른 용도로 사용합니다.

48.3.1 예제

Combo 컴포넌트에서 언어를 선택하면 선택한 언어로 Button 컴포넌트 라벨을 표시합니다.



 [sample_dataobject_02.xfdl](#)

48.3.2 예제에서 사용한 핵심 기능

data

오브젝트 또는 문자열 값을 받아서 데이터 오브젝트로 처리합니다. 넥사크로 스튜디오에서는 데이터를 편집할 수 있는 도구를 제공합니다.

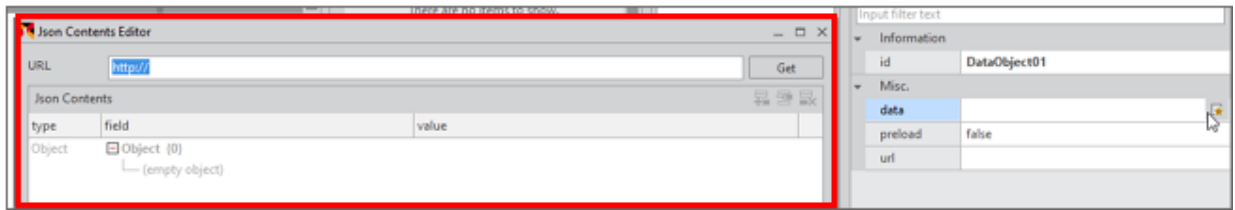
dataobjectpath

DataObject 오브젝트 데이터 중에서 실제 반영할 데이터를 필터링하기 위해 설정하는 값입니다. 스크립트 내에서 값을 변경하면 즉시 반영되어 처리합니다.

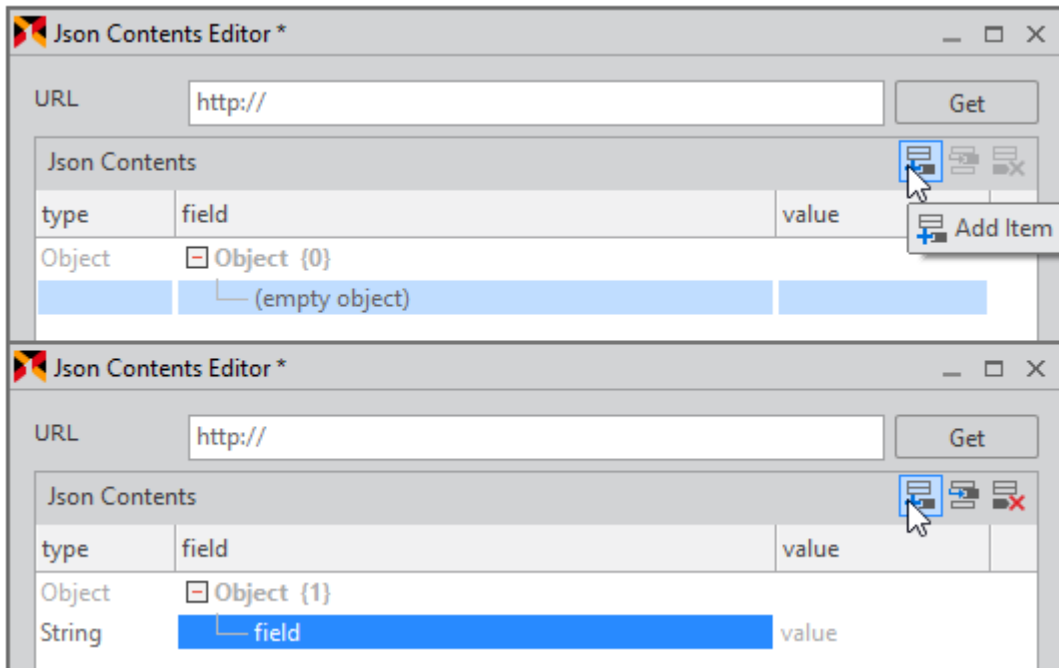
48.3.3 예제 구현 방법

JSON 데이터 작성하기

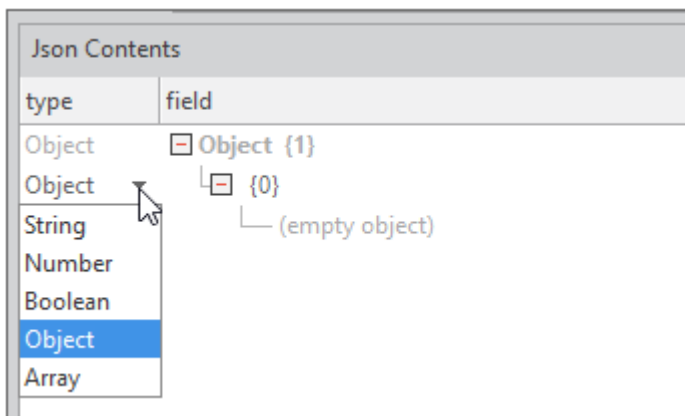
- ① DataObject 오브젝트를 추가합니다.
- ② data 속성 옆에 있는 버튼을 클릭하고 [Json Contents Editor]를 실행합니다.



- 3 (empty object)로 표시된 항목을 선택하고 [Add Item] 버튼을 클릭하면 항목이 추가되며 설정할 수 있는 상태가 됩니다.



- 4 type 항목을 Object 또는 Array로 변경하면 하위 아이টে을 추가할 수 있습니다.



- 5 Source 탭을 클릭하면 JSON 코드값을 직접 입력할 수 있습니다.

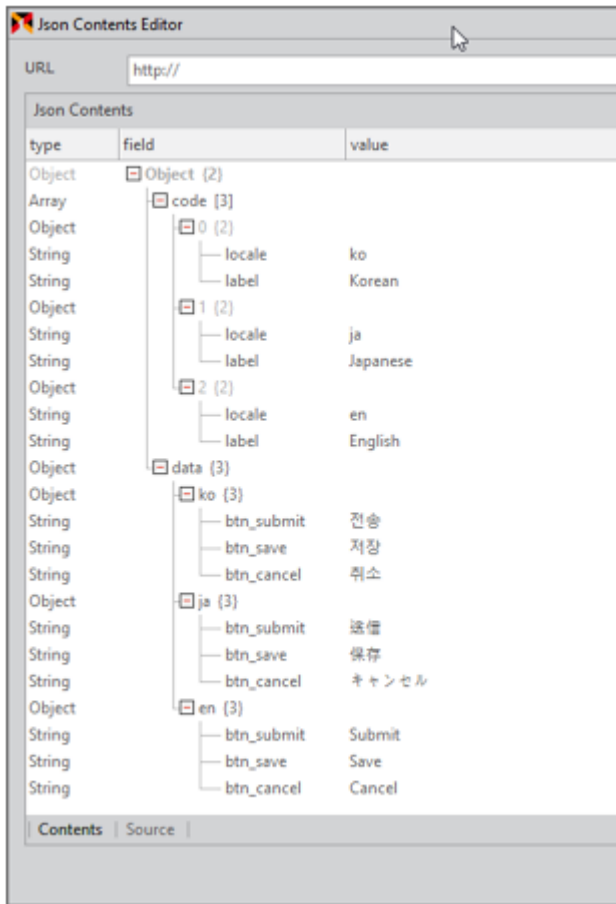
예제에서는 아래와 같은 코드를 입력합니다.

```

1  {
2    "code" : [
3      {
4        "locale" : "ko",
5        "label" : "Korean"
6      },
7      {
8        "locale" : "ja",
9        "label" : "Japanese"
10     },
11     {
12       "locale" : "en",
13       "label" : "English"
14     }
15   ],
16   "data" : {
17     "ko" : {
18       "btn_submit" : "전송",
19       "btn_save" : "저장",
20       "btn_cancel" : "취소"
21     },
22     "ja" : {
23       "btn_submit" : "送信",
24       "btn_save" : "保存",
25       "btn_cancel" : "キャンセル"
26     },
27     "en" : {
28       "btn_submit" : "Submit",
29       "btn_save" : "Save",
30       "btn_cancel" : "Cancel"
31     }
32   }
33 }

```

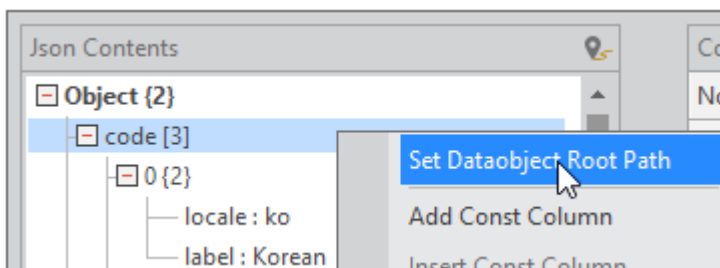
Contents 탭에서는 아래와 같이 표시됩니다.



Combo 컴포넌트에 표시할 기준 데이터 가져오기

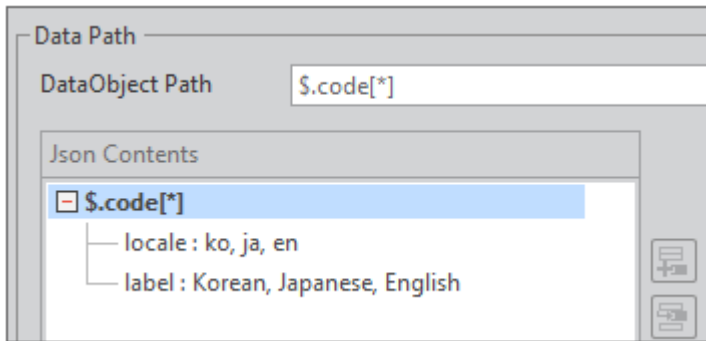
- ① Dataset 오브젝트를 추가하고 binddataobject 속성값을 "DataObject00"으로 지정합니다.
- ② 속성창에서 dataobjectpath 속성 옆에 있는 버튼을 클릭하고 [Data Path Editor]를 실행합니다.
- ③ 데이터에서 "code [3]" 항목을 선택하고 오른쪽 마우스 클릭 후 [Set Dataobject Root Path] 항목을 선택합니다.

데이터 중에서 code 배열 아래의 오브젝트 값을 Combo 컴포넌트의 innerdataset으로 사용할 겁니다.

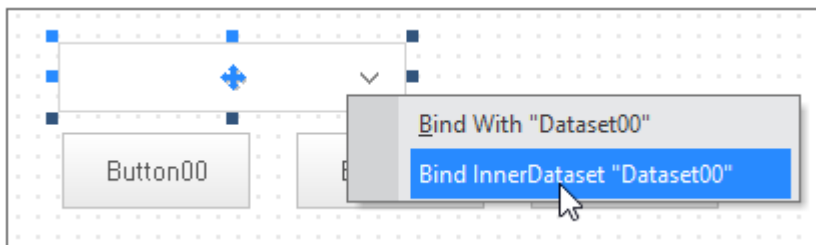


- ④ DataObject Path 값이 "\$.code[*]"로 설정되면 [Refresh json contents] 버튼을 클릭하고 [Filtered Conte

nts] 탭에 아래와 같이 데이터가 표시되는 것을 확인합니다.



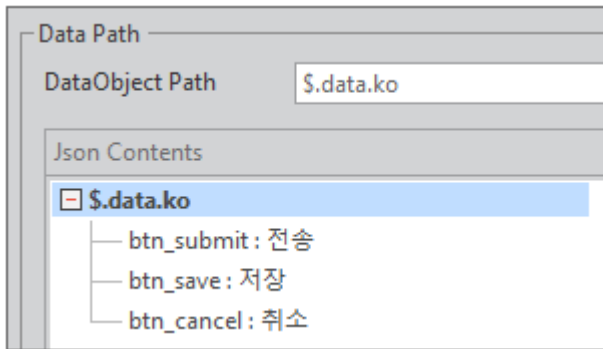
- ⑤ locale, label 항목을 선택하고 컬럼으로 추가한 후 에디터를 닫습니다.
- ⑥ Combo 컴포넌트를 추가합니다.
- ⑦ 앞에서 설정한 Dataset 오브젝트를 Combo 컴포넌트의 InnerDataset으로 설정합니다.
codecolumn 속성값은 "locale", datacolumn 속성값은 "label"로 설정합니다.



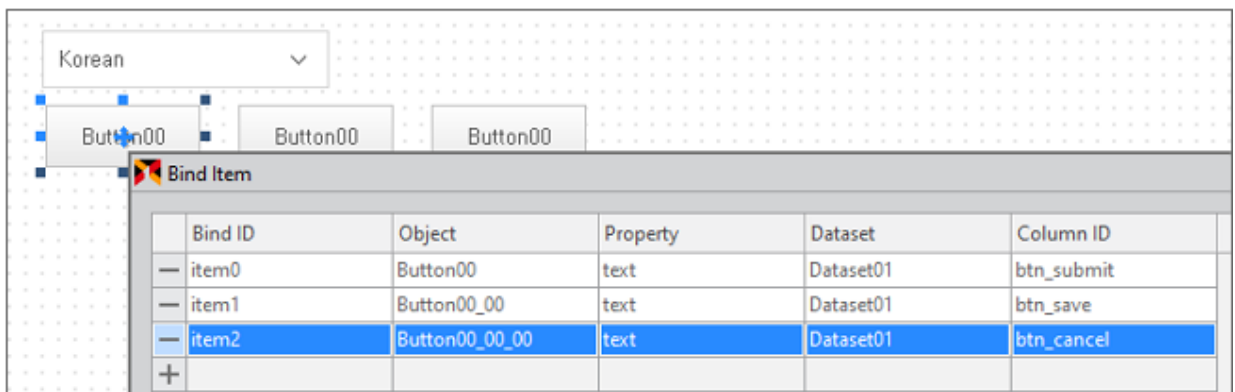
- ⑧ QuickView(Ctrl + F6)로 실행한 후 Combo 컴포넌트의 데이터가 추가됐는지 확인합니다.

Button 컴포넌트에 연결되는 라벨 데이터 가져오기

- ① Dataset 오브젝트를 추가하고 binddataobject 속성값을 "DataObject00"으로 지정합니다.
- ② 속성창에서 dataobjectpath 속성 옆에 있는 버튼을 클릭하고 [Data Path Editor]를 실행합니다.
- ③ 데이터에서 "data {3}" 항목 아래에 있는 "ko {3}" 항목을 선택하고 오른쪽 마우스 클릭 후 [Set Dataobject Root Path] 항목을 선택합니다.
- ④ DataObject Path 값이 "\$.data.ko"로 설정되면 [Refresh json contents] 버튼을 클릭하고 [Filtered Contents] 탭에 아래와 같이 데이터가 표시되는 것을 확인합니다.



- ⑤ btn_submit, btn_save, btn_cancel 항목을 선택하고 컬럼으로 추가한 후 에디터를 닫습니다.
- ⑥ Button 컴포넌트를 3개 추가합니다.
- ⑦ 앞에서 설정한 Dataset 오브젝트를 Button 컴포넌트의 Bind Item으로 설정합니다. text 속성과 각 컬럼을 아래와 같이 연결합니다.



- ⑧ Combo 컴포넌트의 onitemchanged 이벤트 함수를 아래와 같이 작성합니다. 선택한 locale 항목에 따라 Button 컴포넌트에 바인딩된 Dataset 오브젝트의 dataobjectpath 속성값을 변경합니다.

```
this.Combo00_onitemchanged = function(obj:nexacro.Combo,e:nexacro.ItemChangeEventInfo)
{
    this.Dataset01.set_dataobjectpath("$.data."+e.postvalue);
};
```

- ⑨ QuickView(Ctrl + F6)로 실행한 후 Combo 컴포넌트에서 locale 항목 변경 시 Button 컴포넌트에 표시되는 텍스트가 변경되는지 확인합니다.

48.4 메소드 파라미터 지정해서 request 메소드 실행하기

load 메소드는 GET 메소드 파라미터가 설정한 상태로 동작합니다. 하지만 호출하려는 API에 따라서 POST나 PUT 등의 메소드를 설정해야 하는 경우가 있습니다. 또한 인증을 해야 하거나 데이터를 전송하는 경우 postdata를 설정해야 합니다.

48.4.1 예제

값을 입력하고 [Create] 버튼 클릭 시 입력한 값을 설정한 메소드 파라미터로 호출합니다. 정상적으로 실행된 경우 Grid 컴포넌트에 데이터를 표시하고 그렇지 않은 경우에는 TextArea 컴포넌트에 에러 메시지를 표시합니다.



예제 실행을 위해 사용한 서버 연결이 안정적이지 않아 실행할 수 있는 샘플은 제공하지 않습니다. 기능 확인을 위해 가이드 문서만 공개합니다.

샘플 코드는 아래 링크에서 내려받을 수 있습니다.

[sample_dataobject_03.xfdl](#)

48.4.2 예제에서 사용한 핵심 기능

request

메소드 파라미터를 지정할 수 있습니다. header, postdata, async 정보를 저장한 JSON 오브젝트를 추가로 설정할 수 있습니다. 예제에서는 postdata만 전송합니다.

48.4.3 예제 구현 방법

화면 구성하기

- ① Edit 컴포넌트와 Grid 컴포넌트, Button 컴포넌트, Dataset 오브젝트, DataObject 오브젝트를 추가합니다.
- ② Edit 컴포넌트의 displaynulltext 속성값을 설정합니다. 메소드 파라미터를 입력하는 Edit 컴포넌트는 value 속성값을 "POST"로 설정합니다.

request 메소드 준비하기

- ① [Create] Button 컴포넌트의 onclick 이벤트 함수를 아래와 같이 작성합니다.
[Create] 버튼 클릭 시에는 입력한 값과 메소드 파라미터를 설정해서 처리합니다.


```

this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var strPostdata = JSON.stringify({"name":this.Edit_name.value,"salary":this.Edit_
salary.value,"age":this.Edit_age.value});
    this.DataObject00.request("CREATE", this.Edit_method.value, "http://dummy.
restapiexample.com/api/v1/create", {"postdata":strPostdata});
};

```

- ② [Search] Button 컴포넌트의 onclick 이벤트를 함수를 아래와 같이 작성합니다.

[Search] 버튼 클릭시에는 "GET" 메소드 파라미터값을 설정합니다.

```

this.Button00_00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.DataObject00.request("SEARCH", "GET", "http://dummy.restapiexample.com/api/v1/
employees");
};

```

이벤트 처리하기

request 메소드 실행 시에는 onsuccess 이벤트가 먼저 발생하고 그 다음에 data 속성값에 데이터가 설정되고 onload 이벤트가 발생합니다.

- ① onsuccess 이벤트 함수를 아래와 같이 작성합니다.

onsuccess 이벤트 함수 내에서는 실행된 serviceid 값을 확인하고 변수에 저장합니다.

```

var serviceid;
this.DataObject00_onsuccess = function(obj:nexacro.DataObject,e:nexacro.
DataObjectEventInfo)
{
    var strStatus = JSON.parse(e.response).status;
    serviceid = e.serviceid;
    trace(e.serviceid + ":" + strStatus);
};

```

- ② onload 이벤트 함수를 아래와 같이 작성합니다.

onload 이벤트 함수 내에서는 바인딩할 Dataset 오브젝트의 컬럼 정보를 생성하고 binddataobject, dataobjectpath 속성을 설정합니다. Grid 컴포넌트의 binddataset 속성값을 설정하고 createFormat 메소드를 실행

합니다.

```
this.DataObject00_onload = function(obj:nexacro.DataObject,e:nexacro.
DataObjectLoadEventInfo)
{
    if(e.reason == 2)
    {
        this.Dataset00.clear();
        var objData;
        var strDataobjectpath;
        if(serviceid == "CREATE")
        {
            objData = obj.data.data;
            strDataobjectpath = "$.data";
        }
        else if(serviceid == "SEARCH")
        {
            objData = obj.data.data[0];
            strDataobjectpath = "$.data[*]";
        }
        for(var key in objData)
        {
            var objColinfo = new ColumnInfo();
            objColinfo.set_type("String");
            objColinfo.set_datapath("@."+key);
            this.Dataset00.addColumnInfo(key, objColinfo);
        }
        this.Dataset00.set_binddataobject(obj.id);
        this.Dataset00.set_dataobjectpath(strDataobjectpath);
        this.Grid00.set_binddataset(this.Dataset00.name);
        this.Grid00.createFormat();
        this.TextArea00.set_value("");
    }
};
```

- ③ onerror 이벤트 함수를 아래와 같이 작성합니다.

오류가 발생한 경우에는 onerror 이벤트 함수에서 예러 메시지를 TextArea 컴포넌트에 표시해줍니다.

```

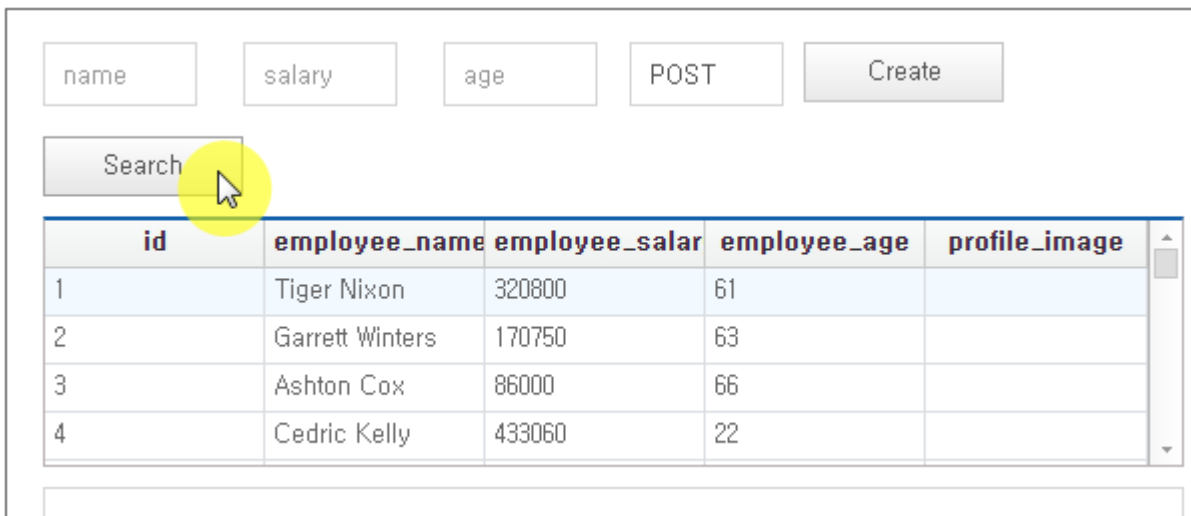
this.DataObject00_onerror = function(obj:nexacro.DataObject,e:nexacro.
DataObjectErrorEventInfo)
{
    trace(e.statuscode + ":" + e.errormsg);
    this.TextArea00.set_value(e.statuscode + ":" + e.errormsg);
};

```

실행 결과 확인하기

QuickView(Ctrl + F6)로 실행한 후 실행 결과를 확인합니다.

[Search] 버튼 클릭 시에는 전체 데이터를 조회합니다. [간단한 데이터 표시하기](#) 예제와 같은 결과지만, 이번 예제에서는 request 메소드를 사용했고, Dataset 오브젝트와 Grid 컴포넌트 설정을 동적으로 처리했습니다.



id	employee_name	employee_salar	employee_age	profile_image
1	Tiger Nixon	320800	61	
2	Garrett Winters	170750	63	
3	Ashton Cox	86000	66	
4	Cedric Kelly	433060	22	

[Create] 버튼 클릭 시에는 입력한 값과 메소드 파라미터를 사용합니다. 데이터를 요청하면 생성된 id 값을 포함한 결과값을 반환합니다.



name	salary	age	id
TEST	45000	12	65

[Create] 버튼 클릭 시 잘못된 메소드 파라미터를 입력하면 에러 처리됩니다. 요청하는 서비스는 Create 요청 시에는 "POST" 메소드 파라미터만 허용하기 때문에 오류로 처리됩니다.

TEST

12312

12312

GET

Create

Search

405:[http://dummy.restapiexample.com/api/v1/create] loading has been failed due to following status code [405]

파트 VII.

넥사크로 오브젝트

49.

nexacro.Decimal

49.1 nexacro.Decimal 오브젝트 소개

자바스크립트 Number 타입의 안정적인 지원 범위를 벗어나는 데이터를 처리하기 위해 제공하는 오브젝트입니다.

자바스크립트에서는 $-(2^{53}-1)$ 부터 $(2^{53}-1)$ 까지의 값을 처리할 수 있습니다. 이 범위를 벗어나게 되면 잘못된 값이 표시될 수 있고, 연산 시 정확한 값을 얻을 수 없습니다. 약 9000조 정도의 값인데 일반적인 업무에서는 사용할 일이 없을 수 있으나, 금융이나 통계 등에서는 발생할 수 있는 수치입니다. 예를 들어 2018년 대한민국 국민순 자산은 약 1경 5511조 7000억원으로 추산하고 있습니다(출처: 통계청 2018 국민대차대조표). 보도자료에서는 뒷자리 단위를 잘라서 "천억 단위"와 같은 식으로 사용하기 때문에 계산이 가능하지만, 실제 값을 다루어야 하는 경우에는 자바스크립트에서 제공하는 지원 범위보다 큰 값을 다루어야 합니다.

nexacro.Decimal 오브젝트는 24자리 숫자(9999해)까지 처리할 수 있습니다.

일부 웹브라우저에서 자바스크립트 내장 객체인 BigInt 오브젝트를 사용할 수 있습니다. BigInt 오브젝트와 nexacro.Decimal 오브젝트의 지원 범위는 아래 표를 참고하세요 (2019년 12월 기준)

	IE8	IE9	IE10	IE11	Edge	Chrome	Safari	Firefox	Opera
nexacro.Decimal	O	O	O	O	O	O	O	O	O
BigInt	X	X	X	X	X	O	X	O	O

49.2 Grid 컴포넌트에서 BIGDECIMAL 타입 사용하기

Dataset 오브젝트에서 컬럼 설정 시 type 값을 "BIGDECIMAL"로 지정하고 Dataset을 바인딩하면 설정된 값을 nexacro.Decimal 오브젝트로 처리합니다.

49.2.1 예제

19자리 숫자값을 다른 type의 Dataset 컬럼에서 어떻게 다루는지 확인합니다. 데이터를 표시하는 방식과 합계를 표시하는 방식을 확인할 수 있습니다.

type이 INT인 경우에는 전혀 다른 값을 표시하고, FLOAT인 경우에는 근사값으로 표시합니다. BIGDECIMAL인 경우에만 입력한 값을 정확하게 표시하고 합계값도 원하는 값을 반환합니다.

INT	FLOAT	BIGDECIMAL
2,112,454,912	1,234,567,890,123,456,800	1,234,567,890,123,456,789
2,112,454,912	1,234,567,890,123,456,800	1,234,567,890,123,456,789
4,224,909,824	2,469,135,780,246,913,500	2,469,135,780,246,913,578

 [sample_nexacro_decimal_01.xfdl](#)

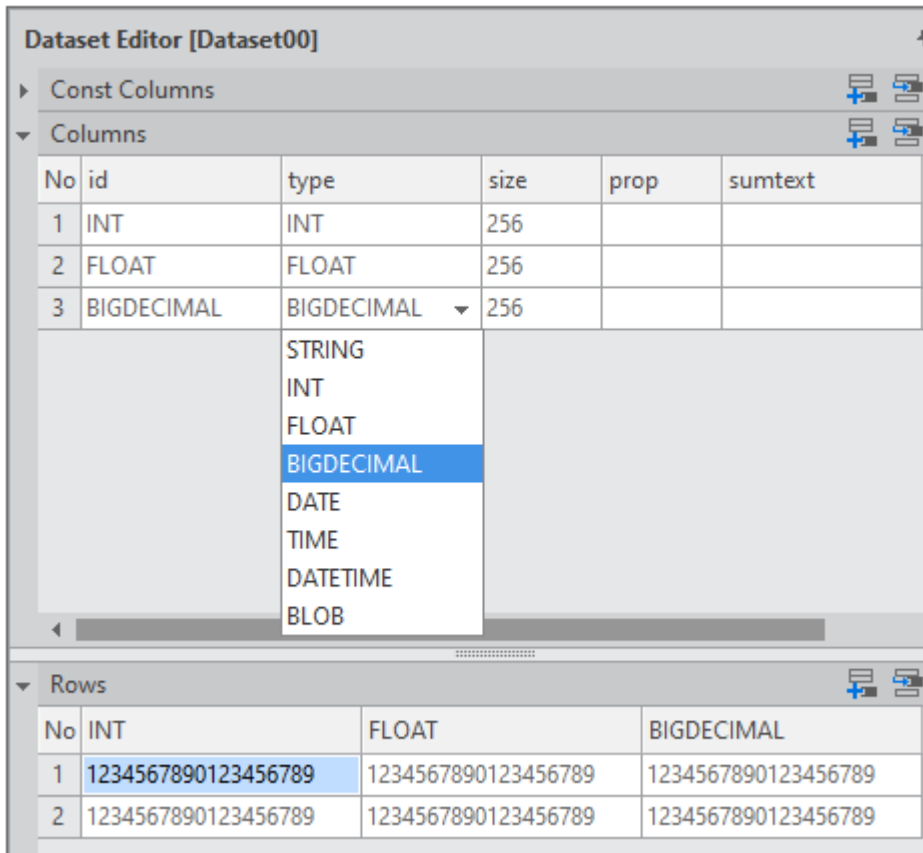
49.2.2 예제에서 사용한 핵심 기능

getSum

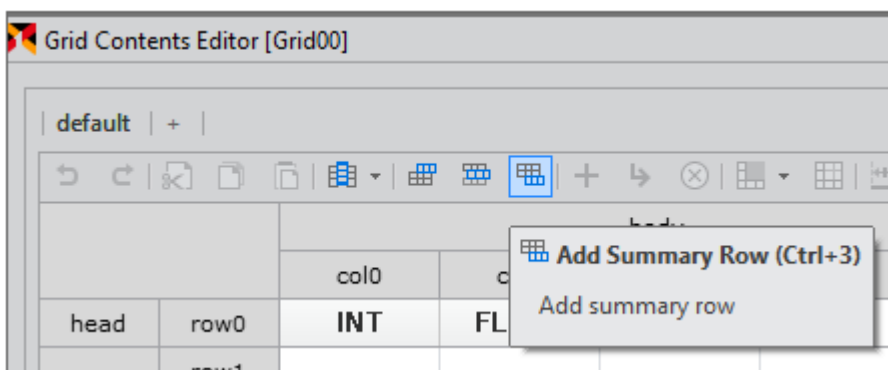
컬럼 ID 값을 지정해 합계값을 반환합니다.

49.2.3 예제 구현 방법

- 1 Grid 컴포넌트를 화면에 배치합니다.
- 2 Dataset 오브젝트를 추가하고 아래와 같이 컬럼 형식과 데이터를 설정합니다.
3개의 컬럼을 추가하고 각 컬럼 type은 INT, FLOAT, BIGDECIMAL로 설정합니다. 데이터는 19자리 숫자값을 같은 값으로 지정합니다.



- ③ Dataset 오브젝트를 Grid 컴포넌트에 바인딩합니다.
- ④ Grid Contents Editor를 실행하고 Summary Row를 추가합니다.



- ⑤ Summary Row의 셀 속성 중에서 displaytype 속성값을 "number"로 설정하고 expr 속성값은 "dataset.get Sum('[Column name]')"으로 설정합니다.

body				displaytype	number
				edittype	none
				expr	dataset.getSum('INT')
				subsumtext	
				suppress	0
				suppressalign	first
summary	row2			text	
				tooltiptext	

6 QuickView(Ctrl + F6)로 실행한 후 Dataset 오브젝트의 type에 따라 값이 어떻게 표시되는지 확인합니다.

49.3 1경(京)보다 큰 값 연산하기

nexacro.Decimal 오브젝트는 숫자가 아닌 오브젝트 형태로 처리합니다. 때문에 일반적인 연산기호를 사용할 수 없으며 연산을 처리하기 위한 별도의 메소드나 속성을 지원합니다.

49.3.1 예제

Edit 컴포넌트에 숫자를 입력하고 연산자를 선택한 후 [Calculate] 버튼을 클릭하면 연산을 처리합니다.

 입력값 또는 연산결과값이 24자리 숫자보다 큰 경우에는 정상적으로 연산 처리가 되지 않습니다.

1234567890123456789

☐ add ☐ sub ☒ mul ☐ div

3

Calculate

3703703670370370367



sample_nexacro_decimal_02.xfdl

49.3.2 예제에서 사용한 핵심 기능

addDecimal, subDecimal, mulDecimal, divDecimal

nexacro.Decimal 오브젝트 연산을 위한 메소드입니다. 연산 결과값을 반환하지 않고 메소드를 실행한 nexacro.Decimal 오브젝트에 연산 결과값이 반영됩니다.

49.3.3 예제 구현 방법

- ① Edit, Radio, Button 컴포넌트를 화면에 배치합니다.
- ② Radio 컴포넌트의 innerdataset 값으로 add, sub, mul, div 값을 설정합니다.
- ③ Button 컴포넌트의 onclick 이벤트 함수를 아래와 같이 작성합니다.

```
this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var a = nexacro.Decimal(this.Edit00.value.toString());
    var b = nexacro.Decimal(this.Edit01.value.toString());

    switch(this.Radio00.value) {
    case 'add':
        a.addDecimal(b);
        break;
    case 'sub':
        a.subDecimal(b);
        break;
    case 'mul':
        a.mulDecimal(b);
        break;
    case 'div':
        a.divDecimal(b);
        break;
    default:
    }
}
```

```
this.Edit02.set_value(a.toString());
};
```

Decimal 오브젝트 설정 시에는 문자열 형태로 값을 지정해주어야 합니다.

```
(O) var a = nexacro.Decimal(this.Edit00.value.toString());
(X) var a = nexacro.Decimal(this.Edit00.value.toNumber());
(O) var a = nexacro.Decimal("1234567890123456789");
(X) var a = nexacro.Decimal(1234567890123456789);
```

연산 처리된 값을 출력하는 경우에도 문자열로 바꾸어서 출력해주어야 합니다. Edit 컴포넌트의 value 값으로 설정한 경우에는 내부 처리 시 문자열로 변환해주기 때문에 오류는 발생하지 않지만, 문자열로 바꾸어서 처리해주는 것이 안전합니다.

```
(O) this.Edit02.set_value(a.toString());
(Δ) this.Edit02.set_value(a);
```

- ④ QuickView(Ctrl + F6)로 실행한 후 입력값의 연산이 처리되는지 확인합니다.

50.

nexacro.DomParser

50.1 nexacro.DomParser 오브젝트 소개

XML 문자열을 DOMDocument 오브젝트로 생성합니다. XML 데이터를 받아서 다른 형식으로 표현해야 하는 경우 사용할 수 있습니다.



nexacro.DomParser 오브젝트는 17.0.0.2200 버전부터 지원합니다.

50.2 XML 형식의 Open API 연결해서 Grid 컴포넌트에 표시하기

도시 별 날씨 정보를 제공하는 Open API를 이용해 선택한 도시의 5일간 날씨 정보를 표시합니다.



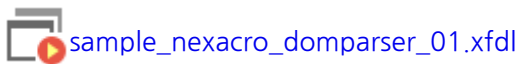
날씨 Open API는 openweathermap 에서 제공하는 정보를 활용했습니다.
데이터 형식과 기타 자세한 내용은 아래 링크를 참고하세요.

<https://openweathermap.org/forecast5#XML>

50.2.1 예제

도시 코드를 미리 설정해놓은 목록 중에서 원하는 항목을 선택하면 현재 시간 이후 5일간의 날씨 정보를 표시합니다.
선택한 국가에 따라 한국어, 일본어, 영어로 날씨 상태 설명을 툴팁으로 표시합니다.

country	city	id	date	temperature	pressure	humidity	icon
KR	Seoul	1835848	2019/12/6 12:00:00	11.17	1,027	36	
KR	Incheon	1843564	2019/12/6 15:00:00	10.72	1,027	32	
JP	Tokyo	1850147	2019/12/6 18:00:00	9.07	1,028	34	
JP	Nagoya	1856057	2019/12/6 21:00:00	7.78	1,028	38	
US	New York	5128581	2019/12/7 0:00:00	6.41	1,027	45	
US	Hawaii	5855797	2019/12/7 3:00:00	6.17	1,026	40	
			2019/12/7 6:00:00	5.94	1,026	42	
			2019/12/7 9:00:00	6	1,026	44	
			2019/12/7 12:00:00	7.36	1,024	43	
			2019/12/7 15:00:00	6.56	1,022	54	
			2019/12/7 18:00:00	6.47	1,022	58	



50.2.2 예제에서 사용한 핵심 기능

onreadystatechange

XMLHttpRequest 오브젝트에서 요청한 호출의 응답 상태에 따라 결과를 처리할 콜백 함수를 설정합니다. 예제에서는 날씨 API 주소를 호출하고 응답받은 XML 값을 nexacro.DomParser 오브젝트에서 처리합니다.

parseFromString

XML 문자열을 DOMDocument 오브젝트 형태로 변환해줍니다. 오브젝트의 속성이나 메소드는 실행 환경에 따라 달라질 수 있습니다. 아래 예제는 NRE, IE11, Chrome에서 확인한 기능을 사용했습니다.

50.2.3 예제 구현 방법

- 1 Grid 컴포넌트를 화면에 배치합니다.
- 2 첫 번째 Grid 컴포넌트와 바인딩할 Dataset 오브젝트를 추가합니다.
- 3 Dataset 오브젝트에 아래와 같이 데이터를 설정합니다.

Columns					
No	id	type	size	prop	sumtext
1	country	STRING	256		
2	city	STRING	256		
3	id	STRING	256		
4	locale	STRING	256		
5	lang	STRING	256		
.....					
Rows					
No	country	city	id	locale	lang
1	KR	Seoul	1835848	ko-KR	kr
2	KR	Incheon	1843564	ko-KR	kr
3	JP	Tokyo	1850147	ja-JP	ja
4	JP	Nagoya	1856057	ja-JP	ja
5	US	New York	5128581	en-US	en
6	US	Hawaii	5855797	en-US	en



사용할 수 있는 도시 정보는 아래 링크에서 확인할 수 있습니다.

<http://bulk.openweathermap.org/sample/>

- ④ 첫 번째 Grid 컴포넌트에 Dataset 오브젝트를 바인딩합니다.
- ⑤ Grid Contents Editor를 실행하고 locale, lang 2개 컬럼은 삭제합니다.
해당 컬럼 데이터는 API 호출 시에만 사용합니다.
- ⑥ 두 번째 Grid 컴포넌트와 바인딩할 Dataset 오브젝트를 추가합니다.
- ⑦ Dataset 오브젝트에 아래와 같이 컬럼 정보만 설정합니다.
데이터는 API 호출 후 처리 과정에서 추가합니다.

Columns				
No	id	type	size	prop
1	date	STRING	256	
2	status	STRING	256	
3	temperature	STRING	256	
4	pressure	STRING	256	
5	humidity	STRING	256	
6	icon	STRING	256	

⑧ 두 번째 Grid 컴포넌트에 Dataset 오브젝트를 바인딩합니다.

⑨ Grid Contents Editor를 실행합니다.

temperature, pressure, humidity 3개의 컬럼은 displaytype 속성값을 "number"로 수정하고 icon 컬럼은 displaytype 속성값은 "imagecontrol", tooltiptext 속성값은 "bind:status", imagestretch 속성값은 "fixaspectratio"로 설정합니다.

⑩ 첫 번째 Grid 클릭 시 API를 호출하는 스크립트를 아래와 같이 작성합니다.

```
this.Grid01_onclick = function(obj:nexacro.Grid,e:nexacro.GridClickEventInfo)
{
    var id = this.Dataset01.getColumn(e.row, 2);
    var lang = this.Dataset01.getColumn(e.row, 4);
    this.fn_getWeatherInfo(id, lang);
};
```

⑪ ID값을 받아서 API를 호출하는 함수를 아래와 같이 작성합니다.

mode 속성값을 지정하지 않으면 JSON 형식으로 데이터를 받습니다. 예제에서는 XML 데이터를 다루기 때문에 mode 속성값을 "xml"로 지정합니다. lang 속성을 설정하면 해당 언어로 데이터값을 받을 수 있습니다.

```
var appid = "[app id]";

this.fn_getWeatherInfo = function(id, lang)
{
    var objXhr = new XMLHttpRequest();
    objXhr.onreadystatechange = this.XHR_onreadystatechange;
    objXhr.open("POST", "https://api.openweathermap.org/data/2.5/forecast?mode=xml&units=metric&appid="+appid+"&id="+id+"&lang="+lang);
    objXhr.send();
};
```



API KEY(appid)는 회원 가입 후 받을 수 있습니다.

<https://openweathermap.org/appid>

⑫ API 호출 후 응답을 처리하는 함수를 아래와 같이 작성합니다.

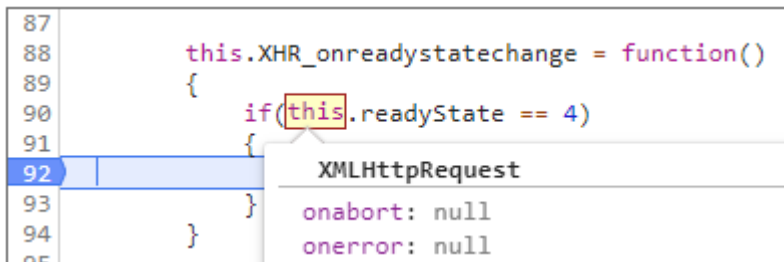
```

var that = this;

this.XHR_onreadystatechange = function()
{
    if(this.readyState == 4)
    {
        that.fn_setData(this.responseText);
    }
}

```

데이터를 처리하기 위한 함수를 호출하기 위해 this(Form) 변수를 전역 변수로 지정해 사용합니다. 아래 함수 내에서 this는 XMLHttpRequest 오브젝트를 가리킵니다.



- ⑬ 날씨 정보를 표시할 Dataset 오브젝트에 데이터를 추가하기 위한 함수를 아래와 같이 작성합니다.

DomParser 오브젝트의 parseFromString 메소드를 사용해 문자열 형태로 전달된 XML 본문을 오브젝트 형태로 변환하고 필요한 데이터에 접근합니다.

```

this.fn_setData = function(strXML)
{
    var domDoc;
    var objDom = new nexacro.DomParser();
    domDoc = objDom.parseFromString(strXML);

    var timezone = domDoc.getElementsByTagName("timezone")[0].childNodes[0]
    timezone = timezone.data;
    var domElement = domDoc.getElementsByTagName("forecast")[0].childNodes;
    var domCnt = domElement.length;
    var msg="";

    this.Dataset00.set_enableevent(false);
    this.Dataset00.clearData();
    for(var i=0;i<domCnt;i++)
    {

```



```

var rIdx = this.Dataset00.addRow();
var domElement = domDoc.getElementsByTagName("forecast")[0].childNodes[i];

this.Dataset00.setColumn(rIdx, 0, this.fn_localDate(domElement.getAttribute("to"),
timezone)); // to date
this.Dataset00.setColumn(rIdx, 1, domElement.getElementsByTagName("symbol")[0].
getAttribute("name")); // status
this.Dataset00.setColumn(rIdx, 2, domElement.getElementsByTagName("temperature")[0].
getAttribute("value")); // temperature
this.Dataset00.setColumn(rIdx, 3, domElement.getElementsByTagName("pressure")[0].
getAttribute("value")); // pressure
this.Dataset00.setColumn(rIdx, 4, domElement.getElementsByTagName("humidity")[0].
getAttribute("value")); // humidity
this.Dataset00.setColumn(rIdx, 5, "http://openweathermap.org/img/wn/"+domElement.
getElementsByTagName("symbol")[0].getAttribute("var")+ ".png");
}
this.Dataset00.set_enableevent(true);
this.Grid00.scrollTo(0,0);

};

```

14 날짜값 처리를 위한 함수를 작성합니다.

API에서는 UTC 기준으로 날짜값을 던져줍니다. 선택한 도시의 타임존에 맞게 시간을 보정해주어야 합니다.

```

this.fn_localDate = function(strDate, timezone)
{
    var d = new nexacro.Date(strDate);
    d.addSeconds(Number(timezone));
    var year = d.getFullYear().toString().padLeft(4, "0");
    var month = (d.getMonth()+1).toString().padLeft(2, "0");
    var day = d.getDate().toString().padLeft(2, "0");
    var hour = d.getHours().toString().padLeft(2, "0");
    var minute = d.getMinutes().toString().padLeft(2, "0");
    return year+"/"+month+"/"+day+" "+hour+":"+minute;
}

```

15 QuickView(Ctrl + F6)로 실행한 후 도시를 선택했을때 날씨 정보가 표시되는지 확인합니다.

파트 VIII.

자바스크립트 표준 내장 객체

51.

Array

51.1 문자열을 분리해 배열로 만들기

입력된 문자열을 지정한 길이만큼 잘라 배열로 만듭니다.

51.1.1 예제

Edit 컴포넌트에 입력된 값을 지정한 길이만큼 잘라 배열로 반환합니다.



51.1.2 예제에서 사용한 핵심 기능

push

배열에 마지막 값을 추가하는 메소드입니다.

51.1.3 예제 구현 방법

① Form 화면 구성하기

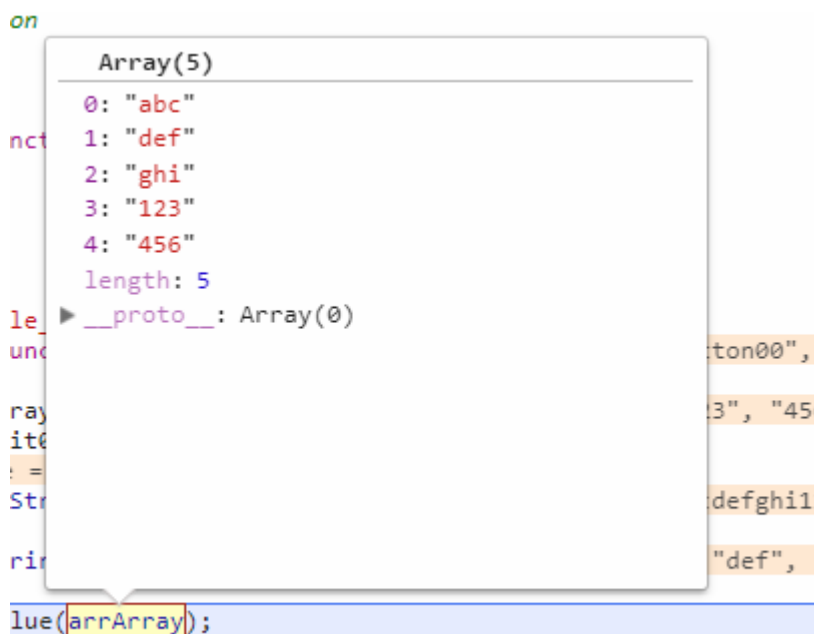
Edit 컴포넌트, Button 컴포넌트를 배치합니다.

② onclick 이벤트 함수 작성하기

버튼 클릭 시 배열을 생성하고 입력된 문자열 길이만큼 반복하면서 배열에 문자열을 잘라서 추가합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var arrArray = new Array();
    var sString = this.Edit00.value;
    var nSize = 3;
    for (var i = 0; i < sString.length; i+=nSize)
    {
        arrArray.push(sString.substr(i, nSize));
    }
    this.Edit01.set_value(arrArray);
};
```

예제의 경우 실제 배열에는 아래와 같이 값이 들어갑니다.



3 QuickView로 확인하기

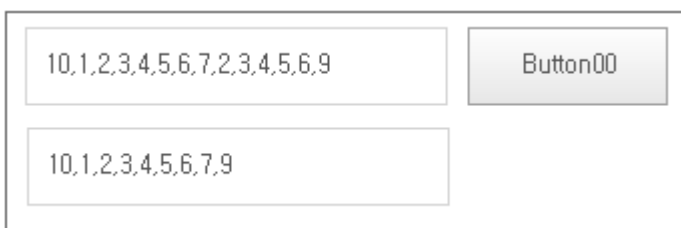
QuickView(Ctrl + F6)로 실행한 후 값을 입력해 확인해봅니다.

51.2 중복된 요소 제거하기

입력된 문자열을 배열로 바꾸고 중복된 요소를 제거합니다.

51.2.1 예제

Edit 컴포넌트에 입력된 값을 콤마(,)를 기준으로 잘라서 배열로 변환하고 중복된 요소를 제거한 결과를 반환합니다.



 [sample_array_02.xfdl](#)

51.2.2 예제에서 사용한 핵심 기능

`split`

문자열을 구분자에 따라 분할해 배열로 반환합니다.

51.2.3 예제 구현 방법

1 Form 화면 구성하기

Edit 컴포넌트, Button 컴포넌트를 배치합니다.

2 onclick 이벤트 함수 작성하기

버튼 클릭 시 배열을 생성하고 입력된 문자열 길이만큼 반복하면서 배열에 문자열을 잘라서 추가합니다.

```

this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var aRtn = this.fn_distinct(this.Edit00.value);
    this.Edit01.set_value(aRtn);
};

this.fn_distinct = function(sParam)
{
    var aData = new Array();
    aData = sParam.split(",");

    var aRtn = new Array();
    var bFlag;
    var vDist;

    for (var i = 0; i < aData.length; i++)
    {
        vDist = aData[i];
        bFlag = false;
        for (var j = 0; j < aRtn.length; j++)
        {
            if (aRtn[j] == vDist)
            {
                bFlag = true;
                break;
            }
        }
        if (bFlag == false)
        {
            aRtn[aRtn.length] = vDist;
        }
    }
    return aRtn;
}

```



예제에서는 배열에 담긴 모든 요소를 하나하나 비교해가며 중복된 값을 제거했습니다. 배열이 크지 않은 경우에는 큰 차이가 없지만, 크기가 큰 경우에는 배열을 먼저 정렬한 다음 중복된 값을 제거하는 것이 더 빠를 수도 있습니다.

3 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 값을 입력해 확인해봅니다.

51.3 숫자 배열 정렬하기

입력된 문자열을 배열로 바꾸고 정렬한 값을 반환합니다.

51.3.1 예제

Edit 컴포넌트에 입력된 값을 콤마(,)를 기준으로 잘라서 배열로 변환하고 문자열 또는 숫자 기준으로 정렬한 값을 표시합니다.



The image shows a UI component with three text input fields stacked vertically. The first field contains the text "1,2,300,4,5". The second field also contains "1,2,300,4,5". The third field contains "1,2,4,5,300". To the right of these fields is a button labeled "Button00".

 [sample_array_03.xfdl](#)

51.3.2 예제에서 사용한 핵심 기능

sort

정렬된 새로운 배열을 반환합니다. 파라미터에 함수를 지정하면 정렬 순서를 원하는 형식으로 변경할 수 있습니다.

51.3.3 예제 구현 방법

① Form 화면 구성하기

Edit 컴포넌트, Button 컴포넌트를 배치합니다.

② onclick 이벤트 함수 작성하기

버튼 클릭 시 배열을 생성하고 입력된 문자열 길이만큼 반복하면서 배열에 문자열을 잘라서 추가합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var arrNo = this.Edit00.value.split(",");
    this.fn_arrayNoSort(arrNo);
    this.fn_arrayStringSort(arrNo);
};

this.fn_arrayStringSort = function(arrNo)
{
    this.Edit01.set_value(arrNo.sort());
};

this.fn_arrayNoSort = function(arrNo)
{
    this.Edit02.set_value(arrNo.sort(function(a, b){return a-b}));
};
```

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 값을 입력해 확인해봅니다.

51.4 2차원 배열 만들기

2차원 배열과 유사한 형식을 구현할 수 있습니다. 정확하게는 배열 안에 배열 요소를 배치하는 방식입니다.

51.4.1 예제

생성할 2차원 배열의 행, 열 갯수를 지정하면 2차원 배열 형태를 생성해서 TextArea 컴포넌트에 표시합니다.



51.4.2 예제에서 사용한 핵심 기능

`new Array()`

`new Array()`로 Array 오브젝트를 생성하지 않고 []를 대입해주어도 Array 오브젝트를 생성합니다.

51.4.3 예제 구현 방법

① Form 화면 구성하기

Edit 컴포넌트, Button 컴포넌트를 배치합니다.

② onclick 이벤트 함수 작성하기

버튼 클릭 시 배열을 생성하고 입력된 문자열 길이만큼 반복하면서 2차원 배열 형식을 만듭니다.

```
this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var arrValue = [];
    var nLen1 = this.Edit00.value;
    var nLen2 = this.Edit01.value;
    for (var i = 0; i < nLen1; i++)
    {
        arrValue[i] = new Array();
```

```
        for (var j = 0; j < nLen2; j++)
        {
            arrValue[i][j] = " [" + i + "][" + j + "];"
        }
    }
    var sRtn = "";
    for (var i = 0; i < nLen1; i++)
    {
        sRtn += arrValue[i] + "\n";
    }
    this.TextArea00.set_value(sRtn);
};
```

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 값을 입력해 확인해봅니다.

52.

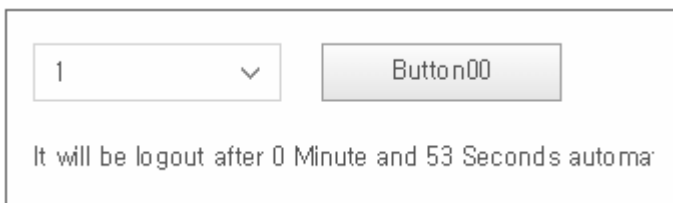
Date

52.1 타이머 구현하기

시간을 선택하고 지정된 시간이 지나면 알람을 표시하는 것이 타이머의 기본 기능입니다. 예제에서는 간단한 타이머 동작 방식을 구현해봅니다.

52.1.1 예제

Combo 컴포넌트에서 시간을 선택하고 버튼을 클릭하면 타이머가 동작합니다.



52.1.2 예제에서 사용한 핵심 기능

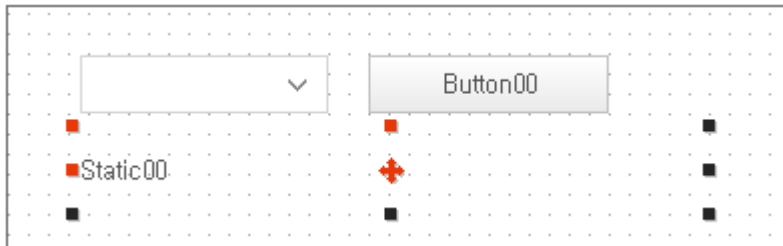
Date

날짜나 시간값을 처리하기 위해 Date 오브젝트를 생성하고 시간 표기를 처리합니다.

52.1.3 예제 구현 방법

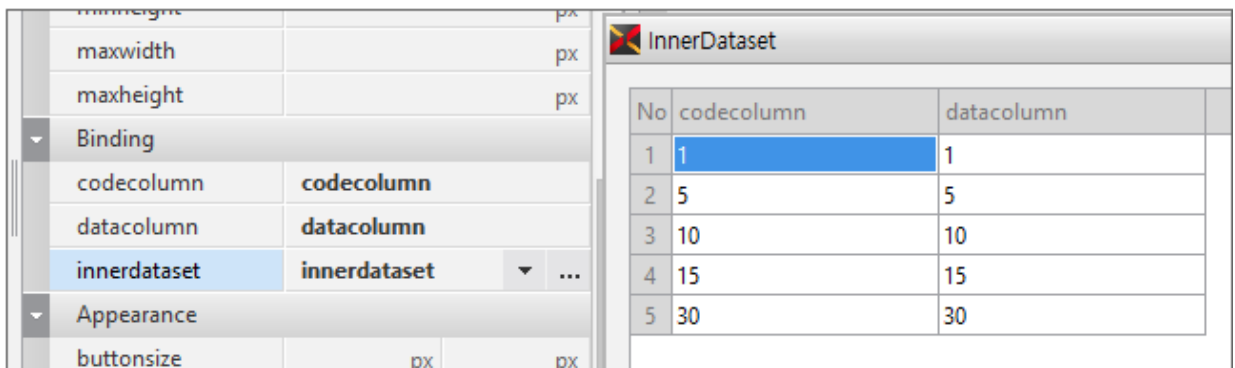
1 화면 구성하기

화면에 Combo, Button, Static 컴포넌트를 배치합니다. Static 컴포넌트는 타이머 시간 표시를 담당합니다.



2 Combo 컴포넌트 innerdataset 설정하기

Combo 컴포넌트는 타이머 시간을 지정하기 위해 사용합니다. 예제에서는 5개의 보기만을 제공합니다. inner dataset 속성 항목 옆에 [...]으로 표시된 버튼을 클릭하면 InnerDataset을 입력할 수 있는 창이 나타납니다.



3 버튼 onclick 이벤트 함수 작성하기

Combo 컴포넌트에서 선택한 값을 기준으로 타이머가 동작할 시점을 계산합니다. 계산을 위해 밀리세컨드로 값을 변환하고 시간을 표시하기 전에 다시 이를 사용자에게 보여줄 수 있는 시간으로 변환합니다.

```

this.session_Endtime= null;
this.curTime = null;

this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var holding_time = this.Combo00.value*60*1000;
    this.session_time = new Date().valueOf();
    this.session_Endtime = new Date(this.session_time + holding_time);
    var millis = this.session_Endtime- this.session_time;
    var t = this.millisToTime(millis);

```

```

        this.Static00.set_text("It will be logout after " + t[1]+"Minute and "+t[2]+ " Seconds
automatically.");
        this.setTimer(100,1000);
    };

    this.millisToTime = function (millis)
    {
        //Hour
        var hours = Math.floor(millis/(1000*60*60), 0);
        //Minute
        var minutes = Math.floor(millis/(1000*60) % 60, 0);
        //Seconds
        var seconds = Math.floor((millis/1000) % 60, 0);
        return [hours,minutes,seconds];
    };

```

④ Form 오브젝트의 ontimer 이벤트 함수 작성하기

ontimer 이벤트가 발생할때마다 Static 컴포넌트에 표시되는 시간을 변경합니다. 표시할 시간이 더 이상 없으면 (if(t[2] <= 0)) 타이머가 끝났음을 표시합니다.

```

this.sample_date_01_ontimer = function(obj:nexacro.Form,e:nexacro.TimerEventInfo)
{
    if(e.timerid == 100)
    {
        this.session_time = new Date().valueOf();
        var millis = this.session_Endtime- this.session_time;
        var t = this.millisToTime(millis);
        if(t[2] <= 0 )
        {
            this.Static00.set_text("Session is terminated, it is logout automatically. ");
            return;
        }
        this.Static00.set_text("It will be logout after " + t[1]+" Minute and "+t[2]+ "
Seconds automatically.");
    }
};

```

⑤ QuickView로 확인하기

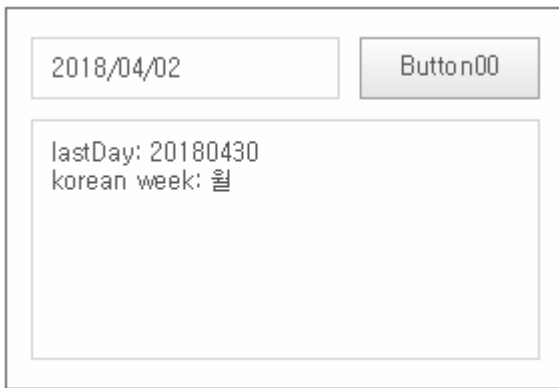
QuickView(Ctrl + F6)를 실행하고 Combo 컴포넌트에서 타이머 시간을 선택해 타이머를 실행해봅니다.

52.2 입력된 날짜에서 정보 구하기

문자열로 입력받은 날짜를 Date 오브젝트로 변환하고 관련된 정보를 추출합니다.

52.2.1 예제

날짜를 입력하고 버튼을 클릭하면 관련 정보를 출력합니다.



 [sample_date_02.xfdl](#)

52.2.2 예제에서 사용한 핵심 기능

addDate

지정한 날짜만큼 더하는 메소드입니다. 음수로 지정하는 경우에는 지정한 날짜만큼 뺄 수 있습니다.

getDay

Date 오브젝트의 날짜에 해당하는 요일값을 반환합니다. 0부터 6까지 인덱스값을 반환하기 때문에 텍스트로 표기할 때는 해당하는 문자열을 매칭해 처리합니다.

52.2.3 예제 구현 방법

① 화면 구성하기

MaskEdit 컴포넌트와 Button 컴포넌트, TextArea 컴포넌트를 배치합니다. MaskEdit 컴포넌트의 format 속성값은 "####/##/##"로 지정하고 type 속성값은 "string"으로 지정합니다.

② onclick 이벤트 함수 작성하기

여러 정보를 처리하고 TextArea 컴포넌트에 표시하기 때문에 정보를 처리하는 함수를 분리해 처리합니다.

```
this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.TextArea00.set_value("lastDay: "+this.fn_lastDay(this.MaskEdit00.value)+"\n"
        +"korean week: "+this.fn_hanGulWeek(this.MaskEdit00.value));
};
```

③ 마지막 날짜 구하기 함수

입력받은 날짜 문자열을 Date 오브젝트에 설정합니다. 입력받은 값이 없는 경우에는 오늘 날짜를 기준으로 처리합니다. Month에 해당하는 값은 0부터 11까지로 처리됩니다. 예를 들어 new Date(2000,11,1)은 2000년 12월 1일입니다.

```
this.fn_lastDay = function(sDate)
{
    var dDate = "";

    if (sDate == null || sDate == "" || sDate == "undefined")
    {
        dDate = (new Date()).addMonth(1);
        dDate.addDate((new Date(dDate)).getDate()*-1);
    }
    else
    {
        dDate = new Date(parseInt(sDate.substr(0,4)), parseInt(sDate.substr(4,2)), 1);
        dDate.addDate(-1);
    }

    var sRtn = dDate.getFullYear()
    + (dDate.getMonth()+1).toString().padLeft(2, "0")
    + dDate.getDate().toString().padLeft(2, "0");

    return sRtn;
};
```

④ 요일 구하기 함수

표시할 요일 문자열을 배열로 만들고 getDay() 메소드에서 구한 값을 배열의 인덱스로 지정합니다.

```
this.fn_hanGulWeek = function(sDate)
{
    var dDate    = new Date(parseInt(sDate.substr(0,4))
                          , parseInt(sDate.substr(4,2))-1
                          , parseInt(sDate.substr(6,2)));

    var sDay     = dDate.getDay();
    var arrweek  = new Array("일", "월", "화", "수", "목", "금", "토");
    var sRtn     = arrweek[sDay];

    return sRtn;
};
```

⑤ QuickView로 확인하기

QuickView(Ctrl + F6)를 실행하고 날짜 문자열을 입력하고 결과값을 확인합니다.

53.

Function

53.1 DECODE, IIF 기능 구현하기

조건에 따라 값을 치환하여 반환하는 방식을 살펴봅니다.

DECODE의 경우에는 아래와 같이 값을 지정합니다. DATA로 전달되는 값에 따라 해당하는 요일값을 반환하는 예제입니다. 함수 내에서 조건식을 정하는 것이 아니라 필요한 조건을 매개변수로 던질 수 있다는 점이 큰 차이입니다. SWITCH 문을 사용하는 것과 비슷합니다.

```
decode(DATA, '0', 'SUN', '1', 'MON', '2', 'TUE', '3', 'WED', '4', 'THU', '5', 'FRI', '6', 'SAT', 'OTHERS');
```

IIF의 경우에는 아래와 같이 값을 지정합니다. true가 되는 조건 자체를 넘기고 그에 따라 결과값을 받습니다.

```
iif(DATA='Y', 'Yes', 'No')
```

53.1.1 예제

첫 번째 Edit 컴포넌트에는 0부터 6까지 숫자 입력 시 해당하는 요일을 반환합니다. 두 번째 Edit 컴포넌트에는 값이 없으면 'No', 값이 있으면 'Yes'를 반환합니다.

1	MON	decode
	No	iif

53.1.2 예제에서 사용한 핵심 기능

arguments

함수 호출 시 전달되는 매개변수를 배열로 처리합니다. decode 함수 같은 경우에는 전달되는 인수의 개수가 정해진 것이 아니기 때문에 arguments 속성을 사용해 넘겨지는 인수의 개수를 확인할 수 있습니다.

53.1.3 예제 구현 방법

① 화면 구성하기

Edit 컴포넌트와 Button 컴포넌트를 배치합니다. Edit 컴포넌트 중에서 결과값을 보여주는 컴포넌트는 enable 속성값을 false로 지정합니다.

② Button 컴포넌트 이벤트 함수 작성하기

예제에서는 첫번째 인자값만 Edit 컴포넌트에서 입력받고 나머지 값은 미리 지정해놓습니다.

```
this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Edit02.set_value(this.fn_decode(this.Edit00.value, '0', 'SUN', '1', 'MON', '2', 'TUE', '3', 'WED', '4', 'THU', '5', 'FRI', '6', 'SAT', 'OTHERS'));
};

this.Button01_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Edit03.set_value(this.fn_iif(this.Edit01.value, 'Yes', 'No'));
};
```

③ fn_decode 함수 작성하기

arguments 속성을 사용해 넘겨받은 매개변수를 확인합니다. 짝수로 입력되었다면 조건식에 지정된 값 외에 기본값이 있다는 의미입니다. 위의 예제에서는 'OTHERS'라는 기본값을 지정했습니다. 그리고 반복문을 사용해 주어진 조건에 해당하는 값을 찾아 반환합니다.

```
this.fn_decode = function()
{
    var sRtnValue = null;
    var arrArgument = this.fn_decode.arguments;
    var sValue = arrArgument[0];
```

```

var bIsDefault = false;
var nCount = 0;

if ((arrArgument.length % 2) == 0)
{
    nCount = arrArgument.length - 1;
    bIsDefault = true;
}
else
{
    nCount = arrArgument.length;
    bIsDefault = false;
}

for (var i = 1; i < nCount; i+=2)
{
    if(sValue == arrArgument[i])
    {
        sRtnValue = arrArgument[i+1];
        i = nCount;
    }
}

if (sRtnValue == null && bIsDefault==true)
{
    sRtnValue = arrArgument[arrArgument.length-1];
}

return sRtnValue;
};

```

④ fn_iif 함수 작성하기

arguments 속성을 사용해 넘겨받은 매개변수를 확인하는 것은 fn_decode 함수와 같습니다. 다만 3개의 값을 넘겨받기 때문에 첫번째 값이 true라면 두번째 값을 반환하고 그렇지 않다면 세번째 값을 반환하는 간단한 기능을 처리합니다.

```

this.fn_iif = function()
{
    var arrArgument = this.fn_iif.arguments;

```

```
    if (arrArgument[0])  
    {  
        return arrArgument[1];  
    }  
    else  
    {  
        return arrArgument[2];  
    }  
};
```

⑤ QuickView로 확인하기

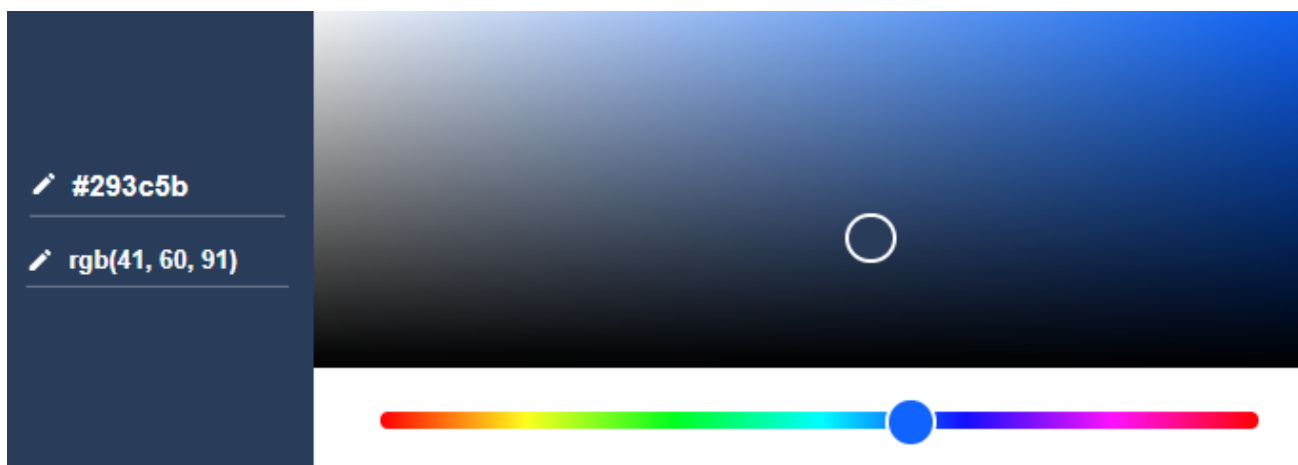
QuickView(Ctrl + F6)로 실행한 후 값을 입력하면서 확인해봅니다.

54.

Global

54.1 RGB 색상 코드 변경하기

색상값을 입력할때 6자리 영문자를 입력하기도 하고 3개의 숫자를 지정하기도 합니다. 같은 값이지만 표시 방식이 16진수인지(HEX) 아니면 우리가 일반적으로 사용하는 10진수인지에 따라 다르게 표시될 뿐입니다. HTML에서는 6자리 영문자를 입력하는 방식이 편하기 때문에 HEX 코드를 주로 사용합니다.



54.1.1 예제

16진수 코드(HEX)를 입력하고 버튼을 클릭하면 RGB 값으로 표시해주고, RGB 값을 입력하고 버튼을 클릭하면 16진수 코드로 표시합니다. 값을 표시하면서 해당 값에 해당하는 색상으로 Form 배경색을 변경합니다.

E9DF0B	HEX to RGB	233	223	11
233	223	11	RGB to HEX	E9DF0B



54.1.2 예제에서 사용한 핵심 기능

parseInt

문자열을 정수로 반환해주는 메소드입니다. 두번째 파라미터로 진법을 지정할 수 있습니다. `parseInt(16진수, 16)` 이런 식으로 지정해주면 10진수로 변환된 값을 반환합니다.

54.1.3 예제 구현 방법

① 화면 구성하기

값을 입력할 수 있는 Edit 컴포넌트와 변환 기능을 처리할 Button 컴포넌트를 배치합니다.



예제에서는 Edit 컴포넌트가 자동으로 생성되는 순서에 따라 id값을 지정하는 기본 상태에서 수정하지 않았습니다. 하지만 이렇게 하는 경우에는 컴포넌트가 많아졌을때 어떤 id값이 어떤 컴포넌트인지 명확하게 아는 것이 쉽지 않습니다. 내부적으로 네이밍 규칙을 정해 지정하는 것이 작업에 도움을 줄 수 있습니다.

② Button 컴포넌트 이벤트 함수 작성하기 (fn_hexToDec)

실제 변환 작업은 `fn_hexToDec` 라는 함수에서 처리합니다. 입력된 6자리 영문자 값을 받아서 대문자로 변환해주고(`toUpperCase`) 2자리씩 잘라서 10진수로 표기하도록 변경합니다(`parseInt`). 변환된 값은 Edit 컴포넌트에 표시하도록 `value` 속성값으로 지정합니다.

```
this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.fn_hexToDec(this.Edit00.value);
};

this.fn_hexToDec = function(sHex)
{
    sHex = sHex.toUpperCase();
    var nRed   = parseInt(sHex.substring(0,2),16);
    var nGreen = parseInt(sHex.substring(2,4),16);
    var nBlue  = parseInt(sHex.substring(4),16);
```

```

    this.Edit01.set_value(nRed);
    this.Edit02.set_value(nGreen);
    this.Edit03.set_value(nBlue);

    this.set_background("#" + sHex);
}

```

3 Button 컴포넌트 이벤트 함수 작성하기 (fn_decToHex)

이번에는 10진수로 입력된 값을 HEX 값으로 표기합니다. 입력받은 값을 숫자로 바꾸어주고(toNumber) 16진수로 변환해줍니다(toString(16)). 이 과정에서 1자리 문자가 나올 수도 있는데 문자열 길이를 체크해서 2보다 작은 경우에는 앞에 "0"을 붙여줍니다.

```

this.Button01_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.fn_decToHex(this.Edit04.value, this.Edit05.value, this.Edit06.value);
};

this.fn_decToHex = function(nRed, nGreen, nBlue)
{
    var sHexRed = nexacro.toNumber(nRed).toString(16);
    var sHexGreen = nexacro.toNumber(nGreen).toString(16);
    var sHexBlue = nexacro.toNumber(nBlue).toString(16);

    sHexRed = sHexRed.length<2?"0"+sHexRed:sHexRed;
    sHexGreen = sHexGreen.length<2?"0"+sHexGreen:sHexGreen;
    sHexBlue = sHexBlue.length<2?"0"+sHexBlue:sHexBlue;

    var sHex = sHexRed.toUpperCase() + sHexGreen.toUpperCase() + sHexBlue.toUpperCase();
    this.Edit07.set_value(sHex);
    this.set_background("#" + sHex);
}

```

4 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 값을 입력하면서 확인해봅니다.



입력할 수 있는 값의 범위를 제한하지 않았기 때문에 범위가 벗어나는 값을 입력하면 Form의 색상이 바뀌지 않습니다.

54.2.3 예제 구현 방법

① 화면 구성하기

Static 컴포넌트와 Edit 컴포넌트, Button 컴포넌트를 배치합니다. Static 컴포넌트는 Edit 컴포넌트 앞에 텍스트를 표시하기 위해 사용합니다.

② Button 컴포넌트의 onclick 이벤트 함수 작성하기

입력된 문자열을 변환하기 위해 encodeURIComponent, decodeURI 메소드를 바로 호출하고 반환된 값을 Edit 컴포넌트에 보여줍니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Edit02.set_value(encodeURIComponent(this.Edit00.value));
    try {
        this.Edit03.set_value(decodeURI(this.Edit01.value));
    } catch (e) {
        this.Edit03.set_value(e.name+": "+e.message);
    }
};
```



decodeURI 메소드 실행 시 인코딩된 URI값이 아닌 값을 입력한 경우에는 URIError로 처리합니다.

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 변환되는 문자를 확인합니다.

54.3 유니코드 변환하기

유니코드로 변환하는 작업은 사용자에게 보여주는 용도보다는 연산이나 다른 변환 작업을 위해 사용합니다.

54.3.1 예제

문자열을 입력하면 유니코드로 변환하고 다시 문자열로 변환합니다.



The image shows a web form with three main components: a text input field containing the Korean text '한글', a text area containing the escaped Unicode string '%uD55C%uAE00', and a button labeled 'change'. The form is enclosed in a light gray border.



54.3.2 예제에서 사용한 핵심 기능

escape, decodeURI

텍스트 문자열을 유니코드로 변환하거나 유니코드값을 텍스트 문자열로 변환합니다.

54.3.3 예제 구현 방법

① 화면 구성하기

Edit 컴포넌트, Button 컴포넌트를 배치합니다. 첫번째 Edit 컴포넌트에 입력된 문자열을 두번째 Edit 컴포넌트에 유니코드로 변환해 보여주고 변환된 유니코드값을 다시 세번째 Edit 컴포넌트에 문자열로 변환해 표시합니다.

② Button 컴포넌트의 onclick 이벤트 함수 작성하기

입력된 문자열을 변환하기 위해 encodeURIComponent, decodeURI 메소드를 바로 호출하고 반환된 값을 Edit 컴포넌트에 보여줍니다.

```
this.Button00.onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Edit01.set_value(escape(this.Edit00.value));
    this.Edit02.set_value(unescape(this.Edit01.value));
};
```

3 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 변환되는 문자를 확인합니다.

54.4 function 찾아서 실행하기

typeof 메소드를 사용하면 Form 오브젝트의 자식이 어떤 형태인지 확인할 수 있습니다. 예제에서는 function을 찾아서 실행하는 방법을 살펴봅니다.

54.4.1 예제

function 이름을 입력하면 해당 이름의 function이 있는지 확인하고 있다면 해당 function을 실행합니다.

fn_test is Function	Button00
---------------------	----------



[sample_global_04.xfdl](#)

54.4.2 예제에서 사용한 핵심 기능

typeof

"number", "string", "boolean", "object", "function" 값을 반환합니다. 선언되지 않는 값인 경우에는 "undefined"를 반환합니다.

54.4.3 예제 구현 방법

1 화면 구성하기

Edit 컴포넌트, Button 컴포넌트를 배치합니다.

2 Button 컴포넌트의 onclick 이벤트 함수 작성하기

입력된 문자열이 Function인지 확인하고 Function인 경우에는 실행하고 그렇지 않은 경우에는 입력된 문자열

의 타입을 반환합니다.

```
this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var strfn = this.Edit00.value.toString();
    if(typeof(this[strfn])=='function')
    {
        this[strfn]();
    }
    else
    {
        this.Edit00.set_value(strfn+' is not Function. is '+typeof(this[strfn]));
    }
};

this.fn_test = function()
{
    this.Edit00.set_value(this.Edit00.value.toString()+' is Function');
}
```

Edit 컴포넌트 클릭 시 출력된 값은 지워줍니다.

```
this.Edit00_oneditclick = function(obj:nexacro.Edit,e:nexacro.EditClickEventInfo)
{
    this.Edit00.set_value();
};
```

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 function 명을 입력하고 버튼을 클릭합니다.

55.

Math

55.1 백분율 구하기

백분율을 구하는 간단한 연산을 Number 오브젝트의 메소드를 사용해 살펴봅니다.

55.1.1 예제

입력한 값에 대한 백분율 연산을 처리하기 결과값을 표시합니다. 반올림 옵션을 추가로 지정할 수 있습니다.

40	is what percent of	300	?
CHECK	FractionDigit	5	
13.33333	%		



sample_math_01.xfdl

55.1.2 예제에서 사용한 핵심 기능

round

소수점을 기준으로 반올림한 값을 반환하는 메소드입니다. 예제에서는 Math 오브젝트의 round 메소드와 nexacroAPI로 제공하는 round 메소드를 사용합니다. Math 오브젝트의 round 메소드는 소수점을 기준으로 반올림한 정수값을 반환합니다. 이에 반해 nexacroAPI의 round 메소드는 반올림할 위치를 지정할 수 있습니다.

55.1.3 예제 구현 방법

1 Form 화면 구성하기

Static 컴포넌트와 Edit 컴포넌트, Button 컴포넌트를 배치합니다.

2 Button 컴포넌트 클릭 이벤트 함수 작성하기

Button 컴포넌트 클릭 시 실행할 이벤트 함수를 작성합니다. fn_percent 함수에서 입력값을 검증하고 검증된 결과값을 Edit 컴포넌트에 표시합니다.

```
this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var nRtn = this.fn_percent(this.Edit00.value, this.Edit01.value, this.Edit03.value);
    this.Edit02.set_value(nRtn);
};
```

3 fn_percent 함수 작성하기

반올림 옵션(nFractionDigit)이 없는 경우에는 Math 오브젝트의 round 메소드를 사용하고 옵션값이 있는 경우에는 nexacroAPI의 round 메소드를 사용합니다.

```
this.fn_percent = function(nTop, nBottom , nFractionDigit)
{
    var nRtn;
    if (nBottom == 0)
    {
        nBottom = 1;
    }

    if (nFractionDigit == null || nFractionDigit == "" || nFractionDigit == "undefined")
    {
        nRtn = Math.round(nTop/nBottom*100);
    } else {
        nRtn = nexacro.round(nTop/nBottom*100, Number(nFractionDigit));
    }

    return nRtn;
}
```

4 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 값을 입력해 확인해봅니다.

56.

Number

56.1 숫자 범위 체크하기

입력폼을 통해 입력받은 대부분의 값은 모두 문자로 처리됩니다. 입력된 값이 숫자인지, 날짜인지 시스템에서는 알 수 없습니다. 사용자가 숫자를 입력하더라도 이를 비교하거나 연산 처리하기 위해서는 명확하게 숫자라는 것을 지정해 주어야 하는 경우가 있습니다.

56.1.1 예제

입력된 두 값(FROM, TO) 사이에 체크하기 원하는 값(INPUT)이 있는지 확인하고 있으면 true, 없으면 false를 반환합니다.

FROM	1
TO	33
INPUT	22,2
RESULT	true

CHECK

 [sample_number_01.xfdl](#)

56.1.2 예제에서 사용한 핵심 기능

Number

Number 오브젝트를 생성하거나 숫자형으로 형변환을 하기 위해 사용합니다. 도움말에서는 new 연산자를 사용해 명시적으로 Number 오브젝트를 생성하는 것은 권장하지 않고 있습니다. 예제에서는 문자열로 전달된

숫자값의 형변환을 위해 사용합니다.

56.1.3 예제 구현 방법

① Form 화면 구성하기

Static 컴포넌트와 Edit 컴포넌트, Button 컴포넌트를 배치합니다.

② Button 컴포넌트 클릭 이벤트 함수 작성하기

Button 컴포넌트 클릭 시 실행할 이벤트 함수를 작성합니다. fn_isNumArea 함수에서 입력값을 검증하고 검증된 결과값을 Edit 컴포넌트에 표시합니다.

```
this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var nRtn = this.fn_isNumArea(this.Edit02.value, this.Edit00.value, this.Edit01.value);
    this.Edit03.set_value(nRtn);
};
```

③ fn_isNumArea 함수 작성하기

입력된 문자를 전달받아 숫자로 변환합니다(Number). 변환된 숫자값을 비교해 반환값을 결정합니다.

```
this.fn_isNumArea = function(sValue, sArea1, sArea2)
{
    var nValue = Number(sValue);
    var nArea1 = Number(sArea1);
    var nArea2 = Number(sArea2);

    if (nArea1 > nArea2)
    {
        return false;
    }

    if (nValue >= nArea1 && nValue <= nArea2)
    {
        return true;
    }
    else
    {
```

```
        return false;
    }
}
```

사용할 값이 정수인 경우에는 parseInt를 사용하더라도 같은 기능을 처리할 수 있습니다. fn_isNumArea 함수의 첫번째 스크립트는 아래와 같이 변경할 수 있습니다.

```
this.fn_isNumArea = function(sValue, sArea1, sArea2)
{
    var nValue = parseInt(sValue);
    var nArea1 = parseInt(sArea1);
    var nArea2 = parseInt(sArea2);
    ...
}
```

4 QuickView로 확인하기

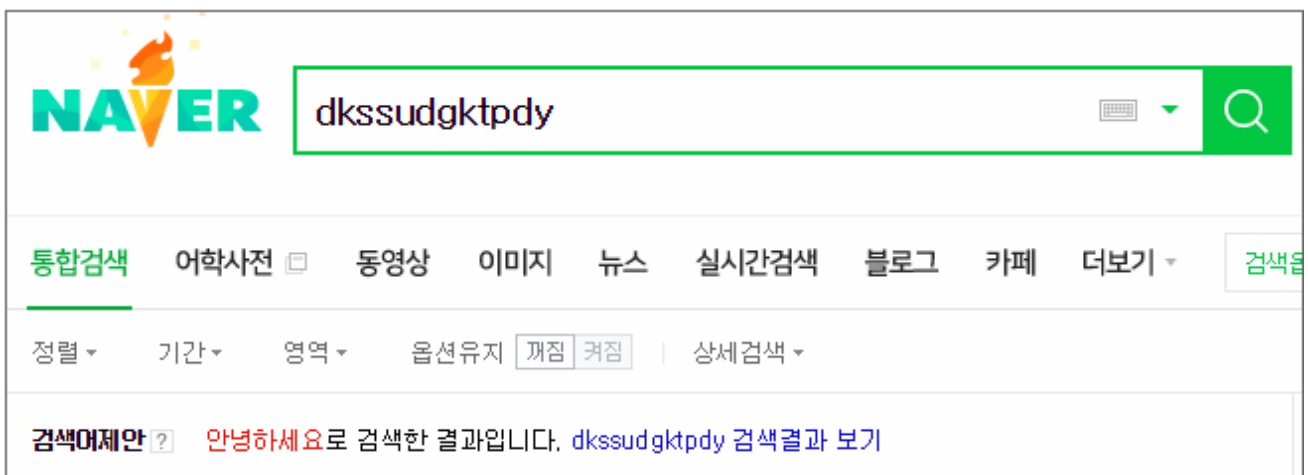
QuickView(Ctrl + F6)로 실행한 후 값을 입력해 확인해봅니다.

57.

String

57.1 영문 자판으로 한글 입력 시 자동 전환하기

간혹 영문 자판 상태에서 한글을 입력하는 경우가 있습니다. 이런 경우 잘못을 인지하지 못하고 계속 입력하는 경우가 있는데 이를 뒤늦게 발견하면 다시 입력해야 합니다. 검색창에서 이런 실수가 많이 나타나고 있어서 아예 검색 포털에서는 사용자가 영문 자판으로 한글을 입력하면 혹시 이걸 찾은 것이 아니냐고 물어보기도 합니다.



이번 예제에서는 어떤 방식으로 영문으로 입력된 텍스트를 한글로 변경할 수 있는지 살펴봅니다.



예제에서 사용한 스크립트는 공개된 스크립트 예제를 참조했습니다.

출처 : <http://www.theyt.net/>

57.1.1 예제

English라고 표시된 TextArea 컴포넌트에 영문을 입력하면 Korean 항목에 한글이 표시됩니다. 반대로 Korean 항목에 한글을 입력하면 English 항목에 영문이 표시됩니다.

English	dkssudgktpdy
Korean	안녕하세요



57.1.2 예제에서 사용한 핵심 기능

oninput

TextArea 컴포넌트에 문자열을 입력할 때 발생하는 이벤트입니다. 예제에서는 별도 명령을 처리하는 버튼이 없고 TextArea 컴포넌트에 입력할 때 영문을 한글로 변경하도록 합니다.

fromCharCode

인수로 전달된 유니코드값을 문자열로 반환하는 메소드입니다.

57.1.3 예제 구현 방법

① Form 화면 구성하기

Static 컴포넌트로 "English", "Korean"이라는 문구를 표시하고 해당 문구 옆에 TextArea 컴포넌트를 배치합니다.

② TextArea 컴포넌트 이벤트 함수 작성하기

TextArea 컴포넌트에 문자열을 입력할 때 이벤트가 처리되도록 합니다. oninput 이벤트 함수를 아래와같이 작성합니다. oninput 이벤트 함수 내에서는 실질적인 변환 작업을 진행하지는 않습니다. TextArea 컴포넌트에 값을 지정하는 부분을 보면 engTypeToKor, KorTypeToEng 라는 함수를 다시 호출하고 있습니다. 실제 변환 작업은 해당 함수에서 처리하고 있다는 것을 알 수 있습니다.


```

if (nJong == -1) {
  nJong = this.JONG_DATA.indexOf(this.KOR_KEY.charAt(p));
  if (nJong == -1) {
    res += this.makeHangul(nCho, nJung, nJong);
    nCho = this.CHO_DATA.indexOf(this.KOR_KEY.charAt(p));
    nJung = -1;
  }
}
}

```



전체 스크립트 코드는 소스를 참조해주세요.

5 makeHangul 함수 작성하기

fromCharCode 메소드는 유니코드 값을 입력하면 문자를 반환해줍니다.

```

this.makeHangul = function (nCho, nJung, nJong) {
  return String.fromCharCode(0xac00 + nCho * 21 * 28 + nJung * 28 + nJong + 1);
};

```

6 korTypeToEng 함수 작성하기

한글을 영문으로 변환하는 경우에는 합쳐져 있는 한글의 초성, 중성, 종성을 분해해주어야 합니다. 한글이 조합되는 경우 "ㄴ्ह"와 같은 경우 5개의 영문 자판과 연결되기 때문에 이를 분해하기 위해 5개 길이의 배열을 생성하고 글자를 분해해주는 작업을 진행합니다.

```

for (var i = 0; i < src.length; i++) {
  var ch = src.charAt(i);
  var nCode = ch.charCodeAt(0);
  var nCho = this.CHO_DATA.indexOf(ch), nJung = this.JUNG_DATA.indexOf(ch), nJong = this
    .JONG_DATA.indexOf(ch);
  var arrKeyIndex = [-1, -1, -1, -1, -1];

  if (0xac00 <= nCode && nCode <= 0xd7a3) {
    nCode -= 0xac00;
    arrKeyIndex[0] = Math.floor(nCode / (21 * 28));
    arrKeyIndex[1] = Math.floor(nCode / 28) % 21;
    arrKeyIndex[3] = nCode % 28 - 1;
  }
}

```



전체 스크립트 코드는 소스를 참조해주세요.

7 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 English, Korean 항목에 다른 언어를 입력해보고 결과를 확인합니다.

57.2 이메일 주소값 검증하기

이메일 주소값을 입력받은 항목이 있는 경우 입력한 값이 유효한 이메일 주소값인지 확인합니다.

57.2.1 예제

e-mail 항목에 이메일 주소값을 입력하고 [check] 버튼을 클릭하면 입력값이 유효한 값인지 아닌지 result 항목에 true/false로 표시합니다.

e-mail	test@@test.com	check
result	false	

 sample_string_02.xfdl

57.2.2 예제에서 사용한 핵심 기능

indexOf

입력받은 문자열 중에 이메일 주소에서 사용할 수 없는 문자가 있는지 확인합니다. 사용할 수 없는 문자가 있는 경우 -1을 반환합니다.

57.2.3 예제 구현 방법

1 Form 화면 구성하기

Static 컴포넌트와 Edit 컴포넌트, Button 컴포넌트를 배치합니다.

2 Button 컴포넌트 클릭 이벤트 함수 작성하기

Button 컴포넌트 클릭 시 실행할 이벤트 함수를 작성합니다. fn_checkEmail 함수에서 이메일 주소값을 검증하고 검증된 결과값을 Edit 컴포넌트에 표시합니다.

```
this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var sRtn = this.fn_checkEmail(this.Edit00.value);
    this.Edit01.set_value(sRtn);
};
```

3 fn_checkEmail 함수 작성하기

이메일 주소값으로 사용할 수 있는 문자를 모두 vChk 라는 변수에 담습니다. 그리고 입력된 이메일 주소값을 f or 문 내에서 indexOf 메소드를 사용해 확인합니다.

```
this.fn_checkEmail = function(sValue)
{
    var vChk = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz1234567890-._@";
    for (var i = 0 ; i<sValue.length ; i++) {
        if (vChk.indexOf(sValue.charAt(i)) < 0) {
            trace(vChk.indexOf(sValue.charAt(i)));
            return false;
        }
    }
    return true;
}
```

위와 같이 작성한 경우에는 잘못된 문자가 포함되는 것만을 방지할 수 있습니다. 중간에 "@" 문자가 없거나 도메인 형식이 잘못된 경우에는 체크하지 못합니다. 나머지 검증 처리를 위해서는 함수의 보완이 필요합니다.



전체 스크립트 코드는 소스를 참조해주세요.

4 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 잘못된 형식의 이메일 주소를 입력하고 확인해봅니다.

57.3 전각문자와 반각문자 변환하기

한국어나 영어만 사용하는 경우에는 유용하지 않지만 일본어를 지원하는 애플리케이션을 만드는 경우에 필요한 기능입니다.

57.3.1 예제

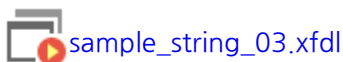
전각 또는 반각문자를 입력하고 버튼을 클릭하면 반각 또는 전각문자로 변환해줍니다.



Half width to Fullwidth n e x a c r o nexacro

Fullwidth to Halfwidth nexacro n e x a c r o

change



57.3.2 예제에서 사용한 핵심 기능

charCodeAt

입력된 문자열 중 특정 인덱스 위치의 문자에 대한 유니코드값을 반환합니다. 예제에서는 유니코드값을 확인하고 변환을 위한 값을 더하거나 빼서 전각 또는 반각문자로 변환합니다.

imemode

IME 모드를 지정할 수 있습니다. 해당 속성값은 IE 브라우저와 넥사크로 브라우저만 지원합니다. IE 브라우저를 사용하는 경우에는 전각 입력은 지원하지 않습니다.

57.3.3 예제 구현 방법

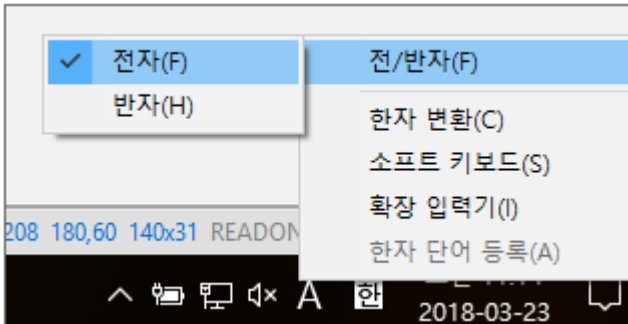
① 화면 구성하기

Static 컴포넌트와 Edit 컴포넌트, Button 컴포넌트를 배치합니다. Static 컴포넌트는 Edit 컴포넌트 앞에 텍스트를 표시하기 위해 사용합니다.

② Edit 컴포넌트 속성 지정하기

전각 문자를 입력할 Edit 컴포넌트의 imemode 속성값을 "alpha,full"으로 지정하고 반각 문자를 입력할 Edit 컴포넌트의 imemode 속성값을 "alpha"로 지정합니다. 해당 속성은 넥사크로 브라우저로 실행하는 경우에만 유효하게 동작합니다.

웹브라우저에서 동작하는 것을 쉽게 확인하기 위해 value 속성값으로 미리 전각문자와 반각문자를 입력해놓습니다. 일반적인 한국어 IME에서 영문 전각문자는 아래와 같이 입력할 수 있습니다.



변환될 문자를 출력하는 Edit 컴포넌트의 enable 속성값은 false로 지정합니다.

③ Button 컴포넌트의 onclick 이벤트 함수 작성하기

입력된 문자열을 변환하기 위해 fn_fullToHalf, fn_halfToFull 함수를 호출하고 반환된 값을 Edit 컴포넌트의 value 속성값으로 지정합니다.

```
this.Button00.onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Edit02.set_value(this.fn_fullToHalf(this.Edit00.value));
    this.Edit03.set_value(this.fn_halfToFull(this.Edit01.value));
};
```

④ 전각문자와 반각문자 변환하기

입력된 문자열을 받아 length 속성값으로 확인된 길이만큼 반복하면서 유니코드값을 확인(charCodeAt)하고 전각 또는 반각문자로 변환하기 위해 연산한 다음 ASCII 문자 형식으로 변환(unescape)합니다.

```

this.fn_fullToHalf = function(sFull)
{
    var sHalf = "";
    var sChar = "";

    for (var i = 0; i < sFull.length; i++)    {
        sChar = sFull.charCodeAt(i);
        if (sChar == 12288)    {
            sHalf += unescape("%20");
        }
        else if ((sChar >= 65281) && (sChar <= 65374))    {
            sHalf += unescape("%" + (sChar-65248).toString(16));
        } else {
            sHalf += sFull.charAt(i);
        }
    }
    return sHalf;
}

```

```

this.fn_halfToFull = function(sHalf)
{
    var sFull = "";
    var sChar = "";

    for (var i = 0; i < sHalf.length; i++) {
        if ((sHalf.charCodeAt(i) >= 32) && (sHalf.charCodeAt(i) <= 126)) {
            if (sHalf.charCodeAt(i) == 32) {
                sChar = unescape("%u" + (12288).toString(16));
            } else {
                sChar = sHalf.charCodeAt(i) + 65248;
            }
        } else {
            sChar = sHalf.charCodeAt(i);
        }
        if (sHalf.charCodeAt(i) == 32) {
            sFull = sFull + (sChar);
        } else {
            sFull = sFull + String.fromCharCode(sChar);
        }
    }
}

```

```
return sFull;
}
```

5 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 변환되는 문자를 확인합니다.

57.4 문자열이 지정된 길이가 되도록 다른 문자로 채우기

특정 자리수만큼 문자열을 입력해야 하는 경우가 있습니다. 예제에서 설명한 방법을 활용하면 "01"을 입력해야 하는 항목에 "1"만 입력하면 앞자리 "0"을 자동으로 채우도록 할 수 있습니다.

57.4.1 예제

문자열을 입력하고 길이를 선택한 후 버튼을 클릭하면 왼쪽과 오른쪽에 "*" 문자로 채운 문자열을 표시합니다.

 sample_string_04.xfdl

57.4.2 예제에서 사용한 핵심 기능

padLeft, padRight

지정한 길이만큼 문자열의 왼쪽 또는 오른쪽 공간을 특정 문자로 채웁니다. 문자열이 지정된 길이보다 긴 경우에는 동작하지 않습니다.

57.4.3 예제 구현 방법

① 화면 구성하기

Edit 컴포넌트, Spin 컴포넌트, Button 컴포넌트를 배치합니다. Spin 컴포넌트는 문자열의 길이를 지정하기 위한 용도로 사용합니다. value 속성값은 20으로 지정합니다.

② Button 컴포넌트의 onclick 이벤트 함수 작성하기

입력된 문자열을 변환하기 위해 padLeft, padRight 메소드를 호출하고 반환된 값을 Edit 컴포넌트의 value 속성값으로 지정합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Edit01.set_value(this.Edit00.value.padLeft(this.Spin00.value, "*"));
    this.Edit02.set_value(this.Edit00.value.padRight(this.Spin00.value, "*"));
};
```

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 변환되는 문자를 확인합니다.

57.5 문자열에 있는 하이픈(-) 제거하기

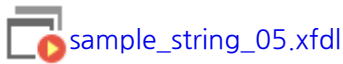
입력된 문자열에 포함된 하이픈을 제거하는 방법을 살펴봅니다.

57.5.1 예제

하이픈(-)이 들어간 문자를 입력받고 해당 문자를 제거해 표시합니다.



The image shows a simple web application interface. At the top, there is a text input field containing the string "010-2222-3333". To the right of this input field is a button labeled "Button00". Below the input field, there is another text input field (or a display area) showing the result "01022223333", which is the original string with all hyphens removed.



57.5.2 예제에서 사용한 핵심 기능

replace

지정된 문자를 다른 문자로 바꿀 수 있습니다. 정규식을 사용하면 여러 문자를 동시에 바꿀 수 있습니다. 예제에서는 하이픈 문자를 모두 제거하기 때문에 정규식을 사용합니다.

57.5.3 예제 구현 방법

① 화면 구성하기

Edit 컴포넌트와 Button 컴포넌트를 배치합니다.

② onclick 이벤트 함수 작성하기

입력받은 문자열 중에서 하이픈(-) 문자를 삭제합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var sRtn = this.Edit00.value;
    var sRemoveHyphen = sRtn.toString().replace(/[-]/g, "");
    this.Edit01.set_value(sRemoveHyphen);
};
```

replace 메소드 대신 replaceAll 메소드를 사용할 수도 있습니다.

```
var sRemoveHyphen = nexacro.replaceAll(sRtn, "-", "");
```

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 변환되는 문자를 확인합니다.

57.6 주민등록번호 유효성 검사하기

입력된 문자열을 규칙에 따라 유효성을 검토하는 방식을 살펴봅니다.

57.6.1 예제

주민등록번호를 입력하면 유효성을 검사하고 성별값을 반환합니다.




57.6.2 예제에서 사용한 핵심 기능

substr

지정된 위치부터 지정된 길이만큼의 문자열을 반환합니다. 원본 문자열 자체를 변경하는 것은 아닙니다.

57.6.3 예제 구현 방법

① 화면 구성하기

Edit 컴포넌트와 Button 컴포넌트를 배치합니다.

② onclick 이벤트 함수 작성하기

입력받은 문자열을 지정된 규칙에 따라 유효성을 검사합니다. 주민등록번호의 유효성을 검증한 후 유효한 경우에는 성별을 체크해 Edit 컴포넌트에 표시합니다.

```
this.Button00.onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var sJuminNo = this.Edit00.value;
```

```

var returnValue = true;

var arrChk = [2, 3, 4, 5, 6, 7, 8, 9, 2, 3, 4, 5];
var sFNum = sJuminNo.substr(0,6).toString();
var sLNum = sJuminNo.substr(6).toString();
var sYY = sFNum.substr(0,2);
var sMM = sFNum.substr(2,2);
var sDD = sFNum.substr(4,2);
var sGenda = sLNum.substr(0,1);
var nSum = 0;
var sCC = "20";

if (sYY < "00" || sYY > "99" || sMM < "01" || sMM > "12" || sDD < "01" || sDD > "31")
{
    returnValue = false;
}

if (sGenda < "1" || sGenda > "4")
{
    returnValue = false;
}

if (sGenda == "1" || sGenda == "2")
{
    sCC = "19";
}

if (this.fn_isYYYYMMDD(parseInt(sCC + sYY), parseInt(sMM), parseInt(sDD)) == false)
{
    returnValue = false;
}

for (var i = 0; i < 12 ; i++)
{
    nSum += parseInt(sJuminNo.substr(i, 1)) * arrChk[i];
}

nSum = 11 - (nSum % 11);
nSum = nSum % 10;

```



```

if (nSum != parseInt(sJuminNo.substr(12, 1)))
{
    returnValue = false;
}

var sRtn;
if(returnValue)
{
    var vGender = sJuminNo.substr(6, 1);
    if (vGender == '1' || vGender == '3' || vGender == '5' || vGender == '7')
    {
        sRtn = "M";
    }
    else if (vGender == '2' || vGender == '4' || vGender == '6' || vGender == '8')
    {
        sRtn = "W";
    }
    else
    {
        sRtn = "X";
    }
} else {
    sRtn = "X";
}

this.Edit01.set_value(sRtn);
};

```

입력된 값 중에서 날짜에 해당하는 부분의 유효성을 검토합니다.

```

this.fn_isYYYYMMDD = function(nYY, nMM, nDD)
{
    switch (nMM)
    {
        case 2:
            if (nDD > 29)
            {
                return false;
            }
            if (nDD == 29)
            {

```

```

        if ((nYY % 4) == 0)
        {
            if ((nYY % 100) != 0 || (nYY % 400) == 0)
            {
                return false;
            }
        }
        break;
    case 4:
    case 6:
    case 9:
    case 11:
        if (nDD == 31)
        {
            return false;
        }
        break;
    }
    if(nMM > 12) return false;

    return true;
};

```

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 반환되는 값을 확인합니다.

57.7 법인등록번호 유효성 검사하기

입력된 문자열을 규칙에 따라 유효성을 검토하는 방식을 살펴봅니다.



법인등록번호의 유효성 검사 방식은 "법인 및 재외국민의 부동산등기용등록번호 부여에 관한 규칙 - 오류검색번호산출방식"에 따릅니다.

57.7.1 예제

법인등록번호를 입력하면 유효성을 검사하고 성별값을 반환합니다.




sample_string_07.xfdl

57.7.2 예제에서 사용한 핵심 기능

substr

지정된 위치부터 지정된 길이만큼의 문자열을 반환합니다. 원본 문자열 자체를 변경하는 것은 아닙니다.

57.7.3 예제 구현 방법

① 화면 구성하기

Edit 컴포넌트와 Button 컴포넌트를 배치합니다.

② onclick 이벤트 함수 작성하기

입력받은 문자열을 지정된 규칙에 따라 유효성을 검사합니다. 주민등록번호의 유효성을 검증한 후 유효한 경우에는 성별을 체크해 Edit 컴포넌트에 표시합니다.

```
this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var arrChk = [1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2];
    var Sum = 0;
    var sComp = this.Edit00.value;

    for (var i = 0; i < 12; i++)
    {
        Sum += arrChk[i] * sComp.charAt(i);
```

```

    }

    Sum = Sum % 10;
    trace("substring : " + sComp.substring(12, 13));
    Sum = 10 - Sum;

    if (Sum > 5 )
    {
        Sum = 0;
    }
    if (Sum == sComp.substring(12, 13))
    {
        this.Edit01.set_value(true);
    } else {
        this.Edit01.set_value(false);
    }
};

```

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 반환되는 값을 확인합니다.

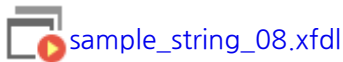
57.8 NULL 값 체크하기

입력된 문자열이 유효한 값인지 확인합니다.

57.8.1 예제

입력된 문자열이 없으면 true를 반환합니다. 버튼을 클릭해 강제로 빈값이나 null, undefined 값을 설정할 수 있습니다.

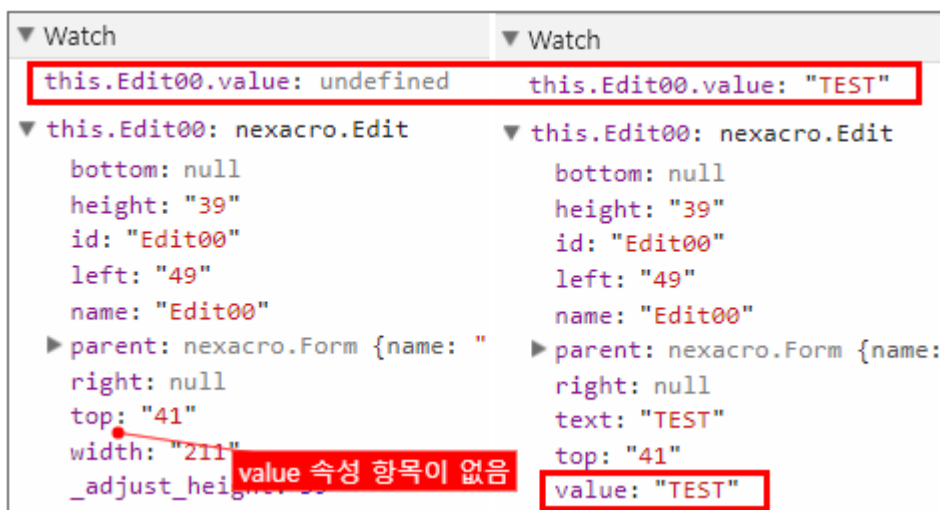
o	NULL check
false	set NULL <input ""="" type="button" value="set "/> set undefined



57.8.2 예제에서 사용한 핵심 기능

undefined

undefined는 변수가 생성된 후 값을 지정하지 않았을 때 주어지는 값입니다. 예제를 실행하고 바로 [NULL check] 버튼을 클릭하면 undefined로 처리가 되는데, 실제 Edit00을 살펴보면 value라는 속성값 자체가 없습니다.



57.8.3 예제 구현 방법

① 화면 구성하기

Edit 컴포넌트와 Button 컴포넌트를 배치합니다.

② onclick 이벤트 함수 작성하기

입력받은 문자열을 지정된 규칙에 따라 유효성을 검사합니다.

```
this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Edit01.set_value(this.fn_checkNull(this.Edit00.value));
};

this.fn_checkNull = function(sValue)
```

```

{
    if (new String(sValue).valueOf() == "undefined")
    {
        return true;
    }

    if (sValue == null)
    {
        return true;
    }

    var sChkStr = new String(sValue);

    if (sChkStr == null)
    {
        return true;
    }

    if (sChkStr.toString().length == 0 )
    {
        return true;
    }

    return false;
};

```

③ 유효하지 않은 값 설정하기

강제적으로 유효하지 않은 값을 설정합니다. [set undefined] 버튼 클릭 시에는 undefined라는 값을 value 속성값으로 설정합니다. 이때 hasOwnProperty 메소드로 value 속성 유무를 확인해보면 false값을 반환합니다.

```

this.Button01_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Edit00.set_value(null);
};

this.Button02_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Edit00.set_value("");
};

```

```

this.Button03_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Edit00.set_value(undefined);
    trace(this.Edit00.hasOwnProperty('value'));
};

```

4 QuickView로 확인하기

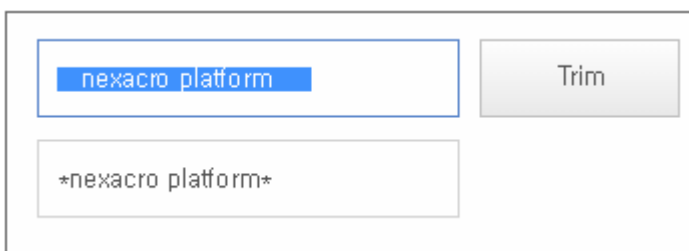
QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 반환되는 값을 확인합니다.

57.9 앞뒤 공백문자 제거하기

입력된 문자열의 앞, 뒤 공백문자를 제거합니다.

57.9.1 예제

버튼 클릭 시 입력된 문자열의 앞, 뒤 공백문자를 제거한 결과값을 반환합니다.



 sample_string_09.xfdl

57.9.2 예제에서 사용한 핵심 기능

trim

예제에서 fn_trim 함수 대신 String 오브젝트의 trim 메소드를 사용해도 같은 결과를 얻을 수 있습니다. 다만, trim 메소드는 값이 null이나 undefined처럼 정해져 있지 않은 경우에는 사용할 수 없습니다.

nexacro.trim

String 오브젝트의 trim 메소드와 비슷하지만, 공백문자 대신 제거할 문자를 지정할 수 있습니다. nexacro.trim("#TEST#", "#")처럼 실행하면 "TEST"라는 문자열을 반환합니다.

57.9.3 예제 구현 방법

① 화면 구성하기

Edit 컴포넌트와 Button 컴포넌트를 배치합니다.

② onclick 이벤트 함수 작성하기

입력받은 문자열에서 공백문자를 제거합니다.

```

this.Button00.onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Edit01.set_value(" "+this.fn_trim(this.Edit00.value)+" ");
};

this.fn_trim = function(sValue)
{
    if (sValue == null)
    {
        return "";
    }
    if (new String(sValue).valueOf() == "undefined")
    {
        return "";
    }
    if (new String(sValue) == null)
    {
        return "";
    }

    var retVal = (new String(sValue)).replace(/^\s+|\s+$/g, '');
    return retVal;
}

```

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 반환되는 값을 확인합니다.

57.10 숫자 형식 여부 확인하기

입력된 문자열이 숫자인지 확인합니다.

57.10.1 예제

입력된 문자열이 숫자인지 확인하고 결과를 표시합니다. nexacro.isNumeric 메소드 결과도 같이 표시하는데, 음수나 소수의 경우에 nexacro.isNumeric 메소드에서는 false 값을 반환합니다.




57.10.2 예제에서 사용한 핵심 기능

isNumeric

입력된 문자코드 또는 문자열이 숫자인지 여부를 확인합니다. 확인할 수 있는 범위의 제약이 있습니다.

charAt

입력된 문자열의 길이만큼 포함된 문자를 모두 체크하는 용도로 사용합니다. 음수 또는 양수를 표시하는 기호와 소숫점 포함 여부를 확인합니다.

57.10.3 예제 구현 방법

- ① 화면 구성하기

Edit 컴포넌트와 Button 컴포넌트를 배치합니다.

2 onclick 이벤트 함수 작성하기

입력받은 문자열이 숫자인지 확인하고 Edit 컴포넌트에 결과를 표시합니다.

```

this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var strchecknumber = this.fn_checkNumber(this.Edit00.value);
    var strisNumeric = nexacro.isNumeric(this.Edit00.value);
    this.Edit01.set_value(strchecknumber+', '+strisNumeric);
};

this.fn_checkNumber = function(sNum)
{
    var sChar;
    var nCnt = 0;
    var bRtn;

    for (var i = 0; i < sNum.length; i++)
    {
        sChar = sNum.charAt(i);

        if (i == 0 && (sChar == "+" || sChar == "-"))
        {
            bRtn = true;
        }
        else if (sChar >= "0" && sChar <= "9")
        {
            bRtn = true;
        }
        else if (sChar == ".")
        {
            nCnt++;
            if (nCnt > 1)
            {
                bRtn = false;
                break;
            }
        }
        else

```

```

    {
        bRtn = false;
        break;
    }

    return bRtn;
}

```

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 반환되는 값을 확인합니다.

57.11 영문이나 숫자 여부 확인하기

입력된 문자열이 영문 또는 숫자인지 확인합니다.

57.11.1 예제

입력된 문자열이 영문인지 또는 숫자인지 확인하고 결과를 표시합니다.

nexacro17	isAlphanumeric
true	

 sample_string_11.xfdl

57.11.2 예제에서 사용한 핵심 기능

charCodeAt

문자열 중 지정된 인덱스에 해당하는 문자의 유니코드 값을 반환합니다.

57.11.3 예제 구현 방법

① 화면 구성하기

Edit 컴포넌트와 Button 컴포넌트를 배치합니다.

② onclick 이벤트 함수 작성하기

입력받은 문자열이 숫자인지 확인하고 Edit 컴포넌트에 결과를 표시합니다.

```

this.Button00.onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var bRtn = this.fn_checkAlpaNum(this.Edit00.value);
    this.Edit01.set_value(bRtn);
};

this.fn_checkAlpaNum = function(sString)
{
    var sVal = "";
    var bVal = false;

    for (var i = 0; i < sString.length; i++)
    {
        var sVal = sString.charCodeAt(i);
        if ((sVal >= 48 && sVal <= 57) || (sVal >= 65 && sVal <= 90) || (sVal >= 97 && sVal
<= 122))
        {
            bVal = true;
        }
        else
        {
            bVal = false;
            break;
        }
    }
    return bVal;
}

```



https://en.wikipedia.org/wiki/ASCII#Character_set

48부터 57까지는 0~9까지의 숫자

65~90까지는 영문 대문자, 97~122까지는 영문 소문자

3 QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 반환되는 값을 확인합니다.

57.12 날짜, 시간 유효성 확인하기

입력된 문자열이 유효한 날짜, 시간값인지 확인합니다.

57.12.1 예제

입력된 문자열이 유효한 날짜, 시간값인지 확인하고 결과를 표시합니다.

2018-12-32	isDate
23:66:01	isTime
false	



[sample_string_12.xfdl](#)

57.12.2 예제에서 사용한 핵심 기능

substr

시작 위치 인덱스부터 지정된 길이만큼의 문자열을 반환합니다. 예제에서는 YYYY-MM-DD 형식으로 텍스트가 입력된다고 가정합니다.

57.12.3 예제 구현 방법

① 화면 구성하기

Edit 컴포넌트와 Button 컴포넌트를 배치합니다.

② onclick 이벤트 함수 작성하기

입력받은 문자열이 유효한 날짜값인지 확인하고 Edit 컴포넌트에 결과를 표시합니다.

```

this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var bRtn = this.fn_checkDate(this.Edit00.value);
    this.Edit01.set_value(bRtn);
};

this.fn_checkDate = function(sDate)
{
    var nYear = Number(sDate.toString().substr(0,4));
    var nMonth = Number(sDate.toString().substr(5,2));
    var nDate = Number(sDate.toString().substr(8,2));

    if ( nMonth > 12 || nMonth < 1 )
    {
        return false;
    }

    switch (nMonth)
    {
        case 1:
        case 3:
        case 5:
        case 7:
        case 8:
        case 10:
        case 12:
            if (nDate > 31 || nDate < 1)
            {
                return false;
            }
            break;
    }
}

```

```

case 4:
case 6:
case 9:
case 11:
    if (nDate > 30 || nDate < 1)
    {
        return false;
    }
    break;
case 2:
    if (((nYear % 4 == 0) && (nYear % 100 != 0)) || (nYear % 400 == 0))
    {
        if (nDate > 29 || nDate < 1)
        {
            return false;
        }
    }
    else
    {
        if (nDate > 28 || nDate < 1)
        {
            return false;
        }
    }
    break;
default:
    break;
}
return true;
}

```



2월인 경우에는 윤년 여부를 확인합니다. 윤년은 4로 나누어지면서 100으로 나누어 떨어지지 않고 400으로 나누어 떨어지는 년입니다.

입력받은 문자열이 유효한 시간값인지 확인하고 Edit 컴포넌트에 결과를 표시합니다.

```

this.Button01_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    var bRtn = this.fn_checkTime(this.Edit02.value);
    this.Edit01.set_value(bRtn);
}

```

```
};

this.fn_checkTime = function(sTime)
{
    var nHH = Number(sTime.toString().substr(0,2));
    var nMM = Number(sTime.toString().substr(3,2));
    var nSS = Number(sTime.toString().substr(6,2));

    if (nHH > 24 || nHH < 0)
    {
        return false;
    }

    if (nMM > 59 || nMM < 0)
    {
        return false;
    }

    if (nSS > 59 || nSS < 0)
    {
        return false;
    }
    return true;
}
```

③ QuickView로 확인하기

QuickView(Ctrl + F6)로 실행한 후 버튼을 클릭해 반환되는 값을 확인합니다.